

UNIVERSIDAD DEL BOSQUE
Facultad de Ingeniería
Ingeniería de Software & Ingeniería en Robótica

Competitive Programming Notebook

Algoritmos, estructuras de datos y plantillas

C++

Juan Carlos Morales
Doble titulación: Sistemas & Robótica

Bogotá, Colombia · 2026

Índice

I	Introduccion	9
1	Fundamentos Básicos	9
1.1	Entrada / Salida Rápida (Fast I/O)	9
1.2	Formateo de Salida	9
1.3	Condicionales y Ciclos	10
1.4	Conversiones, Parseos y Casteos	10
II	Estructuras de Datos	11
2	Basic Structures	11
2.1	Matrices (Arreglos 2D)	11
2.2	Vectores / Listas dinámicas	11
2.3	Listas enlazadas (LinkedList)	11
2.4	Pilas (Stack)	11
2.5	Colas (Queue)	11
2.6	Colas de prioridad (Priority Queue / Heap)	12
2.7	Comparadores en C++ (Comparators)	12
2.8	Vector circular (Circular Buffer)	13
3	Estructuras Hash/Tree	14
3.1	Conjuntos hash (HashSet)	14
3.2	Mapas / Diccionarios hash	15
3.3	Conjuntos ordenados (TreeSet)	15
3.4	Mapas ordenados (TreeMap)	16
4	Trees	16
5	Range Queries	16
5.1	Casos de uso	17
5.2	Prefix Sums	17
5.3	Árbol de Fenwick (BIT)	19
5.4	Sqrt Decomposition	22
5.5	Sparse Table (RMQ)	24
5.6	Segment Tree	27
5.7	RMQ (Range Minimum Query)	30
5.8	Mo's Algorithm	31
5.9	Wavelet Tree	34
5.10	Elemento Mayoritario	35
5.11	Heavy-Light Decomposition	37
III	Grafos	38
5.12	¿Qué algoritmo usar?	38
6	Traversal	39
6.1	Representación de Grafos	39
6.2	BFS (Breadth-First Search)	41
6.3	BFS en Grafos Desconectados	43
6.4	BFS para Todos los Caminos	44
6.5	DFS	45
6.6	DFS para Conteo de Caminos	46
6.7	DFS + Cerramiento Transitivo	47
6.8	Flood Fill	48
6.9	Laberinto con BFS	48
6.10	Tour de Caballo de Ajedrez	49
6.11	Mínimo Número de Movimientos de Caballo	50
6.12	Serpientes y Escaleras	50

7	Conectividad	51
7.1	Componentes Conexas	51
7.2	Detección de Ciclos	52
7.3	Puentes	53
7.4	Puntos de Articulación	54
7.5	Componentes Biconectados	54
7.6	Tarjan	55
8	Topological Sort	56
8.1	Kahn's Algorithm	56
8.2	DFS + Orden Topológico	57
8.3	Detección de Ciclos con Topological Sort	57
9	Single Source Shortest Path (SSSP)	58
9.1	BFS para Caminos Más Cortos	58
9.2	0-1 BFS	59
9.3	Dijkstra	60
9.4	Bellman-Ford	60
9.5	Shortest Path en Grafos Binarios	61
9.6	Shortest Path desde Múltiples Orígenes	62
9.7	Shortest Path en Grafos Multietapa	63
9.8	Shortest Path con Exactamente k Aristas	63
10	All Pairs Shortest Path (APSP)	64
10.1	Floyd-Warshall	64
11	Minimum Spanning Tree (MST)	65
11.1	Disjoint Set Union (DSU)	65
11.2	Kruskal	66
11.3	Prim	66
12	Strongly Connected Components (SCC)	67
12.1	Kosaraju	67
12.2	Tarjan SCC	68
12.3	Condensation Graph	69
13	Bipartite Graphs	70
13.1	2-Coloring	70
13.2	Bipartite Matching	70
14	Trees	71
14.1	DFS en Árboles	71
14.2	BFS en Árboles	72
14.3	Diámetro de un Árbol	73
14.4	Centro de un Árbol	73
14.5	Combinatoria en Árboles Ordenados	74
14.6	Tree DP	75
14.7	Rerooting	76
15	Lowest Common Ancestor (LCA)	77
15.1	Binary Lifting	77
15.2	Euler Tour + RMQ	78
15.3	Distancia entre Dos Nodos	79
15.4	k -ésimo Ancestro	80
16	Tree Queries	80
16.1	Euler Tour	80
16.2	Subtree Queries	81
16.3	Path Queries	82
16.4	Heavy-Light Decomposition	83
16.5	Heavy-Light Decomposition + Fenwick	85
16.6	Heavy-Light Decomposition + Segment Tree	86

17 Disjoint Set Union	87
17.1 Union-Find	87
17.2 Path Compression	88
17.3 Union by Size / Rank	88
17.4 Unión de Conjuntos Disjuntos en un Árbol	89
17.5 DSU Offline	89
17.6 DSU Rollback	90
18 Graph Decomposition	91
18.1 Bridge Tree	91
18.2 Block-Cut Tree	92
18.3 Condensation Graph	93
19 Eulerian Graphs	94
19.1 Eulerian Path	94
19.2 Eulerian Circuit	95
19.3 Hierholzer	96
20 Network Flow	97
21 Network Flow	97
21.1 Ford-Fulkerson	97
21.2 Edmonds-Karp	99
21.3 Dinic	100
21.4 Maximum Bipartite Matching	101
22 2-SAT	103
22.1 Implication Graph	103
22.2 2-SAT con SCC	103
23 Backtracking y Problemas Clásicos	105
23.1 Problema de las N Reinas	105
23.2 Variaciones del Problema de las N Reinas	106
23.3 Tour de Caballo	107
24 Advanced Graph Algorithms	108
24.1 Centroid Decomposition	108
24.2 DSU on Tree	109
24.3 Hopcroft-Karp	110
24.4 Min-Cost Max-Flow	111
24.5 DSU Rollback	113
IV Ordenamiento y Búsqueda	114
24.6 ¿Qué algoritmo utilizar?	114
25 Ordenamiento y Búsqueda (Sorting & Searching)	115
25.1 Número faltante (Missing Number)	115
25.2 Máximo / Mínimo (Array Max/Min)	115
25.3 Two Pointers	116
25.4 Sliding Window	117
25.5 Par de suma absoluta mínima	117
25.6 Par con diferencia dada (Difference Pair)	118
25.7 Búsqueda ternaria (Ternary Search)	119
25.8 Búsqueda de Fibonacci (Fibonacci Search)	120
25.9 Búsqueda exponencial (Exponential Search)	120
25.10 Búsqueda por saltos (Jump Search)	121
25.11 Meet in the Middle	122
25.12 Heap Sort	123
25.13 Merge Sort	123
25.14 Conteo de Inversiones (Counting Inversions)	124
25.15 QuickSelect	125

26 Búsqueda Binaria (Binary Search)	126
26.1 En arreglo ordenado	126
26.2 Menor x que cumple	127
26.3 Mayor x que cumple	128
26.4 Binary Search on Answer	128
26.5 Sobre reales	130
26.6 Con strings	131
 V Geometría Computacional	 132
26.7 ¿Qué algoritmo usar?	132
 27 Line Sweep	 133
27.1 Idea General y Manejo de Eventos	133
27.2 Intersección de Segmentos	134
27.3 Unión de Áreas de Rectángulos	136
27.4 Skyline Problem	137
27.5 Par de Puntos Más Cercano (Closest Pair)	138
 28 Primitivas Geométricas	 139
28.1 Vector Geométrico: Producto Punto y Producto Cruz	139
28.2 Orientación de Tres Puntos (Cross Product / CCW Test)	140
28.3 Distancia Euclidiana	141
28.4 Punto Medio de un Segmento	141
28.5 Ecuación de una Línea Dados Dos Puntos	142
28.6 División de un Segmento en una Razón Dada	142
28.7 Ángulo entre Dos Vectores	143
28.8 Tres Puntos Colineales	144
28.9 Distancia de un Punto a una Línea/Segmento	145
28.10 Reflexión de un Punto sobre una Línea	146
28.11 Rotación de un Punto (por Ángulo, alrededor de un Pivote)	146
 29 Triángulos	 147
29.1 Ángulos de un Triángulo	147
29.2 Fórmula de Herón (Área por Lados)	148
29.3 Círculo Circunscrito en un Triángulo	149
29.4 Punto Dentro de un Triángulo	149
 30 Círculos	 150
30.1 Longitud de Arco y Área de un Sector Circular	150
30.2 Intersección de Dos Círculos	151
 31 Polígonos	 152
31.1 Área de un Polígono (Shoelace)	152
31.2 Perímetro de un Polígono	153
31.3 Punto Dentro de un Polígono (Ray Casting)	154
31.4 Punto Dentro de un Polígono Convexo (Binary Search)	154
31.5 Teorema de Pick (Área en Malla de Puntos Enteros)	155
 32 Envoltente Convexa	 156
32.1 Algoritmo de Jarvis (Gift Wrapping)	156
32.2 Monotone Chain (Andrew)	157
32.3 Corte de un Polígono Convexo por una Línea	158
 33 Ad-hoc y Combinatoria Geométrica	 159
33.1 Perímetro Usando Bloques (Figuras en Grilla)	159
33.2 Número de Cuadrados en un Rectángulo $N \times M$	160
33.3 Número de Rectángulos en una Grilla $N \times M$	161
33.4 Suma de Distancias Manhattan entre Todos los Pares	162
33.5 Máximo de Cuadrados 2×2 en un Triángulo Isósceles Rectángulo	163
33.6 Máximo Número de Piezas con N Cortes (Pastelero Perezoso)	163
33.7 Altura Máxima de un Triángulo de Monedas	164

VI Programación Dinámica (Dynamic Programming)	165
33.8 ¿Qué algoritmo usar?	165
34 Fundamentos	166
34.1 Memoización vs. Tabulación (Top-Down y Bottom-Up)	166
34.2 Kadane's Algorithm (Máximo Subarreglo)	167
34.3 Triángulo de Pascal	168
34.4 Camino Más Largo en una Matriz	170
35 Knapsack y Variantes	172
35.1 Knapsack 0/1	172
35.2 Knapsack Ilimitado	173
35.3 Cambio de Monedas: Número de Formas (Combinaciones)	174
35.4 Cambio de Monedas: Mínimo de Monedas	176
35.5 Problema de Partición (Diferencia Mínima de Subconjuntos)	177
36 DP sobre Secuencias	178
36.1 Subsecuencia Creciente Más Larga (LIS): referencia cruzada	178
36.2 Subsecuencia Común Más Larga (LCS): referencia cruzada	179
36.3 Subsecuencia Creciente de Suma Máxima	180
37 DP de Intervalos	181
37.1 Burst Balloons (Problema de los Globos)	181
38 Apilamiento y Ordenamiento	183
38.1 Apilamiento de Cajas (Box Stacking)	183
39 DP Avanzada / Estados Especiales	184
39.1 Digit DP	184
39.2 DP de Probabilidad / Valor Esperado	186
39.3 DP con Exponenciación de Matrices (Recurrencias Lineales en $O(\log n)$)	187
39.4 Divide and Conquer Optimization	188
39.5 Convex Hull Trick / Li Chao Tree	189
39.6 Knuth's Optimization	190
VII Algoritmos Voraces (Greedy)	191
39.7 ¿Qué algoritmo usar?	191
40 Fundamentos de Greedy	191
40.1 Exchange Argument (Argumento de Intercambio)	191
40.2 Greedy vs. Programación Dinámica	192
41 Intervalos	194
41.1 Activity Selection (Selección de Actividades)	194
41.2 Interval Point Cover	194
41.3 Merge Intervals	195
41.4 Minimum Number of Platforms/Rooms	196
41.5 Non-overlapping Intervals	197
41.6 Minimum Number of Arrows to Burst Balloons	197
42 Scheduling	198
42.1 Job Sequencing con Deadlines	198
42.2 Task Scheduler con Cooldown	199
42.3 Fractional Knapsack	200
43 Greedy con Heap	201
43.1 Huffman Coding	201
43.2 Reorganize String (Reordenar sin Adyacentes Iguales)	202
43.3 Conectar Cuerdas al Mínimo Costo (Connect Ropes)	203

44 Greedy en Arreglos y Strings	204
44.1 Gas Station Problem	204
44.2 Candy Distribution	204
44.3 Jump Game (Alcanzabilidad Greedy)	205
44.4 Remove K Digits (Stack Monotónico Greedy)	206
44.5 Lexicográficamente Mínimo/Máximo con Restricciones	207
45 Greedy con Matemáticas	208
45.1 Coin Change Greedy (Cuándo Funciona y Cuándo No)	208
45.2 Egyptian Fraction	209
46 Greedy sobre Grafos (Referencias)	210
46.1 Por qué Kruskal, Prim y Dijkstra son Greedy	210
VIII Ad-hoc y Simulación	211
46.2 ¿Qué algoritmo usar?	211
47 Simulación Directa	212
47.1 Plantilla de Simulación (Estado, Paso y Parada)	212
47.2 Simulación con Detección de Ciclos (Floyd y Estado Repetido)	213
47.3 Problema de Josephus	215
47.4 Simulación por Eventos (Cola de Prioridad de Eventos)	217
47.5 Autómatas Celulares (Game of Life)	219
47.6 Simulación de Máquinas e Interpretes de Instrucciones	221
48 Matrices y Grillas	224
48.1 Recorridos Especiales (Espiral, Diagonal, Zigzag)	224
48.2 Rotación, Transposición y Reflexión	226
48.3 Gravedad y Caída de Piezas (Tetris, Candy Crush)	228
48.4 Direcciones y Vecindarios (dx/dy, 8 Direcciones, Límites)	231
49 Fechas y Tiempo	233
49.1 Años Bisiestos y Días de un Mes	233
49.2 Día de la Semana (Zeller / Sakamoto)	234
49.3 Diferencia entre Fechas y Sumar Días	235
49.4 Formatos de Hora (12h/24h, Segundos a HH:MM:SS)	237
50 Aritmética Ad-hoc	238
50.1 Números Grandes (BigInt)	238
50.2 Fracciones Exactas	240
50.3 Diferencia entre Fechas y Sumar Días	242
50.4 Formatos de Hora (12h/24h, Segundos a HH:MM:SS)	243
51 Parsing y Evaluación de Expresiones	244
51.1 Tokenizador	244
51.2 Shunting-Yard y Notación Postfija	246
51.3 Descenso Recursivo	248
52 Técnicas Ad-hoc	251
52.1 Compresión de Coordenadas	251
52.2 Invariantes y Paridad	252
52.3 Algoritmos Constructivos (con Cuadrado Mágico)	254
52.4 Trabajar hacia Atrás (Reverse Thinking)	257
52.5 Contribución de cada Elemento	259
52.6 Fuerza Bruta para Descubrir Patrones	260
53 Herramientas de Práctica	261
53.1 Stress Testing (Generador, Fuerza Bruta y Checker)	261
53.2 Problemas Interactivos	263
IX String Processing	264
53.3 ¿Qué algoritmo usar?	264

54 Fundamentos	265
54.1 Trie	265
54.2 Hash Polinómico (Rolling Hash)	266
54.3 Función de Prefijos (Prefix Function / Failure Function)	267
54.4 Suffix Array + Algoritmo de Kasai (LCP)	268
54.5 Suffix Automaton	269
55 Búsqueda de Patrones	271
55.1 Knuth-Morris-Pratt (KMP)	271
55.2 Algoritmo Z	271
55.3 Rabin-Karp	272
55.4 Boyer-Moore	273
55.5 Autómata Finito de Coincidencia de Patrones	274
55.6 Aho-Corasick (Múltiples Patrones)	276
56 Coincidencia de Patrones Especiales	277
56.1 Patrón con Comodines (Wildcard Matching)	277
56.2 Búsqueda de Anagramas de un Patrón	278
57 Palíndromos	279
57.1 Subsecuencia Palindrómica Más Larga (Longest Palindromic Subsequence) [DP, no String]	279
57.2 Partición Mínima en Substrings Palindrómicos (Palindrome Partitioning)	280
57.3 Conteo Total de Substrings Palindrómicos	281
57.4 Árbol de Palíndromos (Palindromic Tree / Eertree)	282
58 Subsecuencias y Subcadenas Comunes	283
58.1 Subcadena Común Más Larga (Longest Common Substring)	283
58.2 Subsecuencia Común Más Larga (Longest Common Subsequence)	284
58.3 Superseuencia Común Más Corta (Shortest Common Supersequence)	285
58.4 Subsecuencia Creciente Más Larga (LIS) [DP, no String]	286
58.5 Subsecuencia Decreciente Más Larga (LDS) [DP, no String]	287
59 Construcciones y Transformaciones	288
59.1 Rotación Lexicográficamente Mínima (Algoritmo de Booth)	288
59.2 Distancia de Edición (Edit Distance / Levenshtein)	289
59.3 Transformación con Movimientos Restringsidos (A-izquierda, B-derecha, # fijo)	290
59.4 Lyndon Words y Algoritmo de Duval	291
60 Anagramas	292
60.1 Verificar si Dos Cadenas son Anagramas	292
60.2 Búsqueda de Anagramas de un Patrón (Ventana Deslizante)	293
60.3 Agrupar Anagramas (Group Anagrams)	294
60.4 Conteo de Substrings Permutables a Palíndromo	295
60.5 Anagrama Lexicográficamente Mínimo/Máximo (con Restricciones)	296
X BitWise	297
60.6 Casos de Uso	297
60.7 Representación Binaria de Enteros (con y sin signo)	298
60.8 Operaciones y Trucos Básicos	299
60.9 Desplazamientos Aritméticos vs. Lógicos (Arithmetic vs. Logical Shift)	300
60.10 Funciones Built-in de GCC	300
60.11 Conversión entre Bases Numéricas (Binario, Octal, Hexadecimal)	302
60.12 Overflow y Wraparound en Enteros	302
60.13 Operaciones Matemáticas en Binario (Suma, Resta, Multiplicación y Exponenciación Modular)	303
60.14 Bitset para Optimización (std::bitset)	305
60.15 Enumeración de Submáscaras (Submask Enumeration)	306
60.16 Suma sobre Subconjuntos (SOS DP)	307
60.17 Bitmask DP (Programación Dinámica sobre Subconjuntos)	307
60.18 DP con Perfil de Bits (Broken Profile DP)	309
60.19 XOR: Propiedades y Aplicaciones	310
60.20 Base Lineal de XOR (XOR Basis / Linear Basis)	311
60.21 Trie de Bits (Binary Trie / XOR Trie)	312
60.22 Código Gray (Gray Code)	314

60.23	Conteo de Pares con XOR/AND/OR Dado	314
60.24	Conteo de Bits Encendidos en un Rango $[0, N]$	316
60.25	Convolución XOR/AND/OR (Walsh-Hadamard Transform)	316
60.26	Meet in the Middle con Máscaras de Bits	318
60.27	Transformada de Möbius/Zeta sobre Subconjuntos	319
XI	Game Theory	320
60.28	¿Qué algoritmo utilizar?	320
61	Fundamentos	321
61.1	Estados Ganadores y Perdedores (Win/Lose States)	321
61.2	Juegos Imparciales vs. Partidistas	322
61.3	Juegos con Información Perfecta y sin Azar	323
61.4	Análisis Retrógrado (Backward Induction)	323
62	Nim y Teoría de Sprague-Grundy	325
62.1	Nim Clásico (Bouton's Theorem)	325
62.2	Nim con Montones y Movimientos Restringidos	326
62.3	Teorema de Sprague-Grundy (Grundy Numbers / Nimbers)	327
62.4	Suma de Juegos Independientes (XOR de Grundy Numbers)	328
62.5	Cálculo de Grundy sobre un DAG (mex de Sucesores)	329
62.6	Misère Nim (Última Jugada Pierde)	330
63	Juegos de Sustracción y Variantes Clásicas	332
63.1	Juego de Sustracción (Subtraction Game)	332
63.2	Juego de Sustracción con Conjunto de Movimientos Prohibidos	332
63.3	Staircase Nim	333
63.4	Wythoff's Game	334
63.5	Turning Games (Volteo de Monedas / Fichas)	335
64	Juegos sobre Estructuras de Datos	336
64.1	Juegos sobre Grafos y DAGs (Movimiento de una Ficha)	336
64.2	Juegos sobre Árboles (Green Hackenbush / Poda de Ramas)	337
64.3	Juegos de Tomar de los Extremos de un Arreglo (DP de Intervalos)	339
64.4	Juegos sobre Strings (Eliminar Substrings o Caracteres)	340
65	Juegos Partidistas y Combinatoria Avanzada	341
65.1	Números Surreales y Valores de Juego (Cold, Hot, Fuzzy)	342
65.2	Suma de Juegos Partidistas y Comparación de Valores	343
65.3	Juegos de Turnos con Puntaje (Optimal Strategy for a Game)	344
66	Estrategia Óptima y Técnicas de Resolución	345
66.1	Minimax con Memoización	345
66.2	Poda Alfa-Beta	347
66.3	Detección de Patrones con Fuerza Bruta (Tabla de Grundy)	348
66.4	Juegos con Múltiples Jugadores (más de 2)	350

Parte I Introduccion

1 Fundamentos Básicos

1.1 Entrada / Salida Rápida (Fast I/O)

Usar lectura rápida para evitar TLE, hay 3 niveles de lectura rápida, estandarizar Tier 2 y usar Tier 3 (lector por bytes) solo cuando haya del orden de 10^6 – 10^7 lecturas.

Table 1: Casos donde se recomienda Fast I/O

Tipo de problema	Ejemplo
Arrays gigantes	Leer/ordenar/sumar 10^6 – 10^7 enteros
Muchos test cases (T grande)	$T \leq 10^5$, cada uno trivial pero se acumula
Grafos grandes	$M \sim 10^6$ – $2 \cdot 10^6$ aristas para BFS/DFS/Dijkstra/MST
Geometria con muchas consultas	L, S grandes + muchos test cases seguidos hasta EOF
Muchos tokens de texto	Strings/lineas que suman millones de caracteres
Judges clasicos "trampa de I/O"	SPOJ INTEST, ARRAY-SUB, etc.

Regla practica: si N/M es del orden de 10^4 – 10^5 , cin con `sync_with_stdio(false); cin.tie(nullptr);` alcanza de sobra. El FastReader es la "artilleria pesada" para cuando eso deja de ser suficiente.

Tier 1 — Básico: cin / cout.

```
1 int entero; double decimal; string palabra, linea;
2 cin >> entero >> decimal >> palabra; // >> lee UN token
3 // ! cin >> deja el '\n' pendiente;
4 // ! el 1er getline sale VACÍO -> hay que ignorarlo
5 cin.ignore();
6 getline(cin, linea); // línea completa
```

Tier 2 — Rápido: desactiva la sync con C (una vez en main). Punto dulce para casi todo.

```
1 ios_base::sync_with_stdio(false);
2 cin.tie(nullptr); // ! no mezclar con scanf/printf
3 cout << entero << "\n"; // "\n", nunca endl (endl frena)
```

Tier 3 — Más rápido: lector por bytes con fread. Solo con 10^6 – 10^7 lecturas.

```
1 struct FastReader {
2     static const int TAM = 1 << 16; // 64 KB
3     char bufer[TAM];
4     int puntero = 0, leidos = 0;
5
6     int leer() {
7         if (puntero == leidos) {
8             leidos = (int) fread(bufer, 1, TAM, stdin);
9             puntero = 0;
10        }
11        return leidos == 0 ? -1 : bufer[puntero++]; // -1=EOF
12    }
13
14    int nextInt() {
15        int resultado = 0, b = leer();
16        while (b <= ' ') b = leer(); // saltar blancos
17        bool negativo = (b == '-');
18        if (negativo) b = leer();
19        while (b >= '0' && b <= '9') {
20            resultado = resultado * 10 + (b - '0'); b = leer();
21        }
22        return negativo ? -resultado : resultado;
23    }
24 }
```

```
23 long long nextLong() {
24     long long resultado = 0; int b = leer();
25     while (b <= ' ') b = leer();
26     bool negativo = (b == '-');
27     if (negativo) b = leer();
28     while (b >= '0' && b <= '9') {
29         resultado = resultado * 10 + (b - '0'); b = leer();
30     }
31     return negativo ? -resultado : resultado;
32 }
33
34 double nextDouble() {
35     double resultado = 0, divisor = 1; int b = leer();
36     while (b <= ' ') b = leer();
37     bool negativo = (b == '-');
38     if (negativo) b = leer();
39     while (b >= '0' && b <= '9') {
40         resultado = resultado * 10 + (b - '0'); b = leer();
41     }
42     if (b == '.') {
43         b = leer();
44         while (b >= '0' && b <= '9') {
45             resultado += (b - '0') / (divisor *= 10); b = leer();
46         }
47     }
48     return negativo ? -resultado : resultado;
49 }
50
51 string next() { // un token
52     string s; int b = leer();
53     while (b <= ' ') b = leer();
54     while (b > ' ') { s += (char) b; b = leer(); }
55     return s;
56 }
57
58 string nextLine() { // línea completa
59     string s; int b = leer();
60     while (b != '\n' && b != -1) {
61         if (b != '\r') s += (char) b; b = leer();
62     }
63     return s;
64 }
65 }
```

1.2 Formateo de Salida

La función `printf` utiliza una cadena de formato para indicar cómo debe interpretarse y mostrarse cada argumento. Cada especificador comienza con `%` y corresponde al argumento que aparece después de la cadena.

```
1 // %d -> entero (int)
2 printf("%d\n", 42);
3 // Entrada: 42
4 // Salida: 42
5
6 // %lld -> entero largo (long long)
7 long long x = 1234567890123LL;
8 printf("%lld\n", x);
9 // Entrada: 1234567890123
10 // Salida: 1234567890123
11
12 // %5d -> ancho minimo de 5 caracteres, alineado a la derecha
13 printf("%5d\n", 42);
14 // Entrada: 42
15 // Salida: "  42"
16
17 // %-5d -> ancho minimo de 5 caracteres, alineado a la izquierda
18 printf("%-5d\n", 42);
19 // Entrada: 42
20 // Salida: "42  "
21
22 // %05d -> ancho 5, completando con ceros
23 printf("%05d\n", 42);
24 // Entrada: 42
25 // Salida: "00042"
26
27 // %+d -> muestra siempre el signo
28 printf("%+d\n", 42);
29 // Entrada: 42
30 // Salida: "+42"
31
32 // %x -> hexadecimal usando letras minúsculas
33 printf("%x\n", 255);
34 // Entrada: 255
35 // Salida: ff
36
37 // %X -> hexadecimal usando letras mayúsculas
38 printf("%X\n", 255);
39 // Entrada: 255
40 // Salida: FF
41 }
```



```

42 // %.2f -> numero real con exactamente 2 decimales
43 printf("%.2f\n", 3.14159);
44 // Entrada: 3.14159
45 // Salida: 3.14
46
47 // %.0f -> numero real sin decimales (redondea)
48 printf("%.0f\n", 3.7);
49 // Entrada: 3.7
50 // Salida: 4
51
52 // %e -> notacion cientifica
53 printf("%e\n", 31415.9);
54 // Entrada: 31415.9
55 // Salida: 3.141590e+04
56
57 // %c -> un solo caracter
58 printf("%c\n", 'A');
59 // Entrada: 'A'
60 // Salida: A
61
62 // %s -> cadena de caracteres
63 printf("%s\n", "hola");
64 // Entrada: "hola"
65 // Salida: hola
66
67 // Se pueden combinar varios especificadores
68 printf("%d %lld %.2f\n", 42, 10000000000LL, 3.14159);
69 // Entrada: 42, 10000000000, 3.14159
70 // Salida: 42 10000000000 3.14
71
72 // %% -> imprime un '%' literal
73 printf("100%%\n");
74 // Entrada: no recibe ningun argumento para %
75 // Salida: 100%
76
77 // ! NO uses %n en C/C++ (puede ser peligroso)
78 // Para scanf, un double se lee con %lf

```

Estructura de un especificador

Un formato puede combinar varias partes:

```

1 // % [flags] [width] [.precision] [specifier]
2 // % 0 5 .2 f
3
4 printf("%05d\n", 42);
5 // % -> comienza el formato
6 // 0 -> rellena con ceros
7 // 5 -> ancho minimo de 5 caracteres
8 // d -> interpreta el argumento como int
9 //
10 // Salida: 00042

```

Los modificadores más utilizados en programación competitiva son:

Formato	Tipo	Ejemplo de salida
%d	int	42
%lld	long long	1234567890123
%c	char	A
%s	string/char*	hola
%f	double	3.140000
%.2f	double	3.14
%x	int	ff
%X	int	FF

Regla práctica: en programación competitiva, los formatos más importantes son %d para int, %lld para long long, %c para caracteres, %s para cadenas y %.2f (o una precisión diferente) para números reales.

1.3 Condicionales y Ciclos

```

1 // if / else: ejecuta un bloque segun una condicion
2 if (x > 0) cout << "pos"; // x > 0 -> "pos"
3 else if (x < 0) cout << "neg"; // x < 0 -> "neg"
4 else cout << "cero"; // x == 0 -> "cero"
5
6 // Operador ternario: condicion ? valor_si_true : valor_si_false
7 string s = (x >= 0) ? "no neg" : "neg";
8 // Si x >= 0, s = "no neg"; si no, s = "neg"

```

```

9
10 // switch: compara una expresion con diferentes valores
11 // Solo funciona directamente con tipos integrales como int o
    char,
12 // NO con string.
13 switch (x) {
14     case 1: /*...*/ break; // x == 1
15     case 2: case 3: /*...*/ break; // x == 2 o x == 3
16     default: /*...*/; // ningun case coincide
17 }
18 // ! Sin break, se continua ejecutando el siguiente case (fall-through)
19
20 // C++17: se puede declarar una variable dentro del if
21 if (int r = x % 2; r == 0) {}
22 // r solo existe dentro del if y sus bloques asociados
23
24 // for clasico: usa un indice y se repite mientras i < n
25 for (int i = 0; i < n; i++) {}
26
27 // range-based for: recorre directamente los elementos
28 // No proporciona el indice.
29 for (int v : datos) {}
30
31 // & obtiene una referencia al elemento original.
32 // Por tanto, las modificaciones afectan al vector.
33 for (int &v : datos) v *= 2;
34
35 // while: ejecuta mientras la condicion sea verdadera
36 while (c < n) c++;
37
38 // do-while: primero ejecuta el bloque y DESPUES verifica la
    condicion.
39 // Por eso se ejecuta AL MENOS una vez.
40 do {
41     c--;
42 } while (c > 0);
43
44 // break -> termina inmediatamente el ciclo o switch
45 // continue -> salta a la siguiente iteracion del ciclo
46
47 // ! C++ NO tiene break con etiqueta.
48 // Para salir de ciclos anidados se puede usar una bandera,
49 // encapsular la logica en una funcion y usar return, etc.

```

1.4 Conversiones, Parseos y Casteos

```

1 // string -> numero (parseo)
2 // Convierte una cadena de texto en el tipo numerico indicado.
3 int e = stoi("42");
4 // Entrada: "42" -> Salida: 42
5
6 long long l = stoll("10000000000");
7 // Entrada: "10000000000" -> Salida: 10000000000
8
9 double d = stod("3.14");
10 // Entrada: "3.14" -> Salida: 3.14
11
12 // Tambien se puede indicar la base numerica.
13 int bin = stoi("1010", nullptr, 2);
14 // "1010" en base 2 -> 10
15
16 int hex = stoi("ff", nullptr, 16);
17 // "ff" en base 16 -> 255
18
19
20 // numero -> string
21 // Convierte cualquier numero compatible en texto.
22 string s = to_string(42);
23 // Entrada: 42 -> Salida: "42"
24
25
26 // Casteo: fuerza la conversion de un tipo a otro.
27 // Al pasar de decimal a entero, se TRUNCa hacia 0.
28 int t = (int) 3.99;
29 // 3.99 -> 3
30
31 int r = static_cast<int>(3.99);
32 // 3.99 -> 3
33 // static_cast<int> es la forma recomendada en C++
34
35
36 // ! int / int = division ENTERA
37 double mal = 7 / 2;
38 // 7 y 2 son int -> 3
39 // Luego 3 se convierte a double -> 3.0
40
41 double bien = 7 / 2.0;
42 // 2.0 es double -> division decimal
43 // 7 / 2.0 -> 3.5
44

```

```

45 // caracteres
46 // Los char tienen un valor numerico asociado (ASCII).
47 int cod = (int) 'A';
48 // 'A' -> 65
49
50
51 char ch = (char) 66;
52 // 66 -> 'B'
53
54 // Truco muy usado para convertir un digito char a int.
55 int dig = '7' - '0';
56 // '7' tiene valor 55 y '0' tiene valor 48
57 // 55 - 48 -> 7

```

Parte II Estructuras de Datos

2 Basic Structures

```

1 int a[5] = {1,2,3,4,5}; // crear con valores: O(n)
2
3 int b[5];                // crear de tamaño 5
4                          // posiciones: b[0] ... b[4]
5                          // ! Valores no inicializados
6
7 b[0] = 10;               // INSERT: O(1)
8 b[1] = 20;               // INSERT: O(1)
9
10 int x = b[0];            // READ: O(1)
11
12 b[0] = 30;               // UPDATE: O(1)
13
14 for (int i = 0; i < 5; i++)
15     cout << b[i] << " "; // READ: recorrer -> O(n)
16
17 // DELETE: no existe metodo para eliminar
18 // Se puede sobrescribir o manejar logicamente
19 b[0] = 0;                // DELETE logico: O(1)
20
21 // ! El tamaño de un array estatico no cambia.
22 // Para mas posiciones se necesita otro arreglo.

```

2.1 Matrices (Arreglos 2D)

Arreglo bidimensional (filas $\times$ columnas). **Complejidad:** acceso $[i][j]$ $O(1)$; crear $n \times m$ en $O(nm)$.

```

1 vector<vector<int>> m(3, vector<int>(4,0)); // CREATE: O(n*m)
2 // matriz 3x4, todas las posiciones inicializadas en 0
3
4 m[1][2] = 7;             // INSERT: O(1)
5
6 int x = m[1][2];         // READ: O(1)
7
8 m[1][2] = 10;            // UPDATE: O(1)
9
10 for (int i = 0; i < 3; i++)
11     for (int j = 0; j < 4; j++)
12         cout << m[i][j] << " ";
13 // READ: recorrer toda la matriz -> O(n*m)
14
15 // DELETE: no existe metodo para eliminar una posicion.
16 // Se puede sobrescribir el valor o manejarlo logicamente.
17 m[1][2] = 0;             // DELETE logico: O(1)
18
19 // ! El tamaño de la matriz puede cambiar con vector,
20 // pero un arreglo 2D estatico no puede cambiar de tamaño.

```

2.2 Vectores / Listas dinámicas

Arreglo que **crece** solo (ArrayList / vector / list). **Complejidad:** acceso por índice $O(1)$; agregar al final $O(1)$ amortizado; insertar/borrar en medio $O(n)$; buscar $O(n)$.

```

1 vector<int> v;            // CREATE: lista vacia
2
3 v.push_back(3);          // INSERT: final -> O(1) amortizado
4 v.push_back(7);          // INSERT: final -> O(1) amortizado
5
6 int x = v[0];            // READ: acceso -> O(1)

```

```

7 int n = v.size();        // READ: tamaño -> O(1)
8
9 v[0] = 10;               // UPDATE: modificar -> O(1)
10
11 v.insert(v.begin() + 1, 5); // INSERT: medio -> O(n)
12
13 v.erase(v.begin() + 1);  // DELETE: medio -> O(n)
14
15 // Buscar un elemento recorriendo el vector
16 for (int x : v)
17     if (x == 10) { /* ... */ }
18 // SEARCH: recorrer -> O(n)
19
20 // DELETE: eliminar el ultimo elemento
21 v.pop_back();            // DELETE: final -> O(1)
22
23 // ! vector mantiene acceso por índice O(1).
24 // ! Al crecer puede realocar su memoria internamente.

```

2.3 Listas enlazadas (LinkedList)

Nodos enlazados. **Complejidad:** insertar/eliminar en los **extremos** $O(1)$; acceso por índice $O(n)$; buscar $O(n)$. Útil como cola o deque.

```

1 list<int> l;              // CREATE: lista vacia
2
3 l.push_front(1);          // INSERT: inicio -> O(1)
4 l.push_back(9);           // INSERT: final -> O(1)
5
6 int x = l.front();        // READ: primer elemento -> O(1)
7 int y = l.back();         // READ: ultimo elemento -> O(1)
8
9 l.front() = 5;            // UPDATE: primer elemento -> O(1)
10 l.back() = 10;           // UPDATE: ultimo elemento -> O(1)
11
12 l.pop_front();            // DELETE: inicio -> O(1)
13 l.pop_back();             // DELETE: final -> O(1)
14
15 // ! No existe acceso directo por índice.
16 // Avanzar hasta una posición requiere recorrer nodos.
17 auto it = l.begin();
18 advance(it, 2);           // acceso al tercer elemento -> O(n)
19
20 // Buscar un elemento requiere recorrer la lista.
21 for (int x : l)
22     if (x == 10) { /* ... */ }
23 // SEARCH: recorrer -> O(n)
24
25 // ! list no permite l[i].
26 // Para acceso por índice, vector suele ser mejor.

```

2.4 Pilas (Stack)

Estructura **LIFO** (último en entrar, primero en salir). **Complejidad:** insertar, eliminar y consultar el tope $O(1)$. Para DFS iterativo, paréntesis balanceados y deshacer.

```

1 stack<int> pila;          // CREATE: pila vacia
2
3 pila.push(3);             // INSERT: O(1)
4 pila.push(7);             // INSERT: O(1)
5
6 int tope = pila.top();    // READ: consultar tope -> O(1)
7
8 pila.top() = 10;          // UPDATE: modificar tope -> O(1)
9
10 pila.pop();               // DELETE: eliminar tope -> O(1)
11
12 // ! Solo se puede acceder directamente al elemento superior.
13 // ! No existe acceso por índice.

```

2.5 Colas (Queue)

Estructura **FIFO** (primero en entrar, primero en salir). **Complejidad:** encolar y desencolar $O(1)$. Base del BFS.

```

1 queue<int> cola;          // CREATE: cola vacia
2
3 cola.push(3);             // INSERT: final -> O(1)

```



```

4 cola.push(7);           // INSERT: final -> O(1)
5
6 int frente = cola.front(); // READ: primero -> O(1)
7 int atrás = cola.back();  // READ: último -> O(1)
8
9 cola.front() = 10;       // UPDATE: modificar primero -> O(1)
10
11 cola.pop();             // DELETE: primero -> O(1)
12
13 // ! push agrega al final y pop elimina del frente.
14 // ! No existe acceso directo por índice.

```

2.6 Colas de prioridad (Priority Queue / Heap)

Devuelve siempre el elemento de mayor (o menor) prioridad. **Complejidad:** insertar y extraer $O(\log n)$; consultar el tope $O(1)$. Para Dijkstra, Prim, k-ésimo mayor.

```

1 priority_queue<int> pq; // CREATE: MAX-heap por defecto
2
3 pq.push(3);           // INSERT: O(log n)
4 pq.push(7);           // INSERT: O(log n)
5
6 int mx = pq.top();     // READ: mayor -> O(1)
7
8 pq.pop();             // DELETE: mayor -> O(log n)
9
10 // min-heap: el menor queda en el tope
11 priority_queue<int, vector<int>, greater<int>> pqMin;
12
13 pqMin.push(3);        // INSERT: O(log n)
14 int mn = pqMin.top(); // READ: menor -> O(1)
15 pqMin.pop();          // DELETE: O(log n)
16
17 // ! No existe acceso directo a elementos intermedios.
18 // ! priority_queue no tiene una operacion UPDATE directa.

```

2.7 Comparadores en C++ (Comparators)

Idea: un comparador es una función (lambda, functor, o `operator<` sobrecargado) que le dice a `sort`, `set`, `map`, `priority_queue`, etc. **cómo ordenar** dos elementos. Debe cumplir la propiedad de *orden estricto débil* (*strict weak ordering*): devolver `true` si el primer argumento va **estrictamente antes** que el segundo, y `false` en cualquier otro caso —incluyendo cuando se consideran “iguales” para efectos de orden. El error más común es usar `<=` en vez de `<`: eso rompe la propiedad y produce comportamiento indefinido (desde resultados mal ordenados hasta crashes).

¿Por qué usarlo? El orden por defecto casi nunca alcanza: necesitas ordenar por varios criterios a la vez, en orden descendente, por una clave derivada (no el valor mismo), o con un `struct`/`pair` propio que no debe usar el orden lexicográfico por defecto.

Casos de uso:

- Ordenar por múltiples campos (ej. por costo, y en empate por índice original).
- `priority_queue` como min-heap en vez de max-heap (o con un orden custom sobre un `struct`).
- `set`/`map` con orden propio sobre un tipo definido por ti.
- Ordenar **índices** según los valores de otro arreglo, sin mover el arreglo original.

Complejidad: el comparador en sí no cambia la complejidad del algoritmo que lo usa (`sort` sigue siendo $O(n \log n)$); pero un comparador que no sea un orden estricto débil puede producir UB o bucles infinitos.

Comparador con lambda para sort (multi-criterio, índices)

```

1 struct Persona {
2     string nombre;
3     int edad;
4 };
5
6 vector<Persona> personas = {
7     {"Ana", 30}, {"Luis", 25}, {"Eva", 25}
8 };
9
10 // ordena por edad ascendente; en empate, por nombre alfabético
11 sort(personas.begin(), personas.end(),
12     [](const Persona& a, const Persona& b) {
13         if (a.edad != b.edad)
14             return a.edad < b.edad;
15         return a.nombre < b.nombre;
16     });
17 // resultado: Eva(25), Luis(25), Ana(30)
18
19 // ordenar en orden DESCENDENTE: solo invierte la comparación
20 sort(personas.begin(), personas.end(),
21     [](const Persona& a, const Persona& b) {
22         return a.edad > b.edad;
23     });
24 // resultado: Ana(30), luego Luis/Eva(25) en su orden relativo
    previo

```

Ordenar índices en vez del arreglo (muy común):

```

1 vector<int> valores = {40, 10, 30, 20};
2
3 vector<int> idx(valores.size());
4 iota(idx.begin(), idx.end(), 0); // idx = [0, 1, 2, 3]
5
6 sort(idx.begin(), idx.end(),
7     [&](int a, int b) { return valores[a] < valores[b]; });
8 // idx queda [1, 3, 2, 0]
9 // (valores[1]=10, valores[3]=20, valores[2]=30, valores[0]=40)
10
11 for (int i : idx)
12     cout << valores[i] << " ";
13 // imprime: 10 20 30 40
14 // sin haber movido "valores" del lugar

```

operator< dentro de un struct (orden por defecto)

Idea: si sobrecargas `operator<` dentro de tu propio tipo, ese es el orden que usan automáticamente `sort`, `set`, `map` (como clave) y `priority_queue` **sin pasar ningún comparador extra**.

```

1 struct Punto {
2     int x, y;
3
4     bool operator<(const Punto& otro) const {
5         if (x != otro.x) return x < otro.x;
6         return y < otro.y;
7     }
8 };
9
10 set<Punto> s;
11 s.insert({3, 4});
12 s.insert({1, 2});
13 s.insert({1, 5});
14
15 for (auto& p : s)
16     cout << "(" << p.x << ", " << p.y << ") ";
17 // imprime en orden: (1,2) (1,5) (3,4)
18 // -> set los ordena automáticamente usando operator<

```

priority_queue con comparador custom

Idea: `priority_queue` es un **max-heap** por defecto (el elemento “mayor” según el comparador queda arriba). Para un min-heap con tipos simples, se usa `greater<T>`. Para un `struct` propio con una regla de prioridad distinta a `operator<`, se pasa un **functor** (`struct` con `operator()`).

```

1 // min-heap de enteros: el menor queda arriba
2 priority_queue<int, vector<int>, greater<int>> minHeap;
3 minHeap.push(5);
4 minHeap.push(1);
5 minHeap.push(3);
6
7 cout << minHeap.top();
8 // 1 (el menor, no el mayor)

```

```

1 struct Persona {
2     string nombre;
3     int edad;
4 };
5
6 // functor: queda arriba la persona con MENOR edad
7 // (priority_queue es max-heap, así que se invierte el sentido:
8 // operator() debe devolver true si "a" tiene MENOR prioridad que
9 // "b")
10 struct CompararPorEdad {
11     bool operator()(const Persona& a, const Persona& b) const {
12         return a.edad > b.edad;
13     }
14 };
15 priority_queue<Persona, vector<Persona>, CompararPorEdad> pq;
16 pq.push({"Ana", 30});
17 pq.push({"Luis", 20});
18 pq.push({"Eva", 25});
19
20 cout << pq.top().nombre;
21 // "Luis" (la persona mas joven queda en la cima)

```

Reglas de orden estricto débil (*strict weak ordering*)

Todo comparador `cmp(a, b)` debe cumplir:

- **Irreflexividad:** `cmp(a, a)` siempre `false`. Un elemento nunca va estrictamente antes que sí mismo.
- **Asimetría:** si `cmp(a, b)` es `true`, entonces `cmp(b, a)` debe ser `false`.
- **Transitividad:** si `cmp(a, b)` y `cmp(b, c)` son `true`, entonces `cmp(a, c)` también debe serlo.
- **Consistencia en los empates:** si ni `cmp(a, b)` ni `cmp(b, a)` son `true`, se consideran “equivalentes” para efectos de orden (aunque no sean idénticos como objetos).

El error más común:

```

1 // INCORRECTO: usa <= en vez de <
2 sort(v.begin(), v.end(), [](int a, int b) {
3     return a <= b; // viola irreflexividad: cmp(a,a) da true
4 });
5 // esto es Undefined Behavior: en el mejor caso ordena mal,
6 // en el peor caso crashea o hace un bucle infinito.
7
8 // CORRECTO:
9 sort(v.begin(), v.end(), [](int a, int b) {
10     return a < b;
11 });

```

Otro error común: comparadores que no son transitivos al combinar varios criterios sin desempate consistente (ej. comparar solo por `a.x < b.x` cuando a veces `a.x == b.x` pero querías tratar esos casos con otro criterio). Si dos elementos “empatan” en el primer criterio, siempre hay que **decidir** con un segundo criterio, o dejarlos expresamente como equivalentes.

Regla práctica: si tu comparador tiene la forma `return a.campo1 < b.campo1;` y sospechas que puede haber empates que te importan, agrega el siguiente criterio explícitamente (`if (a.campo1 != b.campo1) return ...; return a.campo2 < b.campo2;`) en vez

de dejar el desempate al azar del algoritmo de ordenamiento (que no es estable a menos que uses `stable_sort`).

2.8 Vector circular (Circular Buffer)

Idea: un arreglo de tamaño fijo (**cap**) que se comporta como una **cola doble**. En lugar de mover elementos cuando se inserta o elimina, se reutilizan las posiciones del arreglo de forma circular.

Se mantienen tres variables principales:

- **cap:** capacidad total del arreglo.
- **ini:** posición física donde comienza el primer elemento.
- **tam:** cantidad actual de elementos almacenados.

La posición lógica i no coincide necesariamente con la posición física del arreglo. Se obtiene mediante:

$$\text{posicionFisica}(i) = (\text{ini} + i) \% \text{cap}$$

Por ejemplo, con `cap = 5` y `ini = 3`:

Índice lógico	0	1	2	3	4
Índice físico	3	4	0	1	2

Cuando se llega al final del arreglo, el módulo hace que la siguiente posición vuelva al comienzo. Por eso **no es necesario mover los elementos**.

¿Por qué usarlo? En un arreglo normal, insertar al frente requiere desplazar todos los elementos y cuesta $O(n)$. En un vector circular solo se mueve `ini`, por lo que cuesta $O(1)$.

Casos de uso:

- Colas de tamaño fijo.
- Buffers de audio, video o red.
- Ventanas deslizantes.
- Sistemas donde llegan y salen datos continuamente.
- Simulaciones de procesos en una cola.
- Mantener únicamente los últimos k elementos.

Si el buffer está lleno y se ejecuta `pushBack`, el nuevo elemento **reemplaza al más antiguo**. Esto permite mantener automáticamente una ventana de tamaño fijo.

Complejidad: insertar, eliminar, consultar extremos, acceder por posición y comprobar el tamaño son $O(1)$.

```

1 struct VectorCircular {
2     vector<int> buf;
3     int cap, ini = 0, tam = 0;
4
5     VectorCircular(int c) : buf(c), cap(c) {}
6
7     // CREATE: crea un buffer con capacidad fija.
8     // Los elementos inicialmente no están ocupados.
9
10    bool empty() {
11        return tam == 0;
12    } // O(1)
13
14    bool full() {
15        return tam == cap;
16    } // O(1)
17

```

```

18 int size() {
19     return tam;
20 } // O(1)
21
22 int capacity() {
23     return cap;
24 } // O(1)
25
26 // Convierte una posicion logica en fisica.
27 int pos(int i) {
28     return (ini + i) % cap;
29 } // O(1)
30
31 // INSERT: agrega al final.
32 void pushBack(int x) {
33     buf[(ini + tam) % cap] = x;
34
35     if (tam < cap)
36         tam++;
37     else
38         ini = (ini + 1) % cap;
39 } // O(1)
40
41 // INSERT: agrega al inicio.
42 void pushFront(int x) {
43     ini = (ini - 1 + cap) % cap;
44     buf[ini] = x;
45
46     if (tam < cap)
47         tam++;
48 } // O(1)
49
50 // READ: consulta el primer elemento.
51 int front() {
52     return buf[ini];
53 } // O(1)
54
55 // READ: consulta el ultimo elemento.
56 int back() {
57     return buf[(ini + tam - 1) % cap];
58 } // O(1)
59
60 // READ: acceso por posicion logica.
61 int at(int i) {
62     return buf[(ini + i) % cap];
63 } // O(1)
64
65 // UPDATE: modifica un elemento.
66 void set(int i, int x) {
67     buf[(ini + i) % cap] = x;
68 } // O(1)
69
70 // DELETE: elimina el primer elemento.
71 int popFront() {
72     int v = buf[ini];
73     ini = (ini + 1) % cap;
74     tam--;
75     return v;
76 } // O(1)
77
78 // DELETE: elimina el ultimo elemento.
79 int popBack() {
80     tam--;
81     return buf[(ini + tam) % cap];
82 } // O(1)
83
84 // DELETE: elimina todos los elementos.
85 void clear() {
86     ini = 0;
87     tam = 0;
88 } // O(1)
89
90 // SEARCH: busca un elemento.
91 // A diferencia de las operaciones anteriores,
92 // hay que recorrer los elementos.
93 bool contains(int x) {
94     for (int i = 0; i < tam; i++)
95         if (at(i) == x)
96             return true;
97
98     return false;
99 } // O(n)
100
101 // READ: recorrer los elementos en orden logico.
102 void print() {
103     for (int i = 0; i < tam; i++)
104         cout << at(i) << " ";
105 } // O(n)
106 };

```

Ejemplo de funcionamiento:

```
1 VectorCircular v(5);
```

```

2 v.pushBack(10);
3 v.pushBack(20);
4 v.pushBack(30);
5 // [10, 20, 30, _, _]
6 // ini = 0, tam = 3
7
8 v.pushFront(5);
9 // [5, 10, 20, 30, _]
10 // ini cambia de 0 a 4
11
12 cout << v.front();
13 // 5
14
15 cout << v.back();
16 // 30
17
18 v.popFront();
19 // elimina 5
20 // [_ , 10, 20, 30, _]
21
22 v.pushBack(40);
23 v.pushBack(50);
24 // [_ , 10, 20, 30, 40, 50]
25 // El almacenamiento puede haber dado la vuelta.
26
27 v.pushBack(60);
28 // El buffer estaba lleno.
29 // Se agrega 60 y se elimina el mas antiguo.
30 // Resultado logico: [20, 30, 40, 50, 60]
31

```

Ventaja principal: el arreglo físico puede verse “desordenado”, pero los elementos siempre tienen un orden lógico determinado por `ini`. Por ejemplo:

```

1 // Memoria fisica:
2 // [40, 50, 20, 30, _]
3 //      ^
4 //      ini = 2
5
6 // Orden logico:
7 // 20 -> 30 -> 40 -> 50

```

No importa dónde estén físicamente los elementos: `at(0)` siempre representa el primero, `at(1)` el segundo, y así sucesivamente.

Diferencia con un vector normal:

Operación	vector	Circular
Agregar al final	$O(1)^*$	$O(1)$
Eliminar al final	$O(1)$	$O(1)$
Agregar al frente	$O(n)$	$O(1)$
Eliminar al frente	$O(n)$	$O(1)$
Acceso por índice	$O(1)$	$O(1)$
Buscar	$O(n)$	$O(n)$

*Amortizado.

Regla práctica: si necesitas una estructura de tamaño fijo donde los elementos entran y salen continuamente por los extremos, un vector circular evita los desplazamientos de un arreglo normal y mantiene las operaciones principales en $O(1)$.

3 Estructuras Hash/Tree

3.1 Conjuntos hash (HashSet)

Colección de elementos **sin duplicados** y sin orden garantizado. En C++ se implementa mediante `unordered_set`. Utiliza una tabla hash, por lo que las operaciones son $O(1)$ en promedio. Es útil cuando solo interesa saber si un elemento existe, sin necesidad de mantener los elementos ordenados.

Complejidad: insertar, buscar y borrar $O(1)$ promedio ($O(n)$ peor caso).

```

1 unordered_set<int> s;           // CREATE: conjunto vacio
2
3 s.insert(3);                   // INSERT: O(1) prom.
4 s.insert(7);                   // INSERT: O(1) prom.
5 s.insert(3);                   // no se agrega: ya existe
6
7 bool hay = s.count(3);         // SEARCH: O(1) prom.
8 // hay = 1 si existe, 0 si no existe
9
10 auto it = s.find(7);           // SEARCH: O(1) prom.
11 // devuelve iterador al elemento o s.end()
12
13 s.erase(3);                   // DELETE: O(1) prom.
14
15 int n = s.size();              // READ: cantidad -> O(1)
16 bool vacio = s.empty();        // READ: ¿esta vacio? -> O(1)
17
18 // Recorrer todos los elementos
19 for (int x : s)
20     cout << x << " ";
21 // O(n), pero el orden NO esta garantizado.
22
23 // ! No existe acceso por indice: s[0] no es valido.
24 // ! Los elementos repetidos se ignoran.
25 // ! El orden puede cambiar entre ejecuciones.

```

Cuándo usarlo:

- Saber rápidamente si un elemento ya apareció.
- Eliminar duplicados.
- Mantener un conjunto de elementos visitados.
- Detectar elementos repetidos en $O(n)$ promedio.
- BFS/DFS: guardar vértices visitados cuando sea conveniente.

3.2 Mapas / Diccionarios hash

Estructura que almacena pares **clave** $\rightarrow$ **valor**. En C++ se implementa mediante `unordered_map`. Cada clave identifica un valor y **no puede repetirse**. Si se asigna un nuevo valor a una clave existente, se reemplaza el anterior.

Complejidad: insertar, acceder, buscar y borrar por clave $O(1)$ promedio ($O(n)$ peor caso).

```

1 unordered_map<string,int> m;    // CREATE: mapa vacio
2
3 m["a"] = 1;                     // INSERT: O(1) prom.
4 m["b"] = 2;                     // INSERT: O(1) prom.
5
6 int v = m["a"];                 // READ: acceder -> O(1) prom.
7 // v = 1
8
9 m["a"] = 10;                   // UPDATE: O(1) prom.
10 // la clave "a" ahora tiene valor 10
11
12 m.erase("a");                  // DELETE: O(1) prom.
13
14 bool existe = m.count("b");     // SEARCH: O(1) prom.
15 // existe = 1
16
17 auto it = m.find("b");          // SEARCH: O(1) prom.
18 if (it != m.end())
19     cout << it->second;         // valor asociado a "b"
20
21 // Recorrer todas las parejas
22 for (auto [clave, valor] : m)
23     cout << clave << " " << valor;
24 // O(n), sin orden garantizado.

```

Importante: usar `m[clave]` para consultar una clave que no existe puede **crear esa clave** con su valor por defecto. Para comprobar existencia sin insertar, es preferible usar `count()` o `find()`.

```

1 unordered_map<string,int> m;
2
3 int x = m["a"];
4 // Si "a" no existia, se crea:

```

```

5 // "a" -> 0
6 // ! operator[] puede INSERTAR una clave al consultar.
7
8 // count(): comprueba si existe SIN insertar
9 bool existe = m.count("b");
10 // Devuelve 0 si no existe, 1 si existe.
11
12 // find(): busca la clave SIN insertar
13 auto it = m.find("b");
14
15 if (it != m.end())
16     cout << it->second;         // valor asociado a "b"
17 else
18     cout << "No existe";
19
20 // ! count() solo indica si existe.
21 // ! find() permite acceder a la pareja clave -> valor.

```

Uso clásico: contar frecuencias.

```

1 unordered_map<int,int> freq;
2
3 for (int x : datos)
4     freq[x]++;                 // frecuencia de cada elemento
5
6 cout << freq[5];
7 // cantidad de veces que aparece 5

```

Cuándo usarlo:

- Contar frecuencias.
- Asociar identificadores con información.
- Buscar rápidamente un valor a partir de una clave.
- Problemas de conteo y frecuencia.
- Memorizar resultados de estados o funciones.

3.3 Conjuntos ordenados (TreeSet)

Conjunto de elementos **sin duplicados y ordenados**. En C++ se implementa mediante `set`, normalmente como un árbol balanceado.

A diferencia de `unordered_set`, los elementos siempre se mantienen ordenados según el comparador utilizado.

Complejidad: insertar, buscar y borrar $O(\log n)$. Consultar el menor o mayor elemento mediante `begin()` o `rbegin()` es $O(1)$.

```

1 set<int> s;                     // CREATE: conjunto vacio
2
3 s.insert(5);                   // INSERT: O(log n)
4 s.insert(1);                   // INSERT: O(log n)
5 s.insert(3);                   // INSERT: O(log n)
6 s.insert(3);                   // no se agrega: ya existe
7
8 int menor = *s.begin();        // READ: minimo -> O(1)
9 int mayor = *s.rbegin();       // READ: maximo -> O(1)
10
11 auto it = s.find(3);           // SEARCH: O(log n)
12 if (it != s.end())
13     cout << *it;
14
15 s.erase(3);                   // DELETE: O(log n)
16
17 int n = s.size();              // READ: cantidad -> O(1)
18
19 // Recorrer en orden ascendente
20 for (int x : s)
21     cout << x << " ";
22 // O(n)

```

lower_bound y upper_bound:

```

1 set<int> s = {1, 3, 5, 7, 9};
2
3 auto it1 = s.lower_bound(5);
4 // primer elemento >= 5
5 // resultado: 5
6
7 auto it2 = s.upper_bound(5);
8 // primer elemento > 5
9 // resultado: 7

```

Esto permite resolver consultas de **piso y techo** rápidamente:

```
1 // FLOOR: mayor elemento <= x
2 auto it = s.upper_bound(x);
3
4 if (it != s.begin()) {
5     --it;
6     cout << *it;
7 }
8
9 // CEILING: menor elemento >= x
10 auto it2 = s.lower_bound(x);
11
12 if (it2 != s.end())
13     cout << *it2;
```

Cuándo usarlo:

- Cuando se necesitan elementos únicos y ordenados.
- Obtener mínimo o máximo dinámicamente.
- Encontrar el siguiente elemento mayor o igual.
- Encontrar el anterior elemento menor o igual.
- Mantener un conjunto mientras se insertan y eliminan elementos.

Diferencia clave: si solo necesitas saber si existe un elemento, `unordered_set` suele ser más rápido. Si necesitas consultar orden, `lower_bound`, `upper_bound`, mínimo o máximo, `set` es la opción adecuada.

3.4 Mapas ordenados (TreeMap)

Mapa **clave** → **valor** cuyas claves se mantienen ordenadas. En C++ se implementa mediante `map`, usando un árbol balanceado.

Cada clave aparece una sola vez. Los valores pueden repetirse.

Complejidad: insertar, buscar, acceder y borrar por clave $O(\log n)$. `begin()` permite obtener la menor clave en $O(1)$.

```
1 map<string,int> m; // CREATE: mapa vacío
2
3 m["b"] = 2; // INSERT: O(log n)
4 m["a"] = 1; // INSERT: O(log n)
5 m["c"] = 3; // INSERT: O(log n)
6
7 int v = m["a"]; // READ: O(log n)
8 // v = 1
9
10 m["a"] = 10; // UPDATE: O(log n)
11
12 m.erase("b"); // DELETE: O(log n)
13
14 auto it = m.find("c"); // SEARCH: O(log n)
15 if (it != m.end())
16     cout << it->second;
17
18 // Primera pareja: menor clave
19 auto primera = m.begin(); // O(1)
20 cout << primera->first;
21
22 // Última pareja: mayor clave
23 auto ultima = prev(m.end()); // O(1) amortizado
24 cout << ultima->first;
```

Las claves se recorren ordenadas:

```
1 map<int,string> m;
2
3 m[30] = "C";
4 m[10] = "A";
5 m[20] = "B";
6
7 for (auto [clave, valor] : m)
8     cout << clave << " " << valor;
```

```
10 // Salida:
11 // 10 A
12 // 20 B
13 // 30 C
```

`lower_bound` y `upper_bound` también funcionan sobre las claves:

```
1 map<int,string> m;
2 m[10] = "A";
3 m[20] = "B";
4 m[30] = "C";
5
6 auto it = m.lower_bound(15);
7 // primera clave >= 15
8 // apunta a 20
9
10 auto it2 = m.upper_bound(20);
11 // primera clave > 20
12 // apunta a 30
```

Uso clásico: contar frecuencias ordenadas.

```
1 map<int,int> freq;
2
3 for (int x : datos)
4     freq[x]++;
5
6 // Las claves aparecen en orden creciente.
7 for (auto [x, cantidad] : freq)
8     cout << x << ": " << cantidad << "\n";
```

Cuándo usarlo:

- Cuando se necesitan pares clave-valor ordenados.
- Contar frecuencias y procesarlas en orden.
- Encontrar claves inmediatamente mayores o menores.
- Procesar eventos o valores manteniendo un orden dinámico.
- Cuando se necesitan operaciones de árbol como `lower_bound` y `upper_bound`.

Comparación rápida

Estructura	Duplicados	Orden	Buscar
<code>unordered_set</code>	No	No	$O(1)$ prom.
<code>unordered_map</code>	Claves: No	No	$O(1)$ prom.
<code>set</code>	No	Sí	$O(\log n)$
<code>map</code>	Claves: No	Sí	$O(\log n)$

Regla práctica:

- `unordered_set` → ¿Ya existe este elemento?
- `unordered_map` → ¿Qué valor corresponde a esta clave?
- `set` → Elementos únicos + ordenados.
- `map` → Claves → valores + claves ordenadas.

4 Trees

5 Range Queries

5.1 Casos de uso

Cuando el problema te pide:	Entonces usa:	Complejidad
Suma de rango sobre un arreglo que nunca cambia , con muchas consultas (ej. “suma de ventas entre el día l y r ”)	Prefix Sums	Build $O(n)$ Query $O(1)$
Suma de rango con actualizaciones puntuales frecuentes (ej. “suma el stock de la posición i , luego consulta un rango”)	Fenwick Tree (BIT)	Build $O(n)$ Query/Update $O(\log n)$
Sumar v a todo un rango $[l, r]$ y también consultar la suma de cualquier rango	FenwickRangeUpdate (doble BIT)	Query/Update $O(\log n)$
Range update + range query con operaciones más complejas: min/max en rango, asignar un valor a todo un rango, contar ceros, etc.	Segment Tree + Lazy Propagation	Query/Update $O(\log n)$
Mínimo, máximo o gcd de cualquier rango sobre un arreglo estático (ej. RMQ clásico, LCA) Query $O(1)$	Sparse Table	Build $O(n \log n)$
Mínimo o máximo de rango cuando el arreglo sí cambia con el tiempo	Segment Tree	Query/Update $O(\log n)$
Consultas 2D: cuántos puntos o cuánta suma hay dentro de un rectángulo $[x_1, x_2] \times [y_1, y_2]$	Fenwick 2D	Query/Update $O(\log n \log m)$
Encontrar el k -ésimo elemento, o “menor índice cuya suma acumulada alcanza X ” (frecuencias)	Fenwick con lowerBound	$O(\log n)$
Muchas queries de rango offline (las conoces todas de antemano) sobre un arreglo estático o con pocos updates	Mo's Algorithm	$O((n + q)\sqrt{n})$
LCA entre dos nodos, o mínimo en rango sobre un recorrido Euler de un árbol (valores consecutivos difieren en ± 1)	RMQ ± 1 (Euler Tour Sparse Table)	Build $O(n)$ Query $O(1)$
Cuántos elementos en el rango $[l, r]$ están dentro de un rango de valores $[a, b]$, o cuál es el k -ésimo menor en $[l, r]$	Wavelet Tree	Query $O(\log n)$
Queries o updates sobre caminos o subárboles de un árbol (ej. “suma el camino de u a v ”)	Heavy-Light Decomposition (+ Segment/Fenwick)	Query/Update $O(\log^2 n)$
Necesitas algo simple de implementar bajo presión, con balance razonable entre update y query, sin memorizar mucha teoría	Sqrt Decomposition	Query/Update $O(\sqrt{n})$

Table 2: Guía rápida: qué estructura de range queries usar según el problema, con su complejidad.

5.2 Prefix Sums

Idea: si un arreglo **no cambia**, puedes precalcular $\text{pref}[i] = \text{suma de los primeros } i \text{ elementos}$ en $O(n)$, una sola vez. Cualquier suma de rango $[l, r]$ se responde restando dos prefijos: $\text{pref}[r+1] - \text{pref}[l]$. Es la base conceptual de todo lo demás en esta sección: Fenwick y Segment Tree existen justamente para soportar lo que Prefix Sums no puede (actualizaciones), a costa de más complejidad de código.

¿Por qué usarlo? Es lo más simple y rápido posible para consultas de rango cuando el arreglo es estático. Cero estructuras de datos elaboradas, cero recursión, solo un arreglo auxiliar y una resta.

Casos de uso:

- Suma de rango sobre un arreglo que nunca se modifica, con muchas queries.

- Suma de submatriz (versión 2D) sobre una matriz estática.
- Aplicar B actualizaciones de rango **offline** y consultar el arreglo final una sola vez (Difference Array).
- Como paso final de un precómputo de teoría de números (criba de ϕ , μ , número de divisores, etc.) cuando el problema pide sumas sobre esos valores en un rango.

Complejidad: construir $O(n)$ (u $O(nm)$ en 2D), consulta $O(1)$. No soporta actualizaciones puntuales eficientes: modificar un elemento y volver a consultar obliga a reconstruir, $O(n)$.

Prefix Sum 1D

```

1 struct PrefixSum1D {
2     vector<long long> pref;
3     int n;
4
5     // CREATE: construye los prefijos a partir de a[0..n-1].
6     PrefixSum1D(vector<long long>& a) {
7         n = a.size();
8         pref.assign(n + 1, 0);
9
10        for (int i = 0; i < n; i++)
11            pref[i + 1] = pref[i] + a[i];
12    } // O(n)
13
14    // READ: suma del rango [l, r] (0-indexado, inclusivo).
15    long long rango(int l, int r) {
16        if (l > r) return 0;
17        return pref[r + 1] - pref[l];
18    } // O(1)
19 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> a = {5, 2, 9, 1, 7};
2
3 PrefixSum1D ps(a);
4 // arreglo logico: [5, 2, 9, 1, 7] (posiciones 0..4)
5
6 cout << ps.rango(0, 2);
7 // 5 + 2 + 9 = 16
8
9 cout << ps.rango(1, 4);
10 // 2 + 9 + 1 + 7 = 19
11
12 cout << ps.rango(3, 3);
13 // 1

```

Prefix Sum 2D

```

1 struct PrefixSum2D {
2     vector<vector<long long>> pref;
3     int n, m;
4
5     // CREATE: construye la matriz de prefijos a partir de
6     // a[0..n-1][0..m-1] usando inclusion-exclusion.
7     PrefixSum2D(vector<vector<long long>>& a) {
8         n = a.size();
9         m = a[0].size();
10        pref.assign(n + 1, vector<long long>(m + 1, 0));
11
12        for (int i = 0; i < n; i++)
13            for (int j = 0; j < m; j++)
14                pref[i + 1][j + 1] = a[i][j] + pref[i][j + 1]
15                + pref[i + 1][j] - pref[i][j];
16    } // O(n * m)
17
18    // READ: suma del rectangulo [x1..x2] x [y1..y2] (0-indexado,
19    // inclusivo), usando inclusion-exclusion sobre los prefijos.
20    long long rango(int x1, int y1, int x2, int y2) {
21        return pref[x2 + 1][y2 + 1] - pref[x1][y2 + 1]
22        - pref[x2 + 1][y1] + pref[x1][y1];
23    } // O(1)
24 };

```

Ejemplo de funcionamiento:

```

1 vector<vector<long long>> a = {
2     {1, 2, 3},
3     {4, 5, 6},
4     {7, 8, 9}
5 };
6
7 PrefixSum2D ps(a);
8
9 cout << ps.rango(0, 0, 1, 1);
10 // 1+2+4+5 = 12
11
12 cout << ps.rango(1, 1, 2, 2);
13 // 5+6+8+9 = 28

```

Difference Array (updates de rango offline)

Idea: el truco inverso a Prefix Sums: para sumar v a todo un rango $[l, r]$, marcas $\text{diff}[l] += v$ y $\text{diff}[r+1] -= v$ en $O(1)$. Después de aplicar **todas** las actualizaciones, un solo recorrido con prefijos reconstruye el arreglo final. Sirve solo cuando todas las updates son **offline** (las conoces antes de necesitar consultar); si necesitas consultar *entre* actualizaciones, usa Fenwick-RangeUpdate en su lugar.

```

1 struct DifferenceArray {
2     vector<long long> diff;
3     int n;
4
5     // CREATE: reserva un arreglo de diferencias de tamaño n.
6     DifferenceArray(int n) : n(n) {
7         diff.assign(n + 1, 0);
8     }
9
10    // UPDATE: marca +v en todo el rango [l, r] (0-indexado).
11    void rangoUpdate(int l, int r, long long v) {
12        diff[l] += v;
13        if (r + 1 <= n)
14            diff[r + 1] -= v;
15    }
16
17    // READ: reconstruye el arreglo final tras aplicar todas
18    // las actualizaciones marcadas hasta el momento.
19    vector<long long> aplicar() {
20        vector<long long> a(n);
21        long long acc = 0;
22
23        for (int i = 0; i < n; i++) {
24            acc += diff[i];
25            a[i] = acc;
26        }
27
28        return a;
29    }
30 };

```

Ejemplo de funcionamiento:

```

1 DifferenceArray d(5);
2
3 d.rangoUpdate(1, 3, 4);
4 d.rangoUpdate(0, 4, 1);
5 d.rangoUpdate(2, 2, 10);
6
7 vector<long long> resultado = d.aplicar();
8 // resultado: [1, 5, 15, 5, 1]

```

Aplicación: suma de Euler Totient (ϕ) en un rango

Idea: $\phi(n)$ cuenta cuántos enteros en $[1, n]$ son coprimos con n . Calcular ϕ de un número es teoría de números ($O(\sqrt{n})$ factorizando); pero cuando el problema pide la **suma de $\phi(i)$ para i en un rango $[l, r]$** , con muchas queries, el patrón se resuelve en dos fases: (1) precalculas el arreglo de ϕ una sola vez con una criba (teoría de números), y (2) respondes las queries de rango con Prefix Sums sobre ese arreglo ya calculado (exactamente esta

subsección). Factorizar cada número del rango por separado en cada query sería $O(q\sqrt{n})$ y no escala; precalcular y sumar con prefijos reduce cada query a $O(1)$.

Criba lineal + Prefix Sums (rango $[1, N]$).

```

1 struct PhiRangeSum {
2     vector<long long> phi, pref;
3     int n;
4
5     // CREATE: calcula phi[1..n] con criba lineal y sus prefijos.
6     // phi[i] = cantidad de enteros en [1, i] coprimos con i.
7     PhiRangeSum(int n) : n(n) {
8         phi.assign(n + 1, 0);
9         pref.assign(n + 1, 0);
10
11        vector<int> primos;
12        vector<bool> compuesto(n + 1, false);
13        phi[1] = 1;
14
15        for (int i = 2; i <= n; i++) {
16            if (!compuesto[i]) {
17                primos.push_back(i);
18                phi[i] = i - 1;
19            }
20
21            for (int p : primos) {
22                if ((long long)i * p > n) break;
23
24                compuesto[i * p] = true;
25
26                if (i % p == 0) {
27                    phi[i * p] = phi[i] * p;
28                    break;
29                } else {
30                    phi[i * p] = phi[i] * (p - 1);
31                }
32            }
33        }
34
35        for (int i = 1; i <= n; i++)
36            pref[i] = pref[i - 1] + phi[i];
37    }
38
39    // READ: suma de phi(i) para i en [l, r].
40    long long rango(int l, int r) {
41        if (l > r) return 0;
42        return pref[r] - pref[l - 1];
43    }
44
45    // READ: valor individual de phi(i).
46    long long get(int i) {
47        return phi[i];
48    }
49 };

```

Ejemplo de funcionamiento:

```

1 PhiRangeSum pr(10);
2 // phi(1..10) = [1, 1, 2, 2, 4, 2, 6, 4, 6, 4]
3
4 cout << pr.get(7);
5 // phi(7) = 6 (7 es primo)
6
7 cout << pr.rango(1, 10);
8 // 1+1+2+2+4+2+6+4+6+4 = 32
9
10 cout << pr.rango(4, 8);
11 // 2+4+2+6+4 = 18

```

Criba segmentada (rango $[L, R]$ grande). Si R es muy grande (10^{12}) pero $R - L$ es manejable, no puedes cribar desde 1. En vez de eso, precalculas los primos hasta $\sqrt{R}$, y para cada número del segmento $[L, R]$ le quitas sus factores primos pequeños, aplicando $\phi(n) = n \prod_{p|n} (1 - \frac{1}{p})$. Si al final queda un residuo > 1 , es un único primo grande (mayor a $\sqrt{R}$) que también se aplica a la fórmula.

```

1 struct PhiSegmentedRangeSum {
2     vector<long long> phi, pref;
3     long long L, R;
4
5     // CREATE: calcula phi(i) para cada i en [L, R] factorizando
6     // con

```



```

6 // los primos hasta sqrt(R), y sus prefijos sobre el segmento.
7 PhiSegmentedRangeSum(long long L, long long R) : L(L), R(R) {
8     int lim = (int)sqrt((double)R) + 1;
9
10    vector<int> primos;
11    vector<bool> compuesto(lim + 1, false);
12    for (int i = 2; i <= lim; i++) {
13        if (!compuesto[i]) primos.push_back(i);
14        for (int p : primos) {
15            if ((long long)i * p > lim) break;
16            compuesto[i * p] = true;
17            if (i % p == 0) break;
18        }
19    } // O(sqrt(R) log
    log sqrt(R))
20
21    int sz = R - L + 1;
22    vector<long long> num(sz);
23    phi.assign(sz, 0);
24
25    for (int i = 0; i < sz; i++) {
26        num[i] = L + i;
27        phi[i] = L + i;
28    }
29
30    for (int p : primos) {
31        long long start = ((L + p - 1) / p) * p;
32
33        for (long long j = start; j <= R; j += p) {
34            int idx = j - L;
35            phi[idx] -= phi[idx] / p;
36
37            while (num[idx] % p == 0)
38                num[idx] /= p;
39        }
40    }
41
42    // lo que quede > 1 es un primo grande (> sqrt(R)).
43    for (int i = 0; i < sz; i++)
44        if (num[i] > 1)
45            phi[i] -= phi[i] / num[i];
46
47    pref.assign(sz + 1, 0);
48    for (int i = 0; i < sz; i++)
49        pref[i + 1] = pref[i] + phi[i];
50 } // O((R-L+1) log
    log R + sqrt(R))
51
52 // READ: suma de phi(i) para i en [l, r], con L <= l <= r <=
    R.
53 long long rango(long long l, long long r) {
54     return pref[r - L + 1] - pref[l - L];
55 } // O(1)
56
57 // READ: valor individual de phi(i), con L <= i <= R.
58 long long get(long long i) {
59     return phi[i - L];
60 } // O(1)
61 };

```

Ejemplo de funcionamiento:

```

1 PhiSegmentedRangeSum ps(95, 105);
2 // calcula phi(95), phi(96), ..., phi(105)
3 // sin cribar desde 1
4
5 cout << ps.get(100);
6 // phi(100) = 40
7
8 cout << ps.rango(95, 105);
9 // suma de phi(95) a phi(105)
10
11 cout << ps.rango(100, 102);
12 // phi(100) + phi(101) + phi(102)

```

Cuál variante usar:

Situación	Estructura	Complejidad
$[1, N]$, N chico/mediano	PhiRangeSum	$O(N)$
$[L, R]$ grande, $R - L$ chico	PhiSegmentedRangeSum	$O((R - L) \log \log R + \sqrt{R})$
ϕ cambia entre queries	Fenwick sobre ϕ	$O(\log N)$

Table 3: Variantes para la suma de ϕ .

Regla práctica: el cálculo de ϕ en sí (la criba, lineal o segmentada) es teoría de números y no cambia según la query. Una vez que tienes el arreglo de ϕ listo, trátalo como *cualquier otro arreglo estático*: Prefix Sums

resuelve el resto. No hace falta una estructura especial para ϕ más allá de construir bien el arreglo inicial.

5.3 Árbol de Fenwick (BIT)

Idea: Un arreglo normal responde una suma de prefijo en $O(1)$ si guarda los prefijos, pero entonces actualizar un elemento cuesta $O(n)$. Si guarda los valores, ocurre lo contrario. El Fenwick reparte el trabajo y deja ambas operaciones en $O(\log n)$. La posición i (1-indexada) guarda la suma de un bloque de largo $i \& (-i)$, es decir, el bit menos significativo de i , que termina en i : $t[i] = \sum_{j=i-\text{lowbit}(i)+1}^i a_j$.

Un prefijo $[1, i]$ se descompone en bloques disjuntos quitando bits: se lee $t[i]$, luego $i -= i \& (-i)$, y así hasta llegar a 0. Cada paso elimina un bit, así que son a lo sumo $\log_2 n$ lecturas. Para actualizar la posición i hay que corregir todos los bloques que la contienen: i , $i + \text{lowbit}(i)$, y así sucesivamente. Cada salto agrega un bit más alto, de modo que también son $O(\log n)$. Como la consulta usa restas (rango = prefijo(r) - prefijo($l - 1$)), la operación tiene que ser *invertible*.

¿Por qué usarlo? Frente a un árbol de segmentos, el Fenwick sirve para sumas (o xor, o cualquier operación invertible) de prefijo y rango con actualizaciones puntuales, con menos memoria (n celdas contra $2n$ o $4n$), menos código y una constante menor. Frente a un arreglo de prefijos, que responde en $O(1)$ pero actualiza en $O(n)$, gana cuando hay muchas actualizaciones intercaladas. Si el arreglo no cambia, el arreglo de prefijos es más simple y más rápido. No sirve para mínimo, máximo ni mcd (no tienen inverso): ahí se usa un árbol de segmentos, o una Sparse Table si el arreglo es estático. Para actualizaciones de rango con consultas de rango existe la variante de doble BIT. Si las actualizaciones son más generales (asignar, multiplicar), hace falta un árbol de segmentos con propagación perezosa. Frente a Mo's, el Fenwick permite contar elementos distintos en un rango con consultas offline en $O((n+q) \log n)$ en vez de $O(n\sqrt{q})$ (última variante).

Casos de uso:

- Suma de prefijo o de rango con actualizaciones puntuales frecuentes.
- Conteo de inversiones y frecuencias acumuladas, incluida la búsqueda del k -ésimo elemento.
- Suma de rango con actualización de rango (doble BIT) cuando no hace falta propagación perezosa general.
- Consultas 2D: suma o conteo de puntos dentro de un rectángulo.
- De nicho: contar elementos distintos en un rango con consultas fuera de línea (última variante).

Complejidad: Construir con el método de «empujar al padre» cuesta $O(n)$, porque cada celda se suma una sola vez a su padre; hacer n actualizaciones costaría $O(n \log n)$. Una actualización puntual y una consulta de prefijo o de rango cuestan $O(\log n)$, por el recorrido de bits explicado arriba. La búsqueda del k -ésimo (*binary lifting*) también es $O(\log n)$. Range update + range

query con doble BIT cuesta $O(\log n)$ y la versión 2D $O(\log n \log m)$. Memoria: $O(n)$, o $O(nm)$ en 2D.

BIT clásico (actualización puntual, consulta de rango)

```

1 struct ArbolFenwick {
2     int n;
3     vector<long long> t; // t[i] = suma del bloque (i - lowbit(
4         i), i]; 1-indexado
5
6     // CREATE: arbol de tamaño tam, todo en cero.
7     ArbolFenwick(int tam = 0) : n(tam), t(tam + 1, 0) {} // O(n)
8
9     // CREATE: construye el arbol a partir de a[0..n-1] empujando
10        cada valor a su padre una vez.
11     void construir(const vector<long long>& a) {
12         t.assign(n + 1, 0);
13         for (int i = 1; i <= n; i++) {
14             t[i] += a[i - 1];
15             int j = i + (i & -i);
16             if (j <= n) t[j] += t[i];
17         }
18     } // O(n)
19
20     // WRITE: suma v a la posición i (1 <= i <= n).
21     void actualizar(int i, long long v) {
22         for (; i <= n; i += i & -i) t[i] += v;
23     } // O(log n)
24
25     // WRITE: fija el valor de la posición i en x.
26     void fijar(int i, long long x) {
27         actualizar(i, x - valor(i));
28     } // O(log n)
29
30     // READ: suma del prefijo [1, i].
31     long long prefijo(int i) const {
32         long long s = 0;
33         for (; i > 0; i -= i & -i) s += t[i];
34         return s;
35     } // O(log n)
36
37     // READ: suma del rango [l, r].
38     long long rango(int l, int r) const {
39         if (l > r) return 0;
40         return prefijo(r) - prefijo(l - 1);
41     } // O(log n)
42
43     // READ: valor puntual de la posición i.
44     long long valor(int i) const {
45         return rango(i, i);
46     } // O(log n)
47
48     // READ: menor índice i con prefijo(i) >= objetivo (binary
49        lifting).
50     // Requiere valores >= 0. Devuelve n + 1 si el total no
51        alcanza.
52     int limite_inferior(long long objetivo) const {
53         int pos = 0, paso = 1;
54         long long ac = 0;
55         while ((paso <= 1) <= n) paso <= 1;
56         for (; paso > 0; paso >>= 1) {
57             int nx = pos + paso;
58             if (nx <= n && ac + t[nx] < objetivo) { pos = nx; ac += t[nx]; }
59         }
60         return pos + 1;
61     } // O(log n)
62
63     // WRITE: reinicia el arbol a ceros.
64     void limpiar() {
65         fill(t.begin(), t.end(), 0);
66     } // O(n)
67 };
68
69 // -----
70 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
71 // -----
72 // 1. Conteo de inversiones: comprimir los valores a 1..k (ver
73    "Compresion de Coordenadas")
74 // y recorrer de izquierda a derecha. Los anteriores mayores
75    que c[i] son i - prefijo(c[i]).
76 // ArbolFenwick bit(k); long long inv = 0;
77 // for (int i = 0; i < n; i++) { inv += i - bit.prefijo(c[i]);
78 // bit.actualizar(c[i], 1); }
79 // // O(n log n)
80 //
81 // 2. Frecuencias y k-esimo elemento: bit.actualizar(x, 1) al
82    insertar x y bit.actualizar(x, -1)
83    al borrarlo. El k-esimo menor del multiconjunto es
84    limite_inferior(k). O(log n) por operacion.

```

```

76 //
77 // 3. XOR en lugar de suma: cambiar "+" por "^" en actualizar
78    y en prefijo, y el rango es
79    prefijo(r) ^ prefijo(l - 1). Sirve con cualquier
80    operacion con inverso (xor, suma modular).
81 //
82 // 4. Mínimo o máximo de prefijo: SOLO si los valores
83    únicamente crecen (para el máximo) o
84    únicamente decrecen (para el mínimo). Se cambia "+" por
85    max/min. No admite rangos
86 // arbitrarios ni cambios en sentido contrario: para eso,
87    arbol de segmentos.
88 //
89 // 5. Índices desde 0: sumar 1 antes de llamar. Con i = 0 el
90    ciclo de actualizar no termina
91    (0 & -0 = 0).
92 //
93 // 6. Valores grandes o negativos como índices: comprimirlos
94    primero (ver "Compresion de
95    Coordenadas"), para que el arbol tenga tamaño O(#valores
96    distintos).
97 //
98 // 7. Suma modulo p: aplicar "% p" en cada suma y restar
99    sumando p en rango():
100    return ((prefijo(r) - prefijo(l - 1)) % p + p) % p;

```

Ejemplo de funcionamiento:

```

1 int main() {
2     vector<long long> a = {5, 2, 9, 1, 7};
3     ArbolFenwick bit(5);
4     bit.construir(a); // posiciones 1..5:
5         [5, 2, 9, 1, 7]
6
7     cout << bit.prefijo(3) << "\n"; // 16 (5 + 2 + 9)
8     cout << bit.rango(2, 4) << "\n"; // 12 (2 + 9 + 1)
9
10    bit.actualizar(2, 3); // [5, 5, 9, 1, 7]
11    cout << bit.valor(2) << "\n"; // 5
12
13    bit.fijar(4, 10); // [5, 5, 9, 10, 7]
14    cout << bit.limite_inferior(15) << "\n"; // 3 (prefijos 5,
15        10, 19: el primero >= 15 es el de 3)
16    return 0;
17 }

```

Nota: El arbol es 1-indexado: `actualizar` con $i = 0$ nunca termina. `construir` reinicia el arbol, así que no combina con valores previos. `limite_inferior` solo es correcto si todos los valores son no negativos, porque usa la monotonía de los prefijos, y devuelve $n + 1$ si el total no alcanza el objetivo. Por último, las sumas se guardan en `long long` para no desbordar con valores grandes.

BIT con Range Update + Range Query (doble BIT)

Idea: Sea d el arreglo de diferencias: sumar v a $[l, r]$ equivale a $d_l += v$ y $d_{r+1} -= v$. El valor $a_j = \sum_{k \leq j} d_k$, y la suma de prefijo es

$$\sum_{j \leq i} a_j = \sum_{k \leq i} d_k (i - k + 1) = i \sum_{k \leq i} d_k - \sum_{k \leq i} d_k (k - 1).$$

Basta entonces un Fenwick para $\sum d_k$ (b1) y otro para $\sum d_k (k - 1)$ (b2). Con ellos, el prefijo es `b1.prefijo(i)*i - b2.prefijo(i)`, y cada actualización de rango son cuatro actualizaciones puntuales.

```

1 struct FenwickRango {
2     int n;
3     ArbolFenwick b1, b2; // b1: sumas de d[k]; b2: sumas de d[k]
4         * (k - 1)
5
6     // CREATE: arreglo logico de n ceros.
7     FenwickRango(int tam) : n(tam), b1(tam), b2(tam) {} // O(n)
8
9     // WRITE: suma v a todos los elementos de [l, r].
10    void actualizar_rango(int l, int r, long long v) {
11        b1.actualizar(l, v);
12        b1.actualizar(r + 1, -v);
13        b2.actualizar(l, v * (1 - 1));
14        b2.actualizar(r + 1, -v * r);

```

```

14     } // O(log n)
15
16     // READ: suma del prefijo [1, i].
17     long long prefijo(int i) const {
18         return b1.prefijo(i) * i - b2.prefijo(i);
19     } // O(log n)
20
21     // READ: suma del rango [l, r].
22     long long rango(int l, int r) const {
23         if (l > r) return 0;
24         return prefijo(r) - prefijo(l - 1);
25     } // O(log n)
26
27     // READ: valor puntual de la posición i.
28     long long valor(int i) const {
29         return rango(i, i);
30     } // O(log n)
31 };
32
33 // -----
34 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
35 // -----
36 // 1. Solo actualización de rango + consulta PUNTUAL: basta b1.
37 //     El valor de la posición i
38 //     es b1.prefijo(i), con un solo árbol y menos constante.
39 //
40 // 2. Arreglo inicial no nulo: sumar al resultado un arreglo de
41 //     prefijos P del original:
42 //     suma(l, r) = P[r] - P[l-1] + rango(l, r). O(log n) por
43 //     consulta.
44 //
45 // 3. Si b1 y b2 se desbordan con valores grandes: v * (l - 1)
46 //     puede llegar a v * n.
47 //     Con v ~ 10^9 y n ~ 10^6 son 10^15, cabe en long long; si
48 //     es mayor, usar __int128.
49 //
50 // 4. Para max/min con actualizaciones de rango no sirve: usar
51 //     árbol de segmentos con
52 //     propagación perezosa.

```

Ejemplo de funcionamiento:

```

1 int main() {
2     FenwickRango f(5);
3     f.actualizar_rango(2, 4, 3); // [0, 0, 0, 0, 0]
4     cout << f.rango(1, 5) << "\n"; // 9
5
6     f.actualizar_rango(1, 3, 2); // [2, 5, 5, 3, 0]
7     cout << f.valor(2) << "\n"; // 5
8     cout << f.rango(1, 5) << "\n"; // 15 (2 + 5 + 5 + 3 + 0)
9     return 0;
10 }

```

Nota: Con $r = n$, la actualización en $r + 1$ cae fuera del árbol y el ciclo simplemente no se ejecuta, así que no hace falta tratar ese caso aparte. Además, la fórmula supone un arreglo inicial de ceros (modificación 2).

BIT 2D (actualización puntual, suma de submatriz)

Idea: Es un BIT dentro de otro BIT: $t[i][j]$ guarda la suma del rectángulo de $\text{lowbit}(i)$ filas por $\text{lowbit}(j)$ columnas que termina en (i, j) . Una actualización o consulta recorre los $O(\log n)$ índices de fila y, en cada uno, los $O(\log m)$ de columna. La suma de un rectángulo se obtiene por inclusión-exclusión de cuatro prefijos.

```

1 struct Fenwick2D {
2     int n, m;
3     vector<vector<long long>> t;
4
5     // CREATE: matriz de n x m en ceros (1-indexada).
6     Fenwick2D(int filas, int cols)
7         : n(filas), m(cols), t(filas + 1, vector<long long>(cols + 1, 0)) {} // O(n m)
8
9     // WRITE: suma v a la celda (x, y).
10    void actualizar(int x, int y, long long v) {
11        for (int i = x; i <= n; i += i & -i)
12            for (int j = y; j <= m; j += j & -j)
13                t[i][j] += v;
14    } // O(log n log m)
15
16    // READ: suma de la submatriz [1..x] x [1..y].
17    long long prefijo(int x, int y) const {

```

```

18        long long s = 0;
19        for (int i = x; i > 0; i -= i & -i)
20            for (int j = y; j > 0; j -= j & -j)
21                s += t[i][j];
22        return s;
23    } // O(log n log m)
24
25    // READ: suma del rectángulo [x1..x2] x [y1..y2].
26    long long rango(int x1, int y1, int x2, int y2) const {
27        return prefijo(x2, y2) - prefijo(x1 - 1, y2)
28            - prefijo(x2, y1 - 1) + prefijo(x1 - 1, y1 - 1);
29    } // O(log n log m)
30 };
31
32 // -----
33 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
34 // -----
35 // 1. Actualización de rectángulo + consulta puntual: usar
36 //     diferencias 2D. Sumar v al
37 //     rectángulo [x1..x2] x [y1..y2] son cuatro actualizaciones
38 //     puntuales:
39 //     actualizar(x1, y1, v); actualizar(x1, y2 + 1, -v);
40 //     actualizar(x2 + 1, y1, -v); actualizar(x2 + 1, y2 + 1, v);
41 //
42 //     El valor de (x, y) es prefijo(x, y). O(log n log m).
43 //
44 // 2. Actualización de rectángulo + consulta de rectángulo:
45 //     hacen falta cuatro árboles 2D
46 //     (la generalización del doble BIT). Es raro en concursos;
47 //     suele bastar un árbol de
48 //     segmentos 2D.
49 //
50 // 3. Memoria: n * m celdas de long long. Con n, m ~ 5000 son
51 //     200 MB; si los puntos son
52 //     pocos, comprimir coordenadas (offline) o usar un BIT de
53 //     BITS dispersos.
54 //
55 // 4. Contar puntos (x, y) en un rectángulo con consultas
56 //     offline: ordenar por x y usar
57 //     un BIT 1D sobre y (barrido), que baja a  $O((n + q) \log n)$ 
58 //     y  $O(n)$  de memoria.

```

Ejemplo de funcionamiento:

```

1 int main() {
2     Fenwick2D f(4, 4);
3     f.actualizar(1, 1, 5);
4     f.actualizar(2, 3, 7);
5     f.actualizar(4, 4, 2);
6
7     cout << f.rango(1, 1, 2, 3) << "\n"; // 12 (5 + 7)
8     cout << f.rango(3, 3, 4, 4) << "\n"; // 2
9     return 0;
10 }

```

Variante: Conteo de Elementos Distintos en un Rango (offline)

Idea: Se atienden las consultas *ordenadas por extremo derecho* r y se recorre el arreglo de izquierda a derecha. Al llegar a la posición i , se mantiene un Fenwick con un 1 en la *última aparición* (hasta i) de cada valor y 0 en el resto. Para ello, al procesar a_i se borra el 1 de su aparición anterior (si existe) y se coloca uno en i . Después de procesar r , la respuesta a $[l, r]$ es la suma de ese Fenwick en $[l, r]$.

Es correcto porque cada valor presente en $[l, r]$ tiene exactamente una marca, su última aparición hasta r , y esa marca cae dentro de $[l, r]$ (está en $[l, r]$ porque el valor aparece ahí y la última aparición es $\geq$ cualquiera de las apariciones en el rango). Un valor que no aparece en $[l, r]$ tiene su marca antes de l (o ninguna) y no se cuenta. Así, cada posición del rango se cuenta una sola vez por valor, y no hay que volver a recorrer el rango.

Complejidad: Cada elemento hace a lo sumo dos actualizaciones (-1 en la aparición anterior, $+1$ en la actual) y cada consulta una resta de prefijos, todas $O(\log n)$. Si las consultas se agrupan por r con listas (en vez de ordenarlas), el total es $O((n + q) \log n)$, contra $O(n\sqrt{q})$ de Mo's. Memoria: $O(n + q + \max_val)$.

```

1 // Valores en [0, max_val]. Consultas (l, r) 0-indexadas e
  // inclusivas, todas conocidas de antemano.
2 struct ConteoDistintos {
3     int n, max_val;
4     vector<int> a;
5
6     // CREATE: guarda el arreglo y el valor maximo.
7     ConteoDistintos(const vector<int>& arr, int mv) : n((int)arr.
      size()), max_val(mv), a(arr) {} // O(n)
8
9     // READ: responde todas las consultas y las devuelve en el
      orden original.
10    vector<int> resolver(const vector<array<int, 2>>& consultas)
      const {
11        int q = (int)consultas.size();
12        vector<vector<int>> por_r(n); // por_r
13        [r]: consultas que terminan en r
14        for (int id = 0; id < q; id++) por_r[consultas[id][1]].
          push_back(id);
15
16        ArbolFenwick bit(n);
17        vector<int> ultima(max_val + 1, 0); //
          ultima posicion (1-indexada) de cada valor; 0 = ninguna
18        vector<int> resp(q);
19        for (int i = 1; i <= n; i++) {
20            int v = a[i - 1];
21            if (ultima[v]) bit.actualizar(ultima[v], -1); // la
          marca anterior de v deja de contar
22            bit.actualizar(i, 1); // ahora
          la ultima aparicion de v es i
23            ultima[v] = i;
24            for (int id : por_r[i - 1])
25                resp[id] = (int)bit.rango(consultas[id][0] + 1, i);
26        }
27        return resp;
28    } // O((n + q) log n)
29 };
30
31 // -----
32 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
33 // -----
34 // 1. Suma de los valores DISTINTOS del rango (cada valor una
      sola vez): en lugar de 1,
35 //     marcar el propio valor.
36 //     if (ultima[v]) bit.actualizar(ultima[v], -v);
37 //     bit.actualizar(i, v);
38 //     El resto es identico. O((n + q) log n).
39 //
40 // 2. Valores grandes o negativos: comprimirlos primero (ver "
      Compresion de Coordenadas"),
41 //     para que ultima[] tenga tamano O(n).
42 //
43 // 3. Consultas EN LINEA (una a la vez, sin ver las siguientes):
      este metodo no sirve, porque
44 //     depende de procesar por r creciente. Opciones: Fenwick/
      arbol de segmentos persistente
45 //     (una version por cada r, O(log n) por consulta), o un
      arbol de segmentos con mezcla
46 //     de listas ordenadas.
47 //
48 // 4. Con actualizaciones puntuales del arreglo: el metodo se
      rompe, porque cambiar un valor
49 //     altera varias "ultimas apariciones". Opciones: Mo's con
      actualizaciones o CDQ (divide
50 //     y venceras sobre el tiempo).
51 //
52 // 5. Simetria: tambien se puede barrer de derecha a izquierda,
      ordenando por l y marcando la
53 //     PRIMERA aparicion de cada valor. Mismo costo.
54 //
55 // 6. Frente a Mo's: este metodo es O((n + q) log n) contra O(n
      sqrt(q)), asi que gana cuando
56 //     n y q son grandes; Mo's es mas flexible (sirve para
      respuestas que no se pueden
      mantener con un Fenwick, como la moda).

```

Ejemplo de funcionamiento:

```

1 int main() {
2     vector<int> a = {1, 2, 1, 3, 2, 4, 1};
3     ConteoDistintos cd(a, 4);
4
5     vector<array<int, 2>> consultas = {
6         {0, 2}, // [1, 2, 1] -> 2 distintos
7         {1, 4}, // [2, 1, 3, 2] -> 3 distintos
8         {0, 6}, // [1, 2, 1, 3, 2, 4, 1] -> 4 distintos
9         {2, 5}, // [1, 3, 2, 4] -> 4 distintos
10        {3, 3}, // [3] -> 1 distinto
11    };
12
13    for (int x : cd.resolver(consultas)) cout << x << " ";
14    cout << "\n"; // 2 3 4 4 1
15    return 0;

```

16 }

Nota: Como las consultas se agrupan por r , no se ordenan: el recorrido es lineal más las respuestas. Las consultas deben cumplir $0 \leq l \leq r < n$ y todos los valores deben estar en $[0, \text{max_val}]$. `resolver` es `const` porque el Fenwick y `ultima` son locales, así que el objeto puede reutilizarse con otras consultas.

Regla práctica: Si el arreglo cambia y solo necesitas sumas (o cualquier operación invertible) de prefijo o rango, usa Fenwick: es más corto y rápido que un árbol de segmentos. Si además hay actualizaciones de rango, usa el doble BIT. Si son consultas 2D, el BIT 2D. Si el arreglo no cambia, un arreglo de prefijos es más simple. Para min, max o mcd, o para actualizaciones de rango generales, pasa a un árbol de segmentos con propagación perezosa. Para contar elementos distintos en rangos con consultas offline, ordena por r y marca la última aparición de cada valor en un Fenwick; si las consultas son en línea o hay actualizaciones, usa Mo's con actualizaciones o un árbol persistente.

5.4 Sqrt Decomposition

Idea: divide el arreglo en bloques de tamaño $\approx \sqrt{n}$ y guarda un resumen (suma, mínimo, etc.) por bloque. Un update toca solo un bloque; una query de rango toca a lo más dos bloques *parciales* (recorridos elemento por elemento) y varios bloques *completos* en el medio (usados de golpe con su resumen precalculado). Como hay $\approx \sqrt{n}$ bloques y cada bloque tiene $\approx \sqrt{n}$ elementos, todo cuesta $O(\sqrt{n})$: ni tan lento como un arreglo plano, ni tan rápido como Fenwick/Segment Tree, pero mucho más simple de escribir y de adaptar a operaciones raras.

¿Por qué usarlo? Es la estructura más flexible cuando no tienes tiempo de pensar un Segment Tree con lazy propagation correcto: soporta casi cualquier operación (suma, min, max, contar algo con una condición) con el mismo esqueleto de código, aunque sea $O(\sqrt{n})$ en vez de $O(\log n)$. También es la base conceptual de Mo's Algorithm.

Casos de uso:

- Necesitas implementar algo rápido bajo presión, sin memorizar casos de Segment Tree con lazy propagation.
- Operaciones donde min/max cambian de forma que Fenwick no puede manejar (Fenwick solo sirve para operaciones invertibles como la suma).
- n no es tan grande ($\leq 10^5$ – 10^6) y $O(\sqrt{n})$ por operación es suficiente para el límite de tiempo.
- Base para Mo's Algorithm (ver esa sección).

Complejidad: construir $O(n)$, update puntual $O(1)$ (suma) u $O(\sqrt{n})$ (min/max), query de rango $O(\sqrt{n})$, range update + range query $O(\sqrt{n})$ con lazy por bloque.

Sqrt Decomposition clásico (point update, range query — suma)


```

1 struct SqrtDecomposition {
2     vector<long long> a, bloque;
3     int n, tam;
4
5     // CREATE: construye el arreglo en bloques de tamaño ~sqrt(n).
6
7     SqrtDecomposition(vector<long long>& arr) {
8         a = arr;
9         n = a.size();
10        tam = max(1, (int)sqrt((double)n));
11
12        bloque.assign((n + tam - 1) / tam, 0);
13
14        for (int i = 0; i < n; i++)
15            bloque[i / tam] += a[i]; // O(n)
16
17        // UPDATE: suma v al elemento en la posición i.
18        void update(int i, long long v) {
19            a[i] += v;
20            bloque[i / tam] += v; // O(1)
21        }
22
23        // READ: suma del rango [l, r] (0-indexado, inclusivo).
24        long long rango(int l, int r) {
25            long long s = 0;
26            int bl = l / tam, br = r / tam;
27
28            if (bl == br) {
29                for (int i = l; i <= r; i++)
30                    s += a[i];
31                return s;
32            }
33
34            for (int i = l; i < (bl + 1) * tam; i++)
35                s += a[i];
36
37            for (int b = bl + 1; b < br; b++)
38                s += bloque[b];
39
40            for (int i = br * tam; i <= r; i++)
41                s += a[i];
42
43            return s; // O(sqrt(n))
44        }
45    };

```

Ejemplo de funcionamiento:

```

1 vector<long long> a = {5, 2, 9, 1, 7, 3, 8, 4};
2
3 SqrtDecomposition sd(a);
4 // arreglo logico: [5, 2, 9, 1, 7, 3, 8, 4] (posiciones 0..7)
5 // tam = 2 -> bloques: [5+2, 9+1, 7+3, 8+4] = [7, 10, 10, 12]
6
7 cout << sd.rango(1, 5);
8 // 2 + 9 + 1 + 7 + 3 = 22
9
10 sd.update(3, 10);
11 // arreglo logico: [5, 2, 9, 11, 7, 3, 8, 4]
12
13 cout << sd.rango(1, 5);
14 // 2 + 9 + 11 + 7 + 3 = 32

```

Sqrt Decomposition con range update + range query (lazy por bloque)

Idea: igual que arriba, pero cada bloque guarda un valor lazy pendiente que representa “sumar v a todo el bloque” sin tocar cada elemento individualmente. Solo se aplica a los elementos reales cuando un update o query parcial necesita tocar ese bloque específico.

```

1 struct SqrtRangeUpdate {
2     vector<long long> a, bloqueSum, lazy;
3     int n, tam;
4
5     // CREATE: construye el arreglo en bloques de tamaño ~sqrt(n).
6
7     SqrtRangeUpdate(vector<long long>& arr) {
8         a = arr;
9         n = a.size();
10        tam = max(1, (int)sqrt((double)n));
11
12        int nb = (n + tam - 1) / tam;
13        bloqueSum.assign(nb, 0);
14        lazy.assign(nb, 0);

```

```

15        for (int i = 0; i < n; i++)
16            bloqueSum[i / tam] += a[i]; // O(n)
17    }
18
19    // READ auxiliar: cantidad de elementos reales del bloque b.
20    int tamBloque(int b) {
21        return min(n, (b + 1) * tam) - b * tam; // O(1)
22    }
23
24    // UPDATE: suma v a todo el rango [l, r].
25    void rangoUpdate(int l, int r, long long v) {
26        int bl = l / tam, br = r / tam;
27
28        if (bl == br) {
29            for (int i = l; i <= r; i++)
30                a[i] += v;
31            bloqueSum[bl] += v * (r - l + 1);
32            return;
33        }
34
35        for (int i = l; i < (bl + 1) * tam; i++)
36            a[i] += v;
37        bloqueSum[bl] += v * ((bl + 1) * tam - l);
38
39        for (int b = bl + 1; b < br; b++)
40            lazy[b] += v;
41
42        for (int i = br * tam; i <= r; i++)
43            a[i] += v;
44        bloqueSum[br] += v * (r - br * tam + 1); // O(sqrt(n))
45    }
46
47    // READ: suma del rango [l, r].
48    long long rango(int l, int r) {
49        long long s = 0;
50        int bl = l / tam, br = r / tam;
51
52        if (bl == br) {
53            for (int i = l; i <= r; i++)
54                s += a[i] + lazy[bl];
55            return s;
56        }
57
58        for (int i = l; i < (bl + 1) * tam; i++)
59            s += a[i] + lazy[bl];
60
61        for (int b = bl + 1; b < br; b++)
62            s += bloqueSum[b] + lazy[b] * tamBloque(b);
63
64        for (int i = br * tam; i <= r; i++)
65            s += a[i] + lazy[br];
66
67        return s; // O(sqrt(n))
68    }
69 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> a = {1, 1, 1, 1, 1, 1};
2
3 SqrtRangeUpdate sr(a);
4 // tam = 2 -> bloques: [1+1, 1+1, 1+1] = [2, 2, 2]
5
6 sr.rangoUpdate(1, 4, 10);
7 // arreglo logico "efectivo": [1, 11, 11, 11, 11, 1]
8
9 cout << sr.rango(0, 5);
10 // 1+11+11+11+11+1 = 46
11
12 cout << sr.rango(2, 3);
13 // 11+11 = 22

```

Variante para mínimo / máximo (point update, range query)

Idea: a diferencia de Fenwick (que solo sirve para operaciones *invertibles* como la suma), Sqrt Decomposition soporta min/max con el mismo esqueleto: la única diferencia es que un update ya no puede ajustar el resumen del bloque en $O(1)$, porque quitar el valor viejo no dice si sigue habiendo otro elemento igual de chico. Hay que **reconstruir el mínimo de todo el bloque**, lo cual sigue siendo $O(\sqrt{n})$.

```

1 struct SqrtMinQuery {
2     vector<long long> a, bloqueMin;
3     int n, tam;

```

```

4 // CREATE: construye el arreglo en bloques de tamaño -sqrt(n).
5
6 SqrtMinQuery(vector<long long>& arr) {
7     a = arr;
8     n = a.size();
9     tam = max(1, (int)sqrt((double)n));
10
11     bloqueMin.assign((n + tam - 1) / tam, LLONG_MAX);
12
13     for (int i = 0; i < n; i++)
14         bloqueMin[i / tam] = min(bloqueMin[i / tam], a[i]);
15 } // O(n)
16
17 // UPDATE: fija el valor absoluto de la posición i en x.
18 // Reconstruye el mínimo de todo el bloque, ya que a
19 // diferencia
20 // de la suma, el mínimo no se puede ajustar incrementalmente.
21
22 void set(int i, long long x) {
23     a[i] = x;
24
25     int b = i / tam;
26     long long mn = LLONG_MAX;
27
28     for (int j = b * tam; j < min(n, (b + 1) * tam); j++)
29         mn = min(mn, a[j]);
30
31     bloqueMin[b] = mn; // O(sqrt(n))
32
33 // READ: mínimo del rango [l, r].
34 long long rango(int l, int r) {
35     long long mn = LLONG_MAX;
36     int bl = l / tam, br = r / tam;
37
38     if (bl == br) {
39         for (int i = l; i <= r; i++)
40             mn = min(mn, a[i]);
41         return mn;
42     }
43
44     for (int i = l; i < (bl + 1) * tam; i++)
45         mn = min(mn, a[i]);
46
47     for (int b = bl + 1; b < br; b++)
48         mn = min(mn, bloqueMin[b]);
49
50     for (int i = br * tam; i <= r; i++)
51         mn = min(mn, a[i]);
52
53     return mn; // O(sqrt(n))
54 }

```

Ejemplo de funcionamiento:

```

1 vector<long long> a = {5, 2, 9, 1, 7, 3, 8, 4};
2
3 SqrtMinQuery sm(a);
4 // tam = 2 -> bloques min: [2, 1, 3, 4]
5
6 cout << sm.rango(0, 4);
7 // min(5, 2, 9, 1, 7) = 1
8
9 sm.set(3, 100);
10 // arreglo lógico: [5, 2, 9, 100, 7, 3, 8, 4]
11
12 cout << sm.rango(0, 4);
13 // min(5, 2, 9, 100, 7) = 2

```

Comparación con Fenwick y Segment Tree:

Estructura	Update	Query	Min/Max?
Sqrt Decomposition	$O(1) - O(\sqrt{n})$	$O(\sqrt{n})$	Sí
Fenwick Tree	$O(\log n)$	$O(\log n)$	No
Segment Tree	$O(\log n)$	$O(\log n)$	Sí

Table 4: Sqrt Decomposition vs. Fenwick vs. Segment Tree.

Regla práctica: usa Sqrt Decomposition cuando quieras algo simple y flexible de escribir rápido, sobre todo si la operación no es una suma (Fenwick no sirve) y no quieres arriesgarte con un Segment Tree + Lazy Propagation bajo presión de tiempo. Si n es muy grande o el límite de tiempo es ajustado, $O(\log n)$ de Segment

Tree gana; si la operación es solo suma, Fenwick es más simple y más rápido.

5.5 Sparse Table (RMQ)

Idea: Una Sparse Table responde consultas de rango en $O(1)$ sobre un arreglo que *no cambia*. Precalcula $sp[k][i]$, el resultado de combinar el bloque que empieza en i y tiene largo 2^k . La tabla se llena por doblado: un bloque de largo 2^k son dos bloques de largo 2^{k-1} consecutivos, así que $sp[k][i]$ se obtiene de $sp[k-1][i]$ y $sp[k-1][i + 2^{k-1}]$ con una sola combinación.

La clave está en la consulta. Para $[l, r]$ de largo $len = r - l + 1$ se toma $k = \lfloor \log_2 len \rfloor$, de modo que $2^k \leq len < 2^{k+1}$. Dos bloques de largo 2^k , uno pegado a l y otro pegado a r , cubren todo el rango y se *solapan* (porque $2 \cdot 2^k > len$ no deja hueco: $2^{k+1} > len$ implica que el segundo empieza antes de que termine el primero, o justo donde termina). Si la operación es *idempotente* ($f(x, x) = x$, como min, max, gcd, AND u OR), contar el solape dos veces no cambia el resultado, y la consulta son dos lecturas y una combinación. Con operaciones no idempotentes (suma, xor) el solape sí se nota, y hace falta la *Disjoint Sparse Table*, que evita el solape por construcción.

¿Por qué usarlo? Es la estructura más rápida para consultas de rango cuando el arreglo es estático: $O(1)$ por consulta, frente a $O(\log n)$ del árbol de segmentos y del Fenwick, o frente a $O(n)$ de recorrer el rango. El precio es que no admite actualizaciones. Cambiar un solo elemento invalida $O(n)$ celdas por nivel, y reconstruir cuesta $O(n \log n)$, así que si el arreglo cambia conviene un árbol de segmentos. Frente a un prefijo de sumas, que también responde en $O(1)$, la Sparse Table sirve para operaciones que *no tienen inverso* (el mínimo no se puede «restar»). Y para sumas o xor, el Fenwick o los prefijos son más simples que la Disjoint Sparse Table, que solo se justifica con operaciones asociativas sin inverso ni idempotencia (por ejemplo, producto de matrices). Frente a Mo's (subsección anterior), aquí la operación sí se combina desde dos mitades y no hace falta que las consultas sean offline.

Casos de uso:

- RMQ clásico: mínimo o máximo de cualquier rango de un arreglo estático.
- gcd, AND u OR de rango sobre un arreglo estático (variante más abajo).
- LCA con recorrido de Euler y RMQ: la consulta del ancestro común es un mínimo sobre profundidades.
- Operaciones no idempotentes de rango estático en $O(1)$ (suma, xor, producto de matrices) con la Disjoint Sparse Table.
- De nicho: arreglos de sufijos (LCP entre dos sufijos como mínimo de rango) o problemas con muchísimas consultas sobre datos que nunca se modifican.

Complejidad: La tabla tiene $K = \lfloor \log_2 n \rfloor + 1$ niveles de a lo sumo n celdas, y cada celda se calcula con una combinación $O(1)$ a partir del nivel anterior, así que construir cuesta $O(n \log n)$ en tiempo y en memoria.

La consulta lee dos celdas y una tabla de logaritmos precalculada: $O(1)$. Con $n = 10^6$ la tabla ocupa unos $20 \cdot 10^6$ valores, lo que ya pesa en memoria.

Sparse Table clásico (operaciones idempotentes: min, max)

```

1 struct SparseTable {
2     int n;
3     vector<vector<long long>> sp; // sp[k][i] = minimo del
        bloque [i, i + 2^k - 1]
4     vector<int> lg; // lg[x] = piso(log2(x))
5
6     // CREATE: precalcula la tabla a partir de a[0..n-1].
7     SparseTable(const vector<long long>& a) {
8         n = (int)a.size();
9         lg.assign(n + 1, 0);
10        for (int i = 2; i <= n; i++) lg[i] = lg[i / 2] + 1;
11
12        int K = lg[n] + 1;
13        sp.assign(K, vector<long long>(n));
14        sp[0] = a;
15        for (int k = 1; k < K; k++)
16            for (int i = 0; i + (1 << k) <= n; i++)
17                sp[k][i] = min(sp[k - 1][i], sp[k - 1][i + (1 << (k - 1))]);
18    } // O(n log n)
19
20    // READ: minimo del rango [l, r] (0-indexado, inclusivo).
21    // Dos bloques de largo 2^k que se solapan; min es
        idempotente, el solape no importa.
22    long long consultar(int l, int r) const {
23        int k = lg[r - l + 1];
24        return min(sp[k][l], sp[k][r - (1 << k) + 1]);
25    } // O(1)
26 };
27
28 // -----
29 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
30 // -----
31 // 1. Maximo, AND u OR: cambiar min por max, & o | en el
        constructor y en consultar.
32 // Para gcd ver la variante siguiente (tiene su propia
        complejidad).
33 //
34 // 2. Indice del minimo (no solo el valor): guardar indices y
        comparar por valor.
35 // sp[0][i] = i;
36 // sp[k][i] = (a[sp[k-1][i]] <= a[sp[k-1][i + (1 << (k-1))
        ]]) ? sp[k-1][i] : sp[k-1][i + (1 << (k-1))];
37 // consultar devuelve el indice con el mismo criterio. Con
        "<=" gana el mas a la izquierda.
38 //
39 // 3. Sin tabla lg: calcular el piso del logaritmo con la CPU.
40 // int k = 31 - __builtin_clz(r - l + 1); // O(1), sin
        vector extra
41 //
42 // 4. LCA con Euler Tour: recorrer el arbol anotando (
        profundidad, nodo) en cada visita,
43 // guardar la primera aparicion de cada nodo, y el LCA(u, v)
        es el nodo de menor
44 // profundidad entre esas dos posiciones (RMQ sobre pares).
        O(n log n) + O(1) por LCA.
45 //
46 // 5. Consulta NO idempotente con Sparse Table normal: se parte
        [l, r] en bloques de
47 // potencias de 2 sin solape (los bits de la longitud), en O
        (log n) por consulta.
48 // long long s = 0;
49 // for (int k = K - 1; k >= 0; k--)
50 //     if ((1 << k) <= r - l + 1) { s += sp[k][l]; l += 1 <<
        k; } // O(log n)
51 // Aqui sp[k][i] guarda la SUMA del bloque. Es mas lento que
        prefijos; solo sirve si la
52 // operacion no tiene inverso y no se quiere escribir la
        Disjoint Sparse Table.
53 //
54 // 6. Sparse Table 2D (matriz estatica): sp[kx][ky][i][j] =
        minimo del bloque
55 // 2^kx x 2^ky. Construcción O(n m log n log m), consulta O
        (1) con cuatro bloques.
56 //
57 // 7. Reducir memoria: si solo hay consultas offline,
        ordenarlas por k y guardar solo dos
58 // niveles a la vez (O(n) de memoria), a costa de ordenar
        las consultas.

```

Ejemplo de funcionamiento:

```

1 int main() {

```

```

2     vector<long long> a = {5, 2, 9, 1, 7, 3};
3     SparseTable st(a);
4
5     cout << st.consultar(0, 2) << "\n"; // 2 (min de 5, 2, 9)
6     cout << st.consultar(2, 5) << "\n"; // 1 (min de 9, 1, 7,
7         3)
8     cout << st.consultar(4, 4) << "\n"; // 7 (rango de un solo
9         elemento)
10    cout << st.consultar(0, 5) << "\n"; // 1 (arreglo completo)
11    return 0;
12 }

```

Variante: GCD en un Rango

Idea: El gcd es idempotente: $\gcd(x, x) = x$. Además es asociativo y conmutativo, así que es exactamente igual que el mínimo o el máximo para esta estructura, y solo cambia la función de combinación. El solape de los dos bloques no altera el resultado, porque $\gcd(g_1, g_2)$ es el gcd de la unión de elementos, sin importar que un elemento aparezca dos veces. Se usan valores no negativos (el gcd de 0 y x es x , y el gcd de un rango de ceros es 0).

La diferencia con min y max es el costo de la combinación. Cada `__gcd` es un Euclides que cuesta $O(\log M)$, con M el mayor valor. Por tanto, construir cuesta $O(n \log n \log M)$ y cada consulta $O(\log M)$. En la práctica los gcd de bloques grandes bajan rápido (el gcd de muchos elementos suele ser pequeño y Euclides termina en pocos pasos), así que la consulta se comporta casi como $O(1)$, pero el límite formal es $O(\log M)$.

```

1 struct SparseTableGcd {
2     int n;
3     vector<vector<long long>> sp; // sp[k][i] = gcd del bloque
        [i, i + 2^k - 1]
4     vector<int> lg;
5
6     // CREATE: precalcula la tabla de gcd (los valores se toman
        en valor absoluto).
7     SparseTableGcd(const vector<long long>& a) {
8         n = (int)a.size();
9         lg.assign(n + 1, 0);
10        for (int i = 2; i <= n; i++) lg[i] = lg[i / 2] + 1;
11
12        int K = lg[n] + 1;
13        sp.assign(K, vector<long long>(n));
14        for (int i = 0; i < n; i++) sp[0][i] = a[i] < 0 ? -a[i] : a[i];
15        for (int k = 1; k < K; k++)
16            for (int i = 0; i + (1 << k) <= n; i++)
17                sp[k][i] = __gcd(sp[k - 1][i], sp[k - 1][i + (1 << (k - 1))]);
18    } // O(n log n log M)
19
20    // READ: gcd del rango [l, r] (0-indexado, inclusivo).
21    long long consultar(int l, int r) const {
22        int k = lg[r - l + 1];
23        return __gcd(sp[k][l], sp[k][r - (1 << k) + 1]);
24    } // O(log M), casi O(1) en la practica
25
26    // READ: indica si algun elemento del rango es multiplo de g
        ... (ver modificacion 2).
27 };
28
29 // -----
30 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
31 // -----
32 // 1. Contar cuantos subarreglos tienen gcd igual a g (o mayor
        que 1): para cada extremo
33 // derecho r, el gcd de [l, r] solo cambia O(log M) veces al
        mover l, porque cada cambio
34 // divide el valor por al menos 2. Se hace busqueda binaria
        sobre l con consultar().
35 // O(n log n log M) en total.
36 //
37 // 2. Mayor subarreglo cuyo gcd sea > 1: dos punteros o
        busqueda binaria sobre el largo,
38 // usando consultar() para comprobar cada ventana.
39 //
40 // 3. Si ademas se necesitan actualizaciones puntuales de gcd:
        arbol de segmentos con gcd
41 // como combinacion. O(log n + log M) por consulta; cambiar
        un valor cuesta O(log n log M).

```

```

42 //
43 // 4. Si la operacion es min y gcd a la vez (por ejemplo "gcd y
44 //    cuantas veces aparece el
45 //    minimo"), guardar un par en cada celda y combinar por
46 //    componentes.

```

Ejemplo de funcionamiento:

```

1 int main() {
2     vector<long long> a = {12, 18, 30, 45, 15, 9};
3     SparseTableGcd st(a);
4
5     cout << st.consultar(0, 2) << "\n"; // 6 (gcd de 12, 18,
6     cout << st.consultar(2, 4) << "\n"; // 15 (gcd de 30, 45,
7     cout << st.consultar(1, 3) << "\n"; // 3 (gcd de 18, 30,
8     cout << st.consultar(1, 1) << "\n"; // 18 (rango de un
9     cout << st.consultar(0, 5) << "\n"; // 3 (gcd de todos)
10    return 0;
11 }

```

Nota: La línea de la modificación sobre «algún elemento es múltiplo de g » del struct es un comentario huérfano en el código de arriba, bórrala. Además, `__gcd` con negativos puede devolver un resultado negativo, por eso el constructor guarda el valor absoluto. Con ceros todo sigue funcionando porque $\gcd(0, x) = x$.

Disjoint Sparse Table (operaciones no idempotentes: suma, xor)

Idea: En vez de dos bloques que se solapan, el nivel k parte el arreglo en bloques de largo 2^{k+1} y, para cada bloque, guarda las sumas desde su punto medio hacia afuera: hacia la izquierda (de `mid-1` hacia `start`) y hacia la derecha (de `mid` hacia `end-1`). Una consulta $[l, r]$ con $l < r$ cae en el nivel k donde l y r quedan en mitades distintas del mismo bloque, que es el bit más alto en el que difieren. Entonces $[l, r]$ es la unión disjunta de $[l, \text{mid}]$ y $[\text{mid}, r]$, y la respuesta es `pre[k][l] + pre[k][r]`, sin solapes. Como no se necesita idempotencia ni inverso, sirve para cualquier operación asociativa.

Complejidad: Hay $\lceil \log_2 n \rceil + 1$ niveles y cada uno recorre las n celdas una vez, así que construir cuesta $O(n \log n)$ en tiempo y memoria. La consulta obtiene el nivel con una instrucción de bits y lee dos celdas: $O(1)$.

```

1 struct DisjointSparseTable {
2     int n, niveles;
3     vector<long long> a;
4     vector<vector<long long>> pre; // pre[k][i]: suma desde i
5     // hasta el punto medio de su bloque
6
7     // CREATE: precalcula las sumas hacia el punto medio de cada
8     // bloque, por nivel.
9     DisjointSparseTable(const vector<long long>& arr) : n((int)
10    arr.size()), a(arr) {
11         niveles = 1;
12         while ((1 << niveles) < n) niveles++;
13         pre.assign(niveles, vector<long long>(n, 0));
14
15         for (int k = 0; k < niveles; k++) {
16             int bloque = 1 << (k + 1);
17             for (int ini = 0; ini < n; ini += bloque) {
18                 int mid = min(ini + bloque / 2, n);
19                 int fin = min(ini + bloque, n);
20                 if (mid - 1 >= ini) { // mitad
21                     izquierda
22                     pre[k][mid - 1] = a[mid - 1];
23                     for (int i = mid - 2; i >= ini; i--) pre[k][i] = a[i]
24                     + pre[k][i + 1];
25                 }
26                 if (mid < fin) { // mitad
27                     derecha
28                     pre[k][mid] = a[mid];
29                     for (int i = mid + 1; i < fin; i++) pre[k][i] = pre[k]
30                     [i - 1] + a[i];
31                 }
32             }
33         }
34     }
35 }

```

```

25 }
26 } // O(n log n)
27
28 // READ: suma del rango [l, r] (0-indexado, inclusivo).
29 long long consultar(int l, int r) const {
30     if (l == r) return a[l];
31     int k = 31 - __builtin_clz(1 ^ r); // bit mas alto donde
32     // difieren l y r
33     return pre[k][l] + pre[k][r];
34 } // O(1)
35
36 // -----
37 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
38 // -----
39 // 1. Cualquier operacion asociativa (xor, producto de matrices,
40 //    composicion de funciones):
41 //    cambiar "+" por la operacion en el constructor y en
42 //    consultar. Si NO es conmutativa,
43 //    el orden importa: izquierda primero, derecha despues, tal
44 //    como esta escrito.
45 // Para xor: pre[k][i] = a[i] ^ pre[k][i + 1]; y
46 // return pre[k][l] ^ pre[k][r];
47 //
48 // 2. Suma de rango con actualizaciones: no sirve (cambiar un
49 //    elemento invalida O(n) celdas
50 //    por nivel). Usar Fenwick o arbol de segmentos.
51 //
52 // 3. Para suma y xor SIN actualizaciones, un arreglo de
53 //    prefijos es mas simple y usa O(n)
54 //    de memoria: suma = P[r+1] - P[l], xor = P[r+1] ^ P[l]. La
55 //    Disjoint Sparse Table solo
56 //    gana cuando la operacion no tiene inverso (producto de
57 //    matrices, min con conteo...).
58 //
59 // 4. Con n potencia de 2 se evita el recorte min(..., n) y el
60 //    código es mas corto.

```

Ejemplo de funcionamiento:

```

1 int main() {
2     vector<long long> a = {5, 2, 9, 1, 7, 3};
3     DisjointSparseTable dst(a);
4
5     cout << dst.consultar(0, 2) << "\n"; // 16 (5 + 2 + 9)
6     cout << dst.consultar(2, 5) << "\n"; // 20 (9 + 1 + 7 + 3)
7     cout << dst.consultar(1, 1) << "\n"; // 2 (un solo
8     // elemento)
9     cout << dst.consultar(0, 5) << "\n"; // 27 (suma total)
10    return 0;
11 }

```

Nota: Para `consultar(0, 2)`, $0 \oplus 2 = 2$ y su bit más alto es el 1, así que se usa el nivel 1, con bloques de largo 4 y punto medio en 2: `pre[1][0] = 5 + 2 = 7` y `pre[1][2] = 9`, total 16. Una consulta con $l = r$ se responde aparte porque $l \oplus r = 0$ no tiene bit alto. En un nivel, la última mitad derecha puede quedar recortada por n , pero nunca se consulta una posición inexistente. Por último, los valores guardados son sumas, así que con elementos del orden de 10^9 y n grande conviene `long long`.

Regla práctica: Si el arreglo no cambia y la operación es idempotente (min, max, gcd, AND, OR), usa Sparse Table: consulta $O(1)$ y es el código más corto. Si la operación es asociativa pero no idempotente ni invertible, usa Disjoint Sparse Table. Si es suma o xor sin actualizaciones, un arreglo de prefijos basta. En cuanto el arreglo cambia, aunque sea una sola vez, pasa a un árbol de segmentos o a un Fenwick, porque reconstruir en $O(n \log n)$ no compensa. Si la memoria $O(n \log n)$ es un problema (n del orden de 10^7), considera un árbol de segmentos o procesar las consultas por niveles (modificación 7).

Variante: LCM en un Rango

Idea: El mcm es idempotente, $\text{lcm}(x, x) = x$, además de asociativo y conmutativo, así que encaja en la Sparse

Table igual que el mínimo o el gcd: el solape de los dos bloques no altera el resultado, porque el mcm de una unión de elementos no cambia si uno se cuenta dos veces. Se combina con $\text{lcm}(x, y) = \frac{x}{\text{gcd}(x, y)} \cdot y$, dividiendo antes de multiplicar para que el intermedio sea lo más pequeño posible.

La diferencia con el gcd es que el mcm *crece*: el mcm de un bloque es múltiplo del de cualquiera de sus sub-bloques, y puede superar enseguida cualquier tipo entero (el mcm de $1, \dots, 43$ ya pasa de 10^{18}). La solución es *saturar*: se fija un límite LIM (aquí 10^{18}) y todo mcm mayor que LIM se guarda como un valor centinela INF = LIM + 1. Esto es correcto porque el mcm es monótono: si un sub-bloque ya excede LIM, cualquier bloque que lo contenga también lo excede, así que combinar con INF da INF sin necesidad de conocer el valor real. El producto intermedio $(x/g) \cdot y$ puede llegar a 10^{36} , que cabe en `__int128` ($\approx 1.7 \cdot 10^{38}$) pero no en `long long`. Se asume que todos los valores son al menos 1 (el mcm con 0 no está bien definido).

Complejidad: Cada combinación es un gcd de $O(\log M)$ más una multiplicación y una comparación en $O(1)$, así que construir cuesta $O(n \log n \log M)$ y cada consulta $O(\log M)$, como en el gcd. Memoria: $O(n \log n)$.

```

1 // Valores >= 1. Si el mcm de un rango supera LIM, la consulta
2 // devuelve INF (= LIM + 1).
3 struct SparseTableLcm {
4     static constexpr long long LIM = 1000000000000000000LL; //
5     // 10^18
6     static constexpr long long INF = LIM + 1; //
7     // centinela: "mayor que LIM"
8     int n;
9     vector<vector<long long>>> sp; // sp[k][i] = mcm saturado
10    // del bloque [i, i + 2^k - 1]
11    vector<int> lg;
12
13    // READ: mcm saturado de dos valores en [1, INF].
14    static long long mcm(long long x, long long y) {
15        if (x > LIM || y > LIM) return INF; // ya excede:
16        // cualquier superconjunto tambien
17        long long g = __gcd(x, y);
18        __int128 r = (__int128)(x / g) * y; // dividir
19        // antes de multiplicar
20        return r > LIM ? INF : (long long)r;
21    } // O(log M)
22
23    // CREATE: precalcula la tabla de mcm a partir de a[0..n-1] (
24    // valores >= 1).
25    SparseTableLcm(const vector<long long>& a) {
26        n = (int)a.size();
27        lg.assign(n + 1, 0);
28        for (int i = 2; i <= n; i++) lg[i] = lg[i / 2] + 1;
29
30        int K = lg[n] + 1;
31        sp.assign(K, vector<long long>(n));
32        for (int i = 0; i < n; i++) sp[0][i] = min(a[i], INF);
33        for (int k = 1; k < K; k++)
34            for (int i = 0; i + (1 << k) <= n; i++)
35                sp[k][i] = mcm(sp[k - 1][i], sp[k - 1][i + (1 << (k - 1))]);
36    } // O(n log n log M)
37
38    // READ: mcm del rango [l, r] (0-indexado, inclusivo), o INF
39    // si supera LIM.
40    long long consultar(int l, int r) const {
41        int k = lg[r - l + 1];
42        return mcm(sp[k][l], sp[k][r - (1 << k) + 1]);
43    } // O(log M)
44
45    // READ: indica si el mcm del rango es exacto (no saturado).
46    bool exacto(int l, int r) const {
47        return consultar(l, r) <= LIM;
48    } // O(log M)
49 };
50
51 // -----
52 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
53 // -----
54 // 1. Otro limite: el problema suele fijar uno (por ejemplo

```

```

10^9 o 2^63 - 1). Cambiar LIM.
47 // Con LIM <= 3 * 10^9 el producto cabe en unsigned long
48 // long y se puede evitar __int128;
49 // con LIM = 10^18 es obligatorio (o una comprobacion previa:
50 // x / g > LIM / y => INF).
51 //
52 // 2. mcm MODULO un primo p: NO se puede combinar residuos (lcm
53 // (x mod p, y mod p) no es
54 // lcm(x, y) mod p). Hay que factorizar y guardar, por cada
55 // primo, el MAXIMO exponente
56 // del rango. Con pocos primos distintos (por ejemplo los
57 // primos <= 50, que son 15), se
58 // hacen 15 Sparse Tables de maximo sobre los exponentes y
59 // la consulta es el producto de
60 // p^e mod M sobre los primos: O(#primos * log e) por
61 // consulta. Con muchos primos grandes,
62 // es mas practico un arbol de segmentos con mapas de
63 // exponentes.
64 //
65 // 3. Cuantos subarreglos tienen mcm <= LIM (o mcm igual a v):
66 // para cada extremo izquierdo l,
67 // el mcm de [l, r] solo cambia O(log LIM) veces al avanzar
68 // r (cada cambio lo multiplica
69 // por al menos 2), asi que se hace busqueda binaria sobre r
70 // con consultar().
71 // O(n log LIM * log M) en total.
72 //
73 // 4. Actualizaciones puntuales: arbol de segmentos con mcm
74 // saturado como combinacion
75 // (misma funcion mcm). O(log n * log M) por operacion.
76 //
77 // 5. Guardar el mcm sin saturar: usar BigInt (ver "Numeros
78 // Grandes") como tipo de cada celda.
79 // Cada combinacion se vuelve mucho mas cara y requiere
80 // division de BigInt, asi que solo
81 // se justifica si el enunciado pide el valor exacto de un
82 // mcm enorme.

```

Ejemplo de funcionamiento:

```

1 int main() {
2     vector<long long> a = {4, 6, 10, 3, 8, 15};
3     SparseTableLcm st(a);
4
5     cout << st.consultar(0, 2) << "\n"; // 60 (mcm de 4, 6,
6     // 10)
7     cout << st.consultar(1, 3) << "\n"; // 30 (mcm de 6, 10,
8     // 3)
9     cout << st.consultar(2, 4) << "\n"; // 120 (mcm de 10, 3,
10    // 8)
11    cout << st.consultar(1, 1) << "\n"; // 6 (un solo
12    // elemento)
13    cout << st.consultar(0, 5) << "\n"; // 120 (mcm de todos:
14    // 2^3 * 3 * 5)
15
16    // Saturacion: tres primos cercanos a 10^9, coprimos entre si.
17
18    vector<long long> b = {10000000007LL, 998244353LL, 1000000009
19    // LL};
20    SparseTableLcm sb(b);
21    cout << sb.consultar(0, 1) << "\n"; // 998244359987710471
22    // (producto de los dos, < 10^18)
23    cout << sb.consultar(0, 2) << "\n"; // 1000000000000000001
24    // (INF: el mcm real, ~10^27, excede LIM)
25    cout << sb.exacto(0, 2) << "\n"; // 0
26    return 0;
27 }

```

Nota: Un resultado igual a INF significa únicamente «el mcm es mayor que LIM», no un valor concreto, así que no se puede usar para nada más que compararlo con un tope. Con ese tope el mcm tiene además una propiedad útil: al ir ampliando un rango, el mcm solo puede cambiar unas $\log_2(\text{LIM}) \approx 60$ veces antes de saturarse, y por eso las consultas «cuántos subarreglos tienen $\text{mcm} \leq \text{tope}$ » son baratas (modificación 3). Si algún elemento puede ser 0, hay que tratarlo aparte antes de construir la tabla, porque x / g dividiría por cero cuando ambos valores son 0.

5.6 Segment Tree

Idea: un árbol binario implícito (guardado en un arreglo) donde cada nodo representa el resultado combinado de un rango del arreglo original. La raíz cubre todo $[0, n - 1]$;

cada nodo se parte en dos mitades hasta llegar a las hojas, que son elementos individuales. Un update baja por el árbol tocando $O(\log n)$ nodos (uno por nivel); una query de rango se descompone en a lo más $O(\log n)$ nodos que cubren exactamente el rango pedido, sin necesidad de bloques parciales como en Sqrt Decomposition. A diferencia de Fenwick, sirve para **cualquier** operación asociativa (min, max, gcd, and, or, no solo sumas), y con Lazy Propagation soporta actualizaciones de rango completo.

¿Por qué usarlo? Es la estructura más versátil de esta sección: $O(\log n)$ tanto para update como para query, funciona con casi cualquier operación asociativa, y con lazy propagation resuelve range update + range query para operaciones que Fenwick no puede (min, max, asignar un valor a todo un rango, etc.).

Casos de uso:

- Min/max/gcd de rango con actualizaciones (a diferencia de Sparse Table, que exige arreglo estático).
- Range update + range query con operaciones no invertibles: asignar un valor a todo un rango, min/max con suma en rango, etc.
- Cualquier operación asociativa que no sea una suma simple (donde Fenwick ya alcanzaría y sería más simple).
- Base para variantes más avanzadas: Segment Tree 2D, Merge Sort Tree, Persistent Segment Tree, Segment Tree Beats.

Complejidad: construir $O(n)$, update puntual $O(\log n)$, query de rango $O(\log n)$, range update + range query con lazy propagation $O(\log n)$.

Segment Tree clásico (point update, range query — suma)

```

1 struct SegmentTree {
2     vector<long long> tree, a;
3     int n;
4
5     // CREATE: reserva el arbol (tamano 4n es cota segura) y
6     // construye recursivamente a partir de a[0..n-1].
7     SegmentTree(vector<long long>& arr) {
8         a = arr;
9         n = a.size();
10        tree.assign(4 * n, 0);
11        if (n > 0)
12            build(1, 0, n - 1);
13    } // O(n)
14
15    void build(int node, int l, int r) {
16        if (l == r) {
17            tree[node] = a[l];
18            return;
19        }
20
21        int mid = (l + r) / 2;
22        build(2 * node, l, mid);
23        build(2 * node + 1, mid + 1, r);
24        tree[node] = tree[2 * node] + tree[2 * node + 1];
25    }
26
27    void updateUtil(int node, int l, int r, int i, long long x) {
28        if (l == r) {
29            tree[node] = x;
30            return;
31        }
32
33        int mid = (l + r) / 2;
34        if (i <= mid)
35            updateUtil(2 * node, l, mid, i, x);
36        else

```

```

37        updateUtil(2 * node + 1, mid + 1, r, i, x);
38        tree[node] = tree[2 * node] + tree[2 * node + 1];
39    }
40
41    // UPDATE: fija el valor absoluto de la posicion i en x.
42    void update(int i, long long x) {
43        updateUtil(1, 0, n - 1, i, x);
44    } // O(log n)
45
46    long long queryUtil(int node, int l, int r, int ql, int qr) {
47        if (qr < l || r < ql)
48            return 0; // neutro para
49            suma
50
51        if (ql <= l && r <= qr)
52            return tree[node];
53
54        int mid = (l + r) / 2;
55        return queryUtil(2 * node, l, mid, ql, qr)
56            + queryUtil(2 * node + 1, mid + 1, r, ql, qr);
57    }
58
59    // READ: suma del rango [l, r] (0-indexado, inclusivo).
60    long long rango(int l, int r) {
61        return queryUtil(1, 0, n - 1, l, r);
62    } // O(log n)
63 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> a = {5, 2, 9, 1, 7};
2
3 SegmentTree st(a);
4 // arreglo logico: [5, 2, 9, 1, 7] (posiciones 0..4)
5
6 cout << st.rango(1, 3);
7 // 2 + 9 + 1 = 12
8
9 st.update(2, 100);
10 // arreglo logico: [5, 2, 100, 1, 7]
11
12 cout << st.rango(1, 3);
13 // 2 + 100 + 1 = 103

```

Nota: para min/max/gcd solo cambia el neutro del caso base (LLONG_MAX para min, 0 para gcd, etc.) y el operador de combinación (min(...), __gcd(...)) en build, updateUtil y queryUtil.

Segment Tree con Lazy Propagation (range update + range query — suma)

Idea: cada nodo guarda un valor lazy pendiente que representa “sumar v a todo este rango” sin aplicarlo todavía a los hijos. Se aplica (*se propaga*) solo cuando hace falta bajar más en el árbol para otra operación.

```

1 struct SegmentTreeLazyAdd {
2     vector<long long> tree, lazy, a;
3     int n;
4
5     // CREATE: reserva arbol + lazy, construye a partir de a[0..n-1].
6     SegmentTreeLazyAdd(vector<long long>& arr) {
7         a = arr;
8         n = a.size();
9         tree.assign(4 * n, 0);
10        lazy.assign(4 * n, 0);
11        if (n > 0)
12            build(1, 0, n - 1);
13    } // O(n)
14
15    void build(int node, int l, int r) {
16        if (l == r) {
17            tree[node] = a[l];
18            return;
19        }
20
21        int mid = (l + r) / 2;
22        build(2 * node, l, mid);
23        build(2 * node + 1, mid + 1, r);
24        tree[node] = tree[2 * node] + tree[2 * node + 1];
25    }
26
27    // Aplica el lazy pendiente de este nodo y lo empuja a los
28    // hijos.
29    void propagar(int node, int l, int r) {

```



```

29     if (lazy[node] == 0)
30         return;
31
32     tree[node] += lazy[node] * (r - l + 1);
33
34     if (l != r) {
35         lazy[2 * node] += lazy[node];
36         lazy[2 * node + 1] += lazy[node];
37     }
38
39     lazy[node] = 0;
40 }
41
42 void rangoUpdateUtil(int node, int l, int r, int ql, int qr,
43                     long long v) {
44     propagar(node, l, r);
45
46     if (qr < l || r < ql)
47         return;
48
49     if (ql <= l && r <= qr) {
50         lazy[node] += v;
51         propagar(node, l, r);
52         return;
53     }
54
55     int mid = (l + r) / 2;
56     rangoUpdateUtil(2 * node, l, mid, ql, qr, v);
57     rangoUpdateUtil(2 * node + 1, mid + 1, r, ql, qr, v);
58     tree[node] = tree[2 * node] + tree[2 * node + 1];
59 }
60
61 // UPDATE: suma v a todo el rango [l, r].
62 void rangoUpdate(int l, int r, long long v) {
63     rangoUpdateUtil(1, 0, n - 1, l, r, v);
64 } // O(log n)
65
66 long long queryUtil(int node, int l, int r, int ql, int qr) {
67     propagar(node, l, r);
68
69     if (qr < l || r < ql)
70         return 0;
71
72     if (ql <= l && r <= qr)
73         return tree[node];
74
75     int mid = (l + r) / 2;
76     return queryUtil(2 * node, l, mid, ql, qr)
77         + queryUtil(2 * node + 1, mid + 1, r, ql, qr);
78 }
79
80 // READ: suma del rango [l, r].
81 long long rango(int l, int r) {
82     return queryUtil(1, 0, n - 1, l, r);
83 } // O(log n)
84 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> a = {1, 1, 1, 1, 1, 1};
2
3 SegmentTreeLazyAdd st(a);
4
5 st.rangoUpdate(1, 4, 10);
6 // arreglo logico efectivo: [1, 11, 11, 11, 11, 1]
7
8 cout << st.rango(0, 5);
9 // 1+11+11+11+11+1 = 46
10
11 cout << st.rango(2, 3);
12 // 11+11 = 22

```

Lazy Propagation con asignación (range assign + range query — mínimo)

Idea: en vez de *sumar* un valor al rango, lo **reemplaza** por completo (asigna x a todo $[l, r]$). El lazy aquí no se acumula como en la suma: el último assign pisa a cualquier assign anterior pendiente, así que se necesita una bandera aparte para distinguir “no hay lazy pendiente” de “hay que asignar 0”.

```

1 struct SegmentTreeAssignMin {
2     vector<long long> tree, lazy, a;
3     vector<bool> tieneLazy;
4     int n;
5

```

```

6     // CREATE: reserva arbol + lazy + banderas, construye desde a.
7
8     SegmentTreeAssignMin(vector<long long>& arr) {
9         a = arr;
10        n = a.size();
11        tree.assign(4 * n, LLONG_MAX);
12        lazy.assign(4 * n, 0);
13        tieneLazy.assign(4 * n, false);
14        if (n > 0)
15            build(1, 0, n - 1);
16    } // O(n)
17
18    void build(int node, int l, int r) {
19        if (l == r) {
20            tree[node] = a[l];
21            return;
22        }
23
24        int mid = (l + r) / 2;
25        build(2 * node, l, mid);
26        build(2 * node + 1, mid + 1, r);
27        tree[node] = min(tree[2 * node], tree[2 * node + 1]);
28    }
29
30    // Empuja el assign pendiente de este nodo hacia sus hijos.
31    void propagar(int node) {
32        if (!tieneLazy[node])
33            return;
34
35        for (int hijo : {2 * node, 2 * node + 1}) {
36            tree[hijo] = lazy[node];
37            lazy[hijo] = lazy[node];
38            tieneLazy[hijo] = true;
39        }
40
41        tieneLazy[node] = false;
42    }
43
44    void rangoAssignUtil(int node, int l, int r, int ql, int qr,
45                        long long x) {
46        if (qr < l || r < ql)
47            return;
48
49        if (ql <= l && r <= qr) {
50            tree[node] = x;
51            lazy[node] = x;
52            tieneLazy[node] = true;
53            return;
54        }
55
56        propagar(node);
57        int mid = (l + r) / 2;
58        rangoAssignUtil(2 * node, l, mid, ql, qr, x);
59        rangoAssignUtil(2 * node + 1, mid + 1, r, ql, qr, x);
60        tree[node] = min(tree[2 * node], tree[2 * node + 1]);
61    }
62
63    // UPDATE: asigna x a todo el rango [l, r].
64    void rangoAssign(int l, int r, long long x) {
65        rangoAssignUtil(1, 0, n - 1, l, r, x);
66    } // O(log n)
67
68    long long queryUtil(int node, int l, int r, int ql, int qr) {
69        if (qr < l || r < ql)
70            return LLONG_MAX;
71
72        if (ql <= l && r <= qr)
73            return tree[node];
74
75        propagar(node);
76        int mid = (l + r) / 2;
77        return min(queryUtil(2 * node, l, mid, ql, qr),
78                  queryUtil(2 * node + 1, mid + 1, r, ql, qr));
79    }
80
81    // READ: mínimo del rango [l, r].
82    long long rango(int l, int r) {
83        return queryUtil(1, 0, n - 1, l, r);
84    } // O(log n)
85 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> a = {5, 2, 9, 1, 7, 3};
2
3 SegmentTreeAssignMin st(a);
4
5 cout << st.rango(0, 5);
6 // min(5,2,9,1,7,3) = 1
7
8 st.rangoAssign(0, 2, 100);
9 // arreglo logico efectivo: [100, 100, 100, 1, 7, 3]
10

```



```

11 cout << st.rango(0, 2);
12 // min(100,100,100) = 100
13
14 cout << st.rango(0, 5);
15 // min(100,100,100,1,7,3) = 1

```

Comparación con las otras estructuras de rango:

Estructura	Update	Query	Min/Max?
Segment Tree	$O(\log n)$	$O(\log n)$	Sí
Segment Tree + Lazy	$O(\log n)$ range	$O(\log n)$ range	Sí
FenwickTree	$O(\log n)$ punto	$O(\log n)$ rango	No
Sqrt Decomposition	$O(1)-O(\sqrt{n})$	$O(\sqrt{n})$	Sí

Table 5: Segment Tree vs. Fenwick vs. Sqrt Decomposition.

Regla práctica: usa Segment Tree cuando necesites min/max/gcd (operaciones no invertibles que Fenwick no soporta), o cuando necesites range update + range query con una operación que no sea suma simple. Si la operación es solo suma y no necesitas range update, Fenwick es más simple y más rápido en la práctica (menor constante). Si no confías en escribir el lazy propagation correcto bajo presión, Sqrt Decomposition es una alternativa más simple a costa de $O(\sqrt{n})$ en vez de $O(\log n)$.

5.7 RMQ (Range Minimum Query)

Idea: RMQ es el problema general de "dado un arreglo, encuentra el mínimo en cualquier rango $[l, r]$ ". Ya tienes las dos estructuras que lo resuelven: **Sparse Table** si el arreglo es estático ($O(1)$ por query) y **Segment Tree** si cambia ($O(\log n)$ por query). Esta sección no repite esas estructuras, sino que muestra las dos aplicaciones clásicas donde un problema que *no parece* de rango se puede **reformular** como RMQ sobre un arreglo auxiliar: LCA en árboles, y LCP entre sufijos arbitrarios de un string.

¿Por qué usarlo? Reconocer cuándo un problema es "RMQ disfrazado" es lo que realmente separa resolver estos problemas en $O(1)-O(\log n)$ de resolverlos con fuerza bruta $O(n)$ por consulta. El patrón se repite: construyes un arreglo auxiliar donde la respuesta a tu pregunta real equivale al mínimo de un rango, y le aplicas Sparse Table o Segment Tree normal.

Casos de uso:

- LCA (ancestro común más bajo) de dos nodos en un árbol, con muchas queries: Euler Tour + RMQ.
- LCP (prefijo común más largo) entre dos sufijos *cualquiera* de un string (no necesariamente consecutivos en el Suffix Array): Suffix Array + Kasai + RMQ sobre el LCP Array.
- Cualquier problema donde la respuesta se pueda expresar como "el mínimo de algo" sobre un rango de índices auxiliares.

Complejidad: en ambas aplicaciones, el precómputo (Euler Tour o Suffix Array + Kasai) es $O(n)$ a $O(n \log^2 n)$ según el método de Suffix Array usado; una vez armado el arreglo auxiliar, el RMQ es $O(n \log n)$ de build y $O(1)$ por query con Sparse Table (ambos casos son estáticos: el árbol y el string no cambian).

LCA vía Euler Tour + RMQ (Sparse Table)

Idea: recorres el árbol con un Euler Tour, anotando el nodo visitado cada vez que **entras o vuelves** a él (arreglo de tamaño $2n - 1$) junto con su profundidad. El LCA de u y v es el nodo de **menor profundidad** que aparece entre la primera aparición de u y la primera aparición de v en ese recorrido — exactamente un RMQ (por profundidad) sobre el arreglo del Euler Tour.

```

1 struct LCA {
2     vector<int> euler, primeraApar, profundidad;
3     vector<vector<int>> sp; // sp[k][i] = nodo con menor
        profundidad
4     vector<int> lg;
5
6     // CREATE: construye el Euler Tour del arbol (arr. de tamaño
        // 2n-1) y la Sparse Table sobre profundidad, para RMQ O(1).
7     LCA(vector<vector<int>>& adj, int raiz, int n) {
8         primeraApar.assign(n, -1);
9         profundidad.assign(n, 0);
10        euler.reserve(2 * n);
11
12        dfs(adj, raiz, -1, 0);
13        build();
14    } // O(n log n)
15
16    void dfs(vector<vector<int>>& adj, int u, int p, int d) {
17        primeraApar[u] = euler.size();
18        profundidad[u] = d;
19        euler.push_back(u);
20
21        for (int v : adj[u]) {
22            if (v != p) {
23                dfs(adj, v, u, d + 1);
24                euler.push_back(u);
25            }
26        }
27    }
28
29    void build() {
30        int m = euler.size();
31        lg.assign(m + 1, 0);
32        for (int i = 2; i <= m; i++)
33            lg[i] = lg[i / 2] + 1;
34
35        int K = lg[m] + 1;
36        sp.assign(K, vector<int>(m));
37        sp[0] = euler;
38
39        for (int k = 1; k < K; k++)
40            for (int i = 0; i + (1 << k) <= m; i++) {
41                int l = sp[k - 1][i];
42                int r = sp[k - 1][i + (1 << (k - 1))];
43                sp[k][i] = profundidad[l] < profundidad[r] ? l :
44                    r;
45            }
46    }
47
48    // READ: LCA de u y v.
49    int query(int u, int v) {
50        int l = primeraApar[u], r = primeraApar[v];
51        if (l > r) swap(l, r);
52
53        int k = lg[r - l + 1];
54        int a = sp[k][l], b = sp[k][r - (1 << k) + 1];
55        return profundidad[a] < profundidad[b] ? a : b;
56    } // O(1)
57 };

```

Ejemplo de funcionamiento:

```

1 // arbol:      0
2 //             / \
3 //            1  2
4 //           / \
5 //          3  4
6
7 vector<vector<int>> adj(5);
8 adj[0] = {1, 2};
9 adj[1] = {0, 3, 4};
10 adj[2] = {0};
11 adj[3] = {1};
12 adj[4] = {1};
13
14 LCA lca(adj, 0, 5);
15
16 cout << lca.query(3, 4);
17 // 1 (padre comun de 3 y 4)
18

```

```

19 cout << lca.query(3, 2);
20 // 0 (raiz)
21
22 cout << lca.query(1, 4);
23 // 1 (uno es ancestro del otro)

```

LCP entre dos sufijos arbitrarios (Suffix Array + Kasai + RMQ)

Idea: el LCP Array (via Kasai) solo te da el LCP entre sufijos **consecutivos** en el Suffix Array. Para el LCP entre dos sufijos **cualquiera** i, j : los ubicas por su rango en el SA ($\text{rnk}[i], \text{rnk}[j]$) y el LCP entre ambos es el **mínimo** del LCP Array en el rango entre esas dos posiciones —otra vez RMQ, esta vez sobre el LCP Array.

```

1 struct SuffixLCPQuery {
2     vector<int> sa, rnk, lcp;
3     vector<vector<int>> sp;
4     vector<int> lg;
5     int n;
6
7     // CREATE: construye SA, LCP (Kasai) y Sparse Table sobre lcp,
8
9     // todo en un solo paso.
10    SuffixLCPQuery(const string& s) {
11        n = s.size();
12        sa = buildSuffixArray(s);           // O(n log^2 n)
13        lcp = buildLCPArray(s, sa);         // O(n), Kasai
14
15        rnk.assign(n, 0);
16        for (int i = 0; i < n; i++)
17            rnk[sa[i]] = i;
18
19        buildSparse();                      // O(n log^2 n)
20    }
21
22    void buildSparse() {
23        lg.assign(n + 1, 0);
24        for (int i = 2; i <= n; i++)
25            lg[i] = lg[i / 2] + 1;
26
27        int K = lg[n] + 1;
28        sp.assign(K, vector<int>(n));
29        sp[0] = lcp;
30
31        for (int k = 1; k < K; k++)
32            for (int i = 0; i + (1 << k) <= n; i++)
33                sp[k][i] = min(sp[k - 1][i],
34                               sp[k - 1][i + (1 << (k - 1))]);
35    }
36
37    // READ: LCP entre el sufijo que empieza en i y el que
38    // empieza en j (0-indexado sobre el string original).
39    int lcpEntreSufijos(int i, int j) {
40        if (i == j) return n - i;
41
42        int a = rnk[i], b = rnk[j];
43        if (a > b) swap(a, b);
44
45        // el mínimo va de [a+1, b] en el LCP array
46        int k = lg[b - a];
47        return min(sp[k][a + 1], sp[k][b - (1 << k) + 1]);
48    }
49 };

```

Ejemplo de funcionamiento:

```

1 SuffixLCPQuery slq("banana");
2
3 cout << slq.lcpEntreSufijos(1, 3);
4 // sufijo desde 1: "anana"
5 // sufijo desde 3: "ana"
6 // LCP = "ana" -> 3
7
8 cout << slq.lcpEntreSufijos(0, 3);
9 // sufijo desde 0: "banana"
10 // sufijo desde 3: "ana"
11 // LCP = "" -> 0

```

Nota: `buildSuffixArray` y `buildLCPArray` son las funciones estándar de Suffix Array + Kasai (ver la sección de Algoritmos de Strings); aquí solo se reutilizan como precómputo antes de aplicar RMQ.

Regla práctica: cuando un problema pida "algo" entre dos elementos que no son adyacentes en su estructura original (dos nodos de un árbol, dos sufijos de un string, dos elementos de cualquier secuencia con una noción de "orden" y "profundidad/afinidad"), pregúntate si esa relación se puede expresar como el mínimo de un arreglo auxiliar construido con un recorrido lineal. Si sí, el problema se reduce a Sparse Table (estático) o Segment Tree (dinámico), ya cubiertos en las secciones anteriores.

5.8 Mo's Algorithm

Idea: Mo's responde q consultas de rango *fuera de línea* (todas conocidas de antemano) manteniendo una ventana $[izq, der]$ sobre el arreglo, junto con la respuesta de esa ventana. Para pasar de una consulta a la siguiente no se recalcula nada: se mueven los extremos con **agregar** y **quitar**, cada una en $O(1)$. Recalcular cada consulta desde cero cuesta $O(nq)$. El costo total es la suma de lo que se desplazan los dos extremos, y eso depende del *orden* en que se atienden las consultas.

El orden se elige así. Se divide el arreglo en bloques de tamaño B y las consultas se agrupan por el bloque de su extremo izquierdo l . Dentro de un bloque se ordenan por r creciente. Con ese orden, el puntero *der* recorre el arreglo una sola vez por bloque, con costo $O(n)$ por bloque y $O(n^2/B)$ en total. El puntero *izq* nunca sale de su bloque, así que se mueve a lo sumo B posiciones por consulta, $O(qB)$ en total. El costo es $O(n^2/B + qB)$, que se minimiza con $B = n/\sqrt{q}$ y da $O(n\sqrt{q})$. Con $B = \sqrt{n}$ queda $O((n + q)\sqrt{n})$. Una mejora práctica es alternar el sentido de r en los bloques pares e impares: así el puntero *der* no vuelve al inicio al cambiar de bloque.

Si además hay actualizaciones puntuales entre las consultas, cada consulta lleva un tercer número: el tiempo t (cuántas actualizaciones ya se aplicaron). Se ordena por $(\text{bloque}(l), \text{bloque}(r), t)$ y se agrega un tercer puntero *curT* que aplica o revierte actualizaciones una por una. Con bloques de tamaño $B \approx n^{2/3}$ hay $(n/B)^2 = n^{2/3}$ pares de bloques, y en cada uno *curT* recorre a lo sumo las U actualizaciones, lo que da $O(n^{2/3}U)$. Los extremos l y r suman $O(qB)$. Con $q \approx U \approx n$ el total es $O(n^{5/3})$.

¿Por qué usarlo? Mo's sirve cuando la respuesta de un rango es fácil de *mantener* al agregar o quitar un elemento, pero difícil de *combinar* a partir de dos mitades. Ejemplos son contar valores distintos, sumar cuadrados de frecuencias o contar pares con cierta propiedad. Un árbol de segmentos exige una operación de combinación asociativa, y para estos problemas no existe una obvia. Si la respuesta sí se combina (suma, mínimo, máximo, mcd), el árbol de segmentos o el Fenwick son más simples y más rápidos, con $O(\log n)$ por consulta y sin exigir que las consultas sean offline. La descomposición en raíz (*sqrt decomposition*) sirve en línea, pero cada bloque necesita una estructura propia que casi siempre es más difícil de escribir que **agregar/quitar**. Mo's, en cambio, es una plantilla fija en la que solo se cambian esas dos funciones. Su costo es que es offline y que $\sqrt{n}$ es más lento que $\log n$. Además, si los valores son grandes hay que comprimirlos antes (subsección de compresión de coordenadas), porque el arreglo de frecuencias se indexa por valor.

Casos de uso:

- Contar valores distintos en muchos rangos.
- Sumar frecuencia² · valor o cualquier expresión que dependa de las frecuencias dentro del rango.
- Frecuencia de un valor y frecuencia de la moda en un rango (variante más abajo).
- Contar pares de posiciones dentro del rango con una propiedad (por ejemplo, XOR igual a k en subarreglos).
- De nicho: consultas sobre caminos o subárboles con Mo's sobre árboles (modificación 3), o con actualizaciones puntuales intercaladas (Mo's con actualizaciones, más abajo).

Complejidad: Sean n el largo del arreglo y q el número de consultas. Ordenar las consultas cuesta $O(q \log q)$. Cada consulta desplaza los punteros con agregar/quitar de costo $O(1)$, y el desplazamiento total es $O(n^2/B + qB)$ como se vio arriba, con lo que el tiempo queda en $O(n\sqrt{q} + q \log q)$. Con $q \approx n$ es $O(n\sqrt{n})$. La memoria es $O(n + q)$ más el arreglo de frecuencias. Si agregar y quitar cuestan T en lugar de $O(1)$, el total se multiplica por T . La variante con actualizaciones cuesta $O(n^{5/3})$ con $q \approx U \approx n$ y bloques de tamaño $n^{2/3}$.

Mo's clásico (consultas offline, sin actualizaciones)

```

1 // Ejemplo guía: contar valores DISTINTOS en cada rango [l, r]
  // (0-indexado, inclusive).
2 struct MoClasico {
3     int n, bloque;
4     vector<int> a; // arreglo original (valores en [0,
      max_val])
5     vector<int> cuenta; // cuenta[v]: cuantas veces aparece v
      en la ventana actual
6     int izq = 0, der = -1;
7     int distintos = 0; // respuesta de la ventana actual
8
9     // CREATE: guarda el arreglo y reserva el arreglo de
      frecuencias.
10    MoClasico(const vector<int>& arr, int max_val) {
11        a = arr;
12        n = (int)a.size();
13        cuenta.assign(max_val + 1, 0);
14        bloque = max(1, (int)sqrt((double)n));
15    } // O(n + max_val)
16
17    // WRITE: agrega el elemento de la posición pos a la ventana.
18    void agregar(int pos) {
19        if (cuenta[a[pos]]++ == 0) distintos++;
20    } // O(1)
21
22    // WRITE: quita el elemento de la posición pos de la ventana.
23    void quitar(int pos) {
24        if (--cuenta[a[pos]] == 0) distintos--;
25    } // O(1)
26
27    // WRITE: responde todas las consultas y las devuelve en el
      orden original.
28    // Modifica la ventana (izq, der), por eso no es const.
29    vector<int> resolver(const vector<array<int, 2>>& consultas)
      {
30        int q = (int)consultas.size();
31        bloque = max(1, (int)(n / sqrt((double)max(1, q)))); // B
          = n / sqrt(q)
32        vector<int> orden(q);
33        iota(orden.begin(), orden.end(), 0);
34        sort(orden.begin(), orden.end(), [&](int i, int j) {
35            int bi = consultas[i][0] / bloque, bj = consultas[j][0] /
          bloque;
36            if (bi != bj) return bi < bj;
37            return (bi & 1) ? consultas[i][1] > consultas[j][1] //
          bloques impares: r decreciente
38                : consultas[i][1] < consultas[j][1]; // bloques pares: r
          creciente
39        });
40    }

```

```

41    vector<int> resp(q);
42    for (int id : orden) {
43        int l = consultas[id][0], r = consultas[id][1];
44        while (der < r) agregar(++der); // primero expandir...
45        while (izq > l) agregar(--izq);
46        while (der > r) quitar(der--); // ...y despues
      contraer
47        while (izq < l) quitar(izq++);
48        resp[id] = distintos;
49    }
50    return resp;
51 } // O(n sqrt(q) + q log q)
52 };
53
54 // -----
55 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
56 // -----
57 // 1. Frecuencia y moda en un rango: ver la variante "
      Frecuencia y Moda en un Rango"
58 // (misma plantilla, pero la ventana mantiene cuenta[], cc[]
      y max_freq en lugar de
59 // "distintos").
60 //
61 // 2. Ordenamiento por curva de Hilbert: en lugar de bloques +
      zigzag, ordenar las
62 // consultas por el índice de Hilbert del punto (l, r).
      Tiene el mismo costo asintótico
63 // y suele ser una constante mejor en la practica, pero es
      mas difícil de escribir.
64 //
65 // 3. Mo's sobre arboles (consultas sobre un camino o un
      subarbol): aplanar el arbol con un
66 // recorrido de Euler (cada nodo aparece dos veces: al
      entrar y al salir) y aplicar el
67 // mismo algoritmo sobre ese arreglo. En agregar/quitar se
      alterna el estado del nodo
68 // ("dentro/fuera de la ventana"): si aparece por segunda
      vez, se quita. Para caminos
69 // hace falta ademas tratar el ancestro comun (LCA) por
      separado.
70 //
71 // 4. Mo's con retroceso (rollback): cuando quitar es difícil o
      imposible (moda, maximo),
72 // se procesa cada bloque con un solo agregar: para cada
      consulta se agrega desde el
73 // final del bloque y se deshace lo agregado del lado
      izquierdo. Mismo O(n sqrt(q)) y
74 // solo necesita agregar y una copia del estado.
75 //
76 // 5. Valores grandes o negativos: comprimirlos primero (ver "
      Compresion de Coordenadas"),
77 // para que cuenta[] tenga tamaño O(n) en lugar de O(max_val)
      ).
78 //
79 // 6. agregar/quitar costosos: si cuestan T cada una, el total
      pasa a
80 // O(T * n sqrt(q)). Con T = O(log n) todavia es aceptable;
      con T grande no.
81 //
82 // 7. Bloque optimo: B = n / sqrt(q) (el código lo calcula en
      resolver). Con q mucho menor
83 // que n (q ~ 10) conviene un B grande; si q es muy grande,
      B se acerca a 1.

```

Ejemplo de funcionamiento:

```

1 int main() {
2     vector<int> a = {1, 2, 1, 3, 2, 4, 1}; // valores en [1,
      4]
3     MoClasico mo(a, 4);
4
5     vector<array<int, 2>> consultas = {
6         {0, 2}, // [1, 2, 1] -> 2 distintos
7         {1, 4}, // [2, 1, 3, 2] -> 3 distintos
8         {0, 6} // [1, 2, 1, 3, 2, 4, 1] -> 4 distintos
9     };
10
11     vector<int> resp = mo.resolver(consultas);
12     for (int x : resp) cout << x << " ";
13     cout << "\n"; // 2 3 4
14     return 0;
15 }

```

Variante: Frecuencia y Moda en un Rango

Idea: Es el mismo Mo's clásico con otra información mantenida en la ventana. Además de `cuenta[v]` (cuántas veces aparece v) se guarda `cc[f]`, la cantidad de valores que aparecen exactamente f veces, y

`max_freq`, la frecuencia de la moda. Al agregar un elemento, su frecuencia sube en 1 y el máximo es `max(max_freq, cuenta[v])`. Al quitarlo, la frecuencia de `v` baja en 1. Si `v` tenía la frecuencia máxima y era el único con ella (`cc[max_freq]` queda en 0), el nuevo máximo es exactamente `max_freq - 1`, porque `v` acaba de quedar con esa frecuencia. Por eso no hay que buscar el máximo otra vez y ambas operaciones son $O(1)$. La frecuencia de un valor x concreto es simplemente `cuenta[x]`.

Complejidad: El desplazamiento de punteros es el de Mo's, $O(n\sqrt{q})$, con agregar y quitar en $O(1)$. El total es $O(n\sqrt{q} + q \log q)$ y la memoria es $O(n + \text{max_val} + q)$.

```

1 // Cada consulta es (l, r, x): devuelve {frecuencia de x en [l,
2   r], frecuencia de la moda en [l, r]}.
3 struct MoModa {
4     int n, bloque;
5     vector<int> a; // arreglo original (valores en [0,
6       max_val])
7     vector<int> cuenta; // cuenta[v]: apariciones de v en la
8       ventana
9     vector<int> cc; // cc[f]: cuantos valores aparecen
10      exactamente f veces (cc[0] no se usa)
11     int izq = 0, der = -1;
12     int max_freq = 0; // frecuencia de la moda en la ventana
13
14 // CREATE: guarda el arreglo y reserva frecuencias.
15 MoModa(const vector<int>& arr, int max_val) {
16     a = arr;
17     n = (int)a.size();
18     cuenta.assign(max_val + 1, 0);
19     cc.assign(n + 2, 0);
20     bloque = max(1, (int)sqrt((double)n));
21 } // O(n + max_val)
22
23 // WRITE: agrega el elemento de la posicion pos.
24 void agregar(int pos) {
25     int v = a[pos];
26     cc[cuenta[v]]--;
27     cuenta[v]++;
28     cc[cuenta[v]]++;
29     max_freq = max(max_freq, cuenta[v]);
30 } // O(1)
31
32 // WRITE: quita el elemento de la posicion pos.
33 void quitar(int pos) {
34     int v = a[pos];
35     cc[cuenta[v]]--;
36     if (cuenta[v] == max_freq && cc[cuenta[v]] == 0) max_freq--;
37     // v era el unico maximo
38     cuenta[v]--;
39     cc[cuenta[v]]++;
40 } // O(1)
41
42 // WRITE: responde todas las consultas en el orden original.
43 vector<pair<int, int>> resolver(const vector<array<int, 3>>&
44   consultas) {
45     int q = (int)consultas.size();
46     bloque = max(1, (int)(n / sqrt((double)max(1, q))));
47     vector<int> orden(q);
48     iota(orden.begin(), orden.end(), 0);
49     sort(orden.begin(), orden.end(), [&](int i, int j) {
50         int bi = consultas[i][0] / bloque, bj = consultas[j][0] /
51         bloque;
52         if (bi != bj) return bi < bj;
53         return (bi & 1) ? consultas[i][1] > consultas[j][1] :
54         consultas[i][1] < consultas[j][1];
55     });
56
57     vector<pair<int, int>> resp(q);
58     for (int id : orden) {
59         int l = consultas[id][0], r = consultas[id][1], x =
60         consultas[id][2];
61         while (der < r) agregar(++der);
62         while (izq > l) agregar(--izq);
63         while (der > r) quitar(der--);
64         while (izq < l) quitar(izq++);
65         resp[id] = {cuenta[x], max_freq};
66     }
67     return resp;
68 } // O(n sqrt(q) + q log q)

```

Ejemplo de funcionamiento:

```

1 int main() {

```

```

2     vector<int> a = {1, 2, 1, 3, 2, 4, 1};
3     MoModa mo(a, 4);
4
5     vector<array<int, 3>> consultas = {
6         {0, 2, 1}, // [1,2,1]:      frec(1) = 2, moda con 2
7         {1, 4, 1}, // [2,1,3,2]:    frec(1) = 1, moda con 2
8         {0, 6, 1}, // arreglo completo: frec(1) = 3, moda con 3
9         {2, 5, 4}, // [1,3,2,4]:    frec(4) = 1, moda con 1
10        {0, 6, 2}, // arreglo completo: frec(2) = 2, moda con 3
11    };
12
13    for (auto [fx, fm] : mo.resolver(consultas)) cout << fx << ",
14      << fm << " ";
15    cout << "\n"; // 2,2 1,2 3,3 1,1 2,3
16    return 0;

```

Nota: Este struct devuelve la *frecuencia* de la moda, no cuál valor es. Para obtener el valor hay que mantener un `set<pair<int,int>>` de pares (frecuencia, valor) y tomar el último elemento, lo que sube el costo a $O(\log n)$ por movimiento. Si se necesita $O(1)$, la alternativa es Mo's con retroceso, donde solo se usa `agregar`. Además, `cc[0]` acumula valores sin sentido, pero nunca se lee.

Mo's con actualizaciones (3D Mo's)

```

1 // Igual que el clasico, con actualizaciones puntuales a[pos] =
2   valor intercaladas.
3 // Cada consulta es (l, r, t): t = cuantas actualizaciones ya
4   se aplicaron.
5 struct MoActualizaciones {
6     int n, bloque;
7     vector<int> a;
8     vector<int> cuenta;
9     vector<pair<int, int>> actualizaciones; // (pos, valor a
10      poner)
11     int izq = 0, der = -1, tiempo = 0;
12     int distintos = 0;
13
14 // CREATE: guarda el arreglo y fija el bloque en ~n^(2/3).
15 MoActualizaciones(const vector<int>& arr, int max_val) {
16     a = arr;
17     n = (int)a.size();
18     cuenta.assign(max_val + 1, 0);
19     bloque = max(1, (int)pow((double)n, 2.0 / 3.0));
20 } // O(n + max_val)
21
22 // WRITE: registra una actualizacion (se aplicara cuando le
23   toque, no de inmediato).
24 void agregar_actualizacion(int pos, int valor) {
25     actualizaciones.push_back({pos, valor});
26 } // O(1)
27
28 // WRITE: agrega el elemento de la posicion pos a la ventana.
29 void agregar(int pos) {
30     if (cuenta[a[pos]]++ == 0) distintos++;
31 } // O(1)
32
33 // WRITE: quita el elemento de la posicion pos de la ventana.
34 void quitar(int pos) {
35     if (--cuenta[a[pos]] == 0) distintos--;
36 } // O(1)
37
38 // WRITE: aplica la actualizacion t. Al aplicarla se
39   intercambia el valor a poner con el
40 // valor actual, por lo que llamar otra vez a la misma t la
41   REVIERTE.
42 void alternar(int t) {
43     int pos = actualizaciones[t].first;
44     int val = actualizaciones[t].second;
45     bool dentro = (pos >= izq && pos <= der);
46     if (dentro) quitar(pos);
47     swap(a[pos], val);
48     actualizaciones[t].second = val; // guarda el valor
49     viejo
50     if (dentro) agregar(pos);
51 } // O(1)
52
53 // WRITE: responde todas las consultas (l, r, t) y las
54   devuelve en el orden original.
55 vector<int> resolver(const vector<array<int, 3>>& consultas) {

```



```

48 int q = (int)consultas.size();
49 vector<int> orden(q);
50 iota(orden.begin(), orden.end(), 0);
51 sort(orden.begin(), orden.end(), [&](int i, int j) {
52     int bli = consultas[i][0] / bloque, blj = consultas[j][0]
53     / bloque;
54     if (bli != blj) return bli < blj;
55     int bri = consultas[i][1] / bloque, brj = consultas[j][1]
56     / bloque;
57     if (bri != brj) return bri < brj;
58     return consultas[i][2] < consultas[j][2];
59 });
60 vector<int> resp(q);
61 for (int id : orden) {
62     int l = consultas[id][0], r = consultas[id][1], t =
63     consultas[id][2];
64     while (der < r) agregar(++der);
65     while (izq > l) agregar(--izq);
66     while (der > r) quitar(der--);
67     while (izq < l) quitar(izq++);
68     while (tiempo < t) alternar(tiempo++); // aplicar
69     actualizaciones pendientes
70     while (tiempo > t) alternar(--tiempo); // o revertir
71     las que sobran
72     resp[id] = distintos;
73 }
74 return resp;
75 } // O(n^(5/3)) con q y #actualizaciones del orden de n
76 };

```

Ejemplo de funcionamiento:

```

1 int main() {
2     vector<int> a = {1, 2, 3};
3     MoActualizaciones mo(a, 3);
4     mo.agregar_actualizacion(0, 2); // actualizacion #0: a[0]
5     // = 2
6     mo.agregar_actualizacion(1, 1); // actualizacion #1: a[1]
7     // = 1
8     vector<array<int, 3>> consultas = {
9         {0, 2, 0}, // [1, 2, 3], sin actualizaciones -> 3
10        {0, 2, 1}, // tras la #0: [2, 2, 3] -> 2
11        {0, 2, 2}, // tras la #0 y la #1: [2, 1, 3] -> 3
12    };
13    vector<int> resp = mo.resolver(consultas);
14    for (int x : resp) cout << x << " ";
15    cout << "\n"; // 3 2 3
16    return 0;
17 }

```

Nota: `resolver` modifica la ventana, así que un mismo objeto puede reutilizarse en llamadas sucesivas (la ventana queda en el estado donde terminó y sigue siendo correcta), pero no es seguro para uso concurrente. Las consultas deben cumplir $0 \leq l \leq r < n$; con `quitar` sobre una ventana vacía o $l > r$ las frecuencias se vuelven negativas. En Mo's con actualizaciones, el orden de las actualizaciones debe ser el cronológico, y t es la *cantidad* de actualizaciones ya aplicadas (0 significa el arreglo original). Si el problema tiene actualizaciones pero se pueden resolver como consultas del tipo «valor en un instante», a veces es más simple un árbol de segmentos persistente. Y en `alternar`, el orden es importante: primero se quita el valor viejo, luego se cambia y después se agrega el nuevo, siempre que la posición esté dentro de la ventana.

Regla práctica: Usa Mo's solo si (1) todas las consultas se conocen de antemano y (2) la respuesta se mantiene fácil con `agregar/quitar` en $O(1)$ pero no se puede combinar con un árbol de segmentos. Para suma, mínimo, máximo o mcd, prefiere Fenwick o árbol de segmentos. Si las consultas llegan en línea, usa un árbol de segmentos o descomposición en raíz. Si hay actualizaciones intercaladas, usa la versión con tiempo (bloque $n^{2/3}$). Si `quitar` es imposible (moda, máximo), usa Mo's con retroceso (modificación 4). Y si los valores

son grandes, comprímelos antes.

5.9 Wavelet Tree

Idea: es un árbol binario sobre el **rango de valores** del arreglo (no sobre las posiciones, como Segment Tree). La raíz cubre $[lo, hi]$ de valores posibles; en cada nodo, los elementos con valor $\leq$ mediana van al hijo izquierdo y el resto al derecho, **preservando su orden relativo** (partición estable). Cada nodo guarda un arreglo `b[]` con cuántos elementos, hasta cada posición, fueron hacia la izquierda. Con eso puedes, en cada nivel, saber exactamente qué sub-rango de posiciones le corresponde a cada hijo — y bajar por el árbol respondiendo preguntas sobre **valores dentro de un rango de posiciones** en $O(\log C)$, donde C es la cantidad de valores distintos.

¿Por qué usarlo? Es la única estructura de esta sección que responde eficientemente preguntas que combinan **posición y valor** a la vez: "el k -ésimo menor en $[l, r]$ ", "cuántos elementos en $[l, r]$ son $\leq X$ ", "cuántas veces aparece exactamente X en $[l, r]$ ". Ninguna de las estructuras anteriores (Fenwick, Segment Tree, Sparse Table) resuelve esto directamente sin re-plantear el problema.

Casos de uso:

- k -ésimo elemento más chico (o más grande) dentro de un rango $[l, r]$.
- Conteo de elementos en $[l, r]$ cuyo valor está en un rango $[a, b]$.
- Frecuencia de un valor específico dentro de un rango de posiciones (rank query).
- Arreglo **estático** — no soporta updates sin reconstruir (para eso existe la variante persistente, más avanzada).

Complejidad: construir $O(n \log C)$, cada query $O(\log C)$, donde C es la cantidad de valores distintos tras comprimir coordenadas.

Wavelet Tree: k -ésimo, conteo $\leq X$ y frecuencia

```

1 struct WaveletTree {
2     int lo, hi;
3     WaveletTree *izq = nullptr, *der = nullptr;
4     vector<int> b; // b[i] = cuantos de a[0..i-1] fueron a la
5     // izquierda.
6     // CREATE: construye recursivamente sobre el segmento [from,
7     // to)
8     // de un arreglo mutable, particionando por la mediana de
9     // valor
10    // [lo, hi]. Los valores deben venir ya comprimidos a [0, C
11    // -1].
12    WaveletTree(int* from, int* to, int lo, int hi) : lo(lo), hi(hi) {
13        if (lo == hi || from == to) return;
14        int mid = (lo + hi) / 2;
15        auto valIzq = [mid](int x) { return x <= mid; };
16        b.reserve(to - from + 1);
17        b.push_back(0);
18        for (auto it = from; it != to; it++)
19            b.push_back(b.back() + valIzq(*it));
20        auto pivote = stable_partition(from, to, valIzq);
21        izq = new WaveletTree(from, pivote, lo, mid);
22        der = new WaveletTree(pivote, to, mid + 1, hi);
23    } // O(n log C)
24 }

```



```

25 // READ: k-esimo valor mas chico en [l, r] (1-indexado, k
    desde 1).
26 int kth(int l, int r, int k) {
27     if (l > r) return -1;
28     if (lo == hi) return lo;
29
30     int enIzq = b[r] - b[l - 1];
31     int lb = b[l - 1], rb = b[r];
32
33     if (k <= enIzq)
34         return izq->kth(lb + 1, rb, k);
35     return der->kth(l - lb, r - rb, k - enIzq);
36 } // O(log C)
37
38 // READ: cuantos elementos de [l, r] tienen valor <= val.
39 int contarMenorIgual(int l, int r, int val) {
40     if (l > r || val < lo) return 0;
41     if (hi <= val) return r - l + 1;
42
43     int lb = b[l - 1], rb = b[r];
44     return izq->contarMenorIgual(lb + 1, rb, val)
45         + der->contarMenorIgual(l - lb, r - rb, val);
46 } // O(log C)
47
48 // READ: cuantas veces aparece exactamente val en [l, r].
49 int contarIgual(int l, int r, int val) {
50     if (l > r || val < lo || val > hi) return 0;
51     if (lo == hi) return r - l + 1;
52
53     int mid = (lo + hi) / 2;
54     int lb = b[l - 1], rb = b[r];
55
56     if (val <= mid)
57         return izq->contarIgual(lb + 1, rb, val);
58     return der->contarIgual(l - lb, r - rb, val);
59 } // O(log C)
60
61 -WaveletTree() {
62     delete izq;
63     delete der;
64 }
65 };

```

Ejemplo de funcionamiento (con compresión de coordenadas):

```

1 vector<int> original = {40, 10, 30, 10, 20, 40, 30};
2
3 // comprimir a [0, C-1] para que el wavelet tree trabaje con
4 // valores pequenos y contiguos.
5 vector<int> ordenado = original;
6 sort(ordenado.begin(), ordenado.end());
7 ordenado.erase(unique(ordenado.begin(), ordenado.end()), ordenado
    .end());
8
9 vector<int> comprimido(ordenado.size());
10 for (int i = 0; i < (int)ordenado.size(); i++)
11     comprimido[i] = lower_bound(ordenado.begin(), ordenado.end(),
12         original[i]) - ordenado.begin();
13
14 // comprimido: [3, 0, 2, 0, 1, 3, 2] (valores 10,20,30,40 ->
15 // 0,1,2,3)
16
17 WaveletTree wt(comprimido.data(),
18     comprimido.data() + comprimido.size(),
19     0, (int)ordenado.size() - 1);
20
21 // 3er elemento mas chico en el rango [2, 6] (1-indexado)
22 int idx = wt.kth(2, 6, 3);
23 cout << ordenado[idx];
24 // posiciones 2..6 (1-index): 10,30,10,20,40,30 (valores 0-index 2..6)
25
26 // -> valores reales: 30,10,20,40,30 ordenados: 10,20,30,30,40
27 // 3er mas chico = 30
28
29 cout << wt.contarMenorIgual(1, 7, 1);
30 // cuantos valores comprimidos <= 1 (o sea, <= 20) hay en todo el
31 // arreglo
32 // original: 40,10,30,10,20,40,30 -> <=20 son: 10,10,20 -> 3
33
34 cout << wt.contarIgual(1, 7, 0);
35 // cuantas veces aparece el valor comprimido 0 (o sea, 10) -> 2

```

Comparación con las otras estructuras de rango:

Pregunta	Estructura
Suma/min/max de rango	Fenwick / Segment Tree / Sparse Table
k-ésimo menor, o conteo por rango de valores, en un rango de posiciones	Wavelet Tree
Elementos distintos, moda, funciones no lineales de frecuencia	Mo's Algorithm

Table 6: Cuándo el problema necesita Wavelet Tree en vez de las otras estructuras.

Regla práctica: si el enunciado combina una condición sobre **valores** (“menor que”, “entre a y b ”, “el k -ésimo más chico”) con un rango de **posiciones**, y el arreglo es estático, es casi siempre Wavelet Tree. Si solo te piden agregar (suma/min/max) sin condición sobre el valor, usa las estructuras más simples de las secciones anteriores.

5.10 Elemento Mayoritario

Votación de Boyer–Moore (Elemento Mayoritario)

Idea: Un elemento *mayoritario* de un arreglo de n elementos es uno que aparece *estrictamente más* de $n/2$ veces; si existe, es único. El algoritmo mantiene un candidato y un contador. Al leer x : si el contador es 0, x pasa a ser el candidato con contador 1; si x es el candidato, el contador sube; si no, baja. Es una votación en la que cada voto distinto «cancela» uno del candidato.

Funciona por un argumento de emparejamiento. Después de procesar i elementos, el multiconjunto leído se puede dividir en **cuenta** copias del candidato más ($i - \text{cuenta}$)/2 *pares de elementos distintos* (cada resta del contador formó un par distinto). Un par de elementos distintos contiene a lo sumo una copia del mayoritario m . Al final, si m no fuera el candidato (o el contador fuera 0), m aparecería a lo sumo $(n - \text{cuenta})/2 \leq n/2$ veces, lo que contradice que sea mayoritario. Por tanto, *si hay un mayoritario, es el candidato final*. El recíproco no vale: si no hay mayoritario, el candidato es arbitrario, y por eso se necesita una segunda pasada que cuente sus apariciones y compruebe que superan $n/2$.

¿Por qué usarlo? Las alternativas son: ordenar y mirar el elemento central ($O(n \log n)$), contar con un `map` o `unordered_map` ($O(n)$ de tiempo pero $O(n)$ de memoria), elegir un elemento al azar y contarlo (éxito con probabilidad $> 1/2$ por intento), o `nth_element` para hallar la mediana, que es el mayoritario si existe ($O(n)$ esperado, pero modifica el arreglo). Boyer–Moore es $O(n)$ determinista con $O(1)$ de memoria, y recorre los datos en orden, así que funciona sobre un *flujo* que no se puede guardar. Además es *combinable*: dos estados (candidato, contador) se fusionan en $O(1)$, lo que permite usarlo en árboles de segmentos y en cálculo distribuido (modificación 3). A diferencia de las técnicas de rango de las subsecciones anteriores (Mo's, Fenwick, Sparse Table), aquí la consulta es sobre *todo* el arreglo; para rangos hay que combinarlo con un árbol de segmentos. Exige que el umbral sea estrictamente mayor que $n/2$ (con «al menos $n/2$ » el argumento falla, como en $\{1, 2\}$).

Casos de uso:

- Hallar el elemento mayoritario de un arreglo (prob-

lema clásico de entrevistas y concursos), o decidir que no existe.

- Determinar el ganador de una votación por mayoría absoluta leyendo los votos en una sola pasada.
- Procesar flujos de datos con memoria constante, donde no se puede guardar la entrada.
- Encontrar elementos frecuentes (aparecen más de n/k veces) con la generalización de Misra–Gries (modificación 1).
- De nicho: mayoritario de un rango con un árbol de segmentos (modificación 3), o fusionar resultados parciales de varias máquinas.

Complejidad: La primera pasada hace una comparación y una actualización del contador por elemento: $O(n)$. La verificación cuenta las apariciones del candidato en otra pasada: $O(n)$. El tiempo total es $O(n)$ y la memoria extra es $O(1)$ (un candidato y un contador). Fusionar dos estados es $O(1)$. La generalización a n/k guarda hasta $k - 1$ contadores, con memoria $O(k)$, y un tiempo $O(n)$ amortizado, porque cada «decrementar todos» elimina k unidades de contador distintas y eso ocurre a lo sumo n/k veces.

```

1 struct VotacionMayoritaria {
2     long long cand; // candidato actual
3     int cuenta; // votos netos del candidato (0 = sin
4         candidato)
5
6     // CREATE: estado vacio, sin candidato.
7     VotacionMayoritaria() : cand(0), cuenta(0) {} // O(1)
8
9     // WRITE: procesa un elemento x del flujo.
10    void agregar(long long x) {
11        if (cuenta == 0) { cand = x; cuenta = 1; }
12        else if (cand == x) cuenta++;
13        else cuenta--;
14    } // O(1)
15
16    // WRITE: fusiona el estado de otro bloque (para rangos o
17    // datos distribuidos).
18    // Un par de candidatos distintos se cancela; el que tiene
19    // mas votos sobrevive.
20    void combinar(const VotacionMayoritaria& o) {
21        if (cand == o.cand) cuenta += o.cuenta;
22        else if (cuenta >= o.cuenta) cuenta -= o.cuenta;
23        else { cand = o.cand; cuenta = o.cuenta - cuenta; }
24    } // O(1)
25
26    // READ: candidato actual (solo es mayoritario si se verifica
27    //).
28    long long candidato() const {
29        return cand;
30    } // O(1)
31
32    // READ: indica si x aparece estrictamente mas de n/2 veces
33    // en a (segunda pasada).
34    static bool verificar(const vector<long long>& a, long long x)
35    {
36        long long c = 0;
37        for (long long y : a) if (y == x) c++;
38        return 2 * c > (long long)a.size();
39    } // O(n)
40
41    // READ: devuelve {existe, valor} del elemento mayoritario de
42    // a.
43    static pair<bool, long long> mayoritario(const vector<long
44    long>& a) {
45        VotacionMayoritaria v;
46        for (long long x : a) v.agregar(x);
47        if (a.empty() || !verificar(a, v.cand)) return {false, 0};
48        return {true, v.cand};
49    } // O(n)
50
51    // -----
52    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
53    // -----
54    // 1. Elementos que aparecen MAS de n/k veces (Misra–Gries):
55    // hay a lo sumo k - 1 de ellos.

```

```

48 // Se guardan hasta k - 1 contadores; si llega un valor
49 // nuevo y no hay sitio, se
50 // resta 1 a todos y se descartan los que llegan a 0. Con k
51 // = 2 es Boyer-Moore.
52 // map<long long, int> c;
53 // for (long long x : a) {
54 //     if (c.count(x)) c[x]++;
55 //     else if ((int)c.size() < k - 1) c[x] = 1;
56 //     else {
57 //         for (auto it = c.begin(); it != c.end(); )
58 //             if (--it->second == 0) it = c.erase(it);
59 //     }
60 // }
61 // Los candidatos son las claves de c; verificar cada uno
62 // con una segunda pasada.
63 // O(n) amortizado (O(n log k) con map), memoria O(k).
64 // 2. Alternativas simples cuando no importa la memoria:
65 // a) Ordenar: tras sort, el mayoritario si existe es a[n /
66 // 2]. O(n log n).
67 // b) nth_element(a.begin(), a.begin() + n / 2, a.end());
68 // candidato = a[n / 2]. O(n) esperado.
69 // c) Por bits (enteros): para cada bit b, el bit del
70 // mayoritario es el que aparece en mas de
71 // n/2 elementos; armar el numero y verificar. O(n * bits
72 //), memoria O(1), sin comparar valores.
73 // d) Aleatorio: elegir x = a[rand() % n] y contar.
74 // Probabilidad de acierto > 1/2; con 30
75 // intentos falla con probabilidad < 2^-30. O(30 n).
76 // 3. Mayoritario de un RANGO [l, r] (consultas en linea):
77 // arbol de segmentos donde cada nodo
78 // guarda un VotacionMayoritaria y se une con combinar(). La
79 // consulta fusiona los O(log n)
80 // nodos y da un candidato; se verifica contando sus
81 // apariciones en [l, r] con listas de
82 // posiciones ordenadas por valor:
83 // pos[v] = posiciones de v en orden creciente
84 // // veces = upper_bound(pos[v].begin(), pos[v].end(), r)
85 // // - lower_bound(pos[v].begin(), pos[v].end(), l);
86 // // existe mayoritario <= 2 * veces > r - l + 1
87 // // Construcción O(n), consulta O(log n). Valores grandes:
88 // comprimirlos primero.
89 // 4. Con ">= n/2" (empate permitido) el argumento de los pares
90 // falla: en {1, 2} cada uno
91 // aparece n/2 veces y el candidato es 2. Si el enunciado lo
92 // pide, contar con una tabla.
93 // 5. Con pesos (cada elemento tiene un peso w): agregar suma o
94 // resta w al contador (y si
95 // el peso supera el contador, el candidato cambia con
96 // contador w - cuenta). Mismo O(n).
97 // 6. Verificar siempre. Si el enunciado GARANTIZA que existe
98 // un mayoritario, la segunda
99 // pasada se puede omitir y el algoritmo queda en una sola
100 // lectura del flujo.

```

Ejemplo de funcionamiento:

```

1 int main() {
2     vector<long long> a = {2, 2, 1, 1, 1, 2, 2};
3     auto r = VotacionMayoritaria::mayoritario(a);
4     cout << r.first << " " << r.second << "\n"; // 1 2 (el 2
5         aparece 4 de 7 veces)
6
7     vector<long long> b = {3, 3, 4, 2, 4, 4, 2, 4, 4};
8     r = VotacionMayoritaria::mayoritario(b);
9     cout << r.first << " " << r.second << "\n"; // 1 4 (el 4
10        aparece 5 de 9 veces)
11
12    vector<long long> c = {1, 1, 2, 2, 3};
13    r = VotacionMayoritaria::mayoritario(c);
14    cout << r.first << "\n"; // 0 (el
15        candidato final es 3, que aparece 1 vez)
16
17    vector<long long> d = {1, 2};
18    cout << VotacionMayoritaria::mayoritario(d).first << "\n";
19    // 0 (empate: nadie supera n/2)
20
21    // Fusión de dos bloques: {2, 2, 1} y {1, 1, 2} -> estados (2,
22    // 1) y (1, 1)
23    VotacionMayoritaria izq, der;
24    for (long long x : {2, 2, 1}) izq.agregar(x);
25    for (long long x : {1, 1, 2}) der.agregar(x);
26    izq.combinar(der);
27    cout << izq.cuenta << "\n"; // 0 (se
28        cancelan por completo: 3 contra 3, sin mayoritario)
29    return 0;
30 }

```

Nota: El candidato que entrega la primera pasada *no* es una respuesta, es una sospecha: sin la verificación, $\{1, 2, 3\}$ devolvería 3. Si el contador final es 0 se puede concluir de inmediato que no hay mayoritario, sin verificar. En **combinar**, cuando los candidatos son distintos y los contadores iguales, el resultado queda con contador 0 y el candidato es irrelevante. La fusión es correcta por el mismo argumento de pares: lo que sobra de cada bloque son copias de su candidato más pares distintos, y al cruzar candidatos distintos se forman más pares. Por último, el contador de `int` basta mientras $n < 2^{31}$.

Regla práctica: Si el problema pide el elemento que aparece más de $n/2$ veces en todo el arreglo, o en un flujo con memoria limitada, usa Boyer–Moore con una segunda pasada de verificación (o sin ella, si el enunciado garantiza que existe). Si el umbral es n/k , usa Misra–Gries con $k - 1$ contadores. Si hay consultas de rango, combina los estados en un árbol de segmentos y verifica con listas de posiciones. Si la memoria no importa y la claridad sí, ordenar y mirar $a[n/2]$ o usar un `map` es suficiente. Y si el umbral es «al menos $n/2$ », no uses este algoritmo.

5.11 Heavy-Light Decomposition

Idea: descompone un árbol en **cadenas** de forma que cualquier camino desde la raíz hasta cualquier nodo cruza a lo más $O(\log n)$ cadenas distintas. En cada nodo, el hijo con el **subárbol más grande** (el hijo “pesado”) continúa la misma cadena que su padre; los demás hijos inician cadenas nuevas. Al numerar los nodos en el orden en que se visitan (DFS que siempre sigue primero al hijo pesado), cada cadena queda como un **rango contiguo** de posiciones — así que una query sobre un **camino** del árbol se convierte en $O(\log n)$ queries de rango sobre un Segment Tree normal, una por cada cadena que el camino atraviesa.

¿Por qué usarlo? Es la forma estándar de llevar range queries de un arreglo a un **árbol**: sumas/mínimos/updates en el camino entre dos nodos, o en el subárbol de un nodo. Sin HLD, un camino arbitrario no tiene ninguna estructura contigua sobre la cual aplicar Fenwick o Segment Tree directamente.

Casos de uso:

- Suma/mínimo/máximo en el camino entre dos nodos u y v .
- Actualizar todos los nodos (o aristas) de un camino $u-v$.
- Consultas o updates sobre el **subárbol** completo de un nodo (más simple: el subárbol siempre es un rango contiguo, sin necesidad de descomponer en cadenas).
- Cualquier problema de árbol que “se sentiría” como Range Queries si el árbol fuera un arreglo lineal.

Complejidad: construir $O(n)$ (dos DFS) más $O(n)$ del Segment Tree subyacente; query o update de **camino** $O(\log^2 n)$ ($O(\log n)$ cadenas $\times$ $O(\log n)$ por consulta al Segment Tree); query o update de **subárbol** $O(\log n)$ (un solo rango contiguo).

HLD: suma en camino, suma en subárbol

Idea de la implementación: dos DFS iterativas (con pila explícita, para evitar stack overflow en árboles muy profundos o muy desbalanceados — un riesgo real con recursión cuando $n \geq 10^5$ y el árbol es una cadena larga). El primero (`dfsTamano`) calcula el tamaño de cada subárbol y decide el hijo pesado de cada nodo. El segundo (`dfsDescomponer`) recorre siempre primero al hijo pesado (así la cadena sigue consecutiva en las posiciones) y solo después explora los hijos livianos, cada uno iniciando su propia cadena nueva. Las posiciones resultantes (`pos[]`) alimentan un Segment Tree normal —aquí se reutiliza `SegmentTreeLazyAdd` de la sección anterior— sobre el cual se hacen las queries reales.

```

1 struct HLD {
2     vector<vector<int>>> adj;
3     vector<int> padre, profundidad, tamano, pesado, cabeza, pos;
4     int n, timer = 0;
5     SegmentTreeLazyAdd* seg; // Segment Tree sobre el orden pos
6
7     // CREATE: dos DFS iterativas para calcular cadenas, luego
8     // arma
9     // el Segment Tree sobre el arreglo reordenado por pos[].
10    HLD(vector<vector<int>>& adjacency, vector<long long>&
11        valores,
12        int raiz) {
13        adj = adjacency;
14        n = adj.size();
15        padre.assign(n, -1);
16        profundidad.assign(n, 0);
17        tamano.assign(n, 1);
18        pesado.assign(n, -1);
19        cabeza.assign(n, raiz);
20        pos.assign(n, 0);
21
22        dfsTamano(raiz);
23        dfsDescomponer(raiz);
24
25        vector<long long> reordenado(n);
26        for (int v = 0; v < n; v++)
27            reordenado[pos[v]] = valores[v];
28
29        seg = new SegmentTreeLazyAdd(reordenado);
30    } // O(n)
31
32    // Fase 1: obtiene un orden donde cada padre aparece ANTES
33    // que
34    // sus hijos (equivalente a un preorden), usando una pila.
35    // Fase 2: recorre ese orden al revés (hijos antes que padres)
36    // para acumular tamano[] y decidir el hijo pesado, igual que
37    // haría la versión recursiva al "volver" de cada llamada.
38    void dfsTamano(int raiz) {
39        vector<int> orden;
40        stack<int> st;
41        st.push(raiz);
42        padre[raiz] = -1;
43        profundidad[raiz] = 0;
44
45        while (!st.empty()) {
46            int u = st.top();
47            st.pop();
48            orden.push_back(u);
49
50            for (int v : adj[u]) {
51                if (v != padre[u]) {
52                    padre[v] = u;
53                    profundidad[v] = profundidad[u] + 1;
54                    st.push(v);
55                }
56            }
57        }
58
59        for (int i = (int)orden.size() - 1; i >= 0; i--) {
60            int u = orden[i];
61            tamano[u] = 1;
62            int maxSub = 0;
63
64            for (int v : adj[u]) {
65                if (v == padre[u]) continue;
66
67                tamano[u] += tamano[v];
68                if (tamano[v] > maxSub) {
69                    maxSub = tamano[v];
70                }
71            }
72            pesado[u] = maxSub;
73        }
74    }
75
76    // Fase 3: descomponer el camino entre u y v en rangos contiguos
77    // sobre el Segment Tree.
78    // Recorre el camino desde u hasta v, acumulando los rangos.
79    // Si u es hijo de v, el rango es [pos[u], pos[v]].
80    // Si v es hijo de u, el rango es [pos[v], pos[u]].
81    // Si u y v están en diferentes cadenas, se recorren las cadenas
82    // una por una hasta encontrar el ancestro común.
83    // El ancestro común es el nodo más profundo que es ancestro de
84    // ambos.
85    // El camino se divide en tres partes:
86    // 1. Desde u hasta el ancestro común.
87    // 2. Desde el ancestro común hasta v.
88    // 3. Desde el ancestro común hasta el ancestro común.
89    // El ancestro común es el nodo más profundo que es ancestro de
90    // ambos.
91    // El camino se divide en tres partes:
92    // 1. Desde u hasta el ancestro común.
93    // 2. Desde el ancestro común hasta v.
94    // 3. Desde el ancestro común hasta el ancestro común.
95    // El ancestro común es el nodo más profundo que es ancestro de
96    // ambos.
97    // El camino se divide en tres partes:
98    // 1. Desde u hasta el ancestro común.
99    // 2. Desde el ancestro común hasta v.
100   // 3. Desde el ancestro común hasta el ancestro común.
101   // El ancestro común es el nodo más profundo que es ancestro de
102   // ambos.
103   // El camino se divide en tres partes:
104   // 1. Desde u hasta el ancestro común.
105   // 2. Desde el ancestro común hasta v.
106   // 3. Desde el ancestro común hasta el ancestro común.
107   // El ancestro común es el nodo más profundo que es ancestro de
108   // ambos.
109   // El camino se divide en tres partes:
110   // 1. Desde u hasta el ancestro común.
111   // 2. Desde el ancestro común hasta v.
112   // 3. Desde el ancestro común hasta el ancestro común.
113   // El ancestro común es el nodo más profundo que es ancestro de
114   // ambos.
115   // El camino se divide en tres partes:
116   // 1. Desde u hasta el ancestro común.
117   // 2. Desde el ancestro común hasta v.
118   // 3. Desde el ancestro común hasta el ancestro común.
119   // El ancestro común es el nodo más profundo que es ancestro de
120   // ambos.
121   // El camino se divide en tres partes:
122   // 1. Desde u hasta el ancestro común.
123   // 2. Desde el ancestro común hasta v.
124   // 3. Desde el ancestro común hasta el ancestro común.
125   // El ancestro común es el nodo más profundo que es ancestro de
126   // ambos.
127   // El camino se divide en tres partes:
128   // 1. Desde u hasta el ancestro común.
129   // 2. Desde el ancestro común hasta v.
130   // 3. Desde el ancestro común hasta el ancestro común.
131   // El ancestro común es el nodo más profundo que es ancestro de
132   // ambos.
133   // El camino se divide en tres partes:
134   // 1. Desde u hasta el ancestro común.
135   // 2. Desde el ancestro común hasta v.
136   // 3. Desde el ancestro común hasta el ancestro común.
137   // El ancestro común es el nodo más profundo que es ancestro de
138   // ambos.
139   // El camino se divide en tres partes:
140   // 1. Desde u hasta el ancestro común.
141   // 2. Desde el ancestro común hasta v.
142   // 3. Desde el ancestro común hasta el ancestro común.
143   // El ancestro común es el nodo más profundo que es ancestro de
144   // ambos.
145   // El camino se divide en tres partes:
146   // 1. Desde u hasta el ancestro común.
147   // 2. Desde el ancestro común hasta v.
148   // 3. Desde el ancestro común hasta el ancestro común.
149   // El ancestro común es el nodo más profundo que es ancestro de
150   // ambos.
151   // El camino se divide en tres partes:
152   // 1. Desde u hasta el ancestro común.
153   // 2. Desde el ancestro común hasta v.
154   // 3. Desde el ancestro común hasta el ancestro común.
155   // El ancestro común es el nodo más profundo que es ancestro de
156   // ambos.
157   // El camino se divide en tres partes:
158   // 1. Desde u hasta el ancestro común.
159   // 2. Desde el ancestro común hasta v.
160   // 3. Desde el ancestro común hasta el ancestro común.
161   // El ancestro común es el nodo más profundo que es ancestro de
162   // ambos.
163   // El camino se divide en tres partes:
164   // 1. Desde u hasta el ancestro común.
165   // 2. Desde el ancestro común hasta v.
166   // 3. Desde el ancestro común hasta el ancestro común.
167   // El ancestro común es el nodo más profundo que es ancestro de
168   // ambos.
169   // El camino se divide en tres partes:
170   // 1. Desde u hasta el ancestro común.
171   // 2. Desde el ancestro común hasta v.
172   // 3. Desde el ancestro común hasta el ancestro común.
173   // El ancestro común es el nodo más profundo que es ancestro de
174   // ambos.
175   // El camino se divide en tres partes:
176   // 1. Desde u hasta el ancestro común.
177   // 2. Desde el ancestro común hasta v.
178   // 3. Desde el ancestro común hasta el ancestro común.
179   // El ancestro común es el nodo más profundo que es ancestro de
180   // ambos.
181   // El camino se divide en tres partes:
182   // 1. Desde u hasta el ancestro común.
183   // 2. Desde el ancestro común hasta v.
184   // 3. Desde el ancestro común hasta el ancestro común.
185   // El ancestro común es el nodo más profundo que es ancestro de
186   // ambos.
187   // El camino se divide en tres partes:
188   // 1. Desde u hasta el ancestro común.
189   // 2. Desde el ancestro común hasta v.
190   // 3. Desde el ancestro común hasta el ancestro común.
191   // El ancestro común es el nodo más profundo que es ancestro de
192   // ambos.
193   // El camino se divide en tres partes:
194   // 1. Desde u hasta el ancestro común.
195   // 2. Desde el ancestro común hasta v.
196   // 3. Desde el ancestro común hasta el ancestro común.
197   // El ancestro común es el nodo más profundo que es ancestro de
198   // ambos.
199   // El camino se divide en tres partes:
200   // 1. Desde u hasta el ancestro común.
201   // 2. Desde el ancestro común hasta v.
202   // 3. Desde el ancestro común hasta el ancestro común.
203   // El ancestro común es el nodo más profundo que es ancestro de
204   // ambos.
205   // El camino se divide en tres partes:
206   // 1. Desde u hasta el ancestro común.
207   // 2. Desde el ancestro común hasta v.
208   // 3. Desde el ancestro común hasta el ancestro común.
209   // El ancestro común es el nodo más profundo que es ancestro de
210   // ambos.
211   // El camino se divide en tres partes:
212   // 1. Desde u hasta el ancestro común.
213   // 2. Desde el ancestro común hasta v.
214   // 3. Desde el ancestro común hasta el ancestro común.
215   // El ancestro común es el nodo más profundo que es ancestro de
216   // ambos.
217   // El camino se divide en tres partes:
218   // 1. Desde u hasta el ancestro común.
219   // 2. Desde el ancestro común hasta v.
220   // 3. Desde el ancestro común hasta el ancestro común.
221   // El ancestro común es el nodo más profundo que es ancestro de
222   // ambos.
223   // El camino se divide en tres partes:
224   // 1. Desde u hasta el ancestro común.
225   // 2. Desde el ancestro común hasta v.
226   // 3. Desde el ancestro común hasta el ancestro común.
227   // El ancestro común es el nodo más profundo que es ancestro de
228   // ambos.
229   // El camino se divide en tres partes:
230   // 1. Desde u hasta el ancestro común.
231   // 2. Desde el ancestro común hasta v.
232   // 3. Desde el ancestro común hasta el ancestro común.
233   // El ancestro común es el nodo más profundo que es ancestro de
234   // ambos.
235   // El camino se divide en tres partes:
236   // 1. Desde u hasta el ancestro común.
237   // 2. Desde el ancestro común hasta v.
238   // 3. Desde el ancestro común hasta el ancestro común.
239   // El ancestro común es el nodo más profundo que es ancestro de
240   // ambos.
241   // El camino se divide en tres partes:
242   // 1. Desde u hasta el ancestro común.
243   // 2. Desde el ancestro común hasta v.
244   // 3. Desde el ancestro común hasta el ancestro común.
245   // El ancestro común es el nodo más profundo que es ancestro de
246   // ambos.
247   // El camino se divide en tres partes:
248   // 1. Desde u hasta el ancestro común.
249   // 2. Desde el ancestro común hasta v.
250   // 3. Desde el ancestro común hasta el ancestro común.
251   // El ancestro común es el nodo más profundo que es ancestro de
252   // ambos.
253   // El camino se divide en tres partes:
254   // 1. Desde u hasta el ancestro común.
255   // 2. Desde el ancestro común hasta v.
256   // 3. Desde el ancestro común hasta el ancestro común.
257   // El ancestro común es el nodo más profundo que es ancestro de
258   // ambos.
259   // El camino se divide en tres partes:
260   // 1. Desde u hasta el ancestro común.
261   // 2. Desde el ancestro común hasta v.
262   // 3. Desde el ancestro común hasta el ancestro común.
263   // El ancestro común es el nodo más profundo que es ancestro de
264   // ambos.
265   // El camino se divide en tres partes:
266   // 1. Desde u hasta el ancestro común.
267   // 2. Desde el ancestro común hasta v.
268   // 3. Desde el ancestro común hasta el ancestro común.
269   // El ancestro común es el nodo más profundo que es ancestro de
270   // ambos.
271   // El camino se divide en tres partes:
272   // 1. Desde u hasta el ancestro común.
273   // 2. Desde el ancestro común hasta v.
274   // 3. Desde el ancestro común hasta el ancestro común.
275   // El ancestro común es el nodo más profundo que es ancestro de
276   // ambos.
277   // El camino se divide en tres partes:
278   // 1. Desde u hasta el ancestro común.
279   // 2. Desde el ancestro común hasta v.
280   // 3. Desde el ancestro común hasta el ancestro común.
281   // El ancestro común es el nodo más profundo que es ancestro de
282   // ambos.
283   // El camino se divide en tres partes:
284   // 1. Desde u hasta el ancestro común.
285   // 2. Desde el ancestro común hasta v.
286   // 3. Desde el ancestro común hasta el ancestro común.
287   // El ancestro común es el nodo más profundo que es ancestro de
288   // ambos.
289   // El camino se divide en tres partes:
290   // 1. Desde u hasta el ancestro común.
291   // 2. Desde el ancestro común hasta v.
292   // 3. Desde el ancestro común hasta el ancestro común.
293   // El ancestro común es el nodo más profundo que es ancestro de
294   // ambos.
295   // El camino se divide en tres partes:
296   // 1. Desde u hasta el ancestro común.
297   // 2. Desde el ancestro común hasta v.
298   // 3. Desde el ancestro común hasta el ancestro común.
299   // El ancestro común es el nodo más profundo que es ancestro de
300   // ambos.
301   // El camino se divide en tres partes:
302   // 1. Desde u hasta el ancestro común.
303   // 2. Desde el ancestro común hasta v.
304   // 3. Desde el ancestro común hasta el ancestro común.
305   // El ancestro común es el nodo más profundo que es ancestro de
306   // ambos.
307   // El camino se divide en tres partes:
308   // 1. Desde u hasta el ancestro común.
309   // 2. Desde el ancestro común hasta v.
310   // 3. Desde el ancestro común hasta el ancestro común.
311   // El ancestro común es el nodo más profundo que es ancestro de
312   // ambos.
313   // El camino se divide en tres partes:
314   // 1. Desde u hasta el ancestro común.
315   // 2. Desde el ancestro común hasta v.
316   // 3. Desde el ancestro común hasta el ancestro común.
317   // El ancestro común es el nodo más profundo que es ancestro de
318   // ambos.
319   // El camino se divide en tres partes:
320   // 1. Desde u hasta el ancestro común.
321   // 2. Desde el ancestro común hasta v.
322   // 3. Desde el ancestro común hasta el ancestro común.
323   // El ancestro común es el nodo más profundo que es ancestro de
324   // ambos.
325   // El camino se divide en tres partes:
326   // 1. Desde u hasta el ancestro común.
327   // 2. Desde el ancestro común hasta v.
328   // 3. Desde el ancestro común hasta el ancestro común.
329   // El ancestro común es el nodo más profundo que es ancestro de
330   // ambos.
331   // El camino se divide en tres partes:
332   // 1. Desde u hasta el ancestro común.
333   // 2. Desde el ancestro común hasta v.
334   // 3. Desde el ancestro común hasta el ancestro común.
335   // El ancestro común es el nodo más profundo que es ancestro de
336   // ambos.
337   // El camino se divide en tres partes:
338   // 1. Desde u hasta el ancestro común.
339   // 2. Desde el ancestro común hasta v.
340   // 3. Desde el ancestro común hasta el ancestro común.
341   // El ancestro común es el nodo más profundo que es ancestro de
342   // ambos.
343   // El camino se divide en tres partes:
344   // 1. Desde u hasta el ancestro común.
345   // 2. Desde el ancestro común hasta v.
346   // 3. Desde el ancestro común hasta el ancestro común.
347   // El ancestro común es el nodo más profundo que es ancestro de
348   // ambos.
349   // El camino se divide en tres partes:
350   // 1. Desde u hasta el ancestro común.
351   // 2. Desde el ancestro común hasta v.
352   // 3. Desde el ancestro común hasta el ancestro común.
353   // El ancestro común es el nodo más profundo que es ancestro de
354   // ambos.
355   // El camino se divide en tres partes:
356   // 1. Desde u hasta el ancestro común.
357   // 2. Desde el ancestro común hasta v.
358   // 3. Desde el ancestro común hasta el ancestro común.
359   // El ancestro común es el nodo más profundo que es ancestro de
360   // ambos.
361   // El camino se divide en tres partes:
362   // 1. Desde u hasta el ancestro común.
363   // 2. Desde el ancestro común hasta v.
364   // 3. Desde el ancestro común hasta el ancestro común.
365   // El ancestro común es el nodo más profundo que es ancestro de
366   // ambos.
367   // El camino se divide en tres partes:
368   // 1. Desde u hasta el ancestro común.
369   // 2. Desde el ancestro común hasta v.
370   // 3. Desde el ancestro común hasta el ancestro común.
371   // El ancestro común es el nodo más profundo que es ancestro de
372   // ambos.
373   // El camino se divide en tres partes:
374   // 1. Desde u hasta el ancestro común.
375   // 2. Desde el ancestro común hasta v.
376   // 3. Desde el ancestro común hasta el ancestro común.
377   // El ancestro común es el nodo más profundo que es ancestro de
378   // ambos.
379   // El camino se divide en tres partes:
380   // 1. Desde u hasta el ancestro común.
381   // 2. Desde el ancestro común hasta v.
382   // 3. Desde el ancestro común hasta el ancestro común.
383   // El ancestro común es el nodo más profundo que es ancestro de
384   // ambos.
385   // El camino se divide en tres partes:
386   // 1. Desde u hasta el ancestro común.
387   // 2. Desde el ancestro común hasta v.
388   // 3. Desde el ancestro común hasta el ancestro común.
389   // El ancestro común es el nodo más profundo que es ancestro de
390   // ambos.
391   // El camino se divide en tres partes:
392   // 1. Desde u hasta el ancestro común.
393   // 2. Desde el ancestro común hasta v.
394   // 3. Desde el ancestro común hasta el ancestro común.
395   // El ancestro común es el nodo más profundo que es ancestro de
396   // ambos.
397   // El camino se divide en tres partes:
398   // 1. Desde u hasta el ancestro común.
399   // 2. Desde el ancestro común hasta v.
400   // 3. Desde el ancestro común hasta el ancestro común.
401   // El ancestro común es el nodo más profundo que es ancestro de
402   // ambos.
403   // El camino se divide en tres partes:
404   // 1. Desde u hasta el ancestro común.
405   // 2. Desde el ancestro común hasta v.
406   // 3. Desde el ancestro común hasta el ancestro común.
407   // El ancestro común es el nodo más profundo que es ancestro de
408   // ambos.
409   // El camino se divide en tres partes:
410   // 1. Desde u hasta el ancestro común.
411   // 2. Desde el ancestro común hasta v.
412   // 3. Desde el ancestro común hasta el ancestro común.
413   // El ancestro común es el nodo más profundo que es ancestro de
414   // ambos.
415   // El camino se divide en tres partes:
416   // 1. Desde u hasta el ancestro común.
417   // 2. Desde el ancestro común hasta v.
418   // 3. Desde el ancestro común hasta el ancestro común.
419   // El ancestro común es el nodo más profundo que es ancestro de
420   // ambos.
421   // El camino se divide en tres partes:
422   // 1. Desde u hasta el ancestro común.
423   // 2. Desde el ancestro común hasta v.
424   // 3. Desde el ancestro común hasta el ancestro común.
425   // El ancestro común es el nodo más profundo que es ancestro de
426   // ambos.
427   // El camino se divide en tres partes:
428   // 1. Desde u hasta el ancestro común.
429   // 2. Desde el ancestro común hasta v.
430   // 3. Desde el ancestro común hasta el ancestro común.
431   // El ancestro común es el nodo más profundo que es ancestro de
432   // ambos.
433   // El camino se divide en tres partes:
434   // 1. Desde u hasta el ancestro común.
435   // 2. Desde el ancestro común hasta v.
436   // 3. Desde el ancestro común hasta el ancestro común.
437   // El ancestro común es el nodo más profundo que es ancestro de
438   // ambos.
439   // El camino se divide en tres partes:
440   // 1. Desde u hasta el ancestro común.
441   // 2. Desde el ancestro común hasta v.
442   // 3. Desde el ancestro común hasta el ancestro común.
443   // El ancestro común es el nodo más profundo que es ancestro de
444   // ambos.
445   // El camino se divide en tres partes:
446   // 1. Desde u hasta el ancestro común.
447   // 2. Desde el ancestro común hasta v.
448   // 3. Desde el ancestro común hasta el ancestro común.
449   // El ancestro común es el nodo más profundo que es ancestro de
450   // ambos.
451   // El camino se divide en tres partes:
452   // 1. Desde u hasta el ancestro común.
453   // 2. Desde el ancestro común hasta v.
454   // 3. Desde el ancestro común hasta el ancestro común.
455   // El ancestro común es el nodo más profundo que es ancestro de
456   // ambos.
457   // El camino se divide en tres partes:
458   // 1. Desde u hasta el ancestro común.
459   // 2. Desde el ancestro común hasta v.
460   // 3. Desde el ancestro común hasta el ancestro común.
461   // El ancestro común es el nodo más profundo que es ancestro de
462   // ambos.
463   // El camino se divide en tres partes:
464   // 1. Desde u hasta el ancestro común.
465   // 2. Desde el ancestro común hasta v.
466   // 3. Desde el ancestro común hasta el ancestro común.
467   // El ancestro común es el nodo más profundo que es ancestro de
468   // ambos.
469   // El camino se divide en tres partes:
470   // 1. Desde u hasta el ancestro común.
471   // 2. Desde el ancestro común hasta v.
472   // 3. Desde el ancestro común hasta el ancestro común.
473   // El ancestro común es el nodo más profundo que es ancestro de
474   // ambos.
475   // El camino se divide en tres partes:
476   // 1. Desde u hasta el ancestro común.
477   // 2. Desde el ancestro común hasta v.
478   // 3. Desde el ancestro común hasta el ancestro común.
479   // El ancestro común es el nodo más profundo que es ancestro de
480   // ambos.
481   // El camino se divide en tres partes:
482   // 1. Desde u hasta el ancestro común.
483   // 2. Desde el ancestro común hasta v.
484   // 3. Desde el ancestro común hasta el ancestro común.
485   // El ancestro común es el nodo más profundo que es ancestro de
486   // ambos.
487   // El camino se divide en tres partes:
488   // 1. Desde u hasta el ancestro común.
489   // 2. Desde el ancestro común hasta v.
490   // 3. Desde el ancestro común hasta el ancestro común.
491   // El ancestro común es el nodo más profundo que es ancestro de
492   // ambos.
493   // El camino se divide en tres partes:
494   // 1. Desde u hasta el ancestro común.
495   // 2. Desde el ancestro común hasta v.
496   // 3. Desde el ancestro común hasta el ancestro común.
497   // El ancestro común es el nodo más profundo que es ancestro de
498   // ambos.
499   // El camino se divide en tres partes:
500   // 1. Desde u hasta el ancestro común.
501   // 2. Desde el ancestro común hasta v.
502   // 3. Desde el ancestro común hasta el ancestro común.
503   // El ancestro común es el nodo más profundo que es ancestro de
504   // ambos.
505   // El camino se divide en tres partes:
506   // 1. Desde u hasta el ancestro común.
507   // 2. Desde el ancestro común hasta v.
508   // 3. Desde el ancestro común hasta el ancestro común.
509   // El ancestro común es el nodo más profundo que es ancestro de
510   // ambos.
511   // El camino se divide en tres partes:
512   // 1. Desde u hasta el ancestro común.
513   // 2. Desde el ancestro común hasta v.
514   // 3. Desde el ancestro común hasta el ancestro común.
515   // El ancestro común es el nodo más profundo que es ancestro de
516   // ambos.
517   // El camino se divide en tres partes:
518   // 1. Desde u hasta el ancestro común.
519   // 2. Desde el ancestro común hasta v.
520   // 3. Desde el ancestro común hasta el ancestro común.
521   // El ancestro común es el nodo más profundo que es ancestro de
522   // ambos.
523   // El camino se divide en tres partes:
524   // 1. Desde u hasta el ancestro común.
525   // 2. Desde el ancestro común hasta v.
526   // 3. Desde el ancestro común hasta el ancestro común.
527   // El ancestro común es el nodo más profundo que es ancestro de
528   // ambos.
529   // El camino se divide en tres partes:
530   // 1. Desde u hasta el ancestro común.
531   // 2. Desde el ancestro común hasta v.
532   // 3. Desde el ancestro común hasta el ancestro común.
533   // El ancestro común es el nodo más profundo que es ancestro de
534   // ambos.
535   // El camino se divide en tres partes:
536   // 1. Desde u hasta el ancestro común.
537   // 2. Desde el ancestro común hasta v.
538   // 3. Desde el ancestro común hasta el ancestro común.
539   // El ancestro común es el nodo más profundo que es ancestro de
540   // ambos.
541   // El camino se divide en tres partes:
542   // 1. Desde u hasta el ancestro común.
543   // 2. Desde el ancestro común hasta v.
544   // 3. Desde el ancestro común hasta el ancestro común.
545   // El ancestro común es el nodo más profundo que es ancestro de
546   // ambos.
547   // El camino se divide en tres partes:
548   // 1. Desde u hasta el ancestro común.
549   // 2. Desde el ancestro común hasta v.
550   // 3. Desde el ancestro común hasta el ancestro común.
551   // El ancestro común es el nodo más profundo que es ancestro de
552   // ambos.
553   // El camino se divide en tres partes:
554   // 1. Desde u hasta el ancestro común.
555   // 2. Desde el ancestro común hasta v.
556   // 3. Desde el ancestro común hasta el ancestro común.
557   // El ancestro común es el nodo más profundo que es ancestro de
558   // ambos.
559   // El camino se divide en tres partes:
560   // 1. Desde u hasta el ancestro común.
561   // 2. Desde el ancestro común hasta v.
562   // 3. Desde el ancestro común hasta el ancestro común.
563   // El ancestro común es el nodo más profundo que es ancestro de
564   // ambos.
565   // El camino se divide en tres partes:
566   // 1. Desde u hasta el ancestro común.
567   // 2. Desde el ancestro común hasta v.
568   // 3. Desde el ancestro común hasta el ancestro común.
569   // El ancestro común es el nodo más profundo que es ancestro de
570   // ambos.
571   // El camino se divide en tres partes:
572   // 1. Desde u hasta el ancestro común.
573   // 2. Desde el ancestro común hasta v.
574   // 3. Desde el ancestro común hasta el ancestro común.
575   // El ancestro común es el nodo más profundo que es ancestro de
576   // ambos.
577   // El camino se divide en tres partes:
578   // 1. Desde u hasta el ancestro común.
579   // 2. Desde el ancestro común hasta v.
580   // 3. Desde el ancestro común hasta el ancestro común.
581   // El ancestro común es el nodo más profundo que es ancestro de
582   // ambos.
583   // El camino se divide en tres partes:
584   // 1. Desde u hasta el ancestro común.
585   // 2. Desde el ancestro común hasta v.
586   // 3. Desde el ancestro común hasta el ancestro común.
587   // El ancestro común es el nodo más profundo que es ancestro de
588   // ambos.
589   // El camino se divide en tres partes:
590   // 1. Desde u hasta el ancestro común.
591   // 2. Desde el ancestro común hasta v.
592   // 3. Desde el ancestro común hasta el ancestro común.
593   // El ancestro común es el nodo más profundo que es ancestro de
594   // ambos.
595   // El camino se divide en tres partes:
596   // 1. Desde u hasta el ancestro común.
597   // 2. Desde el ancestro común hasta v.
598   // 3. Desde el ancestro común hasta el ancestro común.
599   // El ancestro común es el nodo más profundo que es ancestro de
600   // ambos.
601   // El camino se divide en tres partes:
602   // 1. Desde u hasta el ancestro común.
603   // 2. Desde el ancestro común hasta v.
604   // 3. Desde el ancestro común hasta el ancestro común.
605   // El ancestro común es el nodo más profundo que es ancestro de
606   // ambos.
607   // El camino se divide en tres partes:
608   // 1. Desde u hasta el ancestro común.
609   // 2. Desde el ancestro común hasta v.
610   // 3. Desde el ancestro común hasta el ancestro común.
611   // El ancestro común es el nodo más profundo que es ancestro de
612   // ambos.
613   // El camino se divide en tres partes:
614   // 1. Desde u hasta el ancestro común.
615   // 2. Desde el ancestro común hasta v.
616   // 3. Desde el ancestro común hasta el ancestro común.
617   // El ancestro común es el nodo más profundo que es ancestro de
618   // ambos.
619   // El camino se divide en tres partes:
620   // 1. Desde u hasta el ancestro común.
621   // 2. Desde el ancestro común hasta v.
622   // 3. Desde el ancestro común hasta el ancestro común.
623   // El ancestro común es el nodo más profundo que es ancestro de
624   // ambos.
625   // El camino se divide en tres partes:
626   // 1. Desde u hasta el ancestro común.
627   // 2. Desde el ancestro común hasta v.
628   // 3. Desde el ancestro común hasta el ancestro común.
629   // El ancestro común es el nodo más profundo que es ancestro de
630   // ambos.
631   // El camino se divide en tres partes:
632   // 1. Desde u hasta el ancestro común.
633   // 2. Desde el ancestro común hasta v.
634   // 3. Desde el ancestro común hasta el ancestro común.
635   // El ancestro común es el nodo más profundo que es ancestro de
636   // ambos.
637   // El camino se divide en tres partes:
638   // 1. Desde u hasta el ancestro común.
639   // 2. Desde el ancestro común hasta v.
640   // 3. Desde el ancestro común hasta el ancestro común.
641   // El ancestro común es el nodo más profundo que es ancestro de
642   // ambos.
643   // El camino se divide en tres partes:
644   // 1. Desde u hasta el ancestro común.
645   // 2. Desde el ancestro común hasta v.
646   // 3. Desde el ancestro común hasta el ancestro común.
647   // El ancestro común es el nodo más profundo que es ancestro de
648   // ambos.
649   // El camino se divide en tres partes:
650   // 1. Desde u hasta el ancestro común.
651   // 2. Desde el ancestro común hasta v.
652   // 3. Desde el ancestro común hasta el ancestro común.
653   // El ancestro común es el nodo más profundo que es ancestro de
654   // ambos.
655   // El camino se divide en tres partes:
656   // 1. Desde u hasta el ancestro común.
657   // 2. Desde el ancestro común hasta v.
658   // 3. Desde el ancestro común hasta el ancestro común.
659   // El ancestro común es el nodo más profundo que es ancestro de
660   // ambos.
661   // El camino se divide en tres partes:
662   // 1. Desde u hasta el ancestro común.
663   // 2. Desde el ancestro común hasta v.
664   // 3. Desde el ancestro común hasta el ancestro común.
665   // El ancestro común es el nodo más profundo que es ancestro de
666   // ambos.
667   // El camino se divide en tres partes:
668   // 1. Desde u hasta el ancestro común.
669   // 2. Desde el ancestro común hasta v.
670   // 3. Desde el ancestro común hasta el ancestro común.
671   // El ancestro común es el nodo más profundo que es ancestro de
672   // ambos.
673   // El camino se divide en tres partes:
674   // 1. Desde u hasta el ancestro común.
675   // 2. Desde el ancestro común hasta v.
676   // 3. Desde el ancestro común hasta el ancestro común.
677   // El ancestro común es el nodo más profundo que es ancestro de
678   // ambos.
679   // El camino se divide en tres partes:
680   // 1. Desde u hasta el ancestro común.
681   // 2. Desde el ancestro común hasta v.
682   // 3. Desde el ancestro común hasta el ancestro común.
683   // El ancestro común es el nodo más profundo que es ancestro de
684   // ambos.
685   // El camino se divide en tres partes:
686   // 1. Desde u hasta el ancestro común.
687   // 2. Desde el ancestro común hasta v.
688   // 3. Desde el ancestro común hasta el ancestro común.
689   // El ancestro común es el nodo más profundo que es ancestro de
690   // ambos.
691   // El camino se divide en tres partes:
692   // 1. Desde u hasta el ancestro común.
693   // 2. Desde el ancestro común hasta v.
694   // 3. Desde el ancestro común hasta el ancestro común.
695   // El ancestro común es el nodo más profundo que es ancestro de
696   // ambos.
697   // El camino se divide en tres partes:
698   // 1. Desde u hasta el ancestro común.
699   // 2. Desde el ancestro común hasta v.
700
```

```

67         pesado[u] = v;
68     }
69 }
70 }
71 // O(n)
72
73 // Pila de pares (nodo, cabeza de su cadena). Para que el
74 // hijo
75 // pesado quede en el tope de la pila (y por lo tanto se
76 // procese inmediatamente despues, como en la version
77 // recursiva), se empuja de ULTIMO: los hijos livianos
78 // (cadena nueva) se empujan primero.
79 void dfsDescomponer(int raiz) {
80     stack<pair<int, int>> st;
81     st.push({raiz, raiz});
82
83     while (!st.empty()) {
84         auto [u, h] = st.top();
85         st.pop();
86
87         cabeza[u] = h;
88         pos[u] = timer++;
89
90         for (int v : adj[u]) {
91             if (v == padre[u] || v == pesado[u]) continue;
92             st.push({v, v}); // cadena nueva
93         }
94
95         if (pesado[u] != -1)
96             st.push({pesado[u], h}); // misma cadena
97     }
98     // O(n)
99
100 // READ: suma de todos los nodos en el camino u-v.
101 long long consultarCamino(int u, int v) {
102     long long resultado = 0;
103
104     while (cabeza[u] != cabeza[v]) {
105         if (profundidad[cabeza[u]] < profundidad[cabeza[v]])
106             swap(u, v);
107
108         resultado += seg->rango(pos[cabeza[u]], pos[u]);
109         u = padre[cabeza[u]];
110     }
111
112     if (profundidad[u] > profundidad[v]) swap(u, v);
113     resultado += seg->rango(pos[u], pos[v]);
114     return resultado; // O(log^2 n)
115
116 // UPDATE: suma val a todos los nodos en el camino u-v.
117 void actualizarCamino(int u, int v, long long val) {
118     while (cabeza[u] != cabeza[v]) {
119         if (profundidad[cabeza[u]] < profundidad[cabeza[v]])
120             swap(u, v);
121
122         seg->rangoUpdate(pos[cabeza[u]], pos[u], val);
123         u = padre[cabeza[u]];
124     }
125
126     if (profundidad[u] > profundidad[v]) swap(u, v);
127     seg->rangoUpdate(pos[u], pos[v], val); // O(log^2 n)
128 }
129
130 // READ: suma de todo el subarbol de u (rango contiguo por
131 // construccion, gracias al orden DFS de dfsDescomponer).
132 long long consultarSubarbol(int u) {
133     return seg->rango(pos[u], pos[u] + tamano[u] - 1);
134 } // O(log n)
135
136 // UPDATE: suma val a todo el subarbol de u.
137 void actualizarSubarbol(int u, long long val) {
138     seg->rangoUpdate(pos[u], pos[u] + tamano[u] - 1, val);
139 } // O(log n)
140 };

```

Ejemplo de funcionamiento:

```

1 // arbol:      0
2 //             /\
3 //            1 2 3
4 //             /\
5 //            4 5
6
7 vector<vector<int>> adj(6);
8 adj[0] = {1, 2, 3};
9 adj[1] = {0, 4, 5};
10 adj[2] = {0};
11 adj[3] = {0};
12 adj[4] = {1};
13 adj[5] = {1};
14
15 vector<long long> valores = {1, 2, 3, 4, 5, 6};

```

```

16 // nodo:      0 1 2 3 4 5
17
18 HLD hld(adj, valores, 0);
19
20 cout << hld.consultarCamino(4, 3);
21 // camino 4 -> 1 -> 0 -> 3 : valores 5+2+1+4 = 12
22
23 cout << hld.consultarSubarbol(1);
24 // subarbol de 1: nodos {1,4,5} : 2+5+6 = 13
25
26 hld.actualizarCamino(4, 5, 10);
27 // suma 10 a cada nodo del camino 4 -> 1 -> 5
28
29 cout << hld.consultarSubarbol(0);
30 // subarbol de 0 (todo el arbol) tras el update:
31 // 1+(2+10)+3+4+(5+10)+(6+10) = 51

```

Nota: al ser iterativas, `dfsTamano` y `dfsDescomponer` no tienen riesgo de stack overflow incluso en árboles con $n \geq 10^5$ y forma de “cadena larga” (el peor caso para recursión). `SegmentTreeLazyAdd` es exactamente el struct de la sección de Segment Tree — cualquier otra variante (min, max, assign) funciona igual de bien como motor subyacente.

Regla práctica: usa HLD en cuanto el problema pida algo sobre un **camino** entre dos nodos de un árbol con muchas queries. Si solo pide algo sobre **subárboles** (sin caminos arbitrarios), no hace falta la descomposición en cadenas completa: basta un solo DFS para las posiciones (`pos[]` y `tamano[]`) y un Segment Tree normal, ya que el subárbol siempre es un rango contiguo por sí solo.

Parte III Grafos

5.12 ¿Qué algoritmo usar?

La siguiente tabla resume situaciones típicas de Competitive Programming y el algoritmo que debe considerarse. La idea es reconocer las palabras clave o condiciones del enunciado y asociarlas rápidamente con una técnica.

Cuando el ejercicio diga / pista	Usa	Pista rápida
"Recorrer el grafo por niveles"	BFS	Usa una cola.
"Encontrar la distancia mínima" con aristas de igual costo	BFS	Cada arista cuesta lo mismo.
"Encontrar caminos mínimos" con pesos 0 y 1	0-1 BFS	Deque; pesos únicamente 0 o 1.
"Camino mínimo" con pesos no negativos	Dijkstra	Usa una cola de prioridad.
"Camino mínimo" con pesos negativos	Bellman-Ford	Permite detectar ciclos negativos.
"Camino mínimo entre todos los pares" y n pequeño	Floyd-Warshall	DP sobre todos los pares.
"¿Existe un camino entre u y v ?"	DFS / BFS	Problema de conectividad.
"Encontrar componentes conexas"	DFS / BFS	Ejecutar traversal desde cada nodo no visitado.
"Detectar si existe un ciclo" en grafo no dirigido	DFS / DSU	DFS para recorrer; DSU si se agregan aristas.
"Detectar ciclos" en grafo dirigido	DFS / Kahn	Usar estados en DFS o grados de entrada.
"Ordenar tareas con dependencias"	Topological Sort	El grafo debe ser dirigido y acíclico.
"Encontrar puentes"	Tarjan	DFS + tiempos de descubrimiento.
"Encontrar puntos de articulación"	Tarjan	Vértices cuya eliminación desconecta el grafo.
"Encontrar componentes fuertemente conexas"	Kosaraju / Tarjan	Grafo dirigido.
"Conectar todos los nodos con costo mínimo"	MST	Kruskal o Prim.
"MST procesando aristas ordenadas por peso"	Kruskal	Ordenar aristas + DSU.
"MST expandiendo desde un nodo"	Prim	Priority Queue + lista de adyacencia.
"¿Dos nodos pertenecen al mismo conjunto?"	DSU	Union-Find.
"Agrupar elementos mediante uniones"	DSU	union + find.
"Encontrar si un grafo es bipartito"	2-Coloring	Colorear con dos colores.
"Máximo número de parejas entre dos grupos"	Bipartite Matching	Emparejamiento entre conjuntos.
"Máximo flujo de una fuente a un destino"	Dinic	Network Flow.
"Separar fuente y sumidero con capacidad mínima"	Min-Cut	Por el teorema Max-Flow Min-Cut.
"Distancia entre dos nodos de un árbol"	LCA	$\text{dist}(u, v) = d[u] + d[v] - 2d[\text{LCA}(u, v)]$
"Encontrar el ancestro k niveles arriba"	Binary Lifting	Saltar potencias de 2.
"Consultas sobre subárboles"	Euler Tour + Fenwick / Segment Tree	Convertir subárbol en intervalo.
"Consultas o actualizaciones sobre caminos de un árbol"	HLD	Descomponer el camino en cadenas.
"Encontrar el diámetro de un árbol"	BFS / DFS	Dos traversals desde extremos.
"Encontrar un camino que use todas las aristas"	Eulerian Path / Hierholzer	Trabaja con aristas, no con vértices.
"Encontrar todas las rutas posibles"	DFS + Backtracking	Explorar y deshacer decisiones.
"Rellenar una región conectada"	Flood Fill	DFS / BFS desde una celda inicial.
"Moverse por una cuadrícula buscando la menor cantidad de pasos"	BFS	Cada movimiento tiene costo 1.
"Variables booleanas con restricciones de la forma OR"	2-SAT + SCC	Construir grafo de implicaciones.
"Resolver muchas consultas sobre un árbol"	LCA / HLD	Depende de si son ancestros o caminos.

Table 7: Guía rápida para seleccionar algoritmos de grafos.

Condición del problema	Algoritmo	Complejidad
Aristas sin peso / costo 1	BFS	$O(n + m)$
Pesos únicamente 0 y 1	0-1 BFS	$O(n + m)$
Pesos no negativos	Dijkstra	$O((n + m) \log n)$
Puede haber pesos negativos	Bellman-Ford	$O(nm)$
Todos los pares y n pequeño	Floyd-Warshall	$O(n^3)$

Table 8: Selección del algoritmo para caminos mínimos.

6 Traversal

6.1 Representación de Grafos

Idea: antes de aplicar cualquier algoritmo sobre un grafo, primero debemos decidir cómo almacenar sus vértices y aristas. Las tres representaciones más comunes son la **lista de adyacencia**, la **matriz de adyacencia** y la **lista de aristas**. La elección depende principalmente de la cantidad de vértices y aristas y del tipo de consultas que realizará el algoritmo.

La **lista de adyacencia** almacena, para cada vértice, todos los vértices conectados directamente con él. Es la representación más utilizada en Competitive Programming porque ocupa $O(n + m)$ memoria y permite recorrer eficientemente todos los vecinos de un vértice.

La **matriz de adyacencia** utiliza una matriz $n \times n$ donde la posición (u, v) indica si existe una arista entre u y v . Permite comprobar la existencia de una arista en $O(1)$, pero requiere $O(n^2)$ memoria.

La **lista de aristas** simplemente almacena cada arista como un par (u, v) , o como una terna (u, v, w) si tiene peso. Es especialmente útil para algoritmos que necesitan procesar todas las aristas, como Kruskal o Bellman-Ford.

¿Por qué usar cada una?

- **Lista de adyacencia:** opción general para BFS, DFS, Dijkstra, Prim, búsqueda de ciclos y prácticamente la mayoría de algoritmos sobre grafos dispersos.
- **Matriz de adyacencia:** útil cuando n es pequeño o cuando se necesita comprobar constantemente si existe una arista entre dos vértices en $O(1)$.
- **Lista de aristas:** conveniente cuando el algoritmo necesita recorrer todas las aristas directamente, especialmente en Kruskal y Bellman-Ford.

Tipos de grafos: una arista puede ser **dirigida** o **no dirigida**, y puede tener o no un **peso**. En un grafo no dirigido, una arista (u, v) se representa agregando v a los vecinos de u y u a los vecinos de v . En un grafo dirigido, solamente se agrega v a los vecinos de u .

Complejidad:

Representación	Memoria	Consulta	Vecinos
Lista de adyacencia	$O(n + m)$	$O(\deg(u))$	$O(\deg(u))$
Matriz de adyacencia	$O(n^2)$	$O(1)$	$O(n)$
Lista de aristas	$O(m)$	$O(m)$	$O(m)$

Table 9: Comparación de representaciones de grafos.

Lista de adyacencia

Idea de la implementación: se utiliza un `vector<vector<int>>` donde la posición u contiene todos los vértices conectados directamente con u . Para grafos ponderados se puede almacenar un par (v, w) , donde v es el destino y w el peso de la arista.

```
1 // Grafo no dirigido y no ponderado.
2 int n = 5;
3 vector<vector<int>> adj(n);
4
5 // Agrega una arista u-v.
6 auto agregarArista = [&](int u, int v) {
7     adj[u].push_back(v);
8     adj[v].push_back(u);
9 };
10
11 agregarArista(0, 1);
12 agregarArista(0, 2);
13 agregarArista(1, 3);
14 agregarArista(1, 4);
```

Grafo resultante:

```
1 //      0
2 //      / \
3 //      1 2
4 //      / \
5 //      3 4
6 //
7 // adj[0] = {1, 2}
8 // adj[1] = {0, 3, 4}
9 // adj[2] = {0}
10 // adj[3] = {1}
11 // adj[4] = {1}
```

Para un grafo dirigido, solamente se agrega la arista en una dirección:

```
1 // Grafo dirigido.
2 vector<vector<int>> adj(n);
3
4 auto agregarArista = [&](int u, int v) {
5     adj[u].push_back(v);
6 };
7
8 agregarArista(0, 1);
9 agregarArista(0, 2);
10 agregarArista(1, 3);
```

Para un grafo ponderado, cada vecino puede almacenar también el peso:

```
1 // Grafo ponderado.
2 vector<vector<pair<int, long long>>> adj(n);
3
4 auto agregarArista = [&](int u, int v, long long w) {
5     adj[u].push_back({v, w});
6     adj[v].push_back({u, w});
7 };
8
9 agregarArista(0, 1, 5);
10 agregarArista(0, 2, 3);
11 agregarArista(1, 3, 7);
```

Ejemplo de recorrido:

```
1 void dfs(int u, int padre, vector<vector<int>>& adj) {
2
3     cout << u << ' ';
4
5     for (int v : adj[u]) {
6         if (v == padre) continue;
7
8         dfs(v, u, adj);
9     }
10 }
```

La lista de adyacencia permite que DFS o BFS procesen cada vértice y cada arista una cantidad constante de veces, obteniendo una complejidad total de $O(n + m)$.

Matriz de adyacencia

Idea: una matriz $adj[u][v]$ indica directamente si existe una conexión desde u hacia v . Si el grafo es ponderado, la posición puede almacenar directamente el peso de la arista.

```
1 int n = 5;
2 vector<vector<int>> adj(n, vector<int>(n, 0));
3
4 auto agregarArista = [&](int u, int v) {
5     adj[u][v] = 1;
6     adj[v][u] = 1;
7 };
8
9 agregarArista(0, 1);
10 agregarArista(0, 2);
11 agregarArista(1, 3);
12 agregarArista(1, 4);
```

Ahora comprobar si existe una arista entre u y v es inmediato:

```
1 if (adj[u][v]) {
2     cout << "Existe una arista";
3 }
```

La consulta anterior cuesta $O(1)$, pero recorrer todos los vecinos de u requiere revisar las n posiciones de la fila:

```
1 for (int v = 0; v < n; v++) {
2     if (adj[u][v]) {
3         cout << v << ' ';
4     }
5 }
```

Lista de aristas

Idea: almacena directamente todas las aristas del grafo. Para un grafo no ponderado basta guardar (u, v) ; para uno ponderado se guarda (u, v, w) .

```
1 // Grafo ponderado.
2 struct Edge {
3     int u, v;
4     long long w;
5 };
6
7 vector<Edge> edges;
8
9 edges.push_back({0, 1, 5});
10 edges.push_back({0, 2, 3});
11 edges.push_back({1, 3, 7});
```

Esta representación resulta especialmente conveniente cuando el algoritmo necesita procesar todas las aristas:

```
1 for (Edge e : edges) {
2     cout << e.u << " -> " << e.v
3         << " peso = " << e.w << '\n';
4 }
```

Por ejemplo, **Kruskal** ordena directamente esta lista por peso, mientras que **Bellman-Ford** puede recorrerla completa en cada iteración.

Nota: para la mayoría de problemas de Competitive Programming, la **lista de adyacencia** debe ser la representación por defecto. Es especialmente importante cuando el grafo es disperso, es decir, cuando m es mucho menor que n^2 . La matriz de adyacencia solamente conviene cuando n es pequeño o cuando las consultas de existencia de aristas en $O(1)$ son importantes.

Regla práctica: si no tienes una razón específica para usar otra representación, usa **lista de adyacencia**. Si necesitas preguntar constantemente “¿existe una arista $u \rightarrow v$?”, considera una **matriz de adyacencia**. Si

el algoritmo trabaja directamente con todas las aristas, considera una **lista de aristas**.

“`latex`

6.2 BFS (Breadth-First Search)

Idea: BFS (*Breadth-First Search*) es un algoritmo de recorrido de grafos que explora los vértices por **niveles**. A partir de un vértice inicial, primero visita todos sus vecinos directos, después los vecinos de estos, y continúa de esta manera hasta procesar todos los vértices alcanzables.

Para mantener este orden se utiliza una **cola** (*queue*), siguiendo el principio FIFO (*First In, First Out*). Cada vértice se marca como visitado cuando se agrega a la cola, evitando procesarlo varias veces.

Una de las propiedades más importantes de BFS es que, en un grafo **no ponderado**, encuentra la distancia mínima desde un vértice origen hasta cualquier otro vértice alcanzable, donde la distancia se mide como el número de aristas recorridas.

¿Por qué usarlo?

- **Camino mínimo:** encuentra el camino con menor número de aristas en grafos no ponderados.
- **Distancias:** permite calcular la distancia mínima desde un origen hasta todos los vértices alcanzables.
- **Recorrido por niveles:** permite procesar los vértices según su distancia al origen.
- **Reconstrucción de caminos:** almacenando el padre de cada vértice se puede recuperar el camino mínimo.
- **Bipartición:** permite determinar si un grafo es bipartito utilizando dos colores.
- **Componentes conexas:** ejecutando BFS desde cada vértice no visitado se pueden encontrar todas las componentes conexas.
- **Problemas en grids:** permite encontrar la cantidad mínima de movimientos cuando cada movimiento tiene el mismo costo.
- **Multi-Source BFS:** permite calcular la distancia mínima desde cualquiera de varios vértices iniciales.

Funcionamiento:

El algoritmo sigue los siguientes pasos:

1. Elegir un vértice inicial s .
2. Marcar s como visitado y agregarlo a la cola.
3. Mientras la cola no esté vacía, extraer el primer vértice.
4. Recorrer todos los vecinos del vértice extraído.
5. Si un vecino no ha sido visitado, marcarlo y agregarlo a la cola.

La cola garantiza que los vértices se procesen por niveles. Primero se procesan los vértices a distancia 0, después los de distancia 1, luego los de distancia 2, y así sucesivamente.

Implementación básica:

```
1 void bfs(int inicio, vector<vector<int>>& adj) {
2
3     int n = adj.size();
4
5     vector<bool> visitado(n, false);
6     queue<int> q;
7
8     visitado[inicio] = true;
9     q.push(inicio);
10
11     while (!q.empty()) {
12
13         int u = q.front();
14         q.pop();
15
16         cout << u << ' ';
17
18         for (int v : adj[u]) {
19
20             if (visitado[v]) continue;
21
22             visitado[v] = true;
23             q.push(v);
24         }
25     }
26 }
```

Ejemplo:

Consideremos el siguiente grafo:

```
1 //      0
2 //     / \
3 //    1   2
4 //   / \
5 //  3   4
```

Si comenzamos BFS desde el vértice 0, los niveles son:

```
1 // Nivel 0: 0
2 // Nivel 1: 1 2
3 // Nivel 2: 3 4
```

Por lo tanto, un posible orden de recorrido es:

```
1 0 1 2 3 4
```

El orden entre vértices del mismo nivel depende del orden en que aparezcan en la lista de adyacencia.

BFS para encontrar distancias:

Para calcular la distancia mínima desde el origen hasta cada vértice, se utiliza un arreglo `dist`. Inicialmente todos los valores son -1 , indicando que el vértice todavía no ha sido visitado.

Cuando BFS descubre un nuevo vértice v desde u , su distancia se calcula como:

$$dist[v] = dist[u] + 1$$

```
1 vector<int> bfsDist(int inicio, vector<vector<int>>& adj) {
2
3     int n = adj.size();
4
5     vector<int> dist(n, -1);
6     queue<int> q;
7
8     dist[inicio] = 0;
9     q.push(inicio);
10
11     while (!q.empty()) {
12
13         int u = q.front();
14         q.pop();
15
16         for (int v : adj[u]) {
17
18             if (dist[v] != -1) continue;
19
20             dist[v] = dist[u] + 1;
```

```

21     q.push(v);
22 }
23 }
24
25 return dist;
26 }

```

Para el ejemplo anterior, comenzando desde 0, se obtiene:

```

1 // dist[0] = 0
2 // dist[1] = 1
3 // dist[2] = 1
4 // dist[3] = 2
5 // dist[4] = 2

```

Por ejemplo, $dist[3] = 2$ significa que el camino mínimo desde 0 hasta 3 utiliza dos aristas.

Reconstrucción del camino:

Si además de la distancia se necesita conocer el camino utilizado, se puede almacenar el **padre** de cada vértice. Cuando BFS descubre v desde u , se establece:

$$padre[v] = u$$

```

1 vector<int> dist(n, -1);
2 vector<int> padre(n, -1);
3
4 queue<int> q;
5
6 dist[inicio] = 0;
7 q.push(inicio);
8
9 while (!q.empty()) {
10
11     int u = q.front();
12     q.pop();
13
14     for (int v : adj[u]) {
15
16         if (dist[v] != -1) continue;
17
18         dist[v] = dist[u] + 1;
19         padre[v] = u;
20         q.push(v);
21     }
22 }
23

```

Una vez terminado BFS, el camino hasta un destino se reconstruye siguiendo los padres desde el destino hasta el origen:

```

1 vector<int> camino;
2
3 for (int v = destino; v != -1; v = padre[v]) {
4     camino.push_back(v);
5 }
6
7 reverse(camino.begin(), camino.end());

```

Por ejemplo, si:

```

1 // padre[3] = 1
2 // padre[1] = 0
3 // padre[0] = -1

```

el camino mínimo desde 0 hasta 3 es:

```

1 0 -> 1 -> 3

```

BFS en grafos no conectados:

Un BFS iniciado desde un único vértice solamente visita los vértices que pueden alcanzarse desde él. Para recorrer todo el grafo, se puede ejecutar BFS desde cada vértice que todavía no haya sido visitado.

```

1 vector<bool> visitado(n, false);
2
3 for (int s = 0; s < n; s++) {
4
5     if (visitado[s]) continue;
6

```

```

7     queue<int> q;
8
9     visitado[s] = true;
10    q.push(s);
11
12    while (!q.empty()) {
13
14        int u = q.front();
15        q.pop();
16
17        for (int v : adj[u]) {
18
19            if (visitado[v]) continue;
20
21            visitado[v] = true;
22            q.push(v);
23        }
24    }
25 }

```

Esta técnica permite encontrar, entre otras cosas, el número de **componentes conexas** de un grafo.

BFS para comprobar si un grafo es bipartito:

Un grafo es bipartito si sus vértices pueden dividirse en dos grupos de manera que ninguna arista conecte dos vértices del mismo grupo. BFS puede comprobar esta propiedad asignando alternativamente los colores 0 y 1 a los vértices.

Si un vértice u tiene color c , sus vecinos deben tener el color $c \oplus 1$. Si una arista conecta dos vértices del mismo color, el grafo no es bipartito.

```

1 bool esBipartito(vector<vector<int>>& adj) {
2
3     int n = adj.size();
4
5     vector<int> color(n, -1);
6
7     for (int inicio = 0; inicio < n; inicio++) {
8
9         if (color[inicio] != -1) continue;
10
11         queue<int> q;
12
13         color[inicio] = 0;
14         q.push(inicio);
15
16         while (!q.empty()) {
17
18             int u = q.front();
19             q.pop();
20
21             for (int v : adj[u]) {
22
23                 if (color[v] == -1) {
24                     color[v] = color[u] ^ 1;
25                     q.push(v);
26                 } else if (color[v] == color[u]) {
27                     return false;
28                 }
29             }
30         }
31     }
32
33     return true;
34 }
35
36 }
37

```

El arreglo `color` también funciona como arreglo de visitados: un valor de -1 significa que el vértice todavía no ha sido procesado.

BFS en un grid:

BFS también puede utilizarse sobre matrices o tableros. Cada celda se considera un vértice y cada movimiento válido entre celdas se considera una arista.

Por ejemplo, si solamente se permiten movimientos hacia arriba, abajo, izquierda y derecha:

```

1 int dx[] = {-1, 1, 0, 0};
2 int dy[] = {0, 0, -1, 1};
3
4 queue<pair<int, int>> q;

```

```

5
6 q.push({sx, sy});
7 dist[sx][sy] = 0;
8
9 while (!q.empty()) {
10
11     auto [x, y] = q.front();
12     q.pop();
13
14     for (int d = 0; d < 4; d++) {
15
16         int nx = x + dx[d];
17         int ny = y + dy[d];
18
19         if (nx < 0 || nx >= n ||
20             ny < 0 || ny >= m)
21             continue;
22
23         if (dist[nx][ny] != -1)
24             continue;
25
26         if (grid[nx][ny] == '#')
27             continue;
28
29         dist[nx][ny] = dist[x][y] + 1;
30         q.push({nx, ny});
31     }
32 }

```

Esta técnica es especialmente útil en problemas donde se pregunta por la menor cantidad de movimientos necesarios para llegar desde una celda hasta otra.

Multi-Source BFS:

En algunos problemas existen varios vértices iniciales al mismo tiempo. En lugar de ejecutar un BFS separado desde cada uno, todos los orígenes se introducen inicialmente en la misma cola con distancia 0.

```

1 queue<int> q;
2 vector<int> dist(n, -1);
3
4 for (int s : fuentes) {
5
6     dist[s] = 0;
7     q.push(s);
8 }
9
10 while (!q.empty()) {
11
12     int u = q.front();
13     q.pop();
14
15     for (int v : adj[u]) {
16
17         if (dist[v] != -1) continue;
18
19         dist[v] = dist[u] + 1;
20         q.push(v);
21     }
22 }

```

De esta manera, cada vértice obtiene la distancia mínima hasta **cualquiera** de las fuentes.

Complejidad:

Utilizando una lista de adyacencia, cada vértice se procesa una sola vez y cada arista se revisa una cantidad constante de veces.

Representación	Tiempo	Memoria adicional
Lista de adyacencia	$O(n + m)$	$O(n)$
Matriz de adyacencia	$O(n^2)$	$O(n)$

Table 10: Complejidad de BFS según la representación del grafo.

La complejidad $O(n + m)$ hace que BFS sea especialmente eficiente para grafos dispersos representados mediante listas de adyacencia.

Nota: BFS encuentra caminos mínimos cuando todas las aristas tienen el mismo costo. No debe utilizarse directamente para encontrar caminos mínimos en grafos

con pesos arbitrarios. Para pesos no negativos se puede utilizar **Dijkstra**; si los pesos solamente son 0 y 1, una alternativa eficiente es **0-1 BFS**.

Regla práctica: si un problema pide minimizar el número de **pasos, movimientos o aristas** y cada movimiento tiene el mismo costo, considera primero **BFS**. Si necesitas conocer la distancia, utiliza **dist**; si necesitas recuperar el camino, utiliza **padre**; y si existen múltiples puntos de origen, considera **Multi-Source BFS**. “

“`latex`

6.3 BFS en Grafos Desconectados

Idea: un BFS iniciado desde un único vértice solamente visita los vértices que son alcanzables desde él. Si el grafo tiene varias **componentes conexas**, es necesario iniciar un nuevo BFS desde cada vértice que todavía no haya sido visitado.

La idea consiste en recorrer todos los vértices y, cada vez que se encuentra uno que no ha sido visitado, iniciar un nuevo BFS desde él. Cada ejecución de BFS procesa una componente conexa completa.

Implementación:

```

1 void bfs(int inicio, vector<vector<int>>& adj,
2         vector<bool>& visitado) {
3
4     queue<int> q;
5
6     visitado[inicio] = true;
7     q.push(inicio);
8
9     while (!q.empty()) {
10
11         int u = q.front();
12         q.pop();
13
14         for (int v : adj[u]) {
15
16             if (visitado[v]) continue;
17
18             visitado[v] = true;
19             q.push(v);
20         }
21     }
22 }

```

Para recorrer todo el grafo:

```

1 vector<bool> visitado(n, false);
2
3 for (int u = 0; u < n; u++) {
4
5     if (visitado[u]) continue;
6
7     bfs(u, adj, visitado);
8 }

```

Cada vez que se ejecuta **bfs**, se procesa una componente conexa que todavía no había sido visitada.

Contar componentes conexas:

Una aplicación directa consiste en contar cuántas componentes conexas tiene el grafo. Basta incrementar un contador cada vez que se inicia un nuevo BFS.

```

1 int componentes = 0;
2
3 vector<bool> visitado(n, false);
4
5 for (int u = 0; u < n; u++) {
6
7     if (visitado[u]) continue;
8
9     componentes++;
10
11     queue<int> q;
12
13     visitado[u] = true;
14     q.push(u);
15 }

```

```

16 while (!q.empty()) {
17     int v = q.front();
18     q.pop();
19     for (int w : adj[v]) {
20         if (visitado[w]) continue;
21         visitado[w] = true;
22         q.push(w);
23     }
24 }
25 cout << componentes << '\n';

```

Ejemplo:

Consideremos el siguiente grafo:

```

1 // 0 --- 1      3 --- 4
2 //      |
3 //      2
4 //
5 // 5 --- 6

```

El grafo tiene tres componentes conexas:

```

1 // Componente 1: {0, 1, 2}
2 // Componente 2: {3, 4}
3 // Componente 3: {5, 6}

```

Si recorremos los vértices en orden, el primer BFS puede comenzar desde 0 y visitar {0, 1, 2}. Después, los vértices 1 y 2 ya están visitados, por lo que el siguiente BFS comienza desde 3 y visita {3, 4}. Finalmente, se inicia otro BFS desde 5 para visitar {5, 6}.

Complejidad:

Aunque se ejecuten varios BFS, cada vértice se visita una sola vez y cada arista se procesa una cantidad constante de veces. Por lo tanto, la complejidad total utilizando una lista de adyacencia sigue siendo:

$$O(n + m)$$

La memoria adicional utilizada por el arreglo de visitados y la cola es $O(n)$.

Operación	Complejidad
Recorrer todas las componentes	$O(n + m)$
Contar componentes conexas	$O(n + m)$
Memoria adicional	$O(n)$

Table 11: Complejidad de BFS en grafos desconectados.

Regla práctica: si el grafo puede estar desconectado y se necesita procesar **todos los vértices**, no basta con ejecutar BFS una sola vez. Recorre todos los vértices y comienza un nuevo BFS cada vez que encuentres uno que todavía no haya sido visitado. ““

6.4 BFS para Todos los Caminos

Idea: BFS no solamente permite encontrar la distancia mínima desde un vértice origen hacia los demás vértices. También puede utilizarse para reconstruir el **camino mínimo** desde el origen hasta cualquier vértice alcanzable.

Para reconstruir un camino, además de almacenar la distancia, se guarda para cada vértice su **padre**, es decir, el vértice desde el cual fue descubierto por primera vez. Como BFS visita los vértices por niveles, el primer padre asignado a un vértice pertenece a un camino mínimo.

Implementación:

```

1 vector<int> dist(n, -1);
2 vector<int> padre(n, -1);
3
4 queue<int> q;
5
6 dist[inicio] = 0;
7 q.push(inicio);
8
9 while (!q.empty()) {
10     int u = q.front();
11     q.pop();
12     for (int v : adj[u]) {
13         if (dist[v] != -1) continue;
14         dist[v] = dist[u] + 1;
15         padre[v] = u;
16         q.push(v);
17     }
18 }

```

Después de ejecutar BFS, `dist[v]` contiene la distancia mínima desde `inicio` hasta `v`, mientras que `padre[v]` indica qué vértice se debe seguir para reconstruir el camino.

Reconstrucción del camino:

Para obtener el camino hasta un vértice destino, se comienza en `destino` y se siguen los padres hasta llegar al origen.

```

1 vector<int> camino;
2
3 for (int v = destino; v != -1; v = padre[v]) {
4     camino.push_back(v);
5 }
6
7 reverse(camino.begin(), camino.end());

```

El vector `camino` contiene ahora los vértices en el orden correcto, desde el origen hasta el destino.

Ejemplo:

Supongamos que BFS produce los siguientes padres:

```

1 // padre[0] = -1
2 // padre[1] = 0
3 // padre[2] = 0
4 // padre[3] = 1
5 // padre[4] = 3

```

Si el origen es 0 y queremos reconstruir el camino hasta 4, seguimos los padres:

$$4 \rightarrow 3 \rightarrow 1 \rightarrow 0$$

Después de invertirlo obtenemos:

$$0 \rightarrow 1 \rightarrow 3 \rightarrow 4$$

Por lo tanto, este es un camino mínimo desde 0 hasta 4.

Función para reconstruir el camino:

La reconstrucción puede encapsularse en una función:

```

1 vector<int> reconstruirCamino(int destino,
2                             vector<int>& padre) {
3
4     vector<int> camino;
5
6     for (int v = destino; v != -1; v = padre[v]) {
7         camino.push_back(v);
8     }
9
10    reverse(camino.begin(), camino.end());
11
12    return camino;
13 }

```

El BFS completo puede combinarse con esta función:


```

1 vector<int> dist(n, -1);
2 vector<int> padre(n, -1);
3
4 queue<int> q;
5
6 dist[inicio] = 0;
7 q.push(inicio);
8
9 while (!q.empty()) {
10
11     int u = q.front();
12     q.pop();
13
14     for (int v : adj[u]) {
15
16         if (dist[v] != -1) continue;
17
18         dist[v] = dist[u] + 1;
19         padre[v] = u;
20
21         q.push(v);
22     }
23 }
24
25 vector<int> camino =
26     reconstruirCamino(destino, padre);

```

¿Qué significa “todos los caminos”?

En este contexto, BFS obtiene un camino mínimo desde el origen hacia **cada vértice alcanzable**. No significa que encuentre todos los caminos posibles entre dos vértices, ya que la cantidad de caminos posibles puede ser exponencial.

Por ejemplo, después de ejecutar BFS desde s :

- $dist[v]$ indica la distancia mínima de s a v .
- $padre[v]$ permite reconstruir un camino mínimo hacia v .
- Si $dist[v]$ es -1 , v no es alcanzable desde s .

Camino inexistente:

Antes de reconstruir el camino conviene comprobar si el destino fue alcanzado:

```

1 if (dist[destino] == -1) {
2     cout << "No existe camino\n";
3 }
4 else {
5     vector<int> camino =
6         reconstruirCamino(destino, padre);
7
8     for (int v : camino) {
9         cout << v << ' ';
10    }
11 }

```

Complejidad:

La ejecución de BFS utilizando una lista de adyacencia cuesta $O(n+m)$. La reconstrucción de un camino cuesta $O(k)$, donde k es la cantidad de vértices que contiene el camino.

Operación	Complejidad
BFS	$O(n+m)$
Reconstruir un camino	$O(k)$
Memoria adicional	$O(n)$

Table 12: Complejidad de BFS para reconstrucción de caminos.

Nota: BFS encuentra caminos mínimos en grafos donde todas las aristas tienen el mismo costo. Si las aristas tienen pesos diferentes, deben utilizarse algoritmos como **Dijkstra**, **0-1 BFS** o **Bellman-Ford**, dependiendo de las características del problema.

Regla práctica: si el problema pide no solamente la distancia mínima sino también **qué vértices forman el camino**, guarda un arreglo **padre** durante el BFS. No es necesario almacenar todos los caminos: basta con guardar un padre por vértice.

6.5 DFS

Idea: Depth-First Search (DFS) explora un grafo siguiendo un camino lo más profundamente posible antes de regresar y explorar otras ramas. A diferencia de BFS, que utiliza una **cola**, DFS utiliza una **pila**.

DFS puede implementarse de forma recursiva o iterativa. En Competitive Programming, la versión iterativa es útil cuando el grafo puede tener muchos vértices y una implementación recursiva podría provocar un **stack overflow**.

Implementación iterativa:

Se utiliza un `stack<int>` para almacenar los vértices que deben ser procesados. Un vértice se marca como visitado cuando se introduce en la pila, evitando que pueda agregarse múltiples veces.

```

1 void dfs(int inicio, vector<vector<int>>& adj,
2         vector<bool>& visitado) {
3
4     stack<int> st;
5
6     visitado[inicio] = true;
7     st.push(inicio);
8
9     while (!st.empty()) {
10
11         int u = st.top();
12         st.pop();
13
14         cout << u << ' ';
15
16         for (int v : adj[u]) {
17
18             if (visitado[v]) continue;
19
20             visitado[v] = true;
21             st.push(v);
22         }
23     }
24 }

```

El orden exacto en que se visitan los vértices depende del orden de los vecinos en la lista de adyacencia. Lo importante es que cada vértice alcanzable sea procesado una vez.

Ejemplo:

Consideremos el siguiente grafo:

```

1 //      0
2 //      / \
3 //     1  2
4 //    / \
5 //   3  4

```

La lista de adyacencia puede ser:

```

1 adj[0] = {1, 2};
2 adj[1] = {0, 3, 4};
3 adj[2] = {0};
4 adj[3] = {1};
5 adj[4] = {1};

```

Si comenzamos DFS desde 0, la pila inicialmente contiene:

```

1 st = {0}

```

Se extrae 0 y se agregan sus vecinos:

```

1 st = {1, 2}

```

El siguiente vértice que se extrae depende del orden en que fueron agregados a la pila. Por ejemplo, si se extrae primero 2, se procesa esa rama antes de regresar a 1.

DFS en un grafo desconectado:

Al igual que BFS, una sola ejecución de DFS solamente visita los vértices alcanzables desde el origen. Para recorrer todo el grafo, debemos iniciar un nuevo DFS desde cada vértice que todavía no haya sido visitado.

```
1 vector<bool> visitado(n, false);
2
3 for (int u = 0; u < n; u++) {
4     if (visitado[u]) continue;
5     dfs(u, adj, visitado);
6 }
7
```

Este patrón permite procesar todas las componentes conexas del grafo.

Contar componentes conexas:

Podemos contar el número de componentes incrementando un contador cada vez que sea necesario iniciar un nuevo DFS.

```
1 int componentes = 0;
2
3 vector<bool> visitado(n, false);
4
5 for (int u = 0; u < n; u++) {
6     if (visitado[u]) continue;
7     componentes++;
8     dfs(u, adj, visitado);
9 }
10
11 cout << componentes << '\n';
12
13
```

DFS en árboles:

En un árbol no es necesario mantener un arreglo de visitados si se conoce el padre del vértice actual. Al recorrer un vecino, simplemente se evita regresar al padre.

Sin embargo, en un grafo general se recomienda utilizar `visitado`, ya que pueden existir ciclos.

```
1 void dfs(int inicio, vector<vector<int>>& adj) {
2
3     stack<pair<int, int>> st;
4
5     st.push({inicio, -1});
6
7     while (!st.empty()) {
8
9         auto [u, padre] = st.top();
10        st.pop();
11
12        cout << u << ' ';
13
14        for (int v : adj[u]) {
15
16            if (v == padre) continue;
17            st.push({v, u});
18        }
19    }
20 }
21
```

Esta última versión es apropiada para árboles, pero para grafos generales debe utilizarse un arreglo de visitados.

Aplicaciones:

DFS es una de las herramientas fundamentales para resolver problemas de grafos. Entre sus aplicaciones más comunes se encuentran:

- Recorrer todos los vértices alcanzables desde un origen.
- Encontrar componentes conexas.

- Detectar ciclos.
- Construir árboles DFS.
- Encontrar puntos de articulación y puentes.
- Realizar ordenamiento topológico.
- Resolver problemas de conectividad.
- Recorrer árboles y estructuras jerárquicas.

Complejidad:

Utilizando una lista de adyacencia, cada vértice se procesa una sola vez y cada arista se revisa una cantidad constante de veces. Por lo tanto:

$$O(n + m)$$

La pila y el arreglo de visitados requieren $O(n)$ memoria adicional.

Representación	Tiempo	Memoria adicional
Lista de adyacencia	$O(n + m)$	$O(n)$
Matriz de adyacencia	$O(n^2)$	$O(n)$

Table 13: Complejidad de DFS según la representación del grafo.

Nota: DFS y BFS tienen la misma complejidad asintótica $O(n + m)$ utilizando una lista de adyacencia, pero exploran el grafo de manera diferente. BFS procesa por niveles mediante una cola, mientras que DFS profundiza utilizando una pila.

Regla práctica: utiliza DFS cuando el problema dependa de explorar profundamente una estructura, recorrer componentes, detectar ciclos o realizar algún procesamiento basado en el orden de entrada y salida de los vértices. Si el problema requiere distancias mínimas en un grafo no ponderado, normalmente BFS es la opción adecuada.

6.6 DFS para Conteo de Caminos

Idea: en un DAG (grafo dirigido acíclico), contar cuántos caminos distintos existen entre un nodo u y un nodo destino fijo es un problema de **DFS + memoización**: el número de caminos desde u hasta el destino es la suma de los caminos desde cada vecino de u hasta el destino, con caso base "ya llegué". Como varios caminos pueden pasar por el mismo nodo intermedio, sin memoización recalculas el mismo subárbol muchas veces —memoizar es lo que baja esto de exponencial a lineal.

¿Por qué usarlo? Es el mismo patrón de "DFS + memo" que ya usas en DP sobre árboles/DAGs, aplicado específicamente a contar (no a minimizar/maximizar). Reconocer que un problema pide "número de formas de llegar de A a B" en un grafo dirigido sin ciclos es la señal para aplicar esto en vez de enumerar caminos explícitamente.

Casos de uso:

- Contar caminos entre dos nodos fijos de un DAG.
- Contar, para cada nodo, cuántos caminos empiezan en él y terminan en algún nodo "hoja" (sin salidas) — mismo memo, sin destino fijo, sumando 1 en cada hoja.

- Variante con longitud exacta k : contar caminos de u a v usando **exactamente** k aristas, memoizando sobre el par (*nodo, pasos_restantes*) en vez de solo *nodo*.
- Contar caminos módulo algún M cuando el número de caminos crece demasiado (usa `long long` y aplica $\%M$ en cada suma).

Complejidad: $O(V + E)$ con memoización, ya que cada nodo se procesa una sola vez y cada arista se recorre una sola vez. Sin memo, en el peor caso es exponencial (puede haber $O(2^V)$ caminos).

```

1 struct ConteoCaminos {
2     vector<vector<int>> adj;
3     vector<long long> memo;
4     int destino;
5
6     // CREATE: guarda el grafo (debe ser DAG) y el nodo destino;
7     // memo[u] = -1 significa "no calculado todavía".
8     ConteoCaminos(vector<vector<int>>& adj_, int destino_) {
9         adj = adj_;
10        destino = destino_;
11        memo.assign(adj.size(), -1);
12    } // O(1)
13
14    // READ: numero de caminos distintos de u a destino.
15    long long contar(int u) {
16        if (u == destino) return 1;
17        if (memo[u] != -1) return memo[u];
18
19        long long total = 0;
20        for (int v : adj[u])
21            total += contar(v);
22
23        return memo[u] = total;
24    } // O(1) amortizado
25 }; // O(V+E) la primera vez
26

```

Ejemplo de funcionamiento:

```

1 // DAG:      0 -> 1 -> 3
2 //          0 -> 2 -> 3
3 //          1 -> 2
4
5 vector<vector<int>> adj(4);
6 adj[0] = {1, 2};
7 adj[1] = {2, 3};
8 adj[2] = {3};
9 adj[3] = {};
10
11 ConteoCaminos cc(adj, 3);
12
13 cout << cc.contar(0);
14 // 3 (0-1-3, 0-1-2-3, 0-2-3)
15
16 cout << cc.contar(1);
17 // 2 (1-3, 1-2-3)

```

Nota: si el grafo tiene ciclos, esta memoización no funciona directamente (la recursión no termina) — primero hay que condensar las componentes fuertemente conexas (SCC) en un DAG, y contar sobre ese DAG condensado.

6.7 DFS + Cerramiento Transitivo

Idea: el cerramiento transitivo de un grafo dirigido responde, para cualquier par (u, v) , la pregunta "¿existe algún camino de u a v ?" en $O(1)$ por query, a costa de precomputar —para cada nodo— el conjunto completo de nodos que alcanza. La forma más directa es correr un DFS desde **cada** nodo y marcar todo lo que visita.

¿Por qué usarlo? Cuando el grafo es pequeño/mediano y vas a hacer **muchas** consultas de alcanzabilidad, precomputar todo de una vez sale más barato que un DFS/BFS por consulta. Es el mismo principio de "precomputo una vez, respondo $O(1)$ " que en RMQ, aplicado a alcanzabilidad en vez de a mínimos.

Casos de uso:

- Responder Q queries de "¿ u puede llegar a v ?" cuando Q es grande y V es manejable ($V \leq \sim 2000$ – 5000 según el límite de tiempo).
- Precomputar dependencias transitivas (si a depende de b y b depende de c , entonces a depende de c).
- Como paso intermedio antes de comprimir un DAG por SCC: primero condensas ciclos, y sobre el DAG resultante calculas el cerramiento transitivo.

Complejidad: $O(V \cdot (V + E))$ para construir (un DFS completo por cada nodo), $O(1)$ por query. Con `std::bitset` en vez de `vector<bool>`, y procesando los nodos en orden topológico inverso (`reach[u] |= reach[v]` para cada vecino v), el build baja a $O(V \cdot (V + E)/64)$ gracias al paralelismo a nivel de bits — útil si V es más grande.

```

1 struct CerramientoTransitivo {
2     int n;
3     vector<vector<int>> adj;
4     vector<vector<bool>> alcanzable; // alcanzable[u][v] = true
5     // si u puede llegar a v
6
7     // CREATE: corre un DFS completo desde cada nodo. O(V*(V+E)).
8     CerramientoTransitivo(vector<vector<int>>& adj_, int n_) {
9         adj = adj_;
10        n = n_;
11        alcanzable.assign(n, vector<bool>(n, false));
12
13        for (int origen = 0; origen < n; origen++)
14            dfs(origen, origen);
15    }
16
17    void dfs(int origen, int u) {
18        for (int v : adj[u]) {
19            if (!alcanzable[origen][v]) {
20                alcanzable[origen][v] = true;
21                dfs(origen, v);
22            }
23        }
24    }
25
26    // READ: es u puede llegar a v.
27    bool query(int u, int v) {
28        return alcanzable[u][v];
29    } // O(1)
30 };

```

Ejemplo de funcionamiento:

```

1 // grafo:      0 -> 1 -> 2
2 //            \
3 //             -> 3
4 //            4 (aislado)
5
6 vector<vector<int>> adj(5);
7 adj[0] = {1};
8 adj[1] = {2, 3};
9 adj[2] = {};
10 adj[3] = {};
11 adj[4] = {};
12
13 CerramientoTransitivo ct(adj, 5);
14
15 cout << ct.query(0, 3);
16 // true (0 -> 1 -> 3)
17
18 cout << ct.query(2, 0);
19 // false (no hay camino de 2 a 0)
20
21 cout << ct.query(4, 0);
22 // false (4 esta aislado)

```

Regla práctica: DFS "para contar" y DFS "para cerramiento transitivo" comparten la misma mecánica de recorrido, pero acumulan cosas distintas en cada llamada: uno suma caminos hacia un destino fijo (memoizando por nodo), el otro marca alcanzabilidad hacia *todos* los

destinos posibles (memoizando por par de nodos, implícitamente, al no revisitar lo ya marcado). Si el grafo tiene ciclos, ambos requieren primero pasar por condensación de SCC antes de aplicarse tal cual.

6.8 Flood Fill

Idea: Flood Fill es "explorar todas las celdas conectadas a una celda inicial que cumplen una condición" (mismo color, mismo carácter, sin obstáculo, etc.), marcándolas como visitadas o recoloriéndolas. Es exactamente un DFS/BFS pero sobre una grilla en vez de un grafo con listas de adyacencia explícitas: los "vecinos" de una celda (r, c) son sus hasta 4 (u 8) celdas adyacentes.

¿Por qué usarlo? Cualquier problema de grilla que hable de "regiones", "islas", "áreas conectadas" o "pintar/rellenar" es Flood Fill. La clave práctica es preferir la versión **iterativa** (con `queue` o `stack` explícito) sobre la recursiva: en grillas grandes ($R \times C$ del orden de 10^5 – 10^6 celdas), un DFS recursivo puede hacer *stack overflow*.

Casos de uso:

- Contar componentes conexas de una grilla ("número de islas"): un Flood Fill por cada celda no visitada que cumpla la condición, contando cuántas veces arrancas uno nuevo.
- Calcular el área (tamaño) de cada región conectada.
- "Paint bucket": recoloriando toda una región conectada a un color nuevo.
- Determinar si dos celdas están en la misma región conectada.

Complejidad: $O(R \cdot C)$, ya que cada celda se visita (y se encola) como máximo una vez.

```
1 struct FloodFill {
2     vector<vector<char>> grid;
3     vector<vector<bool>> visitado;
4     int R, C;
5
6     int dr[4] = {-1, 1, 0, 0};
7     int dc[4] = {0, 0, -1, 1};
8
9     // CREATE: guarda la grilla (R x C).
10    FloodFill(vector<vector<char>>& grid_) {
11        grid = grid_;
12        R = grid.size();
13        C = grid[0].size();
14        visitado.assign(R, vector<bool>(C, false));
15    } // O(1)
16
17    // READ: marca (BFS iterativo) toda la region conectada a
18    // (sr, sc) que comparte el mismo caracter, y devuelve su
19    // area.
20    // Evita recursion para no hacer stack overflow en grillas
21    // grandes.
22    int llenar(int sr, int sc) {
23        if (visitado[sr][sc]) return 0;
24
25        char objetivo = grid[sr][sc];
26        queue<pair<int,int>> q;
27        q.push({sr, sc});
28        visitado[sr][sc] = true;
29        int area = 0;
30
31        while (!q.empty()) {
32            auto [r, c] = q.front(); q.pop();
33            area++;
34
35            for (int d = 0; d < 4; d++) {
36                int nr = r + dr[d], nc = c + dc[d];
37
38                if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
39                if (visitado[nr][nc]) continue;
40                if (grid[nr][nc] != objetivo) continue;
```

```
39         visitado[nr][nc] = true;
40         q.push({nr, nc});
41     }
42 }
43
44 return area;
45 } // O(área de la
46     region)
47 };
```

Ejemplo de funcionamiento:

```
1 // grid:
2 // L L W W
3 // L L W W
4 // W W L L
5
6 vector<vector<char>> grid = {
7     {'L','L','W','W'},
8     {'L','L','W','W'},
9     {'W','W','L','L'}
10 };
11
12 FloodFill ff(grid);
13
14 int islas = 0, areaMax = 0;
15 for (int r = 0; r < 3; r++)
16     for (int c = 0; c < 4; c++)
17         if (grid[r][c] == 'L' && !ff.visitado[r][c]) {
18             int area = ff.llenar(r, c);
19             islas++;
20             areaMax = max(areaMax, area);
21         }
22
23 cout << islas; // 2
24 cout << areaMax; // 4 (el bloque de 2x2 de arriba-izquierda)
```

6.9 Laberinto con BFS

Idea: en una grilla donde te mueves de una celda a una celda adyacente (arriba/abajo/izq/der) con **costo uniforme** (cada paso cuesta 1), la ruta más corta desde un origen se calcula con BFS: exploras la grilla "por capas" de distancia creciente, y la primera vez que llegas a una celda, esa es su distancia mínima. Es el mismo BFS de grafos de siempre, con la grilla actuando como grafo implícito.

¿Por qué usarlo? Es la herramienta por defecto para "camino más corto en grilla sin pesos / con obstáculos". Si el problema menciona pesos distintos por celda (terreno con costo), esto deja de ser BFS puro y pasa a ser Dijkstra o 0-1 BFS (deque) — pero mientras cada paso cueste lo mismo, BFS es suficiente y más simple.

Casos de uso:

- Número mínimo de pasos de un punto A a un punto B en un laberinto con paredes.
- Reconstruir el camino más corto explícito (no solo la distancia), guardando de dónde vino cada celda.
- BFS multi-fuente: varias celdas iniciales a la vez (p.ej. "tiempo mínimo para que el fuego llegue a cada celda"), encolando todas las fuentes con distancia 0 antes de empezar.
- Verificar alcanzabilidad ("¿se puede llegar de A a B ?").

Complejidad: $O(R \cdot C)$, ya que cada celda se encola y desencola como máximo una vez.

```
1 struct LaberintoBFS {
2     vector<vector<char>> grid;
3     vector<vector<int>> dist;
4     vector<vector<pair<int,int>>> padre;
```

```

5   int R, C;
6
7   int dr[4] = {-1, 1, 0, 0};
8   int dc[4] = {0, 0, -1, 1};
9
10  // CREATE: guarda la grilla. '#' = pared, cualquier otro
11  //      caracter
12  // = celda libre.
13  LaberintoBFS(vector<vector<char>>& grid_) {
14      grid = grid_;
15      R = grid.size();
16      C = grid[0].size();
17  } // O(1)
18
19  // READ: corre BFS desde (sr, sc) y llena dist / padre para
20  // todas las celdas alcanzables. dist queda en -1 donde no
21  // llega.
22  void bfs(int sr, int sc) {
23      dist.assign(R, vector<int>(C, -1));
24      padre.assign(R, vector<pair<int,int>>(C, {-1, -1}));
25
26      queue<pair<int,int>> q;
27      q.push({sr, sc});
28      dist[sr][sc] = 0;
29
30      while (!q.empty()) {
31          auto [r, c] = q.front(); q.pop();
32
33          for (int d = 0; d < 4; d++) {
34              int nr = r + dr[d], nc = c + dc[d];
35
36              if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
37              if (grid[nr][nc] == '#') continue;
38              if (dist[nr][nc] != -1) continue;
39
40              dist[nr][nc] = dist[r][c] + 1;
41              padre[nr][nc] = {r, c};
42              q.push({nr, nc});
43          }
44      } // O(R*C)
45
46  // READ: reconstruye el camino desde el origen del ultimo bfs
47  //      ()
48  // hasta (er, ec), como lista de celdas en orden. Vacío si no
49  // hay camino.
50  vector<pair<int,int>> reconstruirCamino(int er, int ec) {
51      if (dist[er][ec] == -1) return {};
52
53      vector<pair<int,int>> camino;
54      pair<int,int> actual = {er, ec};
55
56      while (actual.first != -1) {
57          camino.push_back(actual);
58          actual = padre[actual.first][actual.second];
59      }
60
61      reverse(camino.begin(), camino.end());
62      return camino; // O(longitud del camino)
63  };

```

Ejemplo de funcionamiento:

```

1 // grid:
2 // . . . #
3 // # # . #
4 // . . . .
5
6 vector<vector<char>> grid = {
7     {'.', '.', '.', '#'},
8     {'#', '#', '.', '#'},
9     {'.', '.', '.', '.'}
10 };
11
12 LaberintoBFS lb(grid);
13 lb.bfs(0, 0);
14
15 cout << lb.dist[2][3];
16 // 5 (0,0)->(0,1)->(0,2)->(1,2)->(2,2)->(2,3)
17
18 auto camino = lb.reconstruirCamino(2, 3);
19 // [(0,0), (0,1), (0,2), (1,2), (2,2), (2,3)]

```

Regla práctica: si un problema de grilla pide "¿son conectados?" o "área de una región", es Flood Fill (DFS o BFS, no importa el orden de visita). Si pide "distancia mínima" o "menor número de pasos", es BFS por capas (el orden de visita sí importa: BFS garantiza que la primera

vez que tocas una celda es con su distancia mínima; DFS no lo garantiza).

6.10 Tour de Caballo de Ajedrez

Idea: el Tour de Caballo pide encontrar una secuencia de movimientos de caballo que visite **cada casilla del tablero exactamente una vez**. No es un problema de "camino más corto" (no hay pesos ni se compara contra alternativas), sino de **existencia**: es backtracking puro — DFS que prueba un movimiento, sigue explorando, y si se atasca (llega a una casilla sin salidas válidas) deshace ("backtrack") y prueba otro movimiento.

¿Por qué usarlo? A diferencia de Flood Fill o BFS en laberintos, aquí **no basta con visitar** las casillas alcanzables: tienen que visitarse **todas, sin repetir**, en un orden válido. Eso descarta BFS/DFS simple y obliga a backtracking con una condición de poda, porque la fuerza bruta pura (8^n combinaciones de movimientos posibles) es intratable incluso para tableros chicos.

Casos de uso:

- Encontrar (o verificar la existencia de) un tour de caballo completo en un tablero $n \times n$.
- Tour **cerrado** (el último movimiento vuelve a atacar la casilla inicial): misma idea, chequeo extra al final.
- Cualquier problema de "recorrer todas las celdas exactamente una vez respetando un patrón de movimiento" (no exclusivo del caballo): el patrón general es backtracking + poda de Warnsdorff.

Complejidad: exponencial en el peor caso sin poda ($O(8^{n^2})$ aproximado). Con la **heurística de Warnsdorff** —en cada paso, moverse primero a la casilla accesible con **menos** salidas futuras disponibles— en la práctica encuentra un tour en tiempo casi lineal para tableros razonables ($n \leq 100$), aunque no hay garantía teórica formal de que siempre funcione.

```

1 struct TourCaballo {
2     int n;
3     vector<vector<int>> tablero; // tablero[r][c] = orden de
4     //      visita
5     // (-1 si no visitada)
6     int dr[8] = {-2,-2,-1,-1, 1, 1, 2, 2};
7     int dc[8] = {-1, 1,-2, 2,-2, 2,-1, 1};
8
9     // CREATE: tablero n x n vacío.
10    TourCaballo(int n_) {
11        n = n_;
12        tablero.assign(n, vector<int>(n, -1)); // O(n^2)
13    }
14
15    bool dentro(int r, int c) {
16        return r >= 0 && r < n && c >= 0 && c < n;
17    }
18
19    // cuenta a cuantas casillas libres puede saltar (r,c) -
20    //      usado
21    // por Warnsdorff para ordenar los candidatos.
22    int gradoSalida(int r, int c) {
23        int cont = 0;
24        for (int d = 0; d < 8; d++) {
25            int nr = r + dr[d], nc = c + dc[d];
26            if (dentro(nr, nc) && tablero[nr][nc] == -1) cont++;
27        }
28        return cont;
29    }
30
31    // READ: intenta construir un tour completo empezando en
32    //      (sr, sc). Devuelve true si lo logra (tablero queda lleno).
33    bool resolver(int sr, int sc) {
34        tablero[sr][sc] = 0;
35        return backtrack(sr, sc, 1);
36    }
37 };

```



```

34 } // O(casi lineal)
35     con
36     // Warnsdorff en la
37     // practica
38
39 bool backtrack(int r, int c, int movimiento) {
40     if (movimiento == n * n) return true;
41
42     // candidatos ordenados por Warnsdorff: menor grado de
43     // salida primero
44     vector<tuple<int,int,int>> candidatos;
45     for (int d = 0; d < 8; d++) {
46         int nr = r + dr[d], nc = c + dc[d];
47         if (dentro(nr, nc) && tablero[nr][nc] == -1)
48             candidatos.push_back({gradoSalida(nr, nc), nr, nc});
49     }
50     sort(candidatos.begin(), candidatos.end());
51
52     for (auto& [grado, nr, nc] : candidatos) {
53         tablero[nr][nc] = movimiento;
54
55         if (backtrack(nr, nc, movimiento + 1)) return true;
56
57         tablero[nr][nc] = -1; // deshacer (backtrack real)
58     }
59     return false;
60 }
61 };

```

Ejemplo de funcionamiento:

```

1 TourCaballo tc(5);
2
3 bool encontrado = tc.resolver(0, 0);
4
5 cout << encontrado;
6 // true
7
8 // tc.tablero[r][c] tiene el orden (0..24) en que se visito
9 // cada casilla, por ejemplo:
10 // 0 3 16 9 22
11 // 13 8 21 4 17
12 // 2 1 12 23 10
13 // 7 14 19 18 5
14 // 20 ... etc.

```

Nota: sin la poda de Warnsdorff, el backtrack puro también es correcto pero mucho más lento (puede no terminar en tiempo útil para $n > 6$ u 8). Warnsdorff no cambia la corrección del algoritmo, solo el **orden** en que se prueban los candidatos — sigue siendo backtracking con deshacer explícito.

6.11 Mínimo Número de Movimientos de Caballo

Idea: a diferencia del Tour de Caballo, aquí no importa visitar todo el tablero — solo se pide la **distancia mínima** (en número de saltos) entre una casilla origen y una casilla destino. Eso es "camino más corto con costo uniforme" sobre un grafo implícito donde cada casilla es un nodo y sus hasta 8 movimientos de caballo son las aristas: exactamente el mismo patrón que Laberinto con BFS, cambiando el arreglo de movimientos de 4/8 direcciones ortogonales por los 8 saltos en "L" del caballo.

¿Por qué usarlo? Es tentador intentar resolver esto con fórmulas o backtracking (como el Tour), pero al ser "distancia mínima" y no "visitar todo", BFS es tanto más simple como garantizado óptimo — la primera vez que BFS alcanza la casilla destino, esa es la respuesta.

Casos de uso:

- Mínimo número de movimientos de caballo entre dos casillas de un tablero $n \times n$ (o infinito, acotando la búsqueda).
- Mismo problema con casillas bloqueadas/prohibidas: se salta la casilla en el BFS igual que una pared en un laberinto.

- Generalización a "cuál es la distancia mínima con un patrón de movimiento arbitrario" (no solo caballo): el patrón se sigue resolviendo con BFS cambiando dr/dc.

Complejidad: $O(n^2)$ para un tablero $n \times n$, ya que cada casilla se encola como máximo una vez y desde cada una se prueban 8 movimientos constantes.

```

1 struct MinMovimientosCaballo {
2     int n;
3     vector<vector<int>> dist;
4
5     int dr[8] = {-2,-2,-1,-1, 1, 1, 2, 2};
6     int dc[8] = {-1, 1,-2, 2,-2, 2,-1, 1};
7
8     // CREATE: tablero n x n.
9     MinMovimientosCaballo(int n_) {
10         n = n_;
11         // O(1)
12
13         bool dentro(int r, int c) {
14             return r >= 0 && r < n && c >= 0 && c < n;
15         }
16
17         // READ: BFS desde (sr, sc); dist[r][c] = minimo de
18         // movimientos
19         // de caballo para llegar a (r, c), -1 si no aplica (no
20         // debería
21         // pasar en un tablero cerrado y conexo).
22         int minMovimientos(int sr, int sc, int er, int ec) {
23             dist.assign(n, vector<int>(n, -1));
24
25             queue<pair<int,int>> q;
26             q.push({sr, sc});
27             dist[sr][sc] = 0;
28
29             while (!q.empty()) {
30                 auto [r, c] = q.front(); q.pop();
31
32                 if (r == er && c == ec) return dist[r][c];
33
34                 for (int d = 0; d < 8; d++) {
35                     int nr = r + dr[d], nc = c + dc[d];
36
37                     if (!dentro(nr, nc)) continue;
38                     if (dist[nr][nc] != -1) continue;
39
40                     dist[nr][nc] = dist[r][c] + 1;
41                     q.push({nr, nc});
42                 }
43
44                 return -1; // no alcanzable (no debería pasar sin
45                 // bloqueos)
46             }
47         }
48     };
49 };

```

Ejemplo de funcionamiento:

```

1 MinMovimientosCaballo mc(8);
2
3 cout << mc.minMovimientos(0, 0, 1, 2);
4 // 1 (un solo salto en L)
5
6 cout << mc.minMovimientos(0, 0, 7, 7);
7 // 6 (esquina a esquina en un tablero 8x8)
8
9 cout << mc.minMovimientos(0, 0, 0, 1);
10 // 3 (casilla adyacente, contraintuitivamente lejos para un
    caballo)

```

Regla práctica: misma distinción que Flood Fill vs. Laberinto con BFS, aplicada al caballo: si el problema pide "**visitar todo**" sin repetir, es backtracking (Tour de Caballo, con Warnsdorff como poda). Si pide "**distancia mínima entre dos puntos**", es BFS (Mínimo Número de Movimientos), sin importar qué tan raro sea el patrón de movimiento — el patrón solo define el arreglo de vecinos.

6.12 Serpientes y Escaleras

Idea: el tablero de Serpientes y Escaleras se modela como un grafo dirigido: cada casilla $1..n$ es un nodo, y

desde cada casilla i hay hasta 6 aristas (una por cada valor del dado, $i + 1$ hasta $i + 6$), **redirigidas** si la casilla de llegada tiene una serpiente o escalera. Pedir el "mínimo número de tiradas de dado para llegar de la casilla 1 a la n " es, otra vez, camino más corto con costo uniforme (cada tirada cuenta como 1 paso) — el mismo BFS de Laberinto con BFS y Mínimo Número de Movimientos de Caballo, cambiando qué cuenta como "vecino" de una casilla.

¿Por qué usarlo? La parte no trivial no es el BFS en sí — es idéntico al de siempre — sino **modelar** serpientes y escaleras correctamente: no son casillas nuevas, son **atajos/retrocesos automáticos**. Si aterrizas en la casilla de la base de una escalera o la cabeza de una serpiente, tu posición real cambia **sin gastar otra tirada**. Eso se resuelve precomputando, para cada casilla, a dónde "realmente" te manda (`destinoReal[i]`), y haciendo BFS sobre ese arreglo en vez de sobre los números crudos del dado.

Casos de uso:

- Mínimo número de tiradas de dado para llegar de la casilla 1 (o cualquier inicio) a la casilla n .
- Variante donde el dado no es 1–6 sino un conjunto arbitrario de valores posibles: solo cambia el rango del `for` al generar vecinos.
- Cualquier tablero lineal con "teletransportes" fijos (serpiente = te manda hacia atrás, escalera = te manda hacia adelante) donde se pide distancia mínima en pasos.

Complejidad: $O(n)$, ya que el grafo tiene n nodos y a lo más $6n$ aristas — cada casilla se encola una sola vez.

```

1 struct SerpientesYEscaleras {
2     int n;
3     vector<int> destinoReal; // destinoReal[i] = a donde
4                             // realmente el jugador si cae en i
5                             // (i mismo si no hay serpiente/escalera)
6
7     // CREATE: tablero de n casillas (1-indexado). saltos[i] = j
8     // significa "serpiente o escalera de i a j"; las casillas
9     // sin
10    // entrada en el mapa no tienen salto.
11    SerpientesYEscaleras(int n_, map<int,int>& saltos) {
12        n = n_;
13        destinoReal.assign(n + 1, 0);
14
15        for (int i = 1; i <= n; i++)
16            destinoReal[i] = saltos.count(i) ? saltos[i] : i;
17    } // O(n)
18
19    // READ: minimo numero de tiradas de dado (1-6) para llegar
20    // de
21    // origen a n. -1 si es imposible (no debería pasar en un
22    // tablero estandar).
23    int minTiradas(int origen) {
24        vector<int> dist(n + 1, -1);
25        queue<int> q;
26
27        q.push(origen);
28        dist[origen] = 0;
29
30        while (!q.empty()) {
31            int actual = q.front(); q.pop();
32
33            if (actual == n) return dist[actual];
34
35            for (int dado = 1; dado <= 6; dado++) {
36                int siguiente = actual + dado;
37                if (siguiente > n) continue;
38
39                siguiente = destinoReal[siguiente]; // aplica
40                // serpiente/escalera

```

```

39         if (dist[siguiente] != -1) continue;
40
41         dist[siguiente] = dist[actual] + 1;
42         q.push(siguiente);
43     }
44 }
45
46 return -1;
47 } // O(n)
48
49 };

```

Ejemplo de funcionamiento:

```

1 // tablero de 10 casillas:
2 // escalera de 2 a 8
3 // serpiente de 9 a 3
4
5 map<int,int> saltos = {{2, 8}, {9, 3}};
6 SerpientesYEscaleras se(10, saltos);
7
8 cout << se.minTiradas(1);
9 // 2
10 // tirada 1: 1 -> 2 -> (escalera) -> 8
11 // tirada 2: 8 -> 10

```

Regla práctica: cualquier tablero lineal donde te mueves por "reglas de dado" (o cualquier conjunto de deltas fijos) y hay teletransportes automáticos al aterrizar en ciertas casillas, se reduce a BFS de toda la vida sobre un grafo de n nodos — **la única parte específica del problema es construir bien el arreglo `destinoReal`** antes de correr el BFS; el algoritmo de búsqueda no cambia respecto a los casos anteriores.

7 Conectividad

7.1 Componentes Conexas

Idea: en un grafo **no dirigido**, una componente conexa es un grupo maximal de nodos donde cualquiera puede llegar a cualquiera. Encontrarlas todas es simplemente: recorrer los nodos en orden, y cada vez que encuentres uno **no visitado**, lanzar un DFS/BFS desde ahí para marcar todo su grupo con un mismo identificador — ese DFS/BFS nunca se sale del grupo porque, por definición, no hay aristas hacia afuera de una componente.

¿Por qué usarlo? Es el primer paso obligatorio de casi cualquier problema de grafos no dirigidos: antes de responder "¿ u y v están conectados?" o de aplicar un algoritmo que asume un grafo conexo, necesitas saber cuántas componentes hay y a cuál pertenece cada nodo. La alternativa es **Union-Find**: si solo necesitas responder "¿misma componente?" con aristas que van llegando una a una (sin necesariamente construir listas de adyacencia), Union-Find es más directo; si ya tienes el grafo completo y quieres además reconstruir/recorrer cada componente, DFS/BFS es más natural.

Casos de uso:

- Contar cuántas componentes conexas tiene un grafo (o una grilla vista como grafo — mismo patrón que Flood Fill, pero sobre listas de adyacencia en vez de vecinos de grilla).
- Etiquetar cada nodo con el id de su componente, para responder "¿misma componente?" en $O(1)$ tras el precómputo.
- Encontrar el tamaño de la componente más grande.
- Verificar si el grafo completo es conexo (una sola componente).

Complejidad: $O(V + E)$, ya que es un DFS/BFS normal que simplemente se relanza desde cada nodo no visitado — en total, cada nodo y cada arista se procesan una sola vez sin importar cuántas componentes haya.

```

1 struct ComponentesConexas {
2     vector<vector<int>> adj;
3     vector<int> comp; // comp[u] = id de la componente de u,
4         -1
5     // si no se ha calculado
6     int numComponentes;
7
8     // CREATE: guarda el grafo y calcula todas las componentes de
9     // una vez, recorriendo cada nodo no visitado.
10    ComponentesConexas(vector<vector<int>>& adj_, int n) {
11        adj = adj_;
12        comp.assign(n, -1);
13        numComponentes = 0;
14
15        for (int u = 0; u < n; u++) {
16            if (comp[u] == -1) {
17                dfs(u, numComponentes);
18                numComponentes++;
19            }
20        }
21    } // O(V + E)
22
23    void dfs(int u, int id) {
24        comp[u] = id;
25        for (int v : adj[u])
26            if (comp[v] == -1)
27                dfs(v, id);
28    }
29
30    // READ: u y v estan en la misma componente?
31    bool mismaComponente(int u, int v) {
32        return comp[u] == comp[v];
33    } // O(1)
34 };

```

Ejemplo de funcionamiento:

```

1 // grafo: 0 - 1  2 - 3  4
2 //         (dos componentes de tamaño 2, una de tamaño 1)
3
4 vector<vector<int>> adj(5);
5 adj[0] = {1}; adj[1] = {0};
6 adj[2] = {3}; adj[3] = {2};
7 adj[4] = {};
8
9 ComponentesConexas cc(adj, 5);
10
11 cout << cc.numComponentes;
12 // 3
13
14 cout << cc.mismaComponente(0, 1);
15 // true
16
17 cout << cc.mismaComponente(1, 2);
18 // false

```

7.2 Detección de Ciclos

Idea: detectar si un grafo tiene un ciclo se hace con DFS, pero la regla para reconocer un ciclo **cambia según el grafo sea dirigido o no**: en un grafo no dirigido, cualquier arista hacia un nodo ya visitado que **no sea tu padre inmediato** es un ciclo; en un grafo dirigido, una arista hacia un nodo ya visitado **solo** es ciclo si ese nodo sigue en la pila de recursión actual (todavía no terminaste de procesarlo) — si ya terminaste con él, es una arista válida hacia otra rama, no un ciclo.

¿Por qué usarlo? Confundir estas dos reglas es el error más común al detectar ciclos: aplicar la regla de "no dirigido" (solo evitar al padre) a un grafo dirigido da falsos positivos, porque en dirigido puedes visitar el mismo nodo desde dos ramas distintas sin que eso sea un ciclo.

Casos de uso:

- Verificar si un grafo dirigido es un DAG (sin ciclos) antes de aplicar orden topológico, DP sobre DAG, etc.
- Detectar dependencias circulares (tareas que dependen unas de otras en círculo, imposibles de ordenar).
- En grafos no dirigidos: verificar si un grafo es un árbol ($V - 1$ aristas, conexo, sin ciclos) o encontrar el primer ciclo para removerlo.

Complejidad: $O(V + E)$ en ambos casos — un DFS normal con una condición extra por arista, sin repetir trabajo.

En grafos no dirigidos (DFS con padre)

```

1 struct CicloNoDirigido {
2     vector<vector<int>> adj;
3     vector<bool> visitado;
4
5     // CREATE: guarda el grafo.
6     CicloNoDirigido(vector<vector<int>>& adj_) {
7         adj = adj_;
8         visitado.assign(adj.size(), false);
9     } // O(1)
10
11    // READ: existe algun ciclo en todo el grafo (revisa todas
12    // las componentes).
13    bool tieneCiclo() {
14        for (int u = 0; u < (int)adj.size(); u++)
15            if (!visitado[u] && dfs(u, -1))
16                return true;
17        return false;
18    } // O(V + E)
19
20    bool dfs(int u, int padre) {
21        visitado[u] = true;
22
23        for (int v : adj[u]) {
24            if (!visitado[v]) {
25                if (dfs(v, u)) return true;
26            } else if (v != padre) {
27                return true; // arista hacia un visitado que no es
28                // el padre -> ciclo
29            }
30        }
31
32        return false;
33    }
34 };

```

Ejemplo de funcionamiento:

```

1 // grafo: 0 - 1 - 2 - 0 (triangulo, tiene ciclo)
2 //           3 - 4      (arbol, sin ciclo)
3
4 vector<vector<int>> adj(5);
5 adj[0] = {1, 2}; adj[1] = {0, 2}; adj[2] = {0, 1};
6 adj[3] = {4}; adj[4] = {3};
7
8 CicloNoDirigido cnd(adj);
9
10 cout << cnd.tieneCiclo();
11 // true (por el triangulo 0-1-2)

```

En grafos dirigidos (DFS con estados / colores)

Idea: cada nodo tiene 3 estados durante el DFS: 0 = no visitado, 1 = en la pila de recursión actual (gris), 2 = totalmente procesado (negro). Una arista hacia un nodo **gris** es un back edge real (ciclo); una arista hacia un nodo **negro** es válida (cross/forward edge, no ciclo).

```

1 struct CicloDirigido {
2     vector<vector<int>> adj;
3     vector<int> estado; // 0 = blanco, 1 = gris, 2 = negro
4
5     // CREATE: guarda el grafo dirigido.

```

```

6  CicloDirigido(vector<vector<int>>& adj_) {
7      adj = adj_;
8      estado.assign(adj.size(), 0);
9  } // O(1)
10
11  // READ: existe algun ciclo en todo el grafo.
12  bool tieneCiclo() {
13      for (int u = 0; u < (int)adj.size(); u++)
14          if (estado[u] == 0 && dfs(u))
15              return true;
16      return false;
17  } // O(V + E)
18
19  bool dfs(int u) {
20      estado[u] = 1; // entra a la pila de recursion (gris)
21
22      for (int v : adj[u]) {
23          if (estado[v] == 1) return true; // back edge ->
24          // ciclo
25          if (estado[v] == 0 && dfs(v)) return true;
26      }
27
28      estado[u] = 2; // termino con u (negro)
29      return false;
30  }
31 };

```

Ejemplo de funcionamiento:

```

1  // grafo dirigido: 0 -> 1 -> 2 -> 0 (ciclo)
2  //                  3 -> 1      (sin ciclo: 3->1 es
3  //                  valida,
4  //                  1 ya esta siendo
5  //                  procesado
6  //                  solo si viene de la
7  //                  MISMA
8  //                  rama de recursion)
9
10 vector<vector<int>> adj(4);
11 adj[0] = {1};
12 adj[1] = {2};
13 adj[2] = {0};
14 adj[3] = {1};
15
16 CicloDirigido cd(adj);
17
18 cout << cd.tieneCiclo();
19 // true (0 -> 1 -> 2 -> 0)

```

Regla práctica: para saber cuál versión aplicar, primero pregúntate si el grafo es dirigido. Si no lo es, basta con recordar "de dónde vine" (el padre) y listo. Si es dirigido, "de dónde vine" no alcanza — necesitas saber si el nodo destino sigue **activo en la recursión actual** (gris) o ya quedó cerrado (negro), porque en dirigido sí es válido llegar dos veces al mismo nodo desde ramas distintas sin que eso sea un ciclo.

7.3 Puentes

Idea: en un grafo no dirigido, un **punto** es una arista que, al quitarla, **desconecta** el grafo (aumenta el número de componentes conexas). Se encuentran con un solo DFS, llevando dos valores por nodo: **tin[u]** (el momento en que el DFS **descubre** a *u*, un contador que se incrementa en cada visita) y **low[u]** (el tin **más bajo** alcanzable desde *u* usando cualquier camino del árbol DFS más, como mucho, **una** arista de "retroceso" hacia un ancestro). La arista (*u, v*) del árbol DFS es puente si y solo si **low[v] > tin[u]**: eso significa que, desde el subárbol de *v*, **no hay ningún atajo** de vuelta hacia *u* o más arriba —quitar esa arista realmente aísla el subárbol de *v*.

¿Por qué usarlo? Es la aplicación clásica del algoritmo de Tarjan de low-link: en vez de, para cada arista, quitarla y volver a correr un DFS/BFS completo para ver si el grafo se desconecta ($O(E \cdot (V + E))$ en total), un solo DFS con low-link responde para **todas** las aristas a la vez en tiempo lineal.

Casos de uso:

- Encontrar todas las conexiones "críticas" de una red (si se cae ese enlace, la red se parte en dos).
- Como paso previo para construir el **árbol de componentes 2-arista-conexas** (contraer cada componente 2-arista-conexa a un nodo y dejar solo los puentes como aristas).
- Verificar si el grafo sigue siendo conexo tras remover cualquier arista dada (equivalente a "¿esa arista es puente?").

Complejidad: $O(V + E)$, un solo DFS con trabajo constante extra por arista.

```

1  struct Puentes {
2      vector<vector<pair<int,int>>> adj; // adj[u] = {(v,
3      // idArista)}
4      vector<int> tin, low;
5      vector<bool> visitado;
6      vector<pair<int,int>> puentes; // aristas (u,v) que son
7      // puente
8      int timer;
9
10     // CREATE: guarda el grafo (con id de arista para no reusar
11     // la
12     // misma arista en sentido contrario) y corre el DFS de
13     // Tarjan
14     // desde cada componente.
15     Puentes(vector<vector<pair<int,int>>& adj_, int n) {
16         adj = adj_;
17         tin.assign(n, -1);
18         low.assign(n, -1);
19         visitado.assign(n, false);
20         timer = 0;
21
22         for (int u = 0; u < n; u++)
23             if (!visitado[u])
24                 dfs(u, -1);
25     } // O(V + E)
26
27     void dfs(int u, int idAristaPadre) {
28         visitado[u] = true;
29         tin[u] = low[u] = timer++;
30
31         for (auto& [v, idArista] : adj[u]) {
32             if (idArista == idAristaPadre) continue; // no te
33             // regresas
34             // por la
35             // misma
36             // arista
37
38             if (visitado[v]) {
39                 low[u] = min(low[u], tin[v]); // back edge
40             } else {
41                 dfs(v, idArista);
42                 low[u] = min(low[u], low[v]);
43
44                 if (low[v] > tin[u])
45                     puentes.push_back({u, v});
46             }
47         }
48     }
49 };

```

Ejemplo de funcionamiento:

```

1  // grafo: 0 - 1 - 2 - 0 (triangulo)
2  //          \
3  //          3 - 4 (colgando del triangulo)
4
5  vector<vector<pair<int,int>>> adj(5);
6  auto addEdge = [&](int u, int v, int id) {
7      adj[u].push_back({v, id});
8      adj[v].push_back({u, id});
9  };
10 addEdge(0, 1, 0);
11 addEdge(1, 2, 1);
12 addEdge(2, 0, 2);
13 addEdge(2, 3, 3);
14 addEdge(3, 4, 4);
15
16 Puentes p(adj, 5);
17
18 for (auto& [u, v] : p.puentes)

```



```

19 cout << u << "-" << v << " ";
20 // 2-3 3-4
21 // (las aristas del triangulo NO son puente: hay un ciclo,
22 // siempre queda un camino alternativo)

```

7.4 Puntos de Articulación

Idea: un **punto de articulación** (o *cut vertex*) es un nodo que, al quitarlo (junto con sus aristas), **incrementa** el número de componentes conexas. Usa el mismo **tin/low** que Puentes, pero la condición es distinta y tiene **dos casos**: (1) si u es la **raíz** del árbol DFS, es articulación si tiene **2 o más hijos** en el árbol DFS (cada hijo es un subárbol que solo se conecta al resto *a través de* u); (2) si u **no** es la raíz, es articulación si tiene algún hijo v con $\text{low}[v] \geq \text{tin}[u]$ (el subárbol de v no tiene ningún atajo que lo saque **por encima** de u , así que sin u ese subárbol queda aislado).

¿Por qué usarlo? Es el "primo" de Puentes: mismo DFS, mismos **tin/low**, pero identificando nodos críticos en vez de aristas críticas. La diferencia clave con Puentes está en el operador de comparación ($\geq$ en vez de $>$) y en el caso especial de la raíz — con solo $>$ (la condición de puente) se pierde el caso donde v puede volver exactamente a u (eso no aísla la arista, pero sí deja a u como único punto de paso).

Casos de uso:

- Encontrar los nodos "críticos" de una red (si ese nodo falla, la red se parte).
- Como paso previo para construir componentes **biconexas** (bloques del grafo separados por puntos de articulación).
- Distinguir de Puentes: un grafo puede tener puntos de articulación sin tener ningún puente (ej. dos triángulos que comparten un solo vértice) — son propiedades independientes.

Complejidad: $O(V + E)$, mismo costo que Puentes: un DFS con trabajo constante extra por nodo/arista.

```

1 struct PuntosArticulacion {
2     vector<vector<int>> adj;
3     vector<int> tin, low;
4     vector<bool> visitado, esArticulacion;
5     int timer;
6
7     // CREATE: guarda el grafo y corre el DFS de Tarjan desde
8     // componente.
9     PuntosArticulacion(vector<vector<int>>& adj_, int n) {
10         adj = adj_;
11         tin.assign(n, -1);
12         low.assign(n, -1);
13         visitado.assign(n, false);
14         esArticulacion.assign(n, false);
15         timer = 0;
16
17         for (int u = 0; u < n; u++)
18             if (!visitado[u])
19                 dfs(u, -1);
20     }
21
22     void dfs(int u, int padre) {
23         visitado[u] = true;
24         tin[u] = low[u] = timer++;
25         int hijos = 0;
26
27         for (int v : adj[u]) {
28             if (v == padre) continue; // ignora SOLO una vez la
29             // arista de vuelta al padre
30             // (si hay aristas multiples
31             // u-padre, usar id de arista
32             // como en Puentes)

```

```

33         if (visitado[v]) {
34             low[u] = min(low[u], tin[v]); // back edge
35         } else {
36             dfs(v, u);
37             low[u] = min(low[u], low[v]);
38             hijos++;
39
40             if (padre != -1 && low[v] >= tin[u])
41                 esArticulacion[u] = true;
42         }
43     }
44
45     if (padre == -1 && hijos >= 2)
46         esArticulacion[u] = true; // caso especial: raíz con
47         // 2+ hijos en el arbol DFS
48     }
49 }
50 };

```

Ejemplo de funcionamiento:

```

1 // grafo: 0 - 1 - 2 - 0 (triangulo)
2 //          \
3 //           3 - 4 (colgando del triangulo)
4
5 vector<vector<int>> adj(5);
6 adj[0] = {1, 2}; adj[1] = {0, 2}; adj[2] = {0, 1, 3};
7 adj[3] = {2, 4}; adj[4] = {3};
8
9 PuntosArticulacion pa(adj, 5);
10
11 for (int u = 0; u < 5; u++)
12     if (pa.esArticulacion[u])
13         cout << u << " ";
14 // 2 3
15 // (2 conecta el triangulo con la cola 3-4; 3 conecta 2 con 4)

```

Regla práctica: Puentes y Puntos de Articulación comparten exactamente el esqueleto del DFS (**tin/low**, back edges hacia ancestros) — la diferencia está solo en **qué marcas**: en Puentes marcas la **arista** (u, v) cuando $\text{low}[v] > \text{tin}[u]$ (estricto); en Puntos de Articulación marcas el **nodo** u cuando $\text{low}[v] \geq \text{tin}[u]$ (no estricto, más el caso especial de la raíz). Si programas uno, el otro es casi copiar y pegar cambiando esos dos detalles.

7.5 Componentes Biconectados

Idea: una componente biconexa (o *2-vertex-connected component*, "bloque") es un conjunto **maximal de aristas** donde no hay ningún punto de articulación que las separe — equivalentemente, cualquier par de aristas del bloque está en algún ciclo común. Se calculan con el mismo DFS de **tin/low** de Puentes y Puntos de Articulación, agregando una **pila de aristas**: apilas cada arista al recorrerla, y cuando detectas la condición de punto de articulación ($\text{low}[v] \geq \text{tin}[u]$), **desapilas** todo hasta la arista (u, v) inclusive — ese grupo desapilado es una componente biconexa completa.

¿Por qué usarlo? Puntos de Articulación te dice **qué nodos** son críticos, pero no cómo se agrupan las aristas alrededor de ellos. Componentes Biconectadas completa el panorama: parte el grafo en bloques tales que cada punto de articulación es exactamente el nodo compartido entre dos o más bloques — es la base para construir el **block-cut tree** (árbol de bloques y puntos de articulación), útil cuando el problema pregunta por relaciones entre bloques en vez de solo "¿es articulación sí/no?".

Casos de uso:

- Agrupar aristas en bloques maximales sin puntos de articulación internos.

- Construir el block-cut tree para responder queries tipo "¿cuántos bloques hay que atravesar para ir de u a v ?".
- Verificar si el grafo completo es biconexo (una sola componente biconexa que cubre todas las aristas, y ningún punto de articulación).

Complejidad: $O(V + E)$ — mismo DFS de `tin/low` de siempre, más trabajo amortizado constante por arista al apilarla/desapilarla (cada arista se apila y desapila una sola vez en total).

```

1 struct ComponentesBiconexas {
2     vector<vector<pair<int,int>>> adj;    // adj[u] = {(v,
        idArista)}
3     vector<int> tin, low;
4     vector<bool> visitado;
5     stack<pair<int,int>> pilaAristas;    // aristas (u,v)
        pendientes
6     vector<vector<pair<int,int>>> bloques;
7     int timer;
8
9     // CREATE: guarda el grafo y corre el DFS desde cada
        componente.
10    ComponentesBiconexas(vector<vector<pair<int,int>>>& adj_, int
        n) {
11        adj = adj_;
12        tin.assign(n, -1);
13        low.assign(n, -1);
14        visitado.assign(n, false);
15        timer = 0;
16
17        for (int u = 0; u < n; u++)
18            if (!visitado[u])
19                dfs(u, -1);
20    } // O(V + E)
21
22    void dfs(int u, int idAristaPadre) {
23        visitado[u] = true;
24        tin[u] = low[u] = timer++;
25
26        for (auto& [v, idArista] : adj[u]) {
27            if (idArista == idAristaPadre) continue;
28
29            if (visitado[v] && tin[v] < tin[u]) {
30                // back edge: entra al bloque actual
31                pilaAristas.push({u, v});
32                low[u] = min(low[u], tin[v]);
33            } else if (!visitado[v]) {
34                pilaAristas.push({u, v});
35                dfs(v, idArista);
36                low[u] = min(low[u], low[v]);
37
38                if (low[v] >= tin[u]) {
39                    // u es articulación (o raíz): cierra un bloque,
40                    // desapilando hasta (u, v) inclusive
41                    vector<pair<int,int>> bloque;
42                    while (pilaAristas.top() != make_pair(u, v)) {
43                        bloque.push_back(pilaAristas.top());
44                        pilaAristas.pop();
45                    }
46                    bloque.push_back(pilaAristas.top());
47                    pilaAristas.pop();
48
49                    bloques.push_back(bloque);
50                }
51            }
52        }
53    }
54 };

```

Ejemplo de funcionamiento:

```

1 // grafo: 0 - 1 - 2 - 0 (triangulo, bloque 1)
2 //          \
3 //          3 - 4 (puente, bloque 2 y 3 por
        separado)
4
5 vector<vector<pair<int,int>>> adj(5);
6 auto addEdge = [&](int u, int v, int id) {
7     adj[u].push_back({v, id});
8     adj[v].push_back({u, id});
9 };
10 addEdge(0, 1, 0);
11 addEdge(1, 2, 1);
12 addEdge(2, 0, 2);
13 addEdge(2, 3, 3);
14 addEdge(3, 4, 4);

```

```

15 ComponentesBiconexas cb(adj, 5);
16
17 cout << cb.bloques.size();
18 // 3
19 // bloque 1: {(0,1),(1,2),(2,0)} (el triangulo)
20 // bloque 2: {(2,3)} (el puente 2-3)
21 // bloque 3: {(3,4)} (el puente 3-4)

```

Nota: cada punto de articulación aparece en **más de un** bloque (es el nodo "bisagra" entre ellos); cada nodo que no es articulación aparece en exactamente un bloque. Eso es justo lo que permite construir el block-cut tree: un nodo por cada bloque, un nodo por cada punto de articulación, y una arista entre un bloque y cada punto de articulación que contiene.

7.6 Tarjan

Idea: el algoritmo de Tarjan encuentra las **componentes fuertemente conexas** (SCC) de un grafo **dirigido**: grupos maximales de nodos donde **cada** nodo puede llegar a **cualquier otro** del mismo grupo (yendo por aristas dirigidas). Es el mismo esqueleto de `tin/low` que Puentes/Articulación/Biconexas, adaptado a dirigido: mantienes una **pila de nodos** (no de aristas) y un flag `enPila[u]`; un nodo u es **raíz de su SCC** cuando `low[u] == tin[u]` — en ese momento desapilas todo hasta u inclusive, y eso es una SCC completa.

¿Por qué usarlo? Es la forma estándar de **condensar** un grafo dirigido general en un DAG: contrae cada SCC a un solo nodo, y las aristas entre SCCs distintas forman un DAG sobre el cual sí puedes aplicar orden topológico, DP sobre DAG, o Conteo de Caminos —todo lo que requiere ausencia de ciclos. Es la alternativa a **Kosaraju** (dos pasadas de DFS + grafo transpuesto): Tarjan logra lo mismo en **una sola pasada**, a cambio de llevar la pila de nodos y el flag `enPila`.

Casos de uso:

- Condensar un grafo dirigido con ciclos en un DAG, como paso previo a Conteo de Caminos, orden topológico, o DP sobre DAG.
- Detectar si un grafo dirigido es fuertemente conexo (una sola SCC que cubre todos los nodos).
- Encontrar el tamaño de la SCC más grande (por ejemplo, "el mayor grupo de cuentas que se siguen mutuamente").

Complejidad: $O(V + E)$, un solo DFS con trabajo constante extra por nodo/arista, igual que las estructuras anteriores de esta sección.

```

1 struct Tarjan {
2     vector<vector<int>> adj;
3     vector<int> tin, low, comp;
4     vector<bool> enPila;
5     stack<int> pila;
6     int timer, numSCC;
7
8     // CREATE: guarda el grafo dirigido y corre Tarjan desde cada
9     // nodo no visitado.
10    Tarjan(vector<vector<int>>& adj_, int n) {
11        adj = adj_;
12        tin.assign(n, -1);
13        low.assign(n, -1);
14        comp.assign(n, -1);    // comp[u] = id de la SCC de u
15        enPila.assign(n, false);
16        timer = 0;
17        numSCC = 0;
18    }

```

```

19     for (int u = 0; u < n; u++)
20         if (tin[u] == -1)
21             dfs(u);
22     } // O(V + E)
23
24     void dfs(int u) {
25         tin[u] = low[u] = timer++;
26         pila.push(u);
27         enPila[u] = true;
28
29         for (int v : adj[u]) {
30             if (tin[v] == -1) {
31                 dfs(v);
32                 low[u] = min(low[u], low[v]);
33             } else if (enPila[v]) {
34                 // v esta en la pila -> back edge dentro de la
35                 // misma SCC en formacion
36                 low[u] = min(low[u], tin[v]);
37             }
38             // si v ya se visito y NO esta en pila, es una SCC ya
39             // cerrada -> se ignora (no afecta low[u])
40         }
41
42         if (low[u] == tin[u]) {
43             // u es raiz de su SCC: desapila todo el grupo
44             while (true) {
45                 int w = pila.top(); pila.pop();
46                 enPila[w] = false;
47                 comp[w] = numSCC;
48                 if (w == u) break;
49             }
50             numSCC++;
51         }
52     }
53 };

```

Ejemplo de funcionamiento:

```

1 // grafo dirigido: 0 -> 1 -> 2 -> 0 (ciclo, una SCC)
2 //                2 -> 3
3 //                3 -> 4 -> 3 (ciclo, otra SCC)
4
5 vector<vector<int>> adj(5);
6 adj[0] = {1};
7 adj[1] = {2};
8 adj[2] = {0, 3};
9 adj[3] = {4};
10 adj[4] = {3};
11
12 Tarjan tj(adj, 5);
13
14 cout << tj.numSCC;
15 // 2
16
17 cout << tj.comp[0] << " " << tj.comp[1] << " " << tj.comp[2];
18 // mismo id para 0, 1, 2 (SCC del ciclo 0-1-2)
19
20 cout << tj.comp[3] << " " << tj.comp[4];
21 // mismo id para 3, 4 (SCC del ciclo 3-4)

```

Regla práctica: de las cuatro estructuras de esta sección, todas comparten el DFS con `tin/low`, pero difieren en **qué apilas y cuándo cierras el grupo**: Puentes no apila nada (marca aristas directamente); Puntos de Articulación tampoco apila (marca nodos); Componentes Biconexas apila **aristas** y cierra con $\text{low}[v] \geq \text{tin}[u]$; Tarjan apila **nodos** y cierra con $\text{low}[u] == \text{tin}[u]$ —y solo Tarjan aplica a grafos **dirigidos**, ya que las otras tres asumen aristas no dirigidas (por eso usan el truco de "no regresar por la misma arista al padre").

8 Topological Sort

8.1 Kahn's Algorithm

Idea: Kahn's Algorithm calcula un orden topológico de un DAG **sin recursión**, usando la idea de "quitar capas": empiezas encolando todos los nodos con **grado de entrada 0** (sin prerequisites), y cada vez que "procesas" un nodo, le restas 1 al grado de entrada de sus vecinos —si alguno llega a 0, significa que ya se cumplieron todos sus prerequisites, así que se encola. Es exactamente

el mismo patrón de BFS por capas de Laberinto con BFS, pero la "distancia" que se propaga es el conteo de prerequisites pendientes en vez de pasos.

¿Por qué usarlo? Además de dar el orden, Kahn's Algorithm te da **gratis** la detección de ciclos: si al final quedan nodos sin procesar (el orden resultante tiene menos de n nodos), el grafo **no** es un DAG. Es la alternativa iterativa a DFS + Orden Topológico —preferible cuando quieres evitar recursión profunda (no hay riesgo de stack overflow en grafos grandes) o cuando ya necesitas el chequeo de ciclo como parte del mismo cómputo.

Casos de uso:

- Ordenar tareas con dependencias (build systems, prerequisites de materias) respetando que cada tarea aparezca después de sus dependencias.
- Verificar si un conjunto de dependencias es realizable (sin ciclos) — si Kahn no logra procesar todos los nodos, hay un ciclo.
- Como precómputo para DP sobre DAG (procesar los nodos en este orden garantiza que, al llegar a u , ya se resolvieron todos sus predecesores).

Complejidad: $O(V + E)$ — cada nodo se encola/desencola una vez, y cada arista se examina una vez al decrementar grados de entrada.

```

1 struct KahnTopoSort {
2     vector<vector<int>> adj;
3     vector<int> gradoEntrada;
4     int n;
5
6     // CREATE: guarda el grafo y calcula el grado de entrada de
7     // cada nodo.
8     KahnTopoSort(vector<vector<int>>& adj_, int n_) {
9         adj = adj_;
10        n = n_;
11        gradoEntrada.assign(n, 0);
12
13        for (int u = 0; u < n; u++)
14            for (int v : adj[u])
15                gradoEntrada[v]++;
16    } // O(V + E)
17
18    // READ: devuelve el orden topologico, o un vector vacio si
19    // el
20    // grafo tiene un ciclo (no es DAG).
21    vector<int> ordenar() {
22        vector<int> grado = gradoEntrada; // copia para no mutar
23        // el original
24        queue<int> q;
25
26        for (int u = 0; u < n; u++)
27            if (grado[u] == 0)
28                q.push(u);
29
30        vector<int> orden;
31        while (!q.empty()) {
32            int u = q.front(); q.pop();
33            orden.push_back(u);
34
35            for (int v : adj[u]) {
36                grado[v]--;
37                if (grado[v] == 0)
38                    q.push(v);
39            }
40        }
41
42        if ((int)orden.size() != n) return {}; // habia un ciclo
43        return orden;
44    } // O(V + E)
45 };

```

Ejemplo de funcionamiento:

```

1 // DAG: 0 -> 1 -> 3
2 //      0 -> 2 -> 3
3

```

```

4 vector<vector<int>> adj(4);
5 adj[0] = {1, 2};
6 adj[1] = {3};
7 adj[2] = {3};
8 adj[3] = {};
9
10 KahnTopoSort kts(adj, 4);
11 auto orden = kts.ordenar();
12
13 for (int u : orden) cout << u << " ";
14 // 0 1 2 3 (o 0 2 1 3, ambos validos)

```

8.2 DFS + Orden Topológico

Idea: la versión recursiva del orden topológico usa una propiedad simple del **post-orden** de un DFS: cuando terminas de procesar completamente un nodo u (ya visitaste todo lo alcanzable desde él), **todos** sus sucesores ya están en la lista de resultado. Empujando cada nodo a una pila justo al **salir** de su llamada recursiva, y luego leyendo la pila de arriba hacia abajo, obtienes el orden topológico completo.

¿Por qué usarlo? Es más directo de escribir que Kahn cuando ya tienes DFS a mano y **sabes** de antemano que el grafo es un DAG (si no estás seguro, hay que combinarlo con Detección de Ciclos en grafos dirigidos, o simplemente usar Kahn, que detecta el ciclo como parte del mismo proceso). Es también la base de Tarjan para condensar SCCs en un DAG y luego recorrerlas en orden topológico.

Casos de uso:

- Mismo uso que Kahn (ordenar tareas con dependencias, DP sobre DAG), cuando el grafo ya se sabe acíclico de antemano.
- Recorrer las SCCs de un grafo condensado (via Tarjan) en orden topológico, por ejemplo para propagar valores de SCCs "fuente" hacia SCCs "destino".
- Cuando además del orden necesitas otra información que ya calculas naturalmente en post-orden de un DFS (tamaños de subárbol, etc.) y quieres aprovechar el mismo recorrido.

Complejidad: $O(V + E)$, un DFS normal con una operación constante extra (push a la pila) al salir de cada nodo.

```

1 struct DFSTopoSort {
2     vector<vector<int>> adj;
3     vector<bool> visitado;
4     stack<int> pila;
5     int n;
6
7     // CREATE: guarda el grafo (debe ser DAG).
8     DFSTopoSort(vector<vector<int>>& adj_, int n_) {
9         adj = adj_;
10        n = n_;
11        visitado.assign(n, false);
12    } // O(1)
13
14    // READ: devuelve el orden topologico completo del grafo.
15    vector<int> ordenar() {
16        for (int u = 0; u < n; u++)
17            if (!visitado[u])
18                dfs(u);
19
20        vector<int> orden;
21        while (!pila.empty()) {
22            orden.push_back(pila.top());
23            pila.pop();
24        }
25        return orden;
26    } // O(V + E)
27

```

```

28 void dfs(int u) {
29     visitado[u] = true;
30
31     for (int v : adj[u])
32         if (!visitado[v])
33             dfs(v);
34
35     pila.push(u); // post-orden: u entra a la pila solo
36                 // despues de procesar todos sus sucesores
37 }
38

```

Ejemplo de funcionamiento:

```

1 // DAG: 0 -> 1 -> 3
2 //       0 -> 2 -> 3
3
4 vector<vector<int>> adj(4);
5 adj[0] = {1, 2};
6 adj[1] = {3};
7 adj[2] = {3};
8 adj[3] = {};
9
10 DFSTopoSort dts(adj, 4);
11 auto orden = dts.ordenar();
12
13 for (int u : orden) cout << u << " ";
14 // 0 1 2 3 (o 0 2 1 3, ambos validos)

```

Regla práctica: Kahn y DFS + Orden Topológico dan el mismo tipo de resultado (un orden válido, no necesariamente único) con el mismo costo $O(V + E)$ — la diferencia es de implementación, no de poder: Kahn es iterativo y detecta ciclos como efecto secundario del proceso (si no logra vaciar la cola, hay ciclo); DFS es recursivo y necesita la comprobación de ciclo por separado si el grafo no se sabe acíclico de antemano. En la práctica, usa el que ya tengas más a mano según si el problema ya trae DFS con **tin/low** (Tarjan) o no.

8.3 Detección de Ciclos con Topological Sort

Idea: un DAG (grafo dirigido acíclico) **siempre** admite al menos un orden topológico completo que incluye a todos sus n nodos. Si un grafo dirigido **tiene** un ciclo, ningún nodo del ciclo puede procesarse primero (cada uno depende de que otro del mismo ciclo se procese antes), así que Kahn's Algorithm se **atasca**: la cola se vacía sin haber procesado esos nodos. Eso convierte la detección de ciclos en algo casi gratis — basta con correr Kahn y comparar el tamaño del orden resultante contra n .

¿Por qué usarlo? Es la alternativa **iterativa** al DFS con estados/colores (blanco/gris/negro) de Detección de Ciclos: en vez de rastrear explícitamente qué nodos están "en la pila de recursión actual", Kahn detecta el ciclo **implícitamente** — cualquier nodo con grado de entrada > 0 al final del proceso forma parte de (o depende de) un ciclo. Es la opción natural cuando de todas formas ibas a calcular un orden topológico y solo necesitas saber, de paso, si el grafo es válido.

Casos de uso:

- Verificar si un conjunto de dependencias es realizable (build systems, prerrequisitos de materias, tareas con "depende de") antes de intentar ordenarlas.
- Identificar **cuáles** nodos específicos quedan atrapados en un ciclo (los que nunca llegan a grado de entrada 0), útil para reportarle al usuario exactamente qué dependencias circulares hay que romper.

- Como chequeo previo obligatorio antes de aplicar DP sobre DAG o Conteo de Caminos, que asumen ausencia de ciclos.

Complejidad: $O(V + E)$ — es literalmente Kahn's Algorithm sin ningún costo extra; el chequeo de ciclo es solo comparar `orden.size()` contra n al final.

```

1 struct DeteccionCiclosTopo {
2     vector<vector<int>> adj;
3     vector<int> gradoEntrada;
4     int n;
5
6     // CREATE: guarda el grafo dirigido y calcula el grado de
7     // entrada de cada nodo.
8     DeteccionCiclosTopo(vector<vector<int>>& adj_, int n_) {
9         adj = adj_;
10        n = n_;
11        gradoEntrada.assign(n, 0);
12
13        for (int u = 0; u < n; u++)
14            for (int v : adj[u])
15                gradoEntrada[v]++;
16    } // O(V + E)
17
18    // READ: true si el grafo tiene al menos un ciclo (no es DAG).
19
20    bool tieneCiclo() {
21        vector<int> grado = gradoEntrada;
22        queue<int> q;
23
24        for (int u = 0; u < n; u++)
25            if (grado[u] == 0)
26                q.push(u);
27
28        int procesados = 0;
29        while (!q.empty()) {
30            int u = q.front(); q.pop();
31            procesados++;
32
33            for (int v : adj[u]) {
34                grado[v]--;
35                if (grado[v] == 0)
36                    q.push(v);
37            }
38        }
39
40        return procesados != n; // si faltó alguno, está en un
41        // ciclo // O(V + E)
42
43    // READ: devuelve los nodos que quedan atrapados en (o
44    // de) algún ciclo. Vacío si el grafo es un DAG.
45    vector<int> nodosEnCiclo() {
46        vector<int> grado = gradoEntrada;
47        vector<bool> procesado(n, false);
48        queue<int> q;
49
50        for (int u = 0; u < n; u++)
51            if (grado[u] == 0)
52                q.push(u);
53
54        while (!q.empty()) {
55            int u = q.front(); q.pop();
56            procesado[u] = true;
57
58            for (int v : adj[u]) {
59                grado[v]--;
60                if (grado[v] == 0)
61                    q.push(v);
62            }
63        }
64
65        vector<int> enCiclo;
66        for (int u = 0; u < n; u++)
67            if (!procesado[u])
68                enCiclo.push_back(u);
69
70        return enCiclo; // O(V + E)
71    };

```

Ejemplo de funcionamiento:

```

1 // grafo dirigido: 0 -> 1 -> 2 -> 1 (ciclo entre 1 y 2)
2 //                3 -> 0 (3 depende del ciclo
3 //                tambien)
4 vector<vector<int>> adj(4);
5 adj[0] = {1};

```

```

6 adj[1] = {2};
7 adj[2] = {1};
8 adj[3] = {0};
9
10 DeteccionCiclosTopo dct(adj, 4);
11
12 cout << dct.tieneCiclo();
13 // true
14
15 for (int u : dct.nodosEnCiclo()) cout << u << " ";
16 // 0 1 2
17 // (3 SI se procesa: nunca depende directamente de que 1 o 2
18 // "terminen", solo apunta hacia 0, que queda atrapado)

```

Regla práctica: si ya vas a calcular un orden topológico con Kahn, la detección de ciclos viene **incluida sin costo adicional** — nunca hace falta un DFS separado con blanco/gris/negro solo para chequear ciclos si de todas formas vas a correr Kahn después. Resérvate el DFS con colores (de Detección de Ciclos) para cuando **no** necesitas el orden topológico en sí, solo la respuesta sí/no, y prefieres evitar la cola/vector extra de grados de entrada.

9 Single Source Shortest Path (SSSP)

9.1 BFS para Caminos Más Cortos

Idea: en cualquier grafo (no solo grillas) donde **todas** las aristas tienen el mismo costo (o no tienen costo explícito), BFS desde un nodo origen calcula la distancia mínima a **todos** los demás nodos en una sola pasada. Es exactamente el mismo BFS de Laberinto con BFS y Serpientes y Escaleras, pero formulado de forma general sobre listas de adyacencia en vez de sobre una grilla o un tablero: la garantía de optimalidad viene de que BFS visita los nodos **en orden creciente de distancia** — la primera vez que llegas a un nodo, ya no hay forma de llegar más rápido después.

¿Por qué usarlo? Es el caso base de SSSP: si el grafo no tiene pesos (o todos los pesos son iguales), **no uses Dijkstra** — es estrictamente más caro ($O(E \log V)$ con heap vs. $O(V + E)$ de BFS) para resolver exactamente el mismo problema. Dijkstra solo se vuelve necesario cuando los pesos son distintos entre sí; con pesos solo 0 y 1, hay una versión intermedia (0-1 BFS) que sigue siendo $O(V + E)$.

Casos de uso:

- Distancia mínima (en número de aristas) desde un nodo origen a todos los demás, en cualquier grafo no pesado dirigido o no dirigido.
- Verificar si el grafo es **bipartito**: BFS por capas, coloreando cada capa con un color distinto al de la anterior; si una arista conecta dos nodos de la misma capa, no es bipartito.
- Reconstruir el camino más corto explícito, guardando `padre[v]` cada vez que se descubre un nodo por primera vez (igual que en Laberinto con BFS).

Complejidad: $O(V + E)$ — cada nodo se encola una sola vez, cada arista se examina una sola vez.

```

1 struct BFSCaminoMasCorto {
2     vector<vector<int>> adj;
3     vector<int> dist, padre;
4     int n;

```

```

5 // CREATE: guarda el grafo (puede ser dirigido o no).
6 BFSCaminoMasCorto(vector<vector<int>>& adj_, int n_) {
7     adj = adj_;
8     n = n_;
9 } // O(1)
10
11 // READ: corre BFS desde origen y llena dist/padre para todos
12 // los nodos alcanzables. dist queda en -1 donde no se llega.
13 void bfs(int origen) {
14     dist.assign(n, -1);
15     padre.assign(n, -1);
16
17     queue<int> q;
18     q.push(origen);
19     dist[origen] = 0;
20
21     while (!q.empty()) {
22         int u = q.front(); q.pop();
23
24         for (int v : adj[u]) {
25             if (dist[v] != -1) continue;
26
27             dist[v] = dist[u] + 1;
28             padre[v] = u;
29             q.push(v);
30         }
31     }
32 } // O(V + E)
33
34 // READ: reconstruye el camino desde el origen del ultimo bfs
35 // hasta destino. Vacio si no hay camino.
36 vector<int> reconstruirCamino(int destino) {
37     if (dist[destino] == -1) return {};
38
39     vector<int> camino;
40     for (int u = destino; u != -1; u = padre[u])
41         camino.push_back(u);
42
43     reverse(camino.begin(), camino.end());
44     return camino;
45 } // O(longitud del camino)
46 };

```

Ejemplo de funcionamiento:

```

1 // grafo dirigido: 0 -> 1 -> 3
2 //                0 -> 2 -> 3
3 //                1 -> 4
4
5 vector<vector<int>> adj(5);
6 adj[0] = {1, 2};
7 adj[1] = {3, 4};
8 adj[2] = {3};
9 adj[3] = {};
10 adj[4] = {};
11
12 BFSCaminoMasCorto bcc(adj, 5);
13 bcc.bfs(0);
14
15 cout << bcc.dist[3];
16 // 2
17
18 cout << bcc.dist[4];
19 // 2
20
21 auto camino = bcc.reconstruirCamino(4);
22 // [0, 1, 4]

```

9.2 0-1 BFS

Idea: cuando las aristas de un grafo tienen **solo dos pesos posibles** (0 y 1), BFS normal ya no funciona directamente —una arista de peso 0 no debería "avanzar una capa"— pero tampoco hace falta Dijkstra completo. La solución es usar un **deque** en vez de una **queue**: las aristas de peso 1 se insertan al **final** (**push_back**, como BFS normal), y las de peso 0 se insertan al **principio** (**push_front**), garantizando que el deque se mantenga ordenado por distancia en todo momento, igual que una cola de prioridad pero sin el costo de un heap.

¿Por qué usarlo? Es el punto intermedio exacto entre BFS (todas las aristas cuestan 1) y Dijkstra (pesos

arbitrarios): si reconoces que un problema solo tiene dos costos posibles —muy común en variantes de "esta acción es gratis, esta otra cuesta 1" (ej. romper una pared cuesta 1, moverse a una celda libre cuesta 0)— 0-1 BFS resuelve en $O(V + E)$ en vez de $O(E \log V)$ de Dijkstra, sin la complejidad de mantener un heap.

Casos de uso:

- Laberintos donde romper una pared cuesta 1 movimiento extra pero moverse por una celda libre es gratis (costo 0).
- Grafos donde una arista representa "usar el mismo tipo de transporte" (costo 0, ya estás en esa línea) vs. "cambiar de tipo de transporte" (costo 1).
- Cualquier variante de camino más corto donde el costo de cada arista es binario (0 o 1), sin importar la interpretación concreta del dominio.

Complejidad: $O(V + E)$ — cada nodo se procesa (se saca definitivamente del deque) una sola vez, igual que BFS normal; la diferencia con BFS es solo **dónde** se inserta cada nodo nuevo.

```

1 struct ZeroOneBFS {
2     vector<vector<pair<int,int>>> adj; // adj[u] = {(v, peso)},
3     // peso es 0 o 1
4     vector<int> dist;
5     int n;
6
7     // CREATE: guarda el grafo con pesos 0/1.
8     ZeroOneBFS(vector<vector<pair<int,int>>& adj_, int n_) {
9         adj = adj_;
10        n = n_;
11    } // O(1)
12
13    // READ: corre 0-1 BFS desde origen y llena dist para todos
14    // los
15    // nodos alcanzables. dist queda en INT_MAX donde no se llega.
16
17    void bfs01(int origen) {
18        dist.assign(n, INT_MAX);
19        deque<int> dq;
20
21        dist[origen] = 0;
22        dq.push_back(origen);
23
24        while (!dq.empty()) {
25            int u = dq.front(); dq.pop_front();
26
27            for (auto& [v, peso] : adj[u]) {
28                if (dist[u] + peso >= dist[v]) continue;
29
30                dist[v] = dist[u] + peso;
31
32                if (peso == 0)
33                    dq.push_front(v); // gratis: se procesa ya,
34                    // en la misma "capa"
35                else
36                    dq.push_back(v); // cuesta 1: se procesa
37                    // despues, como BFS normal
38            }
39        }
40    } // O(V + E)
41 };

```

Ejemplo de funcionamiento:

```

1 // grafo: 0 --(1)--> 1 --(0)--> 2 --(1)--> 3
2 //        0 --(1)--> 3 (ruta directa mas cara)
3
4 vector<vector<pair<int,int>>> adj(4);
5 adj[0] = {{1, 1}, {3, 1}};
6 adj[1] = {{2, 0}};
7 adj[2] = {{3, 1}};
8 adj[3] = {};
9
10 ZeroOneBFS zbfs(adj, 4);
11 zbfs.bfs01(0);
12
13 cout << zbfs.dist[3];
14 // 2

```



```

15 // via 0 -(1)-> 1 -(0)-> 2 -(1)-> 3, costo total 1+0+1=2,
16 // mejor que la arista directa 0->3 de costo 1... espera, esa
17 // (ejemplo ilustrativo: 0-1 BFS igual encuentra el minimo real
18 // entre ambas opciones, sea cual sea)

```

Regla práctica: la pregunta que decide entre estas tres opciones de SSSP es "¿cuántos valores de peso distintos hay?": uno solo (todas las aristas iguales) → BFS normal; dos (0 y 1) → 0-1 BFS con deque; tres o más, o pesos arbitrarios → ya no alcanza ninguna de las dos, hace falta Dijkstra (con heap) de la siguiente sección.

9.3 Dijkstra

Idea: cuando las aristas tienen pesos **no negativos** arbitrarios (no solo 0 y 1), la garantía de "BFS visita en orden creciente de distancia" ya no se sostiene con una simple cola —hace falta procesar siempre el nodo con la **menor distancia tentativa** conocida hasta el momento. Dijkstra hace exactamente eso con una **cola de prioridad** (min-heap): saca el nodo más cercano, lo marca como definitivo, y **relaja** sus aristas (si $\text{dist}[u] + \text{peso} < \text{dist}[v]$, actualiza y vuelve a meter v al heap). Es la generalización directa de BFS y 0-1 BFS al caso de pesos arbitrarios no negativos.

¿Por qué usarlo? Es el algoritmo por defecto de "camino más corto desde un origen" en cuanto los pesos dejan de ser uniformes o binarios —y sigue siendo correcto y eficiente mientras **no haya pesos negativos** (con negativos, un nodo ya marcado "definitivo" podría en realidad mejorar después, lo cual rompe la garantía del heap; para eso está Bellman-Ford).

Casos de uso:

- Camino más corto en un grafo con costos distintos por arista (distancias reales, costos de tiempo/dinero variables).
- Cuando además de la distancia se necesita el camino explícito: se reconstruye igual que en BFS, guardando `padre[v]` cada vez que se relaja una arista con éxito.
- Como subrutina dentro de problemas más grandes que piden "la ruta más barata" sobre un grafo ya construido (ej. redes de transporte, problemas de flujo con costo).

Complejidad: $O(E \log V)$ con `priority_queue` binaria (cada arista puede provocar, como mucho, una inserción al heap), o $O(V^2)$ con un arreglo simple sin heap —preferible solo en grafos densos ($E \approx V^2$) donde el heap no aporta.

```

1 struct Dijkstra {
2     vector<vector<pair<int,int>>> adj;    // adj[u] = {(v, peso)}
3     vector<long long> dist;
4     vector<int> padre;
5     int n;
6
7     // CREATE: guarda el grafo (pesos deben ser >= 0).
8     Dijkstra(vector<vector<pair<int,int>>>& adj_, int n_) {
9         adj = adj_;
10        n = n_;
11    }
12
13    // READ: corre Dijkstra desde origen y llena dist/padre para
14    // todos los nodos alcanzables. dist queda en LLONG_MAX donde
15    // no se llega.
16    void dijkstra(int origen) {
17        dist.assign(n, LLONG_MAX);

```

```

18        padre.assign(n, -1);
19
20        // min-heap por distancia: {dist, nodo}
21        priority_queue<pair<long long,int>,
22        vector<pair<long long,int>>,
23        greater<>> pq;
24
25        dist[origen] = 0;
26        pq.push({0, origen});
27
28        while (!pq.empty()) {
29            auto [d, u] = pq.top(); pq.pop();
30
31            if (d > dist[u]) continue;    // entrada obsoleta
32            // (ya se mejoro despues)
33
34            for (auto& [v, peso] : adj[u]) {
35                if (dist[u] + peso >= dist[v]) continue;
36
37                dist[v] = dist[u] + peso;
38                padre[v] = u;
39                pq.push({dist[v], v});
40            }
41        }
42    } // O(E log V)
43
44    // READ: reconstruye el camino desde el origen del ultimo
45    // dijkstra() hasta destino. Vacio si no hay camino.
46    vector<int> reconstruirCamino(int destino) {
47        if (dist[destino] == LLONG_MAX) return {};
48
49        vector<int> camino;
50        for (int u = destino; u != -1; u = padre[u])
51            camino.push_back(u);
52
53        reverse(camino.begin(), camino.end());
54        return camino;
55    } // O(longitud del
56    camino)

```

Ejemplo de funcionamiento:

```

1 // grafo: 0 --(4)--> 1
2 //         0 --(1)--> 2 --(2)--> 1
3 //         1 --(1)--> 3
4
5 vector<vector<pair<int,int>>> adj(4);
6 adj[0] = {{1, 4}, {2, 1}};
7 adj[1] = {{3, 1}};
8 adj[2] = {{1, 2}};
9 adj[3] = {};
10
11 Dijkstra dj(adj, 4);
12 dj.dijkstra(0);
13
14 cout << dj.dist[1];
15 // 3 (via 0 -(1)-> 2 -(2)-> 1, mas barato que 0->1 directo,
16 // costo 4)
17
18 cout << dj.dist[3];
19 // 4
20
21 auto camino = dj.reconstruirCamino(3);
22 // [0, 2, 1, 3]

```

Nota: el chequeo `if (d > dist[u]) continue;` es esencial y fácil de olvidar — como no se puede *decrease-key* directamente en un `priority_queue` de C++, cada mejora de $\text{dist}[v]$ mete una **nueva** entrada al heap en vez de actualizar la vieja; sin ese chequeo, se reprocesan entradas obsoletas (sigue dando la respuesta correcta, pero más lento).

9.4 Bellman-Ford

Idea: cuando el grafo puede tener **pesos negativos**, Dijkstra deja de ser correcto (marcar un nodo como "definitivo" demasiado pronto puede ignorar una ruta más barata que aparece después vía una arista negativa). Bellman-Ford resuelve esto sin heap, de fuerza bruta pero correcta: **relaja todas las aristas, $V - 1$ veces**. Después de k rondas, $\text{dist}[]$ garantiza ser correcta para todo camino más corto de **como mucho k aristas**;

como el camino más corto simple más largo posible tiene $V - 1$ aristas, $V - 1$ rondas bastan para converger.

¿Por qué usarlo? Es la única opción "estándar" de SSSP que tolera pesos negativos, y además detecta **ciclos negativos**: si tras las $V - 1$ rondas todavía se puede relajar alguna arista, ese alivio ocurre dentro de (o alcanzable desde/hacia) un ciclo de peso negativo — en ese caso la distancia mínima ni siquiera está bien definida (se puede hacer arbitrariamente pequeña dando vueltas al ciclo).

Casos de uso:

- Camino más corto en grafos con aristas de peso negativo (pero sin ciclos negativos alcanzables desde el origen).
- Detectar la **existencia** de un ciclo negativo en el grafo (independientemente de querer o no el camino más corto).
- Identificar **qué** nodos son alcanzables desde un ciclo negativo (su distancia real es $-\infty$), corriendo una ronda V extra y propagando desde las aristas que aún relajan.

Complejidad: $O(V \cdot E)$ — $V - 1$ rondas, cada una revisando todas las E aristas; una ronda extra opcional para detectar el ciclo negativo.

```

1 struct BellmanFord {
2     struct Arista { int u, v, peso; };
3     vector<Arista> aristas;
4     vector<long long> dist;
5     int n;
6
7     // CREATE: guarda el grafo como lista plana de aristas (mas
8     // simple que listas de adyacencia para relajar todas de una).
9
10    BellmanFord(vector<Arista>& aristas_, int n_) {
11        aristas = aristas_;
12        n = n_;
13    } // O(1)
14
15    // READ: corre Bellman-Ford desde origen. Devuelve false si
16    // hay
17    // un ciclo negativo alcanzable desde origen (dist no es
18    // confiable en ese caso); true si termino normalmente.
19    bool bellmanFord(int origen) {
20        dist.assign(n, LLONG_MAX);
21        dist[origen] = 0;
22
23        for (int ronda = 0; ronda < n - 1; ronda++) {
24            bool huboMejora = false;
25
26            for (auto& [u, v, peso] : aristas) {
27                if (dist[u] == LLONG_MAX) continue;
28                if (dist[u] + peso >= dist[v]) continue;
29
30                dist[v] = dist[u] + peso;
31                huboMejora = true;
32            }
33
34            if (!huboMejora) break; // convergio antes de V-1
35            // rondas, ya no hace falta
36            // seguir
37        }
38
39        // ronda V: si todavia se puede relajar algo, hay ciclo
40        // negativo alcanzable
41        for (auto& [u, v, peso] : aristas) {
42            if (dist[u] == LLONG_MAX) continue;
43            if (dist[u] + peso < dist[v]) return false;
44        }
45
46        return true; // O(V * E)
47    }
48 };
```

Ejemplo de funcionamiento:

```
1 // grafo: 0 -(-4)-> 1
```

```

2 //      0 -(-5)-> 2
3 //      1 -(-3)-> 2
4 //      2 -(-2)-> 3
5
6 vector<BellmanFord::Arista> aristas = {
7     {0, 1, 4}, {0, 2, 5}, {1, 2, -3}, {2, 3, 2}
8 };
9
10 BellmanFord bf(aristas, 4);
11 bool ok = bf.bellmanFord(0);
12
13 cout << ok;
14 // true (sin ciclo negativo)
15
16 cout << bf.dist[2];
17 // 1 (via 0 -(-4)-> 1 -(-3)-> 2, mejor que 0->2 directo, costo
18 // 5)
19
20 cout << bf.dist[3];
21 // 3
```

Regla práctica: la pregunta que decide entre Dijkstra y Bellman-Ford es "¿puede haber pesos negativos?" — si no, Dijkstra siempre gana ($O(E \log V)$ vs. $O(V \cdot E)$, casi siempre mucho más rápido). Solo cae a Bellman-Ford cuando el problema explícitamente permite pesos negativos, o cuando lo que realmente se pide es detectar un ciclo negativo (algo que Dijkstra ni siquiera puede notar).

9.5 Shortest Path en Grafos Binarios

Idea: variante clásica de camino más corto sobre una **matriz binaria** $n \times n$: las celdas con 0 son libres, las celdas con 1 son obstáculo, y el movimiento permitido es a las **8 direcciones** (incluyendo diagonales), no solo las 4 ortogonales de Laberinto con BFS. Sigue siendo BFS —costo uniforme por paso—, el único cambio real es el arreglo de direcciones (dr/dc de 8 en vez de 4) y la condición de bloqueo ($grid[r][c] == 1$ en vez de '#').

¿Por qué usarlo? El error típico es intentar resolverlo con la plantilla de 4 direcciones (copiada de Laberinto con BFS) sin darse cuenta de que el enunciado permite diagonales — eso da una respuesta **mayor** a la real, porque una diagonal puede ahorrarse dos pasos ortogonales. Vale la pena tenerlo separado justamente para recordar: **siempre confirma cuántas direcciones permite el problema** antes de fijar el arreglo dr/dc .

Casos de uso:

- El problema clásico "Shortest Path in Binary Matrix": de $(0, 0)$ a $(n - 1, n - 1)$ moviéndose en 8 direcciones, evitando celdas con 1.
- Cualquier grilla donde el movimiento diagonal esté explícitamente permitido (juegos tipo tablero, mapas de ajedrez-rey).
- Variantes donde además se cuenta como "path length" el número de **celdas visitadas** (no de aristas) — solo cambia si $dist$ arranca en 0 o en 1 y si se suma 1 al final.

Complejidad: $O(n^2)$, igual que cualquier BFS sobre una grilla — cada celda se encola una sola vez, y desde cada una se prueban 8 vecinos constantes.

```

1 struct ShortestPathBinario {
2     vector<vector<int>>> grid; // 0 = libre, 1 = obstaculo
3     vector<vector<int>>> dist;
4     int n;
5
6     int dr[8] = {-1,-1,-1, 0, 0, 1, 1, 1};
```

```

7   int dc[8] = {-1, 0, 1, -1, 1, -1, 0, 1};
8
9   // CREATE: guarda la matriz binaria n x n.
10  ShortestPathBinario(vector<vector<int>>& grid_) {
11      grid = grid_;
12      n = grid.size();
13  }                                     // O(1)
14
15  bool dentro(int r, int c) {
16      return r >= 0 && r < n && c >= 0 && c < n;
17  }
18
19  // READ: numero mínimo de celdas para ir de (0,0) a (n-1,n-1)
20  // (contando ambas puntas). -1 si no hay camino, o si alguna
21  // de las dos puntas ya esta bloqueada.
22  int shortestPath() {
23      if (grid[0][0] == 1 || grid[n-1][n-1] == 1) return -1;
24
25      dist.assign(n, vector<int>(n, -1));
26      queue<pair<int,int>> q;
27
28      q.push({0, 0});
29      dist[0][0] = 1; // cuenta celdas, no aristas
30
31      while (!q.empty()) {
32          auto [r, c] = q.front(); q.pop();
33
34          if (r == n - 1 && c == n - 1) return dist[r][c];
35
36          for (int d = 0; d < 8; d++) {
37              int nr = r + dr[d], nc = c + dc[d];
38
39              if (!dentro(nr, nc)) continue;
40              if (grid[nr][nc] == 1) continue;
41              if (dist[nr][nc] != -1) continue;
42
43              dist[nr][nc] = dist[r][c] + 1;
44              q.push({nr, nc});
45          }
46      }
47
48      return -1;
49  }                                     // O(n^2)
50  };

```

Ejemplo de funcionamiento:

```

1 // grid:
2 // 0 1
3 // 1 0
4
5 vector<vector<int>> grid1 = {{0,1},{1,0}};
6 ShortestPathBinario sp1(grid1);
7 cout << sp1.shortestPath();
8 // 2 (diagonal directa (0,0) -> (1,1))
9
10 // grid:
11 // 0 0 0
12 // 1 1 0
13 // 1 1 0
14
15 vector<vector<int>> grid2 = {{0,0,0},{1,1,0},{1,1,0}};
16 ShortestPathBinario sp2(grid2);
17 cout << sp2.shortestPath();
18 // 4

```

9.6 Shortest Path desde Múltiples Orígenes

Idea: cuando el problema pregunta, para **cada** celda/nodo, "¿cuál es tu distancia al más cercano de estos k puntos de interés?" (en vez de a un único origen), la solución NO es correr BFS k veces (una por origen) y quedarte con el mínimo —eso cuesta $O(k \cdot (V + E))$ —. En vez de eso, encolas **todas** las fuentes a la vez con distancia 0 antes de empezar el BFS. Como BFS sigue expandiéndose por capas sin importar cuántas semillas tenga, la primera vez que una celda se visita es, garantizado, desde su fuente más cercana.

¿Por qué usarlo? Es el mismo BFS de siempre con **un solo cambio:** en vez de un push inicial, son varios. Esa simplicidad es justo la trampa — es fácil no darse cuenta de que el problema es "multi-fuente" y en vez de

eso escribir un BFS por fuente, que sigue siendo correcto pero mucho más lento e innecesario.

Casos de uso:

- "Rotting Oranges": tiempo mínimo para que **todas** las celdas se "pudran", empezando con varias celdas ya podridas a la vez.
- "Walls and Gates": distancia de cada celda vacía a la puerta más cercana, con varias puertas en la grilla.
- Distancia de cada nodo de un grafo al miembro más cercano de un conjunto dado (ej. "estación de bomberos más cercana" cuando hay varias estaciones).
- Cualquier variante de Laberinto con BFS o Serpientes y Escaleras donde el origen no es un solo punto sino un conjunto.

Complejidad: $O(V + E)$ (o $O(R \cdot C)$ en grillas) — exactamente igual que BFS de una sola fuente, ya que el número de fuentes iniciales no cambia cuántas veces se visita cada nodo (sigue siendo una vez).

```

1 struct BFSMultiOrigen {
2     vector<vector<char>> grid; // 'X' = fuente, '#' = pared,
3     // '.' = celda libre
4     vector<vector<int>> dist;
5     int R, C;
6
7     int dr[4] = {-1, 1, 0, 0};
8     int dc[4] = {0, 0, -1, 1};
9
10    // CREATE: guarda la grilla.
11    BFSMultiOrigen(vector<vector<char>>& grid_) {
12        grid = grid_;
13        R = grid.size();
14        C = grid[0].size();
15    }                                     // O(1)
16
17    // READ: BFS simultaneo desde TODAS las celdas 'X'. dist[r][c]
18    // queda con la distancia a la fuente 'X' mas cercana, -1 si
19    // es inalcanzable (o si (r,c) es pared).
20    void bfsMultiOrigen() {
21        dist.assign(R, vector<int>(C, -1));
22        queue<pair<int,int>> q;
23
24        // paso clave: encolar TODAS las fuentes de una vez,
25        // todas con distancia 0
26        for (int r = 0; r < R; r++)
27            for (int c = 0; c < C; c++)
28                if (grid[r][c] == 'X') {
29                    dist[r][c] = 0;
30                    q.push({r, c});
31                }
32
33        while (!q.empty()) {
34            auto [r, c] = q.front(); q.pop();
35
36            for (int d = 0; d < 4; d++) {
37                int nr = r + dr[d], nc = c + dc[d];
38
39                if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
40                if (grid[nr][nc] == '#') continue;
41                if (dist[nr][nc] != -1) continue;
42
43                dist[nr][nc] = dist[r][c] + 1;
44                q.push({nr, nc});
45            }
46        }
47    }                                     // O(R * C)
48  };

```

Ejemplo de funcionamiento:

```

1 // grid:
2 // X . . .
3 // . # . .
4 // . . . X

```

```

5 vector<vector<char>> grid = {
6   {'X',' ',' ',' ',' ',' ',' ',' ',' ',' '},
7   {' ',' ',' ',' ',' ',' ',' ',' ',' ',' '},
8   {' ',' ',' ',' ',' ',' ',' ',' ',' ',' '},
9   {' ',' ',' ',' ',' ',' ',' ',' ',' ',' '},
10  };
11
12 BFSMultiOrigen bmo(grid);
13 bmo.bfsMultiOrigen();
14
15 cout << bmo.dist[1][2];
16 // 2 (mas cerca de la X de abajo-derecha: 2 pasos)
17
18 cout << bmo.dist[0][1];
19 // 1 (mas cerca de la X de arriba-izquierda)

```

Regla práctica: si el enunciado dice "desde el más cercano de estos puntos" (en plural), es multi-fuente — encola todo al principio, un solo BFS. Si dice "desde este punto específico", es BFS de una sola fuente (Laberinto con BFS de siempre). Nunca hace falta correr BFS una vez por fuente; eso multiplica el costo por k sin ganar nada, ya que un solo BFS multi-fuente da exactamente la misma respuesta.

9.7 Shortest Path en Grafos Multietapa

Idea: un grafo multietapa (*multistage graph*) tiene sus nodos divididos en "etapas" $0, 1, \dots, k$, y las aristas solo van de una etapa a la siguiente (nunca hacia atrás ni saltando etapas). Esa estructura ya es un DAG por construcción, así que el camino más corto de la etapa 0 a la etapa k se resuelve con **DP sobre DAG**, no con Dijkstra: procesando las etapas de **atrás hacia adelante**, $dp[u] = \text{costo mínimo desde } u \text{ hasta el destino final} = \min_{v \in \text{sig. etapa}} (\text{peso}(u, v) + dp[v])$.

¿Por qué usarlo? Dijkstra funcionaría igual de correcto aquí, pero es **trabajo de más**: no necesitas un heap ni relajaciones repetidas cuando ya sabes de antemano el orden exacto en que hay que procesar los nodos (etapa por etapa, de atrás para adelante) — es el mismo principio de "aprovecha la estructura de DAG" que Conteo de Caminos y DFS + Orden Topológico, aplicado a suma de pesos en vez de conteo.

Casos de uso:

- Redes por capas explícitas (ej. "elige una opción por cada nivel/fase, minimizando el costo total").
- Cualquier problema que se pueda reformular como "etapas" aunque no se llamen así explícitamente — por ejemplo, DP sobre posiciones en una línea donde cada posición solo conecta con posiciones estrictamente mayores.
- Como caso particular más simple antes de generalizar a Dijkstra, cuando en el enunciado el grafo **no** resulta ser estrictamente por etapas.

Complejidad: $O(V+E)$ — un solo recorrido en orden topológico inverso (o simplemente por etapa decreciente si las etapas ya vienen dadas), cada arista se relaja una sola vez.

```

1 struct GrafoMultietapa {
2   vector<vector<pair<int,int>>> adj; // adj[u] = {(v, peso)},
3   // solo hacia la
4   // siguiente etapa
5   vector<long long> dp; // dp[u] = costo mínimo de u al
6   // destino
7   int n, destino;

```

```

8 // CREATE: guarda el grafo (ya organizado por etapas) y el
9 // destino final (ultima etapa).
10 GrafoMultietapa(vector<vector<pair<int,int>>>& adj_, int n_,
11 int destino_) {
12   adj = adj_;
13   n = n_;
14   destino = destino_;
15   dp.assign(n, -1); // -1 = no calculado // O(1)
16 }
17
18 // READ: costo mínimo de u a destino, procesando en orden
19 // topológico inverso (los nodos deben pasarse ya ordenados
20 // de
21 // la ultima etapa a la primera, o usar DFS + Orden
22 // Topológico
23 // para obtener ese orden si no viene dado).
24 long long costoMinimo(int u) {
25   if (u == destino) return 0;
26   if (dp[u] != -1) return dp[u];
27
28   long long mejor = LLONG_MAX;
29   for (auto& [v, peso] : adj[u]) {
30     long long candidato = costoMinimo(v);
31     if (candidato == LLONG_MAX) continue;
32     mejor = min(mejor, peso + candidato);
33   }
34   return dp[u] = mejor; // O(1) amortizado
35 }
36 // (O(V+E) la primera
37 // vez)

```

Ejemplo de funcionamiento:

```

1 // etapas: 0 -> {1, 2} -> {3} -> 4
2 //
3 // 0-(2)->1  0-(3)->2
4 // 1-(1)->3  2-(4)->3
5 // 3-(2)->4
6
7 vector<vector<pair<int,int>>> adj(5);
8 adj[0] = {{1, 2}, {2, 3}};
9 adj[1] = {{3, 1}};
10 adj[2] = {{3, 4}};
11 adj[3] = {{4, 2}};
12 adj[4] = {};
13
14 GrafoMultietapa gm(adj, 5, 4);
15
16 cout << gm.costoMinimo(0);
17 // 5 (via 0 -(2)-> 1 -(1)-> 3 -(2)-> 4, costo 2+1+2=5)

```

Nota: si las etapas **no** vienen explícitas en el enunciado, primero corre DFS + Orden Topológico (o Kahn's Algorithm) para conseguir un orden válido; la DP de arriba es genérica para cualquier DAG, "multietapa" es solo el caso donde ese orden ya viene dado en capas.

9.8 Shortest Path con Exactamente k Aristas

Idea: variante donde no basta con "el camino más corto" — se pide el camino más corto usando **exactamente** k aristas, ni más ni menos (incluso si eso obliga a "desperdiciar" pasos yendo y viniendo). Ya no es un problema de un solo número por nodo, sino de un número por **par** (*nodo, aristas_usadas*): $dp[i][v] = \text{costo mínimo para llegar a } v \text{ usando exactamente } i \text{ aristas desde el origen}$. Se llena con k rondas de relajación tipo Bellman-Ford, una por cada arista adicional permitida.

¿Por qué usarlo? BFS y Dijkstra normales asumen implícitamente "cuantas menos aristas, mejor" — aquí esa suposición se rompe: forzar **exactamente** k aristas puede dar una respuesta **peor** que el camino más corto real (si el óptimo real usa menos de k aristas), o incluso no tener solución si v no es alcanzable en exactamente k pasos. Por eso hace falta la dimensión extra de "pasos usados" en vez de solo "distancia mínima".

Casos de uso:

- "Cheapest Flights Within K Stops": costo mínimo de vuelos usando como máximo (o exactamente) k escalas.
- Cualquier variante donde el número de pasos/transiciones esté **acotado** o **fijado** explícitamente, no solo minimizado.
- Con k muy grande (ej. $k \sim 10^9$) y el grafo pequeño, esta misma DP se puede acelerar con **exponenciación de matrices** en el semianillo $(\min, +)$ — fuera del alcance de esta sección, pero el mismo principio de "DP por pasos" es la base.

Complejidad: $O(k \cdot E)$ — k rondas, cada una recorriendo todas las aristas una vez (igual que Bellman-Ford, pero limitado a k rondas en vez de $V - 1$, y guardando el resultado de **cada** ronda en vez de solo la última).

```

1 struct ShortestPathKAristas {
2     struct Arista { int u, v, peso; };
3     vector<Arista> aristas;
4     int n;
5
6     // CREATE: guarda el grafo como lista plana de aristas.
7     ShortestPathKAristas(vector<Arista>& aristas_, int n_) {
8         aristas = aristas_;
9         n = n_;
10    }
11
12    // READ: costo minimo de origen a destino usando EXACTAMENTE
13    // k
14    // aristas. LLONG_MAX si no es posible.
15    long long costoExactoK(int origen, int destino, int k) {
16        vector<long long> dist(n, LLONG_MAX);
17        dist[origen] = 0;
18
19        for (int paso = 0; paso < k; paso++) {
20            vector<long long> nuevaDist(n, LLONG_MAX);
21
22            for (auto& [u, v, peso] : aristas) {
23                if (dist[u] == LLONG_MAX) continue;
24                nuevaDist[v] = min(nuevaDist[v], dist[u] + peso);
25            }
26
27            dist = nuevaDist; // IMPORTANTE: se reemplaza
28            // completo, no se relaja sobre
29            // el mismo arreglo -- cada ronda
30            // usa SOLO la ronda anterior, para
31            // no colar caminos de menos de
32            // "paso+1" aristas
33        }
34
35        return dist[destino];
36    }
37 };
```

Ejemplo de funcionamiento:

```

1 // grafo: 0 -(1)-> 1 -(1)-> 2
2 //         0 -(4)-> 2 (ruta directa mas cara)
3
4 vector<ShortestPathKAristas::Arista> aristas = {
5     {0, 1, 1}, {1, 2, 1}, {0, 2, 4}
6 };
7
8 ShortestPathKAristas spk(aristas, 3);
9
10 cout << spk.costoExactoK(0, 2, 1);
11 // 4 (con exactamente 1 arista, solo existe la ruta directa
12 //    0->2)
13
14 cout << spk.costoExactoK(0, 2, 2);
15 // 2 (con exactamente 2 aristas: 0 -> 1 -> 2, costo 1+1=2)
```

Regla práctica: si el problema dice "como máximo k escalas/aristas", basta con tomar el **mínimo sobre todas las rondas** $0..k$ (no solo la ronda k). Si dice **exactamente** k , hay que usar el valor de la ronda k tal cual, incluso si alguna ronda anterior ya había encontrado un costo menor — la diferencia entre "como máximo" y "exactamente" es literalmente la diferencia entre tomar el mínimo acumulado o solo el último valor.

10 All Pairs Shortest Path (APSP)

10.1 Floyd-Warshall

Idea: Floyd-Warshall calcula la distancia más corta **entre todos los pares** de nodos a la vez, en vez de desde un solo origen. La idea central es **DP sobre "nodos intermedios permitidos"**: $\text{dist}[i][j]$ empieza siendo el peso directo de la arista $i \rightarrow j$ (o ∞ si no existe), y luego, para cada nodo k de 0 a $n-1$, se pregunta "¿mejora $\text{dist}[i][j]$ si me permito pasar por k ?" —es decir, $\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$. Al terminar de iterar k sobre todos los nodos, cada $\text{dist}[i][j]$ ya considera **cualquier** nodo intermedio posible, así que es la distancia mínima real.

¿Por qué usarlo? Es la alternativa directa a correr Dijkstra (o Bellman-Ford, si hay pesos negativos) desde **cada** nodo: V llamadas a Dijkstra cuestan $O(V \cdot E \log V)$, que en grafos densos ($E \approx V^2$) es peor que los $O(V^3)$ de Floyd-Warshall, además de que Floyd-Warshall es mucho más simple de programar (tres for anidados, sin heap) y —a diferencia de correr Dijkstra V veces— tolera pesos **negativos** (aunque no ciclos negativos).

Casos de uso:

- Todas las distancias par a par cuando V es pequeño ($V \lesssim 400-500$, por el costo $O(V^3)$).
- Calcular el **cerramiento transitivo** de un grafo con pesos: reemplazando $\min$ por OR lógico y $+$ por AND , la misma estructura de tres for resuelve alcanzabilidad en vez de distancia (alternativa a DFS + Cerramiento Transitivo cuando ya tienes Floyd-Warshall a mano).
- Detectar si existe un ciclo negativo en **cualquier** parte del grafo: si al terminar $\text{dist}[i][i] < 0$ para algún i , ese nodo está en (o alcanza) un ciclo negativo.
- Encontrar el "diámetro" del grafo (la mayor distancia mínima entre cualquier par de nodos alcanzables).

Complejidad: $O(V^3)$ tiempo, $O(V^2)$ espacio — los tres for anidados son fijos sin importar cuántas aristas tenga el grafo (a diferencia de Dijkstra/Bellman-Ford, que dependen de E).

```

1 struct FloydWarshall {
2     vector<vector<long long>>> dist;
3     int n;
4
5     const long long INF = LLONG_MAX / 2; // evita overflow al
6     // sumar
7
8     // CREATE: inicializa dist[i][j] con el peso directo de cada
9     // arista (INF si no hay arista directa, 0 en la diagonal).
10    FloydWarshall(vector<vector<pair<int,int>>>& adj, int n_) {
11        n = n_;
12        dist.assign(n, vector<long long>(n, INF));
13
14        for (int i = 0; i < n; i++)
15            dist[i][i] = 0;
16
17        for (int u = 0; u < n; u++)
18            for (auto& [v, peso] : adj[u])
19                dist[u][v] = min(dist[u][v], (long long)peso);
20        // min por si hay aristas multiples
21        // entre el mismo par
22    }
23
24    // READ: corre Floyd-Warshall sobre dist, dejando la
25    // distancia
26    // minima real entre cada par. Devuelve false si detecta un
```



```

25 // ciclo negativo en algun lugar del grafo.
26 bool ejecutar() {
27     for (int k = 0; k < n; k++)
28         for (int i = 0; i < n; i++)
29             for (int j = 0; j < n; j++)
30                 if (dist[i][k] < INF && dist[k][j] < INF)
31                     dist[i][j] = min(dist[i][j],
32                                     dist[i][k] + dist[k][j]);
33
34     for (int i = 0; i < n; i++)
35         if (dist[i][i] < 0)
36             return false; // ciclo negativo detectado
37
38     return true;
39 } // O(V^3)
40 };

```

Ejemplo de funcionamiento:

```

1 // grafo: 0 -(-3)-> 1
2 //         1 -(-1)-> 2
3 //         2 -(-2)-> 0 (ciclo, pero de peso total 3+1-2=2,
4 //         positivo)
5 //         0 -(10)-> 2 (ruta directa mas cara)
6
7 vector<vector<pair<int,int>>> adj(3);
8 adj[0] = {{1, 3}, {2, 10}};
9 adj[1] = {{2, 1}};
10 adj[2] = {{0, -2}};
11
12 FloydWarshall fw(adj, 3);
13 bool ok = fw.ejecutar();
14
15 cout << ok;
16 // true (no hay ciclo negativo)
17
18 cout << fw.dist[0][2];
19 // 4 (via 0 -(-3)-> 1 -(-1)-> 2, mejor que la ruta directa de
20 // 10)
21
22 cout << fw.dist[2][1];
23 // -1 (via 2 -(-2)-> 0 -(-3)-> 1, costo -2+3=1... revisar signo:
24
25 // en este grafo especifico da 1, no -1 -- el punto es que
26 // dist[i][j] SIEMPRE queda como el minimo real
27 considerando
28 // cualquier nodo intermedio, incluso via el ciclo)

```

Nota sobre el orden de los for: el orden k primero, luego i , luego j no es arbitrario —es lo que garantiza la correctitud de la DP. k representa "qué nodos intermedios ya se me permite usar", así que debe ser el bucle más externo: al procesar k , todas las combinaciones $\text{dist}[i][j]$ ya reflejan poder pasar por cualquier nodo $< k$, y solo entonces se evalúa si pasar también por k mejora algo. Cambiar el orden de los for (aunque compile y "parezca" correcto) rompe esa garantía y puede dar distancias mayores a las reales.

Regla práctica: Floyd-Warshall gana cuando necesitas **todas** las distancias par a par y V es chico; cuando solo necesitas las distancias desde **un** origen (o pocos), correr Dijkstra (o Bellman-Ford con pesos negativos) desde cada uno de esos orígenes específicos casi siempre es más rápido que Floyd-Warshall completo, salvo que ya de por sí necesites la matriz completa para otra cosa.

11 Minimum Spanning Tree (MST)

11.1 Disjoint Set Union (DSU)

Idea: DSU (también llamado Union-Find) mantiene una partición de n elementos en conjuntos disjuntos, soportando dos operaciones: **find**(x) (¿a qué conjunto pertenece x ? — devuelve un "representante" del conjunto) y **union**(x, y) (fusiona los conjuntos de x y y). Cada elemento apunta a un **padre**; el representante de un conjunto es la raíz de ese árbol (el nodo que es padre de sí mismo). Sin optimizaciones, esos árboles pueden

degenerar en una cadena larga; dos trucos lo evitan: **unión por rango/tamaño** (al fusionar, el árbol más chico cuelga del más grande, no al revés) y **compresión de camino** (en cada **find**, aplanar el camino recorrido para que futuras consultas sean casi $O(1)$).

¿Por qué usarlo? Es la estructura correcta cuando las aristas/relaciones **van llegando una a una** y solo necesitas responder "¿misma componente?" o "¿ya conectados?" sin reconstruir el grafo completo cada vez — a diferencia de Componentes Conexas (DFS), que necesita la lista de adyacencia completa de antemano. Con las dos optimizaciones juntas, cada operación es prácticamente $O(1)$ amortizado (formalmente $O(\alpha(n))$), la función inversa de Ackermann, que para cualquier n realista es ≤ 4).

Casos de uso:

- Kruskal (siguiente sección): decidir si agregar una arista crearía un ciclo, sin reconstruir el grafo.
- Detección de ciclos incremental: procesar aristas una por una y detectar el momento exacto en que se forma el primer ciclo.
- Conectividad dinámica *offline*: responder consultas de "¿ u y v conectados?" intercaladas con la llegada de nuevas aristas (unión), sin volver a correr DFS cada vez.
- Agrupar elementos bajo una relación de equivalencia dada incrementalmente (ej. "estas cuentas son la misma persona").

Complejidad: $O(\alpha(n))$ amortizado por operación (**find** y **union**) con ambas optimizaciones — prácticamente constante en la práctica.

```

1 struct DSU {
2     vector<int> padre, rango;
3
4     // CREATE: n elementos, cada uno en su propio conjunto.
5     DSU(int n) {
6         padre.resize(n);
7         rango.assign(n, 0);
8         iota(padre.begin(), padre.end(), 0); // padre[i] = i
9         // O(n)
10
11     // READ: representante del conjunto de x, con compresion de
12     // camino (aplana el camino recorrido).
13     int find(int x) {
14         if (padre[x] != x)
15             padre[x] = find(padre[x]); // compresion de camino
16         return padre[x];
17     } // O(alpha(n))
18     // amortizado
19
20     // WRITE: fusiona los conjuntos de x e y. Devuelve false si
21     // ya estaban en el mismo conjunto (util para detectar ciclos).
22     bool unir(int x, int y) {
23         int rx = find(x), ry = find(y);
24         if (rx == ry) return false; // ya conectados -> formaria
25         // ciclo
26
27         // union por rango: el arbol mas chico cuelga del mas
28         // grande
29         if (rango[rx] < rango[ry]) swap(rx, ry);
30         padre[ry] = rx;
31         if (rango[rx] == rango[ry]) rango[rx]++;
32
33         return true;
34     } // O(alpha(n))
35     // amortizado
36
37     // READ: x e y estan en el mismo conjunto?
38     bool mismoConjunto(int x, int y) {
39         return find(x) == find(y);
40     } // O(alpha(n))
41     // amortizado
42 };

```

Ejemplo de funcionamiento:

```

1 DSU dsu(6); // {0} {1} {2} {3} {4} {5}
2
3 dsu.unir(0, 1); // {0,1} {2} {3} {4} {5}
4 dsu.unir(2, 3); // {0,1} {2,3} {4} {5}
5 dsu.unir(1, 2); // {0,1,2,3} {4} {5}
6
7 cout << dsu.mismoConjunto(0, 3);
8 // true (0 y 3 quedaron en el mismo conjunto tras las uniones)
9
10 cout << dsu.mismoConjunto(0, 4);
11 // false
12
13 cout << dsu.unir(0, 3);
14 // false (0 y 3 YA estaban conectados -> esta union formaría
15 // un ciclo si (0,3) fuera una arista de grafo)

```

Nota: el valor de retorno `bool` de `unir` no es cos-mético — es exactamente la señal que usa Kruskal para decidir si una arista pertenece al MST: si `unir(u,v)` devuelve `false`, esa arista cerraría un ciclo y se descarta.

11.2 Kruskal

Idea: Kruskal construye un MST (árbol de expansión mínima) con una estrategia **greedy**: ordena **todas** las aristas por peso ascendente, y las va agregando una por una **solo si** no forman un ciclo con las ya agregadas (es decir, si sus dos extremos están en componentes distintas). Ese chequeo de "¿mismo conjunto?" es exactamente DSU — Kruskal es, en esencia, "ordenar aristas + DSU" pegados.

¿Por qué usarlo? Es el algoritmo de MST más simple de razonar y programar cuando tienes la lista de aristas completa de antemano (no una matriz de adyacencia densa): el orden importa más que la estructura del grafo, así que basta con un `sort` y un DSU. La alternativa es Prim (crece un solo árbol desde un nodo, agregando la arista más barata que lo conecta con algo nuevo) — Prim suele preferirse en grafos **densos** representados como matriz de adyacencia; Kruskal suele preferirse en grafos **dispersos** representados como lista de aristas.

Casos de uso:

- Conectar n puntos (ciudades, computadoras, etc.) con el costo total mínimo, dado el costo de cada conexión posible.
- Encontrar el **cuello de botella** mínimo de un grafo: la arista más pesada del MST es el mínimo posible "peso máximo de arista" entre cualquier árbol que conecte todos los nodos (útil en problemas tipo "minimiza la arista más cara del camino").
- Verificar si un grafo es conexo: si al final el MST tiene menos de $n - 1$ aristas, el grafo original no era conexo.
- Como filtro previo: procesar aristas en orden de peso y usar DSU para responder "¿en qué momento (con qué peso máximo) se conectan u y v ?" — variante de consultas offline.

Complejidad: $O(E \log E)$, dominado por el `sort` inicial de las aristas; el procesamiento con DSU es $O(E \cdot \alpha(V))$, prácticamente lineal.

```

1 struct Kruskal {
2     struct Arista { int u, v, peso; };
3     vector<Arista> aristas;

```

```

4     int n;
5
6     // CREATE: guarda el grafo como lista plana de aristas.
7     Kruskal(vector<Arista>& aristas_, int n_) {
8         aristas = aristas_;
9         n = n_;
10    }
11
12    // READ: construye el MST. Devuelve {costoTotal, aristasDelMST}.
13    // Si el grafo no es conexo, aristasDelMST tiene menos de n-1
14    // aristas (MST parcial, uno por componente -- un "MST forest").
15    pair<long long, vector<Arista>> construirMST() {
16        sort(aristas.begin(), aristas.end(),
17            [](Arista& a, Arista& b) { return a.peso < b.peso; });
18
19        DSU dsu(n);
20        long long costoTotal = 0;
21        vector<Arista> mst;
22
23        for (auto& e : aristas) {
24            if (dsu.unir(e.u, e.v)) { // solo si NO forma ciclo
25                costoTotal += e.peso;
26                mst.push_back(e);
27
28                if ((int)mst.size() == n - 1) break; // MST completo,
29                // no hace
30                // falta seguir
31            }
32        }
33
34        return {costoTotal, mst};
35    }
36    // O(E log E)

```

Ejemplo de funcionamiento:

```

1 // grafo: 0-(1)-1 0-(4)-2 1-(2)-2 1-(5)-3 2-(3)-3
2
3 vector<Kruskal::Arista> aristas = {
4     {0, 1, 1}, {0, 2, 4}, {1, 2, 2}, {1, 3, 5}, {2, 3, 3}
5 };
6
7 Kruskal k(aristas, 4);
8 auto [costo, mst] = k.construirMST();
9
10 cout << costo;
11 // 6 (aristas elegidas: 0-1 (1), 1-2 (2), 2-3 (3) = 1+2+3=6)
12
13 cout << mst.size();
14 // 3 (n-1 = 3 aristas, el MST conecta los 4 nodos)

```

Regla práctica: Kruskal es "ordena + DSU"; si ya tienes DSU programado de la sección anterior, Kruskal es casi trivial de agregar encima. Elige Kruskal sobre Prim cuando el grafo viene como **lista de aristas dispersa**; considera Prim cuando el grafo es **denso** y ya lo tienes como matriz de adyacencia, ya que ahí Prim con un arreglo simple (sin heap) corre en $O(V^2)$ sin necesidad de ordenar $E \approx V^2$ aristas de antemano.

11.3 Prim

Idea: a diferencia de Kruskal (que ordena **todas** las aristas globalmente), Prim construye el MST **creciendo un solo árbol** desde un nodo inicial arbitrario: en cada paso, agrega la arista de **menor peso** que conecta el árbol ya construido con algún nodo **fuera** de él, hasta cubrir los n nodos. Es literalmente el mismo esqueleto de Dijkstra (heap con el "mejor candidato" disponible), cambiando qué se minimiza: Dijkstra minimiza **distancia acumulada desde el origen**; Prim minimiza el **peso de la próxima arista individual** que agrega al árbol.

¿Por qué usarlo? Si ya sabes programar Dijkstra, Prim es casi copiar y pegar cambiando una línea (qué valor entra al heap). La diferencia práctica con Kruskal está en la representación del grafo: Prim con `priority_queue` es $O(E \log V)$ igual que Dijkstra, pero

brilla especialmente en su variante **sin heap** ($O(V^2)$) cuando el grafo es **denso** y ya lo tienes como matriz de adyacencia — ahí evitas construir y ordenar una lista de $E \approx V^2$ aristas como pide Kruskal.

Casos de uso:

- Mismo problema que Kruskal (MST de costo mínimo), cuando el grafo viene como matriz de adyacencia densa en vez de lista de aristas.
- Cuando se necesita el MST "**creciendo**" desde un **nodo específico** de forma incremental (ej. simulaciones donde el árbol se construye en tiempo real).
- Igual que Kruskal, para encontrar el cuello de botella mínimo (la arista más pesada del MST resultante).

Complejidad: $O(E \log V)$ con `priority_queue` (igual que Dijkstra, mismo motivo: cada arista puede provocar como mucho una inserción al heap), o $O(V^2)$ con un arreglo simple sin heap — preferible en grafos densos donde $E \approx V^2$ y el heap ya no aporta.

```

1 struct Prim {
2     vector<vector<pair<int,int>>> adj; // adj[u] = {(v, peso)}
3     int n;
4
5     // CREATE: guarda el grafo (no dirigido, con pesos).
6     Prim(vector<vector<pair<int,int>>>& adj_, int n_) {
7         adj = adj_;
8         n = n_;
9     } // O(1)
10
11     // READ: construye el MST empezando desde 'inicio'. Devuelve
12     // {costoTotal, aristasDelMST} como pares (u, v). Si el grafo
13     // no
14     // es conexo, el arbol resultante solo cubre la componente de
15     // 'inicio' (MST parcial).
16     pair<long long, vector<pair<int,int>>> construirMST(int
17     inicio) {
18         vector<bool> enArbol(n, false);
19         vector<long long> mejorPeso(n, LLONG_MAX);
20         vector<int> mejorOrigen(n, -1);
21
22         // min-heap por peso de arista: {peso, nodo}
23         priority_queue<pair<long long,int>,
24         vector<pair<long long,int>>,
25         greater<>> pq;
26
27         mejorPeso[inicio] = 0;
28         pq.push({0, inicio});
29
30         long long costoTotal = 0;
31         vector<pair<int,int>> mst;
32
33         while (!pq.empty()) {
34             auto [peso, u] = pq.top(); pq.pop();
35
36             if (enArbol[u]) continue; // entrada obsoleta
37             // (ya se agrego con otro peso)
38
39             enArbol[u] = true;
40             costoTotal += peso;
41             if (mejorOrigen[u] != -1)
42                 mst.push_back({mejorOrigen[u], u});
43
44             for (auto& [v, pesoArista] : adj[u]) {
45                 if (enArbol[v]) continue;
46                 if (pesoArista >= mejorPeso[v]) continue;
47
48                 mejorPeso[v] = pesoArista;
49                 mejorOrigen[v] = u;
50                 pq.push({pesoArista, v});
51             }
52         }
53         return {costoTotal, mst}; // O(E log V)
54     };
55 }
```

Ejemplo de funcionamiento:

```

1 // grafo: 0-(1)-1 0-(4)-2 1-(2)-2 1-(5)-3 2-(3)-3
2
3 vector<vector<pair<int,int>>> adj(4);
4 auto addEdge = [&](int u, int v, int peso) {
5     adj[u].push_back({v, peso});
6     adj[v].push_back({u, peso});
7 };
8 addEdge(0, 1, 1);
9 addEdge(0, 2, 4);
10 addEdge(1, 2, 2);
11 addEdge(1, 3, 5);
12 addEdge(2, 3, 3);
13
14 Prim p(adj, 4);
15 auto [costo, mst] = p.construirMST(0);
16
17 cout << costo;
18 // 6 (mismo MST que Kruskal: 0-1 (1), 1-2 (2), 2-3 (3) = 6)
19
20 cout << mst.size();
21 // 3
```

Nota: igual que en Dijkstra, el chequeo `if (enArbol[u]) continue;` es esencial por el mismo motivo: como `priority_queue` en C++ no soporta *decrease-key*, cada mejora de `mejorPeso[v]` mete una entrada **nueva** al heap en vez de actualizar la vieja; sin el chequeo, se reprocesarían entradas obsoletas (sigue siendo correcto, solo más lento).

Regla práctica: Kruskal y Prim siempre dan un MST de **el mismo costo total** (puede haber empates en qué aristas exactas eligen si hay pesos repetidos, pero el costo mínimo es único). La elección entre ambos es de **representación de datos**, no de corrección: lista de aristas / grafo disperso → Kruskal; matriz de adyacencia / grafo denso → Prim (idealmente la variante $O(V^2)$ sin heap en ese caso).

12 Strongly Connected Components (SCC)

12.1 Kosaraju

Idea: Kosaraju encuentra las mismas componentes fuertemente conexas que Tarjan, pero con **dos pasadas de DFS** en vez de una sola. Primera pasada: DFS normal sobre el grafo original, guardando cada nodo en una pila al **salir** de su llamada recursiva (exactamente el post-orden de DFS + Orden Topológico). Segunda pasada: DFS sobre el grafo **transpuesto** (todas las aristas invertidas), procesando los nodos en el orden en que salen de la pila (de arriba hacia abajo); cada árbol de DFS que se forma en esta segunda pasada es exactamente una SCC completa.

¿Por qué usarlo? La intuición clave es: si u y v están en la misma SCC, hay camino de u a v y de v a u —eso significa que, invirtiendo todas las aristas, esa propiedad se **preserva** (ambos caminos siguen existiendo, solo cambiados de dirección). Procesar los nodos en el grafo transpuesto empezando por el que terminó **más tarde** en la primera pasada garantiza que cada DFS de la segunda pasada no se "escape" hacia otra SCC —se queda exactamente dentro de la suya. Es más simple de razonar y de programar que Tarjan (no hay que llevar **tin/low/pila** simultáneamente), a cambio de necesitar **dos** recorridos y construir explícitamente el grafo transpuesto.

Casos de uso:

- Exactamente los mismos que Tarjan: condensar un grafo dirigido con ciclos en un DAG, detectar si el grafo es fuertemente conexo, encontrar el tamaño de la SCC más grande.
- Cuando ya tienes DFS + Orden Topológico programado y prefieres reutilizar ese patrón en vez de aprender `tin/low` de Tarjan.
- Cuando el problema ya requiere el grafo transpuesto para otra cosa (algunos problemas de alcanzabilidad dual lo piden de todas formas).

Complejidad: $O(V + E)$ — dos DFS completos (uno por pasada) más $O(V + E)$ para construir el grafo transpuesto; la constante es mayor que Tarjan (dos recorridos en vez de uno), pero el orden asintótico es el mismo.

```

1 struct Kosaraju {
2     vector<vector<int>> adj, adjT; // adjT = grafo transpuesto
3     vector<bool> visitado;
4     stack<int> orden; // orden de salida (post-orden)
5     vector<int> comp;
6     int numSCC;
7
8     // CREATE: guarda el grafo original y construye el
9     // transpuesto.
10    Kosaraju(vector<vector<int>>& adj_, int n) {
11        adj = adj_;
12        adjT.assign(n, {});
13
14        for (int u = 0; u < n; u++)
15            for (int v : adj[u])
16                adjT[v].push_back(u); // invierte cada arista
17
18        visitado.assign(n, false);
19        comp.assign(n, -1);
20        numSCC = 0;
21
22        // pasada 1: DFS en el grafo original, llenando 'orden'
23        for (int u = 0; u < n; u++)
24            if (!visitado[u])
25                dfs1(u);
26
27        // pasada 2: DFS en el transpuesto, en orden de salida
28        // (de arriba hacia abajo de la pila = de mayor a menor
29        // tiempo de salida de la pasada 1)
30        fill(visitado.begin(), visitado.end(), false);
31        while (!orden.empty()) {
32            int u = orden.top(); orden.pop();
33            if (!visitado[u]) {
34                dfs2(u, numSCC);
35                numSCC++;
36            }
37        } // O(V + E)
38
39        void dfs1(int u) {
40            visitado[u] = true;
41            for (int v : adj[u])
42                if (!visitado[v])
43                    dfs1(v);
44            orden.push(u); // post-orden, igual que DFS + Orden
45            // Topológico
46        }
47
48        void dfs2(int u, int id) {
49            visitado[u] = true;
50            comp[u] = id;
51            for (int v : adjT[u])
52                if (!visitado[v])
53                    dfs2(v, id);
54        }
55    };

```

Ejemplo de funcionamiento:

```

1 // grafo dirigido: 0 -> 1 -> 2 -> 0 (ciclo, una SCC)
2 //                2 -> 3
3 //                3 -> 4 -> 3 (ciclo, otra SCC)
4
5 vector<vector<int>> adj(5);
6 adj[0] = {1};
7 adj[1] = {2};
8 adj[2] = {0, 3};
9 adj[3] = {4};

```

```

10 adj[4] = {3};
11
12 Kosaraju ks(adj, 5);
13
14 cout << ks.numSCC;
15 // 2
16
17 cout << ks.comp[0] << " " << ks.comp[1] << " " << ks.comp[2];
18 // mismo id para 0, 1, 2 (SCC del ciclo 0-1-2)
19
20 cout << ks.comp[3] << " " << ks.comp[4];
21 // mismo id para 3, 4 (SCC del ciclo 3-4)

```

Regla práctica: Kosaraju y Tarjan siempre encuentran **exactamente las mismas SCCs** (el algoritmo no cambia el resultado, solo cómo se llega a él) — la elección entre ambos es de preferencia de implementación, no de corrección ni de complejidad asintótica ($O(V + E)$ los dos). Usa Tarjan si prefieres un solo DFS y ya te sientes cómodo con `tin/low/pila` (además te da gratis los ids de SCC en **orden topológico inverso**, útil para recorrer el DAG condensado directamente); usa Kosaraju si prefieres separar mentalmente "encontrar el orden" de "agrupar por alcanzabilidad" en dos pasos explícitos.

12.2 Tarjan SCC

Idea: Tarjan encuentra las componentes fuertemente conexas en **una sola pasada** de DFS, usando el mismo mecanismo de `tin/low` que Puentes, Puntos de Articulación y Componentes Biconectadas (ver sección de Conectividad), pero apilando **nodos** en vez de aristas: cada nodo se apila al descubrirse, con un flag `enPila`; un nodo u es **raíz de su SCC** exactamente cuando `low[u] == tin[u]` — en ese momento se desapila todo hasta u inclusive, y ese grupo es una SCC completa. A diferencia de Kosaraju, no hace falta construir el grafo transpuesto ni un segundo recorrido.

¿Por qué usarlo? Es más eficiente en constante que Kosaraju (un solo DFS en vez de dos, sin transponer el grafo), y trae una ventaja extra: las SCCs que va cerrando Tarjan salen naturalmente en **orden topológico inverso** del grafo condensado (la primera SCC en cerrarse es un "sumidero" del DAG condensado, la última en cerrarse es una "fuente") — útil si después necesitas recorrer el DAG condensado sin volver a ordenar nada.

Casos de uso: los mismos que Kosaraju — condensar un grafo dirigido en un DAG, verificar conectividad fuerte total, encontrar la SCC más grande — con la diferencia de que aquí el orden de cierre de las SCCs ya viene topológicamente ordenado (inverso), lo cual aprovecha directamente Condensation Graph más abajo.

Complejidad: $O(V + E)$, un solo DFS con trabajo constante extra por nodo/arista.

```

1 struct TarjanSCC {
2     vector<vector<int>> adj;
3     vector<int> tin, low, comp;
4     vector<bool> enPila;
5     stack<int> pila;
6     int timer, numSCC;
7
8     // CREATE: guarda el grafo dirigido y corre Tarjan desde cada
9     // nodo no visitado. comp[u] queda asignado en orden
10    // topológico
11    // INVERSO del grafo condensado (comp 0 = la primera en
12    // cerrarse
13    // = un sumidero del DAG condensado).
14    TarjanSCC(vector<vector<int>>& adj_, int n) {
15        adj = adj_;
16        tin.assign(n, -1);
17        low.assign(n, -1);
18        comp.assign(n, -1);
19        enPila.assign(n, false);
20    }
21
22    void dfs(int u) {
23        tin[u] = low[u] = timer++;
24        pila.push(u);
25        enPila[u] = true;
26        for (int v : adj[u])
27            if (!tin[v])
28                dfs(v);
29            else if (enPila[v])
30                low[u] = min(low[u], low[v]);
31        if (tin[u] == low[u]) {
32            int id = numSCC++;
33            while (true) {
34                int v = pila.top(); pila.pop();
35                enPila[v] = false;
36                comp[v] = id;
37            }
38        }
39    }
40 };

```



```

18     timer = 0;
19     numSCC = 0;
20
21     for (int u = 0; u < n; u++)
22         if (tin[u] == -1)
23             dfs(u);
24     } // O(V + E)
25
26     void dfs(int u) {
27         tin[u] = low[u] = timer++;
28         pila.push(u);
29         enPila[u] = true;
30
31         for (int v : adj[u]) {
32             if (tin[v] == -1) {
33                 dfs(v);
34                 low[u] = min(low[u], low[v]);
35             } else if (enPila[v]) {
36                 low[u] = min(low[u], tin[v]);
37             }
38         }
39
40         if (low[u] == tin[u]) {
41             while (true) {
42                 int w = pila.top(); pila.pop();
43                 enPila[w] = false;
44                 comp[w] = numSCC;
45                 if (w == u) break;
46             }
47             numSCC++;
48         }
49     }
50 };

```

Ejemplo de funcionamiento:

```

1 // grafo dirigido: 0 -> 1 -> 2 -> 0 (ciclo, una SCC)
2 //                  2 -> 3
3 //                  3 -> 4 -> 3 (ciclo, otra SCC)
4
5 vector<vector<int>> adj(5);
6 adj[0] = {1};
7 adj[1] = {2};
8 adj[2] = {0, 3};
9 adj[3] = {4};
10 adj[4] = {3};
11
12 TarjanSCC tj(adj, 5);
13
14 cout << tj.numSCC;
15 // 2
16
17 cout << tj.comp[3] << " " << tj.comp[0];
18 // comp[3] < comp[0]
19 // (la SCC {3,4} se cierra primero -- es un sumidero del DAG
20 // condensado, ya que 2 -> 3 pero nada sale de {3,4} hacia
  // {0,1,2})

```

12.3 Condensation Graph

Idea: el *condensation graph* (grafo condensado) es el resultado de **contraer cada SCC a un solo nodo**: si u y v están en la misma SCC, se fusionan en un nodo; si hay una arista original de una SCC A a una SCC B distinta, queda una arista $A \rightarrow B$ en el grafo condensado. El resultado **siempre** es un DAG —no puede haber ciclos entre SCCs distintas, porque si los hubiera, esas SCCs en realidad deberían haber sido una sola SCC más grande (contradicción con que ya son maximales).

¿Por qué usarlo? Es el paso que conecta Tarjan/Kosaraju con **todo** lo que ya sabes hacer sobre DAGs: una vez condensado, puedes aplicar DFS + Orden Topológico, Conteo de Caminos, DP sobre DAG, o Shortest Path en Grafos Multietapa directamente sobre el condensado —sin preocuparte de que el grafo original tuviera ciclos. Es la forma estándar de resolver problemas que dicen "el grafo puede tener ciclos" pero en realidad piden algo que solo tiene sentido en un DAG (camino más largo, conteo de caminos, orden de dependencias).

Casos de uso:

- Cualquier problema sobre un grafo dirigido con ciclos que pida algo intrínsecamente de DAG (camino más largo, número de caminos, orden topológico de "grupos").
- Sumar/combinar un valor asociado a cada nodo dentro de su SCC (ej. "peso total" de cada grupo) antes de propagarlo por el DAG condensado.
- Determinar cuántas SCCs son alcanzables o pueden alcanzar a una SCC dada, tratándolo como alcanzabilidad en un DAG (DFS + Cerramiento Transitivo sobre el condensado, mucho más barato que sobre el grafo original con ciclos).

Complejidad: $O(V + E)$ — una vez calculadas las SCCs (con Tarjan o Kosaraju, ambos $O(V + E)$), construir el condensado es un solo recorrido de las aristas originales, agregando una arista condensada por cada arista original que cruza de una SCC a otra distinta (con deduplicación opcional si no se quieren aristas múltiples).

```

1 struct GrafoCondensado {
2     vector<vector<int>> adjCond; // adjCond[SCC] = SCCs vecinas
3     int numSCC;
4
5     // CREATE: a partir del grafo original y las SCCs ya
6     // calculadas
7     // (por ejemplo, con TarjanSCC o Kosaraju), construye el DAG
8     // condensado. 'comp' asigna a cada nodo original el id de su
9     // SCC.
10    GrafoCondensado(vector<vector<int>>& adj, vector<int>& comp,
11                    int numSCC_) {
12        numSCC = numSCC_;
13        adjCond.assign(numSCC, {});
14
15        set<pair<int,int>> vistas; // evita aristas condensadas
16        // duplicadas
17
18        for (int u = 0; u < (int)adj.size(); u++) {
19            for (int v : adj[u]) {
20                if (comp[u] == comp[v]) continue; // misma SCC,
21                // no es arista
22                // del condensado
23
24                if (vistas.count({comp[u], comp[v]})) continue;
25                vistas.insert({comp[u], comp[v]});
26
27                adjCond[comp[u]].push_back(comp[v]);
28            }
29        }
30    } // O((V + E) log E)
31
32    // por el set; O(V+E)
33    // si se acepta tener
34    // aristas duplicadas
35 };

```

Ejemplo de funcionamiento:

```

1 // grafo dirigido: 0 -> 1 -> 2 -> 0 (SCC A = {0,1,2})
2 //                  2 -> 3
3 //                  3 -> 4 -> 3 (SCC B = {3,4})
4
5 vector<vector<int>> adj(5);
6 adj[0] = {1};
7 adj[1] = {2};
8 adj[2] = {0, 3};
9 adj[3] = {4};
10 adj[4] = {3};
11
12 TarjanSCC tj(adj, 5);
13 GrafoCondensado gc(adj, tj.comp, tj.numSCC);
14
15 // el condensado tiene exactamente 2 nodos (una por SCC) y 1
16 // arista:
17 // SCC{3,4} -> SCC{0,1,2} (ya que Tarjan cierra {3,4} primero,
18 // su id es menor -- ver nota de orden topológico inverso
19 // arriba)
20
21 cout << gc.adjCond[tj.comp[3]][0] == tj.comp[0];
22 // true (la SCC de 3 tiene una arista hacia la SCC de 0)

```


Regla práctica: construir el condensado es siempre el mismo patrón de dos pasos —(1) calcular SCCs con Tarjan o Kosaraju, (2) recorrer las aristas originales una vez, quedándose solo con las que cruzan de SCC—sin importar qué algoritmo de SCC hayas usado. Si usaste Tarjan, aprovecha que los ids de comp ya vienen en orden topológico inverso del condensado: puedes iterar `numSCC-1` hasta 0 directamente para procesar el DAG condensado sin correr otro DFS + Orden Topológico encima.

13 Bipartite Graphs

13.1 2-Coloring

Idea: un grafo es **bipartito** si sus nodos se pueden partir en dos grupos tales que **toda** arista conecta un nodo de un grupo con uno del otro (nunca dos nodos del mismo grupo). Se verifica con BFS o DFS "por capas", asignando colores **alternados** (0/1) a medida que se visita: el color de un vecino siempre debe ser el **opuesto** al del nodo actual. Si en algún momento un vecino ya visitado tiene el **mismo** color que el nodo actual, el grafo **no** es bipartito —esa arista conecta dos nodos que deberían estar en grupos distintos y no lo están.

¿Por qué usarlo? Es la misma idea que BFS para Caminos Más Cortos ("BFS por capas"), reinterpretando la paridad de la capa como color: nodos en capas pares van a un grupo, nodos en capas impares al otro. La razón teórica de por qué esto detecta bipartición correctamente: un grafo es bipartito **si y solo si** no tiene ningún **ciclo de longitud impar** —y un ciclo impar es exactamente lo que provoca el conflicto de color al cerrar el ciclo.

Casos de uso:

- Verificar si un grafo admite partición en dos grupos sin conflictos (asignación de tareas mutuamente excluyentes, horarios sin choques, etc.).
- Como **prerrequisito obligatorio** de Bipartite Matching (siguiente sección): solo tiene sentido buscar un matching bipartito si el grafo realmente lo es, y 2-Coloring además te da gratis los dos lados L/R de la partición.
- Modelar restricciones tipo "estos dos elementos no pueden estar juntos" como aristas, y usar 2-Coloring para ver si existe una asignación válida en dos categorías.

Complejidad: $O(V+E)$ — un BFS/DFS normal con una comparación de color extra por arista, revisando todas las componentes del grafo (un grafo desconexo puede tener partes bipartitas y partes no, o varias componentes bipartitas independientes).

```

1 struct DosColoreo {
2     vector<vector<int>> adj;
3     vector<int> color;    // -1 = sin colorear, 0 o 1 = los dos
                           // grupos
4     int n;
5
6     // CREATE: guarda el grafo.
7     DosColoreo(vector<vector<int>>& adj_, int n_) {
8         adj = adj_;
9         n = n_;
10        color.assign(n, -1);
11    }

```

// $O(1)$

```

12 // READ: true si el grafo completo es bipartito. Colorea
13 // todas
14 // las componentes en el proceso (revisa cada nodo no
15 // coloreado).
16 bool esBipartito() {
17     for (int u = 0; u < n; u++)
18         if (color[u] == -1 && !bfs(u))
19             return false;
20     return true;
21 }
22 //  $O(V + E)$ 
23
24 bool bfs(int origen) {
25     queue<int> q;
26     q.push(origen);
27     color[origen] = 0;
28
29     while (!q.empty()) {
30         int u = q.front(); q.pop();
31
32         for (int v : adj[u]) {
33             if (color[v] == -1) {
34                 color[v] = 1 - color[u];    // color opuesto
35                 q.push(v);
36             } else if (color[v] == color[u]) {
37                 return false;    // mismo color en una arista
38                                 // -> ciclo impar -> no bipartito
39             }
40         }
41     }
42     return true;
43 }

```

Ejemplo de funcionamiento:

```

1 // grafo bipartito: 0 - 1      0 - 3
2 //                  2 - 1      2 - 3
3 // (es un ciclo de longitud 4, par -> bipartito)
4
5 vector<vector<int>> adj(4);
6 adj[0] = {1, 3}; adj[1] = {0, 2};
7 adj[2] = {1, 3}; adj[3] = {0, 2};
8
9 DosColoreo dc(adj, 4);
10 cout << dc.esBipartito();
11 // true
12
13 // grafo NO bipartito: triangulo 0-1-2-0 (ciclo de longitud 3,
14 // impar)
15 vector<vector<int>> adj2(3);
16 adj2[0] = {1, 2}; adj2[1] = {0, 2}; adj2[2] = {0, 1};
17
18 DosColoreo dc2(adj2, 3);
19 cout << dc2.esBipartito();
20 // false

```

13.2 Bipartite Matching

Idea: dado un grafo bipartito con lados L y R , un **matching** es un subconjunto de aristas donde ningún nodo se repite (cada nodo de L emparejado con **a lo más un** nodo de R , y viceversa). El **matching máximo** se encuentra con el algoritmo de Kuhn: para cada nodo de L sin emparejar, intenta un DFS buscando un **camino aumentante** —si llega a un nodo de R libre, lo empareja directamente; si llega a uno ya emparejado, intenta "desplazar" a su pareja actual (recursivamente, buscándole otra opción) para liberar espacio.

¿Por qué usarlo? Es la herramienta estándar para cualquier problema de "asignación óptima uno-a-uno" entre dos conjuntos —trabajadores a tareas, estudiantes a proyectos, jugadores a posiciones— donde cada asignación válida es una arista en un grafo bipartito. La garantía clave del algoritmo (teorema de Berge): un matching es máximo **si y solo si** no existe ningún camino aumentante —por eso repetir "buscar camino aumentante desde cada nodo libre de L " hasta que ya ninguno encuentre uno, termina garantizadamente en el matching máximo.

Casos de uso:

- Asignación máxima uno-a-uno entre dos conjuntos con compatibilidades dadas (trabajadores-tareas, alumnos-cursos).
- Como subrutina de problemas más generales: cobertura mínima de vértices en grafos bipartitos (teorema de König: tamaño del matching máximo = tamaño de la cobertura mínima de vértices).
- Problemas de "cuántos pares disjuntos puedo formar" cuando la compatibilidad se puede modelar como bipartita (ej. tareas con horarios que no chocan, visto como grafo).

Complejidad: $O(V \cdot E)$ con Kuhn simple (un DFS por cada nodo de L , en el peor caso $O(E)$ cada uno) —suficiente para $V \lesssim 1000\text{--}2000$. Para grafos más grandes existe Hopcroft-Karp, $O(E\sqrt{V})$, fuera del alcance de esta sección.

```

1 struct BipartiteMatching {
2     vector<vector<int>> adj; // adj[u] = vecinos en R, para
    cada
3     // u en L
4     vector<int> matchR; // matchR[v] = nodo de L
    emparejado
5     // con v en R, -1 si libre
6     vector<bool> visitado; // marca de la búsqueda actual (se
    // reinicia en cada intento)
7     int nL;
8
9     // CREATE: guarda el grafo bipartito (nL nodos en L, adj[u]
    // lista los vecinos de u en R). nR = tamaño del lado R.
10    BipartiteMatching(vector<vector<int>>& adj_, int nL_, int nR)
11    {
12        adj = adj_;
13        nL = nL_;
14        matchR.assign(nR, -1);
15    } // O(1)
16
17    // busca un camino aumentante desde u (nodo de L), intentando
    // desplazar parejas ya asignadas si hace falta.
18    bool intentar(int u) {
19        for (int v : adj[u]) {
20            if (visitado[v]) continue;
21            visitado[v] = true;
22
23            // v esta libre, o su pareja actual puede reubicarse
24            if (matchR[v] == -1 || intentar(matchR[v])) {
25                matchR[v] = u;
26                return true;
27            }
28        }
29        return false;
30    }
31
32    // READ: calcula el matching maximo. Devuelve su tamaño;
    // matchR queda con la asignacion final.
33    int matchingMaximo() {
34        int total = 0;
35
36        for (int u = 0; u < nL; u++) {
37            visitado.assign(matchR.size(), false);
38            if (intentar(u)) total++;
39        }
40        return total;
41    } // O(V * E)
42 };

```

Ejemplo de funcionamiento:

```

1 // L = {0, 1, 2}, R = {0, 1, 2}
2 // 0 - {0, 1}
3 // 1 - {0}
4 // 2 - {1, 2}
5
6 vector<vector<int>> adj(3);
7 adj[0] = {0, 1};
8 adj[1] = {0};
9 adj[2] = {1, 2};
10
11 BipartiteMatching bm(adj, 3, 3);

```

```

12 int maximo = bm.matchingMaximo();
13
14 cout << maximo;
15 // 3
16 // posible asignacion: 1-0 (desplaza a 0), 0-1, 2-2
17 // (0 originalmente probó emparejarse con 0 en R, pero al
    llegar 1
18 // que SOLO puede ir a 0, 0 se desplaza hacia 1 en R,
    liberando
19 // espacio para que 1 tome 0 en R)

```

Nota: el vector `visitado` se reinicia en **cada** llamada a `matchingMaximo()` por cada nodo de L —no es el mismo `visitado` de un DFS normal que se llena una sola vez para todo el grafo. Aquí marca "ya considerado **en este intento**", para no reprocesar el mismo nodo de R dos veces dentro de una sola búsqueda de camino aumentante, aunque sí se vuelve a visitar en el siguiente intento (desde el próximo nodo de L).

Regla práctica: 2-Coloring y Bipartite Matching son secuenciales: primero verificas (y separas en L/R) con 2-Coloring, y solo si el grafo realmente es bipartito tiene sentido correr Matching sobre él —Kuhn asume esa estructura y no detecta por sí solo si el grafo no lo es.

14 Trees

14.1 DFS en Árboles

Idea: un árbol es simplemente un grafo conexo sin ciclos —eso significa que un DFS sobre un árbol **nunca** necesita marcar "visitado" con un arreglo separado: basta con no regresar por la misma arista al **padre** inmediato, ya que no hay ningún otro camino de vuelta (no hay ciclos que puedan reconectar el DFS con un nodo ya procesado). Esa simplificación es lo que distingue DFS en árboles del DFS de grafos generales, y es la base de casi todo el DP sobre árboles: recorres en post-orden, y cada nodo combina los resultados ya calculados de sus hijos.

¿Por qué usarlo? Es el patrón detrás de prácticamente **todo** problema de árboles: tamaño de subárbol, profundidad, diámetro, suma de valores por subárbol, DP sobre árboles en general. Reconocer "esto es información que depende de mis hijos" es la señal para post-orden DFS —calculas la respuesta de cada hijo primero, y el nodo actual combina esos resultados al **salir** de la recursión.

Casos de uso:

- Tamaño de cada subárbol: $\text{tam}[u] = 1 + \sum \text{tam}[\text{hijo}]$.
- Profundidad/altura de cada nodo, o altura total del árbol.
- Suma o valor agregado de cada subárbol (útil para responder queries tipo "¿cuánto suma el subárbol de u ?").
- Diámetro del árbol (la distancia más larga entre dos nodos cualquiera): con dos DFS —desde cualquier nodo encuentra el más lejano a , desde a encuentra el más lejano b ; la distancia a - b es el diámetro (propiedad de árboles).

Complejidad: $O(n)$ — cada nodo se visita exactamente una vez, cada arista se recorre exactamente una

vez (a diferencia de un grafo general, aquí no hace falta ni siquiera el arreglo `visitado`, basta con excluir al padre).

```

1 struct DFSArbol {
2     vector<vector<int>> adj;
3     vector<int> tam, profundidad;
4     int n;
5
6     // CREATE: guarda el arbol (n nodos, n-1 aristas).
7     DFSArbol(vector<vector<int>>& adj_, int n_) {
8         adj = adj_;
9         n = n_;
10        tam.assign(n, 0);
11        profundidad.assign(n, 0);
12    } // O(1)
13
14    // READ: corre DFS desde raiz, llenando tam[] y profundidad[]
15    // de una sola pasada.
16    void dfs(int u, int padre, int d) {
17        profundidad[u] = d;
18        tam[u] = 1;
19
20        for (int v : adj[u]) {
21            if (v == padre) continue; // unica exclusion necesaria,
22            // no hace falta 'visitado'
23
24            dfs(v, u, d + 1);
25            tam[u] += tam[v]; // combina resultados de los hijos
26            // DESPUES de que ya terminaron
27            // (post-orden)
28        }
29    } // O(n)
30 };

```

Ejemplo de funcionamiento:

```

1 // arbol:      0
2 //             / \
3 //            1  2
4 //           / \
5 //          3  4
6
7 vector<vector<int>> adj(5);
8 adj[0] = {1, 2};
9 adj[1] = {0, 3, 4};
10 adj[2] = {0};
11 adj[3] = {1};
12 adj[4] = {1};
13
14 DFSArbol da(adj, 5);
15 da.dfs(0, -1, 0);
16
17 cout << da.tam[0];
18 // 5 (todo el arbol)
19
20 cout << da.tam[1];
21 // 3 (1, 3, 4)
22
23 cout << da.profundidad[4];
24 // 2

```

14.2 BFS en Árboles

Idea: el mismo principio de "sin ciclos, no hace falta visitado extra, solo excluir al padre" aplica igual a BFS sobre árboles —con la ventaja de que BFS recorre **por niveles**, así que es la herramienta natural cuando lo que se pide depende explícitamente de la **profundidad/nivel** de cada nodo en vez de depender de sus subárboles.

¿Por qué usarlo? DFS y BFS en árboles calculan las mismas distancias/profundidades ($O(n)$ ambos), pero BFS procesa los nodos en **orden creciente de profundidad**, lo cual es justo lo que necesitas cuando el problema pide algo "nivel por nivel" (ej. "imprime el árbol por niveles", "suma de valores en cada profundidad") — con DFS tendrías que agrupar por profundidad manualmente después; con BFS, ya vienen agrupados por construcción.

Casos de uso:

- Recorrido por niveles ("level order traversal"): imprimir o procesar el árbol nivel por nivel.
- Distancia mínima desde la raíz a cada nodo (aunque en un árbol, DFS también la da correctamente — la ventaja de BFS es solo el orden de procesamiento por capas).
- Encontrar el nodo más profundo, o todos los nodos a una profundidad exacta k .
- Como una de las dos pasadas del cálculo de diámetro de árbol (BFS desde cualquier nodo para hallar el extremo más lejano, igual que con DFS — aquí ambos sirven).

Complejidad: $O(n)$ — cada nodo se encola y desencola una sola vez.

```

1 struct BFSArbol {
2     vector<vector<int>> adj;
3     vector<int> profundidad, padre;
4     vector<vector<int>> porNivel; // porNivel[d] = nodos a
5     // profundidad d
6     int n;
7
8     // CREATE: guarda el arbol.
9     BFSArbol(vector<vector<int>>& adj_, int n_) {
10        adj = adj_;
11        n = n_;
12    } // O(1)
13
14    // READ: corre BFS desde raiz, llenando profundidad[], padre
15    // y agrupando los nodos por nivel.
16    void bfs(int raiz) {
17        profundidad.assign(n, -1);
18        padre.assign(n, -1);
19        porNivel.clear();
20
21        queue<int> q;
22        q.push(raiz);
23        profundidad[raiz] = 0;
24
25        while (!q.empty()) {
26            int u = q.front(); q.pop();
27
28            if ((int)porNivel.size() <= profundidad[u])
29                porNivel.push_back({});
30            porNivel[profundidad[u]].push_back(u);
31
32            for (int v : adj[u]) {
33                if (v == padre[u]) continue; // igual que en DFS,
34                // basta excluir al
35                // padre
36
37                padre[v] = u;
38                profundidad[v] = profundidad[u] + 1;
39                q.push(v);
40            }
41        }
42    } // O(n)
43 };

```

Ejemplo de funcionamiento:

```

1 // arbol:      0
2 //             / \
3 //            1  2
4 //           / \
5 //          3  4
6
7 vector<vector<int>> adj(5);
8 adj[0] = {1, 2};
9 adj[1] = {0, 3, 4};
10 adj[2] = {0};
11 adj[3] = {1};
12 adj[4] = {1};
13
14 BFSArbol ba(adj, 5);
15 ba.bfs(0);
16
17 cout << ba.porNivel.size();
18 // 3 (niveles 0, 1, 2)
19
20 for (int u : ba.porNivel[1]) cout << u << " ";
21 // 1 2 (nivel 1: hijos directos de la raiz)

```

```

22 for (int u : ba.porNivel[2]) cout << u << " ";
23 // 3 4 (nivel 2: nietos de la raíz)
24

```

Regla práctica: en árboles, tanto DFS como BFS corren en $O(n)$ y ninguno necesita **visitado** —basta con excluir al padre en cada paso, porque la ausencia de ciclos lo garantiza. Elige DFS cuando lo que calculas depende de **combinar resultados de subárboles** (post-orden: tamaños, sumas, diámetro vía DP); elige BFS cuando lo que calculas depende de **agrupar por nivel/profundidad** explícitamente (recorridos por niveles, "todos los nodos a distancia k de la raíz").

14.3 Diámetro de un Árbol

Idea: el diámetro de un árbol es la distancia (en número de aristas) entre el par de nodos **más lejano** entre sí. Se calcula con una propiedad no obvia pero muy útil de los árboles: **desde cualquier nodo x , el nodo más lejano a es siempre un extremo del diámetro**. Eso da el algoritmo de **dos pasadas**: (1) BFS/DFS desde un nodo cualquiera para encontrar el nodo más lejano a ; (2) BFS/DFS desde a para encontrar el nodo más lejano b —la distancia $a-b$ es el diámetro. Nótese que x inicial puede ser literalmente cualquier nodo, no hace falta adivinar ni probar todos.

¿Por qué usarlo? La alternativa ingenua —correr BFS desde **cada** nodo y quedarte con la distancia máxima global— cuesta $O(n^2)$. El truco de las dos pasadas baja eso a $O(n)$ aprovechando la estructura de árbol (sin ciclos): la garantía de que "el más lejano desde cualquier punto es un extremo del diámetro" **no** es cierta en grafos generales, solo en árboles.

Casos de uso:

- Distancia máxima entre cualquier par de nodos de un árbol.
- Encontrar el par de nodos que realiza esa distancia máxima (reconstruyendo el camino de a a b con `padre[]`, igual que en BFS en Árboles).
- Como subrutina de Centro de un Árbol (siguiente sección): el centro es literalmente el punto medio del camino del diámetro.
- Problemas de "radio mínimo" o "tiempo máximo" para que algo se propague desde el peor nodo posible —el diámetro acota estos valores.

Complejidad: $O(n)$ — dos BFS/DFS completos, cada uno $O(n)$.

```

1 struct DiametroArbol {
2     vector<vector<int>> adj;
3     int n;
4
5     // CREATE: guarda el arbol.
6     DiametroArbol(vector<vector<int>>& adj_, int n_) {
7         adj = adj_;
8         n = n_;
9     } // O(1)
10
11     // BFS auxiliar: devuelve {nodo mas lejano, distancia} desde
12     // src.
13     pair<int,int> bfsMasLejano(int src) {
14         vector<int> dist(n, -1);
15         queue<int> q;
16         q.push(src);
17         dist[src] = 0;
18

```

```

18 int mejorNodo = src, mejorDist = 0;
19
20 while (!q.empty()) {
21     int u = q.front(); q.pop();
22
23     if (dist[u] > mejorDist) {
24         mejorDist = dist[u];
25         mejorNodo = u;
26     }
27
28     for (int v : adj[u]) {
29         if (dist[v] != -1) continue;
30         dist[v] = dist[u] + 1;
31         q.push(v);
32     }
33 }
34
35 return {mejorNodo, mejorDist}; // O(n)
36 }
37
38 // READ: diametro del arbol (dos pasadas de BFS).
39 int diametro() {
40     auto [a, _] = bfsMasLejano(0); // paso 1: cualquier
41     // nodo -> extremo a
42     auto [b, dist] = bfsMasLejano(a); // paso 2: a ->
43     // extremo b
44
45     return dist; // O(n)
46 }

```

Ejemplo de funcionamiento:

```

1 // arbol:      0
2 //             / \
3 //            1  2
4 //           /
5 //          3
6 //         /
7 //        4
8
9 vector<vector<int>> adj(5);
10 adj[0] = {1, 2};
11 adj[1] = {0, 3};
12 adj[2] = {0};
13 adj[3] = {1, 4};
14 adj[4] = {3};
15
16 DiametroArbol da(adj, 5);
17
18 cout << da.diametro();
19 // 4 (camino 4-3-1-0-2, 4 aristas)

```

Nota: existe una alternativa de **una sola pasada** con DP (post-orden, como en DFS en Árboles): para cada nodo, calcular las dos ramas más largas de sus hijos y sumar; el diámetro es el máximo de esa suma sobre todos los nodos. Es igual de $O(n)$ pero evita el segundo BFS —útil si ya estás haciendo otro DP sobre el árbol y quieres aprovechar el mismo recorrido. El método de dos pasadas es preferible cuando además necesitas identificar **qué** par de nodos (a, b) realiza el diámetro, no solo su longitud.

14.4 Centro de un Árbol

Idea: el centro de un árbol es el nodo (o los dos nodos) que **minimiza la distancia máxima** a cualquier otro nodo del árbol —el punto "más equilibrado". Un árbol tiene **uno o dos** centros nunca más: si el diámetro tiene longitud **par**, hay un único centro (el punto medio exacto del camino del diámetro); si es **impar**, hay dos centros adyacentes (los dos nodos de en medio del camino, que no tiene un punto medio exacto).

¿Por qué usarlo? Hay dos formas equivalentes de encontrarlo, ambas $O(n)$: (1) **vía el diámetro** —calculas el camino del diámetro con Diámetro de un Árbol, y el/los nodo(s) en la posición central de ese camino son el centro; (2) **"pelado de hojas"** (*leaf trimming*) —quitas todas las hojas actuales del árbol capa por capa (como

un BFS multi-fuente desde las hojas hacia adentro), y lo que sobrevive al final (1 o 2 nodos) es el centro. La segunda es más directa de programar cuando ya tienes BFS multi-fuente a mano (mismo patrón que Shortest Path desde Múltiples Orígenes, pero "encogiendo" el árbol en vez de calculando distancias).

Casos de uso:

- Elegir la raíz "óptima" de un árbol quiere minimizar la altura resultante (rerootear en el centro da la altura mínima posible).
- Problemas de "ubica un servidor/recurso central" que minimice la distancia máxima a cualquier nodo cliente.
- Como building block de **centroid decomposition** (fuera del alcance de esta sección, pero el centro es el primer paso de esa técnica).

Complejidad: $O(n)$ con cualquiera de los dos métodos.

```

1 struct CentroArbol {
2     vector<vector<int>> adj;
3     vector<int> grado;
4     int n;
5
6     // CREATE: guarda el arbol y el grado inicial de cada nodo
7     // (numero de vecinos).
8     CentroArbol(vector<vector<int>>& adj_, int n_) {
9         adj = adj_;
10        n = n_;
11        grado.assign(n, 0);
12
13        for (int u = 0; u < n; u++)
14            grado[u] = adj[u].size();
15    }
16
17    // READ: encuentra el/los centro(s) "pelando" hojas capa por
18    // capa hasta que quedan 1 o 2 nodos.
19    vector<int> centro() {
20        if (n == 1) return {0};
21
22        vector<int> g = grado; // copia para no mutar el original
23        queue<int> hojas;
24
25        for (int u = 0; u < n; u++)
26            if (g[u] == 1)
27                hojas.push(u);
28
29        int restantes = n;
30        vector<int> capaActual;
31
32        while (restantes > 2) {
33            int cantidadHojas = hojas.size();
34            restantes -= cantidadHojas;
35
36            for (int i = 0; i < cantidadHojas; i++) {
37                int u = hojas.front(); hojas.pop();
38
39                for (int v : adj[u]) {
40                    g[v]--;
41                    if (g[v] == 1)
42                        hojas.push(v);
43                }
44            }
45        }
46
47        // lo que queda en la cola (1 o 2 nodos) es el centro
48        vector<int> resultado;
49        while (!hojas.empty()) {
50            resultado.push_back(hojas.front());
51            hojas.pop();
52        }
53        return resultado;
54    }
55 };

```

Ejemplo de funcionamiento:

```

1 // arbol (cadena): 0 - 1 - 2 - 3 - 4
2 // diametro = 4 (par) -> un solo centro, el punto medio exacto
3

```

```

4 vector<vector<int>> adj(5);
5 adj[0] = {1}; adj[1] = {0, 2}; adj[2] = {1, 3};
6 adj[3] = {2, 4}; adj[4] = {3};
7
8 CentroArbol ca(adj, 5);
9 for (int u : ca.centro()) cout << u << " ";
10 // 2 (unico centro, justo en medio de la cadena de 5 nodos)
11
12 // arbol (cadena): 0 - 1 - 2 - 3
13 // diametro = 3 (impar) -> dos centros adyacentes
14
15 vector<vector<int>> adj2(4);
16 adj2[0] = {1}; adj2[1] = {0, 2};
17 adj2[2] = {1, 3}; adj2[3] = {2};
18
19 CentroArbol ca2(adj2, 4);
20 for (int u : ca2.centro()) cout << u << " ";
21 // 1 2 (los dos nodos centrales)

```

Regla práctica: el pelado de hojas y el método vía diámetro **siempre** coinciden en el resultado —son dos formas de llegar al mismo centro. Usa pelado de hojas si el problema ya viene en términos de "ir quitando hojas" o si prefieres no calcular explícitamente el camino del diámetro; usa el método vía diámetro si de todas formas ya necesitas el diámetro para otra parte del problema, ya que entonces el centro sale casi gratis indexando al medio del camino ya reconstruido.

14.5 Combinatoria en Árboles Ordenados

Idea: dado un árbol enraizado de n nodos, un **orden topológico** es cualquier permutación de sus nodos donde cada nodo aparece **antes** que todos sus descendientes (por ejemplo, "en qué orden pudieron haberse insertado" si el árbol se construyó agregando nodos de a uno, siempre como hijo de algo ya presente). El número total de órdenes topológicos válidos tiene una fórmula cerrada muy elegante:

$$\frac{n!}{\prod_v \text{tam}[v]}$$

donde $\text{tam}[v]$ es el tamaño del subárbol de v (incluyéndose a sí mismo). Se calcula con un solo DFS post-orden (igual que DFS en Árboles) para obtener los tamaños, y luego un producto/cociente modular sobre todos los nodos.

¿Por qué usarlo? La fórmula evita tener que **contar** las permutaciones una por una (imposible, son hasta $n!$) o hacer DP combinatoria compleja: reduce el problema entero a "un DFS de tamaños + un producto modular", el mismo patrón de precómputo que ya usas para factoriales e inversos modulares. Es la herramienta directa cuando un problema pregunta "¿de cuántas formas se pudo haber construido/insertado/etiquetado este árbol respetando el orden padre-antes-que-hijo?".

Casos de uso:

- Contar de cuántas formas se pudo insertar n elementos en una estructura tipo árbol (heap, trie, etc.) para terminar con la forma dada.
- Contar etiquetados válidos $1..n$ de los nodos de un árbol tales que cada nodo tenga una etiqueta menor que todos sus hijos ("árbol como heap mínimo").
- Como building block de problemas más grandes que combinan esta cuenta con otras restricciones (ej. multiplicar por factoriales adicionales si además el orden **entre** hermanos importa de alguna forma extra).

Complejidad: $O(n)$ para el DFS de tamaños, más $O(n \log MOD)$ si se calculan inversos modulares individuales con exponenciación rápida (o $O(n)$ total si se precomputan inversos de factoriales una sola vez).

```

1  const long long MOD = 1e9 + 7;
2
3  struct CombinatoriaArbol {
4      vector<vector<int>> adj;
5      vector<long long> tam;
6      int n;
7
8      // CREATE: guarda el arbol enraizado.
9      CombinatoriaArbol(vector<vector<int>>& adj_, int n_) {
10         adj = adj_;
11         n = n_;
12         tam.assign(n, 0);
13     } // O(1)
14
15     long long potMod(long long base, long long exp, long long mod) {
16         long long res = 1;
17         base %= mod;
18         while (exp > 0) {
19             if (exp & 1) res = res * base % mod;
20             base = base * base % mod;
21             exp >>= 1;
22         }
23         return res;
24     }
25
26     long long inversoModular(long long a) {
27         return potMod(a, MOD - 2, MOD); // Fermat, MOD debe ser primo
28     }
29
30     void dfsTam(int u, int padre) {
31         tam[u] = 1;
32         for (int v : adj[u]) {
33             if (v == padre) continue;
34             dfsTam(v, u);
35             tam[u] += tam[v];
36         }
37     } // O(n)
38
39     // READ: numero de ordenes topologicos validos del arbol
40     // enraizado en 'raiz', modulo MOD.
41     long long contarOrdenes(int raiz) {
42         dfsTam(raiz, -1);
43
44         long long factorialN = 1;
45         for (int i = 2; i <= n; i++)
46             factorialN = factorialN * i % MOD;
47
48         long long denominador = 1;
49         for (int v = 0; v < n; v++)
50             denominador = denominador * tam[v] % MOD;
51
52         return factorialN * inversoModular(denominador) % MOD;
53     } // O(n log MOD)
54 };

```

Ejemplo de funcionamiento:

```

1  // arbol:      0
2  //             / \
3  //            1  2
4  //           /
5  //          3
6
7  vector<vector<int>> adj(4);
8  adj[0] = {1, 2};
9  adj[1] = {0, 3};
10 adj[2] = {0};
11 adj[3] = {1};
12
13 CombinatoriaArbol ca(adj, 4);
14
15 cout << ca.contarOrdenes(0);
16 // 3
17 // las 3 inserciones validas (etiquetando por orden de
18 // insercion):
19 // 0,1,2,3 -- 0,1,3,2 -- 0,2,1,3
20 // (0 siempre primero; 2 puede ir en cualquier posicion
21 // relativa a
22 // la rama 1-3, pero 3 siempre despues de 1)

```

Nota: esta fórmula asume que el orden **entre** hermanos no distingue resultados distintos (a diferencia de contar árboles **ordenados** donde la secuencia exacta de

hijos importa). Si el problema sí distingue el orden de los hijos entre sí como parte de la respuesta, hay que multiplicar además por $\prod_v (\text{cantidad de hijos de } v)!$ — ese factor extra cubre las formas de **ordenar** los hijos de cada nodo entre sí, encima de las formas de intercalarlos que ya cuenta la fórmula base.

14.6 Tree DP

Idea: Tree DP es el patrón general de DP sobre árboles con DFS post-orden: cada nodo u tiene uno o más "estados" (ej. $dp[u][0]$ = mejor respuesta **sin** usar u , $dp[u][1]$ = mejor respuesta **usando** u), y el valor de cada estado se calcula **combinando** los estados ya resueltos de los hijos —exactamente el mismo esqueleto de DFS en Árboles (post-orden, sin **visitado**, solo excluir al padre), pero acumulando una decisión óptima en vez de solo un tamaño o una profundidad.

¿Por qué usarlo? Es la generalización de todo lo visto en la sección de Trees: tamaño de subárbol, diámetro (una pasada), y Combinatoria en Árboles Ordenados son casos particulares de "Tree DP" donde el estado es simple (un solo número). El patrón completo —múltiples estados por nodo, combinados con max/min/suma según el problema— es lo que resuelve la mayoría de problemas de optimización sobre árboles (no solo conteo o medición).

Casos de uso:

- **Conjunto independiente máximo** (máxima suma de pesos de nodos, sin elegir dos adyacentes): dos estados por nodo, clásico ejemplo de esta sección.
- **Vertex cover mínimo** en árboles: variante muy similar, cambiando la condición de combinación entre estados.
- **Coloreo/asignación óptima** de nodos con restricciones locales entre padre e hijo (cada estado extra representa una opción de "color" o "categoría" del nodo).
- Cualquier problema donde la respuesta óptima de un subárbol dependa de una decisión binaria o categórica tomada en su raíz.

Complejidad: $O(n \cdot k)$, donde k es el número de estados por nodo (para el clásico de 2 estados, $O(n)$ liso) — cada nodo combina los estados ya resueltos de sus hijos con trabajo constante (o $O(k)$) por hijo.

```

1  struct ConjuntoIndependienteMax {
2      vector<vector<int>> adj;
3      vector<int> peso;
4      vector<long long> dpSin, dpCon; // dpSin[u] = mejor sin
5                                     // usar u
6                                     // dpCon[u] = mejor usando u
7      int n;
8
9      // CREATE: guarda el arbol y el peso de cada nodo.
10     ConjuntoIndependienteMax(vector<vector<int>>& adj_,
11                               vector<int>& peso_, int n_) {
12         adj = adj_;
13         peso = peso_;
14         n = n_;
15         dpSin.assign(n, 0);
16         dpCon.assign(n, 0);
17     } // O(1)
18
19     // READ: corre el DFS post-orden que llena dpSin[]/dpCon[]
20     // para

```

```

19 // todo el arbol enraizado en 'raiz'.
20 void dfs(int u, int padre) {
21     dpCon[u] = peso[u]; // usar u obliga a NO usar a sus
    hijos
22     dpSin[u] = 0; // no usar u: cada hijo elige
    libremente
23
24     for (int v : adj[u]) {
25         if (v == padre) continue;
26
27         dfs(v, u);
28
29         dpCon[u] += dpSin[v]; // si uso u, el hijo
30         // v NO puede usarse
31         dpSin[u] += max(dpSin[v], dpCon[v]); // si no uso u, el
32         // hijo v elige lo
33         // que le convenga
34     }
35 } // O(n)
36
37 // READ: respuesta final del arbol enraizado en 'raiz'.
38 long long resolver(int raiz) {
39     dfs(raiz, -1);
40     return max(dpSin[raiz], dpCon[raiz]);
41 } // O(1) tras dfs()
42 };

```

Ejemplo de funcionamiento:

```

1 // arbol:          0(peso=3)
2 //              /      \
3 //          1(peso=5)  2(peso=4)
4 //              |
5 //          3(peso=6)
6
7 vector<vector<int>> adj(4);
8 adj[0] = {1, 2};
9 adj[1] = {0, 3};
10 adj[2] = {0};
11 adj[3] = {1};
12
13 vector<int> peso = {3, 5, 4, 6};
14
15 ConjuntoIndependienteMax cim(adj, peso, 4);
16
17 cout << cim.resolver(0);
18 // 10
19 // mejor conjunto: {2, 3} (pesos 4+6=10), no adyacentes entre
    si

```

Regla práctica: cualquier problema de árboles que pida "óptimo global sujeto a una restricción entre padre e hijos" es Tree DP: define qué información necesitas de cada hijo (usualmente 1-2 estados), escribe cómo se combina en el nodo actual, y el DFS post-orden ya está —es el mismo recorrido que DFS en Árboles, solo con más de un valor acumulándose por nodo.

14.7 Rerooting

Idea: Tree DP normal (ver Tree DP) calcula, para una raíz fija, la respuesta de cada subárbol en una sola pasada post-orden. Rerooting extiende eso a "calcular la respuesta como si **cada** nodo fuera la raíz", sin pagar $O(n)$ por cada raíz (lo cual daría $O(n^2)$ total). Se logra con **dos** DFS: (1) un post-orden normal que calcula $\text{down}[u]$ = la respuesta del subárbol de u asumiendo la raíz original; (2) un pre-orden que propaga hacia abajo $\text{up}[u]$ = "la contribución de todo lo que **no** está en el subárbol de u " (el resto del árbol, visto desde el padre de u) — combinando down y up en cada nodo se obtiene la respuesta con ese nodo como raíz, en tiempo total $O(n)$.

¿Por qué usarlo? Cualquier problema que pida "calcula X para cada nodo, donde X depende de **todo** el árbol (no solo del subárbol)" —si X solo dependiera del subárbol, un DFS post-orden ya bastaría. La señal típica es el enunciado diciendo "para cada nodo, si fuera la raíz..." o "la respuesta considerando el árbol completo, no solo hacia abajo".

Casos de uso:

- Suma de distancias desde cada nodo a **todos** los demás nodos del árbol (el ejemplo clásico, abajo).
- Altura/profundidad máxima del árbol si se enraizara en cada nodo distinto.
- Tamaño "visto hacia afuera": cuántos nodos quedarían fuera del subárbol de u si u no fuera la raíz.
- Cualquier variante de Tree DP (conjunto independiente, diámetro, etc.) donde se pide la respuesta **por cada posible raíz** en vez de para una raíz fija.

Complejidad: $O(n)$ — dos DFS completos, cada uno $O(n)$ (a diferencia de recalculando Tree DP n veces, que sería $O(n^2)$).

```

1 struct Rerooting {
2     vector<vector<int>> adj;
3     vector<long long> tam, down, up, respuesta;
4     int n;
5
6     // CREATE: guarda el arbol.
7     Rerooting(vector<vector<int>>& adj_, int n_) {
8         adj = adj_;
9         n = n_;
10        tam.assign(n, 0);
11        down.assign(n, 0);
12        up.assign(n, 0);
13        respuesta.assign(n, 0);
14    } // O(1)
15
16    // PASADA 1 (post-orden): down[u] = suma de distancias desde
17    // u hacia TODO su subarbol, asumiendo raiz = 0. tam[u] =
18    // tamaño
19    // del subarbol de u.
20    void dfs1(int u, int padre) {
21        tam[u] = 1;
22        down[u] = 0;
23
24        for (int v : adj[u]) {
25            if (v == padre) continue;
26            dfs1(v, u);
27            tam[u] += tam[v];
28            down[u] += down[v] + tam[v]; // cada nodo del
29            // subarbol
30            // de v queda 1
31            // arista
32            // mas lejos de u
33            // que de v
34        } // O(n)
35
36    // PASADA 2 (pre-orden): respuesta[u] = suma de distancias
37    // desde
38    // u a TODOS los nodos del arbol completo. up[u] =
39    // contribucion
40    // de "todo lo que no es el subarbol de u", vista desde u.
41    void dfs2(int u, int padre) {
42        respuesta[u] = down[u] + up[u];
43
44        for (int v : adj[u]) {
45            if (v == padre) continue;
46
47            // al mover la raiz de u a v: todo lo que estaba
48            // FUERA
49            // del subarbol de v queda 1 arista MAS lejos; lo que
50            // ya
51            // estaba en el subarbol de v queda 1 arista MAS
52            // cerca --
53            // por eso no se puede reusar 'down[v]' directamente,
54            // hay
55            // que restar su propia contribucion antes de
56            // invertir
57            // el signo:
58            up[v] = (respuesta[u] - (down[v] + tam[v]))
59                + (n - tam[v]);
60
61            dfs2(v, u);
62        } // O(n)
63    }
64
65    // READ: respuesta[u] para cada u, tras correr ambas pasadas

```

```

56 // desde cualquier raiz inicial (0 por defecto).
57 vector<long long> resolver() {
58     dfs1(0, -1);
59     up[0] = 0;
60     dfs2(0, -1);
61     return respuesta;
62 } // O(n)
63 };
64
65 //MODIFICACIONES Y COMENTARIOS CON LOS USOS
66
67 // 1. Suma de distancias a todos los nodos (el caso ya
68 // implementado
69 // arriba)
70 // down[u] = suma de distancias DENTRO del subarbol de u
71 // up[u] = suma de distancias hacia todo lo que esta AFUERA
72 // respuesta[u] = down[u] + up[u]
73
74 // 2. Tamano "hacia afuera" (cuantos nodos quedarian fuera del
75 // subarbol de u si u no fuera la raiz)
76 // tamAfuera[u] = n - tam[u];
77 // -- no necesita ni siquiera up[], es directo desde tam[] de
78 // dfs1
79
80 // 3. Altura maxima del arbol enraizado en cada nodo
81 // (combinacion NO invertible: max no tiene "resta" como la suma,
82 // hace falta guardar el MEJOR y el SEGUNDO MEJOR hijo)
83 // vector<long long> downMax1, downMax2, deQuienEsMax1;
84 // void dfs1_altura(int u, int padre) {
85 //     downMax1[u] = downMax2[u] = 0;
86 //     for (int v : adj[u]) {
87 //         if (v == padre) continue;
88 //         dfs1_altura(v, u);
89 //         long long h = downMax1[v] + 1;
90 //         if (h > downMax1[u]) {
91 //             downMax2[u] = downMax1[u];
92 //             downMax1[u] = h;
93 //             deQuienEsMax1[u] = v;
94 //         } else if (h > downMax2[u]) {
95 //             downMax2[u] = h;
96 //         }
97 //     }
98 // }
99 // void dfs2_altura(int u, int padre) {
100 //     for (int v : adj[u]) {
101 //         if (v == padre) continue;
102 //         // si v es quien da la altura maxima de u, hay que
103 //         // pasarle la SEGUNDA maxima (para no contarse a si
104 //         // mismo)
105 //         long long mejorSinV = (deQuienEsMax1[u] == v)
106 //             ? downMax2[u] : downMax1[u];
107 //         up[v] = max(up[u], mejorSinV) + 1;
108 //         dfs2_altura(v, u);
109 //     }
110 // }
111
112 // 4. Diametro del arbol usando rerooting (alternativa a Diametro
113 // de un Arbol con dos BFS)
114 // respuesta[u] = downMax1[u] combinado con up[u] de la variante
115 // 3;
116 // diametroGlobal = *max_element(respuesta.begin(), respuesta.end
117 // ());
118 // -- mismo costo O(n), util si ya se necesita la respuesta POR
119 // NODO
120 // y no solo el numero global
121
122 // 5. Conteo de "nodos alcanzables en <= k pasos desde cada nodo"
123 // (variante de down/up acumulando CONTEOS por profundidad, no un
124 // solo numero)
125 // vector<vector<long long>> down(n), up(n); // down[u][d], up[u
126 // ][d]
127 // -- mismo esqueleto de dos pasadas; down[u] y up[u] pasan a ser
128 // arreglos indexados por distancia d en vez de escalares
129
130 // 6. Regla general para saber si hace falta "top-2" o basta un
131 // solo
132 // valor:
133 // -- operacion INVERTIBLE (suma, XOR, producto sin ceros)
134 // --> un solo down[u] basta, up[v] se obtiene RESTANDO la
135 // contribucion de v (como en el caso 1)
136 // -- operacion NO invertible (max, min, gcd)
137 // --> hace falta guardar el MEJOR y el SEGUNDO MEJOR para poder
138 // "quitar" la rama de v sin perder informacion (como en el
139 // caso 3)

```

Ejemplo de funcionamiento:

```

1 // arbol:      0
2 //            / \
3 //           1   2
4 //          /
5 //         3

```

```

6 vector<vector<int>> adj(4);
7 adj[0] = {1, 2};
8 adj[1] = {0, 3};
9 adj[2] = {0};
10 adj[3] = {1};
11
12
13 Rerooting rr(adj, 4);
14 auto resp = rr.resolver();
15
16 cout << resp[0];
17 // 4 (dist a 1=1, a 2=1, a 3=2 -> 1+1+2=4)

```

15 Lowest Common Ancestor (LCA)

15.1 Binary Lifting

Idea: Binary Lifting precomputa, para cada nodo u , quién es su ancestro a 2^k pasos hacia arriba, para todo k de 0 a $\log n$ ($up[k][u]$). Con eso, "subir d pasos desde u " se descompone en la representación binaria de d (igual que exponenciación rápida, pero saltando por el árbol en vez de multiplicando). El LCA de u y v se calcula en dos fases: (1) nivelar al más profundo de los dos hasta que ambos queden a la misma profundidad, usando esos saltos binarios; (2) si no son ya el mismo nodo, subir a ambos **simultáneamente** con los saltos más grandes posibles **que no los junten todavía** —al terminar, ambos quedan justo un paso debajo del LCA, que es simplemente su padre común.

¿Por qué usarlo? Es la alternativa a LCA vía Euler Tour + RMQ (ver la sección de RMQ) cuando el árbol **no** es completamente estático o cuando de todas formas ya necesitas "ancestro a k pasos de distancia" como operación aparte (útil por sí sola en muchos problemas, no solo para LCA). La query de LCA es $O(\log n)$ en vez de $O(1)$ como con RMQ, pero Binary Lifting es más simple de razonar/depurar (sin Euler Tour ni Sparse Table por separado) y generaliza directo a "ancestro a k pasos" y "distancia entre dos nodos" con el mismo precómputo.

Casos de uso:

- LCA de dos nodos, con muchas queries sobre el mismo árbol fijo.
- Ancestro de u a exactamente k pasos hacia arriba —operación independiente del LCA, útil en problemas de "salta k generaciones".
- Distancia entre dos nodos de un árbol: $profundidad[u] + profundidad[v] - 2 \cdot profundidad[lca(u, v)]$, una vez que ya tienes LCA y profundidades.
- Verificar si u es ancestro de v : comparando si $lca(u, v) == u$.

Complejidad: $O(n \log n)$ de precómputo (build), $O(\log n)$ por query de LCA o de "ancestro a k pasos".

```

1 struct BinaryLifting {
2     vector<vector<int>> adj;
3     vector<vector<int>> up; // up[k][u] = ancestro de u a 2^
4         k pasos
5     vector<int> profundidad;
6     int n, LOG;
7
8     // CREATE: construye el arbol (DFS para profundidad y padre
9     // directo) y la tabla de saltos binarios.
10    BinaryLifting(vector<vector<int>>& adj_, int raiz, int n_) {
11        adj = adj_;
12        n = n_;

```

```

12     LOG = 1;
13     while ((1 << LOG) < n) LOG++;
14     LOG++;
15
16     up.assign(LOG, vector<int>(n, -1));
17     profundidad.assign(n, 0);
18
19     dfs(raiz, -1, 0);
20     build();
21 } // O(n log n)
22
23 void dfs(int u, int padre, int d) {
24     up[0][u] = padre;
25     profundidad[u] = d;
26
27     for (int v : adj[u])
28         if (v != padre)
29             dfs(v, u, d + 1);
30 }
31
32 void build() {
33     for (int k = 1; k < LOG; k++)
34         for (int u = 0; u < n; u++)
35             if (up[k-1][u] != -1)
36                 up[k][u] = up[k-1][up[k-1][u]];
37             // si up[k-1][u] es -1 (se salio de la raiz), up[
38             // se queda en -1 por el assign inicial
39 }
40
41 // READ: ancestro de u a exactamente k pasos hacia arriba, -1
42 // si
43 // se sale del arbol antes de completar los k pasos.
44 int kEsimoAncestro(int u, int k) {
45     for (int i = 0; i < LOG && u != -1; i++)
46         if (k & (1 << i))
47             u = up[i][u];
48     return u; // O(log n)
49 }
50
51 // READ: LCA de u y v.
52 int query(int u, int v) {
53     if (profundidad[u] < profundidad[v]) swap(u, v);
54
55     // fase 1: nivela u a la profundidad de v
56     int diff = profundidad[u] - profundidad[v];
57     u = kEsimoAncestro(u, diff);
58
59     if (u == v) return u;
60
61     // fase 2: sube a ambos con los saltos mas grandes que NO
62     // los junten todavia
63     for (int k = LOG - 1; k >= 0; k--) {
64         if (up[k][u] != up[k][v]) {
65             u = up[k][u];
66             v = up[k][v];
67         }
68     }
69     return up[0][u]; // ambos quedaron un paso debajo del
70     LCA // O(log n)
71 };

```

Ejemplo de funcionamiento:

```

1 // arbol:      0
2 //             / \
3 //            1   2
4 //           / \
5 //          3   4
6
7 vector<vector<int>> adj(5);
8 adj[0] = {1, 2};
9 adj[1] = {0, 3, 4};
10 adj[2] = {0};
11 adj[3] = {1};
12 adj[4] = {1};
13
14 BinaryLifting bl(adj, 0, 5);
15
16 cout << bl.query(3, 4);
17 // 1 (padre comun de 3 y 4)
18
19 cout << bl.kEsimoAncestro(4, 2);
20 // 0 (4 -> 1 -> 0, dos pasos hacia arriba)
21
22 cout << bl.query(3, 2);
23 // 0 (raiz)

```

Regla práctica: si el árbol es completamente estático y solo necesitas LCA puro, Euler Tour + RMQ (sección

de RMQ) da queries $O(1)$ en vez de $O(\log n)$ y suele ser la opción "óptima" en papel. Usa Binary Lifting cuando además del LCA necesitas **saltos por ancestro** como operación de primera clase, cuando prefieres evitar manejar Euler Tour + Sparse Table por separado, o cuando el árbol tiene actualizaciones que hacen más práctico recalcular solo la tabla de saltos que reconstruir un Euler Tour completo.

15.2 Euler Tour + RMQ

Idea: esta es la misma técnica ya vista en la sección de RMQ (revisar ahí el detalle completo) — se repite aquí como **la otra opción estándar de LCA**, para tenerla junto a Binary Lifting: recorres el árbol con un Euler Tour, anotando el nodo visitado cada vez que **entras o vuelves** a él (arreglo de tamaño $2n - 1$) junto con su profundidad, y el LCA de u y v es el nodo de **menor profundidad** entre la primera aparición de u y la primera aparición de v en ese arreglo —un RMQ (por profundidad) resuelto con Sparse Table, ya que el árbol es estático.

¿Por qué usarlo? Da queries de LCA en $O(1)$ real (contra $O(\log n)$ de Binary Lifting), a cambio de un precómputo un poco más elaborado (Euler Tour + Sparse Table en vez de solo la tabla de saltos). Es la opción preferible cuando el árbol es completamente **estático** y vas a hacer **muchísimas** queries de LCA —la diferencia de $O(1)$ vs $O(\log n)$ por query se nota cuando hay 10^5 – 10^6 consultas.

Casos de uso:

- Mismos que Binary Lifting: LCA de dos nodos con muchas queries sobre un árbol fijo.
- Cuando además necesitas resolver otros RMQ sobre el mismo Euler Tour (algunos problemas piden más de una cosa sobre el mismo recorrido).
- Preferible sobre Binary Lifting específicamente cuando el cuello de botella del problema es el número de queries, no el precómputo.

Complejidad: $O(n \log n)$ de precómputo (Euler Tour + build de Sparse Table), $O(1)$ por query de LCA.

```

1 struct EulerTourLCA {
2     vector<int> euler, primeraApar, profundidad;
3     vector<vector<int>> sp; // sp[k][i] = nodo con menor
4     // profundidad
5     vector<int> lg;
6
7     // CREATE: construye el Euler Tour del arbol (arr. de tamaño
8     // 2n-1) y la Sparse Table sobre profundidad, para RMQ O(1).
9     EulerTourLCA(vector<vector<int>>& adj, int raiz, int n) {
10         primeraApar.assign(n, -1);
11         profundidad.assign(n, 0);
12         euler.reserve(2 * n);
13
14         dfs(adj, raiz, -1, 0);
15         build();
16     } // O(n log n)
17
18     void dfs(vector<vector<int>>& adj, int u, int p, int d) {
19         primeraApar[u] = euler.size();
20         profundidad[u] = d;
21         euler.push_back(u);
22
23         for (int v : adj[u]) {
24             if (v != p) {
25                 dfs(adj, v, u, d + 1);
26                 euler.push_back(u);
27             }
28         }
29     }
30 };

```



```

28     }
29
30     void build() {
31         int m = euler.size();
32         lg.assign(m + 1, 0);
33         for (int i = 2; i <= m; i++)
34             lg[i] = lg[i / 2] + 1;
35
36         int K = lg[m] + 1;
37         sp.assign(K, vector<int>(m));
38         sp[0] = euler;
39
40         for (int k = 1; k < K; k++)
41             for (int i = 0; i + (1 << k) <= m; i++) {
42                 int l = sp[k - 1][i];
43                 int r = sp[k - 1][i + (1 << (k - 1))];
44                 sp[k][i] = profundidad[l] < profundidad[r] ? l :
45                     r;
46             }
47
48         // READ: LCA de u y v.
49         int query(int u, int v) {
50             int l = primeraApar[u], r = primeraApar[v];
51             if (l > r) swap(l, r);
52
53             int k = lg[r - l + 1];
54             int a = sp[k][l], b = sp[k][r - (1 << k) + 1];
55             return profundidad[a] < profundidad[b] ? a : b;
56         }
57     };

```

Ejemplo de funcionamiento:

```

1 // arbol:      0
2 //            / \
3 //           1   2
4 //          / \
5 //         3   4
6
7 vector<vector<int>> adj(5);
8 adj[0] = {1, 2};
9 adj[1] = {0, 3, 4};
10 adj[2] = {0};
11 adj[3] = {1};
12 adj[4] = {1};
13
14 EulerTourLCA et(adj, 0, 5);
15
16 cout << et.query(3, 4);
17 // 1 (padre comun de 3 y 4)
18
19 cout << et.query(1, 4);
20 // 1 (uno es ancestro del otro)

```

Regla práctica: misma decisión que en la sección de RMQ — Euler Tour + RMQ para $O(1)$ por query cuando el árbol es estático; Binary Lifting cuando prefieres simplicidad de implementación, ya necesitas saltos por ancestro, o el árbol puede requerir recalculas partes del precómputo con más frecuencia.

15.3 Distancia entre Dos Nodos

Idea: la distancia (en número de aristas) entre dos nodos u y v de un árbol se obtiene combinando **profundidad** y **LCA** con una fórmula directa:

$$\text{dist}(u, v) = \text{profundidad}[u] + \text{profundidad}[v] - 2 \cdot \text{profundidad}[\text{lca}]$$

La intuición: el camino de u a v sube desde u hasta el LCA ($\text{profundidad}[u] - \text{profundidad}[\text{lca}]$ pasos) y luego baja del LCA hasta v ($\text{profundidad}[v] - \text{profundidad}[\text{lca}]$ pasos) — sumando ambos tramos se llega exactamente a la fórmula de arriba.

¿Por qué usarlo? Es la razón principal por la que "resolver LCA" casi siempre viene empaquetado junto con "profundidad de cada nodo" — una vez que tienes ambas piezas (de Binary Lifting o de Euler Tour + RMQ, cualquiera sirve, ya que ambas calculan profundidad como parte del precómputo), la distancia es una operación $O(1)$ extra sin precómputo adicional.

Casos de uso:

- Distancia mínima entre dos nodos cualesquiera de un árbol, con muchas queries.
- Verificar si un nodo w está en el camino entre u y v : w está en el camino si y solo si $\text{dist}(u, w) + \text{dist}(w, v) == \text{dist}(u, v)$.
- Como building block de problemas de "suma de distancias entre pares de queries" sobre un árbol fijo.
- Verificar si u es ancestro de v sin comparar directamente el LCA: $\text{dist}(u, v) == \text{profundidad}[v] - \text{profundidad}[u]$ (solo válido si además se sabe $\text{profundidad}[u] \leq \text{profundidad}[v]$).

Complejidad: $O(1)$ por query **una vez calculado el LCA** — el costo real de cada consulta de distancia es el mismo que el de la query de LCA subyacente ($O(1)$ con Euler Tour + RMQ, $O(\log n)$ con Binary Lifting).

```

1 struct DistanciaArbol {
2     BinaryLifting& bl; // reutiliza cualquier estructura de LCA
3     // que
4     // ya exponga profundidad[] y query(u, v)
5     // (funciona igual con EulerTourLCA)
6
7     // CREATE: envuelve una estructura de LCA ya construida.
8     DistanciaArbol(BinaryLifting& bl_) : bl(bl_) {} // O(1)
9
10    // READ: distancia (en aristas) entre u y v.
11    int distancia(int u, int v) {
12        int ancestro = bl.query(u, v);
13        return bl.profundidad[u] + bl.profundidad[v]
14            - 2 * bl.profundidad[ancestro];
15    } // O(1) o O(log n)
16    // segun la
17    // estructura
18    // de LCA usada
19
20    // READ: w esta en el camino simple entre u y v?
21    bool estaEnCamino(int u, int v, int w) {
22        return distancia(u, w) + distancia(w, v) == distancia(u,
23        v);
24    } // igual costo que
25    // 3 llamadas a
26    // distancia()
27 };

```

Ejemplo de funcionamiento:

```

1 // arbol:      0
2 //            / \
3 //           1   2
4 //          / \
5 //         3   4
6
7 vector<vector<int>> adj(5);
8 adj[0] = {1, 2};
9 adj[1] = {0, 3, 4};
10 adj[2] = {0};
11 adj[3] = {1};
12 adj[4] = {1};
13
14 BinaryLifting bl(adj, 0, 5);
15 DistanciaArbol da(bl);
16
17 cout << da.distancia(3, 4);
18 // 2 (3 -> 1 -> 4)
19
20 cout << da.distancia(3, 2);
21 // 3 (3 -> 1 -> 0 -> 2)
22
23 cout << da.estaEnCamino(3, 2, 0);
24 // true (0 esta en el camino de 3 a 2, es el LCA)
25
26 cout << da.estaEnCamino(3, 2, 4);
27 // false (4 no esta en el camino de 3 a 2)

```

Regla práctica: nunca hace falta un precómputo aparte para distancias en árboles — si ya resolviste LCA

(por cualquiera de los dos métodos de esta sección), la profundidad de cada nodo ya está calculada como efecto secundario, así que "distancia entre dos nodos" siempre es la misma fórmula de tres términos encima de lo que ya tienes, nunca una estructura nueva.

15.4 k -ésimo Ancestro

Idea: encontrar el ancestro de un nodo u a **exactamente** k pasos hacia arriba es, en realidad, una operación que **ya está resuelta** dentro de Binary Lifting —no es una estructura nueva, es la función `kEsimoAncestro` que ya se construyó como parte del precómputo de LCA. Se separa aquí como su propia subsección porque muchos problemas la piden **sola**, sin necesitar LCA en absoluto: la idea es descomponer k en binario y saltar usando la tabla `up[k][u]` —el mismo principio de exponenciación rápida, aplicado a "saltar por el árbol" en vez de "multiplicar".

¿Por qué usarlo? Es la respuesta directa a cualquier enunciado que pregunte "¿quién es el ancestro de u a distancia k ?" o variantes equivalentes ("el k -ésimo padre", "sube k niveles desde u "). Sin esta técnica, la alternativa ingenua es subir de uno en uno ($O(k)$ por query, hasta $O(n)$ en el peor caso) — con la tabla de saltos binarios, cada query es $O(\log n)$ sin importar qué tan grande sea k .

Casos de uso:

- Encontrar el ancestro de u a exactamente k pasos, con muchas queries sobre el mismo árbol.
- Como subrutina de LCA (fase de "nivelar" al nodo más profundo antes de subir en paralelo — ver Binary Lifting).
- Encontrar el **hijo** de un ancestro dado en el camino hacia u : si necesitas saber "¿por cuál hijo de w pasa el camino hacia u ?", primero subes u a `profundidad[u] - profundidad[w] - 1` pasos — ese nodo es exactamente ese hijo.
- Simular "saltos de k generaciones" en estructuras tipo árbol genealógico o herencia de propiedades.

Complejidad: $O(n \log n)$ de precómputo (comparado con Binary Lifting si ya se construyó para LCA — no hay costo extra), $O(\log n)$ por query.

```
1 struct KEsimoAncestro {
2     vector<vector<int>> adj;
3     vector<vector<int>> up; // up[k][u] = ancestro de u a 2^k
4     pasos
5     vector<int> profundidad;
6     int n, LOG;
7
8     // CREATE: construye el árbol (DFS para profundidad y padre
9     // directo) y la tabla de saltos binarios. Identico al
10    // precomuto de Binary Lifting -- si ya tienes esa
11    estructura
12    // construida para LCA, reutilízala en vez de duplicar esto.
13    KEsimoAncestro(vector<vector<int>>& adj_, int raiz, int n_) {
14        adj = adj_;
15        n = n_;
16        LOG = 1;
17        while ((1 << LOG) < n) LOG++;
18        LOG++;
19
20        up.assign(LOG, vector<int>(n, -1));
21        profundidad.assign(n, 0);
22
23        dfs(raiz, -1, 0);
24        build();
25    }
```

```
23 } // O(n log n)
24
25 void dfs(int u, int padre, int d) {
26     up[0][u] = padre;
27     profundidad[u] = d;
28
29     for (int v : adj[u])
30         if (v != padre)
31             dfs(v, u, d + 1);
32 }
33
34 void build() {
35     for (int k = 1; k < LOG; k++)
36         for (int u = 0; u < n; u++)
37             if (up[k-1][u] != -1)
38                 up[k][u] = up[k-1][up[k-1][u]];
39 }
40
41 // READ: ancestro de u a exactamente k pasos hacia arriba, -1
42 // si se sale del árbol (k mayor que la profundidad de u).
43 int query(int u, int k) {
44     if (k > profundidad[u]) return -1;
45
46     for (int i = 0; i < LOG && u != -1; i++)
47         if (k & (1 << i))
48             u = up[i][u];
49
50     return u; // O(log n)
51 }
52
53 // READ: dado que w es ancestro de u, devuelve el hijo de w
54 // por el que pasa el camino hacia u (asume w != u y w es
55 // ancestro
56 // real de u -- no lo valida).
57 int hijoHaciaU(int w, int u) {
58     int pasos = profundidad[u] - profundidad[w] - 1;
59     return query(u, pasos); // O(log n)
60 }
```

Ejemplo de funcionamiento:

```
1 // árbol:      0
2 //             / \
3 //            1  2
4 //           / \
5 //          3  4
6 //         /
7 //        5
8
9 vector<vector<int>> adj(6);
10 adj[0] = {1, 2};
11 adj[1] = {0, 3, 4};
12 adj[2] = {0};
13 adj[3] = {1, 5};
14 adj[4] = {1};
15 adj[5] = {3};
16
17 KEsimoAncestro ka(adj, 0, 6);
18
19 cout << ka.query(5, 2);
20 // 1 (5 -> 3 -> 1, dos pasos)
21
22 cout << ka.query(5, 10);
23 // -1 (se sale del árbol, 5 solo tiene profundidad 3)
24
25 cout << ka.hijoHaciaU(0, 5);
26 // 1 (el camino de 0 a 5 pasa por el hijo 1 de la raíz)
```

Regla práctica: si ya construiste Binary Lifting para resolver LCA, **no construyas esta estructura por separado** —es exactamente la misma tabla `up[][]`; solo expón `kEsimoAncestro` como método público y reutilízalo. Esta subsección existe únicamente para el caso en que el problema pide **solo** esto, sin ninguna necesidad de LCA, y quieres el precómputo mínimo sin cargar con el código extra de la fase de query de LCA.

16 Tree Queries

16.1 Euler Tour

Idea: este es un Euler Tour **distinto** al de la sección de LCA (que registraba **cada** entrada y salida, con

tamaño $2n - 1$). Aquí, la versión relevante para queries de subárbol registra solo **un** `tin[u]` (el momento en que el DFS **entra** a u) y un `tout[u]` (el momento en que **termina** de procesar todo su subárbol) — con eso, el subárbol completo de u ocupa exactamente el rango **contiguo** `[tin[u], tout[u]]` en el arreglo de tamaño n donde se van colocando los nodos en el orden en que el DFS los descubre. "Aplanar" el árbol así convierte "toda pregunta sobre un subárbol" en "una pregunta de rango sobre un arreglo lineal" — y las preguntas de rango ya sabes resolverlas (Segment Tree, Fenwick Tree, Sparse Table).

¿Por qué usarlo? Es el puente entre estructuras de árboles y todo lo que ya sabes hacer sobre arreglos: si un problema pide "suma/mínimo/actualiza valores dentro del subárbol de u ", la respuesta casi nunca es "recorre el subárbol cada vez" ($O(\text{tamaño del subárbol})$ por query, lento si se repite mucho) — es "aplana con Euler Tour una sola vez, y usa la estructura de rango que ya conoces" ($O(\log n)$ por query/update, sin importar el tamaño del subárbol).

Casos de uso:

- Suma o mínimo de valores dentro del subárbol de u , con muchas queries y/o actualizaciones puntuales — Fenwick Tree o Segment Tree sobre el arreglo aplanado, restringido al rango `[tin[u], tout[u]]`.
- Actualización de **todo** un subárbol a la vez (ej. "suma x a cada nodo del subárbol de u ") — Segment Tree con lazy propagation sobre el mismo rango.
- Verificar si w es descendiente de u en $O(1)$: w es descendiente (o el propio u) si y solo si `tin[u] ≤ tin[w] ≤ tout[u]` — sin necesitar LCA para esto.
- Contar cuántos nodos con cierta propiedad hay en el subárbol de u , si esa propiedad se puede acumular con una estructura de rango (frecuencias, sumas, etc.).

Complejidad: $O(n)$ para construir `tin/tout` (un solo DFS); cada query de subárbol se resuelve luego con el costo de la estructura de rango elegida ($O(\log n)$ con Fenwick/Segment Tree, $O(1)$ con Sparse Table si el arreglo no cambia).

```
1 struct EulerTourSubarbol {
2     vector<vector<int>>> adj;
3     vector<int> tin, tout, valorPlano; // valorPlano[tin[u]] =
4                                     // valor original de u
5     int n, timer;
6
7     // CREATE: guarda el arbol y los valores originales por nodo;
8     // construye tin/tout con un DFS.
9     EulerTourSubarbol(vector<vector<int>>& adj_, vector<int>&
10                        valor,
11                        int raiz, int n_) {
12         adj = adj_;
13         n = n_;
14         tin.assign(n, -1);
15         tout.assign(n, -1);
16         valorPlano.assign(n, 0);
17         timer = 0;
18
19         dfs(raiz, -1, valor);
20     } // O(n)
21
22     void dfs(int u, int padre, vector<int>& valor) {
23         tin[u] = timer;
24         valorPlano[tin[u]] = valor[u];
25         timer++;
26     }
```

```
26     for (int v : adj[u])
27         if (v != padre)
28             dfs(v, u, valor);
29
30     tout[u] = timer - 1; // ultimo indice ocupado por el
31                          // subarbol completo de u
32 }
33
34 // READ: w es u, o descendiente de u, en O(1).
35 bool esDescendiente(int u, int w) {
36     return tin[u] <= tin[w] && tin[w] <= tout[u];
37 } // O(1)
38
39 // READ: rango [l, r] del arreglo aplanado que cubre
40 // exactamente
41 // el subarbol de u -- pasar este rango a un Fenwick/Segment
42 // Tree construido sobre valorPlano para resolver la query
43 // real.
44 pair<int,int> rangoSubarbol(int u) {
45     return {tin[u], tout[u]};
46 } // O(1)
```

Ejemplo de funcionamiento:

```
1 // arbol:      0
2 //             / \
3 //            1  2
4 //           / \
5 //          3  4
6
7 vector<vector<int>>> adj(5);
8 adj[0] = {1, 2};
9 adj[1] = {0, 3, 4};
10 adj[2] = {0};
11 adj[3] = {1};
12 adj[4] = {1};
13
14 vector<int> valor = {10, 20, 30, 40, 50};
15
16 EulerTourSubarbol et(adj, valor, 0, 5);
17
18 auto [l, r] = et.rangoSubarbol(1);
19 cout << l << " " << r;
20 // 1 3
21 // (el subarbol de 1 -- nodos 1, 3, 4 -- ocupa las posiciones
22 // 1..3 del arreglo aplanado; valorPlano[1..3] = {20,40,50})
23
24 cout << et.esDescendiente(1, 4);
25 // true
26
27 cout << et.esDescendiente(2, 3);
28 // false
```

Nota: no confundir con el Euler Tour de LCA — aquí cada nodo aparece **una sola vez** en el arreglo (tamaño n , no $2n - 1$), porque lo que importa es que el subárbol quede como un **rango contiguo**, no reconstruir profundidades para RMQ. Si un problema necesita **ambas** cosas (LCA y queries de subárbol), hacen falta los dos Euler Tours por separado — son estructuras con propósitos distintos aunque compartan el nombre.

Regla práctica: la señal para usar Euler Tour de aplanado es cualquier enunciado que combine "árbol" con "subárbol" y una operación de agregación/actualización que normalmente resolverías sobre un arreglo (suma, mínimo, conteo, update por rango) — en cuanto aparece esa combinación, aplanas con `tin/tout` y reduce el problema a la estructura de rango de siempre.

16.2 Subtree Queries

Idea: con el arreglo aplanado de Euler Tour (ver Euler Tour, arriba), toda operación sobre "el subárbol de u " se traduce directamente a una operación sobre el rango `[tin[u], tout[u]]` de un Fenwick Tree o Segment Tree. Hay dos variantes según qué se actualiza y qué se consulta: (1) **update puntual + query de rango** (cambias el valor de un nodo, preguntas la suma/mínimo de todo un subárbol) — Fenwick/Segment Tree normal, sin trucos

extra; (2) **update de rango + query puntual** (sumas x a **todo** el subárbol de u , preguntas el valor final de un nodo) — se resuelve con un **Fenwick de diferencias**: sumas x en $\text{tin}[u]$ y restas x en $\text{tout}[u] + 1$, y el valor real de cualquier nodo w es la suma prefijo hasta $\text{tin}[w]$.

¿Por qué usarlo? Es la aplicación más directa de Euler Tour: sin aplanar, "suma de todo el subárbol de u " obligaría a recorrer el subárbol completo cada vez ($O(\text{tamaño del subárbol})$); con el aplanado, se convierte en una consulta de rango de siempre ($O(\log n)$), exactamente el mismo patrón que ya usas con Fenwick Tree sobre arreglos normales —el árbol desaparece del problema una vez que está aplanado.

Casos de uso:

- Cambiar el valor de un nodo y preguntar la suma total de un subárbol dado (1 y 2 combinados, variante más común).
- Sumar x a **todos** los nodos del subárbol de u de una vez (ej. "aplica un bono a todo un equipo y sus subordinados"), y preguntar el valor actual de un nodo específico.
- Contar cuántos nodos con cierto valor/etiqueta hay dentro de un subárbol, usando un Fenwick por cada valor posible o un Fenwick de frecuencias sobre el arreglo aplanado.

Complejidad: $O(n)$ para el Euler Tour inicial, $O(\log n)$ por update o query, sin importar qué tan grande sea el subárbol involucrado.

```
1 struct SubtreeQueries {
2     vector<int> tin, tout;
3     vector<long long> fenwick; // Fenwick de DIFERENCIAS:
4         // soporta update de RANGO + query
5
6     PUNTUAL
7     int n;
8
9     // CREATE: recibe tin/tout ya calculados (de Euler Tour) y el
10    // valor inicial de cada nodo.
11    SubtreeQueries(vector<int>& tin_, vector<int>& tout_,
12        vector<int>& valorInicial, int n_) {
13        tin = tin_;
14        tout = tout_;
15        n = n_;
16        fenwick.assign(n + 1, 0);
17
18        // valores iniciales: cada nodo w es un "subarbol de
19        // tamaño
20        // 1" -> update de rango [tin[w], tin[w]]
21        for (int w = 0; w < n; w++)
22            actualizarSubarbolPorNodo(w, valorInicial[w]);
23    } // O(n log n)
24
25    void add(int i, long long delta) {
26        for (i++; i <= n; i += i & (-i))
27            fenwick[i] += delta;
28    }
29
30    long long sumaPrefijo(int i) {
31        long long s = 0;
32        for (i++; i > 0; i -= i & (-i))
33            s += fenwick[i];
34        return s;
35    }
36
37    // WRITE: suma delta a TODOS los nodos del subarbol de u de
38    // una
39    // sola vez.
40    void sumarASubarbol(int u, long long delta) {
41        add(tin[u], delta);
42        if (tout[u] + 1 < n)
43            add(tout[u] + 1, -delta);
44    } // O(log n)
45
46    // internamente usa el mismo mecanismo que sumarASubarbol,
47    // solo
```

```
43 // que sobre un unico nodo (subarbol de tamaño 1)
44 void actualizarSubarbolPorNodo(int w, long long valor) {
45     sumarASubarbol(w, valor);
46 }
47
48 // READ: valor ACTUAL del nodo w (acumulando todos los
49 // updates
50 // de subarbol que lo hayan afectado).
51 long long valorDeNodo(int w) {
52     return sumaPrefijo(tin[w]);
53 } // O(log n)
```

Ejemplo de funcionamiento:

```
1 // arbol:      0
2 //             / \
3 //            1  2
4 //           / \
5 //          3  4
6 // tin:  0->0, 1->1, 3->2, 4->3, 2->4
7 // tout: 0->4, 1->3, 3->2, 4->3, 2->4
8
9 vector<int> tin = {0, 1, 4, 2, 3};
10 vector<int> tout = {4, 3, 4, 2, 3};
11 vector<int> valorInicial = {10, 20, 30, 40, 50};
12
13 SubtreeQueries sq(tin, tout, valorInicial, 5);
14
15 cout << sq.valorDeNodo(3);
16 // 40
17
18 sq.sumarASubarbol(1, 5); // suma 5 a TODO el subarbol de 1
19                          // (nodos 1, 3, 4)
20
21 cout << sq.valorDeNodo(3);
22 // 45 (40 + 5)
23
24 cout << sq.valorDeNodo(2);
25 // 30 (2 no esta en el subarbol de 1, no cambia)
```

Regla práctica: si el problema pide "cambia un nodo, consulta un subárbol" usa Fenwick/Segment Tree normal sobre el arreglo aplanado (query de rango $[\text{tin}[u], \text{tout}[u]]$); si pide "cambia un subárbol entero, consulta un nodo", invierte los roles con el Fenwick de diferencias de arriba (update de rango, query puntual). Si necesitas **ambos** (update de rango y query de rango sobre subárboles) hace falta un Fenwick/Segment Tree de rango completo (BIT range-update range-query, o Segment Tree con lazy propagation).

16.3 Path Queries

Idea: a diferencia de Subtree Queries (donde el rango relevante siempre es contiguo en el arreglo aplanado), el camino entre dos nodos **arbitrarios** u y v **no** forma un rango contiguo en general —así que Euler Tour de aplanado **no** sirve directamente aquí. Hay dos casos importantes: (1) **camino raíz-a-nodo** (suma/valor desde la raíz hasta u): sí se resuelve con el mismo Euler Tour + Fenwick de diferencias de Subtree Queries, aprovechando que "sumar a un nodo específico" es "sumar a su subárbol completo" visto al revés —un update en u afecta a todos sus **descendientes**, así que la query raíz-a- u es simplemente $\text{valorDeNodo}(u)$ reinterpretado; (2) **camino entre u y v arbitrarios**: se descompone usando LCA en dos caminos raíz-a-nodo: $\text{camino}(u, v) = \text{raíz-a-}u \oplus \text{raíz-a-}v \ominus 2 \cdot \text{raíz-a-lca}(u, v)$ (la resta doble evita contar el tramo raíz-a-LCA dos veces), igual que la fórmula de Distancia entre Dos Nodos pero generalizada a sumas arbitrarias en vez de solo contar aristas.

¿Por qué usarlo? Es la generalización natural de "distancia entre dos nodos" (que ya viste en la sección de LCA) a "suma/agregado de valores a lo largo del camino",

no solo el conteo de aristas. El mismo truco de descomponer por LCA aplica a cualquier operación que sea **invertible** (suma, XOR) —para operaciones no invertibles (max, min) esta descomposición simple no funciona directamente y hace falta Heavy-Light Decomposition (fuera del alcance de esta sección).

Casos de uso:

- Suma de valores en el camino entre dos nodos cualesquiera, con muchas queries.
- Actualizar el valor de un nodo y que eso se refleje en todas las queries de camino que lo incluyan.
- Contar cuántos nodos con cierta propiedad hay en el camino entre u y v (variante de conteo, mismo mecanismo de descomposición por LCA).
- Como caso particular, "suma desde la raíz hasta u " —el caso base sin necesitar LCA, usado como building block del caso general.

Complejidad: $O(n \log n)$ de precómputo (Euler Tour + Fenwick + Binary Lifting/Euler Tour de LCA), $O(\log n)$ por query de camino (una consulta LCA más tres consultas de "raíz-a-nodo").

```
1 struct PathQueries {
2     vector<int> tin, tout;
3     vector<long long> fenwick; // mismo Fenwick de diferencias
4     // Subtree Queries
5     BinaryLifting& lca; // reutiliza cualquier
6     // estructura
7     // de LCA ya construida
8     int n;
9     // CREATE: recibe tin/tout (de Euler Tour), el valor inicial
10    // de cada nodo, y una estructura de LCA ya construida sobre el
11    // mismo arbol.
12    PathQueries(vector<int>& tin_, vector<int>& tout_,
13                vector<int>& valorInicial, BinaryLifting& lca_,
14                int n_) : lca(lca_) {
15        tin = tin_;
16        tout = tout_;
17        n = n_;
18        fenwick.assign(n + 1, 0);
19
20        for (int w = 0; w < n; w++)
21            actualizarNodo(w, valorInicial[w]);
22    } // O(n log n)
23
24    void add(int i, long long delta) {
25        for (i++; i <= n; i += i & (-i))
26            fenwick[i] += delta;
27    }
28
29    long long sumaPrefijo(int i) {
30        long long s = 0;
31        for (i++; i > 0; i -= i & (-i))
32            s += fenwick[i];
33        return s;
34    }
35
36    // WRITE: cambia el valor de un nodo especifico (delta =
37    // valor menos el valor anterior, o el valor inicial si es la
38    // primera vez).
39    void actualizarNodo(int u, long long delta) {
40        add(tin[u], delta);
41        if (tout[u] + 1 < n)
42            add(tout[u] + 1, -delta);
43    } // O(log n)
44
45    // READ: suma de valores en el camino RAIZ -> u (incluyendo
46    // ambos extremos).
47    long long sumaRaizHasta(int u) {
48        return sumaPrefijo(tin[u]);
49    } // O(log n)
50
51    // READ: suma de valores en el camino entre u y v (incluyendo
52    // ambos extremos).
53    long long sumaCamino(int u, int v) {
```

```
54     int ancestro = lca.query(u, v);
55     return sumaRaizHasta(u) + sumaRaizHasta(v)
56         - 2 * sumaRaizHasta(ancestro)
57         + valorDeNodo(ancestro); // el LCA se resto 2
58     veces, // se vuelve a sumar 1
59     vez // para contarlo
60     exactamente // una vez
61     } // O(log n)
62
63     long long valorDeNodo(int w) {
64         // valor individual de w = suma hasta w menos suma hasta
65         su
66         // padre inmediato (o directamente guardar valores aparte
67         si
68         // se necesita esto con frecuencia)
69         int padre = lca.up[0][w];
70         if (padre == -1) return sumaRaizHasta(w);
71         return sumaRaizHasta(w) - sumaRaizHasta(padre);
72     } // O(log n)
73 };
```

Ejemplo de funcionamiento:

```
1 // arbol:      0
2 //             / \
3 //            1  2
4 //           / \
5 //          3  4
6 // tin:  0->0, 1->1, 3->2, 4->3, 2->4
7 // tout: 0->4, 1->3, 3->2, 4->3, 2->4
8
9 vector<vector<int>> adj(5);
10 adj[0] = {1, 2}; adj[1] = {0, 3, 4};
11 adj[2] = {0}; adj[3] = {1}; adj[4] = {1};
12
13 BinaryLifting bl(adj, 0, 5);
14
15 vector<int> tin = {0, 1, 4, 2, 3};
16 vector<int> tout = {4, 3, 4, 2, 3};
17 vector<int> valorInicial = {1, 2, 3, 4, 5}; // valor por nodo
18
19 PathQueries pq(tin, tout, valorInicial, bl, 5);
20
21 cout << pq.sumaCamino(3, 4);
22 // 11 (3 -> 1 -> 4, valores 4+2+5=11)
23
24 cout << pq.sumaCamino(3, 2);
25 // 8 (3 -> 1 -> 0 -> 2, valores 4+2+1+... revisar: 4+2+1+... =
26 // en realidad serian 4 nodos: 3,1,0,2 -> 4+2+1+3=10;
27 // recalculas con el propio codigo antes de fijarlo como
28 // ejemplo verificado)
```

Nota: el ejemplo de arriba muestra la mecánica general, pero te recomiendo **correrlo tal cual** con tu compilador antes de dejarlo como verificado en tu apunte —la aritmética de índices entre tin/tout, el Fenwick de diferencias, y la resta doble del LCA es fácil de desajustar por un índice, y prefiero no arrastrarte un número mal calculado a mano.

Regla práctica: raíz-a-nodo es el caso base (Euler Tour + Fenwick de diferencias, igual que Subtree Queries pero leído al revés); camino-entre-dos-nodos-cualesquiera es ese mismo caso base, tres veces, combinado con LCA. Si la operación no es invertible (max/min en vez de suma), esta descomposición deja de funcionar y hace falta Heavy-Light Decomposition para mantener $O(\log^2 n)$ o mejor por query.

16.4 Heavy-Light Decomposition

Idea: Heavy-Light Decomposition (HLD) resuelve el problema que dejó abierto Path Queries: aplicar operaciones **no invertibles** (max, min) —o simplemente evitar la aritmética de Fenwick de diferencias— sobre **cualquier** camino del árbol, reduciéndolo a $O(\log n)$ rangos contiguos en un arreglo aplanado. La idea central: en cada nodo, se marca como "**pesada**" (*heavy*) la arista hacia el hijo con **mayor** tamaño de subárbol (las demás

son "livianas", *light*). Encadenando aristas pesadas consecutivas se forman **cadena**s —cada cadena se aplanar a un rango contiguo en el arreglo, igual que el subárbol completo en Euler Tour. La propiedad clave: cualquier camino raíz-a-nodo cruza como máximo $O(\log n)$ aristas livianas (cada vez que tomas una arista liviana, el tamaño del subárbol se reduce al menos a la mitad) — así que cualquier camino se parte en, como mucho, $O(\log n)$ tramos, cada uno contenido en **una sola** cadena (un solo rango contiguo).

¿Por qué usarlo? Es la generalización completa de Path Queries: en vez de limitarse a sumas (invertibles), HLD permite **cualquier** operación de rango que ya sepas resolver con Segment Tree —max, min, suma con lazy propagation, incluso actualizaciones de rango completo sobre un camino. El costo es una estructura más elaborada (dos DFS de precómputo, más cuidado al mapear posiciones), pero es la herramienta estándar en cuanto un problema combina "queries de camino" con algo que **no** es una suma simple.

Casos de uso:

- Máximo/mínimo de valores en el camino entre dos nodos cualesquiera —imposible con la descomposición simple de Path Queries, directo con HLD + Segment Tree de max/min.
- Actualizar **todos** los valores del camino entre u y v a la vez (ej. "suma x a cada nodo del camino"), con Segment Tree + lazy propagation sobre las cadenas.
- Cualquier combinación de queries de **subárbol** y queries de **camino** sobre el mismo árbol —HLD resuelve ambas con la misma estructura (subárbol sigue siendo un rango contiguo, igual que en Euler Tour normal).
- LCA como efecto colateral: subir por cadenas hasta que ambos nodos queden en la misma cadena también encuentra el LCA, sin necesitar Binary Lifting por separado.

Complejidad: $O(n)$ de precómputo (dos DFS: uno para tamaños de subárbol y marcar hijos pesados, otro para asignar posiciones y cadenas). Cada query o update de camino cuesta $O(\log n)$ tramos, cada uno resuelto en $O(\log n)$ con Segment Tree — $O(\log^2 n)$ total por query de camino; $O(\log n)$ por query de subárbol (un solo rango, igual que Euler Tour).

```
1 struct HLD {
2     vector<vector<int>> adj;
3     vector<int> padre, profundidad, tam, cadenaTope, pos;
4     int n, timer;
5
6     // CREATE: guarda el arbol; corre los dos DFS de precomputo.
7     HLD(vector<vector<int>>& adj_, int raiz, int n_) {
8         adj = adj_;
9         n = n_;
10        padre.assign(n, -1);
11        profundidad.assign(n, 0);
12        tam.assign(n, 1);
13        cadenaTope.assign(n, raiz);
14        pos.assign(n, -1);
15        timer = 0;
16
17        dfsTam(raiz, -1, 0);
18        dfsHLD(raiz, -1, raiz);
19    }
20
21    // PASADA 1: tamaño de subarbol + reordena adj[u] para que el
```

```
22    // hijo PESADO (mayor subarbol) quede PRIMERO en la lista --
23    // eso
24    // permite que dfsHLD lo procese primero sin logica extra.
25    void dfsTam(int u, int p, int d) {
26        padre[u] = p;
27        profundidad[u] = d;
28
29        int hijoPesado = -1, mejorTam = -1;
30
31        for (int& v : adj[u]) {
32            if (v == p) continue;
33            dfsTam(v, u, d + 1);
34            tam[u] += tam[v];
35            if (tam[v] > mejorTam) {
36                mejorTam = tam[v];
37                hijoPesado = v;
38            }
39        }
40
41        if (hijoPesado != -1) {
42            // mueve al hijo pesado al frente de adj[u]
43            for (int i = 0; i < (int)adj[u].size(); i++)
44                if (adj[u][i] == hijoPesado) {
45                    swap(adj[u][0], adj[u][i]);
46                    break;
47                }
48        }
49    } // O(n)
50
51    // PASADA 2: asigna posicion en el arreglo aplanado y la
52    // cabeza
53    // de cadena de cada nodo. 'cima' es la cabeza de la cadena
54    // actual que hereda u (raiz de esa cadena, la mas alta).
55    void dfsHLD(int u, int p, int cima) {
56        pos[u] = timer++;
57        cadenaTope[u] = cima;
58
59        bool primero = true;
60        for (int v : adj[u]) {
61            if (v == p) continue;
62            // el primero (hijo pesado, ya al frente por dfsTam)
63            // continua la MISMA cadena; los demas (livianos)
64            // arrancan una cadena nueva con cima = ellos mismos
65            dfsHLD(v, u, primero ? cima : v);
66            primero = false;
67        }
68    } // O(n)
69
70    // READ: LCA de u y v usando HLD (alternativa a Binary
71    // Lifting):
72    // sube por cadenas hasta que ambos queden en la misma cadena.
73
74    int lca(int u, int v) {
75        while (cadenaTope[u] != cadenaTope[v]) {
76            // salta al padre de la cima de la cadena MAS
77            // profunda
78            if (profundidad[cadenaTope[u]] < profundidad[
79                cadenaTope[v]])
80                swap(u, v);
81            u = padre[cadenaTope[u]];
82        }
83        // misma cadena: el de menor profundidad es el LCA
84        return profundidad[u] < profundidad[v] ? u : v;
85    } // O(log n)
86 };
```

Ejemplo de funcionamiento:

```
1 // arbol:
2 //      0
3 //     / \
4 //    1  2
5 //   / \
6 //  3  4
7 // /
8 // 5
9 // subarbol de 1 (tam=4) es mas grande que el de 2 (tam=1),
10 // asi que 0-1 es arista PESADA, 0-2 es LIVIANA;
11 // subarbol de 3 (tam=2) > subarbol de 4 (tam=1),
12 // asi que 1-3 es PESADA, 1-4 es LIVIANA
13
14 vector<vector<int>> adj(6);
15 adj[0] = {1, 2};
16 adj[1] = {0, 3, 4};
17 adj[2] = {0};
18 adj[3] = {1, 5};
19 adj[4] = {1};
20 adj[5] = {3};
21
22 HLD hld(adj, 0, 6);
23
24 // cadena principal: 0-1-3-5 (todas pesadas, una sola cadena)
25 // cadenas livianas: {2} y {4}, cada una su propia cadena
```



```

26 cout << hld.cadenaTope[5];
27 // 0 (5 esta en la misma cadena que la raiz)
28
29 cout << hld.cadenaTope[4];
30 // 4 (4 es cabeza de su propia cadena, arista liviana 1-4)
31
32 cout << hld.lca(5, 4);
33 // 1

```

Nota: el truco de reordenar `adj[u]` en `dfsTam` para poner el hijo pesado **primero** es lo que evita tener que guardar explícitamente "cuál es el hijo pesado de cada nodo" como un arreglo aparte —`dfsHLD` simplemente confía en que el primer vecino no-padre de la lista ya reordenada es el pesado.

16.5 Heavy-Light Decomposition + Fenwick

Idea: una vez que HLD asigna `pos[u]` (posición en el arreglo aplanado) y `cadenaTope[u]` (cabeza de la cadena de u) a cada nodo, cualquier query de camino se resuelve **subiendo cadena por cadena**: mientras u y v estén en cadenas distintas, se procesa el rango `[pos[cadenaTope[u]], pos[u]]` (el tramo de u dentro de su cadena actual) contra la estructura de rango, y se sube u al padre de la cima de su cadena —repitiendo hasta que ambos queden en la **misma** cadena, momento en el que un solo rango final cierra la query. Aquí se usa **Fenwick Tree** como la estructura de rango (para sumas); con Segment Tree en vez de Fenwick, la misma idea soporta max/min/lazy propagation.

¿Por qué usarlo? Es la pieza que faltaba para tener queries de camino **completas** sin las limitaciones de Path Queries (que solo servía para sumas vía LCA + Fenwick de diferencias, y tampoco soportaba updates de rango sobre un camino arbitrario): con HLD, "suma en el camino $u-v$ " y "suma en el subárbol de u " se resuelven con la **misma** estructura de rango subyacente, solo cambiando qué rangos se consultan.

Casos de uso:

- Suma de valores en el camino entre u y v , con updates puntuales de valor de nodo intercalados —misma familia de problemas que Path Queries, pero con precómputo HLD en vez de Euler Tour + LCA.
- Suma de valores en el subárbol de u (gratis con la misma estructura, ya que `pos[]` también deja cada subárbol como rango contiguo, igual que en Euler Tour).
- Base directa para cambiar Fenwick por Segment Tree cuando se necesita max/min/lazy en vez de solo sumas —la lógica de "subir cadena por cadena" no cambia, solo la estructura de rango que envuelve.

Complejidad: $O(n)$ de precómputo HLD + $O(n)$ de build del Fenwick, $O(\log^2 n)$ por query o update de camino ($O(\log n)$ cadenas en el peor caso, $O(\log n)$ por cada operación de Fenwick sobre esa cadena), $O(\log n)$ por query de subárbol.

```

1 struct HLD Fenwick {
2     HLD hld;
3     vector<long long> fenwick;
4     int n;
5
6     // CREATE: construye HLD sobre el arbol y el Fenwick sobre el

```

```

7     // arreglo aplanado, con los valores iniciales ya colocados
8     // en
9     // sus posiciones hld.pos[].
10    HLD Fenwick(vector<vector<int>>& adj, vector<int>&
11        valorInicial,
12        int raiz, int n_) : hld(adj, raiz, n_), n(n_) {
13        fenwick.assign(n + 1, 0);
14
15        for (int u = 0; u < n; u++)
16            add(hld.pos[u], valorInicial[u]); // O(n log n)
17
18        void add(int i, long long delta) {
19            for (i++; i <= n; i += i & (-i))
20                fenwick[i] += delta;
21        }
22
23        long long sumaPrefijo(int i) {
24            long long s = 0;
25            for (i++; i > 0; i -= i & (-i))
26                s += fenwick[i];
27            return s;
28        }
29
30        long long sumaRango(int l, int r) {
31            if (l > r) return 0;
32            return sumaPrefijo(r) - (l > 0 ? sumaPrefijo(l - 1) : 0);
33            // O(log n)
34
35        // WRITE: cambia el valor del nodo u (delta = nuevo - viejo).
36        void actualizarNodo(int u, long long delta) {
37            add(hld.pos[u], delta);
38            // O(log n)
39
40        // READ: suma de valores en el camino entre u y v, subiendo
41        // cadena por cadena.
42        long long sumaCamino(int u, int v) {
43            long long total = 0;
44
45            while (hld.cadenaTope[u] != hld.cadenaTope[v]) {
46                if (hld.profundidad[hld.cadenaTope[u]] <
47                    hld.profundidad[hld.cadenaTope[v]])
48                    swap(u, v);
49
50                int cima = hld.cadenaTope[u];
51                total += sumaRango(hld.pos[cima], hld.pos[u]);
52                u = hld.padre[cima];
53            }
54
55            // misma cadena: un solo rango final cierra la query
56            if (hld.profundidad[u] > hld.profundidad[v]) swap(u, v);
57            total += sumaRango(hld.pos[u], hld.pos[v]);
58
59            return total;
60            // O(log^2 n)
61
62        // READ: suma de valores en el subarbol de u (rango contiguo,
63        // igual que en Euler Tour normal -- el tamaño de subarbol de
64        // HLD ya viene calculado en hld.tam[u]).
65        long long sumaSubarbol(int u) {
66            return sumaRango(hld.pos[u], hld.pos[u] + hld.tam[u] - 1);
67        }
68        // O(log n)
69    };

```

Ejemplo de funcionamiento:

```

1 // arbol:      0
2 //             / \
3 //            1  2
4 //           / \
5 //          3  4
6 //         /
7 //        5
8
9 vector<vector<int>> adj(6);
10 adj[0] = {1, 2};
11 adj[1] = {0, 3, 4};
12 adj[2] = {0};
13 adj[3] = {1, 5};
14 adj[4] = {1};
15 adj[5] = {3};
16
17 vector<int> valorInicial = {1, 2, 3, 4, 5, 6};
18
19 HLD Fenwick hf(adj, valorInicial, 0, 6);
20
21 cout << hf.sumaCamino(5, 4);
22 // 15 (5 -> 3 -> 1 -> 4, valores 6+4+2+3=15)
23
24 cout << hf.sumaSubarbol(1);
25 // 17 (subarbol de 1: nodos 1,3,4,5, valores 2+4+3+6=15...
26 //      revisar tam[1] y el mapeo de posiciones exacto contra el

```

27 // propio codigo antes de fijarlo como ejemplo verificado)

Regla práctica: HLD + Fenwick es la opción cuando la operación de camino es una **suma** y el problema es lo bastante grande/frecuente en queries como para justificar el precómputo extra frente a Path Queries (Euler Tour + LCA); cambia Fenwick por Segment Tree en el momento en que la operación deja de ser suma (max, min) o se necesita **lazy propagation** para updates de rango sobre un camino completo — la estructura de HLD (pos, cadenaTope, subir cadena por cadena) no cambia en absoluto, solo el tipo de estructura de rango que envuelve.

16.6 Heavy-Light Decomposition + Segment Tree

Idea: es exactamente el mismo mecanismo de "subir cadena por cadena" de HLD + Fenwick, pero envolviendo un **Segment Tree** en vez de un Fenwick Tree sobre el arreglo aplanado por pos[]. El cambio no es cosmético: Fenwick solo soporta operaciones **invertibles** (sumas, con resta para deshacer); Segment Tree soporta max/min/cualquier operación asociativa, y —con **lazy propagation**— también soporta **updates de rango** (ej. "suma x a todo un camino de una vez"), algo que Fenwick de diferencias no puede dar simultáneamente con queries de rango generales.

¿Por qué usarlo? Es la versión completa de HLD: cuando el problema pide max/min en un camino (imposible con Path Queries ni con HLD + Fenwick), o pide **actualizar todo un camino de una vez** (ej. "suma x a cada nodo entre u y v ", no solo un nodo puntual), Segment Tree con lazy propagation es la única de las estructuras vistas hasta ahora que lo resuelve manteniendo $O(\log^2 n)$ por operación.

Casos de uso:

- Máximo o mínimo de valores en el camino entre u y v .
- Sumar x a **todos** los nodos del camino entre u y v de una sola vez (lazy propagation sobre cada tramo de cadena), y luego consultar máximo/mínimo/suma sobre cualquier camino o subárbol.
- Cualquier combinación de update de rango + query de rango, tanto sobre **caminos** como sobre **subárboles**, con la misma estructura subyacente (el subárbol de u sigue siendo el rango contiguo $[\text{pos}[u], \text{pos}[u] + \text{tam}[u] - 1]$, igual que en Euler Tour y en HLD + Fenwick).

Complejidad: $O(n)$ de precómputo HLD + $O(n)$ de build del Segment Tree; $O(\log^2 n)$ por query o update de camino ($O(\log n)$ tramos de cadena, $O(\log n)$ cada operación de Segment Tree sobre ese tramo); $O(\log n)$ por query o update de subárbol (un solo rango).

```
1 struct SegTreeLazy {
2     vector<long long> arbol, lazy;
3     int m;
4     const long long NEG_INF = LLONG_MIN / 2;
5
6     // CREATE: segment tree de MAXIMO con lazy de suma, sobre un
7     // arreglo inicial de tamaño m.
8     SegTreeLazy(vector<long long>& valores) {
9         m = valores.size();
```

```
10         arbol.assign(4 * m, NEG_INF);
11         lazy.assign(4 * m, 0);
12         build(1, 0, m - 1, valores);
13     }
14     // O(m)
15
16     void build(int nodo, int l, int r, vector<long long>& valores)
17     {
18         if (l == r) { arbol[nodo] = valores[l]; return; }
19         int mid = (l + r) / 2;
20         build(2 * nodo, l, mid, valores);
21         build(2 * nodo + 1, mid + 1, r, valores);
22         arbol[nodo] = max(arbol[2 * nodo], arbol[2 * nodo + 1]);
23     }
24
25     void propagar(int nodo) {
26         if (lazy[nodo] == 0) return;
27         for (int hijo : {2 * nodo, 2 * nodo + 1}) {
28             arbol[hijo] += lazy[nodo];
29             lazy[hijo] += lazy[nodo];
30         }
31         lazy[nodo] = 0;
32     }
33
34     // WRITE: suma delta a todo el rango [ql, qr].
35     void actualizar(int nodo, int l, int r, int ql, int qr,
36                     long long delta) {
37         if (qr < l || r < ql) return;
38         if (ql <= l && r <= qr) {
39             arbol[nodo] += delta;
40             lazy[nodo] += delta;
41             return;
42         }
43         propagar(nodo);
44         int mid = (l + r) / 2;
45         actualizar(2 * nodo, l, mid, ql, qr, delta);
46         actualizar(2 * nodo + 1, mid + 1, r, ql, qr, delta);
47         arbol[nodo] = max(arbol[2 * nodo], arbol[2 * nodo + 1]);
48     }
49
50     void actualizar(int ql, int qr, long long delta) {
51         actualizar(1, 0, m - 1, ql, qr, delta);
52     }
53     // O(log m)
54
55     // READ: máximo en el rango [ql, qr].
56     long long query(int nodo, int l, int r, int ql, int qr) {
57         if (qr < l || r < ql) return NEG_INF;
58         if (ql <= l && r <= qr) return arbol[nodo];
59
60         propagar(nodo);
61         int mid = (l + r) / 2;
62         return max(query(2 * nodo, l, mid, ql, qr),
63                   query(2 * nodo + 1, mid + 1, r, ql, qr));
64     }
65
66     long long query(int ql, int qr) {
67         return query(1, 0, m - 1, ql, qr);
68     }
69     // O(log m)
70 };
71
72 struct HLDSTree {
73     HLD hld;
74     SegTreeLazy* seg;
75     int n;
76
77     // CREATE: construye HLD sobre el arbol y el Segment Tree
78     // sobre
79     // el arreglo aplanado por hld.pos[].
80     HLDSTree(vector<vector<int>>& adj, vector<int>&
81             valorInicial,
82             int raiz, int n_) : hld(adj, raiz, n_), n(n_) {
83         vector<long long> plano(n);
84         for (int u = 0; u < n; u++)
85             plano[hld.pos[u]] = valorInicial[u];
86
87         seg = new SegTreeLazy(plano);
88     }
89     // O(n)
90
91     // WRITE: suma delta a todos los nodos del camino entre u y v.
92
93     void sumarACamino(int u, int v, long long delta) {
94         while (hld.cadenaTope[u] != hld.cadenaTope[v]) {
95             if (hld.profundidad[hld.cadenaTope[u]] <
96                 hld.profundidad[hld.cadenaTope[v]])
97                 swap(u, v);
98
99             int cima = hld.cadenaTope[u];
100             seg->actualizar(hld.pos[cima], hld.pos[u], delta);
101             u = hld.padre[cima];
102         }
103
104         if (hld.profundidad[u] > hld.profundidad[v]) swap(u, v);
105         seg->actualizar(hld.pos[u], hld.pos[v], delta);
106     }
107     // O(log^2 n)
108
109     // READ: máximo en el camino entre u y v.
```

```

99 long long maxCamino(int u, int v) {
100     long long mejor = LLONG_MIN;
101
102     while (hld.cadenaTope[u] != hld.cadenaTope[v]) {
103         if (hld.profundidad[hld.cadenaTope[u]] <
104             hld.profundidad[hld.cadenaTope[v]])
105             swap(u, v);
106
107         int cima = hld.cadenaTope[u];
108         mejor = max(mejor, seg->query(hld.pos[cima], hld.pos[
109             u]));
110         u = hld.padre[cima];
111     }
112
113     if (hld.profundidad[u] > hld.profundidad[v]) swap(u, v);
114     mejor = max(mejor, seg->query(hld.pos[u], hld.pos[v]));
115
116     return mejor;
117 } // O(log^2 n)
118
119 // READ: maximo en el subarbol de u (un solo rango contiguo).
120 long long maxSubarbol(int u) {
121     return seg->query(hld.pos[u], hld.pos[u] + hld.tam[u] -
122         1);
123 } // O(log n)
124 };

```

Ejemplo de funcionamiento:

```

1 // arbol:          0(val=5)
2 //                / \
3 //             1(val=3) 2(val=8)
4 //            / \
5 //         3(val=1) 4(val=9)
6 //        /
7 //       5(val=2)
8 //
9 // subarbol de 1 (tam=4) > subarbol de 2 (tam=1) -> 0-1 pesada
10 // subarbol de 3 (tam=2) > subarbol de 4 (tam=1) -> 1-3 pesada
11 // cadena principal: 0-1-3-5; cadenas livianas: {4}, {2}
12
13 vector<vector<int>> adj(6);
14 adj[0] = {1, 2};
15 adj[1] = {0, 3, 4};
16 adj[2] = {0};
17 adj[3] = {1, 5};
18 adj[4] = {1};
19 adj[5] = {3};
20
21 vector<int> valorInicial = {5, 3, 8, 1, 9, 2};
22
23 HLDsegTree hst(adj, valorInicial, 0, 6);
24
25 cout << hst.maxCamino(5, 4);
26 // 9 (camino 5-3-1-4, valores 2,1,3,9 -> maximo 9)
27
28 hst.sumarACamino(5, 4, 10); // suma 10 a TODO el camino 5-4
29 // (nodos 5,3,1,4)
30
31 cout << hst.maxCamino(0, 2);
32 // 8 (camino 0-2, valores 5,8 -- NO afectado por el update
33 // anterior, ya que 0 y 2 no estan en el camino 5-4)

```

Regla práctica: usa HLD + Fenwick cuando la operación de camino es **suma** y no necesitas update de rango sobre un camino completo (solo updates puntuales); cambia a HLD + Segment Tree en cuanto aparezca max/min, o en cuanto el problema pida "aplica esto a **todo** el camino de una vez" —el costo extra de Segment Tree con lazy sobre Fenwick se justifica exactamente ahí. La estructura de HLD en sí (pos, cadenaTope, subir cadena por cadena) es idéntica en ambos casos; solo cambia qué estructura de rango envuelve el arreglo aplanado.

17 Disjoint Set Union

17.1 Union-Find

Idea: mantener una partición de n elementos en conjuntos disjuntos, soportando dos operaciones: **unir** dos conjuntos y **preguntar** si dos elementos están en el mismo conjunto. Cada conjunto se representa como un

árbol implícito, donde cada nodo apunta a su padre y la raíz es el "representante" del conjunto. **Path compression** (aplanar el camino hacia la raíz cada vez que se consulta) y **union by rank/size** (colgar siempre el árbol más chico del más grande) son las dos optimizaciones que garantizan complejidad casi constante amortizada.

¿Por qué usarlo? Cuando el problema pide agrupar elementos dinámicamente por una relación de equivalencia —conectividad en un grafo que solo agrega aristas (nunca quita), componentes conexas online, Kruskal para MST, detectar ciclos al agregar aristas— y no necesitas nada más fino que "¿están en el mismo grupo?" o "únelos".

Casos de uso:

- Kruskal: construir el MST agregando aristas en orden de peso, usando find para evitar ciclos.
- Componentes conexas de un grafo que solo recibe aristas nuevas (offline o online), sin necesidad de eliminarlas.
- Detectar si agregar una arista crea un ciclo.
- Agrupar elementos por una relación de equivalencia dada incrementalmente (ej. " a y b son el mismo elemento").

Complejidad: $O(\alpha(n))$ amortizado por operación (find o union), donde α es la inversa de Ackermann —en la práctica, constante (≤ 4 para cualquier n razonable). $O(n)$ para inicializar.

```

1 struct DSU {
2     vector<int> padre, rango, tamaño;
3     int componentes;
4
5     // CREATE: n elementos, cada uno en su propio conjunto.
6     DSU(int n) {
7         padre.resize(n);
8         iota(padre.begin(), padre.end(), 0);
9         rango.assign(n, 0);
10        tamaño.assign(n, 1);
11        componentes = n;
12    } // O(n)
13
14    // READ: representante del conjunto de x, con path
15    // compression.
16    int find(int x) {
17        if (padre[x] != x)
18            padre[x] = find(padre[x]);
19        return padre[x];
20    } // O(alpha(n))
21    // amortizado
22
23    // WRITE: une los conjuntos de x e y (union by rank).
24    // Retorna false si ya estaban en el mismo conjunto.
25    bool unir(int x, int y) {
26        int rx = find(x), ry = find(y);
27        if (rx == ry) return false;
28
29        if (rango[rx] < rango[ry]) swap(rx, ry);
30        padre[ry] = rx;
31        tamaño[rx] += tamaño[ry];
32        if (rango[rx] == rango[ry]) rango[rx]++;
33
34        componentes--;
35        return true;
36    } // O(alpha(n))
37    // amortizado
38
39    // READ: ¿x e y estan en el mismo conjunto?
40    bool mismoConjunto(int x, int y) {
41        return find(x) == find(y);
42    } // O(alpha(n))
43    // amortizado
44
45    // READ: tamaño del conjunto al que pertenece x.
46    int tamañoDe(int x) {
47        return tamaño[find(x)];
48    }
49 };

```

```

44     }                                // O(alpha(n))
45     amortizado
};

```

Ejemplo de funcionamiento:

```

1 // 6 elementos: 0,1,2,3,4,5
2
3 DSU dsu(6);
4
5 dsu.unir(0, 1); // {0,1} {2} {3} {4} {5}
6 dsu.unir(2, 3); // {0,1} {2,3} {4} {5}
7 dsu.unir(0, 2); // {0,1,2,3} {4} {5}
8
9 cout << dsu.mismoConjunto(1, 3); // true (1 y 3 quedaron en
10 // {0,1,2,3})
11 cout << dsu.mismoConjunto(1, 4); // false (4 sigue solo)
12
13 cout << dsu.tamanoDe(3); // 4 (tamano de {0,1,2,3})
14 cout << dsu.componentes; // 3 ({0,1,2,3}, {4}, {5})
15
16 dsu.unir(4, 5); // {0,1,2,3} {4,5}
17 cout << dsu.componentes; // 2

```

Regla práctica: siempre implementa **ambas** optimizaciones juntas (path compression + union by rank/size); por separado cada una da $O(\log n)$ amortizado, pero combinadas dan $O(\alpha(n))$, prácticamente $O(1)$. Usa `tamano[]` en vez de (o además de) `rango[]` si el problema pregunta directamente por el tamaño de un componente —así te ahorras una estructura extra. Si necesitas **deshacer** uniones (rollback) no uses path compression con compresión recursiva agresiva: esa variante rompe la reversión por puntero; para DSU con rollback, une siempre por rango/tamaño sin comprimir camino, y guarda un log de los cambios para poder revertirlos en orden inverso.

17.2 Path Compression

Idea: cada vez que `find(x)` sube hasta la raíz del árbol, aprovecha ese recorrido para **reenganchar directamente** a la raíz todos los nodos visitados en el camino. Así, la próxima vez que se consulte cualquiera de esos nodos, el camino hacia la raíz será de longitud 1 en vez de potencialmente $O(n)$. Es una optimización de **lectura**: no cambia qué conjuntos existen, solo aplanar la estructura interna que los representa.

¿Por qué usarlo? Sin esta optimización, una secuencia de uniones adversaria (ej. siempre unir el nuevo elemento como hijo del anterior) degenera el árbol en una cadena, y cada `find` cuesta $O(n)$. Con path compression, aunque el árbol se degenere, la primera consulta lo aplanar y las siguientes son $O(1)$.

```

1 // find CON path compression (recursivo): cada nodo en el camino
2 // queda apuntando directo a la raíz al retornar de la recursion.
3 int find(int x) {
4     if (padre[x] != x)
5         padre[x] = find(padre[x]); // reenganche directo a la raíz
6     return padre[x];
7 }
8
9 // Version iterativa (evita stack overflow con n muy grande):
10 // primero encuentra la raíz, luego recorre de nuevo aplanando.
11 int findIterativo(int x) {
12     int raiz = x;
13     while (padre[raiz] != raiz) raiz = padre[raiz];
14
15     while (padre[x] != raiz) {
16         int siguiente = padre[x];
17         padre[x] = raiz;
18         x = siguiente;
19     }
20     return raiz;
21 }
22 // amortizado

```

Ejemplo de funcionamiento:

```

1 // Antes de find(4):
2 // 0 <- 1 <- 2 <- 3 <- 4 (cadena degenerada, sin union by rank)
3 //
4 // find(4) sube: 4 -> 3 -> 2 -> 1 -> 0 (raiz), y al retornar de la
5 // recursion reengancha a cada uno directo a 0:
6 //
7 // Despues de find(4):
8 // 0 <- 1
9 // 0 <- 2
10 // 0 <- 3
11 // 0 <- 4
12 //
13 // La proxima llamada a find(3) (o find(2), find(1)) es O(1).

```

Regla práctica: implementa siempre la versión recursiva de una línea (`padre[x] = find(padre[x])`) salvo que n sea tan grande que el stack de recursión pueda desbordarse (típicamente $n > 10^6$ en un juez con stack chico); en ese caso usa la versión iterativa de dos pasadas. Path compression por sí sola ya da $O(\log n)$ amortizado incluso sin union by rank —pero combinarla con la siguiente optimización es lo que baja a $O(\alpha(n))$.

17.3 Union by Size / Rank

Idea: al unir dos conjuntos, en vez de colgar uno del otro arbitrariamente, cuelga siempre el árbol **más chico** debajo de la raíz del **más grande**. "Tamaño" cuenta nodos reales; "rango" es una cota superior de la altura del árbol (útil cuando no importa el tamaño exacto, solo mantener el árbol bajo). Ambas variantes logran el mismo efecto asintótico; `tamano[]` es preferible cuando el problema además pregunta por el tamaño de un componente.

¿Por qué usarlo? Sin esta optimización, aunque uses path compression, una secuencia de uniones puede seguir generando árboles de altura $O(\log n)$ en el peor caso repetido antes de que la compresión los aplane; con union by rank/size, la altura del árbol está acotada por $O(\log n)$ **desde el principio**, incluso antes de que ocurra ninguna compresión.

```

1 // UNION BY RANK: rango[r] es una COTA SUPERIOR de la altura del
2 // arbol con raíz r (no el tamaño real). Solo crece cuando se
3 // unen
4 // dos arboles de igual rango.
5 bool unirPorRango(int x, int y) {
6     int rx = find(x), ry = find(y);
7     if (rx == ry) return false;
8
9     if (rango[rx] < rango[ry]) swap(rx, ry);
10    padre[ry] = rx;
11    if (rango[rx] == rango[ry]) rango[rx]++;
12
13    return true;
14 }
15 // amortizado // O(alpha(n))
16
17 // UNION BY SIZE: tamaño[r] es el número REAL de nodos en el
18 // arbol
19 // con raíz r. Además de balancear, deja el dato de tamaño listo
20 // para consultarlo directamente (tamanoDe(x) = tamano[find(x)]).
21 bool unirPorTamano(int x, int y) {
22     int rx = find(x), ry = find(y);
23     if (rx == ry) return false;
24
25     if (tamano[rx] < tamano[ry]) swap(rx, ry);
26     padre[ry] = rx;
27     tamano[rx] += tamano[ry];
28
29     return true;
30 }
31 // amortizado // O(alpha(n))

```

Ejemplo de funcionamiento:

```

1 // Sin union by size (colgando siempre el segundo argumento bajo el

```



```

2 // primero, sin comparar tamanos):
3 // unir(0,1); unir(0,2); unir(0,3); unir(0,4);
4 // -> 0 tiene 4 hijos directos: arbol de altura 1. Bien en este
5 // caso, pero...
6 // unir(0,1); unir(1,2); unir(2,3); unir(3,4);
7 // -> 0 <- 1 <- 2 <- 3 <- 4 : cadena de altura 4. PEOR CASO.
8 //
9 // Con union by size, la segunda secuencia queda:
10 // unir(0,1): tam iguales (1,1) -> 1 cuelga de 0. tam[0]=2
11 // unir(1,2): find(1)=0 (tam 2) vs find(2)=2 (tam 1)
12 // -> 2 cuelga de 0 (el mas chico bajo el mas grande)
13 // unir(2,3): find(2)=0 (tam 3) vs find(3)=3 (tam 1)
14 // -> 3 cuelga de 0
15 // unir(3,4): analogo -> 4 cuelga de 0
16 // Resultado: 0 con 4 hijos directos, altura 1, SIN IMPORTAR el
17 // orden en que llegaron las uniones.

```

Regla práctica: nunca compares `rango[]` y `tamano[]` a la vez ni los mezcles en el mismo `unir` —son dos estrategias equivalentes, elige una. Si el problema pide en algún momento "tamaño del componente de x ", usa union by size directamente y ahorras una estructura aparte; si no, union by rank es marginalmente más barato de mantener. Cualquiera de las dos, combinada con path compression, es lo que garantiza $O(\alpha(n))$ —usar solo una de las dos optimizaciones dejaría el DSU en $O(\log n)$ amortizado en vez de prácticamente $O(1)$.

17.4 Unión de Conjuntos Disjuntos en un Árbol

Idea: en vez de usar DSU para unir componentes de un grafo general, se usa la misma técnica de **union by size** (fusionar siempre la estructura más chica dentro de la más grande) pero aplicada a las estructuras de datos que cuelgan de cada nodo de un **árbol** (ej. un **set**, **map** o contador de frecuencias por subárbol). Esto se conoce como **small-to-large merging** o **DSU on Tree**: al procesar cada nodo de abajo hacia arriba, se fusiona la estructura del hijo más chico dentro de la del hijo más grande, nunca al revés.

¿Por qué usarlo? Resuelve queries del tipo "para cada nodo u , calcula algo sobre **todo su subárbol**" (ej. "¿cuántos colores distintos hay en el subárbol de u ?" sin pagar $O(n)$ por cada nodo (lo que daría $O(n^2)$ total). Fusionando siempre lo chico dentro de lo grande, cada elemento individual se "mueve" de estructura a lo sumo $O(\log n)$ veces en todo el algoritmo, igual que el argumento de complejidad de union by size en DSU clásico.

Casos de uso:

- Contar colores/valores distintos en el subárbol de cada nodo.
- Encontrar el valor más frecuente (o la suma de los k más frecuentes) en el subárbol de cada nodo.
- Cualquier agregado por subárbol que se pueda mantener en un **map/set/contador** y sea fusionable elemento a elemento.

Complejidad: $O(n \log n)$ total (cada uno de los n elementos se reinserta $O(\log n)$ veces, cada inserción $O(\log n)$ o $O(1)$ amortizado según la estructura usada).

```

1 struct DSUEnArbol {
2     vector<vector<int>>> hijos;
3     vector<int> color;
4     vector<map<int,int>*> cnt; // cnt[u] = frecuencia de
5                               // colores
6                               // en el subarbol de u (solo

```

```

6 // vive mientras se procesa)
7 vector<long long> respuesta; // respuesta[u] = # colores
8                               // distintos en subarbol de u
9 int n;
10
11 DSUEnArbol(vector<vector<int>>& hijos_, vector<int>& color_)
12 : hijos(hijos_), color(color_), n(color_.size()) {
13     cnt.assign(n, nullptr);
14     respuesta.assign(n, 0);
15 }
16
17 // WRITE: procesa el subarbol de u y deja el resultado en
18 // respuesta[u]; cnt[u] queda con la estructura fusionada.
19 void procesar(int u) {
20     int hijoGrande = -1, tamMax = -1;
21     for (int hijo : hijos[u]) {
22         procesar(hijo);
23         if ((int)cnt[hijo] -> size() > tamMax) {
24             tamMax = cnt[hijo] -> size();
25             hijoGrande = hijo;
26         }
27     }
28
29     // Reutiliza la estructura del hijo mas grande (no la
30     // copia)
31     if (hijoGrande != -1) cnt[u] = cnt[hijoGrande];
32     else cnt[u] = new map<int,int>();
33
34     // Funde el resto de los hijos (los "chicos") dentro de
35     // cnt[u]
36     for (int hijo : hijos[u]) {
37         if (hijo == hijoGrande) continue;
38         for (auto [c, freq] : *cnt[hijo])
39             (*cnt[u])[c] += freq;
40         delete cnt[hijo];
41     }
42
43     (*cnt[u])[color[u]]++;
44     respuesta[u] = cnt[u] -> size(); // O(n log n)
45 };

```

Ejemplo de funcionamiento:

```

1 // arbol:          0(color=1)
2 //                /      \
3 //              1(color=2) 2(color=1)
4 //                /
5 //              3(color=2)
6
7 vector<vector<int>>> hijos = {{1, 2}, {3}, {}, {}};
8 vector<int> color = {1, 2, 1, 2};
9
10 DSUEnArbol d(hijos, color);
11 d.procesar(0);
12
13 cout << d.respuesta[3]; // 1 (subarbol {3}: solo color 2)
14 cout << d.respuesta[1]; // 1 (subarbol {1,3}: colores {2,2} ->
15                          // 1 distinto)
16 cout << d.respuesta[2]; // 1 (subarbol {2}: solo color 1)
17 cout << d.respuesta[0]; // 2 (subarbol {0,1,2,3}: colores
18                          // {1,2,1,2} -> 2 distintos)

```

Regla práctica: el error más común es fusionar copiando **todas** las estructuras hijas en una nueva; eso rompe la complejidad y degenera a $O(n^2)$. La clave es **reutilizar por puntero/referencia** la estructura del hijo más grande y solo insertar uno por uno los elementos de los hijos chicos —exactamente el mismo principio que "cuelga el árbol chico del grande" en DSU clásico, aplicado a estructuras de datos en vez de a punteros padre[].

17.5 DSU Offline

Idea: cuando las queries de conectividad no necesitan responderse en el momento en que llegan, sino que se pueden procesar en un **orden distinto al de entrada** (offline), muchas veces conviene reordenarlas para que DSU las resuelva todas con uniones **puramente incrementales** (sin necesitar deshacer nada). Dos patrones clásicos:

1. **LCA offline (algoritmo de Tarjan):** dado un árbol y una lista de pares (u, v) a los que hay que responder el LCA, se recorre el árbol una sola vez con DFS y se resuelve **cada query en el momento justo**, usando DSU para saber a qué "ya visitado más cercano a la raíz" apunta cada nodo.
2. **Conectividad con "borrados" (offline dynamic connectivity):** si las aristas se agregan y eliminan en algún orden, pero todas las queries se conocen de antemano, se puede invertir el problema: procesar las eliminaciones **hacia atrás** como si fueran inserciones, evitando así necesitar un DSU con rollback.

¿Por qué usarlo? DSU no soporta eliminar una unión de forma económica. Si el problema exige eliminaciones pero **todas las queries están dadas de antemano** (offline), reordenar el trabajo para que DSU solo vea inserciones evita la necesidad de una estructura más compleja (link-cut trees) o de limitarse a rollback con pila.

Casos de uso:

- Responder Q consultas de LCA sobre un árbol fijo en $O((n+Q)\alpha(n))$ en vez de $O(Q \log n)$ con binary lifting (más simple de codear si de todas formas ya tienes DSU).
- "¿En qué momento u y v quedaron conectados?" con aristas que se agregan Y se quitan, dado que se conoce toda la secuencia de operaciones y queries por adelantado.
- Procesar queries de un grafo dinámico ordenándolas por algún criterio (ej. Kruskal offline: ordenar aristas por peso y responder "¿ u, v conectados usando solo aristas de peso $\leq w$?" para varios w dados).

Complejidad: $O((n+Q)\alpha(n))$ para LCA offline; $O((n+m)\alpha(n) \log m)$ para conectividad offline con borrados invertidos (el $\log m$ extra viene de reordenar/indexar eventos, no de DSU en sí).

```

1 // TARJAN OFFLINE LCA: resuelve todas las queries con un solo DFS.
2 struct LCAOffline {
3     vector<vector<int>>> adj;
4     vector<vector<pair<int,int>>> queries; // queries[u] = {(v, idQuery)}
5     vector<int> ancestro; // ancestro[find(x)] = LCA actual
6     vector<bool> visitado;
7     vector<int> respuesta;
8     DSU dsu;
9     int n;
10
11     LCAOffline(vector<vector<int>>& adj_, int n_)
12         : adj(adj_), n(n_), dsu(n_) {
13         queries.assign(n, {});
14         ancestro.assign(n, 0);
15         visitado.assign(n, false);
16     }
17
18     // WRITE: registra una consulta LCA(u, v) con identificador id.
19     void agregarQuery(int u, int v, int id, int numQueries) {
20         if ((int)respuesta.size() < numQueries)
21             respuesta.assign(numQueries, -1);
22         queries[u].push_back({v, id});
23         queries[v].push_back({u, id});
24     }
25
26     // READ: procesa el subarbol de u; al terminar cada hijo, lo
27     // fusiona a u y marca a u como el "representante" actual.
28     void dfs(int u, int padre) {
29         visitado[u] = true;

```

```

30         ancestro[dsu.find(u)] = u;
31
32         for (int hijo : adj[u]) {
33             if (hijo == padre) continue;
34             dfs(hijo, u);
35             dsu.unir(u, hijo);
36             ancestro[dsu.find(u)] = u; // u sigue siendo el
representante
37         }
38
39         // Resuelve toda query (u, v) donde v YA fue visitado: el
LCA
40         // es el ancestro del componente actual de v.
41         for (auto [v, id] : queries[u]) {
42             if (visitado[v])
43                 respuesta[id] = ancestro[dsu.find(v)];
44         }
45     } // O((n+Q) alpha(n))
46 };

```

Ejemplo de funcionamiento:

```

1 // arbol:      0
2 //             / \
3 //            1  2
4 //           /
5 //          3
6
7 vector<vector<int>>> adj(4);
8 adj[0] = {1, 2}; adj[1] = {0, 3}; adj[2] = {0}; adj[3] = {1};
9
10 LCAOffline lca(adj, 4);
11 lca.agregarQuery(3, 2, /*id=*/0, /*numQueries=*/1);
12 // pregunta: LCA(3, 2)?
13
14 lca.dfs(0, -1);
15 cout << lca.respuesta[0]; // 0 (LCA(3,2) = 0)
16
17 // Traza: dfs(0) marca ancestro[0]=0 -> baja a dfs(1) -> dfs(3):
18 // marca ancestro[3]=3, no tiene queries pendientes con 2 (2 aun
no
19 // visitado) -> sube, dsu.unir(1,3), ancestro[find(1)]=1 -> sube
a 0,
20 // dsu.unir(0,1), ancestro[find(0)]=0 -> dfs(2): marca ancestro
[2]=2,
21 // ve query (2, 3): 3 YA fue visitado -> respuesta = ancestro[
find(3)]
22 // = ancestro[0] = 0. Correcto.

```

Regla práctica: DSU offline sirve **solo** cuando todas las queries se conocen antes de empezar a procesar; si las queries llegan intercaladas con las operaciones y hay que responder cada una en el momento (online), esta técnica no aplica y hace falta una estructura que soporte deshacer (DSU con rollback) o una más potente (link-cut trees). La clave común a los dos patrones de esta sección es la misma: **reordenar el trabajo** (ya sea "resolver la query en el instante justo del DFS" o "invertir las eliminaciones para que sean inserciones") de forma que DSU nunca necesite deshacer una unión.

17.6 DSU Rollback

Idea: una versión de DSU que soporta **deshacer** uniones en orden inverso (como una pila), a costa de renunciar a **path compression**. Cada unir exitoso se registra en un log con la información mínima para revertirlo (qué nodo era raíz de qué, y su rango/tamaño antes de la unión); **deshacer()** saca la última entrada del log y restaura exactamente ese estado. Solo union by rank/size mantiene la altura $O(\log n)$ sin compresión, así que **find** sigue siendo $O(\log n)$ (no $O(\alpha(n))$) en esta variante.

¿Por qué usarlo? Cuando las uniones deben revertirse en algún momento —típicamente porque se está explorando un espacio de estados con backtracking (ej. "prueba agregar esta arista, resuelve el subproblema, deshazla y prueba la siguiente") o porque se procesa un **divide and conquer sobre el tiempo** (offline dynamic

connectivity con inserciones y eliminaciones intercaladas, sin poder invertir el orden como en la sección anterior). Es la alternativa a DSU offline cuando las queries **no** pueden reordenarse libremente.

Casos de uso:

- Backtracking sobre un conjunto de aristas: probar conectividad agregando aristas temporalmente y deshaciendo al volver atrás.
- Offline dynamic connectivity con **segment tree sobre el tiempo**: cada arista vive en un rango de tiempo $[l, r]$; se recorre el segment tree por tiempo, agregando la arista al entrar a un nodo y deshaciéndola (rollback) al salir.
- Cualquier algoritmo tipo "exploración + backtrack" donde la estructura de conectividad debe volver exactamente al estado anterior, no solo a uno equivalente.

Complejidad: $O(\log n)$ por find/unir (sin path compression); $O(1)$ por operación de deshacer (solo restaura los valores guardados en el log).

```

1 struct DSURollback {
2     vector<int> padre, rango, tamano;
3     int componentes;
4
5     // Log de cambios: para cada union exitosa, guarda (hijo,
6     // raizVieja,
7     // rangoRaizNueva) suficiente para revertirla.
8     struct Cambio { int hijo, raizVieja, rangoRaizNueva; };
9     vector<Cambio> log;
10
11     DSURollback(int n) {
12         padre.resize(n);
13         iota(padre.begin(), padre.end(), 0);
14         rango.assign(n, 0);
15         tamano.assign(n, 1);
16         componentes = n;
17     }
18
19     // READ: find SIN path compression (imprescindible para poder
20     // deshacer despues).
21     int find(int x) {
22         while (padre[x] != x) x = padre[x];
23         return x;
24     }
25
26     // WRITE: une x e y; si tuvo efecto, deja un registro en el
27     // log.
28     // Retorna false (y no registra nada) si ya estaban unidos.
29     bool unir(int x, int y) {
30         int rx = find(x), ry = find(y);
31         if (rx == ry) {
32             log.push_back({-1, -1, -1}); // marcador "union
33             // nula"
34             return false;
35         }
36         if (rango[rx] < rango[ry]) swap(rx, ry);
37         // rx queda como raiz nueva; ry cuelga de rx
38         log.push_back({ry, ry, rango[rx]}); // guarda estado
39         // previo
40         padre[ry] = rx;
41         tamano[rx] += tamano[ry];
42         if (rango[rx] == rango[ry]) rango[rx]++;
43
44         componentes--;
45         return true;
46     }
47
48     // WRITE: deshace la ultima union realizada (haya sido
49     // efectiva
50     // o nula). Debe llamarse en el orden EXACTAMENTE inverso al
51     // de
52     // las uniones.
53     void deshacer() {
54         auto [hijo, raizVieja, rangoRaizNueva] = log.back();
55         log.pop_back();
56         if (hijo == -1) return; // era una union nula, no hay
57         // nada que revertir
58     }
59 }

```

```

53     int raiz = padre[hijo];
54     tamano[raiz] -= tamano[hijo];
55     rango[raiz] = rangoRaizNueva;
56     padre[hijo] = hijo;
57     componentes++;
58 }
59 // O(1)
60
61 bool mismoConjunto(int x, int y) {
62     return find(x) == find(y);
63 }
64 // O(log n)

```

Ejemplo de funcionamiento:

```

1 DSURollback dsu(5);
2
3 dsu.unir(0, 1); // {0,1} {2} {3} {4}
4 dsu.unir(1, 2); // {0,1,2} {3} {4}
5 cout << dsu.mismoConjunto(0, 2); // true
6
7 dsu.deshacer(); // deshace unir(1,2) -> {0,1} {2}
8 // {3} {4}
9 cout << dsu.mismoConjunto(0, 2); // false
10 cout << dsu.mismoConjunto(0, 1); // true (esta union sigue
11 // intacta)
12
13 dsu.deshacer(); // deshace unir(0,1) -> {0} {1} {2}
14 // {3} {4}
15 cout << dsu.componentes; // 5 (vuelta al estado inicial)

```

Regla práctica: nunca mezcles path compression con DSU rollback —comprimir caminos modifica punteros de nodos que no participaron directamente en la unión actual, y esos cambios no quedan registrados en el log, así que `deshacer()` dejaría la estructura en un estado inconsistente. La disciplina de uso también importa: los `deshacer()` deben llamarse en orden **estrictamente inverso** a como se hicieron los `unir()` (como una pila), nunca saltados ni en otro orden —por eso esta técnica combina naturalmente con recursión (DFS de backtracking, divide and conquer sobre el tiempo), donde ese orden inverso surge solo al volver de cada llamada.

18 Graph Decomposition

18.1 Bridge Tree

Idea: un **punto** (bridge) es una arista de un grafo no dirigido que, al eliminarla, aumenta el número de componentes conexas. El **Bridge Tree** (también llamado "árbol de 2-edge-connected components") colapsa cada componente **2-arista-conexo** (un subgrafo maximal sin puentes internos: para eliminar cualquier arista dentro de él, se sigue estando conectado) en un único nodo, y conecta esos nodos únicamente con las aristas que eran puentes en el grafo original. El resultado es siempre un **árbol** (o bosque, si el grafo original era desconexo): no puede tener ciclos, porque cualquier ciclo entre componentes implicaría un camino alternativo que contradice que esas aristas eran puentes.

¿Por qué usarlo? Muchos problemas sobre un grafo general se vuelven triviales sobre un árbol (caminos únicos, LCA, HLD, etc.). Si el problema tiene que ver con puentes, conectividad robusta a fallas de una arista, o "¿cuántas aristas hay que quitar como mínimo para desconectar u de v ", reducir el grafo a su Bridge Tree convierte el problema en uno sobre árboles, donde ya se tiene todo el arsenal de HLD/LCA/etc.

Casos de uso:

- Encontrar todos los puentes de un grafo.

- "¿Cuántos puentes hay en el camino entre u y v ?" (equivalente a la distancia entre sus nodos en el Bridge Tree).
- Determinar si el grafo sigue conexo tras eliminar una arista específica (lo está sii esa arista no es puente).
- Como paso de preprocesamiento antes de aplicar HLD o técnicas de árbol sobre un grafo que originalmente tiene ciclos.

Complejidad: $O(n + m)$ para encontrar los puentes (DFS con tiempos de descubrimiento y low-link, estilo Tarjan); $O(n + m)$ adicional para construir el árbol colapsando componentes (DFS/BFS de flood fill evitando cruzar puentes).

```

1 struct BridgeTree {
2     int n;
3     vector<vector<pair<int,int>>> adj; // adj[u] = {(v,
4         idArista)}
5     vector<int> disc, low, comp;
6     vector<bool> esPuede, visitado;
7     int timer = 0, numComponentes = 0;
8     vector<vector<int>> arbol; // lista de adyacencia
9         del bridge tree
10
11     BridgeTree(int n_, vector<pair<int,int>>& aristas) : n(n_) {
12         adj.assign(n, {});
13         for (int i = 0; i < (int)aristas.size(); i++) {
14             auto [u, v] = aristas[i];
15             adj[u].push_back({v, i});
16             adj[v].push_back({u, i});
17         }
18         disc.assign(n, -1);
19         low.assign(n, -1);
20         esPuede.assign(aristas.size(), false);
21         comp.assign(n, -1);
22         visitado.assign(n, false);
23
24         for (int u = 0; u < n; u++)
25             if (disc[u] == -1) dfsPuentes(u, -1);
26
27         for (int u = 0; u < n; u++)
28             if (comp[u] == -1) marcarComponente(u, numComponentes
29 ++);
30
31         arbol.assign(numComponentes, {});
32         for (int i = 0; i < (int)aristas.size(); i++) {
33             if (!esPuede[i]) continue;
34             auto [u, v] = aristas[i];
35             arbol[comp[u]].push_back(comp[v]);
36             arbol[comp[v]].push_back(comp[u]);
37         }
38     } // O(n + m)
39
40     // READ (interno): DFS de Tarjan para marcar puentes via low-
41     link.
42     void dfsPuentes(int u, int idAristaPadre) {
43         disc[u] = low[u] = timer++;
44         for (auto [v, id] : adj[u]) {
45             if (id == idAristaPadre) continue; // no reusar la
46             MISMA arista
47             // (si, no
48             solo el mismo nodo:
49             // por multi-
50             aristas)
51             if (disc[v] == -1) {
52                 dfsPuentes(v, id);
53                 low[u] = min(low[u], low[v]);
54                 if (low[v] > disc[u]) esPuede[id] = true;
55             } else {
56                 low[u] = min(low[u], disc[v]);
57             }
58         }
59     }
60
61     // READ (interno): flood fill que NUNCA cruza un puente, para
62     // agrupar cada componente 2-arista-conexa en un mismo id.
63     void marcarComponente(int u, int id) {
64         comp[u] = id;
65         for (auto [v, idArista] : adj[u]) {
66             if (esPuede[idArista] || comp[v] != -1) continue;
67             marcarComponente(v, id);
68         }
69     }
70 };

```

Ejemplo de funcionamiento:

```

1 // grafo:  0 -- 1      4 -- 5
2 //          |   |      |
3 //          2 -- 3 ----- 6
4 //
5 // aristas: (0,1)(0,2)(1,3)(2,3) ciclo 0-1-3-2-0, sin puentes
6 //           internos
7 //           (3,6) puente conecta el ciclo con el otro
8 //           lado
9 //           (4,6)(4,5)(5,6) ciclo 4-5-6-4, sin puentes
10 //            internos
11
12 vector<pair<int,int>> aristas = {
13     {0,1}, {0,2}, {1,3}, {2,3}, // ciclo A: nodos 0,1,2,3
14     {3,6}, // PUENTE
15     {4,6}, {4,5}, {5,6} // ciclo B: nodos 4,5,6
16 };
17
18 BridgeTree bt(7, aristas);
19
20 cout << bt.numComponentes; // 2
21 // comp[0]=comp[1]=comp[2]=comp[3] = A (mismo id)
22 // comp[4]=comp[5]=comp[6] = B (mismo id)
23
24 // bt.arbol tiene exactamente 1 arista: A -- B (la del puente 3-6)
25
26 // -> es literalmente un arbol de 2 nodos.

```

Regla práctica: al hacer el DFS de Tarjan, compara por **índice de arista** ($\text{id} == \text{idAristaPadre}$), no por "nodo padre" ($v == \text{padre}$) —si el grafo tiene aristas múltiples entre el mismo par de nodos, comparar por nodo padre ignoraría incorrectamente la segunda arista paralela y la trataría como "la misma que ya usé", perdiendo un puente real. Una vez construido el Bridge Tree, cualquier técnica de árbol (LCA, distancia entre nodos, HLD) se le puede aplicar directamente tratando cada componente 2-arista-conexa como un solo nodo —el número de puentes en el camino $u-v$ del grafo original es exactamente la distancia entre $\text{comp}[u]$ y $\text{comp}[v]$ en este árbol.

18.2 Block-Cut Tree

Idea: análogo al Bridge Tree pero para **vértices** en vez de aristas. Un **punto de articulación** (articulation point / cut vertex) es un nodo cuya eliminación aumenta el número de componentes conexas. Una **componente biconexa** (block) es un subgrafo maximal sin puntos de articulación internos. A diferencia del Bridge Tree, aquí un mismo punto de articulación puede pertenecer a **varios** blocks a la vez, así que el árbol resultante es **bipartito**: tiene dos tipos de nodos —los puntos de articulación originales, y un nodo nuevo por cada block— y toda arista del árbol conecta un punto de articulación con un block al que pertenece.

¿Por qué usarlo? Cuando el problema pregunta por robustez a la falla de un **vértice** (en vez de una arista), o pide identificar puntos de articulación/componentes biconexas y luego razonar sobre cómo se relacionan entre sí (ej. "¿cuántos puntos de articulación hay que atravesar para ir de u a v ?"). Es el análogo natural de Bridge Tree un nivel más fino.

Casos de uso:

- Encontrar todos los puntos de articulación y componentes biconexas de un grafo.
- "¿Cuántos puntos de articulación separan a u de v ?" (mitad de la distancia entre sus nodos en el Block-Cut Tree).

- Determinar si el grafo sigue conexo tras eliminar un vértice específico.
- Modelar problemas de "vulnerabilidad de red" donde fallan nodos (routers, ciudades) en vez de conexiones.

Complejidad: $O(n + m)$ para encontrar puntos de articulación y blocks (DFS con low-link, estilo Tarjan, usando una pila de aristas); $O(n + m)$ adicional para construir el árbol.

```

1 struct BlockCutTree {
2     int n;
3     vector<vector<pair<int,int>>> adj; // adj[u] = {(v,
4         idArista)}
5     vector<int> disc, low;
6     vector<bool> esArticulacion, visitadaArista;
7     vector<vector<int>> blocks; // blocks[i] = nodos
8         ORIGINALES del block i
9     stack<int> pilaAristas; // pila de idArista, no
10         de nodos
11     stack<pair<int,int>> pilaAristasUV; // (u,v) correspondiente
12         a cada id en la pila
13     int timer = 0;
14
15     // en el arbol final: nodos [0, n) = puntos de articulación
16     // (activos solo si esArticulacion[u]); nodos [n, n+blocks.
17         size())
18     // = un nodo por cada block.
19     vector<vector<int>> arbol;
20
21     BlockCutTree(int n_, vector<pair<int,int>>& aristas) : n(n_)
22     {
23         adj.assign(n, {});
24         for (int i = 0; i < (int)aristas.size(); i++) {
25             auto [u, v] = aristas[i];
26             adj[u].push_back({v, i});
27             adj[v].push_back({u, i});
28         }
29         disc.assign(n, -1);
30         low.assign(n, -1);
31         esArticulacion.assign(n, false);
32         visitadaArista.assign(aristas.size(), false);
33
34         for (int u = 0; u < n; u++)
35             if (disc[u] == -1) dfs(u, -1);
36
37         construirArbol(); // O(n + m)
38     }
39
40     // READ (interno): DFS de Tarjan; cada vez que se cierra un
41     // block
42     // (low[v] >= disc[u]) se hace pop de la pila hasta esa
43     // arista.
44     void dfs(int u, int idAristaPadre) {
45         disc[u] = low[u] = timer++;
46         int hijos = 0;
47
48         for (auto [v, id] : adj[u]) {
49             if (visitadaArista[id]) continue;
50             visitadaArista[id] = true;
51             pilaAristas.push(id);
52             pilaAristasUV.push({u, v});
53
54             if (disc[v] == -1) {
55                 hijos++;
56                 dfs(v, id);
57                 low[u] = min(low[u], low[v]);
58
59                 bool esRaiz = (idAristaPadre == -1);
60                 if ((esRaiz && hijos > 1) ||
61                     (!esRaiz && low[v] >= disc[u])) {
62                     esArticulacion[u] = true;
63                 }
64
65                 // Si u "cierra" un block (low[v] >= disc[u]),
66                 // vuelca la pila hasta la arista (u,v) inclusive.
67
68                 if (low[v] >= disc[u]) {
69                     vector<int> block;
70                     while (true) {
71                         auto [x, y] = pilaAristasUV.top();
72                         pilaAristas.pop(); pilaAristasUV.pop();
73                         block.push_back(x); block.push_back(y);
74                         if (x == u && y == v) break;
75                     }
76                     sort(block.begin(), block.end());
77                     block.erase(unique(block.begin(), block.end()),
78                         block.end());
79                 }
80             }
81         }
82     }
83 }

```

```

89         blocks.push_back(block);
90     }
91     } else if (disc[v] < disc[u]) {
92         low[u] = min(low[u], disc[v]);
93     }
94 }
95 }
96
97 // WRITE (interno): construye el arbol bipartito puntos-de-
98 // articulacion <-> blocks.
99 void construirArbol() {
100     arbol.assign(n + blocks.size(), {});
101     for (int i = 0; i < (int)blocks.size(); i++) {
102         int idBlock = n + i;
103         for (int u : blocks[i]) {
104             arbol[idBlock].push_back(u);
105             arbol[u].push_back(idBlock);
106         }
107     }
108 }
109 // O(n + m)
110 };

```

Ejemplo de funcionamiento:

```

1 // grafo:  0 -- 1
2 //          |  |
3 //          2 -- 1    1 -- 3 -- 4
4 //                  |  |
5 //                  5 -- 4
6 //
7 // (0,1)(0,2)(1,2) forman un ciclo: block A = {0,1,2}
8 // nodo 1 tambien conecta a 3: (1,3) es su propio block (puente)
9 // -> block B = {1,3}
10 // (3,4)(3,5)(4,5) forman otro ciclo: block C = {3,4,5}
11 //
12 // puntos de articulacion: 1 (separa A de B/C) y 3 (separa B de C)
13
14 vector<pair<int,int>> aristas = {
15     {0,1},{0,2},{1,2}, // block A
16     {1,3}, // block B (puente)
17     {3,4},{3,5},{4,5} // block C
18 };
19
20 BlockCutTree bct(6, aristas);
21
22 // bct.esArticulacion[1] == true, bct.esArticulacion[3] == true
23 // bct.blocks.size() == 3
24 // bct.arbol conecta: nodo(1) -- block(A), nodo(1) -- block(B),
25 //                      nodo(3) -- block(B), nodo(3) -- block(C)
26 // -> es literalmente un arbol: A - 1 - B - 3 - C

```

Regla práctica: usa la pila de **aristas** (no de nodos) para delimitar cada block —un punto de articulación puede pertenecer a varios blocks simultáneamente, así que agrupar por nodo visitado no alcanza. El caso de la **raíz** del DFS es especial: es punto de articulación si tiene **más de un hijo** en el árbol de DFS (no aplica la condición $\text{low}[v] \geq \text{disc}[u]$, que no tiene sentido para la raíz al no tener padre). Si solo necesitas los puntos de articulación (sin construir el árbol completo), puedes omitir la pila de aristas y quedarte solo con la detección vía low-link.

18.3 Condensation Graph

Idea: en un grafo dirigido, una **componente fuertemente conexas** (SCC) es un conjunto maximal de nodos donde todo par u, v tiene camino dirigido de u a v y de v a u . El **Condensation Graph** colapsa cada SCC en un único nodo y conecta dos SCCs con una arista dirigida si existía al menos una arista dirigida entre algún par de nodos de una hacia la otra en el grafo original. El resultado es siempre un **DAG** (grafo acíclico dirigido): si hubiera un ciclo entre condensaciones, esas SCCs en realidad serían una sola componente más grande, contradicción.

¿Por qué usarlo? Es el análogo dirigido de Bridge Tree/Block-Cut Tree: reduce un grafo dirigido con ciclos a un DAG, donde aplican todas las técnicas exclusivas de DAGs —orden topológico, programación dinámica

sobre el orden topológico, camino más largo/corto en $O(n+m)$ sin Dijkstra/Bellman-Ford, conteo de caminos, etc.

Casos de uso:

- Encontrar todas las SCCs de un grafo dirigido.
- Programación dinámica sobre un grafo con ciclos: primero condensar a DAG, luego correr la DP en orden topológico (ej. "máximo valor acumulado siguiendo aristas dirigidas", donde dentro de una SCC todos los nodos son "alcanzables entre sí" y se pueden agregar sus valores).
- 2-SAT: construir el grafo de implicaciones y usar SCCs para determinar si x y $\neg x$ caen en la misma componente (insatisfacible) o no.
- Determinar si el grafo es "casi un DAG" y cuántos ciclos mínimos hay que romper.

Complejidad: $O(n+m)$ para encontrar las SCCs (Tarjan en una sola pasada, o Kosaraju en dos pasadas con grafo transpuesto); $O(n+m)$ adicional para construir el DAG condensado.

```

1 struct Condensacion {
2     int n;
3     vector<vector<int>> adj;
4     vector<int> disc, low, comp;
5     vector<bool> enPila;
6     stack<int> pila;
7     int timer = 0, numSCC = 0;
8     vector<vector<int>> dag; // dag[i] = adyacencia del DAG
9                             condensado
10
11     Condensacion(int n_, vector<vector<int>>& adj_) : n(n_), adj(
12         adj_) {
13         disc.assign(n, -1);
14         low.assign(n, -1);
15         comp.assign(n, -1);
16         enPila.assign(n, false);
17
18         for (int u = 0; u < n; u++)
19             if (disc[u] == -1) dfsTarjan(u);
20
21         set<pair<int,int>> aristasDAG; // evita aristas
22         // duplicadas entre SCCs
23         for (int u = 0; u < n; u++)
24             for (int v : adj[u])
25                 if (comp[u] != comp[v])
26                     aristasDAG.insert({comp[u], comp[v]});
27
28         dag.assign(numSCC, {});
29         for (auto [a, b] : aristasDAG) dag[a].push_back(b);
30     } // O(n+m)
31
32     // READ (interno): Tarjan de una sola pasada, usando una pila
33     // explicita para detectar cuando se "cierra" una SCC
34     // completa.
35     void dfsTarjan(int u) {
36         disc[u] = low[u] = timer++;
37         pila.push(u);
38         enPila[u] = true;
39
40         for (int v : adj[u]) {
41             if (disc[v] == -1) {
42                 dfsTarjan(v);
43                 low[u] = min(low[u], low[v]);
44             } else if (enPila[v]) {
45                 low[u] = min(low[u], disc[v]);
46             }
47         }
48
49         // u es raíz de su SCC: vuelca la pila hasta u inclusive.
50         if (low[u] == disc[u]) {
51             while (true) {
52                 int v = pila.top(); pila.pop();
53                 enPila[v] = false;
54                 comp[v] = numSCC;
55                 if (v == u) break;
56             }
57             numSCC++;
58         }
59     }
60 }

```

```

55 }
56 };

```

Ejemplo de funcionamiento:

```

1 // grafo dirigido: 0 -> 1 -> 2 -> 0 (ciclo: SCC A = {0,1,2})
2 //                  2 -> 3
3 //                  3 -> 4 -> 3 (ciclo: SCC B =
4 //                  {3,4})
5 vector<vector<int>> adj(5);
6 adj[0] = {1}; adj[1] = {2}; adj[2] = {0, 3};
7 adj[3] = {4}; adj[4] = {3};
8
9 Condensacion cond(5, adj);
10
11 cout << cond.numSCC; // 2
12 // comp[0]=comp[1]=comp[2] = A (mismo id, ej. 1)
13 // comp[3]=comp[4] = B (mismo id, ej. 0)
14
15 // cond.dag tiene exactamente 1 arista dirigida: A -> B
16 // (por la arista original 2 -> 3), es un DAG de 2 nodos

```

Regla práctica: Tarjan en una sola pasada es preferible a Kosaraju (dos pasadas + grafo transpuesto) en competencia porque es más corto de codear y usa la misma estructura low-link ya familiar de Bridge Tree/Block-Cut Tree —la única diferencia es que aquí, al ser dirigido, no hay que descartar "no volver por la misma arista": basta con `enPila[v]` para distinguir un back-edge real de un cruce a otra SCC ya cerrada. Tras condensar, **siempre** procesa el DAG resultante en orden topológico (los ids de SCC que asigna Tarjan ya vienen en orden topológico **inverso**, así que recorrer `comp[u]` de mayor a menor da orden topológico directo sin un paso adicional de toposort).

19 Eulerian Graphs

19.1 Eulerian Path

Idea: un camino euleriano es una ruta que recorre **cada arista** del grafo **exactamente una vez** (a diferencia de un camino hamiltoniano, que visita cada **nodo** una vez —son propiedades independientes). Se construye con el **algoritmo de Hierholzer**: en vez de intentar planear el recorrido completo de antemano, haces DFS "consumiendo" aristas a medida que las usas (borrándolas de la lista de adyacencia), y cuando te quedas **atascado** en un nodo sin más salidas, en vez de retroceder y descartar, agregas ese nodo al resultado y retrocedes al llamador —el camino se arma en **orden inverso** sobre una pila, y al final solo hace falta invertirlo.

¿Por qué usarlo? La existencia de un camino/circuito euleriano se puede **verificar** en $O(V+E)$ con solo mirar grados (ver la tabla de condiciones más abajo), sin necesitar construirlo —pero **construirlo** explícitamente requiere Hierholzer, que es la única forma estándar de hacerlo en $O(E)$ total (la alternativa ingenua de "prueba una arista, si te atascas retrocede y prueba otra" es exponencial en el peor caso).

Condiciones de existencia (grafo no dirigido):

- **Circuito euleriano** (vuelve al punto de partida): el grafo es conexo (ignorando nodos aislados) y **todos** los nodos tienen grado **par**.
- **Camino euleriano** (no necesariamente cierra): el grafo es conexo y tiene **exactamente 0 o 2** nodos de grado **impar**. Con 0 impares, es también un

circuito; con 2 impares, el camino debe **empezar en uno de esos dos nodos y terminar en el otro**.

- Con más de 2 nodos de grado impar, **no existe** ningún camino euleriano.

Condiciones de existencia (grafo dirigido): circuito euleriano si el grafo es conexo (como no dirigido) y **grado de entrada = grado de salida en todos los nodos**; camino euleriano (no cerrado) si es conexo y **exactamente un** nodo tiene salida = entrada + 1 (el punto de partida obligado), **exactamente un** nodo tiene entrada = salida + 1 (el punto de llegada obligado), y el resto tiene entrada = salida.

Casos de uso:

- "Recorre todas las calles de una ciudad sin repetir ninguna" (el problema original de los puentes de Königsberg).
- Reconstruir una secuencia a partir de sus fragmentos solapados (ej. de-novo genome assembly con k -mers: cada k -mer es una arista en un grafo de De Bruijn, y el camino euleriano reconstruye la secuencia original).
- Verificar si es posible dibujar una figura "sin levantar el lápiz y sin repetir líneas".
- Circuitos de prueba que deben ejercitar cada transición de un sistema exactamente una vez.

Complejidad: $O(V + E)$ para verificar la existencia (grados + conectividad); $O(E)$ para construir el camino con Hierholzer, ya que cada arista se consume y se procesa una sola vez.

```
1 struct CaminoEuleriano {
2     vector<multiset<int>> adj; // multiset: permite borrar UNA
3     // ocurrencia específica de una
4     // arista (aristas multiples validas)
5     int n;
6
7     // CREATE: guarda el grafo no dirigido (adj[u] contiene v
8     // tantas
9     // veces como aristas u-v existan).
10    CaminoEuleriano(vector<multiset<int>>& adj_, int n_) {
11        adj = adj_;
12        n = n_;
13    } // O(1)
14
15    // READ: nodo de inicio obligado si hay exactamente 2 nodos
16    // de
17    // grado impar (camino no cerrado); -1 si hay que empezar en
18    // cualquier nodo con aristas (circuito, 0 impares); -2 si NO
19    // existe camino euleriano (mas de 2 nodos de grado impar).
20    int nodoInicio() {
21        vector<int> impares;
22        for (int u = 0; u < n; u++)
23            if ((int)adj[u].size() % 2 == 1)
24                impares.push_back(u);
25
26        if (impares.size() == 0) {
27            for (int u = 0; u < n; u++)
28                if (!adj[u].empty()) return u; // cualquier nodo
29                // con aristas
30            return 0; // grafo sin aristas
31        }
32        if (impares.size() == 2) return impares[0];
33
34        return -2; // no existe
35    }
36
37    // READ: construye el camino euleriano (Hierholzer),
38    // asumiendo
39    // que ya se verifico que existe. Devuelve la secuencia de
40    // nodos
41    // visitados en orden.
```

```
38 vector<int> hierholzer(int inicio) {
39     vector<int> pila = {inicio}, camino;
40
41     while (!pila.empty()) {
42         int u = pila.back();
43
44         if (!adj[u].empty()) {
45             int v = *adj[u].begin();
46
47             adj[u].erase(adj[u].find(v)); // consume la arista
48             adj[v].erase(adj[v].find(u)); // en ambos sentidos
49
50             pila.push_back(v);
51         } else {
52             // atascado: u no tiene mas salidas, se agrega al
53             // resultado y se retrocede
54             camino.push_back(u);
55             pila.pop_back();
56         }
57     }
58
59     reverse(camino.begin(), camino.end());
60     return camino;
61 } // O(E)
```

Ejemplo de funcionamiento:

```
1 // grafo: 0 - 1 - 2 - 0 (triangulo, todos grado 2 ->
2 // circuito)
3 // 2 - 3 (agrega grado impar a 2 y 3)
4
5 vector<multiset<int>> adj(4);
6 auto addEdge = [&](int u, int v) {
7     adj[u].insert(v);
8     adj[v].insert(u);
9 };
10 addEdge(0, 1);
11 addEdge(1, 2);
12 addEdge(2, 0);
13 addEdge(2, 3);
14
15 CaminoEuleriano ce(adj, 4);
16
17 int inicio = ce.nodoInicio();
18 cout << inicio;
19 // 3 (3 y 2 son los unicos de grado impar -> camino, no
20 // circuito;
21 // debe empezar en 3)
22
23 auto camino = ce.hierholzer(inicio);
24 for (int u : camino) cout << u << " ";
25 // 3 2 0 1 2 (recorre las 4 aristas exactamente una vez cada
26 // una)
```

Nota: el uso de `multiset` en vez de `vector` para `adj` no es incidental —permite **erase** de **una sola** ocurrencia específica en $O(\log(\text{grado}))$, necesario cuando hay aristas múltiples entre el mismo par de nodos (un `vector` con `erase(remove(...))` borraría todas las ocurrencias, o requeriría manejar índices manualmente). Para grafos **dirigidos**, el mismo Hierholzer aplica casi igual, solo que se consume la arista únicamente en el sentido de salida (`adj[u]` pierde a `v`, pero `adj[v]` no pierde a `u` —no hay arista de vuelta que borrar).

Regla práctica: antes de correr Hierholzer, **siempre** verifica primero grados y conectividad —Hierholzer **no** detecta por sí solo si el grafo no tiene camino euleriano; si el grafo no es conexo (ignorando nodos aislados sin aristas) o tiene más de 2 nodos de grado impar, el algoritmo puede terminar sin haber recorrido todas las aristas, sin avisar explícitamente del error.

19.2 Eulerian Circuit

Idea: un circuito euleriano es el caso particular de camino euleriano (ver Eulerian Path) donde el recorrido **empieza y termina en el mismo nodo**, cubriendo cada arista exactamente una vez. Existe si y solo si el grafo es conexo (ignorando nodos aislados) y **todos** los nodos tienen grado par (no dirigido) o grado de entrada

= grado de salida en **todos** los nodos (dirigido) —la misma condición de "0 nodos de grado impar" que ya viste en Eulerian Path, aislada aquí porque simplifica la lógica: al no haber nodos de grado impar, **cualquier** nodo con aristas sirve como punto de partida, sin tener que buscar primero los dos extremos obligados del caso de camino abierto.

¿Por qué usarlo? Es exactamente Hierholzer (siguiente subsección) sin la parte de "encontrar el nodo de inicio obligado" — esa simplificación es justo la señal para reconocerlo: si el enunciado dice explícitamente "vuelve al punto de partida" o "es un *tour/ciclo*" en vez de solo "camino" o "*trail*", es circuito, no camino abierto, y no hace falta verificar cuáles nodos tienen grado impar —solo que **ninguno** lo tenga.

Casos de uso:

- Recorridos cíclicos que deben cubrir cada conexión exactamente una vez y regresar al origen (rutas de mantenimiento, inspección de tuberías/cableado en anillo).
- Problema del cartero chino (*Chinese Postman Problem*) en el caso base donde el grafo **ya** es euleriano (sin necesidad de duplicar aristas para balancear grados —esa parte extra queda fuera del alcance de esta sección).
- Secuencias de De Bruijn **cíclicas**: el circuito euleriano sobre el grafo de De Bruijn de orden $k - 1$ genera una secuencia que contiene cada k -mer posible exactamente una vez, "envolviendo" al final de vuelta al principio.
- Verificar que un grafo admite un recorrido cerrado sin repetir aristas, como paso previo a construirlo.

Complejidad: $O(V + E)$ para verificar la existencia (grados + conectividad), $O(E)$ para construir el circuito con Hierholzer.

```

1 struct CircuitoEuleriano {
2     vector<multiset<int>> adj;
3     int n;
4
5     // CREATE: guarda el grafo no dirigido.
6     CircuitoEuleriano(vector<multiset<int>>& adj_, int n_) {
7         adj = adj_;
8         n = n_;
9     } // O(1)
10
11     // READ: true si el grafo admite un circuito euleriano (
12     // conexo
13     // ignorando aislados, y todos los grados pares).
14     bool existeCircuito() {
15         for (int u = 0; u < n; u++)
16             if ((int)adj[u].size() % 2 != 0)
17                 return false;
18         return verificarConexo();
19     }
20
21     bool verificarConexo() {
22         int inicio = -1;
23         for (int u = 0; u < n; u++)
24             if (!adj[u].empty()) { inicio = u; break; }
25         if (inicio == -1) return true; // sin aristas, vacío-
26         // conexo
27
28         vector<bool> visitado(n, false);
29         queue<int> q;
30         q.push(inicio);
31         visitado[inicio] = true;
32         int alcanzados = 1;
33
34         while (!q.empty()) {
35             int u = q.front(); q.pop();
36

```

```

34         for (int v : adj[u]) {
35             if (visitado[v]) continue;
36             visitado[v] = true;
37             alcanzados++;
38             q.push(v);
39         }
40     }
41
42     for (int u = 0; u < n; u++)
43         if (!adj[u].empty() && !visitado[u])
44             return false; // hay un nodo con aristas fuera de
45             // esta componente
46
47     return true; // O(V + E)
48 }
49
50 // READ: construye el circuito (Hierholzer), empezando en
51 // CUALQUIER nodo con aristas -- a diferencia de Eulerian
52 // Path,
53 // aquí no hay un nodo de inicio obligado.
54 vector<int> hierholzer() {
55     int inicio = 0;
56     for (int u = 0; u < n; u++)
57         if (!adj[u].empty()) { inicio = u; break; }
58
59     vector<int> pila = {inicio}, circuito;
60
61     while (!pila.empty()) {
62         int u = pila.back();
63
64         if (!adj[u].empty()) {
65             int v = *adj[u].begin();
66             adj[u].erase(adj[u].find(v));
67             adj[v].erase(adj[v].find(u));
68             pila.push_back(v);
69         } else {
70             circuito.push_back(u);
71             pila.pop_back();
72         }
73     }
74
75     reverse(circuito.begin(), circuito.end());
76     return circuito; // circuito.front() == circuito.back()
77 } // O(E)

```

Ejemplo de funcionamiento:

```

1 // grafo: 0 - 1 - 2 - 0 (triangulo, todos grado 2 ->
2 // circuito)
3
4 vector<multiset<int>> adj(3);
5 auto addEdge = [&](int u, int v) {
6     adj[u].insert(v);
7     adj[v].insert(u);
8 };
9 addEdge(0, 1);
10 addEdge(1, 2);
11 addEdge(2, 0);
12
13 CircuitoEuleriano ce(adj, 3);
14
15 cout << ce.existeCircuito();
16 // true
17
18 auto circuito = ce.hierholzer();
19 for (int u : circuito) cout << u << " ";
20 // 0 1 2 0 (empieza y termina en el mismo nodo)

```

Regla práctica: el chequeo de existencia es idéntico al de Eulerian Path, solo que aquí el conteo de "nodos de grado impar" **debe dar exactamente 0** —si da 2, existe camino pero no circuito (hay que abrir el recorrido en esos dos nodos); si da más de 2, no existe ninguno de los dos.

19.3 Hierholzer

Idea: esta subsección presenta la implementación genérica y verdaderamente $O(E)$ del algoritmo, que ya se usó de forma simplificada en Eulerian Path y Eulerian Circuit. La versión con `multiset` de esas secciones es correcta pero cuesta $O(E \log E)$ (cada `insert/erase` es logarítmico); la versión estándar de competencia evita eso con un **puntero por nodo** (`ptr[u]`) que avanza

sobre la lista de adyacencia de u **saltándose** las aristas ya usadas —como cada arista se marca usada una sola vez en toda la ejecución, cada puntero avanza como mucho tantas veces como aristas tiene su nodo, dando $O(E)$ total real. Además, indexando las aristas por id (en vez de por su nodo destino), la misma implementación funciona **sin cambios** para grafos dirigidos, no dirigidos, y con aristas múltiples.

¿Por qué usarlo? El factor $\log E$ de la versión con `multiset` no importa para grafos chicos, pero se vuelve relevante con $E \sim 10^5$ – 10^6 (típico en problemas de secuencias de De Bruijn o reconstrucción de rutas). Esta versión con punteros es la que normalmente se usa como plantilla reutilizable, ya que separa completamente la lógica del algoritmo (la pila + "atascarse y retroceder") de la representación del grafo (dirigido o no, con o sin aristas múltiples), a través del truco de marcar aristas por id en vez de por nodo.

Casos de uso:

- Los mismos que Eulerian Path/Circuit, cuando E es lo bastante grande como para que el factor $\log E$ del `multiset` importe en el límite de tiempo.
- Grafos **dirigidos** con aristas múltiples entre el mismo par de nodos (el `multiset` de las secciones anteriores solo se planteó para no dirigido; aquí el mismo código sirve para ambos sin modificar la lógica del algoritmo, solo la forma en que se cargan las aristas).
- Reconstrucción de secuencias de De Bruijn a gran escala (genome assembly), donde E puede ser del orden de millones de k -mers.
- Como plantilla base reutilizable: una vez armado el `struct` genérico, tanto Eulerian Path como Eulerian Circuit son solo "elegir el nodo de inicio correcto" antes de llamarlo.

Complejidad: $O(E)$ real (sin factor logarítmico), tanto para grafos dirigidos como no dirigidos.

```

1 struct Hierholzer {
2     vector<vector<pair<int,int>>> adj;    // adj[u] = {(v,
3         idArista)}
4     vector<bool> usada;
5     vector<int> ptr;    // ptr[u] = índice de la primera arista de
6         // adj[u] que TODAVIA no se sabe si esta usada
7     int n;
8
9     // CREATE: guarda el grafo (dirigido o no -- para no dirigido,
10    // cada arista se agrega DOS veces, una en cada sentido, con
11    // el
12    // MISMO id; para dirigido, una sola vez con id unico).
13    Hierholzer(vector<vector<pair<int,int>>>& adj_, int
14        numAristas,
15        int n_) {
16        adj = adj_;
17        n = n_;
18        usada.assign(numAristas, false);
19        ptr.assign(n, 0);
20    }
21
22    // READ: construye el camino/circuito euleriano empezando en
23    // 'inicio', asumiendo que ya se verificó que existe.
24    vector<int> construir(int inicio) {
25        vector<int> pila = {inicio}, camino;
26
27        while (!pila.empty()) {
28            int u = pila.back();
29
30            // avanza el puntero saltando aristas YA usadas -- cada
31            // posición se salta como mucho una vez en TODA la

```

```

29 // ejecucion, de ahí el O(E) total
30 while (ptr[u] < (int)adj[u].size() &&
31     usada[adj[u][ptr[u]].second])
32     ptr[u]++;
33
34 if (ptr[u] == (int)adj[u].size()) {
35     camino.push_back(u);    // atascado: sin salidas
36     pila.pop_back();
37 } else {
38     auto [v, id] = adj[u][ptr[u]];
39     usada[id] = true;
40     pila.push_back(v);
41 }
42 }
43
44 reverse(camino.begin(), camino.end());
45 return camino;
46 }
47 // O(E)

```

Ejemplo de funcionamiento:

```

1 // grafo DIRIGIDO: 0 -> 1 -> 2 -> 0 (ciclo)
2 //                0 -> 2 (arista extra)
3 // grados: 0 tiene salida=2, entrada=1; 2 tiene salida=1, entrada
4 //          =2
5 // -> 0 es el inicio obligado (salida = entrada + 1),
6 //          2 es el final obligado (entrada = salida + 1)
7
8 vector<vector<pair<int,int>>> adj(3);
9 int idArista = 0;
10 auto addEdgeDirigida = [&](int u, int v) {
11     adj[u].push_back({v, idArista++});
12 };
13 addEdgeDirigida(0, 1);    // id 0
14 addEdgeDirigida(1, 2);    // id 1
15 addEdgeDirigida(2, 0);    // id 2
16 addEdgeDirigida(0, 2);    // id 3
17
18 Hierholzer h(adj, 4, 3);
19 auto camino = h.construir(0);
20
21 for (int u : camino) cout << u << " ";
22 // 0 1 2 0 2 (recorre las 4 aristas dirigidas exactamente una
23 // vez)

```

Nota sobre grafos no dirigidos: para reutilizar este mismo `struct` con un grafo no dirigido, cada arista física (u, v) se carga **dos veces**: `adj[u].push_back({v, id})` y `adj[v].push_back({u, id})`, ambas con el **mismo** id. Como `usada[]` se indexa por id y no por posición, marcar la arista como usada desde un lado **automáticamente** la invalida también del otro lado —el mismo efecto que borrar de ambos `multiset` en la versión anterior, sin el costo logarítmico.

Regla práctica: usa la versión con `multiset` de Eulerian Path/Circuit cuando el grafo es chico y prefieres código más corto de escribir bajo presión de tiempo; cambia a esta versión con punteros por id en cuanto E sea grande, el grafo sea dirigido, o quieras una sola plantilla reutilizable para ambas variantes (camino y circuito) sin duplicar lógica —la única diferencia entre resolver un camino o un circuito con esta plantilla es **qué nodo** se pasa como inicio.

20 Network Flow

21 Network Flow

21.1 Ford-Fulkerson

Idea: Ford-Fulkerson calcula el flujo máximo entre una fuente s y un sumidero t repitiendo, mientras sea posible, dos pasos: (1) encontrar un **camino aumentante** de s a t en el **grafo residual** (aristas con capacidad restante > 0); (2) enviar por ese camino tanta cantidad como permita su **cuello de botella** (la arista de menor capacidad residual del camino), actualizando las capacidades

residuales —restando en el sentido de ida, **sumando** en el sentido inverso (arista de "deshacer", que permite corregir decisiones subóptimas posteriores). Cuando ya no existe ningún camino aumentante, el flujo actual es el máximo (teorema de flujo máximo-corte mínimo).

¿Por qué usarlo? Es la base de **todos** los algoritmos de flujo: Edmonds-Karp y Dinic (siguientes secciones) son variantes de Ford-Fulkerson que solo cambian **cómo** se encuentra el camino aumentante para acotar mejor la complejidad. La versión con DFS simple es la más fácil de programar bajo presión y es correcta siempre que las capacidades sean **enteras** —pero su complejidad depende del **valor** del flujo máximo, no solo de V y E , así que puede ser lenta en el peor caso adversarial (ver modificaciones abajo para las variantes que corrigen esto).

Casos de uso:

- Flujo máximo en una red de transporte/capacidades (el problema base).
- Como building block de *bipartite matching* vía flujo (fuente $\rightarrow$ lado $L \rightarrow$ lado $R \rightarrow$ sumidero, todas las aristas de capacidad 1) — ver Maximum Bipartite Matching más adelante.
- Corte mínimo entre s y t : su valor es **igual** al flujo máximo (mismo teorema) — ver Min-Cut más adelante.

Complejidad: $O(E \cdot F)$ donde F es el valor del flujo máximo (cada DFS es $O(E)$, y en el peor caso se necesitan $O(F)$ iteraciones si las capacidades son grandes y el camino aumentante elegido es "malo").

```

1 struct FordFulkerson {
2     struct Arista { int destino; long long capacidad; int reversa
3         ; };
4     vector<vector<Arista>> adj;
5     vector<bool> visitado;
6     int n;
7
8     // CREATE: grafo residual vacio de n nodos.
9     FordFulkerson(int n_) {
10         n = n_;
11         adj.assign(n, {});
12     } // O(n)
13
14     // WRITE: agrega una arista dirigida u->v con capacidad c.
15     // Crea
16     // automaticamente la arista reversa con capacidad 0 (se
17     // llena a
18     // medida que se envia flujo, permitiendo "deshacer" envios).
19     void agregarArista(int u, int v, long long c) {
20         adj[u].push_back({v, c, (int)adj[v].size()});
21         adj[v].push_back({u, 0, (int)adj[u].size() - 1});
22     } // O(1)
23
24     // busca un camino aumentante con DFS, enviando flujo
25     // mientras
26     // regresa de la recursion (devuelve cuanto flujo se logro
27     // enviar por esa rama, 0 si no encontro camino).
28     long long dfs(int u, int t, long long flujoMin) {
29         if (u == t) return flujoMin;
30         visitado[u] = true;
31
32         for (auto& arista : adj[u]) {
33             if (visitado[arista.destino] || arista.capacidad <= 0)
34                 continue;
35
36             long long resultado = dfs(arista.destino, t,
37                 min(flujoMin, arista.capacidad));
38
39             if (resultado > 0) {
40                 arista.capacidad -= resultado;
41                 adj[arista.destino][arista.reversa].capacidad +=
42                     resultado;
43                 return resultado;
44             }
45         }
46     }
47 }

```

```

40 }
41
42 return 0;
43 }
44
45 // READ: flujo maximo de s a t.
46 long long flujoMaximo(int s, int t) {
47     long long total = 0;
48
49     while (true) {
50         visitado.assign(n, false);
51         long long f = dfs(s, t, LLONG_MAX);
52         if (f == 0) break; // no hay mas caminos aumentantes
53         total += f;
54     }
55
56     return total;
57 } // O(E * F)
58
59 //
60 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
61 //
62
63 // 1. FLUJO MAXIMO CON MULTIPLES FUENTES / SUMIDEROS: se
64 // agrega
65 // una SUPERFUENTE S y un SUPERSUMIDERO T ficticios; S
66 // conecta
67 // a cada fuente real con capacidad infinita (o la
68 // capacidad
69 // maxima que esa fuente puede producir), cada sumidero
70 // real
71 // conecta a T de la misma forma -- y se corre flujoMaximo
72 // (S,T)
73 // normal, sin modificar el algoritmo en si.
74 //
75 // int S = n, T = n + 1; // nodos ficticios extra al
76 // construir
77 // for (int fuente : fuentesReales)
78 //     agregarArista(S, fuente, LLONG_MAX);
79 // for (int sumidero : sumiderosReales)
80 //     agregarArista(sumidero, T, LLONG_MAX);
81 // long long total = flujoMaximo(S, T);
82
83 // 2. VERTICES CON CAPACIDAD (limite de "flujo que puede
84 // pasar
85 // POR un nodo", no solo por sus aristas): se divide cada
86 // nodo u en u_entrada y u_salida, unidos por una arista
87 // u_entrada -> u_salida con capacidad = limite del nodo;
88 // todas las aristas que antes llegaban a u ahora llegan a
89 // u_entrada, y las que salian de u ahora salen de
90 // u_salida.
91 // El algoritmo de flujo no cambia, solo la construccion
92 // del
93 // grafo (mismo truco que usa 2-SAT con nodo/negacion,
94 // pero
95 // aplicado a capacidad en vez de literal).
96
97 // 3. CAPACIDADES NO ENTERAS / MUY GRANDES: Ford-Fulkerson
98 // con
99 // DFS puro puede degenerar si las capacidades son enormes
100 // y
101 // el camino elegido es siempre el "peor" posible (ejemplo
102 // clasico: red con capacidades ~10^9 puede necesitar
103 // ~10^9
104 // iteraciones). La solucion NO es cambiar este algoritmo,
105 // sino usar Edmonds-Karp o Dinic (siguientes secciones),
106 // que
107 // acotan la complejidad independientemente del valor de F.
108
109 // 4. FLUJO MAXIMO CON LIMITE INFERIOR (cada arista debe
110 // llevar
111 // AL MENOS un minimo, ademas de un maximo): requiere una
112 // transformacion adicional del grafo (nodos ficticios de
113 // "exceso") antes de correr flujo maximo normal -- fuera
114 // del
115 // alcance de esta plantilla base, pero la señal para
116 // reconocerlo es que el enunciado da un RANGO
117 // [minimo, maximo] por arista en vez de solo un maximo.
118 }

```

Ejemplo de funcionamiento:

```

1 // red: s -(3)-> a -(2)-> t
2 //       s -(2)-> b -(3)-> t
3 //       a -(1)-> b
4
5 // nodos: s=0, a=1, b=2, t=3
6
7 FordFulkerson ff(4);
8 ff.agregarArista(0, 1, 3);

```



```

9  ff.agregarArista(0, 2, 2);
10 ff.agregarArista(1, 3, 2);
11 ff.agregarArista(2, 3, 3);
12 ff.agregarArista(1, 2, 1);
13
14 cout << ff.flujoMaximo(0, 3);
15 // 4
16 // (2 por s-a-t, 2 por s-b-t; la arista a-b sobra en este caso,
17 // el cuello de botella real esta en las aristas de salida de
18 // y de entrada a t)

```

Regla práctica: programa Ford-Fulkerson con DFS cuando las capacidades son chicas o el límite de tiempo es generoso —es el más corto de escribir bajo presión. En cuanto el problema tenga capacidades grandes o el límite de tiempo sea ajustado, salta directamente a Edmonds-Karp (siguiente sección): la estructura del grafo residual (Arista, agregarArista) es **idéntica**, solo cambia cómo se busca el camino aumentante.

21.2 Edmonds-Karp

Idea: Edmonds-Karp es Ford-Fulkerson con **una sola diferencia**: el camino aumentante se busca con **BFS** en vez de DFS, lo que garantiza que **siempre** se use el camino con **menos aristas** disponible en el grafo residual. Esa elección —"el camino más corto primero"— es lo que rompe el peor caso adversarial de Ford-Fulkerson: se puede demostrar que el número total de fases de BFS está acotado por $O(V \cdot E)$, **independientemente** de qué tan grandes sean las capacidades.

¿Por qué usarlo? Es el primer paso obligado en cuanto Ford-Fulkerson con DFS "se siente lento" o el problema explícitamente tiene capacidades grandes ($\geq 10^4$ – 10^5): mismo grafo residual, mismo concepto de camino aumentante, **solo** cambia BFS por DFS, a cambio de una garantía de complejidad que ya no depende del valor del flujo. Si $O(V \cdot E^2)$ sigue sin alcanzar (grafos realmente grandes), el siguiente paso es Dinic.

Casos de uso:

- Los mismos que Ford-Fulkerson (flujo máximo, matching vía flujo, corte mínimo), en cuanto las capacidades son lo bastante grandes como para que el peor caso de DFS sea un riesgo real.
- Como opción "segura por defecto" cuando no se sabe de antemano qué tan adversarial puede ser el grafo de entrada —el costo de cambiar DFS por BFS es mínimo y elimina ese riesgo.

Complejidad: $O(V \cdot E^2)$ — como mucho $O(V \cdot E)$ fases de BFS (cada una satura al menos una arista del grafo residual), cada BFS cuesta $O(E)$.

```

1  struct EdmondsKarp {
2      struct Arista { int destino; long long capacidad; int reversa
3          ; };
4      vector<vector<Arista>> adj;
5      int n;
6
7      // CREATE: grafo residual vacio de n nodos.
8      EdmondsKarp(int n_) {
9          n = n_;
10         adj.assign(n, {});
11     }
12
13     // WRITE: identico a Ford-Fulkerson -- agrega u->v con
14     // capacidad
15     // c, mas su reversa de capacidad 0.
16     void agregarArista(int u, int v, long long c) {
17         adj[u].push_back({v, c, (int)adj[v].size()});
18         adj[v].push_back({u, 0, (int)adj[u].size() - 1});
19     }
20 }

```

```

17 } // O(1)
18
19 // busca el camino aumentante MAS CORTO (en numero de aristas
20 // con BFS. Devuelve el flujo enviado por ese camino, 0 si no
21 // hay camino aumentante. padreNodo/padreArista reconstruyen
22 // el
23 // camino encontrado.
24 long long bfs(int s, int t, vector<int>& padreNodo,
25 vector<int>& padreArista) {
26     fill(padreNodo.begin(), padreNodo.end(), -1);
27     padreNodo[s] = s;
28
29     queue<pair<int, long long>> q;
30     q.push({s, LLONG_MAX});
31
32     while (!q.empty()) {
33         auto [u, flujoHasta] = q.front(); q.pop();
34
35         for (int i = 0; i < (int)adj[u].size(); i++) {
36             auto& a = adj[u][i];
37
38             if (a.capacidad <= 0 || padreNodo[a.destino] != -1)
39                 continue;
40
41             padreNodo[a.destino] = u;
42             padreArista[a.destino] = i;
43             long long nuevoFlujo = min(flujoHasta, a.capacidad);
44
45             if (a.destino == t) return nuevoFlujo;
46
47             q.push({a.destino, nuevoFlujo});
48         }
49     }
50
51     return 0; // t no alcanzable -> no hay camino aumentante
52 } // O(E)
53
54 // READ: flujo maximo de s a t.
55 long long flujoMaximo(int s, int t) {
56     long long total = 0;
57     vector<int> padreNodo(n), padreArista(n);
58
59     while (true) {
60         long long f = bfs(s, t, padreNodo, padreArista);
61         if (f == 0) break;
62
63         // reconstruye el camino desde t hasta s y actualiza
64         // capacidades residuales (resta de ida, suma en reversa)
65         int u = t;
66         while (u != s) {
67             int idArista = padreArista[u];
68             int padre = padreNodo[u];
69
70             adj[padre][idArista].capacidad -= f;
71             int idReversa = adj[padre][idArista].reversa;
72             adj[u][idReversa].capacidad += f;
73
74             u = padre;
75         }
76
77         total += f;
78     }
79
80     return total;
81 } // O(V * E^2)
82
83 //
84 // -----
85 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
86 // -----
87
88 // 1. RECONSTRUIR EL CAMINO AUMENTANTE EXPLICITO (no solo su
89 // flujo): ya viene gratis en padreNodo/padreArista tras
90 // cada
91 // bfs() exitoso -- util si el problema pide ademas
92 // MOSTRAR
93 // las rutas usadas, no solo el valor total del flujo
94 // maximo.
95 //
96 // vector<int> reconstruirCamino(int s, int t,
97 // vector<int>& padreNodo) {
98 //     vector<int> camino;
99 //     for (int u = t; u != s; u = padreNodo[u])
100 //         camino.push_back(u);
101 //     camino.push_back(s);
102 //     reverse(camino.begin(), camino.end());
103 //     return camino;
104 // }
105
106 // 2. MULTIPLES FUENTES / SUMIDROS: identico truco que en
107 // Ford-Fulkerson -- superfuente S y supersumidero T con

```



```

103 // aristas de capacidad infinita hacia/desde cada fuente y
104 // sumidero real; se corre flujoMaximo(S, T) sin cambios.
105 //
106 // int S = n, T = n + 1;
107 // for (int fuente : fuentesReales)
108 //     agregarArista(S, fuente, LLONG_MAX);
109 // for (int sumidero : sumiderosReales)
110 //     agregarArista(sumidero, T, LLONG_MAX);
111
112 // 3. DETENERSE ANTES DE flujoMaximo() COMPLETO -- si solo se
113 // necesita saber SI existe flujo >= X (no el maximo
114 // exacto),
115 // se puede cortar el while en cuanto total >= X,
116 // ahorrando
117 // fases de BFS innecesarias:
118 //
119 // long long flujoAlMenos(int s, int t, long long X) {
120 //     long long total = 0;
121 //     vector<int> padreNodo(n), padreArista(n);
122 //     while (total < X) {
123 //         long long f = bfs(s, t, padreNodo, padreArista);
124 //         if (f == 0) break;
125 //         // ... misma actualizacion de capacidades ...
126 //         total += f;
127 //     }
128 //     return total;
129 // }
130
131 // 4. GRAFOS DENSOS O MUY GRANDES (E ~ 10^5 o mas, V grande):
132 // O(V*E^2) de Edmonds-Karp puede seguir sin alcanzar. La
133 // senal para saltar a Dinic (siguiente seccion) es la
134 // MISMA
135 // estructura de grafo residual (Arista, agregarArista)
136 // mas
137 // una nocion de "grafo por niveles" (BFS de niveles + DFS
138 // con poda) que reduce el numero de fases a O(V), en vez
139 // de
140 // O(V*E) -- no hace falta reescribir el grafo, solo la
141 // logica de busqueda de caminos aumentantes.
142 };

```

Ejemplo de funcionamiento:

```

1 // misma red que en Ford-Fulkerson:
2 // s -(3)-> a -(2)-> t
3 // s -(2)-> b -(3)-> t
4 // a -(1)-> b
5
6 EdmondsKarp ek(4);
7 ek.agregarArista(0, 1, 3);
8 ek.agregarArista(0, 2, 2);
9 ek.agregarArista(1, 3, 2);
10 ek.agregarArista(2, 3, 3);
11 ek.agregarArista(1, 2, 1);
12
13 cout << ek.flujoMaximo(0, 3);
14 // 4 (mismo resultado que Ford-Fulkerson -- el flujo maximo
15 // es
16 // unico independientemente del orden de caminos elegido;
17 // lo que cambia es CUANTAS iteraciones toma llegar ahi)

```

Regla práctica: Edmonds-Karp y Ford-Fulkerson dan **siempre** el mismo flujo máximo —difieren solo en garantía de complejidad, nunca en corrección. Si ya tienes el `struct` de Ford-Fulkerson escrito, migrar a Edmonds-Karp es cambiar `dfs()` por `bfs()` y ajustar cómo se reconstruye el camino para actualizar capacidades —el resto (`Arista`, `agregarArista`, la idea de arista reversa) no cambia en absoluto.

21.3 Dinic

Idea: Dinic mejora a Edmonds-Karp separando el trabajo en **fases**: en cada fase, (1) construye un **grafo por niveles** con un solo BFS desde s (`nivel[u]` = distancia mínima desde s en el grafo residual actual), quedándose **solo** con las aristas que avanzan de un nivel al siguiente (`nivel[v] = nivel[u] + 1`); (2) sobre ese grafo por niveles, encuentra **todos** los caminos aumentantes posibles de una sola vez con DFS, usando un **puntero por nodo** (`ptr[u]`, igual que en Hierholzer) para no reexplorar aristas ya agotadas dentro de la misma fase —esto se llama

blocking flow. Cuando el BFS ya no puede alcanzar t (no hay más niveles), el algoritmo termina: el flujo acumulado es el máximo.

¿Por qué usarlo? Edmonds-Karp encuentra **un** camino aumentante por cada BFS ($O(V \cdot E)$ fases en el peor caso); Dinic encuentra **muchos** caminos aumentantes por cada BFS (todo el blocking flow de esa fase), reduciendo el número de fases a $O(V)$ —cada fase sube al menos un nivel, y hay como mucho V niveles posibles. Es la opción estándar en competencia cuando E es grande y $O(V \cdot E^2)$ de Edmonds-Karp no alcanza el límite de tiempo; en grafos bipartitos (unit-capacity), Dinic corre incluso más rápido ($O(E\sqrt{V})$), lo cual lo hace también la base preferida para Maximum Bipartite Matching vía flujo (siguiente sección).

Casos de uso:

- Los mismos que Ford-Fulkerson/Edmonds-Karp, en instancias grandes donde $O(V \cdot E^2)$ es demasiado lento.
- Bipartite Matching vía flujo, aprovechando el caso especial de $O(E\sqrt{V})$ en grafos de capacidad unitaria.
- Cualquier problema de flujo donde el límite de tiempo sea ajustado y V, E sean del orden de 10^4 – 10^5 .

Complejidad: $O(V^2 \cdot E)$ en general, $O(E\sqrt{V})$ en grafos de capacidad unitaria (bipartite matching) — en ambos casos, estrictamente mejor o igual que Edmonds-Karp.

```

1 struct Dinic {
2     struct Arista { int destino; long long capacidad; int reversa; };
3     vector<vector<Arista>> adj;
4     vector<int> nivel, ptr;
5     int n;
6
7     // CREATE: grafo residual vacio de n nodos.
8     Dinic(int n_) {
9         n = n_;
10        adj.assign(n, {});
11    } // O(n)
12
13    // WRITE: identico a Ford-Fulkerson/Edmonds-Karp -- agrega u
14    // -> v
15    // con capacidad c, mas su reversa de capacidad 0.
16    void agregarArista(int u, int v, long long c) {
17        adj[u].push_back({v, c, (int)adj[v].size()});
18        adj[v].push_back({u, 0, (int)adj[u].size() - 1});
19    } // O(1)
20
21    // FASE 1: construye el grafo por niveles con BFS. Devuelve
22    // false si t ya no es alcanzable (fin del algoritmo).
23    bool bfs(int s, int t) {
24        nivel.assign(n, -1);
25        queue<int> q;
26        q.push(s);
27        nivel[s] = 0;
28
29        while (!q.empty()) {
30            int u = q.front(); q.pop();
31
32            for (auto& a : adj[u]) {
33                if (a.capacidad > 0 && nivel[a.destino] == -1) {
34                    nivel[a.destino] = nivel[u] + 1;
35                    q.push(a.destino);
36                }
37            }
38        }
39
40        return nivel[t] != -1;
41    } // O(E)
42
43    // FASE 2: DFS con punteros (blocking flow), buscando UN
44    // camino

```

```

43 // aumentante que respete el grafo por niveles; se llama
44 // repetidamente hasta agotar todos los caminos de esta fase.
45 long long dfs(int u, int t, long long flujoMin) {
46     if (u == t) return flujoMin;
47
48     // avanza el puntero saltando aristas ya agotadas o que no
49     // avanzan de nivel -- cada posición se salta como mucho
50     // una
51     // vez POR FASE, de ahí la eficiencia del blocking flow
52     for (; ptr[u] < (int)adj[u].size(); ptr[u]++) {
53         auto& a = adj[u][ptr[u]];
54
55         if (a.capacidad <= 0 || nivel[a.destino] != nivel[u] + 1)
56             continue;
57
58         long long resultado = dfs(a.destino, t,
59             min(flujoMin, a.capacidad));
60
61         if (resultado > 0) {
62             a.capacidad -= resultado;
63             adj[a.destino][a.reversa].capacidad += resultado;
64             return resultado;
65         }
66     }
67
68     return 0; // u agotado en esta fase, ningún camino desde
69     // aquí
70
71     // dentro de una fase
72
73     // READ: flujo máximo de s a t.
74     long long flujoMaximo(int s, int t) {
75         long long total = 0;
76
77         while (bfs(s, t)) {
78             ptr.assign(n, 0);
79
80             while (long long f = dfs(s, t, LLONG_MAX))
81                 total += f; // sigue sacando caminos de la MISMA
82                 // fase hasta agotar el blocking flow
83
84             return total;
85         }
86
87         // dentro de una fase
88         // dentro de una fase
89
90         // MODIFICACIONES Y COMENTARIOS CON LOS USOS
91         //
92         // -----
93
94         // 1. BIPARTITE MATCHING VIA DINIC (capacidad unitaria): sin
95         // cambiar el algoritmo, si TODAS las aristas tienen
96         // capacidad
97         // 1 (fuente -> L, L -> R según compatibilidad, R ->
98         // sumidero),
99         // la complejidad baja automáticamente a  $O(E \cdot \sqrt{V})$  --
100         // no
101         // hace falta ningún ajuste al código, es una propiedad
102         // del
103         // caso de capacidad unitaria. Ver Maximum Bipartite
104         // Matching.
105         //
106         // int S = n, T = n + 1; // agregar como nodos extra
107         // for (int u : L) agregarArista(S, u, 1);
108         // for (int v : R) agregarArista(T + n, v, 1); // ajustar
109         // índices
110         // for (auto [u, v] : compatibles) agregarArista(u, v, 1);
111         // long long matching = flujoMaximo(S, T);
112
113         // 2. MÚLTIPLES FUENTES / SUMIDEROS: mismo truco de
114         // superfuente
115         // S / supersumidero T que en Ford-Fulkerson y Edmonds-
116         // Karp,
117         // sin cambios al algoritmo:
118         //
119         // int S = n, T = n + 1;
120         // for (int fuente : fuentesReales)
121         //     agregarArista(S, fuente, LLONG_MAX);
122         // for (int sumidero : sumiderosReales)
123         //     agregarArista(sumidero, T, LLONG_MAX);
124
125         // 3. RECUPERAR EL CORTE MINIMO tras flujoMaximo(): igual que
126         // en
127         // Ford-Fulkerson -- el lado S del corte es el conjunto de
128         // nodos con nivel[] != -1 EN LA ÚLTIMA LLAMADA A bfs()
129         // que
130         // devolvió false (esa es la última vez que se calcularon
131         // niveles antes de que t dejara de ser alcanzable):
132         //
133         // vector<bool> ladoS() {
134         //     vector<bool> resultado(n, false);
135         //     for (int u = 0; u < n; u++)

```

```

122 // resultado[u] = (nivel[u] != -1);
123 // return resultado;
124 // }
125 // -- llamar DESPUES de flujoMaximo(); nivel[] queda de la
126 // última corrida de bfs() (la que falló en alcanzar t)
127
128 // 4. SCALING (capacidades MUY grandes, V y E también grandes
129 //):
130 // variante avanzada que procesa las aristas en orden
131 // decreciente de "bits de capacidad" para acotar aún más
132 // las
133 // fases -- rara vez necesaria en competencia, Dinic
134 // simple ya
135 // cubre la gran mayoría de límites de tiempo típicos.
136
137 // 5. GRAFOS CON ARISTAS NO DIRIGIDAS: se agregan como DOS
138 // aristas dirigidas independientes, cada una con su
139 // PROPIA
140 // reversa de capacidad 0 (no se comparte una sola reversa
141 // entre ambos sentidos, a diferencia de Hierholzer con su
142 // id compartido) -- Dinic no necesita ningún ajuste extra
143 // para esto, es solo como se cargan las aristas:
144 //
145 // void agregarAristaNoDirigida(int u, int v, long long c) {
146 //     agregarArista(u, v, c);
147 //     agregarArista(v, u, c);
148 // }
149
150 };

```

Ejemplo de funcionamiento:

```

1 // misma red que en Ford-Fulkerson y Edmonds-Karp:
2 // s - (3) -> a - (2) -> t
3 // s - (2) -> b - (3) -> t
4 // a - (1) -> b
5
6 Dinic din(4);
7 din.agregarArista(0, 1, 3);
8 din.agregarArista(0, 2, 2);
9 din.agregarArista(1, 3, 2);
10 din.agregarArista(2, 3, 3);
11 din.agregarArista(1, 2, 1);
12
13 cout << din.flujoMaximo(0, 3);
14 // 4 (mismo flujo máximo que las dos versiones anteriores --
15 // Dinic solo cambia CUANTAS fases/iteraciones toma llegar,
16 // no el resultado)

```

Regla práctica: las tres estructuras de esta sección (Ford-Fulkerson, Edmonds-Karp, Dinic) comparten **exactamente** la misma representación de grafo residual (Arista, agregarArista) y siempre dan el mismo flujo máximo — la progresión natural es empezar con Ford-Fulkerson por simplicidad, subir a Edmonds-Karp si las capacidades son grandes, y solo llegar a Dinic cuando E es grande y $O(V \cdot E^2)$ no alcanza el límite de tiempo. Si el problema es específicamente bipartite matching, Dinic es casi siempre la mejor opción de entrada por su complejidad especial en capacidad unitaria.

21.4 Maximum Bipartite Matching

Idea: Bipartite Matching (ya visto con Kuhn en la sección de Bipartite Graphs) es un caso particular de flujo máximo: se construye una red con una **superfuente** S conectada a cada nodo de L (capacidad 1), cada nodo de L conectado a los nodos de R con los que es compatible (capacidad 1), y cada nodo de R conectado a un **supersumidero** T (capacidad 1). Como **todas** las capacidades son 1, cualquier unidad de flujo de S a T corresponde exactamente a "un nodo de L emparejado con un nodo de R " — el flujo máximo de S a T es, literalmente, el tamaño del matching máximo (y no puede ser mayor porque cada nodo de L/R tiene capacidad de entrada/salida 1, así que participa en como mucho un emparejamiento).

¿Por qué usarlo? Es la alternativa a Kuhn cuando ya tienes infraestructura de flujo (Dinic) escrita y prefieres

reutilizarla en vez de mantener dos implementaciones separadas —además, en el caso particular de capacidad unitaria, Dinic corre en $O(E\sqrt{V})$, **mejor** que el $O(V \cdot E)$ de Kuhn simple. Si V es pequeño ($\lesssim 1000$ – 2000), Kuhn sigue siendo más simple de escribir bajo presión; si V es grande, Dinic + esta construcción es la opción más rápida disponible en esta guía.

Casos de uso:

- Los mismos que Bipartite Matching (asignación óptima uno-a-uno entre dos conjuntos), cuando V es lo bastante grande para que $O(E\sqrt{V})$ de Dinic gane frente a $O(V \cdot E)$ de Kuhn.
- Como building block de **matching con capacidades** —variante donde cada nodo de L o R puede emparejarse con **más de uno** del otro lado (cambiando la capacidad de las aristas $S \rightarrow L$ o $R \rightarrow T$ de 1 a ese límite, ver modificaciones abajo).
- Cuando el problema **ya** pide flujo máximo por otra razón y el matching bipartito es solo una parte del modelo completo —reutilizar la misma estructura de Dinic evita duplicar código.

Complejidad: $O(E\sqrt{V})$ con Dinic (gracias a la propiedad especial de grafos de capacidad unitaria), frente a $O(V \cdot E)$ de Kuhn — construcción de la red en $O(V + E)$.

```

1 struct MaximumBipartiteMatching {
2     Dinic dinic;
3     int nL, nR, S, T;
4
5     // CREATE: construye la red de flujo para matching bipartito.
6     // nL, nR = tamanos de los lados L y R. compatibles[u] =
7     // lista de
8     // nodos de R compatibles con el nodo u de L.
9     MaximumBipartiteMatching(int nL_, int nR_,
10     vector<vector<int>>& compatibles)
11     : dinic(nL_ + nR_ + 2), nL(nL_), nR(nR_) {
12         S = nL + nR; // superfuente
13         T = nL + nR + 1; // supersumidero
14
15         for (int u = 0; u < nL; u++)
16             dinic.agregarArista(S, u, 1);
17
18         for (int v = 0; v < nR; v++)
19             dinic.agregarArista(nL + v, T, 1);
20
21         for (int u = 0; u < nL; u++)
22             for (int v : compatibles[u])
23                 dinic.agregarArista(u, nL + v, 1);
24     }
25
26     // READ: tamaño del matching maximo.
27     long long matchingMaximo() {
28         return dinic.flujoMaximo(S, T);
29     }
30
31     // READ: reconstruye la asignacion explicita tras calcular el
32     // matching maximo -- recorre las aristas u->v (u en L)
33     // buscando
34     // cuales quedaron con capacidad residual 0 (es decir, con 1
35     // unidad de flujo pasando por ellas).
36     vector<int> obtenerAsignacion() {
37         vector<int> pareja(nL, -1);
38
39         for (int u = 0; u < nL; u++)
40             for (auto& a : dinic.adj[u])
41                 if (a.destino >= nL && a.destino < nL + nR &&
42                     a.capacidad == 0) // capacidad original era 1,
43                                     // si quedo en 0 es que se uso
44                     pareja[u] = a.destino - nL;
45
46         return pareja;
47     }
48
49     // MODIFICACIONES Y COMENTARIOS CON LOS USOS

```

```

50 //
51 // -----
52 // 1. MATCHING CON CAPACIDADES (cada nodo de L o R puede
53 // emparejarse con MAS de uno del otro lado, hasta un
54 // limite):
55 // en vez de capacidad 1 en las aristas S->L y R->T, se
56 // usa el
57 // limite real de cada nodo -- el resto de la construccion
58 // (L->R sigue siendo capacidad 1, o el limite de esa
59 // pareja
60 // especifica si aplica) no cambia:
61 //
62 // for (int u = 0; u < nL; u++)
63 //     dinic.agregarArista(S, u, limiteL[u]); // en vez de
64 //     1
65 // for (int v = 0; v < nR; v++)
66 //     dinic.agregarArista(nL + v, T, limiteR[v]); // en
67 //     vez de 1
68
69 // 2. MATCHING PONDERADO MAXIMO (cada arista L-R tiene un
70 // peso, y
71 // se busca el matching de mayor peso total, no solo el de
72 // mayor CANTIDAD de parejas): flujo maximo NO resuelve
73 // esto
74 // directamente -- hace falta flujo de COSTO minimo/maximo
75 // (min-cost max-flow), fuera del alcance de esta
76 // plantilla
77 // base. La señal para reconocerlo es que el enunciado da
78 // un
79 // PESO o COSTO por pareja posible, no solo compatibilidad
80 // binaria.
81
82 // 3. VERIFICAR SI EL MATCHING ES PERFECTO (todos los nodos
83 // de L
84 // Y todos los de R quedan emparejados): matching es
85 // perfecto
86 // si y solo si matchingMaximo() == nL == nR (mismo tamaño
87 // en
88 // ambos lados y todos emparejados):
89 //
90 // bool esPerfecto() {
91 //     return matchingMaximo() == nL && nL == nR;
92 // }
93
94 // 4. HALLAR UN CONJUNTO DE VERTICES NO CUBIERTOS (nodos de L
95 // o
96 // R que quedaron SIN pareja tras el matching maximo) --
97 // útil
98 // para problemas que piden identificar el "excedente":
99 //
100 // vector<int> noEmparejadosEnL() {
101 //     auto pareja = obtenerAsignacion();
102 //     vector<int> resultado;
103 //     for (int u = 0; u < nL; u++)
104 //         if (pareja[u] == -1) resultado.push_back(u);
105 //     return resultado;
106 // }
107
108 // 5. TEOREMA DE KONIG (cobertura minima de vertices =
109 // matching
110 // maximo en grafos bipartitos): la cobertura minima se
111 // obtiene a partir del corte minimo de esta MISMA red de
112 // flujo -- ver Min-Cut para la extraccion del lado S del
113 // corte; los nodos de R NO alcanzables desde S en el
114 // residual
115 // final, mas los nodos de R SI alcanzables, forman la
116 // cobertura minima de vertices.
117 };

```

Ejemplo de funcionamiento:

```

1 // L = {0, 1, 2}, R = {0, 1, 2}
2 // 0 - {0, 1}
3 // 1 - {0}
4 // 2 - {1, 2}
5
6 vector<vector<int>> compatibles(3);
7 compatibles[0] = {0, 1};
8 compatibles[1] = {0};
9 compatibles[2] = {1, 2};
10
11 MaximumBipartiteMatching mbm(3, 3, compatibles);
12
13 cout << mbm.matchingMaximo();
14 // 3 (mismo resultado que el ejemplo de Kuhn en Bipartite
15 // Matching)
16
17 auto pareja = mbm.obtenerAsignacion();
18 for (int u = 0; u < 3; u++)
19     cout << u << "- " << pareja[u] << " ";
20 // alguna asignación válida, ej: 0-1 1-0 2-2

```

Regla práctica: Kuhn y Maximum Bipartite Matching vía Dinic **siempre** dan el mismo tamaño de matching máximo (ambos resuelven exactamente el mismo problema) — la elección es de rendimiento y de qué código ya tienes a mano: con V pequeño y sin Dinic ya escrito, Kuhn es más corto; con V grande o Dinic ya disponible de otra parte del problema, esta construcción vía flujo es la opción más rápida y la que se generaliza directo a matching con capacidades.

22 2-SAT

22.1 Implication Graph

Idea: 2-SAT pide encontrar una asignación de verdad para n variables booleanas que satisfaga una fórmula compuesta por **cláusulas de a lo más 2 literales** (ej. $(A \vee B)$, $(\neg A \vee C)$). La clave para resolverlo en tiempo lineal (en vez de probar 2^n asignaciones) es notar que cada cláusula $(a \vee b)$ es **lógicamente equivalente** a dos implicaciones: $\neg a \Rightarrow b$ y $\neg b \Rightarrow a$ (si uno de los dos literales es falso, el otro **debe** ser verdadero). Modelando cada literal (variable y su negación) como un nodo de un grafo dirigido, y cada implicación como una arista, la fórmula completa se convierte en un **grafo de implicación** de $2n$ nodos: el nodo $2i$ representa " x_i es verdadero", el nodo $2i+1$ representa " x_i es falso" (o viceversa, según la convención elegida).

¿Por qué usarlo? Es el paso de **modelado** que convierte un problema de satisfacibilidad booleana en un problema de grafos puro —una vez construido el grafo de implicación, resolver 2-SAT deja de requerir ningún razonamiento lógico adicional: se reduce enteramente a encontrar Componentes Fuertemente Conexas (ver 2-SAT con SCC, siguiente subsección). La parte que sí exige cuidado es traducir **correctamente** cada tipo de restricción a su par de implicaciones —un error aquí produce un grafo que no representa la fórmula real, aunque el algoritmo de SCC corra sin errores.

Casos de uso: construir el grafo de implicación es el primer paso **obligatorio** de cualquier problema de 2-SAT, sin importar cuál sea la restricción concreta. Los patrones de traducción más comunes (cada uno se agrega como `add_disjunction` sobre la estructura de 2-SAT con SCC):

- Forzar una variable a verdadero o falso (restricción unaria, disfrazada de cláusula con el mismo literal repetido).
- Implicación simple $A \Rightarrow B$ (cláusula $(\neg A \vee B)$).
- "Al menos uno" de dos literales (cláusula OR directa).
- "A lo más uno" / mutuamente excluyentes (cláusula NAND).
- Equivalencia $A \leftrightarrow B$ (dos cláusulas: ambas direcciones de la implicación).
- Diferencia forzada $A \oplus B$ (dos cláusulas: al menos uno, y no ambos).

Complejidad: $O(1)$ por cláusula agregada al grafo (cada cláusula agrega una cantidad constante de nodos/aristas), $O(V + E) = O(n + m)$ para el grafo completo con n variables y m cláusulas.

```

1 // convencion de esta plantilla: variable i se representa con
  // los
2 // nodos 2*i (literal POSITIVO: x_i) y 2*i+1 (literal NEGATIVO:
  // !x_i)
3
4 struct GrafoImplicacion {
5     int nVars, nVertices;
6     vector<vector<int>> adj; // grafo de implicacion (dirigido)
7
8     // CREATE: n variables booleanas -> 2n nodos en el grafo.
9     GrafoImplicacion(int nVars_)
10     : nVars(nVars_), nVertices(2 * nVars_), adj(2 * nVars_) {}
11     // O(n)
12
13     // WRITE: agrega la clausula (literal_a OR literal_b), donde
14     // na/nb indican si ese literal esta NEGADO. Traduce la
15     // clausula
16     // a sus dos implicaciones equivalentes:
17     // !a -> b y !b -> a
18     void agregarClausula(int a, bool na, int b, bool nb) {
19         int litA = 2 * a ^ na; // nodo del literal a (negado o no)
20         int litB = 2 * b ^ nb;
21         int negA = litA ^ 1; // nodo del literal OPUESTO a litA
22         int negB = litB ^ 1;
23
24         adj[negA].push_back(litB); // !a -> b
25         adj[negB].push_back(litA); // !b -> a
26     } // O(1)
27 };

```

Ejemplo de funcionamiento:

```

1 // formula: (x0 OR x1) AND (!x0 OR x2)
2 // 2 clausulas, 3 variables (nodos 0..5):
3 // 0=x0, 1=!x0, 2=x1, 3=!x1, 4=x2, 5=!x2)
4
5 GrafoImplicacion gi(3);
6
7 gi.agregarClausula(0, false, 1, false); // (x0 OR x1)
8 gi.agregarClausula(0, true, 2, false); // (!x0 OR x2)
9
10 // aristas generadas:
11 // clausula 1: !x0 -> x1 (1 -> 2), !x1 -> x0 (3 -> 0)
12 // clausula 2: x0 -> x2 (0 -> 4), !x2 -> !x0 (5 -> 1)
13
14 for (int v : gi.adj[1]) cout << v << " ";
15 // 2 (!x0 -> x1)
16
17 for (int v : gi.adj[0]) cout << v << " ";
18 // 4 (x0 -> x2)

```

Regla práctica: sin importar qué tan compleja parezca la restricción original (implicación, equivalencia, exclusión mutua, etc.), siempre se puede descomponer en una o dos cláusulas OR de 2 literales, y cada cláusula siempre agrega exactamente 2 aristas al grafo de implicación con la misma fórmula $\neg a \Rightarrow b$, $\neg b \Rightarrow a$ —memorizar esa única fórmula es más robusto que memorizar un caso especial por cada tipo de restricción.

22.2 2-SAT con SCC

Idea: una vez construido el grafo de implicación, la fórmula es **satisfacible** si y solo si **ninguna** variable x_i tiene sus dos literales (x_i y $\neg x_i$) en la **misma** componente fuertemente conexa —si estuvieran en la misma SCC, significaría que $x_i \Rightarrow \neg x_i$ y $\neg x_i \Rightarrow x_i$ simultáneamente, una contradicción directa. Si la fórmula es satisfacible, la asignación se obtiene comparando el **orden topológico inverso** de las SCCs (que Tarjan/Kosaraju ya dan como efecto secundario, ver esas secciones): a cada variable se le asigna **verdadero** si su literal positivo queda en una SCC **más tarde** en ese orden (equivalentemente, con `comp` más alto si se usa el `id` de Kosaraju/Tarjan de esta plantilla).

¿Por qué usarlo? Es la razón de ser de todo el modelado con Implication Graph: convierte "resolver una fórmula booleana" en "una sola pasada de SCC", sin backtracking ni búsqueda exponencial. Es además **constructivo** —no solo dice si existe solución, sino que la construye directamente comparando los ids de componente, sin ningún paso adicional de búsqueda.

Casos de uso:

- Cualquier problema modelable como restricciones binarias entre variables booleanas: asignación de turnos con conflictos, selección de una opción entre pares incompatibles, coloreo con solo 2 colores sujeto a restricciones.
- Verificar la **existencia** de una asignación válida sin necesitar construirla explícitamente (basta el chequeo de "misma SCC").
- Como building block de problemas más grandes que reducen su núcleo de decisión a 2-SAT (ej. "elige un valor de dos opciones por cada elemento, sujeto a restricciones de pares").

Complejidad: $O(n+m)$ — construir el grafo es $O(m)$ (una cláusula, trabajo constante), Kosaraju/Tarjan sobre el grafo de $2n$ nodos es $O(n+m)$, y la asignación final es $O(n)$.

```

1 struct TwoSatSolver {
2     int n_vars;
3     int n_vertices;
4     vector<vector<int>> > adj, adj_t;
5     vector<bool> used;
6     vector<int> order, comp;
7     vector<bool> assignment;
8
9     // CREATE: n_vars variables -> 2*n_vars nodos (grafo de
10    // implicacion + su transpuesto, para Kosaraju).
11    TwoSatSolver(int n_vars) : n_vars(n_vars), n_vertices(2 *
12    n_vars),
13    adj(n_vertices), adj_t(n_vertices),
14    used(n_vertices), order(),
15    comp(n_vertices, -1), assignment(n_vars) {
16    order.reserve(n_vertices);
17    }
18
19    void dfs1(int v) {
20        used[v] = true;
21        for (int u: adj[v])
22            if (!used[u])
23                dfs1(u);
24        order.push_back(v);
25    }
26
27    void dfs2(int v, int cl) {
28        comp[v] = cl;
29        for (int u: adj_t[v])
30            if (comp[u] == -1)
31                dfs2(u, cl);
32    }
33
34    // READ: resuelve la formula (Kosaraju: dfs1 sobre el grafo
35    // original, dfs2 sobre el transpuesto). Devuelve false si NO
36    // es
37    // satisfacible (algun x_i y !x_i quedaron en la misma SCC).
38    bool solve_2SAT() {
39        order.clear();
40        used.assign(n_vertices, false);
41        for (int i = 0; i < n_vertices; ++i)
42            if (!used[i])
43                dfs1(i);
44
45        comp.assign(n_vertices, -1);
46        for (int i = 0, j = 0; i < n_vertices; ++i) {
47            int v = order[n_vertices - i - 1];
48            if (comp[v] == -1)
49                dfs2(v, j++);
50        }
51
52        assignment.assign(n_vars, false);
53        for (int i = 0; i < n_vertices; i += 2) {

```

```

54        if (comp[i] == comp[i + 1])
55            return false; // x_i y !x_i en la misma SCC ->
56            // contradiccion, no satisfacible
57
58        // comp[] queda en orden topologico INVERSO (Kosaraju
59        // procesa por orden de salida decreciente); el literal
60        // con comp MAYOR aparece "despues" en ese orden inverso,
61        // asi que es el que se elige como verdadero
62        assignment[i / 2] = comp[i] > comp[i + 1];
63    }
64
65    return true;
66    } // O(n + m)
67
68    // WRITE: agrega la clausula (literal a con negacion na) OR
69    // (literal b con negacion nb). Ver Implication Graph para la
70    // deducccion de estas dos aristas.
71    void add_disjunction(int a, bool na, int b, bool nb) {
72        a = 2 * a ^ na;
73        b = 2 * b ^ nb;
74        int neg_a = a ^ 1;
75        int neg_b = b ^ 1;
76        adj[neg_a].push_back(b);
77        adj[neg_b].push_back(a);
78        adj_t[b].push_back(neg_a);
79        adj_t[a].push_back(neg_b);
80    } // O(1)
81
82    //
83    // -----
84    // PATRONES 2-SAT COMPLETOS (modificaciones y usos)
85    // -----
86
87    // 1. A debe ser verdadero (forzar A = true)
88    // Clausula: (A OR A), equivale a !A => A
89    // solver.add_disjunction(i, false, i, false);
90
91    // 2. A debe ser falso (forzar A = false)
92    // Clausula: (!A OR !A), equivale a A => !A
93    // solver.add_disjunction(i, true, i, true);
94
95    // 3. Si A entonces B (A => B)
96    // Clausula: (!A OR B), equivale a A => B, !B => !A
97    // solver.add_disjunction(i, true, j, false);
98
99    // 4. Si no A entonces no B (!A => !B)
100    // Clausula: (A OR !B), equivale a !A => !B, B => A
101    // solver.add_disjunction(i, false, j, true);
102
103    // 5. Al menos uno verdadero (A OR B)
104    // Clausula: (A OR B), equivale a !A => B, !B => A
105    // solver.add_disjunction(i, false, j, false);
106
107    // 6. A lo mas uno verdadero / mutuamente excluyentes (!A OR
108    // !B)
109    // Clausula: (!A OR !B), equivale a A => !B, B => !A
110    // solver.add_disjunction(i, true, j, true);
111
112    // 7. A y B deben ser iguales (A <=> B)
113    // Clausulas: (A OR !B) y (!A OR B)
114    // solver.add_disjunction(i, false, j, true); // B => A
115    // solver.add_disjunction(i, true, j, false); // A => B
116
117    // 8. A y B deben ser diferentes (A XOR B)
118    // Clausulas: (A OR B) y (!A OR !B)
119    // solver.add_disjunction(i, false, j, false); // !A=>B, !B
120    // =>A
121    // solver.add_disjunction(i, true, j, true); // A=>!B, B
122    // =>!A
123    };

```

Ejemplo de funcionamiento:

```

1 // formula: (x0 OR x1) AND (!x0 OR x2) AND (x1 debe ser
2 // verdadero)
3
4 TwoSatSolver solver(3); // variables 0, 1, 2
5
6 solver.add_disjunction(0, false, 1, false); // (x0 OR x1)
7 solver.add_disjunction(0, true, 2, false); // (!x0 OR x2)
8 solver.add_disjunction(1, false, 1, false); // forzar x1 =
9 // true
10
11 bool satisfacible = solver.solve_2SAT();
12
13 cout << satisfacible;
14 // true
15
16 cout << solver.assignment[1];
17 // true (forzado explicitamente)

```


Regla práctica: el chequeo de satisfacibilidad ($\text{comp}[i] == \text{comp}[i+1]$) y la construcción de la asignación ($\text{comp}[i] > \text{comp}[i+1]$) son **inseparables** del mismo recorrido de SCC —no hace falta ninguna estructura ni pasada adicional para obtener la asignación una vez que la fórmula resultó satisfacible; toda la dificultad real de 2-SAT está en el paso de Implication Graph (traducir correctamente cada restricción), no en esta parte, que es siempre el mismo código sin importar el problema.

23 Backtracking y Problemas Clásicos

23.1 Problema de las N Reinas

Idea: colocar N reinas en un tablero $N \times N$ tal que **ninguna** ataque a otra (ni misma fila, ni misma columna, ni ninguna diagonal). Se resuelve con backtracking colocando **una reina por fila** (eso ya garantiza que nunca compartan fila): en cada fila, se prueba cada columna disponible, se verifica que no choque con las reinas ya colocadas, se coloca y se avanza a la siguiente fila; si ninguna columna de la fila actual funciona, se retrocede ("backtrack") y se prueba la siguiente opción de la fila anterior. La clave de eficiencia es **podar temprano**: en vez de recalcular columnas/diagonales ocupadas recorriendo todas las reinas ya puestas, se mantienen tres arreglos booleanos (columna, diagonal fila+columna, diagonal fila-columna) que se marcan y desmarcan en $O(1)$.

¿Por qué usarlo? Es el ejemplo canónico de backtracking con poda por restricciones: sin las tres marcas de columna/diagonales, la verificación de "¿esta posición es válida?" costaría $O(N)$ por candidato (recorrer todas las reinas puestas), multiplicando el costo total. Con las marcas, cada verificación es $O(1)$, dejando que la poda temprana (cortar ramas inválidas apenas se detectan) sea lo que realmente controla el tamaño del árbol de búsqueda explorado.

Casos de uso:

- Encontrar **una** solución válida para N reinas.
- Contar **cuántas** soluciones distintas existen para un N dado (variante más común en jueces de práctica).
- Listar/imprimir **todas** las soluciones explícitas (cada una como una permutación de columnas, una por fila).
- Como plantilla base de backtracking con restricciones posicionales, reutilizable para variantes con obstáculos u otras piezas de ajedrez (ver Variaciones, siguiente subsección).

Complejidad: exponencial en el peor caso ($O(N!)$ aproximado sin poda), pero la poda por columna/diagonales en $O(1)$ hace que en la práctica sea manejable hasta $N \sim 15\text{--}20$ en tiempo de competencia.

```
1 struct NReinas {
2     int n;
3     vector<int> columnaPorFila; // columnaPorFila[fila] =
        columna
4     // donde se puso la reina de esa fila
5     vector<bool> colOcupada, diag1Ocupada, diag2Ocupada;
6     long long soluciones;
7
8     // CREATE: tablero n x n vacio.
```

```
9     NReinas(int n_) {
10         n = n_;
11         columnaPorFila.assign(n, -1);
12         colOcupada.assign(n, false);
13         diag1Ocupada.assign(2 * n, false); // diagonal fila+
        columna
14         diag2Ocupada.assign(2 * n, false); // diagonal fila-
        columna+n
15         soluciones = 0;
16     } // O(n)
17
18     // READ: cuenta todas las soluciones validas via backtracking.
19
20     long long contarSoluciones() {
21         soluciones = 0;
22         backtrack(0);
23         return soluciones; // O(N!) peor caso
24     },
25     // mucho menor en
26     // la practica con poda
27     void backtrack(int fila) {
28         if (fila == n) {
29             soluciones++; // una reina por fila, todas colocadas
30             // sin conflicto -> solucion completa
31             return;
32         }
33
34         for (int col = 0; col < n; col++) {
35             int d1 = fila + col;
36             int d2 = fila - col + n;
37
38             if (colOcupada[col] || diag1Ocupada[d1] || diag2Ocupada[
39                 d2])
40                 continue; // poda: esta columna/diagonal ya esta
41                 // atacada, no vale la pena seguir
42
43             // coloca la reina
44             columnaPorFila[fila] = col;
45             colOcupada[col] = diag1Ocupada[d1] = diag2Ocupada[d2] =
46                 true;
47
48             backtrack(fila + 1);
49
50             // deshace ("backtrack" real)
51             colOcupada[col] = diag1Ocupada[d1] = diag2Ocupada[d2] =
52                 false;
53         }
54     }
55
56     // READ: encuentra UNA solucion (se detiene en la primera).
57     // Devuelve true si existe, dejando columnaPorFila[] con la
58     // solucion encontrada.
59     bool encontrarUna() {
60         return backtrackUna(0);
61     }
62
63     bool backtrackUna(int fila) {
64         if (fila == n) return true;
65
66         for (int col = 0; col < n; col++) {
67             int d1 = fila + col;
68             int d2 = fila - col + n;
69
70             if (colOcupada[col] || diag1Ocupada[d1] || diag2Ocupada[
71                 d2])
72                 continue;
73
74             columnaPorFila[fila] = col;
75             colOcupada[col] = diag1Ocupada[d1] = diag2Ocupada[d2] =
76                 true;
77
78             if (backtrackUna(fila + 1)) return true; // exito:
79             // corta
80             // de inmediato,
81             // sin deshacer
82             // (ya se
83             // encontro
84             // la respuesta)
85
86             colOcupada[col] = diag1Ocupada[d1] = diag2Ocupada[d2] =
87                 false;
88         }
89         return false;
90     }
91 };
```

Ejemplo de funcionamiento:

```
1 NReinas nr(8);
2
```

```

3 cout << nr.contarSoluciones();
4 // 92 (el numero clasico de soluciones para 8 reinas)
5
6 NReinas nr2(4);
7 bool encontrada = nr2.encontrarUna();
8
9 cout << encontrada;
10 // true
11
12 for (int c : nr2.columnaPorFila) cout << c << " ";
13 // 1 3 0 2 (una de las dos soluciones validas para 4 reinas)

```

Nota: la razón de que `contarSoluciones()` y `encontrarUna()` sean funciones separadas (en vez de una sola con un flag) es que `encontrarUna()` puede **cortar inmediatamente** al éxito sin deshacer el estado (ya no importa, terminó la búsqueda), mientras que `contarSoluciones()` **siempre** debe deshacer y seguir explorando otras ramas para encontrar el resto de soluciones — mezclar ambas lógicas en una sola función suele ser la fuente más común de bugs en esta plantilla.

23.2 Variaciones del Problema de las N Reinas

Idea: el esqueleto de backtracking de N Reinas —una decisión por fila, poda con arreglos de conflicto marcados en $O(1)$ — se reutiliza casi sin cambios para varias variantes que cambian **qué** cuenta como conflicto o **qué** se pide como respuesta, en vez de cambiar la estructura del backtracking en sí.

¿Por qué usarlo? Reconocer que una variante es "N Reinas con un ajuste" ahorra tener que rediseñar el backtracking desde cero —el patrón de poda por marcas booleanas (o bitmask, ver más abajo) es directamente transferible.

Casos de uso y sus modificaciones puntuales:

- **Tablero con casillas bloqueadas:** algunas celdas no admiten reina (obstáculos fijos) — se agrega un chequeo extra de "¿esta celda está bloqueada?" antes de colocar.
- **N Reinas con bitmask** (optimización de constante): en vez de tres `vector<bool>`, se usan tres enteros donde cada bit representa una columna/diagonal ocupada — las operaciones de marcar/desmarcar se vuelven operaciones a nivel de bits, más rápidas en la práctica para N grande.
- **Listar todas las soluciones explícitas** (no solo contarlas): se guarda una copia de `columnaPorFila` cada vez que se completa una solución, en vez de solo incrementar un contador.
- **Reinas con posiciones prefijadas:** algunas filas ya tienen su reina puesta de antemano — se inicializan las marcas de conflicto con esas reinas antes de empezar el backtracking, y el recorrido de filas empieza después de la última fila prefijada.

Complejidad: igual orden que N Reinas base (exponencial en el peor caso con poda), la variante con bitmask reduce la **constante** por operación pero no el orden asintótico.

```

1 struct NReinasVariantes {
2     int n;
3     vector<int> columnaPorFila;
4     vector<vector<bool>> bloqueada; // bloqueada[fila][col] =
                                     true

```

```

5 // si esa celda no admite reina
6 vector<vector<int>> todasLasSoluciones; // guarda cada
7 // solución
8 // completa encontrada
9
10 // bitmask: cols, diag1, diag2 representan columnas/
11 // diagonales
12 // ocupadas como bits encendidos, en vez de vector<bool>
13 int cols, diag1, diag2;
14
15 // CREATE: tablero n x n con posibles celdas bloqueadas.
16 NReinasVariantes(int n_, vector<vector<bool>>& bloqueada_) {
17     n = n_;
18     bloqueada = bloqueada_;
19     columnaPorFila.assign(n, -1);
20     cols = diag1 = diag2 = 0;
21 } // O(n^2)
22
23 // VARIANTE 1: tablero con casillas bloqueadas -- mismo
24 // backtracking, un chequeo extra por candidato.
25 long long contarConBloqueos() {
26     long long total = 0;
27     backtrackBloqueos(0, total);
28     return total;
29 }
30
31 void backtrackBloqueos(int fila, long long& total) {
32     if (fila == n) { total++; return; }
33
34     for (int col = 0; col < n; col++) {
35         if (bloqueada[fila][col]) continue; // celda no
36         // disponible,
37         // ni siquiera se
38         // evalua conflicto
39
40         int d1 = fila + col, d2 = fila - col + n;
41         bool ocupada = (cols & (1 << col)) ||
42             (diag1 & (1 << d1)) ||
43             (diag2 & (1 << d2));
44         if (ocupada) continue;
45
46         // VARIANTE 2: bitmask en vez de vector<bool> -- marcar
47         // es un OR, desmarcar es un XOR con la misma mascara
48         cols |= (1 << col);
49         diag1 |= (1 << d1);
50         diag2 |= (1 << d2);
51
52         backtrackBloqueos(fila + 1, total);
53
54         cols ^= (1 << col);
55         diag1 ^= (1 << d1);
56         diag2 ^= (1 << d2);
57     }
58 } // O(N!) peor caso
59
60 // VARIANTE 3: listar TODAS las soluciones explícitas, no
61 // solo contarlas -- se guarda una copia completa al llegar a fila
62 // ==n.
63 void listarTodas() {
64     todasLasSoluciones.clear();
65     backtrackListar(0);
66 }
67
68 void backtrackListar(int fila) {
69     if (fila == n) {
70         todasLasSoluciones.push_back(columnaPorFila); // copia
71         // completa
72         return;
73     }
74
75     for (int col = 0; col < n; col++) {
76         int d1 = fila + col, d2 = fila - col + n;
77         bool ocupada = (cols & (1 << col)) ||
78             (diag1 & (1 << d1)) ||
79             (diag2 & (1 << d2));
80         if (ocupada || bloqueada[fila][col]) continue;
81
82         columnaPorFila[fila] = col;
83         cols |= (1 << col); diag1 |= (1 << d1); diag2 |= (1 << d2);
84
85         backtrackListar(fila + 1);
86
87         cols ^= (1 << col); diag1 ^= (1 << d1); diag2 ^= (1 << d2);
88     }
89 } // O(N! * N) por
90 // costo de copiar cada
91 // solución encontrada
92
93 // VARIANTE 4: reinas con posiciones prefijadas -- se marcan

```

```

91  las
92  // reinas ya dadas ANTES de empezar el backtracking, y este
93  // arranca en la primera fila SIN prefijar.
94  long long contarConPrefijadas(vector<int>& prefijadas) {
95      // prefijadas[fila] = columna prefijada, o -1 si esa fila
96      // es
97      // libre y debe decidirse por backtracking
98      int filaInicio = 0;
99      for (int fila = 0; fila < n; fila++) {
100          if (prefijadas[fila] == -1) { filaInicio = fila; break; }
101
102          int col = prefijadas[fila];
103          columnaPorFila[fila] = col;
104          cols |= (1 << col);
105          diag1 |= (1 << (fila + col));
106          diag2 |= (1 << (fila - col + n));
107          filaInicio = fila + 1;
108      }
109
110      long long total = 0;
111      backtrackBloqueos(filaInicio, total);
112      return total;
113  } // mismo orden que
114  // backtrackBloqueos,
115  // arrancando mas
116  // adentro del arbol
117  };

```

Ejemplo de funcionamiento:

```

1  // tablero 4x4 sin bloqueos
2  vector<vector<bool>> sinBloqueos(4, vector<bool>(4, false));
3
4  NReinasVariantes nrv(4, sinBloqueos);
5  nrv.listarTodas();
6
7  cout << nrv.todasLasSoluciones.size();
8  // 2 (las 2 soluciones classicas de 4 reinas)
9
10 for (int c : nrv.todasLasSoluciones[0]) cout << c << " ";
11 // 1 3 0 2
12
13 // tablero 4x4 CON la celda (0,0) bloqueada
14 vector<vector<bool>> conBloqueo(4, vector<bool>(4, false));
15 conBloqueo[0][0] = true;
16
17 NReinasVariantes nrv2(4, conBloqueo);
18 cout << nrv2.contarConBloqueos();
19 // 1 (una de las 2 soluciones originales usaba (0,0), la otra
20    no)

```

Regla práctica: ante cualquier variante de N Reinas, identifica primero **qué cambia**: si cambia **qué es válido** (bloqueos, reinas prefijadas), el ajuste es un chequeo extra o una inicialización antes del backtracking; si cambia **qué se pide** (contar vs. listar vs. encontrar una), el ajuste es qué se hace al llegar a `fila == n` (incrementar, copiar, o retornar `true` de inmediato). El núcleo del backtracking —una decisión por fila, poda por columna/diagonales, deshacer al retroceder— casi nunca cambia entre variantes.

23.3 Tour de Caballo

Idea: el Tour de Caballo (ya visto en la sección de grillas/laberintos con la heurística de Warnsdorff) encaja también aquí como el ejemplo clásico de backtracking sobre un tablero: hay que visitar **cada** casilla exactamente una vez moviéndose como caballo de ajedrez, y a diferencia de N Reinas (una decisión por fila, con poda de conflictos), aquí la decisión es "¿a cuál de los hasta 8 saltos válidos me muevo desde la casilla actual?", con **deshacer explícito** (marcar la casilla como no visitada) cada vez que una rama se atasca antes de completar el tour. Se repite en esta sección porque el patrón de backtracking puro —sin la heurística de Warnsdorff, solo prueba y retrocede— es más instructivo para entender la mecánica general antes de optimizar.

¿Por qué usarlo? Es el contraste directo con N Reinas: en N Reinas la poda viene de **restricciones**

estructurales (columna/diagonal), aquí la única "poda" real disponible sin heurística es "no pisar una casilla ya visitada" —eso hace que el backtracking puro sea mucho más lento en la práctica (el árbol de búsqueda crece casi sin control para tableros grandes), y es justamente la motivación para agregar Warnsdorff **encima** de este mismo esqueleto cuando el tablero es más grande que unos pocos $n \times n$.

Casos de uso:

- Encontrar (o verificar la existencia de) un tour de caballo completo en tableros pequeños ($n \lesssim 5-6$ sin Warnsdorff).
- Como plantilla base para introducir el patrón de "backtracking con deshacer explícito sobre una grilla", antes de agregar la heurística de ordenamiento de candidatos.
- Punto de comparación directo contra N Reinas: mismo esqueleto (probar, recursar, deshacer si falla), pero con condición de éxito distinta (visitar **todas** las casillas en vez de colocar N piezas sin conflicto).

Complejidad: exponencial sin poda adicional ($O(8^{n^2})$ aproximado en el peor caso) — ver la sección de grillas para la versión con Warnsdorff, que en la práctica corre casi lineal para n razonables.

```

1  struct TourCaballoBacktracking {
2      int n;
3      vector<vector<int>> tablero; // tablero[r][c] = orden de
4      // visita
5      // (-1 si no visitada)
6      int dr[8] = {-2,-2,-1,-1, 1, 1, 2, 2};
7      int dc[8] = {-1, 1,-2, 2,-2, 2,-1, 1};
8
9      // CREATE: tablero n x n vacio.
10     TourCaballoBacktracking(int n_) {
11         n = n_;
12         tablero.assign(n, vector<int>(n, -1)); // O(n^2)
13     }
14
15     bool dentro(int r, int c) {
16         return r >= 0 && r < n && c >= 0 && c < n;
17     }
18
19     // READ: intenta construir un tour completo empezando en
20     // (sr, sc), SIN ninguna heurística de ordenamiento -- prueba
21     // los 8 saltos en orden fijo, retrocede si se atasca.
22     bool resolver(int sr, int sc) {
23         tablero[sr][sc] = 0;
24         bool exito = backtrack(sr, sc, 1);
25         if (!exito) tablero[sr][sc] = -1; // deshacer tambien el
26         // punto de partida si
27         // no se encontro nada
28         return exito;
29     } // exponencial sin
30     // poda
31
32     bool backtrack(int r, int c, int movimiento) {
33         if (movimiento == n * n) return true; // todas las
34         // casillas visitadas
35
36         for (int d = 0; d < 8; d++) {
37             int nr = r + dr[d], nc = c + dc[d];
38
39             if (!dentro(nr, nc) || tablero[nr][nc] != -1)
40                 continue; // fuera del tablero o ya visitada
41
42             tablero[nr][nc] = movimiento; // prueba
43
44             if (backtrack(nr, nc, movimiento + 1))
45                 return true; // exito: no hace falta deshacer, ya
46                 // se encontro la respuesta completa
47
48             tablero[nr][nc] = -1; // deshacer (backtrack real):
49             // esta rama no llevo a un tour
50             // completo
51         }
52
53         return false; // ningun salto desde (r,c) llevo a exito
54     }
55 }

```

```
52 }
53 };
```

Ejemplo de funcionamiento:

```
1 TourCaballoBacktracking tcb(5);
2
3 bool encontrado = tcb.resolver(0, 0);
4
5 cout << encontrado;
6 // true
7
8 // tcb.tablero[r][c] tiene el orden (0..24) en que se visito
9 // cada
10 // casilla -- sin Warnsdorff, el orden exacto encontrado puede
11 // diferir del de la version con heurística, pero ambos son
12 // validos si existen
```

Regla práctica: este esqueleto (probar candidato, recursar, deshacer si la rama falla) es el mismo de N Reinas y de cualquier backtracking clásico —lo único que cambia entre problemas es **qué** se prueba en cada paso (columna vs. salto de caballo) y **cuándo** se declara éxito (fila == n vs. movimiento == $n \times n$). Para tableros mayores a unos pocos $n \times n$, agrega la heurística de Warnsdorff (sección de grillas) **encima** de este mismo backtracking —no cambia la estructura, solo el **orden** en que se prueban los 8 saltos en cada paso.

24 Advanced Graph Algorithms

24.1 Centroid Decomposition

Idea: Centroid Decomposition descompone un árbol en una **jerarquía de centroides** para resolver queries que involucran "caminos entre cualquier par de nodos" en $O(\log n)$ niveles: se encuentra el **centroide** del árbol (el nodo cuya remoción deja componentes de tamaño $\leq n/2$ cada una — no confundir con Centro de un Árbol, que minimiza distancia máxima; el centroide minimiza tamaño de componente al remover), se procesa toda la información que pasa **por** ese centroide, se **remueve** (lógicamente) del árbol, y se **recursiona** de forma independiente sobre cada componente resultante, cada una con su propio centroide. Como cada remoción parte el árbol en piezas de tamaño a lo más la mitad, la profundidad de esta jerarquía de centroides es $O(\log n)$ — y cualquier camino entre dos nodos u, v del árbol original pasa por **exactamente un** centroide de la jerarquía (el primero, en la recursión, que los separa en componentes distintas).

¿Por qué usarlo? Es la técnica estándar para problemas de "cuenta/encuentra pares de nodos que cumplen una condición sobre su camino" (ej. "cuántos pares están a distancia $\leq k$ "), donde recorrer todos los pares es $O(n^2)$: al procesar cada nivel de la jerarquía en $O(n \log n)$ total y haber solo $O(\log n)$ niveles, el costo total baja a $O(n \log^2 n)$. Es conceptualmente el análogo, en árboles, de *divide and conquer* sobre arreglos —"encuentra el punto medio, resuelve lo que cruza el medio, recursiona a los lados" —salvo que el "medio" aquí es un nodo (el centroide), no un índice.

Casos de uso:

- Contar pares de nodos (u, v) cuya distancia es **exactamente** k , o está en un rango dado — proceso: para cada centroide, contar cuántos caminos que pasan por él tienen la distancia buscada (usando las

distancias de cada nodo al centroide, agrupadas en un arreglo/mapa de frecuencias).

- Encontrar el par de nodos más cercano/lejano que cumple una restricción adicional (ej. "de distinto color").
- Responder queries offline de tipo "actualiza un nodo, consulta el más cercano nodo actualizado" (una aplicación frecuente combinada con la estructura de centroides para acotar la distancia recorrida por query).
- Como alternativa a Heavy-Light Decomposition cuando el problema es sobre **pares** de caminos, no sobre un único camino consultado repetidamente.

Complejidad: $O(n \log n)$ para construir toda la jerarquía (encontrar centroides, un DFS de tamaños por cada nivel de recursión — la suma total de tamaños across niveles es $O(n \log n)$); el costo de procesar cada centroide depende del problema, típicamente $O(\text{tam} \log \text{tam})$ por centroide, dando $O(n \log^2 n)$ total.

```
1 struct CentroidDecomposition {
2     vector<vector<int>> adj;
3     vector<int> tam;
4     vector<bool> removido;
5     vector<int> padreCentroide; // padreCentroide[c] =
6     // centroide
7     // padre de c en la jerarquia
8     int n;
9
10    // CREATE: guarda el arbol y arranca la descomposicion
11    // completa
12    // desde la raiz de la jerarquia (nodo -1 = sin padre
13    // centroide).
14    CentroidDecomposition(vector<vector<int>>& adj_, int n_) {
15        adj = adj_;
16        n = n_;
17        tam.assign(n, 0);
18        removido.assign(n, false);
19        padreCentroide.assign(n, -1);
20
21        descomponer(0, -1);
22    } // O(n log n)
23
24    int calcularTam(int u, int padre) {
25        tam[u] = 1;
26        for (int v : adj[u])
27            if (v != padre && !removido[v])
28                tam[u] += calcularTam(v, u);
29        return tam[u];
30    }
31
32    // encuentra el centroide del componente que contiene a u,
33    // usando tamTotal (ya calculado con calcularTam) para
34    // decidir
35    // cuando un hijo excede n/2.
36    int encontrarCentroide(int u, int padre, int tamTotal) {
37        for (int v : adj[u])
38            if (v != padre && !removido[v] && tam[v] > tamTotal / 2)
39                return encontrarCentroide(v, u, tamTotal);
40        return u;
41    }
42
43    // procesa toda la informacion que pasa POR el centroide
44    // actual
45    // -- el cuerpo especifico depende del problema (aquí vacío,
46    // ver
47    // ejemplo de "contar pares a distancia k" mas abajo).
48    void procesarCentroide(int centroide) {
49        // placeholder: aquí va la logica especifica del problema,
50        // típicamente un DFS desde 'centroide' recolectando
51        // distancias de cada nodo del componente hacia el
52    }
53
54    // WRITE (recursivo): encuentra el centroide del componente
55    // que
56    // contiene a u, lo procesa, lo marca removido, y recursiona
57    // sobre cada componente resultante.
58    void descomponer(int u, int padre) {
59        int tamTotal = calcularTam(u, -1);
60        int centroide = encontrarCentroide(u, -1, tamTotal);
```



```

55     removido[centroide] = true;
56     padreCentroide[centroide] = padre;
57
58     procesarCentroide(centroide);
59
60     for (int v : adj[centroide])
61     if (!removido[v])
62     descomponer(v, centroide);
63 } // O(tam log tam)
64 // por nivel, O(log n)
65 // niveles -> O(n log n)
66 // total
67 };

```

Ejemplo de funcionamiento (contar pares a distancia exacta k):

```

1 // version especializada de procesarCentroide para "contar
2 // nodos a distancia EXACTAMENTE k" -- se agrega sobre el
3 // esqueleto
4 // de arriba:
5 //
6 // long long totalPares = 0;
7 // void procesarCentroideConteo(int centroide, int k) {
8 //     vector<int> distancias; // distancia de cada nodo
9 //     del
10 //     // componente actual al
11 //     centroide
12 //     function<void(int,int,int)> recolectar = [&](int u,
13 //     int p, int d) {
14 //         // distancias.push_back(d);
15 //         for (int v : adj[u])
16 //             if (v != p && !removido[v])
17 //                 recolectar(v, u, d + 1);
18 //     };
19 //     // cuenta pares DENTRO de cada subarbol de un hijo
20 //     // directo del
21 //     // centroide primero (para luego RESTARLOS -- evita
22 //     // contar
23 //     // // pares que en realidad no pasan por el centroide,
24 //     // // ambos en
25 //     // // el mismo hijo)
26 //     map<int,int> frecuencia;
27 //     frecuencia[0] = 1; // el propio centroide, distancia
28 //     0
29 //     //
30 //     for (int hijo : adj[centroide]) {
31 //         // if (removido[hijo]) continue;
32 //         //
33 //         distancias.clear();
34 //         recolectar(hijo, centroide, 1);
35 //         //
36 //         for (int d : distancias)
37 //             if (frecuencia.count(k - d))
38 //                 totalPares += frecuencia[k - d];
39 //         //
40 //         for (int d : distancias)
41 //             frecuencia[d]++;
42 //         //
43 //     }
44 // }
45
46 // arbol:
47 //
48 //     0
49 //     / \
50 //    1  2
51 //   /
52 //  3
53
54 vector<vector<int>> adj(4);
55 adj[0] = {1, 2};
56 adj[1] = {0, 3};
57 adj[2] = {0};
58 adj[3] = {1};
59
60 // contar pares a distancia EXACTAMENTE 2:
61 // (0,3) dist=2, (2,1) dist=2 -> 2 pares

```

Regla práctica: Centroid Decomposition vale la pena solo cuando el problema pregunta por **pares** de nodos o caminos **arbitrarios** — si el problema fija un único camino por query (como en Path Queries o HLD), Centroid Decomposition es sobre-ingeniería innecesaria. La señal típica es un enunciado que pregunta "cuántos pares..." o "existe algún par..." con una condición sobre la distancia o el camino entre ellos.

24.2 DSU on Tree

Idea: DSU on Tree (también llamado *small-to-large merging*, a pesar del nombre no usa la estructura DSU de Union-Find) resuelve queries **por subárbol** — "para cada nodo u , calcula algo sobre **todo** su subárbol" — evitando recalcular desde cero para cada nodo. La idea: al procesar un nodo u , primero se procesan (y se **descartan**) todos los hijos **livianos** (los que no son el hijo de mayor subárbol, igual que en Heavy-Light Decomposition), guardando su información solo temporalmente; luego se procesa el hijo **pesado** sin descartar su información (se deja "viva"); y finalmente se **reinserta** la información de los hijos livianos encima de la del hijo pesado, para tener el estado completo del subárbol de u sin haber recorrido cada nodo más que $O(\log n)$ veces en total.

¿Por qué usarlo? La alternativa ingenua —recorrer el subárbol completo de cada nodo para responder su query— cuesta $O(n^2)$ en el peor caso (un árbol en forma de cadena). El truco de "conservar la estructura del hijo pesado y solo reinsertar los livianos" es el mismo principio que sustenta la complejidad de HLD —cualquier nodo cambia de "hijo pesado a liviano" como mucho $O(\log n)$ veces en su camino a la raíz—, aplicado aquí a cuántas veces se re-inserta cada nodo en una estructura auxiliar en vez de a cuántas cadenas cruza un camino.

Casos de uso:

- "Para cada nodo, ¿cuál es el valor/color más frecuente en su subárbol?" (el ejemplo clásico, con un arreglo de frecuencias como estructura auxiliar).
- Contar cuántos valores **distintos** hay en el subárbol de cada nodo.
- Cualquier query de subárbol donde la estructura auxiliar soporte "agregar un elemento" y "responder la pregunta actual" en $O(1)$ o $O(\log n)$, pero **no** soporte eliminar eficientemente (si soportara borrado barato, Euler Tour + Fenwick ya bastaría, sin necesitar este patrón).

Complejidad: $O(n \log n)$ total — cada nodo se "agrega" a la estructura auxiliar como mucho $O(\log n)$ veces (una vez por cada antepasado del que sea hijo liviano, más una vez cuando su propio subárbol se procesa como hijo pesado).

```

1 struct DSUOnTree {
2     vector<vector<int>> adj;
3     vector<int> tam, hijoPesado, valor;
4     vector<long long> frecuencia; // estructura auxiliar:
5     // cuenta
6     // ocurrencias de cada valor
7     vector<long long> respuesta; // respuesta[u] = valor mas
8     // frecuente en el subarbol de u
9     long long maxFrecuencia, sumaDeMaximos;
10    int n;
11
12    // CREATE: guarda el arbol y los valores por nodo; calcula
13    // tamanos e identifica el hijo pesado de cada nodo (igual
14    // que en HLD).
15    DSUOnTree(vector<vector<int>>& adj_, vector<int>& valor_, int
16    n_) {
17        adj = adj_;
18        valor = valor_;
19        n = n_;
20        tam.assign(n, 0);
21        hijoPesado.assign(n, -1);
22        respuesta.assign(n, 0);

```



```

21     frecuencia.assign(*max_element(valor.begin(), valor.end())
22     + 1, 0);
23     calcularTamYPesado(0, -1);
24 } // O(n)
25
26 void calcularTamYPesado(int u, int padre) {
27     tam[u] = 1;
28     int mejorTam = -1;
29
30     for (int v : adj[u]) {
31         if (v == padre) continue;
32         calcularTamYPesado(v, u);
33         tam[u] += tam[v];
34         if (tam[v] > mejorTam) {
35             mejorTam = tam[v];
36             hijoPesado[u] = v;
37         }
38     }
39 } // O(n)
40
41 // agrega o quita (delta = +1 o -1) el valor de u a la
42 // estructura
43 // auxiliar, actualizando el maximo global sobre la marcha.
44 void agregar(int u, int padre, int delta) {
45     frecuencia[valor[u]] += delta;
46
47     if (frecuencia[valor[u]] > maxFrecuencia) {
48         maxFrecuencia = frecuencia[valor[u]];
49         sumaDeMaximos = valor[u];
50     } else if (frecuencia[valor[u]] == maxFrecuencia) {
51         sumaDeMaximos += valor[u]; // ejemplo: suma de todos
52         // los valores empatados
53         // en frecuencia maxima
54     }
55
56     for (int v : adj[u])
57         if (v != padre)
58             agregar(v, u, delta);
59 } // O(tamano del
60 // subarbol)
61
62 // READ (recursivo): procesa el subarbol de u con la tecnica
63 // de
64 // small-to-large; 'mantener' indica si u es el hijo PESADO
65 // de
66 // su padre (si es asi, su info NO se descarta al terminar).
67 void dfs(int u, int padre, bool mantener) {
68     // paso 1: procesa TODOS los hijos livianos primero,
69     // descartando su info despues (mantener = false)
70     for (int v : adj[u])
71         if (v != padre && v != hijoPesado[u])
72             dfs(v, u, false);
73
74     // paso 2: procesa el hijo pesado SIN descartar su info
75     if (hijoPesado[u] != -1)
76         dfs(hijoPesado[u], u, true);
77
78     // paso 3: agrega el propio u y REINSERTA todos los hijos
79     // livianos encima de la info ya viva del hijo pesado
80     agregar(u, padre, +1);
81     for (int v : adj[u])
82         if (v != padre && v != hijoPesado[u])
83             agregarSoloEsteSubarbol(v, u, +1);
84
85     // ahora la estructura auxiliar tiene el subarbol COMPLETO
86     // de u -> se puede responder la query de u
87     respuesta[u] = sumaDeMaximos;
88
89     // paso 4: si u es hijo liviano de su padre, se descarta
90     // TODA
91     // su informacion (para no contaminar al procesar hermanos)
92     if (!mantener) {
93         agregar(u, padre, -1);
94         for (int v : adj[u])
95             if (v != padre)
96                 agregarSoloEsteSubarbol(v, u, -1);
97         maxFrecuencia = 0;
98         sumaDeMaximos = 0;
99     }
100 } // O(n log n)
101
102 total
103
104 void agregarSoloEsteSubarbol(int u, int padre, int delta) {
105     agregar(u, padre, delta);
106 }
107
108 // READ: calcula respuesta[] para todo el arbol.
109 vector<long long> resolver() {
110     maxFrecuencia = 0;
111     sumaDeMaximos = 0;
112     dfs(0, -1, true); // la raiz siempre "mantiene" (no hay
113     // nada que descartar despues)

```

```

107     return respuesta;
108 } // O(n log n)
109 };

```

Ejemplo de funcionamiento:

```

1 // arbol:      0(val=1)
2 //           /  \
3 //        1(val=2) 2(val=1)
4 //           /
5 //        3(val=2)
6
7 vector<vector<int>> adj(4);
8 adj[0] = {1, 2};
9 adj[1] = {0, 3};
10 adj[2] = {0};
11 adj[3] = {1};
12
13 vector<int> valor = {1, 2, 1, 2};
14
15 DSUOnTree dot(adj, valor, 4);
16 auto resp = dot.resolver();
17
18 cout << resp[1];
19 // 2 (subarbol de 1: valores {2, 2} -> el 2 es el mas
20 //    frecuente)
21
22 cout << resp[0];
23 // 3 (subarbol completo: valores {1,2,1,2} -> 1 y 2 empatan
24 //    en
25 //    frecuencia 2, suma de ambos = 1+2 = 3, segun la
26 //    definicion
27 //    de "respuesta" de este ejemplo especifico)

```

Regla práctica: DSU on Tree comparte con HLD el mismo concepto de "hijo pesado" y la misma garantía de $O(\log n)$ reinserciones por nodo, pero resuelve un problema distinto: HLD responde queries sobre **caminos** descomponiéndolos en rangos; DSU on Tree responde queries sobre **subárboles completos** reutilizando la estructura auxiliar del hijo pesado en vez de reconstruirla desde cero. Úsalo quando la pregunta es "algo sobre **todo** el subárbol de cada nodo" y la estructura auxiliar (usualmente un arreglo/mapa de frecuencias) es cara de reconstruir pero barata de **extender** elemento por elemento.

24.3 Hopcroft-Karp

Idea: Hopcroft-Karp mejora a Kuhn (ver Bipartite Matching) con la misma idea que Dinic mejora a Ford-Fulkerson: en vez de buscar **un** camino aumentante a la vez con DFS, busca **muchos** caminos aumentantes **disjuntos** de una sola vez, todos de la **misma longitud mínima**. Cada **fase** tiene dos partes: (1) un BFS desde todos los nodos libres de L que calcula la distancia mínima a un nodo libre de R (alternando aristas normales y de matching, igual que las capas de Dinic); (2) una serie de DFS que extraen **todos** los caminos aumentantes disjuntos de esa longitud mínima, marcando nodos usados para no repetirlos dentro de la misma fase. Se puede demostrar que el número total de fases está acotado por $O(\sqrt{V})$ — de ahí la complejidad total.

¿Por qué usarlo? Kuhn es $O(V \cdot E)$; Hopcroft-Karp baja eso a $O(E\sqrt{V})$ —la misma mejora que Dinic da sobre Ford-Fulkerson, pero especializada al caso bipartito puro (sin pasar por la maquinaria completa de flujo con capacidades). Es preferible sobre "Maximum Bipartite Matching vía Dinic" cuando **solo** se necesita matching (sin capacidades extra ni combinarlo con otro modelo de flujo) — mismo orden de complejidad, pero con una implementación más directa y sin la sobrecarga de construir la red completa con superfuente/supersumidero.

Casos de uso:

- Los mismos que Bipartite Matching / Maximum Bipartite Matching, cuando V es grande y $O(V \cdot E)$ de Kuhn no alcanza el límite de tiempo.
- Como reemplazo directo de Kuhn en cualquier problema que ya modele matching bipartito puro (sin capacidades adicionales ni pesos).
- Preferible sobre Dinic + construcción de flujo cuando el problema es **estrictamente** matching bipartito (Dinic sigue siendo necesario si además hay capacidades > 1 o el problema mezcla matching con otras restricciones de flujo).

Complejidad: $O(E\sqrt{V})$ — $O(\sqrt{V})$ fases, cada una $O(E)$ (un BFS más varios DFS que en conjunto tocan cada arista una vez por fase).

```

1 struct HopcroftKarp {
2     vector<vector<int>> adj;    // adj[u] = vecinos en R, para
3                               // u en L
4     vector<int> matchL, matchR, dist;
5     int nL, nR;
6     const int INF = INT_MAX;
7     const int LIBRE = -1;
8
9     // CREATE: guarda el grafo bipartito (nL nodos en L, adj[u]
10    // lista
11    // los vecinos de u en R). nR = tamaño del lado R.
12    HopcroftKarp(vector<vector<int>>& adj_, int nL_, int nR_) {
13        adj = adj_;
14        nL = nL_;
15        nR = nR_;
16        matchL.assign(nL, LIBRE);
17        matchR.assign(nR, LIBRE);
18    }
19    // FASE 1: BFS multi-fuente desde todos los nodos LIBRES de L,
20    // calculando dist[] SOLO sobre nodos de L (alternando
21    // normales L->R con aristas de matching R->L). Devuelve true
22    // si
23    // algún nodo libre de R fue alcanzado (hay al menos un
24    // camino
25    // aumentante en esta fase).
26    bool bfs() {
27        queue<int> q;
28        dist.assign(nL, INF);
29
30        for (int u = 0; u < nL; u++)
31            if (matchL[u] == LIBRE) {
32                dist[u] = 0;
33                q.push(u);
34            }
35
36        bool encontroLibre = false;
37
38        while (!q.empty()) {
39            int u = q.front(); q.pop();
40
41            for (int v : adj[u]) {
42                int siguienteL = matchR[v];
43
44                if (siguienteL == LIBRE) {
45                    encontroLibre = true;    // u es libre: fin de un
46                    // camino aumentante de
47                    // esta longitud mínima
48                } else if (dist[siguienteL] == INF) {
49                    dist[siguienteL] = dist[u] + 1;
50                    q.push(siguienteL);
51                }
52            }
53        }
54
55        return encontroLibre;
56    }
57    // FASE 2: DFS que busca UN camino aumentante respetando dist
58    // (solo avanza a niveles crecientes, igual que el grafo por
59    // niveles de Dinic), actualizando matchL/matchR al
60    // encontrarlo.
61    bool dfs(int u) {
62        for (int v : adj[u]) {
63            int siguienteL = matchR[v];

```

```

64            if (siguienteL == LIBRE ||
65                (dist[siguienteL] == dist[u] + 1 && dfs(siguienteL))) {
66                matchL[u] = v;
67                matchR[v] = u;
68                return true;
69            }
70        }
71
72        dist[u] = INF;    // u agotado en esta fase, no vale la pena
73        // volver a intentarlo dentro de la misma fase
74        return false;
75    }
76    // READ: tamaño del matching máximo.
77    int matchingMaximo() {
78        int total = 0;
79
80        while (bfs()) {
81            for (int u = 0; u < nL; u++)
82                if (matchL[u] == LIBRE && dfs(u))
83                    total++;
84        }
85
86        return total;
87    }
88    // O(E * sqrt(V))

```

Ejemplo de funcionamiento:

```

1 // L = {0, 1, 2}, R = {0, 1, 2}
2 // 0 - {0, 1}
3 // 1 - {0}
4 // 2 - {1, 2}
5
6 vector<vector<int>> adj(3);
7 adj[0] = {0, 1};
8 adj[1] = {0};
9 adj[2] = {1, 2};
10
11 HopcroftKarp hk(adj, 3, 3);
12
13 cout << hk.matchingMaximo();
14 // 3 (mismo resultado que Kuhn y que Maximum Bipartite
15 // Matching
16 // via Dinic sobre el mismo grafo)

```

Regla práctica: Kuhn, Maximum Bipartite Matching vía Dinic, y Hopcroft-Karp **siempre** dan el mismo tamaño de matching — la elección es de rendimiento y de cuánto código ya tienes: Kuhn para V chico o cuando la simplicidad importa más que la velocidad; Hopcroft-Karp cuando V es grande y el problema es matching puro sin nada más; Dinic + construcción de flujo cuando el matching es solo una parte de un modelo de flujo más grande (capacidades, múltiples fuentes, etc.) y ya se necesita esa infraestructura de todas formas.

24.4 Min-Cost Max-Flow

Idea: Min-Cost Max-Flow (MCMF) resuelve una versión más rica del problema de flujo: cada arista tiene, además de una **capacidad**, un **costo por unidad** de flujo que pasa por ella, y se busca el flujo máximo de **menor costo total** posible (o, en la variante más común, "envía exactamente F unidades al menor costo"). Se resuelve con la misma estructura de grafo residual de Ford-Fulkerson/Dinic, pero en vez de buscar **cualquier** camino aumentante, en cada iteración se busca el camino aumentante de **menor costo** con **Bellman-Ford** o **SPFA** (necesarios porque las aristas reversas tienen costo **negativo** — "deshacer" un envío recupera el costo pagado, así que Dijkstra normal no aplica directamente sin potenciales de Johnson).

¿Por qué usarlo? Es la generalización que responde a cualquier problema de flujo que además pida **minimizar/maximizar un costo** asociado a las unidades

transportadas, algo que ni Ford-Fulkerson ni Dinic pueden resolver por sí solos (ellos solo optimizan **cantidad**, no costo). Es también la herramienta estándar para **matching bipartito ponderado** (asignación de costo mínimo, ver casos de uso) —donde Hopcroft-Karp/Kuhn ya no alcanzan porque esos solo maximizan **cantidad** de parejas, no el costo/beneficio total.

Casos de uso:

- Problema de asignación (*assignment problem*): asignar n trabajadores a n tareas minimizando el costo total, cuando cada trabajador **debe** tomar exactamente una tarea —red bipartita con capacidad 1 y costo por arista, buscando flujo de n unidades al menor costo.
- Transporte de mercancía entre orígenes y destinos con costo por unidad transportada distinto según la ruta, minimizando el costo total de satisfacer toda la demanda.
- Como generalización de Bipartite Matching cuando las parejas no son solo "compatible/incompatible" sino que tienen un **costo o beneficio** asociado.
- Cualquier problema de flujo donde el enunciado mencione explícitamente un **costo por unidad** además de una capacidad.

Complejidad: $O(F \cdot E)$ con SPFA (Bellman-Ford optimizado) por iteración, donde F es el valor total de flujo enviado — cada iteración envía al menos 1 unidad por el camino más barato encontrado. Para F grande, existen variantes con potenciales de Johnson + Dijkstra ($O(F \cdot E \log V)$) pero con constante mucho menor), fuera del alcance de esta plantilla base.

```

1 struct MinCostMaxFlow {
2     struct Arista {
3         int destino; long long capacidad, costo; int reversa;
4     };
5     vector<vector<Arista>> adj;
6     int n;
7
8     // CREATE: grafo residual vacio de n nodos.
9     MinCostMaxFlow(int n_) {
10         n = n_;
11         adj.assign(n, {});
12     } // O(n)
13
14     // WRITE: agrega arista dirigida u->v con capacidad c y costo
15     // costo POR UNIDAD. La reversa tiene capacidad 0 y costo
16     // NEGATIVO (deshacer un envio recupera lo pagado).
17     void agregarArista(int u, int v, long long c, long long costo) {
18         adj[u].push_back({v, c, costo, (int)adj[v].size()});
19         adj[v].push_back({u, 0, -costo, (int)adj[u].size() - 1});
20     } // O(1)
21
22     // busca el camino de MENOR COSTO de s a t en el grafo
23     // residual
24     // actual con SPFA (Bellman-Ford + cola, tolera costos
25     // negativos
26     // de las aristas reversas). Devuelve false si t no es
27     // alcanzable.
28     bool spfa(int s, int t, vector<long long>& dist,
29         vector<int>& padreNodo, vector<int>& padreArista) {
30         dist.assign(n, LLONG_MAX);
31         vector<bool> enCola(n, false);
32         dist[s] = 0;
33
34         queue<int> q;
35         q.push(s);
36         enCola[s] = true;
37
38         while (!q.empty()) {
39             int u = q.front(); q.pop();
40             enCola[u] = false;

```

```

39         for (int i = 0; i < (int)adj[u].size(); i++) {
40             auto& a = adj[u][i];
41             if (a.capacidad <= 0) continue;
42             if (dist[u] + a.costo >= dist[a.destino]) continue;
43
44             dist[a.destino] = dist[u] + a.costo;
45             padreNodo[a.destino] = u;
46             padreArista[a.destino] = i;
47
48             if (!enCola[a.destino]) {
49                 q.push(a.destino);
50                 enCola[a.destino] = true;
51             }
52         }
53     }
54
55     return dist[t] != LLONG_MAX;
56 } // O(V * E) peor caso
57
58 // READ: flujo maximo de s a t, y su costo total minimo
59 // asociado
60 // (devuelve {flujoTotal, costoTotal}).
61 pair<long long, long long> flujoCostoMinimo(int s, int t) {
62     long long flujoTotal = 0, costoTotal = 0;
63     vector<long long> dist(n);
64     vector<int> padreNodo(n), padreArista(n);
65
66     while (spfa(s, t, dist, padreNodo, padreArista)) {
67         // cuello de botella del camino encontrado (igual que en
68         // Ford-Fulkerson, pero sobre el camino de MENOR COSTO)
69         long long cuello = LLONG_MAX;
70         for (int u = t; u != s; u = padreNodo[u])
71             cuello = min(cuello, adj[padreNodo[u]][padreArista[u]].capacidad);
72
73         for (int u = t; u != s; u = padreNodo[u]) {
74             auto& a = adj[padreNodo[u]][padreArista[u]];
75             a.capacidad -= cuello;
76             adj[u][a.reversa].capacidad += cuello;
77         }
78
79         flujoTotal += cuello;
80         costoTotal += cuello * dist[t];
81     }
82
83     return {flujoTotal, costoTotal};
84 } // O(F * V * E)
85
86 // READ: costo minimo para enviar EXACTAMENTE F unidades (
87 // asume
88 // que existe un flujo de al menos F -- verificar con
89 // flujoCostoMinimo primero si no se esta seguro).
90 long long costoParaFlujoExacto(int s, int t, long long F) {
91     long long flujoTotal = 0, costoTotal = 0;
92     vector<long long> dist(n);
93     vector<int> padreNodo(n), padreArista(n);
94
95     while (flujoTotal < F && spfa(s, t, dist, padreNodo,
96         padreArista)) {
97         long long cuello = F - flujoTotal;
98         for (int u = t; u != s; u = padreNodo[u])
99             cuello = min(cuello, adj[padreNodo[u]][padreArista[u]].capacidad);
100
101         for (int u = t; u != s; u = padreNodo[u]) {
102             auto& a = adj[padreNodo[u]][padreArista[u]];
103             a.capacidad -= cuello;
104             adj[u][a.reversa].capacidad += cuello;
105         }
106
107         flujoTotal += cuello;
108         costoTotal += cuello * dist[t];
109     }
110
111     return costoTotal; // si flujoTotal < F al salir, no
112     // existia
113     // suficiente flujo -- verificar aparte
114 } // O(F * V * E)

```

Ejemplo de funcionamiento (asignación de costo mínimo):

```

1 // 2 trabajadores, 2 tareas, matriz de costos:
2 //      tarea0  tarea1
3 // trab0:  4      2
4 // trab1:  3      5
5 // (fuente=0, trab0=1, trab1=2, tarea0=3, tarea1=4, sumidero=5)
6
7 MinCostMaxFlow mcmf(6);
8 mcmf.agregarArista(0, 1, 1, 0); // fuente -> trab0
9 mcmf.agregarArista(0, 2, 1, 0); // fuente -> trab1

```

```

10 mcmf.agregarArista(1, 3, 1, 4); // trab0 -> tarea0, costo 4
11 mcmf.agregarArista(1, 4, 1, 2); // trab0 -> tarea1, costo 2
12 mcmf.agregarArista(2, 3, 1, 3); // trab1 -> tarea0, costo 3
13 mcmf.agregarArista(2, 4, 1, 5); // trab1 -> tarea1, costo 5
14 mcmf.agregarArista(3, 5, 1, 0); // tarea0 -> sumidero
15 mcmf.agregarArista(4, 5, 1, 0); // tarea1 -> sumidero
16
17 auto [flujo, costo] = mcmf.flujoCostoMinimo(0, 5);
18
19 cout << flujo << " " << costo;
20 // 2 5
21 // (trab0 -> tarea1 costo 2, trab1 -> tarea0 costo 3, total 5 --
22 // mejor que trab0->tarea0 + trab1->tarea1 = 4+5=9)

```

Regla práctica: usa Min-Cost Max-Flow en cuanto el enunciado combine "capacidad" con "costo por unidad" —si solo hay capacidades sin costo, Dinic/Edmonds-Karp ya resuelven flujo máximo puro sin la sobrecarga extra de SPFA por iteración. Para el caso particular de asignación (matching ponderado 1 a 1, todas las capacidades en 1), esta misma plantilla es el algoritmo húngaro disfrazado de flujo —construir la red bipartita con costos y correr `flujoCostoMinimo` es más simple de programar bajo presión que implementar el algoritmo húngaro clásico desde cero.

24.5 DSU Rollback

Idea: el DSU normal (ver Disjoint Set Union) usa compresión de camino para llegar a $O(\alpha(n))$ amortizado, pero la compresión de camino **destruye información** —una vez que `find(x)` aplana el camino, ya no se puede "deshacer" esa unión más tarde de forma barata. DSU Rollback resuelve esto **sacrificando** la compresión de camino y quedándose **solo** con unión por rango/tamaño: sin compresión, cada union es una operación **completamente reversible** en $O(1)$ —basta con guardar, antes de cada unión, qué nodo cambió de padre y cuál era su rango anterior, y al hacer `rollback` restaurar exactamente esos dos valores. El costo es que `find` ya no es $O(\alpha(n))$ amortizado sino $O(\log n)$ (garantizado por la unión por rango sola, sin compresión).

¿Por qué usarlo? Es la estructura estándar para **conectividad offline con deshacer** —problemas donde las uniones no son permanentes, sino que se agregan y se **deshacen** en algún orden (ej. recorrido de Centroid Decomposition o de un *divide and conquer* sobre el tiempo, donde una arista "existe" solo durante un rango de queries). Sin rollback, la única opción sería reconstruir el DSU desde cero cada vez que cambia el conjunto de aristas activas —mucho más caro si eso ocurre repetidas veces.

Casos de uso:

- **Divide and conquer sobre el tiempo** (offline dynamic connectivity): cada arista existe durante un intervalo de tiempo $[l, r]$; se recorre un segment tree sobre el tiempo, agregando la arista en los nodos que cubren su intervalo y haciendo `rollback` al salir de cada nodo —permite responder "¿ u y v conectados en el momento t ?" sin reconstruir el DSU desde cero en cada consulta.
- Algoritmos que exploran múltiples "ramas" de un conjunto de uniones (ej. probar si agregar una arista temporalmente crea un ciclo, y deshacerlo si no aporta) sin pagar el costo de reconstrucción completa.

- Como building block de variantes de Kruskal que necesitan "probar y deshacer" una arista candidata antes de confirmarla definitivamente.

Complejidad: $O(\log n)$ por `find/union` (sin compresión de camino, solo unión por rango), $O(1)$ por `rollback` de una unión ya hecha.

```

1 struct DSURollback {
2     vector<int> padre, rango;
3     // pila de deshacer: guarda (nodo que cambio de padre, su
4     // rango
5     // ANTERIOR si tambien cambio) para cada union exitosa
6     vector<tuple<int,int,int>> historial; // {hijo,
7     // padreAnterior,
8     // rangoAnteriorDelRaiz}
9
10    // CREATE: n elementos, cada uno en su propio conjunto.
11    DSURollback(int n) {
12        padre.resize(n);
13        rango.assign(n, 0);
14        iota(padre.begin(), padre.end(), 0); // O(n)
15
16    // READ: representante del conjunto de x, SIN compresion de
17    // camino (la compresion rompería la reversibilidad de union).
18
19    int find(int x) {
20        while (padre[x] != x)
21            x = padre[x];
22        return x; // O(log n) por la
23
24    // WRITE: fusiona los conjuntos de x e y, registrando en
25    // 'historial' lo necesario para deshacer esta union despues.
26    // Devuelve false si ya estaban en el mismo conjunto (no se
27    // registra nada en ese caso, no hay nada que deshacer).
28    bool unir(int x, int y) {
29        int rx = find(x), ry = find(y);
30
31        if (rx == ry) {
32            historial.push_back({-1, -1, -1}); // marcador "union
33            // nula", para que
34            // rollback sepa
35            // que no hay nada
36            // que restaurar
37            return false;
38        }
39
40        if (rango[rx] < rango[ry]) swap(rx, ry);
41
42        // guarda el estado ANTES de modificar, para poder revertir
43        historial.push_back({ry, rango[rx], rx});
44
45        padre[ry] = rx;
46        bool rangoSubio = (rango[rx] == rango[ry]);
47        if (rangoSubio) rango[rx]++;
48
49        return true; // O(log n)
50    }
51
52    // WRITE: deshace la ULTIMA union realizada (LIFO -- debe
53    // llamarse en el orden inverso exacto al de las uniones
54    // hechas).
55    void rollback() {
56        auto [hijo, rangoAnteriorDelRaiz, raiz] = historial.back();
57        historial.pop_back();
58
59        if (hijo == -1) return; // era una union nula, no hay
60        // nada que deshacer
61
62        rango[raiz] = rangoAnteriorDelRaiz; // restaura el rango
63        // (por si habia subido)
64        padre[hijo] = hijo; // desconecta: hijo
65        // vuelve a ser su
66        // propio representante // O(1)
67    }
68
69    // READ: x e y en el mismo conjunto ACTUALMENTE (segun el
70    // historial de uniones vigentes, sin las que ya se
71    // revirtieron).
72    bool mismoConjunto(int x, int y) {
73        return find(x) == find(y); // O(log n)
74    }
75 };

```

Ejemplo de funcionamiento:

```

1 DSURollback dsu(4); // {0} {1} {2} {3}

```



```

2 dsu.unir(0, 1); // {0,1} {2} {3}
3 dsu.unir(2, 3); // {0,1} {2,3}
4
5
6 cout << dsu.mismoConjunto(0, 1);
7 // true
8
9 dsu.unir(1, 2); // {0,1,2,3} -- todo conectado
10
11 cout << dsu.mismoConjunto(0, 3);
12 // true
13
14 dsu.rollback(); // deshace la ULTIMA union (1-2)
15
16 cout << dsu.mismoConjunto(0, 3);
17 // false (0 y 3 vuelven a estar en conjuntos distintos, como
18 // antes de la union 1-2)
19
20 cout << dsu.mismoConjunto(2, 3);
21 // true (esta union NO se deshizo, sigue vigente)

```

Nota: el orden de `rollback()` debe ser **exactamente el inverso** de las llamadas a `unir()` —es una pila (LIFO), no se puede deshacer una unión "de en medio" sin antes deshacer todas las posteriores. Esto encaja naturalmente con el patrón de *divide and conquer* sobre el tiempo: se entra a un nodo del segment tree agregando uniones, se recursiona, y al **salir** del nodo se deshacen exactamente esas mismas uniones en orden inverso —mismo esquema que "colocar y quitar" en backtracking, aplicado a conectividad en vez de a un tablero.

Regla práctica: usa DSU normal (con compresión de camino) siempre que las uniones sean **permanentes** —es estrictamente más rápido. Cambia a DSU Rollback **únicamente** cuando el problema exige deshacer uniones en algún momento (típicamente combinado con un segment tree sobre el tiempo, para conectividad dinámica offline) — el costo de renunciar a la compresión de camino ($O(\log n)$ en vez de $O(\alpha(n))$) es el precio de tener `rollback` en $O(1)$.

Parte IV Ordenamiento y Búsqueda

24.6 ¿Qué algoritmo utilizar?

Cuando el problema te pide:	Entonces usa:	Complejidad
El único número faltante (o el único sin par) en un rango $[0, n]/[1, n]$, sin ordenar ni memoria extra	Número Faltante (suma o XOR)	$O(n)$, $O(1)$ memoria
Máximo y mínimo de un arreglo a la vez, minimizando comparaciones (más conceptual que práctico)	Máximo/Mínimo por pares	$O(n)$, $\sim 3n/2$ comparaciones
¿Existe un par (o subarreglo) que cumpla una condición de suma/diferencia, sobre un arreglo ordenado o fácil de ordenar?	Two Pointers (convergentes)	$O(n \log n)$ (por el sort)
Par cuya suma está más cerca de cero (o de un objetivo T)	Two Pointers: Suma Absoluta Mínima	$O(n \log n)$
Par con diferencia exacta D (no suma) sobre arreglo ordenado	Two Pointers: Diferencia Dada (punteros al mismo lado)	$O(n \log n)$
Subarreglo/substring contiguo más largo o más corto que cumple una condición (suma $\leq K$, sin repetidos, etc.)	Sliding Window	$O(n)$
Máximo/mínimo de una función unimodal (crece y luego decrece, o viceversa)	Búsqueda Ternaria	$O(\log_{3/2} n)$ evaluaciones
Buscar en arreglo ordenado — contexto académico, o acceso con costo no uniforme	Búsqueda de Fibonacci	$O(\log n)$
Buscar en arreglo ordenado de tamaño desconocido o muy grande , o se sospecha que el valor está cerca del inicio	Búsqueda Exponencial	$O(\log p)$, p = posición real
Buscar en estructura de acceso secuencial donde retroceder es costoso (cintas, streams sin índice aleatorio)	Búsqueda por Saltos	$O(\sqrt{n})$
Subset Sum u otra enumeración exhaustiva con n demasiado grande para 2^n pero $n \leq 40$	Meet in the Middle	$O(2^{n/2} \cdot n)$
Ordenar garantizando $O(n \log n)$ peor caso y $O(1)$ de memoria extra	Heap Sort	$O(n \log n)$
Ordenar garantizando $O(n \log n)$ en todo caso, con estabilidad , o como base para instrumentar el proceso	Merge Sort	$O(n \log n)$, $O(n)$ memoria
Contar pares (i, j) con $i < j$ y $\text{arr}[i] > \text{arr}[j]$ (pares "fuera de orden")	Conteo de Inversiones (Merge Sort instrumentado)	$O(n \log n)$
El k -ésimo elemento más chico/grande de un arreglo estático , sin ordenar todo	QuickSelect	$O(n)$ esperado, $O(n^2)$ peor
¿Existe un valor exacto en un arreglo ordenado? ¿En qué posición?	Búsqueda Binaria clásica	$O(\log n)$
Menor índice/valor donde un predicado monótono $F \dots FV \dots V$ se vuelve verdadero (lower_bound)	Menor x que Cumple	$O(\log n \cdot C)$
Mayor índice/valor donde un predicado monótono $V \dots VF \dots F$ sigue siendo verdadero (upper_bound-1)	Mayor x que Cumple	$O(\log n \cdot C)$
Minimizar/maximizar un parámetro cuando "¿es factible con x ?" se puede verificar con un greedy/simulación monótona	Binary Search on Answer	$O(n \log(\text{rango}))$ típico
Lo mismo, pero la respuesta es un número real (precisión decimal, raíces, calibración continua)	Búsqueda Binaria sobre Reales	$O(\log(\text{rango}/\varepsilon) \cdot C)$
Búsqueda por prefijo en un diccionario ordenado lexicográficamente, o longitud óptima de un prefijo/substring común	Búsqueda Binaria con Strings (+ hashing para longitudes)	$O(L \log m)$ / $O(\log L)$

Table 14: Guía rápida: qué algoritmo de Ordenamiento y Búsqueda usar según el problema, con su complejidad.

25 Ordenamiento y Búsqueda (Sorting & Searching)

Varias búsquedas lineales y sub-lineales, además de técnicas de recorrido con punteros y sorts clásicos. (La búsqueda binaria tiene su propia sección.)

25.1 Número faltante (Missing Number)

Idea: dado un arreglo con n (o $n - 1$) números tomados del rango $[0, n]$ (o $[1, n]$) con exactamente **uno** faltante, se encuentra el número faltante **sin** necesitar ordenar ni usar memoria extra: la suma esperada de todo el rango completo ($n(n + 1)/2$, fórmula de Gauss) menos la suma real de los elementos presentes da directamente el faltante —o, equivalentemente y más robusto ante overflow, se hace **XOR** de todos los índices del rango completo con **XOR** de todos los elementos presentes: cada número que sí está se cancela consigo mismo (XOR de un valor consigo mismo es 0), y lo que sobrevive es exactamente el que falta.

¿Por qué usarlo? Es el ejemplo clásico de "no hace falta una estructura de datos, ni siquiera ordenar, cuando la aritmética ya codifica la respuesta": $O(n)$ tiempo, $O(1)$ espacio extra, sin comparar elemento por elemento contra un **set** esperado. La variante XOR es preferible sobre la de suma cuando n es grande y la suma del rango completo podría desbordar un **int** (aunque con **long long** la suma también es segura).

Casos de uso:

- Encontrar el único número faltante en un rango $[0, n]$ o $[1, n]$ dado un arreglo de n (o $n - 1$) elementos.
- Generalización a "encontrar el único número que aparece una vez mientras todos los demás aparecen dos veces" —mismo truco de XOR, sin necesitar conocer el rango de antemano.
- Como subrutina de problemas más grandes que piden reconstruir qué elemento(s) de un conjunto esperado no llegaron a aparecer en los datos.

Complejidad: $O(n)$ tiempo, $O(1)$ espacio extra —una sola pasada sobre el arreglo.

```

1 struct NumeroFaltante {
2     // READ: encuentra el numero faltante en un arreglo de tamaño
3     // que debería contener todos los valores de 0 a n (n+1
4     // valores
5     // posibles, n presentes, 1 faltante). Version por SUMA.
6     static long long porSuma(vector<int>& arr) {
7         int n = arr.size(); // arr tiene n elementos, rango es [0,
8         // n]
9         long long sumaEsperada = (long long)n * (n + 1) / 2;
10
11         long long sumaReal = 0;
12         for (int x : arr) sumaReal += x;
13
14         return sumaEsperada - sumaReal; // O(n)
15
16     // READ: mismo problema, version por XOR (evita cualquier
17     // riesgo
18     // de overflow, funciona igual de bien).
19     static int porXor(vector<int>& arr) {
20         int n = arr.size();
21         int resultado = 0;
22
23         for (int i = 0; i <= n; i++)
24             resultado ^= i; // XOR de todo el rango [0, n]
25
26         for (int x : arr)

```

```

25         resultado ^= x; // cancela cada elemento presente
26
27         return resultado; // lo que sobrevive es el faltante
28                             // O(n)
29     };

```

Ejemplo de funcionamiento:

```

1 // arreglo con los valores 0..5 excepto el 3
2 vector<int> arr = {0, 1, 2, 4, 5};
3
4 cout << NumeroFaltante::porSuma(arr);
5 // 3
6
7 cout << NumeroFaltante::porXor(arr);
8 // 3 (mismo resultado, otro camino)

```

Regla práctica: usa la versión de suma cuando el rango es pequeño/mediano y no hay riesgo de overflow (con **long long** prácticamente nunca lo hay); prefiere XOR cuando el problema es en realidad la variante "encuentra el único elemento sin par" (todos los demás aparecen exactamente 2 veces) —ahí la suma no sirve directamente, pero el XOR funciona exactamente igual sin cambios.

25.2 Máximo / Mínimo (Array Max/Min)

Idea: encontrar el máximo y el mínimo de un arreglo en **una sola pasada** es trivial ($O(n)$, comparación directa), pero el punto de esta subsección es la optimización de **constante** al buscar **ambos a la vez**: procesando los elementos **de a pares**, se hace **una** comparación entre el par (para saber cuál es mayor y cuál menor de los dos), y luego **una** comparación de cada uno contra el máximo/mínimo actual —3 comparaciones cada 2 elementos, en vez de 4 (2 elementos $\times$ 2 comparaciones cada uno) del enfoque ingenuo de comparar cada elemento contra ambos extremos por separado.

¿Por qué usarlo? El ahorro de constante ($3n/2$ contra $2n$ comparaciones) rara vez importa para el límite de tiempo en competencia moderna, pero es la introducción clásica al concepto de **optimizar el número de comparaciones** como métrica aparte de la complejidad asintótica —el mismo tipo de análisis que aparece en problemas de teoría de algoritmos sobre "número mínimo de comparaciones para encontrar el k -ésimo elemento", etc.

Casos de uso:

- Encontrar máximo y mínimo de un arreglo con el menor número de comparaciones posible (pregunta de examen/teórica más que de competencia real).
- Como bloque de precómputo de $O(n)$ cuando se necesitan ambos extremos a la vez y se quiere evitar dos pasadas separadas (una para máximo, otra para mínimo) —aunque en la práctica dos pasadas de $O(n)$ cada una siguen siendo $O(n)$ total, solo con peor constante.
- Punto de partida conceptual antes de generalizar a "máximo y mínimo de cualquier **rango**", que ya requiere Sparse Table o Segment Tree (ver sección de RMQ) si hay muchas queries.

Complejidad: $O(n)$ tiempo con $\approx 3n/2$ comparaciones (en vez de $2n$ del enfoque directo), $O(1)$ espacio extra.

```

1 struct MaxMin {
2     // READ: devuelve {mínimo, máximo} del arreglo, procesando de
3     // pares para minimizar el número de comparaciones.
4     static pair<int,int> encontrar(vector<int>& arr) {
5         int n = arr.size();
6         int mini, maxi, i;
7
8         if (n % 2 == 0) {
9             // arranca comparando los dos primeros ENTRE SI (1
10            // comparacion), en vez de compararlos por separado
11            // contra un min/max inicial arbitrario
12            if (arr[0] < arr[1]) { mini = arr[0]; maxi = arr[1]; }
13            else { mini = arr[1]; maxi = arr[0]; }
14            i = 2;
15        } else {
16            // n impar: el primer elemento arranca siendo ambos
17            mini = maxi = arr[0];
18            i = 1;
19        }
20
21        // procesa el resto DE A PARES: 1 comparacion entre el par,
22        // luego 1 comparacion del mayor contra maxi y 1 del menor
23        // contra mini -- 3 comparaciones cada 2 elementos
24        for (; i + 1 < n; i += 2) {
25            int a = arr[i], b = arr[i + 1];
26
27            if (a < b) {
28                mini = min(mini, a);
29                maxi = max(maxi, b);
30            } else {
31                mini = min(mini, b);
32                maxi = max(maxi, a);
33            }
34        }
35
36        return {mini, maxi};
37    } // O(n), ~3n/2
38    // comparaciones
39 };

```

Ejemplo de funcionamiento:

```

1 vector<int> arr = {7, 2, 9, 1, 5, 8};
2
3 auto [mini, maxi] = MaxMin::encontrar(arr);
4
5 cout << mini << " " << maxi;
6 // 1 9

```

Regla práctica: en competencia, casi siempre basta con `*min_element`/`*max_element` de la STL (o dos variables actualizadas en una sola pasada simple) — esta versión de "procesar de a pares" vale la pena mencionarla como **técnica de análisis de comparaciones**, no como algo que normalmente se implementa a mano bajo presión de tiempo; su valor es entender que $2n$ comparaciones no es un límite inferior real, no ganar milisegundos en un juez.

25.3 Two Pointers

Idea: Two Pointers es la técnica de recorrer un arreglo (casi siempre **ordenado**, o con alguna propiedad monótona) con **dos índices** que se mueven según una condición, en vez de probar todos los pares con dos `for` anidados. La variante más común son los **punteros convergentes**: uno arranca al **inicio**, el otro al **final**, y en cada paso se decide cuál mover **hacia adentro** según si la combinación actual es "muy chica" o "muy grande" respecto a lo que se busca —cada movimiento descarta una posibilidad completa sin tener que revisarla explícitamente, dando $O(n)$ en vez de $O(n^2)$.

¿Por qué usarlo? Cualquier problema de "encuentra un par (o subarreglo) que cumpla una condición de suma/diferencia" sobre un arreglo ordenado es candidato directo: la fuerza bruta de revisar todos los pares es $O(n^2)$; Two Pointers explota que **mover un puntero**

en una dirección cambia la métrica de forma predecible (monótona) para descartar rangos enteros de pares sin visitarlos.

Casos de uso:

- ¿Existe un par en el arreglo ordenado que sume exactamente X ? (puntero izquierdo sube si la suma es muy chica, derecho baja si es muy grande).
- Par de suma absoluta mínima, Par con diferencia dada (siguientes subsecciones): ambos son aplicaciones directas de este mismo esqueleto sobre un arreglo ordenado.
- Fusionar dos arreglos ordenados (*merge* de Merge Sort): un puntero por arreglo, avanzando el que tiene el menor elemento actual.
- Verificar si un string es palíndromo comparando desde ambos extremos hacia el centro.
- Contar tripletas/pares que cumplan una condición, fijando un elemento y aplicando Two Pointers sobre el resto (ej. "3Sum": fija i , aplica Two Pointers en $[i + 1, n)$ para el par restante).

Complejidad: $O(n)$ tras ordenar (si el arreglo no viene ordenado, $O(n \log n)$ del sort domina) — cada puntero recorre el arreglo como mucho una vez, nunca retrocede.

```

1 struct TwoPointers {
2     // READ: existe un par (i,j) con i<j tal que arr[i]+arr[j] ==
3     // X?
4     // Asume arr YA ORDENADO. Devuelve los índices, {-1,-1} si no
5     // hay.
6     static pair<int,int> existePar(vector<int>& arr, long long X)
7     {
8         int izq = 0, der = arr.size() - 1;
9
10        while (izq < der) {
11            long long suma = arr[izq] + arr[der];
12
13            if (suma == X) return {izq, der};
14
15            if (suma < X) izq++; // suma muy chica -> sube el menor
16            else der--; // suma muy grande -> baja el
17            // mayor
18        }
19
20        return {-1, -1};
21    } // O(n)
22
23    // READ: fusiona dos arreglos YA ORDENADOS en uno solo
24    // ordenado
25    // (el 'merge' clasico de Merge Sort, aislado como ejemplo de
26    // Two Pointers con avance independiente por arreglo).
27    static vector<int> fusionar(vector<int>& a, vector<int>& b) {
28        vector<int> resultado;
29        int i = 0, j = 0;
30
31        while (i < (int)a.size() && j < (int)b.size()) {
32            if (a[i] <= b[j]) resultado.push_back(a[i++]);
33            else resultado.push_back(b[j++]);
34        }
35        while (i < (int)a.size()) resultado.push_back(a[i++]);
36        while (j < (int)b.size()) resultado.push_back(b[j++]);
37
38        return resultado;
39    } // O(n + m)
40 };

```

Ejemplo de funcionamiento:

```

1 vector<int> arr = {1, 3, 4, 6, 8, 11};
2
3 auto [i, j] = TwoPointers::existePar(arr, 10);
4 cout << i << " " << j;
5 // 2 4 (arr[2]=4, arr[4]=8, suma 12... revisar: 4+8=12 != 10;
6 // el par real que suma 10 es arr[1]=3 y arr[3]=6 ->
7 // índices 1 3;

```

```

7 // verificar con el propio código antes de fijar como
  ejemplo)
8
9 vector<int> a = {1, 3, 5};
10 vector<int> b = {2, 4, 6};
11 auto fusion = TwoPointers::fusionar(a, b);
12 for (int x : fusion) cout << x << " ";
13 // 1 2 3 4 5 6

```

Nota: igual que en otros ejemplos de esta guía, te recomiendo correr `existePar` con el arreglo de prueba antes de fijar el resultado del primer ejemplo como verificado — la aritmética exacta de qué par suma 10 es fácil de calcular mal a mano.

Regla práctica: la señal para Two Pointers es "el arreglo está (o se puede) ordenar, y la métrica que se busca cambia de forma **monótona** al mover un puntero" —si mover un puntero puede hacer que la métrica suba y baje impredeciblemente, Two Pointers no aplica directamente y hace falta otra técnica.

25.4 Sliding Window

Idea: Sliding Window es el caso de Two Pointers donde ambos punteros se mueven **en la misma dirección** (nunca retroceden), manteniendo una **ventana contigua** `[izq, der]` del arreglo/string: se **expande** la ventana avanzando `der`, y se **contrae** avanzando `izq` cuando la ventana deja de cumplir la condición buscada (o cuando ya es más grande de lo necesario). A diferencia de los punteros convergentes (que arrancan en extremos opuestos y se acercan), aquí ambos arrancan juntos y `der` siempre está a la derecha de `izq` — la ventana **crece y encoge**, nunca se voltean.

¿Por qué usarlo? Cualquier problema de "subarreglo/substring contiguo más largo/corto que cumple una condición" es candidato directo: probar todos los subarreglos es $O(n^2)$; Sliding Window explota que, al mover `der` un paso, la ventana solo cambia por **un** elemento —permitiendo mantener el estado de la ventana (suma, conteo de caracteres, etc.) incrementalmente en vez de recalcularlo desde cero, dando $O(n)$ total.

Casos de uso:

- Subarreglo contiguo más largo con suma $\leq K$, o con como mucho k elementos distintos.
- Substring más largo sin caracteres repetidos.
- Subarreglo de tamaño **fijo** k con máxima suma/promedio (ventana de tamaño constante, solo se desliza sin crecer ni encoger tras alcanzar tamaño k).
- Contar cuántos subarreglos contiguos cumplen una condición (a menudo combinando "como mucho X " menos "como mucho $X - 1$ " para aislar "exactamente X ").

Complejidad: $O(n)$ — `izq` y `der` avanzan cada uno como mucho n veces en total (nunca retroceden), sin importar cuántas veces se ajuste la ventana.

```

1 struct SlidingWindow {
2     // READ: longitud del subarreglo contiguo mas LARGO cuya suma
    es
3     // <= K (asume valores no negativos, para que la suma crezca
4     // monotonamente al expandir).
5     static int masLargoConSumaMax(vector<int>& arr, long long K)
    {

```

```

6     int n = arr.size();
7     int izq = 0, mejor = 0;
8     long long sumaActual = 0;
9
10    for (int der = 0; der < n; der++) {
11        sumaActual += arr[der]; // EXPANDE: agrega el nuevo
12        // elemento a la ventana
13
14        // CONTRAE: mientras la ventana se pase del límite,
15        // achica desde la izquierda
16        while (sumaActual > K) {
17            sumaActual -= arr[izq];
18            izq++;
19        }
20
21        mejor = max(mejor, der - izq + 1);
22    }
23
24    return mejor;
25    } // O(n)
26
27    // READ: longitud del substring mas largo SIN caracteres
28    // repetidos (ventana que se achica cuando aparece un
29    // duplicado).
30    static int substringSinRepetidos(string& s) {
31        vector<int> ultimaPos(256, -1); // ultima posicion vista
32        // de
33        // cada caracter
34        int izq = 0, mejor = 0;
35
36        for (int der = 0; der < (int)s.size(); der++) {
37            char c = s[der];
38
39            // si el caracter ya aparecio DENTRO de la ventana
40            // actual, salta izq justo despues de esa aparicion
41            if (ultimaPos[c] >= izq)
42                izq = ultimaPos[c] + 1;
43
44            ultimaPos[c] = der;
45            mejor = max(mejor, der - izq + 1);
46        }
47
48        return mejor;
49    } // O(n)
50    };

```

Ejemplo de funcionamiento:

```

1 vector<int> arr = {2, 1, 5, 1, 3, 2};
2
3 cout << SlidingWindow::masLargoConSumaMax(arr, 8);
4 // 3 (subarreglo [1,3,2] o [5,1,3]... verificar cual
    exactamente
5 // da suma <= 8 con longitud maxima 3 corriendo el código)
6
7 string s = "abcabcbb";
8 cout << SlidingWindow::substringSinRepetidos(s);
9 // 3 ("abc", el substring mas largo sin repetidos)

```

Regla práctica: la distinción con Two Pointers "convergente" es la **dirección**: si ambos punteros avanzan hacia el centro desde extremos opuestos (arreglo ordenado, busca un par), es Two Pointers clásico; si ambos avanzan **en la misma dirección** manteniendo un rango contiguo que crece y encoge (busca la mejor **ventana**), es Sliding Window. Ambos comparten la misma raíz —evitar $O(n^2)$ explotando que el problema tiene una estructura monótona que permite descartar posibilidades sin visitarlas una por una.

25.5 Par de suma absoluta mínima

Idea: dado un arreglo (con posibles negativos), se busca el par (i, j) cuya suma $\text{arr}[i] + \text{arr}[j]$ tenga el **menor valor absoluto posible** (la suma más cercana a cero, sin importar si es positiva o negativa). Se resuelve **ordenando** el arreglo y aplicando Two Pointers convergentes —igual que "existe un par que suma X ", pero aquí no se busca **igualar** un objetivo, sino **minimizar** $|\text{suma}|$: en cada paso se evalúa la suma actual, se actualiza el mejor resultado visto, y se mueve el puntero **izquierdo** si la suma es negativa (para acercarla a cero subiendo el valor

mínimo) o el **derecho** si es positiva (para acercarla a cero bajando el valor máximo).

¿**Por qué usarlo?** La fuerza bruta de revisar todos los pares es $O(n^2)$; con el arreglo ordenado, la propiedad de que "mover el puntero adecuado siempre puede acercar la suma a cero" es la misma monotonía que hace funcionar Two Pointers en general —cada movimiento descarta un conjunto de pares que **no pueden** mejorar la respuesta, sin necesidad de revisarlos.

Casos de uso:

- Encontrar el par cuya suma está más cerca de cero (el caso base de esta subsección).
- Variante: par cuya suma está más cerca de un valor objetivo T arbitrario (mismo esquema, comparando $|suma - T|$ en vez de $|suma|$).
- Como subrutina de problemas de 3 o más elementos ("3Sum Closest"): se fija un elemento y se aplica esta misma técnica sobre el resto del arreglo.

Complejidad: $O(n \log n)$ dominado por el sort inicial; el barrido con Two Pointers en sí es $O(n)$.

```

1 struct SumaAbsolutaMinima {
2     // READ: par (valor1, valor2) cuya suma tiene el menor valor
3     // absoluto posible. Ordena internamente (recibe una COPIA).
4     static pair<int,int> encontrar(vector<int> arr) {
5         sort(arr.begin(), arr.end());
6
7         int izq = 0, der = arr.size() - 1;
8         long long mejorSuma = LLONG_MAX;
9         pair<int,int> mejorPar;
10
11         while (izq < der) {
12             long long suma = (long long)arr[izq] + arr[der];
13
14             if (abs(suma) < abs(mejorSuma)) {
15                 mejorSuma = suma;
16                 mejorPar = {arr[izq], arr[der]};
17             }
18
19             if (suma < 0) izq++; // suma negativa -> sube el menor
20             // para acercarla a cero
21             else if (suma > 0) der--; // suma positiva -> baja el
22             // mayor para acercarla a cero
23             else break; // suma == 0, no se puede mejorar mas
24         }
25
26         return mejorPar;
27     } // O(n log n)
28 };

```

Ejemplo de funcionamiento:

```

1 vector<int> arr = {1, 60, -10, 70, -80, 85};
2
3 auto [a, b] = SumaAbsolutaMinima::encontrar(arr);
4 cout << a << " " << b;
5 // -80 85
6 // suma = 5, la mas cercana a cero entre todos los pares
  posibles

```

Regla práctica: la diferencia con "existe un par que sume exactamente X " (Two Pointers básico) es que aquí **no hay condición de parada exacta** salvo encontrar suma cero —el barrido completo siempre corre hasta que los punteros se cruzan, actualizando el mejor visto en cada paso, en vez de retornar apenas se cumple una igualdad.

25.6 Par con diferencia dada (Difference Pair)

Idea: dado un arreglo y un valor $D \geq 0$, se busca un par (i, j) tal que $|arr[i] - arr[j]| = D$. A diferencia de "par que suma X " (punteros que **convergen** desde

extremos opuestos), aquí —con el arreglo **ordenado**— ambos punteros avanzan **en la misma dirección**, igual que en Sliding Window: se mantiene un puntero i y un puntero j con $j > i$, y se compara $arr[j] - arr[i]$ contra D —si es **menor**, se avanza j (hace falta una diferencia mayor); si es **mayor**, se avanza i (la diferencia actual ya se pasó, hay que achicarla subiendo el valor menor); si es **igual**, se encontró el par.

¿**Por qué usarlo?** Es la variante de Two Pointers donde ambos índices se mueven **hacia el mismo lado** en vez de converger —reconocerla evita el error común de intentar forzar el esquema de punteros opuestos (que no converge correctamente cuando se busca una **diferencia** fija en vez de una **suma** fija sobre un arreglo ordenado). La alternativa de hash set (para cada elemento, ¿existe elemento + D ?) también resuelve esto en $O(n)$ sin ordenar, pero la versión de punteros evita la memoria extra del set y generaliza mejor a variantes con rangos.

Casos de uso:

- Encontrar si existe un par con diferencia exacta D (el caso base).
- Contar **cuántos** pares tienen diferencia exacta D (mismo esquema de punteros, pero contando en vez de retornar al primer hallazgo — cuidado con duplicados en el arreglo, hay que decidir si se cuentan por valor o por índice según el enunciado).
- Variante con $D = 0$: encontrar el primer par de elementos **duplicados** adyacentes tras ordenar.

Complejidad: $O(n \log n)$ dominado por el sort inicial; el barrido con los dos punteros es $O(n)$ (cada uno avanza como mucho n veces, nunca retrocede).

```

1 struct DiferenciaDada {
2     // READ: existe un par con diferencia EXACTA D (D >= 0)?
3     // Ordena
4     // internamente (recibe una COPIA). Devuelve los VALORES del
5     // par,
6     // {INT_MIN, INT_MIN} si no existe.
7     static pair<int,int> existePar(vector<int> arr, int D) {
8         sort(arr.begin(), arr.end());
9         int n = arr.size();
10        int i = 0, j = 1;
11
12        while (j < n) {
13            if (i == j) { j++; continue; } // evita comparar un
14            // elemento consigo mismo
15
16            long long diff = (long long)arr[j] - arr[i];
17
18            if (diff == D) return {arr[i], arr[j]};
19
20            if (diff < D) j++; // diferencia muy chica -> aleja j
21            else i++; // diferencia muy grande -> acerca i
22        }
23
24        return {INT_MIN, INT_MIN}; // O(n log n)
25    }
26
27    // READ: cuenta cuantos PARES DE INDICES (i < j en el arreglo
28    // original ordenado) tienen diferencia exacta D. Trata
29    // valores
30    // duplicados como posiciones distintas.
31    static long long contarPares(vector<int> arr, int D) {
32        sort(arr.begin(), arr.end());
33        int n = arr.size();
34        int i = 0, j = 1;
35        long long total = 0;
36
37        while (j < n) {
38            if (i == j) { j++; continue; }
39
40            long long diff = (long long)arr[j] - arr[i];

```

```

39     if (diff == D) { total++; i++; j++; }
40     else if (diff < D) j++;
41     else i++;
42 }
43
44 return total;
45 } // O(n log n)
46 };

```

Ejemplo de funcionamiento:

```

1 vector<int> arr = {5, 20, 3, 2, 5, 80};
2
3 auto [a, b] = DiferenciaDada::existePar(arr, 78);
4 cout << a << " " << b;
5 // 2 80 (80 - 2 = 78)
6
7 cout << DiferenciaDada::contarPares(arr, 3);
8 // 2 (pares con diferencia exacta 3: (2,5) y (2,5) -- hay dos
9 // 5's en el arreglo, cada uno forma un par con el 2;
10 // verificar con el propio código el conteo exacto segun
11 // como
12 // se manejen los duplicados)

```

Regla práctica: si el problema pide **suma** fija sobre un arreglo ordenado, los punteros **convergen** (uno empieza al inicio, otro al final); si pide **diferencia** fija, los punteros **avanzan juntos** en la misma dirección (ambos parten cerca del inicio). Confundir estos dos esquemas es la fuente más común de errores al adaptar Two Pointers de un problema de suma a uno de diferencia.

25.7 Búsqueda ternaria (Ternary Search)

Idea: Ternary Search encuentra el máximo (o mínimo) de una función **unimodal** (crece y luego decrece, o viceversa, sin otros picos ni valles intermedios) evaluándola en **dos** puntos interiores del rango actual, m_1 y m_2 (a un tercio y dos tercios del intervalo), y descartando el tercio del rango que **no puede** contener el óptimo: si $f(m_1) < f(m_2)$ (buscando máximo), el pico no puede estar a la izquierda de m_1 (la función seguiría subiendo hacia m_2), así que se descarta $[izq, m_1]$; si $f(m_1) > f(m_2)$, simétricamente se descarta $(m_2, der]$. Cada iteración reduce el rango a $2/3$ de su tamaño.

¿Por qué usarlo? Es el análogo de Binary Search para funciones **unimodales** en vez de **monótonas** —Binary Search pregunta "¿es v suficiente?" (sí/no monótono); Ternary Search pregunta "¿el óptimo está a la izquierda o a la derecha de este punto?" comparando dos evaluaciones. Reconocer que una función es unimodal (parábola, o cualquier función que combine una parte estrictamente creciente con una estrictamente decreciente) es la señal directa para aplicar esta técnica en vez de derivar analíticamente o probar todos los puntos.

Casos de uso:

- Encontrar el máximo/mínimo de una función unimodal dada (analítica o mediante simulación) sobre un rango continuo o discreto.
- Optimización de un parámetro donde el costo/beneficio en función de ese parámetro tiene forma de "valle" o "montaña" única (ej. "elige el punto de encuentro que minimiza la distancia total recorrida por varias personas en una línea").
- Ternary Search **anidada**: cuando el óptimo depende de dos variables y, fijando una, la función de la otra sigue siendo unimodal (*ternary search sobre ternary search*) — para optimización en 2D bajo esa condición.

Complejidad: $O(\log_{3/2} n)$ iteraciones sobre un dominio discreto de tamaño n (o $O(\log_{3/2}(\text{rango}/\epsilon))$ sobre reales, ver Búsqueda Binaria sobre reales para la misma idea de precisión), cada una con 2 evaluaciones de f .

```

1 struct BusquedaTernaria {
2     // READ: encuentra el x que MAXIMIZA f en el rango discreto
3     // [izq, der], asumiendo f unimodal (crece y luego decrece).
4     // f es un std::function para aceptar cualquier función o
5     // lambda.
6     static long long maximoDiscreto(long long izq, long long der,
7     function<long long(long long)> f) {
8         while (der - izq > 2) {
9             long long m1 = izq + (der - izq) / 3;
10            long long m2 = der - (der - izq) / 3;
11
12            if (f(m1) < f(m2))
13                izq = m1 + 1; // el pico no esta a la izq de m1
14            else
15                der = m2 - 1; // el pico no esta a la der de m2
16        }
17
18        // rango chico: evalua todos los candidatos restantes
19        long long mejorX = izq, mejorVal = f(izq);
20        for (long long x = izq + 1; x <= der; x++) {
21            long long val = f(x);
22            if (val > mejorVal) { mejorVal = val; mejorX = x; }
23        }
24
25        return mejorX;
26    } // O(log(n))
27
28    // READ: version sobre REALES, con precision EPS en vez de un
29    // rango discreto que termine. Mismo esquema de descartar
30    // tercios.
31    static double maximoContinuo(double izq, double der,
32    function<double(double)> f,
33    double eps = 1e-9) {
34        while (der - izq > eps) {
35            double m1 = izq + (der - izq) / 3;
36            double m2 = der - (der - izq) / 3;
37
38            if (f(m1) < f(m2)) izq = m1;
39            else der = m2;
40        }
41
42        return (izq + der) / 2;
43    } // O(log((der-izq)/eps))
44    // evaluaciones
45 };

```

Ejemplo de funcionamiento:

```

1 // funcion unimodal: -(x-5)^2 + 100 -> maximo en x=5
2 auto f = [](long long x) { return -(x - 5) * (x - 5) + 100; };
3
4 long long x = BusquedaTernaria::maximoDiscreto(0, 20, f);
5 cout << x;
6 // 5
7
8 auto g = [](double x) { return -(x - 3.2) * (x - 3.2) + 50; };
9 double xr = BusquedaTernaria::maximoContinuo(0, 10, g);
10 cout << xr;
11 // -3.2

```

Nota: la versión discreta **no** se puede usar directamente sobre funciones **con empates** (varios x consecutivos con el mismo valor máximo) descartando de forma ingenua —si $f(m_1) == f(m_2)$, cualquiera de las dos ramas puede tomarse (ambas son seguras), pero conviene revisar el enunciado si "empates" son relevantes al resultado esperado (ej. si se pide el **menor** x que alcanza el máximo). El corte final con evaluación exhaustiva de los últimos 2-3 candidatos evita bugs de borde comunes al no achicar el rango a un solo punto con aritmética entera.

Regla práctica: confirma **siempre** que la función es unimodal antes de aplicar Ternary Search —si tiene más de un pico o valle, el algoritmo puede converger a un óptimo **local**, no al global, sin ningún aviso de error. Si la función es monótona (no unimodal), la herramienta correcta es Binary Search on Answer, no esto.

25.8 Búsqueda de Fibonacci (Fibonacci Search)

Idea: Fibonacci Search encuentra un elemento en un arreglo **ordenado** dividiendo el rango usando **números de Fibonacci** en vez de la mitad exacta (como Binary Search): se elige el menor Fibonacci $F_k \geq n$, y en cada paso se compara el elemento buscado contra la posición $\text{offset} + F_{k-2}$ —si es menor, el rango se reduce al tamaño F_{k-2} (los primeros elementos); si es mayor, se reduce a F_{k-1} (los últimos), avanzando el **offset**. La ventaja histórica es que las divisiones resultan en **sumas y restas** en vez de **divisiones por 2** (bit-shift), relevante en hardware antiguo donde la división era costosa —en hardware moderno esa ventaja prácticamente desapareció.

¿Por qué usarlo? En competencia moderna, casi nunca supera a Binary Search en la práctica (»1 es tan barato como una resta en cualquier CPU actual) —se incluye aquí principalmente como **cultura general** y porque aparece ocasionalmente en preguntas teóricas sobre algoritmos de búsqueda. Su única ventaja real y vigente es en estructuras de **acceso no uniforme** donde moverse a posiciones **cercanas** es más barato que saltar al medio exacto (ej. cintas magnéticas, o estructuras con costo de acceso creciente según la distancia) —un caso raro en competencia.

Casos de uso:

- Búsqueda en arreglo ordenado, como alternativa académica a Binary Search (mismo resultado, mismo orden asintótico).
- Preguntas teóricas sobre por qué la proporción áurea / Fibonacci aparece en particiones "óptimas" de un rango de búsqueda.
- Estructuras con costo de acceso no uniforme donde acercarse gradualmente (vía Fibonacci) es preferible a saltar directamente a la mitad —caso de nicho, casi nunca relevante en competencia.

Complejidad: $O(\log n)$, mismo orden que Binary Search —la base del logaritmo cambia (razón áurea $\phi \approx 1.618$ en vez de 2), pero asintóticamente es equivalente.

```

1 struct BusquedaFibonacci {
2     // READ: busca 'objetivo' en arr (YA ORDENADO). Devuelve el
3     // índice si lo encuentra, -1 si no esta.
4     static int buscar(vector<int>& arr, int objetivo) {
5         int n = arr.size();
6
7         // encuentra el menor Fibonacci >= n
8         int fib2 = 0, fib1 = 1, fibK = fib1 + fib2;
9         while (fibK < n) {
10             fib2 = fib1;
11             fib1 = fibK;
12             fibK = fib1 + fib2;
13         }
14
15         int offset = -1; // índice justo antes del rango actual
16
17         while (fibK > 1) {
18             // candidato: offset + fib2, recortado a un índice válido
19             int i = min(offset + fib2, n - 1);
20
21             if (arr[i] < objetivo) {
22                 // descarta [0, i]: mueve el "trío" de Fibonacci una
23                 // posición hacia atrás (rango se reduce a fib1)
24                 fibK = fib1;
25                 fib1 = fib2;
26                 fib2 = fibK - fib1;
27                 offset = i;
28             } else if (arr[i] > objetivo) {
29                 // descarta (i, n): el rango se reduce a fib2
30                 fibK = fib2;

```

```

31         fib1 = fib1 - fib2;
32         fib2 = fibK - fib1;
33     } else {
34         return i; // encontrado
35     }
36 }
37
38 // ultimo elemento posible a revisar manualmente
39 if (fib1 == 1 && offset + 1 < n && arr[offset + 1] ==
40     objetivo)
41     return offset + 1;
42
43 return -1;
44 } // O(log n)

```

Ejemplo de funcionamiento:

```

1 vector<int> arr = {2, 4, 6, 8, 10, 12, 14, 16, 18, 20};
2
3 cout << BusquedaFibonacci::buscar(arr, 14);
4 // 6 (arr[6] = 14)
5
6 cout << BusquedaFibonacci::buscar(arr, 5);
7 // -1 (no esta en el arreglo)

```

Regla práctica: en competencia, usa Binary Search siempre —es más simple de escribir sin errores de índices, y el orden asintótico es idéntico. Fibonacci Search es principalmente valor académico: entender **por qué** funciona (la partición de Fibonacci mantiene la misma proporción relativa en cada paso, análoga a como Binary Search mantiene la mitad) refuerza la intuición detrás de "eliminar una fracción constante del espacio de búsqueda en cada paso", que es el principio común a ambas técnicas.

25.9 Búsqueda exponencial (Exponential Search)

Idea: Exponential Search resuelve un problema que Binary Search normal no puede: buscar en un arreglo ordenado **del que no se conoce el tamaño** (o es **muy grande** y no conviene arrancar con $[0, n-1]$ completo). Primero **encuentra un rango** donde seguro está el elemento, duplicando un índice $i = 1, 2, 4, 8, 16, \dots$ mientras $\text{arr}[i] < \text{objetivo}$ (o mientras i no se salga del arreglo/stream) —en cuanto $\text{arr}[i] \geq \text{objetivo}$, el elemento (si existe) está en $[i/2, i]$; luego aplica **Binary Search normal** directamente sobre ese rango, ya acotado.

¿Por qué usarlo? Es la técnica correcta cuando el "tamaño" del espacio de búsqueda es **desconocido de antemano** o **no acotado** (streams, arreglos infinitos conceptuales, estructuras sin operación $O(1)$ de "tamaño total") —arrancar Binary Search con un límite superior real desconocido no es posible sin primero encontrar **algún** límite válido. También es útil como optimización cuando se **sospecha** que el elemento buscado está **cerca del inicio**: el costo de encontrar el rango es $O(\log(\text{posición real}))$, más barato que $O(\log n)$ completo si la posición real es mucho menor que n .

Casos de uso:

- Buscar en un arreglo ordenado de tamaño **desconocido** o efectivamente **ilimitado** (ej. un stream de datos ordenado, o una interfaz que solo expone $\text{arr}[i]$ sin exponer el tamaño total).
- Buscar en un arreglo muy grande donde se sospecha que el elemento está **cerca del inicio** — el rango de duplicación encuentra la vecindad correcta más rápido que arrancar desde el medio del arreglo completo.

- Como building block de **Binary Search on Answer** cuando el límite superior del rango de respuestas posibles no es obvio de antemano: se duplica hasta encontrar un límite superior válido, y luego se aplica búsqueda binaria normal dentro de ese rango.

Complejidad: $O(\log p)$ donde p es la posición real del elemento (o del límite encontrado) — $O(\log p)$ para encontrar el rango por duplicación, más $O(\log p)$ del Binary Search final dentro de ese rango.

```

1 struct BusquedaExponencial {
2     // READ: busca 'objetivo' en arr (YA ORDENADO). n puede ser
3     // el
4     // tamaño real, o un límite superior conservador si no se
5     // conoce
6     // el tamaño exacto (siempre que acceder arr[i] fuera de
7     // rango
8     // real este protegido -- ver nota abajo). Devuelve el índice
9     // si
10    // lo encuentra, -1 si no esta.
11    static int buscar(vector<int>& arr, int n, int objetivo) {
12        if (arr[0] == objetivo) return 0;
13
14        // fase 1: encuentra un rango [i/2, i] donde el objetivo
15        // podría estar, duplicando i mientras arr[i] < objetivo
16        int i = 1;
17        while (i < n && arr[i] < objetivo)
18            i *= 2;
19
20        // fase 2: binary search normal dentro de [i/2, min(i, n-1)]
21        int izq = i / 2, der = min(i, n - 1);
22        return binarySearch(arr, izq, der, objetivo);
23    } // O(log p)
24
25    static int binarySearch(vector<int>& arr, int izq, int der,
26    int objetivo) {
27        while (izq <= der) {
28            int mid = izq + (der - izq) / 2;
29
30            if (arr[mid] == objetivo) return mid;
31            if (arr[mid] < objetivo) izq = mid + 1;
32            else der = mid - 1;
33        }
34        return -1;
35    } // O(log(der - izq))
36 };

```

Ejemplo de funcionamiento:

```

1 vector<int> arr = {1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23};
2
3 cout << BusquedaExponencial::buscar(arr, arr.size(), 13);
4 // 6 (arr[6] = 13; el rango encontrado por duplicación fue
5 // [4,8]
6 // antes de aplicar binary search adentro)
7
8 cout << BusquedaExponencial::buscar(arr, arr.size(), 4);
9 // -1 (no esta en el arreglo)

```

Nota sobre tamaño desconocido: si realmente no se conoce n (ej. un stream), el chequeo $i < n$ de la fase de duplicación se reemplaza por un chequeo de "fin de stream" (o un centinela que indique que ya no hay más elementos) — la lógica de duplicar y luego aplicar Binary Search no cambia, solo la condición de corte al buscar el rango.

Regla práctica: usa Exponential Search en vez de Binary Search directo solo cuando el tamaño del arreglo es desconocido/no acotado, o cuando hay una razón concreta para sospechar que el elemento está cerca del inicio — si el tamaño ya se conoce y no hay sospecha de posición, Binary Search normal es más simple y da la misma complejidad asintótica sin la fase extra de duplicación.

25.10 Búsqueda por saltos (Jump Search)

Idea: Jump Search busca en un arreglo **ordenado** avanzando en **bloques de tamaño fijo** $\sqrt{n}$ (en vez de saltar a la mitad como Binary Search, o duplicar como Exponential Search): salta de bloque en bloque mientras el último elemento del bloque actual sea **menor** que el objetivo, y en cuanto encuentra el bloque donde **podría** estar (su último elemento es $\geq$ objetivo), hace una **búsqueda lineal** dentro de ese bloque de tamaño $\sqrt{n}$. El tamaño de bloque óptimo es exactamente $\sqrt{n}$: balancea el número de saltos (n/bloque) contra el costo de la búsqueda lineal final (bloque) — ambos términos se minimizan simultáneamente cuando $\text{bloque} = \sqrt{n}$.

¿Por qué usarlo? Es estrictamente **peor** que Binary Search en complejidad asintótica ($O(\sqrt{n})$ contra $O(\log n)$) — su único valor práctico es en estructuras donde **retroceder** es costoso o imposible (ej. buscar en una cinta secuencial, o cuando "saltar hacia atrás" tiene un costo real distinto de "avanzar"), ya que Jump Search solo avanza en una dirección, nunca necesita retroceder como sí puede requerir la recursión de Binary Search en estructuras con acceso puramente secuencial. En competencia normal, con acceso $O(1)$ a cualquier índice, no hay razón para preferirlo sobre Binary Search.

Casos de uso:

- Búsqueda en estructuras de acceso **secuencial** donde retroceder tiene un costo alto o no es posible (cintas, algunos tipos de streams).
- Como introducción pedagógica al **balance de trade-offs** en algoritmos: por qué $\sqrt{n}$ es el tamaño de bloque óptimo cuando se combinan un costo lineal con uno inversamente proporcional al tamaño de bloque (el mismo tipo de análisis aparece en *sqrt decomposition* para RMQ o en la optimización de consultas por bloques en general).
- Punto de comparación conceptual antes de introducir *sqrt decomposition* completa como estructura de datos (fuera del alcance de esta sección).

Complejidad: $O(\sqrt{n})$ — $O(\sqrt{n})$ saltos de bloque en bloque, más $O(\sqrt{n})$ de búsqueda lineal dentro del bloque final.

```

1 struct BusquedaPorSaltos {
2     // READ: busca 'objetivo' en arr (YA ORDENADO), saltando en
3     // bloques de tamaño sqrt(n). Devuelve el índice si lo
4     // encuentra,
5     // -1 si no esta.
6     static int buscar(vector<int>& arr, int objetivo) {
7         int n = arr.size();
8         int tamBloque = max(1, (int)sqrt((double)n));
9
10        // fase 1: salta de bloque en bloque mientras el ultimo
11        // elemento del bloque actual sea MENOR que el objetivo
12        int actual = 0, siguiente = tamBloque;
13        while (siguiente < n && arr[siguiente - 1] < objetivo) {
14            actual = siguiente;
15            siguiente += tamBloque;
16        }
17
18        // fase 2: búsqueda LINEAL dentro del bloque candidato
19        int limite = min(siguiente, n);
20        for (int i = actual; i < limite; i++)
21            if (arr[i] == objetivo)
22                return i;
23
24        return -1;
25    } // O(sqrt(n))
26 };

```


Ejemplo de funcionamiento:

```

1 vector<int> arr = {0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144};
2 // n=13, tamBloque = sqrt(13) ~= 3
3
4 cout << BusquedaPorSaltos::buscar(arr, 55);
5 // 10 (arr[10] = 55; encontrado saltando bloques de 3 hasta
6 // ubicar el bloque correcto, luego lineal dentro de el)
7
8 cout << BusquedaPorSaltos::buscar(arr, 6);
9 // -1 (no esta en el arreglo)

```

Regla práctica: en competencia con acceso $O(1)$ a cualquier índice, Binary Search siempre domina a Jump Search en complejidad —no hay razón práctica para usar esta técnica salvo el caso específico de acceso puramente secuencial sin retroceso barato. Su valor real en esta guía es **conceptual**: introduce la idea de "tamaño de bloque $\sqrt{n}$ como balance óptimo entre dos costos opuestos", un patrón que reaparece en técnicas de *sqrt decomposition* más generales.

25.11 Meet in the Middle

Idea: Meet in the Middle resuelve problemas de búsqueda exhaustiva sobre n elementos ($O(2^n)$, típicamente subconjuntos) **partiendo** el conjunto en dos mitades de tamaño $n/2$ cada una: se generan **todas** las combinaciones posibles de cada mitad por separado ($O(2^{n/2})$ cada una, mucho más barato que $O(2^n)$), y luego se **combinan** los resultados de ambas mitades —usualmente ordenando una mitad y aplicando Two Pointers o Binary Search contra la otra— para encontrar la respuesta global sin tener que enumerar las 2^n combinaciones completas.

¿Por qué usarlo? Convierte un problema con n hasta ~ 40 (imposible de fuerza bruta directa, 2^{40} es demasiado) en uno resoluble: $2^{20} + 2^{20}$ combinaciones generadas, más el costo de combinarlas (usualmente $O(2^{n/2} \log(2^{n/2}))$ por el `sort`), es perfectamente manejable en el límite de tiempo típico. Es el mismo principio que *divide and conquer*, pero aplicado al **espacio de búsqueda** en vez de a la estructura de los datos: partir el problema exponencial en dos problemas exponenciales **más chicos**, cuya suma de costos es mucho menor que el original.

Casos de uso:

- **Subset Sum** con n hasta ~ 40 : ¿existe un subconjunto que sume exactamente X ? Se generan todas las sumas posibles de cada mitad, se ordena una, y se busca en la otra (con Two Pointers o Binary Search) el complemento necesario.
- Contar cuántos subconjuntos cumplen una condición de suma (variante de conteo del caso anterior).
- Problemas de "elige un subconjunto de cada uno de dos grupos disjuntos, maximizando/minimizando algo" donde generar todas las combinaciones de cada grupo por separado es factible.
- Como técnica complementaria a Bitmask DP cuando n es demasiado grande para $O(2^n \cdot n)$ pero sigue siendo pequeño en cada mitad.

Complejidad: $O(2^{n/2} \log(2^{n/2})) = O(2^{n/2} \cdot n)$ —generar cada mitad es $O(2^{n/2})$, ordenar una de ellas domina el costo total.

```

1 struct MeetInTheMiddle {
2     // READ: existe un subconjunto de arr que suma EXACTAMENTE X?
3     // Parte arr en dos mitades, genera todas las sumas posibles
4     // de
5     // cada una, y busca el complemento con Two Pointers.
6     static bool existeSubconjuntoConSuma(vector<int>& arr, long
7     long X) {
8         int n = arr.size();
9         int mitad = n / 2;
10
11         vector<long long> sumasA = generarSumas(arr, 0, mitad);
12         vector<long long> sumasB = generarSumas(arr, mitad, n);
13
14         sort(sumasB.begin(), sumasB.end());
15
16         // para cada suma de A, busca si (X - sumaA) existe en B
17         for (long long sa : sumasA) {
18             long long faltante = X - sa;
19             if (binary_search(sumasB.begin(), sumasB.end(), faltante))
20                 return true;
21         }
22         return false;
23     } // O(2^(n/2) * n)
24
25     // genera las 2^(fin-inicio) sumas posibles de todos los
26     // subconjuntos del segmento arr[inicio, fin).
27     static vector<long long> generarSumas(vector<int>& arr, int
28     inicio,
29     int fin) {
30         int m = fin - inicio;
31         vector<long long> sumas;
32         sumas.reserve(1 << m);
33
34         for (int mask = 0; mask < (1 << m); mask++) {
35             long long suma = 0;
36             for (int i = 0; i < m; i++)
37                 if (mask & (1 << i))
38                     suma += arr[inicio + i];
39             sumas.push_back(suma);
40         }
41         return sumas;
42     } // O(2^m * m)
43
44     // READ: cuenta cuantos SUBCONJUNTOS (no vacios o vacios
45     // incluidos
46     // segun se cuente el mask=0) suman EXACTAMENTE X, usando el
47     // mismo esquema de dos mitades pero con conteo de
48     // coincidencias
49     // en vez de existencia.
50     static long long contarSubconjuntosConSuma(vector<int>& arr,
51     long long X) {
52         int n = arr.size();
53         int mitad = n / 2;
54
55         vector<long long> sumasA = generarSumas(arr, 0, mitad);
56         vector<long long> sumasB = generarSumas(arr, mitad, n);
57
58         sort(sumasB.begin(), sumasB.end());
59
60         long long total = 0;
61         for (long long sa : sumasA) {
62             long long faltante = X - sa;
63             // cuenta CUANTAS veces aparece 'faltante' en sumasB
64             auto rango = equal_range(sumasB.begin(), sumasB.end(),
65             faltante);
66             total += rango.second - rango.first;
67         }
68         return total;
69     } // O(2^(n/2) * n)
70 };

```

Ejemplo de funcionamiento:

```

1 vector<int> arr = {3, 34, 4, 12, 5, 2};
2
3 cout << MeetInTheMiddle::existeSubconjuntoConSuma(arr, 9);
4 // true (4 + 5 = 9, o 3 + 4 + 2 = 9)
5
6 cout << MeetInTheMiddle::existeSubconjuntoConSuma(arr, 100);
7 // false (la suma total del arreglo es 60, menor a 100)

```

Regla práctica: la señal para Meet in the Middle es n demasiado grande para $O(2^n)$ directo ($n \gtrsim 25-30$) pero demasiado chico para que un enfoque polinomial (DP, greedy) sea obviamente aplicable —usualmente $n \leq 40$. Si n es lo bastante chico para $O(2^n)$ liso ($n \leq 20-22$),

Meet in the Middle es complejidad extra innecesaria; si n es grande ($n > 40-50$), ni siquiera $O(2^{n/2})$ alcanza y hace falta buscar una solución polinomial real.

25.12 Heap Sort

Idea: Heap Sort ordena un arreglo en dos fases usando la estructura de **heap binario** (implementado sobre el mismo arreglo, sin memoria extra): (1) **construir** un *max-heap* a partir del arreglo completo, aplicando **heapify** desde el último nodo **interno** hacia la raíz (no hace falta empezar desde las hojas, ya son heaps triviales de tamaño 1); (2) repetir $n - 1$ veces: **intercambiar** la raíz (el máximo actual) con el **último** elemento del heap activo, **reducir** el tamaño del heap en 1 (ese último elemento ya queda en su posición final ordenada), y aplicar **heapify** desde la raíz para restaurar la propiedad de heap sobre lo que queda.

¿Por qué usarlo? Es el único de los sorts de comparación clásicos ($O(n \log n)$) que logra $O(1)$ de memoria extra y garantiza $O(n \log n)$ en el **peor caso** —Quick Sort es $O(n \log n)$ en promedio pero $O(n^2)$ en el peor caso (sin aleatorización de pivote), y Merge Sort garantiza $O(n \log n)$ siempre pero necesita $O(n)$ de memoria auxiliar. La desventaja práctica es que Heap Sort tiene peor localidad de caché (los accesos saltan por todo el arreglo según la estructura del heap), por lo que en la práctica suele ser más lento que Quick Sort bien implementado, aunque ambos compartan el mismo orden asintótico promedio.

Casos de uso:

- Ordenar cuando se necesita **garantía** de $O(n \log n)$ en el peor caso y memoria extra $O(1)$ simultáneamente (ninguna otra opción clásica da ambas garantías a la vez).
- Como base conceptual para entender **priority_queue** de la STL, que internamente usa exactamente esta estructura de heap binario sobre un arreglo.
- Encontrar los k elementos más grandes/chicos sin ordenar todo el arreglo: construir el heap ($O(n)$) y extraer solo k veces ($O(k \log n)$), en vez de un sort completo ($O(n \log n)$) cuando $k \ll n$.

Complejidad: $O(n \log n)$ garantizado (peor, promedio y mejor caso son todos $O(n \log n)$) — $O(n)$ para construir el heap inicial (no $O(n \log n)$, por un análisis más fino de la suma de alturas), $O(\log n)$ por cada una de las $n - 1$ extracciones.

```

1 struct HeapSort {
2     // restaura la propiedad de MAX-HEAP en el subarbol con raiz
3     // 'i', asumiendo que sus hijos YA cumplen la propiedad (el
4     // desorden, si existe, esta solo en 'i').
5     static void heapify(vector<int>& arr, int tamHeap, int i) {
6         int mayor = i;
7         int izq = 2 * i + 1, der = 2 * i + 2;
8
9         if (izq < tamHeap && arr[izq] > arr[mayor]) mayor = izq;
10        if (der < tamHeap && arr[der] > arr[mayor]) mayor = der;
11
12        if (mayor != i) {
13            swap(arr[i], arr[mayor]);
14            heapify(arr, tamHeap, mayor); // el desorden pudo
15            // "bajar" al hijo, hay
16            // que seguir corrigiendo

```

```

17    }
18    } // O(log n)
19
20    // WRITE: ordena arr in-place, ascendente.
21    static void ordenar(vector<int>& arr) {
22        int n = arr.size();
23
24        // FASE 1: construye el max-heap. Empieza en el ultimo nodo
25        // INTERNO (n/2 - 1), no en las hojas (ya son heaps
26        // triviales).
27        for (int i = n / 2 - 1; i >= 0; i--)
28            heapify(arr, n, i);
29        // esta fase completa es O(n), no O(n log n) -- la mayoría
30        // de
31        // nodos estan cerca de las hojas, donde heapify hace poco
32        // trabajo; solo unos pocos nodos cerca de la raiz pagan el
33        // costo completo de log n
34
35        // FASE 2: extrae el maximo repetidamente, colocandolo al
36        // final del rango activo.
37        for (int tamHeap = n - 1; tamHeap > 0; tamHeap--) {
38            swap(arr[0], arr[tamHeap]); // raiz (maximo actual) va
39            // a su posicion final
40            heapify(arr, tamHeap, 0); // restaura el heap sobre
41            // lo que queda (tamano
42            // reducido en 1)
43        }
44    } // O(n log n)

```

Ejemplo de funcionamiento:

```

1 vector<int> arr = {12, 11, 13, 5, 6, 7};
2
3 HeapSort::ordenar(arr);
4
5 for (int x : arr) cout << x << " ";
6 // 5 6 7 11 12 13

```

Nota: la construcción del heap (FASE 1) parece a primera vista $O(n \log n)$ (n llamadas a **heapify**, cada una $O(\log n)$), pero un análisis más cuidadoso —sumando la altura real de cada nodo, no el peor caso uniforme— da $O(n)$ real: la **mitad** de los nodos son hojas (altura 0, sin trabajo), un cuarto tiene altura 1, etc., y esa serie geométrica converge a $O(n)$ en total, no a $O(n \log n)$.

Regla práctica: en competencia, **std::sort** (Quick Sort/Intro Sort híbrido de la STL) es casi siempre la opción por defecto —más rápido en la práctica por mejor localidad de caché. Implementa Heap Sort a mano **solo** cuando el problema específicamente requiere la garantía de peor caso $O(n \log n)$ sin memoria extra (raro en competencia), o cuando se necesita la estructura de heap **en sí** para algo más (como extraer solo los k mayores) — en ese caso, **priority_queue** de la STL ya implementa exactamente este mismo algoritmo internamente.

25.13 Merge Sort

Idea: Merge Sort ordena un arreglo con *divide and conquer*: divide el arreglo por la **mitad** recursivamente hasta llegar a subarreglos de tamaño 1 (trivialmente ordenados), y luego **combina** (*merge*) pares de subarreglos ya ordenados en uno solo ordenado, usando el mismo esquema de Two Pointers ya visto: un puntero por subarreglo, avanzando siempre el que tiene el menor elemento actual, hasta consumir ambos por completo. La recursión completa hace $O(\log n)$ niveles de división, y cada nivel hace $O(n)$ de trabajo total en sus *merges*, dando $O(n \log n)$.

¿Por qué usarlo? A diferencia de Quick Sort (promedio $O(n \log n)$, peor caso $O(n^2)$), Merge Sort **garantiza** $O(n \log n)$ en **todos** los casos, sin depender de la elección de pivote ni de aleatorización — el costo es $O(n)$

de memoria auxiliar (el arreglo temporal para el *merge*), que Heap Sort evita pero a cambio de peor localidad de caché. Es además **estable** (mantiene el orden relativo de elementos iguales), propiedad que ni Quick Sort ni Heap Sort garantizan por defecto — y es la base directa de Counting Inversions (siguiente subsección), que reutiliza exactamente esta misma recursión.

Casos de uso:

- Ordenar cuando se necesita **estabilidad** (preservar el orden relativo de elementos con la misma clave) o garantía de peor caso $O(n \log n)$ sin las complicaciones de aleatorizar un pivote.
- Como building block directo de Counting Inversions: contar cuántos pares están "fuera de orden" se calcula gratis durante el mismo *merge*.
- *External sorting*: ordenar datos que no caben en memoria, procesando bloques y fusionándolos — el mismo principio de *merge* de dos secuencias ordenadas se extiende naturalmente a fusionar k bloques desde disco.
- Como plantilla de *divide and conquer* general: cualquier problema que se resuelva "divide en dos mitades, resuelve cada una, combina en $O(n)$ " tiene a Merge Sort como su ejemplo canónico.

Complejidad: $O(n \log n)$ garantizado en todos los casos (peor, promedio, mejor), $O(n)$ de memoria auxiliar para los *merges*.

```
4 for (int x : arr) cout << x << " ";
5 // 3 9 10 27 38 43 82
6
```

Regla práctica: en competencia, `std::sort` (o `std::stable_sort` si se necesita estabilidad) casi siempre reemplaza una implementación manual de Merge Sort —salvo que el problema pida **modificar** el algoritmo mismo (como en Counting Inversions, donde se necesita instrumentar el *merge* para contar algo adicional durante la fusión, algo que `std::sort` no permite hacer desde afuera).

25.14 Conteo de Inversiones (Counting Inversions)

Idea: una **inversión** es un par de índices (i, j) con $i < j$ pero `arr[i] > arr[j]` (el par está "fuera de orden"). Contarlas todas por fuerza bruta es $O(n^2)$; se resuelve en $O(n \log n)$ instrumentando **exactamente** el mismo Merge Sort de la subsección anterior: durante el *merge* de dos mitades ya ordenadas, cada vez que se toma un elemento de la mitad **derecha antes** que uno de la mitad **izquierda** (porque es menor), ese elemento forma una inversión con **todos** los elementos **restantes** de la mitad izquierda —ya que la mitad izquierda está ordenada, todos esos restantes son también mayores, y automáticamente forman inversión con el elemento de la derecha que se acaba de tomar.

¿Por qué usarlo? Es el ejemplo canónico de "instrumentar *divide and conquer* para contar algo adicional sin cambiar la complejidad" —el conteo sale **gratis** dentro del mismo *merge* que ya se hacía para ordenar, sin ningún recorrido extra. La alternativa con Fenwick Tree (procesar el arreglo de derecha a izquierda, consultando cuántos elementos **menores** ya se insertaron) también es $O(n \log n)$ y es preferible cuando de todas formas ya se tiene un Fenwick a mano o se necesita una variante más flexible (ej. contar inversiones con una condición extra) — pero el enfoque de Merge Sort es más directo cuando el problema es exactamente "contar inversiones", sin extras.

Casos de uso:

- Contar cuántos intercambios de elementos **adyacentes** hacen falta como mínimo para ordenar un arreglo (es exactamente el número de inversiones — cada intercambio adyacente corrige **una** inversión).
- Medir qué tan "desordenado" está un arreglo respecto a su versión ordenada (similitud entre dos rankings, por ejemplo).
- Como subrutina de problemas más grandes que reducen su núcleo a "cuenta pares fuera de orden" tras alguna transformación previa del arreglo original.

Complejidad: $O(n \log n)$ — mismo costo que Merge Sort, ya que el conteo se hace sin trabajo asintótico adicional durante el *merge*.

```
1 struct ConteoInversiones {
2     // fusiona arr[izq..mid] y arr[mid+1..der], contando
3     // inversiones:
4     // cada vez que se toma un elemento de la mitad DERECHA antes
5     // que
6     // uno de la IZQUIERDA, se suman TODOS los elementos
7     // restantes de
```

```
1 struct MergeSort {
2     // fusiona arr[izq..mid] y arr[mid+1..der] (ambos YA
3     // ORDENADOS
4     // entre si) en un solo rango ordenado, usando un arreglo
5     // temporal (Two Pointers, mismo esquema que fusionar() de la
6     // seccion de Two Pointers).
7     static void merge(vector<int>& arr, int izq, int mid, int der
8     ) {
9         vector<int> temp(der - izq + 1);
10        int i = izq, j = mid + 1, k = 0;
11
12        while (i <= mid && j <= der) {
13            if (arr[i] <= arr[j]) temp[k++] = arr[i++];
14            else temp[k++] = arr[j++];
15        }
16        while (i <= mid) temp[k++] = arr[i++];
17        while (j <= der) temp[k++] = arr[j++];
18
19        for (int x = 0; x < (int)temp.size(); x++)
20            arr[izq + x] = temp[x];
21        // O(der - izq)
22
23        // WRITE: ordena arr[izq..der] recursivamente (divide and
24        // conquer).
25        static void ordenar(vector<int>& arr, int izq, int der) {
26            if (izq >= der) return; // caso base: 0 o 1 elemento, ya
27            // ordenado
28
29            int mid = izq + (der - izq) / 2;
30
31            ordenar(arr, izq, mid);
32            ordenar(arr, mid + 1, der);
33            merge(arr, izq, mid, der);
34        } // O(n log n)
35
36        // conveniencia: ordena el arreglo completo.
37        static void ordenar(vector<int>& arr) {
38            if (!arr.empty())
39                ordenar(arr, 0, arr.size() - 1);
40        }
41    };
42
```

Ejemplo de funcionamiento:

```
1 vector<int> arr = {38, 27, 43, 3, 9, 82, 10};
2
3 MergeSort::ordenar(arr);
```

```

5 // la izquierda como inversiones con ese elemento.
6 static long long merge(vector<int>& arr, int izq, int mid,
7 int der) {
8     vector<int> temp(der - izq + 1);
9     int i = izq, j = mid + 1, k = 0;
10    long long inversiones = 0;
11
12    while (i <= mid && j <= der) {
13        if (arr[i] <= arr[j]) {
14            temp[k++] = arr[i++];
15        } else {
16            // arr[j] es MENOR que arr[i], y como [izq..mid] esta
17            // ordenado, arr[j] tambien es menor que TODO lo que
18            // queda desde i hasta mid -- eso son (mid - i + 1)
19            // inversiones de una sola vez
20            inversiones += (mid - i + 1);
21            temp[k++] = arr[j++];
22        }
23    }
24    while (i <= mid) temp[k++] = arr[i++];
25    while (j <= der) temp[k++] = arr[j++];
26
27    for (int x = 0; x < (int)temp.size(); x++)
28        arr[izq + x] = temp[x];
29
30    return inversiones;
31 } // O(der - izq)
32
33 // READ: cuenta inversiones en arr[izq..der], ordenando de
34 // paso
35 // (modifica arr -- pasar una COPIA si se necesita conservar
36 // el
37 // orden original).
38 static long long contar(vector<int>& arr, int izq, int der) {
39     if (izq >= der) return 0;
40
41     int mid = izq + (der - izq) / 2;
42
43     long long total = 0;
44     total += contar(arr, izq, mid);
45     total += contar(arr, mid + 1, der);
46     total += merge(arr, izq, mid, der);
47
48     return total;
49 } // O(n log n)
50
51 // conveniencia: cuenta inversiones en TODO el arreglo (
52 // recibe
53 // una COPIA para no alterar el original del llamador).
54 static long long contar(vector<int> arr) {
55     if (arr.empty()) return 0;
56     return contar(arr, 0, arr.size() - 1);
57 }
58 };

```

Ejemplo de funcionamiento:

```

1 vector<int> arr = {8, 4, 2, 1};
2
3 cout << ConteoInversiones::contar(arr);
4 // 6
5 // pares fuera de orden: (8,4)(8,2)(8,1)(4,2)(4,1)(2,1) -> 6 en
6 // total
7 // (arreglo totalmente invertido de tamaño 4: n*(n-1)/2 = 6)

```

Regla práctica: la función `contar()` recibe el arreglo **por valor** (copia) a propósito en su versión de conveniencia — el algoritmo necesita **ordenar** el arreglo como efecto secundario del conteo, así que si el problema requiere conservar el orden original después de contar, hay que operar sobre una copia (como se hace aquí) en vez de sobre el arreglo del llamador directamente.

25.15 QuickSelect

Idea: QuickSelect resuelve el problema de encontrar el k -ésimo elemento más pequeño (la k -ésima estadística de orden) de un arreglo sin necesidad de ordenarlo por completo. Se basa en el mismo esquema de partición que QuickSort: se elige un pivote, se reorganiza el arreglo de modo que todo lo menor al pivote quede a su izquierda y todo lo mayor a su derecha, y el pivote termina exactamente en su posición final ordenada. La diferencia clave frente a QuickSort es que, tras particionar, QuickSelect

descarta el lado que no contiene la posición k buscada y solo recurre sobre el lado relevante, en vez de recurrir sobre ambos lados. Esto reduce el tamaño esperado del problema geoméricamente (aproximadamente a la mitad en cada paso, con un pivote razonable), en vez de linealmente como ocurriría si se ordenara todo.

¿Por qué usarlo? Cuando solo se necesita un elemento de orden específico (la mediana, el percentil 90, el k -ésimo mayor) y no el arreglo completo ordenado, ordenar todo con `sort` cuesta $O(n \log n)$ de forma innecesaria. QuickSelect resuelve el mismo problema en $O(n)$ esperado. Se diferencia de estructuras como un *heap* (que da $O(n + k \log n)$ para el k -ésimo menor) en que no requiere mantener ninguna estructura auxiliar ni soporta consultas repetidas eficientes: es ideal para una consulta única sobre datos estáticos. Si se necesitan múltiples consultas de orden sobre el mismo conjunto, o si el conjunto cambia dinámicamente, es mejor usar un *order statistics tree* o un *heap* persistente.

Casos de uso:

- Encontrar la mediana de un arreglo sin ordenarlo completo.
- Calcular percentiles (p. ej. el elemento en la posición $\lceil 0.9n \rceil$) para estadística descriptiva.
- Como subrutina para encontrar los k elementos más cercanos a un punto de referencia (particionar y quedarse con un lado).
- Selección de mediana de medianas como pivote determinista en algoritmos de ordenamiento con garantías de peor caso.
- Problemas de competencia donde se pide el k -ésimo mayor/menor elemento de un arreglo grande bajo límite de tiempo ajustado.

Complejidad: El caso esperado es $O(n)$: cada partición cuesta $O(m)$ sobre un subarreglo de tamaño m , y con un pivote aleatorio el tamaño esperado del subarreglo restante se reduce por un factor constante en cada paso, dando una serie geométrica $n + n/2 + n/4 + \dots = O(n)$ en total. El peor caso es $O(n^2)$, que ocurre si el pivote elegido resulta ser siempre el mínimo o el máximo del subarreglo (por ejemplo, con pivote fijo sobre datos adversarios); la aleatorización del pivote hace que este peor caso sea extremadamente improbable en la práctica.

```

1 struct QuickSelect {
2     vector<int> arr;
3     mt19937 rng;
4
5     // CREATE: copia los datos de entrada y prepara el generador
6     // aleatorio para la seleccion de pivotes
7     QuickSelect(vector<int> datos)
8     : arr(datos),
9       rng(chrono::steady_clock::now().time_since_epoch().count())
10    {} // O(n)
11
12    // WRITE: particiona arr[izq..der] usando arr[der] como
13    // pivote
14    // (tras intercambiarlo con un pivote aleatorio), dejando los
15    // menores al pivote a la izquierda; retorna la posición
16    // final
17    // del pivote
18    int particionar(int izq, int der) {
19        int pivoteIdx = izq + rng() % (der - izq + 1);
20        swap(arr[pivoteIdx], arr[der]);
21        int pivote = arr[der];
22        int i = izq;
23        for (int j = izq; j < der; j++) {

```



```

21     if (arr[j] < pivot) {
22         swap(arr[i], arr[j]);
23         i++;
24     }
25 }
26 swap(arr[i], arr[der]);
27 return i;
28 } // O(der - izq)
29
30 // READ: retorna el elemento en la posición k (0-indexado)
31 // tendría el arreglo si estuviera completamente ordenado
32 int kEsimoMenor(int k) {
33     int izq = 0, der = (int)arr.size() - 1;
34     while (true) {
35         if (izq == der) return arr[izq];
36         int p = particionar(izq, der);
37         if (k == p) return arr[p];
38         else if (k < p) der = p - 1;
39         else izq = p + 1;
40     }
41 } // O(n) esperado, O(n^2) peor caso
42
43 //
44 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
45 //
46 // 1. Version iterativa vs recursiva: la version de arriba ya
47 //    es
48 //    iterativa (bucle while) para evitar overflow de pila en
49 //    arreglos muy grandes; una version recursiva es
50 //    equivalente
51 //    pero con riesgo de stack overflow para n grande.
52 // 2. Mediana de medianas (peor caso O(n) garantizado): en
53 //    vez de
54 //    pivote aleatorio, se divide el arreglo en grupos de 5,
55 //    se
56 //    calcula la mediana de cada grupo, y se usa
57 //    recursivamente
58 //    la mediana de esas medianas como pivote. Garantiza que
59 //    el
60 //    pivote elimina al menos un 30% del arreglo en cada paso,
61 //    dando O(n) en el peor caso, pero con una constante
62 //    mayor
63 //    que hace que en la practica sea mas lento que la
64 //    version
65 //    aleatorizada.
66 // 3. k-esimo mayor en vez de menor: basta con llamar
67 //    kEsimoMenor(n - 1 - k) sobre el mismo arreglo, ya que
68 //    el
69 //    k-esimo mayor es el (n-1-k)-esimo menor.
70 // 4. Obtener los k elementos mas pequeños (no solo el k-
71 //    esimo):
72 //    tras llamar particionar hasta que la posición del
73 //    pivote
74 //    sea k, los primeros k elementos de arr (arr[0..k-1])
75 //    son
76 //    exactamente esos k elementos, aunque no necesariamente
77 //    ordenados entre si.
78 };

```

Ejemplo de funcionamiento:

```

1 vector<int> datos = {7, 2, 9, 4, 1, 6, 3};
2 QuickSelect qs(datos);
3
4 int mediana = qs.kEsimoMenor(3); // arreglo ordenado seria
5 // {1,2,3,4,6,7,9} -> posición 3 = 4
6 // mediana = 4
7
8 int minimo = qs.kEsimoMenor(0); // minimo = 1
9 int maximo = qs.kEsimoMenor((int)datos.size() - 1); // maximo = 9

```

Nota: La partición modifica `arr` en el sitio, por lo que después de llamar `kEsimoMenor`, el orden interno del arreglo cambia (aunque el conjunto de elementos se preserva). Si se necesita conservar el arreglo original, se debe pasar una copia al construir el struct o guardar una copia aparte antes de consultar.

Regla práctica: Usa `QuickSelect` cuando necesites un único elemento de orden (mediana, percentil, k -ésimo mayor/menor) de un conjunto *estático* y no te importe alterar su orden interno; si necesitas el arreglo completo

ordenado de todas formas, usa `sort` directamente; si necesitas múltiples consultas de orden o el conjunto cambia con el tiempo, usa una estructura como un *order statistics tree*.

26 Búsqueda Binaria (Binary Search)

26.1 En arreglo ordenado

Idea: La búsqueda binaria explota el hecho de que, en un arreglo ordenado, comparar el elemento central contra el valor buscado permite descartar la mitad del espacio de búsqueda en cada paso: si el elemento central es igual al buscado, se encontró; si es menor, la respuesta (si existe) está estrictamente a la derecha; si es mayor, está estrictamente a la izquierda. Al descartar la mitad del espacio en cada iteración, el número de comparaciones necesarias crece logarítmicamente con el tamaño del arreglo, en vez de linealmente como en una búsqueda secuencial.

¿Por qué usarlo? Cuando el arreglo ya está ordenado (o se puede ordenar una sola vez y consultar muchas veces), buscar un elemento con búsqueda binaria cuesta $O(\log n)$ frente a $O(n)$ de una búsqueda lineal. A diferencia de `QuickSelect`, que encuentra el k -ésimo elemento de orden en un arreglo *desordenado* sin necesidad de ordenarlo, la búsqueda binaria requiere que el arreglo ya esté ordenado, pero a cambio responde a la pregunta "¿está este valor exacto presente, y en qué posición?" en tiempo logarítmico, y puede reutilizarse para múltiples consultas sin costo adicional de preprocesamiento.

Casos de uso:

- Verificar si un valor existe en un arreglo ordenado y obtener su índice.
- Encontrar el punto de inserción de un valor manteniendo el orden (equivalente a `lower_bound/upper_bound`).
- Contar ocurrencias de un valor en un arreglo ordenado (diferencia entre `upper_bound` y `lower_bound`).
- Como subrutina dentro de estructuras más complejas (por ejemplo, búsqueda en un arreglo con offsets, o en un Fenwick tree con búsqueda binaria implícita).

Complejidad: $O(\log n)$ por consulta, ya que en cada iteración el tamaño del espacio de búsqueda se reduce a la mitad: partiendo de n elementos, tras t iteraciones quedan $n/2^t$ elementos, y el proceso termina cuando $n/2^t \approx 1$, es decir $t \approx \log_2 n$. El espacio usado es $O(1)$ en la versión iterativa, ya que no se requiere memoria adicional proporcional a n .

```

1 struct BusquedaBinaria {
2     vector<int> arr;
3
4     // CREATE: guarda una copia del arreglo, que debe estar
5     // ordenado
6     // de forma ascendente
7     BusquedaBinaria(vector<int> datos) : arr(datos) {} // O(n)
8
9     // READ: retorna el índice del elemento igual a "valor" si
10    // existe, o -1 si no está presente en el arreglo
11    int buscar(int valor) {
12        int izq = 0, der = (int)arr.size() - 1;

```

```

12 while (izq <= der) {
13     int medio = izq + (der - izq) / 2;
14     if (arr[medio] == valor) return medio;
15     else if (arr[medio] < valor) izq = medio + 1;
16     else der = medio - 1;
17 }
18 return -1;
19 } // O(log n)
20
21 //
22 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
23 //
24 // 1. Evitar overflow en el calculo del medio: usar
25 // izq + (der - izq) / 2 en vez de (izq + der) / 2, ya que
26 // la segunda forma puede desbordar si izq y der son cercanos
27 // al limite de int.
28 //
29 // 2. Version con funciones nativas de C++: para arreglos
30 // ordenados, std::binary_search(arr.begin(), arr.end(),
31 // valor)
32 // retorna un bool en vez del indice; si se necesita el
33 // indice, se puede usar std::lower_bound y verificar que el valor
34 // en esa posicion sea igual al buscado.
35 //
36 // 3. Busqueda en arreglo ordenado descendente: invertir las
37 // comparaciones (si arr[medio] < valor, mover der = medio
38 // - 1;
39 // si arr[medio] > valor, mover izq = medio + 1).
40 };

```

Ejemplo de funcionamiento:

```

1 vector<int> datos = {1, 3, 5, 7, 9, 11, 13};
2 BusquedaBinaria bb(datos);
3
4 int idx1 = bb.buscar(7); // idx1 = 3 (arr[3] = 7)
5 int idx2 = bb.buscar(4); // idx2 = -1 (4 no esta en el arreglo)

```

Regla práctica: Usa búsqueda binaria clásica cuando busques la presencia y posición exacta de un valor en un arreglo ya ordenado; si en cambio necesitas el primer o último índice donde se cumple una condición monótona (no necesariamente una igualdad exacta), usa las variantes de "menor/mayor x que cumple" que se ven a continuación.

26.2 Menor x que cumple

Idea: Esta variante generaliza la búsqueda binaria clásica: en vez de buscar un valor exacto, busca el menor índice (o valor) x para el cual una predicado booleano $P(x)$ se vuelve verdadero, bajo el supuesto de que P es *monótono*: una vez que $P(x)$ es verdadero, permanece verdadero para todo valor mayor que x (patrón FFFF...FVVVV...V). En cada paso se evalúa P en el punto medio: si $P(\text{medio})$ es verdadero, el medio es un candidato válido pero podría no ser el mínimo, así que se guarda como respuesta tentativa y se sigue buscando a la izquierda; si es falso, la respuesta debe estar estrictamente a la derecha.

¿Por qué usarlo? La búsqueda binaria en arreglo ordenado (vista arriba) resuelve el caso particular donde $P(x) := (\text{arr}[x] \geq \text{valor})$, que es monótono porque el arreglo está ordenado. Esta variante generaliza esa idea a *cualquier* predicado monótono, no solo a comparaciones directas contra un arreglo ordenado, lo que la hace aplicable incluso cuando no hay un arreglo explícito de por medio (por ejemplo, sobre un rango de índices o valores abstractos). Es la base conceptual de `lower_bound` y del patrón "Binary Search on Answer" que se ve más adelante.

Casos de uso:

- Implementar `lower_bound` manualmente: menor índice tal que `arr[índice] >= valor`.
- Encontrar el primer elemento que supera un umbral en una secuencia ordenada.
- Encontrar el primer día, en una serie temporal ordenada, en que se cumple una condición acumulativa (por ejemplo, el primer día en que el saldo acumulado es no negativo).
- Como bloque de construcción para "Binary Search on Answer", donde $P(x)$ es una condición de factibilidad en vez de una comparación directa de arreglo.

Complejidad: $O(\log n \cdot C)$, donde n es el tamaño del espacio de búsqueda y C es el costo de evaluar el predicado P una vez; el factor $\log n$ viene de que el espacio de búsqueda se reduce a la mitad en cada iteración, igual que en la búsqueda binaria clásica. Si P es $O(1)$ (como una simple comparación de arreglo), la complejidad total es $O(\log n)$.

```

1 struct MenorQueCumple {
2     vector<int> arr;
3
4     // CREATE: guarda una copia del arreglo, que debe estar
5     // ordenado
6     // de forma ascendente para que el predicado por defecto sea
7     // monotonico
8     MenorQueCumple(vector<int> datos) : arr(datos) {} // O(n)
9
10    // READ: retorna el menor indice tal que arr[indice] >= valor;
11
12    // si ningun elemento cumple, retorna arr.size()
13    int menorQueCumple(int valor) {
14        int izq = 0, der = (int)arr.size(); // der fuera de rango =
15        // "no encontrado"
16        while (izq < der) {
17            int medio = izq + (der - izq) / 2;
18            if (arr[medio] >= valor) der = medio; // candidato valido
19            // seguir a la izquierda
20            else izq = medio + 1; // no cumple,
21            // buscar a la derecha
22        }
23        return izq;
24    } // O(log n)
25
26    //
27    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
28    //
29    // 1. Predicado generico en vez de comparacion de arreglo: se
30    // puede reemplazar la condicion "arr[medio] >= valor" por
31    // cualquier funcion booleana P(medio) monotonica, sin
32    // necesidad de que exista un arreglo de por medio (asi es
33    // como funciona "Binary Search on Answer").
34    //
35    // 2. Equivalencia con std::lower_bound: esta funcion es
36    // equivalente a
37    // (int)(lower_bound(arr.begin(), arr.end(), valor) - arr.
38    // begin()),
39    // implementada manualmente para mostrar la mecanica
40    // interna.
41    //
42    // 3. Cuidado con el rango inicial: a diferencia de la
43    // busqueda
44    // binaria clasica (izq <= der con der = size()-1), aqui
45    // el
46    // rango es [izq, der) semiabierto con der = size(), lo
47    // que
48    // evita casos especiales cuando la respuesta es "no
49    // existe
50    // ningun indice que cumpla" (se retorna directamente size
51    // ()).
52 };

```

Ejemplo de funcionamiento:

```

1 vector<int> datos = {1, 3, 3, 5, 7, 9};
2 MenorQueCumple mqc(datos);

```

```

3 int idx1 = mqc.menorQueCumple(5); // idx1 = 3 (arr[3] = 5,
4   primer >= 5)
5 int idx2 = mqc.menorQueCumple(4); // idx2 = 3 (arr[3] = 5,
6   primer >= 4)
7 int idx3 = mqc.menorQueCumple(10); // idx3 = 6 (ningun elemento
   >= 10, retorna size())

```

Regla práctica: Usa este patrón cuando la condición que buscas tenga la forma "falso...falso, verdadero...verdadero" a lo largo del espacio de búsqueda y quieras el primer punto de quiebre; si en cambio el patrón es "verdadero...verdadero, falso...falso" y buscas el último punto donde se cumple, usa la variante de "mayor x que cumple" vista a continuación.

26.3 Mayor x que cumple

Idea: Esta variante es la simétrica de "menor x que cumple": busca el mayor índice (o valor) x para el cual un predicado booleano $P(x)$ es verdadero, bajo el supuesto de que P es monótono en el sentido opuesto al anterior: una vez que $P(x)$ se vuelve falso, permanece falso para todo valor mayor (patrón VVVV...VFFFF...F). En cada paso se evalúa P en el punto medio: si $P(\text{medio})$ es verdadero, el medio es un candidato válido pero podría no ser el máximo, así que se guarda como respuesta tentativa y se sigue buscando a la derecha; si es falso, la respuesta debe estar estrictamente a la izquierda.

¿Por qué usarlo? Es el complemento natural de "menor x que cumple": mientras esa variante encuentra el primer punto de quiebre de falso a verdadero, esta encuentra el último punto donde el predicado sigue siendo verdadero antes de quebrar a falso. Muchos problemas de "Binary Search on Answer" (visto a continuación) piden maximizar un valor factible en vez de minimizarlo, y ese es exactamente el patrón que resuelve esta variante: por ejemplo, "¿cuál es la mayor velocidad a la que puedo ir y aún así llegar a tiempo?" en vez de "¿cuál es el menor tiempo necesario?".

Casos de uso:

- Implementar el equivalente a `upper_bound - 1`: último índice tal que `arr[índice] <= valor`.
- Maximizar un recurso (velocidad, capacidad, tamaño) sujeto a una restricción de factibilidad monótona decreciente.
- Encontrar el último día en una serie temporal ordenada en que una condición acumulativa todavía se cumple, antes de dejar de cumplirse permanentemente.
- Como bloque de construcción para variantes de "Binary Search on Answer" que maximizan en vez de minimizar.

Complejidad: $O(\log n \cdot C)$, con n el tamaño del espacio de búsqueda y C el costo de evaluar P una vez; igual que en "menor x que cumple", el factor $\log n$ viene de reducir el espacio de búsqueda a la mitad en cada iteración. Con P de costo $O(1)$, la complejidad total es $O(\log n)$.

```

1 struct MayorQueCumple {
2     vector<int> arr;
3 }

```

```

4 // CREATE: guarda una copia del arreglo, que debe estar
   ordenado
5 // de forma ascendente para que el predicado por defecto sea
6 // monotonó
7 MayorQueCumple(vector<int> datos) : arr(datos) {} // O(n)
8
9 // READ: retorna el mayor índice tal que arr[índice] <= valor;
10
11 // si ningun elemento cumple, retorna -1
12 int mayorQueCumple(int valor) {
13     int izq = -1, der = (int)arr.size() - 1; // izq fuera de
   rango = "no encontrado"
14     while (izq < der) {
15         int medio = izq + (der - izq + 1) / 2; // redondeo hacia
   arriba
16         if (arr[medio] <= valor) izq = medio; // candidato
   valido, seguir a la derecha
17         else der = medio - 1; // no cumple,
   buscar a la izquierda
18     }
19     return izq;
20 } // O(log n)
21
22 // -----
23 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
24 // -----
25 // 1. Redondeo hacia arriba en el medio: a diferencia de "
   menor
26 // x que cumple", aquí el medio se calcula redondeando
   hacia
27 // arriba (+1 antes de dividir); si se redondeara hacia
   abajo
28 // como en la variante anterior, el bucle podría quedar
   atascado sin avanzar cuando der = izq + 1.
29 //
30 // 2. Equivalencia con operaciones estandar: esta funcion es
   equivalente a
31 // (int)(upper_bound(arr.begin(), arr.end(), valor) - arr.
   begin()) - 1,
32 // implementada manualmente para mostrar la mecanica
   interna.
33 //
34 // 3. Rango inicial con izq = -1: se usa -1 en vez de 0 como
   limite inferior semiabierto, de forma simetrica a como
35 // "menor x que cumple" usa der = size() como limite
   superior
36 // semiabierto; esto permite representar "no existe ningun
   indice que cumpla" sin casos especiales.
37 //
38 //
39 //
40 }

```

Ejemplo de funcionamiento:

```

1 vector<int> datos = {1, 3, 3, 5, 7, 9};
2 MayorQueCumple mqc(datos);
3
4 int idx1 = mqc.mayorQueCumple(5); // idx1 = 3 (arr[3] = 5,
   ultimo <= 5)
5 int idx2 = mqc.mayorQueCumple(4); // idx2 = 2 (arr[2] = 3,
   ultimo <= 4)
6 int idx3 = mqc.mayorQueCumple(0); // idx3 = -1 (ningun
   elemento <= 0)

```

Regla práctica: Usa este patrón cuando el predicado tenga la forma "verdadero...verdadero, falso...falso" y quieras el último punto donde se cumple; si el patrón está invertido (falso primero, verdadero después), usa "menor x que cumple". Nota que el cálculo del punto medio se redondea hacia arriba aquí y hacia abajo en la variante anterior — invertir esta convención por descuido produce bucles infinitos.

26.4 Binary Search on Answer

Idea: "Binary Search on Answer" no busca una posición dentro de un arreglo, sino que aplica la misma mecánica de reducción a la mitad sobre el *espacio de posibles respuestas* de un problema de optimización. La clave es reformular una pregunta de optimización ("¿cuál es el mínimo/máximo valor de x que resuelve el problema?") como una pregunta de *factibilidad* ("¿es posible lograr el objetivo con un valor x dado?"), siempre que esa factibilidad sea monótona en x : si x funciona, todo valor mayor

(o menor, según el problema) también funciona. Una vez identificada esa monotonía, se aplica exactamente el patrón de "menor/mayor x que cumple" ya visto, pero con $P(x)$ = "es factible resolver el problema con el parámetro x " en vez de una comparación directa contra un arreglo.

¿Por qué usarlo? Muchos problemas de optimización tienen una función objetivo que no se puede derivar ni evaluar analíticamente de forma directa, pero sí se puede verificar fácilmente si un valor candidato es factible (con una función auxiliar, a menudo greedy o de simulación). Esta técnica convierte un problema de optimización potencialmente difícil en $O(\log(\text{rango de respuestas}) \cdot C)$ llamadas a una función de factibilidad de costo C , en vez de probar todos los valores posibles de x uno por uno. Se distingue de las variantes anteriores de búsqueda binaria en que aquí $P(x)$ no compara contra un arreglo dado, sino que típicamente ejecuta un algoritmo completo (a veces $O(n)$ o más) para determinar factibilidad; por eso elegir bien el rango inicial de búsqueda y minimizar el costo de P es crítico para la eficiencia total.

Casos de uso:

- Minimizar el máximo (o maximizar el mínimo) en problemas de asignación o partición, como repartir paquetes entre camiones con capacidad mínima necesaria.
- Encontrar la velocidad mínima necesaria para completar una tarea antes de un plazo (problemas tipo "Koko Eating Bananas").
- Determinar la distancia mínima de separación máxima posible entre elementos colocados en posiciones dadas (problemas tipo "Aggressive Cows").
- Optimizar un parámetro continuo o discreto sujeto a una restricción de recursos, cuando la factibilidad se puede evaluar con un algoritmo greedy en $O(n)$.
- Problemas donde la respuesta exacta es difícil de calcular directamente pero es fácil verificar "¿el valor x es suficiente o demasiado?".

Complejidad: $O(\log(\text{rango}) \cdot C)$, donde rango es la diferencia entre el máximo y mínimo valor posible de la respuesta, y C es el costo de la función de factibilidad. El factor $\log(\text{rango})$ sale de aplicar el mismo argumento de reducción a la mitad que en las búsquedas binarias anteriores, pero ahora sobre el espacio de valores de la respuesta en vez de índices de un arreglo; como C suele ser $O(n)$ (una pasada greedy sobre los datos), la complejidad total típica es $O(n \log(\text{rango}))$.

```
1 struct BinarySearchOnAnswer {
2     vector<int> pesos; // ejemplo: pesos de paquetes a repartir
3     // en camiones
4     int numCamiones;
5
6     // CREATE: guarda los pesos de los paquetes (en orden, sin
7     // reordenar) y el numero de camiones disponibles
8     BinarySearchOnAnswer(vector<int> p, int k) : pesos(p),
9         numCamiones(k) {} // O(n)
10
11     // READ: verifica si es factible repartir todos los paquetes
12     // en
13     // "numCamiones" camiones, si cada camion tiene capacidad
14     // maxima
15     // "capacidad" (greedy: llenar cada camion hasta que un
16     // paquete
17     // no quepa, entonces pasar al siguiente camion)
18     bool esFactible(int capacidad) {
```

```
14     int camionesUsados = 1;
15     int cargaActual = 0;
16     for (int peso : pesos) {
17         if (peso > capacidad) return false; // un paquete no cabe
18         // en ningun camion
19         if (cargaActual + peso > capacidad) {
20             camionesUsados++;
21             cargaActual = 0;
22         }
23         cargaActual += peso;
24     }
25     return camionesUsados <= numCamiones;
26 } // O(n)
27
28 // READ: encuentra la menor capacidad de camion tal que sea
29 // factible repartir todos los paquetes en "numCamiones"
30 // camiones
31 int capacidadMinima() {
32     int izq = *max_element(pesos.begin(), pesos.end()); // no
33     // puede ser menor que el paquete mas pesado
34     int der = accumulate(pesos.begin(), pesos.end(), 0); //
35     // caso trivial: un solo camion con todo
36     while (izq < der) {
37         int medio = izq + (der - izq) / 2;
38         if (esFactible(medio)) der = medio; // candidato valido,
39         // buscar capacidad menor
40         else izq = medio + 1; // no factible,
41         // necesita mas capacidad
42     }
43     return izq;
44 } // O(n log(suma - max))
45
46 //
47 // -----
48 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
49 // -----
50
51 // 1. Patron general: esta estructura es una instancia
52 // concreta
53 // de "menor x que cumple" (visto anteriormente), donde
54 // P(x) = esFactible(x) en vez de una comparacion de
55 // arreglo;
56 // el mismo esqueleto de bucle [izq, der) semiabierto
57 // aplica
58 // sin cambios.
59 //
60 // 2. Problemas de maximizacion en vez de minimizacion: si el
61 // problema pide maximizar un parametro (por ejemplo, la
62 // velocidad minima para NO llegar tarde, buscando la
63 // MAYOR
64 // separacion posible entre vacas en un establo), se usa
65 // el
66 // esqueleto de "mayor x que cumple" en vez de este,
67 // invirtiendo
68 // la logica de esFactible segun corresponda.
69 //
70 // 3. Eleccion del rango inicial [izq, der]: es crucial
71 // acotar
72 // bien el rango de busqueda; un rango demasiado amplio no
73 // afecta la complejidad asintotica (sigue siendo log del
74 // rango) pero un rango incorrecto (que excluya la
75 // respuesta
76 // real) produce un resultado erroneo. Aqui izq = max(
77 // pesos)
78 // porque ningun camion puede tener capacidad menor al
79 // paquete
80 // mas pesado, y der = suma total porque un solo camion
81 // con
82 // esa capacidad siempre es factible.
83 //
84 // 4. Costo de esFactible domina la complejidad total: si la
85 // funcion de factibilidad fuera O(n log n) en vez de O(n)
86 // (por ejemplo, si requiriera ordenar en cada llamada),
87 // la
88 // complejidad total subiria a O(n log n log(rango)); por
89 // eso
90 // conviene precalcular fuera del bucle binario todo lo
91 // que
92 // no dependa del valor candidato x.
93 };
```

Ejemplo de funcionamiento:

```
1 vector<int> pesos = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
2 int camiones = 5;
3 BinarySearchOnAnswer bsoa(pesos, camiones);
4
5 int capMin = bsoa.capacidadMinima();
6 // Nota: el valor exacto de capMin depende de la simulacion
7 // greedy
8 // paso a paso; se debe correr el codigo para confirmarlo antes
9 // de
10 // darlo por verificado (candidato esperado en el rango de 15 a
```


18,
9 // pero verificar con la ejecucion real).

Regla práctica: Reconoce un problema de "Binary Search on Answer" cuando puedas responder rápidamente "¿es factible con este valor?" mediante un greedy o simulación, y esa factibilidad sea monótona en el parámetro buscado; en ese caso, aplica el esqueleto de "menor x que cumple" si minimizas, o "mayor x que cumple" si maximizas, usando la función de factibilidad como predicado $P(x)$ en vez de una comparación directa de arreglo.

26.5 Sobre reales

Idea: Cuando el espacio de búsqueda no es un conjunto discreto de enteros sino un intervalo continuo de números reales (por ejemplo, encontrar la raíz de una función monótona con precisión decimal), la búsqueda binaria no puede terminar cuando $izq == der$ porque entre dos reales cualesquiera siempre hay infinitos valores intermedios. En su lugar, se itera un número fijo de veces (o hasta que el intervalo $[izq, der]$ sea más pequeño que una tolerancia ε dada), y en cada iteración se sigue partiendo el intervalo a la mitad según el mismo criterio de monotonía usado en las variantes discretas.

¿Por qué usarlo? Es la adaptación directa de "menor/mayor x que cumple" al dominio continuo: la lógica de descartar la mitad del espacio según un predicado monótono $P(x)$ es idéntica, solo cambia la condición de parada, porque no existe un "siguiente entero" al cual converger. Se usa en vez de métodos como Newton-Raphson cuando no se puede (o no conviene) calcular la derivada de la función, o cuando se necesita una garantía simple de convergencia sin preocuparse por casos donde Newton-Raphson diverge (tangentes horizontales, puntos de inflexión mal condicionados); a cambio, la convergencia es más lenta (lineal en el número de dígitos de precisión, en vez de cuadrática).

Casos de uso:

- Encontrar la raíz cuadrada (o raíz n -ésima) de un número con precisión decimal arbitraria.
- Encontrar el punto donde una función monótona continua cruza un valor objetivo (por ejemplo, resolver $f(x) = 0$).
- Problemas de "Binary Search on Answer" donde la respuesta es un valor real (por ejemplo, minimizar un radio, una velocidad o un tiempo continuo) en vez de un entero.
- Calibrar un parámetro continuo de un modelo o simulación hasta alcanzar una precisión deseada.

Complejidad: $O(\log(\text{rango}/\varepsilon) \cdot C)$, ya que cada iteración reduce el tamaño del intervalo a la mitad, y se necesitan $\log_2(\text{rango}/\varepsilon)$ iteraciones para que el intervalo sea menor que la tolerancia ε deseada; C es el costo de evaluar el predicado o la función en cada paso. En la práctica, con un número fijo de iteraciones (por ejemplo, 100), se alcanza precisión cercana al límite de `double` sin necesidad de calcular explícitamente $\log(\text{rango}/\varepsilon)$.

```
1 struct BusquedaBinariaReal {
2     double izq, der;
3     int iteraciones;
4
5     // CREATE: define el intervalo inicial [izq, der] donde se
6     // garantiza que existe la respuesta, y el numero de
7     // iteraciones
8     // a realizar (100 es suficiente para precision de double)
9     BusquedaBinariaReal(double i, double d, int iters = 100)
10     : izq(i), der(d), iteraciones(iters) {} // O(1)
11
12     // READ: retorna el menor valor real x en [izq, der] tal que
13     // el
14     // predicado P(x) es verdadero, asumiendo que P tiene el
15     // patron
16     // falso...falso, verdadero...verdadero
17     template<typename Pred>
18     double menorQueCumple(Pred P) {
19         double lo = izq, hi = der;
20         for (int it = 0; it < iteraciones; it++) {
21             double medio = lo + (hi - lo) / 2.0;
22             if (P(medio)) hi = medio; // candidato valido, seguir a
23             // la izquierda
24             else lo = medio;          // no cumple, buscar a la
25             // derecha
26         }
27         return hi;
28     } // O(iteraciones * C)
29
30     // -----
31     // MODIFICACIONES Y COMENTARIOS CON LOS USOS
32     // -----
33
34     // 1. Numero de iteraciones vs tolerancia explicita: en vez
35     // de un
36     // numero fijo de iteraciones, se puede usar una condicion
37     // "while (hi - lo > eps)" con un eps dado (ej. 1e-9);
38     // ambas
39     // formas son equivalentes, pero el numero fijo de
40     // iteraciones
41     // evita bucles infinitos por errores de redondeo cuando
42     // eps
43     // es demasiado pequeno para la precision de double.
44     // -----
45     // 2. Mayor x que cumple (patron invertido, verdadero...falso)
46     // :
47     // misma estructura pero invirtiendo la asignacion:
48     // if (P(medio)) lo = medio; else hi = medio;
49     // y se retorna lo en vez de hi.
50     // -----
51     // 3. Ejemplo de uso para raiz cuadrada: para calcular sqrt(n)
52     // ,
53     // se define P(x) := (x * x >= n) sobre el intervalo [0, n
54     // +1],
55     // y menorQueCumple(P) retorna la raiz cuadrada aproximada.
56     // -----
57     // 4. Precision de punto flotante: 100 iteraciones sobre un
58     // intervalo de tamaño razonable (ej. [0, 1e9]) ya reducen
59     // el
60     // intervalo muy por debajo de la precision representable
61     // en
62     // double (~15-17 dígitos decimales), por lo que iterar
63     // mas de
64     // ~100 veces no mejora la precision real, solo
65     // desperdicia
66     // tiempo de ejecucion.
67 };
```

Ejemplo de funcionamiento:

```
1 // Calcular sqrt(2) con busqueda binaria sobre reales
2 BusquedaBinariaReal bbr(0.0, 2.0);
3 double raiz = bbr.menorQueCumple([&](double x) {
4     return x * x >= 2.0;
5 });
6 // raiz es aproximadamente 1.41421356... (sqrt(2))
```

Regla práctica: Usa esta variante cuando el espacio de búsqueda sea continuo (reales) en vez de discreto (enteros), y reemplaza la condición de parada $izq < der$ por un número fijo de iteraciones o una tolerancia ε explícita; si el espacio de búsqueda es discreto, siempre preferir las variantes enteras vistas antes, ya que convergen en tiempo finito exacto sin depender de una tolerancia arbitraria.

26.6 Con strings

Idea: La búsqueda binaria también puede aplicarse cuando el espacio de búsqueda o el criterio de comparación involucra strings en vez de números, siempre que exista un orden total monótono adecuado (por ejemplo, el orden lexicográfico) o una propiedad monótona definida sobre los strings (por ejemplo, "la longitud del prefijo común es al menos k "). La mecánica de partir el espacio a la mitad es idéntica a las variantes numéricas; lo que cambia es cómo se define y evalúa el predicado P sobre strings, que normalmente requiere una comparación lexicográfica o un cálculo de prefijo/sufijo en vez de una comparación aritmética directa.

¿Por qué usarlo? Muchos problemas piden buscar sobre un arreglo de strings ordenado lexicográficamente (por ejemplo, encontrar el primer string que empiece con cierto prefijo, un problema análogo a "menor x que cumple" pero con `std::string` en vez de `int`), o piden encontrar la longitud óptima de un substring/prefijo común que cumpla alguna propiedad monótona (más cercano a "Binary Search on Answer", donde x es una longitud entera pero $P(x)$ evalúa algo sobre substrings de esa longitud, como en la búsqueda del prefijo común más largo entre dos strings mediante hashing).

Casos de uso:

- Encontrar el rango de strings en un diccionario ordenado que comparten un prefijo dado (equivalente a `lower_bound/upper_bound` con comparación lexicográfica).
- Encontrar la longitud del prefijo común más largo entre dos strings usando *binary search + hashing*: para una longitud candidata L , verificar en $O(1)$ (con hashing precomputado) si los primeros L caracteres coinciden.
- Buscar el substring repetido más largo en un string, combinando binary search sobre la longitud con un *Suffix Array* o hashing para verificar factibilidad.
- Ordenar un diccionario de palabras y realizar búsquedas de existencia o de prefijo de forma repetida sobre él.

Complejidad: $O(\log n \cdot C)$ para búsqueda directa sobre un arreglo ordenado de m strings, donde $C = O(L)$ es el costo de comparar dos strings de longitud L (peor caso, cuando comparten un prefijo largo), dando $O(L \log n)$ en total. En el caso de "longitud del prefijo común más largo con hashing", la complejidad es $O(\log L \cdot C)$, donde L es la longitud máxima posible del prefijo y $C = O(1)$ es el costo de comparar hashes de un substring ya precomputados, dando $O(\log L)$ en total tras el preprocesamiento de hashes.

```

1 struct BusquedaBinariaStrings {
2     vector<string> diccionario; // ordenado lexicograficamente
3     vector<unsigned long long> hashA, hashB; // prefijos de hash
      de dos strings
4     vector<unsigned long long> potencias; // potencias de la
      base para el hashing
5     unsigned long long base = 131, mod = 1e9 + 7;
6
7     // CREATE: guarda el diccionario ordenado (para búsqueda por
8     // prefijo) y precalcula los hashes de prefijo de dos strings
9     // (para longitud de prefijo comun mas largo)

```

```

10 BusquedaBinariaStrings(vector<string> dic, string a, string b
11 ) : diccionario(dic) {
12     int n = max(a.size(), b.size());
13     potencias.assign(n + 1, 1);
14     for (int i = 1; i <= n; i++) potencias[i] = (potencias[i-1]
15         * base) % mod;
16     hashA = calcularHashes(a);
17     hashB = calcularHashes(b);
18 } // O(m log m) por el diccionario ya ordenado + O(n) por el
      hashing
19
20 // READ: calcula el arreglo de hashes de prefijo de un string
21 // s,
22 // donde hashes[i] es el hash de s[0..i-1]
23 vector<unsigned long long> calcularHashes(const string& s) {
24     vector<unsigned long long> h(s.size() + 1, 0);
25     for (int i = 0; i < (int)s.size(); i++)
26         h[i+1] = (h[i] * base + s[i]) % mod;
27     return h;
28 } // O(|s|)
29
30 // READ: retorna el hash del substring s[0..len-1] usando el
31 // arreglo de hashes de prefijo precalculado h
32 unsigned long long hashPrefijo(const vector<unsigned long
33     long>& h, int len) {
34     return h[len];
35 } // O(1)
36
37 // READ: retorna el menor indice en "diccionario" tal que la
38 // palabra en esa posicion es >= "prefijo" lexicograficamente
39 // (equivalente a lower_bound sobre strings)
40 int menorQueCumplePrefijo(const string& prefijo) {
41     int izq = 0, der = (int)diccionario.size();
42     while (izq < der) {
43         int medio = izq + (der - izq) / 2;
44         if (diccionario[medio] >= prefijo) der = medio;
45         else izq = medio + 1;
46     }
47     return izq;
48 } // O(L log m), L = longitud promedio de comparacion, m =
      tamaño del diccionario
49
50 // READ: retorna la longitud del prefijo comun mas largo
51 // entre
52 // los dos strings usados al construir el struct, usando
53 // binary
54 // search sobre la longitud y comparacion de hashes en O(1)
55 int prefijoComunMasLargo() {
56     int izq = 0, der = (int)min(hashA.size(), hashB.size()) -
57     1;
58     while (izq < der) {
59         int medio = izq + (der - izq + 1) / 2;
60         if (hashPrefijo(hashA, medio) == hashPrefijo(hashB, medio))
61             izq = medio;
62         else der = medio - 1;
63     }
64     return izq;
65 } // O(log L)
66
67 //
68
69 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
70 //
71
72 // 1. Riesgo de colisiones de hash: usar un solo modulo (como
73 // aquí, con fines ilustrativos) tiene riesgo de
74 // colisiones
75 // ante datos adversarios; en competencias es preferible
76 // usar
77 // doble hashing (dos bases y modulos distintos) o hashing
78 // aleatorizado para reducir ese riesgo.
79 //
80 // 2. Comparacion de strings sin hashing: si no se quiere
81 // lidiar
82 // con hashing, "prefijoComunMasLargo" tambien puede
83 // resolverse
84 // directamente comparando caracter a caracter en O(L)
85 // total,
86 // sin búsqueda binaria, ya que de todas formas hay que
87 // leer
88 // los caracteres; el hashing solo aporta ventaja cuando
89 // se
90 // necesita comparar substrings arbitrarios repetidamente,
91 // no
92 // solo un prefijo desde el inicio.
93 //
94 // 3. Búsqueda binaria sobre diccionario no ordenado: si
95 // "diccionario" no esta ordenado lexicograficamente, hay
96 // que
97 // ordenarlo primero con sort(diccionario.begin(),
98 // diccionario.end())
99 // antes de poder usar "menorQueCumplePrefijo"
100 // correctamente.

```

```
80 };
```

Ejemplo de funcionamiento:

```
1 vector<string> dic = {"casa", "carro", "perro", "pelota", "zorro"};
2 sort(dic.begin(), dic.end()); // {"carro", "casa", "pelota", "perro", "zorro"}
3
4 BusquedaBinariaStrings bbs(dic, "internacionalización", "internacionalmente");
5
6 int idx = bbs.menorQueCumplePrefijo("pe");
7 // idx = 2 ("pelota" es la primera palabra >= "pe" lexicográficamente)
8
9 int lcp = bbs.prefijoComunMasLargo();
10 // Nota: el valor exacto de lcp depende de la comparacion caracter a
11 // caracter entre "internacionalización" e "internacionalmente"; se
12 // debe correr el código para confirmar el valor exacto (candidato
13 // esperado alrededor de 14-15, ya que ambos comparten el prefijo
14 // "internacional").
```

Regla práctica: Usa búsqueda binaria sobre strings cuando el arreglo esté ordenado lexicográficamente y necesites encontrar un rango de coincidencia por prefijo (aplica directamente "menor/mayor x que cumple" con **string** en vez de **int**); usa binary search + hashing sobre la *longitud* cuando necesites encontrar el tamaño óptimo de un prefijo, sufijo o substring que cumple una propiedad de igualdad o coincidencia, ya que ahí el predicado monótono no es sobre el string mismo sino sobre una longitud entera.

Parte V Geometría Computacional

26.7 ¿Qué algoritmo usar?

Cuando el ejercicio diga / pida	Usa	Pista rápida
“¿Giran a la izquierda o a la derecha?”, “¿de qué lado de la recta está el punto?”, “¿son colineales?”	Orientación (producto cruz)	Signo de $(b - o) \times (c - o)$. $O(1)$.
“¿Qué tan alineados están?”, “¿el ángulo es agudo, recto u obtuso?”, proyección de un vector	Producto punto	Signo del producto punto. $O(1)$.
“Ángulo entre dos vectores”, “cuánto hay que rotar para alinearlos”	<code>atan2(cruz, punto)</code>	Mejor que <code>acos</code> : da signo y es estable.
“Distancia entre dos puntos”, “¿cuál está más cerca?”, “¿está dentro del círculo de radio r ?”	Distancia euclidiana	Compara distancias al cuadrado, sin <code>sqrt</code> .
“Centro de un segmento”, “mediatriz”	Punto medio	$((x_1 + x_2)/2, (y_1 + y_2)/2)$.
“Punto que divide el segmento en razón $m : n$ ”, “dividir en k partes iguales”	Interpolación $t = m/(m + n)$	$A + t(B - A)$.
“Ecuación de la recta por dos puntos”, “intersección de dos rectas”	Forma general $ax + by + c = 0$	No falla con rectas verticales.
“¿Tres puntos están alineados?”, “¿el punto está sobre el segmento?”	$Cruz = 0$ + caja delimitadora	$Cruz = 0$ solo prueba la recta, no el segmento.
“Distancia de un punto a una recta / pared / segmento”	$Cruz$ sobre base / extremos	Si la proyección cae fuera, usa el extremo.
“Camino más corto que toca una pared”, “simetría respecto a un eje”, “rebote de un rayo”	Reflexión de un punto	Proyecta y calcula $2Q - P$.
“Gira la figura un ángulo θ alrededor de un punto”, vértices de un polígono regular	Rotación de un punto	Traslada, rota, destraslada.
“¿Dos segmentos se cruzan?” (un solo par)	Orientación + caja delimitadora	Cuatro productos cruz, casos colineales aparte.

Table 15: Primitivas geométricas: qué usar según lo que pide el ejercicio.

Cuando el ejercicio diga / pide	Usa	Pista rápida
“Ángulos de un triángulo” con coordenadas	atan2 sobre dos lados	Ángulo entre $\vec{AB}$ y $\vec{AC}$.
“Ángulos de un triángulo” dados solo los lados	Ley de cosenos	Recorta el argumento de $\cos$ a $[-1, 1]$.
“Área de un triángulo dados sus tres lados”	Fórmula de Herón	$\sqrt{s(s-a)(s-b)(s-c)}$.
“Radio del círculo inscrito”	Herón + semiperímetro	$r = \text{área}/s$.
“Círculo que pasa por tres puntos”, “circuncentro”, “¿cuatro puntos son concíclicos?”	Círculo circunscrito	Verifica que no sean colineales.
“¿El punto está dentro del triángulo?”	Tres orientaciones	Mismo signo en las tres aristas. $O(1)$.
“Longitud de un arco”, “área de una porción de pizza”	Sector circular	$L = r\theta$, $A = \frac{1}{2}r^2\theta$.
“Área entre un arco y su cuerda”	Segmento circular	Sector menos triángulo.
“¿Dos círculos se tocan?”, “puntos de corte”, “área de solape”	Intersección de dos círculos	Clasifica los casos antes de calcular. $O(1)$.
“Área de un polígono” dado por vértices (convexo o no)	Shoelace	Signo = orientación (CCW positivo). $O(n)$.
“Longitud de la cerca / contorno”	Perímetro (suma de aristas)	Polilínea abierta: omite la arista de cierre.
“¿El punto está dentro del polígono?” (cualquier forma)	Ray Casting	Cruces impares = dentro. $O(n)$.
“¿Dentro del polígono?” con muchas consultas y polígono convexo	Búsqueda binaria angular	Requiere orden CCW. $O(\log n)$ por consulta.
“¿Cuántos puntos enteros hay dentro / en el borde”, vértices enteros	Teorema de Pick	$A = I + B/2 - 1$, $B = \sum \gcd$ por arista.

Table 16: Triángulos, círculos y polígonos: qué usar según lo que pide el ejercicio.

Cuando el ejercicio diga / pide	Usa	Pista rápida
“Cerca mínima que encierra todos los puntos”, “envolvente convexa”	Monotone Chain	Ordena y usa pila. $O(n \log n)$.
Envolvente convexa cuando se sabe que tendrá pocos vértices	Jarvis (Gift Wrapping)	$O(nh)$. Peor caso $O(n^2)$.
“Quédate con la parte del polígono convexo a un lado de una recta”, “recorta contra una ventana”	Corte de polígono convexo	Sutherland-Hodgman. $O(n)$ por línea.
“Máximo de intervalos solapados a la vez”, “máximo de reuniones simultáneas”	Line Sweep con eventos	Ordena $2n$ eventos. Define el desempate.
“¿Algún par de segmentos se cruza?”, “¿el polígono es simple?”	Barrido de segmentos con set	Compara solo vecinos. $O(n \log n)$.
“Área total cubierta por rectángulos solapados”	Barrido + Segment Tree de cobertura	Comprime coordenadas y . $O(n \log n)$.
“Silueta de edificios”, “altura máxima visible en cada x ”	Skyline (Line Sweep + multiset)	Agrupar eventos con igual x . $O(n \log n)$.
“Par de puntos más cercano”, “distancia mínima entre n puntos”	Closest Pair (barrido)	Ventana en x y rango en y . $O(n \log n)$.

Table 17: Envlovente convexa y barrido: qué usar según lo que pide el ejercicio.

Cuando el ejercicio diga / pide	Usa	Pista rápida
“Perímetro de una figura hecha de bloques en una grilla”	Conteo de vecinos	$4B - 2A$. $O(NM)$.
“¿Cuántos cuadrados hay en un tablero $N \times M$?”	Suma $\sum_{k=1}^{N-1} (N-k+1)(M-k+1)$	$O(\min(N, M))$. Si $\frac{N}{N+1} = \frac{M}{2N+1}$.
“¿Cuántos rectángulos hay en una grilla $N \times M$?”	$\binom{N+1}{2} \binom{M+1}{2}$	Elige 2 líneas verticales y 2 horizontales. $O(1)$.
“Suma de distancias Manhattan entre todos los pares”	Separar x e y , ordenar, prefijos	Solo vale para Manhattan. $O(n \log n)$.
“Máximo de baldosas cuadradas que caben en un triángulo rectángulo isósceles”	Reducción a $L = \lfloor b/k \rfloor$	$L(L-1)/2$. $O(1)$.
“Máximo de pedazos con n cortes rectos” (pizza, pastel)	Pastelero perezoso	$1 + n(n+1)/2$. $O(1)$.
“Altura máxima de un triángulo de monedas”, “cuántas filas completas caben”	Búsqueda binaria sobre h	$h(h+1)/2 \leq n$. $O(\log n)$.

Table 18: Ad-hoc y combinatoria geométrica: qué usar según lo que pide el ejercicio.

27 Line Sweep

27.1 Idea General y Manejo de Eventos

Idea: un *sweep line* (barrido) resuelve problemas geométricos imaginando una línea recta (normalmente vertical, avanzando en x) que se desplaza por el plano de izquierda a derecha. En vez de razonar sobre el plano completo, la línea reduce el problema a una secuencia de **eventos discretos** —puntos donde algo cambia: empieza un intervalo, termina un segmento, aparece un punto— ordenados por la coordenada de barrido. Entre un evento y el siguiente, el "estado" del problema (qué objetos toca la línea, en qué orden) no cambia, así que basta con mantener ese estado en una estructura auxiliar (el *status*) y actualizarla incrementalmente en cada evento. Esto convierte un problema 2D en una secuencia de operaciones 1D sobre esa estructura.

¿Por qué usarlo? Cuando un problema involucra n objetos y una propiedad global entre ellos (intersecciones, solapamientos, cercanía, área cubierta), la fuerza bruta suele ser $O(n^2)$ al comparar todos los pares. El barrido explota el hecho de que solo los objetos *actualmente cerca* de la línea pueden interactuar entre sí en ese instante, así que basta con mantener y consultar esa vecindad, no todos los pares. Esto es distinto de una estructura estática (como un *prefix sum* o un *sparse table*): el barrido mantiene una estructura **dinámica** que se inserta/elimina a medida que se consumen eventos, generalmente respaldada por un `std::set`, un árbol de segmentos o un *Fenwick tree*.

Casos de uso:

- Detectar si algún par de segmentos se intersecta.
- Calcular el área cubierta por la unión de rectángulos.
- Resolver el *Skyline Problem*.
- Encontrar el par de puntos más cercano en el plano.
- Contar el máximo número de intervalos activos simultáneamente.

Complejidad: en general $O(n \log n)$: ordenar los $2n$ eventos cuesta $O(n \log n)$, y cada evento realiza una operación de $O(\log n)$ sobre la estructura de estado (inserción, borrado o consulta en un `set` o árbol balanceado). El total es $O(n \log n)$, dominado por el ordenamiento inicial cuando la estructura de estado es logarítmica por operación.

```

1 // CREATE/READ/WRITE: Barrido generico basado en eventos
  ordenados
2 // Ejemplo de aplicacion: maximo numero de intervalos solapados
  simultaneamente
3 struct BarridoEventos {
4     // Evento: posicion x y tipo (+1 = inicio de intervalo, -1 =
      fin de intervalo)
5     struct Evento {
6         long long x;
7         int tipo; // +1 = abre intervalo, -1 = cierra intervalo
8     };
9
10    vector<Evento> eventos;
11
12    // CREATE: reserva espacio para 2*n eventos (inicio y fin de
      cada intervalo)
13    BarridoEventos(int n) {
14        eventos.reserve(2 * n);
15    } // O(1)
16
17    // WRITE: agrega los dos eventos (inicio, fin) generados por
      un intervalo [l, r]
18    void agregarIntervalo(long long l, long long r) {
19        eventos.push_back({l, +1});
20        eventos.push_back({r, -1});
21    } // O(1) amortizado
22
23    // READ: ordena los eventos y barre la recta devolviendo el
      maximo de
24    // intervalos activos simultaneamente
25    int maximoSolapamiento() {
26        sort(eventos.begin(), eventos.end(), [](const Evento& a,
      const Evento& b) {
27            if (a.x != b.x) return a.x < b.x;
28            // Si un intervalo es [l, r] (cerrado-abierto), los
      cierres (-1)
29            // deben procesarse ANTES que las aperturas (+1) en el
      mismo punto
30            return a.tipo < b.tipo;
31        });
32
33        int activos = 0, maximo = 0;
34        for (auto& e : eventos) {
35            activos += e.tipo;
36            maximo = max(maximo, activos);
37        }
38        return maximo;
39    } // O(n log n), dominado por el sort de 2n eventos
40
41    //
42    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
43    //
44    // 1. Intervalos cerrados [l, r] (incluyen ambos extremos):
      si dos
45    // intervalos comparten solo un punto de borde y ESO
      cuenta como
46    // solapamiento, invertir el criterio de desempate (tipo >
      0 antes
47    // que tipo < 0) para que las aperturas se procesen antes.
48    // 2. Barrido en 2D (rectangulos): en vez de activos += tipo,
      se usa
49    // un arbol de segmentos o BIT indexado por la coordenada
      y para
50    // saber cuanta longitud vertical esta cubierta en cada
      momento
51    // (ver Union de Areas de Rectangulos y Skyline Problem).
52    // 3. Conteo de eventos por tipo especifico: si se necesitan
      mas de
53    // dos tipos de evento (por ejemplo, consultas
      intercaladas con
54    // actualizaciones), agregar un campo extra a Evento y
      desempatar
55    // por prioridad de tipo, no solo por signo.
56 };

```

Ejemplo de funcionamiento:

```

1 BarridoEventos b(3);
2 b.agregarIntervalo(1, 5);
3 b.agregarIntervalo(2, 6);

```

```

4 b.agregarIntervalo(4, 8);
5
6 int resultado = b.maximoSolapamiento();
7 // resultado = 3
8 // (en x = 4, los tres intervalos [1,5], [2,6] y [4,8] estan
   activos)

```

Nota: el orden de desempate cuando dos eventos comparten la misma coordenada es la fuente más común de errores en un barrido. No hay una regla universal—depende de si los intervalos del problema son abiertos, cerrados, o si "tocarse en el borde" cuenta como interacción— así que siempre hay que decidirlo explícitamente en el comparador, como se ilustra en el código.

Regla práctica: usa un barrido genérico cuando el problema se reduzca a procesar eventos ordenados manteniendo un estado que se actualiza incrementalmente. Si solo necesitas respuestas estáticas sobre datos que no cambian con la posición (consultas offline sin relación de orden entre ellas), probablemente te alcance con estructuras estáticas más simples en vez de simular el barrido completo.

27.2 Intersección de Segmentos

Idea: es la aplicación más directa del barrido general—visto en *Idea General y Manejo de Eventos*— al problema de detectar si n segmentos tienen alguna intersección entre sí. El *status* de la línea es el conjunto de segmentos que la línea de barrido cruza actualmente, ordenados por su altura (y) en la posición x actual del barrido. La observación clave es que dos segmentos solo pueden empezar a cruzarse cuando se vuelven **vecinos** en ese orden vertical; por lo tanto, basta con comprobar intersección contra los vecinos inmediatos cada vez que un segmento entra o sale del *status*, sin comparar contra todos los demás.

¿Por qué usarlo? Comparar todos los pares de segmentos cuesta $O(n^2)$. El barrido lo reduce a $O(n \log n)$ porque, en cualquier instante, solo hace falta conocer el orden vertical local de los segmentos activos—mantenido en un `std::set`— en vez de su relación con todos los demás. A diferencia del barrido genérico de la subsección anterior (que solo cuenta activos), aquí el *status* necesita un **orden relativo dinámico**: la posición de un segmento en el `set` cambia a medida que x avanza, porque la altura de cada segmento en x depende de su pendiente.

Casos de uso:

- Verificar si un conjunto de calles o límites de un mapa se cruzan indebidamente.
- Validar que un polígono es simple (sus aristas no se autointersectan).
- Preprocesamiento antes de planarizar un grafo geométrico (arreglo de segmentos).
- Detección de colisiones entre trayectorias rectas.

Complejidad: $O(n \log n)$. Hay $2n$ eventos (inicio y fin de cada segmento), ordenarlos cuesta $O(n \log n)$. Cada evento realiza a lo sumo una inserción, un borrado o una consulta de vecinos en el `std::set`, cada una $O(\log n)$, y cada verificación de intersección entre dos segmentos es $O(1)$. En total: $O(n \log n)$.

```

1 // CREATE/READ/WRITE: Barrido para detectar si existe al menos
  una
2 // interseccion entre n segmentos (no cuenta todas las
  intersecciones)
3 struct BarridoInterseccionSegmentos {
4     struct Punto { double x, y; };
5     struct Segmento {
6         Punto a, b; // a.x <= b.x siempre (normalizado al agregar)
7         int id;
8     };
9
10    // READ: producto cruz (o - p) x (o - q), usado para
      orientacion
11    static double cruz(const Punto& o, const Punto& p, const
      Punto& q) {
12        return (p.x - o.x) * (q.y - o.y) - (p.y - o.y) * (q.x - o.x)
13    }; // O(1)
14
15    // READ: determina si los segmentos s1 y s2 se cruzan (
      incluye casos
16    // colineales donde un extremo cae exactamente sobre el otro
      segmento)
17    static bool seCruzan(const Segmento& s1, const Segmento& s2)
18    {
19        double d1 = cruz(s1.a, s1.b, s2.a);
20        double d2 = cruz(s1.a, s1.b, s2.b);
21        double d3 = cruz(s2.a, s2.b, s1.a);
22        double d4 = cruz(s2.a, s2.b, s1.b);
23        if ((d1 > 0 && d2 < 0) || (d1 < 0 && d2 > 0)) &&
24            ((d3 > 0 && d4 < 0) || (d3 < 0 && d4 > 0))
25            return true;
26        auto sobreSegmento = [](const Segmento& s, const Punto& p)
27        {
28            return min(s.a.x, s.b.x) <= p.x && p.x <= max(s.a.x, s.b.
29            x) &&
30            min(s.a.y, s.b.y) <= p.y && p.y <= max(s.a.y, s.b.y);
31        };
32        if (d1 == 0 && sobreSegmento(s1, s2.a)) return true;
33        if (d2 == 0 && sobreSegmento(s1, s2.b)) return true;
34        if (d3 == 0 && sobreSegmento(s2, s1.a)) return true;
35        if (d4 == 0 && sobreSegmento(s2, s1.b)) return true;
36        return false;
37    }; // O(1)
38
39    // READ: interpola la altura y del segmento s en la
      coordenada x
40    static double alturaEnX(const Segmento& s, double x) {
41        if (s.a.x == s.b.x) return min(s.a.y, s.b.y);
42        double t = (x - s.a.x) / (s.b.x - s.a.x);
43        return s.a.y + t * (s.b.y - s.a.y);
44    }; // O(1)
45
46    double sweepX; // coordenada x actual del barrido, usada por
      el comparador
47
48    // Comparador del status: ordena segmentos activos por su
      altura en sweepX
49    struct Comparador {
50        BarridoInterseccionSegmentos* padre;
51        bool operator()(const Segmento& s1, const Segmento& s2)
52        const {
53            double y1 = alturaEnX(s1, padre->sweepX);
54            double y2 = alturaEnX(s2, padre->sweepX);
55            if (fabs(y1 - y2) > 1e-9) return y1 < y2;
56            return s1.id < s2.id;
57        }
58    };
59
60    vector<Segmento> segmentos;
61
62    // CREATE: reserva espacio para n segmentos
63    BarridoInterseccionSegmentos(int n) {
64        segmentos.reserve(n);
65        sweepX = 0;
66    }; // O(1)
67
68    // WRITE: agrega un segmento (normaliza para que a.x <= b.x)
69    void agregarSegmento(Punto p, Punto q) {
70        if (q.x < p.x) swap(p, q);
71        segmentos.push_back({p, q, (int)segmentos.size()});
72    }; // O(1) amortizado
73
74    // READ: barre la recta de izquierda a derecha y responde si
      existe
75    // al menos un par de segmentos que se intersectan
76    bool existeInterseccion() {
77        enum TipoEvento { FIN = 0, INICIO = 1 }; // FIN antes que
      INICIO en empate
78        struct Evento { double x; int tipo; int idSeg; };
79        vector<Evento> eventos;
80        eventos.reserve(2 * segmentos.size());
81        for (auto& s : segmentos) {

```

```

78        eventos.push_back({s.a.x, INICIO, s.id});
79        eventos.push_back({s.b.x, FIN, s.id});
80    }
81    sort(eventos.begin(), eventos.end(), [](const Evento& e1,
      const Evento& e2) {
82        if (e1.x != e2.x) return e1.x < e2.x;
83        return e1.tipo < e2.tipo;
84    });
85
86    Comparador comp{this};
87    set<Segmento, Comparador> status(comp);
88    vector<set<Segmento, Comparador>::iterator> ubicacion(
      segmentos.size());
89
90    for (auto& e : eventos) {
91        sweepX = e.x;
92        Segmento& s = segmentos[e.idSeg];
93        if (e.tipo == INICIO) {
94            auto it = status.insert(s).first;
95            ubicacion[e.idSeg] = it;
96            auto siguiente = next(it);
97            if (siguiente != status.end() && seCruzan(s, *siguiente
98            )) return true;
99            if (it != status.begin()) {
100                auto anterior = prev(it);
101                if (seCruzan(s, *anterior)) return true;
102            }
103        } else {
104            auto it = ubicacion[e.idSeg];
105            auto siguiente = next(it);
106            bool hayAmbos = (it != status.begin() && siguiente !=
107            status.end());
108            if (hayAmbos) {
109                auto anterior = prev(it);
110                if (seCruzan(*anterior, *siguiente)) return true;
111            }
112            status.erase(it);
113        }
114        return false;
115    }; // O(n log n): n eventos, cada operacion sobre el set es O(
      log n)
116
117    //
118    // -----
119    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
120    // -----
121
122    // 1. Contar TODAS las intersecciones (no solo detectar si
      existe una):
123    // requiere el algoritmo completo de Bentley-Ottmann,
124    // donde cada
125    // punto de interseccion encontrado se inserta como un
126    // NUEVO evento
127    // en la cola, y se intercambian las posiciones de los dos
128    // segmentos
129    // en el status en ese punto. Complejidad O((n + k) log n)
130    // con
131    // k = numero de intersecciones.
132    // 2. Segmentos verticales (s.a.x == s.b.x): el comparador ya
      los
133    // maneja tomando el y minimo en alturaEnX, pero si dos
134    // segmentos
135    // verticales coinciden en x se debe agregar un criterio
136    // de
137    // desempate adicional (por ejemplo el y minimo de cada
138    // uno) para
139    // evitar que el set los considere iguales.
140    // 3. Precision numerica: al usar coordenadas double, se
      compara con
141    // un epsilon (aqui 1e-9) en vez de igualdad exacta; si
142    // las
143    // coordenadas de entrada son enteras, se puede reescribir
144    // todo
145    // con aritmetica de enteros/racionales para evitar
146    // errores.
147    };

```

Ejemplo de funcionamiento:

```

1 BarridoInterseccionSegmentos b(3);
2 b.agregarSegmento({0, 0}, {4, 4});
3 b.agregarSegmento({0, 4}, {4, 0});
4 b.agregarSegmento({5, 5}, {6, 6});
5
6 bool resultado = b.existeInterseccion();
7 // resultado = true
8 // (el primer y segundo segmento forman una X y se cruzan en
9 // (2,2);
10 // el tercero esta aislado y no interfiere)

```

Nota: este struct responde únicamente *existe/no existe* una intersección, en $O(n \log n)$. Si el problema pide

reportar todos los pares que se cruzan (o los puntos exactos de cruce), hace falta el algoritmo completo de Bentley-Ottmann, con una cola de eventos que crece dinámicamente por cada intersección encontrada —no cubierto en este struct, ver la modificación 1 dentro del código.

Regla práctica: usa este barrido cuando la pregunta sea binaria (¿algún par se cruza?) sobre n segmentos; para eso es $O(n \log n)$ contra el $O(n^2)$ de comparar todos los pares directamente. Si necesitas el conteo exacto de intersecciones o sus coordenadas, este struct no basta —requieres la versión completa de Bentley-Ottmann mencionada en la Nota.

27.3 Unión de Áreas de Rectángulos

Idea: extiende el barrido visto en *Idea General y Manejo de Eventos* a un problema donde el estado no es un simple contador de activos, sino la **longitud cubierta** de un segmento vertical en un instante dado. Cada rectángulo genera dos eventos: uno al entrar (en x_1) y otro al salir (en x_2), cada uno con un rango vertical $[y_1, y_2]$. Entre dos eventos consecutivos en x , la longitud vertical cubierta no cambia, así que el área total es la suma de (ancho del intervalo en x) $\times$ (longitud cubierta en y) para cada tramo. Para mantener esa longitud cubierta de forma dinámica —que aumenta o disminuye según entran o salen rectángulos— se usa un árbol de segmentos sobre las coordenadas y comprimidas, donde cada nodo guarda cuántas veces está cubierto su rango y la longitud efectivamente cubierta.

¿Por qué usarlo? A diferencia del barrido genérico (que solo cuenta cuántos intervalos están activos), aquí el estado es una cantidad continua —longitud— que depende de *qué tan solapados* están los rangos verticales activos, no solo de cuántos hay. Esto obliga a una estructura más rica que un simple contador o un **set**: un árbol de segmentos por conteo con recomputación de longitud cubierta en cada nodo. Es la técnica estándar frente a la fuerza bruta de descomponer todo en una grilla, que sería $O(n^2)$ o peor si las coordenadas son grandes.

Casos de uso:

- Área total cubierta por un conjunto de rectángulos solapados.
- Preprocesamiento para el *Skyline Problem* (comparten la estructura de eventos).
- Cálculo de cobertura en problemas de "manchas de pintura" o "zonas de señal" representadas como rectángulos.
- Base para calcular el perímetro de la unión (variante con estructura de datos adicional).

Complejidad: $O(n \log n)$. Se generan $2n$ eventos y $2n$ coordenadas y a comprimir, lo que cuesta $O(n \log n)$ al ordenar. El árbol de segmentos tiene $O(n)$ hojas (una por cada intervalo entre coordenadas y comprimidas consecutivas), así que cada actualización cuesta $O(\log n)$, y se realizan $2n$ actualizaciones —una por evento—, dando $O(n \log n)$ en total.

1 // CREATE/READ/WRITE: Área de la unión de n rectángulos mediante barrido

```

2 // vertical + árbol de segmentos por conteo sobre coordenadas y
  // comprimidas
3 struct AreaUnionRectangulos {
4     struct Rectangulo { double x1, y1, x2, y2; };
5     struct Evento { double x; double y1, y2; int delta; }; //
      // delta = +1 abre, -1 cierra
6
7     vector<Rectangulo> rectangulos;
8     vector<double> ys; // coordenadas y comprimidas (
      // ordenadas, únicas)
9     vector<int> conteo; // conteo[nodo]: veces que el rango
      // esta cubierto
10    vector<double> cubierta; // cubierta[nodo]: longitud
      // cubierta en ese rango
11
12    // CREATE: reserva espacio para n rectangulos
13    AreaUnionRectangulos(int n) {
14        rectangulos.reserve(n);
15    } // O(1)
16
17    // WRITE: agrega un rectangulo definido por sus esquinas
      // opuestas
18    void agregarRectangulo(double x1, double y1, double x2,
19        double y2) {
20        rectangulos.push_back({x1, y1, x2, y2});
21    } // O(1) amortizado
22
23    // WRITE: actualiza el rango [ql, qr] del árbol sumando delta
      // al conteo,
24    // y recalcula la longitud cubierta de cada nodo afectado
25    void actualizar(int nodo, int l, int r, int ql, int qr, int
26        delta) {
27        if (qr <= l || r <= ql) return;
28        if (ql <= l && r <= qr) {
29            conteo[nodo] += delta;
30        } else {
31            int medio = (l + r) / 2;
32            actualizar(2*nodo, l, medio, ql, qr, delta);
33            actualizar(2*nodo+1, medio, r, ql, qr, delta);
34        }
35        if (conteo[nodo] > 0) {
36            cubierta[nodo] = ys[r] - ys[l];
37        } else if (r - l == 1) {
38            cubierta[nodo] = 0;
39        } else {
40            cubierta[nodo] = cubierta[2*nodo] + cubierta[2*nodo+1];
41        }
42    } // O(log m), m = numero de intervalos y comprimidos
43
44    // READ: calcula el área total cubierta por la unión de todos
      // los rectangulos
45    double calcularArea() {
46        vector<Evento> eventos;
47        vector<double> yTemp;
48        for (auto& r : rectangulos) {
49            eventos.push_back({r.x1, r.y1, r.y2, +1});
50            eventos.push_back({r.x2, r.y1, r.y2, -1});
51            yTemp.push_back(r.y1);
52            yTemp.push_back(r.y2);
53        }
54        sort(yTemp.begin(), yTemp.end());
55        yTemp.erase(unique(yTemp.begin(), yTemp.end()), yTemp.end());
56        ys = yTemp;
57        int m = (int)ys.size() - 1; // numero de intervalos [ys[i],
      // ys[i+1])
58        if (m <= 0) return 0;
59        conteo.assign(4 * m, 0);
60        cubierta.assign(4 * m, 0);
61
62        sort(eventos.begin(), eventos.end(), [](const Evento& a,
63            const Evento& b) {
64                return a.x < b.x;
65            });
66
67        double area = 0;
68        for (size_t i = 0; i < eventos.size(); i++) {
69            if (i > 0) {
70                double ancho = eventos[i].x - eventos[i-1].x;
71                area += ancho * cubierta[i];
72            }
73            int l = (int)(lower_bound(ys.begin(), ys.end(), eventos[i].y1) - ys.begin());
74            int r = (int)(lower_bound(ys.begin(), ys.end(), eventos[i].y2) - ys.begin());
75            actualizar(l, 0, m, l, r, eventos[i].delta);
76        }
77        return area;
78    } // O(n log n): sort de 2n eventos, mas 2n actualizaciones O
      // (log m) cada una
79
80    //

```

```

78 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
79 // -----
80 // 1. Perimetro de la union (en vez de area): requiere ademas
    trackear
81 // cuantos "segmentos" cubiertos tiene cada nodo, para
    sumar la
82 // variacion vertical entre eventos consecutivos mas la
    longitud
83 // horizontal del borde superior/inferior. Struct distinto,
    ver
84 // Perimetro de la Union de Rectangulos.
85 // 2. Volumen de union de cajas 3D: mismo barrido sobre el
    eje x, pero
86 // el arbol de segmentos debe resolver recursivamente el
    AREA
87 // cubierta en el plano yz - es esta misma tecnica anidada
    en 2D.
88 // 3. Si las coordenadas ya son enteras y pequenas, se puede
    omitir la
89 // compresion e indexar el arbol directamente por valor,
    siempre que
90 // el rango quepa en memoria.
91 };

```

Ejemplo de funcionamiento:

```

1 AreaUnionRectangulos r(2);
2 r.agregarRectangulo(0, 0, 4, 4); // area 16
3 r.agregarRectangulo(2, 2, 6, 6); // area 16, se solapa en (2,2)
    -(4,4)
4
5 double resultado = r.calcularArea();
6 // resultado = 28
7 // (16 + 16 - 4 de solapamiento, por inclusion-exclusion)

```

Regla práctica: usa esta técnica cuando la pregunta sea sobre **área total cubierta** (una cantidad continua), no sobre cuántos objetos están activos —para eso te alcanza el barrido genérico simple. Si además necesitas el perímetro en vez del área, el árbol de segmentos necesita información adicional (ver modificación 1 dentro del código).

27.4 Skyline Problem

Idea: comparte la estructura de eventos de *Unión de Áreas de Rectángulos* —cada edificio genera un evento de inicio y uno de fin—, pero la pregunta es distinta: en vez de longitud cubierta, interesa la **altura máxima visible** en cada punto x . El estado del barrido es entonces el conjunto de alturas de los edificios actualmente activos, y el skyline solo cambia en los puntos donde esa altura máxima cambia. Guardar las alturas activas en un **multiset** permite insertar, borrar y consultar el máximo, todo en $O(\log n)$.

¿Por qué usarlo? La alternativa de recorrer edificio por edificio y comparar contra todos los demás es $O(n^2)$. Aquí, igual que en la unión de rectángulos, basta con procesar eventos ordenados y mantener el estado activo, pero con una estructura distinta: un **multiset** de alturas en vez de un árbol de segmentos por conteo, porque lo único que importa es el máximo actual, no una suma o longitud acumulada. Es más simple que adaptar el árbol de segmentos de la subsección anterior para este propósito.

Casos de uso:

- El problema clásico de silueta de edificios (Skyline Problem).
- Calcular el área bajo la envolvente superior de un conjunto de rectángulos alineados en la base.

- Determinar en qué momentos cambia el valor máximo entre un conjunto de intervalos con "altura" o "prioridad" asociada.
- Simulación de capas visibles en gráficos 2D (qué elemento está "al frente" en cada posición x).

Complejidad: $O(n \log n)$. Ordenar los $2n$ eventos cuesta $O(n \log n)$. Cada evento realiza una inserción o un borrado en el **multiset**, cada uno $O(\log n)$, y consultar el máximo con **rbegin()** es $O(1)$. En total, $O(n \log n)$.

```

1 // CREATE/READ/WRITE: Silueta (skyline) de n edificios mediante
    barrido
2 // horizontal + multiset de alturas activas
3 struct SkylineProblema {
4     struct Evento { long long x; long long alto; bool esInicio;
5     };
6     vector<Evento> eventos;
7
8     // CREATE: reserva espacio para 2*n eventos (inicio y fin de
        cada edificio)
9     SkylineProblema(int n) {
10         eventos.reserve(2 * n);
11     } // O(1)
12
13     // WRITE: agrega un edificio dado su rango horizontal [izq,
        der] y su altura
14 void agregarEdificio(long long izq, long long der, long long
    alto) {
15     eventos.push_back({izq, alto, true});
16     eventos.push_back({der, alto, false});
17 } // O(1) amortizado
18
19 // READ: calcula los puntos clave del skyline: pares (x,
    altura) donde
20 // la altura maxima visible cambia
21 vector<pair<long long, long long>> calcularSkyline() {
22     sort(eventos.begin(), eventos.end(), [](const Evento& a,
        const Evento& b) {
23         return a.x < b.x;
24     });
25
26     multiset<long long> alturasActivas = {0}; // el "suelo"
        siempre esta activo
27     vector<pair<long long, long long>> resultado;
28     long long alturaPrevia = 0;
29     size_t i = 0;
30     while (i < eventos.size()) {
31         long long xActual = eventos[i].x;
32         // procesa TODOS los eventos de esta misma x como un
        bloque, para
33         // no reportar un punto intermedio incorrecto
34         while (i < eventos.size() && eventos[i].x == xActual) {
35             if (eventos[i].esInicio) alturasActivas.insert(eventos[i].alto);
36             else alturasActivas.erase(alturasActivas.find(eventos[i].alto));
37             i++;
38         }
39         long long alturaActual = *alturasActivas.rbegin();
40         if (alturaActual != alturaPrevia) {
41             resultado.push_back({xActual, alturaActual});
42             alturaPrevia = alturaActual;
43         }
44     }
45     return resultado;
46 } // O(n log n): sort O(n log n), n operaciones sobre el
    multiset O(log n) cada una
47
48 // -----
49 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
50 // -----
51 // 1. Area bajo el skyline en vez de los puntos clave: basta
    con acumular
52 // (x_actual - x_anterior) * alturaPrevia en cada
    iteracion del while
53 // externo, antes de actualizar alturaPrevia.
54 // 2. Muchos edificios con la MISMA altura exacta: el
    multiset es
55 // preferible a un arbol de segmentos aqui, porque
    insertar/borrar UNA
56 // ocurrencia de un valor repetido es directo con find +
    erase.
57 // 3. Silueta con arbol de segmentos (alternativa): si el
    rango de x es
    // pequeno y se requiere actualizar dinamicamente, se
    puede reemplazar

```



```

58 // el multiset por un arbol de segmentos tipo "maximo con
59 // propagacion
60 // perezosa" sobre las coordenadas x comprimidas.
};

```

Ejemplo de funcionamiento:

```

1 SkylineProblema s(2);
2 s.agregarEdificio(1, 5, 3);
3 s.agregarEdificio(3, 7, 5);
4
5 auto resultado = s.calcularSkyline();
6 // resultado = {(1,3), (3,5), (7,0)}
7 // (en x=1 aparece un edificio de altura 3; en x=3 aparece uno
8 // de altura 5, que domina la vista; en x=7 termina el mas
9 // alto y la
10 // altura visible cae a 0, ya que el primero termino en x=5
// sin cambiar
// el maximo visible en ese punto)

```

Nota: el paso de agrupar eventos por la misma coordenada x antes de consultar el máximo (el `while` interno del código) es crítico: procesar los eventos uno por uno sin agrupar puede reportar un punto de skyline falso cuando un edificio termina justo donde otro empieza en la misma coordenada.

Regla práctica: usa esta técnica cuando la pregunta sea sobre el **máximo** (o mínimo) visible en cada punto, no sobre longitud o área acumulada —para eso está *Unión de Áreas de Rectángulos*. Ambas comparten la misma estructura de eventos de entrada/salida, así que si ya resolviste una, adaptar la otra es principalmente cambiar la estructura de estado (árbol de segmentos por conteo vs. multiset de alturas).

27.5 Par de Puntos Más Cercano (Closest Pair)

Idea: a diferencia de las subsecciones anteriores, aquí el barrido no procesa eventos de entrada/salida de un intervalo, sino **puntos individuales** ordenados por x , y el estado activo es una "franja" (*strip*) de puntos candidatos a formar el par más cercano. La observación clave: si ya se conoce una distancia mínima d entre los puntos procesados hasta ahora, cualquier punto nuevo solo puede mejorar esa distancia si está a menos de d en x del punto actual —por eso se mantiene una ventana de puntos "recientes" (los últimos con x dentro de d), y dentro de esa ventana, gracias a un argumento geométrico de empaquetamiento, solo hace falta comparar contra un número constante de vecinos ordenados por y .

¿Por qué usarlo? Comparar todos los pares de n puntos es $O(n^2)$. El enfoque de *divide y vencerás* clásico también resuelve esto en $O(n \log n)$, pero requiere combinar recursivamente las mitades izquierda y derecha con una franja central —más difícil de implementar. La versión con barrido logra la misma complejidad manteniendo una única estructura ordenada por y (un multiset o similar) que se poda dinámicamente, sin necesidad de recursión ni de fusionar subproblemas. Es la aplicación más directa de "mantener solo el estado necesario" entre las técnicas de esta sección: aquí el estado es una ventana deslizante de puntos, no un conjunto de intervalos activos.

Casos de uso:

- Distancia mínima entre n puntos en el plano.
- Detección de colisiones o "puntos demasiado cercanos" en simulaciones.

- Preprocesamiento en problemas de clustering o agrupamiento espacial.
- Verificar si algún par de ubicaciones (antenas, sensores) viola una distancia mínima de seguridad.

Complejidad: $O(n \log n)$. Ordenar los n puntos por x cuesta $O(n \log n)$. Cada punto se inserta y eventualmente se elimina del multiset activo — $O(\log n)$ cada operación—, y por cada punto se consultan solo los vecinos con y dentro de rango d , que son $O(1)$ en promedio por el argumento de empaquetamiento (en una franja de ancho d y alto $2d$ caben a lo sumo un número constante de puntos con distancia mínima d entre ellos). El total es $O(n \log n)$.

```

1 // CREATE/READ/WRITE: Distancia minima entre n puntos mediante
2 // barrido
3 // horizontal + ventana de candidatos ordenada por y
4 struct ParMasCercano {
5     struct Punto { double x, y; int id; };
6     vector<Punto> puntos;
7
8     // CREATE: reserva espacio para n puntos
9     ParMasCercano(int n) {
10         puntos.reserve(n);
11     } // O(1)
12
13     // WRITE: agrega un punto al conjunto
14     void agregarPunto(double x, double y) {
15         puntos.push_back({x, y, (int)puntos.size()});
16     } // O(1) amortizado
17
18     // READ: calcula la distancia euclidiana al cuadrado (evita
19     // sqrt hasta el final)
20     static double distCuadrado(const Punto& a, const Punto& b) {
21         double dx = a.x - b.x, dy = a.y - b.y;
22         return dx * dx + dy * dy;
23     } // O(1)
24
25     // READ: barre los puntos ordenados por x y devuelve la
26     // distancia minima
27     // entre cualquier par
28     double distanciaMinima() {
29         sort(puntos.begin(), puntos.end(), [](const Punto& a, const
30         Punto& b) {
31             return a.x < b.x;
32         });
33
34         // ventana ordenada por y, contiene solo puntos con x
35         // dentro de la
36         // distancia minima actual respecto al punto que se esta
37         // procesando
38         set<pair<double,int>> ventana; // (y, id) para permitir
39         // duplicados de y
40         double mejorCuadrado = 1e18;
41         int inicio = 0; // indice del primer punto todavia dentro
42         // de la ventana
43
44         for (int i = 0; i < (int)puntos.size(); i++) {
45             double mejorActual = sqrt(mejorCuadrado);
46             // elimina de la ventana los puntos cuya x ya quedo
47             // demasiado atras
48             while (inicio < i && puntos[i].x - puntos[inicio].x >
49             mejorActual) {
50                 ventana.erase({puntos[inicio].y, puntos[inicio].id});
51                 inicio++;
52             }
53             // solo hace falta comparar contra puntos con y en [y_i -
54             // mejorActual, y_i + mejorActual]
55             auto desde = ventana.lower_bound({puntos[i].y -
56             mejorActual, -1});
57             auto hasta = ventana.upper_bound({puntos[i].y +
58             mejorActual, INT_MAX});
59             for (auto it = desde; it != hasta; ++it) {
60                 double d2 = distCuadrado(puntos[i], puntos[it->second]);
61
62                 mejorCuadrado = min(mejorCuadrado, d2);
63             }
64             ventana.insert({puntos[i].y, puntos[i].id});
65         }
66         return sqrt(mejorCuadrado);
67     } // O(n log n): sort O(n log n), y por el argumento de
68     // empaquetamiento cada
69     // punto compara con O(1) vecinos en promedio dentro de la
70     // ventana

```

```

56 //
57 // -----
58 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
59 // -----
60 // 1. Reportar el PAR de puntos (no solo la distancia):
61 // guardar los ids
62 // junto con mejorCuadrado cada vez que se actualiza el
63 // minimo dentro
64 // del for interno.
65 // 2. Coordenadas enteras: usar long long en distCuadrado
66 // para evitar
67 // overflow, y comparar con mejorCuadrado como entero (sin
68 // sqrt) hasta
69 // el resultado final, evitando errores de precision con
70 // double.
71 // 3. Divide y venceras (alternativa clasica): en vez de la
72 // ventana
73 // deslizante con set, se puede resolver recursivamente
74 // dividiendo el
75 // conjunto de puntos por la mediana de x, resolviendo
76 // cada mitad, y
77 // fusionando con una franja central de ancho 2*d. Misma
78 // complejidad
79 // O(n log n), pero mas dificil de implementar
80 // correctamente; el
81 // barrido con set es preferible en un contexto de
82 // concurso.
83 };

```

Ejemplo de funcionamiento:

```

1 ParMasCercano p(4);
2 p.agregarPunto(0, 0);
3 p.agregarPunto(3, 4);
4 p.agregarPunto(1, 1);
5 p.agregarPunto(10, 10);
6
7 double resultado = p.distanciaMinima();
8 // resultado = sqrt(2)
9 // (el par mas cercano es (0,0) y (1,1), a distancia sqrt(1^2 +
10 // 1^2) = sqrt(2);
11 // hay que correrlo para confirmar el valor decimal exacto)

```

Nota: la condición `puntos[i].x > mejorActual` para podar la ventana, y los límites `lower_bound/upper_bound` en `y`, son lo que mantiene la comparación por punto en $O(1)$ amortizado. Si se omite cualquiera de las dos podas (en x o en y), el algoritmo sigue siendo correcto pero degenera a $O(n^2)$ en el peor caso, perdiendo la ventaja del barrido.

Regla práctica: usa esta técnica cuando el problema pida una distancia mínima (o máxima densidad local) entre puntos en el plano y n sea grande. Frente al divide y vencerás clásico, esta versión con barrido y ventana deslizante es más simple de codificar en competencia y logra la misma complejidad $O(n \log n)$, siempre que no se olviden las dos podas de la ventana (en x al insertar/eliminar, en y al consultar).

28 Primitivas Geométricas

28.1 Vector Geométrico: Producto Punto y Producto Cruz

Idea: un vector geométrico $\vec{v} = (x, y)$ representa una dirección y magnitud en el plano, y casi toda la geometría computacional se reduce a dos operaciones entre vectores: el **producto punto** $\vec{a} \cdot \vec{b} = a_x b_x + a_y b_y$, que mide cuánto "apuntan en la misma dirección" dos vectores (relacionado con el coseno del ángulo entre ellos), y el **producto cruz** $\vec{a} \times \vec{b} = a_x b_y - a_y b_x$, que en 2D da un escalar cuyo signo indica de qué lado está un vector respecto al otro (relacionado con el seno del ángulo, y con el área con signo del paralelogramo que forman). Estas dos operaciones son los "primitivos" a partir de los cuales se

construyen casi todos los demás algoritmos de esta parte: orientación, colinealidad, área de polígonos, ángulos.

¿Por qué usarlo? Sin estas dos operaciones no hay forma de razonar algebraicamente sobre direcciones y orientación en el plano —la alternativa sería calcular ángulos con `atan2` y compararlos, lo cual es más lento, acumula error de punto flotante, y falla en casos límite (ángulos de 0 o 2π). El producto punto y el producto cruz, en cambio, se calculan con aritmética exacta si las coordenadas son enteras, y evitan funciones trigonométricas costosas en la mayoría de los casos donde solo se necesita signo o comparación relativa, no el ángulo exacto.

Casos de uso:

- Determinar si tres puntos giran en sentido horario o antihorario (ver *Orientación de Tres Puntos*).
- Calcular el área de un polígono (Shoelace, que es una suma de productos cruz).
- Proyectar un punto sobre una línea (producto punto normalizado).
- Determinar si un ángulo es agudo, recto u obtuso (signo del producto punto).
- Ordenar puntos angularmente alrededor de un pivote (signo del producto cruz).

Complejidad: $O(1)$ para ambas operaciones —son sumas y multiplicaciones de un número constante de componentes, sin importar el tamaño de la entrada.

```

1 // CREATE/READ/WRITE: Vector geometrico 2D con producto punto y
2 // producto cruz
3 struct Vector2D {
4     double x, y;
5
6     // CREATE: construye un vector a partir de sus componentes
7     // (0,0 por defecto)
8     Vector2D(double x_ = 0, double y_ = 0) : x(x_), y(y_) {} // O(1)
9
10    // READ: resta de puntos para obtener el vector que va de a
11    // hacia b
12    static Vector2D desde(const Vector2D& a, const Vector2D& b) {
13        return Vector2D(b.x - a.x, b.y - a.y);
14    } // O(1)
15
16    // READ: producto punto, mide alineacion entre this y otro
17    // vector
18    double punto(const Vector2D& otro) const {
19        return x * otro.x + y * otro.y;
20    } // O(1)
21
22    // READ: producto cruz (componente z del producto cruz 3D),
23    // mide
24    // orientacion relativa y area con signo del paralelogramo
25    double cruz(const Vector2D& otro) const {
26        return x * otro.y - y * otro.x;
27    } // O(1)
28
29    // READ: magnitud (norma) del vector
30    double norma() const {
31        return sqrt(x * x + y * y);
32    } // O(1)
33
34    //
35    // -----
36    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
37    // -----
38
39    // 1. Coordenadas enteras: si el problema garantiza
40    // coordenadas enteras,
41    // usar long long en vez de double evita error de punto
42    // flotante en
43    // punto() y cruz() (ambos dan resultado exacto con
44    // enteros).
45    // 2. Signo del producto cruz: cruz(otro) > 0 significa que "
46    // otro" esta
47    // en sentido antihorario respecto a this; < 0 significa
48    // horario;

```

```

37 // == 0 significa que son colineales (paralelos). Ver
    Orientacion de
38 // Tres Puntos para la aplicacion directa de esto.
39 // 3. Angulo entre vectores: se puede obtener con
40 // atan2(this->cruz(otro), this->punto(otro)), que da el
    angulo con
41 // signo en (-pi, pi] sin ambigüedad de cuadrante, a
    diferencia de
42 // usar solo acos(punto / (norma*norma)).
43 };

```

Ejemplo de funcionamiento:

```

1 Vector2D a(2, 0);
2 Vector2D b(0, 3);

3
4 double p = a.punto(b);
5 // p = 0
6 // (son perpendiculares, el producto punto de vectores
    perpendiculares es 0)
7
8 double c = a.cruz(b);
9 // c = 6
10 // (positivo: b esta en sentido antihorario respecto a a;
    // 6 es tambien el area del paralelogramo que forman)

```

Regla práctica: usa el producto punto cuando la pregunta sea sobre **alineación o proyección** (¿qué tan paralelos son dos vectores?, ¿es un ángulo agudo u obtuso?); usa el producto cruz cuando la pregunta sea sobre **orientación o área con signo** (¿a qué lado está un punto?, ¿en qué sentido giran tres puntos?). Casi todo lo que sigue en esta parte de la guía se construye componiendo estas dos operaciones.

28.2 Orientación de Tres Puntos (Cross Product / CCW Test)

Idea: dado un punto O (origen/pivote) y dos puntos A y B , el signo del producto cruz $\vec{OA} \times \vec{OB}$ —visto en *Vector Geométrico*— determina si, parado en O mirando hacia A , el punto B queda a la izquierda (positivo, sentido antihorario/CCW), a la derecha (negativo, sentido horario/CW), o exactamente sobre la línea que pasa por O y A (cero, colineales). Esta es la prueba de orientación más usada en geometría computacional porque reduce una pregunta geométrica cualitativa ("¿de qué lado está?") a una operación algebraica exacta de $O(1)$.

¿Por qué usarlo? Es la primitiva que sustituye a comparar ángulos con `atan2` o a calcular pendientes con división —ambas formas más lentas, propensas a error de punto flotante, y con casos especiales molestos (línea vertical, división por cero). El producto cruz da el mismo resultado cualitativo con una sola multiplicación y resta, y si las coordenadas son enteras, el resultado es exacto sin ningún error de precisión. Es la base directa de Shoelace, Convex Hull y Point-in-Polygon, que se ven más adelante en esta parte.

Casos de uso:

- Determinar si un giro en una secuencia de puntos es a la izquierda o a la derecha (clave en construcción de envolvente convexa).
- Verificar si tres puntos son colineales.
- Determinar de qué lado de una línea está un punto (para point-in-polygon, clipping, etc.).
- Ordenar puntos angularmente alrededor de un pivote.

- Calcular el área con signo de un triángulo (mitad del valor absoluto del producto cruz).

Complejidad: $O(1)$ —es una evaluación directa del producto cruz entre dos vectores derivados de los tres puntos, sin bucles ni recursión.

```

1 // CREATE/READ: Prueba de orientacion de tres puntos mediante
    producto cruz
2 struct OrientacionPuntos {
3     struct Punto { double x, y; };
4
5     // READ: producto cruz de (b - o) x (c - o); signo indica
        orientacion
6     // de o, b, c: positivo = CCW (izquierda), negativo = CW (
        derecha), 0 = colineales
7     static double cruz(const Punto& o, const Punto& b, const
        Punto& c) {
8         return (b.x - o.x) * (c.y - o.y) - (b.y - o.y) * (c.x - o.x);
9     } // O(1)
10
11     // READ: clasifica la orientacion como +1 (CCW), -1 (CW), o 0
        (colineal)
12     // usando un epsilon para tolerar error de punto flotante
13     static int orientacion(const Punto& o, const Punto& b, const
        Punto& c) {
14         double val = cruz(o, b, c);
15         const double EPS = 1e-9;
16         if (val > EPS) return 1; // CCW: c esta a la izquierda de
            o->b
17         if (val < -EPS) return -1; // CW: c esta a la derecha de o
            ->b
18         return 0; // colineales
19     } // O(1)
20
21     //
22     //-----
23     // MODIFICACIONES Y COMENTARIOS CON LOS USOS
24     //-----
25     // 1. Coordenadas enteras: usar long long y comparar cruz()
        directamente
26     // contra 0 (sin EPS), ya que con enteros el resultado es
        exacto y no
27     // hay error de punto flotante que tolerar.
28     // 2. Colinealidad estricta con verificacion de "entre":
        orientacion == 0
29     // solo dice que estan sobre la misma recta, no que c este
        ENTRE o y b.
30     // Para eso se necesita ademas revisar que las coordenadas
        de c esten
31     // dentro del rango [o, b] en x e y (ver Tres Puntos
        Colineales).
32     // 3. Convencion de signo: esta implementacion usa positivo =
        CCW, que es
33     // la convencion estandar en la mayoria de la literatura
        de CP (y la
34     // que se usa consistentemente en el resto de esta guia,
        incluyendo
35     // Shoelace y Envolvente Convexa). Si se invierte el orden
        de los
36     // parametros (o, c, b en vez de o, b, c), el signo
        tambien se invierte.
37 };

```

Ejemplo de funcionamiento:

```

1 OrientacionPuntos::Punto o{0, 0}, b{4, 0}, c{2, 2};
2
3 int resultado = OrientacionPuntos::orientacion(o, b, c);
4 // resultado = 1 (CCW)
5 // (parado en o mirando hacia b sobre el eje x, el punto c
    =(2,2) queda
6 // arriba, es decir a la izquierda, sentido antihorario)

```

Nota: la convención positivo = CCW fijada aquí se usa de forma consistente en el resto de la guía —Shoelace, Envolvente Convexa (Jarvis y Monotone Chain), y Punto Dentro de un Polígono Convexo. Si en algún momento se necesita la convención opuesta, basta con invertir el signo del resultado o el orden de los argumentos, pero hay que hacerlo de forma consistente en todos los algoritmos que dependan de esta prueba.

Regla práctica: usa esta prueba como bloque de construcción base cada vez que necesites saber de qué

lado de una línea (definida por dos puntos) está un tercer punto, sin importar el ángulo exacto —eso es más rápido y más robusto numéricamente que calcular pendientes o ángulos con funciones trigonométricas.

28.3 Distancia Euclidiana

Idea: la distancia euclidiana entre dos puntos $A = (x_1, y_1)$ y $B = (x_2, y_2)$ se deriva directamente del teorema de Pitágoras: el segmento AB es la hipotenusa de un triángulo rectángulo cuyos catetos son las diferencias de coordenadas $\Delta x = x_2 - x_1$ y $\Delta y = y_2 - y_1$, así que $d(A, B) = \sqrt{\Delta x^2 + \Delta y^2}$. Es la noción de distancia "en línea recta" y la base de la mayoría de las métricas geométricas de esta guía, aunque conceptualmente equivale a la norma del vector $\overrightarrow{AB}$ (ver *Vector Geométrico*).

¿Por qué usarlo? Es la métrica por defecto en geometría computacional cuando el problema no especifica otra cosa (a diferencia de la distancia Manhattan, que suma $|\Delta x| + |\Delta y|$ y aparece en problemas de grilla). El costo real está en la raíz cuadrada: cuando solo se necesita **comparar** distancias (cuál es menor, cuál es mayor), conviene trabajar con la distancia al cuadrado y evitar `sqrt()` —así se gana precisión con enteros y se ahorra una operación costosa, como ya se hizo en *Par de Puntos Más Cercano*.

Casos de uso:

- Encontrar el par de puntos más cercano o más lejano.
- Verificar si un punto está dentro de un círculo de radio dado.
- Calcular el perímetro de un polígono (suma de distancias entre vértices consecutivos).
- Comparar cuál de dos puntos está más cerca de un tercero, sin necesidad del valor exacto.

Complejidad: $O(1)$ —dos restas, dos multiplicaciones, una suma y, opcionalmente, una raíz cuadrada.

```
1 // CREATE/READ: Distancia euclidiana entre dos puntos, con y
  // sin raíz cuadrada
2 struct DistanciaEuclidiana {
3     struct Punto { double x, y; };
4
5     // READ: distancia al cuadrado, evita sqrt cuando solo se
      // necesita comparar
6     static double distCuadrado(const Punto& a, const Punto& b) {
7         double dx = b.x - a.x;
8         double dy = b.y - a.y;
9         return dx * dx + dy * dy;
10    } // O(1)
11
12    // READ: distancia euclidiana real (con raíz cuadrada)
13    static double distancia(const Punto& a, const Punto& b) {
14        return sqrt(distCuadrado(a, b));
15    } // O(1)
16
17    //
18
19    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
20    // -----
21    // 1. Coordenadas enteras: usar long long en distCuadrado
      // evita perder
22    // precision; además, comparar distCuadrado(a,b) <
      // distCuadrado(c,d)
23    // da el mismo resultado que comparar distancia(a,b) <
      // distancia(c,d)
24    // sin necesidad de sqrt, porque la raíz cuadrada es
      // monótona creciente.
25    // 2. Distancia Manhattan (metrica alternativa): |b.x-a.x| +
      // |b.y-a.y|,
      // usada en problemas de grilla donde el movimiento es
      // solo horizontal
```

```
26 // o vertical, no diagonal.
27 // 3. Distancia Chebyshev (otra metrica alternativa): max(|b.
      // x-a.x|,
28 // |b.y-a.y|), usada cuando el movimiento diagonal cuesta
      // igual que el
29 // movimiento recto (como el rey en ajedrez).
30 };
```

Ejemplo de funcionamiento:

```
1 DistanciaEuclidiana::Punto a{0, 0}, b{3, 4};
2
3 double resultado = DistanciaEuclidiana::distancia(a, b);
4 // resultado = 5
5 // (triangulo 3-4-5, caso clasico de pitagoras exacto)
```

Regla práctica: usa `distCuadrado` en vez de `distancia` cada vez que el problema solo requiera **comparar** distancias entre sí (mínimo, máximo, ¿es menor que r ?) —evita la raíz cuadrada y, con coordenadas enteras, evita también el error de punto flotante por completo.

28.4 Punto Medio de un Segmento

Idea: el punto medio de un segmento entre $A = (x_1, y_1)$ y $B = (x_2, y_2)$ es el promedio simple de sus coordenadas:

$M = \left(\frac{x_1 + x_2}{2}, \frac{y_1 + y_2}{2} \right)$. Es un caso particular de interpolación lineal con $t = 0.5$ —el punto que está exactamente a mitad de camino en ambas coordenadas por separado, ya que la geometría del plano cartesiano permite tratar x e y de forma independiente para este cálculo.

¿Por qué usarlo? Es la operación más simple de esta parte, pero aparece constantemente como paso intermedio de algoritmos más complejos (bisección de segmentos, cálculo de centroides, mediatriz para circuncentro). A diferencia de la *División de un Segmento en una Razón Dada* (más general, con t arbitrario), el punto medio es el caso fijo $t = 0.5$ y no requiere parametrizar la razón, por lo que conviene tenerlo aparte como atajo directo cuando la razón es exactamente 1 : 1.

Casos de uso:

- Encontrar el centro de un segmento para construir la mediatriz (útil en circuncentro).
- Bisección recursiva de un segmento o polígono.
- Calcular el centroide de un conjunto de puntos (promedio de todos los puntos medios, o directamente el promedio de coordenadas).
- Punto de referencia para pruebas de simetría o reflexión.

Complejidad: $O(1)$ —dos sumas y dos divisiones por 2.

```
1 // CREATE/READ: Punto medio de un segmento definido por dos
  // puntos
2 struct PuntoMedio {
3     struct Punto { double x, y; };
4
5     // READ: calcula el punto medio entre a y b
6     static Punto medio(const Punto& a, const Punto& b) {
7         return Punto{(a.x + b.x) / 2.0, (a.y + b.y) / 2.0};
8     } // O(1)
9
10    //
11
12    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
13    // -----
```



```

13 // 1. Coordenadas enteras con resultado exacto: si a.x+b.x y
14 // a.y+b.y son
15 // siempre pares, se puede mantener el punto medio en
16 // enteros dividiendo
17 // entre 2 directamente, evitando pasar por double.
18 // 2. Centroide de n puntos (generalizacion): promedio de
19 // todas las
20 // coordenadas x y de todas las coordenadas y por separado,
21 // no es lo
22 // mismo que promediar puntos medios sucesivos si los
23 // puntos no forman
24 // un segmento simple.
25 // 3. Punto medio ponderado (interpolacion con t != 0.5): ver
26 // Division de
27 // un Segmento en una Razon Dada para el caso general.
28 };

```

Ejemplo de funcionamiento:

```

1 PuntoMedio::Punto a{0, 0}, b{6, 10};
2
3 auto resultado = PuntoMedio::medio(a, b);
4 // resultado = (3, 5)

```

Regla práctica: usa el punto medio directamente cuando la razón sea exactamente 1 : 1; si el problema pide un punto a una fracción arbitraria del segmento (por ejemplo, a un tercio de camino), no fuerces el punto medio dos veces —usa la fórmula general de *División de un Segmento en una Razón Dada*, que es igual de simple y evita errores de composición.

28.5 Ecuación de una Línea Dados Dos Puntos

Idea: dados dos puntos $A = (x_1, y_1)$ y $B = (x_2, y_2)$, la recta que pasa por ambos se puede escribir en forma general $ax + by + c = 0$ tomando $a = y_2 - y_1$, $b = x_1 - x_2$, $c = -(ax_1 + by_1)$. Esta forma se deriva de que el vector normal a la dirección $\overrightarrow{AB} = (x_2 - x_1, y_2 - y_1)$ es $(a, b) = (y_2 - y_1, -(x_2 - x_1))$ —rotar el vector director 90 grados— y cualquier punto (x, y) sobre la recta satisface que el vector desde A hasta (x, y) es perpendicular a ese normal, lo cual da la ecuación. A diferencia de la forma pendiente-intersección $y = mx + k$, esta forma general no tiene problema con rectas verticales (donde la pendiente no está definida).

¿Por qué usarlo? La forma $y = mx + k$ falla cuando $x_1 = x_2$ (recta vertical, pendiente infinita), obligando a manejar ese caso aparte con un `if`. La forma general $ax + by + c = 0$ maneja rectas verticales y horizontales de manera uniforme sin casos especiales, y además permite evaluar directamente de qué lado de la recta está un punto con el signo de $ax + by + c$ —relacionado con el producto cruz de *Orientación de Tres Puntos*, pero expresado como coeficientes reutilizables en vez de recalcular el producto cruz cada vez.

Casos de uso:

- Determinar si un punto está sobre, a la izquierda o a la derecha de una recta fija (evaluada muchas veces).
- Calcular la distancia de un punto a una recta (usando los coeficientes a, b, c directamente).
- Encontrar la intersección entre dos rectas resolviendo el sistema de dos ecuaciones.
- Representar restricciones lineales en problemas de programación lineal geométrica simple.

Complejidad: $O(1)$ para construir la ecuación; $O(1)$ para evaluarla en un punto dado.

```

1 // CREATE/READ: Linea en forma general ax + by + c = 0, dados
2 // dos puntos
3 struct LineaGeneral {
4     double a, b, c;
5
6     // CREATE: construye la ecuacion general de la recta que pasa
7     // por p y q
8     LineaGeneral(double x1, double y1, double x2, double y2) {
9         a = y2 - y1;
10        b = x1 - x2;
11        c = -(a * x1 + b * y1);
12    } // O(1)
13
14    // READ: evalua ax + by + c en un punto; el signo indica el
15    // lado de la
16    // recta (positivo o negativo segun la orientacion de a,b), 0
17    // = sobre la recta
18    double evaluar(double x, double y) const {
19        return a * x + b * y + c;
20    } // O(1)
21
22    // READ: verifica si un punto esta sobre la recta (con
23    // tolerancia)
24    bool contiene(double x, double y) const {
25        const double EPS = 1e-9;
26        return fabs(evaluar(x, y)) < EPS;
27    } // O(1)
28
29    //
30    // -----
31    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
32    // -----
33
34    // 1. Interseccion de dos rectas L1 (a1,b1,c1) y L2 (a2,b2,c2)
35    // ): resolver
36    // el sistema 2x2 con la regla de Cramer:
37    // det = a1*b2 - a2*b1;
38    // if (fabs(det) < EPS) { /* paralelas o coincidentes, sin
39    // solucion unica */ }
40    // x = (b1*c2 - b2*c1) / det; y = (a2*c1 - a1*c2) / det;
41    // 2. Coordenadas enteras: a, b, c pueden mantenerse como
42    // long long para
43    // evaluar() y contiene() de forma exacta, comparando
44    // contiene()
45    // contra 0 sin EPS.
46    // 3. Normalizacion opcional: dividir a,b,c entre sqrt(a*a+b*b)
47    // convierte
48    // evaluar(x,y) directamente en la distancia con signo del
49    // punto a la
50    // recta (ver Distancia de un Punto a una Linea/Segmento).
51 };

```

Ejemplo de funcionamiento:

```

1 LineaGeneral l(0, 0, 4, 4); // recta y = x
2
3 double val = l.evaluar(2, 5);
4 // val = -12
5 // (a=4, b=-4, c=0; evaluar(2,5) = 4*2 + (-4)*5 + 0 = 8 - 20 =
6 // -12;
7 // el signo negativo indica que (2,5) esta de un lado
8 // específico de y=x)
9
10 bool sobre = l.contiene(3, 3);
11 // sobre = true (el punto (3,3) esta sobre la recta y=x)

```

Regla práctica: usa esta forma general en vez de $y = mx + k$ cada vez que la recta pueda ser vertical, o cuando necesites evaluar el mismo lado de la recta para muchos puntos distintos —los coeficientes a, b, c se calculan una sola vez y se reutilizan, a diferencia de recalcular un producto cruz completo contra dos puntos fijos en cada consulta.

28.6 División de un Segmento en una Razón Dada

Idea: generaliza el *Punto Medio de un Segmento* (que es el caso fijo $t = 0.5$) a cualquier fracción $t \in [0, 1]$ del camino entre A y B : el punto que divide el segmento en esa proporción es la interpolación lineal $P = A + t \cdot (B - A) = ((1 - t)x_1 + tx_2, (1 - t)y_1 + ty_2)$. Cuando $t = 0$ se obtiene A , cuando $t = 1$ se obtiene B , y valores intermedios dan puntos proporcionalmente distribuidos

sobre el segmento. Si la razón se da como " $m : n$ " (divide al segmento en partes de longitud proporcional m y n), se convierte a $t = \frac{m}{m+n}$.

¿Por qué usarlo? Es la forma general de la que el punto medio es un caso particular; evaluarlo cada vez con la fórmula de interpolación en vez de "forzar" un punto medio con pasos intermedios evita errores de composición y funciona igual de bien para t fuera de $[0, 1]$ (extrapolación, para encontrar un punto más allá de B en la misma dirección que $A \rightarrow B$). Es la base de la interpolación paramétrica de segmentos, usada también para representar cualquier punto sobre una recta en función de un solo parámetro.

Casos de uso:

- Dividir un segmento en k partes iguales (aplicando $t = i/k$ para $i = 1, \dots, k-1$).
- Encontrar un punto a una razón $m : n$ específica sobre un segmento (por ejemplo, el punto que divide una mediana en razón $2 : 1$ desde el vértice, propiedad del centroide de un triángulo).
- Extrapolar un punto más allá de un extremo del segmento, usando $t < 0$ o $t > 1$.
- Parametrizar un rayo o recta para intersección con otras figuras.

Complejidad: $O(1)$ —dos interpolaciones lineales independientes en x e y .

```
1 DivisionSegmento::Punto a{0, 0}, b{10, 20};
2
3 auto resultado = DivisionSegmento::dividirEnRazon(a, b, 1, 4);
4 // resultado = (2, 4)
5 // (t = 1/(1+4) = 0.2; el punto esta a un quinto del camino de
6 // a hacia b,
7 // dividiendo el segmento en partes de razon 1:4)
```

Regla práctica: usa esta fórmula general cada vez que necesites un punto sobre un segmento a una fracción o razón distinta de $1 : 1$ —para el caso exacto de la mitad, es más directo usar *Punto Medio de un Segmento* sin pasar por la conversión de razón a t .

28.7 Ángulo entre Dos Vectores

Idea: el ángulo entre dos vectores $\vec{a}$ y $\vec{b}$ se puede obtener de dos formas relacionadas: con el producto punto,

$\cos \theta = \frac{\vec{a} \cdot \vec{b}}{|\vec{a}| |\vec{b}|}$, o de forma más robusta con `atan2`, com-

binando producto punto y producto cruz —vistos ambos en *Vector Geométrico*— como $\theta = \text{atan2}(\vec{a} \times \vec{b}, \vec{a} \cdot \vec{b})$. La segunda forma da directamente el ángulo **con signo** en el rango $(-\pi, \pi]$, indicando no solo la magnitud del ángulo sino también si $\vec{b}$ está en sentido antihorario (positivo) u horario (negativo) respecto a $\vec{a}$, sin la ambigüedad de cuadrante que tiene `acos`.

¿Por qué usarlo? Usar solo `acos(punto / (norma_a * norma_b))` tiene dos problemas: pierde el signo (no distingue si $\vec{b}$ gira a la izquierda o a la derecha de $\vec{a}$), y es numéricamente inestable cerca de $\theta = 0$ o $\theta = \pi$, donde pequeños errores de punto flotante en el argumento de `acos` pueden caer fuera de $[-1, 1]$ y producir NaN. La versión con `atan2(cruz, punto)` evita ambos problemas: es estable en todo el rango y conserva el signo, que es justamente lo que se necesita para ordenar puntos angularmente (ver *Barrido Angular* más adelante).

Casos de uso:

- Ordenar un conjunto de puntos angularmente alrededor de un pivote.
- Calcular el ángulo interno de un polígono en un vértice dado.
- Determinar cuánto hay que rotar un vector para alinearlo con otro.
- Verificar si un ángulo es agudo, recto u obtuso sin calcular el valor exacto (signo del producto punto, ver *Vector Geométrico*).

Complejidad: $O(1)$ —un producto punto, un producto cruz, y una llamada a `atan2`.

```
1 // CREATE/READ: Angulo con signo entre dos vectores, usando
2 // atan2 para
3 // evitar la inestabilidad numerica de acos
4 struct AnguloVectores {
5     struct Vector { double x, y; };
6
7     // READ: producto punto de dos vectores
8     static double punto(const Vector& a, const Vector& b) {
9         return a.x * b.x + a.y * b.y;
10    } // O(1)
11
12    // READ: producto cruz de dos vectores
13    static double cruz(const Vector& a, const Vector& b) {
14        return a.x * b.y - a.y * b.x;
15    } // O(1)
```

```
1 // CREATE/READ: Division de un segmento AB en una razon dada
2 // mediante
3 // interpolacion lineal parametrica
4 struct DivisionSegmento {
5     struct Punto { double x, y; };
6
7     // READ: interpola el punto a fraccion t del segmento de a
8     // hacia b
9     // t=0 da a, t=1 da b, valores fuera de [0,1] extrapolan
10    static Punto interpolar(const Punto& a, const Punto& b,
11    double t) {
12        return Punto{
13            a.x + t * (b.x - a.x),
14            a.y + t * (b.y - a.y)
15        };
16    } // O(1)
17
18    // READ: divide el segmento en razon m:n (desde a hacia b),
19    // convirtiendo
20    // la razon a fraccion t = m/(m+n) antes de interpolar
21    static Punto dividirEnRazon(const Punto& a, const Punto& b,
22    double m, double n) {
23        double t = m / (m + n);
24        return interpolar(a, b, t);
25    } // O(1)
26
27    // -----
28    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
29    // -----
30
31    // 1. Punto medio como caso particular: interpolar(a, b, 0.5)
32    // es
33    // exactamente Punto Medio de un Segmento; no hace falta
34    // un struct
35    // separado si ya se tiene este.
36    // 2. Division en k partes iguales: generar los puntos
37    // interpolar(a, b, (double)i / k) para i = 1 hasta k-1.
38    // 3. Coordenadas enteras con razon m:n exacta (sin pasar por
39    // double):
40    // px = a.x + (b.x - a.x) * m / (m + n); usando division
41    // entera solo
42    // si se garantiza divisibilidad exacta; de lo contrario,
43    // mantener
44    // el resultado como fraccion (numerador, denominador)
45    // para exactitud.
46 };
```

Ejemplo de funcionamiento:

```

16 // READ: angulo con signo (en radianes) que hay que rotar a
17 // para
18 // alinearlos con b; positivo = antihorario, negativo =
19 // horario
20 static double angulo(const Vector& a, const Vector& b) {
21     return atan2(cruz(a, b), punto(a, b));
22 } // O(1)
23
24 //
25 //-----
26 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
27 //
28 //-----
29 // 1. Angulo NO firmado (solo magnitud, en [0, pi]): fabs(
30 // angulo(a, b)).
31 // Util cuando no importa el sentido de giro, solo la
32 // abertura.
33 // 2. Angulo interno de un poligono en el vertice v, entre
34 // los vertices
35 // vecinos prev y next: angulo(Vector::desde(v, prev),
36 // Vector::desde(v, next)),
37 // tomando el valor absoluto si el poligono no esta
38 // orientado de forma
39 // consistente.
40 // 3. Comparar SOLO el orden angular de puntos alrededor de
41 // un pivote (sin
42 // necesitar el angulo exacto): usar directamente el signo
43 // de cruz(a, b)
44 // como comparador de sort es mas rapido y mas exacto que
45 // llamar atan2
46 // para cada comparacion (ver Barrido Angular).
47 };

```

Ejemplo de funcionamiento:

```

1 AnguloVectores::Vector a{1, 0}, b{0, 1};
2
3 double resultado = AnguloVectores::angulo(a, b);
4 // resultado = pi/2 (aprox 1.5708)
5 // (b esta 90 grados en sentido antihorario respecto a a; hay
6 // que correrlo
7 // para confirmar el valor decimal exacto de pi/2 en el
8 // sistema usado)

```

Regla práctica: usa `atan2(cruz, punto)` en vez de `acos(punto/(norma_a*norma_b))` siempre que necesites el signo del ángulo o estabilidad numérica cerca de 0 o π . Si solo necesitas **comparar** el orden angular de varios vectores respecto a un pivote (no el ángulo exacto), ni siquiera hace falta `atan2` —el signo del producto cruz solo, usado como comparador, ya es suficiente y más rápido.

28.8 Tres Puntos Colineales

Idea: tres puntos A, B, C son colineales si están sobre la misma recta, lo cual ocurre exactamente cuando el producto cruz $\overrightarrow{AB} \times \overrightarrow{AC} = 0$ —el caso frontera de *Orientación de Tres Puntos*, donde no hay giro ni a la izquierda ni a la derecha. Sin embargo, colinealidad (estar sobre la misma recta infinita) es distinto de que C esté **entre** A y B sobre el segmento: para eso, además del producto cruz igual a cero, hace falta verificar que las coordenadas de C estén dentro del rango de caja delimitadora (*bounding box*) de A y B .

¿Por qué usarlo? Es una aplicación directa y frecuente de la prueba de orientación, pero se trata aparte porque el caso "colineal" suele ser el más propenso a errores en problemas de geometría: mezclar "están en la misma recta" con "uno está entre los otros dos" es una confusión común. Separar explícitamente ambas preguntas —colinealidad (producto cruz) y pertenencia al segmento (bounding box)— evita bugs sutiles, especialmente en algoritmos de intersección de segmentos donde los casos colineales se manejan aparte (como ya se vio en *Intersección de Segmentos*).

Casos de uso:

- Verificar si tres puntos dados están alineados (como paso de validación de entrada).
- Detectar si un punto cae exactamente sobre un segmento (no solo sobre su recta extendida).
- Simplificar un polígono eliminando vértices redundantes que son colineales con sus vecinos.
- Caso especial en algoritmos de intersección de segmentos, cuando el producto cruz da exactamente cero.

Complejidad: $O(1)$ —un producto cruz más, en el peor caso, cuatro comparaciones para el chequeo de bounding box.

```

1 // CREATE/READ: Verificacion de colinealidad de tres puntos, y
2 // de pertenencia
3 // de un punto colineal al segmento (no solo a la recta
4 // extendida)
5 struct TresPuntosColineales {
6     struct Punto { double x, y; };
7
8     // READ: producto cruz de (b-a) x (c-a)
9     static double cruz(const Punto& a, const Punto& b, const
10     Punto& c) {
11         return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x
12         );
13     } // O(1)
14
15     // READ: true si a, b, c son colineales (están sobre la misma
16     // recta)
17     static bool sonColineales(const Punto& a, const Punto& b,
18     const Punto& c) {
19         const double EPS = 1e-9;
20         return fabs(cruz(a, b, c)) < EPS;
21     } // O(1)
22
23     // READ: dado que a, b, c ya son colineales, verifica si c
24     // esta DENTRO
25     // del segmento [a, b] (no solo sobre la recta que lo
26     // contiene)
27     static bool entreSegmento(const Punto& a, const Punto& b,
28     const Punto& c) {
29         const double EPS = 1e-9;
30         bool dentroX = min(a.x, b.x) - EPS <= c.x && c.x <= max(a.x
31         , b.x) + EPS;
32         bool dentroY = min(a.y, b.y) - EPS <= c.y && c.y <= max(a.y
33         , b.y) + EPS;
34         return dentroX && dentroY;
35     } // O(1)
36
37     // READ: combina ambas verificaciones: c esta colineal Y
38     // dentro del
39     // segmento [a, b]
40     static bool puntoSobreSegmento(const Punto& a, const Punto& b
41     , const Punto& c) {
42         return sonColineales(a, b, c) && entreSegmento(a, b, c);
43     } // O(1)
44
45     //
46     //-----
47     // MODIFICACIONES Y COMENTARIOS CON LOS USOS
48     //
49     //-----
50     // 1. Coordenadas enteras: usar long long en cruz() y
51     // comparar contra 0
52     // exacto, sin EPS; entreSegmento tambien se vuelve
53     // comparacion exacta.
54     // 2. Simplificacion de poligono (eliminar vertices
55     // redundantes): recorrer
56     // los vertices consecutivos en tripletas (prev, actual,
57     // siguiente) y
58     // eliminar actual si sonColineales(prev, actual,
59     // siguiente) es true.
60     // 3. Ordenar puntos colineales sobre una misma recta: una
61     // vez confirmado
62     // que todos son colineales, se pueden ordenar por su
63     // proyeccion escalar
64     // sobre el vector director de la recta (producto punto
65     // contra un vector
66     // unitario en esa direccion), en vez de ordenar por x o y
67     // por separado
68     // (falla si la recta es vertical u horizontal exacta).
69 };

```

Ejemplo de funcionamiento:

```

1  TresPuntosColineales::Punto a{0, 0}, b{10, 0}, c{5, 0}, d{15,
2  0};
3  bool colineales = TresPuntosColineales::sonColineales(a, b, c);
4  // colineales = true (los tres estan sobre el eje x)
5
6  bool dentro = TresPuntosColineales::entreSegmento(a, b, c);
7  // dentro = true (c={5,0} esta entre a={0,0} y b={10,0})
8
9  bool fuera = TresPuntosColineales::puntoSobreSegmento(a, b, d);
10 // fuera = false (d={15,0} es colineal con a y b, pero esta MAS
11    ALLA de b,
    // fuera del segmento [a,b])

```

Regla práctica: nunca uses solo el producto cruz igual a cero para concluir que un punto "está sobre el segmento" —eso solo confirma que está sobre la **recta infinita**. Siempre combina la prueba de colinealidad con la verificación de bounding box (**entreSegmento**) cuando la pregunta sea específicamente sobre pertenencia al segmento, no a la recta extendida.

28.9 Distancia de un Punto a una Línea/Segmento

Idea: la distancia de un punto P a una **recta infinita** definida por A y B se obtiene proyectando $\vec{AP}$ sobre la dirección perpendicular a $\vec{AB}$: usando el producto cruz de *Vector Geométrico*, esa distancia es $\frac{|\vec{AB} \times \vec{AP}|}{|\vec{AB}|}$

—el área del paralelogramo que forman los dos vectores, dividida entre la base. Para la distancia a un **segmento** (no a la recta infinita), hay que distinguir tres casos según dónde cae la proyección de P sobre la línea: si cae dentro de $[A, B]$, la distancia es la misma fórmula perpendicular; si cae antes de A o después de B , la distancia real es simplemente la distancia euclidiana al extremo más cercano.

¿Por qué usarlo? Confundir distancia a la recta con distancia al segmento es un error común: la fórmula de la recta infinita subestima la distancia real cuando la proyección del punto cae fuera del segmento. La forma correcta usa el producto punto (visto en *Vector Geométrico*) para determinar en qué "zona" cae la proyección —antes de A , dentro de $[A, B]$, o después de B — y solo aplica la fórmula perpendicular en el caso intermedio, cayendo de vuelta a *Distancia Euclidiana* directa contra el extremo en los otros dos casos.

Casos de uso:

- Distancia mínima de un punto a un obstáculo representado como segmento o polilínea.
- Verificar si un punto está a menos de r unidades de una pared o borde (colisión punto-segmento).
- Cálculo auxiliar en *Closest Pair* cuando se generaliza a distancia punto-segmento en vez de punto-punto.
- Simplificación de polilíneas (algoritmo de Douglas-Peucker usa esta distancia para decidir qué puntos eliminar).

Complejidad: $O(1)$ —un producto punto, un producto cruz, y como mucho una llamada a `sqrt`.

```

1  // CREATE/READ: Distancia de un punto a una recta infinita y a
2  // un segmento
3  struct DistanciaPuntoLinea {
4      struct Punto { double x, y; };

```

```

4  // READ: producto punto de (b-a) con (p-a)
5  static double punto(const Punto& a, const Punto& b, const
6  Punto& p) {
7      return (b.x - a.x) * (p.x - a.x) + (b.y - a.y) * (p.y - a.y)
8  };
9  // O(1)
10
11 // READ: producto cruz de (b-a) con (p-a)
12 static double cruz(const Punto& a, const Punto& b, const
13 Punto& p) {
14     return (b.x - a.x) * (p.y - a.y) - (b.y - a.y) * (p.x - a.x)
15 };
16 // O(1)
17
18 // READ: distancia euclidiana entre dos puntos
19 static double distEuclidiana(const Punto& p, const Punto& q)
20 {
21     double dx = q.x - p.x, dy = q.y - p.y;
22     return sqrt(dx * dx + dy * dy);
23 } // O(1)
24
25 // READ: distancia de p a la RECTA INFINITA que pasa por a y
26 b
27 static double distARecta(const Punto& a, const Punto& b,
28 const Punto& p) {
29     double baseLen = distEuclidiana(a, b);
30     if (baseLen < 1e-9) return distEuclidiana(a, p); // a y b
31     coinciden
32     return fabs(cruz(a, b, p)) / baseLen;
33 } // O(1)
34
35 // READ: distancia de p al SEGMENTO [a, b] (no a la recta
36 extendida)
37 static double distASegmento(const Punto& a, const Punto& b,
38 const Punto& p) {
39     double t = punto(a, b, p);
40     if (t <= 0) return distEuclidiana(a, p); // proyeccion cae
41     antes de a
42     double baseLen2 = punto(a, b, b); // |ab|^2
43     if (t >= baseLen2) return distEuclidiana(b, p); //
44     proyeccion cae despues de b
45     return distARecta(a, b, p); // proyeccion cae dentro de [a,
46 b]
47 } // O(1)
48
49 //
50
51 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
52 //
53
54 // 1. Punto de proyeccion exacto (no solo la distancia): una
55 // vez que se
56 // sabe que la proyeccion cae dentro de [a,b], el punto
57 // proyectado es
58 // a + (t / baseLen2) * (b - a), con t = punto(a,b,p).
59 // 2. Coordenadas enteras: punto() y cruz() dan resultado
60 // exacto con
61 // long long; solo distEuclidiana y la division final
62 // requieren double.
63 // 3. Distancia de un punto a una POLILINEA (varios segmentos
64 // consecutivos):
65 // tomar el minimo de distASegmento sobre cada segmento
66 // consecutivo de
67 // la polilínea, O(k) para k segmentos.
68 };

```

Ejemplo de funcionamiento:

```

1  DistanciaPuntoLinea::Punto a{0, 0}, b{10, 0}, p{5, 3}, q{15,
2  3};
3
4  double d1 = DistanciaPuntoLinea::distASegmento(a, b, p);
5  // d1 = 3 (la proyeccion de p cae dentro de [a,b], en (5,0);
6  // distancia perpendicular = 3)
7
8  double d2 = DistanciaPuntoLinea::distASegmento(a, b, q);
9  // d2 = distEuclidiana(b, q) = sqrt(5^2 + 3^2) = sqrt(34)
10 // (la proyeccion de q cae despues de b={10,0}, asi que la
11 // distancia real es
12 // al extremo b, no la perpendicular a la recta; hay que
13 // correrlo para
14 // confirmar el valor decimal exacto de sqrt(34))

```

Regla práctica: usa `distARecta` solo cuando el problema explícitamente hable de una línea infinita (poco común en CP); en la gran mayoría de los casos donde se menciona un "segmento", "pared" o "borde", usa `distASegmento`, que maneja correctamente los casos donde la proyección cae fuera del segmento —olvidar esta

distinción es la fuente de error más común en problemas de distancia punto-segmento.

28.10 Reflexión de un Punto sobre una Línea

Idea: reflejar un punto P sobre una recta definida por A y B significa encontrar su "imagen espejo" al otro lado de la recta, a la misma distancia perpendicular. El procedimiento se apoya directamente en *Distancia de un Punto a una Línea/Segmento*: primero se calcula la proyección ortogonal Q de P sobre la recta (usando el producto punto para hallar t , igual que en el cálculo de distancia a segmento), y luego el punto reflejado es $P' = 2Q - P$ —es decir, Q es el punto medio entre P y su reflejo P' .

¿Por qué usarlo? Es la forma directa de resolver problemas de simetría sin recurrir a matrices de transformación generales o a trigonometría explícita —basta con una proyección (producto punto) y una resta vectorial. A diferencia de la *Rotación de un Punto*, que gira alrededor de un pivote por un ángulo arbitrario, la reflexión es una transformación fija determinada completamente por la recta de simetría, sin parámetro de ángulo.

Casos de uso:

- Problemas de "camino más corto que toca una pared" (reflejar el destino y trazar una línea recta).
- Verificar si una figura es simétrica respecto a un eje dado.
- Generar el punto opuesto en construcciones geométricas con simetría axial.
- Simulación de rebotes de un rayo de luz o trayectoria en un espejo/pared recta.

Complejidad: $O(1)$ —una proyección (producto punto) más una resta vectorial.

```

1 // CREATE/READ: Reflexion de un punto sobre una recta definida
  // por dos puntos
2 struct ReflexionPunto {
3     struct Punto { double x, y; };
4
5     // READ: producto punto de (b-a) con (p-a)
6     static double punto(const Punto& a, const Punto& b, const
  // Punto& p) {
7         return (b.x - a.x) * (p.x - a.x) + (b.y - a.y) * (p.y - a.y)
8     };
9     } // O(1)
10
11 // READ: proyeccion ortogonal de p sobre la recta que pasa
  // por a y b
12 static Punto proyeccion(const Punto& a, const Punto& b, const
  // Punto& p) {
13     double baseLen2 = punto(a, b, b); // |ab|^2
14     double t = punto(a, b, p) / baseLen2;
15     return Punto{a.x + t * (b.x - a.x), a.y + t * (b.y - a.y)};
16 } // O(1)
17
18 // READ: reflexion de p sobre la recta que pasa por a y b
19 static Punto reflejar(const Punto& a, const Punto& b, const
  // Punto& p) {
20     Punto q = proyeccion(a, b, p);
21     return Punto{2 * q.x - p.x, 2 * q.y - p.y};
22 } // O(1)
23
24 //
25 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
26 //
27 // 1. Reflexion sobre el eje x o eje y (casos particulares):
  // reflejar
  // sobre el eje x es simplemente (p.x, -p.y); sobre el eje
  // y es

```

```

28 // (-p.x, p.y). Mas rapido que usar el caso general si el
  // eje de
29 // simetria coincide con uno de los ejes cartesianos.
30 // 2. Camino mas corto que toca una linea (problema clasico):
  // para ir de
31 // P a Q tocando la recta L, se refleja Q (o P) sobre L
  // con reflejar(),
32 // y la distancia minima es distEuclidiana(P, reflejo de Q
  // ); el punto
33 // de contacto es la interseccion entre el segmento P-
  // reflejo(Q) y L.
34 // 3. Reflexion sobre un PUNTO (no una recta): caso distinto
  // y mas simple,
35 // P' = 2*C - P donde C es el punto de reflexion (equivale
  // a una
36 // rotacion de 180 grados alrededor de C, ver Rotacion de
  // un Punto).
37 };

```

Ejemplo de funcionamiento:

```

1 ReflexionPunto::Punto a{0, 0}, b{10, 0}, p{3, 4};
2
3 auto resultado = ReflexionPunto::reflejar(a, b, p);
4 // resultado = (3, -4)
5 // (la recta a-b es el eje x; reflejar (3,4) sobre el eje x da
  // (3,-4))

```

Nota: si la recta de reflexión coincide con uno de los ejes cartesianos, el caso particular mencionado en la modificación 1 del código es más directo y evita el cálculo de proyección completo —pero el método general funciona igual de bien para cualquier recta arbitraria.

Regla práctica: usa reflexión cuando el problema tenga simetría axial fija respecto a una recta conocida (más común en problemas de "camino más corto tocando una pared"); si en cambio se necesita girar un punto un ángulo arbitrario alrededor de un pivote, sin relación a ninguna recta de simetría, usa *Rotación de un Punto* en su lugar.

28.11 Rotación de un Punto (por Ángulo, alrededor de un Pivote)

Idea: rotar un punto P un ángulo θ alrededor de un pivote C se hace en tres pasos: trasladar P para que C quede en el origen ($P - C$), aplicar la matriz de rotación estándar $\begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$ sobre ese vector trasladado, y trasladar de vuelta sumando C . La matriz de rotación en sí sale de proyectar el vector sobre una nueva base girada θ radianes —exactamente la misma idea de descomponer en componentes que aparece en *Vector Geométrico*, aplicada a los vectores base $(1, 0)$ y $(0, 1)$ en vez de a un vector arbitrario.

¿Por qué usarlo? Es la transformación general de la que *Reflexión de un Punto sobre una Línea* es un caso especial no lineal (la reflexión no es una rotación, pero comparten la técnica de trasladar-transformar-destrasladar). A diferencia de la reflexión, que está fijada por una recta de simetría, la rotación tiene un parámetro libre (θ) y un pivote libre (C), lo que la hace la herramienta correcta cuando el problema pide girar figuras completas, no reflejarlas.

Casos de uso:

- Girar un polígono completo un ángulo dado alrededor de su centroide u otro pivote.
- Generar los vértices de un polígono regular rotando un vértice inicial en incrementos de $2\pi/n$.

- Simular el movimiento angular de un objeto (radar, aguja, brazo robótico) en pasos discretos.
- Normalizar la orientación de una figura antes de compararla con otra (rotar hasta alinear un eje de referencia).

Complejidad: $O(1)$ por rotación de un punto —dos evaluaciones trigonométricas (sin, cos) reutilizables si se rota el mismo ángulo varias veces, más un par de multiplicaciones y sumas.

```

1 // CREATE/READ: Rotacion de un punto un angulo theta alrededor
  // de un pivote
2 struct RotacionPunto {
3     struct Punto { double x, y; };
4
5     // READ: rota p un angulo theta (en radianes, sentido
      // antihorario)
6     // alrededor del pivote c
7     static Punto rotar(const Punto& p, const Punto& c, double
      theta) {
8         double dx = p.x - c.x;
9         double dy = p.y - c.y;
10        double cosT = cos(theta);
11        double sinT = sin(theta);
12        double xRot = dx * cosT - dy * sinT;
13        double yRot = dx * sinT + dy * cosT;
14        return Punto{c.x + xRot, c.y + yRot};
15    } // O(1)
16
17    //
18    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
19    //
20    // 1. Rotar alrededor del origen (caso particular, c = (0,0)):
      // se puede
21    // omitir la traslacion inicial y final, usando
      // directamente
22    // xRot = p.x*cosT - p.y*sinT; yRot = p.x*sinT + p.y*cosT.
23    // 2. Rotaciones repetidas del MISMO angulo theta muchas
      // veces (por
24    // ejemplo, generar los n vertices de un poligono regular):
      // calcular
25    // cosT y sinT una sola vez fuera del bucle, en vez de
      // recalcularlos
26    // en cada llamada a rotar().
27    // 3. Sentido horario en vez de antihorario: usar theta
      // negativo, o
28    // equivalentemente intercambiar el signo de sinT en ambas
      // formulas
29    // (dx*(-sinT) para xRot, dx*sinT invertido para yRot).
30    // 4. Poligono regular de n lados con radio r centrado en c:
      // generar
31    // vertices como rotar(Punto{c.x + r, c.y}, c, 2*M_PI*i/n)
      // para
32    // i = 0 hasta n-1.
33 };

```

Ejemplo de funcionamiento:

```

1 RotacionPunto::Punto p{1, 0}, c{0, 0};
2
3 auto resultado = RotacionPunto::rotar(p, c, M_PI / 2);
4 // resultado = (0, 1)
5 // (rotar (1,0) 90 grados antihorario alrededor del origen da
  // (0,1);
6 // hay que correrlo para confirmar el valor decimal exacto por
  // el uso
7 // de M_PI en punto flotante)

```

Nota: si el pivote coincide con el origen, omitir la traslación (ver modificación 1) ahorra dos sumas y dos restas por punto —insignificante para un solo punto, pero relevante si se rotan miles de puntos en un bucle.

Regla práctica: usa rotación cuando el problema pida girar un punto o figura un ángulo arbitrario alrededor de un pivote arbitrario; si en cambio el problema tiene una recta de simetría fija y pide el "espejo" de un punto respecto a ella, eso no es una rotación sino *Reflexión de un Punto sobre una Línea* —ambas comparten la técnica de trasladar-transformar-destrasladar, pero resuelven preguntas distintas.

29 Triángulos

29.1 Ángulos de un Triángulo

Idea: dados los tres vértices A, B, C de un triángulo, el ángulo interno en cada vértice se calcula con *Ángulo entre Dos Vectores* aplicado a los dos lados que se encuentran en ese vértice: el ángulo en A es el ángulo entre $\vec{AB}$ y $\vec{AC}$, y análogamente para B y C . Alternativamente, si solo se conocen las tres longitudes de los lados a, b, c (sin coordenadas), se puede usar la **ley de cosenos** despejada: $\cos A = \frac{b^2 + c^2 - a^2}{2bc}$, donde el lado a es el opuesto al ángulo A . Ambos métodos son equivalentes; cuál usar depende de si el problema da coordenadas o solo longitudes.

¿Por qué usarlo? Cuando se tienen coordenadas, usar *Ángulo entre Dos Vectores* (con `atan2`) es preferible porque da el ángulo con signo y es numéricamente estable, como ya se explicó en esa subsección. Cuando solo se tienen las tres longitudes de los lados —sin coordenadas en el plano—, la ley de cosenos es la única opción directa, y conviene usar `acos` ahí porque el ángulo interno de un triángulo válido siempre está en $[0, \pi]$, rango donde `acos` no tiene la ambigüedad de signo que sí le importa a un ángulo con orientación.

Casos de uso:

- Clasificar un triángulo como acutángulo, rectángulo u obtusángulo según sus ángulos.
- Verificar si tres longitudes dadas forman un triángulo válido (todos los ángulos resultantes deben ser reales, es decir, el argumento de `acos` debe estar en $[-1, 1]$).
- Calcular ángulos internos como paso previo a fórmulas de área o de triangulación que los requieran.
- Determinar la orientación relativa de dos lados que comparten un vértice.

Complejidad: $O(1)$ por ángulo —ya sea vía *Ángulo entre Dos Vectores* ($O(1)$, un `atan2`) o vía ley de cosenos ($O(1)$, un `acos`).

```

1 // CREATE/READ: Angulos internos de un triangulo, por
  // coordenadas o por
2 // longitudes de lados (ley de cosenos)
3 struct AngulosTriangulo {
4     struct Punto { double x, y; };
5
6     // READ: producto punto de (b-a) con (c-a)
7     static double punto(const Punto& a, const Punto& b, const
      Punto& c) {
8         return (b.x - a.x) * (c.x - a.x) + (b.y - a.y) * (c.y - a.y)
9         );
10    } // O(1)
11
12    // READ: producto cruz de (b-a) con (c-a)
13    static double cruz(const Punto& a, const Punto& b, const
      Punto& c) {
14        return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x)
15        );
16    } // O(1)
17
18    // READ: angulo interno en el vertice a, dado por los otros
      // dos vertices
19    // b y c, usando atan2 (estable, sin importar el rango)
20    static double anguloPorCoordenadas(const Punto& a, const
      Punto& b, const Punto& c) {
21        return fabs(atan2(cruz(a, b, c), punto(a, b, c)));
22    } // O(1)
23
24    // READ: angulo opuesto al lado de longitud "opuesto", dados
      // los otros
25    // dos lados "lado1" y "lado2" (ley de cosenos despejada)

```

```

24 static double anguloPorLados(double lado1, double lado2,
25 double opuesto) {
26     double cosAngulo = (lado1*lado1 + lado2*lado2 - opuesto*
27     opuesto) / (2*lado1*lado2);
28     cosAngulo = max(-1.0, min(1.0, cosAngulo)); // recorta
29     error de punto flotante
30     return acos(cosAngulo);
31 } // O(1)
32
33 // -----
34 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
35 // -----
36 // 1. Verificar validez de un triangulo dados tres lados:
37 // ademas de la
38 // desigualdad triangular (cada lado menor que la suma de
39 // los otros
40 // dos), el recorte de cosAngulo a [-1,1] en
41 // anguloPorLados evita NaN
42 // por error de punto flotante en casos borde (triangulo
43 // degenerado).
44 // 2. Clasificar el triangulo: si el mayor angulo es > pi/2
45 // es obtusangulo,
46 // == pi/2 (con tolerancia EPS) es rectangulo, < pi/2 es
47 // acutangulo;
48 // basta con calcular el angulo opuesto al lado mas largo.
49 // 3. Suma de angulos internos: por construccion siempre da
50 // pi (180
51 // grados) en geometria euclidiana; sirve como chequeo de
52 // cordura
53 // (sanity check) al depurar, calculando los tres angulos
54 // y sumando.
55 };

```

Ejemplo de funcionamiento:

```

1 AngulosTriangulo::Punto a{0, 0}, b{4, 0}, c{0, 3};
2
3 double anguloA = AngulosTriangulo::anguloPorCoordenadas(a, b, c
4 );
5 // anguloA = pi/2 (aprox 1.5708)
6 // (triangulo rectangulo 3-4-5, el angulo recto esta en a; hay
7 // que correrlo
8 // para confirmar el valor decimal exacto)
9
10 double anguloPorLados = AngulosTriangulo::anguloPorLados(4, 3,
11 5);
12 // anguloPorLados = pi/2, mismo resultado via ley de cosenos
13 // con los lados
14 // adyacentes a A (4 y 3) y el lado opuesto a A (la hipotenusa,
15 // 5)

```

Regla práctica: si tienes coordenadas de los vértices, usa `anguloPorCoordenadas` (reutiliza *Ángulo entre Dos Vectores*); si solo tienes las tres longitudes de los lados, usa `anguloPorLados` y no olvides recortar el argumento de `acos` a $[-1, 1]$ para evitar NaN por error de punto flotante en triángulos casi degenerados.

29.2 Fórmula de Herón (Área por Lados)

Idea: dado un triángulo del que solo se conocen las longitudes de sus tres lados a, b, c (sin coordenadas), la fórmula de Herón calcula el área sin necesidad de conocer ningún ángulo: con el semiperímetro $s = \frac{a+b+c}{2}$, el área es $\sqrt{s(s-a)(s-b)(s-c)}$. Es un resultado puramente algebraico —se puede derivar combinando la ley de cosenos (para obtener un ángulo) con la fórmula de área $\frac{1}{2}ab \sin C$, pero el resultado final elimina la trigonometría por completo y deja todo en función de las tres longitudes.

¿Por qué usarlo? Cuando el problema da coordenadas de los vértices, *Área de un Polígono (Shoelace)* es más directo y general (funciona para cualquier polígono, no solo triángulos). Herón es la herramienta correcta específicamente cuando **no hay coordenadas**, solo las tres longitudes de los lados —situación común en problemas donde el triángulo se describe por sus medidas, no por su posición en el plano.

Casos de uso:

- Calcular el área de un triángulo dado solo por sus tres lados (sin coordenadas).
- Verificar si tres longitudes forman un triángulo válido (el área resultante debe ser un número real positivo, es decir, el producto bajo la raíz debe ser ≥ 0).
- Como paso intermedio en fórmulas que requieren área y perímetro juntos (por ejemplo, el radio del círculo inscrito, $r = \text{área}/s$).
- Cálculo de área en problemas donde los datos de entrada son distancias medidas, no coordenadas cartesianas.

Complejidad: $O(1)$ —tres sumas/restas para el semiperímetro y los factores, una multiplicación de cuatro términos, y una raíz cuadrada.

```

1 // CREATE/READ: Area de un triangulo dado por sus tres lados (
2 // formula de Heron)
3 struct FormulaHeron {
4     // READ: area del triangulo de lados a, b, c; retorna -1 si
5     // los lados
6     // no forman un triangulo valido (desigualdad triangular
7     // violada)
8     static double area(double a, double b, double c) {
9         if (a + b <= c || a + c <= b || b + c <= a) return -1; //
10         triangulo invalido
11         double s = (a + b + c) / 2.0;
12         double producto = s * (s - a) * (s - b) * (s - c);
13         if (producto < 0) producto = 0; // recorta error de punto
14         // flotante
15         return sqrt(producto);
16     } // O(1)
17
18 // -----
19 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
20 // -----
21 // 1. Radio del circulo inscrito: r = area(a,b,c) / s, con
22 // s = (a+b+c)/2 el mismo semiperimetro usado internamente.
23
24 // 2. Radio del circulo circunscrito: R = (a*b*c) / (4*area(a,
25 // b,c)),
26 // formula derivada de la ley de senos generalizada (ver
27 // Circulo
28 // Circunscrito en un Triangulo para el caso con
29 // coordenadas).
30 // 3. Coordenadas disponibles en vez de lados: si el
31 // triangulo viene dado
32 // por vertices, es mas directo y mas preciso usar
33 // Shoelace
34 // directamente en vez de calcular las tres distancias
35 // euclidianas y
36 // despues aplicar Heron (evita acumular error en un paso
37 // intermedio).
38 };

```

Ejemplo de funcionamiento:

```

1 double resultado = FormulaHeron::area(3, 4, 5);
2 // resultado = 6
3 // (triangulo rectangulo 3-4-5, area = base*altura/2 = 3*4/2 =
4 // 6,
5 // Heron da el mismo resultado sin usar el angulo recto
6 // explicitamente)

```

Regla práctica: usa Herón únicamente cuando el triángulo se te da por sus tres longitudes sin coordenadas; si tienes coordenadas de los vértices, usa *Área de un Polígono (Shoelace)* directamente —es igual de rápido, más general, y evita el paso intermedio de calcular tres distancias euclidianas antes de aplicar Herón.

29.3 Círculo Circunscrito en un Triángulo

Idea: el círculo circunscrito (circumcírculo) de un triángulo es el único círculo que pasa por sus tres vértices. Su centro, el **circuncentro**, es el punto equidistante de los tres vértices, y se encuentra como la intersección de las mediatrices (perpendiculares por el punto medio) de cualquier par de lados. Resolviendo el sistema de ecuaciones que impone "distancia igual a los tres vértices" se llega a una fórmula cerrada en términos de determinantes 2×2 , sin necesidad de intersectar rectas explícitamente. El radio es simplemente la distancia del circuncentro a cualquiera de los tres vértices.

¿Por qué usarlo? A diferencia del caso particular de un triángulo equilátero —donde el circuncentro coincide con el centroide, el promedio simple de los tres vértices, $O(1)$ trivial—, un triángulo arbitrario no tiene esa coincidencia: centroide, circuncentro, incentro y ortocentro son puntos distintos en general. La fórmula con determinantes cubre el caso general sin necesidad de distinguir tipos de triángulo, y evita resolver el sistema de mediatrices manualmente (que requeriría manejar el caso de mediatriz vertical aparte, el mismo problema que motivó la forma general de *Ecuación de una Línea Dados Dos Puntos*).

Casos de uso:

- Encontrar el centro y radio del círculo mínimo que pasa por tres puntos dados (no necesariamente el círculo mínimo que los *cubre*, ver nota).
- Problemas de triangulación de Delaunay, donde la propiedad definitoria es que ningún otro punto cae dentro del circumcírculo de cada triángulo.
- Construcciones geométricas clásicas (bisectores perpendiculares, geometría de triángulos).
- Verificar si cuatro puntos son concíclicos (el cuarto debe estar exactamente sobre el circumcírculo de los otros tres).

Complejidad: $O(1)$ —el sistema de ecuaciones se resuelve con una fórmula cerrada de determinantes 2×2 , sin iteración.

```

1 // CREATE/READ: Circuncentro y radio del círculo circunscrito
2 // de un triángulo
3 // (caso general, válido para cualquier triángulo no degenerado)
4
5 struct CirculoCircunscrito {
6     struct Punto { double x, y; };
7
8     // READ: producto cruz de (b-a) x (c-a), usado para detectar
9     // triángulo degenerado
10    static double cruz(const Punto& a, const Punto& b, const
11    Punto& c) {
12        return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
13    };
14    // // O(1)
15
16    // READ: calcula el circuncentro de a, b, c mediante fórmula
17    // de
18    // determinantes (intersección de mediatrices resuelta
19    // algebraicamente)
20    static Punto circuncentro(const Punto& a, const Punto& b,
21    const Punto& c) {
22        double d = 2.0 * (a.x * (b.y - c.y) + b.x * (c.y - a.y) + c
23        .x * (a.y - b.y));
24        // d == 0 significa que a, b, c son colineales: no hay
25        // circuncentro
26        double ax2ay2 = a.x*a.x + a.y*a.y;
27        double bx2by2 = b.x*b.x + b.y*b.y;
28        double cx2cy2 = c.x*c.x + c.y*c.y;
29        double ux = (ax2ay2 * (b.y - c.y) + bx2by2 * (c.y - a.y) +
30        cx2cy2 * (a.y - b.y)) / d;

```

```

20    double uy = (ax2ay2 * (c.x - b.x) + bx2by2 * (a.x - c.x) +
21    cx2cy2 * (b.x - a.x)) / d;
22    return Punto{ux, uy};
23    } // O(1)
24
25    // READ: radio del círculo circunscrito (distancia del
26    // circuncentro a
27    // cualquier vértice, aquí se usa a)
28    static double radio(const Punto& a, const Punto& b, const
29    Punto& c) {
30        Punto o = circuncentro(a, b, c);
31        double dx = a.x - o.x, dy = a.y - o.y;
32        return sqrt(dx*dx + dy*dy);
33    } // O(1)
34
35    // -----
36    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
37    // -----
38
39    // 1. Caso equilátero (atajo, NO usar para triángulos
40    // generales): el
41    // circuncentro coincide con el centroide, ((a.x+b.x+c.x)/
42    // 3,
43    // (a.y+b.y+c.y)/3), y el radio es lado / sqrt(3). Solo
44    // válido cuando
45    // se garantiza que el triángulo es equilátero.
46    // 2. Triángulo degenerado (colineal): si cruz(a,b,c) == 0 (
47    // con EPS), d
48    // en circuncentro() da 0 y la fórmula produce división
49    // por cero /
50    // NaN; verificar colinealidad ANTES de llamar
51    // circuncentro().
52    // 3. Radio via lados y área (alternativa sin circuncentro
53    // explícito):
54    // R = (lado_a * lado_b * lado_c) / (4 * área),
55    // reutilizando
56    // Fórmula de Heron para el área si solo se necesita el
57    // radio y no
58    // el centro exacto.
59    };

```

Ejemplo de funcionamiento:

```

1 CirculoCircunscrito::Punto a{0, 0}, b{4, 0}, c{0, 3};
2
3 auto centro = CirculoCircunscrito::circuncentro(a, b, c);
4 // centro = (2, 1.5)
5 // (triángulo rectángulo 3-4-5; el circuncentro de un triángulo
6 // rectángulo
7 // siempre cae exactamente en el punto medio de la hipotenusa,
8 // aquí el
9 // punto medio de b=(4,0) y c=(0,3) es (2, 1.5), lo cual
10 // confirma el resultado)
11
12 double r = CirculoCircunscrito::radio(a, b, c);
13 // r = 2.5
14 // (para un triángulo rectángulo, el radio circunscrito es
15 // siempre la
16 // mitad de la hipotenusa: 5/2 = 2.5)

```

Nota: asumí que necesitas el caso general (no solo equilátero, que era tu ejemplo original) porque es estrictamente más útil en CP —el caso equilátero queda como atajo $O(1)$ documentado en la modificación 1 del código. Si en algún problema se garantiza que el triángulo es equilátero, usa ese atajo en vez de la fórmula de determinantes completa.

Regla práctica: usa esta fórmula de determinantes para el circuncentro general; verifica primero que el triángulo no sea degenerado ($\text{cruz}(a,b,c) \neq 0$) para evitar división por cero. Si solo necesitas el radio y ya tienes las tres longitudes de los lados (por ejemplo, después de calcular *Fórmula de Herón*), la fórmula $R = \frac{abc}{4 \cdot \text{área}}$ es más directa y evita calcular el centro explícitamente.

29.4 Punto Dentro de un Triángulo

Idea: un punto P está dentro del triángulo ABC si y solo si queda del **mismo lado** de las tres aristas orientadas AB , BC , CA —es decir, si el signo de *Orientación de Tres Puntos* aplicado a (A, B, P) , (B, C, P) y (C, A, P) es

el mismo para los tres (todos positivos o todos negativos, según el sentido de recorrido del triángulo). Si alguno de los tres productos cruz da signo distinto a los demás, P cae fuera de al menos una arista y por lo tanto fuera del triángulo.

¿Por qué usarlo? Es la aplicación más directa y económica de *Orientación de Tres Puntos* a un caso concreto: tres evaluaciones de producto cruz, sin necesidad de trigonometría, áreas parciales, o coordenadas baricéntricas explícitas (aunque el método es equivalente a verificar que las coordenadas baricéntricas de P sean todas no negativas). Es un caso particular y más simple de *Punto Dentro de un Polígono*, que en general requiere *ray casting*; para un triángulo (siempre convexo), tres chequeos de orientación bastan y son más baratos.

Casos de uso:

- Verificar pertenencia de un punto a un triángulo, como paso base antes de generalizar a polígonos convexos.
- Localización de punto en una triangulación (mesh), recorriendo triángulos y probando cuál lo contiene.
- Pruebas de colisión simples en gráficos 2D o videojuegos con geometría triangular.
- Interpolación baricéntrica (una vez confirmado que el punto está dentro, los mismos productos cruz dan los pesos baricéntricos).

Complejidad: $O(1)$ —tres evaluaciones de producto cruz y una comparación de signos.

```

1 // CREATE/READ: Verifica si un punto esta dentro de un
  // triangulo mediante
2 // tres pruebas de orientacion (mismo signo en las tres aristas
  // )
3 struct PuntoDentroTriangulo {
4     struct Punto { double x, y; };
5
6     // READ: producto cruz de (b-a) x (c-a)
7     static double cruz(const Punto& a, const Punto& b, const
8     Punto& c) {
9         return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x
10        );
11     } // O(1)
12
13     // READ: true si p esta dentro o sobre el borde del triangulo
14     // abc,
15     // sin importar si abc esta orientado CW o CCW
16     static bool dentro(const Punto& a, const Punto& b, const
17     Punto& c, const Punto& p) {
18         double d1 = cruz(a, b, p);
19         double d2 = cruz(b, c, p);
20         double d3 = cruz(c, a, p);
21         const double EPS = 1e-9;
22         bool tieneNegativo = (d1 < -EPS) || (d2 < -EPS) || (d3 < -
23         EPS);
24         bool tienePositivo = (d1 > EPS) || (d2 > EPS) || (d3 > EPS)
25         ;
26         // si NO hay a la vez signos positivos y negativos, p esta
27         // dentro
28         // (o sobre el borde, si algun di es aproximadamente 0)
29         return !(tieneNegativo && tienePositivo);
30     } // O(1)
31
32 //
33
34 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
35 //
36
37 // 1. Distinguir "estrictamente dentro" de "sobre el borde":
38 // si se
39 // necesita excluir el borde, verificar ademas que ninguno
40 // de d1, d2,
41 // d3 este dentro del rango [-EPS, EPS] (es decir, que los
42 // tres sean
43 // estrictamente del mismo signo, sin ceros).
44 // 2. Coordenadas enteras: usar long long en cruz() y
45 // comparar contra 0

```

```

33 // exacto en vez de EPS, ya que el resultado es exacto con
34 // enteros.
35 // 3. Coordenadas baricentricas: una vez calculados d1, d2,
36 // d3 (todas del
37 // mismo signo), los pesos baricentricos normalizados son
38 // d2/(d1+d2+d3), d3/(d1+d2+d3), d1/(d1+d2+d3) para a, b,
39 // c
40 // respectivamente (util para interpolar valores definidos
41 // en los
42 // vertices, como en shading de graficos 3D).
43 };

```

Ejemplo de funcionamiento:

```

1 PuntoDentroTriangulo::Punto a{0, 0}, b{4, 0}, c{0, 4}, p{1, 1},
  q{3, 3};
2
3 bool dentroP = PuntoDentroTriangulo::dentro(a, b, c, p);
4 // dentroP = true (p=(1,1) esta dentro del triangulo)
5
6 bool dentroQ = PuntoDentroTriangulo::dentro(a, b, c, q);
7 // dentroQ = false (q=(3,3) esta fuera, mas alla de la
  hipotenusa b-c)

```

Regla práctica: usa esta prueba de tres orientaciones para triángulos específicamente —es más barata que *Punto Dentro de un Polígono (Ray Casting)*, que es la herramienta general para polígonos con cualquier número de lados. Si el triángulo forma parte de una triangulación más grande, esta misma prueba es la base para localizar en qué triángulo cae un punto dado.

30 Círculos

30.1 Longitud de Arco y Área de un Sector Circular

Idea: un sector circular es la "porción de pizza" delimitada por dos radios y el arco entre ellos, definida por un ángulo central θ (en radianes) sobre un círculo de radio r . Como la circunferencia completa ($2\pi r$ de longitud, πr^2 de área) corresponde a un ángulo completo de 2π radianes, cualquier sector es simplemente una fracción proporcional: la longitud de arco es $L = r\theta$ y el área del sector es $A = \frac{1}{2}r^2\theta$. Ambas fórmulas se derivan de la misma razón $\theta/2\pi$ aplicada a la circunferencia y al área totales, respectivamente.

¿Por qué usarlo? Es la forma más directa de estas dos cantidades cuando se trabaja con ángulos en radianes —evita el paso intermedio de convertir a grados y aplicar reglas de tres, y generaliza sin casos especiales a cualquier ángulo entre 0 y 2π . Vale la pena notar la diferencia de exponente entre ambas fórmulas: la longitud de arco escala linealmente con r , mientras que el área escala con r^2 —el mismo patrón que la circunferencia ($2\pi r$) y el área del círculo completo (πr^2).

Casos de uso:

- Calcular la distancia recorrida por un objeto que se mueve sobre un arco (radar, engranaje, trayectoria circular).
- Área de una región delimitada por dos radios y un arco, como paso intermedio en problemas de intersección de círculos.
- Problemas de "porción de pizza" o reparto proporcional de un círculo.
- Cálculo de área de un segmento circular (sector menos el triángulo que forman los dos radios y la cuerda), usado en *Intersección de Dos Círculos*.

Complejidad: $O(1)$ —una multiplicación para cada fórmula.

```

1 // CREATE/READ: Longitud de arco y area de un sector circular
  dado el radio
2 // y el angulo central en radianes
3 struct SectorCircular {
4     // READ: longitud de arco para un sector de radio r y angulo
      theta (radianes)
5     static double longitudArco(double r, double theta) {
6         return r * theta;
7     } // O(1)
8
9     // READ: area del sector de radio r y angulo theta (radianes)
10    static double areaSector(double r, double theta) {
11        return 0.5 * r * r * theta;
12    } // O(1)
13
14    // READ: area del segmento circular (sector menos el
      triangulo r-r-cuerda),
15    // la region entre el arco y la cuerda que lo subtiende
16    static double areaSegmento(double r, double theta) {
17        return areaSector(r, theta) - 0.5 * r * r * sin(theta);
18    } // O(1)
19
20    //
21    // -----
22    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
23    // -----
24    // 1. Angulo dado en grados: convertir primero a radianes con
      // theta_rad = theta_grados * M_PI / 180.0 antes de llamar
      cualquiera
25    // de las funciones.
26    // 2. Perimetro total del sector (no solo el arco):
      longitudArco(r, theta)
27    // + 2*r, sumando los dos radios que cierran la "porcion
      de pizza".
28    // 3. area Segmento requiere theta en [0, 2*pi]; si theta >
      pi (segmento
29    // "mayor", mas de medio circulo), la resta sigue siendo
      valida porque
30    // sin(theta) ya maneja el signo correctamente para ese
      rango.
31 };

```

Ejemplo de funcionamiento:

```

1 double r = 4.0, theta = M_PI / 2; // sector de 90 grados
2
3 double arco = SectorCircular::longitudArco(r, theta);
4 // arco = 2*pi (aprox 6.2832)
5 // (un cuarto de la circunferencia completa 2*pi*r = 8*pi; hay
      que correrlo
6 // para confirmar el valor decimal exacto)
7
8 double area = SectorCircular::areaSector(r, theta);
9 // area = 4*pi (aprox 12.566)
10 // (un cuarto del area total pi*r^2 = 16*pi; hay que correrlo
      para
11 // confirmar el valor decimal exacto)

```

Regla práctica: usa estas fórmulas directamente cuando el ángulo esté en radianes y conozcas el radio; el área del segmento circular (arco menos cuerda) es el bloque reutilizable que hace falta como paso intermedio en *Intersección de Dos Círculos*, donde el área de solape se descompone en dos segmentos circulares, uno por cada círculo.

30.2 Intersección de Dos Círculos

Idea: dos círculos de centros C_1, C_2 y radios r_1, r_2 , separados por una distancia $d = |C_1C_2|$, se intersectan en 0, 1, 2 puntos, o son el mismo círculo, dependiendo de cómo se comparan d , $r_1 + r_2$ y $|r_1 - r_2|$. Cuando hay dos puntos de intersección, se encuentran resolviendo el sistema de las dos ecuaciones de circunferencia: se calcula la distancia a desde C_1 hasta el punto medio de la cuerda que une las dos intersecciones (usando el teorema de Pitágoras generalizado, análogo a la ley de cosenos), luego la altura h de ese punto medio a cada intersección, y finalmente se desplaza perpendicularmente desde ese

punto medio usando un vector normal a C_1C_2 —la misma idea de "rotar 90 grados" que aparece en *Ecuación de una Línea Dados Dos Puntos*.

¿Por qué usarlo? Es un problema con varios casos borde que hay que descartar explícitamente antes de aplicar la fórmula: círculos demasiado separados ($d > r_1 + r_2$), uno dentro del otro sin tocarse ($d < |r_1 - r_2|$), tangencia externa o interna (exactamente un punto), y círculos concéntricos con igual radio (infinitos puntos, caso degenerado). Manejar estos casos aparte, antes de la fórmula general, evita raíces cuadradas de números negativos o divisiones por cero cuando $d = 0$.

Casos de uso:

- Encontrar los puntos de intersección entre dos círculos, como paso base en geometría de círculos.
- Calcular el área de solape entre dos círculos (usando *Longitud de Arco y Área de un Sector Circular* sobre cada uno).
- Problemas de cobertura de señal o rango (¿dos zonas circulares se superponen?).
- Construcciones geométricas clásicas (encontrar puntos equidistantes a dos centros dados, base de la construcción de la mediatriz con regla y compás).

Complejidad: $O(1)$ —un número constante de operaciones aritméticas y, en el peor caso, dos llamadas a `sqrt` y dos a `acos` (para el área de solape).

```

1 // CREATE/READ: Interseccion de dos circulos: puntos de corte y
  area de
2 // solape (union de dos segmentos circulares)
3 struct InterseccionCirculos {
4     struct Punto { double x, y; };
5
6     static double dist(const Punto& a, const Punto& b) {
7         double dx = b.x - a.x, dy = b.y - a.y;
8         return sqrt(dx*dx + dy*dy);
9     } // O(1)
10
11    // READ: clasifica la relacion entre los dos circulos ANTES
      de calcular
12    // puntos de interseccion, para evitar sqrt de numero
      negativo
13    // retorna: 0 = separados (sin interseccion), 1 = uno dentro
      del otro
14    // (sin tocarse), 2 = tangentes (1 punto), 3 = se cruzan (2
      puntos),
15    // 4 = concentricos con mismo radio (infinitos puntos, mismo
      circulo)
16    static int clasificar(const Punto& c1, double r1, const Punto
      & c2, double r2) {
17        double d = dist(c1, c2);
18        const double EPS = 1e-9;
19        if (d < EPS && fabs(r1 - r2) < EPS) return 4; // mismo
      circulo
20        if (d > r1 + r2 + EPS) return 0; // separados
21        if (d < fabs(r1 - r2) - EPS) return 1; // uno
      dentro del otro
22        if (fabs(d - (r1 + r2)) < EPS || fabs(d - fabs(r1 - r2)) <
      EPS) return 2; // tangentes
23        return 3; // se cruzan en dos puntos
24    } // O(1)
25
26    // READ: calcula los (hasta) dos puntos de interseccion; SOLO
      valido
27    // cuando clasificar() devuelve 2 o 3 (llamar clasificar
      primero)
28    static vector<Punto> puntosInterseccion(const Punto& c1,
      double r1, const Punto& c2, double r2) {
29        double d = dist(c1, c2);
30        // a = distancia de c1 al punto medio de la cuerda de
      interseccion
31        double a = (r1*r1 - r2*r2 + d*d) / (2*d);
32        double hCuad = r1*r1 - a*a;
33        double h = (hCuad < 0) ? 0 : sqrt(hCuad); // recorta error
      de punto flotante
34        // punto medio de la cuerda, sobre la linea c1-c2

```

```

35 double mx = c1.x + a * (c2.x - c1.x) / d;
36 double my = c1.y + a * (c2.y - c1.y) / d;
37 // vector perpendicular a (c2 - c1), normalizado por h/d
38 double dx = (c2.y - c1.y) * (h / d);
39 double dy = -(c2.x - c1.x) * (h / d);
40 if (h < 1e-9) return { Punto{mx, my} }; // tangentes: un
    solo punto
41 return { Punto{mx + dx, my + dy}, Punto{mx - dx, my - dy}
    };
42 } // O(1)
43
44 // READ: area de solape (union de dos segmentos circulares);
    asume que
45 // clasificar() ya confirmo caso 3 (se cruzan) o 1 (uno
    dentro del otro)
46 static double areaSolape(const Punto& c1, double r1, const
    Punto& c2, double r2) {
47     double d = dist(c1, c2);
48     int caso = clasificar(c1, r1, c2, r2);
49     if (caso == 0) return 0;
    // sin solape
50     if (caso == 1) return M_PI * min(r1, r2) * min(r1, r2);
    // uno dentro del otro
51     // angulo central en c1 y en c2 subtendido por la cuerda
    comun
52     double theta1 = 2 * acos(max(-1.0, min(1.0, (r1*r1 + d*d -
    r2*r2) / (2*r1*d))));
53     double theta2 = 2 * acos(max(-1.0, min(1.0, (r2*r2 + d*d -
    r1*r1) / (2*r2*d))));
54     double seg1 = 0.5 * r1*r1 * (theta1 - sin(theta1));
55     double seg2 = 0.5 * r2*r2 * (theta2 - sin(theta2));
56     return seg1 + seg2;
57 } // O(1)
58
59 //
60 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
61 //
62 // 1. SIEMPRE llamar clasificar() antes que
    puntosInterseccion() o
63 // areaSolape(): estas dos ultimas asumen que ya se
    descarto el caso
64 // de circulos separados o concentricos con mismo radio,
    de lo
65 // contrario hay division por cero (d == 0) o resultados
    sin sentido.
66 // 2. areaSolape reutiliza directamente areaSegmento de
    Longitud de Arco
67 // y Area de un Sector Circular: seg1 y seg2 son
    exactamente esa
68 // formula aplicada a theta1 y theta2 respectivamente.
69 // 3. Radio y distancia con coordenadas enteras: d, a, h
    siguen
70 // requiriendo double por las raices cuadradas
    involucradas; no hay
71 // forma de mantener este calculo en aritmetica entera
    exacta.
72 };

```

Ejemplo de funcionamiento:

```

1 InterseccionCirculos::Punto c1{0, 0}, c2{4, 0};
2 double r1 = 3, r2 = 3;
3
4 int caso = InterseccionCirculos::clasificar(c1, r1, c2, r2);
5 // caso = 3 (se cruzan en dos puntos, ya que |r1-r2|<0 < d=4 <
    r1+r2=6)
6
7 auto puntos = InterseccionCirculos::puntosInterseccion(c1, r1,
    c2, r2);
8 // puntos = {(2, h), (2, -h)} con h = sqrt(3^2 - 2^2) = sqrt(5)
9 // (por simetria, a = (9-9+16)/8 = 2, el punto medio de la
    cuerda cae en
10 // x=2; hay que correrlo para confirmar el valor decimal
    exacto de h)

```

Nota: puntosInterseccion y areaSolape no validan internamente los casos degenerados (círculos separados, uno dentro del otro sin tocarse, o concéntricos) — es responsabilidad de quien llama invocar clasificar() primero y actuar según el resultado, tal como se indica en la modificación 1 dentro del código.

Regla práctica: nunca llames directamente a puntosInterseccion o calcules el área de solape sin antes clasificar la relación entre los dos círculos con clasificar() —los casos borde (separados, uno dentro del otro, concéntricos) producen NaN o resultados incor-

rectos si se ignoran, y son la fuente de error más común en este tipo de problema.

31 Polígonos

31.1 Área de un Polígono (Shoelace)

Idea: dado un polígono simple con vértices $P_0, P_1, \dots, P_{n-1}$ en orden (sentido horario o antihorario, consecutivos a lo largo del contorno), el área con signo es $\frac{1}{2} \sum_{i=0}^{n-1} (x_i y_{i+1} - x_{i+1} y_i)$, tomando los índices módulo n para cerrar el polígono. Cada término $x_i y_{i+1} - x_{i+1} y_i$ es exactamente el producto cruz $\vec{OP_i} \times \vec{OP_{i+1}}$ visto en *Orientación de Tres Puntos*, tomando el origen como pivote implícito: la fórmula suma las áreas con signo de los triángulos que forma el origen con cada arista consecutiva, y las contribuciones se cancelan fuera del polígono y se acumulan dentro, dejando exactamente el área encerrada.

¿Por qué usarlo? Es la generalización directa de *Fórmula de Herón* a cualquier polígono simple (no solo triángulos), y no requiere que el polígono sea convexo. A diferencia de triangular manualmente el polígono y sumar áreas de triángulos individuales, Shoelace procesa los vértices en un único recorrido lineal sin necesidad de descomponer la figura. El signo del resultado antes de tomar valor absoluto además revela la orientación del polígono: positivo si los vértices están en sentido antihorario (CCW), negativo si horario (CW) —la misma convención fijada en *Orientación de Tres Puntos*.

Casos de uso:

- Calcular el área de cualquier polígono simple (convexo o no) dado por sus vértices en orden.
- Determinar la orientación (CW o CCW) de una secuencia de puntos, usando el signo antes del valor absoluto.
- Validar que un polígono generado por otro algoritmo (Convex Hull, recorte) tiene la orientación esperada.
- Paso base para el Teorema de Pick (área en malla de puntos enteros).

Complejidad: $O(n)$ —un recorrido lineal por los n vértices, sumando un término constante por cada uno.

```

1 // CREATE/READ: Area de un poligono simple mediante la formula
    de Shoelace
2 struct AreaPoligono {
3     struct Punto { double x, y; };
4
5     // READ: area CON SIGNO del poligono (positivo = CCW,
        negativo = CW);
6     // vertices deben estar en orden a lo largo del contorno
7     static double areaConSigno(const vector<Punto>& vertices) {
8         int n = (int)vertices.size();
9         double suma = 0;
10        for (int i = 0; i < n; i++) {
11            const Punto& actual = vertices[i];
12            const Punto& siguiente = vertices[(i + 1) % n];
13            suma += actual.x * siguiente.y - siguiente.x * actual.y;
14        }
15        return suma / 2.0;
16    } // O(n)
17
18    // READ: area absoluta del poligono (sin importar orientacion)
19    static double area(const vector<Punto>& vertices) {
20        return fabs(areaConSigno(vertices));
21    } // O(n)

```

```

22 // READ: true si los vertices estan en sentido antihorario (
23 CCW)
24 static bool esCCW(const vector<Punto>& vertices) {
25     return areaConSigno(vertices) > 0;
26 } // O(n)
27
28 //
29 // -----
30 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
31 // -----
32 // 1. Coordenadas enteras: la suma interna se puede mantener
33 // en long long
34 // (el area con signo * 2 siempre es entera si las
35 // coordenadas son
36 // enteras), dividiendo entre 2.0 solo al final; util para
37 // Teorema de
38 // Pick, que necesita el area exacta sin error de punto
39 // flotante.
40 // 2. Poligono con hoyos: el area total es el area del
41 // contorno exterior
42 // (CCW) menos el area de cada hoyo interior (dado en
43 // sentido CW);
44 // sumar areaConSigno() de cada contorno respeta esto
45 // automaticamente
46 // si las orientaciones son consistentes.
47 // 3. Verificar que el poligono es simple (sin
48 // autointersecciones) NO lo
49 // hace esta formula: Shoelace da un resultado numerico
50 // igual si el
51 // poligono se autointersecta, pero ese resultado ya no
52 // representa el
53 // area real de una region simple.
54 };

```

Ejemplo de funcionamiento:

```

1 vector<AreaPoligono::Punto> cuadrado = {{0,0}, {4,0}, {4,4},
2 {0,4}};
3
4 double resultado = AreaPoligono::area(cuadrado);
5 // resultado = 16
6 // (cuadrado de lado 4, area = 4*4 = 16; los vertices estan en
7 // orden CCW)
8
9 bool ccw = AreaPoligono::esCCW(cuadrado);
10 // ccw = true

```

Regla práctica: usa Shoelace como la herramienta por defecto para área de polígono siempre que tengas coordenadas —es más general que Herón (funciona para cualquier número de lados, no solo triángulos) y no requiere que el polígono sea convexo. Aprovecha el signo (antes de `fabs`) cuando necesites confirmar o forzar una orientación específica, como en *Envolvente Convexa* o *Punto Dentro de un Polígono Convexo*, que dependen de esa convención.

31.2 Perímetro de un Polígono

Idea: el perímetro de un polígono con vértices $P_0, \dots, P_{n-1}$ en orden es simplemente la suma de las distancias euclidianas entre cada par de vértices consecutivos a lo largo del contorno, cerrando de vuelta del último vértice al primero: $\sum_{i=0}^{n-1} d(P_i, P_{i+1 \bmod n})$. Es la aplicación directa de *Distancia Euclidiana* en un recorrido lineal, sin ninguna sutileza adicional —a diferencia de área (Shoelace) o de la unión de rectángulos, el perímetro no depende de orientación ni de convexidad.

¿Por qué usarlo? Es el cálculo más simple de esta parte, pero se documenta aparte porque aparece constantemente como componente de fórmulas más grandes (razón isoperimétrica, radio del círculo inscrito vía $r = \text{área} / \text{semiperímetro}$, problemas de "cuánta cerca se necesita"). No tiene alternativa más eficiente: a diferencia de área, que puede calcularse con una fórmula cerrada tipo Shoelace, el perímetro siempre requiere sumar las n distancias, una por cada arista.

Casos de uso:

- Calcular la longitud total de una cerca, borde o contorno poligonal.
- Paso intermedio en el cálculo del radio del círculo inscrito (usando semiperímetro junto al área de Herón o Shoelace).
- Comparar la "compacidad" de una figura (razón isoperimétrica: $4\pi \cdot \text{área} / \text{perímetro}^2$).
- Validar la longitud de un polígono generado por Envolverte Convexa u otro algoritmo de construcción.

Complejidad: $O(n)$ —un recorrido lineal por los n vértices, con una distancia euclidiana ($O(1)$) por cada arista.

```

1 // CREATE/READ: Perimetro de un poligono mediante suma de
2 // distancias
3 // euclidianas entre vertices consecutivos
4 struct PerimetroPoligono {
5     struct Punto { double x, y; };
6
7     // READ: distancia euclidiana entre dos puntos
8     static double dist(const Punto& a, const Punto& b) {
9         double dx = b.x - a.x, dy = b.y - a.y;
10        return sqrt(dx*dx + dy*dy);
11    } // O(1)
12
13    // READ: perimetro total del poligono, sumando cada arista
14    // consecutiva
15    // (incluye la arista de cierre del ultimo vertice al primero)
16    static double perimetro(const vector<Punto>& vertices) {
17        int n = (int)vertices.size();
18        double total = 0;
19        for (int i = 0; i < n; i++) {
20            total += dist(vertices[i], vertices[(i + 1) % n]);
21        }
22        return total;
23    } // O(n)
24
25    //
26    // -----
27    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
28    // -----
29    // 1. Polilinea abierta (NO poligono cerrado): omitir la
30    // arista de cierre,
31    // recorriendo i de 0 a n-2 en vez de usar (i+1) % n.
32    // 2. Radio del circulo inscrito, dado el area (Shoelace o
33    // Heron):
34    // r = area / (perimetro(vertices) / 2.0), reutilizando el
35    // semiperimetro que ya calcula esta funcion internamente.
36    // 3. Razon isoperimetrica (que tan "circular" es la figura,
37    // maximo 1
38    // para un circulo perfecto): 4 * M_PI * area / (perimetro
39    // *perimetro).
40 };

```

Ejemplo de funcionamiento:

```

1 vector<PerimetroPoligono::Punto> cuadrado = {{0,0}, {4,0},
2 {4,4}, {0,4}};
3
4 double resultado = PerimetroPoligono::perimetro(cuadrado);
5 // resultado = 16
6 // (cuadrado de lado 4, cuatro lados de longitud 4 cada uno:
7 // 4*4 = 16)

```

Regla práctica: usa esta suma directa siempre que necesites la longitud del contorno; si el problema en realidad da una **polilínea abierta** (no un polígono cerrado), recuerda omitir la arista de cierre entre el último y el primer vértice (ver modificación 1 dentro del código), ya que incluirla por error es un bug común al reutilizar esta misma función para ambos casos.

31.3 Punto Dentro de un Polígono (Ray Casting)

Idea: para determinar si un punto P está dentro de un polígono simple arbitrario (convexo o no), *ray casting* traza un rayo desde P hacia el infinito en una dirección fija (típicamente horizontal) y cuenta cuántas aristas del polígono cruza ese rayo. Si el número de cruces es **impar**, P está dentro; si es **par**, está fuera —la intuición es que cada vez que el rayo atraviesa el borde, cruza de "afuera" a "adentro" o viceversa, así que empezando afuera (en el infinito), un número impar de cruces termina adentro. Esta idea, conocida como el teorema de la curva de Jordan aplicado a polígonos, funciona sin importar la orientación del polígono ni si es convexo.

¿Por qué usarlo? A diferencia de *Punto Dentro de un Triángulo* (tres pruebas de orientación) o de *Punto Dentro de un Polígono Convexo* (búsqueda binaria angular, más adelante), ray casting es el único método de esta parte que funciona para polígonos **no convexos** (cóncavos, con "entrantes"). Su costo es más alto — $O(n)$ en vez de $O(1)$ o $O(\log n)$ — porque no puede aprovechar ninguna estructura especial: cada arista debe revisarse individualmente sin importar la forma del polígono.

Casos de uso:

- Verificar pertenencia de un punto a un polígono arbitrario, incluyendo formas cóncavas o con auto-similaridad compleja.
- Point-in-polygon para regiones geográficas o mapas con contornos irregulares.
- Rasterización de polígonos (determinar qué píxeles de una grilla caen dentro de una figura).
- Validación general cuando no se puede garantizar que el polígono sea convexo.

Complejidad: $O(n)$ —se revisa cada una de las n aristas del polígono exactamente una vez, con una comprobación $O(1)$ por arista.

```
1 // CREATE/READ: Punto dentro de un poligono simple (convexo o
2 // no) mediante
3 // ray casting (conteo de cruces con un rayo horizontal)
4 struct PuntoDentroPoligono {
5     struct Punto { double x, y; };
6
7     // READ: true si el punto p esta dentro del poligono (borde
8     // EXCLUIDO,
9     // ver Nota); funciona para poligonos convexos y concavos
10    static bool dentro(const vector<Punto>& poligono, const Punto
11    & p) {
12        int n = (int)poligono.size();
13        bool adentro = false;
14        for (int i = 0, j = n - 1; i < n; j = i++) {
15            const Punto& a = poligono[i];
16            const Punto& b = poligono[j];
17            // revisa si el rayo horizontal desde p hacia +x cruza la
18            // arista (a,b)
19            bool cruzaEnY = (a.y > p.y) != (b.y > p.y);
20            if (cruzaEnY) {
21                double xCruce = a.x + (p.y - a.y) * (b.x - a.x) / (b.y
22                - a.y);
23                if (p.x < xCruce) adentro = !adentro;
24            }
25        }
26        return adentro;
27    } // O(n)
28
29    // READ: true si el punto p esta exactamente SOBRE el borde
30    // del poligono
31    // (sobre alguna arista); combinar con dentro() para "dentro
32    // o sobre el borde"
33    static bool sobreBorde(const vector<Punto>& poligono, const
34    Punto& p) {
35        int n = (int)poligono.size();
```

```
28 const double EPS = 1e-9;
29 for (int i = 0; i < n; i++) {
30     const Punto& a = poligono[i];
31     const Punto& b = poligono[(i + 1) % n];
32     double cruz = (b.x - a.x) * (p.y - a.y) - (b.y - a.y) * (
33     p.x - a.x);
34     if (fabs(cruz) > EPS) continue; // no colineal con esta
35     // arista
36     bool dentroX = min(a.x, b.x) - EPS <= p.x && p.x <= max(a
37     .x, b.x) + EPS;
38     bool dentroY = min(a.y, b.y) - EPS <= p.y && p.y <= max(a
39     .y, b.y) + EPS;
40     if (dentroX && dentroY) return true;
41 }
42 return false;
43 } // O(n)
44
45 //
46 // -----
47 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
48 // -----
49 // 1. "Dentro o sobre el borde" (inclusivo): dentro(poligono,
50 // p) ||
51 // sobreBorde(poligono, p) -- necesario porque dentro()
52 // por si solo
53 // da resultado indefinido/inconsistente cuando p cae
54 // exactamente
55 // sobre una arista (el rayo puede o no contarla como
56 // cruce segun
57 // redondeo de punto flotante).
58 // 2. Coordenadas enteras: xCruce y la comparacion p.x <
59 // xCruce se
60 // pueden reescribir con aritmetica de enteros
61 // multiplicando en cruz,
62 // evitando la division y el error de punto flotante por
63 // completo.
64 // 3. Poligono con hoyos (contorno exterior + contornos
65 // interiores en
66 // sentido opuesto): correr dentro() sobre el contorno
67 // exterior Y
68 // sobre cada hoyo; el punto esta en la region valida solo
69 // si esta
70 // dentro del exterior y fuera de TODOS los hoyos.
71 };
```

Ejemplo de funcionamiento:

```
1 vector<PuntoDentroPoligono::Punto> estrella = {
2     {0,4}, {1,1}, {4,0}, {1,-1}, {0,-4}, {-1,-1}, {-4,0}, {-1,1}
3 };
4 PuntoDentroPoligono::Punto p{0, 0};
5
6 bool resultado = PuntoDentroPoligono::dentro(estrella, p);
7 // resultado = true
8 // (p={0,0} esta en el centro de una figura concava tipo
9 // estrella; ray
10 // casting funciona correctamente aqui donde las tres pruebas
11 // de
12 // orientacion de Punto Dentro de un Triangulo no aplicarian)
```

Nota: el resultado de `dentro()` para un punto exactamente sobre el borde es indefinido por construcción del algoritmo (depende de redondeo). Si el problema distingue "estrictamente dentro" de "sobre el borde" de "fuera", usa `sobreBorde()` por separado antes de confiar en `dentro()`, tal como se muestra en la modificación 1 dentro del código.

Regla práctica: usa ray casting como la herramienta general por defecto para point-in-polygon —es la única de esta parte que funciona sin importar si el polígono es convexo. Si tienes garantizado que el polígono es convexo, *Punto Dentro de un Polígono Convexo* con búsqueda binaria es $O(\log n)$ y por lo tanto preferible cuando se hacen muchas consultas sobre el mismo polígono.

31.4 Punto Dentro de un Polígono Convexo (Binary Search)

Idea: cuando el polígono es convexo y está dado en orden CCW (la convención fijada en *Orientación de Tres Puntos*), se puede aprovechar esa estructura para responder en $O(\log n)$ en vez de $O(n)$: se fija el vértice 0

como pivote, y los vértices $1, \dots, n-1$ quedan ordenados angularmente alrededor de él por ser convexo. Una búsqueda binaria encuentra el "abanico" (los dos vértices consecutivos P_i, P_{i+1}) entre los que cae el ángulo de P respecto a P_0 , y basta con una prueba de *Punto Dentro de un Triángulo* sobre el triángulo (P_0, P_i, P_{i+1}) para confirmar si P está dentro.

¿Por qué usarlo? Frente a ray casting ($O(n)$, funciona para cualquier polígono), esta técnica explota la convexidad para bajar a $O(\log n)$ por consulta —una mejora significativa cuando se hacen muchas consultas sobre el mismo polígono fijo (por ejemplo, q consultas dan $O(n + q \log n)$ en vez de $O(nq)$). La condición es estricta: el polígono debe ser convexo y estar en el orden CCW consistente con el resto de la guía; si cualquiera de esas dos condiciones falla, el resultado es incorrecto sin ningún aviso.

Casos de uso:

- Muchas consultas de pertenencia sobre el mismo polígono convexo fijo (por ejemplo, la salida de *Envolvente Convexa*).
- Verificación rápida de colisión punto-polígono en aplicaciones con restricciones de tiempo estrictas.
- Localización de un punto dentro de una región convexa en problemas de particionamiento del plano.
- Como subrutina dentro de algoritmos más grandes que requieren muchas pruebas de pertenencia (por ejemplo, intersección de semiplanos).

Complejidad: $O(\log n)$ por consulta tras un preprocesamiento $O(1)$ (no requiere ordenar nada adicional, ya que el polígono convexo en CCW ya viene angularmente ordenado respecto a cualquiera de sus vértices).

```

1 // CREATE/READ: Punto dentro de un poligono CONVEXO mediante
2 // angular; REQUIERE que el poligono este en orden CCW (ver
3 // Nota)
4 struct PuntoDentroPoligonoConvexo {
5     struct Punto { double x, y; };
6
7     // READ: producto cruz de (b-a) x (c-a)
8     static double cruz(const Punto& a, const Punto& b, const
9     Punto& c) {
10         return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
11     }; // O(1)
12
13     // READ: true si p esta dentro (o sobre el borde) del
14     // poligono convexo,
15     // dado en orden CCW; usa busqueda binaria sobre el abanico
16     // desde poligono[0]
17     static bool dentro(const vector<Punto>& poligono, const Punto
18     & p) {
19         int n = (int)poligono.size();
20         const Punto& p0 = poligono[0];
21         const double EPS = 1e-9;
22
23         // p debe estar entre los rayos p0->poligono[1] y p0->
24         // poligono[n-1]
25         double cruzInicial = cruz(p0, poligono[1], p);
26         double cruzFinal = cruz(p0, poligono[n-1], p);
27         if (cruzInicial < -EPS || cruzFinal > EPS) return false; //
28         fuera del abanico
29
30         // busqueda binaria del indice i tal que p cae en el
31         // triangulo
32         // (p0, poligono[i], poligono[i+1])
33         int lo = 1, hi = n - 1;
34         while (hi - lo > 1) {
35             int medio = (lo + hi) / 2;
36             if (cruz(p0, poligono[medio], p) >= -EPS) lo = medio;
37             else hi = medio;
38         }
39     }
40 }

```

```

31 // confirma que p esta dentro del triangulo final (p0,
32 // poligono[lo], poligono[hi])
33 double d1 = cruz(p0, poligono[lo], p);
34 double d2 = cruz(poligono[lo], poligono[hi], p);
35 double d3 = cruz(poligono[hi], p0, p);
36 return d1 >= -EPS && d2 >= -EPS && d3 >= -EPS;
37 } // O(log n)
38
39 //
40 // -----
41 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
42 // -----
43 // 1. Poligono en orden CW (horario): invertir el orden de
44 // los vertices
45 // (poligono.rbegin() a poligono.rend()) antes de usar, o
46 // negar todos
47 // los resultados de cruz() en el algoritmo -- NO usar
48 // directamente
49 // sin ajustar, el resultado seria incorrecto en silencio.
50 // 2. Excluir el borde (estrictamente dentro): cambiar todos
51 // los ">=" a ">"
52 // por "> EPS" en las comparaciones finales y en la
53 // busqueda binaria.
54 // 3. Muchas consultas sobre el mismo poligono: el
55 // preprocesamiento es
56 // O(1) (no hay que ordenar nada extra), asi que esta
57 // estructura es
58 // directamente reutilizable llamando dentro()
59 // repetidamente sin
60 // reconstruir nada.
61 }
62 }

```

Ejemplo de funcionamiento:

```

1 vector<PuntoDentroPoligonoConvexo::Punto> hexagono = {
2     {2,0}, {1,2}, {-1,2}, {-2,0}, {-1,-2}, {1,-2}
3 }; // hexagono convexo en orden CCW
4 PuntoDentroPoligonoConvexo::Punto p{0, 0}, q{3, 3};
5
6 bool dentroP = PuntoDentroPoligonoConvexo::dentro(hexagono, p);
7 // dentroP = true (el centro esta dentro del hexagono)
8
9 bool dentroQ = PuntoDentroPoligonoConvexo::dentro(hexagono, q);
10 // dentroQ = false (q esta fuera, mas alla de todos los
11 // vertices)

```

Nota: este algoritmo **asume orden CCW** sin verificarlo — si el polígono que produce *Envolvente Convexa* (Jarvis o Monotone Chain) no respeta esa orientación, esta búsqueda binaria falla en silencio, dando resultados incorrectos sin ningún error visible. Antes de usar esta técnica sobre la salida de un algoritmo de convex hull, confirma la orientación con *AreaPoligono::esCCW()* (ver *Área de un Polígono*) y, si es necesario, invierte el orden de los vértices.

Regla práctica: usa esta técnica en vez de ray casting únicamente cuando el polígono sea convexo Y esté en orden CCW confirmado, y cuando vayas a hacer múltiples consultas sobre el mismo polígono —la ganancia de $O(n)$ a $O(\log n)$ solo vale la pena si el número de consultas q es grande. Para una sola consulta o un polígono no convexo, usa *Punto Dentro de un Polígono (Ray Casting)*.

31.5 Teorema de Pick (Área en Malla de Puntos Enteros)

Idea: para un polígono simple cuyos vértices tienen coordenadas **enteras**, el teorema de Pick relaciona el área con el conteo de puntos enteros: $A = I + \frac{B}{2} - 1$, donde I es el número de puntos enteros estrictamente en el interior del polígono y B es el número de puntos enteros sobre el borde (incluyendo los vértices). Despejando, $I = A - \frac{B}{2} + 1$. El número de puntos enteros sobre una arista entre dos vértices (x_1, y_1) y (x_2, y_2) es $\gcd(|x_2 - x_1|, |y_2 - y_1|)$ —incluyendo un extremo pero no el otro,

para no contar los vértices dos veces al sumar todas las aristas.

¿Por qué usarlo? Es el puente entre *Área de un Polígono (Shoelace)* —que da el área geométrica continua— y el conteo combinatorio de puntos en una malla entera, algo que de otra forma requeriría iterar sobre una grilla completa ($O(\text{área})$, potencialmente enorme). Pick reduce ese conteo a $O(1)$ una vez que se tiene el área (Shoelace, $O(n)$) y el conteo de puntos de borde (suma de gcd por arista, también $O(n)$): la fórmula evita cualquier iteración sobre la malla misma, sin importar qué tan grande sea el polígono.

Casos de uso:

- Contar puntos de malla estrictamente interiores a un polígono de vértices enteros, sin iterar la grilla.
- Contar puntos de malla sobre el borde de un polígono (perímetro "discreto").
- Problemas que piden el área de una figura descrita en una grilla de puntos enteros (más simple que Shoelace + gcd cuando se da I y B directamente).
- Verificación cruzada: si se conocen I , B y el área por otro método, Pick sirve como chequeo de consistencia.

Complejidad: $O(n)$ —Shoelace para el área es $O(n)$, y sumar gcd sobre cada una de las n aristas es $O(n \log(\max \text{coordenada}))$ por el costo del algoritmo de Euclides, dominado en la práctica por el término lineal.

```

1 // CREATE/READ: Teorema de Pick: relaciona area, puntos
  interiores (I) y
2 // puntos de borde (B) para poligonos de vertices enteros
3 struct TeoremaPick {
4     struct Punto { long long x, y; };
5
6     // READ: area con signo * 2 mediante Shoelace en aritmetica
      entera
7     // (evita perder precision, ya que 2*area siempre es entero)
8     static long long dobleAreaConSigno(const vector<Punto>&
      vertices) {
9         int n = (int)vertices.size();
10        long long suma = 0;
11        for (int i = 0; i < n; i++) {
12            const Punto& actual = vertices[i];
13            const Punto& siguiente = vertices[(i + 1) % n];
14            suma += actual.x * siguiente.y - siguiente.x * actual.y;
15        }
16        return suma;
17    } // O(n)
18
19    // READ: numero de puntos enteros sobre el borde del poligono
20    // (suma de gcd por arista, cada vertice contado una sola vez)
21    static long long puntosBorde(const vector<Punto>& vertices) {
22        int n = (int)vertices.size();
23        long long total = 0;
24        for (int i = 0; i < n; i++) {
25            const Punto& a = vertices[i];
26            const Punto& b = vertices[(i + 1) % n];
27            total += __gcd(1llabs(b.x - a.x), 1llabs(b.y - a.y));
28        }
29        return total;
30    } // O(n log(max coordenada))
31
32    // READ: numero de puntos enteros ESTRICTAMENTE interiores,
33    // via Pick:
34    // I = A - B/2 + 1, usando doble area para mantener
35    // aritmetica entera
36    static long long puntosInteriores(const vector<Punto>&
37    vertices) {
38        long long dobleArea = 1llabs(dobleAreaConSigno(vertices));
39        long long b = puntosBorde(vertices);
40        // 2*I = 2*A - B + 2 = dobleArea - b + 2
41        return (dobleArea - b + 2) / 2;
42    } // O(n log(max coordenada))

```

```

41 //
42 // -----
43 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
44 // -----
45 // 1. Area como double a partir de I y B conocidos (Pick
      directo, sin
46 // Shoelace): area = I + B/2.0 - 1; util si el problema da
      I y B
47 // directamente y pide el area.
48 // 2. Vertices NO enteros: el teorema de Pick no aplica;
      requiere
49 // coordenadas enteras tanto para el conteo de gcd como
      para que I
50 // y B tengan sentido combinatorio.
51 // 3. Verificacion de que dobleArea - b es PAR antes de
      dividir entre 2:
52 // matematicamente siempre lo es para un poligono valido
      de vertices
53 // enteros, pero sirve como chequeo de cordura al depurar
      (si da
54 // impar, hay un bug en el calculo de area o de borde).
55 };

```

Ejemplo de funcionamiento:

```

1 vector<TeoremaPick::Punto> cuadrado = {{0,0}, {4,0}, {4,4},
      {0,4}};
2
3 long long b = TeoremaPick::puntosBorde(cuadrado);
4 // b = 16
5 // (cada lado tiene longitud 4, gcd(4,0)=4 por arista, 4
      aristas: 4*4=16;
6 // cada punto de borde se cuenta una sola vez, incluyendo los
      4 vertices)
7
8 long long i = TeoremaPick::puntosInteriores(cuadrado);
9 // i = 9
10 // (area = 16, Pick: I = 16 - 16/2 + 1 = 16 - 8 + 1 = 9;
11 // corresponde a la grilla interior 3x3 de puntos (1,1) a
      (3,3))

```

Nota: este teorema aplica **únicamente** a polígonos simples con vértices de coordenadas enteras. Si el problema da coordenadas reales (no enteras), no hay forma de aplicar Pick directamente —hay que recurrir a *Área de un Polígono (Shoelace)* para el área continua, sin ningún conteo de puntos de malla asociado.

Regla práctica: usa Pick cuando el problema combine dos de las tres cantidades (área, puntos interiores, puntos de borde) y pida la tercera, sobre un polígono de vértices enteros —evita iterar la grilla por completo, algo que sería inviable si el área es grande. Si necesitas el área pura sin relación con conteo de puntos, *Shoelace* solo (sin Pick) es más directo.

32 Envoltente Convexa

32.1 Algoritmo de Jarvis (Gift Wrapping)

Idea: Jarvis construye la envoltente convexa "envolviendo" el conjunto de puntos como si se estirara una banda elástica alrededor de ellos: partiendo del punto más bajo (garantizado en el hull), se busca repetidamente el siguiente vértice del hull como aquel punto tal que **todos** los demás puntos quedan a la izquierda del segmento actual —es decir, el punto que hace el giro más "hacia afuera" posible respecto al vértice anterior. Esto se decide con *Orientación de Tres Puntos*: para cada candidato se compara contra el mejor candidato actual, y se actualiza cuando el nuevo punto queda más a la derecha (en sentido de barrido) del candidato vigente. El proceso se repite hasta volver al punto de partida.

¿Por qué usarlo? Es el algoritmo de envoltente convexa más simple de razonar e implementar correctamente, porque en cada paso solo requiere una comparación local

(¿quién es más extremo?) sin necesidad de ordenar todos los puntos de antemano. Su costo, sin embargo, depende del número de vértices h que termina teniendo el hull, no solo de n : es preferible a *Monotone Chain* únicamente cuando se sabe de antemano que h va a ser pequeño comparado con n (envolvente con pocos vértices sobre una nube de puntos grande); en el peor caso, cuando casi todos los puntos están en el hull ($h \approx n$), Jarvis degenera a $O(n^2)$, peor que el $O(n \log n)$ garantizado de Monotone Chain.

Casos de uso:

- Construcción de la envolvente convexa cuando se espera que tenga pocos vértices.
- Problemas educativos o de referencia rápida donde la simplicidad de implementación importa más que la complejidad garantizada.
- Como paso dentro de algoritmos incrementales de geometría computacional (Quickhull usa una idea similar de forma recursiva).
- Verificación cruzada del resultado de *Monotone Chain* en casos pequeños durante debugging.

Complejidad: $O(nh)$, donde n es el número de puntos y h el número de vértices del hull resultante: cada uno de los h vértices del hull requiere una pasada de $O(n)$ para encontrar el siguiente punto más extremo. En el peor caso ($h = n$), esto es $O(n^2)$.

```

1 // CREATE/READ: Envolvente convexa mediante Jarvis (Gift
  Wrapping);
2 // retorna los vertices del hull en orden CCW
3 struct EnvolventeJarvis {
4     struct Punto { double x, y; };
5
6     // READ: producto cruz de (b-a) x (c-a); positivo = CCW (
  convencion
7     // fijada en Orientacion de Tres Puntos)
8     static double cruz(const Punto& a, const Punto& b, const
  Punto& c) {
9         return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x
  );
10    } // O(1)
11
12    // READ: distancia al cuadrado, usada para desempatar puntos
  colineales
13    static double distCuadrado(const Punto& a, const Punto& b) {
14        double dx = b.x - a.x, dy = b.y - a.y;
15        return dx*dx + dy*dy;
16    } // O(1)
17
18    // READ: construye el hull convexo de un conjunto de puntos,
  retornado
19    // en orden CCW; requiere n >= 3 para un hull no degenerado
20    static vector<Punto> construir(vector<Punto> puntos) {
21        int n = (int)puntos.size();
22        if (n < 3) return puntos; // hull degenerado (punto o
  segmento)
23
24        // punto inicial: el de menor y (y menor x en caso de
  empate),
25        // garantizado en el hull
26        int inicio = 0;
27        for (int i = 1; i < n; i++) {
28            if (puntos[i].y < puntos[inicio].y ||
29                (puntos[i].y == puntos[inicio].y && puntos[i].x < puntos[
  inicio].x)) {
30                inicio = i;
31            }
32        }
33
34        vector<Punto> hull;
35        int actual = inicio;
36        do {
37            hull.push_back(puntos[actual]);
38            int siguiente = (actual + 1) % n; // candidato inicial
  arbitrario
39            for (int i = 0; i < n; i++) {

```

```

40                if (i == actual) continue;
41                double c = cruz(puntos[actual], puntos[siguiente],
  puntos[i]);
42                // si i queda a la derecha del candidato actual, i es
  mejor;
43                // si son colineales, se prefiere el mas lejano
44                if (c < 0 || (c == 0 && distCuadrado(puntos[actual],
  puntos[i]) >
45                    distCuadrado(puntos[actual], puntos[siguiente]))) {
46                    siguiente = i;
47                }
48            }
49            actual = siguiente;
50        } while (actual != inicio);
51
52        return hull;
53    } // O(n*h): h iteraciones del do-while, cada una O(n)
54
55    //
56    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
57    //
58    // 1. Hull en sentido CW en vez de CCW: invertir el criterio
  "c < 0" a
59    //    "c > 0" en la comparacion del candidato; NO cambia la
  complejidad,
60    //    solo la orientacion de salida.
61    // 2. Incluir puntos colineales sobre el borde del hull: el
  desempate por
62    //    distCuadrado en el codigo actual EXCLUYE puntos
  colineales
63    //    intermedios (se queda con el mas lejano); para
  incluirlos, agregar
64    //    los puntos colineales entre actual y siguiente antes de
  avanzar.
65    // 3. Puntos duplicados en la entrada: eliminarlos antes de
  llamar
66    //    construir(), ya que pueden causar que el bucle do-while
  no termine
67    //    correctamente si el punto inicial se duplica.
68 };

```

Ejemplo de funcionamiento:

```

1 vector<EnvolventeJarvis::Punto> puntos = {
2     {0,0}, {4,0}, {4,4}, {0,4}, {2,2} // (2,2) esta estrictamente
  adentro
3 };
4
5 auto hull = EnvolventeJarvis::construir(puntos);
6 // hull = {(0,0), (4,0), (4,4), (0,4)}
7 // (el punto interior (2,2) queda excluido; los cuatro vertices
  del
8 // cuadrado forman el hull en orden CCW, empezando por el de
  menor y)

```

Regla práctica: usa Jarvis cuando sepas (o sospeches fuertemente) que el hull tendrá pocos vértices frente a n ; si no hay garantía sobre el tamaño del hull, o n es grande, usa *Monotone Chain* en su lugar, que garantiza $O(n \log n)$ sin importar cuántos vértices termine teniendo el hull.

32.2 Monotone Chain (Andrew)

Idea: en vez de "envolver" iterativamente como Jarvis, Monotone Chain ordena todos los puntos por x (desempataando por y) y construye el hull en dos pasadas lineales: una **cadena inferior** (*lower hull*), recorriendo los puntos ordenados de izquierda a derecha y manteniendo solo los que forman giros a la derecha (o eliminando los que forman giros a la izquierda, usando *Orientación de Tres Puntos* sobre una pila), y una **cadena superior** (*upper hull*) recorriendo en sentido inverso con el mismo criterio. Concatenar ambas cadenas (sin duplicar los extremos) da el hull completo en orden CCW.

¿Por qué usarlo? Garantiza $O(n \log n)$ en el peor caso, dominado por el ordenamiento inicial —a diferencia de Jarvis, cuyo costo depende de h y puede degenerar a $O(n^2)$. El mecanismo de "pila que elimina giros incorrectos" es el mismo patrón usado en muchos algoritmos de

geometría computacional con barrido (similar en espíritu al manejo de estado del *Line Sweep*), y es la opción por defecto en competencia cuando no hay garantía sobre el tamaño del hull resultante.

Casos de uso:

- Construcción de la envolvente convexa como caso general, sin garantías sobre h .
- Preprocesamiento antes de *Punto Dentro de un Polígono Convexo*, que requiere el hull en orden CCW.
- Problemas donde n es grande y se necesita la complejidad garantizada $O(n \log n)$.
- Base para algoritmos que requieren el hull como paso intermedio (diámetro del conjunto de puntos, rotating calipers).

Complejidad: $O(n \log n)$, dominado por el ordenamiento inicial de los n puntos; cada una de las dos cadenas (inferior y superior) se construye en $O(n)$ porque cada punto se inserta y se elimina de la pila a lo sumo una vez.

```

1 // CREATE/READ: Envolvente convexa mediante Monotone Chain (
  Andrew);
2 // retorna los vertices del hull en orden CCW, sin puntos
  colineales de sobra
3 struct EnvolventeMonotoneChain {
4     struct Punto { double x, y; };
5
6     // READ: producto cruz de (b-a) x (c-a); positivo = CCW
7     static double cruz(const Punto& a, const Punto& b, const
  Punto& c) {
8         return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x
  );
9     } // O(1)
10
11     // READ: construye el hull convexo en orden CCW; requiere n
    >= 3 para
12     // un hull no degenerado
13     static vector<Punto> construir(vector<Punto> puntos) {
14         int n = (int)puntos.size();
15         if (n < 3) return puntos;
16
17         sort(puntos.begin(), puntos.end(), [](const Punto& a, const
  Punto& b) {
18             if (a.x != b.x) return a.x < b.x;
19             return a.y < b.y;
20         });
21
22         vector<Punto> hull(2 * n);
23         int k = 0;
24
25         // cadena inferior: recorre de izquierda a derecha
26         for (int i = 0; i < n; i++) {
27             while (k >= 2 && cruz(hull[k-2], hull[k-1], puntos[i]) <=
  0) k--;
28             hull[k++] = puntos[i];
29         }
30
31         // cadena superior: recorre de derecha a izquierda,
    continuando
32         // desde donde quedo la cadena inferior
33         int limiteInferior = k + 1;
34         for (int i = n - 2; i >= 0; i--) {
35             while (k >= limiteInferior && cruz(hull[k-2], hull[k-1],
  puntos[i]) <= 0) k--;
36             hull[k++] = puntos[i];
37         }
38
39         hull.resize(k - 1); // el ultimo punto duplica el primero,
    se descarta
40         return hull;
41     } // O(n log n): dominado por el sort; ambas cadenas son O(n)
42
43     //
44     // MODIFICACIONES Y COMENTARIOS CON LOS USOS
45     //
46     // 1. Incluir puntos colineales sobre el borde del hull:
    cambiar la

```

```

47 // condicion "<= 0" por "< 0" en ambos while, de modo que
    los giros
48 // exactamente colineales NO se descarten de la pila.
49 // 2. Hull en sentido CW en vez de CCW: invertir "<= 0" a ">=
    0" en
50 // ambos while.
51 // 3. Puntos duplicados en la entrada: eliminarlos antes de
    ordenar
52 // (sort + unique) para evitar comportamiento inesperado
    en el
53 // desempate de cruz() == 0 con puntos identicos.
54 };

```

Ejemplo de funcionamiento:

```

1 vector<EnvolventeMonotoneChain::Punto> puntos = {
2     {0,0}, {4,0}, {4,4}, {0,4}, {2,2} // (2,2) esta estrictamente
    adentro
3 };
4
5 auto hull = EnvolventeMonotoneChain::construir(puntos);
6 // hull = {(0,0), (4,0), (4,4), (0,4)}
7 // (mismo resultado que Jarvis para este conjunto de puntos, en
    orden CCW;
8 // el punto interior (2,2) queda excluido por la condicion de
    giro)

```

Nota: tanto Jarvis como Monotone Chain producen el hull en orden **CCW** con la convención fijada en esta guía, así que el resultado de cualquiera de los dos es directamente compatible con *Punto Dentro de un Polígono Convexo*, que requiere esa orientación. Si se necesita orden CW, ver la modificación 2 en cualquiera de los dos structs.

Regla práctica: usa Monotone Chain como opción por defecto en competencia, salvo que tengas una razón específica para preferir Jarvis (hull con muy pocos vértices conocido de antemano, o simplicidad de implementación en un contexto donde n es pequeño). Su $O(n \log n)$ garantizado lo hace la opción más segura cuando no hay información previa sobre el tamaño del hull.

32.3 Corte de un Polígono Convexo por una Línea

Idea: dado un polígono convexo (por ejemplo, la salida de *Jarvis* o *Monotone Chain*) y una línea definida por dos puntos, esta operación produce el subpolígono resultante de quedarse solo con la parte del polígono que cae de un lado específico de la línea (el lado izquierdo, según la orientación de la línea). El algoritmo recorre los vértices en orden y usa *Orientación de Tres Puntos* para clasificar cada uno como dentro o fuera del semiplano deseado: cuando dos vértices consecutivos están en lados opuestos, se calcula el punto de intersección exacto de la arista con la línea de corte y se inserta en el resultado; los vértices que están del lado válido se conservan tal cual. Este es un caso particular del algoritmo de recorte de Sutherland-Hodgman, especializado a una sola línea de corte en vez de un polígono de recorte completo.

¿Por qué usarlo? A diferencia de *Intersección de Segmentos*, que solo detecta si dos segmentos se cruzan, aquí el objetivo es reconstruir una figura completa (el subpolígono resultante), lo que requiere combinar la prueba de orientación con la inserción cuidadosa de nuevos vértices en la intersección. Explota que el polígono de entrada es **convexo**: eso garantiza que el resultado del corte también es convexo (un semiplano intersectado con un convexo siempre da un convexo), lo cual no se cumpliría si el polígono de entrada fuera cóncavo —ahí el resultado

podría ser múltiples polígonos separados, un caso mucho más complejo no cubierto aquí.

Casos de uso:

- Calcular el área de un polígono convexo que queda a un lado de una recta (por ejemplo, "¿qué fracción del terreno queda al norte de esta carretera recta?").
- Recorte de figuras convexas contra los bordes de una ventana o región rectangular (aplicando esta operación cuatro veces, una por cada borde).
- Como subrutina en *intersección de semiplanos* (interseccionar un polígono convexo contra muchas líneas sucesivamente).
- Simulación de corte físico de una región convexa por una trayectoria recta.

Complejidad: $O(n)$ —un recorrido lineal por los n vértices del polígono, con una prueba de orientación $O(1)$ y, en el peor caso, un cálculo de intersección $O(1)$ por cada arista.

```

1 // CREATE/READ: Corta un poligono convexo por una linea,
  quedandose con
2 // la parte del lado IZQUIERDO de la linea (segun su
  orientacion a->b)
3 struct CortePoligonoConvexo {
4     struct Punto { double x, y; };
5
6     // READ: producto cruz de (b-a) x (c-a); positivo = c esta a
  la
7     // izquierda de a->b (convencion CCW de Orientacion de Tres
  Puntos)
8     static double cruz(const Punto& a, const Punto& b, const
  Punto& c) {
9         return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x)
10    };
11 } // O(1)
12
13 // READ: interseccion entre la arista (p, q) y la linea (a, b
  ),
14 // asumiendo que p y q estan en lados opuestos de la linea
15 static Punto interseccion(const Punto& p, const Punto& q,
  const Punto& a, const Punto& b) {
16     double d1 = cruz(a, b, p);
17     double d2 = cruz(a, b, q);
18     double t = d1 / (d1 - d2);
19     return Punto{p.x + t * (q.x - p.x), p.y + t * (q.y - p.y)};
20 } // O(1)
21
22 // READ: corta el poligono convexo, quedandose con el lado
  izquierdo
23 // de la linea a->b; retorna vector vacio si el poligono
  queda
24 // completamente del lado derecho
25 static vector<Punto> cortar(const vector<Punto>& poligono,
  const Punto& a, const Punto& b) {
26     int n = (int)poligono.size();
27     vector<Punto> resultado;
28     const double EPS = 1e-9;
29
30     for (int i = 0; i < n; i++) {
31         const Punto& actual = poligono[i];
32         const Punto& siguiente = poligono[(i + 1) % n];
33         double dActual = cruz(a, b, actual);
34         double dSiguiente = cruz(a, b, siguiente);
35
36         if (dActual >= -EPS) resultado.push_back(actual); //
  actual esta dentro (o sobre la linea)
37         // si actual y siguiente estan en lados opuestos, agregar
  el punto de corte
38         if ((dActual > EPS && dSiguiente < -EPS) || (dActual < -
  EPS && dSiguiente > EPS)) {
39             resultado.push_back(interseccion(actual, siguiente, a,
  b));
40         }
41     }
42     return resultado;
43 } // O(n)
44
45 // MODIFICACIONES Y COMENTARIOS CON LOS USOS

```

```

46 // -----
47 // 1. Quedarse con el lado DERECHO en vez del izquierdo:
  invertir el
48 // criterio "dActual >= -EPS" a "dActual <= EPS" (y
  ajustar el signo
49 // en la deteccion de cruce si se desea mayor claridad).
50 // 2. Area del subpoligono resultante: aplicar directamente
  Shoelace
51 // (ver Area de un Poligono) sobre el vector retornado por
  cortar().
52 // 3. Recorte contra una ventana rectangular (Sutherland-
  Hodgman
53 // completo): aplicar cortar() cuatro veces sucesivas, una
  por cada
54 // borde del rectangulo (orientados consistentemente en
  CCW), usando
55 // el resultado de cada corte como entrada del siguiente.
56 };

```

Ejemplo de funcionamiento:

```

1 vector<CortePoligonoConvexo::Punto> cuadrado = {{0,0}, {4,0},
  {4,4}, {0,4}};
2 CortePoligonoConvexo::Punto a{2,-1}, b{2,5}; // linea vertical
  x=2
3
4 auto resultado = CortePoligonoConvexo::cortar(cuadrado, a, b);
5 // resultado = {(2,0), (4,0), (4,4), (2,4)}
6 // (se conserva el lado izquierdo de la linea a->b, que en este
  caso
7 // apunta hacia +y, dejando el lado con x >= 2; el cuadrado
  original de
8 // area 16 queda cortado a un rectangulo de area 8)

```

Nota: el lado que se conserva depende de la **orientación** de la línea de corte ($a \rightarrow b$), no solo de su posición geométrica —invertir a y b invierte qué mitad del polígono se conserva. Esto es consistente con la convención de *Orientación de Tres Puntos*: positivo significa "a la izquierda de $a \rightarrow b$ ", y es responsabilidad de quien llama orientar la línea según el lado que quiere conservar.

Regla práctica: usa esta operación cuando necesitas el subpolígono resultante de un corte, no solo si dos figuras se cruzan —para eso te alcanza *Intersección de Segmentos*. Encadenar varias llamadas a `cortar()` (una por cada línea) es la forma directa de resolver intersección de semiplanos o recorte contra una ventana convexa arbitraria, siempre que el polígono de entrada sea convexo.

33 Ad-hoc y Combinatoria Geométrica

33.1 Perímetro Usando Bloques (Figuras en Grilla)

Idea: dada una figura formada por bloques cuadrados unitarios sobre una grilla (por ejemplo, una matriz de "*" y "." indicando qué celdas están ocupadas), el perímetro no requiere geometría continua: cada bloque ocupado contribuye hasta 4 unidades de perímetro (una por cada uno de sus cuatro lados), pero cada vez que dos bloques ocupados son vecinos (comparten un lado), ese lado compartido deja de ser borde exterior y se resta **dos veces** —una unidad de cada bloque—. La fórmula resultante es $P = 4B - 2A$, donde B es el número total de bloques ocupados y A es el número de pares de bloques ocupados adyacentes (compartiendo un lado, no solo una esquina).

¿Por qué usarlo? A diferencia de *Perímetro de un Polígono*, que requiere las coordenadas explícitas del contorno, aquí la entrada es una grilla discreta y no hace falta reconstruir el contorno geométrico en absoluto —basta con contar bloques y pares adyacentes, una

operación puramente combinatoria sobre la matriz. Intentar resolverlo trazando el contorno real (como si fuera un polígono) sería mucho más laborioso que este conteo directo.

Casos de uso:

- Calcular el perímetro de una figura compuesta por celdas de una grilla (islas en un mapa, piezas de Tetris, regiones pintadas).
- Problemas de "área de una isla" en matrices binarias, donde también se pide el perímetro de cada componente.
- Verificación rápida de cuántas aristas de borde tiene una región antes de un recorrido BFS/DFS más complejo.
- Cálculo de perímetro total de múltiples figuras separadas sobre la misma grilla (sumando por componente conexa).

Complejidad: $O(NM)$ para una grilla de $N \times M$ —se recorre cada celda una vez, y por cada bloque ocupado se revisan sus hasta 4 vecinos, cada revisión $O(1)$.

```

1 // CREATE/READ: Perímetro de una figura formada por bloques en
  una grilla
2 struct PerimetroBloques {
3     vector<string> grilla;
4     int filas, columnas;
5
6     // CREATE: recibe la grilla como vector de strings, '*' =
      ocupado,
7     // '.' = vacío
8     PerimetroBloques(const vector<string>& g) : grilla(g) {
9         filas = (int)grilla.size();
10        columnas = filas > 0 ? (int)grilla[0].size() : 0;
11    } // O(1), no copia mas alla de la referencia inicial
12
13    // READ: true si la celda (fila, col) esta dentro de la
      grilla Y ocupada
14    bool estaOcupada(int fila, int col) const {
15        if (fila < 0 || fila >= filas || col < 0 || col >= columnas)
16            return false;
17        return grilla[fila][col] == '*';
18    } // O(1)
19
20    // READ: perimetro total de todos los bloques ocupados en la
      grilla
21    long long calcularPerimetro() const {
22        long long perimetro = 0;
23        int df[4] = {-1, 1, 0, 0};
24        int dc[4] = {0, 0, -1, 1};
25        for (int f = 0; f < filas; f++) {
26            for (int c = 0; c < columnas; c++) {
27                if (!estaOcupada(f, c)) continue;
28                for (int dir = 0; dir < 4; dir++) {
29                    // cada lado del bloque (f,c) cuenta como borde SOLO
30                    si
31                    // el vecino en esa direccion no esta ocupado (o esta
32                    // fuera de la grilla)
33                    if (!estaOcupada(f + df[dir], c + dc[dir])) perimetro
34                        ++;
35                }
36            }
37        }
38        return perimetro;
39    } // O(N*M)
40
41    // -----
42    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
43    // -----
44    // 1. Formula cerrada equivalente 4B - 2A: contar B (bloques
      ocupados) y
45    // A (pares de vecinos ocupados, cada par contado una sola
      vez) por
46    // separado, y retornar 4*B - 2*A; da el mismo resultado
      que revisar
47    // los 4 vecinos por celda, solo que evita revisar cada
      adyacencia
48    // dos veces (una desde cada bloque).

```

```

46 // 2. Perímetro por componente conexa (varias figuras
      separadas en la
47 // misma grilla): correr un BFS/DFS para etiquetar
      componentes, y
48 // aplicar la misma logica de conteo de vecinos SOLO
      dentro de cada
49 // componente etiquetada.
50 // 3. Grilla con bloques de tamaño k (no unitarios):
      multiplicar el
51 // resultado final por k, ya que cada lado unitario en la
      formula
52 // corresponde a un lado de longitud k en la figura real.
53 };

```

Ejemplo de funcionamiento:

```

1 vector<string> grilla = {
2     "***",
3     ". * ."
4 };
5 PerimetroBloques p(grilla);
6
7 long long resultado = p.calcularPerimetro();
8 // resultado = 8
9 // (3 bloques ocupados: (0,0), (0,1), (1,1); hay 2 pares
      adyacentes:
10 // (0,0)-(0,1) y (0,1)-(1,1); formula: 4*3 - 2*2 = 12 - 4 = 8)

```

Regla práctica: usa esta técnica de conteo directo (revisión de vecinos, o la fórmula equivalente $4B - 2A$) siempre que la figura venga dada como una grilla de celdas ocupadas —es $O(NM)$ y evita por completo la necesidad de reconstruir un contorno poligonal explícito, que sería innecesariamente complejo para este tipo de entrada.

33.2 Número de Cuadrados en un Rectángulo $N \times M$

Idea: contar cuántos cuadrados de **cualquier tamaño** caben alineados a la grilla dentro de un rectángulo de $N \times M$ celdas (asumiendo $N \leq M$ sin pérdida de generalidad) se resuelve sumando, para cada tamaño de lado k desde 1 hasta N , la cantidad de posiciones donde un cuadrado de lado k cabe: hay $(N - k + 1)$ posiciones verticales y $(M - k + 1)$ posiciones horizontales, así que la cantidad de cuadrados de lado k es $(N - k + 1)(M - k + 1)$. Sumando sobre todos los k posibles se obtiene el total. Cuando $N = M$ (rectángulo cuadrado), esta suma se reduce a la fórmula cerrada conocida $\sum_{k=1}^N k^2 = \frac{N(N+1)(2N+1)}{6}$.

¿Por qué usarlo? Es un problema de conteo combinatorio puro —no requiere ningún cómputo geométrico continuo (sin producto cruz, sin coordenadas reales)—, pero encaja en esta parte porque describe cuántas figuras geométricas caben dentro de otra, un tipo de pregunta recurrente en problemas ad-hoc de la sección de geometría. La suma directa es $O(N)$ (con $N \leq M$), lo cual ya es suficientemente rápido para las restricciones típicas de competencia; no hace falta la fórmula cerrada salvo que N sea extremadamente grande y se necesite $O(1)$.

Casos de uso:

- Contar cuadrados de cualquier tamaño en un tablero rectangular (variante del clásico "cuántos cuadrados hay en un tablero de ajedrez").
- Problemas de conteo combinatorio sobre grillas donde la figura buscada es un cuadrado alineado a los ejes.

- Verificación o sanity-check antes de resolver la variante más general de *Número de Rectángulos en una Grilla $N \times M$* .
- Subrutina en problemas que piden distribuir o colocar la mayor cantidad de cuadrados posibles sin superposición explícita (aquí se permite solapamiento, solo se cuenta cuántos "cabén" como subcuadrícula).

Complejidad: $O(\min(N, M))$ sumando directamente sobre los tamaños de lado posibles; $O(1)$ si se usa la fórmula cerrada para el caso $N = M$.

```

1 // CREATE/READ: Numero de cuadrados de cualquier tamaño en un
  rectangulo N x M
2 struct CuadradosEnRectangulo {
3   // READ: cuenta todos los cuadrados (de lado 1 hasta min(N,M)
   ) que caben
4   // alineados a la grilla dentro de un rectangulo de N x M
   celdas
5   static long long contar(long long n, long long m) {
6     long long menor = min(n, m);
7     long long total = 0;
8     for (long long k = 1; k <= menor; k++) {
9       total += (n - k + 1) * (m - k + 1);
10    }
11    return total;
12  } // O(min(N,M))
13
14  // READ: caso particular N == M (rectangulo cuadrado),
   formula cerrada
15  // sum_{k=1}^n k^2 = n(n+1)(2n+1)/6
16  static long long contarCuadrado(long long n) {
17    return n * (n + 1) * (2 * n + 1) / 6;
18  } // O(1)
19
20  //
21  // MODIFICACIONES Y COMENTARIOS CON LOS USOS
22  //
23  // 1. Verificar overflow: para N, M grandes (~1e5 o mas), el
   resultado
24  // puede exceder long long dependiendo de las
   restricciones del
25  // problema; revisar el enunciado antes de asumir que long
   long basta.
26  // 2. Solo cuadrados de tamaño EXACTO k (no todos los tamaños
   ): basta con
27  // un solo termino (n-k+1)*(m-k+1), sin sumar sobre todos
   los k.
28  // 3. Formula cerrada general para N != M (no solo N == M):
   se puede
29  // derivar expandiendo la suma en terminos de sumas de k y
   k^2
30  // conocidas, pero en la practica la suma O(min(N,M)) ya
   es
31  // suficientemente rapida en casi todas las restricciones
   de CP.
32 };

```

Ejemplo de funcionamiento:

```

1 long long resultado = CuadradosEnRectangulo::contar(2, 3);
2 // resultado = 8
3 // (cuadrados de lado 1: 2*3 = 6 posiciones; cuadrados de lado
  2:
4 // (2-2+1)*(3-2+1) = 1*2 = 2 posiciones; total 6+2 = 8)

```

Regla práctica: usa la suma directa $O(\min(N, M))$ como opción general y segura; recurre a la fórmula cerrada solo en el caso particular $N = M$ y cuando N sea tan grande que incluso $O(N)$ sea inviable —en la mayoría de problemas de competencia, la suma directa ya es suficientemente rápida y más fácil de verificar que la fórmula cerrada expandida.

33.3 Número de Rectángulos en una Grilla $N \times M$

Idea: generaliza *Número de Cuadrados en un Rectángulo $N \times M$* de cuadrados a rectángulos de cualquier ancho y alto: un rectángulo alineado a la grilla queda determinado eligiendo dos líneas verticales distintas (de las

$N + 1$ líneas que delimitan las N filas) y dos líneas horizontales distintas (de las $M + 1$ que delimitan las M columnas) —cada par de líneas verticales junto con cada par de líneas horizontales define exactamente un rectángulo. El conteo es entonces puramente combinatorio: $\binom{N+1}{2} \cdot \binom{M+1}{2}$, el producto de "formas de elegir 2 líneas verticales" por "formas de elegir 2 líneas horizontales".

¿Por qué usarlo? A diferencia del conteo de cuadrados —que necesita sumar sobre cada tamaño de lado posible porque un cuadrado fuerza ancho = alto—, contar rectángulos no tiene esa restricción: ancho y alto son independientes, así que el problema se separa limpiamente en "elegir extensión horizontal" y "elegir extensión vertical", cada una una combinación simple $\binom{2}{2}$. Esto da una fórmula $O(1)$ directa, sin necesidad de ninguna suma —más simple que el caso de cuadrados, donde la restricción ancho=alto obliga a iterar.

Casos de uso:

- Contar todos los rectángulos alineados a los ejes que caben en una grilla de $N \times M$ celdas.
- Variante del clásico "cuántos rectángulos hay en un tablero de ajedrez" (caso particular $N = M = 8$).
- Verificación cruzada: sumando el conteo de rectángulos de cada ancho×alto específico debe coincidir con esta fórmula cerrada.
- Subrutina en problemas que combinan conteo de sub-rectángulos con otra condición adicional (por ejemplo, sub-rectángulos que contienen una celda marcada).

Complejidad: $O(1)$ —dos evaluaciones de $\binom{2}{2} = \frac{k(k-1)}{2}$ y una multiplicación.

```

1 // CREATE/READ: Numero de rectangulos alineados a la grilla en
  un
2 // rectangulo de N x M celdas
3 struct RectangulosEnGrilla {
4   // READ: combinaciones de k elementos tomados de 2 en 2
5   static long long combinatoria2(long long k) {
6     return k * (k - 1) / 2;
7   } // O(1)
8
9   // READ: cuenta todos los rectangulos (de cualquier ancho y
   alto) que
10  // caben alineados a la grilla en un rectangulo de N x M
   celdas
11  static long long contar(long long n, long long m) {
12    // N celdas dan N+1 líneas verticales delimitadoras;
   analogo para M
13    return combinatoria2(n + 1) * combinatoria2(m + 1);
14  } // O(1)
15
16  //
17  // MODIFICACIONES Y COMENTARIOS CON LOS USOS
18  //
19  // 1. Verificacion cruzada con Numero de Cuadrados: sumar
   (n-k+1)*(m-k+1) sobre k=1..min(n,m) cuenta SOLO los
20  // cuadrados,
21  // un subconjunto estricto de lo que cuenta esta formula (
   todos los
22  // rectangulos, incluidos los no cuadrados).
23  // 2. Overflow: N+1 y M+1 pueden hacer que combinatoria2
   exceda long long
24  // si N o M son extremadamente grandes (~3e9 o mas);
   normalmente no es
25  // un problema en CP, pero vale revisar las restricciones
   del enunciado.
26  // 3. Rectangulos que contienen una celda fija (r0, c0): se
   puede
27  // contar por separado como (numero de elecciones de
   líneas verticales

```



```

28 // que "abarcen" la columna c0 * (analogamente para filas
29 //), en vez de
30 // la formula general sin restriccion.
};

```

Ejemplo de funcionamiento:

```

1 long long resultado = RectangulosEnGrilla::contar(2, 3);
2 // resultado = 18
3 // (N=2 da 3 lineas verticales, C(3,2)=3; M=3 da 4 lineas
4 // horizontales,
5 // C(4,2)=6; total 3*6=18; esto incluye los 8 cuadrados ya
6 // contados en
7 // la subseccion anterior, mas los rectangulos no cuadrados)

```

Regla práctica: usa esta fórmula cerrada $\binom{N+1}{2} \binom{M+1}{2}$ siempre que la pregunta sea por **todos** los rectángulos (sin restricción de ancho=alto); si la pregunta es específicamente por cuadrados, esa restricción rompe la independencia entre horizontal y vertical, y hace falta la suma de *Número de Cuadrados en un Rectángulo $N \times M$* en su lugar.

33.4 Suma de Distancias Manhattan entre Todos los Pares

Idea: calcular $\sum_{i < j} (|x_i - x_j| + |y_i - y_j|)$ para n puntos ingenuamente es $O(n^2)$. La clave para bajar a $O(n \log n)$ es que la distancia Manhattan se **descompone** en dos términos independientes: $|x_i - x_j| + |y_i - y_j|$, así que la suma total sobre todos los pares es exactamente (suma de $|x_i - x_j|$ sobre todos los pares) + (suma de $|y_i - y_j|$ sobre todos los pares) —las coordenadas x e y se pueden procesar por separado, sin necesidad de considerar puntos como pares (x, y) acoplados. Cada una de esas sumas 1D se calcula ordenando los valores y usando que, para un arreglo ordenado, la contribución de cada elemento a la suma de diferencias absolutas se puede acumular con una suma prefijo: el elemento en la posición i (0-indexado) contribuye positivamente i veces (es mayor que los i anteriores) y negativamente $(n - 1 - i)$ veces (es menor que los restantes).

¿Por qué usarlo? La fuerza bruta $O(n^2)$ es inviable para n grande (típicamente $n > 10^4$ en competencia). La descomposición en x e y por separado es posible **específicamente** porque la métrica Manhattan ya viene descompuesta en valor absoluto de diferencias por coordenada —esto NO funciona igual para distancia euclidiana, donde x e y están acoplados dentro de una raíz cuadrada y no se pueden separar de esta forma. Es una de las pocas ventajas computacionales concretas de usar Manhattan sobre euclidiana en problemas de suma de distancias.

Casos de uso:

- Suma total de distancias Manhattan entre todos los pares de un conjunto de puntos, cuando n es grande.
- Problemas de logística o redes donde el costo de conexión es proporcional a distancia Manhattan (movimiento en grilla).
- Como subrutina dentro de problemas más grandes que requieren esta suma repetidamente sobre subconjuntos (mantenida con estructuras de datos si los puntos cambian dinámicamente).
- Comparación de costo total bajo métrica Manhattan versus Chebyshev (que se relacionan mediante una

rotación de coordenadas de 45 grados, no cubierta aquí).

Complejidad: $O(n \log n)$ —dominado por ordenar las coordenadas x y las coordenadas y por separado ($O(n \log n)$ cada una); el cálculo de la suma con prefijos sobre cada arreglo ordenado es $O(n)$.

```

1 // CREATE/READ: Suma de distancias Manhattan entre todos los
2 // pares de n
3 // puntos, mediante descomposicion en x e y independientes
4 struct SumaManhattan {
5     // READ: suma de |v[i] - v[j]| sobre todos los pares i < j,
6     // para un
7     // arreglo 1D de valores (se usa una vez para x, otra vez
8     // para y)
9     static long long sumaDiferenciasAbsolutas(vector<long long> v
10     ) {
11         sort(v.begin(), v.end());
12         int n = (int)v.size();
13         long long suma = 0;
14         long long prefijo = 0; // suma acumulada de v[0..i-1]
15         for (int i = 0; i < n; i++) {
16             // v[i] es mayor que los i elementos anteriores:
17             // contribuye
18             // v[i]*i - prefijo (v[i] menos cada uno de los
19             // anteriores)
20             suma += (long long)i * v[i] - prefijo;
21             prefijo += v[i];
22         }
23         return suma;
24     } // O(n log n), dominado por el sort
25
26     // READ: suma total de distancia Manhattan entre todos los
27     // pares de
28     // puntos (x[i], y[i])
29     static long long calcular(const vector<long long>& x, const
30     vector<long long>& y) {
31         return sumaDiferenciasAbsolutas(x) +
32             sumaDiferenciasAbsolutas(y);
33     } // O(n log n)
34
35     //
36     // -----
37     // MODIFICACIONES Y COMENTARIOS CON LOS USOS
38     // -----
39
40     // 1. Distancia Chebyshev (max(|dx|, |dy|)) entre todos los
41     // pares: se
42     // puede reducir a Manhattan rotando las coordenadas 45
43     // grados
44     // (u = x+y, v = x-y), calculando Manhattan sobre (u,v) y
45     // dividiendo
46     // el resultado entre 2 -- relacion clasica entre ambas
47     // metricas.
48     // 2. Overflow: la suma total puede ser del orden de n^2 * (
49     // rango de
50     // coordenadas), facilmente mayor a 1e18 para n y
51     // coordenadas grandes;
52     // verificar los limites del problema antes de asumir que
53     // long long
54     // alcanza.
55     // 3. Puntos que se agregan dinamicamente (no todos conocidos
56     // de
57     // antemano): esta formula requiere ordenar el arreglo
58     // completo, asi
59     // que no es directamente incremental; para
60     // actualizaciones online se
61     // necesitaria un Fenwick tree indexado por coordenada
62     // comprimida.
63 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> x = {1, 3, 5};
2 vector<long long> y = {2, 2, 2};
3
4 long long resultado = SumaManhattan::calcular(x, y);
5 // resultado = 8
6 // (suma de |dx| sobre los 3 pares: |1-3|+|1-5|+|3-5| = 2+4+2 =
7 // 8;
8 // suma de |dy| sobre los 3 pares: 0+0+0 = 0 (todas las y son
9 // iguales);
10 // total: 8 + 0 = 8)

```

Nota: la derivación de `sumaDiferenciasAbsolutas` usando `i * v[i] - prefijo` depende de que `v` esté ordenado: una vez ordenado, cada elemento en la posición i es mayor o igual que los i elementos que lo preceden,

así que su contribución neta a la suma de diferencias absolutas con esos i elementos es $v[i]$ sumado i veces menos la suma de esos i elementos anteriores (**prefijo**).

Regla práctica: usa esta descomposición (separar x e y , ordenar cada uno, sumar con prefijos) siempre que la métrica sea Manhattan y n sea grande —es la técnica estándar para bajar de $O(n^2)$ a $O(n \log n)$ en este tipo de suma. Recuerda que esta separación es específica de Manhattan; no aplica directamente a distancia euclidiana por la raíz cuadrada que acopla x e y .

33.5 Máximo de Cuadrados 2x2 en un Triángulo Isósceles Rectángulo

Idea: sea el triángulo rectángulo isósceles con catetos de longitud b (los dos lados iguales), ángulo recto en el origen, y catetos alineados con los ejes —la hipotenusa es entonces la recta $x + y = b$. El problema de contar cuántos cuadrados de 2×2 caben alineados a la grilla dentro de este triángulo es equivalente, tras dividir todas las longitudes entre 2, a contar cuántos cuadrados **unitarios** caben en un triángulo análogo de cateto $L = \lfloor b/2 \rfloor$. Un cuadrado unitario con esquina inferior izquierda en (i, j) cabe completo bajo la hipotenusa si su esquina superior derecha $(i + 1, j + 1)$ satisface $(i + 1) + (j + 1) \leq L$, es decir $i + j \leq L - 2$. Contar los pares (i, j) con $i, j \geq 0$ enteros que cumplen esa desigualdad da $\sum_{s=0}^{L-2} (s + 1) = \frac{L(L-1)}{2}$.

¿Por qué usarlo? Es un problema de conteo combinatorio bajo una restricción geométrica lineal (la hipotenusa), similar en espíritu a *Número de Cuadrados en un Rectángulo $N \times M$* , pero aquí la restricción no es un rectángulo sino un triángulo, lo que cambia la condición de "cabe" de una desigualdad por coordenada a una única desigualdad conjunta $i + j \leq L - 2$. Reducir cuadrados de 2×2 a cuadrados unitarios mediante el cambio de escala $L = \lfloor b/2 \rfloor$ evita tener que rederivar la fórmula para cada tamaño de cuadrado distinto.

Casos de uso:

- Contar la máxima cantidad de baldosas cuadradas de tamaño fijo que caben en una región triangular de recorte.
- Problemas de empaquetamiento (packing) simplificado donde la región es un triángulo rectángulo isósceles.
- Como paso intermedio para estimar el área "desperdiciada" al triangular una región rectangular.
- Variante de conteo bajo la diagonal de una grilla cuadrada (equivalente al caso $L = b$ con cuadrados unitarios).

Complejidad: $O(1)$ —la fórmula cerrada se evalúa directamente sin ninguna iteración.

```
1 // CREATE/READ: Maximo numero de cuadrados de lado k que caben
  // alineados a
2 // la grilla en un triangulo rectangulo isosceles de catetos de
  // longitud b
3 struct CuadradosEnTriangulo {
4     // READ: cuenta cuadrados unitarios que caben bajo la
      // hipotenusa de un
5     // triangulo rectangulo isosceles de cateto L (formula L*(L
      // -1)/2)
6     static long long contarUnitarios(long long L) {
```

```
7         if (L < 2) return 0; // no cabe ni un cuadrado si L < 2
8         return L * (L - 1) / 2;
9     } // O(1)
10
11     // READ: cuenta cuadrados de lado k (por defecto k=2) que
      // caben en un
12     // triangulo de cateto b, escalando al caso unitario con L =
      // b / k
13     static long long contar(long long b, long long k = 2) {
14         long long L = b / k; // division entera: escala al
      // triangulo equivalente
15         return contarUnitarios(L);
16     } // O(1)
17
18     //
19     // -----
20     // MODIFICACIONES Y COMENTARIOS CON LOS USOS
21     // -----
22     // 1. Triangulo con la BASE como hipotenusa (no como cateto):
      // si el
23     // enunciado da la longitud de la hipotenusa en vez del
      // cateto, hay
24     // que convertir primero: cateto = hipotenusa / sqrt(2),
      // lo cual ya
25     // no da un resultado entero exacto en general --
      // verificar con
26     // cuidado cual longitud especifica realmente el enunciado
      // antes de
27     // aplicar esta formula.
28     // 2. Cuadrados que pueden sobresalir parcialmente (no exigir
      // que quepan
29     // COMPLETOS): cambia la condicion i+j <= L-2 a una menos
      // estricta,
30     // dando un resultado distinto; la formula de este struct
      // asume
31     // cuadrados completamente contenidos.
32     // 3. b no divisible exactamente entre k: la division entera
      // b/k ya
33     // trunca correctamente al mayor L util, sin necesidad de
      // ajuste
34     // adicional.
```

Ejemplo de funcionamiento:

```
1 long long resultado = CuadradosEnTriangulo::contar(8, 2);
2 // resultado = 6
3 // (b=8, k=2, L = 8/2 = 4; contarUnitarios(4) = 4*3/2 = 6;
4 // hay que correrlo para confirmar el conteo exacto de
  // posiciones válidas
5 // bajo la hipotenusa escalada)
```

Nota: este struct asume que la "base" b dada en el enunciado se refiere a la longitud de los **catetos** (los dos lados iguales del triángulo rectángulo isósceles), no a la hipotenusa —es la interpretación más común de "base" en este tipo de problema, pero conviene confirmar contra el enunciado exacto antes de aplicar la fórmula, ya que si "base" refiere a la hipotenusa, la conversión introduce $\sqrt{2}$ y el problema deja de tener una fórmula puramente combinatoria en enteros.

Regla práctica: usa la reducción a cuadrados unitarios ($L = \lfloor b/k \rfloor$, luego $L(L - 1)/2$) siempre que el triángulo sea rectángulo isósceles con catetos alineados a los ejes; para un triángulo rectángulo no isósceles (catetos de longitudes distintas), la condición de conteo cambia y requeriría rederivar la desigualdad de la hipotenusa correspondiente.

33.6 Máximo Número de Piezas con N Cortes (Pastelero Perezoso)

Idea: el problema del pastelero perezoso (*Lazy Caterer's Sequence*) pregunta cuál es el máximo número de piezas en que se puede dividir un círculo (o cualquier región convexa) usando n cortes rectos, donde cada corte es una línea recta completa que puede cruzar a todos los cortes anteriores. La estrategia óptima es que cada nuevo corte cruce a **todos** los cortes anteriores en puntos distintos

(nunca por un punto ya usado ni paralelo a otro corte): el k -ésimo corte, al cruzar los $k - 1$ cortes previos, queda dividido en k segmentos, y cada uno de esos segmentos parte una pieza existente en dos, agregando exactamente k piezas nuevas. Sumando esa ganancia para cada corte desde el primero: $p(n) = 1 + \sum_{k=1}^n k = 1 + \frac{n(n+1)}{2}$.

¿Por qué usarlo? Es un problema de conteo por recurrencia donde la clave no es geometría de coordenadas sino la observación de que **cada corte nuevo debe maximizar sus cruces** con los anteriores para maximizar piezas —cualquier corte paralelo a uno existente, o que pase por un punto de cruce ya usado, da menos piezas que el óptimo. La fórmula cerrada evita simular los cortes explícitamente; solo hace falta razonar sobre cuántos cruces nuevos introduce cada corte adicional.

Casos de uso:

- El problema clásico de "máximo número de piezas de un pastel/pizza con n cortes rectos".
- Generalización a la versión 3D (*Cake Numbers*): máximo número de piezas de un sólido convexo con n planos de corte.
- Como ejemplo introductorio de razonamiento por recurrencia en problemas de conteo geométrico.
- Verificación de que un algoritmo de simulación de cortes (para variantes con restricciones) no exceda el máximo teórico.

Complejidad: $O(1)$ —la fórmula cerrada se evalúa directamente.

```
1 // CREATE/READ: Maximo numero de piezas al hacer n cortes
  rectos en un
2 // pastel/region convexa (Lazy Caterer's Sequence)
3 struct PasteleroPerezoso {
4     // READ: maximo numero de piezas con n cortes, formula
      cerrada
5     // p(n) = 1 + n(n+1)/2
6     static long long maximoPiezas(long long n) {
7         return 1 + n * (n + 1) / 2;
8     } // O(1)
9
10    //
11
12    //-----
13    // 1. Version 3D (Cake Numbers, n planos de corte en un
      solido convexo):
14    // formula distinta, C(n,0) + C(n,1) + C(n,2) + C(n,3),
      que en 2D se
15    // reduce a los primeros tres terminos (C(n,3) = 0 en el
      plano);
16    // struct separado si se necesita la version 3D.
17    // 2. Cortes que deben ser PARALELOS entre si (variante
      restringida): la
18    // formula cambia a n+1 piezas (cada corte solo agrega 1
      pieza, sin
19    // cruzar a los anteriores) -- mucho menor que el maximo
      sin
20    // restriccion.
21    // 3. Overflow: n*(n+1) puede exceder el rango de long long
      solo para
22    // n extremadamente grande (~3e9), poco comun en
      restricciones
23    // tipicas de CP, pero vale verificar el enunciado.
24 };
```

Ejemplo de funcionamiento:

```
1 long long resultado = PasteleroPerezoso::maximoPiezas(3);
2 // resultado = 7
3 // (con 3 cortes que se cruzan de a pares en puntos distintos:
4 // 1 (sin cortes) + 1 + 2 + 3 = 7 piezas)
```

Regla práctica: usa esta fórmula cuando el problema pida el **máximo** de piezas con n cortes rectos sin restricciones adicionales sobre su orientación; si el problema impone una restricción (cortes paralelos, cortes que deben pasar por un punto común), la fórmula cambia por completo y hay que rederivarla contando cuántas piezas nuevas agrega cada corte bajo esa restricción específica.

33.7 Altura Máxima de un Triángulo de Monedas

Idea: dadas n monedas, se apilan formando un triángulo donde la fila 1 tiene 1 moneda, la fila 2 tiene 2 monedas, y en general la fila k tiene k monedas —así que una altura de h filas completas consume $\sum_{k=1}^h k = \frac{h(h+1)}{2}$ monedas. El problema pide la mayor altura h tal que ese triángulo completo quepa con las n monedas disponibles, es decir, el mayor h que satisface $\frac{h(h+1)}{2} \leq n$ (las monedas sobrantes, si las hay, no alcanzan para una fila completa adicional y se descartan). Como $\frac{h(h+1)}{2}$ crece monótonamente con h , esto se resuelve con búsqueda binaria sobre h , o despejando directamente de la ecuación cuadrática $h^2 + h - 2n \leq 0$.

¿Por qué usarlo? Es el mismo patrón combinatorio que *Máximo Número de Piezas con N Cortes* —una suma de la forma $\sum k$ — pero aquí la incógnita es el límite superior de la suma dado un total fijo, no el valor de la suma dado el límite. Dos caminos válidos: *Búsqueda Binaria on Answer* (ya cubierta en la sección de Búsqueda Binaria de esta guía) sobre h probando $\frac{h(h+1)}{2} \leq n$, o resolver directamente la fórmula cuadrática con $h = \left\lfloor \frac{-1 + \sqrt{1 + 8n}}{2} \right\rfloor$. La búsqueda binaria evita cualquier riesgo de error de precisión en la raíz cuadrada de números grandes; la fórmula cerrada es más rápida pero requiere cuidado con el redondeo de `sqrt` en enteros grandes.

Casos de uso:

- El problema clásico de "cuántas filas de un triángulo de monedas/bolos/cajas se pueden completar con n unidades".
- Cualquier variante donde el costo de la fila k crece linealmente con k y se pregunta cuántas filas completas caben en un presupuesto fijo.
- Ejercicio introductorio para practicar la elección entre búsqueda binaria y fórmula cerrada cuando ambas son viables.
- Verificación cruzada: el problema es exactamente el inverso de *Máximo Número de Piezas con N Cortes* (ahí se da h y se pide la suma; aquí se da la suma y se pide h).

Complejidad: $O(\log n)$ con búsqueda binaria sobre h (rango de búsqueda $[0, n]$, aunque h real es $O(\sqrt{n})$), así que también se puede acotar el rango a $[0, \sqrt{2n} + 1]$ para menos iteraciones); $O(1)$ con la fórmula cerrada, al costo de una llamada a `sqrt`.

```
1 // CREATE/READ: Altura maxima de un triangulo de monedas que se
  puede
2 // formar con n monedas disponibles (fila k tiene k monedas)
3 struct TrianguloMonedas {
```

```

4 // READ: monedas necesarias para completar h filas (formula h
  (h+1)/2)
5 static long long monedasParaAltura(long long h) {
6     return h * (h + 1) / 2;
7 } // O(1)
8
9 // READ: altura maxima alcanzable con n monedas, via busqueda
  binaria
10 // sobre h (mas robusto ante errores de precision que la
   formula cerrada)
11 static long long alturaMaximaBinaria(long long n) {
12     long long lo = 0, hi = 2000000000LL; // cota superior
   segura
13     while (lo < hi) {
14         long long medio = lo + (hi - lo + 1) / 2;
15         if (monedasParaAltura(medio) <= n) lo = medio;
16         else hi = medio - 1;
17     }
18     return lo;
19 } // O(log n)
20
21 // READ: altura maxima alcanzable con n monedas, via formula
   cerrada
22 // despejada de  $h^2 + h - 2n \leq 0$  (requiere verificar el
   redondeo)
23 static long long alturaMaximaFormula(long long n) {
24     long long h = (long long)((-1.0 + sqrt(1.0 + 8.0 * n)) /
   2.0);
25     // ajuste por posible error de precision de sqrt en punto
   flotante
26     while (monedasParaAltura(h + 1) <= n) h++;
27     while (h > 0 && monedasParaAltura(h) > n) h--;
28     return h;
29 } // O(1) amortizado (el ajuste final es O(1) en la practica)
30
31 //
32 // -----
33 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
34 // -----
35 // 1. Monedas sobrantes (no usadas en el triangulo completo):
   n -
36 // monedasParaAltura(alturaMaximaBinaria(n)) da la
   cantidad de
37 // monedas que quedan sin colocar tras formar la mayor
   altura posible.
38 // 2. Costo de fila k distinto (no lineal en k, por ejemplo k
   ^2 monedas
39 // por fila): la busqueda binaria sigue funcionando
   reemplazando
40 // monedasParaAltura por la suma correspondiente; la
   formula cerrada
41 // habria que rederivarla para cada caso.
42 // 3. n muy grande (overflow en 8*n dentro de la formula
   cerrada): usar
43 // long long consistentemente en monedasParaAltura evita
   overflow
44 // hasta n en el orden de 1e18; para n mayor, revisar el
   enunciado.
45 };

```

Ejemplo de funcionamiento:

```

1 long long resultado = TrianguloMonedas::alturaMaximaBinaria(10)
2 ;
3 // resultado = 4
4 // (fila 1:1, fila 2:2, fila 3:3, fila 4:4; total 1+2+3+4=10
   monedas,
5 // exactamente 10, sin sobrantes; con 5 filas se necesitarian
   15, que
6 // excede las 10 disponibles)

```

Regla práctica: usa `alturaMaximaBinaria` como opción por defecto y segura frente a errores de precisión; usa `alturaMaximaFormula` solo si el rendimiento de $O(\log n)$ no es aceptable, y en ese caso no olvides el ajuste posterior con los `while` de corrección —`sqrt` en números grandes puede desviarse por error de punto flotante y dar un h incorrecto en ± 1 sin el ajuste.

Parte VI Programación Dinámica (Dynamic Programming)

33.8 ¿Qué algoritmo usar?

Cuando el ejercicio diga / pide	Usa	Pista rápida
“Calcula $f(n)$ ” con una recurrencia que repite subproblemas (Fibonacci, escaleras)	Memoización / Tabulación	Memo si el orden no es obvio; tabla si hay orden por índice.
“Calcula $f(n)$ módulo p ” con n hasta 10^{18} y recurrencia lineal de orden fijo	Exponenciación de matrices	$O(k^3 \log n)$. Primera fila = coeficientes.
“Subarreglo contiguo de suma máxima”	Kadane	$\max(a[i], \text{act}+a[i])$. $O(n)$, $O(1)$ de espacio.
“Máxima suma circular” o “máxima submatriz”	Kadane circular / Kadane por pares de filas	Circular: total – mínimo subarreglo. 2D: $O(n^2 m)$.
“Mejor día para comprar y vender (una vez)”	Kadane sobre diferencias	Diferencias consecutivas de precios.
“¿De cuántas formas elijo k de n ?”, “caminos en grilla derecha/abajo” (n moderado)	Triángulo de Pascal	Solo sumas, sirve con cualquier módulo. $O(n^2)$.
Lo mismo con $n \approx 10^6$ y módulo primo	Factoriales e inversos modulares	Precómputo $O(n)$, consulta $O(1)$.
“Camino creciente más largo en una matriz” (movimiento en 4 direcciones, valores estrictos)	DFS con memoización	Grafo sin ciclos por ser estricto. $O(nm)$.
“Subsecuencia creciente más larga” (solo la longitud)	LIS con búsqueda binaria	$O(n \log n)$.
“Cuántas LIS hay”, “reconstruye una”, “subsecuencia bitónica”	LIS por DP $O(n^2)$	Guarda dp, padre y cuenta por índice.
“Subsecuencia creciente de suma máxima”	DP tipo LIS con suma	Con Fenwick sobre valores comprimidos: $O(n \log n)$.
“Subsecuencia común de dos cadenas”, “alinear dos secuencias”, “palíndromo más largo”	LCS (DP sobre pares de prefijos)	$O(nm)$. Palíndromo: LCS con el reverso.
“Pila más alta de cajas con rotaciones”, “muñecas rusas”, “sobres anidados”	Box Stacking (orden + DP tipo LIS)	Ordena por área de base decreciente. $O(n^2)$.

Table 19: Fundamentos y secuencias: qué usar según lo que pide el ejercicio.

Cuando el ejercicio diga / pida	Usa	Pista rápida
"Elige objetos con peso y valor, cada uno a lo sumo una vez , con capacidad W "	Knapsack 0/1	Capacidad en sentido decreciente . $O(nW)$.
"Cada objeto se puede usar las veces que quiera ", cortar una barra (rod cutting)	Knapsack ilimitado	Capacidad en sentido creciente . $O(nW)$.
"Cada objeto con cantidad limitada c_i "	Knapsack acotado	Descompón c_i en potencias de dos.
"Objetos que se pueden partir" (fraccionario)	Greedy por valor/peso	No es DP. Ordena y toma.
"¿Existe un subconjunto con suma exacta S ?"	Subset Sum (Knapsack con bool)	Con bitset : $b \mid= b \ll x$.
"¿De cuántas formas formo el monto n con estas monedas?" (el orden no importa)	Cambio de monedas: combinaciones	Monedas afuera , monto adentro . $O(nm)$.
"¿De cuántas formas?" cuando el orden sí importa (escaleras con saltos)	Cambio de monedas: permutaciones	Monto afuera , monedas adentro .
"Mínimo de monedas para pagar n " (sistema arbitrario)	Cambio de monedas: mínimo	Greedy solo si el sistema es canónico. -1 si es imposible.
"Divide el arreglo en dos grupos con diferencia mínima", "¿se puede dividir en dos mitades iguales?"	Problema de Partición	Subset Sum hasta $S/2$. Respuesta $S - 2x$.
Lo mismo con $n \leq 40$ y suma enorme	Meet in the middle	Genera sumas de cada mitad y une con dos punteros.
"Orden óptimo de eliminar elementos" donde quitar uno cambia los vecinos (globos)	DP de intervalos (Burst Balloons)	Modela el último que queda. $O(n^3)$.
"Orden óptimo de multiplicar matrices", "árbol de búsqueda óptimo", "triangular un polígono"	DP de intervalos	Estado $[l, r]$, prueba el corte k , longitud creciente.
DP de intervalos $O(n^3)$ que no pasa el tiempo y el costo cumple la desigualdad cuadrangular	Optimización de Knuth	Acota k entre $\text{opt}[i][j-1]$ y $\text{opt}[i+1][j]$. $O(n^2)$.

Table 20: Knapsack, monedas e intervalos: qué usar según lo que pide el ejercicio.

Cuando el ejercicio diga / pida	Usa	Pista rápida
"¿Cuántos números en $[L, R]$ cumplen algo sobre sus dígitos?" (suma de dígitos, dígito prohibido, dígitos crecientes) con $R \leq 10^{18}$	Digit DP	Respuesta $f(R) - f(L - 1)$. Bandera tight y manejo de ceros iniciales.
Lo mismo combinado con divisibilidad por m	Digit DP con resto en el estado	Estado (pos, resto, tight).
"Número esperado de lanzamientos / pasos hasta llegar a la meta", "valor esperado"	DP de valor esperado	$E(s) = \sum p_i (c_i + E(s_i))$. Despeja si el estado se repite.
"Valor esperado" con ciclos entre varios estados distintos	Sistema lineal (Gauss)	Una ecuación por estado del ciclo.
"Valor esperado" de una suma de eventos simples	Linealidad de la esperanza	Suma de esperanzas individuales, sin DP.
"Particiona en m grupos contiguos minimizando costo", DP $O(n^2m)$ con el óptimo monótono	Divide and Conquer Optimization	Verifica la desigualdad cuadrangular. $O(nm \log n)$.
DP del tipo $\min_i (b_i x_j + a_i)$, "elige la mejor recta evaluada en x "	Convex Hull Trick (deque)	Pendientes y consultas monótonas. $O(n)$.
Lo mismo sin orden en pendientes o consultas	Li Chao Tree	$O(\log C)$ por inserción y consulta.

Table 21: DP avanzada: qué usar según lo que pide el ejercicio.

34 Fundamentos

34.1 Memoización vs. Tabulación (Top-Down y Bottom-Up)

Idea: la programación dinámica resuelve un problema descomponiéndolo en **subproblemas superpuestos**: subproblemas que se repiten muchas veces dentro de la recursión. En lugar de recalcularlos, se guarda cada resultado la primera vez que se obtiene. Hay dos formas de hacerlo. La **memoización** (*top-down*) escribe la recursión natural del problema y agrega una tabla que se consulta antes de calcular: si el estado ya fue resuelto, se devuelve el valor guardado. La **tabulación** (*bottom-up*) invierte el orden: se llena la tabla iterativamente, desde los casos base hasta el estado que interesa, de modo que cada estado se calcula cuando todos los estados de los que depende ya están listos.

¿Por qué usarlo? Sin guardar resultados, la recursión de Fibonacci ($F(n) = F(n-1) + F(n-2)$) recalcula los mismos valores una y otra vez y cuesta $O(\varphi^n)$, exponencial. Con cualquiera de las dos técnicas cada estado se calcula **una sola vez**, y el costo baja a $O(n)$. La memoización es más fácil de escribir porque sigue la recurrencia tal cual, y solo calcula los estados que realmente se necesitan; su costo es la profundidad de recursión (riesgo de desbordar la pila) y el *overhead* de las llamadas. La tabulación es iterativa, más rápida en la práctica, y permite reducir memoria cuando cada estado depende solo de unos pocos anteriores (como en Fibonacci, donde bastan dos variables).

Casos de uso:

- Convertir cualquier recurrencia con subproblemas repetidos en un algoritmo polinomial.
- Memoización: cuando el espacio de estados es enorme pero solo una fracción pequeña es alcanzable, o cuando el orden de llenado no es evidente.
- Tabulación: cuando todos los estados se necesitan de todas formas y se quiere evitar el límite de recursión.
- Reducción de memoria: cuando cada fila o valor depende solo de la anterior (rolling array).
- Base conceptual de todas las demás secciones de esta parte (Knapsack, DP de Intervalos, DP sobre Secuencias).

Complejidad: $O(n)$ tiempo en ambas versiones, porque hay $n + 1$ estados y cada uno se calcula una vez con trabajo $O(1)$. La memoización usa además $O(n)$ de pila de recursión; la tabulación usa $O(n)$ de tabla, reducible a $O(1)$ con dos variables.

```

1 // CREATE/READ/WRITE: Fibonacci resuelto de tres formas para
  // comparar
2 // memoizacion (top-down), tabulacion (bottom-up) y espacio
  // constante
3 struct FibonacciDP {
4     vector<long long> memo; // memo[i] = F(i), o -1 si aun no se
      calculo
5
6     // CREATE: prepara la tabla de memoizacion con centinela -1
7     FibonacciDP(int n) {
8         memo.assign(n + 1, -1);
9     } // O(n)
10
11     // READ: memoizacion (top-down): recursion natural + consulta
      // a la tabla
12     long long memoizado(int n) {
13         if (n <= 1) return n; // casos base
14         if (memo[n] != -1) return memo[n]; // estado ya resuelto

```

```

15     memo[n] = memoizado(n - 1) + memoizado(n - 2);
16     return memo[n];
17 } // O(n) tiempo, O(n) pila de recursion
18
19 // READ: tabulacion (bottom-up): llena la tabla de menor a
    mayor
20 static long long tabulado(int n) {
21     if (n <= 1) return n;
22     vector<long long> dp(n + 1, 0);
23     dp[1] = 1;
24     for (int i = 2; i <= n; i++) dp[i] = dp[i - 1] + dp[i - 2];
25     return dp[n];
26 } // O(n) tiempo, O(n) espacio
27
28 // READ: tabulacion con espacio constante: solo se conservan
    los dos
29 // ultimos valores, porque F(i) depende unicamente de F(i-1)
    y F(i-2)
30 static long long espacioConstante(int n) {
31     if (n <= 1) return n;
32     long long anterior = 0, actual = 1;
33     for (int i = 2; i <= n; i++) {
34         long long siguiente = anterior + actual;
35         anterior = actual;
36         actual = siguiente;
37     }
38     return actual;
39 } // O(n) tiempo, O(1) espacio
40
41 //
42 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
43 //
44 // 1. Estados dispersos o muy grandes (por ejemplo indices
    hasta 1e18
45 // o claves compuestas): reemplazar el vector por
46 // unordered_map<long long, long long> memo y consultar
    con
47 // memo.find(n); la tabla solo guarda los estados
    realmente visitados.
48 // 2. Recursion demasiado profunda (n ~ 1e6 puede desbordar
    la pila):
49 // convertir la memoizacion a tabulacion, o aumentar el
    limite de
50 // pila del sistema; la tabulacion no tiene este problema.
51 // 3. Resultado modular: sumar y reducir en cada paso,
52 // dp[i] = (dp[i-1] + dp[i-2]) % MOD, para evitar overflow
    en n grande.
53 // Sin modulo, F(90) es el ultimo Fibonacci que cabe en
    long long.
54 // 4. n astronomico (n ~ 1e18): ni la memoizacion ni la
    tabulacion
55 // alcanzan; se usa exponenciacion de matrices
56 // [[1,1],[1,0]]^n en O(log n).
57 // 5. Centinela de la tabla: usar -1 como "no calculado" solo
    es valido
58 // si -1 no puede ser una respuesta real; si lo fuera,
    usar un
59 // vector<bool> visitado aparte.
60 // 6. Rolling array en DP de dos dimensiones: si dp[i][j]
    solo depende
61 // de la fila i-1, guardar dos filas (o una sola,
    recorriendo j en
62 // el orden correcto) en vez de la tabla completa.
63 };

```

Ejemplo de funcionamiento:

```

1 FibonacciDP f(50);
2
3 long long a = f.memoizado(10);
4 // a = 55
5
6 long long b = FibonacciDP::tabulado(10);
7 // b = 55 (mismo resultado, calculado de menor a mayor)
8
9 long long c = FibonacciDP::espacioConstante(50);
10 // c = 12586269025 (con solo dos variables en memoria)

```

Nota: las tres versiones dan siempre el mismo resultado; lo que cambia es el orden de cálculo y la memoria. Un error frecuente al pasar de memoización a tabulación es llenar la tabla en un orden en que un estado se calcula **antes** de que sus dependencias estén listas. Antes de escribir el bucle, conviene identificar de qué estados depende cada uno.

Regla práctica: empieza siempre por la recurrencia y los casos base. Si el orden de llenado no es obvio o solo

una parte de los estados es alcanzable, usa memoización. Si el problema tiene un orden natural (por índice, por longitud creciente) y n es grande, usa tabulación, y reduce la memoria con *rolling array* cuando cada estado dependa solo de los anteriores inmediatos. Esta comparación se aplica a todas las subsecciones siguientes de esta parte.

34.2 Kadane's Algorithm (Máximo Subarreglo)

Idea: dado un arreglo de n enteros (que pueden ser negativos), se busca el **subarreglo contiguo** cuya suma sea máxima. Sea $dp[i]$ la suma máxima de un subarreglo contiguo que **termina exactamente** en el índice i . En cada posición hay solo dos opciones: **extender** el mejor subarreglo que termina en $i - 1$ sumándole $a[i]$, o **empezar de nuevo** en i si arrastrar lo anterior perjudica más de lo que ayuda —es decir, $dp[i] = \max(a[i], dp[i - 1] + a[i])$ —. La decisión de "empezar de nuevo" tiene sentido exactamente cuando $dp[i - 1] < 0$: arrastrar una suma negativa nunca ayuda. La respuesta final es $\max_i dp[i]$, no $dp[n - 1]$, porque el subarreglo óptimo puede terminar en cualquier posición.

¿Por qué usarlo? Probar todos los $O(n^2)$ subarreglos (o $O(n^3)$ sin sumas de prefijo) es la fuerza bruta directa. Kadane resuelve el mismo problema en $O(n)$ con un único recorrido, porque $dp[i]$ resume todo lo que hace falta saber de los subarreglos que terminan en i sin necesidad de volver a mirar atrás. Es el ejemplo más simple de DP de un solo arreglo con **espacio** $O(1)$: como $dp[i]$ solo depende de $dp[i - 1]$, no hace falta guardar la tabla completa, solo el valor anterior — el mismo patrón de *rolling array* mencionado en *Memoización vs. Tabulación*.

Casos de uso:

- Máximo subarreglo contiguo en un arreglo con valores positivos y negativos mezclados.
- Máxima ganancia de una única transacción de compra-venta (diferencia de precios consecutivos transformada en un arreglo de diferencias, y Kadane sobre ese arreglo).
- Como subrutina de *Stress Testing*: ejemplo estándar de solución rápida a verificar contra fuerza bruta $O(n^2)$.
- Base conceptual de variantes 2D (máxima submatriz) que fijan dos filas y aplican Kadane 1D sobre la suma de columnas entre esas filas.
- Punto de partida para variantes con restricciones: longitud máxima o mínima del subarreglo, o exactamente k elementos.

Complejidad: $O(n)$ tiempo con un único recorrido del arreglo; $O(1)$ espacio, manteniendo solo el mejor valor que termina en la posición actual y el mejor global visto hasta ahora.

```

1 // CREATE/READ: Kadane's Algorithm: suma maxima de un
    subarreglo contiguo,
2 // con reconstruccion de los indices del subarreglo optimo
3 struct Kadane {
4     vector<long long> a;
5
6     // CREATE: guarda el arreglo

```

Ejemplo de funcionamiento:

```

1 Kadane k({-2, 1, -3, 4, -1, 2, 1, -5, 4});
2
3 long long resultado = k.sumaMaxima();
4 // resultado = 6
5 // (el subarreglo 4, -1, 2, 1 suma 6, el maximo posible)
6
7 auto rango = k.reconstruir();
8 // rango = {3, 6}
9 // (índices 3 a 6, correspondientes a los valores 4, -1, 2, 1)

```

Nota: el error más común es inicializar `mejorActual` en 0 en vez de `a[0]` cuando el problema exige que el subarreglo sea **no vacío**. Con esa inicialización incorrecta, un arreglo completamente negativo (como `{-5, -3, -8}`) devuelve 0 en vez del valor correcto `-3` (el menos negativo), porque el algoritmo "tiene permitido" no tomar nada. Revisa siempre si el enunciado exige un subarreglo no vacío antes de decidir la inicialización.

Regla práctica: usa Kadane como la solución por defecto para "máxima suma de subarreglo contiguo" — es $O(n)$, $O(1)$ de espacio, y casi nunca hace falta nada más complejo. Antes de escribirlo, confirma si el subarreglo debe ser no vacío (inicializa en `a[0]`) o puede ser vacío con suma mínima 0 (inicializa en 0, ver modificación 2), y si el arreglo es circular (ver modificación 4) — son los dos detalles que más distinguen un enunciado de otro.

34.3 Triángulo de Pascal

Idea: el triángulo de Pascal organiza los coeficientes binomiales $\binom{n}{k}$ en filas: la fila n contiene $\binom{n}{0}, \binom{n}{1}, \dots, \binom{n}{n}$. Cada número es la suma de los dos que tiene encima: $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$, con $\binom{n}{0} = \binom{n}{n} = 1$. La recurrencia tiene una lectura combinatoria directa. Para elegir k elementos de n , se fija un elemento especial: o se incluye (faltan $k-1$ por elegir entre los otros $n-1$) o no se incluye (faltan k entre los otros $n-1$). Como los subproblemas $\binom{n-1}{k}$ se reutilizan en toda la fila siguiente, es un caso de DP muy claro sobre *Memoización vs. Tabulación*: la tabla se llena fila por fila, y cada celda solo necesita la fila anterior.

¿Por qué usarlo? La fórmula cerrada $\binom{n}{k} = \frac{n!}{k!(n-k)!}$ desborda con facilidad (ya $21!$ no cabe en `long long`) y requiere inversos modulares cuando se trabaja módulo un primo. El triángulo evita por completo divisiones y factoriales: solo usa **sumas**, así que funciona con cualquier módulo (primo o no) y es exacto mientras el resultado quepa en el tipo. Su costo es $O(n^2)$ de precomputación, que es aceptable cuando n es del orden de 10^3 a $5 \cdot 10^3$ y se necesitan muchas consultas $\binom{n}{k}$ distintas. Si n es muy grande (por ejemplo 10^6), conviene la fórmula con factoriales precomputados e inversos modulares.

Casos de uso:

- Precomputar una tabla de $\binom{n}{k}$ para responder muchas consultas en $O(1)$.
- Contar caminos en una grilla que solo avanza a la derecha y hacia abajo: $\binom{f+c}{f}$ para f pasos hacia abajo y c pasos a la derecha.
- Contar subconjuntos de tamaño k , o formas de repartir objetos en problemas combinatorios de conteo.
- Coeficientes del desarrollo de $(x+y)^n$ y de otros polinomios sencillos.

```

7 Kadane(const vector<long long>& v) : a(v) {
8 } // O(n)
9
10 // READ: suma maxima de un subarreglo contiguo no vacío, en O
11 // (1) espacio
12 long long sumaMaxima() const {
13     long long mejorActual = a[0], mejorGlobal = a[0];
14     for (size_t i = 1; i < a.size(); i++) {
15         mejorActual = max(a[i], mejorActual + a[i]);
16         mejorGlobal = max(mejorGlobal, mejorActual);
17     }
18     return mejorGlobal;
19 } // O(n) tiempo, O(1) espacio
20
21 // READ: retorna {inicio, fin} (índices 0-indexados,
22 // inclusive) del
23 // subarreglo optimo
24 pair<int,int> reconstruir() const {
25     long long mejorActual = a[0], mejorGlobal = a[0];
26     int inicioActual = 0, mejorInicio = 0, mejorFin = 0;
27     for (size_t i = 1; i < a.size(); i++) {
28         if (a[i] > mejorActual + a[i]) {
29             mejorActual = a[i];
30             inicioActual = (int)i; // conviene "empezar de nuevo"
31             aqui
32         } else {
33             mejorActual += a[i];
34         }
35         if (mejorActual > mejorGlobal) {
36             mejorGlobal = mejorActual;
37             mejorInicio = inicioActual;
38             mejorFin = (int)i;
39         }
40     }
41     return {mejorInicio, mejorFin};
42 } // O(n) tiempo, O(1) espacio
43
44 //
45 // -----
46 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
47 // -----
48 // 1. Arreglo con TODOS los elementos negativos: el código de
49 // arriba
50 // ya maneja este caso correctamente (devuelve el elemento
51 // negativo
52 // menos malo), porque el subarreglo debe ser NO VACIO por
53 // definicion del problema; no inicializar mejorActual en
54 // 0, sino
55 // en a[0], es lo que garantiza esto.
56 // 2. Permitir subarreglo VACIO (suma minima posible es 0, no
57 // un
58 // numero negativo): inicializar mejorActual y mejorGlobal
59 // en 0 en
60 // vez de a[0], y en cada paso hacer
61 // mejorActual = max(0LL, mejorActual + a[i]).
62 // 3. Minimo subarreglo (en vez de maximo): negar todos los
63 // valores,
64 // aplicar Kadane normal, y negar el resultado; o invertir
65 // los max
66 // por min directamente en el código.
67 // 4. Maxima suma circular (el subarreglo puede "dar la
68 // vuelta" del
69 // final al principio): la respuesta es
70 // max(Kadane normal, sumaTotal - Kadane del MINIMO
71 // subarreglo),
72 // con el caso especial de que si TODO el arreglo es
73 // negativo, el
74 // segundo termino da 0 por error y hay que devolver solo
75 // el
76 // Kadane normal.
77 // 5. Maxima submatriz 2D: fijar cada par de filas (fSup,
78 // fInf) en
79 // O(n^2) combinaciones, calcular el arreglo de sumas de
80 // columna
81 // entre esas filas en O(m), y aplicar Kadane 1D sobre ese
82 // arreglo
83 // en O(m); total O(n^2 * m).
84 // 6. Restriccion de longitud MINIMA k: mantener una suma de
85 // prefijo y,
86 // para cada posicion i >= k, comparar prefijo[i] - min(
87 // prefijo[0..i-k]);
88 // Kadane simple no respeta un minimo de longitud por si
89 // solo.
90 // 7. Ganancia maxima de UNA transaccion de compra-venta de
91 // acciones:
92 // aplicar Kadane sobre el arreglo de diferencias
93 // consecutivas
94 // precio[i] - precio[i-1]; el resultado es la maxima
95 // ganancia de
96 // comprar en un dia y vender en uno posterior.
97 };

```

- Trabajar con un módulo compuesto (no primo), donde los inversos modulares no siempre existen.

Complejidad: $O(n^2)$ tiempo y $O(n^2)$ espacio para construir la tabla hasta la fila n , ya que hay $\frac{(n+1)(n+2)}{2}$ celdas y cada una se calcula con una suma en $O(1)$. Una consulta posterior es $O(1)$. Para obtener solo una fila, $O(n^2)$ tiempo pero $O(n)$ espacio.

```

1 // CREATE/READ/WRITE: Triangulo de Pascal para coeficientes
  binomiales,
2 // con modulo opcional (mod = 0 significa sin modulo)
3 struct TrianguloPascal {
4     vector<vector<long long>> tabla; // tabla[i][j] = C(i, j)
5     long long mod;
6
7     // CREATE: construye las filas 0..n; cada celda es la suma de
      las dos
8     // que tiene encima (tabulacion fila por fila)
9     TrianguloPascal(int n, long long m = 0) : mod(m) {
10         tabla.assign(n + 1, vector<long long>());
11         for (int i = 0; i <= n; i++) {
12             tabla[i].assign(i + 1, 1); // bordes C(i,0) = C(i,i) = 1
13             for (int j = 1; j < i; j++) {
14                 long long v = tabla[i - 1][j - 1] + tabla[i - 1][j];
15                 if (mod > 0) v %= mod;
16                 tabla[i][j] = v;
17             }
18         }
19     } // O(n^2)
20
21     // READ: coeficiente binomial C(n, k); devuelve 0 si k esta
      fuera de rango
22     long long combinacion(int n, int k) const {
23         if (k < 0 || k > n) return 0;
24         return tabla[n][k];
25     } // O(1)
26
27     // READ: fila completa n del triangulo
28     const vector<long long>& fila(int n) const {
29         return tabla[n];
30     } // O(1)
31
32     // READ: una sola fila n con espacio O(n), sin guardar la
      tabla completa
33     // (rolling array: se actualiza de derecha a izquierda para
      no pisar
34     // valores que todavia hacen falta)
35     static vector<long long> filaUnica(int n) {
36         vector<long long> f(n + 1, 0);
37         f[0] = 1;
38         for (int i = 1; i <= n; i++) {
39             for (int j = i; j >= 1; j--) f[j] += f[j - 1];
40         }
41         return f;
42     } // O(n^2) tiempo, O(n) espacio
43
44     //
45     // MODIFICACIONES Y COMENTARIOS CON LOS USOS
46     //
47     // 1. Overflow: sin modulo, C(n,k) cabe en long long solo
      hasta n = 66
48     // (C(66,33) ~ 7.2e18). Para n mayor, usar un modulo (por
      ejemplo
49     // 1e9+7) o __int128 si se necesita el valor exacto pero
      acotado.
50     // 2. Modulo compuesto: este triangulo funciona con cualquier
      modulo,
51     // incluso no primo, porque solo suma; la formula con
      factoriales e
52     // inversos modulares exige un modulo primo.
53     // 3. n grande (~1e6) con muchas consultas: precomputar fact[
      i] e
54     // invFact[i] mod p primo, y responder
55     // C(n,k) = fact[n] * invFact[k] % p * invFact[n-k] % p en
      O(1),
56     // con precomputo O(n) en vez de O(n^2).
57     // 4. Caminos en grilla que solo avanza derecha o abajo, con
      f pasos
58     // hacia abajo y c hacia la derecha: combinacion(f + c, f).
59
60     // 5. Suma de la fila n: siempre 2^n, ya que cada elemento de
      un
61     // conjunto de n se incluye o no; util como chequeo de
      cordura.
62     // 6. Identidades utiles: C(n,k) = C(n,n-k) (simetria de cada
      fila) y
63     // C(n,0)+C(n,1)+...+C(n,n) = 2^n; permiten calcular solo
      la mitad

```

```

83 // de cada fila si se necesita ahorrar tiempo.
84 // 7. Aplicar el modulo en cada suma (como hace el
      constructor con
85 // mod): reducir solo al final no evita el desbordamiento
      intermedio.
86 };

```

Ejemplo de funcionamiento:

```

1 TrianguloPascal t(10);
2
3 long long a = t.combinacion(5, 2);
4 // a = 10 (formas de elegir 2 elementos entre 5)
5
6 vector<long long> fila5 = t.fila(5);
7 // fila5 = {1, 5, 10, 10, 5, 1}
8 // (simetrica; su suma es 32 = 2^5)
9
10 long long b = t.combinacion(10, 3);
11 // b = 120
12 // (por la recurrencia: C(10,3) = C(9,2) + C(9,3) = 36 + 84 =
      120)
13
14 vector<long long> f6 = TrianguloPascal::filaUnica(6);
15 // f6 = {1, 6, 15, 20, 15, 6, 1} (misma fila, calculada con
      memoria O(n))

```

Regla práctica: usa el triángulo de Pascal cuando n sea moderado (hasta unos pocos miles), el módulo sea arbitrario o no exista, y necesites muchas consultas $\binom{n}{k}$ en $O(1)$. Si n llega a 10^5 o más y el módulo es primo, cambia a factoriales con inversos modulares, que precomputan en $O(n)$. Si solo necesitas una fila, usa `filaUnica` para no gastar $O(n^2)$ de memoria.

Idea: el triángulo de Pascal organiza los coeficientes binomiales $\binom{n}{k}$ en filas: la fila n contiene $\binom{n}{0}, \binom{n}{1}, \dots, \binom{n}{n}$. Cada número es la suma de los dos que tiene encima: $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$, con $\binom{n}{0} = \binom{n}{n} = 1$. La recurrencia tiene una lectura combinatoria directa. Para elegir k elementos de n , se fija un elemento especial: o se incluye (faltan $k-1$ por elegir entre los otros $n-1$) o no se incluye (faltan k entre los otros $n-1$). Como los subproblemas $\binom{n-1}{\cdot}$ se reutilizan en toda la fila siguiente, es un caso de DP muy claro sobre *Memoización vs. Tabulación*: la tabla se llena fila por fila, y cada celda solo necesita la fila anterior.

¿Por qué usarlo? La fórmula cerrada $\binom{n}{k} = \frac{n!}{k!(n-k)!}$ desborda con facilidad (ya $20!$ es el límite de `long long`) y requiere inversos modulares cuando se trabaja módulo un primo. El triángulo evita por completo divisiones y factoriales: solo usa **sumas**, así que funciona con cualquier módulo (primo o no) y es exacto mientras el resultado quepa en el tipo. Su costo es $O(n^2)$ de precomputación, que es aceptable cuando n es del orden de 10^3 a $5 \cdot 10^3$ y se necesitan muchas consultas $\binom{n}{k}$ distintas. Si n es muy grande (por ejemplo 10^6), conviene la fórmula con factoriales precomputados e inversos modulares.

Casos de uso:

- Precomputar una tabla de $\binom{n}{k}$ para responder muchas consultas en $O(1)$.
- Contar caminos en una grilla que solo avanza a la derecha y hacia abajo: $\binom{f+c}{f}$ para f filas y c columnas de pasos.
- Contar subconjuntos de tamaño k , o formas de repartir objetos en problemas combinatorios de conteo.
- Coeficientes del desarrollo de $(x+y)^n$ y de otros polinomios sencillos.

- Trabajar con un módulo compuesto (no primo), donde los inversos modulares no siempre existen.

Complejidad: $O(n^2)$ tiempo y $O(n^2)$ espacio para construir la tabla hasta la fila n , ya que hay $\frac{(n+1)(n+2)}{2}$ celdas y cada una se calcula con una suma en $O(1)$. Una consulta posterior es $O(1)$. Para obtener solo una fila, $O(n^2)$ tiempo pero $O(n)$ espacio.

```

1 // CREATE/READ/WRITE: Triangulo de Pascal para coeficientes
  binomiales,
2 // con modulo opcional (mod = 0 significa sin modulo)
3 struct TrianguloPascal {
4     vector<vector<long long>> tabla; // tabla[i][j] = C(i, j)
5     long long mod;
6
7     // CREATE: construye las filas 0..n; cada celda es la suma de
      las dos
8     // que tiene encima (tabulacion fila por fila)
9     TrianguloPascal(int n, long long m = 0) : mod(m) {
10         tabla.assign(n + 1, vector<long long>());
11         for (int i = 0; i <= n; i++) {
12             tabla[i].assign(i + 1, 1); // bordes C(i,0) = C(i,i) = 1
13             for (int j = 1; j < i; j++) {
14                 long long v = tabla[i - 1][j - 1] + tabla[i - 1][j];
15                 if (mod > 0) v %= mod;
16                 tabla[i][j] = v;
17             }
18         }
19     } // O(n^2)
20
21     // READ: coeficiente binomial C(n, k); devuelve 0 si k esta
      fuera de rango
22     long long combinacion(int n, int k) const {
23         if (k < 0 || k > n) return 0;
24         return tabla[n][k];
25     } // O(1)
26
27     // READ: fila completa n del triangulo
28     const vector<long long>& fila(int n) const {
29         return tabla[n];
30     } // O(1)
31
32     // READ: una sola fila n con espacio O(n), sin guardar la
      tabla completa
33     // (rolling array: se actualiza de derecha a izquierda para
      no pisar
34     // valores que todavia hacen falta)
35     static vector<long long> filaUnica(int n) {
36         vector<long long> f(n + 1, 0);
37         f[0] = 1;
38         for (int i = 1; i <= n; i++) {
39             for (int j = i; j >= 1; j--) f[j] += f[j - 1];
40         }
41         return f;
42     } // O(n^2) tiempo, O(n) espacio
43
44     //
45     // MODIFICACIONES Y COMENTARIOS CON LOS USOS
46     //
47     // 1. Overflow: sin modulo, C(n,k) cabe en long long solo
      hasta n = 66
48     // (C(66,33) ~ 7.2e18). Para n mayor, usar un modulo (por
      ejemplo
49     // 1e9+7) o __int128 si se necesita el valor exacto pero
      acotado.
50     // 2. Modulo compuesto: este triangulo funciona con cualquier
      modulo,
51     // incluso no primo, porque solo suma; la formula con
      factoriales e
52     // inversos modulares exige un modulo primo.
53     // 3. n grande (~1e6) con muchas consultas: precomputar fact[
      i] e
54     // invFact[i] mod p primo, y responder
55     // C(n,k) = fact[n] * invFact[k] % p * invFact[n-k] % p en
      O(1),
56     // con precomputo O(n) en vez de O(n^2).
57     // 4. Caminos en grilla que solo avanza derecha o abajo, con
      f pasos
58     // hacia abajo y c hacia la derecha: combinacion(f + c, f).
59     // 5. Suma de la fila n: siempre 2^n, ya que cada elemento de
      un
60     // conjunto de n se incluye o no; util como chequeo de
      cordura.
61     // 6. Identidades utiles: C(n,k) = C(n,n-k) (simetria de cada
      fila) y
62     // C(n,0)+C(n,1)+...+C(n,n) = 2^n; permiten calcular solo
      la mitad

```

```

53 // de cada fila si se necesita ahorrar tiempo.
54 };

```

Ejemplo de funcionamiento:

```

1 TrianguloPascal t(10);
2
3 long long a = t.combinacion(5, 2);
4 // a = 10 (formas de elegir 2 elementos entre 5)
5
6 vector<long long> fila5 = t.fila(5);
7 // fila5 = {1, 5, 10, 10, 5, 1}
8 // (simetrica; su suma es 32 = 2^5)
9
10 long long b = t.combinacion(10, 3);
11 // b = 120 (C(10,3) = 45 + 75? no: se obtiene como C(9,2)+C
    (9,3) = 36 + 84 = 120)
12
13 vector<long long> f6 = TrianguloPascal::filaUnica(6);
14 // f6 = {1, 6, 15, 20, 15, 6, 1} (misma fila, calculada con
    memoria O(n))

```

Nota: en el comentario del ejemplo de $\binom{10}{3}$ hay una frase mal escrita por mi parte al redactarlo. El resultado 120 es correcto y proviene de $\binom{9}{2} + \binom{9}{3} = 36 + 84$; borra la parte "45 + 75? no:" al pegarlo en tu guía. Además, cuando el problema pide el valor módulo un número, aplica el módulo en cada suma dentro del constructor (como hace el código con mod); reducir solo al final no evita el desbordamiento intermedio.

Regla práctica: usa el triángulo de Pascal cuando n sea moderado (hasta unos pocos miles), el módulo sea arbitrario o no exista, y necesites muchas consultas $\binom{n}{k}$ en $O(1)$. Si n llega a 10^5 o más y el módulo es primo, cambia a factoriales con inversos modulares, que precomputan en $O(n)$. Si solo necesitas una fila, usa `filaUnica` para no gastar $O(n^2)$ de memoria.

34.4 Camino Más Largo en una Matriz

Idea: dada una matriz de enteros, se busca la longitud del camino **estrictamente creciente** más largo, donde desde cada celda se puede avanzar a sus 4 vecinos (arriba, abajo, izquierda, derecha). Sea $dp[f][c]$ la longitud del camino creciente más largo que **empieza** en la celda (f, c) . Su recurrencia es $dp[f][c] = 1 + \max\{dp[f'][c'] : (f', c') \text{ vecino, } a[f'][c'] > a[f][c]\}$, y vale 1 si ningún vecino es mayor. La condición de **estrictamente creciente** es lo que hace válida la DP: como solo se avanza hacia valores mayores, nunca se puede volver a una celda ya visitada, así que las dependencias forman un grafo sin ciclos y cada estado se puede calcular una sola vez, tal como se explicó en *Memoización vs. Tabulación*.

¿Por qué usarlo? Sin memoización, una búsqueda en profundidad desde cada celda explora todos los caminos crecientes posibles, que pueden ser exponenciales en el tamaño de la matriz. Al guardar el valor de cada celda la primera vez que se calcula, cada una se resuelve una sola vez con trabajo $O(1)$ (revisar 4 vecinos), y el total baja a $O(nm)$. Aquí la memoización es la opción natural: el orden de llenado depende de los valores de la matriz y no de los índices, así que no hay un recorrido fila por fila que sirva para tabular directamente. Si se prefiere evitar la recursión, se puede tabular procesando las celdas ordenadas por valor (ver modificación 1 dentro del código).

Casos de uso:

- Encontrar la ruta creciente más larga en una matriz de alturas o valores (problema clásico de *Longest Increasing Path in a Matrix*).

- Caminos más largos en un grafo dirigido acíclico (DAG) dado implícitamente por una grilla.
- Juegos o simulaciones donde cada movimiento debe llevar a una casilla de mayor valor, y se pide la mayor cantidad de pasos posibles.
- Reconstruir la ruta concreta que logra ese máximo, no solo su longitud.
- Base conceptual para DP sobre DAG: el mismo esquema (memoizar la mejor respuesta desde cada nodo) sirve para cualquier grafo sin ciclos.

Complejidad: $O(nm)$ tiempo para una matriz de n filas y m columnas: hay nm estados, cada uno se calcula una sola vez y examina a lo sumo 4 vecinos en $O(1)$. El espacio es $O(nm)$ para la tabla, más hasta $O(nm)$ de pila de recursión en el peor caso (una matriz cuyos valores forman una sola cadena creciente).

```

1 // CREATE/READ/WRITE: Camino estrictamente creciente mas largo
  // en una
2 // matriz (movimiento en 4 direcciones), con memoizacion sobre
  // cada celda
3 struct CaminoMasLargoMatriz {
4     vector<vector<int>>> a; // matriz de valores
5     vector<vector<int>>> memo; // memo[f][c] = longitud del mejor
      camino que EMPIEZA en (f,c); 0 = sin calcular
6     int filas, cols;
7     const int DF[4] = {-1, 1, 0, 0};
8     const int DC[4] = {0, 0, -1, 1};
9
10    // CREATE: guarda la matriz y prepara la tabla de memoizacion
      en 0
11    // (0 sirve como centinela porque todo camino real tiene
      longitud >= 1)
12    CaminoMasLargoMatriz(const vector<vector<int>>>& m) : a(m) {
13        filas = (int)a.size();
14        cols = filas > 0 ? (int)a[0].size() : 0;
15        memo.assign(filas, vector<int>(cols, 0));
16    } // O(n*m)
17
18    // READ: longitud del mejor camino creciente que empieza en (
      f,c),
19    // memoizado (top-down)
20    int desde(int f, int c) {
21        if (memo[f][c] != 0) return memo[f][c]; // estado ya
      resuelto
22        int mejor = 1; // el camino formado solo por esta celda
23        for (int d = 0; d < 4; d++) {
24            int nf = f + DF[d], nc = c + DC[d];
25            if (nf < 0 || nf >= filas || nc < 0 || nc >= cols)
      continue;
26            if (a[nf][nc] > a[f][c]) mejor = max(mejor, 1 + desde(nf,
      nc));
27        }
28        return memo[f][c] = mejor;
29    } // O(1) amortizado por celda, O(n*m) en total
30
31    // READ: longitud del camino creciente mas largo de toda la
      matriz
32    int longitudMaxima() {
33        int mejor = 0;
34        for (int f = 0; f < filas; f++)
35            for (int c = 0; c < cols; c++)
36                mejor = max(mejor, desde(f, c));
37        return mejor;
38    } // O(n*m)
39
40    // READ: reconstruye UN camino optimo como lista de celdas (
      fila, col),
41    // siguiendo en cada paso un vecino mayor con memo
      exactamente uno menos
42    vector<pair<int,int>>> reconstruir() {
43        int mejor = longitudMaxima();
44        vector<pair<int,int>>> camino;
45        if (mejor == 0) return camino;
46        int f = 0, c = 0;
47        for (int i = 0; i < filas; i++)
48            for (int j = 0; j < cols; j++)
49                if (memo[i][j] == mejor) { f = i; c = j; }
50        camino.push_back({f, c});
51        while (memo[f][c] > 1) {
52            for (int d = 0; d < 4; d++) {
53                int nf = f + DF[d], nc = c + DC[d];

```

```

54        if (nf < 0 || nf >= filas || nc < 0 || nc >= cols)
      continue;
55        if (a[nf][nc] > a[f][c] && memo[nf][nc] == memo[f][c] -
      1) {
56            f = nf; c = nc;
57            camino.push_back({f, c});
58            break;
59        }
60    }
61    }
62    return camino;
63 } // O(n*m)
64
65 // READ: version iterativa (bottom-up) sin recursion: se
      ordenan las
66 // celdas por valor DECRECIENTE, asi cuando se procesa una
      celda todos
67 // sus vecinos mayores ya tienen su dp calculado
68 static int longitudMaximaIterativa(const vector<vector<int>>>&
      m) {
69     int n = (int)m.size();
70     if (n == 0) return 0;
71     int k = (int)m[0].size();
72     vector<array<int,3>>> celdas; // {valor, fila, col}
73     for (int i = 0; i < n; i++)
74         for (int j = 0; j < k; j++) celdas.push_back({m[i][j], i, j
      });
75     sort(celdas.begin(), celdas.end(), [](const array<int,3>& x
      , const array<int,3>& y) {
76         return x[0] > y[0];
77     });
78     int df[4] = {-1, 1, 0, 0}, dc[4] = {0, 0, -1, 1};
79     vector<vector<int>>> dp(n, vector<int>(k, 1));
80     int mejor = 1;
81     for (auto& cel : celdas) {
82         int f = cel[1], c = cel[2];
83         for (int d = 0; d < 4; d++) {
84             int nf = f + df[d], nc = c + dc[d];
85             if (nf < 0 || nf >= n || nc < 0 || nc >= k) continue;
86             if (m[nf][nc] > m[f][c]) dp[f][c] = max(dp[f][c], 1 +
      dp[nf][nc]);
87         }
88         mejor = max(mejor, dp[f][c]);
89     }
90     return mejor;
91 } // O(n*m log(n*m)), dominado por el sort
92
93 //
94 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
95 //
96 // 1. Sin recursion (matrices grandes, riesgo de desbordar la
      pila):
97 //     usar longitudMaximaIterativa(), que ordena las celdas
      por valor
98 //     decreciente y tabula; cambia O(n*m) por O(n*m log(n*m))
      pero
99 //     elimina la profundidad de recursion. Tambien se puede
      hacer en
100 //     O(n*m) con orden topologico (algoritmo de Kahn) sobre
      el grafo
101 //     de aristas "celda -> vecino mayor".
102 // 2. Camino DECRECIENTE en vez de creciente: cambiar a[nf][
      nc] > a[f][c]
103 //     por a[nf][nc] < a[f][c] en la condicion de vecino
      valido (y en
104 //     el orden del sort, si se usa la version iterativa).
105 // 3. Con 8 direcciones (se permiten diagonales): ampliar DF
      y DC a 8
106 //     entradas y recorrer d de 0 a 7; el resto del codigo no
      cambia.
107 // 4. Camino NO estricto (se permiten valores iguales): la DP
      deja de
108 //     ser valida, porque dos celdas vecinas iguales se
      podrian recorrer
109 //     de ida y vuelta formando ciclos; el problema pasaria a
      pedir
110 //     caminos simples y se vuelve mucho mas dificil (NP-
      dificil en
111 //     general), asi que solo se resuelve con la condicion
      estricta.
112 // 5. CONTAR el numero de caminos crecientes (con cualquier
      longitud):
113 //     cuentaDesde(f,c) = 1 + suma de cuentaDesde(vecino mayor
      ); el 1
114 //     cuenta el camino formado solo por la celda; la
      respuesta total es
115 //     la suma sobre todas las celdas (aplicar modulo si piden
      ).
116 // 6. Otras reglas de movimiento (por ejemplo saltos de tipo
      caballo
117 //     de ajedrez): solo cambia el arreglo de desplazamientos

```

```

118 // DF y DC,
119 // mientras la condicion de valor estrictamente mayor
120 // mantenga el
121 // grafo sin ciclos.
122 // 7. Ganancia acumulada en vez de longitud: si cada celda
123 // aporta un
124 // peso w[f][c] y se pide maximizar la suma, usar
125 // w[f][c] + max(desde(vecino mayor)) en lugar de 1 + max
126 (...).
127 };

```

Ejemplo de funcionamiento:

```

1 vector<vector<int>> m = {
2     {9, 9, 4},
3     {6, 6, 8},
4     {2, 1, 1}
5 };
6 CaminoMasLargoMatriz p(m);
7
8 int resultado = p.longitudMaxima();
9 // resultado = 4
10 // (el camino 1 -> 2 -> 6 -> 9 recorre las celdas (2,1), (2,0),
11 // (1,0), (0,0))
12
13 auto camino = p.reconstruir();
14 // camino = {(2,1), (2,0), (1,0), (0,0)}
15 // (valores 1, 2, 6, 9: cada paso va a un vecino estrictamente
16 // mayor)
17
18 vector<vector<int>> m2 = {
19     {3, 4, 5},
20     {3, 2, 6},
21     {2, 2, 1}
22 };
23 int r2 = CaminoMasLargoMatriz::longitudMaximaIterativa(m2);
24 // r2 = 4
25 // (el camino 3 -> 4 -> 5 -> 6 recorre (1,0), (0,0)... o (0,0),
26 // (0,1), (0,2), (1,2))

```

Nota: la DP funciona únicamente porque la condición es **estrictamente creciente**: eso garantiza que el grafo de celdas no tiene ciclos y que el valor guardado en memo nunca depende de sí mismo. Si el problema permite pasar entre celdas de igual valor, esta técnica deja de ser válida (ver la modificación 4 dentro del código). Además, usar 0 como centinela de memo es seguro aquí porque toda longitud real es al menos 1.

Regla práctica: usa memoización con DFS desde cada celda cuando la regla de movimiento obligue a ir siempre a valores estrictamente mayores (o menores): así es $O(nm)$ y la más corta de escribir. Si la matriz es tan grande que la recursión puede desbordar la pila, pasa a la versión iterativa ordenando las celdas por valor. Si en cambio la regla es solo “moverse a la derecha o hacia abajo” (sin condición de valores), no hace falta memoizar por valor: basta la tabulación fila por fila, como en *Triángulo de Pascal*.

35 Knapsack y Variantes

35.1 Knapsack 0/1

Idea: se tienen n objetos, cada uno con un peso p_i y un valor v_i , y una mochila de capacidad W . Cada objeto se puede tomar **una sola vez** o no tomarse (de ahí el “0/1”). Se busca el máximo valor total cuyo peso no exceda W . Sea $dp[i][w]$ el mejor valor usando solo los primeros i objetos con capacidad w . Para el objeto i hay dos opciones: no tomarlo ($dp[i-1][w]$) o tomarlo ($dp[i-1][w-p_i] + v_i$, solo si $p_i \leq w$), y se elige la mejor: $dp[i][w] = \max(dp[i-1][w], dp[i-1][w-p_i] + v_i)$. Como la fila i solo depende de la fila $i-1$, se puede guardar un único arreglo $dp[w]$ si se recorre w de **mayor a menor**: así $dp[w-p_i]$ todavía pertenece a la fila anterior y el objeto no se cuenta dos veces.

¿Por qué usarlo? La fuerza bruta prueba los 2^n subconjuntos de objetos. La DP baja el costo a $O(nW)$, porque el estado (i, w) ya resume todas las formas de decidir los primeros i objetos. Un *greedy* por razón valor/peso no sirve aquí: con objetos indivisibles puede dar una respuesta subóptima (la versión fraccionaria sí es greedy, ver modificación 7). Frente a *Cambio de Monedas*, la diferencia es que aquí cada objeto se usa a lo sumo una vez, y eso obliga a recorrer la capacidad en sentido descendente. Es **pseudopolinomial**: el costo depende del valor numérico de W , no solo del tamaño de la entrada.

Casos de uso:

- Elegir el subconjunto de proyectos, ítems o tareas de mayor beneficio con un presupuesto o tiempo limitado.
- Decidir si existe un subconjunto con suma exacta (*Subset Sum*), que es Knapsack con valor igual al peso.
- Problemas de partición y reparto equilibrado, como *Problema de Partición*.
- Selección de objetos con dos restricciones a la vez, usando una dimensión de DP adicional.
- Base de muchas variantes de mochila: acotada, de múltiples grupos, con dependencias.

Complejidad: $O(nW)$ tiempo, porque hay $n \cdot (W + 1)$ estados y cada uno se calcula en $O(1)$. El espacio es $O(W)$ con el arreglo unidimensional y $O(nW)$ si se guarda la tabla completa para reconstruir los objetos elegidos.

```

1 // CREATE/READ/WRITE: Knapsack 0/1 con arreglo unidimensional y
2 // reconstruccion de los objetos elegidos con tabla completa
3 struct Knapsack01 {
4     int n;
5     int capacidad;
6     vector<int> peso;
7     vector<long long> valor;
8
9     // CREATE: guarda los objetos y la capacidad de la mochila
10    Knapsack01(const vector<int>& p, const vector<long long>& v,
11               int W)
12    : n((int)p.size()), capacidad(W), peso(p), valor(v) {
13    } // O(n)
14
15    // READ: maximo valor con peso total <= capacidad (espacio O(W))
16    long long maximoValor() const {
17        vector<long long> dp(capacidad + 1, 0);
18        for (int i = 0; i < n; i++) {
19            // w decreciente: dp[w - peso[i]] aun es de la fila
20            // anterior
21            for (int w = capacidad; w >= peso[i]; w--) {
22                dp[w] = max(dp[w], dp[w - peso[i]] + valor[i]);
23            }
24        }
25        return dp[capacidad];
26    } // O(n*W) tiempo, O(W) espacio
27
28    // READ: indices (0-indexados, ascendentes) de un conjunto
29    // optimo de
30    // objetos; usa la tabla completa dp[i][w] para poder
31    // retroceder
32    vector<int> reconstruir() const {
33        vector<vector<long long>> dp(n + 1, vector<long long>(
34            capacidad + 1, 0));
35        for (int i = 1; i <= n; i++) {
36            for (int w = 0; w <= capacidad; w++) {
37                dp[i][w] = dp[i-1][w];
38                if (w >= peso[i-1]) {
39                    dp[i][w] = max(dp[i][w], dp[i-1][w - peso[i-1]] +
40                        valor[i-1]);
41                }
42            }
43        }
44    }

```

```

36     }
37 }
38 vector<int> elegidos;
39 int w = capacidad;
40 for (int i = n; i >= 1; i--) {
41     if (dp[i][w] != dp[i - 1][w]) { // el objeto i fue
necesario
42         elegidos.push_back(i - 1);
43         w -= peso[i - 1];
44     }
45 }
46 reverse(elegidos.begin(), elegidos.end());
47 return elegidos;
48 } // O(n*W) tiempo y espacio
49
50 //
51 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
52 //
53 // 1. Peso EXACTO igual a la capacidad (mochila que debe
llenarse):
54 //     inicializar dp[0] = 0 y dp[w] = -INF para w > 0 (por
ejemplo
55 //     -1e18), y al actualizar ignorar los estados -INF. La
respuesta es
56 //     dp[capacidad], o "imposible" si sigue siendo -INF.
57 // 2. Solo factibilidad (Subset Sum): usar vector<bool>
alcanzable(W+1)
58 //     con alcanzable[0] = true, y para cada objeto, w
decreciente:
59 //     alcanzable[w] = alcanzable[w] || alcanzable[w - peso[i]
]].
60 //     Con bitset<MAXW> se reduce a: mascara |= (mascara <<
peso[i]);
61 //     ver Bitset para Optimizacion en la parte de BitWise.
62 // 3. Contar el NUMERO de subconjuntos con suma exacta W:
cuenta[0] = 1,
63 //     y para cada objeto, w decreciente:
64 //     cuenta[w] += cuenta[w - peso[i]] (aplicar modulo si lo
piden).
65 // 4. Pesos enormes pero valores pequenos (W ~ 1e9, suma de
valores
66 //     <= -1e5): invertir los roles del estado. Sea minPeso[w]
el minimo
67 //     peso para lograr valor v: minPeso[0] = 0, resto INF, y
para cada
68 //     objeto, v decreciente: minPeso[v] = min(minPeso[v],
69 //     minPeso[v - valor[i]] + peso[i]). La respuesta es el
mayor v con
70 //     minPeso[v] <= W. Costo O(n * suma de valores).
71 // 5. Objetos con cantidad limitada c_i (mochila acotada):
descomponer
72 //     c_i en potencias de dos (1, 2, 4, ..., resto) y tratar
cada trozo
73 //     como un objeto 0/1 de peso k*peso[i] y valor k*valor[i];
el costo
74 //     baja a O(W * sum log c_i). Si c_i es ilimitado, ver
Knapsack
75 //     Ilimitado.
76 // 6. n pequeno (n <= 40) pero W enorme: DP inviable, usar
meet in the
77 //     middle sobre subconjuntos (ver Meet in the Middle con
Mascaras
78 //     de Bits en BitWise).
79 // 7. Objetos DIVISIBLES (mochila fraccionaria): no es DP. Se
ordenan
80 //     por valor/peso decreciente y se toman completos
mientras quepan,
81 //     y del ultimo se toma la fraccion restante (greedy
correcto solo
82 //     en esta version).
83 // 8. Dos restricciones (por ejemplo peso y volumen): agregar
una
84 //     dimension a dp, dp[w1][w2], recorriendo ambas en
sentido
85 //     decreciente; el costo pasa a O(n * W1 * W2).
86 // 9. Orden de los bucles: el bucle de objetos va AFUERA y el
de
87 //     capacidad ADENTRO en sentido decreciente. Si se recorre
w en
88 //     sentido creciente, un mismo objeto se puede usar varias
veces y
89 //     se obtiene Knapsack Ilimitado, no el 0/1.
90 };

```

Ejemplo de funcionamiento:

```

1 vector<int> peso = {1, 3, 4, 5};
2 vector<long long> valor = {1, 4, 5, 7};
3 Knapsack01 k(peso, valor, 7);
4
5 long long mejor = k.maximoValor();

```

```

6 // mejor = 9
7 // (tomar los objetos de peso 3 y 4: peso total 7, valor 4 + 5
= 9;
8 // la alternativa 1 + 5 pesa 6 y vale 1 + 7 = 8, que es menor)
9
10 vector<int> elegidos = k.reconstruir();
11 // elegidos = {1, 2}
12 // (los indices 1 y 2 corresponden a los objetos de peso 3 y 4)

```

Nota: el error más común es recorrer la capacidad de menor a mayor en la versión unidimensional. Eso permite reutilizar el mismo objeto dentro de una misma pasada y convierte silenciosamente el problema en Knapsack Ilimitado, sin ningún error visible. Recuerda también que la complejidad $O(nW)$ solo es aceptable si W es moderado (típicamente hasta 10^5 o 10^6); si W es enorme, revisa la modificación 4 o la 6.

Regla práctica: usa Knapsack 0/1 cuando cada objeto se pueda tomar como máximo una vez y la capacidad sea numéricamente manejable. Recorre siempre la capacidad en sentido decreciente. Si los objetos se pueden repetir, usa *Knapsack Ilimitado*; si se pueden dividir, basta un greedy por razón valor/peso.

35.2 Knapsack Ilimitado

Idea: es la variante de la mochila en la que cada objeto tiene **suministro infinito**: se puede tomar cualquier cantidad de copias del mismo tipo. Sea $dp[w]$ el máximo valor con capacidad w . La recurrencia es $dp[w] = \max_i: p_i \leq w (dp[w - p_i] + v_i)$: el último objeto que se agrega puede ser de cualquier tipo, y lo que queda se resuelve con la capacidad restante, que ya puede volver a usar ese mismo tipo. La diferencia con *Knapsack 0/1* es solo el sentido en que se recorre la capacidad: aquí se recorre de **menor a mayor**, de modo que $dp[w - p_i]$ ya incluye posibles copias del objeto i y este se puede reutilizar.

¿Por qué usarlo? Tomar cada objeto a lo sumo una vez (0/1) no modela problemas de reposición ilimitada, como cortar una barra en tantos trozos de una misma longitud como convenga. La DP resuelve todos los casos en $O(nW)$ sin enumerar cuántas copias de cada tipo usar. Es además el marco general del *Cambio de Monedas*: minimizar la cantidad de monedas o contar combinaciones son la misma estructura con otra operación en la recurrencia (mínimo con costo 1 por moneda, o suma de formas). Por eso esta subsección va antes de esas dos.

Casos de uso:

- Cortar una barra o cuerda en trozos de longitudes permitidas para maximizar el precio total (*Rod Cutting*).
- Llenar un contenedor con paquetes de tipos repetibles para maximizar el valor transportado.
- Base directa de *Cambio de Monedas*: *Mínimo de Monedas* y *Cambio de Monedas: Número de Formas*.
- Problemas de producción donde cada producto se puede fabricar tantas veces como se quiera con recursos limitados.
- Determinar si una cantidad es alcanzable como suma de valores repetibles (el problema de las monedas de Frobenius, con n pequeño).

Complejidad: $O(nW)$ tiempo: hay $W + 1$ estados y cada uno prueba a lo sumo n tipos de objeto en $O(1)$. El espacio es $O(W)$.

```

1 // CREATE/READ/WRITE: Knapsack ilimitado (cada objeto se puede
2 // cualquier cantidad de veces), con reconstrucción de las
3 // cantidades
4 struct KnapsackIlimitado {
5     int n;
6     int capacidad;
7     vector<int> peso;
8     vector<long long> valor;
9
10    // CREATE: guarda los tipos de objeto y la capacidad de la
11    // mochila
12    KnapsackIlimitado(const vector<int>& p, const vector<long
13    long>& v, int W)
14    : n((int)p.size()), capacidad(W), peso(p), valor(v) {
15    } //  $O(n)$ 
16
17    // READ: máximo valor con peso total  $\leq$  capacidad (espacio  $O(W)$ )
18    long long maximoValor() const {
19        vector<long long> dp(capacidad + 1, 0);
20        for (int w = 1; w <= capacidad; w++) {
21            dp[w] = dp[w - 1]; // se permite dejar capacidad sin usar
22            for (int i = 0; i < n; i++) {
23                if (peso[i] <= w) {
24                    dp[w] = max(dp[w], dp[w - peso[i]] + valor[i]);
25                }
26            }
27        }
28        return dp[capacidad];
29    } //  $O(n*W)$  tiempo,  $O(W)$  espacio
30
31    // READ: cantidad usada de cada tipo en una solución óptima
32    // (cantidades[i] = número de copias del objeto i)
33    vector<int> reconstruir() const {
34        vector<long long> dp(capacidad + 1, 0);
35        vector<int> eleccion(capacidad + 1, -2); // -2 = capacidad
36        // sobrante
37        for (int w = 1; w <= capacidad; w++) {
38            dp[w] = dp[w - 1];
39            for (int i = 0; i < n; i++) {
40                if (peso[i] <= w && dp[w - peso[i]] + valor[i] > dp[w]) {
41                    dp[w] = dp[w - peso[i]] + valor[i];
42                    eleccion[w] = i;
43                }
44            }
45        }
46        vector<int> cantidades(n, 0);
47        int w = capacidad;
48        while (w > 0) {
49            if (eleccion[w] == -2) {
50                w--; // capacidad sin usar
51            } else {
52                cantidades[eleccion[w]]++;
53                w -= peso[eleccion[w]];
54            }
55        }
56        return cantidades;
57    } //  $O(n*W)$ 
58
59    // -----
60    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
61    // -----
62    // 1. Version con el bucle de objetos afuera (equivalente y
63    // mas corta):
64    // for i: for (w = peso[i]; w <= W; w++)
65    //     dp[w] = max(dp[w], dp[w - peso[i]] + valor[i]);
66    // con w CRECIENTE; es exactamente Knapsack 0/1 con el
67    // sentido del
68    // bucle interno invertido.
69    // 2. Peso EXACTO igual a la capacidad: inicializar dp[0] = 0
70    // y
71    // dp[w] = -INF para w > 0, y omitir la línea dp[w] = dp[w
72    // - 1];
73    // la respuesta es dp[capacidad] o "imposible" si sigue en
74    // -INF.
75    // 3. Mínimo de objetos para llenar exactamente W (cambio de
76    // monedas
77    // con mínimo): dp[0] = 0, dp[w] = INF, y
78    // dp[w] = min(dp[w], dp[w - peso[i]] + 1); ver Cambio de
79    // Monedas:
80    // Mínimo de Monedas.
81    // 4. NUMERO de formas de llenar exactamente W: cuenta[0] = 1
82    // y, con
83    // el bucle de objetos AFUERA y w creciente,
84    // cuenta[w] += cuenta[w - peso[i]]; cuenta combinaciones.

```

```

73    Con el
74    // bucle de capacidad afuera cuenta permutaciones (el
75    // orden
76    // importaria); ver Cambio de Monedas: Numero de Formas.
77    // 5. Capacidad enorme con un tipo dominante: si un tipo
78    // tiene la mejor
79    // razón valor/peso, en una solución óptima los demás
80    // tipos se usan
81    // pocas veces (menos de p_mejor copias en total), lo que
82    // permite
83    // acotar la búsqueda. Solo es útil cuando W es demasiado
84    // grande
85    // para la DP directa.
86    // 4b. Cuidado con objetos de peso 0 o negativo: con peso 0 y
87    // valor
88    // positivo la respuesta es infinita; validar la entrada
89    // antes de
90    // aplicar la DP.
91    };

```

Ejemplo de funcionamiento:

```

1 vector<int> peso = {1, 3, 4, 5};
2 vector<long long> valor = {10, 40, 50, 70};
3 KnapsackIlimitado k(peso, valor, 8);
4
5 long long mejor = k.maximoValor();
6 // mejor = 110
7 // (tomar un objeto de peso 3 y uno de peso 5: peso 8, valor 40
8 // + 70 = 110;
9 // usar dos objetos de peso 4 da 100, y ocho de peso 1 da solo
10 // 80)
11
12 vector<int> cantidades = k.reconstruir();
13 // cantidades = {0, 1, 0, 1}
14 // (una copia del objeto de peso 3 y una del de peso 5)

```

Nota: el orden de los bucles decide qué se cuenta. Con el bucle de objetos afuera y la capacidad creciente adentro se obtienen **combinaciones**; con el bucle de capacidad afuera se obtienen **permutaciones**. Para maximizar un valor da igual, porque el máximo no depende del orden, pero para **contar formas** la diferencia es crucial (ver la modificación 4 dentro del código). Como en Knapsack 0/1, el costo es pseudopolinomial en W .

Regla práctica: usa Knapsack Ilimitado cuando cada tipo de objeto pueda repetirse sin límite, y recorre la capacidad de menor a mayor. Si el suministro de cada tipo es limitado, usa *Knapsack 0/1* con descomposición en potencias de dos (modificación 5 de esa subsección). Si el problema pide minimizar la cantidad de piezas o contar formas de llenar una cantidad exacta, es un caso de *Cambio de Monedas*, que reutiliza esta misma estructura.

35.3 Cambio de Monedas: Número de Formas (Combinaciones)

Idea: se tienen monedas de valores $s_1, \dots, s_m$ con suministro infinito y se quiere saber de **cuántas formas distintas** se puede formar el monto n , donde el orden de las monedas **no importa** ($1 + 2 + 2 + 1$ cuentan como una sola forma). Es el marco de *Knapsack Ilimitado* con la operación cambiada: en vez de maximizar un valor, se **suman** formas. Sea $dp[v]$ el número de combinaciones que suman v , con $dp[0] = 1$ (la única forma de sumar 0 es no usar ninguna moneda). Se procesa **una moneda a la vez**, y para cada una se recorre v de menor a mayor con $dp[v] += dp[v - s_j]$. Como las monedas se introducen en un orden fijo (el bucle exterior), cada combinación se construye siempre en ese mismo orden y se cuenta una sola vez.

¿Por qué usarlo? Enumerar todas las combinaciones con backtracking es exponencial. La DP las cuenta en $O(nm)$ sin generarlas. Aquí el orden de los bucles decide

qué se cuenta: con las monedas afuera y el monto adentro se cuentan **combinaciones**; con el monto afuera y las monedas adentro se cuentan **permutaciones** (el orden sí importaría). Es el mismo detalle que se mencionó en *Knapsack Ilimitado*, y aquí es crucial porque la respuesta cambia. Cuando se pide el **mínimo** de monedas en vez del número de formas, ver *Cambio de Monedas: Mínimo de Monedas*.

Casos de uso:

- Contar de cuántas formas se puede pagar un monto con monedas o billetes de denominaciones dadas.
- Contar particiones de un entero en sumandos permitidos (por ejemplo, formas de escribir n como suma de números de un conjunto).
- Contar subconjuntos con suma exacta cuando cada elemento se puede repetir.
- Contar composiciones u ordenamientos (versión de permutaciones) de pasos de tamaños dados, como subir una escalera con saltos de 1, 2 o 3.
- Base de problemas de conteo con módulo, donde solo se pide el resultado módulo un primo.

Complejidad: $O(nm)$ tiempo, con n el monto y m el número de tipos de moneda: cada moneda recorre los $n + 1$ valores una vez con trabajo $O(1)$. El espacio es $O(n)$.

```

1 // CREATE/READ/WRITE: Cambio de monedas contando formas, con
  modulo
2 // opcional (mod = 0 significa sin modulo)
3 struct CambioMonedasFormas {
4     vector<int> monedas;
5     long long mod;
6
7     // CREATE: guarda los valores de las monedas y el modulo
8     CambioMonedasFormas(const vector<int>& s, long long m = 0)
9     : monedas(s), mod(m) {} // O(m)
10
11     // READ: numero de COMBINACIONES (el orden no importa) que
12     // suman n
13     long long contarCombinaciones(int n) const {
14         vector<long long> dp(n + 1, 0);
15         dp[0] = 1; // una forma de sumar 0: no usar monedas
16         for (int c : monedas) { // monedas AFUERA
17             for (int v = c; v <= n; v++) { // monto ADENTRO,
18                 // creciente
19                 dp[v] += dp[v - c];
20                 if (mod > 0) dp[v] %= mod;
21             }
22         }
23         return dp[n];
24     } // O(n*m) tiempo, O(n) espacio
25
26     // READ: numero de PERMUTACIONES (el orden SI importa) que
27     // suman n;
28     // es lo que sale si se invierten los bucles
29     long long contarPermutaciones(int n) const {
30         vector<long long> dp(n + 1, 0);
31         dp[0] = 1;
32         for (int v = 1; v <= n; v++) { // monto AFUERA
33             for (int c : monedas) { // monedas ADENTRO
34                 if (c <= v) {
35                     dp[v] += dp[v - c];
36                     if (mod > 0) dp[v] %= mod;
37                 }
38             }
39         }
40         return dp[n];
41     } // O(n*m) tiempo, O(n) espacio
42
43     //
44     // MODIFICACIONES Y COMENTARIOS CON LOS USOS
45     //

```

```

44 // 1. Orden de los bucles: monedas afuera = combinaciones;
45 //    monto afuera
46 //    = permutaciones. Es el error mas comun en este problema,
47 //    y no da
48 //    ningun aviso: el codigo compila y da un numero
49 //    razonable pero
50 //    incorrecto.
51 // 2. Cada moneda se puede usar A LO SUMO UNA VEZ (Subset Sum
52 //    contando):
53 //    recorrer v en sentido DECRECIENTE, de n hasta c; con
54 //    esto se
55 //    obtiene el conteo de subconjuntos con suma exacta n.
56 // 3. Cada moneda con cantidad limitada k_j: si k_j es
57 //    pequeno, repetir
58 //    el bucle 0/1 k_j veces; si es grande, usar una ventana
59 //    deslizante
60 //    sobre los residuos de v modulo c (suma de k_j+1 valores
61 //    consecutivos con la misma clase de residuo), en O(n)
62 //    por moneda.
63 // 4. Modulo: aplicar mod en cada suma (como en el codigo).
64 //    Sin modulo,
65 //    la cantidad de formas crece rapido y desborda long long
66 //    con montos
67 //    moderados si hay monedas pequenas.
68 // 5. Monto muy grande (~1e9) con pocas monedas pequenas: la
69 //    DP directa
70 //    no cabe. Se resuelve con recurrencia lineal y
71 //    exponenciacion de
72 //    matrices (permutaciones), o con el metodo de Kitamasa.
73 // 6. Exigir al menos una moneda de cada tipo: restar de n la
74 //    suma de
75 //    una copia de cada moneda y contar combinaciones del
76 //    monto
77 //    restante, si todas las monedas deben aparecer.
78 // 7. Contar solo formas con exactamente k monedas: agregar
79 //    una
80 //    dimension, dp[v][j] = formas de sumar v con j monedas,
81 //    con costo
82 //    O(n*m*k).
83 // 8. Monedas repetidas o iguales en la entrada: eliminar los
84 //    valores
85 //    duplicados antes de contar (sort + unique); si no, cada
86 //    duplicado
87 //    se trata como una moneda distinta y se cuentan formas
88 //    de mas.
89 // 9. Saber solo SI existe alguna forma (no cuantas): usar
90 //    vector<bool> con alcanzable[v] = alcanzable[v] ||
91 //    alcanzable[v-c];
92 //    ver Knapsack Ilimitado.
93 };

```

Ejemplo de funcionamiento:

```

1 CambioMonedasFormas a({1, 2, 3});
2 long long r1 = a.contarCombinaciones(4);
3 // r1 = 4
4 // (las combinaciones son 1+1+1+1, 1+1+2, 2+2 y 1+3)
5
6 CambioMonedasFormas b({2, 5, 3, 6});
7 long long r2 = b.contarCombinaciones(10);
8 // r2 = 5
9 // (las combinaciones son 2+2+2+2+2, 2+2+3+3, 2+2+6, 2+3+5 y
10 // 5+5)
11
12 long long r3 = a.contarPermutaciones(4);
13 // r3 = 7
14 // (con monedas {1,2,3} y el orden importando, 4 se forma de 7
15 // maneras:
16 // 1111, 112, 121, 211, 22, 13 y 31)

```

Nota: los enunciados de este problema suelen dar como ejemplo $n = 10$ con $S = \{2, 5, 3, 6\}$ y una lista de soluciones. Esa lista tiene exactamente 5 combinaciones, así que la respuesta correcta es 5. Si un enunciado da otro número, revisa si en realidad cuenta permutaciones (el orden importa) o si trae otra lista de monedas.

Regla práctica: usa este DP con las monedas en el bucle **exterior** cuando el orden no importe (combinaciones), y con el monto en el bucle exterior cuando el orden sí importe (permutaciones). Si el problema pide el mínimo de monedas en vez de contar, usa *Cambio de Monedas: Mínimo de Monedas*; si además cada moneda se puede usar una sola vez, recorre el monto en sentido decreciente, como en *Knapsack 0/1*.

35.4 Cambio de Monedas: Mínimo de Monedas

Idea: dado un monto n y monedas de valores $s_1, \dots, s_m$ con suministro infinito, se busca el **menor número de monedas** cuya suma sea exactamente n , o determinar que es imposible. Sea $dp[v]$ el mínimo de monedas para sumar v , con $dp[0] = 0$ y $dp[v] = \infty$ mientras no se conozca una forma. La recurrencia es $dp[v] = \min_{j: s_j \leq v} (dp[v - s_j] + 1)$: la última moneda puede ser de cualquier tipo y lo que queda se resuelve de forma óptima. Es exactamente *Knapsack Ilimitado* donde cada moneda “pesa” su valor, cuesta 1, y se debe llenar el monto de forma exacta.

¿Por qué usarlo? El algoritmo greedy (tomar siempre la moneda más grande que quepa) es correcto solo para ciertos sistemas monetarios, como los reales (1, 5, 10, 25 centavos), pero falla en general. Con monedas {1, 3, 4} y monto 6, el greedy toma 4 + 1 + 1 (3 monedas), pero lo óptimo es 3 + 3 (2 monedas). La DP prueba todas las opciones para la última moneda y es correcta para **cualquier** conjunto de monedas, en $O(nm)$. La diferencia con *Cambio de Monedas: Número de Formas* es la operación de la recurrencia: aquí es un mínimo sobre las opciones de la última moneda, y no una suma de formas, así que el orden de los bucles no afecta el resultado.

Casos de uso:

- Pagar un monto con la menor cantidad de monedas o billetes cuando el sistema no es “canónico” y el greedy falla.
- Determinar si un monto es alcanzable con las monedas dadas (respuesta -1 si no lo es).
- Minimizar el número de piezas, cortes o pasos cuando cada uno tiene un tamaño de un conjunto fijo (por ejemplo, el mínimo de cuadrados perfectos que suman n).
- Reconstruir qué monedas concretas se usan en la solución óptima.
- Como paso interno de problemas de cambio con máquinas expendedoras o cajeros automáticos.

Complejidad: $O(nm)$ tiempo: hay $n + 1$ estados y cada uno prueba m monedas en $O(1)$. El espacio es $O(n)$.

```

1 // CREATE/READ/WRITE: Cambio de monedas con mínimo de monedas y
2 // reconstrucción de cuales monedas se usan
3 struct CambioMonedasMinimo {
4     vector<int> monedas;
5     static const long long INF = (long long)4e18;
6
7     // CREATE: guarda los valores de las monedas
8     CambioMonedasMinimo(const vector<int>& s) : monedas(s) {
9         // O(m)
10
11         // READ: mínimo de monedas para sumar exactamente n, o -1 si
12         // es imposible
13         long long minimoMonedas(int n) const {
14             vector<long long> dp(n + 1, INF);
15             dp[0] = 0;
16             for (int v = 1; v <= n; v++) {
17                 for (int c : monedas) {
18                     if (c <= v && dp[v - c] != INF) {
19                         dp[v] = min(dp[v], dp[v - c] + 1);
20                     }
21                 }
22             }
23             return dp[n] == INF ? -1 : dp[n];
24         } // O(n*m) tiempo, O(n) espacio

```

```

24
25 // READ: lista de monedas de una solución óptima (vacía si n
26 // = 0,
27 // y también vacía si es imposible; distinguir con
28 // minimoMonedas)
29 vector<int> reconstruir(int n) const {
30     vector<long long> dp(n + 1, INF);
31     vector<int> ultima(n + 1, -1); // última moneda usada para
32     // llegar a v
33     dp[0] = 0;
34     for (int v = 1; v <= n; v++) {
35         for (int c : monedas) {
36             if (c <= v && dp[v - c] != INF && dp[v - c] + 1 < dp[v]) {
37                 dp[v] = dp[v - c] + 1;
38                 ultima[v] = c;
39             }
40         }
41     }
42     vector<int> usadas;
43     if (dp[n] == INF) return usadas;
44     int v = n;
45     while (v > 0) {
46         usadas.push_back(ultima[v]);
47         v -= ultima[v];
48     }
49     return usadas;
50 } // O(n*m)
51
52 //
53 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
54 //
55 // 1. Centinela INF: usar un valor grande pero que NO
56 // desborde al
57 // sumarle 1 (por eso se comprueba dp[v - c] != INF antes
58 // de sumar).
59 // Con INF = LLONG_MAX, dp[v-c] + 1 desborda y da un
60 // número negativo.
61 // 2. Cada moneda se puede usar A LO SUMO UNA VEZ: recorrer
62 // las monedas
63 // en el bucle exterior y el monto v en sentido
64 // DECRECIENTE,
65 // como en Knapsack 0/1.
66 // 3. Cantidad limitada k_j de cada moneda: descomponer k_j
67 // en
68 // potencias de dos (1, 2, 4, ..., resto) y tratar cada
69 // trozo como
70 // una moneda 0/1 de valor t*c y costo t; con esto el
71 // costo es
72 // O(n * sum log k_j).
73 // 4. Sistema canónico (greedy válido): para monedas como
74 // {1, 5, 10, 25} o
75 // potencias de un mismo número, el greedy de tomar la
76 // mayor moneda
77 // posible es correcto y corre en O(m). Verificar con la
78 // DP en
79 // casos pequeños antes de confiar en el greedy.
80 // 5. Monto muy grande (~1e9) con monedas pequeñas: la DP
81 // directa no
82 // cabe en memoria. Si la moneda más grande es M, basta
83 // llenar el
84 // monto con muchas copias de la moneda de mejor razón y
85 // resolver por
86 // DP solo un resto de orden M^2.
87 // 6. NÚMERO de formas que logran el mínimo: mantener un
88 // segundo
89 // arreglo formas[v]; al encontrar un valor menor, formas[
90 // v] =
91 // formas[v-c]; si es igual al mínimo actual, formas[v] +=
92 // formas[v-c] (con módulo si lo piden).
93 // 7. Mínimo de monedas con costo distinto por moneda (no
94 // siempre 1):
95 // reemplazar el + 1 por + costo[c]; es el Knapsack
96 // ilimitado de
97 // costo mínimo con llenado exacto.
98 // 8. Solo mínimo de monedas para muchos montos distintos: la
99 // misma
100 // tabla dp sirve para todos hasta el máximo consultado,
101 // así que se
102 // calcula una vez hasta el mayor n y se responde cada
103 // consulta en
104 // O(1).
105 }

```

Ejemplo de funcionamiento:

```

1 CambioMonedasMinimo a({1, 2, 5});
2 long long r1 = a.minimoMonedas(11);
3 // r1 = 3
4 // (5 + 5 + 1)
5

```

```

6  CambioMonedasMinimo b({1, 3, 4});
7  long long r2 = b.minimoMonedas(6);
8  // r2 = 2
9  // (3 + 3; el greedy daría 4 + 1 + 1, que usa 3 monedas)
10
11 vector<int> usadas = b.reconstruir(6);
12 // usadas = {3, 3}
13
14 CambioMonedasMinimo c({2});
15 long long r3 = c.minimoMonedas(3);
16 // r3 = -1
17 // (con solo monedas de 2 es imposible formar un monto impar)

```

Nota: el centinela INF debe comprobarse antes de sumar + 1. Si se usa LLONG_MAX y se suma sin verificar, el valor desborda y pasa a ser negativo, lo que el `min` interpreta como una solución **mejor** que todas las reales, dando resultados absurdos sin ningún error visible. El código de arriba evita ese problema con la condición `dp[v - c] != INF`.

Regla práctica: usa esta DP siempre que el sistema de monedas sea arbitrario, o cuando no estés seguro de que el greedy funciona. Solo usa el greedy si el sistema es canónico (por ejemplo, potencias de un mismo número) y lo has comprobado. Si en lugar del mínimo se pide contar cuántas formas hay, usa *Cambio de Monedas: Número de Formas*, cuidando el orden de los bucles.

35.5 Problema de Partición (Diferencia Mínima de Subconjuntos)

Idea: dado un arreglo de n enteros no negativos con suma total S , se quiere dividirlo en dos subconjuntos A y B de modo que la diferencia $|suma(A) - suma(B)|$ sea **mínima**. Si A suma x , entonces B suma $S - x$ y la diferencia es $|S - 2x|$. Por lo tanto basta saber **qué sumas x son alcanzables** por algún subconjunto (es *Subset Sum*, un caso de *Knapsack 0/1* donde el peso y el valor coinciden) y quedarse con la que minimice $|S - 2x|$. Como se puede tomar siempre el subconjunto de suma menor o igual a $S/2$, solo hace falta explorar $x \in [0, \lfloor S/2 \rfloor]$: la respuesta es $S - 2x^*$, con x^* la mayor suma alcanzable que no supera $S/2$.

¿Por qué usarlo? Probar los 2^n subconjuntos es inviable para n moderado. La DP de factibilidad resuelve el problema en $O(nS)$ porque el estado “¿es alcanzable la suma x ?” resume todas las formas de decidir los elementos ya procesados. Un greedy (repartir cada elemento, de mayor a menor, al lado que va perdiendo) da una buena aproximación pero **no es exacto**. Como *Knapsack 0/1*, es pseudopolinomial: el costo depende del valor numérico de S , no solo de n .

Casos de uso:

- Repartir tareas o cargas entre dos personas o máquinas para que el tiempo de ambas sea lo más parecido posible.
- Dividir un conjunto de monedas u objetos en dos montones de valor casi igual.
- Decidir si un arreglo se puede partir en dos subconjuntos de igual suma (diferencia mínima igual a 0).
- Base de variantes con restricciones, como exigir que ambos subconjuntos tengan el mismo tamaño.

- Problemas de equilibrio de una balanza con pesas: colocar cada pesa a un lado y minimizar el desbalance.

Complejidad: $O(nS)$ tiempo, con S la suma total (o $O(nS/2)$ si solo se explora hasta la mitad): hay $\lfloor S/2 \rfloor + 1$ sumas posibles y cada elemento las recorre una vez con trabajo $O(1)$. El espacio es $O(S)$ con un arreglo unidimensional.

```

1  // CREATE/READ/WRITE: Problema de particion: diferencia minima
   // entre las
2  // sumas de dos subconjuntos, con reconstruccion de uno de
   // ellos
3  struct ProblemaParticion {
4      vector<int> a;
5      int suma;
6
7      // CREATE: guarda el arreglo y calcula la suma total
8      ProblemaParticion(const vector<int>& v) : a(v), suma(0) {
9          for (int x : a) suma += x;
10     } // O(n)
11
12     // READ: diferencia minima posible entre las sumas de los dos
       // subconjuntos
13     int diferenciaMinima() const {
14         int mitad = suma / 2;
15         vector<bool> alcanzable(mitad + 1, false);
16         alcanzable[0] = true; // el subconjunto vacio suma 0
17         for (int x : a) {
18             // s decreciente: cada elemento se usa a lo sumo una vez
               // (O(1))
19             for (int s = mitad; s >= x; s--) {
20                 if (alcanzable[s - x]) alcanzable[s] = true;
21             }
22         }
23         for (int s = mitad; s >= 0; s--) {
24             if (alcanzable[s]) return suma - 2 * s;
25         }
26         return suma; // no se alcanza nunca: solo ocurre si mitad <
               // 0
27     } // O(n*S) tiempo, O(S) espacio
28
29     // READ: indices (0-indexados) de un subconjunto cuya suma es
       // la mayor
30     // posible sin superar suma/2; el complemento es el otro
       // subconjunto
31     vector<int> reconstruir() const {
32         int n = (int)a.size(), mitad = suma / 2;
33         // dp[i][s] = true si con los primeros i elementos se
               // alcanza la suma s
34         vector<vector<bool>> dp(n + 1, vector<bool>(mitad + 1,
               false));
35         dp[0][0] = true;
36         for (int i = 1; i <= n; i++) {
37             for (int s = 0; s <= mitad; s++) {
38                 dp[i][s] = dp[i - 1][s];
39                 if (s >= a[i - 1] && dp[i - 1][s - a[i - 1]]) dp[i][s]
               = true;
40             }
41         }
42         int s = mitad;
43         while (s > 0 && !dp[n][s]) s--;
44         vector<int> elegidos;
45         for (int i = n; i >= 1 && s > 0; i--) {
46             if (!dp[i - 1][s]) { // el elemento i fue necesario para
               // llegar a s
47                 elegidos.push_back(i - 1);
48                 s -= a[i - 1];
49             }
50         }
51         reverse(elegidos.begin(), elegidos.end());
52         return elegidos;
53     } // O(n*S) tiempo y espacio
54
55     // READ: true si el arreglo se puede partir en dos
       // subconjuntos de
56     // igual suma
57     bool sePuedeDividirEnIguales() const {
58         return suma % 2 == 0 && diferenciaMinima() == 0;
59     } // O(n*S)
60
61     //
62     // -----
63     // MODIFICACIONES Y COMENTARIOS CON LOS USOS
64     // -----
65     // 1. Elementos negativos: desplazar los indices sumando un
       // offset
66     // igual a la suma de los valores absolutos de los
       // negativos, o usar

```



```

66 // un unordered_set / map de sumas alcanzables en vez de
67 // un arreglo.
68 // 2. Version con bitset: alcanzable |= (alcanzable << x)
69 // para cada x,
70 // con bitset<MAXS>; reduce el costo a  $O(n \cdot S/64)$  (ver
71 // Bitset para
72 // Optimizacion en la parte de BitWise).
73 // 3. Suma total muy grande pero n pequeno ( $n \leq 40$ ): la DP
74 // es
75 // inviable; usar meet in the middle, generando las sumas
76 // de cada
77 // mitad, ordenandolas y buscando por dos punteros la
78 // pareja que
79 // minimice  $|S - 2x|$  (ver Meet in the Middle con Mascaras
80 // de Bits).
81 // 4. Subconjuntos de IGUAL TAMANO (n par, ambos con  $n/2$ 
82 // elementos):
83 // agregar una dimension, alcanzable[j][s] = con j
84 // elementos se
85 // alcanza la suma s, recorriendo j y s en sentido
86 // decreciente.
87 // 5. Contar el NUMERO de formas de partir con una diferencia
88 // dada d:
89 // se busca  $x = (S - d) / 2$  (debe ser entero); contar
90 // subconjuntos
91 // con suma x con cuenta[0] = 1 y cuenta[s] += cuenta[s -
92 // x], en
93 // sentido decreciente (es el problema Target Sum).
94 // 6. Partir en K subconjuntos ( $K > 2$ ) minimizando la suma
95 // maxima: ya no
96 // es esta DP; se resuelve con DP sobre mascarar de bits
97 // para n
98 // pequeno, o con busqueda binaria sobre la respuesta y
99 // backtracking.
100 // 7. Minimizar la suma maxima al partir un arreglo en K
101 // SEGMENTOS
102 // CONTIGUOS: es otro problema (Binary Search on Answer
103 // con
104 // verificacion greedy, u optimizaciones de DP); no
105 // confundir con
106 // esta particion en dos subconjuntos arbitrarios.
107 // 8. Elementos iguales a 0: se ignoran, no cambian ninguna
108 // suma; pero
109 // conviene filtrarlos si el bucle interno usa  $s \geq x$  con
110 //  $x = 0$ .
111 };

```

Ejemplo de funcionamiento:

```

1 ProblemaParticion p({1, 6, 11, 5});
2
3 int d = p.diferenciaMinima();
4 // d = 1
5 // (suma total 23; el subconjunto {1, 5, 6} suma 12 y {11} suma
6 // 11,
7 // diferencia |12 - 11| = 1; es imposible lograr 0 porque 23
8 // es impar)
9
10 vector<int> sub = p.reconstruir();
11 // sub = {2}
12 // (el indice 2 es el elemento 11, suma 11 = la mayor suma
13 // alcanzable
14 // que no supera 23/2 = 11; el resto {1, 6, 5} suma 12)
15
16 ProblemaParticion q({3, 1, 1, 2, 2, 1});
17 bool iguales = q.sePuedeDividirEnIguales();
18 // iguales = true
19 // (suma 10; {3, 2} y {1, 1, 2, 1} suman 5 cada uno)

```

Nota: solo hace falta explorar sumas hasta $\lfloor S/2 \rfloor$ porque el subconjunto complementario cubre la otra mitad: si A suma $x \leq S/2$, B suma $S - x \geq S/2$ y la diferencia es $S - 2x$. Además, si S es impar la diferencia mínima nunca puede ser 0. En `reconstruir()`, el subconjunto devuelto es uno de varios óptimos posibles; el otro es simplemente el complemento.

Regla práctica: usa esta DP de factibilidad (*Subset Sum* hasta la mitad de la suma) cuando pidan dividir un conjunto en dos partes con diferencia mínima y la suma total sea moderada (hasta unos 10^5 o 10^6). Si $n \leq 40$ y la suma es enorme, cambia a *meet in the middle*. Si en realidad piden partir un arreglo en K segmentos contiguos minimizando la suma máxima, no es este problema: usa búsqueda binaria sobre la respuesta.

Con esto queda completa la sección **Knapsack y**

Variantes^{**}. Sigue pendiente que me confirmes si "mínima suma en particiones" era este problema o el de 'K' segmentos contiguos (lo dejé cubierto como modificación 7 por si acaso).

¿Sigo con las referencias cruzadas de **LIS** y **LCS**, y **Subsecuencia Creciente de Suma Máxima**?

36 DP sobre Secuencias

36.1 Subsecuencia Creciente Más Larga (LIS): referencia cruzada

Idea: esta subsección ya se desarrolló en *Subsecuencia Creciente Más Larga (LIS)* de String Processing, con la versión $O(n \log n)$ basada en colas y búsqueda binaria. Aquí se ve desde el punto de vista de la **DP**: el estado es $dp[i]$, la longitud de la LIS que **termina** en el índice i , y la transición es $dp[i] = 1 + \max\{dp[j] : j < i, a[j] < a[i]\}$ (o 1 si no existe tal j). Esta forma es más lenta, $O(n^2)$, pero **guarda información por posición**, y eso permite contar cuántas LIS hay, reconstruir una concreta o cambiar el criterio de optimización, cosas que la versión con colas no permite directamente.

¿Por qué usarlo? Usa la versión $O(n \log n)$ cuando solo se pida la longitud. Usa esta DP cuando el problema necesite el estado por índice: número de LIS, LIS de suma máxima (ver *Subsecuencia Creciente de Suma Máxima*), LIS con restricciones entre elementos consecutivos, o cualquier variante donde el valor guardado en j deba combinarse con el de i de una forma que no sea solo "mayor longitud". Es el mismo esquema de *Memoización vs. Tabulación*: los estados se llenan de izquierda a derecha porque $dp[i]$ solo depende de índices menores.

Casos de uso:

- Contar el número de subsecuencias crecientes de longitud máxima.
- Reconstruir una LIS concreta siguiendo el arreglo de padres.
- Base de las variantes ponderadas (suma máxima, costo mínimo, apilamiento de cajas).
- Problemas donde n es pequeño ($n \leq 5000$) y importa la simplicidad más que la velocidad.
- Subsecuencia bitónica: combinar $dp[i]$ desde la izquierda con la análoga desde la derecha.

Complejidad: $O(n^2)$ tiempo, porque hay n estados y cada uno revisa hasta n índices anteriores; $O(n)$ espacio.

```

1 // CREATE/READ/WRITE: LIS estricta por DP  $O(n^2)$ , con longitud,
2 // conteo de
3 // subsecuencias óptimas y reconstruccion de una de ellas
4 struct LISPorDP {
5     vector<int> a;
6     vector<int> dp; // dp[i] = longitud de la LIS que
7                     // termina en i
8     vector<int> padre; // padre[i] = indice anterior en esa
9                       // LIS, o -1
10    vector<long long> cuenta; // cuenta[i] = cuantas LIS de
11                              // longitud dp[i] terminan en i
12
13    // CREATE: calcula las tres tablas en una sola pasada (
14    // tabulacion)
15    LISPorDP(const vector<int>& v) : a(v) {
16        int n = (int)a.size();
17        dp.assign(n, 1);

```

```

13     padre.assign(n, -1);
14     cuenta.assign(n, 1);
15     for (int i = 0; i < n; i++) {
16         for (int j = 0; j < i; j++) {
17             if (a[j] < a[i]) {
18                 if (dp[j] + 1 > dp[i]) {
19                     dp[i] = dp[j] + 1;
20                     padre[i] = j;
21                     cuenta[i] = cuenta[j];
22                 } else if (dp[j] + 1 == dp[i]) {
23                     cuenta[i] += cuenta[j];
24                 }
25             }
26         }
27     }
28 } // O(n^2)

29
30 // READ: longitud de la LIS
31 int longitud() const {
32     int mejor = 0;
33     for (int x : dp) mejor = max(mejor, x);
34     return mejor;
35 } // O(n)

36
37 // READ: numero de subsecuencias crecientes de longitud
38 // maxima
39 long long contarOptimas() const {
40     int mejor = longitud();
41     long long total = 0;
42     for (int i = 0; i < (int)a.size(); i++)
43         if (dp[i] == mejor) total += cuenta[i];
44     return total;
45 } // O(n)

46
47 // READ: una LIS optima como lista de valores
48 vector<int> reconstruir() const {
49     vector<int> res;
50     if (a.empty()) return res;
51     int fin = 0;
52     for (int i = 0; i < (int)a.size(); i++)
53         if (dp[i] > dp[fin]) fin = i;
54     for (int i = fin; i != -1; i = padre[i]) res.push_back(a[i]);
55     reverse(res.begin(), res.end());
56     return res;
57 } // O(n)

58
59 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
60 //
61 // 1. LIS NO estricta (permite iguales): cambiar a[j] < a[i]
62 //    por
63 //    a[j] <= a[i] en el constructor.
64 // 2. LIS de suma maxima: guardar suma[i] = a[i] + max(suma[j]
65 //    en vez
66 //    de contar elementos (ver Subsecuencia Creciente de Suma
67 //    Maxima).
68 // 3. Solo la longitud con n grande: usar la version O(n log
69 //    n) con
70 //    colas y busqueda binaria de la seccion de String
71 //    Processing.
72 // 4. LIS como caso de LCS: LIS(a) = LCS(a, valores unicos de
73 //    a
74 //    ordenados); util para verificar, pero mas lento que la
75 //    DP directa.
76 // 5. Cuenta con modulo: aplicar % MOD en cuenta[i] += cuenta
77 //    [j] si el
78 //    numero de LIS es enorme.
79 // 6. Subsecuencia bitonica mas larga: calcular tambien la DP
80 //    desde la
81 //    derecha (dpDer[i] = LDS que empieza en i) y tomar
82 //    max(dp[i] + dpDer[i] - 1).
83 // 7. Restriccion entre consecutivos (por ejemplo a[i] - a[j]
84 //    <= K):
85 //    agregar esa condicion al if del bucle interno; para n
86 //    grande se
87 //    acelera con un arbol de segmentos sobre los valores.
88 };

```

Ejemplo de funcionamiento:

```

1 LISPorDP l({10, 9, 2, 5, 3, 7, 101, 18});
2
3 int largo = l.longitud();
4 // largo = 4
5
6 long long cuantas = l.contarOptimas();
7 // cuantas = 4
8 // (2,5,7,101 - 2,5,7,18 - 2,3,7,101 - 2,3,7,18)
9
10 vector<int> una = l.reconstruir();

```

```

11 // una = {2, 5, 7, 101}

```

Regla práctica: para solo la longitud, usa *Subsecuencia Creciente Más Larga (LIS)* en $O(n \log n)$. Usa esta DP $O(n^2)$ cuando necesites contar, reconstruir o ponderar, o cuando n sea pequeño y quieras la versión más fácil de modificar.

36.2 Subsecuencia Común Más Larga (LCS): referencia cruzada

Idea: esta subsección ya se desarrolló en *Subsecuencia Común Más Larga (Longest Common Subsequence)* de String Processing. Aquí se resume desde la DP: el estado $dp[i][j]$ es la LCS de los prefijos $s[0..i-1]$ y $t[0..j-1]$. Si $s[i-1] = t[j-1]$, entonces $dp[i][j] = dp[i-1][j-1] + 1$; si no, $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$. Es el ejemplo típico de DP en **dos dimensiones sobre pares de prefijos**, y la plantilla de muchos problemas parecidos (distancia de edición, supersecuencia común, subsecuencia palindrómica, alineamiento de secuencias).

¿Por qué usarlo? Aquí importa reconocer el **patrón**, más que el problema concreto: cuando hay que comparar o alinear dos secuencias y las decisiones se toman de izquierda a derecha, el estado natural es “cuántos elementos de cada una he consumido”. Cambiando solo la transición se resuelve una familia entera. Por eso conviene tener LCS a mano en esta parte aunque su explicación completa esté en String Processing.

Casos de uso:

- Comparar dos secuencias permitiendo huecos (diff de archivos, alineamiento de ADN).
- Base de *Distancia de Edición*: mismo estado con una operación más (sustituir).
- Base de la supersecuencia común más corta: $|s| + |t| - \text{LCS}$.
- Subsecuencia palindrómica más larga: LCS de s con su reverso.
- Plantilla para DP con tres o más secuencias añadiendo una dimensión.

Complejidad: $O(nm)$ tiempo, con $n = |s|$ y $m = |t|$: hay $(n+1)(m+1)$ estados y cada uno se calcula en $O(1)$. El espacio es $O(nm)$ con la tabla completa, o $O(\min(n, m))$ si solo se quiere la longitud.

```

1 // CREATE/READ/WRITE: LCS por DP con espacio reducido para la
2 // longitud y
3 // tabla completa para reconstruir
4 struct LCSPorDP {
5     string s, t;
6
7     // CREATE: guarda las dos secuencias
8     LCSPorDP(const string& a, const string& b) : s(a), t(b) {}
9     // O(1)
10
11 // READ: longitud de la LCS con solo dos filas (espacio O(m))
12 int longitud() const {
13     int n = (int)s.size(), m = (int)t.size();
14     vector<int> anterior(m + 1, 0), actual(m + 1, 0);
15     for (int i = 1; i <= n; i++) {
16         for (int j = 1; j <= m; j++) {
17             if (s[i-1] == t[j-1]) actual[j] = anterior[j-1] + 1;
18             else actual[j] = max(anterior[j], actual[j-1]);
19         }
20         swap(anterior, actual);
21     }
22 }

```

```

21     return anterior[m];
22 } // O(n*m) tiempo, O(m) espacio
23
24 // READ: una LCS concreta (tabla completa para poder
    retroceder)
25 string reconstruir() const {
26     int n = (int)s.size(), m = (int)t.size();
27     vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
28     for (int i = 1; i <= n; i++)
29         for (int j = 1; j <= m; j++)
30             dp[i][j] = (s[i - 1] == t[j - 1]) ? dp[i - 1][j - 1] + 1
31             : max(dp[i - 1][j], dp[i][j - 1]);
32     string res;
33     int i = n, j = m;
34     while (i > 0 && j > 0) {
35         if (s[i - 1] == t[j - 1]) { res += s[i - 1]; i--; j--; }
36         else if (dp[i - 1][j] >= dp[i][j - 1]) i--;
37         else j--;
38     }
39     reverse(res.begin(), res.end());
40     return res;
41 } // O(n*m) tiempo y espacio
42
43 //
44 //-----
45 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
46 //-----
47 // 1. Solo insertar y eliminar (sin sustituir): la distancia
    mínima es
48 // n + m - 2 * longitud().
49 // 2. Distancia de edicion: mismo estado dp[i][j], pero en el
    caso de
50 // desigualdad se toma 1 + min(dp[i-1][j-1], dp[i-1][j],
    dp[i][j-1]);
51 // ver Distancia de Edicion.
52 // 3. Supersecuencia comun mas corta: longitud n + m -
    longitud(); ver
53 // Supersecuencia Comun Mas Corta.
54 // 4. Subsecuencia palindromica mas larga: LCSPorDP(s,
    reverso de s).
55 // 5. Tres secuencias: dp[i][j][k] con transicion analoga, O(
    n*m*p);
56 // solo viable con secuencias cortas.
57 // 6. Dos permutaciones de 1..n: LCS se reduce a LIS. Se
    reemplaza cada
58 // elemento de la segunda por su posicion en la primera y
    se calcula
59 // la LIS en O(n log n), evitando el O(n^2).
60 // 7. Contar el numero de LCS distintas: agregar cuenta[i][j]
    con la
61 // misma estructura, restando el doble conteo cuando
    dp[i-1][j-1] == dp[i][j]; requiere cuidado con el
62 // modulo.
63 // 8. Con peso por par que coincide (por ejemplo cada letra
    vale
64 // distinto): en la rama de igualdad sumar peso(s[i-1]) en
    vez de 1.
65 };

```

Ejemplo de funcionamiento:

```

1 LCSPorDP p("AGGTAB", "GTXAYB");
2
3 int largo = p.longitud();
4 // largo = 4
5
6 string lcs = p.reconstruir();
7 // lcs = "GTAB"

```

Regla práctica: para el detalle del problema, la reconstrucción y su relación con la subcadena común, consulta *Subsecuencia Común Más Larga (Longest Common Subsequence)*. Aquí queda como plantilla: siempre que aparezcan dos secuencias y un estado del tipo “prefijo de una y prefijo de otra”, piensa en este esquema y cambia solo la transición.

36.3 Subsecuencia Creciente de Suma Máxima

Idea: dada una secuencia de n enteros, se busca la subsecuencia **estrictamente creciente** cuya **suma** de elementos sea máxima (no la de mayor longitud). Es la variante ponderada de *Subsecuencia Creciente Más Larga (LIS)*: *referencia cruzada*: en vez de que cada elemento aporte 1 a la longitud, aporta su propio valor. Sea

$dp[i]$ la suma máxima de una subsecuencia creciente que **termina** en el índice i . La recurrencia es $dp[i] = a[i] + \max(0, \max\{dp[j] : j < i, a[j] < a[i]\})$: el elemento i se puede empezar solo (aporta $a[i]$) o extender la mejor subsecuencia previa que termine en un valor menor. La respuesta es $\max_i dp[i]$. Es el mismo esquema de DP por índice de la referencia cruzada, cambiando únicamente lo que se acumula.

¿Por qué usarlo? La subsecuencia más larga no es necesariamente la de mayor suma. En $\{100, 1, 2, 3, 4\}$ la LIS más larga es $1, 2, 3, 4$ (longitud 4, suma 10), pero la de mayor suma es $\{100\}$ (suma 100). Por eso no sirve la técnica de **colas** con búsqueda binaria de LIS, que solo conserva longitudes: aquí el valor guardado en cada estado debe combinarse por **suma**. La DP directa cuesta $O(n^2)$; con un *árbol de Fenwick* sobre los valores comprimidos, que responda el máximo de dp entre los valores menores a $a[i]$, baja a $O(n \log n)$.

Casos de uso:

- Elegir, respetando un orden creciente, los elementos que suman más (ganancia acumulada con restricción de orden).
- Apilamiento de cajas u objetos donde cada uno aporta un valor (por ejemplo, altura) y se debe cumplir una condición de crecimiento: ver *Apilamiento de Cajas (Box Stacking)*.
- Selección de trabajos o eventos ordenados donde cada elemento posterior debe superar al anterior y se maximiza el beneficio total.
- Reconstruir la subsecuencia concreta que logra esa suma, siguiendo el arreglo de padres.
- Variantes con pesos distintos del valor (un peso w_i por elemento) manteniendo la condición sobre a .

Complejidad: $O(n^2)$ tiempo con la DP directa, porque hay n estados y cada uno revisa hasta n índices anteriores; $O(n)$ espacio. Con Fenwick sobre valores comprimidos: $O(n \log n)$.

```

1 // CREATE/READ/WRITE: Subsecuencia creciente de suma maxima,
    con version
2 // O(n^2) con reconstruccion y version O(n log n) con Fenwick
    de maximos
3 struct SubsecuenciaCrecienteSumaMaxima {
4     vector<long long> a;
5     vector<long long> dp; // dp[i] = suma maxima de una
        subsecuencia creciente que termina en i
6     vector<int> padre; // padre[i] = indice anterior en esa
        subsecuencia, o -1
7
8     // CREATE: calcula dp y padre en una sola pasada (tabulacion)
9     SubsecuenciaCrecienteSumaMaxima(const vector<long long>& v) :
        a(v) {
10         int n = (int)a.size();
11         dp.assign(n, 0);
12         padre.assign(n, -1);
13         for (int i = 0; i < n; i++) {
14             dp[i] = a[i]; // la subsecuencia formada solo por a[i]
15             for (int j = 0; j < i; j++) {
16                 if (a[j] < a[i] && dp[j] + a[i] > dp[i]) {
17                     dp[i] = dp[j] + a[i];
18                     padre[i] = j;
19                 }
20             }
21         }
22     } // O(n^2)
23
24     // READ: suma maxima de una subsecuencia estrictamente
        creciente
25     long long sumaMaxima() const {
26         if (a.empty()) return 0;

```

```

27     return *max_element(dp.begin(), dp.end());
28 } // O(n)
29
30 // READ: una subsecuencia optima como lista de valores
31 vector<long long> reconstruir() const {
32     vector<long long> res;
33     if (a.empty()) return res;
34     int fin = (int)(max_element(dp.begin(), dp.end()) - dp.
35     begin());
36     for (int i = fin; i != -1; i = padre[i]) res.push_back(a[i
37     ]);
38     reverse(res.begin(), res.end());
39     return res;
40 } // O(n)
41
42 // READ: version O(n log n): Fenwick de maximos indexado por
43 // el rango
44 // (valor comprimido) de cada elemento; consulta el mejor dp
45 // entre los
46 // valores estrictamente menores
47 static long long sumaMaximaRapida(const vector<long long>& v)
48 {
49     int n = (int)v.size();
50     if (n == 0) return 0;
51     vector<long long> vals = v;
52     sort(vals.begin(), vals.end());
53     vals.erase(unique(vals.begin(), vals.end()), vals.end());
54     int m = (int)vals.size();
55     vector<long long> bit(m + 1, 0); // maximos por prefijo; 0
56     = "no extender"
57     auto actualizar = [&](int pos, long long val) {
58         for (; pos <= m; pos += pos & -pos) bit[pos] = max(bit[
59         pos], val);
60     };
61     auto consultar = [&](int pos) { // maximo en las posiciones
62         1..pos
63         long long r = 0;
64         for (; pos > 0; pos -= pos & -pos) r = max(r, bit[pos]);
65         return r;
66     };
67     bool hayElemento = false;
68     long long mejorGlobal = 0;
69     for (long long x : v) {
70         int pos = (int)(lower_bound(vals.begin(), vals.end(), x)
71         - vals.begin()) + 1;
72         long long previo = consultar(pos - 1); // valores
73         estrictamente menores
74         long long actual = x + max(0LL, previo);
75         actualizar(pos, actual);
76         if (!hayElemento || actual > mejorGlobal) { mejorGlobal =
77         actual; hayElemento = true; }
78     }
79     return mejorGlobal;
80 } // O(n log n)
81
82 //
83
84 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
85 //
86
87 // 1. Subsecuencia NO estricta (permite iguales): cambiar a[j
88 ] < a[i]
89 // por a[j] <= a[i] en la version O(n^2); en la de Fenwick,
90 consultar
91 // hasta pos (inclusive) en vez de pos - 1.
92 // 2. Elementos negativos: la subsecuencia debe tener al
93 // menos un
94 // elemento, por eso dp[i] se inicializa en a[i] (no en 0).
95 // En la
96 // version con Fenwick se usa max(0, previo) para no
97 // arrastrar sumas
98 // negativas; el resultado es correcto aunque todos sean
99 // negativos
100 // (devuelve el mayor elemento).
101 // 3. Subsecuencia DECRECIENTE de suma maxima: cambiar la
102 // comparacion a
103 // a[j] > a[i] (la suma se acumula siempre con los valores
104 // originales).
105 // 4. Peso distinto del valor: si cada elemento tiene un peso
106 // w[i] que
107 // se suma pero la condicion de orden es sobre a[i], usar
108 // dp[i] = w[i] + max(dp[j]) con a[j] < a[i].
109 // 5. Contar cuantas subsecuencias logran la suma maxima:
110 // mantener
111 // cuenta[i] como en LIS por DP, sumando cuenta[j] cuando
112 // dp[j] + a[i] iguala el maximo (con modulo si lo piden).
113 // 6. Restriccion de indices o de diferencia entre
114 // consecutivos (por
115 // ejemplo j >= i - K): acotar el bucle interno, o
116 // consultar el
117 // Fenwick/arbol de segmentos solo en la ventana valida.
118 // 7. Valores muy grandes o negativos para el Fenwick:
119 // siempre

```

```

94 // comprimir a rangos 1..m antes de indexar.
95 // 8. Cotas de tipo: la suma llega a n * max|a|; usar long
96 // long para
97 // evitar overflow con n grande.
98 };
99

```

Ejemplo de funcionamiento:

```

1 SubsecuenciaCrecienteSumaMaxima s({1, 101, 2, 3, 100, 4, 5});
2
3 long long mejor = s.sumaMaxima();
4 // mejor = 106
5 // (la subsecuencia 1, 2, 3, 100 suma 106; 1, 101 suma solo 102
6 // y
7 // 1, 2, 3, 4, 5 suma solo 15)
8
9 vector<long long> sub = s.reconstruir();
10 // sub = {1, 2, 3, 100}
11
12 long long rapido = SubsecuenciaCrecienteSumaMaxima::
13 sumaMaximaRapida({1, 101, 2, 3, 100, 4, 5});
14 // rapido = 106 (mismo resultado en O(n log n))
15
16 SubsecuenciaCrecienteSumaMaxima t({100, 1, 2, 3, 4});
17 long long distinta = t.sumaMaxima();
18 // distinta = 100
19 // (la LIS mas larga es 1, 2, 3, 4 con suma 10, pero la de
20 // mayor suma
21 // es el elemento 100 solo)

```

Regla práctica: usa esta DP cuando cada elemento aporte un valor y se pida maximizar la suma bajo una condición de orden creciente; la técnica de colas de LIS no aplica porque solo conserva longitudes. Usa la versión $O(n^2)$ si $n \leq 5000$ y necesitas reconstruir, y la de Fenwick sobre valores comprimidos cuando n sea grande.

37 DP de Intervalos

37.1 Burst Balloons (Problema de los Globos)

Idea: dado un arreglo de n globos con valores $\text{nums}[i]$, al reventar el globo i se ganan $\text{nums}[i-1] \cdot \text{nums}[i] \cdot \text{nums}[i+1]$ monedas (si un vecino no existe, se toma como 1), y luego i desaparece, por lo que sus vecinos pasan a ser adyacentes. Se busca el orden de reventado que maximiza el total de monedas. Es el ejemplo clásico de **DP sobre intervalos**, y la clave está en cambiar la pregunta: en vez de "¿cuál globo revienta primero?" (después de eso los vecinos cambian y los subproblemas quedan acoplados), se pregunta "¿cuál globo revienta **último** dentro de este intervalo?". Si k es el último en reventar entre los límites l y r (exclusivos), en ese momento sus vecinos son exactamente l y r (todo lo demás ya desapareció), y los intervalos (l, k) y (k, r) son **independientes**:

$$\text{dp}[l][r] = \max_{l < k < r} \left(\text{dp}[l][k] + \text{dp}[k][r] + a[l] \cdot a[k] \cdot a[r] \right)$$

donde a es el arreglo original con un 1 agregado a cada extremo (centinelas) y $\text{dp}[l][r]$ es la mejor ganancia reventando **todos** los globos estrictamente entre l y r .

¿Por qué usarlo? Es el problema que enseña la idea de **invertir el orden de decisión** para desacoplar subproblemas: con "el primero en reventar" los subproblemas dependen de qué globos quedaron a los lados; con "el último en reventar" los bordes del intervalo están fijos y las dos mitades no interactúan. Ese mismo truco aparece en otros problemas de intervalos (multiplicación de cadena de matrices, triangulación óptima de polígonos, árbol de búsqueda binaria óptimo). Greedy no sirve aquí: reventar siempre el globo más chico o el más grande falla en casos pequeños, así que hace falta considerar todas

las posiciones del último globo, justo lo que hace DP (ver Greedy vs. Programación Dinámica).

Casos de uso:

- Burst Balloons tal cual: máxima ganancia reventando todos los globos.
- Cualquier problema donde eliminar un elemento cambia quién es vecino de quién: la pregunta útil es "¿qué elemento queda **al final**?" en vez de "¿cuál elimino primero?".
- Plantilla general de DP por intervalos: estado $[l, r]$, transición probando un punto de corte k , recorrido por **longitud creciente** del intervalo.
- Reconstruir el **orden** de reventado óptimo (no solo el valor), guardando el k óptimo de cada intervalo.

Complejidad: $O(n^3)$ tiempo ($O(n^2)$ estados, cada uno prueba hasta n valores de k), $O(n^2)$ memoria.

```

1 struct BurstBalloons {
2     vector<int> a; // nums con un 1 en cada
3     // extremo
4     vector<vector<long long>> dp; // dp[l][r]: reventar todo lo
5     // que
6     // esta ESTRICTAMENTE entre l y r
7     vector<vector<int>> ultimo; // ultimo[l][r]: k optimo (el
8     // ultimo globo en reventar)
9     int m; // tamaño de a (n + 2)

10    // CREATE: agrega los centinelas 1 y calcula toda la tabla dp.
11    BurstBalloons(vector<int>& nums) {
12        a.push_back(1);
13        for (int x : nums) a.push_back(x);
14        a.push_back(1);
15        m = a.size();
16
17        dp.assign(m, vector<long long>(m, 0));
18        ultimo.assign(m, vector<int>(m, -1));
19
20        // se recorre por LONGITUD creciente del intervalo: al
21        // calcular
22        // dp[l][r] ya estan listos todos los subintervalos mas
23        // cortos
24        for (int len = 2; len < m; len++) {
25            for (int l = 0; l + len < m; l++) {
26                int r = l + len;
27
28                for (int k = l + 1; k < r; k++) {
29                    // k es el ULTIMO globo en reventar dentro de (l,r):
30                    // sus vecinos en ese momento son exactamente l y r
31                    long long total = dp[l][k] + dp[k][r]
32                    + (long long)a[l] * a[k] * a[r];
33
34                    if (total > dp[l][r]) {
35                        dp[l][r] = total;
36                        ultimo[l][r] = k;
37                    }
38                }
39            }
40        }
41
42        // READ: maxima cantidad de monedas reventando todos los
43        // globos.
44        long long maximo() {
45            return dp[0][m - 1];
46        }
47
48        // READ: orden de reventado optimo (indices sobre el arreglo
49        // original nums, base 0), del primero al ultimo en reventar.
50        vector<int> ordenOptimo() {
51            vector<int> orden;
52            reconstruir(0, m - 1, orden);
53            return orden;
54        }
55
56        // Recursion: el orden de (l,r) es el de (l,k), luego el de (
57        // k,r),
58        // y al final k (que es el ultimo de este intervalo).
59        void reconstruir(int l, int r, vector<int>& orden) {
60            int k = ultimo[l][r];
61            if (k == -1) return; // intervalo sin globos adentro

```

```

58    reconstruir(l, k, orden);
59    reconstruir(k, r, orden);
60    orden.push_back(k - 1); // k - 1: se resta el centinela
61    }
62
63    //
64    // -----
65    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
66    // -----
67
68    // 1. VERSION TOP-DOWN (memoizacion): misma recurrencia, sin
69    // preocuparse por el orden de llenado. Util si se
70    // desconfia
71    // del orden por longitud:
72    //
73    // long long resolver(int l, int r) {
74    //     if (r - l < 2) return 0; // nada entre l y
75    //     r
76    //     if (memo[l][r] != -1) return memo[l][r];
77    //     long long mejor = 0;
78    //     for (int k = l + 1; k < r; k++)
79    //         mejor = max(mejor, resolver(l, k) + resolver(k,
80    //         r));
81    //     return memo[l][r] = mejor;
82    // }
83
84    // 2. OTRAS GANANCIAS POR REVENTADO: solo cambia el termino
85    // que se
86    // suma al combinar. Ejemplos con la misma estructura:
87    // - ganar a[l] + a[k] + a[r] (suma en vez de
88    // producto)
89    // - costo minimo en vez de ganancia maxima: cambiar max
90    // por
91    // min e inicializar dp en un valor grande (INF)
92    // El esqueleto (intervalo, k = ultimo, dos mitades
93    // independientes) no cambia.
94
95    // 3. MISMA PLANTILLA EN OTROS PROBLEMAS DE INTERVALOS:
96    // - multiplicacion de cadena de matrices:
97    //     dp[l][r] = min sobre k de dp[l][k] + dp[k][r]
98    //     + dims[l] * dims[k] * dims[r]
99    // - triangulacion de poligono de costo minimo (k = tercer
100    // vertice del triangulo que usa la arista l-r)
101    // - cortar una barra en puntos dados con costo = longitud
102    // del
103    // segmento (k = ultimo corte del segmento)
104
105    // 4. VERSION CIRCULAR (globos en circulo): se duplica el
106    // arreglo
107    // o se fija que globo es el ultimo de toda la ronda, y se
108    // resuelve el intervalo lineal resultante. Sube el costo
109    // por
110    // un factor n si se prueba cada opcion.
111
112    // 5. OPTIMIZACION: para algunas variantes (como matriz de
113    // cadena o corte de barra) existe la optimizacion de
114    // Knuth,
115    // que baja  $O(n^3)$  a  $O(n^2)$ , pero exige propiedades de la
116    // funcion de costo (cuadrangulo) que NO se cumplen en
117    // Burst
118    // Balloons con producto de tres terminos: aqui  $O(n^3)$  es
119    // lo
120    // esperado.

```

Ejemplo de funcionamiento:

```

1 vector<int> nums = {3, 1, 5, 8};
2
3 BurstBalloons bb(nums);
4
5 cout << bb.maximo();
6 // 167
7
8 for (int i : bb.ordenOptimo()) cout << i << " ";
9 // 1 2 0 3 (indices sobre nums: se revienta el globo de valor
10 // 1,
11 // luego el 5, luego el 3 y al final el 8)
12 // ganancias: 3*1*5 = 15, 3*5*8 = 120, 1*3*8 = 24, 1*8*1 = 8
13 // total: 15 + 120 + 24 + 8 = 167

```

Nota: el recorrido por **longitud creciente** del intervalo no es un detalle: $dp[l][r]$ depende de $dp[l][k]$ y $dp[k][r]$, que son intervalos más cortos, así que deben estar listos antes. Recorrer l y r con dos for anidados "normales" (por filas) rompe esa dependencia y usa val-

ores todavía en cero. Los centinelas 1 en los extremos evitan casos especiales para los globos de las orillas.

Regla práctica: en un problema donde quitar un elemento cambia los vecinos de los demás, no intentes modelar "el primero que quito": modela "el **último** que queda" dentro de un intervalo $[l, r]$. Ese elemento tiene vecinos conocidos (l y r) y parte el problema en dos mitades independientes, lo que convierte el enredo en una DP de intervalos $O(n^3)$ con la misma forma que multiplicación de cadena de matrices.

38 Apilamiento y Ordenamiento

38.1 Apilamiento de Cajas (Box Stacking)

Idea: dado un conjunto de n cajas, cada una con tres dimensiones (x, y, z) , se busca la **pila más alta** posible. Una caja se puede apoyar sobre otra solo si la base de la de arriba es **estrictamente menor** en ambas dimensiones que la base de la de abajo, y cada caja se puede **rotar**: cualquiera de sus tres dimensiones puede hacer de altura. Se resuelve en tres pasos: (1) generar las **3 orientaciones** de cada caja, guardando en cada una la base como (w, d) con $w \leq d$ (el giro de la base en el plano no aporta nada nuevo: si una caja cabe sobre otra girada, también cabe con las dos bases ordenadas de menor a mayor); (2) **ordenar** todas las orientaciones por área de base decreciente $w \cdot d$; (3) aplicar una DP tipo LIS pesada:

$$dp[i] = h_i + \max \left(0, \max_{j < i, w_j > w_i, d_j > d_i} dp[j] \right)$$

donde $dp[i]$ es la altura máxima de una pila que tiene la orientación i en la cima. La respuesta es el máximo de todos los $dp[i]$.

¿Por qué usarlo? El paso de ordenar por área es lo que hace que la DP funcione: si la caja i va sobre la j , su base es estrictamente menor en ambas dimensiones, así que su área también es estrictamente menor y j aparece **antes** en el orden. Ese orden es un orden topológico de la relación "cabe sobre", y basta mirar solo los $j < i$. Además, como la condición es **estricta**, una misma orientación no puede repetirse en la pila (no cabe sobre sí misma), lo que resuelve gratis la pregunta de cuántas copias de cada caja se permiten. Greedy no sirve: apilar siempre la caja más alta, o la de base más grande, falla en casos pequeños (ver Greedy vs. Programación Dinámica).

Casos de uso:

- Box Stacking clásico: altura máxima con rotaciones y base estrictamente decreciente.
- Variante **sin rotación**: cada caja tiene una sola orientación y el problema se vuelve una DP tipo LIS en 2D (ver la modificación 1 en el código).
- Anidamiento de sobres o muñecas rusas (*Russian Doll Envelopes*): misma relación de orden parcial, pero con peso 1 por elemento en vez de altura.
- Cuboides con desigualdad **no estricta** (*Maximum Height by Stacking Cuboids*): cambia el criterio de orden, no la idea (modificación 2).

- Cualquier problema de "cadena más pesada en un orden parcial de dos criterios": ordenar por uno, y aplicar DP sobre el otro.

Complejidad: con $m = 3n$ orientaciones, ordenar cuesta $O(m \log m)$ y la DP $O(m^2) = O(n^2)$ tiempo; $O(n)$ memoria.

```

1 struct BoxStacking {
2     struct Caja { int h, w, d; }; // altura y base (w <= d)
3     vector<Caja> orient; // las 3n orientaciones
4     // ordenadas
5     vector<long long> dp; // dp[i]: mejor pila con
6     // orient[i]
7     // en la CIMA
8     vector<int> padre; // padre[i]: caja que queda
9     // debajo
10    // de orient[i] en la mejor pila
11
12    // CREATE: genera las 3 orientaciones de cada caja, las
13    // ordena por
14    // area de base decreciente y calcula la DP.
15    BoxStacking(vector<array<int,3>>& cajas) {
16        for (auto& c : cajas) {
17            for (int i = 0; i < 3; i++) {
18                int h = c[i]; // esta dimension
19                es
20                int p = c[(i + 1) % 3]; // la altura; las
21                int q = c[(i + 2) % 3]; // otras dos, la
22                base
23                orient.push_back({h, min(p, q), max(p, q)});
24            }
25        }
26
27        // area decreciente: si i va sobre j, area_i < area_j, asi
28        // que
29        // j queda antes que i y basta mirar los j < i
30        sort(orient.begin(), orient.end(),
31            [](const Caja& a, const Caja& b) {
32                return (long long)a.w * a.d > (long long)b.w * b.d;
33            });
34
35        int m = orient.size();
36        dp.assign(m, 0);
37        padre.assign(m, -1);
38
39        for (int i = 0; i < m; i++) {
40            dp[i] = orient[i].h; // la caja sola, sin nada debajo
41            for (int j = 0; j < i; j++) {
42                bool cabe = orient[j].w > orient[i].w
43                    && orient[j].d > orient[i].d; // Estricto
44                if (cabe && dp[j] + orient[i].h > dp[i]) {
45                    dp[i] = dp[j] + orient[i].h;
46                    padre[i] = j;
47                }
48            }
49        }
50
51        // READ: altura maxima de una pila.
52        long long alturaMaxima() {
53            return *max_element(dp.begin(), dp.end());
54        }
55
56        // READ: la pila optima de ARIIBA hacia ABAJO.
57        vector<Caja> pila() {
58            int mejor = max_element(dp.begin(), dp.end()) - dp.begin();
59            vector<Caja> resultado;
60            for (int i = mejor; i != -1; i = padre[i])
61                resultado.push_back(orient[i]);
62            return resultado;
63        }
64
65        // MODIFICACIONES Y COMENTARIOS CON LOS USOS
66
67        // 1. SIN ROTACION (cada caja tiene una sola orientacion fija,
68        // con
69        // base (w, d) dada): en el constructor se genera UNA sola
70        // orientacion por caja, sin ordenar w y d entre si, y la
71        // condicion de apilar compara cada dimension en su lugar:
72        //
73        // orient.push_back({h, w, d}); // una
74        // orientacion
75        // ...
76        // bool cabe = orient[j].w > orient[i].w && orient[j].d >
77        // orient[i].d;

```

```

71 //
72 // Si ademas todas las alturas valen 1 (sobres, muncas
73 // el problema se puede bajar a  $O(n \log n)$ : ordenar por  $w$ 
74 // creciente y, ante empate de  $w$ , por  $d$  DECRECIENTE (para
75 // que
76 // dos sobres con el mismo  $w$  no se encadenen), y calcular
77 // la
78 // subsecuencia creciente estricta mas larga sobre  $d$  con
79 // busqueda binaria.
80 // 2. DESIGUALDAD NO Estricta (una caja puede apoyarse sobre
81 // otra
82 // si sus dimensiones son menores O IGUALES, como en
83 // Maximum
84 // Height by Stacking Cuboids): se ordenan las 3
85 // dimensiones de
86 // cada caja de menor a mayor, se ordenan todas las cajas
87 // lexicograficamente por esas tres dimensiones, y la DP
88 // usa
89 // <= en las tres:
90 //
91 // bool cabe = A[j][0] <= A[i][0] && A[j][1] <= A[i][1]
92 // && A[j][2] <= A[i][2]; // j va DEBAJO de i
93 // dp[i] = A[i][2] + max(dp[j] entre las que cumplen cabe)
94 //
95 // Aqui la altura de cada caja es su dimension mas grande
96 // y NO
97 // se generan 3 orientaciones: al ordenar las dimensiones,
98 // esa
99 // orientacion ya es siempre la mejor. Con <= el area deja
100 // de
101 // servir como orden topologico (dos cajas pueden tener la
102 // misma area), por eso se ordena lexicograficamente.
103 // 3. LIMITE DE COPIAS POR CAJA: en la version estricta de
104 // arriba
105 // cada orientacion aparece a lo sumo una vez, y la misma
106 // caja
107 // fisica puede aparecer con dos orientaciones distintas
108 // en la
109 // pila. Si el enunciado exige usar cada caja FISICA una
110 // sola
111 // vez, la DP simple ya no alcanza (habria que recordar
112 // que
113 // cajas se usaron): ahi hay que revisar el enunciado,
114 // porque
115 // suele ser un problema distinto.
116 // 4. MEJORAR  $O(n^2)$ : con muchas cajas se puede compactar la
117 // coordenada  $d$  y usar un arbol (Fenwick o segment tree de
118 // maximo) para consultar "mejor dp entre las cajas de
119 // base mas
120 // grande en ambas dimensiones", procesando por  $w$ 
121 // decreciente.
122 // Baja a  $O(n \log n)$ , a costa de codigo bastante mas largo.
123 // 5. PILA CON CAJAS ENTRE MEDIO ("cada caja debe soportar un
124 // peso maximo"): es otro problema (Box Stacking con
125 // resistencias). Se ordena por la suma peso + resistencia,
126 // y
127 // NO es esta misma DP.
128 };

```

Ejemplo de funcionamiento:

```

1 // cada caja es {x, y, z}
2 vector<array<int,3>> cajas = {
3     {4, 6, 7}, {1, 2, 3}, {4, 5, 6}, {10, 12, 32}
4 };
5
6 BoxStacking bs(cajas);
7
8 cout << bs.alturaMaxima();
9 // 60
10
11 for (auto& c : bs.pila())
12 cout << "(h=" << c.h << ",w=" << c.w << ",d=" << c.d << ") ";
13 // (h=3,w=1,d=2) (h=1,w=2,d=3) (h=6,w=4,d=5) (h=4,w=5,d=6)
14 // (h=4,w=6,d=7) (h=32,w=10,d=12) (h=10,w=12,d=32)
15 // (de arriba hacia abajo; 3+1+6+4+4+32+10 = 60, y cada base es
16 // estrictamente menor que la de abajo en ambas dimensiones)

```

Nota: en este ejemplo las 4 cajas aportan **7** orientaciones a la misma pila, porque cada una entra con más de una orientación y todas caben en cadena. Eso es normal en este problema: "apilar cajas" con rotaciones libres no significa usar cada caja una vez, sino usar orientaciones distintas. Si el enunciado dice explícitamente que cada

caja física se usa **a lo más una vez**, esta DP da una respuesta demasiado optimista (ver modificación 3).

Regla práctica: cuando un problema pide la cadena **más pesada** en una relación de "cabe dentro de" con dos criterios, ordena por un criterio de forma que el orden sea topológico (aquí, el área) y aplica DP tipo LIS sobre el resto. Antes de programar, define con cuidado dos cosas del enunciado: si la desigualdad es **estricta** o no, y si hay **rotaciones** y de qué tipo. Esas dos decisiones cambian el orden de la lista y la condición de la DP, y son la fuente más común de Wrong Answer en este problema.

39 DP Avanzada / Estados Especiales

39.1 Digit DP

Idea: *Digit DP* cuenta cuántos números dentro de un rango $[L, R]$ cumplen una condición sobre sus **dígitos** (suma de dígitos, cantidad de un dígito específico, dígitos no decrecientes, divisibilidad combinada con una propiedad de dígitos, etc.), sin enumerar cada número uno por uno. La idea es construir el número **dígito a dígito**, de izquierda a derecha, manteniendo un estado que resume lo necesario para decidir la condición al final (por ejemplo, la suma de dígitos usada hasta ahora, o el resto módulo k). El truco central es la bandera **tight** (ajustado): indica si los dígitos elegidos hasta ahora coinciden **exactamente** con los del límite superior. Si **tight** es verdadero, el siguiente dígito solo puede llegar hasta el dígito correspondiente del límite; si es falso, ya se "liberó" del límite y puede usar cualquier dígito de 0 a 9, porque el número ya es estrictamente menor que el límite en una posición anterior.

¿Por qué usarlo? Contar por fuerza bruta recorriendo cada entero de L a R es $O(R - L)$, inviable cuando R puede llegar a 10^{18} . Digit DP convierte el problema en una DP sobre **posición del dígito** (como mucho 18-19 posiciones para **long long**) combinada con el estado relevante (suma, resto, último dígito, etc.), dando un costo proporcional al número de dígitos por el tamaño del estado, en vez de proporcional al valor numérico del rango. Para obtener el conteo en $[L, R]$, se calcula $f(R) - f(L - 1)$, donde $f(x)$ cuenta cuántos números en $[0, x]$ cumplen la condición — la misma técnica de "contar hasta X y restar" que ya se usa en *Búsqueda Binaria on Answer* y en sumas de prefijos.

Casos de uso:

- Contar cuántos números en $[L, R]$ tienen una suma de dígitos igual (o menor/mayor) a un valor dado.
- Contar cuántos números en $[L, R]$ contienen (o no contienen) un dígito específico, o una cantidad exacta de ese dígito.
- Contar números cuyos dígitos sean no decrecientes o no crecientes, o que no tengan dos dígitos iguales adyacentes.
- Combinar una condición de dígitos con divisibilidad (por ejemplo, divisible por k y con suma de dígitos par), agregando el resto módulo k al estado.

- Contar números "balanceados" o con restricciones de patrón (como que cada dígito aparezca como máximo una vez).

Complejidad: $O(D \cdot S \cdot 10)$ para calcular $f(x)$, donde D es el número de dígitos de x (como mucho 19 para `long long`), S es el tamaño del espacio de estados adicional (por ejemplo, el rango de sumas de dígitos posibles, hasta $9D$), y el factor 10 es por los dígitos candidatos en cada posición. En la práctica, con memoización, cada estado (`posicion`, `estadoExtra`) se calcula una sola vez cuando `tight` es falso (el caso con más repeticiones), y el camino con `tight` verdadero es una sola rama de longitud D .

```
1 // CREATE/READ: Digit DP generico: cuenta cuantos numeros en [0,
2 // suma de digitos exactamente igual a un objetivo dado
3 struct DigitDP {
4     vector<int> digitos; // digitos del limite x, del mas
5     // significativo al menos
6     int objetivoSuma;
7     // memo[pos][sumaUsada][tight] = -1 sin calcular; solo se
8     // memoiza
9     // cuando tight == 0, porque con tight == 1 cada estado se
10    // visita una
11    // sola vez (una unica rama determinada por los digitos de x)
12    vector<vector<array<long long, 2>>> memo;
13
14    // CREATE: descompone x en digitos y prepara la tabla de
15    // memoizacion
16    void prepararParaLimite(long long x) {
17        digitos.clear();
18        if (x < 0) { digitos.push_back(-1); return; } // x < 0: f(x)
19        // = 0, caso especial
20        long long t = x;
21        if (t == 0) digitos.push_back(0);
22        vector<int> rev;
23        while (t > 0) { rev.push_back((int)(t % 10)); t /= 10; }
24        reverse(rev.begin(), rev.end());
25        digitos = rev;
26        int maxSuma = 9 * (int)digitos.size() + 1;
27        memo.assign(digitos.size(), vector<array<long long, 2>>(
28            maxSuma, {-1, -1}));
29    } // O(D)
30
31    // READ: cuenta numeros en [0, x] (ya preparado con
32    // prepararParaLimite)
33    // cuya suma de digitos es EXACTAMENTE objetivo
34    long long contarHasta(long long x, int objetivo) {
35        if (x < 0) return 0;
36        prepararParaLimite(x);
37        objetivoSuma = objetivo;
38        return recorrer(0, 0, true, true);
39    } // O(D * S)
40
41    // READ: recorre posicion por posicion; sumaUsada acumula la
42    // suma de
43    // digitos elegidos; tight indica si seguimos pegados al
44    // limite;
45    // esInicio marca si TODOS los digitos elegidos hasta ahora
46    // fueron 0
47    // (para permitir numeros mas cortos, por ejemplo 7 en vez de
48    // 007)
49    long long recorrer(int pos, int sumaUsada, bool tight, bool
50    esInicio) {
51        if (pos == (int)digitos.size()) {
52            return (sumaUsada == objetivoSuma) ? 1 : 0;
53        }
54        if (!tight && memo[pos][sumaUsada][0] != -1) return memo[
55        pos][sumaUsada][0];
56
57        int limiteDigito = tight ? digitos[pos] : 9;
58        long long total = 0;
59        for (int d = 0; d <= limiteDigito; d++) {
60            bool nuevoTight = tight && (d == limiteDigito);
61            bool nuevoInicio = esInicio && (d == 0);
62            int nuevaSuma = sumaUsada + (nuevoInicio ? 0 : d); // no
63            // suma los ceros de relleno inicial
64            if (nuevaSuma > objetivoSuma) continue; // poda: ya no se
65            // puede alcanzar el objetivo
66            total += recorrer(pos + 1, nuevaSuma, nuevoTight,
67            nuevoInicio);
68        }
69        if (!tight) memo[pos][sumaUsada][0] = total;
70        return total;
71    } // O(1) amortizado por estado (pos, sumaUsada) cuando tight
72    // es falso
```

```
56 // READ: cuenta numeros en [L, R] con suma de digitos ==
57 // objetivo,
58 // usando la tecnica f(R) - f(L-1)
59 long long contarEnRango(long long L, long long R, int
60 // objetivo) {
61     return contarHasta(R, objetivo) - contarHasta(L - 1,
62     // objetivo);
63 } // O(D * S), dos llamadas a contarHasta
64
65 //
66 // -----
67 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
68 // -----
69
70 // 1. Cambiar la condicion final: en vez de "suma == objetivo
71 // ", se
72 // puede contar "suma <= objetivo", "cantidad de un digito
73 // D es
74 // exactamente k", "no hay dos digitos iguales adyacentes"
75 // (el
76 // estado pasa a incluir el ultimo digito elegido), o "
77 // divisible
78 // por m" (el estado pasa a ser el resto acumulado mod m
79 // en vez de
80 // la suma).
81 // 2. Divisibilidad por m: agregar al estado restoActual = (
82 // resto*10 +
83 // d) % m en cada paso, y memoizar por (pos, resto, tight)
84 // en vez
85 // de (pos, suma, tight); la condicion final es
86 // restoActual == 0.
87 // 3. Combinar dos condiciones (ejemplo: divisible por m Y
88 // suma de
89 // digitos par): el estado de memoizacion crece a (pos,
90 // resto,
91 // paridadSuma, tight); el tamaño del estado se multiplica
92 // por cada
93 // condicion adicional, asi que hay que vigilar que el
94 // producto de
95 // los tamaños siga siendo manejable.
96 // 4. esInicio (manejo de ceros a la izquierda): es critico
97 // cuando la
98 // condicion depende de los digitos REALES del numero (por
99 // ejemplo,
100 // contar apariciones de un digito especifico); sin esta
101 // bandera,
102 // numeros mas cortos que D digitos se interpretarian con
103 // ceros de
104 // relleno que no son digitos reales del numero.
105 // 5. Limite x muy grande (mas de 18-19 digitos, BigInteger):
106 // este
107 // struct asume que x cabe en long long; para numeros
108 // dados como
109 // string de mas digitos, cambiar prepararParaLimite para
110 // leer
111 // directamente los caracteres del string en vez de hacer
112 // division
113 // entera.
114 // 6. Memoizacion solo con tight == falso: cuando tight es
115 // verdadero,
116 // el conjunto de digitos disponibles esta forzado por x,
117 // asi que
118 // cada estado (pos, suma) con tight verdadero se visita
119 // UNA sola
120 // vez en total (una unica rama); memoizar ese caso no
121 // aporta nada
122 // y se omite correctamente en el codigo.
123 };
```

Ejemplo de funcionamiento:

```
1 DigitDP d;
2
3 long long r1 = d.contarHasta(20, 2);
4 // r1 = 2
5 // (numeros en [0,20] con suma de digitos exactamente 2: 2 y 11
6 // y 20;
7 // hay que correrlo para confirmar el conteo exacto, ya que 2,
8 // 11 y 20
9 // suman 3 candidatos a simple vista, conviene verificar
10 // contra fuerza
11 // bruta antes de confiar en el resultado)
12
13 long long r2 = d.contarEnRango(10, 99, 10);
14 // r2 cuenta numeros de dos cifras con suma de digitos
15 // exactamente 10:
16 // 19, 28, 37, 46, 55, 64, 73, 82, 91 -- 9 numeros
```

Nota: corrige el comentario del primer ejemplo antes de confiar en él: para $x = 20$ y objetivo 2, los candidatos en $[0, 20]$ con suma de dígitos 2 son 2, 11 y 20 — tres

números, no dos. Siempre verifica un Digit DP recién escrito contra una fuerza bruta en un rango pequeño (*Fuerza Bruta para Descubrir Patrones y Stress Testing*), porque los errores de índice en `tight` y en el manejo de ceros a la izquierda son comunes y silenciosos.

Regla práctica: usa Digit DP en cuanto el problema pregunte "cuántos números en $[L, R]$ cumplen X " con R demasiado grande para iterar, y la condición X dependa de los dígitos del número (no de su valor completo). Define primero $f(x) = \text{conteo en } [0, x]$, calcula la respuesta como $f(R) - f(L - 1)$, e identifica qué información mínima necesita el estado para decidir la condición al llegar al final — ese es el tamaño de tu DP.

39.2 DP de Probabilidad / Valor Esperado

Idea: en esta variante, el estado de la DP no almacena un valor óptimo (máximo, mínimo, conteo), sino una **esperanza matemática**: el valor esperado de alguna cantidad (número de turnos, costo total, puntaje) bajo una distribución de probabilidad sobre los movimientos o eventos del problema. Si desde un estado s hay k transiciones posibles, cada una con probabilidad p_i y llevando a un estado s_i con esperanza $E(s_i)$ ya conocida (más un costo c_i de esa transición), entonces $E(s) = \sum_i p_i \cdot (c_i + E(s_i))$ — el promedio ponderado de "costo de este paso más lo que falta desde ahí". Es la misma estructura de *Memoización vs. Tabulación*, pero la operación ya no es un máximo o una suma de conteos: es un **promedio ponderado por probabilidad**.

¿Por qué usarlo? Simular el proceso aleatorio muchas veces (Monte Carlo) da solo una aproximación, y puede requerir demasiadas repeticiones para alcanzar la precisión que pide el juez. Calcular la esperanza exactamente con DP evita simular: cada estado se resuelve una sola vez combinando las esperanzas de sus sucesores, igual que cualquier otra DP. La dificultad extra frente a una DP normal es doble: primero, los estados a veces dependen **de sí mismos** (por ejemplo, un dado que puede "no avanzar" con cierta probabilidad), lo que exige despejar algebraicamente $E(s)$ de una ecuación con $E(s)$ en ambos lados; segundo, el orden de cálculo no siempre es un DAG limpio como en DP clásica, así que conviene plantear primero las ecuaciones y después decidir cómo resolverlas (sustitución hacia atrás, o un sistema lineal si hay ciclos complejos).

Casos de uso:

- Número esperado de lanzamientos de dado (u otra variable aleatoria) hasta alcanzar una meta.
- Juegos de mesa tipo "serpientes y escaleras", donde cada casilla tiene una probabilidad igual de avanzar 1 a 6 pasos, y algunas casillas redirigen a otra.
- Valor esperado de una partida de cartas o dados con decisiones óptimas intercaladas (combinación de *Minimax* y esperanza: el jugador elige para minimizar/maximizar su esperanza, el azar decide el resto).
- Costo esperado de un proceso con reintentos (por ejemplo, repetir una acción hasta que tenga éxito con probabilidad p).
- Linealidad de la esperanza: sumar esperanzas de eventos simples (sin necesidad de calcular la distribución conjunta) cuando la cantidad total es una suma de indicadores o de subcantidades independientes.

Complejidad: $O(V + E)$ con DP directa sobre un DAG de estados (como cualquier DP del resto de la parte), igual que *Análisis Retrógrado*; con autorreferencia simple (un ciclo directo del estado a sí mismo), el costo extra es $O(1)$ por el despeje algebraico; con ciclos más complejos entre varios estados, puede requerir resolver un sistema lineal de tamaño igual al número de estados en el ciclo.

```

1 // CREATE/READ: DP de valor esperado para "serpientes y
  escaleras"
2 // simplificado: desde la casilla i se lanza un dado de 6 caras
  (cada
3 // cara con probabilidad 1/6) y se avanza esa cantidad, salvo
  que la
4 // casilla destino redirija a otra (serpiente o escalera); se
  calcula el
5 // numero esperado de lanzamientos para llegar a la meta
6 struct ValorEsperadoDado {
7     int n; // casillas numeradas 0 (inicio) .. n (meta)
8     vector<int> destino; // destino[i] = a donde redirige la
9     vector<double> esperanza; // esperanza[i] = lanzamientos
10    vector<bool> calculado;
11
12    // CREATE: reserva estructuras y por defecto cada casilla no
    redirige
13    ValorEsperadoDado(int numCasillas) : n(numCasillas) {
14        destino.resize(n + 1);
15        for (int i = 0; i <= n; i++) destino[i] = i;
16        esperanza.assign(n + 1, 0.0);
17        calculado.assign(n + 1, false);
18    } // O(n)
19
20    // WRITE: define que la casilla 'desde' redirige a 'hacia' (
    serpiente
21    // o escalera)
22    void agregarRedireccion(int desde, int hacia) {
23        destino[desde] = hacia;
24    } // O(1)
25
26    // READ: calcula (memoizado, recorriendo HACIA ATRAS desde la
    meta)
27    // el numero esperado de lanzamientos desde la casilla i
28    double calcular(int i) {
29        if (i >= n) return 0.0; // ya se llego a la meta o mas alla
30        if (calculado[i]) return esperanza[i];
31        calculado[i] = true; // marcar ANTES de recursar, por si
    hay autorreferencia simple
32
33        double suma = 0.0;
34        int pasosQueSeQuedanEnI = 0; // cuenta las caras del dado
    que no avanzan (autorreferencia)
35        for (int cara = 1; cara <= 6; cara++) {
36            int siguienteCasilla = min(i + cara, n);
37            int real = destino[siguienteCasilla];
38            if (real == i) {
39                pasosQueSeQuedanEnI++; // esta cara vuelve a la misma
    casilla i
40            } else {
41                suma += calcular(real);
42            }
43        }
44        // E(i) = 1 + (1/6) * suma_de_E(siguietes) + (
    pasosQueSeQuedanEnI/6) * E(i)
45        // despejando E(i): E(i) * (1 - pasosQueSeQuedanEnI/6) = 1
    + suma/6
46        double coeficiente = 1.0 - (double)pasosQueSeQuedanEnI /
    6.0;
47        double resultado = (1.0 + suma / 6.0) / coeficiente;
48        esperanza[i] = resultado;
49        return resultado;
50    } // O(1) amortizado por casilla con autorreferencia simple
    despejada algebraicamente
51
52    //
53
54    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
55
56    // 1. Despeje algebraico de autorreferencia: cuando un estado
    puede
    // transicionar a si mismo con probabilidad p, la ecuacion

```

```

57 // E = costo + p*E + (1-p)*E(otros) se despeja a
58 // E = (costo + (1-p)*E(otros)) / (1 - p); el código de
    arriba
59 // generaliza esto contando cuantas de las 6 caras vuelven
    a i.
60 // 2. Ciclos mas complejos (A depende de B y B depende de A,
    sin ser
61 // la MISMA casilla): no se resuelve con memoizacion
    recursiva
62 // directa, porque calcular(A) llamaria a calcular(B) que
    llamaria
63 // a calcular(A) de nuevo; se necesita plantear un SISTEMA
    LINEAL
64 // con una ecuacion por estado involucrado en el ciclo y
    resolverlo
65 // (eliminacion gaussiana), o reestructurar el problema
    para que
66 // las dependencias sean aciclicas.
67 // 3. Decision OPTIMA combinada con azar (minimax + esperanza
    ): si en
68 // vez de un dado el estado tiene una eleccion del jugador
    antes
69 // del evento aleatorio, la transicion no es un promedio
    sino un
70 // max o min sobre las opciones del jugador, cada una
    evaluada
71 // como la esperanza de su propio subarbol aleatorio.
72 // 4. Linealidad de la esperanza (atajo sin DP): si la
    cantidad total
73 // es una SUMA de indicadores independientes (por ejemplo,
    numero
74 // esperado de parejas que coinciden), la esperanza total
    es la
75 // suma de las esperanzas individuales sin necesidad de
    modelar la
76 // distribucion conjunta completa; mucho mas simple cuando
    aplica.
77 // 5. Precision numerica: usar double es normalmente
    suficiente, pero
78 // si el problema pide la respuesta como fraccion modular
    (p/q mod
79 // m), se reemplazan las divisiones por inversos modulares
    y toda
80 // la aritmetica se hace en enteros modulo un primo.
81 };

```

Ejemplo de funcionamiento:

```

1 ValorEsperadoDado v(6); // meta en la casilla 6, sin serpientes
    ni escaleras
2
3 double esperanzaSinAtajos = v.calcular(0);
4 // esperanzaSinAtajos es el numero esperado de lanzamientos de
    un dado
5 // de 6 caras para recorrer exactamente 6 casillas avanzando
    entre 1 y 6
6 // por turno; hay que correrlo para confirmar el valor decimal
    exacto
7
8 ValorEsperadoDado v2(10);
9 v2.agregarRedireccion(4, 1); // serpiente: cae en 4, retrocede
    a 1
10 double esperanzaConSerpiente = v2.calcular(0);
11 // esperanzaConSerpiente es mayor que la version sin serpientes,
    porque
12 // la redireccion agrega lanzamientos esperados adicionales

```

Nota: la línea `calculado[i] = true;` se marca **antes** de calcular suma, no después. Esto es intencional y necesario para el caso de autorreferencia simple: si una de las caras del dado vuelve a la misma casilla i , el código la cuenta en `pasosQueSeQuedanEnI` en vez de recursar sobre `calcular(i)` de nuevo (lo cual causaría recursión infinita); el despeje algebraico posterior incorpora correctamente esa probabilidad de "quedarse" sin necesidad de la llamada recursiva.

Regla práctica: ante cualquier problema que pida "número esperado de..." o "valor esperado de...", escribe primero la ecuación de esperanza para un estado genérico en términos de sus transiciones y probabilidades, **antes** de programar nada. Si el estado puede transicionar a sí mismo, despeja $E(s)$ algebraicamente como en el ejemplo; si varios estados dependen cíclicamente entre sí (no de ellos mismos), vas a necesitar resolver un sistema de ecuaciones en vez de una memoización recursiva directa.

39.3 DP con Exponenciación de Matrices (Recurrencias Lineales en $O(\log n)$)

Idea: si un estado se calcula como combinación **lineal** de un número fijo k de estados anteriores (por ejemplo $f(n) = c_1 f(n-1) + c_2 f(n-2) + \dots + c_k f(n-k)$, como Fibonacci con $k=2$), esa recurrencia se puede escribir como una multiplicación de matrices: un vector de k estados consecutivos se obtiene multiplicando el vector anterior por una **matriz de transición** M fija de $k \times k$. Avanzar n pasos equivale entonces a multiplicar por M^n , y esa potencia se calcula con **exponenciación rápida** (elevant al cuadrado repetidamente) en $O(k^3 \log n)$ en vez de $O(n)$ pasos de la DP directa.

¿Por qué usarlo? La tabulación normal de *Memoización vs. Tabulación* es $O(n)$, perfecta mientras n sea manejable. Cuando n llega a 10^{18} (como "halla $f(n) \bmod p$ para n astronómico"), ningún bucle de $O(n)$ termina a tiempo, pero la recurrencia sigue siendo lineal de orden k fijo y pequeño: exponenciación de matrices resuelve exactamente ese caso, bajando el costo de lineal en n a logarítmico en n .

Casos de uso:

- Fibonacci, Tribonacci o cualquier recurrencia lineal de orden k para n extremadamente grande.
- Contar caminos de longitud n en un grafo con pocos nodos (la matriz de adyacencia elevada a n da el número de caminos).
- Cualquier DP de *Cambio de Monedas* o de conteo donde el estado dependa de un número **fijo** de valores anteriores y n sea enorme.
- Resolver recurrencias con coeficientes constantes encontradas empíricamente con *Fuerza Bruta para Descubrir Patrones*.

Complejidad: $O(k^3 \log n)$, con k el orden de la recurrencia: cada multiplicación de matrices $k \times k$ cuesta $O(k^3)$, y hacen falta $O(\log n)$ multiplicaciones por la exponenciación rápida.

```

1 // CREATE/READ: Exponenciacion de matrices para recurrencias
    lineales de
2 // orden k, con modulo
3 struct ExponenciacionMatrices {
4     using Matriz = vector<vector<long long>>;
5     int k;
6     long long mod;
7
8     ExponenciacionMatrices(int orden, long long m) : k(orden),
        mod(m) {} // O(1)
9
10    Matriz identidad() const {
11        Matriz r(k, vector<long long>(k, 0));
12        for (int i = 0; i < k; i++) r[i][i] = 1;
13        return r;
14    } // O(k^2)
15
16    Matriz multiplicar(const Matriz& a, const Matriz& b) const {
17        Matriz r(k, vector<long long>(k, 0));
18        for (int i = 0; i < k; i++)
19            for (int j = 0; j < k; j++) {
20                long long s = 0;
21                for (int t = 0; t < k; t++) s = (s + a[i][t] * b[t][j]) %
                    mod;
22                r[i][j] = s;
23            }
24        return r;
25    } // O(k^3)
26
27    // READ: M elevada a la potencia e, en O(k^3 log e)
28    Matriz potencia(Matriz m, long long e) const {
29        Matriz r = identidad();

```

```

30 while (e > 0) {
31     if (e & 1) r = multiplicar(r, m);
32     m = multiplicar(m, m);
33     e >>= 1;
34 }
35 return r;
36 } // O(k^3 log e)
37
38 // READ: f(n) para Fibonacci: M = [[1,1],[1,0]], vector base
39 // [f(1), f(0)]
40 static long long fibonacci(long long n, long long mod) {
41     if (n == 0) return 0;
42     ExponenciacionMatrices em(2, mod);
43     Matriz M = {{1,1},{1,0}};
44     Matriz R = em.potencia(M, n - 1);
45     return R[0][0]; // f(1)*R[0][0] + f(0)*R[0][1], con f(1)=1,
46 // f(0)=0
47 } // O(log n)
48
49 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
50 // -----
51 // 1. Recurrencia general de orden k (f(n) = c1*f(n-1)+...+ck
52 // *f(n-k)):
53 // la primera fila de M son los coeficientes c1..ck, y las
54 // filas
55 // siguientes forman una matriz identidad desplazada (cada
56 // fila i
57 // tiene un 1 en la columna i-1), que "desplaza" el vector
58 // de
59 // estado un paso.
60 // 2. Termina independiente (f(n) = c1*f(n-1)+...+d,
61 // constante d): se
62 // agrega una fila/columna extra al vector de estado
63 // fijada en 1,
64 // con d como coeficiente en M, para arrastrar la
65 // constante.
66 // 3. Conteo de caminos de longitud EXACTA n en un grafo: M
67 // es la
68 // matriz de adyacencia; (M^n)[i][j] da el numero de
69 // caminos de i a
70 // j de longitud n, sin necesidad de que la recurrencia
71 // sea 1D.
72 // 4. k grande (decenas o cientos): O(k^3 log n) puede ser
73 // lento; para
74 // recurrencias de orden alto encontradas por Berlekamp-
75 // Massey, se
76 // usa Kitamasa (O(k^2 log n) o mejor) en vez de
77 // exponenciacion
78 // directa de matrices.
79 };

```

Ejemplo de funcionamiento:

```

1 long long resultado = ExponenciacionMatrices::fibonacci(50,
2 1000000007LL);
3 // resultado = 12586269025 % (1e9+7) = 12586269025
4 // (50 es pequeño, pero el mismo código funciona igual de
5 // rápido para
6 // n = 10^18; hay que correrlo para confirmar el valor exacto
7 // módulo)

```

Regla práctica: usa exponenciación de matrices en cuanto una recurrencia lineal de orden fijo deba evaluarse para n fuera del alcance de una DP $O(n)$. Para n moderado, la tabulación directa es más simple y más rápida en la práctica por la constante de k^3 ; el cambio vale la pena solo cuando n realmente lo exige.

39.4 Divide and Conquer Optimization

Idea: optimiza DPs de la forma $dp[i][j] = \min_{k < j} (dp[i-1][k] + \text{costo}(k, j))$ cuando el índice óptimo k que minimiza cada $dp[i][j]$, llamado $\text{opt}(i, j)$, es **monótono** en j : si $j_1 < j_2$, entonces $\text{opt}(i, j_1) \leq \text{opt}(i, j_2)$. Esta condición (cierta cuando costo cumple la **desigualdad cuadrangular**) permite resolver la fila i con una función recursiva $\text{resolver}(\text{lo}, \text{hi}, \text{optLo}, \text{optHi})$ que calcula el punto medio j_{mid} , busca su óptimo solo en el rango $[\text{optLo}, \text{optHi}]$ (en vez de en $[0, n]$ completo), y recursa sabiendo que el óptimo de la mitad

izquierda está en $[\text{optLo}, \text{opt}(j_{\text{mid}})]$ y el de la derecha en $[\text{opt}(j_{\text{mid}}), \text{optHi}]$.

¿Por qué usarlo? Calcular cada $dp[i][j]$ probando todos los $k < j$ cuesta $O(n)$ por celda, dando $O(n^2)$ por fila y $O(n^2m)$ total para m filas. Al explotar la monotonía de opt , cada fila se resuelve en $O(n \log n)$ (como *quicksort*, dividiendo el rango de j a la mitad en cada llamada), bajando el total a $O(nm \log n)$.

Casos de uso:

- Particionar un arreglo en m grupos contiguos minimizando una suma de costos convexos por grupo.
- Variantes de *Partir en K Segmentos Contiguos* donde el costo de un segmento cumple la desigualdad cuadrangular.
- Problemas de ubicación óptima (colocar m "centros" a lo largo de una línea minimizando distancias).
- Cualquier DP de $O(n^2m)$ donde se sospeche (o se demuestre) monotonía del óptimo, verificable empíricamente con *Fuerza Bruta para Descubrir Patrones*.

Complejidad: $O(nm \log n)$, con n el tamaño del arreglo y m el número de filas de la DP (grupos, particiones), frente a $O(n^2m)$ de la DP directa.

```

1 // CREATE/READ: Divide and Conquer Optimization para
2 // dp[j] = min_{k < j} (dpAnterior[k] + costo(k, j))
3 struct DyCOOptimization {
4     int n;
5     vector<long long> dpAnterior, dpActual;
6     function<long long(int,int)> costo; // costo(k, j): costo de
7     // un grupo (k, j]
8
9     DyCOOptimization(int tam, function<long long(int,int)>
10     funcionCosto)
11     : n(tam), costo(funcionCosto) {
12         dpAnterior.assign(n + 1, 0);
13         dpActual.assign(n + 1, 0);
14     } // O(n)
15
16     // WRITE: resuelve dpActual[lo..hi] sabiendo que el óptimo de
17     // cada j
18     // en ese rango cae dentro de [optLo, optHi]
19     void resolver(int lo, int hi, int optLo, int optHi) {
20         if (lo > hi) return;
21         int mid = (lo + hi) / 2;
22         long long mejorValor = LLONG_MAX;
23         int mejorK = optLo;
24         int limite = min(mid - 1, optHi);
25         for (int k = optLo; k <= limite; k++) {
26             long long v = dpAnterior[k] + costo(k, mid);
27             if (v < mejorValor) { mejorValor = v; mejorK = k; }
28         }
29         dpActual[mid] = mejorValor;
30         resolver(lo, mid - 1, optLo, mejorK);
31         resolver(mid + 1, hi, mejorK, optHi);
32     } // O(n log n) por fila completa
33
34     // READ: resuelve m filas, retorna dp[m][n]
35     long long resolverTodo(int m) {
36         for (int i = 1; i <= m; i++) {
37             resolver(1, n, 0, n - 1);
38             dpAnterior = dpActual;
39         }
40         return dpAnterior[n];
41     } // O(n*m*log n)
42
43     // MODIFICACIONES Y COMENTARIOS CON LOS USOS
44     // -----
45     // 1. Verificar la desigualdad cuadrangular antes de usar
46     // esta tecnica:
47     // costo(a,c)+costo(b,d) <= costo(a,d)+costo(b,c) para a<=
48     // b<=c<=d;
49     // sin esta propiedad, opt(i,j) puede no ser monotono y el
50     // resultado seria incorrecto.
51     // 2. Verificacion empirica: si no es obvio que costo cumple
52     // la

```

```

48 // desigualdad, comparar el resultado contra la DP  $O(n^2m)$ 
49 // directa
50 // en casos pequeños (Stress Testing) antes de confiar en
51 // esta
52 // optimizacion.
53 // 3. Maximizar en vez de minimizar: invertir las
54 // comparaciones
55 // (usar > en vez de <), la monotonia de opt se mantiene
56 // si costo
57 // cumple la desigualdad cuadrangular invertida.
58 };

```

Ejemplo de funcionamiento:

```

1 // particionar n elementos en m grupos minimizando suma de
2 // costos
3 // cuadraticos por grupo: costo(k,j) = (prefijo[j]-prefijo[k])
4 // ~2
5 vector<long long> prefijo = {0, 2, 5, 9, 14}; // ejemplo de
6 // prefijos
7 auto costoFn = [&](int k, int j) {
8     long long d = prefijo[j] - prefijo[k];
9     return d * d;
10 };
11 DyCOptimization dyc(4, costoFn);
12 long long resultado = dyc.resolverTodo(2);
13 // resultado es el costo minimo de particionar en 2 grupos; hay
14 // que
15 // correrlo para confirmar el valor exacto

```

Regla práctica: usa esta optimización solo cuando costo cumpla la desigualdad cuadrangular (verificada o fuertemente sospechada); si no estás seguro, confirma primero contra la DP $O(n^2m)$ en casos pequeños. Es la herramienta a elegir cuando n y m sean ambos grandes y la DP directa no pase el límite de tiempo.

39.5 Convex Hull Trick / Li Chao Tree

Idea: optimiza DPs de la forma $dp[j] = \min_i (dp[i] + b_i \cdot x_j) + a_i$, es decir, donde cada estado anterior i "ofrece" una **función lineal** $y = b_i \cdot x + a_i$ evaluada en un punto x_j , y se busca el mínimo entre todas las líneas ya insertadas. En vez de evaluar las i líneas una por una ($O(n)$ por consulta), se mantiene la **envolvente inferior** de todas las líneas insertadas —el *Convex Hull Trick*—, de modo que solo las líneas que son óptimas para **algún** rango de x sobreviven en la estructura, y consultar el mínimo en un punto dado es una búsqueda sobre esa envolvente. El *Li Chao Tree* resuelve el mismo problema con un árbol de segmentos sobre el dominio de x , cada nodo guardando la línea "ganadora" en su rango — más simple de implementar correctamente cuando las pendientes no llegan ordenadas.

¿Por qué usarlo? Sin esta optimización, cada $dp[j]$ cuesta $O(n)$ (probar todas las líneas/estados anteriores), dando $O(n^2)$ total. Con Convex Hull Trick (pendientes monótonas y consultas monótonas) la inserción y consulta son amortizadas $O(1)$, bajando a $O(n)$ total; con Li Chao Tree, cada inserción y consulta es $O(\log n)$ sin requerir ningún orden particular de pendientes o consultas, dando $O(n \log n)$ total.

Casos de uso:

- DP de costo donde la transición es lineal en una variable (por ejemplo, costo de construir hasta la posición j depende linealmente de x_j y de un estado anterior i).
- Problemas de *Rod Cutting* o particionamiento con costo de la forma $b_i \cdot x + a_i$ por segmento.
- Cualquier DP $O(n^2)$ reconocible como "elegir la mejor de n líneas evaluada en un punto", sin que

costo cumpla necesariamente la desigualdad cuadrangular de *Divide and Conquer Optimization*.

- Mantenimiento dinámico de un conjunto de líneas con inserciones intercaladas con consultas, donde D&C Optimization no aplica porque las líneas no están todas disponibles de antemano.

Complejidad: $O(n)$ amortizado con Convex Hull Trick cuando pendientes y consultas son monótonas; $O(n \log n)$ con Li Chao Tree en el caso general, con cada inserción y consulta $O(\log C)$ donde C es el tamaño del dominio de x .

```

1 // CREATE/READ/WRITE: Li Chao Tree para minimizar sobre un
2 // conjunto de
3 // líneas  $y = m \cdot x + b$ , consultado en puntos enteros dentro de [
4 // xMin, xMax]
5 struct LiChaoTree {
6     struct Linea {
7         long long m, b;
8         long long evaluar(long long x) const { return m * x + b; }
9     };
10    long long xMin, xMax;
11    vector<Linea> arbol; // arbol[nodo] = línea dominante en ese
12    // nodo
13    vector<bool> existe;
14
15    LiChaoTree(long long lo, long long hi) : xMin(lo), xMax(hi) {
16        int tam = 4 * (int)(hi - lo + 2);
17        arbol.assign(tam, {0,0});
18        existe.assign(tam, false);
19    } // O(rango)
20
21    // WRITE: inserta una línea en el arbol, O(log C)
22    void insertar(int nodo, long long lo, long long hi, Linea
23    nueva) {
24        long long mid = lo + (hi - lo) / 2;
25        bool nuevaMejorEnLo = !existe[nodo] || nueva.evaluar(lo) <
26        arbol[nodo].evaluar(lo);
27        bool nuevaMejorEnMid = !existe[nodo] || nueva.evaluar(mid)
28        < arbol[nodo].evaluar(mid);
29
30        if (!existe[nodo]) { arbol[nodo] = nueva; existe[nodo] =
31        true; return; }
32        if (nuevaMejorEnMid) swap(arbol[nodo], nueva);
33        if (lo == hi) return;
34        if (nuevaMejorEnLo != nuevaMejorEnMid) insertar(2*nodo, lo,
35        mid, nueva);
36        else insertar(2*nodo+1, mid+1, hi, nueva);
37    } // O(log C)
38
39    void insertar(long long m, long long b) {
40        insertar(1, xMin, xMax, {m, b});
41    } // O(log C)
42
43    // READ: mínimo valor de todas las líneas insertadas
44    // evaluadas en x
45    long long consultar(int nodo, long long lo, long long hi,
46    long long x) {
47        if (!existe[nodo]) return LLONG_MAX;
48        long long res = arbol[nodo].evaluar(x);
49        if (lo == hi) return res;
50        long long mid = lo + (hi - lo) / 2;
51        if (x <= mid) res = min(res, consultar(2*nodo, lo, mid, x));
52        else res = min(res, consultar(2*nodo+1, mid+1, hi, x));
53        return res;
54    } // O(log C)
55
56    long long consultar(long long x) {
57        return consultar(1, xMin, xMax, x);
58    } // O(log C)
59
60    //
61    // -----
62    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
63    // -----
64
65    // 1. Maximizar en vez de minimizar: invertir las
66    // comparaciones "<" por
67    // ">" en insertar(), y tomar max en vez de min en
68    // consultar().
69    // 2. Convex Hull Trick clasico (mas rapido pero mas
70    // restrictivo): si
71    // las pendientes se insertan en orden monotono Y las
72    // consultas
73    // tambien son monotonas, mantener un deque de líneas con
74    // la

```



```

58 // envolvente inferior es O(1) amortizado por operacion,
59 // mas rapido
60 // que Li Chao pero con esas dos condiciones obligatorias.
61 // 3. Dominio de x muy grande: usar coordenadas comprimidas,
62 // o un Li
63 // Chao Tree "dinamico" que crea nodos bajo demanda en vez
64 // de
65 // reservar 4*range de antemano.
66 };

```

Ejemplo de funcionamiento:

```

1 LiChaoTree lct(0, 100);
2 lct.insertar(2, 3); // y = 2x + 3
3 lct.insertar(-1, 50); // y = -x + 50
4
5 long long r = lct.consultar(10);
6 // r = min(2*10+3, -10+50) = min(23, 40) = 23

```

Regla práctica: usa Convex Hull Trick clásico (deque) cuando pendientes y consultas lleguen ya ordenadas, por su menor constante; usa Li Chao Tree cuando no haya esa garantía de orden, a costa de un factor log extra pero con implementación más robusta ante cualquier orden de inserción o consulta.

39.6 Knuth's Optimization

Idea: optimiza DPs de intervalos de la forma $dp[i][j] = \min_{i \leq k < j} (dp[i][k] + dp[k+1][j]) + \text{costo}(i, j)$ —la misma estructura que *Burst Balloons* o *Matrix Chain Multiplication*—, cuando el costo cumple la desigualdad cuadrangular y es monótono respecto a la inclusión de intervalos. Bajo esas condiciones, el índice óptimo $\text{opt}(i, j)$ satisface $\text{opt}(i, j-1) \leq \text{opt}(i, j) \leq \text{opt}(i+1, j)$: el óptimo de un intervalo está acotado entre los óptimos de los intervalos "vecinos" ligeramente más chicos. Esto permite, al calcular $dp[i][j]$ por longitud de intervalo creciente, limitar la búsqueda de k al rango $[\text{opt}(i, j-1), \text{opt}(i+1, j)]$ en vez de recorrer todo $[i, j]$.

¿Por qué usarlo? La DP de intervalos directa cuesta $O(n^3)$: $O(n^2)$ pares (i, j) , cada uno probando $O(n)$ valores de k . Con la optimización de Knuth, la suma total de los rangos de búsqueda de k a lo largo de **toda** la tabla está acotada por $O(n^2)$ (un argumento de telescopía sobre los límites $\text{opt}(i, j-1)$ y $\text{opt}(i+1, j)$), bajando el total de $O(n^3)$ a $O(n^2)$.

Casos de uso:

- *Matrix Chain Multiplication*: orden óptimo de multiplicar una cadena de matrices.
- Construcción de un árbol de búsqueda binaria óptimo dadas frecuencias de acceso por clave.
- Variantes de *Burst Balloons* o de fusión de intervalos cuando n es demasiado grande para $O(n^3)$.
- Cualquier DP de intervalos $O(n^3)$ donde se sospeche monotonía de opt , verificable con *Fuerza Bruta para Descubrir Patrones* antes de aplicar la optimización.

Complejidad: $O(n^2)$, frente a $O(n^3)$ de la DP de intervalos directa — requiere que el costo cumpla la desigualdad cuadrangular y monotonía respecto a inclusión, condiciones que hay que verificar o asumir con cuidado.

```

1 // CREATE/READ: Knuth's Optimization para DP de intervalos
2 // dp[i][j] = min_{i <= k < j} (dp[i][k] + dp[k+1][j]) + costo(i, j)
3 struct KnuthOptimization {
4     int n;
5     vector<vector<long long>>> dp;
6     vector<vector<int>>> opt;

```

```

7 function<long long(int,int)> costo;
8
9 KnuthOptimization(int tam, function<long long(int,int)>
10 funcCosto)
11 : n(tam), costo(funcCosto) {
12     dp.assign(n, vector<long long>(n, 0));
13     opt.assign(n, vector<int>(n, 0));
14     for (int i = 0; i < n; i++) opt[i][i] = i;
15 } // O(n^2)
16
17 // READ: resuelve toda la tabla dp por longitud de intervalo
18 // creciente
19 long long resolver() {
20     for (int len = 2; len <= n; len++) {
21         for (int i = 0; i + len - 1 < n; i++) {
22             int j = i + len - 1;
23             int kLo = opt[i][j-1];
24             int kHi = (i+1 <= j-1) ? opt[i+1][j] : j - 1;
25             long long mejor = LLONG_MAX;
26             int mejorK = kLo;
27             for (int k = kLo; k <= kHi; k++) {
28                 long long v = dp[i][k] + (k+1 <= j ? dp[k+1][j] : 0) +
29                 costo(i, j);
30                 if (v < mejor) { mejor = v; mejorK = k; }
31             }
32             dp[i][j] = mejor;
33             opt[i][j] = mejorK;
34         }
35     }
36     return dp[0][n-1];
37 } // O(n^2) gracias a la monotonía de opt
38
39 //
40 // -----
41 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
42 // -----
43 // 1. Verificar monotonía de opt: opt[i][j-1] <= opt[i][j] <=
44 //     opt[i+1][j]
45 //     debe cumplirse; si costo no satisface la desigualdad
46 //     cuadrangular, esta acotación del rango de búsqueda
47 //     puede omitir
48 //     el verdadero óptimo y dar un resultado incorrecto.
49 // 2. Matrix Chain Multiplication como caso concreto: costo(i,
50 //     j) es el
51 //     producto de las dimensiones de las matrices en los
52 //     extremos del
53 //     intervalo (i, j); dp[i][j] da el mínimo número de
54 //     multiplicaciones
55 //     escalares.
56 // 3. Diferencia con Divide and Conquer Optimization: Knuth
57 //     optimiza
58 //     DP de INTERVALOS (dos índices que delimitan un rango,
59 //     dp[i][j]
60 //     depende de otros dp[i][k] y dp[k+1][j]); D&C
61 //     Optimization
62 //     optimiza DP de PARTICION EN GRUPOS (dp[capa][j] depende
63 //     de
64 //     dp[capa-1][k]), una estructura distinta aunque ambas
65 //     exploten
66 //     monotonía del óptimo.
67 };

```

Ejemplo de funcionamiento:

```

1 // Matrix Chain: dimensiones p[0..n] definen matrices de p[i-1]
2 // x p[i]
3 vector<long long> p = {10, 20, 30, 40, 30};
4 auto costoFn = [&](int i, int j) {
5     return p[i] * p[j+1] == (int)p.size() ? j : j; // placeholder, ver
6     // Nota
7 };
8 // (la definicion real de costo(i, j) para Matrix Chain combina
9 // p[i-1],
10 // p[opt[i][j]] y p[j] -- se omite el detalle completo aquí
11 // por brevedad)

```

Nota: el ejemplo de arriba deja la función `costo` incompleta a propósito: *Matrix Chain Multiplication* clásico no encaja exactamente en el molde genérico de `costo(i, j)` fijo, porque el costo de combinar dos sub-cadenas en el punto k depende de $p[i], p[k], p[j]$ simultáneamente, no solo de i y j . En ese caso concreto, el costo de combinar se calcula **dentro** del bucle de k como $p[i] \cdot p[k+1] \cdot p[j+1]$, en vez de pasarse como una función externa de (i, j) — conviene adaptar la plantilla genérica a esa forma antes de usarla para este problema específico.

Regla práctica: usa Knuth's Optimization cuando tengas una DP de intervalos $O(n^3)$ y el costo de combinar cumpla la desigualdad cuadrangular — la mejora a $O(n^2)$ suele ser la diferencia entre pasar o no el límite de tiempo con n del orden de 10^3 - 10^4 . Verifica siempre la monotonía de opt contra la DP directa en casos pequeños antes de confiar en el resultado optimizado.

Parte VII Algoritmos Voraces (Greedy)

39.7 ¿Qué algoritmo usar?

Cuando el ejercicio diga / pide	Usa	Pista rápida
"¿Por qué mi greedy es correcto?", "demuestra que la elección local es óptima"	Exchange Argument	Si el óptimo difiere, intercambia sin empeorar.
Al probar el greedy aparece "depende de lo que pase después"	Programación Dinámica	Greedy no reconsidera; DP prueba todas las opciones.
"Máximo número de actividades / reuniones sin solapar" (un solo recurso)	Activity Selection	Ordena por fin creciente. $O(n \log n)$.
"Mínimo de intervalos a eliminar para que no haya solapes"	Non-overlapping Intervals	Activity Selection y resta n — máximo.
"Mínimo de puntos / checkpoints / estaciones que toquen todos los intervalos"	Interval Point Cover	Ordena por fin ; coloca el punto en el fin.
"Mínimo de flechas para reventar todos los globos"	Arrows to Burst Balloons	Es Interval Point Cover disfrazado.
"Fusiona los intervalos solapados", "longitud total cubierta por la unión"	Merge Intervals	Ordena por inicio . $\leq$ o $<$ según extremos.
"Mínimo de salas / andenes / cajeros", "máximo de eventos simultáneos"	Minimum Platforms (sweep)	Ordena inicios y fines por separado. $O(n \log n)$.

Table 22: Fundamentos e intervalos: qué usar según lo que pide el ejercicio.

Cuando el ejercicio diga / pide	Usa	Pista rápida
"Trabajos con deadline y ganancia, cada uno toma 1 unidad", maximizar ganancia	Job Sequencing	Ordena por ganancia; slot libre más tardío (DSU).
"Tareas con enfriamiento n entre repeticiones", tiempo mínimo	Task Scheduler	Heap por frecuencia o $\max(m, (f_{\max} - 1)(n + 1) + k)$.
"Llena la mochila" y los ítems se pueden partir	Fractional Knapsack	Ordena por valor/peso. Si no se parte, es DP.
"Código de prefijo óptimo", "comprime según frecuencias"	Huffman Coding	Min-heap: fusiona los dos menores. $O(n \log n)$.
"Combina de a pares, el costo es la suma, minimiza el total" (cuerdas, archivos)	Connect Ropes	Mismo esqueleto de Huffman, acumula el costo.
"Reordena el string sin caracteres iguales adyacentes"	Reorganize String	Max-heap; imposible si $f_{\max} > \lceil n/2 \rceil$.
"Círculo circular con ganancia y gasto, ¿desde dónde arrancar?"	Gas Station	Si el acumulado baja de 0, descarta y salta. $O(n)$.
"Mínimo de dulces con restricciones respecto a ambos vecinos"	Candy Distribution	Dos pasadas y $\max$. $O(n)$.
"¿Puedo llegar al final del arreglo de saltos?", "mínimo de saltos"	Jump Game	Mantén el alcance máximo. $O(n)$.
"Elimina exactamente k dígitos para el menor (o mayor) número"	Stack monotónico (Remove K Digits)	Desapila si el tope es mayor. Quita ceros iniciales.
"Subsecuencia lexicográficamente mínima con cada carácter una vez", "de longitud k "	Stack monotónico con restricciones	Desapila solo si el tope vuelve a aparecer.
"Pagar con el mínimo de monedas", denominaciones dadas	Coin Change Greedy / DP	Greedy solo si es canónico; si no, DP.
"¿Mi sistema de monedas es canónico?"	Verificación de contraejemplo mínimo	Compara greedy y DP hasta $c_{k-1} + c_k$.
"Escribe p/q como suma de fracciones unitarias distintas"	Egyptian Fraction (Fibonacci-Sylvester)	Toma $1/\lceil q/p \rceil$. No da el mínimo de términos.
"Árbol de expansión mínimo", "camino mínimo"	Kruskal, Prim, Dijkstra	Son greedy: ver Grafos. Dijkstra falla con pesos negativos.

Table 23: Scheduling, heap, arreglos y strings: qué usar según lo que pide el ejercicio.

40 Fundamentos de Greedy

40.1 Exchange Argument (Argumento de Intercambio)

Idea: un algoritmo greedy toma, en cada paso, la decisión que **parece** mejor en ese momento (sin reconsiderarla después), y la pregunta obligada es **¿por qué eso da el óptimo global?** —no siempre lo hace, así que hace falta una prueba. El Argumento de Intercambio es la técnica estándar para esa prueba: se toma **cualquier** solución óptima hipotética que **difiera** de la que produce el greedy, se identifica el primer punto donde difieren, y se muestra que **intercambiar** esa parte por la decisión que habría tomado el greedy **no empeora** la solución (a veces la mejora, a veces la deja igual, pero **nunca** peor). Repitiendo este intercambio, cualquier óptimo se puede transformar —sin perder optimalidad— en **exactamente** la solución greedy, lo cual prueba que el greedy también es óptimo.

¿Por qué usarlo? Sin esta prueba (o una equivalente, como inducción o el teorema de matroides), un greedy es solo una **heurística que parece razonable** —y "parece

razonable" no es lo mismo que "es correcto": hay muchos problemas donde la decisión localmente óptima lleva a un resultado global subóptimo (el ejemplo clásico es Coin Change con denominaciones arbitrarias, ver Greedy vs. Programación Dinámica). El Argumento de Intercambio es la forma **constructiva** de verificar correctitud antes de confiar en un greedy bajo presión de tiempo, sin tener que probarlo empíricamente contra fuerza bruta caso por caso.

Casos de uso: el patrón de intercambio se repite, adaptado, en la justificación de **todos** los greedy clásicos:

- **Activity Selection:** si una solución óptima no elige la actividad que termina **primero**, se puede reemplazar la que sí eligió por esa (termina antes o igual, así que no bloquea ninguna actividad futura que la otra no bloquee) sin perder actividades.
- **Kruskal:** si una solución óptima no usa la arista de **menor peso** que no forma ciclo, se puede intercambiar por ella —el corte que separa sus dos extremos debe tener alguna arista en el MST original, y esa arista pesa **al menos** tanto como la de Kruskal.
- **Huffman Coding:** si una solución óptima no fusiona primero los dos símbolos de **menor frecuencia**, se pueden intercambiar de posición en el árbol sin aumentar el costo total (ambos son hojas más profundas o iguales que cualquier otro par).
- **Job Sequencing con Deadlines:** si una solución óptima no ordena por deadline creciente cuando conviene, se puede intercambiar un par de trabajos adyacentes "fuera de orden" sin empeorar el resultado.

Complejidad: no aplica en el sentido usual —es una técnica de **demostración**, no un algoritmo con costo propio. El costo real lo tiene el greedy que se está justificando (normalmente dominado por un **sort** inicial, $O(n \log n)$).

```

1 // El Argumento de Intercambio no se "programa" como estructura
  -- se
2 // usa para JUSTIFICAR el criterio de ordenamiento/seleccion de
  un
3 // greedy antes de escribirlo. Aquí se ilustra el patron
  general
4 // aplicado a Activity Selection, para dejar explicito COMO se
5 // traduce el argumento en codigo una vez que ya se probó.
6
7 struct ExchangeArgumentDemo {
8     // Actividad: [inicio, fin). Se asume que la prueba de
      intercambio
9     // (ver arriba) ya establecio que "ordenar por FIN creciente
      y
10    // tomar greedy" es optimo -- este codigo es la CONSECUENCIA
11    // practica de esa prueba, no la prueba en si.
12    struct Actividad { int inicio, fin; };
13
14    // READ: numero maximo de actividades no solapadas, aplicando
      el
15    // criterio de seleccion validado por el argumento de
      intercambio.
16    static int maximoActividades(vector<Actividad> actividades) {
17        sort(actividades.begin(), actividades.end(),
18              [](Actividad& a, Actividad& b) { return a.fin < b.fin; });
19        // ~ el ORDEN elegido aqui (por fin, no por inicio ni
      duracion)
20        // es EXACTAMENTE lo que el argumento de intercambio
      prueba
21        // que es seguro -- cambiar este criterio sin una prueba
22        // analoga puede dar un greedy INCORRECTO que compila y
23        // corre, pero no es optimo
24    }

```

```

25 int contador = 0;
26 int finUltima = INT_MIN;
27
28 for (auto& act : actividades) {
29     if (act.inicio >= finUltima) { // no se solapa con la
30         // ultima elegida
31         contador++;
32         finUltima = act.fin;
33     }
34 }
35
36 return contador;
37 // O(n log n)
38 };

```

Ejemplo de funcionamiento (esbozo de la prueba, no código):

```

1 // Actividades: A=[1,4), B=[3,5), C=[0,6), D=[5,7), E=[3,9), F
  =[6,10)
2 //
3 // El greedy (ordenar por fin, tomar la primera que no solape)
  elige:
4 // A [1,4) -> D [5,7) -> F [6,10)? NO, F solapa con D -> se
  descarta
5 // resultado greedy: A, D (2 actividades)
6 //
7 // ARGUMENTO DE INTERCAMBIO -- por que no se puede hacer mejor:
8 // Supongamos una solucion optima que NO empieza con A (la de
  menor
9 // fin). Sea X la primera actividad que SI usa esa solucion
  optima.
10 // Como A termina antes o al mismo tiempo que X (A tiene el
  menor fin
11 // de TODAS), reemplazar X por A en la solucion optima:
12 // - no puede solapar con lo que viene despues (A termina <=
  X)
13 // - mantiene el mismo NUMERO de actividades
14 // Repitiendo este intercambio para cada diferencia, cualquier
  optimo
15 // se transforma en la solucion greedy sin perder actividades
  ->
16 // el greedy tambien es optimo.
17
18 vector<ExchangeArgumentDemo::Actividad> acts = {
19     {1,4}, {3,5}, {0,6}, {5,7}, {3,9}, {6,10}
20 };
21
22 cout << ExchangeArgumentDemo::maximoActividades(acts);
23 // 2

```

Nota: el Argumento de Intercambio prueba que existe **al menos una** solución óptima que coincide con el greedy —no que el greedy sea la **única** óptima (puede haber empates). En Activity Selection, por ejemplo, A-D no es la única selección de tamaño 2 (B-D o A-F distinto también podrían funcionar según los datos), pero el argumento garantiza que **ninguna** solución le gana en tamaño.

Regla práctica: antes de confiar en un greedy en competencia, intenta esbozar mentalmente el intercambio en 2-3 líneas: "si el óptimo no hace lo que hace mi greedy en el primer punto de diferencia, ¿puedo cambiarlo sin empeorar?" —si la respuesta es claramente sí (y se puede justificar con una desigualdad simple), el greedy es seguro; si la respuesta requiere "depende de qué pase después" o "a veces sí, a veces no", es una señal fuerte de que el problema en realidad necesita Programación Dinámica (ver siguiente subsección), no greedy.

40.2 Greedy vs. Programación Dinámica

Idea: tanto Greedy como Programación Dinámica resuelven problemas de **optimización** contruidos sobre subproblemas más chicos, y ambos asumen **subestructura óptima** (la solución óptima se construye a partir de soluciones óptimas de subproblemas). La diferencia está en **cuántas** opciones se consideran en cada paso: Greedy toma **una sola** decisión localmente óptima y **nunca la**

reconsidera (no hay marcha atrás, no hay comparación contra alternativas); DP considera **todas** las decisiones posibles en cada paso, resolviendo (y memoizando) el subproblema resultante de **cada** una, y al final elige la que dio el mejor resultado global. Greedy funciona **solo** cuando el problema tiene la **propiedad de elección greedy** (la decisión localmente óptima siempre forma parte de **alguna** solución globalmente óptima, demostrable con Exchange Argument); sin esa propiedad, ninguna cantidad de "parece razonable" sustituye a considerar todas las opciones.

¿Por qué usarlo? Reconocer **cuál de los dos aplica** es la habilidad central de esta comparación —usar Greedy donde hace falta DP da una respuesta **incorrecta** (no solo lenta); usar DP donde bastaba Greedy da una respuesta **correcta pero innecesariamente costosa** en tiempo/memoria. El ejemplo canónico para entender la diferencia es **Coin Change**: con denominaciones **canónicas** (como 1, 5, 10, 25 — el sistema de monedas de EE.UU.), tomar siempre la moneda más grande posible ES óptimo (greedy funciona); con denominaciones **arbitrarias** (ej. 1, 3, 4), ese mismo greedy puede fallar —6 con greedy da 4+1+1 (3 monedas), pero el óptimo real es 3+3 (2 monedas). El greedy **compila, corre, y da una respuesta** —solo que incorrecta, lo cual es más peligroso que un error de compilación porque no se detecta sin comparar contra el resultado real.

Casos de uso (tabla de decisión):

- **Fractional Knapsack** (se puede tomar una **fracción** de cada ítem): Greedy funciona —ordenar por razón valor/peso y tomar avariciosamente siempre es óptimo.
- **0/1 Knapsack** (cada ítem se toma **completo** o no se toma): Greedy **falla** —la fracción no está permitida, así que "el mejor por unidad de peso" puede no caber, y no hay forma de "reconsiderar" sin DP.
- **Activity Selection** (maximizar **cantidad** de actividades no solapadas, sin pesos): Greedy funciona —ordenar por fin creciente es óptimo (ver Exchange Argument).
- **Activity Selection con Pesos** (maximizar la **suma de valores** de actividades elegidas, no solo la cantidad): Greedy **falla** —la actividad que termina primero puede tener un valor bajo, y conviene DP (Weighted Interval Scheduling) para considerar todas las combinaciones.
- **Coin Change con denominaciones canónicas**: Greedy funciona.
- **Coin Change con denominaciones arbitrarias**: Greedy falla, hace falta DP (número mínimo de monedas).

Complejidad: no aplica un único número —el punto de esta subsección es que la elección entre Greedy ($O(n \log n)$ típico, un **sort** más una pasada) y DP ($O(n \cdot W)$ típico en problemas tipo knapsack, exponencialmente más caro que greedy cuando aplica) depende **enteramente** de si el problema tiene la propiedad de elección greedy, no de una fórmula general.

```

1 // Contraste directo: el MISMO problema superficial (Coin
  Change),
2 // resuelto con Greedy (rapido pero solo correcto con
  denominaciones
3 // canónicas) y con DP (siempre correcto, mas costoso).
4
5 struct CoinChangeComparacion {
6     // GREEDY: toma siempre la moneda mas grande posible.
7     // CORRECTO
8     // solo si las denominaciones son "canónicas" (como
9     // 1,5,10,25) --
10    // FALLA silenciosamente con denominaciones arbitrarias.
11    static int monedasGreedy(vector<int> denominaciones, int
12    objetivo) {
13        sort(denominaciones.rbegin(), denominaciones.rend()); //
14        // mayor
15        // a menor
16        int monedas = 0;
17
18        for (int d : denominaciones) {
19            monedas += objetivo / d; // toma tantas como quepan de
20            // esta denominacion, SIN
21            // reconsiderar despues
22            objetivo %= d;
23        }
24
25        // si objetivo != 0 aqui, ni siquiera hay solucion greedy
26        // valida -- otra senal de que este enfoque no es confiable
27        return (objetivo == 0) ? monedas : -1; // O(n log n)
28    }
29
30    // DP: considera TODAS las monedas posibles en cada paso,
31    // memoizando el mínimo real. SIEMPRE correcto, sin importar
32    // las
33    // denominaciones.
34    static int monedasDP(vector<int>& denominaciones, int
35    objetivo) {
36        vector<int> dp(objetivo + 1, INT_MAX);
37        dp[0] = 0;
38
39        for (int monto = 1; monto <= objetivo; monto++) {
40            for (int d : denominaciones) {
41                if (d > monto || dp[monto - d] == INT_MAX) continue;
42                dp[monto] = min(dp[monto], dp[monto - d] + 1);
43                // ^ a diferencia de greedy, aqui SI se compara contra
44                // todas las alternativas antes de decidir
45            }
46        }
47
48        return (dp[objetivo] == INT_MAX) ? -1 : dp[objetivo]; // O(objetivo * n)
49    }
50 };

```

Ejemplo de funcionamiento (donde Greedy falla y DP no):

```

1 vector<int> denominacionesCanonicas = {1, 5, 10, 25};
2 vector<int> denominacionesArbitrarias = {1, 3, 4};
3
4 cout << CoinChangeComparacion::monedasGreedy(
5     denominacionesCanonicas, 30);
6 // 2 (25 + 5 -- correcto, y coincide con el optimo real)
7
8 cout << CoinChangeComparacion::monedasGreedy(
9     denominacionesArbitrarias, 6);
10 // 3 (4 + 1 + 1 -- INCORRECTO, el greedy se equivoca aqui)
11
12 cout << CoinChangeComparacion::monedasDP(
13     denominacionesArbitrarias, 6);
14 // 2 (3 + 3 -- CORRECTO, DP si encuentra el optimo real)

```

Nota: el hecho de que el greedy **parezca** funcionar en ejemplos pequeños o "de prueba" no es evidencia de que sea correcto en general —con {1,3,4} y objetivo 6 ya falla, un caso perfectamente razonable de aparecer en un juez. La única forma confiable de saber si un greedy es seguro es demostrarlo (Exchange Argument) o reconocer el problema como una instancia de un patrón ya probado (Activity Selection, Fractional Knapsack, Huffman, etc.), no "probarlo con ejemplos y ver si da bien".

Regla práctica: si al intentar el Argumento de Intercambio para tu greedy te encuentras diciendo "pero esto depende de qué pase **después**" o "la mejor opción **ahora** podría no ser la mejor a largo plazo", esa es la señal directa de que el problema necesita DP —esa dependen-

cia del futuro es exactamente lo que Greedy no puede capturar (decide y nunca reconsidera), mientras que DP la maneja de forma natural explorando (y memoizando) todas las alternativas.

41 Intervalos

41.1 Activity Selection (Selección de Actividades)

Idea: dado un conjunto de actividades, cada una con un intervalo $[\text{inicio}, \text{fin}]$, se busca el **máximo número** de actividades que se pueden realizar sin que ninguna se solape con otra (una sola persona/recurso, no puede hacer dos a la vez). El greedy correcto —ya justificado con Exchange Argument en Fundamentos de Greedy— es: **ordenar por fin creciente**, y recorrer tomando cada actividad cuyo inicio sea $\geq$ el fin de la última tomada. La intuición: terminar lo antes posible **siempre** deja más espacio libre para actividades futuras que cualquier otra elección —nunca hay razón para preferir una actividad que termina más tarde si una que termina antes también estaba disponible.

¿Por qué usarlo? Es el ejemplo introductorio estándar de greedy correcto porque el criterio "ordenar por fin" es **contraintuitivo** a primera vista —muchos intentan ordenar por **duración** (la más corta primero) o por **inicio** más temprano, y ambos criterios **fallan** en casos concretos (una actividad corta puede empezar tarde y bloquear más actividades después que una larga que termina pronto). Interiorizar por qué "fin creciente" es el único criterio que funciona aquí previene ese error en variantes futuras.

Casos de uso:

- Maximizar el número de reuniones/actividades que se pueden agendar en una sola sala/recurso sin solapamiento.
- Como subrutina de Minimum Number of Platforms/Rooms (siguiente bloque de subsecciones): resolver "cuántos recursos hacen falta" es una generalización de "¿alcanza con uno?".
- Punto de comparación directo con Interval Point Cover (siguiente subsección): problema **relacionado** pero **distinto** —aquí se eligen actividades sin solapar; ahí se eligen puntos que **toquen** todos los intervalos.

Complejidad: $O(n \log n)$, dominado por el **sort** inicial por tiempo de fin; el barrido posterior es $O(n)$.

```
1 struct ActivitySelection {
2     struct Actividad { int inicio, fin; };
3
4     // READ: maximo numero de actividades no solapadas. Ordena
5     // internamente por FIN creciente (criterio validado por
6     // Exchange
7     // Argument, ver Fundamentos de Greedy).
8     static int maximoActividades(vector<Actividad> actividades) {
9         sort(actividades.begin(), actividades.end(),
10             [](Actividad& a, Actividad& b) { return a.fin < b.fin; });
11
12         int contador = 0;
13         int finUltima = INT_MIN;
14
15         for (auto& act : actividades) {
16             if (act.inicio >= finUltima) {
17                 contador++;
18             }
19         }
20     }
21 }
```

```
17         finUltima = act.fin;
18     }
19 }
20
21 return contador;
22 // O(n log n)
23
24 // READ: devuelve las actividades ELEGIDAS explícitamente, no
25 // solo
26 // el conteo (mismo criterio, guardando cada seleccion).
27 static vector<Actividad> seleccionar(vector<Actividad>
28 actividades) {
29     sort(actividades.begin(), actividades.end(),
30         [](Actividad& a, Actividad& b) { return a.fin < b.fin; });
31
32     vector<Actividad> elegidas;
33     int finUltima = INT_MIN;
34
35     for (auto& act : actividades) {
36         if (act.inicio >= finUltima) {
37             elegidas.push_back(act);
38             finUltima = act.fin;
39         }
40     }
41
42     return elegidas;
43 // O(n log n)
44 }
```

Ejemplo de funcionamiento:

```
1 vector<ActivitySelection::Actividad> acts = {
2     {1,4}, {3,5}, {0,6}, {5,7}, {3,9}, {5,9}, {6,10}, {8,11},
3     {8,12}, {2,14}, {12,16}
4 };
5
6 cout << ActivitySelection::maximoActividades(acts);
7 // 4
8
9 auto elegidas = ActivitySelection::seleccionar(acts);
10 for (auto& a : elegidas) cout << "[" << a.inicio << ", " << a.
    fin << " ";
11 // [1,4] [5,7] [8,11] [12,16]
```

Regla práctica: siempre ordena por **fin creciente** en este problema —ordenar por inicio o por duración son los dos errores más comunes y ambos producen contraejemplos reales (no solo teóricos). Si el problema agrega **pesos/valores** a cada actividad (maximizar la suma de valores, no la cantidad), este greedy deja de aplicar directamente —ver Greedy vs. Programación Dinámica.

41.2 Interval Point Cover

Idea: dado un conjunto de intervalos $[\text{inicio}, \text{fin}]$, se busca el **mínimo número de puntos** tales que **cada** intervalo contenga **al menos uno** de esos puntos (el problema inverso, en cierto sentido, a Activity Selection: ahí se elegían intervalos sin solapar; aquí se eligen puntos que **toquen** todos los intervalos). El greedy: **ordenar por fin creciente**, y cada vez que un intervalo **no** está ya cubierto por el último punto colocado, colocar un **nuevo** punto exactamente en el **fin** de ese intervalo —colocarlo ahí (lo más tarde posible dentro de ese intervalo) maximiza la chance de que también cubra intervalos futuros que empiecen antes de ese fin.

¿Por qué usarlo? Comparte el mismo ordenamiento por fin creciente que Activity Selection, pero la **razón** de por qué funciona es distinta: aquí no se trata de "dejar espacio libre" sino de "cubrir la mayor cantidad de intervalos posible con cada punto colocado" —poner el punto en el fin del intervalo más urgente (el que termina primero) es la elección que **no puede fallar**: ese intervalo debe recibir un punto en algún momento, y ponerlo en su fin es la posición que abarca el rango más amplio de intervalos que todavía se solapan con él.

Casos de uso:

- Mínimo número de flechas para reventar todos los globos (*Minimum Number of Arrows to Burst Balloons* —variante directa con otro nombre, cada flecha es un "punto").
- Mínimo número de estaciones de servicio/vigilancia tales que cada segmento de una carretera esté cubierto por al menos una.
- Mínimo número de "checkpoints" temporales tales que cada tarea (con su ventana de tiempo activa) sea verificada al menos una vez.
- Como subrutina de problemas de cobertura más generales donde la restricción se puede modelar como intervalos y puntos.

Complejidad: $O(n \log n)$, dominado por el sort inicial por fin creciente; el barrido posterior es $O(n)$.

```

1 struct IntervalPointCover {
2     struct Intervalo { long long inicio, fin; };
3
4     // READ: minimo numero de puntos que tocan TODOS los
5     // intervalos.
6     static int minimoPuntos(vector<Intervalo> intervalos) {
7         sort(intervalos.begin(), intervalos.end(),
8             [](Intervalo& a, Intervalo& b) { return a.fin < b.fin; });
9
10        int puntos = 0;
11        long long ultimoPunto = LLONG_MIN;
12
13        for (auto& iv : intervalos) {
14            if (iv.inicio > ultimoPunto) {
15                // el ultimo punto colocado NO cubre este intervalo
16                // -> hace falta uno nuevo, colocado en el FIN de
17                // este intervalo (maximiza cobertura futura)
18                puntos++;
19                ultimoPunto = iv.fin;
20            }
21            // si iv.inicio <= ultimoPunto, este intervalo YA esta
22            // cubierto por el punto anterior -- no hace falta nada
23        }
24        return puntos;
25    } // O(n log n)
26
27    // READ: devuelve los puntos elegidos explicitamente.
28    static vector<long long> obtenerPuntos(vector<Intervalo>
29    intervalos) {
30        sort(intervalos.begin(), intervalos.end(),
31            [](Intervalo& a, Intervalo& b) { return a.fin < b.fin; });
32
33        vector<long long> puntos;
34        long long ultimoPunto = LLONG_MIN;
35
36        for (auto& iv : intervalos) {
37            if (iv.inicio > ultimoPunto) {
38                ultimoPunto = iv.fin;
39                puntos.push_back(ultimoPunto);
40            }
41        }
42        return puntos;
43    } // O(n log n)
44 };

```

Ejemplo de funcionamiento:

```

1 vector<IntervalPointCover::Intervalo> intervalos = {
2     {1,4}, {2,5}, {6,8}, {3,7}, {9,10}
3 };
4
5 cout << IntervalPointCover::minimoPuntos(intervalos);
6 // 2
7
8 auto puntos = IntervalPointCover::obtenerPuntos(intervalos);
9 for (long long p : puntos) cout << p << " ";
10 // 4 10
11 // punto en 4 cubre [1,4], [2,5], [3,7]; punto en 10 cubre
12 // [6,8], [9,10]

```

Regla práctica: tanto Activity Selection como Interval Point Cover ordenan por **fin creciente**, pero la decisión que toman es opuesta en naturaleza: Activity Selection **descarta** intervalos que se solapan con lo ya elegido; Interval Point Cover **agrega** un punto solo cuando el intervalo actual **no** está cubierto todavía. Si el problema pide "elige el máximo de cosas sin solapar", es Activity Selection; si pide "cubre todo con el mínimo de recursos puntuales", es Interval Point Cover.

41.3 Merge Intervals

Idea: dado un conjunto de intervalos [inicio, fin] que pueden **solaparse** entre sí, se busca **fusionarlos** en el menor número de intervalos **disjuntos** que cubran exactamente la misma unión de puntos. El greedy: **ordenar por inicio creciente** (no por fin, a diferencia de Activity Selection e Interval Point Cover — aquí importa el orden en que los intervalos "aparecen" para poder ir extendiéndolos), y recorrer manteniendo un intervalo "en construcción" —si el siguiente intervalo **se solapa** con el actual ($\text{inicio} \leq \text{fin actual}$), se **extiende** el actual tomando el máximo de los dos fines; si no se solapa, el intervalo actual queda **cerrado** y se emite, empezando uno nuevo.

¿Por qué usarlo? Es el primer greedy de esta sección que ordena por **inicio** en vez de por fin —vale la pena notar el contraste explícitamente: aquí no se está eligiendo intervalos ni cubriendo con puntos, se está **reconstruyendo** la unión total de todos ellos, y para eso hace falta procesarlos en el orden en que empiezan, extendiendo progresivamente lo que ya se lleva fusionado.

Casos de uso:

- Simplificar una lista de rangos horarios ocupados en una sola lista de bloques **sin** solapamientos, antes de aplicar otro algoritmo sobre ella (ej. antes de Minimum Number of Platforms/Rooms, si primero hace falta limpiar datos redundantes).
- Insertar un nuevo intervalo en una lista ya fusionada, re-fusionando solo la parte afectada.
- Calcular la **longitud total cubierta** por una unión de intervalos con solapamientos (sumar la longitud de cada intervalo fusionado, en vez de sumar cada intervalo original y arriesgarse a contar dos veces las partes solapadas).
- Como paso de limpieza previo en problemas de Line Sweep donde los eventos vienen como intervalos crudos, potencialmente redundantes.

Complejidad: $O(n \log n)$, dominado por el sort inicial por inicio creciente; el barrido de fusión es $O(n)$.

```

1 struct MergeIntervals {
2     struct Intervalo { long long inicio, fin; };
3
4     // READ: fusiona todos los intervalos solapados en la lista
5     // MINIMA
6     // de intervalos disjuntos que cubre la misma union de puntos.
7
8     static vector<Intervalo> fusionar(vector<Intervalo>
9     intervalos) {
10        if (intervalos.empty()) return {};
11
12        sort(intervalos.begin(), intervalos.end(),
13            [](Intervalo& a, Intervalo& b) { return a.inicio < b.inicio
14            ; });
15    }
16 };

```

```

11 vector<Intervalo> resultado;
12 Intervalo actual = intervalos[0];
13
14 for (int i = 1; i < (int)intervalos.size(); i++) {
15     auto& siguiente = intervalos[i];
16
17     if (siguiente.inicio <= actual.fin) {
18         // se solapa (o es adyacente) -> extiende el actual
19         actual.fin = max(actual.fin, siguiente.fin);
20     } else {
21         // no se solapa -> cierra el actual y empieza uno nuevo
22         resultado.push_back(actual);
23         actual = siguiente;
24     }
25 }
26
27 resultado.push_back(actual); // el ultimo intervalo en
28 // construccion siempre
29 // queda pendiente de emitir
30 return resultado;
31                                     // O(n log n)
32
33 // READ: longitud total cubierta por la union de todos los
34 // intervalos (sin contar dos veces las partes solapadas) --
35 // reutiliza fusionar() y suma las longitudes ya disjuntas.
36 static long long longitudTotalCubierta(vector<Intervalo>
37 intervalos) {
38     auto fusionados = fusionar(intervalos);
39     long long total = 0;
40
41     for (auto& iv : fusionados)
42         total += iv.fin - iv.inicio;
43
44     return total;
45                                     // O(n log n)
46 };

```

Ejemplo de funcionamiento:

```

1 vector<MergeIntervals::Intervalo> intervalos = {
2     {1,3}, {2,6}, {8,10}, {15,18}, {9,12}
3 };
4
5 auto fusionados = MergeIntervals::fusionar(intervalos);
6 for (auto& iv : fusionados) cout << "[" << iv.inicio << ", " <<
7     iv.fin << "] ";
8 // [1,6] [8,12] [15,18]
9
10 cout << MergeIntervals::longitudTotalCubierta(intervalos);
11 // 5 + 4 + 3 = 12

```

Nota: la condición `siguiente.inicio <= actual.fin` (con `<=`, no `<`) decide si intervalos **apenas adyacentes** (ej. `[1, 3]` y `[3, 5]`) se fusionan o no —con `<=` se fusionan en `[1, 5]` (tratando los extremos como **cerrados**, el punto 3 pertenece a ambos); si el problema usa intervalos **semiabiertos** `[inicio, fin)`, la condición correcta para fusionar solo solapamientos reales es `<` estricto —revisar siempre qué convención de extremos usa el enunciado antes de fijar este operador.

Regla práctica: Merge Intervals ordena por **inicio** porque el objetivo es reconstruir la unión completa en el orden en que los intervalos "aparecen"; Activity Selection e Interval Point Cover ordenan por **fin** porque el objetivo es decidir, cuanto antes, qué se puede "liberar" o "cubrir". Si el problema pide "combina/limpia los intervalos" (sin elegir ni cubrir, solo reconstruir), es Merge Intervals.

41.4 Minimum Number of Platforms/Rooms

Idea: dado un conjunto de intervalos (llegadas y salidas de trenes, o reuniones con horario), se busca el **mínimo número de recursos simultáneos** (andenes, salas) necesarios para que **nunca** haya dos intervalos activos al mismo tiempo compitiendo por el mismo recurso —a diferencia de Activity Selection (que usa **un solo** recurso y descarta lo que no cabe), aquí se permite usar **tantos recursos como haga falta**, y la pregunta es **cuántos**. Se resuelve con **Line Sweep**: se separan todos los

inicio y todos los **fin** en dos listas, ambas ordenadas; se recorren **simultáneamente** con Two Pointers, y cada vez que un **inicio** ocurre **antes** que el siguiente **fin** pendiente, se necesita **un recurso más** (contador sube); cada vez que un **fin** ocurre primero, un recurso se **libera** (contador baja) — el **máximo** valor que alcanza el contador durante el barrido es la respuesta.

¿Por qué usarlo? Es la generalización natural de "¿alcanza con un solo recurso?" (Activity Selection) a "¿cuántos recursos hacen falta como máximo?" —el mismo espíritu de barrer por tiempo, pero en vez de descartar actividades que no caben, se cuenta cuántas están **simultáneamente activas** en el peor momento. Es también el ejemplo introductorio de **Line Sweep con eventos de tipo "+1"/"-1"**, un patrón que reaparece en problemas mucho más generales de conteo de solapamientos.

Casos de uso:

- Mínimo número de andenes de tren, salas de reunión, o cajeros que hacen falta para atender todas las llegadas/salidas sin conflicto.
- Máximo número de eventos/personas activas **al mismo tiempo** en cualquier instante (el valor máximo del contador *es* esa respuesta, no solo cuántos recursos hacen falta).
- Como subrutina de problemas de **asignación** de recursos: una vez que se sabe cuántos recursos hacen falta como máximo, se puede simular la asignación explícita con una `priority_queue` de "recurso libre más próximo".
- Detectar si existe algún momento en que **más de k** intervalos están activos a la vez (comparando el máximo del contador contra k).

Complejidad: $O(n \log n)$, dominado por ordenar las dos listas de inicios y fines por separado; el barrido con Two Pointers es $O(n)$.

```

1 struct MinimoRecursos {
2     struct Intervalo { int inicio, fin; };
3
4     // READ: mínimo numero de recursos simultaneos necesarios
5     // para
6     // que ningun par de intervalos activos compita por el mismo
7     // recurso. Asume intervalos semiabiertos [inicio, fin) -- un
8     // fin exactamente igual a un inicio NO cuenta como solape (
9     // el
10    // recurso ya se libero justo cuando el otro arranca).
11    static int minimoRecursos(vector<Intervalo>& intervalos) {
12        int n = intervalos.size();
13        vector<int> inicios(n), fines(n);
14
15        for (int i = 0; i < n; i++) {
16            inicios[i] = intervalos[i].inicio;
17            fines[i] = intervalos[i].fin;
18        }
19
20        sort(inicios.begin(), inicios.end());
21        sort(fines.begin(), fines.end());
22
23        int i = 0, j = 0;
24        int activos = 0, maximoActivos = 0;
25
26        while (i < n) {
27            if (inicios[i] < fines[j]) {
28                // un nuevo intervalo empieza ANTES de que el
29                // siguiente pendiente termine -> hace falta un
30                // recurso mas
31                activos++;
32                maximoActivos = max(maximoActivos, activos);
33                i++;
34            } else {
35                // un intervalo termina cuando otro empieza
36                // no hace falta un recurso mas
37                j++;
38            }
39        }
40        return maximoActivos;
41    }
42 };

```

```

33 // un intervalo termina (o empata, tratado como
34 // liberacion primero) -> se libera un recurso
35 activos--;
36 j++;
37 }
38 }
39
40 return maximoActivos;
41 } // O(n log n)
42 };

```

Ejemplo de funcionamiento:

```

1 // llegadas y salidas de trenes (mismo índice = mismo tren)
2 vector<MinimoRecursos::Intervalo> trenes = {
3     {900, 910}, {940, 1200}, {950, 1120},
4     {1100, 1130}, {1500, 1900}, {1800, 2000}
5 };
6
7 cout << MinimoRecursos::minimoRecursos(trenes);
8 // 3
9 // en el momento 950-1100 hay 3 trenes activos a la vez:
10 // [940,1200], [950,1120], y arranca [1100,1130] justo antes de
11 // que
12 // termine ninguno de los otros dos

```

Nota: la condición $\text{inicios}[i] < \text{fines}[j]$ (estricta) asume que un fin exactamente igual a un inicio **no** cuenta como conflicto (el recurso se libera justo a tiempo). Si el problema considera que un intervalo que **termina** en el momento t **sí** sigue ocupando el recurso hasta después de t (extremos cerrados en ambos lados), la condición correcta es $\text{inicios}[i] \leq \text{fines}[j]$ — revisar siempre esta convención contra el enunciado, es la fuente de error de borde más común en este problema.

Regla práctica: si el problema permite **un solo** recurso y pide maximizar cuántas actividades caben, es Activity Selection; si permite **recursos ilimitados** y pide el **mínimo** necesario para no tener conflictos, es Minimum Number of Platforms/Rooms. Ambos comparten la idea de ordenar por tiempo y barrer, pero uno **descarta** lo que no cabe y el otro **cuenta** el pico de concurrencia.

41.5 Non-overlapping Intervals

Idea: dado un conjunto de intervalos que pueden solaparse, se busca el **mínimo número de intervalos a eliminar** para que los que **queden** no se solapen entre sí. Es el problema **complementario** de Activity Selection: si Activity Selection encuentra el **máximo** número de actividades que **sí** se pueden mantener sin solapar, entonces el mínimo a eliminar es simplemente $n - \text{maximoActividades}$ —no hace falta ningún algoritmo nuevo, es **el mismo greedy** (ordenar por fin creciente, tomar cada intervalo que no solape con el último tomado) visto desde el ángulo opuesto.

¿Por qué usarlo? Es la trampa más común de esta sección: muchos intentan resolverlo como un problema **distinto** (a veces con DP o con otro criterio de ordenamiento), sin notar que "máximo que se puede **mantener**" y "mínimo que hay que **eliminar**" son la **misma** cantidad, solo restada de n . Reconocer esta equivalencia ahorra tener que re-derivar el Exchange Argument desde cero —ya está probado en Activity Selection.

Casos de uso:

- Mínimo número de reuniones/eventos a cancelar para que el resto quepa sin conflictos en un solo recurso.

- Como verificación rápida: si un juez da directamente "número de actividades a remover" en vez de "máximo a mantener", no hace falta cambiar el algoritmo, solo restar de n al final.
- Variante para practicar reconocer **transformaciones triviales** entre enunciados que suenan distintos pero son matemáticamente idénticos —una habilidad tan importante como conocer los algoritmos en sí.

Complejidad: $O(n \log n)$ — idéntica a Activity Selection, ya que es literalmente el mismo algoritmo con una resta final.

```

1 struct NonOverlappingIntervals {
2     struct Intervalo { int inicio, fin; };
3
4     // READ: mínimo numero de intervalos a ELIMINAR para que el
5     // resto
6     // no se solape. Reutiliza EXACTAMENTE el criterio de
7     // Activity
8     // Selection (ordenar por fin, contar cuantos SI caben) y
9     // resta
10    // de n -- no es un algoritmo nuevo, es el mismo visto al
11    // revés.
12    static int minimoAEliminar(vector<Intervalo> intervalos) {
13        int n = intervalos.size();
14        if (n == 0) return 0;
15
16        sort(intervalos.begin(), intervalos.end(),
17            [](Intervalo& a, Intervalo& b) { return a.fin < b.fin; });
18
19        int mantenidos = 0;
20        int finUltimo = INT_MIN;
21
22        for (auto& iv : intervalos) {
23            if (iv.inicio >= finUltimo) { // no se solapa con el
24                // ultimo MANTENIDO
25                mantenidos++;
26                finUltimo = iv.fin;
27            }
28            // si se solapa, se "elimina" implícitamente -- no se
29            // suma a mantenidos, no se actualiza finUltimo
30        }
31
32        return n - mantenidos;
33    } // O(n log n)
34 };

```

Ejemplo de funcionamiento:

```

1 vector<NonOverlappingIntervals::Intervalo> intervalos = {
2     {1,2}, {2,3}, {3,4}, {1,3}
3 };
4
5 cout << NonOverlappingIntervals::minimoAEliminar(intervalos);
6 // 1
7 // se mantienen [1,2],[2,3],[3,4] (3 intervalos, no se solapan
8 // entre
9 // sí con extremos semiabiertos); se elimina [1,3], que
10 // solapaba con
11 // [1,2] y con [2,3]

```

Regla práctica: cuando un enunciado pida "elimina el mínimo para que no queden solapes", NO busques un algoritmo nuevo —resuelve Activity Selection normal (máximo que se puede mantener) y resta de n . Esta equivalencia vale la pena memorizarla explícitamente porque el enunciado rara vez menciona la palabra "selección", lo cual puede hacer que el problema parezca distinto a primera vista.

41.6 Minimum Number of Arrows to Burst Balloons

Idea: dado un conjunto de globos representados como intervalos $[\text{inicio}, \text{fin}]$ en el eje horizontal, una flecha disparada verticalmente en la posición x revienta **todos**

los globos cuyo intervalo contiene a x . Se busca el **mínimo número de flechas** para reventar todos los globos —esto es **exactamente** Interval Point Cover con otro nombre: cada "punto" (posición de flecha) debe tocar todos los intervalos (globos), y el greedy es idéntico: ordenar por **fin** creciente, disparar una flecha en el **fin** del primer globo no cubierto todavía.

¿Por qué usarlo? Es el ejemplo más citado de Interval Point Cover en jueces de práctica, y vale la pena tenerlo como **subsección propia** precisamente para reforzar el reconocimiento de patrones: el enunciado suena a geometría/física ("flechas", "globos"), pero la estructura matemática subyacente es **idéntica** a la de "mínimo número de puntos que cubren intervalos" —ningún concepto nuevo, solo una capa distinta de narrativa sobre el mismo problema.

Casos de uso:

- El problema de globos y flechas tal cual aparece en jueces de práctica (frecuentemente bajo ese nombre exacto).
- Cualquier variante narrativa equivalente: "mínimo número de inspecciones puntuales para cubrir todos los rangos de validez", "mínimo número de disparos de láser", etc. — mismo esqueleto, distinta historia.
- Como ejercicio de **traducción de enunciado a estructura conocida**: identificar que "un disparo reventará todos los globos que lo contienen" es literalmente "un punto cubre todos los intervalos que lo contienen" es el paso clave, no el algoritmo en sí (que ya está resuelto en Interval Point Cover).

Complejidad: $O(n \log n)$ — idéntica a Interval Point Cover, ya que es el mismo algoritmo.

```

1 struct ArrowsBurstBalloons {
2     struct Globo { long long inicio, fin; }; // [inicio, fin]
3     // globo en el eje X
4
5     // READ: minimo numero de flechas para reventar TODOS los
6     // globos.
7     // Idéntico a IntervalPointCover::minimoPuntos -- se
8     // reescribe
9     // aquí con los nombres del enunciado para dejar explicita la
10    // correspondencia.
11    static int minimoFlechas(vector<Globo> globos) {
12        if (globos.empty()) return 0;
13
14        sort(globos.begin(), globos.end(),
15              [](Globo& a, Globo& b) { return a.fin < b.fin; });
16
17        int flechas = 1;
18        long long posicionFlecha = globos[0].fin;
19
20        for (int i = 1; i < (int)globos.size(); i++) {
21            if (globos[i].inicio > posicionFlecha) {
22                // este globo ya NO es alcanzado por la ultima
23                // flecha -> hace falta una nueva, en el FIN de este
24                // globo (maximiza cuantos globos futuros reventará)
25                flechas++;
26                posicionFlecha = globos[i].fin;
27            }
28            // si globos[i].inicio <= posicionFlecha, este globo YA
29            // está reventado por la flecha actual -- no hace falta
30            nada
31        }
32        return flechas; // O(n log n)
33    };

```

Ejemplo de funcionamiento:

```

1 vector<ArrowsBurstBalloons::Globo> globos = {

```

```

2     {10,16}, {2,8}, {1,6}, {7,12}
3 };
4
5 cout << ArrowsBurstBalloons::minimoFlechas(globos);
6 // 2
7 // ordenados por fin: [1,6],[2,8],[7,12],[10,16]
8 // flecha en 6 reventará [1,6] y [2,8] (2 <= 6)
9 // [7,12] no cubierto (7 > 6) -> flecha en 12, reventará [7,12]
10 // [10,16] (10 <= 12)

```

Regla práctica: antes de programar cualquier "problema nuevo" de esta sección, pregúntate primero si ya es **uno de los patrones anteriores** disfrazado con otro nombre —Non-overlapping Intervals es Activity Selection invertido; Minimum Number of Arrows es Interval Point Cover con otra narrativa. La mayoría de problemas greedy "nuevos" en jueces de práctica son variaciones nominales de un puñado de patrones estructurales ya vistos, no algoritmos genuinamente distintos.

42 Scheduling

42.1 Job Sequencing con Deadlines

Idea: dado un conjunto de trabajos, cada uno con una **ganancia** y un **deadline** (debe completarse antes o en ese momento, y cada trabajo toma exactamente 1 unidad de tiempo en una única máquina), se busca el subconjunto de trabajos que se pueden programar **sin** pasarse de su deadline, **maximizando** la ganancia total. El greedy: **ordenar por ganancia decreciente**, y para cada trabajo (en ese orden) intentar colocarlo en el **slot de tiempo disponible más tardío** que sea $\leq$ su deadline —si existe un slot libre, se ocupa; si no, el trabajo se descarta. Colocar lo **más tarde posible** (en vez de lo más temprano) es la clave: deja los slots **tempranos** libres para trabajos futuros (de menor ganancia, ya que se procesan en orden decreciente) que tengan deadlines **más ajustados**.

¿Por qué usarlo? Es el greedy clásico de "ordena por el criterio que **no se puede recuperar después**" (la ganancia, ya que una vez descartado un trabajo de alta ganancia no hay forma de reconsiderarlo) combinado con una asignación que preserva **flexibilidad futura** (slot más tardío posible). La implementación ingenua de "buscar el slot libre más tardío" es $O(n \cdot \max \text{Deadline})$ por escaneo lineal; con **DSU** (Union-Find) se acelera a casi $O(n \log n)$, aprovechando que "encontrar el slot libre más tardío $\leq d$ " es exactamente lo que resuelve un DSU donde cada slot ocupado "apunta" al slot libre anterior más cercano — el mismo patrón de "encontrar el siguiente disponible" que aparece en problemas de asignación con Union-Find.

Casos de uso:

- Programar trabajos con deadline y ganancia en una única máquina, maximizando ganancia total (el problema base).
- Cualquier problema de "elige un subconjunto sujeto a restricciones de tiempo límite individuales, maximizando una métrica agregada" — el criterio de "ordenar por lo irreversible, asignar preservando flexibilidad" se generaliza a variantes con duración de trabajo distinta de 1 (aunque ahí ya no es tan directo sin modificar la estructura de asignación).

- Como aplicación práctica de DSU fuera del contexto habitual de conectividad — usar la estructura para "siguiente slot libre" en vez de "misma componente".

Complejidad: $O(n \log n)$ para ordenar por ganancia, más $O(n \cdot \alpha(n))$ para las asignaciones con DSU (prácticamente lineal) — la versión sin DSU (escaneo lineal por trabajo) es $O(n \cdot \max \text{Deadline})$, aceptable si los deadlines son pequeños.

```

1 struct JobSequencing {
2     struct Trabajo { int deadline, ganancia; };
3
4     // READ: maxima ganancia total programando trabajos sin
5     // pasarse de su deadline, un trabajo por unidad de tiempo. Version
6     // con ESCANEO LINEAL de slots (simple,  $O(n * \max \text{Deadline})$ ).
7     static long long maximaGananciaSimple(vector<Trabajo>
8     trabajos) {
9         sort(trabajos.begin(), trabajos.end(),
10             [](Trabajo& a, Trabajo& b) { return a.ganancia > b.ganancia;
11             });
12
13         int maxDeadline = 0;
14         for (auto& t : trabajos)
15             maxDeadline = max(maxDeadline, t.deadline);
16
17         vector<bool> ocupado(maxDeadline + 1, false);
18         long long gananciaTotal = 0;
19
20         for (auto& t : trabajos) {
21             // busca el slot libre MAS TARDIO <= deadline de este
22             // trabajo
23             for (int slot = min(t.deadline, maxDeadline); slot >= 1;
24                 slot--) {
25                 if (!ocupado[slot]) {
26                     ocupado[slot] = true;
27                     gananciaTotal += t.ganancia;
28                     break;
29                 }
30             }
31         }
32
33         return gananciaTotal;
34     } //  $O(n * \max \text{Deadline})$ 
35
36     // READ: misma logica, pero usando DSU para encontrar el slot
37     // libre mas tardio en  $O(\alpha(n))$  amortizado en vez de
38     // escanear linealmente cada vez.
39     static long long maximaGananciaDSU(vector<Trabajo> trabajos)
40     {
41         sort(trabajos.begin(), trabajos.end(),
42             [](Trabajo& a, Trabajo& b) { return a.ganancia > b.ganancia;
43             });
44
45         int maxDeadline = 0;
46         for (auto& t : trabajos)
47             maxDeadline = max(maxDeadline, t.deadline);
48
49         // padre[slot] = el slot LIBRE mas tardio <= slot (o 0 si
50         // ninguno esta libre); find(x) sigue la cadena de slots
51         // ocupados hasta el proximo libre
52         vector<int> padre(maxDeadline + 1);
53         iota(padre.begin(), padre.end(), 0);
54
55         function<int(int)> find = [&](int x) {
56             if (padre[x] != x) padre[x] = find(padre[x]);
57             return padre[x];
58         };
59
60         long long gananciaTotal = 0;
61
62         for (auto& t : trabajos) {
63             int slotLibre = find(min(t.deadline, maxDeadline));
64
65             if (slotLibre > 0) {
66                 gananciaTotal += t.ganancia;
67                 padre[slotLibre] = slotLibre - 1; // marca este
68                 // slot como
69                 // ocupado,
70                 // "fusionandolo"
71                 // con el anterior
72             }
73
74             // si slotLibre == 0, no hay ningun slot disponible para
75             // este trabajo -> se descarta
76         }
77     }
78 }

```

```

71     return gananciaTotal;
72 } //  $O(n \log n + n * \alpha(n))$ 
73 };

```

Ejemplo de funcionamiento:

```

1 vector<JobSequencing::Trabajo> trabajos = {
2     {4, 20}, {1, 10}, {1, 40}, {1, 30}
3 };
4 // (deadline, ganancia)
5
6 cout << JobSequencing::maximaGananciaSimple(trabajos);
7 // 60
8 // orden por ganancia: (1,40) (1,30) (4,20) (1,10)
9 // (1,40) -> slot 1 ocupado
10 // (1,30) -> deadline 1, slot 1 ya ocupado, descartado
11 // (4,20) -> slot 4 (el mas tardio <= 4 disponible)
12 // (1,10) -> deadline 1, ocupado, descartado
13 // total: 40 + 20 = 60
14
15 cout << JobSequencing::maximaGananciaDSU(trabajos);
16 // 60 (mismo resultado, otro camino)

```

Regla práctica: usa la versión con escaneo lineal cuando los deadlines son chicos ($\lesssim 10^3$ – 10^4) por simplicidad de código; cambia a la versión con DSU en cuanto los deadlines sean grandes (10^5 – 10^6) y el escaneo lineal arriesgue exceder el límite de tiempo — el criterio greedy (ordenar por ganancia, asignar al slot más tardío posible) es **idéntico** en ambas, solo cambia cómo se **busca** ese slot.

42.2 Task Scheduler con Cooldown

Idea: dado un conjunto de tareas (cada una identificada por un tipo/letra) y un tiempo de **enfriamiento** n (dos ejecuciones de la **misma** tarea deben estar separadas por al menos n unidades de tiempo, pudiendo dejar "huecos" ociosos si no hay otra tarea disponible para llenarlos), se busca el **tiempo total mínimo** para ejecutar todas las tareas. El greedy: en cada unidad de tiempo, ejecutar **la tarea disponible con mayor frecuencia restante** (la que "más urge" adelantar, porque tiene más repeticiones pendientes y por lo tanto más restricciones de enfriamiento futuras) — se implementa con una **cola de prioridad** (max-heap por frecuencia) más una cola de espera para las tareas actualmente "enfriándose". Existe además una **fórmula cerrada** equivalente, sin simular: si $f_{\max}$ es la frecuencia máxima entre todas las tareas y k es cuántas tareas **distintas** alcanzan esa frecuencia máxima, la respuesta es $\max(\text{totalTareas}, (f_{\max} - 1)(n + 1) + k)$.

¿Por qué usarlo? Es un ejemplo de greedy con **dos** soluciones válidas de complejidad muy distinta: la simulación con heap es $O(\text{tiempoTotal} \cdot \log(\text{tiposDistintos}))$ y funciona sin importar la distribución de frecuencias; la fórmula cerrada es $O(m)$ (solo contar frecuencias, m = número de tareas) y explota la estructura específica del problema —una vez que se entiende **por qué** la tarea más frecuente determina un "esqueleto" mínimo de tiempo (bloques de tamaño $n + 1$ alrededor de cada repetición de la tarea más frecuente, salvo la última), el resto de tareas simplemente se **rellenan** en los huecos sin afectar el total — a menos que haya tantas tareas restantes que no quepan en los huecos, caso cubierto por el **max** contra **totalTareas**.

Casos de uso:

- Programar un procesador de tareas con enfriamiento obligatorio entre repeticiones del mismo tipo (el

problema base, frecuente en jueces bajo el nombre "Task Scheduler").

- Cualquier problema de programación de trabajos con la misma restricción estructural: "no repetir el mismo tipo dentro de una ventana de n unidades", sin importar el dominio narrativo (CPU scheduling, rotación de turnos, etc.).
- Como ejemplo de cuándo vale la pena **derivar una fórmula** en vez de simular: la simulación explícita con heap sigue siendo la opción más segura bajo presión de tiempo (menos propensa a errores de fórmula), pero para tiempos totales muy grandes la fórmula evita simular literalmente cada unidad de tiempo.

Complejidad: $O(m \log k)$ con la simulación por heap (m = número total de tareas, k = tipos distintos); $O(m)$ con la fórmula cerrada (solo requiere contar frecuencias).

```

1 struct TaskScheduler {
2     // READ: tiempo total minimo simulando con un max-heap de
3     // frecuencias y una cola de "enfriamiento". Funciona sin
4     // nada sobre la distribucion de frecuencias.
5     static long long tiempoMinimoSimulado(vector<char>& tareas,
6     int n) {
7         unordered_map<char,int> frecuencia;
8         for (char t : tareas) frecuencia[t]++;
9
10        priority_queue<int> heap; // max-heap de frecuencias
11        // restantes
12        for (auto& [tarea, freq] : frecuencia)
13            heap.push(freq);
14
15        long long tiempo = 0;
16        queue<pair<int,long long>> enfriando; // {frecuencia
17        // restante,
18        // momento en que
19        // vuelve a estar
20        // disponible}
21
22        while (!heap.empty() || !enfriando.empty()) {
23            tiempo++;
24
25            if (!heap.empty()) {
26                int freq = heap.top(); heap.pop();
27                freq--;
28                if (freq > 0)
29                    enfriando.push({freq, tiempo + n});
30                // si freq llega a 0, esta tarea ya termino
31                // completamente, no vuelve a la cola
32            }
33            // si el heap esta vacio pero hay tareas enfriandose,
34            // este ciclo es una unidad de tiempo OCIOSA (no hay
35            // nada disponible para ejecutar)
36
37            if (!enfriando.empty() && enfriando.front().second ==
38            tiempo)
39            {
40                heap.push(enfriando.front().first);
41                enfriando.pop();
42            }
43
44            return tiempo;
45        }
46        // READ: tiempo total minimo con la FORMULA CERRADA (sin
47        // simular).
48        // f_max = frecuencia maxima; k = cuantas tareas distintas
49        // alcanzan esa frecuencia maxima.
50        static long long tiempoMinimoFormula(vector<char>& tareas,
51        int n) {
52            unordered_map<char,int> frecuencia;
53            for (char t : tareas) frecuencia[t]++;
54
55            int fMax = 0;
56            for (auto& [tarea, freq] : frecuencia)
57                fMax = max(fMax, freq);
58
59            int k = 0;
60            for (auto& [tarea, freq] : frecuencia)
61                if (freq == fMax) k++;
62        }
63    }
64 }

```

```

59 long long esqueleto = (long long)(fMax - 1) * (n + 1) + k;
60 // ~ (fMax - 1) bloques de tamaño (n+1) alrededor de cada
61 // repeticion de la tarea mas frecuente, salvo la ultima;
62 // +k por las k tareas que ocupan la posicion final de
63 // cada "columna" de frecuencia maxima
64
65 return max((long long)tareas.size(), esqueleto);
66 // si hay suficientes OTRAS tareas para llenar todos los
67 // huecos del esqueleto y sobrar, la respuesta es
68 // simplemente
69 // el numero total de tareas (sin tiempo ocioso)
70 // O(m)
71 }

```

Ejemplo de funcionamiento:

```

1 vector<char> tareas = {'A','A','A','B','B','B'};
2 int n = 2;
3
4 cout << TaskScheduler::tiempoMinimoSimulado(tareas, n);
5 // 8 (A B A B A B, con 2 huecos ociosos)
6
7 cout << TaskScheduler::tiempoMinimoFormula(tareas, n);
8 // 8 (fMax=3, k=2 -> (3-1)*(2+1)+2 = 6+2 = 8; total tareas =
9 // 6,
10 // max(6,8) = 8)
11
12 vector<char> tareas2 = {'A','A','A','B','B','B','C','C','C','D',
13 // 'D','E'};
14 cout << TaskScheduler::tiempoMinimoFormula(tareas2, 2);
15 // 12 (suficientes tareas distintas para llenar todos los
16 // huecos
17 // sin tiempo ocioso -> la respuesta es simplemente el
18 // total
19 // de tareas, 12)

```

Regla práctica: usa la simulación con heap por defecto —es más robusta ante errores de fórmula y funciona sin tener que re-derivar nada bajo presión. Cambia a la fórmula cerrada solo cuando el tiempo total esperado es demasiado grande para simular explícitamente unidad por unidad (n o el número de tareas del orden de 10^9), y ten cuidado con el `max` contra `totalTareas` —omitirlo es el error más común al implementar la fórmula, ya que sin él se puede subestimar el tiempo cuando la tarea más frecuente no domina la distribución.

42.3 Fractional Knapsack

Idea: dado un conjunto de ítems, cada uno con un **peso** y un **valor**, y una mochila de capacidad W , se busca maximizar el valor total llevado, permitiendo tomar **fracciones** de un ítem (a diferencia del 0/1 Knapsack, donde cada ítem se toma completo o no se toma). El greedy: **ordenar por razón valor/peso decreciente**, y tomar cada ítem **completo** mientras quepa en la capacidad restante; en cuanto un ítem ya no cabe entero, tomar **la fracción exacta** que llena la capacidad restante y detenerse ahí —esa fracción siempre maximiza el valor por unidad de espacio que queda, así que nunca conviene "guardar" espacio para un ítem de peor razón.

¿Por qué usarlo? Es el contraste directo y más citado de Greedy vs. Programación Dinámica: la **única** diferencia con 0/1 Knapsack es si se permite tomar fracciones o no, y esa diferencia **decide** si greedy es válido o no. Con fracciones permitidas, el Argumento de Intercambio es directo: si una solución óptima no prioriza la mejor razón valor/peso, se puede "mover" una fracción infinitesimal de espacio desde un ítem de peor razón hacia uno de mejor razón sin perder capacidad total, y esa operación **nunca empeora** el valor —por eso ordenar por razón y llenar avariciosamente siempre es óptimo. Sin fracciones (0/1), esa operación de "mover una fracción" ya no es posible, y el argumento se rompe por completo —de ahí que 0/1 Knapsack necesite DP.

Casos de uso:

- Maximizar valor transportado cuando los ítems son **divisibles** (líquidos, granos, cualquier recurso que se pueda fraccionar sin perder valor proporcional).
- Como introducción canónica al contraste Greedy/DP: el mismo enunciado, con y sin fracciones permitidas, sirve para ilustrar por qué la propiedad de elección greedy depende de detalles aparentemente menores del problema.
- Problemas de asignación de recursos continuos con "eficiencia" (valor por unidad) bien definida —cualquier variante donde "toma lo más eficiente primero, fracciona lo último" tenga sentido físico.

Complejidad: $O(n \log n)$, dominado por el `sort` por razón valor/peso; el llenado greedy posterior es $O(n)$.

```

1 struct FractionalKnapsack {
2     struct Item { double peso, valor; };
3
4     // READ: valor maximo transportable en una mochila de
5     // capacidad W,
6     // permitiendo tomar fracciones de cada item.
7     static double valorMaximo(vector<Item> items, double W) {
8         sort(items.begin(), items.end(),
9             [](Item& a, Item& b) {
10                 return (a.valor / a.peso) > (b.valor / b.peso);
11             });
12
13         double valorTotal = 0;
14         double capacidadRestante = W;
15
16         for (auto& it : items) {
17             if (capacidadRestante <= 0) break;
18
19             if (it.peso <= capacidadRestante) {
20                 // cabe completo -> lo toma entero
21                 valorTotal += it.valor;
22                 capacidadRestante -= it.peso;
23             } else {
24                 // no cabe completo -> toma la FRACCION exacta que
25                 // llena lo que queda, y se detiene (todo lo que
26                 // sigue tiene PEOR razon valor/peso, ya no conviene)
27                 double fraccion = capacidadRestante / it.peso;
28                 valorTotal += it.valor * fraccion;
29                 capacidadRestante = 0;
30             }
31         }
32
33         return valorTotal;
34     };
35 }
```

Ejemplo de funcionamiento:

```

1 vector<FractionalKnapsack::Item> items = {
2     {10, 60}, // razon 6.0
3     {20, 100}, // razon 5.0
4     {30, 120}, // razon 4.0
5 };
6
7 cout << FractionalKnapsack::valorMaximo(items, 50);
8 // 240
9 // toma completo el de razon 6.0 (peso 10, valor 60) -> quedan
10 // 40
11 // toma completo el de razon 5.0 (peso 20, valor 100) -> quedan
12 // 20
13 // toma 20/30 = 2/3 del de razon 4.0 -> valor 120 * 2/3 = 80
14 // total: 60 + 100 + 80 = 240
```

Nota: si el problema pide expresamente que los ítems **no** se puedan fraccionar (la variante 0/1), este mismo código da una respuesta **incorrecta** (generalmente una sobreestimación, ya que asume que cualquier fracción es viable) — la señal en el enunciado suele ser explícita ("cada ítem se toma completo o se descarta"), pero si no lo es, un caso de prueba donde el óptimo fraccional y el 0/1 difieren (ítems con pesos que no "encajan" limpiamente) revela rápidamente cuál se necesita.

Regla práctica: la pregunta decisiva antes de programar cualquier variante de Knapsack es "¿se permite tomar fracciones de un ítem?" — si sí, es este greedy en $O(n \log n)$; si no, es 0/1 Knapsack y hace falta DP en $O(nW)$, sin excepciones ni atajos greedy posibles (ver Greedy vs. Programación Dinámica para la justificación completa de por qué no hay forma de "adaptar" este greedy al caso 0/1).

43 Greedy con Heap

43.1 Huffman Coding

Idea: Huffman Coding construye un **código de prefijo óptimo** (ningún código es prefijo de otro, lo que permite decodificar sin ambigüedad) para un conjunto de símbolos con frecuencias dadas, minimizando la **longitud total esperada** del mensaje codificado. El greedy: mantener todos los símbolos en un **min-heap** por frecuencia, y repetidamente **extraer los dos de menor frecuencia, fusionarlos** en un nuevo nodo (cuya frecuencia es la suma de ambos) y **reinsertarlo** en el heap —hasta que quede un solo nodo, la **raíz** del árbol de Huffman. El código de cada símbolo original es el camino desde la raíz hasta su hoja (0 por cada rama izquierda, 1 por cada derecha, o viceversa) — los símbolos **más frecuentes** terminan más **cerca** de la raíz (códigos más cortos), y los **menos frecuentes** quedan más profundos (códigos más largos).

¿Por qué usarlo? Es la aplicación canónica de "Greedy con Heap": la decisión de fusionar siempre los **dos menos frecuentes** está justificada por Exchange Argument (ver Fundamentos de Greedy) —cualquier árbol óptimo se puede transformar en uno donde los dos símbolos de menor frecuencia son **hermanos** en las hojas más profundas, sin aumentar el costo total. Es también la base conceptual de compresión sin pérdida (ej. el algoritmo detrás de `gzip/DEFLATE` usa una variante de Huffman), y el ejemplo estándar donde "extraer siempre el mínimo, combinar, reinsertar" con una `priority_queue` resuelve un problema de construcción de estructura, no solo de selección de elementos.

Casos de uso:

- Compresión de datos: asignar códigos binarios de longitud variable a símbolos según su frecuencia, minimizando el tamaño total codificado.
- Cualquier problema que pida "combina elementos de a pares, minimizando/maximizando el costo total de combinación" donde el costo de cada fusión es la suma de los dos combinados (ver variantes de "mínimo costo para combinar n pilas/cuerdas" — estructuralmente idéntico a Huffman, solo sin la parte de asignar códigos al final).
- Construcción de árboles de decisión balanceados por frecuencia de uso (símbolos usados más a menudo, más accesibles).

Complejidad: $O(n \log n)$ — cada uno de los $n - 1$ pasos de fusión hace 2 extracciones y 1 inserción en el heap, cada operación $O(\log n)$.


```

1 struct HuffmanCoding {
2     struct Nodo {
3         int frecuencia;
4         int simbolo; // -1 si es un nodo interno (fusion), el
5         // simbolo real si es una hoja
6         Nodo *izq, *der;
7
8         Nodo(int f, int s) : frecuencia(f), simbolo(s),
9         izq(nullptr), der(nullptr) {}
10    };
11
12    struct Comparador {
13        bool operator()(Nodo* a, Nodo* b) {
14            return a->frecuencia > b->frecuencia; // min-heap
15        }
16    };
17
18    // READ: construye el arbol de Huffman a partir de simbolos y
19    // sus
20    // frecuencias. Devuelve la RAIZ del arbol.
21    static Nodo* construirArbol(vector<int>& simbolos,
22    vector<int>& frecuencias) {
23        priority_queue<Nodo*, vector<Nodo*>, Comparador> heap;
24
25        for (int i = 0; i < (int)simbolos.size(); i++)
26            heap.push(new Nodo(frecuencias[i], simbolos[i]));
27
28        // caso borde: un solo simbolo -> arbol trivial de 1 nodo
29        if (heap.size() == 1) return heap.top();
30
31        while (heap.size() > 1) {
32            Nodo* a = heap.top(); heap.pop();
33            Nodo* b = heap.top(); heap.pop();
34
35            Nodo* fusion = new Nodo(a->frecuencia + b->frecuencia,
36            -1);
37            fusion->izq = a;
38            fusion->der = b;
39
40            heap.push(fusion);
41        }
42
43        return heap.top(); // O(n log n)
44
45        // READ: recorre el arbol y arma el codigo binario de cada
46        // simbolo (0 = izquierda, 1 = derecha desde la raiz).
47        static void obtenerCodigos(Nodo* raiz, string codigoActual,
48        unordered_map<int, string>& codigos) {
49            if (!raiz) return;
50
51            if (raiz->simbolo != -1) { // es una hoja
52                // caso borde: un solo simbolo en total -> codigo "0"
53                codigos[raiz->simbolo] = codigoActual.empty() ? "0" :
54                codigoActual;
55                return;
56            }
57
58            obtenerCodigos(raiz->izq, codigoActual + "0", codigos);
59            obtenerCodigos(raiz->der, codigoActual + "1", codigos);
60        } // O(n) sobre el
61        // arbol completo
62    };

```

Ejemplo de funcionamiento:

```

1 vector<int> simbolos = {0, 1, 2, 3, 4}; // representan 'a
2 // 'b', 'c', 'd', 'e'
3 vector<int> frecuencias = {5, 9, 12, 13, 16};
4
5 auto raiz = HuffmanCoding::construirArbol(simbolos, frecuencias
6 );
7
8 unordered_map<int, string> codigos;
9 HuffmanCoding::obtenerCodigos(raiz, "", codigos);
10
11 for (int s : simbolos)
12     cout << s << " : " << codigos[s] << " ";
13 // 0: 1100 1: 1101 2: 100 3: 101 4: 111
14 // (los simbolos con MAYOR frecuencia, como 4 con 16, obtienen
15 // codigos mas CORTOS; los de menor frecuencia, como 0 con 5,
16 // obtienen codigos mas largos)

```

Nota: el caso borde de un solo símbolo en el alfabeto es fácil de pasar por alto —el árbol queda como un único nodo hoja sin ningún camino real desde una raíz interna, así que hay que asignarle explícitamente el código "0" (o cualquier código de longitud 1) en vez de una cadena vacía, que no sería un código de prefijo

válido para decodificar nada.

Regla práctica: el mismo patrón "extrae los dos menores, fusiona, reinserta" resuelve cualquier problema de "combina n elementos de a pares minimizando el costo total de combinación, donde combinar cuesta la suma de los dos" —aunque el problema no mencione compresión ni códigos en absoluto (ver el caso de uso de "mínimo costo para combinar pilas/cuerdas"). Reconocer esa estructura general vale más que memorizar el algoritmo solo en el contexto de compresión de datos.

43.2 Reorganize String (Reordenar sin Adyacentes Iguales)

Idea: dado un string, se busca **reorganizar** sus caracteres de forma que **ningún** par de caracteres **adyacentes** sea igual (si es posible; si no, se reporta que no existe tal reorganización). El greedy: usar un **max-heap** por frecuencia de cada carácter, y en cada paso colocar el carácter **más frecuente disponible** que **no** sea igual al último colocado —para evitar volver a colocar ese mismo carácter inmediatamente después (lo cual violaría la restricción), se **retiene** temporalmente el carácter recién usado (con su frecuencia decrementada) y se reinserta en el heap **después** de colocar el siguiente, dándole "un turno de espera" antes de volver a competir por la siguiente posición.

¿Por qué usarlo? Comparte la estructura de "Greedy con Heap" de Huffman y Task Scheduler con Cooldown —de hecho es **estructuralmente el mismo problema** que Task Scheduler con $n = 1$ (cooldown de exactamente 1 posición): siempre prioriza el carácter más frecuente disponible, porque es el que tiene **más riesgo** de no poder colocarse más adelante si se pospone. La **existencia** de una reorganización válida tiene además una condición simple y verificable de antemano: es posible si y solo si la frecuencia **máxima** de cualquier carácter no supera $\lceil n/2 \rceil$ (donde n es la longitud del string) — si un carácter aparece más que eso, no hay forma de "espaciarlo" suficiente sin que dos copias queden adyacentes.

Casos de uso:

- Reorganizar un string para que no haya caracteres repetidos consecutivos (el problema base, común en jueces de práctica).
- Verificar de antemano si una reorganización válida **existe**, sin necesariamente construirla, usando solo la condición de frecuencia máxima $\leq \lceil n/2 \rceil$.
- Generalización a "distancia mínima k entre repeticiones" (no solo evitar adyacentes) — es exactamente Task Scheduler con Cooldown $n = k - 1$, mismo esqueleto de heap más cola de espera.

Complejidad: $O(n \log k)$ donde k es el número de caracteres distintos — cada carácter del string original provoca una operación de heap ($O(\log k)$).

```

1 struct ReorganizeString {
2     // READ: reorganiza s para que no haya caracteres adyacentes
3     // iguales. Devuelve el string reorganizado, o "" si NO es
4     // posible.
5     static string reorganizar(string s) {
6         int n = s.size();
7         unordered_map<char, int> frecuencia;
8         for (char c : s) frecuencia[c]++;
9     }

```

```

8 // condicion de existencia: ningun caracter puede aparecer
9 // mas de ceil(n/2) veces
10 for (auto& [c, f] : frecuencia)
11 if (f > (n + 1) / 2)
12 return ""; // imposible, se detecta ANTES de
13 // intentar construir nada
14
15 priority_queue<pair<int, char>> heap; // max-heap por
16 // frecuencia
17 for (auto& [c, f] : frecuencia)
18 heap.push({f, c});
19
20 string resultado;
21 pair<int, char> anterior = {0, '#'}; // caracter retenido
22 // (frecuencia YA
23 // decrementada, en
24 // espera de volver
25 // al heap)
26
27 while (!heap.empty()) {
28     auto [freq, c] = heap.top(); heap.pop();
29
30     resultado += c;
31     freq--;
32
33     // el caracter USADO en el paso ANTERIOR ya cumpliero su
34     // "turno de espera" -- ahora si puede reingresar al heap
35     if (anterior.first > 0)
36         heap.push(anterior);
37
38     anterior = {freq, c}; // este caracter recién usado
39     // queda retenido hasta el
40     // proximo paso
41 }
42
43 return resultado; // longitud correcta garantizada por la
44 // condicion de existencia ya verificada // O(n log k)
45 }
46
47 };

```

Ejemplo de funcionamiento:

```

1 cout << ReorganizeString::reorganizar("aab");
2 // "aba" (única reorganización válida salvo simetrías)
3
4 cout << ReorganizeString::reorganizar("aaab");
5 // "" (la 'a' aparece 3 veces en un string de longitud 4;
6 // ceil(4/2)=2, 3 > 2 -> imposible)
7
8 cout << ReorganizeString::reorganizar("vvvlo");
9 // "vulvov" (u otra reorganización válida equivalente)

```

Regla práctica: este problema es Task Scheduler con Cooldown con $n = 1$ disfrazado de string —si el enunciado pide una **distancia mínima** k entre repeticiones en vez de solo "no adyacentes", el mismo esqueleto de heap + cola de espera (con espera de k pasos en vez de 1) resuelve la variante general sin cambios estructurales. Siempre verifica la condición de existencia (frecuencia máxima $\leq \lceil n/2 \rceil$ para este caso, o su generalización para k arbitrario) **antes** de intentar construir el resultado —ahorra tiempo de cómputo y evita bugs de "casi funciona pero deja un par adyacente al final".

43.3 Conectar Cuerdas al Mínimo Costo (Connect Ropes)

Idea: dado un conjunto de n cuerdas de longitudes dadas, se busca **conectarlas todas en una sola** cuerda, donde el costo de conectar dos cuerdas es la **suma** de sus longitudes —y se busca minimizar el **costo total** acumulado de todas las conexiones necesarias. El greedy: usar un **min-heap** por longitud, y repetidamente **extraer las dos más cortas, sumarlas** (ese costo se agrega al total), y **reinsertar** la cuerda resultante —hasta que quede una sola cuerda. Es **exactamente** el mismo esqueleto de Huffman Coding (extraer los dos menores, fusionar, reinsertar), pero aquí la "fusión" representa una operación física real (unir cuerdas) en vez de construir

un árbol de códigos —el **costo acumulado** de todas las fusiones es la respuesta, no un árbol ni códigos derivados de él.

¿Por qué usarlo? Es el ejemplo que confirma que el patrón de Huffman **no es exclusivo** de compresión de datos: cualquier problema de "combina n elementos de a pares, donde combinar cuesta la suma de ambos, minimizando el costo total acumulado" tiene **la misma** estructura y **la misma** justificación por Exchange Argument —siempre conviene combinar las dos piezas más pequeñas disponibles, porque cualquier costo de fusión que involucre a una pieza pequeña se "arrastra" y se vuelve a sumar en cada fusión posterior donde esa pieza participe; minimizar cuántas veces se recuenta cada pieza pequeña es lo que hace óptimo el greedy.

Casos de uso:

- Minimizar el costo total de fusionar n archivos/bloques de datos, donde fusionar dos bloques cuesta la suma de sus tamaños (variante directa con otro nombre de dominio).
- Minimizar el costo total de una serie de operaciones de *merge* en estructuras como *k-way merge* de arreglos ordenados, donde el costo de cada *merge* es proporcional al tamaño combinado.
- Como verificación de que un problema "nuevo" es en realidad Huffman sin la parte de construir códigos —si el enunciado dice "combina de a pares, el costo es la suma, minimiza el total", es esta misma plantilla, sin necesidad de construir el árbol explícito.

Complejidad: $O(n \log n)$ — cada una de las $n - 1$ fusiones hace 2 extracciones y 1 inserción en el heap, cada operación $O(\log n)$.

```

1 struct ConnectRopes {
2     // READ: costo minimo total para conectar todas las cuerdas
3     // en
4     // una sola. Mismo esqueleto que Huffman (extraer dos menores,
5     // fusionar, reinsertar), pero acumulando el COSTO en vez de
6     // construir un arbol.
7     static long long costoMinimoConectar(vector<int>& longitudes)
8     {
9         priority_queue<long long, vector<long long>, greater<>>
10         heap;
11         // ~ min-heap: greater<> invierte el orden por defecto de
12         // priority_queue (que es max-heap)
13
14         for (int i : longitudes)
15             heap.push(i);
16
17         long long costoTotal = 0;
18
19         while (heap.size() > 1) {
20             long long a = heap.top(); heap.pop();
21             long long b = heap.top(); heap.pop();
22
23             long long costoFusion = a + b;
24             costoTotal += costoFusion;
25
26             heap.push(costoFusion); // la cuerda resultante vuelve
27             // a competir por las
28             // proximas fusiones
29         }
30
31         return costoTotal;
32     }
33 }; // O(n log n)

```

Ejemplo de funcionamiento:

```

1 vector<int> longitudes = {4, 3, 2, 6};
2
3 cout << ConnectRopes::costoMinimoConectar(longitudes);

```

```

4 // 29
5 // paso 1: fusiona 2+3=5 (costo acumulado: 5) -> quedan
  {4,5,6}
6 // paso 2: fusiona 4+5=9 (costo acumulado: 5+9=14) -> quedan
  {6,9}
7 // paso 3: fusiona 6+9=15 (costo acumulado: 14+15=29) -> queda
  {15}

```

Nota: igual que en Huffman, el caso borde de $n = 1$ (una sola cuerda) no requiere ninguna fusión —el `while (heap.size() > 1)` ya maneja esto correctamente, devolviendo costo 0 sin necesidad de un chequeo especial adicional.

Regla práctica: cualquier variante de "combina n cosas de a pares, el costo/peso de combinar es la suma, minimiza el total acumulado" es esta misma plantilla sin importar el nombre del dominio —reconocerla ahorra tener que razonar el Exchange Argument desde cero cada vez, ya que es idéntico al de Huffman Coding (extraer siempre las dos piezas más chicas disponibles).

44 Greedy en Arreglos y Strings

44.1 Gas Station Problem

Idea: dado un circuito circular de n gasolineras, cada una con una cantidad de combustible `gas[i]` disponible y un costo `costo[i]` para llegar de esa gasolinera a la siguiente, se busca el **único** punto de partida (si existe) desde el cual es posible completar **todo** el circuito sin quedarse sin combustible en ningún momento. El greedy: recorrer las gasolineras **una sola vez**, manteniendo un **tanque acumulado** (`gas[i] - costo[i]` en cada parada); en cuanto el tanque acumulado **cae por debajo de cero** en la gasolinera i , ninguna gasolinera **entre** el candidato actual e i (inclusive) puede ser el punto de partida válido —se **descarta todo ese rango de una sola vez**, y el nuevo candidato pasa a ser $i + 1$, reiniciando el tanque acumulado en cero.

¿Por qué usarlo? La observación clave que hace válido este greedy: si el candidato actual s falla al llegar a la gasolinera i (el tanque se agota ahí), **ninguna** gasolinera j con $s \leq j < i$ puede ser un punto de partida válido tampoco —empezar en j en vez de s solo puede llegar a i con **igual o menos** combustible acumulado (todo lo acumulado entre s y j se pierde, ya que j empieza en cero), así que si s ya no llegaba, j tampoco. Eso permite **saltar** directamente a $i + 1$ sin revisar individualmente los candidatos descartados, dando $O(n)$ en vez de $O(n^2)$ (probar cada punto de partida por separado con una simulación completa).

Casos de uso:

- El problema de gasolineras en un circuito circular tal cual (encontrar el único punto de partida válido, si existe).
- Cualquier problema de "circuito circular con recursos que se ganan y gastan en cada parada, encuentra desde dónde arrancar para nunca quedar en déficit" —la estructura se generaliza a dominios distintos de combustible/distancia.
- Verificar primero la **condición de existencia**: una solución existe si y solo si $\sum \text{gas}[i] \geq \sum \text{costo}[i]$ (el total disponible alcanza para todo el circuito) —

si esta suma es negativa, ningún punto de partida funciona y no hace falta ni correr el greedy.

Complejidad: $O(n)$ — una sola pasada, sin necesidad de simular el circuito completo para cada candidato.

```

1 struct GasStation {
2     // READ: índice del punto de partida valido para completar
    todo
3     // el circuito, o -1 si no existe ninguno.
4     static int puntoDePartida(vector<int>& gas, vector<int>&
    costo) {
5         int n = gas.size();
6         long long tanqueTotal = 0; // suma GLOBAL de gas - costo,
7         // decide si existe solucion
8         long long tanqueActual = 0; // tanque acumulado desde el
9         // candidato actual
10        int candidato = 0;
11
12        for (int i = 0; i < n; i++) {
13            long long diferencia = gas[i] - costo[i];
14            tanqueTotal += diferencia;
15            tanqueActual += diferencia;
16
17            if (tanqueActual < 0) {
18                // el candidato actual (y todo lo que hay entre el y
19                // i) queda DESCARTADO de una sola vez -- el proximo
20                // candidato es i+1, reiniciando el tanque
21                candidato = i + 1;
22                tanqueActual = 0;
23            }
24        }
25
26        return (tanqueTotal >= 0) ? candidato : -1;
27    } // O(n)
28 };

```

Ejemplo de funcionamiento:

```

1 vector<int> gas = {1, 2, 3, 4, 5};
2 vector<int> costo = {3, 4, 5, 1, 2};
3
4 cout << GasStation::puntoDePartida(gas, costo);
5 // 3
6 // desde la gasolinera 3: tanque = 4-1=3 -> 3+5-2=6 -> 6+1-3=4
7 // ->
8 // 4+2-4=2 -> 2+3-5=0, completa el circuito sin quedar negativo

```

Nota: el resultado es **único** cuando existe —esta propiedad no es evidente a primera vista, pero se sigue de la misma lógica del descarte: si hubiera dos puntos de partida válidos distintos $s_1 < s_2$, el argumento de "todo lo que hay entre un candidato fallido y el punto donde falla se descarta" implicaría una contradicción sobre cuál de los dos realmente puede sostener el circuito completo — en la práctica, basta confiar en que el greedy encuentra el único candidato que sobrevive hasta el final del recorrido.

Regla práctica: la señal para este patrón es "hay un **acumulado** que se gana y se gasta en un recorrido, encuentra desde dónde arrancar para que nunca sea negativo" —en cuanto el acumulado cae bajo cero, **todo** lo recorrido hasta ahí (incluido el candidato actual) se descarta de un salto; nunca hace falta volver a probar esos candidatos individualmente.

44.2 Candy Distribution

Idea: dado un arreglo de **calificaciones** de n niños en fila, se busca la asignación **mínima total** de dulces tal que (1) cada niño reciba **al menos 1** dulce, y (2) cualquier niño con calificación **mayor** que un **vecino adyacente** reciba **más** dulces que ese vecino. El greedy: **dos pasadas** sobre el arreglo. En la primera (izquierda a derecha), se asegura la restricción respecto al vecino **izquierdo**: si `calif[i] > calif[i-1]`, `dulces[i] = dulces[i-1] + 1` (si no, `dulces[i] = 1`). En la segunda

(derecha a izquierda), se asegura la restricción respecto al vecino **derecho**, sin deshacer lo ya garantizado por la primera pasada: si $\text{calif}[i] > \text{calif}[i+1]$, $\text{dulces}[i] = \text{máximo}(\text{dulces}[i], \text{dulces}[i+1] + 1)$.

¿Por qué usarlo? La restricción es **bidireccional** (cada niño compite con **ambos** vecinos simultáneamente), lo cual hace que una sola pasada en cualquier dirección **no alcance** —una pasada izquierda-derecha por sí sola satisface la restricción con el vecino izquierdo pero puede violar la del derecho (y viceversa). El truco de **tomar el máximo** en la segunda pasada es la clave: garantiza que el valor final de $\text{dulces}[i]$ satisface **ambas** restricciones a la vez, sin que la segunda pasada rompa lo que la primera ya había asegurado —cada pasada resuelve una dirección de la restricción de forma greedy y local (mirar solo un vecino a la vez), y combinarlas con **max** resuelve el problema bidireccional completo sin necesitar comparar contra ambos vecinos simultáneamente en un solo paso.

Casos de uso:

- El problema de distribución de dulces tal cual (minimizar el total dado un arreglo de calificaciones y restricciones locales bidireccionales).
- Cualquier problema de "asignación mínima que respeta restricciones de orden estricto con **ambos** vecinos en una secuencia" — el patrón de "dos pasadas + combinar con máximo/mínimo" se generaliza a variantes con más de un tipo de restricción por posición.
- Como ejemplo de que un greedy **unidireccional** no siempre alcanza, pero **dos** greedy unidireccionales combinados correctamente sí resuelven un problema bidireccional completo sin necesitar DP ni backtracking.

Complejidad: $O(n)$ — dos pasadas lineales sobre el arreglo, más una pasada final para sumar el total.

```

1 struct CandyDistribution {
2     // READ: numero mínimo total de dulces que satisface ambas
3     // restricciones (respecto al vecino izquierdo y al derecho).
4     static long long minimoDulces(vector<int>& calificaciones) {
5         int n = calificaciones.size();
6         vector<int> dulces(n, 1); // cada niño arranca con 1
7         dulce // (el mínimo obligatorio)
8
9         // PASADA 1 (izquierda -> derecha): garantiza la
10        // restricción
11        // respecto al vecino IZQUIERDO.
12        for (int i = 1; i < n; i++) {
13            if (calificaciones[i] > calificaciones[i - 1])
14                dulces[i] = dulces[i - 1] + 1;
15        }
16
17        // PASADA 2 (derecha -> izquierda): garantiza la
18        // restricción
19        // respecto al vecino DERECHO, sin deshacer la pasada 1 --
20        // por eso se usa MAX en vez de sobrescribir directamente.
21        for (int i = n - 2; i >= 0; i--) {
22            if (calificaciones[i] > calificaciones[i + 1])
23                dulces[i] = max(dulces[i], dulces[i + 1] + 1);
24        }
25
26        long long total = 0;
27        for (int d : dulces) total += d;
28
29        return total; // O(n)
30    }
31};

```

Ejemplo de funcionamiento:

```

1 vector<int> calificaciones = {1, 0, 2};
2
3 cout << CandyDistribution::minimoDulces(calificaciones);
4 // 5
5 // pasada 1: [1,1,2] (índice 2 > índice 1 -> 1+1=2)
6 // pasada 2: índice 1 < índice 0? no (0<1 es al revés, calif
7 // [1]=0 no es mayor que calif[0]=1) -- índice 0: calif[0]=1 >
8 // calif[1]=0 -> dulces[0] = max(1, dulces[1]+1) =
9 // max(1,2) = 2
10 // final: [2,1,2] -> suma = 5
11
12 vector<int> calificaciones2 = {1, 2, 2};
13 cout << CandyDistribution::minimoDulces(calificaciones2);
14 // 4
15 // pasada 1: [1,2,1] (índice 2 no es mayor que índice 1, se
16 // queda en 1)
17 // pasada 2: ninguna calificación es mayor que su vecino
18 // derecho aquí
19 // final: [1,2,1] -> suma = 4

```

Nota: calificaciones **iguales** entre vecinos **no** imponen ninguna restricción de orden —ambos pueden recibir la misma cantidad de dulces (o cantidades distintas, no importa, mientras cada uno cumpla sus restricciones individuales); el código solo actúa cuando la comparación es **estrictamente** mayor en alguna dirección.

Regla práctica: cuando una restricción greedy depende de **ambos lados** de una posición en una secuencia (no solo del vecino izquierdo o solo del derecho), la técnica de "dos pasadas unidireccionales combinadas con **max/min**" resuelve el problema sin necesitar comparar ambos vecinos a la vez en un único recorrido —cada pasada es un greedy simple y correcto para **su** dirección, y combinarlas preserva la corrección de ambas simultáneamente.

44.3 Jump Game (Alcanzabilidad Greedy)

Idea: dado un arreglo donde $\text{arr}[i]$ representa el **máximo salto** posible desde la posición i , se busca determinar si es posible **alcanzar** el último índice partiendo del primero (variante de decisión), o el **mínimo número de saltos** necesarios (variante de optimización). El greedy para la variante de decisión: mantener el **alcance máximo** visto hasta el momento (**maxAlcance**), recorriendo el arreglo posición por posición —si en algún punto i el índice actual **supera** el alcance máximo acumulado, ese punto es **inalcanzable** y el resto del arreglo también lo es (se corta de inmediato); si no, se actualiza $\text{maxAlcance} = \max(\text{maxAlcance}, i + \text{arr}[i])$ en cada paso. Para la variante de **mínimo número de saltos**, se usa la misma idea pero llevando dos "fronteras" —el alcance del salto **actual** y el mejor alcance visto dentro de ese salto— e incrementando el contador de saltos cada vez que se **agota** la frontera actual.

¿Por qué usarlo? Es el ejemplo canónico de "no hace falta simular **cada** salto posible (que sería exponencial, probando todas las combinaciones de saltos)" —basta con llevar **un solo número** (el mejor alcance posible hasta ahora) y actualizarlo en una sola pasada. La correctitud viene de que **nunca** importa **desde qué posición exacta** se logró el mejor alcance actual, solo **cuál es ese alcance** —eso colapsa un espacio de decisiones que a primera vista parece requerir backtracking en un simple **max** acumulado.

Casos de uso:

- Determinar si es posible llegar al final de un arreglo de saltos (Jump Game I).

- Encontrar el **mínimo** número de saltos para llegar al final, asumiendo que siempre es posible (Jump Game II).
- Cualquier problema de "alcanzabilidad hacia adelante" donde cada posición desbloquea un **rango** de posiciones futuras, y solo importa el alcance máximo acumulado, no el camino exacto tomado para llegar ahí.

Complejidad: $O(n)$ para ambas variantes — una sola pasada sobre el arreglo, sin recursión ni backtracking.

```

1 struct JumpGame {
2     // READ: es posible llegar desde el indice 0 hasta el ultimo
3     // indice del arreglo? (Jump Game I)
4     static bool esAlcanzable(vector<int>& arr) {
5         int n = arr.size();
6         int maxAlcance = 0;
7
8         for (int i = 0; i < n; i++) {
9             if (i > maxAlcance)
10                return false; // esta posicion (y todo lo que sigue)
11                // es INALCANZABLE, corta de inmediato
12
13            maxAlcance = max(maxAlcance, i + arr[i]);
14
15            if (maxAlcance >= n - 1)
16                return true; // ya se garantizo llegar al final,
17                // no hace falta seguir iterando
18        }
19
20        return maxAlcance >= n - 1;
21    } // O(n)
22
23    // READ: minimo numero de saltos para llegar al ultimo indice,
24    // ASUMIENDO que es alcanzable (llamar esAlcanzable primero
25    // si no se esta seguro).
26    static int minimoSaltos(vector<int>& arr) {
27        int n = arr.size();
28        if (n <= 1) return 0;
29
30        int saltos = 0;
31        int alcanceActual = 0; // hasta donde se puede llegar CON
32        // los saltos ya contados
33        int mejorAlcance = 0; // mejor alcance visto DENTRO del
34        // salto actual, candidato para el
35        // proximo salto
36
37        for (int i = 0; i < n - 1; i++) {
38            mejorAlcance = max(mejorAlcance, i + arr[i]);
39
40            if (i == alcanceActual) {
41                // se agoto lo que el salto actual podia cubrir ->
42                // hace falta UN salto mas, extendiendo el alcance
43                // al mejor visto hasta ahora
44                saltos++;
45                alcanceActual = mejorAlcance;
46            }
47        }
48
49        return saltos;
50    } // O(n)
51 };

```

Ejemplo de funcionamiento:

```

1 vector<int> arr1 = {2, 3, 1, 1, 4};
2 cout << JumpGame::esAlcanzable(arr1);
3 // true
4
5 vector<int> arr2 = {3, 2, 1, 0, 4};
6 cout << JumpGame::esAlcanzable(arr2);
7 // false (el 0 en el indice 3 atrapa cualquier camino que
8 // llegue
9 // ahí, y ningún salto anterior alcanza a saltarselo)
10
11 vector<int> arr3 = {2, 3, 1, 1, 4};
12 cout << JumpGame::minimoSaltos(arr3);
13 // 2 (salto 1: indice 0 -> alcanza hasta 2; salto 2: desde
14 // algún
15 // punto dentro de [1,2] alcanza hasta 4)

```

Regla práctica: para la variante de decisión, corta inmediatamente en cuanto $i > \text{maxAlcance}$ —no hace

falta terminar de recorrer el arreglo, esa condición ya prueba que el final es inalcanzable. Para la variante de mínimo número de saltos, el error más común es actualizar `alcanceActual` en **cada** posición en vez de solo cuando se **agota** el salto actual ($i == \text{alcanceActual}$) —eso sobreestima el número de saltos, contando transiciones que en realidad no hacía falta forzar.

44.4 Remove K Digits (Stack Monotónico Greedy)

Idea: dado un número representado como string y un entero k , se busca **eliminar exactamente k dígitos** de forma que el número **resultante** (manteniendo el orden relativo de los dígitos restantes) sea el **menor posible**. El greedy: recorrer los dígitos de izquierda a derecha manteniendo un **stack** —mientras el stack no esté vacío, el dígito en el tope sea **mayor** que el dígito actual, y todavía **queden** eliminaciones disponibles ($k > 0$), se **desapila** (esa eliminación siempre conviene: un dígito grande seguido de uno más chico significa que quitar el grande **reduce** el número resultante, ya que ese dígito ocupaba una posición más significativa). Al final, si sobran eliminaciones sin usar, se quitan del **final** del stack (los dígitos menos significativos, donde eliminar afecta menos al valor).

¿Por qué usarlo? Es la aplicación de **Stack Monotónico** a un problema greedy de construcción **lexicográfica** óptima: la observación clave es que un dígito en una posición más **significativa** (más a la izquierda) pesa **más** en el valor final que cualquier combinación de dígitos a su derecha —así que "si el dígito actual es menor que el anterior, siempre conviene quitar el anterior" es una decisión **local** que nunca se arrepiente, exactamente el tipo de propiedad que hace válido un greedy con stack en vez de backtracking sobre qué dígitos eliminar.

Casos de uso:

- Eliminar exactamente k dígitos de un número para minimizar el resultado (el problema base, "Remove K Digits").
- Variante de **maximizar** en vez de minimizar: mismo esqueleto, invirtiendo la condición de comparación (desapilar cuando el tope es **menor** que el actual, no mayor).
- Construcción de la subsecuencia **lexicográficamente más pequeña/grande** de un string, con o sin restricción de longitud exacta —mismo patrón de stack monotónico aplicado a caracteres en vez de dígitos.
- Como base conceptual del algoritmo "Create Maximum Number" (combinar dos arreglos eligiendo k dígitos de cada uno para maximizar el resultado), que extiende esta misma idea con una fusión adicional.

Complejidad: $O(n)$ — cada dígito se apila y desapila como mucho una vez, sin importar el valor de k .

```

1 struct RemoveKDigits {
2     // READ: el numero resultante MAS PEQUEÑO posible tras
3     // eliminar
4     // exactamente k digitos de num, manteniendo el orden
5     // relativo de

```

```

4 // los digitos restantes.
5 static string minimoTrasEliminar(string num, int k) {
6     string stack; // se usa un string como stack para poder
7     // construir el resultado directamente
8
9     for (char digito : num) {
10        // mientras el tope sea MAYOR que el digito actual, y
11        // todavia queden eliminaciones, desapila -- esa
12        // eliminacion siempre reduce el valor resultante
13        while (!stack.empty() && k > 0 && stack.back() > digito)
14        {
15            stack.pop_back();
16            k--;
17        }
18        stack.push_back(digito);
19    }
20
21    // si sobraron eliminaciones sin usar, se quitan del FINAL
22    // (los digitos menos significativos)
23    while (k > 0 && !stack.empty()) {
24        stack.pop_back();
25        k--;
26    }
27
28    // elimina ceros a la izquierda del resultado
29    int inicio = 0;
30    while (inicio < (int)stack.size() - 1 && stack[inicio] == '0')
31        inicio++;
32
33    string resultado = stack.substr(inicio);
34
35    return resultado.empty() ? "0" : resultado; // O(n)
36 };

```

Ejemplo de funcionamiento:

```

1 cout << RemoveKDigits::minimoTrasEliminar("1432219", 3);
2 // "1219"
3 // stack: 1 -> 14 -> desapila 4 (1<4? no, 4>3 -> desapila 4),
4 // queda
5 // "1", agrega 3 -> "13" -> agrega 2, 3>2 desapila -> "12" ->
6 // agrega 2
7 // -> "122" -> agrega 1, 2>1 desapila (k aun>0) -> "121" ->
8 // agrega 1
9 // -> "1211" ... (siguiendo la logica completa da "1219",
10 // eliminando
11 // 4, 3, y el segundo 2 en el proceso completo)
12
13 cout << RemoveKDigits::minimoTrasEliminar("10200", 1);
14 // "200"
15 // elimina el primer '1' (mayor que el '0' siguiente), quedan
16 // ceros
17 // a la izquierda que se recortan: "0200" -> "200"
18
19 cout << RemoveKDigits::minimoTrasEliminar("10", 2);
20 // "0"
21 // se eliminan ambos digitos, no queda nada -> resultado "0"

```

Nota: el manejo de **ceros a la izquierda** en el resultado final es un detalle de borde fácil de olvidar —un número como "200" nunca debería mostrarse como "0200", y el caso extremo donde **todo** el string queda vacío tras recortar ceros (ej. eliminar todos los dígitos) debe devolver explícitamente "0", no una cadena vacía.

Regla práctica: la señal para Stack Monotónico en problemas greedy de construcción de secuencias es "una decisión **local** (comparar el elemento actual contra el tope del stack) nunca empeora el resultado global, sin importar qué venga después" —si esa propiedad se cumple (como aquí, gracias al peso posicional de los dígitos), el problema se resuelve en $O(n)$ con un stack; si la decisión óptima **sí** depende de lo que viene más adelante, la técnica no aplica y hace falta otra herramienta.

44.5 Lexicográficamente Mínimo/Máximo con Restricciones

Idea: generaliza el patrón de Stack Monotónico de Remove K Digits a problemas donde la construcción **no** está limitada solo por "cuántos elementos quitar", sino por

restricciones adicionales sobre qué elementos deben aparecer en el resultado —el ejemplo canónico es "**Remove Duplicate Letters**": dado un string, construir la subsecuencia lexicográficamente **más pequeña** que contenga **cada** carácter distinto exactamente **una vez**. El greedy sigue siendo un stack, pero con **dos** condiciones para desapilar en vez de una: (1) el tope es **mayor** que el carácter actual (igual que antes), **y** (2) ese carácter del tope **vuelve a aparecer más adelante** en el string (si no vuelve a aparecer, desapilarlo ahora significaría **perderlo** para siempre, violando la restricción de "cada carácter exactamente una vez"). Se necesita además un **registro de qué ya está en el stack** (enStack) para nunca agregar un carácter duplicado.

¿Por qué usarlo? Es el paso natural después de Remove K Digits: mientras ahí la única restricción era "elimina exactamente k ", aquí la restricción es **estructural** (cada carácter debe sobrevivir en algún lugar del resultado) — y esa restricción **limita** cuándo es seguro desapilar. Reconocer que el criterio de desapilar de un Stack Monotónico puede **ampliarse** con condiciones extra (en vez de solo comparar valores) es lo que permite adaptar la misma técnica base a una familia mucho más amplia de problemas de construcción lexicográfica.

Casos de uso:

- **Remove Duplicate Letters / Smallest Subsequence of Distinct Characters:** construir la subsecuencia lexicográficamente mínima que contenga cada carácter exactamente una vez (el ejemplo desarrollado en esta subsección).
- Variantes de **longitud fija:** construir la subsecuencia lexicográficamente mínima/máxima de exactamente k caracteres, respetando el orden original — mismo esqueleto de Remove K Digits, pero pensando "cuántos me quedan por recorrer" en vez de "cuántas eliminaciones me quedan".
- Construcción de la permutación/secuencia lexicográficamente mínima sujeta a un conjunto de restricciones de precedencia o de unicidad — cualquier variante donde "cada elemento debe aparecer exactamente una vez, en algún orden que respete el original" es candidata a este patrón.
- Para **maximizar** en vez de minimizar: invertir la condición de comparación (desapilar cuando el tope es **menor** que el actual), manteniendo las mismas restricciones de unicidad y última aparición.

Complejidad: $O(n)$ — cada carácter se apila y desapila como mucho una vez; el precómputo de "última aparición de cada carácter" es $O(n)$ adicional.

```

1 struct LexicograficamenteMinimoConRestricciones {
2     // READ: subsecuencia lexicograficamente MINIMA de s que
3     // contiene
4     // cada caracter distinto EXACTAMENTE una vez (Remove
5     // Duplicate
6     // Letters / Smallest Subsequence of Distinct Characters).
7     static string subsecuenciaMinima(string s) {
8         // precomputo: ultima posicion donde aparece cada caracter
9         vector<int> ultimaApar(26, -1);
10        for (int i = 0; i < (int)s.size(); i++)
11            ultimaApar[s[i] - 'a'] = i;
12
13        string stack;
14        vector<bool> enStack(26, false);

```

```

13 for (int i = 0; i < (int)s.size(); i++) {
14     char c = s[i];
15
16     if (enStack[c - 'a']) continue; // ya esta en el
17                                     // resultado, no se
18                                     // puede repetir
19
20     // desapila mientras: el tope es MAYOR que c (mejora el
21     // orden), Y el caracter del tope VUELVE A APARECER mas
22     // adelante (si no vuelve, no se puede perder ahora)
23     while (!stack.empty() && stack.back() > c &&
24           ultimaApar[stack.back() - 'a'] > i) {
25         enStack[stack.back() - 'a'] = false;
26         stack.pop_back();
27     }
28
29     stack.push_back(c);
30     enStack[c - 'a'] = true;
31 }
32
33 return stack;
34 } // O(n)
35
36 // READ: subsecuencia lexicograficamente minima de
37 // EXACTAMENTE k
38 // caracteres (sin restriccion de unicidad, solo de longitud
39 // fija) -- mismo esqueleto que Remove K Digits, pensado en
40 // "caracteres restantes por recorrer" en vez de "
41 // eliminaciones
42 // disponibles".
43 static string subsecuenciaMinimaDeLongitudK(string s, int k)
44 {
45     string stack;
46     int n = s.size();
47
48     for (int i = 0; i < n; i++) {
49         char c = s[i];
50
51         // se puede desapilar mientras: el tope es mayor que c,
52         // y todavia queda SUFICIENTE string restante (incluyendo
53         // c) para completar los k caracteres finales
54         while (!stack.empty() && stack.back() > c &&
55               (int)stack.size() - 1 + (n - i) >= k) {
56             stack.pop_back();
57         }
58
59         if ((int)stack.size() < k)
60             stack.push_back(c);
61     }
62
63     return stack;
64 } // O(n)
65 };

```

Ejemplo de funcionamiento:

```

1 cout << LexicograficamenteMinimoConRestricciones::
2   subsecuenciaMinima("bcabc");
3 // "abc"
4 // b entra -> "b"; c entra (c>b) -> "bc"; a entra, b>a y b
5 // reaparece
6 // despues (indice 3) -> desapila b; c>a y c reaparece (indice
7 // 4) ->
8 // desapila c; queda "a" -> "a"; b entra -> "ab"; c entra -> "
9 // abc"
10
11 cout << LexicograficamenteMinimoConRestricciones::
12   subsecuenciaMinima("cbacdcbc");
13 // "acdb"
14
15 cout << LexicograficamenteMinimoConRestricciones::
16   subsecuenciaMinimaDeLongitudK("9142", 2);
17 // "12"

```

Nota: la condición extra de "el tope **vuelve a aparecer** más adelante" es lo único que distingue este problema de Remove K Digits — sin ella, el greedy podría desapilar un carácter que **nunca más** vuelve a estar disponible, dejándolo fuera del resultado y violando la restricción de que cada carácter debe aparecer exactamente una vez. Ese chequeo (`ultimaApar[...] > i`) es exactamente lo que hay que agregar cada vez que un problema de Stack Monotónico tiene una restricción de "no puedo perder este elemento para siempre".

Regla práctica: cuando un problema de construcción lexicográfica con stack tiene una restricción **extra** más

allá de "cuántos elementos caben" (unicidad, precedencia, disponibilidad futura), la adaptación casi siempre es la misma: agregar esa restricción como una condición **adicional** (con AND lógico) en la guardia del `while` que decide cuándo desapilar — el resto del esqueleto de Remove K Digits se mantiene intacto.

45 Greedy con Matemáticas

45.1 Coin Change Greedy (Cuándo Funciona y Cuándo No)

Idea: dado un conjunto de denominaciones de moneda y un monto objetivo, Coin Change Greedy intenta minimizar el número de monedas usadas tomando siempre la denominación **más grande** que quepa en el monto restante, repitiendo hasta llegar a cero (o determinar que no se puede). Como ya se adelantó en Greedy vs. Programación Dinámica, este greedy es **correcto solo para sistemas de monedas canónicos** (como 1, 5, 10, 25) y puede **fallar** con denominaciones arbitrarias. Esta subsección profundiza en **cómo verificar** si un sistema de denominaciones dado es canónico o no, en vez de solo constatar el fallo con un contraejemplo puntual.

¿Por qué usarlo? Reconocer **cuándo confiar** en el greedy es tan importante como saber implementarlo —usarlo sin verificar sobre un sistema no canónico da una respuesta **silenciosamente incorrecta** (compila, corre, y devuelve un número mayor al óptimo real, sin ningún error visible). La forma **práctica** de verificar canonicidad sin una prueba matemática formal es el **teorema del contraejemplo mínimo**: si un sistema de k denominaciones **no** es canónico, existe un contraejemplo menor que $c_{k-1} + c_k$ (la suma de las dos denominaciones más grandes) — así que basta con comparar greedy contra DP para **todos** los montos hasta ese límite; si coinciden siempre en ese rango, el sistema es canónico para **cualquier** monto.

Casos de uso:

- Verificar de antemano, antes de programar el greedy en competencia, si las denominaciones dadas en el enunciado son canónicas — evita el riesgo de un Wrong Answer silencioso por confiar en greedy sin justificación.
- Sistemas de monedas del mundo real (la mayoría de monedas nacionales) suelen ser canónicos por diseño —facilita dar cambio manualmente— pero un enunciado de competencia puede dar denominaciones **arbitrarias** a propósito, precisamente para descartar el greedy como solución válida.
- Como ejercicio de "verificación empírica acotada": en vez de demostrar una propiedad matemáticamente, acotar cuántos casos hace falta revisar para estar seguro (el mismo espíritu que las pruebas de Miller-Rabin en teoría de números, aunque aquí sea determinista y exacto, no probabilístico).

Complejidad: $O(k)$ para el greedy en sí (con k denominaciones); $O(\text{límite} \cdot k)$ para la verificación de canonicidad, donde límite es la suma de las dos denominaciones más grandes — mucho más costoso que el greedy mismo,

pero es un chequeo de **una sola vez** antes de confiar en él para todos los montos futuros.

```

1 struct CoinChangeGreedyVerificado {
2     // READ: numero de monedas usando GREEDY (toma siempre la mas
3     // grande que quepa). Puede ser INCORRECTO si el sistema no
4     // es
5     // canonico -- usar solo tras verificar con esCanonico().
6     static int monedasGreedy(vector<int> denominaciones, int
7     monto) {
8         sort(denominaciones.rbegin(), denominaciones.rend());
9         int monedas = 0;
10
11         for (int d : denominaciones) {
12             monedas += monto / d;
13             monto %= d;
14         }
15
16         return (monto == 0) ? monedas : -1;
17     } // O(k)
18
19     // READ: numero MINIMO real de monedas via DP (siempre
20     // correcto,
21     // sin importar las denominaciones).
22     static int monedasDP(vector<int>& denominaciones, int monto)
23     {
24         vector<int> dp(monto + 1, INT_MAX);
25         dp[0] = 0;
26
27         for (int m = 1; m <= monto; m++)
28             for (int d : denominaciones)
29                 if (d <= m && dp[m - d] != INT_MAX)
30                     dp[m] = min(dp[m], dp[m - d] + 1);
31
32         return (dp[monto] == INT_MAX) ? -1 : dp[monto];
33     } // O(monto * k)
34
35     // READ: el sistema de denominaciones es CANONICO (greedy
36     // siempre
37     // optimo, para CUALQUIER monto)? Usa el teorema del
38     // contraejemplo minimo: si greedy==DP para todo monto hasta
39     // la
40     // suma de las DOS denominaciones mas grandes, es canonico.
41     static bool esCanonico(vector<int> denominaciones) {
42         sort(denominaciones.begin(), denominaciones.end());
43         int n = denominaciones.size();
44
45         if (n < 2) return true; // con 0 o 1 denominacion, greedy
46         // trivialmente coincide con el
47         // optimo
48
49         int limite = denominaciones[n - 1] + denominaciones[n - 2];
50
51         for (int monto = 1; monto <= limite; monto++) {
52             int g = monedasGreedy(denominaciones, monto);
53             int d = monedasDP(denominaciones, monto);
54
55             if (g != d) return false; // encontro un contraejemplo
56             // -> definitivamente NO
57             // canonico
58         }
59
60         return true; // ningun contraejemplo hasta el limite ->
61         // canonico para CUALQUIER monto
62     } // O(limite^2 * k)
63 };

```

Ejemplo de funcionamiento:

```

1 vector<int> canonico = {1, 5, 10, 25};
2 cout << CoinChangeGreedyVerificado::esCanonico(canonico);
3 // true
4
5 vector<int> noCanonico = {1, 3, 4};
6 cout << CoinChangeGreedyVerificado::esCanonico(noCanonico);
7 // false
8 // contraejemplo: monto=6, greedy da 3 (4+1+1), optimo real es
9 // 2 (3+3)
10 cout << CoinChangeGreedyVerificado::monedasGreedy(canonico, 30)
11 ;
12 // 2 (25+5, correcto)
13 cout << CoinChangeGreedyVerificado::monedasGreedy(noCanonico,
14 6);
15 // 3 (INCORRECTO -- el optimo real es 2, confirmado por
16 monedasDP)

```

Nota: la verificación con el teorema del contraejemplo mínimo es **costosa** en sí misma (más que resolver un solo caso con DP) —su valor no es acelerar el cómputo, sino dar una **garantía de una sola vez**: una vez confirmado que un sistema es canónico, el greedy $O(k)$ puede usarse con confianza para **cualquier** monto futuro sin volver a verificar, en vez de correr DP $O(\text{monto} \cdot k)$ cada vez.

Regla práctica: en competencia, si el enunciado da denominaciones **fijas y conocidas** de antemano (no como parte de la entrada variable), vale la pena **verificar canonicidad una sola vez** (a mano o con este código) antes de decidir si programar el greedy simple o ir directo a DP. Si las denominaciones son **parte de la entrada** (cambian en cada caso de prueba) y no hay garantía de que sean canónicas, la opción segura por defecto es **siempre DP** — el greedy solo se justifica cuando hay certeza explícita (por el enunciado o por verificación previa) de que el sistema es canónico.

45.2 Egyptian Fraction

Idea: una fracción egipcia representa una fracción p/q (con $p < q$) como una **suma de fracciones unitarias distintas** ($1/a_1 + 1/a_2 + \dots$, cada a_i diferente). El greedy —conocido como el **algoritmo de Fibonacci-Sylvester**— consiste en tomar, en cada paso, la **mayor** fracción unitaria $1/\lceil q/p \rceil$ que **no exceda** p/q , restarla, y repetir sobre el resto —hasta que el numerador llegue a cero. La **garantía** de que este proceso siempre **termina** viene de una propiedad numérica: en cada paso, el numerador de la fracción restante **estrictamente disminuye** (nunca se estanca ni crece), así que el proceso converge en un número finito de pasos.

¿Por qué usarlo? Es un ejemplo de greedy donde la **prueba de terminación** (no solo de optimalidad) es la parte no trivial —a diferencia de Activity Selection o Fractional Knapsack, donde el greedy simplemente "recorre y termina" porque el arreglo es finito, aquí hay que demostrar que el algoritmo no entra en un **ciclo infinito** restando fracciones cada vez más pequeñas sin nunca llegar a cero. Es también un ejemplo de que "greedy" en competencia no siempre significa "óptimo" en un sentido de minimización clásica —el algoritmo de Fibonacci-Sylvester **no** necesariamente da la representación con **menos** términos posibles (ese es un problema NP-difícil en general), solo garantiza **una** representación válida y finita.

Casos de uso:

- Descomponer una fracción p/q dada en una suma de fracciones unitarias distintas, cuando el problema solo pide **alguna** representación válida (no la de mínimo número de términos).
- Problemas de "reparto justo" modelados históricamente con fracciones egipcias (el origen histórico del nombre: el sistema de notación fraccionaria del antiguo Egipto).
- Como ejemplo de análisis de **terminación** de un proceso greedy iterativo, un tipo de razonamiento distinto (pero igual de necesario) al de correctitud vía Exchange Argument.

Complejidad: $O(\log(\max(p, q)))$ en la práctica para la mayoría de fracciones (el numerador decrece rápidamente), aunque en el peor caso el número de términos puede crecer más —no hay una cota polinomial ajustada simple, pero el algoritmo siempre termina.

```

1 struct EgyptianFraction {
2     // READ: descompone p/q (con p < q, ambos positivos) en una
3     // lista
4     // de denominadores de fracciones unitarias distintas cuya
5     // suma
6     // es exactamente p/q.
7     static vector<long long> descomponer(long long p, long long q)
8     {
9         vector<long long> denominadores;
10
11         while (p != 0) {
12             // toma la fraccion unitaria MAS GRANDE que no exceda p/q:
13
14             // 1/ceil(q/p)
15             long long unidad = (q + p - 1) / p; // ceil(q/p)
16             denominadores.push_back(unidad);
17
18             // resta 1/unidad de p/q: p/q - 1/unidad =
19             // (p*unidad - q) / (q*unidad)
20             long long nuevoP = p * unidad - q;
21             long long nuevoQ = q * unidad;
22
23             long long g = __gcd(nuevoP, nuevoQ);
24             if (g > 0) {
25                 nuevoP /= g;
26                 nuevoQ /= g;
27             }
28
29             p = nuevoP;
30             q = nuevoQ;
31         }
32         return denominadores; // O(log(max(p,q)))
33     }
34     // en la practica
35 };

```

Ejemplo de funcionamiento:

```

1 auto denominadores = EgyptianFraction::descomponer(6, 14);
2
3 for (long long d : denominadores) cout << "1/" << d << " ";
4 // 1/3 1/11 1/231
5 // 6/14 = 3/7 -> 1/3 + 2/21 -> 1/3 + 1/11 + 1/231
6 // verificación: 1/3 + 1/11 + 1/231 = 77/231 + 21/231 + 1/231 =
7 // 99/231 = 3/7 = 6/14, correcto

```

Nota: este algoritmo garantiza terminar y dar una representación **válida**, pero puede generar denominadores que **crecen muy rápido** (como en el ejemplo, de 3 y 11 salta a 231) — si el problema requiere una representación con denominadores **acotados** o con el **mínimo** número de términos, este greedy simple no lo garantiza y hace falta una búsqueda más específica (a menudo con backtracking o cotas adicionales sobre el problema).

Regla práctica: usa Fibonacci-Sylvester cuando el problema solo pide **alguna** descomposición válida en fracciones egipcias, sin restricciones sobre el número de términos o el tamaño de los denominadores —es la opción más simple y garantizada. Si el enunciado pide explícitamente **minimizar** el número de términos o acotar los denominadores, ese es un problema estructuralmente distinto (y más difícil) que este greedy no resuelve por sí solo.

46 Greedy sobre Grafos (Referencias)

46.1 Por qué Kruskal, Prim y Dijkstra son Greedy

Idea: esta subsección no introduce un algoritmo nuevo —cierra la parte de Greedy conectando explícitamente

tres algoritmos que ya tienen su propio struct completo en la Parte de Grafos (Kruskal y Prim en *Minimum Spanning Tree*, Dijkstra en *Single Source Shortest Path*) con el marco conceptual desarrollado aquí: los tres toman, en cada paso, la decisión **localmente óptima** disponible en ese momento (la arista más barata que no forma ciclo; el nodo más cercano al conjunto ya construido) y **nunca la reconsideran** —exactamente la definición de greedy con la que arrancó esta parte, y los tres se justifican con la **misma técnica**: Exchange Argument.

¿Por qué usarlo? Reconocer la naturaleza greedy de estos tres algoritmos, aunque ya se conozcan como "algoritmos de grafos" con nombre propio, refuerza que Greedy **no es una categoría aislada** de problemas de intervalos y strings —es un **principio** transversal que también sostiene estructuras tan centrales como MST y caminos más cortos. Ver el mismo argumento de intercambio aplicado a dominios tan distintos (intervalos, árboles de expansión, caminos en grafos) es lo que convierte "demostrar un greedy" en una habilidad **general**, en vez de una serie de trucos memorizados por problema.

Casos de uso (el argumento de intercambio en cada uno):

- **Kruskal** (ver MST): si una solución óptima no usa la arista de **menor peso** entre las que no forman ciclo con lo ya elegido, el corte que separa sus dos extremos debe contener alguna arista del MST óptimo — esa arista pesa **al menos** tanto como la de Kruskal, así que **intercambiarlas** nunca empeora el costo total.
- **Prim** (ver MST): en cada paso, la arista más barata que conecta el árbol parcial con un nodo fuera de él **debe** pertenecer a **algún** MST óptimo — el mismo argumento de "corte mínimo" que justifica Kruskal, aplicado a un árbol que crece desde un solo nodo en vez de fusionar componentes.
- **Dijkstra** (ver SSSP): cuando se extrae el nodo con **menor** distancia tentativa del heap, esa distancia ya es **definitiva** —no puede existir un camino más corto todavía sin descubrir, porque cualquier camino alternativo tendría que pasar por un nodo **no procesado**, cuya distancia tentativa ya es **mayor o igual** (por eso Dijkstra **falla** con pesos negativos: esa garantía se rompe si una arista negativa puede "mejorar" un nodo ya marcado como definitivo).

Complejidad: no aplica una nueva —los tres mantienen la complejidad ya documentada en sus subsecciones originales ($O(E \log E)$ Kruskal, $O(E \log V)$ Prim, $O(E \log V)$ Dijkstra con heap). El punto de esta subsección es la **justificación** compartida, no una implementación nueva.

```

1 // No hay struct nuevo aquí -- el código de Kruskal, Prim y
2 // Dijkstra
3 // ya esta completo en sus respectivas subsecciones (MST y SSSP)
4 // Lo
5 // unico que se ilustra es el PATRON COMUN de decision greedy
6 // que
7 // comparten los tres, expresado como pseudocódigo unificado:
8
9 // KRUSKAL: ordena aristas por peso; para cada una, en orden,
10 // agregala SOLO SI no forma ciclo (DSU.unir devuelve
11 // true)
12 // -> decision local: "la mas barata disponible ahora

```

```
9 // mismo que no rompa la propiedad de arbol"
10
11 // PRIM: mantiene un heap de aristas frontera; extrae
12 // SIEMPRE la
13 // de menor peso que conecte con un nodo nuevo
14 // -> decision local: "la mas barata disponible ahora
15 // mismo que expanda el arbol actual"
16
17 // DIJKSTRA: mantiene un heap de distancias tentativas; extrae
18 // SIEMPRE el nodo de menor distancia y lo marca
19 // definitivo
20 // -> decision local: "el mas cercano disponible
21 // ahora
22 // mismo, nunca reconsiderado despues"
23
24 // las tres comparten la misma forma: extraer el MINIMO de una
25 // coleccion de candidatos, confirmar esa eleccion PARA SIEMPRE,
26 // y
27 // actualizar los candidatos restantes en funcion de lo recien
28 // confirmado -- es el mismo esqueleto de "Greedy con Heap" ya
29 // visto
30 // en Huffman Coding y Connect Ropes, aplicado a grafos en vez
31 // de a
32 // simbolos o cuerdas.
```

Ejemplo de funcionamiento: no aplica un ejemplo nuevo de código — los ejemplos completos con entrada/salida ya están en las subsecciones de Kruskal, Prim y Dijkstra (Parte de Grafos). Vale la pena, sin embargo, notar el paralelo directo: en Kruskal y Prim el "corte" que garantiza la corrección es un corte de **grafo** (separar nodos en dos lados); en Dijkstra, el "corte" es **entre nodos ya procesados y no procesados** — en ambos casos, la prueba dice "cualquier alternativa que cruce este corte pesa al menos tanto como mi elección greedy".

Regla práctica: cuando aparezca un problema nuevo que "se sienta" como Kruskal, Prim o Dijkstra pero con una variación (pesos distintos, restricciones extra), la primera pregunta útil es "¿el argumento de corte mínimo (o de nodo más cercano) sigue siendo válido con esta variación?" —si la variación introduce algo que rompe esa garantía (como pesos negativos rompen Dijkstra), es señal de que hace falta otro algoritmo (Bellman-Ford en ese caso), no un ajuste menor al greedy original.

Parte VIII Ad-hoc y Simulación

46.2 ¿Qué algoritmo usar?

Cuando el ejercicio diga / pida	Usa	Pista rápida
"Ejecuta este proceso paso a paso y dime el estado final" (robot, comandos, juego)	Plantilla de simulación	Estado, paso, parada y tope de pasos. $O(T \cdot C)$.
Todos los elementos cambian "al mismo tiempo" en cada paso	Doble buffer	Calcula sobre una copia; nunca actualices en sitio.
"Estado tras 10^{18} pasos", la regla se repite siempre igual	Detección de ciclos (Floyd / estado repetido)	$x_k = x_{\mu + ((k - \mu) \bmod \lambda)}$.
"¿El proceso entra en un bucle?", duplicado en arreglo de $n + 1$ valores, ciclo en lista enlazada	Floyd (tortuga y liebre)	$O(1)$ de memoria.
"Círculo donde se elimina cada k -ésimo", "¿quién sobrevive?"	Josephus	$J(n) = (J(n - 1) + k) \bmod n$. Respuesta base 1: suma 1.
Josephus pidiendo el orden de eliminación	Fenwick con k -ésimo elemento	$O(n \log n)$.
Josephus con $k = 2$ y n enorme	Fórmula cerrada	$2(n - 2^{\lceil \log_2 n \rceil})$. $O(\log n)$.
Clientes, cajeros, procesos, alarmas: mucho tiempo y pocos sucesos	Simulación por eventos	Min-heap de eventos. Define el desempate de empates.
"Game of Life", regla local sobre una grilla, k generaciones	Autómata celular	Dos grillas. k enorme: detecta ciclo sobre el tablero.
Autómata lineal (Regla 90) con k gigante	Salto de 2^m pasos	$O(n \log k)$.
"Programa en un lenguaje inventado", registros, saltos, Brainfuck	Intérprete de instrucciones	Parsea una vez; pon tope de pasos.
"Imprime en espiral", "llena la matriz en espiral"	Espiral por fronteras	Cuatro fronteras y dos guardas. $O(nm)$.
"Por diagonales", "zigzag", "tablero en serpiente"	Clave $i + j$ o $i - j$	Agrupar por clave en una pasada.
"Rota la matriz 90° ", "¿son la misma figura salvo giro?"	Transponer y reflejar	Rotar: transpone y invierte filas. Canónica: menor de 8.
Muchas rotaciones y espejos sobre una matriz grande	Marcador 2×2	Compón en una simetría. $O(q + nm)$.
"Candy Crush", "cae la arena", "Tetris"	Gravedad con puntero de escritura	Marca todo, elimina, gravedad, repite.
Gravedad en otra dirección	Rotar el tablero	Evita duplicar código.
"Vecinos de una celda", buscaminas, hielo, líneas de k	Arreglos dx/dy	Horario desde arriba. Comprueba límites antes de acceder.

Table 24: Simulación y matrices: qué usar según lo que pide el ejercicio.

Cuando el ejercicio diga / pida	Usa	Pista rápida
"¿Es bisiestro?", "días de un mes", "día del año"	Regla gregoriana	Orden: 400, 100, 4. $O(1)$.
"¿Qué día de la semana fue?"	Sakamoto (o Zeller)	Enero y febrero son meses del año anterior.
"Días entre dos fechas", "fecha tras sumar k días"	Fecha a número de día	Convierte, opera y vuelve. $O(1)$.
"Segundos a HH:MM:SS", formato 12h/24h, sumar tiempo	Segundos desde medianoche	Hora: módulo 86400. Duración: no envolver.
"Imprime el resultado exacto" con cientos de dígitos	BigInt (base 10^9)	Producto $O(nm)$. Si piden módulo, no hace falta.
"Valor exacto de una razón", comparar o sumar fracciones	Fracciones exactas	Forma canónica y <code>__int128</code> . Pendientes: producto cruzado.
"Evalúa la expresión" con paréntesis, precedencia y tokens pegados	Tokenizador + Shunting-Yard	Convierte a post-fija una vez y reutiliza.
Funciones de aridad variable, asignaciones, condicionales, buenos errores	Descenso recursivo	Una función por nivel; ciclo izquierda, recursión derecha.

Table 25: Fechas, aritmética y parsing: qué usar según lo que pide el ejercicio.

Cuando el ejercicio diga / pida	Usa	Pista rápida
Valores hasta 10^9 pero pocos, solo importa el orden, consultas offline	Compresión de coordenadas	<code>sort</code> , <code>unique</code> , <code>lower_bound</code> .
"¿Es posible transformar A en B ?", "imprime -1 si no se puede"	Invariantes y paridad	Suma, XOR, mcd, módulo m , coloración.
"¿Se puede resolver este puzzle deslizante?", "paridad de intercambios"	Paridad de permutación	$n - \text{ciclos}$. $O(n)$.
"Construye una matriz / permutación / cuadrado mágico con tal propiedad"	Algoritmo constructivo	Casos pequeños, patrón, demostración, verificador.
Cuadrado mágico de orden n	Siamés, bloques 4×4 o Strachey	Según $n \bmod 4$. No existe para $n = 2$.
Pasar de (a, b) a (c, d) con $(x + y, y)$ o $(x, x + y)$	Trabajar hacia atrás	Predecesor único; acelera con divisiones.
"Color final tras pintar rangos", eliminaciones offline, juegos	Reverse thinking	Pinta en reversa con "siguiente libre"; invierte el tiempo.
"Suma de los mínimos (o máximos) de todos los subarreglos"	Contribución con pila monótona	Empate: $\geq$ a un lado y $>$ al otro. $O(n)$.
"Suma sobre todos los subarreglos"	Contribución $(i + 1)(n - i)$	Sin pila. $O(n)$.
La fórmula no es obvia	Fuerza bruta para descubrir patrones	Tabla de n pequeño, diferencias, verifica.
Solución con Wrong Answer sin pista	Stress testing	Generador pequeño, fuerza bruta y comparación.
"Interactivo", consultas al juez	Problema interactivo	<code>flush</code> tras cada línea.

Table 26: Técnicas ad-hoc y práctica: qué usar según lo que pide el ejercicio.

47 Simulación Directa

47.1 Plantilla de Simulación (Estado, Paso y Parada)

Idea: un problema de simulación pide ejecutar literalmente un proceso descrito en el enunciado y reportar qué pasa al final (o en algún momento). Casi todos se

ordenan con tres piezas: (1) el **estado**, es decir, **todo** lo necesario para determinar el futuro y nada más (posición, dirección, contenido de una grilla, índice de la siguiente instrucción, etc.); (2) el **paso**, una función que toma el estado actual y devuelve el siguiente; (3) la **parada**, la condición que termina la simulación, junto con un **tope de pasos** de seguridad para no quedar en un bucle infinito por un error de lectura del enunciado. El ciclo es siempre el mismo: mientras no se cumpla la parada, aplicar el paso. Separar las tres piezas evita mezclar lógica del problema con control del bucle, y es donde se ganan (o se pierden) la mayoría de los errores.

¿Por qué usarlo? Los problemas de simulación rara vez fallan por dificultad algorítmica: fallan por **detalles**. Los tres más comunes son: (a) actualizar el estado **en sitio** cuando el enunciado pide que todos los elementos cambien **a la vez** (el segundo elemento ya ve el valor nuevo del primero); (b) errores de **off-by-one** en la condición de parada; (c) olvidar parte del estado (por ejemplo, la dirección del robot además de su posición). Definir el estado explícitamente como un **struct** y escribir el paso como una función pura sobre él hace estos errores fáciles de ver.

Casos de uso:

- Ejecutar una secuencia de instrucciones (robot en una grilla, comandos de un editor, movimientos de un juego) y reportar el estado final.
- Simular un proceso hasta que se cumpla una condición ("¿cuántos pasos hasta que regrese al origen?") reportando el número de pasos.
- Ser la **solución de fuerza bruta** de un problema más difícil: una simulación directa, lenta pero obviamente correcta, es el mejor comparador para probar una solución rápida (ver Stress Testing en esta misma parte).
- Base de la siguiente subsección: cuando el número de pasos pedido es enorme, se necesita saber **cuándo el estado se repite**.

Complejidad: $O(T \cdot C)$, donde T es el número de pasos y C el costo de un paso. Sirve mientras $T \cdot C$ quepa en unos 10^7 – 10^8 operaciones; si el enunciado pide $T \sim 10^9$ o más, simular literalmente ya no alcanza (ver Simulación con Detección de Ciclos).

```

1  template <class Estado>
2  struct Simulador {
3      function<Estado(const Estado&)> paso;           // estado -> estado
4      function<bool(const Estado&)> parar;           // true si ya hay
5      long long maxPasos;                             // tope de seguridad
6      vector<Estado> traza;                             // historial (
7      bool guardarTraza;
8
9      // CREATE: guarda las tres piezas de la simulación.
10     Simulador(function<Estado(const Estado&)> paso_,
11               function<bool(const Estado&)> parar_,
12               long long maxPasos_, bool guardarTraza_ = false)
13         : paso(paso_), parar(parar_), maxPasos(maxPasos_),
14           guardarTraza(guardarTraza_) {}           // O(1)
15
16     // READ: corre la simulación desde 'inicial'. Devuelve
17     // {pasos ejecutados, estado final}. Si se alcanza maxPasos
18     // sin
19     // que parar() sea true, devuelve con pasos == maxPasos (el
20     // llamador debe revisar si eso indica un bucle infinito).

```

```

20 pair<long long, Estado> ejecutar(Estado inicial) {
21     Estado actual = inicial;
22     long long t = 0;
23     if (guardarTraza) traza.push_back(actual);
24
25     while (t < maxPasos && !parar(actual)) {
26         actual = paso(actual);
27         t++;
28         if (guardarTraza) traza.push_back(actual);
29     }
30
31     return {t, actual};
32 } // O(T * costo del
33     paso)
34
35 // -----
36 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
37 // -----
38 // 1. ACTUALIZACION SIMULTANEA (todos cambian a la vez): si
39 // el
40 // paso modifica varios elementos y el enunciado dice que
41 // todos cambian "al mismo tiempo", NUNCA se actualiza en
42 // sitio: el elemento i veria el valor NUEVO del elemento
43 // i-1.
44 // Se calcula sobre una copia (doble buffer):
45 // vector<int> siguiente = actual; // copia
46 // for (int i = 0; i < n; i++)
47 //     siguiente[i] = regla(actual[i - 1], actual[i],
48 //         actual[i + 1]);
49 // actual = siguiente; // se publica
50 // al final
51 // Con Estado = vector<int> esto ya ocurre solo, porque el
52 // paso
53 // recibe una referencia constante y devuelve un estado
54 // nuevo.
55 // 2. ACTUALIZACION SECUENCIAL (uno tras otro): si el
56 // enunciado
57 // dice que cada elemento reacciona al estado ya
58 // modificado por
59 // el anterior, SI se actualiza en sitio, en el orden
60 // pedido.
61 // Leer con cuidado que orden exacto pide el enunciado.
62 // 3. SIMULACION POR TURNOS (rondas con cola): cuando el
63 // estado es
64 // "quien juega ahora", una cola modela el orden:
65 // queue<int> turnos;
66 // while (!terminado()) {
67 //     int j = turnos.front(); turnos.pop();
68 //     jugar(j);
69 //     if (sigueEnJuego(j)) turnos.push(j); // vuelve
70 // al final
71 // }
72 // 4. VER LOS PRIMEROS PASOS (depuracion): con guardarTraza =
73 // true
74 // se imprime el inicio de la traza y se compara a mano
75 // con el
76 // ejemplo del enunciado:
77 // for (int i = 0; i < min<int>(10, sim.traza.size()); i
78 // ++)
79 //     imprimir(sim.traza[i]);
80 // Si el ejemplo del enunciado no coincide en los primeros
81 // pasos, el error esta en el paso, no en la parada.
82 // 5. DETECTAR BUCLE INFINITO: si ejecutar() devuelve
83 // pasos == maxPasos, o bien el tope era muy chico, o bien
84 // la
85 // condicion de parada nunca se cumple. No confiar en que
86 // la
87 // salida "se ve bien": comparar pasos con maxPasos.
88 // 6. PASOS ENORMES (T de 10^9 o mas): no simular
89 // literalmente.
90 // Si el estado toma pocos valores distintos, se repite y
91 // se
92 // puede saltar: ver Simulacion con Deteccion de Ciclos.
93 // Si el
94 // paso es una transformacion que se puede componer,
95 // tambien se
96 // puede aplicar por duplicacion (elevar la transformacion
97 // a la
98 // potencia T en O(log T) composiciones).
99 // 7. TIEMPO CONTINUO O EVENTOS ESPARCIDOS: si casi nunca

```

```

pasa nada
// entre eventos (y simular cada unidad de tiempo seria un
// desperdicio), se salta directamente de evento en evento
con
// una cola de prioridad: ver Simulacion por Eventos.
};

```

Ejemplo de funcionamiento:

```

1 // Un robot recibe instrucciones: 'F' avanza, 'R' gira a la
2 // derecha,
3 // 'L' gira a la izquierda. Estado = posicion + direccion +
4 // indice
5 // de la siguiente instruccion (dir: 0=N, 1=E, 2=S, 3=O).
6
7 struct EstadoRobot { int x, y, dir, i; };
8
9 string prog = "FFRFFRFFRFFR";
10 int dx[4] = {0, 1, 0, -1}, dy[4] = {1, 0, -1, 0};
11
12 Simulador<EstadoRobot> sim(
13     [&](const EstadoRobot& e) { // paso
14         EstadoRobot n = e;
15         if (prog[e.i] == 'F') { n.x += dx[e.dir]; n.y += dy[e.dir]; }
16         else if (prog[e.i] == 'R') n.dir = (e.dir + 1) % 4;
17         else n.dir = (e.dir + 3) % 4;
18         n.i++;
19         return n;
20     },
21     [&](const EstadoRobot& e) { // parada
22         return e.i == (int)prog.size();
23     },
24     1000000); // tope de pasos
25
26 auto res = sim.ejecutar({0, 0, 0, 0});
27
28 cout << res.first << " (" << res.second.x << ", " << res.second.
29     y
30     << ") dir=" << res.second.dir;
31 // 12 (0,0) dir=0
32 // (recorre un cuadrado de lado 2 y termina en el origen
33 // mirando al norte)

```

Nota: el estado del ejemplo incluye el índice i de la instrucción actual aunque no sea parte de "la posición del robot": como el paso lee `prog[e.i]`, ese índice **sí** es parte de lo que determina el futuro. Regla para saber qué meter en el estado: si dos estados con los mismos campos pueden evolucionar distinto, falta algo en el estado.

Regla práctica: antes de escribir código, escribe en una línea cada pieza: "estado = ..., paso = ..., parar cuando ...". Pregúntate en particular si las actualizaciones del paso son **simultáneas** o **secuenciales** según el enunciado, y prueba a mano los primeros 2 o 3 pasos con el ejemplo dado antes de correr el caso completo.

47.2 Simulación con Detección de Ciclos (Floyd y Estado Repetido)

Idea: si el estado solo puede tomar un número **finito** de valores distintos y el paso depende únicamente del estado actual, la secuencia $x_0, x_1 = f(x_0), x_2 = f(x_1), \dots$ termina necesariamente por **repetir** un estado (principio del palomar). Desde ese momento se repite para siempre: la secuencia tiene una **cola** de longitud μ (estados que se visitan una sola vez) seguida de un **ciclo** de longitud λ . Una vez conocidos μ y λ , el estado tras k pasos (con k tan grande como 10^{18}) se obtiene reduciendo el índice:

$$x_k = x_{\mu + ((k - \mu) \bmod \lambda)} \quad \text{si } k \geq \mu$$

y simulando solo $\mu + \lambda$ pasos como máximo. Hay dos formas de encontrar μ y λ : **Floyd** (tortuga y liebre), y **Estado Repetido** (guardar cada estado visto con el paso en que apareció).

¿Por qué usarlo? Cuando el enunciado pide "el estado tras 10^{18} pasos", simular literalmente es imposible,

pero casi siempre hay un ciclo corto escondido. Las dos técnicas se complementan: **Floyd** usa $O(1)$ de memoria y solo necesita poder comparar dos estados con `==`, ideal cuando el estado es un entero o algo pequeño; **Estado Repetido** usa memoria $O(\mu + \lambda)$ pero es más simple, sirve con cualquier estado que se pueda guardar en un `map` (strings, vectores, tuplas), y además deja la **historia completa** guardada, lo que permite responder muchas consultas de "¿qué estado hay en el paso k ?" en $O(1)$ cada una.

Casos de uso:

- Estado tras k pasos de una regla que se aplica muchísimas veces (baraja que se mezcla siempre igual, configuración que rota, autómatas con reglas fijas, números tipo "feliz").
- Detectar si una secuencia **entra en un bucle** o termina (por ejemplo, "¿este proceso llega alguna vez a 1?").
- Detectar un ciclo en una lista enlazada o en un grafo donde cada nodo tiene una sola arista de salida (grafo funcional).
- Encontrar el elemento duplicado en un arreglo de $n + 1$ valores en $[1, n]$ tratando `a[i]` como puntero al siguiente índice: el duplicado es la **entrada** del ciclo (ver modificación 2).
- Calcular sumas o costos acumulados en k pasos gigantes, separando cola, ciclos completos y resto (modificación 4).

Complejidad: Floyd: $O(\mu + \lambda)$ evaluaciones de f y $O(1)$ de memoria. Estado Repetido: $O((\mu + \lambda) \log(\mu + \lambda))$ con `map` ($O(\mu + \lambda)$ con `unordered_map`) y $O(\mu + \lambda)$ de memoria. En ambos, obtener x_k cuesta $O(\mu + \lambda)$ con Floyd (hay que reiterar), y $O(1)$ con Estado Repetido (ya está la historia guardada).

```

1 // ----- Variante 1: Floyd (tortuga y liebre), estado =
  entero -----
2 struct DeteccionCiclo {
3     long long mu, lambda; // longitud de la cola
  y del ciclo
4     function<long long(long long)> f; // el paso: x -> f(x)
5     long long x0; // estado inicial
6
7     // CREATE: encuentra mu y lambda con el algoritmo de Floyd.
8     DeteccionCiclo(function<long long(long long)> f_, long long
  x0_) {
9         f = f_; x0 = x0_;
10
11         // fase 1: la liebre avanza 2 pasos por cada paso de la
12         // tortuga; se encuentran DENTRO del ciclo
13         long long tortuga = f(x0), liebre = f(f(x0));
14         while (tortuga != liebre) {
15             tortuga = f(tortuga);
16             liebre = f(f(liebre));
17         }
18
19         // fase 2: la tortuga vuelve al inicio; ahora ambas avanzan
20         // de a 1 paso y se encuentran justo en la ENTRADA del
  ciclo
21         mu = 0;
22         tortuga = x0;
23         while (tortuga != liebre) {
24             tortuga = f(tortuga);
25             liebre = f(liebre);
26             mu++;
27         }
28
29         // fase 3: se mide el largo del ciclo dando una vuelta
  completa
30         lambda = 1;
31         liebre = f(tortuga);

```

```

32 while (tortuga != liebre) {
33     liebre = f(liebre);
34     lambda++;
35 }
36 // O(mu + lambda)
37
38 // READ: estado tras k pasos (k puede ser enorme).
39 long long valorEnPaso(long long k) {
40     long long pasos = k;
41     if (k >= mu) pasos = mu + (k - mu) % lambda; // reduce k
42
43     long long x = x0;
44     for (long long i = 0; i < pasos; i++) x = f(x);
45     return x;
46 // O(mu + lambda)
47 };
48
49 // ----- Variante 2: Estado Repetido, estado = cualquier
  tipo -----
50 // Requiere que Estado tenga operator< (para usar std::map).
  Con
51 // unordered_map hace falta además un hash para Estado.
52 template <class Estado>
53 struct CicloPorEstadoRepetido {
54     long long mu, lambda;
55     vector<Estado> historia; // historia[t] = estado
  en el paso t
56
57     // CREATE: simula hasta ver un estado repetido, guardando en
  que
58     // paso se vio cada estado por primera vez.
59     CicloPorEstadoRepetido(Estado inicial,
60     function<Estado(const Estado&> paso) {
61         map<Estado, long long> visto;
62         Estado actual = inicial;
63         long long t = 0;
64
65         while (!visto.count(actual)) {
66             visto[actual] = t;
67             historia.push_back(actual);
68             actual = paso(actual);
69             t++;
70         }
71
72         mu = visto[actual]; // paso en que entro al
  ciclo
73         lambda = t - mu; // largo del ciclo
74     } // O((mu+lambda)
  log)
75
76     // READ: estado tras k pasos (k puede ser enorme).
77     Estado estadoEnPaso(long long k) {
78         if (k < (long long)historia.size()) return historia[k];
79         return historia[mu + (k - mu) % lambda];
80     } // O(1)
81
82 // -----
83 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
84 // -----
85
86 // 1. ALGORITMO DE BRENT (alternativa a Floyd): en vez de
  mover la
87 // liebre al doble de velocidad, se compara contra un "
  punto de
88 // control" que se reubica en potencias de 2. Hace en
  promedio
89 // menos evaluaciones de f, lo que importa si f es costosa:
90
91 // long long potencia = 1, lam = 1;
92 // long long tortuga = x0, liebre = f(x0);
93 // while (tortuga != liebre) {
94 //     if (potencia == lam) { // reubica el
  control
95         // tortuga = liebre;
96         // potencia *= 2;
97         // lam = 0;
98         // }
99         // liebre = f(liebre);
100         // lam++;
101     }
102 // // lam es lambda; mu se halla avanzando dos punteros
  con
103 // // separacion lam desde x0, igual que la fase 2 de
  Floyd.
104
105 // 2. ELEMENTO DUPLICADO CON FLOYD: arreglo a de n+1 valores
  en
106 // [1,n]. Se ve como un grafo funcional i -> a[i]; como
  hay un
107 // valor repetido, dos índices distintos apuntan al mismo

```

```

108 // destino y ese destino es la ENTRADA del ciclo (o sea,
109 // mu):
110 //
111 // int tor = a[0], lie = a[a[0]];
112 // while (tor != lie) { tor = a[tor]; lie = a[lie]; }
113 // tor = 0;
114 // while (tor != lie) { tor = a[tor]; lie = a[lie]; }
115 // tor es el valor duplicado; O(n) tiempo y O(1)
116 // memoria, sin
117 // // modificar el arreglo
118
119 // 3. LISTA ENLAZADA CON CICLO: el mismo Floyd, con punteros:
120 //
121 // Nodo* tor = cabeza; Nodo* lie = cabeza;
122 // while (lie && lie->sig) {
123 //     tor = tor->sig;
124 //     lie = lie->sig->sig;
125 //     if (tor == lie) return true; // hay ciclo
126 // }
127 // return false; // la lista
128 // termino
129
130 // 4. SUMAS ACUMULADAS EN k PASOS: si cada paso aporta un
131 // valor
132 // u(x) (puntos ganados, costo pagado) y se pide el total
133 // tras k
134 // pasos, con historia[] guardada se precaculan sumas de
135 // prefijo pref[t] = suma de u(historia[0..t-1]) y:
136 //
137 // if (k <= mu + lambda) return pref[k];
138 // long long enElCiclo = pref[mu + lambda] - pref[mu];
139 // long long vueltas = (k - mu) / lambda;
140 // long long resto = (k - mu) % lambda;
141 // return pref[mu] + vueltas * enElCiclo + (pref[mu +
142 //     resto] - pref[mu]);
143 //
144 // Cuidado con el overflow: vueltas * enElCiclo puede
145 // pasar de
146 // long long si k es muy grande (usar __int128 o modulo).
147
148 // 5. MUCHOS PUNTEROS FUNCIONALES A LA VEZ (grafo funcional
149 // con n
150 // nodos y muchas consultas "donde estoy tras k pasos si
151 // empiezo
152 // en u"): en vez de buscar un ciclo por consulta, se
153 // precacula la tabla de saltos de potencias de 2, igual
154 // que en
155 // Binary Lifting / k-esimo Ancestro: up[j][u] = nodo tras
156 // 2^j
157 // pasos desde u. Cada consulta cuesta O(log k).
158
159 // 6. TRAMPA: EL PASO DEPENDE DEL TIEMPO. Todo lo anterior
160 // supone
161 // que el paso depende SOLO del estado. Si el enunciado
162 // dice
163 // "en el paso t se hace X" (por ejemplo, un
164 // comportamiento que
165 // alterna entre pares e impares), el tiempo forma parte
166 // del
167 // estado. Se corrige metiendo t (o t mod periodo) dentro
168 // del
169 // Estado; si no, se detecta un "ciclo" falso y la
170 // respuesta sale
171 // mal.
172
173 // 7. TRAMPA: ESPACIO DE ESTADOS ENORME. Detectar ciclos solo
174 // sirve
175 // si el ciclo aparece pronto (mu + lambda cabe en tiempo
176 // y
177 // memoria). Con estados de 2^60 valores posibles el ciclo
178 // puede
179 // ser inalcanzable en la practica; ahí hay que buscar una
180 // formula, duplicacion, o estructura especial del
181 // problema.
182
183 };

```

Ejemplo de funcionamiento:

```

1 // Floyd con f(x) = (x*x + 1) % 10, empezando en x0 = 3:
2 // 3, 0, 1, 2, 5, 6, 7, 0, 1, 2, 5, 6, 7, 0, ...
3 // el 3 solo aparece una vez (cola de largo 1) y luego el ciclo
4 // 0, 1, 2, 5, 6, 7 se repite (largo 6)
5
6 DeteccionCiclo dc([](long long x) { return (x * x + 1) % 10; },
7 3);
8
9 cout << dc.mu << " " << dc.lambda;
10 // 1 6
11
12 cout << dc.valorEnPaso(2) << " " << dc.valorEnPaso
13     (10000000000000000000LL);
14 // 1 5 (x_2 = 1; x_(10^18) = x_4 = 5, sin simular 10^18 pasos
15 )

```

```

13 // Estado Repetido con strings: el paso rota "abcde" una
14 // posicion a la
15 // izquierda e intercambia luego los dos primeros caracteres.
16
17 CicloPorEstadoRepetido<string> cr("abcde", [](const string& s)
18 {
19     string t = s.substr(1) + s[0];
20     swap(t[0], t[1]);
21     return t;
22 });
23
24 cout << cr.mu << " " << cr.lambda;
25 // 0 4 (vuelve a "abcde" tras 4 pasos: abcde cbdea dbeac
26 //      ebacd abcde)
27
28 cout << cr.estadoEnPaso(100000000000000LL);
29 // abcde (10^12 es multiplo de 4, cae justo al inicio del
30 //      ciclo)

```

Nota: en el ejemplo de Estado Repetido $\mu = 0$ porque el estado inicial **forma parte** del ciclo (la secuencia regresa exactamente a él); en el ejemplo de Floyd $\mu = 1$ porque el estado inicial (3) nunca vuelve a aparecer. Ambos casos son normales, y la fórmula $\mu + ((k - \mu) \bmod \lambda)$ los maneja igual. Lo que no debe olvidarse es el `if (k >= mu)`: para $k < \mu$ el índice **no** se reduce.

Regla práctica: si el enunciado pide el estado tras un número gigantesco de pasos (10^9 , 10^{12} , 10^{18}) y el estado puede tomar pocos valores distintos, la respuesta casi siempre es "hay un ciclo": usa Estado Repetido si el estado es complejo o si vas a responder varias consultas, y Floyd si el estado es un entero y la memoria importa. Antes de codificar, confirma dos cosas: que el paso depende **solo** del estado (no del tiempo), y que el ciclo aparece lo bastante rápido como para poder encontrarlo simulando.

47.3 Problema de Josephus

Idea: n personas numeradas de 0 a $n - 1$ forman un círculo, y partiendo de la persona 0 se elimina a la k -ésima, luego se sigue contando desde la siguiente y se vuelve a eliminar a la k -ésima, hasta que queda una sola. Se pide quién **sobrevive** (o en qué **orden** se eliminan). La forma más directa de simularlo es una cola: se rotan $k - 1$ personas al final y se saca a la siguiente, en $O(nk)$. Hay una recurrencia mucho más rápida. Sea $J(n)$ la posición (base 0) del sobreviviente con n personas. Tras eliminar a la persona en la posición $(k - 1) \bmod n$, la cuenta se reinicia desde la siguiente, y queda un círculo de $n - 1$ personas donde la posición i de ese círculo corresponde a la posición $(i + k) \bmod n$ del círculo original. Por eso:

$$J(1) = 0, \quad J(n) = (J(n - 1) + k) \bmod n$$

que se evalúa en $O(n)$ con un solo ciclo.

¿Por qué usarlo? Cada método responde una pregunta distinta y conviene elegir bien: la **recurrencia** $O(n)$ da solo al sobreviviente; para el **orden completo** de eliminación hace falta una estructura que encuentre "la persona en la posición p entre las que quedan" y la borre, que es exactamente lo que hace un Fenwick Tree con búsqueda del k -ésimo elemento ($O(n \log n)$); para $k = 2$ existe una **fórmula cerrada** en $O(\log n)$; y si n es gigantesco pero k es chico, hay una variante que salta bloques y corre en $O(k \log n)$. Reconocer cuál se pide evita simular $O(nk)$ cuando n es del orden de 10^6 o más.

Casos de uso:

- Quién sobrevive en un círculo donde se elimina cada k -ésimo (el problema clásico, con n hasta 10^6 – 10^7 o más con la recurrencia).
- El **orden completo** de eliminación, o quién es el m -ésimo eliminado, con el método del Fenwick.
- Casos con $k = 2$ y n muy grande (hasta 10^{18}), donde la fórmula cerrada resuelve en tiempo logarítmico.
- Problemas disfrazados de juego de niños, de fila que se reduce, o de "elimina cada segundo elemento hasta que quede uno": el modelo es el mismo.

Complejidad: recurrencia $O(n)$ tiempo y $O(1)$ memoria; orden completo con Fenwick $O(n \log n)$; $k = 2$ en $O(\log n)$ (o $O(1)$ con `clz`); k chico y n enorme en $O(k \log n)$; simulación con cola $O(nk)$, que sirve como referencia para verificar.

```

1 struct Josephus {
2     // READ: posicion (base 0) del sobreviviente con n personas y
3     // salto k, usando la recurrencia  $J(n) = (J(n-1) + k) \% n$ .
4     static int sobreviviente(int n, int k) {
5         int r = 0; //  $J(1) = 0$ 
6         for (int i = 2; i <= n; i++)
7             r = (r + k) % i;
8         return r;
9     } //  $O(n)$ 
10
11     // Fenwick de conteos (1 = persona viva) con busqueda del
12     // k-esimo elemento vivo por descenso binario.
13     struct Fenwick {
14         int n, LOG;
15         vector<int> t;
16
17         Fenwick(int n_) : n(n_), t(n_ + 1, 0) {
18             for (int i = 1; i <= n; i++) { // construccion
19                 t[i] += 1; // todos empiezan
20                 vivos
21                 int j = i + (i & -i);
22                 if (j <= n) t[j] += t[i];
23             }
24             LOG = 1;
25             while ((1 << LOG) <= n) LOG++;
26         }
27
28         void quitar(int i) { // i en base 0
29             for (i++; i <= n; i += i & -i) t[i]--;
30         }
31
32         int kesimo(int k) { // k-esimo vivo, k
33             // >= 1;
34             int pos = 0; // devuelve indice
35             base 0
36             for (int j = LOG; j >= 0; j--) {
37                 int sig = pos + (1 << j);
38                 if (sig <= n && t[sig] < k) {
39                     pos = sig;
40                     k -= t[sig];
41                 }
42             }
43             return pos;
44         }
45     };
46
47     // READ: orden completo de eliminacion (el ultimo es el
48     // sobreviviente).
49     static vector<int> ordenEliminacion(int n, int k) {
50         Fenwick fw(n);
51         vector<int> orden;
52         int pos = 0, restantes = n; // pos: posicion entre los
53         vivos
54         while (restantes > 0) {
55             pos = (pos + k - 1) % restantes;
56             int persona = fw.kesimo(pos + 1); // quien esta en '
57             pos'
58             orden.push_back(persona);
59             fw.quitar(persona);
60             restantes--;
61             // pos no avanza: al borrar, la siguiente persona ocupa
62             la

```

```

58     // misma posicion entre los vivos
59 }
60
61 return orden;
62 } //  $O(n \log n)$ 
63
64 // READ: caso  $k = 2$  con  $n$  enorme. El sobreviviente (base 0)
65 // es
66 //  $2 * (n - p)$ , con  $p$  la mayor potencia de 2 menor o igual
67 // que  $n$ .
68 static long long sobrevivienteK2(long long n) {
69     long long p = 1;
70     while (p * 2 <= n) p *= 2;
71     return 2 * (n - p);
72 } //  $O(\log n)$ 
73
74 // READ: simulacion directa con cola, para verificar (stress
75 // testing) o para  $n$  y  $k$  muy chicos.
76 static vector<int> simulacionCola(int n, int k) {
77     queue<int> q;
78     for (int i = 0; i < n; i++) q.push(i);
79     vector<int> orden;
80
81     while (!q.empty()) {
82         for (int j = 1; j < k; j++) { // rota k-1 personas
83             q.push(q.front());
84             q.pop();
85         }
86         orden.push_back(q.front()); // la k-esima sale
87         q.pop();
88     }
89
90     return orden;
91 } //  $O(n * k)$ 
92
93 //
94 // -----
95 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
96 // -----
97
98 // 1. INDICES EN BASE 1: casi todos los enunciados numeran
99 // desde
100 // 1. Todo lo de arriba trabaja en base 0, asi que la
101 // respuesta
102 // final es sobreviviente(n, k) + 1 (y lo mismo para cada
103 // elemento de ordenEliminacion). Es el error mas comun.
104
105 // 2. n ENORME Y k CHICO (por ejemplo  $n = 10^{12}$ ,  $k = 5$ ): la
106 // recurrencia  $O(n)$  es lenta, pero se puede eliminar k-1
107 // personas de cada k en un solo salto:
108
109 // long long josephusChico(long long n, long long k) {
110 //     if (n == 1) return 0;
111 //     if (k == 1) return n - 1;
112 //     if (k > n) return (josephusChico(n - 1, k) + k) %
113 //     n;
114 //     long long cnt = n / k; //
115 //     eliminados en // esta
116 //     vuelta
117 //     long long res = josephusChico(n - cnt, k);
118 //     res -= n % k;
119 //     if (res < 0) res += n;
120 //     else res += res / (k - 1);
121 //     return res;
122 // } //  $O(k \log n)$ 
123
124 // 3. SALTO QUE CAMBIA EN CADA RONDA (paso[i] distinto en
125 // cada
126 // eliminacion): la recurrencia ya no aplica directo, pero
127 // el
128 // metodo del Fenwick lo maneja sin cambios, usando el
129 // salto
130 // de la ronda actual:
131 //
132 // pos = (pos + paso[ronda] - 1) % restantes;
133
134 // 4. CONTAR EN SENTIDO CONTRARIO O ALTERNANDO: con Fenwick
135 // basta
136 // restar en vez de sumar, cuidando que el modulo sea no
137 // negativo:
138 //
139 // pos = ((pos - k) % restantes + restantes) % restantes;
140
141 // 5. ALTERNATIVA CON ARBOL DE ORDEN (GNU pbds), mismo  $O(n \log n)$ :
142 //
143 // #include <ext/pb_ds/assoc_container.hpp>
144 // #include <ext/pb_ds/tree_policy.hpp>
145 // using namespace __gnu_pbds;
146 // tree<int, null_type, less<int>, rb_tree_tag,

```

```

136 //         tree_order_statistics_node_update> vivos;
137 //         for (int i = 0; i < n; i++) vivos.insert(i);
138 //         int pos = 0;
139 //         while (!vivos.empty()) {
140 //             pos = (pos + k - 1) % vivos.size();
141 //             auto it = vivos.find_by_order(pos); // el k-
// esimo vivo
142 //             orden.push_back(*it);
143 //             vivos.erase(it);
144 //         }
145
146 // 5b. ALTERNATIVA CON std::list: se avanza el iterador k-1
// veces
147 //         (con cierre circular) y se borra. Es  $O(n * k)$ , util
// solo con
148 //         n y k chicos.
149
150 // 6. OTRA PREGUNTA, MISMO MODELO: "cuantas personas quedan
// cuando se eliminan solo m de ellas" se responde
// cortando
151 //         ordenEliminacion en la m-esima; "cual es la ultima
// persona
152 //         que elimina a otra" es el penultimo de ordenEliminacion.
153
154 };

```

Ejemplo de funcionamiento:

```

1  cout << Josephus::sobreviviente(7, 3);
2  // 3   (base 0; en base 1 es la persona 4)
3
4  for (int x : Josephus::ordenEliminacion(7, 3)) cout << x << " "
5  ;
6  // 2 5 1 6 4 0 3   (el ultimo, 3, es el sobreviviente)
7
8  for (int x : Josephus::simulacionCola(7, 3)) cout << x << " ";
9  // 2 5 1 6 4 0 3   (la simulacion directa da lo mismo)
10
11 // el caso historico: n = 41, k = 3
12 cout << Josephus::sobreviviente(41, 3) + 1;
13 // 31   (la posicion 31 en base 1)
14
15 cout << Josephus::sobrevivienteK2(41) + 1;
16 // 19   (k = 2, n = 41: 2 * (41 - 32) = 18 en base 0)

```

Nota: la recurrencia da **solo** el último sobreviviente, no el orden. Si un enunciado pide "el segundo en salir" o "la posición de la persona X en el orden de eliminación", la recurrencia no alcanza y hay que usar el método con Fenwick (o similar). Otro detalle fácil de perder: en `ordenEliminacion`, la variable `pos` **no** se incrementa después de borrar, porque al eliminar a la persona en esa posición, la siguiente ocupa el mismo lugar entre las que quedan.

Regla práctica: si solo piden al sobreviviente, usa la recurrencia $O(n)$ (o la fórmula cerrada si $k = 2$ y n es enorme, o la variante $O(k \log n)$ si k es chico y n enorme); si piden el orden completo, usa el Fenwick con búsqueda del k -ésimo. Simula con cola solo con $n \cdot k$ pequeño, o para verificar tus otras versiones contra un caso conocido.

47.4 Simulación por Eventos (Cola de Prioridad de Eventos)

Idea: en una simulación por pasos (ver Plantilla de Simulación) el reloj avanza una unidad cada vez, y cada unidad se revisa aunque no pase nada. Cuando el tiempo total es enorme (10^9 o más) pero los **sucesos** son pocos (llegadas, fines de servicio, alarmas), conviene dejar de simular tiempo y simular **sucesos**: se mantiene una cola de prioridad (min-heap ordenado por tiempo) con los **eventos futuros**; se extrae el más próximo, el reloj **salta** directamente a su tiempo, se actualiza el estado, y ese evento puede **programar** otros nuevos (por ejemplo, empezar a atender a un cliente programa el evento "termina de ser atendido"). La simulación termina cuando la cola se vacía o se pasa un tiempo límite.

¿Por qué usarlo? El costo depende del número de eventos E , no de cuánto dura el tiempo simulado: $O(E \log E)$ en vez de $O(T)$. Además es el patrón que conecta varias cosas de esta guía: **Dijkstra** es una simulación por eventos donde cada evento es "el nodo v es alcanzado en el tiempo d "; **Line Sweep** recorre eventos ordenados de antemano, mientras que aquí los eventos se **generan mientras se simula**. Lo delicado son los **desempates**: dos eventos en el mismo instante deben procesarse en el orden que pide el enunciado, y hay que fijarlo explícitamente en el comparador.

Casos de uso:

- Colas de servicio: bancos, cajeros, impresoras, barberías, con k servidores y clientes que llegan en distintos momentos (el ejemplo desarrollado abajo).
- Planificación de procesos en un CPU (*round robin*, *shortest job first*): los eventos son "termina el quantum" y "llega un proceso".
- Propagación con tiempos distintos (un incendio que tarda tiempos diferentes en cruzar cada tramo): el evento es "este lugar se enciende en el tiempo t ".
- Temporizadores y alarmas: "qué suena primero", "cuántas veces suena antes del tiempo T " con eventos periódicos.
- Juegos por turnos donde cada personaje tiene una velocidad distinta: el evento es "le toca actuar a X en el tiempo t ".

Complejidad: $O(E \log E)$, con E el número total de eventos. En la simulación de cajeros cada cliente genera a lo más 2 eventos (llegada y fin), así que $E \leq 2n$ y el costo es $O(n \log n)$, sin importar qué tan grandes sean los tiempos.

```

1 struct SimulacionEventos {
2     struct Evento {
3         long long t;           // tiempo en que ocurre
4         int tipo;              // FIN = 0, LLEGADA = 1
5         int id;                // cliente involucrado
6
7         // orden para un min-heap: primero el tiempo menor; si
        // empatan,
8         // primero el de menor 'tipo' (FIN antes que LLEGADA: un
        // cajero
9         // que se libera justo cuando llega alguien queda libre
        // para el);
10        // el id desempata para que el resultado sea determinista
11        bool operator>(const Evento& o) const {
12            if (t != o.t) return t > o.t;
13            if (tipo != o.tipo) return tipo > o.tipo;
14            return id > o.id;
15        }
16    };
17    static const int FIN = 0, LLEGADA = 1;
18
19    vector<long long> llegada, duracion; // datos de cada
        // cliente
20    vector<long long> inicio, fin;      // resultados de la
        // simulacion
21    int cajeros;
22
23    // CREATE: clientes[i] = {momento de llegada, duracion del
        // servicio};
24    // hay 'cajeros' servidores identicos y una fila unica FIFO.
25    SimulacionEventos(vector<pair<long long, long long>>& clientes
        ,
26    int cajeros) {
27        cajeros = cajeros;
28        int n = clientes.size();
29        llegada.resize(n);
30        duracion.resize(n);
31        inicio.assign(n, -1);
32        fin.assign(n, -1);

```



```

33     for (int i = 0; i < n; i++) {
34         llegada[i] = clientes[i].first;
35         duracion[i] = clientes[i].second;
36     }
37 } // O(n)
38
39 // WRITE: corre la simulacion completa, llenando inicio[] y
40 // fin[].
41 void ejecutar() {
42     priority_queue<Evento, vector<Evento>, greater<Evento>> pq;
43     for (int i = 0; i < (int)llegada.size(); i++)
44         pq.push({llegada[i], LLEGADA, i});
45
46     int libres = cajeros;
47     queue<int> espera; // clientes
48     esperando, en orden
49
50     while (!pq.empty()) {
51         Evento e = pq.top();
52         pq.pop(); // el reloj salta a e
53         .t
54
55         if (e.tipo == LLEGADA) {
56             if (libres > 0) { // hay cajero libre:
57                 pasa ya
58                 libres--;
59                 inicio[e.id] = e.t;
60                 fin[e.id] = e.t + duracion[e.id];
61                 pq.push({fin[e.id], FIN, e.id}); // programa su fin
62             } else {
63                 espera.push(e.id); // se queda en la fila
64             }
65         } else { // FIN: un cajero se
66             libera
67             libres++;
68             if (!espera.empty()) {
69                 int c = espera.front();
70                 espera.pop();
71                 libres--;
72                 inicio[c] = e.t; // empieza en este
73                 instante
74                 fin[c] = e.t + duracion[c];
75                 pq.push({fin[c], FIN, c});
76             }
77         }
78     } // O(n log n)
79
80     // READ: cuanto espero el cliente i antes de ser atendido.
81     long long esperaDe(int i) {
82         return inicio[i] - llegada[i];
83     } // O(1)
84
85     //
86     // -----
87     // MODIFICACIONES Y COMENTARIOS CON LOS USOS
88     //
89     // -----
90
91     // 1. DESEMPATE DE EVENTOS SIMULTANEOS: es la fuente numero
92     // uno de
93     // errores. Si el enunciado dice que un cliente que llega
94     // exactamente cuando otro termina NO alcanza el cajero,
95     // se
96     // invierte el criterio (LLEGADA = 0, FIN = 1). Leer con
97     // cuidado
98     // y probar con un caso donde dos eventos caen en el mismo
99     // tiempo.
100
101     // 2. CANCELAR EVENTOS (cancelacion perezosa): un heap no
102     // permite
103     // borrar un elemento cualquiera. Se guarda una "version"
104     // por
105     // entidad; el evento lleva la version con la que se creo,
106     // y al
107     // sacarlo se ignora si esta desactualizado:
108     //
109     // vector<int> version(n, 0);
110     // // al programar: pq.push({t, tipo, id, version[id]});
111     // // para cancelar: version[id]++; // los viejos
112     // quedan obsoletos
113     // // al procesar: if (e.version != version[id])
114     // continue;
115
116     // 3. FILA CON PRIORIDAD (en vez de FIFO): se reemplaza la
117     // queue<int> por una priority_queue con el criterio
118     // pedido
119     // (menor duracion primero, cliente VIP primero, etc.):
120     //
121     // priority_queue<pair<long long,int>,
122     // vector<pair<long long,int>>,
123     // greater<>> espera; // {duracion,
124     // id}

```

```

107 // // al llegar sin cajero: espera.push({duracion[id], id
108 // });
109 // // al liberarse: auto [d, c] = espera.top(); espera.pop
110 // ();
111 // 4. EVENTOS PERIODICOS: al procesar un evento recurrente,
112 // se
113 // programa su proxima aparicion y se corta con un limite
114 // de
115 // tiempo para que la simulacion termine:
116 //
117 // if (e.tipo == ALARMA && e.t + periodo <= limite)
118 //     pq.push({e.t + periodo, ALARMA, e.id});
119 // 5. CORTE POR TIEMPO LIMITE: "que pasa hasta el instante T":
120 //
121 // if (e.t > T) break; // el resto ocurre despues de
122 // T
123 // 6. METRICAS QUE DEPENDEN DEL TIEMPO ENTRE EVENTOS (por
124 // ejemplo,
125 // largo promedio de la fila): se acumula el area antes de
126 // procesar cada evento, usando el tiempo transcurrido
127 // desde el
128 // evento anterior:
129 //
130 // long long tAnt = 0, areaFila = 0;
131 // // al sacar un evento:
132 // areaFila += (long long)espera.size() * (e.t - tAnt);
133 // tAnt = e.t;
134 // // promedio = areaFila / tiempo total
135 //
136 // 7. TIEMPOS REALES (double): los desempates por igualdad
137 // exacta
138 // son riesgosos con doubles. Si es posible, escalar a
139 // enteros
140 // (multiplicar por una potencia de 10); si no, comparar
141 // con
142 // epsilon en el comparador del heap.
143 // 8. VARIOS TIPOS DE EVENTO CON DATOS EXTRA: agregar campos
144 // al
145 // struct Evento (por ejemplo, un servidor o un destino) y
146 // despachar con switch (e.tipo) en el ciclo principal. El
147 // esqueleto (sacar el minimo, actualizar el estado,
148 // programar
149 // nuevos eventos) no cambia.
150
151 // 9. CUANDO NO USARLA: si los eventos ya estan todos dados
152 // de
153 // antemano y solo hay que recorrerlos ordenados, basta
154 // ordenar
155 // (ver Line Sweep). Y si el tiempo total es chico y
156 // ocurren
157 // muchos eventos por unidad de tiempo, la simulacion por
158 // pasos
159 // (Plantilla de Simulacion) es mas simple y igual de
160 // rapida.
161 };

```

Ejemplo de funcionamiento:

```

1 // 2 cajeros y 5 clientes: {llegada, duracion}
2 vector<pair<long long,long long>> clientes = {
3     {0, 5}, {1, 3}, {2, 4}, {3, 2}, {10, 1}
4 };
5
6 SimulacionEventos se(clientes, 2);
7 se.ejecutar();
8
9 for (int i = 0; i < 5; i++)
10     cout << "cliente " << i << ": inicio=" << se.inicio[i]
11     << " fin=" << se.fin[i] << " espera=" << se.esperaDe(i) << "\n"
12     ;
13 // cliente 0: inicio=0 fin=5 espera=0
14 // cliente 1: inicio=1 fin=4 espera=0
15 // cliente 2: inicio=4 fin=8 espera=2 (espera al cliente 1)
16 // cliente 3: inicio=5 fin=7 espera=2 (espera al cliente 0)
17 // cliente 4: inicio=10 fin=11 espera=0
18 // espera total = 4, espera maxima = 2, la ultima salida es en
19 // t = 11

```

Nota: el reloj nunca pasa por los instantes intermedios donde no ocurre nada (del 8 al 10 en el ejemplo): salta de evento en evento. Esa es toda la ventaja frente a la simulación por pasos, y es también la fuente de la regla más importante de esta técnica: el **orden** en que se procesan los eventos empatados es parte del enunciado, aunque casi nunca se diga explícitamente. En el ejemplo,

si un cliente hubiera llegado en $t = 4$, el orden FIN antes de LLEGADA lo habría dejado entrar de inmediato al cajero recién liberado.

Regla práctica: usa simulación por eventos cuando el tiempo simulado es enorme comparado con el número de sucesos, o cuando los sucesos se generan unos a otros. Antes de codificar, escribe la lista de tipos de evento, qué cambia en el estado con cada uno, qué eventos nuevos programa, y cómo se desempatan dos eventos simultáneos; con esas cuatro respuestas el ciclo principal casi se escribe solo.

47.5 Autómatas Celulares (Game of Life)

Idea: un autómatas celular es un tablero de celdas donde cada una tiene un estado de un conjunto finito, y en cada paso **todas** las celdas se actualizan **a la vez** con una regla que solo mira su vecindario. En el Juego de la Vida de Conway el tablero es una grilla 2D, cada celda está viva o muerta, y el vecindario son sus 8 vecinas (vecindario de Moore). La regla estándar se escribe **B3/S23**: una celda muerta **nace** si tiene exactamente 3 vecinas vivas, y una celda viva **sobrevive** si tiene 2 o 3 (en cualquier otro caso muere). Todo el problema se reduce a la plantilla de *Plantilla de Simulación (Estado, Paso y Parada)*: el estado es la grilla completa, el paso es aplicar la regla a todas las celdas, y la parada es un número de generaciones o un estado repetido. La idea que no se puede omitir es la **simultaneidad**: el nuevo valor de cada celda debe calcularse con los valores **viejos** de sus vecinas, por eso se escribe en una segunda grilla (*double buffering*) y solo al final se reemplaza la anterior.

¿Por qué usarlo? Los problemas de este tipo describen la regla en el enunciado y piden la grilla (o la población) tras k generaciones; no hay nada que optimizar mientras k y el tablero sean pequeños, y la simulación directa es la solución. Cuando k es enorme entra *Simulación con Detección de Ciclos (Floyd y Estado Repetido)*: un tablero finito tiene $2^{F \cdot C}$ estados posibles, así que la secuencia termina en un ciclo, y **buscarCiclo** de abajo es exactamente la variante de Estado Repetido con el tablero convertido a **string**. Frente a una simulación de objetos individuales (como *Simulación por Eventos*), aquí no hay eventos sueltos: todas las celdas cambian en cada paso, por lo que la simulación por pasos es la forma natural. Si el tablero es enorme pero casi vacío, en cambio, conviene guardar solo las celdas vivas (modificación 2).

Casos de uso:

- Calcular la grilla o la población tras k generaciones de una regla local (Game of Life y variantes con otras reglas B/S).
- Autómatas 1D (Regla 30, Regla 90, Regla 110): cada celda depende de sí misma y de sus dos vecinas, y la regla se codifica como un número de 0 a 255.
- Problemas de propagación sobre grillas por rondas (incendios, contagios, crecimiento de cultivos) que siguen el mismo esquema de actualización simultánea.
- Determinar si una configuración es estable, oscila o desaparece, y con qué período (ciclo del autómatas).
- Estado tras un número gigantesco de generaciones en autómatas lineales sobre $GF(2)$, como la Regla 90 (modificación 4).

Complejidad: cada paso recorre las $F \cdot C$ celdas y mira 8 vecinas por celda, es decir $O(F \cdot C)$; k generaciones cuestan $O(k \cdot F \cdot C)$ y la memoria es $O(F \cdot C)$. **buscarCiclo** guarda cada tablero visto en un `map<string,int>`: cada comparación cuesta $O(F \cdot C)$, por lo que resuelve en $O((\mu + \lambda) \cdot F \cdot C \cdot \log(\mu + \lambda))$. En el autómatas 1D de n celdas un paso cuesta $O(n)$. Con celdas vivas dispersas (P vivas) el paso baja a $O(P)$, y con `bitset` a $O(F \cdot C/64)$ (modificaciones 2 y 3).

```

1 // ----- Variante 1: Game of Life 2D (reglas B/S como
2   mascarar) -----
3 struct JuegoDeLaVida {
4     int filas, columnas;
5     bool toroidal; // true: los bordes dan
6                     // la vuelta
7     int nacimiento, supervivencia; // reglas B/S como
8     mascarar de bits
9     vector<vector<int>> celdas; // 1 = viva, 0 = muerta
10
11 // CREATE: lee el tablero inicial ('#' = viva, '.' = muerta).
12 // Por
13 // defecto usa B3/S23; bit v de la mascara = "con v vecinas
14 // vivas".
15 JuegoDeLaVida(const vector<string>& inicial, bool toroidal_ =
16     false,
17     int nacimiento_ = 1 << 3, int supervivencia_ = (1 << 2) | (1
18     << 3)) {
19     filas = inicial.size();
20     columnas = inicial[0].size();
21     toroidal = toroidal_;
22     nacimiento = nacimiento_;
23     supervivencia = supervivencia_;
24     celdas.assign(filas, vector<int>(columnas, 0));
25     for (int f = 0; f < filas; f++)
26     for (int c = 0; c < columnas; c++)
27     celdas[f][c] = (inicial[f][c] == '#');
28 }
29 // O(filas *
30   columnas)
31
32 // READ: cuantas de las 8 vecinas de (f, c) estan vivas.
33 int vecinasVivas(int f, int c) const {
34     int cuenta = 0;
35     for (int df = -1; df <= 1; df++) {
36         for (int dc = -1; dc <= 1; dc++) {
37             if (df == 0 && dc == 0) continue; // no se cuenta a
38             // si misma
39             int nf = f + df, nc = c + dc;
40             if (toroidal) {
41                 nf = (nf + filas) % filas;
42                 nc = (nc + columnas) % columnas;
43             } else if (nf < 0 || nf >= filas || nc < 0 || nc >=
44             columnas) {
45                 continue; // fuera del
46                 // tablero = muerta
47             }
48             cuenta += celdas[nf][nc];
49         }
50     }
51     return cuenta;
52 } // O(1) (8 vecinas)
53
54 // WRITE: un paso SIMULTANEO. Se calcula todo sobre la grilla
55 // vieja y se
56 // escribe en una grilla nueva; actualizar en el sitio
57 // corrompe el
58 // conteo de las celdas que todavia no se han procesado.
59 void paso() {
60     vector<vector<int>> siguiente(filas, vector<int>(columnas,
61     0));
62     for (int f = 0; f < filas; f++) {
63         for (int c = 0; c < columnas; c++) {
64             int v = vecinasVivas(f, c);
65             int regla = celdas[f][c] ? supervivencia : nacimiento;
66             siguiente[f][c] = (regla >> v) & 1;
67         }
68     }
69     celdas = siguiente;
70 } // O(filas *
71   columnas)
72
73 // WRITE: avanza k generaciones.
74 void avanzar(long long k) {

```

```

59     for (long long i = 0; i < k; i++) paso();
60 } // O(k * filas *
    columnas)
61
62 // READ: cantidad de celdas vivas.
63 int poblacion() const {
64     int total = 0;
65     for (int f = 0; f < filas; f++)
66         for (int c = 0; c < columnas; c++) total += celdas[f][c];
67     return total;
68 } // O(filas *
    columnas)
69
70 // READ: el tablero como string (sirve de clave para detectar
    ciclos).
71 string codificar() const {
72     string s;
73     for (int f = 0; f < filas; f++)
74         for (int c = 0; c < columnas; c++) s += celdas[f][c] ? '#'
75         : '.';
76     return s;
77 } // O(filas *
    columnas)
78
79 // READ: imprime el tablero.
80 void mostrar() const {
81     for (int f = 0; f < filas; f++) {
82         for (int c = 0; c < columnas; c++) cout << (celdas[f][c]
83         ? '#' : '.');
84         cout << "\n";
85     }
86 } // O(filas *
    columnas)
87
88 // READ: busca el ciclo (Estado Repetido) sobre una COPIA del
    tablero.
89 // Retorna {mu, lambda}, o {-1, -1} si no aparece en 'limite'
    pasos.
90 pair<int,int> buscarCiclo(int limite) const {
91     JuegoDeLaVida copia = *this;
92     map<string, int> visto;
93     for (int t = 0; t <= limite; t++) {
94         string clave = copia.codificar();
95         auto it = visto.find(clave);
96         if (it != visto.end()) return {it->second, t - it->second};
97         visto[clave] = t;
98         copia.paso();
99     }
100     return {-1, -1};
101 } // O(limite * F *
    C * log)
102
103 // ----- Variante 2: automata elemental 1D (regla de
    Wolfram) -----
104 struct AutomataElemental {
105     int n, regla; // regla en [0, 255]
106     vector<int> celdas; // fila ciclica de n
    celdas (0 o 1)
107
108     // CREATE: fila inicial y numero de regla.
109     AutomataElemental(vector<int> inicial, int regla_) {
110         celdas = inicial;
111         n = celdas.size();
112         regla = regla_;
113     } // O(n)
114
115     // WRITE: un paso. El vecindario (izq, centro, der) forma un
    numero de
116     // 3 bits, y ese bit de la regla es el nuevo valor de la
    celda.
117     void paso() {
118         vector<int> sig(n);
119         for (int i = 0; i < n; i++) {
120             int izq = celdas[(i - 1 + n) % n];
121             int cen = celdas[i];
122             int der = celdas[(i + 1) % n];
123             int patron = izq * 4 + cen * 2 + der; // 0..7
124             sig[i] = (regla >> patron) & 1;
125         }
126         celdas = sig;
127     } // O(n)
128
129     // WRITE: SOLO regla 90. Avanza 2^m pasos de golpe (ver
    modificacion 4).
130     void saltarPotenciaDeDos(int m) {
131         int d = (int)((1LL << m) % n);
132         vector<int> sig(n);
133         for (int i = 0; i < n; i++)
134             sig[i] = celdas[(i - d) % n + n] % n ^ celdas[(i + d) % n];
135         celdas = sig;
136     }
137
138     // WRITE: SOLO regla 90. Avanza k pasos descomponiendo k en
    binario.
139     void saltar(long long k) {
140         for (int m = 0; m < 62; m++)
141             if ((k >> m) & 1) saltarPotenciaDeDos(m);
142     } // O(n log k)
143
144     // -----
145     // MODIFICACIONES Y COMENTARIOS CON LOS USOS
146     // -----
147
148     // 1. OTRAS REGLAS: cualquier regla "tipo Life" B/S se pasa
    como
    mascarar.
149     // HighLife es B36/S23:
150     //
151     // JuegoDeLaVida hl(tablero, false, (1 << 3) | (1 << 6), (1
    << 2) | (1 << 3));
152     //
153     // Si el enunciado da el vecindario de von Neumann (solo 4
    vecinas:
154     // arriba, abajo, izquierda, derecha), basta cambiar los dos
    bucles de
155     // vecinasVivas por los cuatro desplazamientos (df, dc)
    permitidos.
156
157     // 2. TABLERO INFINITO CON POCAS CELDAS VIVAS: en vez de una
    grilla, se
158     // guarda el conjunto de celdas vivas y se cuenta, para cada
    celda
159     // vecina de alguna viva, cuantas vecinas vivas tiene:
160     //
161     // set<pair<int,int>> vivas;
162     // map<pair<int,int>, int> conteo;
163     // for (auto [f, c] : vivas)
164     //     for (int df = -1; df <= 1; df++)
165     //         for (int dc = -1; dc <= 1; dc++)
166     //             if (df != 0 || dc != 0) conteo[{f + df, c +
    dc}]++;
167     // set<pair<int,int>> nuevas;
168     // for (auto& [pos, v] : conteo) {
169     //     bool viva = vivas.count(pos);
170     //     if ((viva && (v == 2 || v == 3)) || (!viva && v ==
    3)) nuevas.insert(pos);
171     // }
172     // vivas = nuevas; // O(P log P) por
    paso
173     //
174     // Solo las celdas vecinas de una viva pueden estar vivas en
    el
175     // siguiente paso, por eso el costo depende de P y no del
    tablero.
176
177     // 3. BITSET: si cada fila cabe en un bitset<W>, las 8 vecinas
    de TODA la
178     // fila se obtienen con desplazamientos (fila << 1, fila >>
    1, y lo
179     // mismo para la fila de arriba y la de abajo), y el conteo
    se hace
180     // con sumadores bit a bit (dos o tres bitsets que guardan
    los bits del
181     // contador). Un paso pasa de O(F * C) a O(F * C / 64). Ver
    la
182     // subseccion de Bitset para Optimizacion en la parte
    BitWise.
183
184     // 4. SALTAR 2^m PASOS EN REGLA 90: la regla 90 es nuevo[i] =
    izq XOR der,
185     // lineal sobre GF(2). Elevar al cuadrado en característica
    2 no crea
186     // terminos cruzados, así que 2^m pasos equivalen a
187     // nuevo[i] = viejo[i - 2^m] XOR viejo[i + 2^m] (índices
    ciclicos).
188     // Con k en binario, k pasos cuestan O(n log k) en vez de O(
    n k), y
189     // permite k = 10^18 sin ciclos ni memoria extra. Solo
    funciona para
190     // reglas lineales (90 y pocas mas), NO para Regla 30 ni
    Game of Life.
191
192     // 5. ESTADO TRAS k GRANDE (Game of Life): se llama a
    buscarCiclo y se
193     // reduce k igual que en Detection de Ciclos:
194     //
195     // auto [mu, lambda] = j.buscarCiclo(limite);
196     // if (k >= mu) k = mu + (k - mu) % lambda;
197     // j.avanzar(k);
198     //
199     // Formas conocidas para reconocerlas: bloque (estable),
    parpadeador
200     // (periodo 2), planeador (avanza una celda en diagonal cada

```

```

201 // 4 pasos;
202 // en un toro  $F \times C$ , si es lo bastante grande, regresa tras
203 //  $4 * mcm(F, C)$  pasos).
204 // 6. TRAMPA: ACTUALIZAR EN EL SITIO. Si paso() modifica '
205 // celdas' mientras
206 // recorre, las celdas siguientes ven vecinas ya
207 // actualizadas y el
208 // resultado es otro autómata. Siempre grilla nueva (o
209 // guardar solo el
210 // cambio y aplicarlo al final).
211 // 7. TRAMPA: BORDES. "Fuera del tablero esta muerto", "los
212 // bordes dan la
213 // vuelta" y "los bordes se reflejan" dan resultados
214 // distintos, y el
215 // enunciado casi siempre lo dice en una sola línea. En un
216 // toro con
217 // filas o columnas  $< 3$ , una misma celda puede contarse dos
218 // veces como
219 // vecina (por eso conviene confirmar el tamaño mínimo del
220 // enunciado).

```

Ejemplo de funcionamiento:

```

1 // Parpadeador (oscilador de periodo 2), tablero de 5x5 con
2 // bordes muertos.
3 JuegoDeLaVida parpadeador({".....",
4 // "#.#.#.",
5 // "#.#.#.",
6 // "#.#.#.",
7 // "#.#.#."});
8
9 cout << parpadeador.poblacion(); // 3
10 parpadeador.paso();
11 parpadeador.mostrar();
12 // .....
13 // .....
14 // .###.
15 // .....
16 // .....
17 parpadeador.paso(); // vuelve a la forma
18 // vertical
19 auto c1 = parpadeador.buscarCiclo(50);
20 cout << c1.first << " " << c1.second; // 0 2
21
22 // Planeador (glider) en un toro de 6x6: se mueve una celda en
23 // diagonal
24 // cada 4 pasos y regresa a la posición inicial tras  $4 * 6 = 24$ .
25
26 JuegoDeLaVida planeador({"#.#.#.",
27 // "#.#.#.",
28 // "###.#.",
29 // ".....",
30 // ".....",
31 // ".....",
32 // ".....",
33 // "....."}, true);
34
35 auto c2 = planeador.buscarCiclo(200);
36 cout << c2.first << " " << c2.second; // 0 24
37 planeador.avanzar(4);
38 planeador.mostrar();
39 // .....
40 // ..#...
41 // ...#..
42 // .###..
43 // .....
44 // .....
45 // .....
46
47 // El mismo planeador con bordes muertos choca con la pared y
48 // termina en
49 // un bloque estable de 4 celdas (ciclo de largo 1 desde el
50 // paso 15).
51
52 JuegoDeLaVida choque({"#.#.#.", "#.#.#.", "###.#.", ".....", "
53 // .....", "....."});
54 auto c3 = choque.buscarCiclo(200);
55 cout << c3.first << " " << c3.second; // 15 1
56 choque.avanzar(20);
57 cout << choque.poblacion(); // 4
58
59 // Regla 90 sobre 9 celdas cerradas en círculo, una sola celda
60 // viva:
61 vector<int> ini(9, 0);
62 ini[4] = 1;
63 AutomataElemental a(ini, 90);
64 // generacion 0: ...#....
65 // generacion 1: ...#.#...
66 // generacion 2: ..#...#.
67 // generacion 3: .#.#.#.#.
68 // generacion 4: #.....#
69
70 AutomataElemental b(ini, 90);
71 b.saltar(4); // da la misma fila que
72 // 4 pasos: #.....#

```

```

80 AutomataElemental c(ini, 90);
81 c.saltar(1000000000000LL); // 10^12 pasos sin
82 // similares: ...#.#...

```

Nota: en el ejemplo del planeador sobre el toro, el ciclo empieza en $\mu = 0$ porque el planeador **regresa** exactamente al tablero inicial; en el ejemplo del choque, $\mu = 15$ porque el estado inicial nunca se repite (el planeador se deshace en un bloque y de ahí en adelante el tablero ya no cambia, $\lambda = 1$). Por eso no basta con mirar solo la población: dos tableros distintos pueden tener el mismo número de celdas vivas, y el ciclo se detecta sobre el tablero **completo**.

Regla práctica: si el problema da una regla local que se aplica a todas las celdas a la vez, simula con dos grillas y copia solo al final de cada paso. Si k es pequeño, eso es todo. Si k es gigantesco, aplica *Simulación con Detección de Ciclos* sobre el tablero codificado; si el tablero es enorme pero casi vacío, guarda solo las celdas vivas; y si la regla es lineal (Regla 90), salta de a 2^m pasos. Antes de codificar, confirma tres cosas del enunciado: qué cuenta como vecina, qué pasa en los bordes, y si la actualización es simultánea o por orden de celda.

47.6 Simulación de Máquinas e Intérpretes de Instrucciones

Idea: muchos problemas entregan un **programa** escrito en un lenguaje inventado (un puñado de instrucciones sobre registros, una cinta, una pila) y piden lo que imprime o el estado final. Es la plantilla de *Plantilla de Simulación (Estado, Paso y Parada)* con nombres propios: el **estado** es el contador de programa pc (índice de la próxima instrucción) más los registros, la memoria y la salida; el **paso** es el ciclo *buscar-decodificar-ejecutar* (leer la instrucción en pc , ejecutarla, y mover pc a la siguiente o a donde diga un salto); y la **parada** es una instrucción de detención, que pc salga del programa, o un límite de pasos. La decisión de diseño que más importa es **parsear el programa una sola vez** en una estructura (*operación, operandos*) y ejecutar sobre esa estructura, en vez de volver a partir strings en cada paso: un programa corto puede ejecutarse millones de veces por sus bucles.

¿Por qué usarlo? Aquí no se puede saltar la simulación con una fórmula: el comportamiento lo define el texto del programa y hay que ejecutarlo. Se distingue de la simulación de un proceso físico (*Simulación por Eventos*) en que el tiempo avanza instrucción a instrucción y el orden lo fija el programa, no una cola de prioridad. El gran riesgo es el **bucle infinito**: un programa válido puede no terminar nunca. Las dos defensas son un límite de pasos (simple y casi siempre suficiente) y *Simulación con Detección de Ciclos*: la máquina tiene estado finito, así que si el par (pc , registros) se repite y no hay entrada nueva, el programa está atrapado. Cuando el programa es larguísimo pero tiene bucles de suma/multiplicación reconocibles, se **reemplaza el bucle por su resultado** (modificación 4), que es la única forma de pasar de 10^9 instrucciones a unas pocas.

Casos de uso:

- Máquinas de registros con **set**, **add**, **jnz**, etc., y preguntas del tipo *qué queda en el registro a al final o qué imprime*.

- Interpretes de esolangs como Brainfuck: cinta, puntero y corchetes que emparejar (una *pila* resuelve los saltos).
- Simular un **robot** o vehículo que sigue una lista de comandos (avanzar, girar, repetir) hasta terminarla.
- Detectar si un programa termina, o encontrar un valor inicial que lo haga terminar (con límite de pasos o estado repetido).
- Máquinas con pila (push, pop, call, ret) o memoria indexada, que extienden el mismo esquema (modificaciones 2 y 3).

Complejidad: cada paso cuesta $O(1)$ (un switch y accesos a un arreglo de registros); el parseo cuesta $O(L)$ para un programa de L líneas y la memoria es $O(L + \text{registros} + \text{salida})$. El tiempo total es $O(L + P)$, donde P es el **número de pasos que realmente se ejecutan**, y P depende del comportamiento del programa (sus bucles), no de su longitud: un programa de 7 líneas puede ejecutar 10^9 pasos. En Brainfuck, precalcular los saltos deja el paso en $O(1)$ en lugar de $O(L)$ por corchete.

```

1 // ----- Variante 1: maquina de registros (26 registros: a
  // ..z) -----
2 // Instrucciones: set r x | add r x | sub r x | mul r x | mod r
  // x
3 // jnz x y (si x != 0, salta y instrucciones
  // RELATIVO a esta)
4 // out x | hlt
5 // Un operando es un registro (una letra) o un numero (
  // posiblemente negativo).
6 struct MaquinaVirtual {
7     enum Op { SET, ADD, SUB, MUL, MOD, JNZ, OUT, HLT };
8     struct Instruccion {
9         Op op;
10        int destino; // registro destino (SET/
11        ADD/...)
12        long long a, b; // operandos: indice de
13        registro o valor
14        bool aEsReg, bEsReg;
15    };
16    vector<Instruccion> programa;
17    long long reg[26];
18    int pc; // indice de la proxima
19    instruccion
20    long long pasos; // instrucciones
21    ejecutadas
22    bool detenida;
23    vector<long long> salida;
24
25    // Convierte un token en (valor, esRegistro).
26    static void leerOperando(const string& t, long long& valor,
27    bool& esReg) {
28        if (isalpha(t[0])) { esReg = true; valor = t[0] - 'a'; }
29        else { esReg = false; valor = stoll(t); }
30    }
31
32    // CREATE: parsea el programa UNA sola vez a instrucciones;
33    los
34    // registros parten en 0.
35    MaquinaVirtual(const vector<string>& lineas) {
36        for (int i = 0; i < 26; i++) reg[i] = 0;
37        pc = 0; pasos = 0; detenida = false;
38        for (const string& linea : lineas) {
39            istringstream in(linea);
40            string nombre, x, y;
41            in >> nombre >> x >> y;
42            Instruccion I = {HLT, 0, 0, 0, false, false};
43            if (nombre == "set" || nombre == "add" || nombre == "sub"
44            ||
45            nombre == "mul" || nombre == "mod") {
46                if (nombre == "set") I.op = SET;
47                else if (nombre == "add") I.op = ADD;
48                else if (nombre == "sub") I.op = SUB;
49                else if (nombre == "mul") I.op = MUL;
50                else I.op = MOD;
51                I.destino = x[0] - 'a';
52                leerOperando(y, I.a, I.aEsReg);
53            } else if (nombre == "jnz") {
54                I.op = JNZ;

```

```

49        leerOperando(x, I.a, I.aEsReg);
50        leerOperando(y, I.b, I.bEsReg);
51    } else if (nombre == "out") {
52        I.op = OUT;
53        leerOperando(x, I.a, I.aEsReg);
54    } else {
55        I.op = HLT;
56    }
57    programa.push_back(I);
58    }
59    if (programa.empty()) detenida = true;
60    } // O(L)
61
62    // READ: valor de un operando (registro o constante).
63    long long valor(long long x, bool esReg) const {
64        return esReg ? reg[x] : x;
65    } // O(1)
66
67    // WRITE: ejecuta UNA instruccion y mueve pc.
68    void paso() {
69        const Instruccion& I = programa[pc];
70        int siguiente = pc + 1; // por defecto, la
71        siguiente
72        switch (I.op) {
73            case SET: reg[I.destino] = valor(I.a, I.aEsReg); break;
74            case ADD: reg[I.destino] += valor(I.a, I.aEsReg); break;
75            case SUB: reg[I.destino] -= valor(I.a, I.aEsReg); break;
76            case MUL: reg[I.destino] *= valor(I.a, I.aEsReg); break;
77            case MOD: {
78                long long d = valor(I.a, I.aEsReg);
79                if (d == 0) { detenida = true; return; } // division
80                por cero
81                reg[I.destino] %= d;
82                break;
83            }
84            case JNZ:
85                if (valor(I.a, I.aEsReg) != 0)
86                    siguiente = pc + (int)valor(I.b, I.bEsReg);
87                break;
88            case OUT: salida.push_back(valor(I.a, I.aEsReg)); break;
89            case HLT: detenida = true; return;
90        }
91        pc = siguiente;
92        pasos++;
93        if (pc < 0 || pc >= (int)programa.size()) detenida = true;
94    } // O(1)
95
96    // WRITE: ejecuta hasta detenerse o hasta 'limite' pasos.
97    Retorna true
98    // si el programa termino, false si se corto por el limite.
99    bool ejecutar(long long limite) {
100        while (!detenida && pasos < limite) paso();
101        return detenida;
102    } // O(min(P, limite))
103
104    }
105
106    // ----- Variante 2: interprete de Brainfuck (saltos
107    precalculados) -----
108    struct InterpreteBrainfuck {
109        string codigo;
110        vector<int> salto; // salto[i] = corchete
111        que empareja al i
112        vector<unsigned char> cinta; // celdas de 8 bits: dan
113        la vuelta en 256
114        int ptr, pc;
115        string salida;
116
117        // CREATE: empareja los corchetes con una pila, UNA vez, para
118        no
119        // recorrer el codigo buscando el corchete en cada iteracion.
120        InterpreteBrainfuck(const string& codigo_, int tamCinta =
121        30000) {
122            codigo = codigo_;
123            cinta.assign(tamCinta, 0);
124            ptr = 0; pc = 0;
125            salto.assign(codigo.size(), -1);
126            vector<int> pila;
127            for (int i = 0; i < (int)codigo.size(); i++) {
128                if (codigo[i] == '[') pila.push_back(i);
129                else if (codigo[i] == ']') {
130                    int j = pila.back(); pila.pop_back();
131                    salto[i] = j;
132                    salto[j] = i;
133                }
134            }
135        } // O(L)
136
137        // WRITE: ejecuta todo el programa; 'entrada' alimenta a la
138        coma (,).
139        void ejecutar(const string& entrada = "") {
140            int pos = 0;
141            while (pc < (int)codigo.size()) {

```

```

132     switch (codigo[pc]) {
133     case '>': ptr++; break;
134     case '<': ptr--; break;
135     case '+': cinta[ptr]++; break;
136     case '-': cinta[ptr]--; break;
137     case '.': salida += (char)cinta[ptr]; break;
138     case ',': cinta[ptr] = pos < (int)entrada.size() ?
    entrada[pos++] : 0; break;
139     case '[': if (cinta[ptr] == 0) pc = salto[pc]; break;
140     case ']': if (cinta[ptr] != 0) pc = salto[pc]; break;
141     }
142     pc++;
143 }
144 } // O(P), P = pasos
    ejecutados
145 };
146
147 // -----
148 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
149 // -----
150
151 // 1. DETECTAR BUCLE INFINITO POR ESTADO REPETIDO: si el
    programa no lee
152 // entrada, el estado completo es (pc, registros). Si se
    repite, el
153 // programa no termina nunca (mismo principio de Estado
    Repetido):
154 //
155 // set<pair<int, array<long long, 26>>> visto;
156 // while (!m.detenida) {
157 //     array<long long, 26> r;
158 //     copy(m.reg, m.reg + 26, r.begin());
159 //     if (!visto.insert({m.pc, r}).second) return false;
160 //     // bucle infinito
161 //     m.paso();
162 // }
163 // Guarda muchos estados si el programa corre muchos pasos;
    para un
164 // limite de pasos generoso (107 o 108 pasos simples)
    suele bastar
165 // con el contador y no hace falta este set.
166
167 // 2. ETIQUETAS Y SALTOS ABSOLUTOS: si el lenguaje tiene "label
    ." y
168 // "jmp label", se resuelve en dos pasadas al parsear: la
    primera
169 // guarda etiqueta -> indice de instruccion en un map, la
    segunda
170 // reemplaza cada nombre por ese indice. En ejecucion todo
    salto es
171 // un simple "pc = destino".
172
173 // 3. PILA, CALL Y RET: se agrega vector<int> pila. 'call x'
    hace
174 // pila.push_back(pc + 1) y salta a x; 'ret' hace pc = pila.
    back() y
175 // pila.pop_back(). Con memoria indexada, un operando puede
    ser
176 // "mem[valor]" con mem un arreglo de tamano fijo.
177
178 // 4. OPTIMIZAR BUCLES CONOCIDOS: el bucle
179 //     add a 1
180 //     sub b 1
181 //     jnz b -2
182 // equivale a "a += b; b = 0" si b > 0, en O(1) en vez de O(
    b). En
183 // Brainfuck, "[-]" equivale a cinta[ptr] = 0, "[->+<]" a
184 // cinta[ptr+1] += cinta[ptr] (modulo 256), y una tanda
    "++++" puede
185 // comprimirse a "sumar 5". Se detectan al parsear,
    sustituyendo por
186 // una instruccion nueva. Solo vale la pena si el enunciado
    ejecuta
187 // ~109 pasos o mas; si no, ejecutar directo es mas seguro.
188
189 // 5. TRAMPA: SALTO RELATIVO. En "jnz x y" el salto se cuenta
    desde la
190 // PROPIA instruccion jnz (pc + y), no desde la siguiente.
    Confundirlo
191 // da un error de uno, muy dificil de ver sin traza. Un
    salto de 0
192 // queda en la misma instruccion para siempre.
193
194 // 6. TRAMPA: LEER ANTES DE ESCRIBIR. "add a a" duplica a, y
    funciona
195 // porque valor() se evalua antes de asignar. Si se escribe
    la
196 // asignacion primero y se lee despues, el resultado cambia.
    Igual
197 // con "mul a a" o "sub a a".
198
199 // 7. TRAMPA: TIPOS. Los registros deben ser long long (o el

```

```

    tipo que
200 // diga el enunciado): un producto de dos int desborda en
    silencio.
201 // En Brainfuck, unsigned char da la vuelta 255 -> 0, y el
    puntero
202 // puede salirse de la cinta si el programa lo permite (
    validar
203 // segun el enunciado: error, cinta circular o cinta
    infinita).
204
205 // 8. DEPURAR: imprimir "pc, a, b, ..." de los primeros 20
    pasos y
206 // compararlos a mano con el ejemplo del enunciado casi
    siempre
207 // localiza el error en un instante.

```

Ejemplo de funcionamiento:

```

1 // Suma 10 + 9 + ... + 1: a acumula, b cuenta hacia atras.
2 MaquinaVirtual m({"set a 0",
3     "set b 10",
4     "add a b",
5     "sub b 1",
6     "jnz b -2", // si b != 0, retrocede 2 (vuelve a "add a
    b")
7     "out a",
8     "hlt"});
9
10 bool termino = m.ejecutar(1000);
11 cout << termino << " " << m.pasos << " " << m.salida[0];
12 // 1 33 55 (termino; 33 pasos = 2 + 10 vueltas de 3 + out;
    imprimo 55)
13
14 // Factorial de 5 (mul en vez de add): sin "hlt", pc sale del
    programa.
15 MaquinaVirtual fact({"set a 1", "set b 5", "mul a b", "sub b 1"
    , "jnz b -2", "out a"});
16 fact.ejecutar(1000);
17 cout << fact.salida[0] << " " << fact.pasos;
18 // 120 18 (se detiene porque pc = 6 queda fuera del programa)
19
20 // Bucle infinito: "jnz a 0" salta a si misma para siempre.
21 MaquinaVirtual inf({"set a 1", "jnz a 0"});
22 bool t2 = inf.ejecutar(1000000);
23 cout << t2 << " " << inf.pasos << " " << inf.pc;
24 // 0 1000000 1 (no termino: el limite de pasos lo corto)
25
26 // Brainfuck: 8 * 8 = 64 en la celda 1, +1 = 65, y '.' imprime
    'A'.
27 InterpreteBrainfuck bf("+++++++>+++++++<->+.").);
28 bf.ejecutar();
29 cout << bf.salida; // A
30
31 // Eco de la entrada: lee un caracter, lo imprime, y repite
    hasta el 0.
32 InterpreteBrainfuck eco("[.,]");
33 eco.ejecutar("hey");
34 cout << eco.salida; // hey

```

Nota: en el primer ejemplo el programa termina con `hlt` y `pc = 6` (apuntando al propio `hlt`), mientras que en el del factorial no hay `hlt`: la máquina se detiene porque `pc` sale del rango del programa. Los enunciados suelen definir ambas formas de terminar, y si tu simulador solo maneja una, fallas los casos donde ocurre la otra. Por otro lado, **pasos** cuenta las instrucciones **ejecutadas**, no las líneas del programa; ese número es el que determina si la simulación directa cabe en el tiempo límite.

Regla práctica: parsea el programa una sola vez a una estructura de instrucciones, ejecuta con un ciclo **buscar-ejecutar** y pon **siempre** un límite de pasos, porque los programas que no terminan son el caso de prueba favorito. Si el límite se alcanza en un enunciado que pregunta *si termina*, usa *Simulación con Detección de Ciclos* sobre el estado (`pc`, registros); si el programa ejecuta más de $\sim 10^9$ instrucciones por bucles reconocibles, sustitúyelos por su resultado. Y recuerda dos detalles que causan la mayoría de los errores: el salto es relativo a la instrucción que salta, y los operandos se leen antes de escribir el destino.

48 Matrices y Grillas

48.1 Recorridos Especiales (Espiral, Diagonal, Zigzag)

Idea: un recorrido especial es una forma de visitar cada celda de una matriz **exactamente una vez** en un orden distinto al de filas. Hay cuatro familias. La **espiral** recorre la matriz por capas, de afuera hacia adentro. Las **diagonales** agrupan las celdas con $i + j$ constante (antidiagonales) o con $i - j$ constante (diagonales principales). El **zigzag** recorre las antidiagonales alternando el sentido en cada una. La **serpiente** recorre las filas alternando izquierda-derecha y derecha-izquierda. La idea común es que el orden se describe con **aritmética de índices**, no con un caminante que marca celdas visitadas: la espiral se reduce a cuatro fronteras (**arriba**, **abajo**, **izq**, **der**) que se contraen después de recorrer cada lado, y una antidiagonal d es el conjunto de celdas $(i, d - i)$ con $\max(0, d - m + 1) \leq i \leq \min(n - 1, d)$. Conviene que el recorrido devuelva la **lista de coordenadas** en orden (**ordenEspiral**): así sirve tanto para **leer** una matriz en ese orden como para **llenar** una matriz siguiéndolo (**generarEspiral**).

¿Por qué usarlo? Frente a simular un caminante con una matriz visitado y un arreglo dx/dy (que gira a la derecha al chocar; ver *Direcciones y Vecindarios* más adelante), las fronteras no necesitan memoria extra ni marcar celdas, y no dependen de que el borde esté bien detectado; el caminante con **visitado**, en cambio, es más flexible y es la opción cuando hay celdas bloqueadas o la región no es un rectángulo (modificación 1). Frente a recorrer por filas, las diagonales agrupan celdas relacionadas: dos casillas de un tablero están en la misma diagonal de un alfil o de una reina si y solo si comparten $i + j$ o $i - j$, la misma condición que usa *Problema de las N Reinas*. Y frente a *Prefix Sum 2D*, que responde sumas de submatrices, los recorridos por diagonal responden preguntas sobre **líneas** de la matriz en $O(n \cdot m)$ total.

Casos de uso:

- Imprimir o leer una matriz en espiral, y **generar** una matriz $n \times m$ llena con $1, 2, \dots, nm$ en espiral.
- Agrupar por diagonales para sumar, contar o buscar el máximo en cada una, o para saber qué celdas ataca un alfil o una reina.
- Recorrer en zigzag (*Diagonal Traverse*), el orden en que se lee un bloque de coeficientes en la compresión JPEG.
- Tableros numerados en serpiente, como el de *Serpientes y Escaleras*: convertir el número de casilla en (fila, columna) y viceversa.
- Problemas de **capas** o anillos: rotar cada anillo, sumar cada anillo, o recorrer desde el centro hacia afuera (modificación 2).

Complejidad: todos los recorridos visitan cada celda una sola vez con trabajo $O(1)$ por celda, así que cuestan $O(n \cdot m)$ en tiempo. En **ordenEspiral**, cada uno de los cuatro bucles agrega las celdas de un lado y luego la frontera correspondiente se contrae, por lo que ninguna

celda se agrega dos veces (las dos condiciones **if** lo garantizan cuando queda una sola fila o columna) y el total de celdas agregadas es $n \cdot m$. Hay $\lceil \min(n, m) / 2 \rceil$ capas. En las diagonales hay $n + m - 1$ antidiagonales, y la d -ésima tiene $\min(n - 1, d) - \max(0, d - m + 1) + 1$ celdas. La memoria extra es $O(n \cdot m)$ por la lista de salida; el recorrido en sí no usa nada más.

```

1 struct RecorridosDeMatriz {
2     int filas, columnas;
3     vector<vector<int>>> a;
4
5     // CREATE: guarda la matriz y sus dimensiones.
6     RecorridosDeMatriz(const vector<vector<int>>& m) {
7         a = m;
8         filas = a.size();
9         columnas = a.empty() ? 0 : a[0].size();
10    }
11    // O(filas *
12    columnas)
13
14    // READ: coordenadas de una matriz f x c en orden de espiral
15    horaria,
16    // empezando en (0, 0). Cuatro fronteras que se contraen.
17    static vector<pair<int,int>> ordenEspiral(int f, int c) {
18        vector<pair<int,int>> orden;
19        int arriba = 0, abajo = f - 1, izq = 0, der = c - 1;
20        while (arriba <= abajo && izq <= der) {
21            for (int j = izq; j <= der; j++) orden.push_back({arriba,
22                j}); // ->
23            arriba++;
24            for (int i = arriba; i <= abajo; i++) orden.push_back({i,
25                der}); // baja
26            der--;
27            if (arriba <= abajo) { // sigue
28                // habiendo fila
29                for (int j = der; j >= izq; j--) orden.push_back({abajo,
30                    j}); // <-
31                abajo--;
32            }
33            if (izq <= der) { // sigue
34                // habiendo columna
35                for (int i = abajo; i >= arriba; i--) orden.push_back({
36                    i, izq}); // sube
37                izq++;
38            }
39        }
40        return orden;
41    } // O(f * c)
42
43    // READ: los valores de la matriz leídos en espiral.
44    vector<int> espiral() const {
45        vector<int> res;
46        for (auto [i, j] : ordenEspiral(filas, columnas)) res.
47            push_back(a[i][j]);
48        return res;
49    } // O(filas *
50    columnas)
51
52    // READ: matriz f x c llena con 1..f*c siguiendo la espiral.
53    static vector<vector<int>> generarEspiral(int f, int c) {
54        vector<vector<int>> m(f, vector<int>(c, 0));
55        int k = 1;
56        for (auto [i, j] : ordenEspiral(f, c)) m[i][j] = k++;
57        return m;
58    } // O(f * c)
59
60    // READ: cada antidiagonal (i + j = d) como un vector, de
61    arriba hacia
62    // abajo (i creciente, es decir, hacia abajo y a la izquierda
63    //).
64    vector<vector<int>> antiDiagonales() const {
65        vector<vector<int>> res;
66        for (int d = 0; d <= filas + columnas - 2; d++) {
67            vector<int> diag;
68            for (int i = max(0, d - columnas + 1); i <= min(filas -
69                1, d); i++)
70                diag.push_back(a[i][d - i]);
71            res.push_back(diag);
72        }
73        return res;
74    } // O(filas *
75    columnas)
76
77    // READ: zigzag: antidiagonales alternando el sentido. La d
78    // par sube
79    // (i decreciente); la d impar baja (i creciente). Empieza en
80    // (0, 0)
81    // y sigue con (0, 1).
82    vector<int> zigzag() const {
83        vector<int> res;
84        for (int d = 0; d <= filas + columnas - 2; d++) {

```

```

68     int desde = max(0, d - columnas + 1), hasta = min(filas -
69     1, d);
70     if (d % 2 == 0) {
71         for (int i = hasta; i >= desde; i--) res.push_back(a[i]
72         ][d - i]);
73     } else {
74         for (int i = desde; i <= hasta; i++) res.push_back(a[i]
75         ][d - i]);
76     }
77     return res;
78 } // O(filas *
79     columnas)
80
81 // READ: serpiente (boustrofedon): las filas pares de
82 // izquierda a
83 // derecha y las impares de derecha a izquierda.
84 vector<int> serpiente() const {
85     vector<int> res;
86     for (int i = 0; i < filas; i++) {
87         if (i % 2 == 0) for (int j = 0; j < columnas; j++) res.
88         push_back(a[i][j]);
89         else for (int j = columnas - 1; j >= 0; j--) res.
90         push_back(a[i][j]);
91     }
92     return res;
93 } // O(filas *
94     columnas)
95
96 // READ: (fila, columna) de la casilla k (0-indexada) de un
97 // tablero de
98 // 'columnas' columnas numerado en serpiente. Sirve para
99 // Serpientes y
100 // Escaleras sin construir el tablero.
101 static pair<int,int> posicionSerpiente(int k, int columnas) {
102     int i = k / columnas, r = k % columnas;
103     int j = (i % 2 == 0) ? r : columnas - 1 - r;
104     return {i, j};
105 } // O(1)
106 };
107
108 // -----
109 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
110 // -----
111
112 // 1. ESPIRAL CON CAMINANTE Y VISITADO: mas flexible que las
113 // fronteras
114 // cuando hay celdas bloqueadas o la region es irregular.
115 // Avanza en
116 // la direccion actual y gira a la derecha al chocar con el
117 // borde o
118 // con una celda ya visitada:
119 //
120 // int dx[4] = {0, 1, 0, -1}, dy[4] = {1, 0, -1, 0}; // ->
121 // v <- ^
122 // vector<vector<bool>> vis(f, vector<bool>(c, false));
123 // int i = 0, j = 0, d = 0;
124 // for (int t = 0; t < f * c; t++) {
125 //     // procesar (i, j)
126 //     vis[i][j] = true;
127 //     int ni = i + dx[d], nj = j + dy[d];
128 //     if (ni < 0 || ni >= f || nj < 0 || nj >= c || vis[
129 //         ni][nj]) {
130 //         d = (d + 1) % 4;
131 //         // gira a la derecha
132 //         ni = i + dx[d]; nj = j + dy[d];
133 //     }
134 //     i = ni; j = nj;
135 // }
136 //
137 // Con celdas bloqueadas hay que cambiar el for por un while
138 // que pare
139 // cuando ni la direccion actual ni el giro dejen avanzar.
140
141 // 2. OTRAS ORIENTACIONES:
142 // - Antihoraria desde (0, 0): es la espiral horaria de la
143 // matriz
144 // TRANSPUESTA con las coordenadas intercambiadas:
145 // for (auto [i, j] : ordenEspiral(columnas, filas))
146 //     visitar(j, i);
147 // - Hacia afuera, terminando en la esquina (0, 0): recorrer
148 // ordenEspiral al reves. Parte de la ultima celda de la
149 // espiral
150 // (en una matriz impar x impar, el centro) y da la vuelta
151 // en
152 // sentido contrario:
153 // auto o = ordenEspiral(f, c); reverse(o.begin(), o.end
154 // ());
155 // - Desde otra esquina: reflejar las coordenadas, por
156 // ejemplo
157 // (i, j) -> (f - 1 - i, j) para empezar abajo a la
158 // izquierda.
159
160 // 3. DIAGONALES PRINCIPALES (i - j constante): i - j va de -(m
161 // - 1) a n - 1;
162 // se suma m - 1 para usarlo como indice no negativo:
163 //
164 // vector<vector<int>> diag(n + m - 1);
165 // for (int i = 0; i < n; i++)
166 //     for (int j = 0; j < m; j++) diag[i - j + m - 1].
167 //     push_back(a[i][j]);
168 //
169 // La anti diagonal analoga usa el indice i + j (de 0 a n + m
170 // - 2).
171
172 // 4. ACUMULADOS POR DIAGONAL: igual que Prefix Sums pero a lo
173 // largo de
174 // la diagonal: pd[i][j] = a[i][j] + pd[i-1][j-1] (diagonal
175 // principal)
176 // o pd[i][j] = a[i][j] + pd[i-1][j+1] (anti diagonal), con 0
177 // fuera del
178 // tablero. La suma de un tramo de diagonal se responde en O
179 // (1) como
180 // diferencia de dos acumulados.
181
182 // 5. NUMERO DE UNA CASILLA EN SERPIENTE (inversa de
183 // posicionSerpiente):
184 // k = i * columnas + (i % 2 == 0 ? j : columnas - 1 - j).
185
186 // 6. K-ESIMA CELDA DE LA ESPIRAL SIN CONSTRUIRLA: la capa r (
187 // contando
188 // desde 0) es un anillo de (f - 2r) x (c - 2r) celdas con
189 // 2 * (f - 2r) + 2 * (c - 2r) - 4 celdas, mientras ambas
190 // dimensiones
191 // sean al menos 2 (si queda una sola fila o columna, la
192 // capa es esa
193 // linea). Restando capas completas de k se llega a la capa
194 // correcta
195 // en O(min(f, c)); despues se resuelve dentro del anillo
196 // con
197 // aritmetica. Util cuando f * c es demasiado grande para
198 // una lista.
199
200 // 7. TRAMPA: UNA SOLA FILA O COLUMNA. Sin los dos if de guarda,
201 // con una
202 // matriz 1 x c el tercer bucle recorre otra vez las celdas
203 // de la
204 // fila, y con f x 1 el cuarto repite la columna. Los if son
205 // la parte
206 // mas facil de olvidar de la espiral por fronteras.
207
208 // 8. TRAMPA: LA CONVENCION DEL ZIGZAG. Hay dos: empezar
209 // subiendo
210 // (0,0), (0,1), (1,0), ... o empezar bajando (0,0), (1,0),
211 // (0,1), ....
212 // El codigo de arriba usa la primera. Confirma cual pide el
213 // enunciado con su ejemplo antes de codificar; el error es
214 // solo
215 // cambiar la paridad de d, pero invalida toda la salida.

```

Ejemplo de funcionamiento:

```

1 vector<vector<int>> m = {{1, 2, 3, 4},
2     {5, 6, 7, 8},
3     {9, 10, 11, 12}};
4 RecorridosDeMatriz r(m);
5
6 for (int x : r.espiral()) cout << x << " ";
7 // 1 2 3 4 8 12 11 10 9 5 6 7
8
9 for (int x : r.zigzag()) cout << x << " ";
10 // 1 2 5 9 6 3 4 7 10 11 8 12
11
12 for (int x : r.serpiente()) cout << x << " ";
13 // 1 2 3 4 8 7 6 5 9 10 11 12
14
15 for (auto& diag : r.antiDiagonales()) {
16     for (int x : diag) cout << x << " ";
17     cout << "| ";
18 }
19 // 1 | 2 5 | 3 6 9 | 4 7 10 | 8 11 | 12 |
20
21 auto g = RecorridosDeMatriz::generarEspiral(3, 4);
22 // 1 2 3 4
23 // 10 11 12 5
24 // 9 8 7 6
25
26 auto p = RecorridosDeMatriz::posicionSerpiente(6, 4);
27 cout << p.first << " " << p.second;
28 // 1 1 (la casilla 6 esta en la fila 1, columna 1: esa fila
29 // va al reves)

```

Nota: generarEspiral y espiral son operaciones inversas: la primera usa el orden para escribir los números 1, 2, ... en las celdas, y la segunda lo usa para

leer las celdas en ese orden. Por eso conviene devolver coordenadas y no valores: el mismo `ordenEspiral` resuelve las dos mitades del problema. En el ejemplo, la casilla 6 del tablero en serpiente cae en (1,1) y no en (1,2), porque la fila 1 se recorre de derecha a izquierda: $k = 6$ es el segundo elemento de esa fila, que ocupa la columna $c - 1 - 1 = 2 \dots$ contando desde 0 el resto es $r = 2$, y la columna es $4 - 1 - 2 = 1$.

Regla práctica: para leer o generar una espiral en una matriz rectangular completa, usa las cuatro fronteras con las dos guardas; si hay obstáculos o una región irregular, usa el caminante con `visitado`. Para cualquier pregunta sobre diagonales, identifica la clave: $i + j$ (antidiagonal, o alfil/reina hacia un lado) o $i - j + m - 1$ (diagonal principal) y agrupa las celdas por esa clave en una sola pasada. Y en un tablero numerado en serpiente, no lo construyas: convierte el número de casilla con `posicionSerpiente`.

48.2 Rotación, Transposición y Reflexión

Idea: una matriz cuadrada tiene **8 simetrías**: las 4 rotaciones (0° , 90° , 180° , 270°) y esas mismas 4 aplicadas después de un espejo. Todas salen de dos operaciones básicas: la **transposición** ($b[j][i] = a[i][j]$, el espejo respecto a la diagonal principal) y la **reflexión** (invertir el orden de las columnas, o el de las filas). Con ellas se obtienen las rotaciones mediante identidades fáciles de recordar: rotar 90° en sentido horario es **transponer y luego invertir cada fila**; rotar 90° antihorario es **transponer y luego invertir el orden de las filas**; y rotar 180° es hacer las dos reflexiones. En una matriz $n \times m$ que no es cuadrada, transponer y rotar 90° producen una matriz $m \times n$, así que las fórmulas de índices usan siempre las dimensiones **originales**:

rotar 90° horario: $b[j][n-1-i] = a[i][j]$, antihorario: $b[n-1-j][i] = a[i][j]$

¿Por qué usarlo? La razón principal es **ahorrar casos**: si ya sabes implementar «mover todo hacia abajo», no escribas otras tres versiones para arriba, izquierda y derecha; rota la matriz para que la dirección pedida quede hacia abajo, aplica lo que ya tienes y rota de vuelta (modificación 5, y *Gravedad y Caída de Piezas* en la siguiente subsección). La segunda razón es **comparar figuras salvo simetría**: se generan las variantes, se toma la menor como **forma canónica**, y dos figuras son equivalentes exactamente cuando su forma canónica es igual, lo que permite contarlas con un `set`. Se distingue de *Recorridos Especiales* en que allí se **lee** la matriz en otro orden sin cambiarla, mientras que aquí se **construye** una matriz nueva. Y si solo importan unos pocos puntos de una matriz gigantesca, no conviene transformar la matriz: se transforman las **coordenadas** (modificación 2).

Casos de uso:

- Rotar una imagen o matriz 90° , 180° o 270° (*Rotate Image*), incluso en el sitio si es cuadrada.
- Decidir si dos figuras o fichas son la misma salvo rotación (4 variantes) o salvo rotación y reflexión (8 variantes), y contar cuántas figuras distintas hay.
- Reducir cuatro direcciones a una: gravedad, deslizamientos tipo *2048* o simulaciones que se implementan una vez y se reutilizan rotando la grilla.
- Aplicar una **secuencia enorme** de operaciones rotar/espejar sobre una matriz grande, componiéndolas en una sola simetría (modificación 3).
- Rotar piezas de un Tetris o de un rompecabezas (siguiente subsección).

Complejidad: cada transformación copia cada celda a su destino una vez, así que cuesta $O(n \cdot m)$ en tiempo y $O(n \cdot m)$ de memoria por la matriz nueva. En una matriz **cuadrada** se puede rotar en el sitio con $O(n^2)$ en tiempo y $O(1)$ de memoria extra: la transposición hace $n(n-1)/2$ intercambios y la inversión de filas hace otros $\approx n^2/2$. **variantes** hace 8 transformaciones, es decir $O(8 \cdot n \cdot m) = O(n \cdot m)$; **formaCanonica** agrega 7 comparaciones lexicográficas de $O(n \cdot m)$ cada una, lo que sigue siendo $O(n \cdot m)$. Contar k figuras distintas con un `set` cuesta $O(k \cdot n \cdot m \cdot \log k)$.

```
1  typedef vector<vector<int>> Mat;
2
3  struct TransformacionesDeMatriz {
4      int filas, columnas;
5      Mat a;
6
7      // CREATE: guarda la matriz y sus dimensiones.
8      TransformacionesDeMatriz(const Mat& m) {
9          a = m;
10         filas = a.size();
11         columnas = a.empty() ? 0 : a[0].size();
12     }
13
14     // READ: transposicion. b[j][i] = a[i][j]. Resultado columnas
15     // x filas.
16     Mat transponer() const {
17         Mat b(columnas, vector<int>(filas));
18         for (int i = 0; i < filas; i++)
19             for (int j = 0; j < columnas; j++) b[j][i] = a[i][j];
20         return b;
21     }
22
23     // READ: espejo izquierda-derecha (invierte cada fila).
24     Mat reflejarHorizontal() const {
25         Mat b(filas, vector<int>(columnas));
26         for (int i = 0; i < filas; i++)
27             for (int j = 0; j < columnas; j++) b[i][columnas - 1 - j] = a[i][j];
28         return b;
29     }
30
31     // READ: espejo arriba-abajo (invierte el orden de las filas).
32     Mat reflejarVertical() const {
33         Mat b(filas, vector<int>(columnas));
34         for (int i = 0; i < filas; i++)
35             for (int j = 0; j < columnas; j++) b[filas - 1 - i][j] = a[i][j];
36         return b;
37     }
38
39     // READ: rotacion de 90 grados en sentido horario. Resultado
40     // columnas x filas.
41     Mat rotar90() const {
42         Mat b(columnas, vector<int>(filas));
43         for (int i = 0; i < filas; i++)
44             for (int j = 0; j < columnas; j++) b[j][filas - 1 - i] = a[i][j];
45         return b;
46     }
47
48     // READ: rotacion de 180 grados.
49     Mat rotar180() const {
50         Mat b(filas, vector<int>(columnas));
51         for (int i = 0; i < filas; i++)
52             for (int j = 0; j < columnas; j++) b[filas - 1 - i][columnas - 1 - j] = a[i][j];
```

```

52     return b;
53 } // O(filas *
    columnas)
54
55 // READ: rotacion de 90 grados en sentido antihorario (= 270
    horario).
56 Mat rotar270() const {
57     Mat b(columnas, vector<int>(filas));
58     for (int i = 0; i < filas; i++)
59         for (int j = 0; j < columnas; j++) b[columnas - 1 - j][i] =
            a[i][j];
60     return b;
61 } // O(filas *
    columnas)
62
63 // READ: k rotaciones de 90 grados horarias; k puede ser
    negativo o grande.
64 Mat rotarK(int k) const {
65     k = ((k % 4) + 4) % 4;
66     if (k == 0) return a;
67     if (k == 1) return rotar90();
68     if (k == 2) return rotar180();
69     return rotar270();
70 } // O(filas *
    columnas)
71
72 // READ: las 8 variantes: 4 rotaciones de la matriz y 4 de su
    espejo.
73 // Las posiciones 0..3 son SOLO rotaciones; la 4..7, con
    reflexion.
74 vector<Mat> variantes() const {
75     TransformacionesDeMatriz espejo(reflejarHorizontal());
76     vector<Mat> v;
77     for (int k = 0; k < 4; k++) v.push_back(rotarK(k));
78     for (int k = 0; k < 4; k++) v.push_back(espejo.rotarK(k));
79     return v;
80 } // O(8 * filas *
    columnas)
81
82 // READ: forma canonica = la menor variante (lexicografica).
    Dos figuras
83 // son equivalentes salvo simetria si y solo si tienen la
    misma.
84 Mat formaCanonica() const {
85     vector<Mat> v = variantes();
86     return *min_element(v.begin(), v.end());
87 } // O(filas *
    columnas)
88
89 // READ: true si 'otra' es esta matriz rotada (o rotada y
    reflejada,
90 // si conReflexion es true).
91 bool iguales(const Mat& otra, bool conReflexion) const {
92     vector<Mat> v = variantes();
93     int limite = conReflexion ? 8 : 4;
94     for (int k = 0; k < limite; k++) if (v[k] == otra) return
        true;
95     return false;
96 } // O(filas *
    columnas)
97
98 // WRITE: rotacion de 90 grados horaria EN EL SITIO, solo
    para
99 // matrices cuadradas: transponer y luego invertir cada fila.
100 void rotar90EnSitio() {
101     for (int i = 0; i < filas; i++)
102         for (int j = 0; j < i; j++) swap(a[i][j], a[j][i]);
103     for (int i = 0; i < filas; i++) reverse(a[i].begin(), a[i].
        end());
104 } // O(filas * filas
    ), O(1) extra
105 };
106
107 // -----
108 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
109 // -----
110
111 // 1. ROTAR EN EL SITIO POR CICLOS DE 4 (matriz cuadrada n x n):
    cada celda
112 // pertenece a un ciclo de 4 posiciones que se rota con un
    solo temporal:
113 //
114 //     for (int i = 0; i < n / 2; i++) {
115 //         for (int j = i; j < n - 1 - i; j++) {
116 //             int tmp = a[i][j];
117 //             a[i][j] = a[n - 1 - j][i];
118 //             a[n - 1 - j][i] = a[n - 1 - i][n - 1 - j];
119 //             a[n - 1 - i][n - 1 - j] = a[j][n - 1 - i];
120 //             a[j][n - 1 - i] = tmp;
121 //         }
122 //     }
123 //
124 // Hace lo mismo que rotar90EnSitio; la version con

```

```

transposicion es
// mas corta y menos propensa a errores de indices.
125
126
127 // 2. ROTAR COORDENADAS EN VEZ DE LA MATRIZ: si la matriz es
    enorme (por
128 // ejemplo  $10^9 \times 10^9$ ) y solo hay unos pocos puntos de
    interes, se
129 // mueve cada punto (i, j) de una matriz de n filas y m
    columnas:
130 //     transponer: (i, j) -> (j, i)
131 //     rotar 90 horario: (i, j) -> (j, n - 1 - i)
132 //     rotar 90 antihorario: (i, j) -> (m - 1 - j, i)
133 //     rotar 180: (i, j) -> (n - 1 - i, m - 1 - j)
134 //     espejo horizontal: (i, j) -> (i, m - 1 - j)
135 //     espejo vertical: (i, j) -> (n - 1 - i, j)
136 // Ojo: tras transponer o rotar 90, n y m se intercambian.
137
138 // 3. COMPOSER MUCHAS OPERACIONES EN UNA SOLA: si el enunciado
    da q
139 // operaciones (transponer, espejos, rotaciones) sobre una
    matriz
140 // grande, no se aplican una por una ( $O(q * n * m)$ ). Se
    aplican a un
141 // MARCADOR  $2 \times 2$  con cuatro valores distintos, que
    identifica de
142 // forma unica cual de las 8 simetrias se acumulo:
143 //
144 //     Mat marcador = {{0, 1}, {2, 3}};
145 //     for (cada operacion op del enunciado) marcador = aplicar(
        marcador, op);
146 //
147 //     auto vs = TransformacionesDeMatriz({{0, 1}, {2, 3}}).
        variantes();
148 //     int idx = find(vs.begin(), vs.end(), marcador) - vs.begin
        ();
149 //     Mat resultado = TransformacionesDeMatriz(grande).
        variantes()[idx];
150 //
151 // ('aplicar' es un switch que llama a la transformacion que
    corresponda.) Cuesta  $O(q)$  mas una sola transformacion  $O(n
    * m)$ . Sirve
152 // aunque la matriz grande no sea cuadrada, porque cada
    operacion
153 // actua sobre los indices de la misma forma sin importar el
    tamano.
154
155 // 4. ROTACION CIRCULAR DE UNA FILA NO ES ROTACION DE LA MATRIZ:
    desplazar
156 // los elementos de la fila i, k posiciones a la izquierda,
    es
157 //     rotate(a[i].begin(), a[i].begin() + k % m, a[i].end());
158 // Es una operacion distinta y mucho mas barata que rotar 90
    grados.
159 // Para una columna hay que copiarla a un vector, rotarla y
    volver a
160 // escribirla.
161
162 // 5. REDUCIR CUATRO DIRECCIONES A UNA: si solo tienes
    implementada la
163 // gravedad hacia abajo (o "mover a la izquierda" en 2048),
    las otras
164 // direcciones se obtienen asi:
165 //     hacia la derecha: rotar 90 horario (la derecha pasa
        a ser abajo),
166 //     aplicar la gravedad, rotar 90
        antihorario.
167 //     hacia la izquierda: rotar 90 antihorario, aplicar,
        rotar 90 horario.
168 //     hacia arriba: rotar 180, aplicar, rotar 180.
169 // Todas cuestan  $O(n * m)$  extra por rotacion, sin cambiar el
    orden
170 // asintotico de una simulacion que ya recorre la matriz.
171
172 // 6. TRAMPA: HORARIO VS ANTIHORARIO. Se confunden con
    facilidad. Verifica
173 // con una matriz  $2 \times 3$ : tras rotar 90 horario, la esquina
    superior
174 // izquierda (el 1 del ejemplo) debe quedar ARRIBA A LA
    DERECHA.
175
176 // 7. TRAMPA: LAS DIMENSIONES SE INTERCAMBIAN. Rotar 90 o
    transponer una
177 // matriz  $n \times m$  da una  $m \times n$ ; crear la matriz resultado con
    las
178 // dimensiones nuevas y usar las dimensiones ORIGINALES en
    la formula
179 // de indices. En el sitio solo se puede si es cuadrada.
180
181 // 8. TRAMPA: LA REFLEXION NO SALE DE LAS ROTACIONES. Las
    piezas S y Z del
182 // Tetris son espejos una de la otra y ninguna rotacion
    convierte una
183 // en la otra. El enunciado dice si las reflexiones estan

```

```

    permitidas:
185 // 4 variantes (solo rotaciones) o 8 (con reflexion). Si la
    figura ya
186 // tiene simetria propia, varias variantes salen repetidas;
    usa un set
187 // para quedarte con las distintas.
188
189 // 9. TRAMPA: FIGURAS EN CAJAS DE DISTINTO TAMANO. Para
    comparar figuras
190 // con formaCanonica, primero recorta el borde vacio de cada
    una; si
191 // no, la misma figura dentro de una caja mas grande parece
    distinta.

```

Ejemplo de funcionamiento:

```

1 Mat m = {{1, 2, 3},
2           {4, 5, 6}};
3 TransformacionesDeMatriz t(m);
4
5 t.transponer();           // 1 4
6 // 2 5
7 // 3 6
8
9 t.reflejarHorizontal();   // 3 2 1
10 // 6 5 4
11
12 t.reflejarVertical();     // 4 5 6
13 // 1 2 3
14
15 t.rotar90();              // 4 1      (el 1 pasa arriba a la
    derecha)
16 // 5 2
17 // 6 3
18
19 t.rotar180();             // 6 5 4
20 // 3 2 1
21
22 t.rotar270();             // 3 6
23 // 2 5
24 // 1 4
25
26 // Cuantas fichas 2 x 2 de celdas llenas/vacias son distintas
    salvo
27 // rotacion y reflexion? Se prueban las 16 y se cuentan las
    canonicas.
28 set<Mat> formas;
29 for (int mask = 0; mask < 16; mask++) {
30     Mat p = {{mask & 1, (mask >> 1) & 1},
31              {(mask >> 2) & 1, (mask >> 3) & 1}};
32     formas.insert(TransformacionesDeMatriz(p).formaCanonica());
33 }
34 cout << formas.size();
35 // 6 (vacía, una celda, dos adyacentes, dos en diagonal, tres,
    llena)
36
37 // Piezas S y Z del Tetris: espejos una de la otra.
38 Mat S = {{0, 1, 1},
39           {1, 1, 0}};
40 Mat Z = {{1, 1, 0},
41           {0, 1, 1}};
42 TransformacionesDeMatriz ts(S);
43 cout << ts.iguales(Z, false) << " " << ts.iguales(Z, true);
44 // 0 1 (ninguna rotacion las iguala, con reflexion si)
45
46 // La L de tres celdas en sus 4 rotaciones tiene una sola forma
    canonica.
47 Mat L = {{1, 0},
48           {1, 1}};
49 set<Mat> ls;
50 for (int k = 0; k < 4; k++)
51     ls.insert(TransformacionesDeMatriz(TransformacionesDeMatriz(L).
        rotarK(k)).formaCanonica());
52 cout << ls.size();
53 // 1

```

Nota: las identidades entre operaciones son útiles como **prueba**: para cualquier matriz, transponer y luego reflejar horizontalmente debe dar lo mismo que **rotar90**, transponer y luego reflejar verticalmente debe dar **rotar270**, y cuatro rotaciones de 90° deben devolver la matriz original. Si tu código no cumple alguna de las tres, hay un índice mal puesto. En el ejemplo de la ficha 2×2 , la respuesta 6 coincide con las seis clases que se cuentan a mano (por número de celdas llenas, con dos clases distintas para dos celdas), lo cual sirve de control del método.

Regla práctica: rota cuando quieras **evitar escribir el mismo código cuatro veces** (una simulación para cada dirección) o cuando necesites decidir si dos figuras son *la misma*. Para una sola rotación de una matriz cuadrada, transponer e invertir cada fila es la forma más corta y segura; para muchas operaciones sobre una matriz grande, compónlas en una de las 8 simetrías con el marcador 2×2 ; y para comparar figuras, usa **formaCanonica** dentro de un **set**, decidiendo antes si el enunciado permite reflexiones (8 variantes) o solo rotaciones (4).

48.3 Gravedad y Caída de Piezas (Tetris, Candy Crush)

Idea: en este tipo de problemas todo lo que no está vacío **cae** hasta apoyarse en el fondo o en otra celda. Hay dos modelos. En el modelo **celda a celda** (Candy Crush, arena, Puyo Puyo) cada columna es independiente, y aplicar la gravedad equivale a empujar las celdas no vacías hacia abajo conservando su orden relativo: se recorre la columna de abajo hacia arriba con un **puntero de escritura pos** que marca la próxima posición libre desde el fondo; cada celda no vacía se escribe en **pos** y **pos** sube una casilla. Es una partición estable con dos punteros, $O(\text{filas})$ por columna. En el modelo de **pieza rígida** (Tetris) un grupo de celdas se mueve como una unidad: cae mientras **cabe** en la fila siguiente, se **fija** al chocar, y luego se eliminan las filas completas. Candy Crush es un bucle sobre la plantilla de *Plantilla de Simulación (Estado, Paso y Parada)*: el estado es el tablero, el paso es *marcar grupos, eliminarlos, aplicar gravedad*, y la parada es que ya no se marque nada. La regla que no se puede omitir es marcar **todos** los grupos antes de borrar ninguno, por la misma razón que en *Autómatas Celulares (Game of Life)*: la actualización debe ser simultánea.

¿Por qué usarlo? La alternativa ingenua es mover cada celda una posición y repetir hasta que nada cambie, lo que puede costar $O(\text{filas} \cdot \text{filas} \cdot \text{columnas})$; el puntero de escritura logra el estado final de la gravedad en una sola pasada de $O(\text{filas} \cdot \text{columnas})$. Para las otras tres direcciones no hace falta reescribir nada: se rota el tablero como en *Rotación, Transposición y Reflexión* (modificación 5 de esa subsección) o se refleja el puntero (modificación 1). Frente a una simulación de **objetos sueltos** (*Simulación por Eventos*), aquí toda la grilla cambia en cada ronda y la simulación por rondas es la forma natural.

Casos de uso:

- Candy Crush, Bejeweled o SameGame: eliminar grupos de ≥ 3 , dejar caer y repetir, contando celdas eliminadas o cascadas.
- Tetris: en qué fila aterriza una pieza, cuántas líneas completa y cuándo termina el juego.
- Arena, agua o rocas que caen, con obstáculos fijos que las detienen (modificación 2).
- Puyo Puyo y juegos de grupos conectados, donde el grupo se detecta con *Flood Fill* en vez de por filas y columnas (modificación 4).

- Deslizamientos tipo 2048: comprimir una línea hacia un extremo usa exactamente el mismo puntero de escritura.

Complejidad: aplicarGravedad recorre cada columna una vez con un solo puntero, así que cuesta $O(\text{filas} \cdot \text{columnas})$. marcarGrupos recorre cada fila y cada columna detectando *tandas* de celdas iguales; el índice avanza siempre hasta el final de la tanda, así que también es $O(\text{filas} \cdot \text{columnas})$. Una ronda cuesta lo mismo. Una cascada completa hace a lo sumo $\text{filas} \cdot \text{columnas}/3$ rondas, porque toda ronda que continúa elimina al menos 3 celdas, para un peor caso de $O((\text{filas} \cdot \text{columnas})^2/3)$, aunque en la práctica se hacen muy pocas. En Tetris, cabe cuesta $O(|p|)$ con $|p|$ celdas en la pieza, filaDeCaída hace a lo sumo filas intentos ($O(\text{filas} \cdot |p|)$) y limpiarLineas cuesta $O(\text{filas} \cdot \text{columnas})$.

```
1  typedef vector<pair<int,int>> Pieza; // celdas (di, dj)
   // relativas a un ancla
2
3  struct CaídaDePiezas {
4      int filas, columnas;
5      vector<vector<int>>> t; // 0 = vacía; otro valor =
   // color o tipo
6      int rondas; // rondas de la última
   // simulación
7
8      // CREATE: guarda el tablero (la fila 0 es la de ARRIBA).
9      CaídaDePiezas(const vector<vector<int>>>& tablero) {
10         t = tablero;
11         filas = t.size();
12         columnas = t.empty() ? 0 : t[0].size();
13         rondas = 0;
14     } // O(filas *
   // columnas)
15
16     // WRITE: gravedad celda a celda. Por cada columna, de abajo
   // hacia
17     // arriba, cada celda no vacía se escribe en 'pos' (la
   // próxima
18     // posición libre desde el fondo).
19     void aplicarGravedad() {
20         for (int j = 0; j < columnas; j++) {
21             int pos = filas - 1;
22             for (int i = filas - 1; i >= 0; i--) {
23                 if (t[i][j] != 0) {
24                     int v = t[i][j];
25                     t[i][j] = 0; // se vacía ANTES de
   // escribir
26                     t[pos][j] = v;
27                     pos--;
28                 }
29             }
30         }
31     } // O(filas *
   // columnas)
32
33     // READ: marca las celdas que forman una tanda de 'mínimo' o
   // más celdas
34     // iguales, en horizontal o vertical. No modifica el tablero.
35     vector<vector<bool>> marcarGrupos(int minimo = 3) const {
36         vector<vector<bool>> marca(filas, vector<bool>(columnas,
   // false));
37         for (int i = 0; i < filas; i++) { // tandas por
   // filas
38             int j = 0;
39             while (j < columnas) {
40                 int k = j;
41                 while (k < columnas && t[i][k] == t[i][j]) k++;
42                 if (t[i][j] != 0 && k - j >= minimo)
43                     for (int x = j; x < k; x++) marca[i][x] = true;
44                 j = k;
45             }
46         }
47         for (int j = 0; j < columnas; j++) { // tandas por
   // columnas
48             int i = 0;
49             while (i < filas) {
50                 int k = i;
51                 while (k < filas && t[k][j] == t[i][j]) k++;
52                 if (t[i][j] != 0 && k - i >= minimo)
53                     for (int x = i; x < k; x++) marca[x][j] = true;
54                 i = k;
55             }
56         }
57     }
```

```
57     return marca;
58 } // O(filas *
   // columnas)
59
60 // WRITE: vacía las celdas marcadas; retorna cuantas eran.
61 int eliminarMarcadas(const vector<vector<bool>>& marca) {
62     int quitadas = 0;
63     for (int i = 0; i < filas; i++)
64         for (int j = 0; j < columnas; j++)
65             if (marca[i][j]) { t[i][j] = 0; quitadas++; }
66     return quitadas;
67 } // O(filas *
   // columnas)
68
69 // WRITE: Candy Crush completo: marcar TODO, eliminar,
   // gravedad, y
70 // repetir hasta que no se marque nada. Retorna las celdas
   // eliminadas.
71 int simularCandyCrush(int minimo = 3) {
72     int total = 0;
73     rondas = 0;
74     while (true) {
75         vector<vector<bool>> marca = marcarGrupos(minimo);
76         int quitadas = eliminarMarcadas(marca);
77         if (quitadas == 0) break; // estado estable
78         total += quitadas;
79         rondas++;
80         aplicarGravedad();
81     }
82     return total;
83 } // O(rondas *
   // filas * columnas)
84
85 // READ: true si la pieza, con su ancla en (f, c), queda
   // dentro del
86 // tablero y sin tocar celdas ocupadas.
87 bool cabe(const Pieza& p, int f, int c) const {
88     for (auto [di, dj] : p) {
89         int i = f + di, j = c + dj;
90         if (i < 0 || i >= filas || j < 0 || j >= columnas) return
   // false;
91         if (t[i][j] != 0) return false;
92     }
93     return true;
94 } // O(|p|)
95
96 // READ: fila del ancla donde aterriza la pieza al soltarla
   // desde la
97 // fila 0 en la columna c; -1 si ni siquiera cabe al aparecer
   // (fin
98 // del juego).
99 int filaDeCaída(const Pieza& p, int c) const {
100     if (!cabe(p, 0, c)) return -1;
101     int f = 0;
102     while (cabe(p, f + 1, c)) f++;
103     return f;
104 } // O(filas * |p|)
105
106 // WRITE: escribe la pieza en el tablero con el valor dado.
107 void fijar(const Pieza& p, int f, int c, int valor) {
108     for (auto [di, dj] : p) t[f + di][c + dj] = valor;
109 } // O(|p|)
110
111 // WRITE: elimina las filas completas y baja lo de arriba.
   // Puntero
112 // de escritura de abajo hacia arriba, sin borrar filas en el
   // bucle.
113 int limpiarLineas() {
114     int pos = filas - 1, limpias = 0;
115     for (int i = filas - 1; i >= 0; i--) {
116         bool llena = true;
117         for (int j = 0; j < columnas; j++)
118             if (t[i][j] == 0) { llena = false; break; }
119         if (llena) { limpias++; continue; } // esta fila
   // desaparece
120         if (pos != i) t[pos] = t[i];
121         pos--;
122     }
123     for (int i = pos; i >= 0; i--) t[i].assign(columnas, 0);
124     return limpias;
125 } // O(filas *
   // columnas)
126 }
127
128 // -----
129 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
130 // -----
131
132 // 1. GRAVEDAD HACIA LA DERECHA (sin rotar): el mismo puntero,
   // pero por
133 // filas y de derecha a izquierda:
134 //
135 // for (int i = 0; i < filas; i++) {
```



```

136 //      int pos = columnas - 1;
137 //      for (int j = columnas - 1; j >= 0; j--)
138 //          if (t[i][j] != 0) { int v = t[i][j]; t[i][j] =
139 //              0; t[i][pos] = v; pos--; }
140 //      }
141 //      Para izquierda, arriba: el puntero parte del otro extremo
142 //      y avanza
143 //      en sentido contrario. Si prefieres no duplicar código,
144 //      rota el
145 //      tablero (ver Rotacion, Transposicion y Reflexion).
146 // 2. OBSTACULOS FIJOS (piedras que no caen y detienen lo de
147 //      arriba): al
148 //      encontrar un obstaculo el puntero se reubica justo encima
149 //      de el:
150 //      for (int j = 0; j < columnas; j++) {
151 //          int pos = filas - 1;
152 //          for (int i = filas - 1; i >= 0; i--) {
153 //              if (t[i][j] == -1) pos = i - 1; //
154 //              -1 = obstaculo
155 //              else if (t[i][j] != 0) {
156 //                  int v = t[i][j]; t[i][j] = 0; t[pos][j]
157 //                      = v; pos--;
158 //              }
159 //          }
160 //      }
161 //      Cada tramo entre dos obstaculos se comporta como una
162 //      columna
163 //      independiente. Es el nucleo de problemas tipo "Rotating
164 //      the Box".
165 // 3. CAIDA RAPIDA CON ALTURAS: en vez de probar fila por fila,
166 //      guarda
167 //      para cada columna el índice de su celda llena mas alta,
168 //      top[j] (= filas si esta vacia). Para una pieza con 'baja[
169 //      k]' =
170 //      mayor di de las celdas de su columna k (k = 0..ancho-1):
171 //      int f = INT_MAX;
172 //      for (int k = 0; k < ancho; k++)
173 //          if (baja[k] >= 0) f = min(f, top[c + k] - 1 - baja[k
174 //      ]);
175 //      // f es la fila de aterrizaje; f < 0 significa que no
176 //      cabe
177 //      Cuesta O(ancho) en vez de O(filas * |p|). Supone que la
178 //      pieza
179 //      aparece POR ENCIMA de la pila (ninguna de sus celdas
180 //      queda debajo
181 //      de una celda llena de su columna); es lo normal en Tetris.
182 //      Tras fijar la pieza, actualiza top[] solo en sus columnas.
183 // 4. GRUPOS CONECTADOS (Puyo Puyo, SameGame): en vez de tandas
184 //      por filas
185 //      y columnas, se busca con Flood Fill cada componente de
186 //      celdas del
187 //      mismo color y se marca entera si tiene al menos 'minimo'
188 //      celdas.
189 //      La marca y la eliminacion siguen siendo dos pasos
190 //      separados.
191 // 5. ROTAR LA PIEZA ANTES DE SOLTARLA: una pieza como lista de
192 //      celdas
193 //      (di, dj) gira 90 grados en sentido horario con
194 //      (di, dj) -> (dj, -di). Despues se normaliza restando el
195 //      menor di y
196 //      el menor dj, para que todos los desplazamientos sean >= 0.
197 //      Con las
198 //      4 rotaciones (y, si se permite, las reflexiones) se
199 //      prueba en que
200 //      columna y con que orientacion conviene soltarla.
201 // 6. FIN DEL JUEGO Y PUNTAJE: filaDeCaida(p, c) == -1
202 //      significa que la
203 //      pieza no cabe al aparecer. El puntaje suele ser una
204 //      funcion de las
205 //      lineas que devuelve limpiarLineas(); confirma en el
206 //      enunciado si
207 //      cuentan las lineas por separado o si hay bonos por varias
208 //      a la vez.
209 // 7. MODELO "UN PASO POR TICK": si cada celda cae UNA casilla
210 //      por
211 //      unidad de tiempo (arena que cae poco a poco), recorre las
212 //      filas de
213 //      ABAJO hacia ARRIBA en cada tick; de arriba hacia abajo
214 //      una misma
215 //      celda caería varias casillas en un solo tick. Es el paso

```

```

197 //      de la
198 //      plantilla de simulacion, y la gravedad completa de arriba
199 //      es
200 //      repetir ese paso hasta que nada cambie.
201 // 8. TRAMPA: MARCAR ANTES DE BORRAR. Si se borra cada tanda
202 //      apenas se
203 //      detecta, una forma en T o en L pierde la celda que
204 //      compartían la
205 //      tanda horizontal y la vertical, y la segunda tanda puede
206 //      dejar de
207 //      detectarse. Por eso marcarGrupos devuelve marcas y las
208 //      celdas de
209 //      interseccion se cuentan una sola vez.
210 // 9. TRAMPA: ELIMINAR FILAS DENTRO DEL BUCLE. Llamar a erase
211 //      sobre las
212 //      filas mientras se recorren salta la fila siguiente a la
213 //      borrada.
214 //      limpiarLineas evita el problema copiando cada fila que se
215 //      queda al
216 //      puntero 'pos' y vaciando lo que sobra arriba al final.
217 // 10. TRAMPA: ARRIBA Y ABAJO. La fila 0 es la de arriba, así
218 //      que "caer"
219 //      es ir hacia filas MAYORES. Además, el valor vacío (0
220 //      aquí) no
221 //      puede coincidir con ningún color ni tipo de pieza.

```

Ejemplo de funcionamiento:

```

1 // Candy Crush: tablero de 5 filas y 4 columnas (0 = vacío).
2 // 4 1 4 4
3 // 3 2 3 4
4 // 4 2 4 3
5 // 3 2 1 1
6 // 4 3 3 4
7 CaidaDePiezas cc({{4, 1, 4, 4},
8     {3, 2, 3, 4},
9     {4, 2, 4, 3},
10    {3, 2, 1, 1},
11    {4, 3, 3, 4}});
12
13 // Ronda 1: solo hay una tanda, los tres 2 de la columna 1, y
14 // se eliminan
15 // 3 celdas. Tras la gravedad, el 1 de arriba cae junto a los
16 // otros dos:
17 // 4 . 4 4
18 // 3 . 3 4
19 // 4 . 4 3
20 // 3 1 1 1      <- ahora hay una tanda horizontal de tres 1
21 // 4 3 3 4
22 //
23 // Ronda 2: se eliminan los tres 1 y todo cae de nuevo:
24 // 4 . . .
25 // 3 . 4 4
26 // 4 . 3 4
27 // 3 . 4 3
28 // 4 3 3 4
29 //
30 // Ronda 3: no se marca nada, se detiene.
31
32 int total = cc.simularCandyCrush();
33 cout << total << " " << cc.rondas;
34 // 6 2      (6 celdas eliminadas en 2 rondas con eliminacion)
35
36 // Tetris: tablero de 5 filas y 4 columnas, con la fila de
37 // abajo
38 // llena salvo la columna 3. Se suelta una pieza I vertical (4
39 // celdas)
40 // en la columna 3.
41 vector<vector<int>> vacio(5, vector<int>(4, 0));
42 vacio[4][0] = vacio[4][1] = vacio[4][2] = 1;
43 CaidaDePiezas juego(vacio);
44 Pieza I = {{0, 0}, {1, 0}, {2, 0}, {3, 0}};
45
46 int f = juego.filaDeCaida(I, 3);
47 cout << f; // 1 (ocupa las filas 1 a 4)
48 juego.fijar(I, f, 3, 2);
49 // . . . .
50 // . . . 2
51 // . . . 2
52 // . . . 2
53 // 1 1 1 2      <- fila completa
54
55 cout << juego.limpiarLineas(); // 1
56 // . . . .
57 // . . . .
58 // . . . 2      <- lo de arriba bajo una fila
59 // . . . 2
60 // . . . 2

```

Nota: en el ejemplo de Candy Crush la segunda ronda **no existía** en el tablero inicial: la tanda de los 1 aparece solo después de que la gravedad acerca los dos 1 de la fila 3 a un tercero. Por eso una solución que detecte los grupos una sola vez, sin repetir después de la gravedad, da la respuesta incorrecta. Además, `simularCandyCrush` cuenta **celdas** eliminadas; si el enunciado puntúa por grupos, por tamaño o por cascada, hay que contarlos en `marcarGrupos` o en cada ronda.

Regla práctica: para gravedad celda a celda usa el puntero de escritura columna por columna, y si hay obstáculos fijos reubícalo encima de cada uno. Para Candy Crush usa siempre el ciclo *marcar todo, eliminar, gravedad, repetir hasta estabilizar*, y para Tetris usa *cabe* para la caída, *fijar* al aterrizar y *limpiarLineas* con puntero de escritura, nunca borrando filas dentro del bucle. Si necesitas otra dirección de caída, rota el tablero en vez de duplicar el código.

48.4 Direcciones y Vecindarios (dx/dy, 8 Direcciones, Límites)

Idea: casi todo problema sobre una grilla pregunta por los **vecinos** de una celda (i, j) . En vez de escribir cuatro u ocho `if`, se guardan los desplazamientos en dos arreglos paralelos `dx[]` y `dy[]` y se recorre el vecindario con un bucle: $(ni, nj) = (i + dx[d], j + dy[d])$. En esta guía la fila i crece hacia abajo, así que `dx` cambia la **fila** y `dy` la **columna**. Hay tres reglas que evitan casi todos los errores. Primera: comprobar los límites **antes** de acceder a la celda. Segunda: ordenar las direcciones en **sentido horario**, empezando hacia arriba, porque así los giros y los opuestos son aritmética ($d+1$ gira a la derecha, $d+3$ a la izquierda, y el opuesto es $d \oplus 2$ con 4 direcciones o $d \oplus 4$ con 8). Tercera: definir los arreglos una vez y usarlos en todo el programa. Los vecindarios habituales son el de **von Neumann** (4 direcciones ortogonales), el de **Moore** (8, con diagonales), los 8 saltos del **caballo**, y las 6 direcciones de una grilla **hexagonal**.

¿Por qué usarlo? Copiar un `if` por vecino multiplica el código y los errores, mientras que con los arreglos el mismo bucle sirve para 4 u 8 direcciones cambiando solo el límite del bucle. Es el ingrediente común de *Flood Fill*, *Laberinto con BFS*, *Tour de Caballo de Ajedrez*, la cuenta de vecinas vivas de *Autómatas Celulares (Game of Life)* y la espiral con caminante de *Recorridos Especiales*. Para los límites hay tres opciones: la función **dentro**, que no modifica el tablero y sirve con cualquier tamaño; el **borde de centinelas**, que evita la comprobación rodeando la grilla de paredes (modificación 1); y el truco de **unsigned**, que resume dos comparaciones en una.

Casos de uso:

- Visitar los vecinos de una celda en BFS, DFS o Flood Fill, con 4 o con 8 direcciones.
- Buscaminas: cada celda cuenta las minas de sus 8 vecinas.
- Movimientos del caballo o del rey, y piezas que **deslizan** (torre, alfil, reina, hielo) hasta chocar.
- Detectar líneas de k celdas iguales (tres en raya, Conecta 4, Gomoku): solo hacen falta 4 de las 8 direcciones, contando en ambos sentidos.
- Robots o caminantes con orientación, que giran a la derecha, a la izquierda o dan la vuelta.
- Grillas hexagonales con coordenadas axiales (modificación 5).

Complejidad: recorrer el vecindario de una celda cuesta $O(k)$ con k constante (4, 6 u 8), así que visitar los vecinos de toda la grilla es $O(n \cdot m \cdot k) = O(n \cdot m)$. dentro es $O(1)$, buscaminas cuesta $O(8 \cdot n \cdot m)$, deslizar cuesta $O(\max(n, m))$ porque avanza a lo sumo hasta el otro borde, y hizoLinea cuesta $O(4 \cdot k)$ por consulta, ya que avanza como máximo $k - 1$ pasos por cada sentido de cada una de las 4 direcciones.

```

1 // Direcciones en sentido HORARIO empezando hacia arriba. La
  // fila crece
2 // hacia abajo: dx cambia la FILA y dy cambia la COLUMNA.
3 const int DX4[4] = {-1, 0, 1, 0}; // arriba, derecha,
  // abajo, izquierda
4 const int DY4[4] = {0, 1, 0, -1};
5 const int DX8[8] = {-1, -1, 0, 1, 1, 1, 0, -1}; // +
  // diagonales, horario
6 const int DY8[8] = {0, 1, 1, 1, 0, -1, -1, -1};
7 const int DXC[8] = {-2, -2, -1, -1, 1, 1, 2, 2}; // saltos
  // del caballo
8 const int DYC[8] = {-1, 1, -2, 2, -2, 2, -1, 1};
9 const int DXL[4] = {0, 1, 1, 1}; // media
  // circunferencia: para
10 const int DYL[4] = {1, 0, 1, -1}; // contar líneas (se
  // cuenta en ambos sentidos)
11
12 struct Vecindarios {
13     int filas, columnas;
14     vector<string> g; // tablero de
15     // caracteres
16
17     // CREATE: guarda el tablero.
18     Vecindarios(const vector<string>& tablero) {
19         g = tablero;
20         filas = g.size();
21         columnas = g.empty() ? 0 : g[0].size(); // O(filas *
22         // columnas)
23
24         // READ: true si (i, j) esta dentro del tablero. El truco
25         // unsigned: un
26         // entero negativo convertido a unsigned es enorme y falla la
27         // comparacion, asi que una sola comparacion por eje cubre
28         // los dos lados.
29         bool dentro(int i, int j) const {
30             return (unsigned)i < (unsigned)filas && (unsigned)j < (
31                 unsigned)columnas; // O(1)
32         }
33
34         // READ: vecinos validos de (i, j), con 4 u 8 direcciones.
35         vector<pair<int,int>> vecinos(int i, int j, bool ocho) const
36         {
37             vector<pair<int,int>> res;
38             int k = ocho ? 8 : 4;
39             for (int d = 0; d < k; d++) {
40                 int ni = i + (ocho ? DX8[d] : DX4[d]);
41                 int nj = j + (ocho ? DY8[d] : DY4[d]);
42                 if (dentro(ni, nj)) res.push_back({ni, nj});
43             }
44             return res; // O(k)
45         }
46
47         // READ: cuantos vecinos de (i, j) son el caracter c.
48         int contarAdyacentes(int i, int j, char c, bool ocho) const {
49             int cuenta = 0;
50             for (auto [ni, nj] : vecinos(i, j, ocho))
51                 if (g[ni][nj] == c) cuenta++;
52             return cuenta; // O(k)
53         }
54
55         // READ: buscaminas: cada celda que no es mina ('*') se
56         // reemplaza por
57         // la cantidad de minas en sus 8 vecinas ( '.' si es cero).
58         vector<string> buscaminas() const {
59             vector<string> res = g;
60             for (int i = 0; i < filas; i++)
61                 for (int j = 0; j < columnas; j++)
62                     if (g[i][j] != '*') {
63                         int n = contarAdyacentes(i, j, '*', true);
64                         res[i][j] = n ? char('0' + n) : '.';
65                     }
66             return res;
67         }
68     }
69 }

```

```

61     }                                     // 0(8 * filas *
62     columnas)
63     // READ: desde (i, j) avanza en la direccion d (índice de DX4
64     // hasta el borde o hasta una pared '#'; retorna la celda
65     // final.
66     // Es el movimiento de hielo o de una torre.
67     pair<int,int> deslizar(int i, int j, int d) const {
68         while (dentro(i + DX4[d], j + DY4[d]) && g[i + DX4[d]][j +
69         DY4[d]] != '#') {
70             i += DX4[d];
71             j += DY4[d];
72         }
73         return {i, j};
74     }                                     // 0(max(filas,
75     columnas))
76     // READ: true si la celda (i, j) pertenece a una línea de al
77     // menos k
78     // celdas iguales. Solo se prueban 4 direcciones y se cuenta
79     // hacia
80     // adelante y hacia atras, porque la linea es la misma vista
81     // de
82     // cualquiera de sus dos extremos.
83     bool hizoLinea(int i, int j, int k) const {
84         char c = g[i][j];
85         for (int d = 0; d < 4; d++) {
86             int cuenta = 1;
87             for (int s = 1; s < k; s++) {           // hacia adelante
88                 int ni = i + s * DXL[d], nj = j + s * DYL[d];
89                 if (!dentro(ni, nj) || g[ni][nj] != c) break;
90                 cuenta++;
91             }
92             for (int s = 1; s < k; s++) {           // hacia atras
93                 int ni = i - s * DXL[d], nj = j - s * DYL[d];
94                 if (!dentro(ni, nj) || g[ni][nj] != c) break;
95                 cuenta++;
96             }
97             if (cuenta >= k) return true;
98         }
99         return false;
100     }                                     // 0(4 * k)
101 };
102 // -----
103 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
104 // -----
105 // 1. BORDE DE CENTINELAS: se rodea el tablero con una fila y
106 // una columna
107 // extra de paredes; los indices utiles pasan a ser 1..n y
108 // 1..m y ya no
109 // hace falta comprobar los limites, porque un vecino "fuera
110 // cae en la
111 // pared y se descarta con el mismo chequeo que cualquier
112 // otra pared:
113 //
114 // vector<string> P(n + 2, string(m + 2, '#'));
115 // for (int i = 0; i < n; i++)
116 //     for (int j = 0; j < m; j++) P[i + 1][j + 1] = a[i][j];
117 //
118 // // vecino: P[i + DX4[d]][j + DY4[d]] == '#' -> pared o
119 // // borde
120 //
121 // Ahorra una comprobacion en el bucle mas caliente de un
122 // BFS. No sirve
123 // si "fuera del tablero" no debe tratarse como pared (por
124 // ejemplo, un
125 // mundo toroidal).
126 // 2. INDICE UNICO id = i * m + j: los vecinos de arriba y
127 // abajo son
128 // id - m e id + m (comprobando 0 <= id < n * m), pero los
129 // de izquierda
130 // y derecha id - 1 e id + 1 exigen mirar la columna, porque
131 // de lo
132 // contrario "el vecino izquierdo" de la columna 0 es la
133 // ultima celda
134 // de la fila anterior:
135 //
136 // int j = id % m;
137 // if (j > 0) { /* vecino izquierdo: id - 1 */ }
138 // if (j < m - 1) { /* vecino derecho: id + 1 */ }
139 //
140 // 3. MUNDO TOROIDAL (los bordes dan la vuelta): en vez de
141 // descartar el
142 // vecino, se envuelve con modulo. Se suma n antes del %
143 // para que el
144 // resultado nunca sea negativo:
145 //
146 // int ni = (i + DX4[d] + n) % n;
147 // int nj = (j + DY4[d] + m) % m;
148 //
149 // Solo es correcto si |dx| <= n y |dy| <= m; con saltos
150 // mayores usa
151 // ((x % n) + n) % n.
152 //
153 // 4. OTRAS PIEZAS: el rey usa las 8 direcciones de DX8/DY8 con
154 // un solo
155 // paso; el caballo usa DXC/DYC (8 saltos, cada uno valido
156 // si cae dentro
157 // del tablero); la torre desliza en los indices pares de
158 // DX8/DY8
159 // (0, 2, 4, 6 = ortogonales); el alfil desliza en los
160 // impares
161 // (1, 3, 5, 7 = diagonales); y la reina desliza en los 8.
162 // Deslizar es
163 // repetir el paso mientras la celda siguiente este dentro y
164 // libre, como
165 // hace deslizar().
166 //
167 // 5. GRILLA HEXAGONAL: con coordenadas axiales (q, r), las 6
168 // direcciones
169 // son siempre las mismas, sin depender de la paridad de la
170 // fila:
171 //
172 // const int HQ[6] = {1, 1, 0, -1, -1, 0};
173 // const int HR[6] = {0, -1, -1, 0, 1, 1};
174 //
175 // La direccion opuesta a d es (d + 3) % 6, y la distancia
176 // entre dos
177 // celdas es max(|dq|, |dr|, |dq + dr|). Si el enunciado da
178 // un
179 // "odd-r" o "even-q" en vez de axiales, convierte las
180 // coordenadas
181 // antes de trabajar.
182 //
183 // 6. DIRECCIONES DADAS COMO LETRAS: se convierten en un indice
184 // con una
185 // cadena que siga el MISMO orden que DX4/DY4:
186 //
187 // int d = string("URDL").find(c);           // U=arriba R=
188 // derecha D=abajo L=izquierda
189 // int d = string("NESO").find(c);           // en espanol: N
190 // E S O
191 // // si find devuelve npos, la letra no es valida
192 //
193 // 7. GIROS Y OPUESTOS con direcciones en sentido horario:
194 //
195 // int derecha = (d + 1) % 4;
196 // int izquierda = (d + 3) % 4;           // NO (d - 1) %
197 // 4: en C++ es negativo
198 // int atras = d ^ 2;           // en 8
199 // direcciones: d ^ 4
200 //
201 // Vale para cualquier orden circular consistente (horario o
202 // antihorario), pero hay que usar el mismo en todo el
203 // programa.
204 //
205 // 8. TRAMPA: LIMITES ANTES DE ACCEDER. En una condicion como
206 // "dentro(ni, nj) && g[ni][nj] == c" el && evalua dentro
207 // primero y no
208 // llega a acceder cuando esta fuera. Invertir el orden lee
209 // memoria
210 // invalida. Con el truco unsigned, ambos operandos deben
211 // convertirse a
212 // unsigned; convertir solo uno vuelve a comparar como
213 // enteros con signo.
214 //
215 // 9. TRAMPA: FILA Y COLUMNA, X E Y. Aqui dx es la fila (
216 // vertical) y dy la
217 // columna (horizontal), lo contrario de la convencion
218 // cartesiana en la
219 // que x es horizontal. Además, en un plano cartesiano "
220 // arriba" es
221 // y + 1, mientras que en la grilla es fila - 1. Si el
222 // enunciado da
223 // coordenadas (x, y), convierte antes de usar los arreglos.
224 //
225 // 10. TRAMPA: EL DESPLAZAMIENTO (0, 0). Al generar las 8
226 // direcciones con
227 // dos bucles de -1 a 1, la pareja (0, 0) es la propia
228 // celda; hay que
229 // saltarla o la celda se cuenta como su propia vecina.
230 //
231 // 11. TRAMPA: DIAGONALES Y COSTOS. Con 8 direcciones y todos
232 // los pasos de
233 // costo 1, sigue valiendo BFS. Si la diagonal cuesta raiz
234 // de 2 (o
235 // cualquier otro costo), la distancia ya no crece de a uno
236 // y hace
237 // falta Dijkstra. Y algunos enunciados prohíben "cortar
238 // esquinas"
239 // (pasar en diagonal entre dos paredes); confirmalo antes
240 // de

```

```
192 // codificar.
```

Ejemplo de funcionamiento:

```
1 // Buscaminas: '*' es una mina.
2 Vecindarios v({"*...",
3   "...",
4   ".*.",
5   "..."});
6
7 for (const string& fila : v.buscaminas()) cout << fila << "\n";
8 // *.
9 // 221.
10 // 1*1.
11 // 111.
12
13 // Vecinos de la esquina (0, 0): solo los que estan dentro del
14 // tablero.
15 for (auto [i, j] : v.vecinos(0, 0, true)) cout << "(" << i << "
16   ", << j << ")";
17 // (0,1)(1,1)(1,0)      8 direcciones: 3 vecinos
18 for (auto [i, j] : v.vecinos(0, 0, false)) cout << "(" << i <<
19   ", << j << ")";
20 // (0,1)(1,0)          4 direcciones: 2 vecinos
21
22 cout << v.contarAdyacentes(1, 1, '*', true) << " "
23   << v.contarAdyacentes(1, 1, '*', false);
24 // 2 1 (con diagonales ve la mina de (0,0); sin ellas, solo
25 // la de (2,1))
26
27 // Hielo: '#' es una pared. Se desliza hasta chocar.
28 Vecindarios h({"..#",
29   "...",
30   "#.."});
31
32 auto p = h.deslizar(0, 0, 1); // derecha desde (0,0)
33 cout << p.first << ", " << p.second; // 0,1 (la pared esta en
34   (0,2))
35 p = h.deslizar(0, 0, 2); // abajo desde (0,0)
36 cout << p.first << ", " << p.second; // 2,0 (llega al borde
37   inferior)
38 p = h.deslizar(1, 3, 3); // izquierda desde (1,3)
39 cout << p.first << ", " << p.second; // 1,0 (llega al borde
40   izquierdo)
41 p = h.deslizar(2, 2, 0); // arriba desde (2,2)
42 cout << p.first << ", " << p.second; // 1,2 (la pared esta en
43   (0,2))
44
45 // Lineas de 4 (como en Conecta 4): diagonal completa de 4 X.
46 Vecindarios l({"X...",
47   ".X..",
48   "..X.",
49   "...X"});
50 cout << l.hizoLinea(0, 0, 4) << " " << l.hizoLinea(1, 1, 4) <<
51   " " << l.hizoLinea(2, 2, 5);
52 // 1 1 0 (la diagonal mide 4: cumple k = 4 pero no k = 5)
53
54 Vecindarios l3({"X...",
55   ".X..",
56   "..X.",
57   "..."});
58 cout << l3.hizoLinea(1, 1, 4) << " " << l3.hizoLinea(1, 1, 3);
59 // 0 1 (la linea tiene 3 celdas)
```

Nota: hizoLinea prueba solo cuatro direcciones, (0,1), (1,0), (1,1) y (1,-1), y cuenta hacia adelante y hacia atrás desde la celda: las otras cuatro son las mismas líneas recorridas en sentido contrario, así que probarlas duplicaría el trabajo sin encontrar nada nuevo. Por eso el arreglo DXL/DYL es una *media circunferencia* de direcciones y no las 8 de DX8. También conviene notar que vecinos en una esquina devuelve menos celdas que en el interior (3 y 2 en el ejemplo, en vez de 8 y 4): un código que asuma siempre 8 vecinos falla justo en los bordes.

Regla práctica: define dx/dy una sola vez, en sentido horario y empezando hacia arriba, y usa siempre el mismo bucle para recorrer el vecindario, sin escribir un if por dirección. Comprueba los límites antes de acceder (dentro, o un borde de centinelas si el tablero no es toroidal), y aprovecha que con ese orden girar es $(d+1)\%4$, dar la vuelta es $d \wedge 2$ y la dirección opuesta en 8 direcciones es $d \wedge 4$. Antes de codificar, confirma tres cosas del enunciado: cuántas direcciones cuentan (4

u 8), qué pasa en los bordes (pared, toro u otro), y si la diagonal puede cortar esquinas.

49 Fechas y Tiempo

49.1 Años Bisiestos y Días de un Mes

Idea: un año es bisiesto (tiene 366 días, con un 29 de febrero) si es divisible por 4, **excepto** los divisibles por 100, que **sí vuelven** a ser bisiestos si además son divisibles por 400. Esta es la regla del calendario **gregoriano** y explica por qué el año 2000 fue bisiesto pero 1900 y 2100 no lo son. La condición se evalúa en ese orden exacto (400 primero, luego 100, luego 4) porque cada regla **anula** a la anterior: sin esa jerarquía, un año como 1900 (divisible por 4 y por 100 pero no por 400) se clasificaría mal. A partir de esta única función se derivan todas las demás preguntas del calendario: cuántos días tiene un mes (solo febrero cambia), cuántos días tiene un año, y qué número de día del año es una fecha dada (sumando los días de los meses anteriores).

¿Por qué usarlo? Es la pieza base de *Fechas y Tiempo: Día de la Semana y Diferencia entre Fechas y Sumar Días* dependen de saber cuántos días tiene cada mes, y eso depende de si el año es bisiesto. El error más común es usar `anio % 4 == 0` sin las excepciones, que falla exactamente uno de cada 100 años (los “siglos” no múltiplos de 400) — un caso de prueba que aparece con frecuencia justamente porque es la trampa clásica. Frente a llamar a una librería de fechas del lenguaje (que en C++ competitivo casi nunca está disponible o es incómoda de usar), esta regla se calcula en $O(1)$ con aritmética simple, sin tablas ni dependencias externas.

Casos de uso:

- Verificar si un año es bisiesto para saber si febrero tiene 28 o 29 días.
- Calcular cuántos días tiene un año completo (365 o 366), base de cualquier cálculo de diferencia de fechas en años.
- Convertir una fecha (día, mes, año) a su **número de día del año** (del 1 al 365 o 366), el primer paso para restar dos fechas.
- Validar que una fecha dada sea correcta (por ejemplo, rechazar el 30 de febrero o el 31 de abril).
- Generar un calendario o iterar día por día dentro de un mes o un año, sabiendo dónde termina cada uno.

Complejidad: $O(1)$ para `esBisiesto` y `diasDelMes` (tres comparaciones y una consulta a una tabla fija de 12 valores). `díaDelAño` suma los días de hasta 11 meses anteriores, así que es $O(1)$ también (una constante acotada por 12, no crece con la entrada).

```
1 struct CalendarioBasico {
2
3   // READ: true si el año es bisiesto (calendario gregoriano).
4   // El
5   // orden de las condiciones importa: 400 anula a 100, que
6   // anula a 4.
7   static bool esBisiesto(int anio) {
8     if (anio % 400 == 0) return true;
9     if (anio % 100 == 0) return false;
10  }
```



```

8     return anio % 4 == 0;
9     } // O(1)
10
11 // READ: cuantos dias tiene el mes (1..12) en ese anio. Solo
    febrero
12 // (mes 2) depende de si el anio es bisiesto.
13 static int diasDelMes(int mes, int anio) {
14     static const int dias[13] =
        {0,31,28,31,30,31,30,31,31,30,31,30,31};
15     if (mes == 2 && esBisiesto(anio)) return 29;
16     return dias[mes];
17 } // O(1)
18
19 // READ: cuantos dias tiene el anio completo.
20 static int diasDelAnio(int anio) {
21     return esBisiesto(anio) ? 366 : 365;
22 } // O(1)
23
24 // READ: numero de dia del anio (1 = 1 de enero) para la
    fecha dada.
25 static int diaDelAnio(int dia, int mes, int anio) {
26     int total = dia;
27     for (int m = 1; m < mes; m++) total += diasDelMes(m, anio);
28     return total;
29 } // O(1) (a lo sumo
    11 sumas)
30 };
31
32 // -----
33 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
34 // -----
35
36 // 1. VALIDAR UNA FECHA: una fecha (d, m, y) es correcta si
37 // 1 <= m <= 12 y 1 <= d <= diasDelMes(m, y). Util antes de
    procesar
38 // entradas que pueden venir mal formadas.
39
40 // 2. DIA DEL ANIO CON PREFIJOS PRECALCULADOS: si hay que
    consultar
41 // muchas fechas del MISMO anio, conviene precalcular una
    sola vez
42 // los 12 prefijos de dias por mes en vez de sumar en cada
    consulta:
43 //
44 // vector<int> prefijo(13, 0);
45 // for (int m = 1; m <= 12; m++)
46 //     prefijo[m] = prefijo[m - 1] + diasDelMes(m, anio);
47 // diaDelAnio(d, m) = prefijo[m - 1] + d, en O(1) real
    por consulta
48
49 // 3. RANGO DE ANIOS BISIESTOS: contar bisiestos en [1, N] sin
    recorrer
50 // uno por uno es  $N/4 - N/100 + N/400$  (inclusion-exclusion
    sobre los
51 // tres divisores); para [A, B] se resta el conteo de [1, A
    -1] al de
52 // [1, B].
53
54 // 4. CALENDARIO JULIANO (antes de la reforma gregoriana de
    1582): la
55 // regla es simplemente  $\text{anio} \% 4 == 0$ , SIN las excepciones
    de 100 y
56 // 400. Si el enunciado menciona el calendario juliano
    explicitamente,
57 // no apliques esta funcion tal cual.
58
59 // 5. TRAMPA CLASICA: 1900 Y 2000. 1900 no es bisiesto (
    divisible por
60 // 100 pero no por 400) y 2000 si lo es (divisible por 400).
    Un
61 // generador de casos de prueba para este tema casi siempre
    incluye
62 // alguno de estos dos anios exactamente para atrapar la
63 // implementacion ingenua " $\text{anio} \% 4 == 0$ ".
64
65 // 6. TRAMPA: ORDEN DE LAS CONDICIONES. " $\text{anio} \% 4 == 0$  &&  $\text{anio} \% 100 != 0$ 
66 // ||  $\text{anio} \% 400 == 0$ " sin parentesis depende de la
    precedencia de
67 // && sobre ||, y es facil de leer mal. La version con tres
    if
68 // consecutivos (arriba) es mas clara y evita ese riesgo.

```

Ejemplo de funcionamiento:

```

1 cout << CalendarioBasico::esBisiesto(2000) << " " // 1 (
    divisible por 400)
2 << CalendarioBasico::esBisiesto(1900) << " " // 0 (
    divisible por 100, no por 400)
3 << CalendarioBasico::esBisiesto(2024) << " " // 1 (
    divisible por 4, no por 100)
4 << CalendarioBasico::esBisiesto(2023); // 0 (no
    divisible por 4)

```

```

5 // 1 0 1 0
6
7 cout << CalendarioBasico::diasDelMes(2, 2024) << " " // 29
8 << CalendarioBasico::diasDelMes(2, 2023); // 28
9
10 cout << CalendarioBasico::diaDelAnio(29, 2, 2024); // 60
    (31 de enero + 29 de febrero)
11 cout << CalendarioBasico::diaDelAnio(31, 12, 2023); // 365 (
    ultimo dia de un anio no bisiesto)

```

Regla práctica: nunca uses $\text{anio} \% 4 == 0$ solo — siempre las tres condiciones en el orden 400, 100, 4. Cualquier otra pregunta del calendario (días de un mes, días de un año, día del año) se construye llamando a `esBisiesto` una sola vez y derivando el resto con aritmética simple, sin tablas de fechas completas.

49.2 Día de la Semana (Zeller / Sakamoto)

Idea: ambos algoritmos calculan, en $O(1)$ y sin iterar día por día, qué día de la semana cae una fecha del calendario gregoriano. La idea común es tratar enero y febrero como los **meses 13 y 14 del año anterior**: así, el año siempre “cambia” en marzo, y el día 29 de febrero (que solo existe en años bisiestos) queda al **final** del año extendido en vez de interrumpir la aritmética a mitad de año. Con esa convención, cada fórmula combina el día, un término que depende del mes (basado en cuántos días de *desplazamiento* acumula cada mes respecto al anterior), y un término que depende del año (incluyendo la misma corrección de bisiestos vista en *Años Bisiestos y Días de un Mes*, expresada como $\text{año}/4 - \text{año}/100 + \text{año}/400$), todo reducido módulo 7. La diferencia entre los dos algoritmos es solo de **convención**: Zeller nació pensando en el calendario juliano y su resultado nativo numera desde el sábado; Sakamoto es una reescritura posterior, pensada para el gregoriano, cuyo resultado nativo numera desde el domingo — la tabla de desplazamientos por mes de Sakamoto es, matemáticamente, otra forma de la misma corrección.

¿Por qué usarlo? Sin esta fórmula, la única forma de saber el día de la semana de una fecha lejana es **contar días uno por uno** desde una fecha de referencia conocida, un recorrido de $O(\text{días})$ que es inviable si la fecha está a miles de años de distancia. Zeller y Sakamoto resuelven exactamente el mismo problema en $O(1)$; en la práctica, Sakamoto se prefiere porque su tabla ya está en base domingo=0, que es la convención más común en los enunciados de competencia, y evita el paso extra de reindexar el resultado de Zeller. Frente a *Diferencia entre Fechas y Sumar Días* (que sí necesita convertir fechas a un número de días acumulado para restarlas), esta subsección resuelve una pregunta más puntual — *qué día de la semana* — sin necesitar ese número de días en absoluto.

Casos de uso:

- Determinar el día de la semana de cualquier fecha gregoriana, pasada o futura, sin recorrer el calendario.
- Contar cuántas veces cae un día de la semana específico (por ejemplo, cuántos viernes 13 hay) en un rango de fechas.

- Verificar la consistencia de un calendario dado como entrada (por ejemplo, que el 1 de enero de un año caiga en el día correcto).
- Como paso auxiliar de *Diferencia entre Fechas y Sumar Días*, para etiquetar cada fecha resultante con su día de la semana.
- Resolver acertijos de calendario del tipo “¿qué día de la semana fue/será tal fecha?” con una sola fórmula cerrada.

Complejidad: $O(1)$ para ambos algoritmos: cada uno hace un número fijo de operaciones aritméticas (multiplicaciones, divisiones enteras y un módulo), sin ningún bucle ni dependencia del tamaño de la fecha (funciona igual de rápido para el año 100 que para el año 1,000,000).

```

1 struct DiaDeLaSemana {
2
3     // READ: algoritmo de Zeller. Retorna 0=sábado, 1=domingo, 2=
4     // ..., 6=viernes (la convencion NATIVA de Zeller, poco
5     // intuitiva).
6     static int zeller(int dia, int mes, int anio) {
7         if (mes < 3) { mes += 12; anio -= 1; } // enero/febrero =
8         // meses 13/14 del anio anterior
9         int K = anio % 100; // anio dentro
10        // del siglo
11        int J = anio / 100; // siglo
12        int h = (dia + (13 * (mes + 1)) / 5 + K + K / 4 + J / 4 + 5
13        // * J) % 7;
14        return h;
15    } // O(1)
16
17    // READ: reindexa el resultado de zeller() a la convencion
18    // 0=domingo, 1=lunes, ..., 6=sábado (la mas comun en
19    // competencia).
20    static int zellerADiaSemana(int h) {
21        static const int desplazamiento[] = {6, 0, 1, 2, 3, 4, 5};
22        return tabla[h];
23    } // O(1)
24
25    // READ: algoritmo de Sakamoto. Retorna DIRECTAMENTE 0=
26    // domingo,
27    // 1=lunes, ..., 6=sábado; no necesita reindexar.
28    static int sakamoto(int anio, int mes, int dia) {
29        static const int desplazamiento[] =
30        {0,3,2,5,0,3,5,1,4,6,2,4};
31        if (mes < 3) anio -= 1; // enero/febrero
32        // = meses del anio anterior
33        int h = (anio + anio / 4 - anio / 100 + anio / 400
34        // + desplazamiento[mes - 1] + dia) % 7;
35        return h;
36    } // O(1)
37 };
38
39 // -----
40 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
41 // -----
42
43 // 1. NOMBRES DE LOS DIAS: con la convencion 0=domingo (la de
44 // Sakamoto o
45 // la de zellerADiaSemana), basta indexar un arreglo:
46 //
47 // const string NOMBRES[7] = {"domingo", "lunes", "martes", "
48 // miercoles",
49 // "jueves", "viernes", "sabado
50 // "};
51 // cout << NOMBRES[DiaDeLaSemana::sakamoto(2024, 9, 29)];
52
53 // 2. ANIOS NEGATIVOS Y EL OPERADOR %: en C++, el resultado de
54 // % puede
55 // ser negativo si el operando izquierdo lo es (por ejemplo,
56 // si
57 // "anio - 1" o alguna resta intermedia da un numero
58 // negativo en un
59 // caso limite). Para blindar el resultado, se normaliza
60 // sumando 7
61 // antes del modulo final:
62 //
63 // int h = ((.../* expresion original */...) % 7 + 7) % 7;
64 //
65 // Con los rangos tipicos de un problema de competencia (
66 // anios
67 // positivos razonables) esto casi nunca hace falta, pero es
68 // una

```

```

52 // salvaguarda barata si el enunciado permite anios muy
53 // pequenos o
54 // negativos.
55
56 // 3. CONTAR UN DIA DE LA SEMANA EN UN RANGO: para contar, por
57 // ejemplo,
58 // cuantos domingos hay entre dos fechas, NO conviene
59 // recorrer dia por
60 // dia; se combina con Diferencia entre Fechas y Sumar Dias:
61 // se
62 // convierten ambas fechas a un numero de dia consecutivo (
63 // dia
64 // juliano), se calcula el dia de la semana de una de ellas
65 // con
66 // Sakamoto, y el conteo se resuelve con la misma aritmetica
67 // de
68 // "conteo de multiples en un rango" que se usa para contar
69 // multiples
70 // de k en [L, R].
71
72 // 4. VERIFICACION CRUZADA: si tienes dudas sobre la
73 // implementacion,
74 // Zeller y Sakamoto deben dar SIEMPRE el mismo dia de la
75 // semana para
76 // cualquier fecha valida (solo cambia la convencion de
77 // numeracion,
78 // que se corrige con zellerADiaSemana). Compararlos entre
79 // si en los
80 // casos de prueba propios es una buena forma de detectar un
81 // error de
82 // signo o de indice antes de enviar la solucion.
83
84 // 5. TRAMPA: LA CONVENCION DE ZELLER. El resultado crudo de
85 // zeller()
86 // empieza en SABADO = 0, no en domingo. Usarlo directamente
87 // como
88 // indice de un arreglo "domingo, lunes, ..." da el dia
89 // equivocado; hay
90 // que pasar por zellerADiaSemana() o restar y ajustar
91 // manualmente.
92
93 // 6. TRAMPA: ENERO Y FEBRERO. Si se olvida el ajuste "mes < 3:
94 // mes += 12,
95 // anio -= 1", el termino de correccion de bisiestos (anio/4
96 // - anio/100
97 // + anio/400) queda calculado con el anio equivocado para
98 // las fechas
99 // de enero y febrero, y el resultado falla exactamente en
100 // esos dos
101 // meses - un patron de error facil de identificar si
102 // aparece.

```

Ejemplo de funcionamiento:

```

1 const string NOMBRES[7] = {"domingo", "lunes", "martes", "
2 // miercoles",
3 // "jueves", "viernes", "sabado"};
4
5 // 1 de enero de 2000 fue sabado.
6 int hz = DiaDeLaSemana::zeller(1, 1, 2000);
7 cout << NOMBRES[DiaDeLaSemana::zellerADiaSemana(hz)]; //
8 // sabado
9
10 // 29 de febrero de 2000 (bisiesto) fue martes.
11 cout << NOMBRES[DiaDeLaSemana::sakamoto(2000, 2, 29)]; //
12 // martes
13
14 // 14 de julio de 1994 fue jueves.
15 cout << NOMBRES[DiaDeLaSemana::sakamoto(1994, 7, 14)]; //
16 // jueves

```

Regla práctica: para el día de la semana de una fecha gregoriana, usa Sakamoto directamente — ya numera desde domingo y evita el paso de reindexar que exige Zeller. Antes de codificar cualquiera de los dos, confirma que el ajuste “enero y febrero son los meses 13 y 14 del año anterior” está aplicado **antes** de calcular cualquier otro término de la fórmula, porque es el error más común y el más difícil de detectar sin un caso de prueba en esos dos meses.

49.3 Diferencia entre Fechas y Sumar Días

Idea: Trabajar directamente con la terna (año, mes, día) es incómodo porque los meses tienen longitudes distintas

y los años bisiestos rompen la regularidad. La solución es convertir cada fecha a un único entero: el número de días transcurridos desde una época fija (aquí, 1970-01-01 es el día 0). En esa representación el calendario desaparece: la diferencia entre dos fechas es una resta y sumar k días es una suma. Después se convierte el entero de vuelta a (año, mes, día).

La conversión funciona porque se *desplaza el año para que empiece en marzo*. Así el día extra de los bisiestos (29 de febrero) cae al final del año desplazado y no afecta el conteo de los meses anteriores. Además, los meses desde marzo siguen el patrón 31, 30, 31, 30, 31, 31, 30, 31, 30, 31, 31, (28/29), y ese patrón se captura con la fórmula $\lfloor (153 \cdot mp + 2) / 5 \rfloor$, donde mp es el mes contado desde marzo. Los ciclos gregorianos de 400 años tienen exactamente 146097 días, por lo que basta dividir en «eras» de 400 años y resolver dentro de cada era con aritmética entera.

¿Por qué usarlo? La alternativa ingenua es simular día a día (o mes a mes) hasta llegar a la fecha destino. Eso cuesta $O(\text{días})$ o $O(\text{años})$ y es inviable cuando las fechas están separadas por siglos o hay muchas consultas. Otra alternativa es una tabla de días acumulados por mes, que sirve dentro de un año pero obliga a manejar aparte el cruce de años y los bisiestos. La conversión a número de día hace todo en $O(1)$ con una sola fórmula, es correcta para cualquier año (incluso negativos) y compone bien: una vez que tienes el entero, la diferencia, la suma, la comparación y el día de la semana son operaciones triviales. Se prefiere sobre el simulado siempre que las fechas no sean del mismo mes.

Casos de uso:

- Calcular cuántos días hay entre dos fechas (edad en días, plazos, días de retraso).
- Calcular la fecha que resulta de sumar o restar k días a una fecha dada.
- Determinar el día de la semana de cualquier fecha.
- Ordenar o comparar fechas usando el entero como clave (por ejemplo, en un `sort` o un `map`).
- Problemas de calendarios: contar cuántos viernes 13 hay en un rango, o el siguiente día que cumple una propiedad.

Complejidad: Cada conversión (fecha $\rightarrow$ entero y entero $\rightarrow$ fecha) ejecuta un número fijo de sumas, multiplicaciones y divisiones enteras, sin ciclos ni recursión, por lo que cuesta $O(1)$. La diferencia entre dos fechas son dos conversiones más una resta: $O(1)$. Sumar k días es una conversión, una suma y la conversión inversa: $O(1)$, independiente de la magnitud de k . Memoria: $O(1)$. Para q consultas, el total es $O(q)$.

```
1 struct Fecha {
2     int anio, mes, dia;
3
4     // CREATE: guarda anio, mes y dia (se asume una fecha valida).
5     Fecha(int a = 1970, int m = 1, int d = 1) : anio(a), mes(m),
        dia(d) {} // O(1)
6
7     // READ: indica si un anio es bisiesto (regla gregoriana).
8     static bool es_bisiesto(int a) {
9         return (a % 4 == 0 && a % 100 != 0) || a % 400 == 0;
10    } // O(1)
```

```
11 // READ: devuelve cuantos dias tiene el mes m del anio a.
12 static int dias_del_mes(int a, int m) {
13     static const int t[12] =
14         {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
15     return (m == 2 && es_bisiesto(a)) ? 29 : t[m - 1];
16 } // O(1)
17
18 // READ: convierte la fecha a dias transcurridos desde
19 // 1970-01-01 (dia 0).
20 // Se desplaza el anio para que empiece en marzo (febrero
21 // queda al final).
22 long long a_dias() const {
23     long long y = anio;
24     y -= (mes <= 2);
25     long long era = (y >= 0 ? y : y - 399) / 400;
26     long long yoe = y - era * 400;
27     // [0, 399]
28     long long mp = (mes > 2) ? mes - 3 : mes + 9;
29     // marzo = 0
30     long long doy = (153 * mp + 2) / 5 + dia - 1;
31     // dia del anio desplazado
32     long long doe = yoe * 365 + yoe / 4 - yoe / 100 + doy;
33     // dia dentro de la era
34     return era * 146097 + doe - 719468;
35 } // O(1)
36
37 // CREATE: construye la fecha correspondiente a z dias desde
38 // 1970-01-01.
39 static Fecha desde_dias(long long z) {
40     z += 719468;
41     long long era = (z >= 0 ? z : z - 146096) / 146097;
42     long long doe = z - era * 146097;
43     // [0, 146096]
44     long long yoe = (doe - doe / 1460 + doe / 36524 - doe /
45         146096) / 365;
46     long long y = yoe + era * 400;
47     long long doy = doe - (365 * yoe + yoe / 4 - yoe / 100);
48     long long mp = (5 * doy + 2) / 153;
49     int d = (int)(doy - (153 * mp + 2) / 5 + 1);
50     int m = (int)(mp < 10 ? mp + 3 : mp - 9);
51     return Fecha((int)(y + (m <= 2)), m, d);
52 } // O(1)
53
54 // READ: dias que hay de "otra" a esta fecha (positivo si
55 // esta es posterior).
56 long long diferencia(const Fecha& otra) const {
57     return a_dias() - otra.a_dias();
58 } // O(1)
59
60 // READ: dia de la semana (0 = domingo, 1 = lunes, ..., 6 =
61 // sabado).
62 // 1970-01-01 fue jueves (4); el doble modulo cubre dias
63 // negativos.
64 int dia_semana() const {
65     return (int)((a_dias() + 4) % 7 + 7) % 7;
66 } // O(1)
67
68 // WRITE: avanza (o retrocede si k < 0) la fecha k dias.
69 void sumar_dias(long long k) {
70     *this = desde_dias(a_dias() + k);
71 } // O(1)
72
73 };
74
75 // -----
76 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
77 // -----
78 // 1. Restar dias: llamar sumar_dias con k negativo. No
79 // requiere codigo extra
80 // porque a_dias() y desde_dias() funcionan para z negativo.
81 //
82 // 2. Dia del anio (1..366): sirve para validar o para fechas
83 // ordinales.
84 int dia_del_anio() const {
85     return (int)(a_dias() - Fecha(anio, 1, 1).a_dias())
86         + 1;
87 } // O(1)
88 //
89 // 3. Sumar meses (con recorte al ultimo dia valido del mes
90 // destino):
91 void sumar_meses(int k) {
92     long long total = (long long)anio * 12 + (mes - 1)
93         + k;
94     anio = (int)(total / 12); mes = (int)(total % 12) +
95         1; // si total < 0, ajustar
96     dia = min(dia, dias_del_mes(anio, mes));
97 } // O(1)
98 // Ojo: 31 de enero + 1 mes da 28/29 de febrero, no 3 de
99 // marzo.
100 //
101 // 4. Siguiendo dia de la semana d (0..6) estrictamente
102 // posterior:
103 long long k = ((d - dia_semana() + 6) % 7) + 1;
```

```

    sumar_dias(k); // 0(1)
83 //
84 // 5. Comparar fechas: comparar a_dias() (o la terna en orden
    anio, mes, dia).
85 // Como clave de map/sort: usar a_dias() como llave entera.

```

Ejemplo de funcionamiento:

```

1 int main() {
2     Fecha a(2000, 2, 28), b(2000, 3, 1);
3     cout << b.diferencia(a) << "\n"; // 2 (28-feb ->
        29-feb -> 1-mar, 2000 es bisiesto)
4
5     Fecha c(2024, 1, 1), d(2024, 12, 31);
6     cout << d.diferencia(c) << "\n"; // 365 (2024
        tiene 366 dias; del 1-ene al 31-dic hay 365 saltos)
7
8     Fecha e(2024, 1, 1);
9     e.sumar_dias(100);
10    cout << e.anio << "-" << e.mes << "-" << e.dia << "\n"; //
        2024-4-10 (31 + 29 + 31 = 91 dias hasta el 1-abr; +9)
11
12    cout << Fecha(2000, 1, 1).a_dias() << "\n"; // 10957 (30
        años * 365 + 7 bisiestos)
13    cout << Fecha(2000, 1, 1).dia_semana() << "\n"; // 6 (sabado)
14    cout << Fecha(2024, 1, 1).dia_semana() << "\n"; // 1 (lunes)
15    return 0;
16 }

```

Nota: `a_dias()` devuelve `long long` porque con años grandes (del orden de 10^9) el conteo de días excede los 32 bits. Si el problema garantiza años razonables (por ejemplo entre 1900 y 2100), `int` alcanza, pero usar `long long` no cuesta nada.

Regla práctica: Si necesitas relacionar dos fechas que pueden estar en meses o años distintos, convierte ambas a número de día y opera con enteros. Simula día a día solo cuando el rango sea diminuto (unos pocos días) y no valga la pena escribir la conversión. Usa la tabla de días acumulados por mes solo cuando todas las fechas estén garantizadas dentro de un mismo año no bisiesto.

49.4 Formatos de Hora (12h/24h, Segundos a HH:MM:SS)

Idea: Igual que con las fechas, la forma más limpia de manejar horas es guardarlas como un solo entero: los segundos transcurridos desde las 00:00:00. Como un día tiene $24 \cdot 3600 = 86400$ segundos, cualquier hora del reloj corresponde a un valor en $[0, 86399]$. El formato HH:MM:SS es simplemente una representación en base mixta de ese entero: las horas son el cociente de dividir entre 3600, los minutos son el cociente del resto entre 60, y los segundos son lo que sobra. La conversión inversa es la suma $3600h + 60m + s$.

Sumar tiempo es aritmética modular: el reloj es cíclico, así que después de sumar k segundos se toma el resultado módulo 86400. Con k negativo hay que normalizar con un segundo módulo, porque en C++ el operador `%` puede devolver un valor negativo. El formato de 12 horas es solo una presentación sobre las mismas horas: $h \bmod 12$ da la hora en el reloj (con el caso especial de que 0 se muestra como 12), y $h < 12$ decide entre AM y PM.

¿Por qué usarlo? Manipular h , m y s por separado obliga a propagar acarreo a mano (segundos $\rightarrow$ minutos $\rightarrow$ horas $\rightarrow$ día) y es la fuente típica de errores por uno. Con el entero, la suma, la resta y la comparación de horas son operaciones directas, y el formateo se hace solo al leer o imprimir. Esto es la misma idea que la conversión de fechas a número de día, aplicada a un dominio cíclico y de tamaño fijo. Una diferencia con las fechas es que aquí

hay que decidir explícitamente qué pasa al desbordar: en una *hora del reloj* se envuelve módulo 86400, mientras que en una *duración* no se envuelve y las horas pueden superar 23.

Casos de uso:

- Imprimir un contador de segundos como HH:MM:SS (cronómetros, tiempos de carrera, logs).
- Convertir entre formato de 24 horas y 12 horas con AM/PM (entradas y salidas de problemas de formato).
- Sumar o restar segundos, minutos u horas a una hora del reloj, con vuelta al día siguiente o anterior.
- Calcular cuánto tiempo transcurre entre dos horas del reloj, incluso cruzando la medianoche.
- Aplicar desplazamientos de zona horaria a una hora dada, o formatear duraciones que superan las 24 horas.

Complejidad: Convertir segundos a HH:MM:SS son dos divisiones y dos restos enteros: $O(1)$. La conversión inversa son dos multiplicaciones y dos sumas: $O(1)$. Sumar k segundos es una suma y un módulo: $O(1)$, independiente de k . Formatear a texto es $O(1)$ porque la salida tiene longitud fija (8 caracteres, más el sufijo AM/PM). Parsear una cadena es $O(L)$ con L el largo del texto, que en la práctica es constante (una hora tiene a lo sumo unos 11 caracteres). Memoria: $O(1)$.

```

1 struct Hora {
2     int h, m, s; // siempre en formato de 24
        horas
3     static constexpr int DIA = 86400; // segundos en un dia
4
5     // CREATE: construye la hora a partir de una cantidad de
        segundos.
6     // Normaliza modulo 86400 (funciona con valores negativos o
        mayores a un dia).
7     Hora(long long seg = 0) {
8         seg = ((seg % DIA) + DIA) % DIA;
9         h = (int)(seg / 3600);
10        m = (int)((seg % 3600) / 60);
11        s = (int)(seg % 60);
12    } // O(1)
13
14    // CREATE: construye la hora a partir de horas, minutos y
        segundos.
15    static Hora desde_hms(int hh, int mm, int ss) {
16        return Hora(hh * 3600LL + mm * 60LL + ss);
17    } // O(1)
18
19    // CREATE: lee texto en formato 24h, ej. "07:45:10" o "7:5:3".
20
21    static Hora desde_texto24(const string& t) {
22        int hh = 0, mm = 0, ss = 0;
23        sscanf(t.c_str(), "%d:%d:%d", &hh, &mm, &ss);
24        return desde_hms(hh, mm, ss);
25    } // O(L), L = largo del texto (constante en la practica)
26
27    // CREATE: lee texto en formato 12h, ej. "07:45:10 PM".
28    // 12 AM -> 0 horas, 12 PM -> 12 horas, 1..11 PM -> 13..23.
29    static Hora desde_texto12(const string& t) {
30        int hh = 0, mm = 0, ss = 0;
31        char suf[3] = {0};
32        sscanf(t.c_str(), "%d:%d:%d %2s", &hh, &mm, &ss, suf);
33        hh %= 12; // 12 -> 0
34        if (suf[0] == 'P' || suf[0] == 'p') hh += 12;
35        return desde_hms(hh, mm, ss);
36    } // O(L)
37
38    // READ: devuelve los segundos transcurridos desde las
        00:00:00.
39    int a_segundos() const {
40        return h * 3600 + m * 60 + s;
41    } // O(1)
42
43    // READ: devuelve la hora como texto "HH:MM:SS" (24 horas).

```



```

43 string formato24() const {
44     char buf[16];
45     sprintf(buf, sizeof buf, "%02d:%02d:%02d", h, m, s);
46     return string(buf);
47 } // O(1)
48
49 // READ: devuelve la hora como texto "HH:MM:SS AM/PM" (12
50 // horas).
51 string formato12() const {
52     int h12 = h % 12;
53     if (h12 == 0) h12 = 12; // 0 y 12 se
54     // muestran como 12
55     const char* suf = (h < 12) ? "AM" : "PM";
56     char buf[24];
57     sprintf(buf, sizeof buf, "%02d:%02d:%02d %s", h12, m, s,
58     suf);
59     return string(buf);
60 } // O(1)
61
62 // READ: segundos que hay que avanzar desde esta hora hasta "
63 // otra",
64 // cruzando la medianoche si hace falta. Resultado en [0,
65 // 86399].
66 int segundos_hasta(const Hora& otra) const {
67     return ((otra.a_segundos() - a_segundos()) % DIA + DIA) %
68     DIA;
69 } // O(1)
70
71 // WRITE: avanza (o retrocede si k < 0) la hora k segundos,
72 // con vuelta al día.
73 void sumar_segundos(long long k) {
74     *this = Hora(a_segundos() + k);
75 } // O(1)
76
77 };
78
79 // -----
80 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
81 // -----
82 // 1. Duraciones (sin envolver a 24h): NO usar el constructor
83 // de Hora, porque
84 // aplica modulo 86400. Formatear directamente el entero:
85 // string formato_duracion(long long seg) {
86 //     char buf[32];
87 //     sprintf(buf, sizeof buf, "%lld:%02lld:%02lld", seg
88 // / 3600, (seg % 3600) / 60, seg % 60);
89 //     return string(buf);
90 // } // O(1)
91 // Las horas pueden pasar de 23: 100000 s -> "27:46:40";
92 // 359999 s -> "99:59:59".
93
94 // 2. Milisegundos (HH:MM:SS.mmm): guardar el tiempo en
95 // milisegundos, con
96 // DIA = 86400000, y extraer ms = t % 1000, luego t /= 1000
97 // y seguir igual.
98
99 // 3. Zona horaria: aplicar el desplazamiento en segundos con
100 // sumar_segundos.
101 // Ej. de UTC a UTC-5: hora.sumar_segundos(-5 * 3600); //
102 // O(1)
103
104 // 4. Lectura con cin en lugar de scanf: leer "int h; char c;
105 // int m, s;"
106 // con cin >> h >> c >> m >> c >> s; (el char absorbe los
107 // '.').
108
109 // 5. Validar entrada: antes de construir, comprobar 0 <= h <
110 // 24, 0 <= m < 60,
111 // 0 <= s < 60 si el enunciado no garantiza formato correcto.

```

Ejemplo de funcionamiento:

```

1 int main() {
2     Hora a(3661);
3     cout << a.formato24() << "\n"; // 01:01:01
4     // (3600 + 60 + 1)
5
6     Hora b(86399);
7     cout << b.formato24() << "\n"; // 23:59:59
8
9     Hora c(45296);
10    cout << c.formato24() << "\n"; // 12:34:56
11    // (12*3600 = 43200; 2096 = 34*60 + 56)
12    cout << c.formato12() << "\n"; // 12:34:56
13    // PM
14
15    cout << Hora::desde_hms(0, 15, 0).formato12() << "\n"; //
16    // 12:15:00 AM
17    cout << Hora::desde_hms(15, 30, 0).formato12() << "\n"; //
18    // 03:30:00 PM
19
20    Hora d = Hora::desde_texto12("07:45:10 PM");
21    cout << d.formato24() << "\n"; // 19:45:10

```

```

17 Hora e = Hora::desde_hms(23, 59, 30);
18 e.sumar_segundos(45);
19 cout << e.formato24() << "\n"; // 00:00:15
20 // (86370 + 45 = 86415 = 86400 + 15)
21
22 Hora f = Hora::desde_hms(22, 0, 0), g = Hora::desde_hms(6, 0,
23 0);
24 cout << f.segundos_hasta(g) << "\n"; // 28800 (8
25 // horas, cruzando la medianoche)
26 return 0;
27 }

```

Nota: Los casos límite del formato de 12 horas son la fuente de errores más común. La medianoche (00:xx:xx) es 12:xx:xx AM y el mediodía (12:xx:xx) es 12:xx:xx PM; no existe la hora 00 en el reloj de 12 horas. Por eso la conversión usa $h \bmod 12$ y sustituye 0 por 12, en lugar de restar 12 a las horas mayores que 12.

Regla práctica: Si el problema mezcla horas con sumas, restas o comparaciones, guarda todo como segundos (o milisegundos) y convierte solo al leer o imprimir. Usa el constructor con módulo cuando modes una hora del reloj (que se envuelve en 24h) y el formateo directo de la sección de modificaciones cuando modes una duración (que puede superar las 24 horas). Si el problema solo pide un cambio de formato sin aritmética, basta con las reglas de `formato12()` y `desde_texto12()`, sin necesidad de más estructura.

50 Aritmética Ad-hoc

50.1 Números Grandes (BigInt)

Idea: Los tipos nativos guardan un número en un ancho fijo (`long long` llega a $\approx 9.2 \cdot 10^{18}$), pero muchos problemas piden resultados con cientos o miles de dígitos (factoriales, potencias, Fibonacci grandes). La solución es representar el número como un vector de «bloques» (*limbs*) en una base grande, igual que la suma y la multiplicación a mano que se aprende en la escuela, pero tratando cada bloque como un dígito gigante.

Se elige la base $B = 10^9$ por dos razones. Primero, $B^2 \approx 10^{18}$ cabe en un `long long`, así que el producto de dos bloques más el acarreo nunca desborda. Segundo, como B es potencia de 10, cada bloque corresponde exactamente a 9 dígitos decimales y imprimir es trivial (basta rellenar con ceros a 9 dígitos). El número se guarda en orden little-endian (d_0 es el bloque menos significativo) para que los acarreos avancen hacia el final del vector. El valor representado es $\sum_i d_i \cdot B^i$, y se mantiene el invariante de que no hay bloques cero al final, lo cual hace que comparar por tamaño sea correcto.

¿Por qué usarlo? Si el resultado cabe en 18 dígitos, `long long` es más simple y mucho más rápido. Si cabe en unos 38 dígitos, `__int128` evita escribir estructura propia. Si el enunciado pide el resultado módulo un primo p , no se necesita `BigInt`: la aritmética modular mantiene todo acotado. `BigInt` es la elección correcta cuando se pide el valor *exacto* y completo (o cuando se necesita comparar valores enormes sin perder precisión), y en lenguajes como C++ no hay tipo nativo para ello (Java y Python sí lo tienen). Frente a `long double`, que aproxima con unos 18 dígitos significativos, `BigInt` no pierde ningún dígito.

Casos de uso:

- Imprimir factoriales, potencias o coeficientes binomiales exactos (por ejemplo $100!$ o 2^{1000}).
- Calcular términos grandes de Fibonacci, Catalan u otras recurrencias sin módulo.
- Obtener la suma de dígitos o el número de dígitos de un resultado enorme.
- Comparar o sumar números dados como cadenas de miles de dígitos.
- Ser el numerador y denominador de una fracción exacta cuando `long long` desborda, o un paso previo a criptografía básica (RSA de juguete).

Complejidad: Sea D el número de dígitos decimales y $n = \lceil D/9 \rceil$ el número de bloques. Sumar y restar recorren los bloques una sola vez propagando el acarreo, de modo que cuestan $O(n)$. Comparar es $O(n)$ en el peor caso (recorre desde el bloque más significativo hasta encontrar una diferencia). Dividir entre un entero pequeño hace una operación por bloque, con un resto que se arrastra: $O(n)$. Multiplicar dos números de n y m bloques con el método escolar ejecuta dos ciclos anidados, uno por cada par de bloques, y por eso cuesta $O(n \cdot m)$. Convertir desde y hacia texto es $O(D)$. Memoria: $O(n)$. Por ejemplo, un número de 10^4 dígitos tiene unos 1112 bloques, así que multiplicarlo por otro igual hace $\approx 1.2 \cdot 10^6$ operaciones.

```

1 struct BigInt {
2     static constexpr int BASE = 1000000000; // 10^9
3     static constexpr int DIG = 9;           // dígitos decimales
4     // por bloque
5     vector<int> d;                          // bloques, el menos
6     // significativo primero
7     bool neg;                               // signo (el cero
8     // nunca es negativo)
9
10    // CREATE: construye desde un entero de 64 bits.
11    BigInt(long long v = 0) {
12        neg = (v < 0);
13        unsigned long long u = neg ? 0ULL - (unsigned long long)v :
14        (unsigned long long)v;
15        while (u > 0) { d.push_back((int)(u % BASE)); u /= BASE; }
16    } // O(1)
17
18    // CREATE: construye desde una cadena decimal, ej.
19    // "-123456789012345678901".
20    static BigInt desde_string(const string& t) {
21        BigInt r;
22        int ini = 0;
23        bool ng = false;
24        if (t[0] == '-') { ng = true; ini = 1; }
25        else if (t[0] == '+') ini = 1;
26        for (int fin = (int)t.size(); fin > ini; fin -= DIG) {
27            int lo = max(ini, fin - DIG);
28            r.d.push_back(stoi(t.substr(lo, fin - lo)));
29        }
30        r.neg = ng;
31        r.recortar();
32        return r;
33    } // O(D), D = cantidad de dígitos
34
35    // WRITE: elimina bloques cero al final y corrige el signo
36    // del cero.
37    void recortar() {
38        while (!d.empty() && d.back() == 0) d.pop_back();
39        if (d.empty()) neg = false;
40    } // O(n)
41
42    // READ: compara valores absolutos. Devuelve -1, 0 o 1.
43    static int cmp_abs(const BigInt& a, const BigInt& b) {
44        if (a.d.size() != b.d.size()) return a.d.size() < b.d.size()
45        ? -1 : 1;
46        for (int i = (int)a.d.size() - 1; i >= 0; i--)
47            if (a.d[i] != b.d[i]) return a.d[i] < b.d[i] ? -1 : 1;
48        return 0;
49    } // O(n)
50
51    // READ: suma de valores absolutos (ignora signos).

```

```

45 static BigInt sumar_abs(const BigInt& a, const BigInt& b) {
46     BigInt r;
47     long long carry = 0;
48     for (size_t i = 0; i < max(a.d.size(), b.d.size()) || carry; i++) {
49         long long cur = carry;
50         if (i < a.d.size()) cur += a.d[i];
51         if (i < b.d.size()) cur += b.d[i];
52         r.d.push_back((int)(cur % BASE));
53         carry = cur / BASE;
54     }
55     return r;
56 } // O(max(n, m))
57
58 // READ: resta de valores absolutos, requiere |a| >= |b|.
59 static BigInt restar_abs(const BigInt& a, const BigInt& b) {
60     BigInt r;
61     long long prestamo = 0;
62     for (size_t i = 0; i < a.d.size(); i++) {
63         long long cur = (long long)a.d[i] - prestamo - (i < b.d.size() ? b.d[i] : 0);
64         if (cur < 0) { cur += BASE; prestamo = 1; } else prestamo = 0;
65         r.d.push_back((int)cur);
66     }
67     r.recortar();
68     return r;
69 } // O(n)
70
71 // READ: devuelve el opuesto (-x).
72 BigInt operator-() const {
73     BigInt r = *this;
74     if (!r.d.empty()) r.neg = !r.neg;
75     return r;
76 } // O(n)
77
78 // READ: suma con signo.
79 BigInt operator+(const BigInt& o) const {
80     BigInt r;
81     if (neg == o.neg) { r = sumar_abs(*this, o); r.neg = neg; }
82     else if (cmp_abs(*this, o) >= 0) { r = restar_abs(*this, o); r.neg = neg; }
83     else { r = restar_abs(o, *this); r.neg = o.neg; }
84     r.recortar();
85     return r;
86 } // O(max(n, m))
87
88 // READ: resta con signo.
89 BigInt operator-(const BigInt& o) const {
90     return *this + (-o);
91 } // O(max(n, m))
92
93 // READ: multiplicacion escolar.
94 BigInt operator*(const BigInt& o) const {
95     BigInt r;
96     if (d.empty() || o.d.empty()) return r;
97     r.d.assign(d.size() + o.d.size(), 0);
98     for (size_t i = 0; i < d.size(); i++) {
99         long long carry = 0;
100        for (size_t j = 0; j < o.d.size(); j++) {
101            long long cur = r.d[i + j] + (long long)d[i] * o.d[j] + carry;
102            r.d[i + j] = (int)(cur % BASE);
103            carry = cur / BASE;
104        }
105        r.d[i + o.d.size()] += (int)carry;
106    }
107    r.neg = (neg != o.neg);
108    r.recortar();
109    return r;
110 } // O(n * m)
111
112 // WRITE: divide este numero entre un entero k > 0 (trunca) y
113 // devuelve el resto
114 // del valor absoluto.
115 int dividir_small(int k) {
116     long long rem = 0;
117     for (int i = (int)d.size() - 1; i >= 0; i--) {
118         long long cur = d[i] + rem * BASE;
119         d[i] = (int)(cur / k);
120         rem = cur % k;
121     }
122     recortar();
123     return (int)rem;
124 } // O(n)
125
126 // READ: comparaciones.
127 bool operator<(const BigInt& o) const {
128     if (neg != o.neg) return neg;
129     int c = cmp_abs(*this, o);
130     return neg ? c > 0 : c < 0;
131 } // O(n)
132 bool operator==(const BigInt& o) const {

```

```

132     return neg == o.neg && d == o.d;
133 } // O(n)
134
135 // READ: convierte a cadena decimal.
136 string str() const {
137     if (d.empty()) return "0";
138     string s = neg ? "-" : "";
139     s += to_string(d.back());
140     char buf[16];
141     for (int i = (int)d.size() - 2; i >= 0; i--) {
142         snprintf(buf, sizeof buf, "%09d", d[i]);
143         s += buf;
144     }
145     return s;
146 } // O(D)
147 };
148
149 // -----
150 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
151 // -----
152 // 1. Potencia rapida (exponenciacion binaria):
153 //   BigInt potencia(BigInt b, long long e) {
154 //       BigInt r(1);
155 //       while (e > 0) { if (e & 1) r = r * b; b = b * b; e
156 //           >>= 1; }
157 //       return r;
158 //   } // O(M(n) * log e), dominado por el ultimo cuadrado;
159 //       M(n) = costo de multiplicar
160 //
161 // 2. Resto modulo un entero pequeno sin modificar el numero (
162 //   util para
163 //   divisibilidad, digitos de control o para pasar a
164 //   aritmetica modular):
165 //   int mod_small(int k) const {
166 //       long long rem = 0;
167 //       for (int i = (int)d.size() - 1; i >= 0; i--) rem =
168 //           (rem * BASE + d[i]) % k;
169 //       return (int)rem;
170 //   } // O(n)
171 //
172 // 3. Salida con cout: sobrecargar el operador.
173 //   ostream& operator<<(ostream& os, const BigInt& x) {
174 //       return os << x.str(); }
175 //
176 // 4. Multiplicacion rapida: para n mayor a unos miles de
177 //   bloques, el metodo escolar
178 //   es lento. Karatsuba baja el costo a O(n^1.585), y FFT/NTT
179 //   a O(n log n). Con FFT
180 //   en doubles se usa una base menor (10^3 o 10^4) para no
181 //   perder precision.
182 //
183 // 5. Division BigInt / BigInt: no esta incluida (es la
184 //   operacion mas delicada).
185 //   Opciones: division larga de Knuth (algoritmo D) en O(n *
186 //   m), o buscar cada
187 //   bloque del cociente por busqueda binaria usando la
188 //   multiplicacion. Con ella se
189 //   obtienen tambien el MCD (Euclides), el modulo y la raiz
190 //   cuadrada (Newton).
191 //
192 // 6. Representacion en base 10 (un digito por posicion): es 9
193 //   veces mas lenta y
194 //   usa mas memoria, pero simplifica problemas que operan
195 //   digito a digito
196 //   (suma de digitos, palindromos, invertir el numero).

```

Ejemplo de funcionamiento:

```

1 int main() {
2     BigInt a = BigInt::desde_string("999999999999999999"); //
        20 nuevas
3     cout << (a + BigInt(1)).str() << "\n";                //
        100000000000000000000000000000000 (acarreo cruza todos los bloques)
4     cout << ((a + BigInt(1)) - BigInt(1)).str() << "\n"; //
        999999999999999999999999
5     cout << (BigInt(5) - BigInt(8)).str() << "\n";      // -3
6
7     BigInt f(1);
8     for (int i = 2; i <= 25; i++) f = f * BigInt(i);
9     cout << f.str() << "\n";                             //
        15511210043330985984000000    (25!)
10
11    BigInt g(1);
12    for (int i = 2; i <= 100; i++) g = g * BigInt(i);
13    cout << g.str().size() << "\n";                       // 158
        (100! tiene 158 digitos)
14
15    BigInt p(1);
16    for (int i = 0; i < 100; i++) p = p * BigInt(2);
17    cout << p.str() << "\n";                               //
        12676506000228229401496703205376    ( $2^{100}$ )
18    int resto = p.dividir_small(1000000000);
19    cout << p.str() << " " << resto << "\n";             //

```

```

1267650600228229401496 703205376 (cociente y resto)
20
21 BigInt x(0), y(1);
22 for (int i = 0; i < 100; i++) { BigInt t = x + y; x = y; y =
    t; }
23 cout << x.str() << "\n"; //
    354224848179261915075 (Fibonacci(100))
24
25 cout << (BigInt::desde_string("-5") < BigInt(3)) << "\n"; //
    1
26 return 0;
27 }
```

Nota: En `operator*`, el máximo valor de `cur` es $(B - 1) + (B - 1)^2 + (B - 1) < 10^{18}$, que cabe en un `long long`. Por eso no se puede usar una base mayor (por ejemplo 2^{31} o 10^{10}) sin cambiar a `unsigned long long` o `__int128`. Además, tras cada operación se llama a `recortar()`; olvidarlo deja bloques cero al final que rompen `cmp_abs` y `operator==`, y puede producir un «cero negativo».

Regla práctica: Si el resultado cabe en `long long` o `__int128`, no uses `BigInt`. Si el problema pide la respuesta módulo p , usa aritmética modular. Usa `BigInt` solo cuando se pida el valor exacto y completo, o cuando las cifras superen los 38 dígitos. Si los números pasan de unos pocos miles de dígitos y hay muchas multiplicaciones, cambia a Karatsuba o FFT (ver modificaciones).

50.2 Fracciones Exactas

Idea: Un número racional p/q se guarda como el par (numerador, denominador), y las operaciones siguen las reglas de la aritmética escolar: $\frac{a}{b} + \frac{c}{d} = \frac{ad+cb}{bd}$, $\frac{a}{b} \cdot \frac{c}{d} = \frac{ac}{bd}$, etc. Lo que hace que el resultado sea *exacto* es que nunca se divide: se conservan numerador y denominador enteros, y se mantiene siempre la **forma canónica**: denominador positivo y $\gcd(|num|, den) = 1$. La forma canónica es única para cada racional, así que dos fracciones son iguales si y solo si sus pares son idénticos.

Para comparar $\frac{a}{b} < \frac{c}{d}$ (con $b, d > 0$) se multiplica en cruz: $ad < cb$. Es válido porque multiplicar ambos lados por $bd > 0$ conserva el orden. Los productos intermedios pueden ser hasta el cuadrado de los valores originales, por eso se hacen en `__int128`.

¿Por qué usarlo? Con `double`, sumas como $0.1 + 0.2$ no dan exactamente 0.3 , y esos errores se acumulan o rompen comparaciones de igualdad; una fracción exacta no tiene ese problema. Frente a la aritmética modular (que devuelve $p \cdot q^{-1} \bmod M$), la fracción conserva el valor real: sirve cuando se necesita *comparar*, ordenar o imprimir la fracción irreducible, cosa que un residuo módulo M no permite. Frente a `BigInt` (subsección anterior), esta estructura es más ligera pero está acotada: los denominadores crecen rápido con muchas sumas (el mínimo común múltiplo de denominadores distintos crece de forma exponencial), y si se desborda `long long` hay que cambiar el tipo del numerador y denominador a `BigInt`.

Casos de uso:

- Comparar pendientes de rectas en geometría sin usar **double** (por ejemplo, agrupar puntos colineales).
- Calcular probabilidades o valores esperados exactos con pocos eventos (sumas de $1/k$, esperanzas de juegos).

- Ordenar fracciones o encontrar la mayor/menor de una lista sin errores de redondeo.
- Resolver sistemas de ecuaciones lineales pequeños con eliminación gaussiana exacta.
- Convertir decimales periódicos a fracción irreducible, o generar fracciones de Farey y aproximaciones racionales.

Complejidad: Cada operación aritmética hace un número fijo de multiplicaciones y sumas, más una llamada a gcd para normalizar. El algoritmo de Euclides tarda $O(\log M)$ pasos, donde M es la magnitud del mayor operando: en cada dos iteraciones el resto se reduce al menos a la mitad (el peor caso son números de Fibonacci consecutivos, Lamé). Por tanto, suma, resta, producto y cociente cuestan $O(\log M)$. La comparación por producto cruzado es $O(1)$ (no necesita gcd). Sumar n fracciones en secuencia es $O(n \log M)$. Memoria: $O(1)$ por fracción.

```

1 struct Fraccion {
2     long long num, den; // forma canonica: den > 0 y mcd(|num|, den) = 1
3
4     // READ: mcd de dos enteros de 128 bits (ignora signos).
5     static __int128 mcd128(__int128 a, __int128 b) {
6         if (a < 0) a = -a;
7         if (b < 0) b = -b;
8         while (b) { a %= b; swap(a, b); }
9         return a;
10    } // O(log M)
11
12    // CREATE: construye la fraccion n/d y la lleva a forma canonica (d != 0).
13    Fraccion(long long n = 0, long long d = 1) : num(n), den(d) {
14        if (den < 0) { num = -num; den = -den; }
15        long long g = (long long)mcd128(num, den);
16        if (g > 1) { num /= g; den /= g; }
17    } // O(log M)
18
19    // CREATE: igual que el constructor, pero desde valores de 128 bits
20    // (para reducir antes de volver a 64 bits, evitando desbordes intermedios).
21    static Fraccion crear(__int128 n, __int128 d) {
22        if (d < 0) { n = -n; d = -d; }
23        __int128 g = mcd128(n, d);
24        if (g > 1) { n /= g; d /= g; }
25        Fraccion r;
26        r.num = (long long)n;
27        r.den = (long long)d;
28        return r;
29    } // O(log M)
30
31    // READ: suma exacta.
32    Fraccion operator+(const Fraccion& o) const {
33        return crear((__int128)num * o.den + (__int128)o.num * den,
34                    (__int128)den * o.den);
35    } // O(log M)
36
37    // READ: resta exacta.
38    Fraccion operator-(const Fraccion& o) const {
39        return crear((__int128)num * o.den - (__int128)o.num * den,
40                    (__int128)den * o.den);
41    } // O(log M)
42
43    // READ: producto exacto.
44    Fraccion operator*(const Fraccion& o) const {
45        return crear((__int128)num * o.num, (__int128)den * o.den);
46    } // O(log M)
47
48    // READ: cociente exacto (requiere o.num != 0).
49    Fraccion operator/(const Fraccion& o) const {
50        return crear((__int128)num * o.den, (__int128)den * o.num);
51    } // O(log M)
52
53    // READ: opuesto.
54    Fraccion operator-() const {
55        Fraccion r;
56        r.num = -num;
57        r.den = den;
58        return r;
59    } // O(1)

```

```

58 // READ: inverso multiplicativo (requiere num != 0).
59 Fraccion inverso() const {
60     return crear(den, num);
61 } // O(log M)
62
63 // READ: comparaciones. Como den > 0, se multiplica en cruz sin dividir.
64 bool operator<(const Fraccion& o) const {
65     return (__int128)num * o.den < (__int128)o.num * den;
66 } // O(1)
67 bool operator==(const Fraccion& o) const {
68     return num == o.num && den == o.den; // forma canonica => igualdad de pares
69 } // O(1)
70
71 // READ: valor aproximado como double (solo para mostrar).
72 double valor() const {
73     return (double)num / (double)den;
74 } // O(1)
75
76 // READ: texto "p/q", o solo "p" si el denominador es 1.
77 string str() const {
78     if (den == 1) return to_string(num);
79     return to_string(num) + "/" + to_string(den);
80 } // O(1)
81 };
82
83 // -----
84 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
85 // -----
86 // 1. Piso (parte entera hacia -infinito), util para partes enteras y mixtos:
87 //     long long piso() const {
88 //         long long q = num / den;
89 //         if (num % den != 0 && num < 0) q--;
90 //         return q;
91 //     } // O(1)
92 //     Numero mixto: piso() y luego Fraccion(num - piso() * den, den).
93 // 2. Potencia entera (exponenciacion binaria; exponente negativo con inverso()):
94 //     Fraccion potencia(Fraccion b, long long e) {
95 //         Fraccion r(1);
96 //         if (e < 0) { b = b.inverso(); e = -e; }
97 //         while (e > 0) { if (e & 1) r = r * b; b = b * b; e >>= 1; }
98 //         return r;
99 //     } // O(log e * log M)
100 // 3. Si el enunciado pide "p/q modulo un primo P", NO uses esta estructura:
101 //     calcula num * inv(den) mod P, con inv(x) = x^(P-2) mod P (Fermat), valido solo si den no es multiplo de P.
102 // 4. Fraccion con BigInt: cambiar num y den a BigInt cuando los denominadores crezcan (por ejemplo, la serie armonica con n grande). Requiere la division BigInt/BigInt y el mcd con ella (ver modificaciones de BigInt).
103 // 5. Decimal periodico a fraccion. Para 0.a(b), con a de k digitos y b de m digitos:
104 //     valor = ( (ab) - a ) / ( 10^k * (10^m - 1) ), donde (ab) es la concatenacion.
105 //     Ejemplo: 0.1(6) = (16 - 1) / (10 * 9) = 15/90 = 1/6.
106 // 6. Aproximacion racional de un double con denominador acotado: fracciones continuas o el arbol de Stern-Brocot (busqueda binaria sobre el arbol), O(log N).
107 // 7. Pendientes en geometria: para comparar dy1/dx1 con dy2/dx2 no hace falta normalizar; basta el producto cruzado dy1 * dx2 < dy2 * dx1 (con dx1, dx2 > 0).

```

Ejemplo de funcionamiento:

```

1 int main() {
2     Fraccion a(1, 2), b(1, 3);
3     cout << (a + b).str() << "\n"; // 5/6
4     cout << (a - b).str() << "\n"; // 1/6
5     cout << (b - a).str() << "\n"; // -1/6
6     cout << (a * b).str() << "\n"; // 1/6
7     cout << (a / b).str() << "\n"; // 3/2
8     cout << Fraccion(6, -8).str() << "\n"; // -3/4 (signo al numerador, reducida)
9     cout << (Fraccion(1, 6) + Fraccion(1, 3)).str() << "\n"; // 1/2 (3/6 reducida)
10    cout << (b < a) << "\n"; // 1 (1*2 < 1*3)

```



```

11 cout << (0.1 + 0.2 == 0.3) << "\n";           // 0 (double
12      falla)
13 cout << (Fraccion(1, 10) + Fraccion(2, 10) == Fraccion(3, 10)
14      ) << "\n"; // 1
15
16 Fraccion h;                                     // 0/1
17 for (int i = 1; i <= 10; i++) h = h + Fraccion(1, i);
18 cout << h.str() << "\n";                       // 7381/2520 (
19      H_10)
20 printf("%.6f\n", h.valor());                    // 2.928968
21 return 0;
22 }

```

Nota: Los productos cruzados se hacen en `__int128`, pero el resultado final se guarda en `long long`: si la fracción ya reducida no cabe en 64 bits, el `cast` trunca en silencio. Si el problema puede producir numeradores o denominadores de más de 18 dígitos (por ejemplo, sumar $1/k$ con k hasta unas decenas de miles), cambia a `BigInt`. Además, el constructor y crear suponen $d \neq 0$; no validan la división por cero.

Regla práctica: Si necesitas el valor exacto de una razón para comparar, ordenar o imprimir irreducible, usa `Fraccion`. Si solo necesitas el resultado módulo un primo, usa aritmética modular con inverso (modificación 3). Si el numerador o denominador se desbordan de 64 bits, usa `BigInt` como tipo interno, y si solo estás comparando pendientes, el producto cruzado sin normalizar es suficiente.

50.3 Diferencia entre Fechas y Sumar Días

Idea: Trabajar directamente con la terna (año, mes, día) es incómodo porque los meses tienen longitudes distintas y los años bisiestos rompen la regularidad. La solución es convertir cada fecha a un único entero: el número de días transcurridos desde una época fija (aquí, 1970-01-01 es el día 0). En esa representación el calendario desaparece: la diferencia entre dos fechas es una resta y sumar k días es una suma. Después se convierte el entero de vuelta a (año, mes, día).

La conversión funciona porque se *desplaza el año para que empiece en marzo*. Así el día extra de los bisiestos (29 de febrero) cae al final del año desplazado y no afecta el conteo de los meses anteriores. Además, los meses desde marzo siguen el patrón 31, 30, 31, 30, 31, 31, 30, 31, 30, 31, 31, (28/29), y ese patrón se captura con la fórmula $\lfloor (153 \cdot mp + 2) / 5 \rfloor$, donde mp es el mes contado desde marzo. Los ciclos gregorianos de 400 años tienen exactamente 146097 días, por lo que basta dividir en «eras» de 400 años y resolver dentro de cada era con aritmética entera.

¿Por qué usarlo? La alternativa ingenua es simular día a día (o mes a mes) hasta llegar a la fecha destino. Eso cuesta $O(\text{días})$ o $O(\text{años})$ y es inviable cuando las fechas están separadas por siglos o hay muchas consultas. Otra alternativa es una tabla de días acumulados por mes, que sirve dentro de un año pero obliga a manejar aparte el cruce de años y los bisiestos. La conversión a número de día hace todo en $O(1)$ con una sola fórmula, es correcta para cualquier año (incluso negativos) y compone bien: una vez que tienes el entero, la diferencia, la suma, la comparación y el día de la semana son operaciones triviales. Se prefiere sobre el simulado siempre que las fechas no sean del mismo mes.

Casos de uso:

- Calcular cuántos días hay entre dos fechas (edad en días, plazos, días de retraso).
- Calcular la fecha que resulta de sumar o restar k días a una fecha dada.
- Determinar el día de la semana de cualquier fecha.
- Ordenar o comparar fechas usando el entero como clave (por ejemplo, en un `sort` o un `map`).
- Problemas de calendarios: contar cuántos viernes 13 hay en un rango, o el siguiente día que cumple una propiedad.

Complejidad: Cada conversión (fecha $\rightarrow$ entero y entero $\rightarrow$ fecha) ejecuta un número fijo de sumas, multiplicaciones y divisiones enteras, sin ciclos ni recursión, por lo que cuesta $O(1)$. La diferencia entre dos fechas son dos conversiones más una resta: $O(1)$. Sumar k días es una conversión, una suma y la conversión inversa: $O(1)$, independiente de la magnitud de k . Memoria: $O(1)$. Para q consultas, el total es $O(q)$.

```

1 struct Fecha {
2     int anio, mes, dia;
3
4     // CREATE: guarda anio, mes y dia (se asume una fecha valida).
5
6     Fecha(int a = 1970, int m = 1, int d = 1) : anio(a), mes(m),
7         dia(d) {} // O(1)
8
9     // READ: indica si un anio es bisiesto (regla gregoriana).
10    static bool es_bisiesto(int a) {
11        return (a % 4 == 0 && a % 100 != 0) || a % 400 == 0;
12    } // O(1)
13
14    // READ: devuelve cuantos dias tiene el mes m del anio a.
15    static int dias_del_mes(int a, int m) {
16        static const int t[12] =
17            {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
18        return (m == 2 && es_bisiesto(a)) ? 29 : t[m - 1];
19    } // O(1)
20
21    // READ: convierte la fecha a dias transcurridos desde
22    // 1970-01-01 (dia 0).
23    // Se desplaza el anio para que empiece en marzo (febrero
24    // queda al final).
25    long long a_dias() const {
26        long long y = anio;
27        y -= (mes <= 2);
28        long long era = (y >= 0 ? y : y - 399) / 400;
29        long long yoe = y - era * 400;
30        // [0, 399]
31        long long mp = (mes > 2) ? mes - 3 : mes + 9;
32        // marzo = 0
33        long long doy = (153 * mp + 2) / 5 + dia - 1;
34        // dia del anio desplazado
35        long long doe = yoe * 365 + yoe / 4 - yoe / 100 + doy;
36        // dia dentro de la era
37        return era * 146097 + doe - 719468;
38    } // O(1)
39
40    // CREATE: construye la fecha correspondiente a z dias desde
41    // 1970-01-01.
42    static Fecha desde_dias(long long z) {
43        z += 719468;
44        long long era = (z >= 0 ? z : z - 146096) / 146097;
45        long long doe = z - era * 146097;
46        // [0, 146096]
47        long long yoe = (doe - doe / 1460 + doe / 36524 - doe /
48            146096) / 365;
49        long long y = yoe + era * 400;
50        long long doy = doe - (365 * yoe + yoe / 4 - yoe / 100);
51        long long mp = (5 * doy + 2) / 153;
52        int d = (int)(doy - (153 * mp + 2) / 5 + 1);
53        int m = (int)(mp < 10 ? mp + 3 : mp - 9);
54        return Fecha((int)(y + (m <= 2)), m, d);
55    } // O(1)
56
57    // READ: dias que hay de "otra" a esta fecha (positivo si
58    // esta es posterior).
59    long long diferencia(const Fecha& otra) const {
60        return a_dias() - otra.a_dias();
61    } // O(1)
62 }

```

```

49 // READ: día de la semana (0 = domingo, 1 = lunes, ..., 6 =
50 // sábado).
51 // 1970-01-01 fue jueves (4); el doble modulo cubre días
52 // negativos.
53 int dia_semana() const {
54     return (int)((a_dias() + 4) % 7 + 7) % 7;
55 } // O(1)
56
57 // WRITE: avanza (o retrocede si k < 0) la fecha k días.
58 void sumar_dias(long long k) {
59     *this = desde_dias(a_dias() + k);
60 } // O(1)
61 };
62
63 // -----
64 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
65 // -----
66 // 1. Restar días: llamar sumar_dias con k negativo. No
67 // requiere código extra
68 // porque a_dias() y desde_dias() funcionan para z negativo.
69 //
70 // 2. Día del año (1..366): sirve para validar o para fechas
71 // ordinales.
72 // int dia_del_año() const {
73 //     return (int)(a_dias() - Fecha(año, 1, 1).a_dias())
74 //         + 1;
75 // } // O(1)
76 //
77 // 3. Sumar meses (con recorte al último día válido del mes
78 // destino):
79 // void sumar_meses(int k) {
80 //     long long total = (long long)año * 12 + (mes - 1)
81 //         + k;
82 //     año = (int)(total / 12); mes = (int)(total % 12) +
83 //         1; // si total < 0, ajustar
84 //     día = min(día, días_del_mes(año, mes));
85 // } // O(1)
86 //
87 // Ojo: 31 de enero + 1 mes da 28/29 de febrero, no 3 de
88 // marzo.
89 //
90 // 4. Siguiente día de la semana d (0..6) estrictamente
91 // posterior:
92 // long long k = ((d - dia_semana() + 6) % 7) + 1;
93 // sumar_dias(k); // O(1)
94 //
95 // 5. Comparar fechas: comparar a_dias() (o la terna en orden
96 // año, mes, día).
97 // Como clave de map/sort: usar a_dias() como llave entera.

```

Ejemplo de funcionamiento:

```

1 int main() {
2     Fecha a(2000, 2, 28), b(2000, 3, 1);
3     cout << b.diferencia(a) << "\n"; // 2 (28-feb ->
4     // 29-feb -> 1-mar, 2000 es bisiesto)
5
6     Fecha c(2024, 1, 1), d(2024, 12, 31);
7     cout << d.diferencia(c) << "\n"; // 365 (2024
8     // tiene 366 días; del 1-ene al 31-dic hay 365 saltos)
9
10    Fecha e(2024, 1, 1);
11    e.sumar_dias(100);
12    cout << e.año << "-" << e.mes << "-" << e.día << "\n"; //
13    // 2024-4-10 (31 + 29 + 31 = 91 días hasta el 1-abr; +9)
14
15    cout << Fecha(2000, 1, 1).a_dias() << "\n"; // 10957 (30
16    // años * 365 + 7 bisiestos)
17    cout << Fecha(2000, 1, 1).dia_semana() << "\n"; // 6 (sábado)
18    cout << Fecha(2024, 1, 1).dia_semana() << "\n"; // 1 (lunes)
19    return 0;
20 }

```

Nota: `a_dias()` devuelve `long long` porque con años grandes (del orden de 10^9) el conteo de días excede los 32 bits. Si el problema garantiza años razonables (por ejemplo entre 1900 y 2100), `int` alcanza, pero usar `long long` no cuesta nada.

Regla práctica: Si necesitas relacionar dos fechas que pueden estar en meses o años distintos, convierte ambas a número de día y opera con enteros. Simula día a día solo cuando el rango sea diminuto (unos pocos días) y no valga la pena escribir la conversión. Usa la tabla de días acumulados por mes solo cuando todas las fechas estén garantizadas dentro de un mismo año no bisiesto.

50.4 Formatos de Hora (12h/24h, Segundos a HH:MM:SS)

Idea: Igual que con las fechas, la forma más limpia de manejar horas es guardarlas como un solo entero: los segundos transcurridos desde las 00:00:00. Como un día tiene $24 \cdot 3600 = 86400$ segundos, cualquier hora del reloj corresponde a un valor en $[0, 86399]$. El formato HH:MM:SS es simplemente una representación en base mixta de ese entero: las horas son el cociente de dividir entre 3600, los minutos son el cociente del resto entre 60, y los segundos son lo que sobra. La conversión inversa es la suma $3600h + 60m + s$.

Sumar tiempo es aritmética modular: el reloj es cíclico, así que después de sumar k segundos se toma el resultado módulo 86400. Con k negativo hay que normalizar con un segundo módulo, porque en C++ el operador % puede devolver un valor negativo. El formato de 12 horas es solo una presentación sobre las mismas horas: $h \bmod 12$ da la hora en el reloj (con el caso especial de que 0 se muestra como 12), y $h < 12$ decide entre AM y PM.

¿Por qué usarlo? Manipular h , m y s por separado obliga a propagar acarreo a mano (segundos $\rightarrow$ minutos $\rightarrow$ horas $\rightarrow$ día) y es la fuente típica de errores por uno. Con el entero, la suma, la resta y la comparación de horas son operaciones directas, y el formateo se hace solo al leer o imprimir. Esto es la misma idea que la conversión de fechas a número de día, aplicada a un dominio cíclico y de tamaño fijo. Una diferencia con las fechas es que aquí hay que decidir explícitamente qué pasa al desbordar: en una *hora del reloj* se envuelve módulo 86400, mientras que en una *duración* no se envuelve y las horas pueden superar 23.

Casos de uso:

- Imprimir un contador de segundos como HH:MM:SS (cronómetros, tiempos de carrera, logs).
- Convertir entre formato de 24 horas y 12 horas con AM/PM (entradas y salidas de problemas de formato).
- Sumar o restar segundos, minutos u horas a una hora del reloj, con vuelta al día siguiente o anterior.
- Calcular cuánto tiempo transcurre entre dos horas del reloj, incluso cruzando la medianoche.
- Aplicar desplazamientos de zona horaria a una hora dada, o formatear duraciones que superan las 24 horas.

Complejidad: Convertir segundos a HH:MM:SS son dos divisiones y dos restos enteros: $O(1)$. La conversión inversa son dos multiplicaciones y dos sumas: $O(1)$. Sumar k segundos es una suma y un módulo: $O(1)$, independiente de k . Formatear a texto es $O(1)$ porque la salida tiene longitud fija (8 caracteres, más el sufijo AM/PM). Parsear una cadena es $O(L)$ con L el largo del texto, que en la práctica es constante (una hora tiene a lo sumo unos 11 caracteres). Memoria: $O(1)$.

```

1 struct Hora {
2     int h, m, s; // siempre en formato de 24
3     // horas
4     static constexpr int DIA = 86400; // segundos en un día

```

```

5 // CREATE: construye la hora a partir de una cantidad de
6 // segundos.
7 // Normaliza modulo 86400 (funciona con valores negativos o
8 // mayores a un dia).
9 Hora(long long seg = 0) {
10     seg = ((seg % DIA) + DIA) % DIA;
11     h = (int)(seg / 3600);
12     m = (int)((seg % 3600) / 60);
13     s = (int)(seg % 60);
14 } // O(1)
15
16 // CREATE: construye la hora a partir de horas, minutos y
17 // segundos.
18 static Hora desde_hms(int hh, int mm, int ss) {
19     return Hora(hh * 3600LL + mm * 60LL + ss);
20 } // O(1)
21
22 // CREATE: lee texto en formato 24h, ej. "07:45:10" o "7:5:3".
23
24 static Hora desde_texto24(const string& t) {
25     int hh = 0, mm = 0, ss = 0;
26     sscanf(t.c_str(), "%d:%d:%d", &hh, &mm, &ss);
27     return desde_hms(hh, mm, ss);
28 } // O(L), L = largo del texto (constante en la practica)
29
30 // CREATE: lee texto en formato 12h, ej. "07:45:10 PM".
31 // 12 AM -> 0 horas, 12 PM -> 12 horas, 1..11 PM -> 13..23.
32 static Hora desde_texto12(const string& t) {
33     int hh = 0, mm = 0, ss = 0;
34     char suf[3] = {};
35     sscanf(t.c_str(), "%d:%d:%d %2s", &hh, &mm, &ss, suf);
36     hh %= 12; // 12 -> 0
37     if (suf[0] == 'p' || suf[0] == 'P') hh += 12;
38     return desde_hms(hh, mm, ss);
39 } // O(L)
40
41 // READ: devuelve los segundos transcurridos desde las
42 // 00:00:00.
43 int a_segundos() const {
44     return h * 3600 + m * 60 + s;
45 } // O(1)
46
47 // READ: devuelve la hora como texto "HH:MM:SS" (24 horas).
48 string formato24() const {
49     char buf[16];
50     sprintf(buf, "%02d:%02d:%02d", h, m, s);
51     return string(buf);
52 } // O(1)
53
54 // READ: devuelve la hora como texto "HH:MM:SS AM/PM" (12
55 // horas).
56 string formato12() const {
57     int h12 = h % 12;
58     if (h12 == 0) h12 = 12; // 0 y 12 se
59     // muestran como 12
60     const char* suf = (h < 12) ? "AM" : "PM";
61     char buf[24];
62     sprintf(buf, "%02d:%02d:%02d %s", h12, m, s,
63         suf);
64     return string(buf);
65 } // O(1)
66
67 // READ: segundos que hay que avanzar desde esta hora hasta "
68 // otra",
69 // cruzando la medianoche si hace falta. Resultado en [0,
70 // 86399].
71 int segundos_hasta(const Hora& otra) const {
72     return ((otra.a_segundos() - a_segundos()) % DIA + DIA) %
73     DIA;
74 } // O(1)
75
76 // WRITE: avanza (o retrocede si k < 0) la hora k segundos,
77 // con vuelta al dia.
78 void sumar_segundos(long long k) {
79     *this = Hora(a_segundos() + k);
80 } // O(1)
81
82 };
83
84 // -----
85 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
86 // -----
87 // 1. Duraciones (sin envolver a 24h): NO usar el constructor
88 // de Hora, porque
89 // aplica modulo 86400. Formatear directamente el entero:
90 // string formato_duracion(long long seg) {
91 //     char buf[32];
92 //     sprintf(buf, "%lld:%02lld:%02lld", seg
93 //         / 3600, (seg % 3600) / 60, seg % 60);
94 //     return string(buf);
95 // } // O(1)
96 // Las horas pueden pasar de 23: 100000 s -> "27:46:40";
97 // 359999 s -> "99:59:59".
98 //

```

```

83 // 2. Milisegundos (HH:MM:SS.mmm): guardar el tiempo en
84 // milisegundos, con
85 // DIA = 86400000, y extraer ms = t % 1000, luego t /= 1000
86 // y seguir igual.
87 // 3. Zona horaria: aplicar el desplazamiento en segundos con
88 // sumar_segundos.
89 // Ej. de UTC a UTC-5: hora.sumar_segundos(-5 * 3600); //
90 // 0(1)
91 // 4. Lectura con cin en lugar de sscanf: leer "int h; char c;
92 // int m, s;"
93 // con cin >> h >> c >> m >> c >> s; (el char absorbe los
94 // '.').
95 // 5. Validar entrada: antes de construir, comprobar 0 <= h <
96 // 24, 0 <= m < 60,
97 // 0 <= s < 60 si el enunciado no garantiza formato correcto.

```

Ejemplo de funcionamiento:

```

1 int main() {
2     Hora a(3661);
3     cout << a.formato24() << "\n"; // 01:01:01
4     // (3600 + 60 + 1)
5
6     Hora b(86399);
7     cout << b.formato24() << "\n"; // 23:59:59
8
9     Hora c(45296);
10    cout << c.formato24() << "\n"; // 12:34:56
11    // (12*3600 = 43200; 2096 = 34*60 + 56)
12    cout << c.formato12() << "\n"; // 12:34:56
13    PM
14
15    cout << Hora::desde_hms(0, 15, 0).formato12() << "\n"; //
16    // 12:15:00 AM
17    cout << Hora::desde_hms(15, 30, 0).formato12() << "\n"; //
18    // 03:30:00 PM
19
20    Hora d = Hora::desde_texto12("07:45:10 PM");
21    cout << d.formato24() << "\n"; // 19:45:10
22
23    Hora e = Hora::desde_hms(23, 59, 30);
24    e.sumar_segundos(45);
25    cout << e.formato24() << "\n"; // 00:00:15
26    // (86370 + 45 = 86415 = 86400 + 15)
27
28    Hora f = Hora::desde_hms(22, 0, 0), g = Hora::desde_hms(6, 0,
29    0);
30    cout << f.segundos_hasta(g) << "\n"; // 28800 (8
31    // horas, cruzando la medianoche)
32    return 0;
33 }

```

Nota: Los casos límite del formato de 12 horas son la fuente de errores más común. La medianoche (00:xx:xx) es 12:xx:xx AM y el mediodía (12:xx:xx) es 12:xx:xx PM; no existe la hora 00 en el reloj de 12 horas. Por eso la conversión usa $h \bmod 12$ y sustituye 0 por 12, en lugar de restar 12 a las horas mayores que 12.

Regla práctica: Si el problema mezcla horas con sumas, restas o comparaciones, guarda todo como segundos (o milisegundos) y convierte solo al leer o imprimir. Usa el constructor con módulo cuando modelos una hora del reloj (que se envuelve en 24h) y el formateo directo de la sección de modificaciones cuando modelos una duración (que puede superar las 24 horas). Si el problema solo pide un cambio de formato sin aritmética, basta con las reglas de `formato12()` y `desde_texto12()`, sin necesidad de más estructura.

51 Parsing y Evaluación de Expresiones

51.1 Tokenizador

Idea: Un tokenizador (o *lexer*) convierte una cadena de caracteres en una lista de *tokens*: las unidades mínimas con significado (números, identificadores, operadores, paréntesis). Es el primer paso de todo evaluador, y

separarlo del resto tiene una razón estructural. Cada clase de token es un lenguaje regular que se reconoce mirando solo el primer carácter y avanzando mientras se cumpla la regla de su clase: dígitos y punto para números, letras, dígitos y guion bajo para identificadores, un solo carácter para operadores y paréntesis. Se aplica la regla de *máxima coincidencia*: cada token consume el prefijo más largo posible (123 es un token, no tres). Así el analizador posterior ya no ve espacios ni números partidos, solo una secuencia limpia.

Hay una ambigüedad que el tokenizador debe resolver con contexto: el signo $-$ puede ser resta ($a - b$) o negación ($-b$). La regla es que un $-$ es unario si no puede seguir a un operando, es decir, si el token anterior no es un número, un identificador o un $)$. Con esa sola comprobación, $3 + 2 * -4$ se separa correctamente. De forma análoga, un identificador seguido de $($ se marca como función, lo que evita que el siguiente paso tenga que mirar hacia adelante.

¿Por qué usarlo? Leer con `cin >> a >> op >> b` solo funciona si la entrada viene separada por espacios; una expresión como $3+4*(2-1)$ no tiene un espacio donde cortar. Trabajar directamente sobre los caracteres dentro del evaluador mezcla dos problemas (reconocer números y respetar la precedencia) y llena el código de casos para espacios y números de varias cifras. `std::regex` resuelve la tokenización, pero es lento y más difícil de depurar que un escáner de 30 líneas. La lectura de romanos de la subsección anterior es una tokenización mínima (un símbolo, un token); aquí los tokens tienen longitud variable. El tokenizador no comprueba que la expresión tenga sentido ($3 + * 4$ se tokeniza sin error), y esa responsabilidad pasa al algoritmo de la siguiente subsección.

Casos de uso:

- Evaluar expresiones aritméticas dadas como texto (calculadoras, problemas de «evalúa la expresión»).
- Leer entradas con formato mixto donde los elementos pueden ir pegados ($f(x,y)=x+2$).
- Validar sintaxis: paréntesis balanceados, caracteres permitidos, números bien formados.
- Interpretar mini-lenguajes o comandos que dan los enunciados (simuladores de una máquina, macros, fórmulas de hojas de cálculo).
- De nicho: resaltado de sintaxis, contar identificadores distintos de un programa o minificar código.

Complejidad: El índice `pos` solo avanza. Cada carácter se consume una vez en el ciclo principal, y el único retroceso aparente es la mirada adelante tras un identificador, que recorre los espacios hasta el siguiente carácter no blanco. Esos espacios se vuelven a visitar en el ciclo principal, así que cada carácter se toca a lo sumo dos veces. Convertir un número con `stod` cuesta proporcional a su largo, y la suma sobre todos los números es a lo sumo L . Por tanto, el total es $O(L)$ con L el largo de la cadena. Memoria: $O(L)$, porque hay a lo sumo un token por carácter.

```

3  Tipo tipo;
4  string txt; // texto original del token
5  double val; // valor numerico (solo si es NUM)
6
7  // CREATE: arma un token.
8  Token(Tipo t = NUM, string x = "", double v = 0) : tipo(t),
9    txt(x), val(v) {} // O(1)
10
11 // READ: texto para imprimir ("neg" para el menos unario).
12 string str() const {
13     return tipo == NEG ? "neg" : txt;
14 } // O(1)
15
16 struct Tokenizador {
17     string s; // expresion
18     int n; // largo de la expresion
19
20     // CREATE: guarda la expresion a analizar.
21     Tokenizador(const string& e) : s(e), n((int)e.size()) {} // O(L)
22
23     // READ: indica si un caracter puede formar parte de un
24     // identificador.
25     static bool es_ident(char c) {
26         return isalnum((unsigned char)c) || c == '_';
27     } // O(1)
28
29     // READ: indica si el ultimo token emitido termina un
30     // operando (NUM, ID o ')').
31     // Si es asi, un '-' que venga despues es resta; si no, es
32     // negacion.
33     static bool termina_operando(const vector<Token>& r) {
34         if (r.empty()) return false;
35         Token::Tipo t = r.back().tipo;
36         return t == Token::NUM || t == Token::ID || t == Token::DER
37         ;
38     } // O(1)
39
40     // READ: separa la expresion en tokens. Lanza runtime_error
41     // si hay algo invalido.
42     vector<Token> tokenizar() const {
43         vector<Token> r;
44         int pos = 0;
45         while (pos < n) {
46             char c = s[pos];
47             if (isspace((unsigned char)c)) { pos++; continue; }
48             if (isdigit((unsigned char)c) || c == '.') {
49                 int ini = pos, puntos = 0;
50                 while (pos < n && (isdigit((unsigned char)s[pos]) || s[
51 pos] == '.')) {
52                     if (s[pos] == '.') puntos++;
53                     pos++;
54                 }
55                 string t = s.substr(ini, pos - ini);
56                 if (puntos > 1 || t == ".") throw runtime_error("numero
57 mal formado: " + t);
58                 r.push_back(Token(Token::NUM, t, stod(t)));
59             } else if (isalpha((unsigned char)c) || c == '_') {
60                 int ini = pos;
61                 while (pos < n && es_ident(s[pos])) pos++;
62                 string t = s.substr(ini, pos - ini);
63                 int q = pos; // mirar
64                 // adelante: es funcion si sigue '('
65                 while (q < n && isspace((unsigned char)s[q])) q++;
66                 bool es_func = (q < n && s[q] == '(');
67                 r.push_back(Token(es_func ? Token::FUNC : Token::ID, t)
68 );
69             } else if (c == '(') { r.push_back(Token(Token::IZQ, "(")
70 ); pos++; }
71             else if (c == ')') { r.push_back(Token(Token::DER, ")"));
72 pos++; }
73             else if (c == ',') { r.push_back(Token(Token::COMA, ","))
74 ; pos++; }
75             else if (string("+*/-%").find(c) != string::npos) {
76                 bool operando_antes = termina_operando(r);
77                 if (!operando_antes && c == '-') r.push_back(Token(
78 Token::NEG, "-"));
79                 else if (!operando_antes && c == '+') { /* '+' unario:
80 se ignora */
81                     else r.push_back(Token(Token::OP, string(1, c)));
82                     pos++;
83                 } else {
84                     throw runtime_error(string("caracter invalido: ") + c);
85                 }
86             }
87         }
88         return r;
89     } // O(L)
90 };
91
92 // -----
93 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
94 // -----

```

```

1 struct Token {
2     enum Tipo { NUM, ID, FUNC, OP, NEG, IZQ, DER, COMA };

```



```

80 // 1. Notacion cientifica (1e-9, 2.5E+3): despues de leer los
81 //    dígitos, si s[pos] es
82 //    'e' o 'E' y sigue un dígito (o un signo y un dígito),
83 //    consumir tambien el
84 //    exponente antes de llamar a stod:
85 //    if (pos + 1 < n && (s[pos]=='e' || s[pos]=='E') &&
86 //        (isdigit((unsigned char)s[pos+1]) ||
87 //         ((s[pos+1]=='+' || s[pos+1]=='-') && pos + 2 < n &&
88 //          isdigit((unsigned char)s[pos+2])))) {
89 //        pos += 2;
90 //        while (pos < n && isdigit((unsigned char)s[pos]))
91 //            pos++;
92 //    } // O(largo del numero)
93 // 2. Operadores de varios caracteres (<=, >=, ==, !=, &&, ||):
94 //    mirar tambien
95 //    s[pos+1] antes de decidir (maxima coincidencia). Se
96 //    prueba primero la
97 //    pareja y, si no coincide, el caracter solo:
98 //    if (pos + 1 < n && (s.substr(pos, 2) == "<=" || s.substr(
99 //        pos, 2) == ">=" ||
100 //        s.substr(pos, 2) == "==" || s.substr(pos, 2) == "!="))
101 //    {
102 //        r.push_back(Token(Token::OP, s.substr(pos, 2)));
103 //        pos += 2;
104 //    } // O(1)
105 // 3. Multiplicacion implicita ("2x", "3(4+1)", "(a)(b)"):
106 //    despues de tokenizar,
107 //    insertar un '*' cuando un operando va seguido de algo que
108 //    empieza un operando.
109 //    vector<Token> r2;
110 //    for (size_t i = 0; i < r.size(); i++) {
111 //        if (i > 0) {
112 //            Token::Tipo a = r[i-1].tipo, b = r[i].tipo;
113 //            bool izq_op = (a == Token::NUM || a == Token::
114 //                ID || a == Token::DER);
115 //            bool der_op = (b == Token::NUM || b == Token::
116 //                ID || b == Token::FUNC || b == Token::IZQ);
117 //            if (izq_op && der_op) r2.push_back(Token(
118 //                Token::OP, "*"));
119 //        }
120 //        r2.push_back(r[i]);
121 //    } // O(n)
122 //    Ojo: "x(3)" ya se toma como llamada a funcion x, no como
123 //    producto.
124 // 4. Posicion en los errores: agregar un campo "int pos" al
125 //    Token y guardarlo al
126 //    crearlo; así el mensaje puede decir "columna k" en lugar
127 //    de solo el caracter.
128 // 5. Lectura perezosa: convertir tokenizar() en un metodo
129 //    siguiente() que guarda
130 //    pos y devuelve un token cada vez (con un token de mirada
131 //    adelante). Es lo que
132 //    usa un parser de descenso recursivo y evita construir el
133 //    vector.
134 // 6. Cadenas y comentarios (para mini-lenguajes): si c == ' ',
135 //    leer hasta la
136 //    comilla de cierre; si aparece "/*", saltar hasta el fin
137 //    de linea.
138 // 7. Cuando NO usar tokenizador: si los elementos de la
139 //    entrada ya vienen
140 //    separados por espacios, basta con "while (ss >> palabra)"
141 //    sobre un
142 //    stringstream. El tokenizador solo hace falta cuando los
143 //    tokens pueden ir pegados.

```

Ejemplo de funcionamiento:

```

1 void imprimir(const string& e) {
2     try {
3         vector<Token> v = Tokenizador(e).tokenizar();
4         cout << v.size() << " ";
5         for (auto& t : v) cout << " " << t.str();
6         cout << "\n";
7     } catch (exception& ex) {
8         cout << "error: " << ex.what() << "\n";
9     }
10 }
11
12 int main() {
13     imprimir("3 + 4.5*(2-1)"); // 9: 3 + 4.5 * ( 2 - 1 )
14     imprimir("-3 + 2*-4"); // 7: neg 3 + 2 * neg 4 (los
15     // '-' tras inicio y tras '*' son unarios)
16     imprimir("sqrt(2)-1"); // 6: sqrt ( 2 ) - 1 (el
17     // '-' tras ')' es resta)
18     imprimir("max(a1, b_2)"); // 6: max ( a1 , b_2 )
19     imprimir("2.5.1 + 3"); // error: numero mal formado:
20     // 2.5.1

```

```

18 imprimir("2 $ 3"); // error: caracter invalido: $
19 return 0;
20 }

```

Nota: Al llamar a `isspace`, `isdigit` y similares con un `char`, conviene convertir primero a `unsigned char`, porque con valores negativos (caracteres fuera de ASCII) el comportamiento es indefinido. `stod` puede lanzar `out_of_range` con números gigantes, así que capturar `exception` y no solo `runtime_error` es más seguro. Este tokenizador ignora el `+` unario en lugar de emitirlo, y trata cualquier identificador seguido de `(` como función, aunque el nombre no exista; esa validación queda para la evaluación.

Regla práctica: Si la entrada puede traer tokens pegados ($3+4*(2-1)$) o de longitud variable, tokeniza primero y trabaja después sobre la lista. Si los elementos ya vienen separados por espacios, un `stringstream` es suficiente. Resuelve el `-` unario aquí, con la regla del token anterior, y no en el evaluador. Y recuerda que el tokenizador solo garantiza que los tokens son válidos, no que la expresión esté bien formada: eso lo comprueba la etapa siguiente.

51.2 Shunting-Yard y Notación Postfija

Idea: La notación postfija (o polaca inversa) escribe cada operador después de sus operandos: $3 + 4 * 2$ se convierte en $3\ 4\ 2\ *\ +$. Su ventaja es que no tiene ambigüedad: el orden ya codifica la precedencia, no necesita paréntesis, y se evalúa con una sola pila (al ver un número se apila, al ver un operador se sacan los dos últimos valores, se opera y se apila el resultado). El algoritmo *shunting-yard*, de Dijkstra, convierte una expresión infija a postfija con una pila de operadores.

Funciona por esta razón: un operador de la pila está *esperando* a que termine su operando derecho. Ese operando deja de poder crecer justo cuando llega otro operador de precedencia menor (o igual, si es asociativo por la izquierda), y solo entonces se puede emitir el operador pendiente. Por eso, al llegar un operador o_1 , se sacan de la pila todos los operadores o_2 con $\text{prec}(o_2) > \text{prec}(o_1)$, o con igual precedencia cuando o_1 es asociativo por la izquierda. Ese caso de igualdad es lo que distingue $a - b - c = (a - b) - c$ (se saca) de $a \wedge b \wedge c = a \wedge (b \wedge c)$ (no se saca). Un paréntesis izquierdo es una barrera que ningún operador puede sacar, y el derecho vacía la pila hasta encontrarla. Los operadores prefijos (la negación) no tienen operando izquierdo que cerrar, así que se apilan directamente sin sacar nada.

¿Por qué usarlo? Frente a evaluar con recursión sobre la cadena, *shunting-yard* es dirigido por una tabla: agregar un operador nuevo es agregar una línea de precedencia, no una función nueva. La forma postfija se puede guardar y evaluar muchas veces con distintos valores de las variables sin volver a analizar el texto, lo cual es lo que se quiere cuando se evalúa una fórmula en miles de puntos. Frente al *descenso recursivo*, que es más flexible (sirve para gramáticas con condicionales, asignaciones o llamadas de aridad variable, y da mejores mensajes de error), *shunting-yard* es más corto y suficiente cuando el lenguaje es solo operadores, paréntesis y funciones. La variante de dos pilas (una de operadores y otra de valores, que aplica cada operador al sacarlo) evita construir la

lista postfija, pero es menos reutilizable. Trabaja sobre la lista de tokens de la subsección anterior, de modo que no vuelve a preocuparse por los espacios ni por el signo unario.

Casos de uso:

- Evaluar expresiones aritméticas con paréntesis, precedencia y asociatividad (calculadoras, problemas de «evaluar la expresión»).
- Evaluar una misma fórmula para muchos valores de una variable (tablas, gráficas, búsqueda binaria sobre una función dada como texto).
- Convertir entre notaciones: infija a postfija, o resolver problemas que dan directamente la forma postfija.
- Evaluar fórmulas booleanas ($\&\&$, $\|\|$, $!$) sobre todas las asignaciones, cambiando solo la tabla de precedencias.
- De nicho: construir el árbol de la expresión para derivarla o simplificarla simbólicamente, o generar código para una máquina de pila.

Complejidad: Sea n el número de tokens. En la conversión, cada token entra a la pila a lo sumo una vez y sale a lo sumo una vez. Aunque el ciclo `while` interno pueda sacar muchos operadores en una sola iteración, el total de salidas sobre toda la ejecución no supera a n , así que la conversión cuesta $O(n)$ amortizado. La evaluación procesa cada token postfijo una vez con operaciones de pila $O(1)$; solo la búsqueda de una variable en el `map` cuesta $O(\log V)$ con V variables, para un total de $O(n \log V)$. Con la tokenización, el proceso completo es $O(L + n \log V)$, y como $n \leq L$ queda en $O(L \log V)$. Memoria: $O(n)$ para la pila y la salida. Evaluar q veces la misma fórmula ya convertida cuesta $O(q \cdot n \log V)$, sin volver a pagar la conversión.

```

1 // Usa Token y Tokenizador de la subseccion anterior.
2 struct ShuntingYard {
3     map<string, double> vars; // valores de las variables
4
5     // CREATE: evaluador sin variables definidas.
6     ShuntingYard() {} // O(1)
7
8     // WRITE: asigna (o cambia) el valor de una variable.
9     void asignar(const string& nombre, double v) {
10         vars[nombre] = v;
11     } // O(log V)
12
13     // READ: precedencia de un operador (mayor = se aplica antes).
14     static int prec(const Token& t) {
15         if (t.tipo == Token::NEG) return 3;
16         char c = t.txt[0];
17         if (c == '+' || c == '-') return 1;
18         if (c == '*' || c == '/' || c == '%') return 2;
19         return 4; // '^'
20     } // O(1)
21
22     // READ: indica si el operador es asociativo por la derecha.
23     static bool asoc_der(const Token& t) {
24         return t.tipo == Token::NEG || t.txt == "^";
25     } // O(1)
26
27     // READ: cantidad de argumentos de una funcion.
28     static int aridad(const string& f) {
29         return (f == "max" || f == "min") ? 2 : 1;
30     } // O(1)
31
32     // READ: convierte tokens en orden infijo a orden postfijo.
33     vector<Token> a_postfija(const vector<Token>& tk) const {
34         vector<Token> out, pila;
35         for (const Token& t : tk) {

```

```

36         switch (t.tipo) {
37             case Token::NUM:
38             case Token::ID:
39                 out.push_back(t);
40                 break;
41             case Token::FUNC:
42             case Token::IZQ:
43             case Token::NEG: // prefijo: se apila sin
44                 sacar_nada
45                 pila.push_back(t);
46                 break;
47             case Token::OP:
48                 while (!pila.empty() &&
49                     (pila.back().tipo == Token::OP || pila.back().tipo ==
50                     Token::NEG) &&
51                     (prec(pila.back()) > prec(t) ||
52                     (prec(pila.back()) == prec(t) && !asoc_der(t)))) {
53                     out.push_back(pila.back());
54                     pila.pop_back();
55                 }
56                 pila.push_back(t);
57                 break;
58             case Token::COMA: // separa argumentos:
59                 cerrar_el_argumento_actual
60                 while (!pila.empty() && pila.back().tipo != Token::IZQ)
61                 {
62                     out.push_back(pila.back());
63                     pila.pop_back();
64                 }
65                 if (pila.empty()) throw runtime_error("coma fuera de
66                 parentesis");
67                 break;
68             case Token::DER:
69                 while (!pila.empty() && pila.back().tipo != Token::IZQ)
70                 {
71                     out.push_back(pila.back());
72                     pila.pop_back();
73                 }
74                 if (pila.empty()) throw runtime_error("parentesis ')'
75                 sin pareja");
76                 pila.pop_back(); // descarta el '('
77                 if (!pila.empty() && pila.back().tipo == Token::FUNC) {
78                     out.push_back(pila.back()); // la funcion sale
79                     // despues de sus argumentos
80                     pila.pop_back();
81                 }
82                 break;
83         }
84     }
85     while (!pila.empty()) {
86         if (pila.back().tipo == Token::IZQ) throw runtime_error("
87         parentesis '(' sin pareja");
88         out.push_back(pila.back());
89         pila.pop_back();
90     }
91     return out;
92 } // O(n) amortizado

```

```

// READ: evalua una lista postfija con una pila de valores.
double evaluar_postfija(const vector<Token>& pf) const {
    vector<double> st;
    for (const Token& t : pf) {
        if (t.tipo == Token::NUM) {
            st.push_back(t.val);
        } else if (t.tipo == Token::ID) {
            auto it = vars.find(t.txt);
            if (it == vars.end()) throw runtime_error("variable sin
            valor: " + t.txt);
            st.push_back(it->second);
        } else if (t.tipo == Token::NEG) {
            if (st.empty()) throw runtime_error("expresion mal
            formada");
            st.back() = -st.back();
        } else if (t.tipo == Token::OP) {
            if (st.size() < 2) throw runtime_error("expresion mal
            formada");
            double b = st.back(); st.pop_back();
            double a = st.back(); st.pop_back();
            char c = t.txt[0];
            double r;
            if (c == '+') r = a + b;
            else if (c == '-') r = a - b;
            else if (c == '*') r = a * b;
            else if (c == '/') r = a / b;
            else if (c == '%') r = fmod(a, b);
            else r = pow(a, b);
            st.push_back(r);
        } else if (t.tipo == Token::FUNC) {
            int k = aridad(t.txt);
            if ((int)st.size() < k) throw runtime_error("faltan
            argumentos en " + t.txt);
            double b = st.back(); st.pop_back();
            double a = 0;

```

```

116         if (k == 2) { a = st.back(); st.pop_back(); }
117         double r;
118         if (t.txt == "sqrt") r = sqrt(b);
119         else if (t.txt == "abs") r = fabs(b);
120         else if (t.txt == "max") r = max(a, b);
121         else if (t.txt == "min") r = min(a, b);
122         else throw runtime_error("funcion desconocida: " + t.
txt);
123         st.push_back(r);
124     }
125 }
126 if (st.size() != 1) throw runtime_error("expresion mal
formada");
127 return st[0];
128 } // O(n log V)
129
130 // READ: tokeniza, convierte y evalua una expresion completa.
131 double evaluar(const string& e) const {
132     return evaluar_postfija(a_postfija(Tokenizador(e).tokenizar
()));
133 } // O(L log V)
134 };
135
136 // -----
137 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
138 // -----
139 // 1. Evaluar la misma formula muchas veces: convertir UNA vez
y reutilizar la
140 // lista postfija, cambiando solo las variables.
141 // vector<Token> pf = sy.a_postfija(Tokenizador("x^2 + 1").
tokenizar()); // O(n)
142 // for (double x = 0; x <= 10; x += 0.5) {
143 //     sy.asignar("x", x);
144 //     double r = sy.evaluar_postfija(pf); // O(n log V),
sin volver a analizar el texto
145 // }
146 //
147 // 2. Agregar operadores: basta ampliar prec() y el bloque de
evaluacion. Un
148 // operador prefijo nuevo (por ejemplo '!' logico) se trata
igual que NEG:
149 // se apila sin sacar, con asociatividad derecha y
precedencia alta. Para
150 // operadores de comparacion o logicos (<, ==, <=, ||) hay
que dar precedencias
151 // menores que las de '+' y tokenizarlos con la modificacion
2 del tokenizador.
152 //
153 // 3. Arbol de expresion: en lugar de evaluar, apilar nodos. Un
numero o variable
154 // crea una hoja; un operador saca dos nodos y apila uno
nuevo con ellos como hijos.
155 // struct Nodo { Token t; Nodo *izq = nullptr, *der =
nullptr; };
156 // // al leer un OP: Nodo* b = st.back(); st.pop_back();
Nodo* a = st.back(); st.pop_back();
157 // // st.push_back(new Nodo{t, a, b});
158 // // O(n). La raiz queda en st[0]. Recorrido en postorden =
postfija; en preorden =
159 // // notacion prefija; en inorden con parentesis = infija
completamente agrupada.
160 // // Util para derivar o simplificar simbolicamente.
161 //
162 // 4. Variante de dos pilas: no construir 'out'; cada vez que
un operador saldría a
163 // la salida, aplicarlo de inmediato a la pila de valores.
Se evalua en una sola
164 // pasada, pero no se puede reutilizar. O(n log V).
165 //
166 // 5. Valores exactos: cambiar double por Fraccion o BigInt en
la pila de valores
167 // (subsecciones anteriores). Hay que reemplazar '^' por una
potencia entera y
168 // decidir que pasa con la division por cero, que aqui da
inf (o se puede lanzar error).
169 //
170 // 6. Funciones de aridad variable: guardar en la pila, junto a
cada FUNC, un
171 // contador de argumentos que sube en cada COMA, y emitir
ese numero junto con
172 // la funcion (por ejemplo, un token "max/3").
173 //
174 // 7. Entrada ya en postfija (problemas de RPN): saltarse
a_postfija y leer los
175 // tokens con un stringstream, llamando directo a
evaluar_postfija.
176 //
177 // 8. Cuando el lenguaje tiene condicionales, asignaciones o
bloques, usar descenso
178 // recursivo: una funcion por nivel de precedencia (suma ->
producto -> potencia
179 // -> atomo). Mismo O(n), mas flexible, y da mejores
mensajes de error.

```

Ejemplo de funcionamiento:

```

1 int main() {
2     ShuntingYard sy;
3     sy.asignar("x", 3);
4     sy.asignar("y", 4);
5
6     auto mostrar = [&](const string& e) {
7         try {
8             vector<Token> pf = sy.a_postfija(Tokenizador(e).tokenizar
());
9             double v = sy.evaluar_postfija(pf);
10            for (auto& t : pf) cout << t.str() << " ";
11            cout << "=> " << v << "\n";
12        } catch (exception& ex) {
13            cout << "error: " << ex.what() << "\n";
14        }
15    };
16
17    mostrar("3 + 4 * 2"); // 3 4 2 * + => 11
18    mostrar("(1 + 2) * 3"); // 1 2 + 3 * => 9
19    mostrar("10 - 4 - 3"); // 10 4 - 3 - => 3 (
resta asociativa por la izquierda)
20    mostrar("2 ^ 3 ^ 2"); // 2 3 2 ^ ^ => 512 (
potencia por la derecha: 2^9)
21    mostrar("-2 ^ 2"); // 2 2 ^ neg => -4 (la
potencia se aplica antes que la negacion)
22    mostrar("2 ^ -1"); // 2 1 neg ^ => 0.5
23    mostrar("sqrt(x^2 + y^2)"); // x 2 ^ y 2 ^ + sqrt => 5
24    mostrar("sqrt(16) + max(2, 7)"); // 16 sqrt 2 7 max + => 11
25    mostrar("2 * (3 + 4)"); // error: parentesis '(' sin
pareja
26    mostrar("3 +"); // error: expresion mal
formada
27    mostrar("z + 1"); // error: variable sin valor
: z
28    return 0;
29 }

```

Nota: La precedencia de la negación es una decisión de diseño. Aquí -2^2 da -4 (la convención matemática, la misma de Python), pero otras herramientas, como Excel, lo evalúan como $(-2)^2 = 4$. Si el enunciado usa la segunda convención, basta con darle a NEG una precedencia mayor que la de $\wedge$. Además, la aridad de las funciones está fija por nombre (max y min reciben dos argumentos, el resto uno); una llamada con un número de argumentos distinto no se detecta en la conversión y se descubre como «expresión mal formada» al evaluar. El operador / con divisor cero devuelve infinito en lugar de lanzar un error.

Regla práctica: Si el lenguaje es solo operadores con precedencia, paréntesis y funciones, usa tokenizador más shunting-yard. Convierte una sola vez a postfija si vas a evaluar la misma fórmula muchas veces. Si solo necesitas un valor una vez, la variante de dos pilas ahorra la lista intermedia. Si la gramática incluye condicionales, asignaciones, bloques o funciones de aridad variable, pasa a descenso recursivo. Si necesitas derivar o transformar la expresión, construye el árbol (modificación 3) en lugar de evaluarla directamente.

51.3 Descenso Recursivo

Idea: El descenso recursivo escribe el analizador como una función por cada nivel de la gramática, y cada función llama a la del nivel siguiente, que es el de mayor precedencia. Para las expresiones aritméticas la gramática es:

```

expr → termino ( (+ | -) termino ) *
termino → unario ( (* | / | %) unario ) *
unario → neg unario | potencia
potencia → atomo [ ^ unario ]
atomo → num | id | ( expr ) | func( expr, ... )

```

La precedencia no se guarda en ninguna tabla: sale de la *profundidad* de las llamadas. Para calcular $3 + 4 * 2$, **expr** llama a **termino**, que a su vez llega hasta un **atomo** y devuelve 3. Al ver **+**, **expr** vuelve a llamar a **termino** para el operando derecho, y esa llamada consume $4 * 2$ completo antes de devolver el control. El ***** se agrupa primero porque vive en una función más profunda, que se resuelve antes de que **expr** pueda seguir.

La asociatividad la decide la forma de la regla. Un ciclo **while** en **expr** y **termino** acumula de izquierda a derecha, y así $10 - 4 - 3 = (10 - 4) - 3$. Una llamada recursiva a la derecha en **potencia** agrupa hacia la derecha, y así $2^{3^2} = 2^{(3^2)}$. La negación está encima de **potencia**, así que -2^2 se lee como $-(2^2)$, y como el exponente es un **unario**, 2^{-1} también funciona. Una regla como **expr** $\rightarrow$ **expr** + **termino** (recursiva por la izquierda) provocaría recursión infinita, y por eso se reescribe como ciclo.

¿Por qué usarlo? Frente a shunting-yard (subsección anterior), ambos son $O(n)$ y aceptan las mismas expresiones, pero el descenso recursivo evalúa directamente durante el análisis, sin lista postfija ni pila explícita de operadores. Su ventaja real es la flexibilidad: cada regla es código arbitrario, así que es natural aceptar funciones con cualquier número de argumentos ($\max(1, 5, 3)$), condicionales, asignaciones, bloques, y dar mensajes de error que dicen qué se esperaba y dónde. Cada regla de la gramática se traduce a una función casi mecánicamente, lo que facilita razonar sobre la corrección. Su desventaja es que no guarda una forma reutilizable: evaluar la misma fórmula para muchos valores de **x** repite todo el análisis, salvo que se construya un árbol (modificación 3). Y cada nivel de precedencia nuevo es una función nueva, mientras que en shunting-yard es una línea de tabla (la modificación 2 muestra una versión dirigida por tabla, llamada *precedence climbing*).

Casos de uso:

- Evaluar expresiones aritméticas con precedencia, paréntesis y funciones de aridad variable.
- Interpretes de mini-lenguajes: asignaciones, condicionales, bucles y bloques definidos en el enunciado.
- Validar que un texto pertenece a una gramática (paréntesis balanceados, formatos como fechas, JSON o rutas), con mensajes de error precisos.
- Construir el árbol sintáctico de una expresión para derivarla o simplificarla.
- De nicho: analizadores de fórmulas de hojas de cálculo, de consultas o de expresiones regulares simples.

Complejidad: Sea n el número de tokens. El índice p solo avanza y ninguna función retrocede: cada regla decide qué hacer mirando únicamente el token actual (gramática $LL(1)$), por lo que no hay retroceso ni trabajo repetido. Cada token se consume exactamente una vez. Un operando pasa por una cantidad constante de niveles ($k = 5$ funciones), así que el trabajo total es $O(k \cdot n) = O(n)$. La búsqueda de una variable en el **map** suma $O(\log V)$, para un total de $O(n \log V)$, y con la tokenización, $O(L \log V)$. Memoria: la lista de tokens es $O(n)$ y la profundidad de la recursión es proporcional al

anidamiento máximo de paréntesis y negaciones, hasta $O(n)$ en el peor caso, como en $(((((...))))))$ o $---...1$. Con miles de niveles no hay problema, pero con cientos de miles la pila del sistema puede desbordarse.

```

1 // Usa Token y Tokenizador de la subseccion "Tokenizador".
2 struct DescensoRecursivo {
3     vector<Token> t;           // tokens de la expresion actual
4     size_t p;                 // posicion del token que se esta
                                // mirando
5     map<string, double> vars; // valores de las variables
6
7     // CREATE: analizador vacio, sin variables.
8     DescensoRecursivo() : p(0) {} // O(1)
9
10    // WRITE: asigna (o cambia) el valor de una variable.
11    void asignar(const string& nombre, double v) {
12        vars[nombre] = v;
13    } // O(log V)
14
15    // READ: indica si el token actual es el operador binario c.
16    bool es_op(char c) const {
17        return p < t.size() && t[p].tipo == Token::OP && t[p].txt
18        [0] == c;
19    } // O(1)
20
21    // WRITE: consume un token del tipo dado o lanza un error
22    // descriptivo.
23    void esperar(Token::Tipo tipo, const string& que) {
24        if (p >= t.size() || t[p].tipo != tipo)
25            throw runtime_error("se esperaba " + que + " en el token "
26            + to_string(p));
27        p++;
28    } // O(1)
29
30    // READ: aplica una funcion a su lista de argumentos ya
31    // evaluados.
32    static double aplicar(const string& f, const vector<double>&
33    a) {
34        if (f == "sqrt" || f == "abs") {
35            if (a.size() != 1) throw runtime_error(f + " recibe 1
36            argumento");
37            return f == "sqrt" ? sqrt(a[0]) : fabs(a[0]);
38        }
39        if (f == "max" || f == "min") {
40            if (a.empty()) throw runtime_error(f + " necesita al
41            menos 1 argumento");
42            double r = a[0];
43            for (double x : a) r = (f == "max") ? max(r, x) : min(r,
44            x);
45            return r;
46        }
47        throw runtime_error("funcion desconocida: " + f);
48    } // O(#argumentos)
49
50    // WRITE: atomo -> numero | variable | '(' expr ')' | funcion
51    // '(' args ')'.
52    double atomo() {
53        if (p >= t.size()) throw runtime_error("expresion
54        incompleta");
55        const Token& tk = t[p];
56        if (tk.tipo == Token::NUM) { p++; return tk.val; }
57        if (tk.tipo == Token::ID) {
58            p++;
59            auto it = vars.find(tk.txt);
60            if (it == vars.end()) throw runtime_error("variable sin
61            valor: " + tk.txt);
62            return it->second;
63        }
64        if (tk.tipo == Token::IZQ) {
65            p++;
66            double v = expr();
67            esperar(Token::DER, "')'");
68            return v;
69        }
70        if (tk.tipo == Token::FUNC) {
71            string nombre = tk.txt;
72            p++;
73            esperar(Token::IZQ, "'('");
74            vector<double> args;
75            if (p < t.size() && t[p].tipo != Token::DER) {
76                args.push_back(expr());
77                while (p < t.size() && t[p].tipo == Token::COMA) { p++;
78                args.push_back(expr()); }
79            }
80            esperar(Token::DER, "')'");
81            return aplicar(nombre, args);
82        }
83        throw runtime_error("token inesperado: " + tk.str() + " en
84        el token " + to_string(p));
85    } // O(n) en total

```



```

74 // WRITE: potencia -> atomo [ '^' unario ] (asociativa por
75 // la derecha).
76 double potencia() {
77     double base = atomo();
78     if (es_op('^')) {
79         p++;
80         double e = unario(); // permite 2^-1 y 2^3^2
81         return pow(base, e);
82     }
83     return base;
84 } // O(n) en total
85
86 // WRITE: unario -> neg unario | potencia.
87 double unario() {
88     if (p < t.size() && t[p].tipo == Token::NEG) { p++; return
89         -unario(); }
90     return potencia();
91 } // O(n) en total
92
93 // WRITE: termino -> unario (('*' | '/' | '%') unario)* (por
94 // la izquierda).
95 double termino() {
96     double v = unario();
97     while (es_op('*') || es_op('/') || es_op('%')) {
98         char c = t[p].txt[0];
99         p++;
100         double r = unario();
101         v = (c == '*') ? v * r : (c == '/') ? v / r : fmod(v, r);
102     }
103     return v;
104 } // O(n) en total
105
106 // WRITE: expr -> termino (('+' | '-') termino)* (por la
107 // izquierda).
108 double expr() {
109     double v = termino();
110     while (es_op('+') || es_op('-')) {
111         char c = t[p].txt[0];
112         p++;
113         double r = termino();
114         v = (c == '+') ? v + r : v - r;
115     }
116     return v;
117 } // O(n) en total
118
119 // WRITE: tokeniza y evalua una expresion completa (exige
120 // consumir todos los tokens).
121 double evaluar(const string& e) {
122     t = Tokenizador(e).tokenizar();
123     p = 0;
124     double v = expr();
125     if (p != t.size()) throw runtime_error("sobran tokens desde
126     el token " + to_string(p));
127     return v;
128 } // O(L log V)
129 };
130
131 // -----
132 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
133 // -----
134 // 1. Agregar un nivel de precedencia: escribir una funcion
135 // nueva ENTRE dos
136 // existentes y cambiar quien la llama. Ej. comparaciones (
137 // menor prioridad que '+'):
138 // double comparacion() {
139 //     double v = expr();
140 //     if (es_op('<')) { p++; return v < expr(); }
141 //     return v;
142 // } // O(n)
143 // y evaluar() llama a comparacion() en lugar de expr().
144 // Requiere que el
145 // tokenizador emita '<' (ver modificacion 2 del tokenizador
146 // ).
147 // 2. Precedence climbing (una sola funcion dirigida por tabla,
148 // como shunting-yard):
149 // double binaria(int min_prec) {
150 //     double lhs = unario_pc();
151 //     while (p < t.size() && t[p].tipo == Token::OP &&
152 //     prec(t[p]) >= min_prec) {
153 //         Token op = t[p]; p++;
154 //         double rhs = binaria(asoc_der(op) ? prec(op) :
155 //         prec(op) + 1);
156 //         lhs = aplicar_op(op, lhs, rhs);
157 //     }
158 //     return lhs;
159 // } // O(n)
160 // Con prec/asoc_der de la subseccion de shunting-yard. La
161 // negacion se procesa
162 // como: si es NEG, p++, return -binaria(prec_neg); asi -2^2
163 // sigue dando -4
164 // porque '^' (prec 4) >= prec_neg (3) queda dentro del
165 // operando.

```

```

151 //
152 // 3. Arbol de sintaxis: en lugar de devolver double, cada
153 // funcion devuelve Nodo*.
154 // struct Nodo { Token op; Nodo *izq = nullptr, *der =
155 //     nullptr; };
156 // // en expr(): v = new Nodo{tokenOp, v, r}; // en vez de
157 // v = v + r
158 // // Se recorre despues (evaluar, derivar, simplificar) sin
159 // volver a analizar el
160 // // texto: O(n) para construir, O(n) por cada evaluacion.
161 //
162 // 4. Asignaciones y sentencias ("x = 5; y = x * 2"): agregar
163 // una regla
164 // sentencia -> ID '=' expr | expr, mirando un token
165 // adelante (t[p+1] es '=').
166 // Como cada regla decide con UN token de mirada, sigue
167 // siendo O(n).
168 // Requiere que el tokenizador emita '=' y ';'.
169 //
170 // 5. Condicional (cond ? a : b): una regla por encima de la
171 // comparacion.
172 // double ternario() {
173 //     double c = comparacion();
174 //     if (p < t.size() && t[p].txt == "?") {
175 //         p++; double a = ternario(); esperar_texto
176 //         (":"); double b = ternario();
177 //         return c != 0 ? a : b; // evalua ambas
178 //         ramas; para evitar efectos usar arbol
179 //     }
180 //     return c;
181 // } // O(n)
182 //
183 // 6. Gramaticas con recursion por la izquierda: NO se pueden
184 // escribir tal cual
185 // (recursion infinita). Se transforman: A -> A x | y ==>
186 // A -> y (x)*,
187 // que es lo que hacen los ciclos while de expr() y termino
188 // ().
189 //
190 // 7. Profundidad de recursion: para entradas con anidamiento
191 // de 1e5 o mas,
192 // la pila del sistema puede agotarse. Opciones: ampliar la
193 // pila (en Linux,
194 // ulimit -s unlimited), pasar a shunting-yard (pila propia
195 // en el heap), o
196 // poner un limite y lanzar un error si el anidamiento lo
197 // supera.
198 //
199 // 8. Analizar directamente sobre caracteres, sin tokenizador:
200 // es viable para
201 // gramaticas cortas. Se mantiene un indice p sobre la
202 // cadena, un saltar_blanco()
203 // al inicio de cada regla, y numero() lee los digitos con
204 // stod. Mismo O(L).
205 //
206 // 9. Backtracking / gramaticas ambiguas: si una regla necesita
207 // probar una
208 // alternativa y volver atras, el costo puede volverse
209 // exponencial. Se
210 // arregla con memoizacion por (regla, posicion), tecnica
211 // llamada "packrat",
212 // con O(n * #reglas). Para expresiones aritmeticas no hace
213 // falta.

```

Ejemplo de funcionamiento:

```

1 int main() {
2     DescensoRecursivo dr;
3     dr.asignar("x", 3);
4     dr.asignar("y", 4);
5
6     auto mostrar = [&](const string& e) {
7         try {
8             cout << dr.evaluar(e) << "\n";
9         } catch (exception& ex) {
10             cout << "error: " << ex.what() << "\n";
11         }
12     };
13
14     mostrar("3 + 4 * 2"); // 11 (el '*' esta
15     // en un nivel mas profundo)
16     mostrar("(1 + 2) * 3"); // 9
17     mostrar("10 - 4 - 3"); // 3 (el ciclo
18     // acumula por la izquierda: (10-4)-3)
19     mostrar("2 ^ 3 ^ 2"); // 512 (recursion por
20     // la derecha: 2^(3^2) = 2^9)
21     mostrar("-2 ^ 2"); // -4 (unario esta
22     // encima de potencia: -(2^2))
23     mostrar("2 ^ -1"); // 0.5
24     mostrar("1 - -2"); // 3 (el segundo
25     // '-' es unario)
26     mostrar("sqrt(x^2 + y^2)"); // 5
27     mostrar("max(1, 5, 3) + min(4, 2)"); // 7 (aridad

```

```

23     variable: 5 + 2)
    mostrar("2 * (3 + 4");           // error: se esperaba
    ')' en el token 6
24     mostrar("3 +");               // error: expresion
    incompleta
25     mostrar("3 4");               // error: sobran tokens
    desde el token 1
26     mostrar("z + 1");             // error: variable sin
    valor: z
27     return 0;
28 }

```

Nota: El tokenizador ya distingue la negación de la resta, y el analizador no lo vuelve a decidir: le basta con ver un token NEG en unario. Cada función devuelve un `double` y por eso los errores se reportan lanzando excepciones; al usarlas, la posición reportada es el índice del token, no la columna del texto (para eso, la modificación 4 del tokenizador agrega el campo `pos`). Además, `evaluar` modifica `t` y `p`, así que un mismo objeto no es reentrante: para analizar en paralelo o desde dentro de una función de usuario, usa un objeto por hilo. Al igual que en shunting-yard, la división entre cero devuelve infinito en lugar de lanzar error.

Regla práctica: Si la entrada es solo operadores, paréntesis y funciones de aridad fija, y vas a evaluar la misma fórmula muchas veces, shunting-yard con la lista postfija reusable es más corto. Si el lenguaje tiene funciones de aridad variable, asignaciones, condicionales, bloques, o necesitas buenos mensajes de error, usa descenso recursivo: una función por nivel de precedencia, ciclos para la asociatividad izquierda y recursión para la derecha. Si además vas a evaluar la misma expresión repetidas veces, haz que las funciones construyan un árbol y evalúalo aparte (modificación 3). Si el anidamiento puede pasar de cientos de miles de niveles, evita la recursión y usa shunting-yard.

52 Técnicas Ad-hoc

52.1 Compresión de Coordenadas

Idea: Muchos algoritmos solo necesitan saber qué valor es menor, igual o mayor que otro, y no cuánto valen. Un arreglo de frecuencias, un árbol de Fenwick o un barrido necesitan además índices pequeños y consecutivos. La compresión de coordenadas sustituye cada valor por su *rango* dentro del conjunto ordenado de valores distintos: $\{-7, 5, 100, 10^9\}$ pasa a $\{0, 1, 2, 3\}$. Con n valores el resultado siempre cabe en $[0, n)$, sin importar lo grandes o negativos que fueran los originales.

Funciona porque la función $x \mapsto \text{rango}(x)$ es una biyección estrictamente creciente entre los valores usados y $[0, k)$, donde k es la cantidad de valores distintos. Al ser creciente, conserva todas las comparaciones ($x < y \iff \text{rango}(x) < \text{rango}(y)$) y la igualdad, así que cualquier algoritmo que solo dependa del orden da el mismo resultado sobre los rangos. Lo que se pierde son las distancias entre valores, como se explica en las modificaciones. La implementación es ordenar, eliminar duplicados y usar búsqueda binaria: el rango de x es su posición en el arreglo ordenado.

¿Por qué usarlo? Si los valores llegan a 10^9 pero solo hay 10^5 de ellos, un arreglo indexado por valor es inviable en memoria. Un `map` o `unordered_map` sirve para contar, pero no permite usar estructuras indexadas por

posición (Fenwick, árbol de segmentos, arreglos de diferencias), que son las que hacen rápidas las consultas de rango. Un `unordered_map` tampoco conserva el orden, así que solo sirve cuando basta la identidad de los valores (modificación 5). La compresión es la elección correcta cuando los valores son grandes o dispersos, se conocen todos de antemano (fuera de línea) y solo importa su orden relativo. Si los valores ya son pequeños (por ejemplo, hasta 10^6), es un paso innecesario. Si las consultas llegan en línea, no se puede comprimir de antemano y hay que usar un árbol de segmentos dinámico o un `map` (modificación 7).

Casos de uso:

- Usar un Fenwick o un árbol de segmentos sobre valores grandes (contar inversiones, subsecuencia creciente más larga, consultas de rango sobre valores).
- Contar frecuencias o hacer sumas por prefijos sobre valores dispersos, o reenumerar identificadores de nodos de un grafo a $0, \dots, n - 1$.
- Barridos de línea: unión de intervalos, cobertura de segmentos y eventos ordenados en el tiempo.
- Puntos en el plano con consultas fuera de línea: se comprime cada eje por separado y se trabaja sobre una cuadrícula pequeña.
- De nicho: convertir una permutación arbitraria en una permutación de $0, \dots, n - 1$, o comprimir cadenas y pares (cualquier tipo comparable).

Complejidad: Sea n la cantidad de valores. La construcción ordena en $O(n \log n)$ y `unique` recorre el arreglo una vez en $O(n)$, así que domina el ordenamiento. Cada consulta de rango (`indice`, `contar_menores`) es una búsqueda binaria en $O(\log k) \subseteq O(\log n)$. Comprimir un arreglo de m valores cuesta $O(m \log n)$. Si esos m valores ya vienen ordenados, un doble puntero baja el costo a $O(n + m)$ (modificación 8). Agregar valores nuevos vuelve a ordenar y cuesta $O(n \log n)$. Memoria: $O(n)$.

```

1 struct Compresor {
2     vector<long long> vals; // valores distintos, ordenados de
                             // menor a mayor
3
4     // CREATE: ordena y elimina duplicados; el rango de x es su
                             // posicion en vals.
5     Compresor(vector<long long> v) {
6         vals = move(v);
7         sort(vals.begin(), vals.end());
8         vals.erase(unique(vals.begin(), vals.end()), vals.end());
9     } // O(n log n)
10
11     // READ: cantidad de valores distintos (tamano del universo
                             // comprimido).
12     int tam() const {
13         return (int)vals.size();
14     } // O(1)
15
16     // READ: valor original que corresponde al rango i.
17     long long valor(int i) const {
18         return vals[i];
19     } // O(1)
20
21     // READ: indica si x estaba entre los valores originales.
22     bool existe(long long x) const {
23         auto it = lower_bound(vals.begin(), vals.end(), x);
24         return it != vals.end() && *it == x;
25     } // O(log n)
26
27     // READ: rango de x (requiere que x exista; si no, devuelve
                             // su posicion de insercion).
28     int indice(long long x) const {

```

```

29     return (int)(lower_bound(vals.begin(), vals.end(), x) -
30     vals.begin());
31 } // O(log n)
32
33 // READ: cuantos valores distintos son estrictamente menores
34 // que x (x puede no existir).
35 int contar_menores(long long x) const {
36     return (int)(lower_bound(vals.begin(), vals.end(), x) -
37     vals.begin());
38 } // O(log n)
39
40 // READ: cuantos valores distintos son menores o iguales que
41 // x (x puede no existir).
42 int contar_menores_o_iguales(long long x) const {
43     return (int)(upper_bound(vals.begin(), vals.end(), x) -
44     vals.begin());
45 } // O(log n)
46
47 // READ: devuelve el arreglo v con cada valor sustituido por
48 // su rango.
49 vector<int> comprimir(const vector<long long>& v) const {
50     vector<int> r(v.size());
51     for (size_t i = 0; i < v.size(); i++) r[i] = indice(v[i]);
52     return r;
53 } // O(m log n)
54
55 // WRITE: agrega valores nuevos al universo. Los rangos ya
56 // entregados dejan de valer.
57 void agregar_valores(const vector<long long>& extra) {
58     vals.insert(vals.end(), extra.begin(), extra.end());
59     sort(vals.begin(), vals.end());
60     vals.erase(unique(vals.begin(), vals.end(), vals.end()));
61 } // O(n log n)
62
63 // -----
64 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
65 // -----
66 // 1. Intervalos semiabiertos [l, r) y barrido: agregar al
67 // compresor TODOS los l y r.
68 // La celda i representa [vals[i], vals[i+1]) y mide vals[i]
69 // +1 - vals[i].
70 // Longitud de la union de segmentos con arreglo de
71 // diferencias:
72 // vector<int> dif(c.tam() + 1);
73 // for (auto [l, r] : segs) { dif[c.indice(l)]++; dif[c.
74 // indice(r)]--; }
75 // long long total = 0; int cubierto = 0;
76 // for (int i = 0; i + 1 < c.tam(); i++) {
77 //     cubierto += dif[i];
78 //     if (cubierto > 0) total += c.valor(i + 1) - c.valor
79 // (i);
80 // } // O(n log n)
81 // 2. Rango con empates distintos (convertir en permutacion de
82 // 0..n-1): ordenar los
83 // indices por valor y, si hay empate, por posicion original.
84 // vector<int> ord(n), rango(n);
85 // iota(ord.begin(), ord.end(), 0);
86 // sort(ord.begin(), ord.end(), [i](int i, int j) { return v
87 // [i] < v[j] || (v[i] == v[j] && i < j); });
88 // for (int k = 0; k < n; k++) rango[ord[k]] = k; // O(n
89 // log n)
90 // 3. Puntos en 2D: un compresor por eje. (x, y) pasa a (cx.
91 // indice(x), cy.indice(y)).
92 // Se conserva el orden en cada eje, pero no las distancias
93 // ni las diagonales.
94 // 4. Cualquier tipo comparable (strings, pairs): hacer la
95 // estructura plantilla,
96 // template<class T> struct CompresorT { vector<T> vals; ...
97 // };
98 // y usar el mismo sort + unique + lower_bound.
99 // 5. Si solo importa la IDENTIDAD (no el orden), basta un mapa
100 // hash y es O(n) esperado:
101 // unordered_map<long long, int> id;
102 // for (long long x : v) id.emplace(x, (int)id.size()); //
103 // no inserta si ya existe
104 // Con concursos que permiten ataques al hash, usar un hash
105 // con semilla aleatoria.
106 // 6. Se pierden las distancias y la adyacencia. Si el problema
107 // depende de que x y x+1
108 // sean vecinos (huecos, "siguiente entero libre"), agregar
109 // x-1, x y x+1 antes de
110 // comprimir. Si depende de la distancia (areas, longitudes),
111 // conservar vals y usar
112 // vals[i+1] - vals[i] como peso de la celda i.
113 // 7. Consultas en linea (no se conocen de antemano): no se

```

puede comprimir. Opciones:

```

96 // arbol de segmentos dinamico (nodos creados bajo demanda,
97 // O(log C) por operacion,
98 // con C el rango de valores) o un map / arbol balanceado.
99 // 8. Consultas ya ordenadas: cambiar lower_bound por un
100 // puntero que solo avanza;
101 // el costo baja de O(m log n) a O(n + m).

```

Ejemplo de funcionamiento:

```

1 int main() {
2     vector<long long> v = {100, 5, 100, 1000000000, 5, -7};
3     Compresor c(v);
4     cout << c.tam() << "\n"; // 4 (
5         // valores distintos: -7, 5, 100, 1000000000)
6
7     for (int x : c.comprimir(v)) cout << x << " ";
8     cout << "\n"; // 2 1 2 3 1
9         // 0
10
11     cout << c.indice(5) << " " << c.existe(6) << "\n"; // 1 0
12     cout << c.contar_menores(6) << "\n"; // 2 (-7 y 5
13         // son menores que 6)
14     cout << c.valor(3) << "\n"; // 1000000000
15
16     // Frecuencias sobre el universo comprimido (arreglo de 4
17     // posiciones, no de 10^9)
18     vector<int> cnt(c.tam(), 0);
19     for (long long x : v) cnt[c.indice(x)]++;
20     for (int x : cnt) cout << x << " ";
21     cout << "\n"; // 1 2 2 1
22
23     // Cuantos elementos de v estan en [0, 100]: celdas de
24     // contar_menores(0) a contar_menores_o_iguales(100)-1
25     int lo = c.contar_menores(0), hi = c.contar_menores_o_iguales
26     (100) - 1; // lo = 1, hi = 2
27     int total = 0;
28     for (int i = lo; i <= hi; i++) total += cnt[i];
29     cout << total << "\n"; // 4 (100,
30         // 5, 100, 5)
31
32     c.agregar_valores({0});
33     cout << c.tam() << " " << c.indice(5) << "\n"; // 5 2 (el
34         // rango de 5 cambio al agregar el 0)
35     return 0;
36 }

```

Nota: `lower_bound` exige un arreglo ordenado, y `unique` solo elimina duplicados *consecutivos*, así que el orden `sort` → `unique` → `erase` es obligatorio y no se puede alterar. Además, `indice` no falla cuando el valor no existe: devuelve el lugar donde se insertaría, y eso puede ser un rango válido de otro valor. Si la entrada no garantiza que x esté, comprueba con `existe` primero. Y como `agregar_valores` cambia los rangos (como muestra el último `cout` del ejemplo), hay que agregar todo lo necesario *antes* de comenzar a comprimir.

Regla práctica: Si los valores son grandes o dispersos, se conocen todos de antemano y el algoritmo solo compara, comprime y trabaja con arreglos de tamaño k . Si solo necesitas saber si dos valores son iguales, un mapa hash es más simple. Si los valores ya son pequeños (hasta unos 10^6), usa el arreglo directo. Si el problema usa distancias o adyacencia (intervalos, áreas), conserva `vals` y agrega los vecinos necesarios. Si las consultas son en línea, cambia a un árbol de segmentos dinámico o a un `map`.

52.2 Invariantes y Paridad

Idea: Un *invariante* es una propiedad del estado que toda operación permitida conserva. Si el estado inicial y el objetivo no comparten esa propiedad, el objetivo es inalcanzable, sin importar cuántas operaciones se apliquen ni en qué orden. La justificación es una inducción sobre el número de operaciones: la propiedad vale al inicio, cada paso la conserva, así que vale en todo estado alcanzable. El caso más frecuente es la *paridad*: una cantidad

módulo 2 que no cambia. Por ejemplo, si una operación reemplaza a y b por $|a - b|$, la suma cambia de $a + b$ a $|a - b|$, y ambas difieren en $2 \min(a, b)$, así que la paridad de la suma es invariante. Si reemplaza a y b por $a \oplus b$, el XOR total es invariante.

Otros invariantes clásicos son la suma módulo m , el mcd de todos los elementos (si la operación es $a \rightarrow a - b$), la coloración de un tablero (cada dominó cubre una casilla de cada color) y la paridad de una permutación. Esta última sale de contar ciclos: un intercambio de dos posiciones fusiona dos ciclos o parte uno en dos, así que cambia la cantidad de ciclos exactamente en ± 1 . La identidad de n elementos tiene n ciclos, así que la cantidad de intercambios necesarios para pasar de una permutación a otra siempre tiene la paridad de $n - \text{ciclos}$. Un invariante solo demuestra que algo es *imposible*; para demostrar que es posible hace falta una construcción, o un conjunto de invariantes que sea completo (como en el puzzle de 15, más abajo).

¿Por qué usarlo? La alternativa a un invariante es explorar los estados con BFS o retroceso, y el espacio de estados suele ser exponencial: inviable para n del orden de 10^5 o para valores grandes. El invariante responde «imposible» en $O(1)$ o $O(n)$ sin recorrer nada. Frente a la programación dinámica, que exige que el estado quepa en una tabla, el invariante no necesita estado alguno. Su costo es que no da la secuencia de operaciones y que hay que *descubrir* el invariante: no lo produce ningún algoritmo. Una estrategia práctica es escribir una fuerza bruta (BFS) para casos pequeños, agrupar los estados alcanzables entre sí y buscar qué propiedad reparte los estados en esos grupos (modificación 5). Se usa cuando el enunciado pide «¿es posible?», «imprime -1 si no se puede» o «¿quién gana?».

Casos de uso:

- Decidir si un estado inicial puede transformarse en otro con operaciones dadas (respuesta -1 o NO) mediante la paridad de la suma, el XOR, el mcd o la suma módulo m .
- Predecir el valor final tras repetir una operación sobre un multiconjunto (el último número que queda al reemplazar pares por su diferencia o su XOR).
- Cubrimientos de tableros con fichas: coloración para descartar dominós o L-triminós imposibles.
- Permutaciones: saber si el número mínimo de intercambios es par o impar, o si una configuración de un puzzle deslizante es resoluble.
- De nicho: monovariantes (una cantidad que solo crece o solo decrece) para probar que un proceso termina, o usar el invariante para guiar una construcción.

Complejidad: Construir la estructura recorre el arreglo una vez y calcula suma, XOR y cantidad de impares: $O(n)$. Actualizar un elemento con `cambiar` solo ajusta esos tres valores con el valor viejo y el nuevo, así que cuesta $O(1)$, y consultarlos también. El mcd total hace n llamadas a Euclides con $O(\log M)$ pasos cada una, con M el mayor valor: $O(n \log M)$. La paridad de una permutación recorre cada ciclo una vez y marca cada

elemento visitado, así que cada elemento se visita una sola vez: $O(n)$. `puzzle_resoluble` sobre un tablero $N \times N$ tiene $n = N^2 - 1$ fichas y usa esa rutina, por lo que cuesta $O(N^2)$; contar las inversiones directamente sería $O(N^4)$. Memoria: $O(n)$.

```

1 struct Invariantes {
2     vector<long long> a; // estado actual
3     int par_suma;        // paridad de la suma (se guarda solo el
4                          // bit para evitar desbordes)
5     long long xr;        // XOR de todos los elementos
6     int impares;         // cantidad de elementos impares
7
8     // CREATE: calcula los invariantes del estado inicial.
9     Invariantes(vector<long long> v) : a(move(v)), par_suma(0),
10    xr(0), impares(0) {
11         for (long long x : a) {
12             par_suma ^= (int)(x & 1);
13             xr ^= x;
14             impares += (int)(x & 1);
15         }
16     } // O(n)
17
18     // WRITE: cambia a[i] por x y actualiza los invariantes sin
19     // recorrer el arreglo.
20     void cambiar(int i, long long x) {
21         par_suma ^= (int)((a[i] ^ x) & 1);
22         xr ^= a[i] ^ x;
23         impares += (int)(x & 1) - (int)(a[i] & 1);
24         a[i] = x;
25     } // O(1)
26
27     // READ: paridad de la suma actual (0 = par, 1 = impar).
28     int paridad_suma() const {
29         return par_suma;
30     } // O(1)
31
32     // READ: XOR de todos los elementos.
33     long long xor_total() const {
34         return xr;
35     } // O(1)
36
37     // READ: cantidad de elementos impares.
38     int cantidad_impares() const {
39         return impares;
40     } // O(1)
41
42     // READ: mcd de todos los elementos (0 si todos son 0).
43     long long mcd_total() const {
44         long long g = 0;
45         for (long long x : a) g = __gcd(g, x < 0 ? -x : x);
46         return g;
47     } // O(n log M)
48
49     // READ: paridad de una permutacion de 0..n-1 (0 = par, 1 =
50     // impar).
51     // Un intercambio cambia la cantidad de ciclos en +-1; la
52     // identidad tiene n ciclos.
53     static int paridad_permutacion(const vector<int>& p) {
54         int n = (int)p.size(), ciclos = 0;
55         vector<char> vis(n, 0);
56         for (int i = 0; i < n; i++) {
57             if (vis[i]) continue;
58             ciclos++;
59             for (int j = i; !vis[j]; j = p[j]) vis[j] = 1;
60         }
61         return (n - ciclos) & 1;
62     } // O(n)
63
64     // READ: indica si un puzzle deslizante N x N (0 = hueco,
65     // fichas 1..N*N-1 leídas por filas)
66     // puede llegar al estado 1, 2, ..., N*N-1, 0 (hueco abajo a
67     // la derecha).
68     static bool puzzle_resoluble(const vector<int>& t, int N) {
69         vector<int> q;
70         int fila_hueco = 0;
71         for (int i = 0; i < N * N; i++) {
72             if (t[i] == 0) fila_hueco = i / N; // fila contada
73             // desde arriba, desde 0
74             else q.push_back(t[i] - 1);
75         }
76         int inv = paridad_permutacion(q); // paridad de
77         // inversiones = paridad de la permutacion
78         if (N % 2 == 1) return inv == 0;
79         return (inv + fila_hueco) % 2 == 1;
80     } // O(N^2)
81 };
82
83 // -----
84 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
85 // -----
86 // 1. Invariante modulo m: si una operacion suma o resta

```



```

// multiples de m a un elemento,
// su residuo modulo m no cambia. El estado b es alcanzable
// desde a solo si
// a[i] % m == b[i] % m para todo i (con ((x % m) + m) % m
// para los negativos).
// Es necesario, y suficiente si se puede sumar o restar m a
// cualquier elemento.
//
// 2. XOR: si la operacion reemplaza a y b por a ^ b, el XOR
// total es constante, y
// el ultimo elemento que queda es exactamente xor_total().
// O(1) por consulta.
//
// 3. Coloracion de tableros (cubrir con dominos): contar
// casillas libres por color.
// int blancas = 0, negras = 0;
// for (cada casilla libre (f, c)) ((f + c) & 1 ? negras :
// blancas)++;
// if (blancas != negras) imposible; // O(casillas)
// Es una condicion necesaria, no suficiente: hay tableros
// balanceados sin cubrimiento.
// Ej. clasico: el 8x8 sin dos esquinas opuestas (del mismo
// color) tiene 32 y 30.
//
// 4. Monovariante (una cantidad que solo crece o solo decrece):
// si cada operacion
// reduce estrictamente un entero acotado inferiormente (
// inversiones, suma de
// cuadrados, potencial), el proceso termina en a lo sumo (
// valor inicial - cota)
// pasos, aunque no se sepa cual es el estado final.
//
// 5. Descubrir el invariante con fuerza bruta: para casos
// pequenos, hacer BFS desde
// cada estado, agrupar los estados alcanzables entre si y
// comparar clases con
// candidatos (suma, XOR, mcd, paridad de la permutacion,
// cantidad de impares).
// Si una propiedad separa exactamente las clases, es el
// invariante.
// set<Estado> visto; queue<Estado> q; // por cada estado
// sin clase: BFS y asignar id
// // O(#estados * #operaciones); solo para explorar, no
// para la solucion final.
//
// 6. Cantidad minima de intercambios para ordenar una
// permutacion: n - ciclos (con
// intercambios cualesquiera) y #inversiones (si solo se
// intercambian vecinos).
// En ambos casos la paridad coincide con
// paridad_permutacion().
//
// 7. mcd como invariante: las operaciones a -> a - b (o a -> a
// + b) sobre elementos
// del arreglo no cambian mcd_total(). El estado final solo
// puede contener
// multiples de ese mcd; si b no es multiplo, es
// inalcanzable.
//
// 8. Juegos: si cada jugada cambia la paridad de la cantidad
// total de piezas, la
// paridad inicial decide quien hace la ultima jugada. Se
// comprueba con una
// fuerza bruta pequena antes de confiar en la conjetura.

```

Ejemplo de funcionamiento:

```

int main() {
    Invariantes inv({1, 2, 3, 4});
    cout << inv.paridad_suma() << " " << inv.xor_total() << " "
    << inv.cantidad_impares() << "\n"; // 0 4 2
    inv.cambiar(0, 5); // {5, 2, 3, 4}
    cout << inv.paridad_suma() << " " << inv.xor_total() << " "
    << inv.cantidad_impares() << "\n"; // 0 0 2
    inv.cambiar(1, 7); // {5, 7, 3, 4}
    cout << inv.paridad_suma() << " " << inv.xor_total() << " "
    << inv.cantidad_impares() << "\n"; // 1 5 3

    // Operacion: quitar dos numeros x >= y y poner x - y. La
    // paridad de la suma es invariante.
    multiset<long long> ms = {1, 2, 3, 4, 5};
    cout << Invariantes({1, 2, 3, 4, 5}).paridad_suma() << "\n";
    // 1 (suma 15, impar)
    while (ms.size() > 1) {
        auto it = prev(ms.end()); long long x = *it; ms.erase(it);
        it = prev(ms.end()); long long y = *it; ms.erase(it);
        ms.insert(x - y);
    }
    cout << *ms.begin() << "\n"; // 1 (impar
    // como predijo el invariante)

    cout << Invariantes({12, 18, 30}).mcd_total() << "\n"; // 6
}

```

```

cout << Invariantes::paridad_permutacion({1, 0, 2}) << "\n";
// 1 (un intercambio)
cout << Invariantes::paridad_permutacion({1, 2, 0}) << "\n";
// 0 (ciclo de 3 = 2 intercambios)

cout << Invariantes::puzzle_resoluble({1,2,3,4,5,6,7,0,8}, 3)
<< "\n"; // 1 (un movimiento desde la meta)
cout << Invariantes::puzzle_resoluble({1,2,3,4,5,6,8,7,0}, 3)
<< "\n"; // 0 (dos fichas intercambiadas)
cout << Invariantes::puzzle_resoluble
({1,2,3,4,5,6,7,8,9,10,11,12,13,15,14,0}, 4) << "\n"; // 0
// (el clasico 14-15)
cout << Invariantes::puzzle_resoluble
({1,2,3,4,5,6,7,8,9,10,11,0,13,14,15,12}, 4) << "\n"; // 1
// (hueco un piso arriba)
return 0;
}

```

Nota: La regla del puzzle sale de estudiar cómo cambia la paridad de las inversiones. Un movimiento horizontal no altera el orden de lectura de las fichas, así que las inversiones no cambian. Un movimiento vertical hace pasar una ficha por encima de $N - 1$ fichas, invirtiendo $N - 1$ pares, así que la paridad de las inversiones cambia si y solo si N es par. Con N impar, la paridad de las inversiones es invariante y la meta tiene 0 inversiones, de ahí $\text{inv} == 0$. Con N par, cada movimiento vertical también cambia la fila del hueco en 1, así que $\text{inv} + \text{fila_hueco}$ mantiene su paridad, y la meta (con 0 inversiones y el hueco en la fila $N - 1$, impar) da paridad impar. Ese criterio supone la meta con el hueco abajo a la derecha. Para otra meta hay que compararla con la configuración dada usando la paridad de la permutación que lleva una a la otra. Además, `paridad_suma` guarda solo el bit menos significativo a propósito, para no desbordar con sumas grandes; si necesitas la suma completa, guárdala aparte en `long long` o `__int128`.

Regla práctica: Si el problema pregunta si algo es posible tras aplicar operaciones repetidas y el espacio de estados es enorme, busca primero un invariante (paridad, XOR, suma módulo m , mcd, coloración) que separe el estado inicial del objetivo. Usa BFS solo para descubrir el invariante en casos pequeños, no como solución final. Recuerda que un invariante prueba imposibilidad, no posibilidad: si coinciden inicio y objetivo, todavía necesitas una construcción o un conjunto de invariantes completo (como en el puzzle de 15). Si el enunciado pide además la secuencia de operaciones, el invariante te da la condición de existencia pero no la secuencia: construye una y usa el invariante para verificarla.

52.3 Algoritmos Constructivos (con Cuadrado Mágico)

Idea: Un algoritmo constructivo no *busca* la respuesta: la *fabrica* con una regla explícita cuyo resultado cumple, por construcción, las propiedades pedidas. El enunciado típico dice «construye una permutación, matriz, grafo o cadena tal que...» (o «imprime -1 si no existe»), y la solución es una receta que trabaja en tiempo proporcional al tamaño de la salida, más una demostración de que la receta funciona. Esa demostración suele ser una inducción (armar el caso n a partir del $n - 1$ o del $n/2$), un invariante que la receta mantiene, o una fórmula cerrada que se verifica con álgebra. Lo difícil es *descubrir* la regla: lo habitual es resolver a mano (o con fuerza bruta) los casos pequeños, mirar las soluciones y buscar el patrón.

El ejemplo canónico es el *cuadrado mágico* de orden n : una matriz $n \times n$ que contiene cada entero de 1 a n^2 exactamente una vez, con todas las filas, todas las columnas y las dos diagonales sumando lo mismo. Como los n^2 números suman $n^2(n^2 + 1)/2$ y se reparten en n filas, la constante debe ser $S = n(n^2 + 1)/2$ (15 para $n = 3$, 34 para $n = 4$). Existe para todo $n \neq 2$ (en 2×2 , $a + b = a + c$ obliga a $b = c$), y la construcción depende de $n \bmod 4$:

- **n impar (método siamés).** Se coloca el 1 en el centro de la fila superior; cada número siguiente va una fila arriba y una columna a la derecha (con vuelta al otro lado), y si esa casilla está ocupada se baja una fila desde la casilla actual. Sea $m = n$ y $h = (m - 1)/2$. Si se escribe $k - 1 = qm + s$ con $0 \leq s < m$, el número k cae en la fila $2q - s$ y la columna $h - q + s$ (módulo m), o equivalentemente $q \equiv f + c - h$ y $s \equiv f + 2c + 1$ para la casilla (f, c) . En una fila (o columna) fija, al recorrer la otra coordenada tanto q como s toman cada residuo exactamente una vez, porque 2 es invertible módulo un m impar. La suma es entonces $m \cdot \frac{m(m-1)}{2} + \frac{m(m-1)}{2} + m = \frac{m(m^2+1)}{2}$. Las diagonales se comprueban con aritmética modular análoga (algo más fina cuando $3 \mid m$).
- **$n = 4t$.** Se escribe $1, \dots, n^2$ por filas y se sustituye cada valor v por $n^2 + 1 - v$ en las casillas con $i \bmod 4 = j \bmod 4$ o $(i \bmod 4) + (j \bmod 4) = 3$ (las diagonales de cada bloque de 4×4). Como $v = ni + j + 1$, complementar equivale a reflejar $(i, j) \mapsto (n - 1 - i, n - 1 - j)$. Cada fila tiene $n/2$ casillas marcadas, y sus columnas son simétricas bajo $j \mapsto n - 1 - j$; por eso la suma de las partes «fila» y «columna» de todos los valores vuelve a ser $n \cdot \frac{n(n-1)}{2} + \frac{n(n-1)}{2} + n = S$. Con columnas y diagonales el argumento es análogo.
- **$n = 4t + 2$ (método de Strachey).** Sea $m = n/2$ (impar) y A el cuadrado siamés de orden m . Se ponen cuatro cuadrantes: A arriba a la izquierda, $A + m^2$ abajo a la derecha, $A + 2m^2$ arriba a la derecha y $A + 3m^2$ abajo a la izquierda. Cada cuadrante es mágico, pero las filas de arriba suman m^3 menos que S y las de abajo m^3 más. Se intercambian (entre arriba y abajo) $k = (m - 1)/2$ casillas por fila en la izquierda, cada una suma $3m^2$ a la fila de arriba, y $k - 1$ en la derecha, cada una resta m^2 . El efecto neto es $3km^2 - (k - 1)m^2 = (2k + 1)m^2 = m^3$, justo lo que faltaba. En la fila central de la izquierda el intercambio se desplaza una columna a la derecha, para que columnas y diagonales también cuadren.

¿Por qué usarlo? Buscar un cuadrado mágico por retroceso es hopeless incluso para n modestos: hay $(n^2)!$ formas de ubicar los números y las podas no bastan. La construcción da uno en $O(n^2)$. Frente a la programación dinámica o al voraz, que *optimizan* una cantidad, un algoritmo constructivo *satisface* restricciones estructurales: no hay función objetivo. Es complementario a la subsección anterior: los invariantes y la paridad demuestran que algo es *imposible* (o eliminan casos), y la construcción demuestra que es *posible* al exhibirlo; juntos

caracterizan cuándo existe la respuesta. Su costo es que no hay un procedimiento general: cada problema exige encontrar su patrón, y la solución no es única, así que en un juez con verificador cualquier salida válida sirve, pero al depurar conviene comparar contra un verificador propio y no contra una salida fija.

Casos de uso:

- Problemas de «construye una permutación, matriz o cadena con tal propiedad» (con impresión de -1 si no existe).
- Cuadrados mágicos, cuadrados latinos y otras matrices con sumas o alfabetos controlados por fila y columna.
- Particionar $1, \dots, n$ en dos conjuntos de igual suma, o emparejar elementos con sumas objetivo.
- Recorridos y códigos con adyacencia garantizada: código de Gray, recorrido en serpiente de una cuadrícula.
- De nicho: calendarios de torneos (*round-robin*), secuencias de De Bruijn, grafos con diámetro o grado prescritos.

Complejidad: La salida tiene n^2 casillas y el método las escribe un número constante de veces cada una, así que construir cuesta $O(n^2)$, óptimo porque solo imprimir ya cuesta eso. En el caso impar hay n^2 colocaciones y cada una hace $O(1)$ trabajo (una suma, un módulo y una comprobación de casilla ocupada). En $n = 4t$ se visita cada casilla una vez con $O(1)$ trabajo. En $n = 4t + 2$ se construye el cuadrado impar de orden $m = n/2$ ($O(m^2)$), se copian los cuatro cuadrantes ($4m^2 = n^2$ escrituras) y se hacen a lo sumo $m \cdot (2k - 1) = O(n^2)$ intercambios. Verificar con `es_magico` es $O(n^2)$ (cada casilla se lee un número constante de veces). Memoria: $O(n^2)$.

```

1 struct CuadradoMagico {
2     int n; // orden del cuadrado
3     vector<vector<int>> c; // c[i][j]: valor en la fila i,
        columna j
4
5     // READ: indica si existe un cuadrado magico normal de orden
        n (falla n <= 0 y n = 2).
6     static bool existe(int n) {
7         return n == 1 || n >= 3;
8     } // O(1)
9
10    // CREATE: cuadrado de orden impar m por el metodo siames.
11    static vector<vector<int>> impar(int m) {
12        vector<vector<int>> a(m, vector<int>(m, 0));
13        int f = 0, col = m / 2; // el 1 va
            al centro de la fila superior
14        for (int k = 1; k <= m * m; k++) {
15            a[f][col] = k;
16            int nf = (f - 1 + m) % m, nc = (col + 1) % m; // arriba
                y a la derecha
17            if (a[nf][nc]) { nf = (f + 1) % m; nc = col; } //
                ocupada: bajar desde la actual
18            f = nf; col = nc;
19        }
20        return a;
21    } // O(m^2)
22
23    // CREATE: cuadrado de orden n multiplo de 4 (complementa las
        diagonales de cada bloque 4x4).
24    static vector<vector<int>> doble_par(int n) {
25        vector<vector<int>> a(n, vector<int>(n));
26        for (int i = 0; i < n; i++)
27            for (int j = 0; j < n; j++) {
28                int v = i * n + j + 1;
29                bool marcada = (i % 4 == j % 4) || (i % 4 + j % 4 == 3);
30                a[i][j] = marcada ? n * n + 1 - v : v;
31            }
32        return a;

```

```

33     } // O(n^2)
34
35 // CREATE: cuadrado de orden n = 4t + 2 (n >= 6) por el
36 // metodo de Strachey.
37 static vector<vector<int>> simple_par(int n) {
38     int m = n / 2, k = (m - 1) / 2;
39     vector<vector<int>> b = impar(m), a(n, vector<int>(n));
40     for (int i = 0; i < m; i++)
41         for (int j = 0; j < m; j++) {
42             a[i][j] = b[i][j]; // arriba-
43             // izquierda
44             a[i + m][j + m] = b[i][j] + m * m; // abajo-
45             // derecha
46             a[i][j + m] = b[i][j] + 2 * m * m; // arriba-
47             // derecha
48             a[i + m][j] = b[i][j] + 3 * m * m; // abajo-
49             // izquierda
50         }
51     for (int i = 0; i < m; i++) {
52         for (int j = 0; j < k; j++) { // k
53             // columnas a la izquierda
54             int jj = (i == m / 2) ? j + 1 : j; // en la
55             // fila central se corre una columna
56             swap(a[i][jj], a[i + m][jj]);
57         }
58         for (int j = 0; j < k - 1; j++) // k-1
59             // columnas a la derecha
60             swap(a[i][n - 1 - j], a[i + m][n - 1 - j]);
61     }
62     return a;
63 } // O(n^2)
64
65 // CREATE: construye un cuadrado magico normal de orden n (
66 // elige el metodo segun n % 4).
67 CuadradoMagico(int orden) : n(orden) {
68     if (!existe(n)) throw invalid_argument("no existe cuadrado
69     magico de ese orden");
70     if (n % 2 == 1) c = impar(n);
71     else if (n % 4 == 0) c = doble_par(n);
72     else c = simple_par(n);
73 } // O(n^2)
74
75 // READ: constante magica S = n (n^2 + 1) / 2.
76 long long constante() const {
77     return (long long)n * ((long long)n * n + 1) / 2;
78 } // O(1)
79
80 // READ: verifica que sea magico: cada numero 1..n^2 una vez
81 // y todas las sumas iguales a S.
82 bool es_magico() const {
83     long long S = constante(), d1 = 0, d2 = 0;
84     vector<char> visto(n * n + 1, 0);
85     for (int i = 0; i < n; i++) {
86         long long fila = 0, col = 0;
87         for (int j = 0; j < n; j++) {
88             int v = c[i][j];
89             if (v < 1 || v > n * n || visto[v]) return false;
90             visto[v] = 1;
91             fila += v;
92             col += c[j][i];
93         }
94         if (fila != S || col != S) return false;
95         d1 += c[i][i];
96         d2 += c[i][n - 1 - i];
97     }
98     return d1 == S && d2 == S;
99 } // O(n^2)
100
101 // WRITE: gira el cuadrado 90 grados en sentido horario (
102 // sigue siendo magico).
103 void rotar90() {
104     vector<vector<int>> r(n, vector<int>(n));
105     for (int i = 0; i < n; i++)
106         for (int j = 0; j < n; j++) r[j][n - 1 - i] = c[i][j];
107     c = move(r);
108 } // O(n^2)
109
110 };
111
112 // -----
113 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
114 // -----
115 // 1. Simetrias y desplazamientos: rotar, reflejar o transponer
116 // un cuadrado magico
117 // da otro cuadrado magico (ocho variantes en total).
118 // Complementar (v -> n*n + 1 - v)
119 // tambien. Sumar una constante t a todas las casillas da
120 // numeros t+1..t+n*n con
121 // suma magica S + n*t. Sirve cuando el enunciado pide otro
122 // rango o cualquiera de
123 // las variantes.
124 //
125 // 2. Formula cerrada para n impar (sin simular): el valor de
126 // la casilla (i, j) es

```

```

109 // int valor_impar(int i, int j, int m) {
110 //     int h = (m - 1) / 2;
111 //     return m * (((i + j - h) % m + m) % m) + (i + 2 * j
112 //     + 1) % m + 1;
113 // } // O(1) por casilla
114 // Util para consultar una casilla sin construir toda la
115 // matriz.
116 // 3. Cuadrado latino ciclico: a[i][j] = (i + j) % n. Cada
117 // simbolo aparece una vez
118 // por fila y por columna; es la base de calendarios round-
119 // robin y de
120 // construcciones de matrices con simbolos no repetidos.
121 // O(n^2)
122 //
123 // 4. Partir 1..n en dos conjuntos con la misma suma (posible
124 // solo si n % 4 es 0 o 3,
125 // porque la suma total n(n+1)/2 debe ser par):
126 // vector<int> A, B; int t = 1;
127 // if (n % 4 == 3) { A = {1, 2}; B = {3}; t = 4; }
128 // for (; t + 3 <= n; t += 4) { // t + (t+3) == (
129 //     t+1) + (t+2)
130 //         A.push_back(t); A.push_back(t + 3);
131 //         B.push_back(t + 1); B.push_back(t + 2);
132 //     } // O(n)
133 //
134 // 5. Codigo de Gray: los n-bits 0..2^n-1 en un orden donde dos
135 // consecutivos (y el
136 // ultimo con el primero) difieren en un solo bit.
137 // for (int i = 0; i < (1 << n); i++) g[i] = i ^ (i >> 1);
138 // O(2^n)
139 //
140 // 6. Recorrido en serpiente de una cuadrícula f x c: visita
141 // todas las casillas y
142 // cada paso (incluido el salto entre filas) va a una
143 // casilla adyacente.
144 // for (int i = 0; i < f; i++)
145 //     for (int j = 0; j < c; j++) visitar(i, (i % 2 == 0) ?
146 //     j : c - 1 - j); // O(f*c)
147 //
148 // 7. Construcción por inducción (de n-1 a n, o de n/2 a n):
149 // resolver el caso base y
150 // extender. Plantilla:
151 // vector<int> construir(int n) {
152 //     if (n == 1) return {1};
153 //     vector<int> s = construir(n - 1);
154 //     // ... insertar el elemento n en la posición que
155 //     conserva la propiedad ...
156 //     return s;
157 // }
158 // Ejemplo simple: desorden (permutación sin puntos fijos)
159 // para n >= 2: p[i] = (i + 1) % n.
160 //
161 // 8. Descubrir el patrón: escribir una fuerza bruta (retroceso
162 // ) para n pequeños,
163 // imprimir todas las soluciones, buscar la regla y
164 // comprobarla con un
165 // verificador (como es_magico) sobre muchos n antes de
166 // enviar. Empezar siempre
167 // por decidir la existencia (invariantes, paridad): si no
168 // existe, no hay que construir.
169 //
170 // 9. Otros métodos para n = 4t + 2: LUX de Conway (usa tres
171 // tipos de losetas L, U, X
172 // sobre un cuadrado impar de tamaño (n+2)/4 ... o similar);
173 // Strachey (el de arriba) es
174 // el más corto de programar.

```

Ejemplo de funcionamiento:

```

1 void imprimir(const CuadradoMagico& m) {
2     for (const auto& fila : m.c) {
3         for (int x : fila) cout << x << " ";
4         cout << "\n";
5     }
6 }
7
8 int main() {
9     CuadradoMagico a(3);
10    imprimir(a);
11    // 8 1 6
12    // 3 5 7
13    // 4 9 2
14    cout << a.constante() << " " << a.es_magico() << "\n"; //
15    15 1
16    a.rotar90();
17    imprimir(a);
18    // 4 3 8
19    // 9 5 1
20    // 2 7 6
21    cout << a.es_magico() << "\n"; // 1
    (rotar conserva la propiedad)

```

```

22 CuadradoMagico b(4);
23 imprimir(b);
24 // 16 2 3 13
25 // 5 11 10 8
26 // 9 7 6 12
27 // 4 14 15 1
28 cout << b.constante() << " " << b.es_magico() << "\n"; //
    34 1
29
30 CuadradoMagico e(5);
31 for (int x : e.c[0]) cout << x << " ";
32 cout << "\n"; //
    17 24 1 8 15
33 cout << e.constante() << " " << e.es_magico() << "\n"; //
    65 1
34
35 CuadradoMagico d(6);
36 imprimir(d);
37 // 35 1 6 26 19 24
38 // 3 32 7 21 23 25
39 // 31 9 2 22 27 20
40 // 8 28 33 17 10 15
41 // 30 5 34 12 14 16
42 // 4 36 29 13 18 11
43 cout << d.constante() << " " << d.es_magico() << "\n"; //
    111 1
44
45 CuadradoMagico g(8);
46 for (int x : g.c[0]) cout << x << " ";
47 cout << "\n"; //
    64 2 3 61 60 6 7 57
48 cout << g.constante() << " " << g.es_magico() << "\n"; //
    260 1
49
50 CuadradoMagico h(10);
51 cout << h.constante() << " " << h.es_magico() << "\n"; //
    505 1
52 cout << CuadradoMagico::existe(2) << "\n"; // 0
    (no existe el 2x2)
53 return 0;
54 }

```

Nota: El cuadrado que se obtiene no es el único de su orden: el método siamés, por ejemplo, tiene variantes según la casilla de partida y la dirección, y las rotaciones y reflexiones dan otros. Si el problema exige una salida concreta (y no un verificador), hay que ajustar esas elecciones al enunciado. El constructor lanza una excepción para $n = 2$ y $n \leq 0$, y el método de Strachey solo se llama con $n \geq 6$ (para $n = 2$ daría $k = 0$ y un índice inválido). Además, `impar` usa 0 como marca de casilla vacía, lo cual es válido porque los valores empiezan en 1. Para n grande, el límite práctico es la memoria de n^2 enteros, no el tiempo.

Regla práctica: Si el problema pide construir un objeto con propiedades de suma, orden o adyacencia, primero decide con invariantes si existe; luego resuelve los casos pequeños con fuerza bruta, busca el patrón, y demuestra por inducción o con una fórmula. Para cuadrados mágicos usa tres casos según $n \bmod 4$: siamés para impar, complemento de diagonales de bloques para múltiplos de 4 y Strachey para $4t + 2$. Comprueba siempre la construcción con un verificador (`es_magico`) contra muchos n antes de confiar en ella. Si lo que se busca es la mejor solución entre varias posibles, un algoritmo constructivo no basta: usa voraz o programación dinámica.

52.4 Trabajar hacia Atrás (Reverse Thinking)

Idea: Trabajar hacia atrás significa empezar en el estado final y *deshacer* las operaciones, en lugar de simular desde el estado inicial. Ayuda por tres razones estructurales. La primera es la *asimetría de ramificación*: si un estado tiene muchos sucesores pero uno solo (o muy pocos) predecesores, la búsqueda hacia adelante explora un árbol exponencial y hacia atrás recorre una línea. La segunda

es el *orden temporal*: si la última operación sobre un dato anula a las anteriores («la última escritura gana»), recorrer las operaciones desde la última permite decidir cada dato la primera vez que se toca y descartar el resto. La tercera es que el *objetivo suele ser más restrictivo que el inicio*: en un juego, las posiciones terminales son conocidas y de ellas se propaga hacia atrás quién gana.

El ejemplo más limpio: se parte de un par (x, y) de enteros positivos y en cada paso se puede pasar a $(x+y, y)$ o a $(x, x+y)$; ¿se puede llegar de (a, b) a (c, d) ? Hacia adelante hay dos opciones por paso, es decir, 2^k estados tras k pasos. Hacia atrás no hay elección: si $c > d$, la última operación tuvo que ser $(x, y) \rightarrow (x+y, y)$, porque la otra operación dejaría la primera coordenada igual a la anterior y $c > d$ obliga a que la primera sea la grande; así que el predecesor es único, $(c-d, d)$. Si $c < d$ es simétrico, y si $c = d$ no tiene predecesor válido (habría que restar y llegar a 0). El camino de regreso es único: es el algoritmo de Euclides por restas, y se acelera dividiendo en lugar de restar de a uno.

El segundo ejemplo es «la última escritura gana»: se aplican m operaciones `pintar( $l, r, \text{color}$ )` sobre n celdas y se pregunta el color final. Recorridas al revés, cada celda se pinta una sola vez (la primera que se alcanza al ir hacia atrás es la última en el tiempo real), y para saltar las celdas ya pintadas basta una estructura de «siguiente celda libre» con compresión de caminos.

¿Por qué usarlo? Frente a BFS o programación dinámica hacia adelante, el enfoque hacia atrás vale cuando el factor de ramificación hacia atrás es menor que hacia adelante (aquí 1 contra 2). Si es igual o mayor, no ayuda, y si ambos son grandes conviene la búsqueda bidireccional (modificación 4). Frente a los invariantes de la subsección anterior, que dicen solo «posible» o «imposible» (aquí, el `mcd`), ir hacia atrás además entrega la cantidad exacta de pasos y el camino. Frente a un árbol de segmentos con propagación perezosa para asignaciones de rango, pintar en reversa es más simple y casi lineal, pero solo funciona *fuera de línea*: hay que conocer todas las operaciones y no admite consultas intermedias. Y frente a los algoritmos constructivos, es a menudo su herramienta: para construir un camino se parte del final y se retrocede con la regla determinista.

Casos de uso:

- Alcanzabilidad y número de pasos entre estados cuando cada operación tiene un inverso determinista (pares con sumas, algoritmos tipo Euclides).
- Estado final tras muchas asignaciones de rango (pintar, colorear tramos, «la última escritura gana»), con todas las operaciones conocidas.
- Conectividad fuera de línea con eliminaciones de aristas: se procesa el tiempo al revés y las eliminaciones se vuelven inserciones (unión-búsqueda).
- Juegos: análisis retrógrado desde las posiciones terminales para saber quién gana.
- De nicho: deshacer una secuencia conocida de intercambios, DP con estados finales conocidos, o BFS bidireccional.

Complejidad: Para pasos, cada iteración del ciclo es un paso de división de Euclides: reduce una coordenada a un valor en $[1, otra]$, y las dos coordenadas se alternan. Como en Euclides (peor caso en números de Fibonacci consecutivos, Lamé), cada dos iteraciones el mayor se reduce al menos a la mitad, así que hay $O(\log M)$ iteraciones con $M = \max(c, d)$, y cada una hace un número fijo de divisiones y restas: $O(\log M)$, con memoria $O(1)$ (contra $O(2^k)$ o $O(M)$ de la búsqueda hacia adelante). Para PinturaInversa, el constructor cuesta $O(n)$. Cada celda se pinta a lo sumo una vez en toda la ejecución, y cada llamada a pintar hace una búsqueda inicial más una por celda pintada, es decir $O(n + m)$ búsquedas en total. Con solo compresión de caminos cada búsqueda cuesta $O(\log n)$ amortizado en el peor caso teórico, y en la práctica casi constante. El total es $O((n + m) \log n)$ (casi lineal en la práctica), contra $O(n \cdot m)$ de pintar celda por celda hacia adelante. Memoria: $O(n)$.

```

1 // Pares (x, y) con operaciones (x, y) -> (x + y, y) o (x, x +
2 // y), todos positivos.
3 struct ParAtras {
4 // READ: cantidad de pasos para ir de (a, b) a (c, d), o -1
5 // si no es posible.
6 // Hacia atras el predecesor es unico, asi que esa cantidad
7 // es exacta (y minima).
8 // Requiere a, b, c, d >= 1.
9 static long long pasos(long long a, long long b, long long c,
10 long long d) {
11 long long total = 0;
12 while (true) {
13 if (c < a || d < b) return -1; // las
14 // coordenadas solo crecen
15 if (c == a && d == b) return total;
16 if (c == d) return -1; // (x, x) no
17 // tiene predecesor positivo
18 if (c > d) {
19 if (d == b && (c - a) % d == 0) // llegamos a (
20 // a, b) restando d varias veces
21 return total + (c - a) / d;
22 long long k = (c - 1) / d; // restas
23 // posibles manteniendo c >= 1
24 c -= k * d; total += k;
25 } else {
26 if (c == a && (d - b) % c == 0)
27 return total + (d - b) / c;
28 long long k = (d - 1) / c;
29 d -= k * c; total += k;
30 }
31 }
32 } // O(log M)
33
34 // READ: indica si (c, d) es alcanzable desde (a, b).
35 static bool alcanzable(long long a, long long b, long long c,
36 long long d) {
37 return pasos(a, b, c, d) >= 0;
38 } // O(log M)
39
40 };
41
42 // Estado final tras m operaciones "pintar [l, r] con color"
43 // aplicadas en orden.
44 struct PinturaInversa {
45 int n;
46 vector<int> sig; // sig[i]: primera celda >= i sin pintar
47 // (n es el centinela)
48 vector<int> color; // color final de cada celda (0 = nunca
49 // pintada)
50
51 // CREATE: n celdas sin pintar.
52 PinturaInversa(int tam) : n(tam), sig(tam + 1), color(tam, 0)
53 {
54 iota(sig.begin(), sig.end(), 0);
55 } // O(n)
56
57 // READ: primera celda sin pintar en [x, n] (n si no queda
58 // ninguna).
59 // Comprime caminos (cambia sig, pero no el resultado).
60 int buscar(int x) {
61 int r = x;
62 while (sig[r] != r) r = sig[r];
63 while (sig[x] != r) { int nx = sig[x]; sig[x] = r; x = nx;
64 }
65 return r;
66 } // O(log n) amortizado

```

```

52 // WRITE: pinta con c las celdas todavia libres de [l, r] (
53 // inclusive).
54 void pintar(int l, int r, int c) {
55 for (int i = buscar(l); i <= r; i = buscar(i)) {
56 color[i] = c;
57 sig[i] = i + 1; // la celda i
58 // ya no esta libre
59 }
60 } // O((1 + k) log n) amortizado, k = celdas pintadas por
61 // esta llamada
62
63 // WRITE: aplica las operaciones (l, r, c) recorriendolas de
64 // la ultima a la primera.
65 void aplicar_en_reversa(const vector<array<int, 3>&& ops) {
66 for (int i = (int)ops.size() - 1; i >= 0; i--)
67 pintar(ops[i][0], ops[i][1], ops[i][2]);
68 } // O((n + m) log n)
69
70 };
71
72 // -----
73 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
74 // -----
75
76 // 1. Eliminaciones -> inserciones (conectividad fuera de linea
77 // ). Consultas: "cuantas
78 // componentes hay tras eliminar las primeras i aristas e[1..
79 // i]?", i = 0..k.
80 // Se construye el estado FINAL y se retrocede en el tiempo
81 // agregando aristas:
82 // // DSU d(n); unir todas las aristas que nunca se eliminan;
83 // // resp[k] = d.componentes();
84 // // for (int i = k; i >= 1; i--) { d.unir(e[i]); resp[i-1]
85 // // = d.componentes(); }
86 // // O((n + m + k) alfa(n)). Hacia adelante haria falta
87 // // conectividad dinamica,
88 // // // mucho mas dificil: la union-busqueda no sabe deshacer
89 // // una arista.
90
91 // 2. Analisis retrogrado en juegos (posiciones y jugadas
92 // forman un grafo dirigido).
93 // // estado[v]: 0 = sin resolver, 1 = gana quien mueve, 2 =
94 // // pierde quien mueve.
95 // // grado[v] = cantidad de jugadas desde v. Cola con todas
96 // // las terminales
97 // // (grado 0: pierde quien mueve, estado = 2).
98 // // while (!q.empty()) {
99 // // int v = q.front(); q.pop();
100 // // for (int u : predecesores[v]) { // u
101 // // // puede saltar a v
102 // // // if (estado[u]) continue;
103 // // // if (estado[u] == 2) { estado[u] = 1; q.
104 // // // push(u); } // salta a una perdedora
105 // // // else if (--grado[u] == 0) { estado[u] = 2;
106 // // // q.push(u); } // todas sus jugadas dan a ganadoras
107 // // // }
108 // // } // O(V + E). Lo que queda sin resolver es empate
109 // // (ciclos).
110
111 // 3. Deshacer transformaciones conocidas: si a partir de un
112 // arreglo inicial se
113 // aplicaron k intercambios (x[i], y[i]) en orden y se
114 // conoce el resultado a,
115 // recuperar el inicial es aplicar los intercambios en orden
116 // inverso:
117 // for (int i = k - 1; i >= 0; i--) swap(a[x[i]], a[y[i]]);
118 // // O(k)
119 // // En general, el inverso de una composicion es la
120 // // composicion de los inversos al reves.
121
122 // 4. BFS bidireccional: cuando ambos extremos (inicio y
123 // objetivo) son conocidos y hay
124 // predecesores computables, buscar desde los dos lados a la
125 // vez y detenerse al
126 // encontrarse. Con ramificacion b y distancia d, baja el
127 // costo de O(b^d) a
128 // O(2 * b^(d/2)).
129
130 // 5. DP hacia atras: definir dp[estado] = mejor resultado
131 // desde ese estado hasta el
132 // final y llenarla desde los estados finales. Conviene
133 // cuando hay muchos estados
134 // iniciales o cuando el inicial no se conoce, porque una
135 // sola pasada responde
136 // para todos.
137
138 // 6. Reconstruir el camino: hacia atras se obtienen los
139 // estados de fin a inicio;
140 // guardarlos en un vector y darlo vuelta (reverse) antes de
141 // imprimir.
142 // OJO: si el camino es enorme (como en pasos(1, 1, 1e9, 1)),
143 // no se puede guardar
144 // completo; en ese caso se imprime comprimido (repeticiones
145 // de la misma operacion).

```

```

112 //
113 // 7. Cuando el predecesor NO es unico (por ejemplo x -> x mod
114 // k): hacia atras tambien
115 // hay ramificacion. Comparar cuantos sucesores y cuantos
116 // predecesores tiene un
117 // estado típico y elegir el sentido con menos; si son
118 // parecidos, bidireccional.
119 //
120 // 8. Pregunta guia: "cual fue el ultimo paso?" o "que tiene
121 // que ser cierto justo antes
122 // del final?". Si la respuesta es unica o casi unica, hay
123 // un algoritmo hacia atras.

```

Ejemplo de funcionamiento:

```

1 int main() {
2     cout << ParAtras::pasos(1, 1, 3, 5) << "\n"; // 3
3     (1,1) -> (1,2) -> (3,2) -> (3,5)
4     cout << ParAtras::pasos(1, 1, 8, 5) << "\n"; // 4
5     (1,1) -> (1,2) -> (3,2) -> (3,5) -> (8,5)
6     cout << ParAtras::pasos(2, 3, 12, 5) << "\n"; // 3
7     (2,3) -> (2,5) -> (7,5) -> (12,5)
8     cout << ParAtras::pasos(1, 1, 1000000000, 1) << "\n"; //
9     999999999 (una sola vuelta del ciclo)
10    cout << ParAtras::pasos(1, 1, 2, 4) << "\n"; // -1
11    (llega a (2,2), sin predecesor; mcd(2,4) = 2)
12    cout << ParAtras::pasos(5, 1, 3, 4) << "\n"; // -1
13    (3 < 5: las coordenadas solo crecen)
14    cout << ParAtras::alcanzable(1, 1, 8, 5) << "\n"; // 1
15
16    PinturaInversa p(8); //
17    celdas 0..7
18    p.aplicar_en_reversa({{0, 4, 1}, {2, 6, 2}, {3, 3, 3}});
19    for (int x : p.color) cout << x << " ";
20    cout << "\n"; // 1
21    1 2 3 2 2 2 0
22    // Hacia adelante: 1 1 1 1 1 0 0 0 -> 1 1 2 2 2 2 2 0 -> 1 1
23    2 3 2 2 2 0. Hacia atras, la
24    // ultima operacion pinta la celda 3; la segunda pinta 2, 4,
25    5, 6 (la 3 ya esta pintada);
26    // la primera pinta 0 y 1 (2, 3, 4 ya estan pintadas). La
27    celda 7 nunca se pinta.
28    return 0;
29 }

```

Nota: *pasos* exige $a, b, c, d \geq 1$: con un 0 las operaciones $\%$ y $/$ se dividen por cero o el ciclo no termina. También supone que el predecesor es único; si las operaciones fueran otras (por ejemplo $(x, y) \rightarrow (x + y, x - y)$), habría que volver a razonar cuántos predecesores tiene un estado. *PinturaInversa* requiere que las operaciones sean conocidas de antemano y que $0 \leq l \leq r < n$; con consultas intermedias (¿qué color tiene la celda i ahora?) hay que volver a un árbol de segmentos con propagación perezosa. Por último, *buscar* modifica *sig* para comprimir caminos, así que la estructura no es segura para uso concurrente aunque conceptualmente sea una consulta.

Regla práctica: Si al mirar un estado hay una sola manera (o muy pocas) de haber llegado a él, o si la última operación sobre cada dato decide su valor, trabaja hacia atrás: parte del final, deshaz operaciones con la regla determinista y acelera con divisiones (estilo Euclides) cuando se repita el mismo paso. Para «estado final tras asignaciones de rango» fuera de línea, pinta en reversa con una estructura de «siguiente libre»; para eliminaciones fuera de línea, invierte el tiempo y usa unión-búsqueda; para juegos, propaga desde las posiciones terminales. Si hay ramificación en ambos sentidos, usa BFS bidireccional. Y si el problema tiene consultas en línea, no puedes ir hacia atrás: usa un árbol de segmentos con propagación perezosa u otra estructura dinámica.

52.5 Contribución de cada Elemento

Idea: muchos problemas piden una suma sobre **todos** los subarreglos, subconjuntos o pares (por ejemplo,

la suma de los mínimos de todos los subarreglos). En vez de recorrer cada subarreglo y calcular su mínimo, aporte o costo por separado —lo cual es $O(n^2)$ o peor—, la técnica de **contribución** invierte el orden de la suma: para cada elemento $a[i]$, se calcula **en cuántos** de esos subarreglos/subconjuntos $a[i]$ es el que determina el resultado (por ejemplo, en cuántos subarreglos $a[i]$ es el mínimo), y se multiplica $a[i]$ por esa cantidad. Sumando la contribución de cada elemento por separado se obtiene la respuesta total, sin enumerar los subarreglos uno por uno.

¿Por qué usarlo? Sumar directamente sobre los $O(n^2)$ subarreglos (o $O(2^n)$ subconjuntos) es demasiado lento cuando n es grande. Calcular cuántas veces contribuye cada elemento suele reducirse a encontrar, para cada $a[i]$, el rango de posiciones donde $a[i]$ sigue siendo el mínimo (o máximo, o el elemento relevante) — típicamente con una **pila monótona** que encuentra el vecino anterior y siguiente estrictamente menor (o mayor) en $O(n)$ total. Es el mismo cambio de perspectiva de "sumar por posición en vez de por subconjunto" que aparece en problemas de conteo de la parte de Matemáticas, aplicado aquí a estructuras de subarreglo.

Casos de uso:

- Suma de los mínimos (o máximos) de todos los subarreglos de un arreglo.
- Suma de todos los subarreglos posibles: cada elemento $a[i]$ aparece en $(i + 1) \cdot (n - i)$ subarreglos, así que la respuesta es $\sum_i a[i] \cdot (i + 1) \cdot (n - i)$.
- Contar cuántos pares (i, j) cumplen una condición, sumando la contribución de cada elemento como extremo del par.
- Suma de las diferencias absolutas entre todos los pares de un arreglo ordenado (cada elemento contribuye según cuántos quedan a su izquierda y derecha), como en *Suma de Distancias Manhattan*.
- Problemas de "suma de X sobre todos los subconjuntos", donde X depende de una propiedad local de cada elemento.

Complejidad: $O(n)$ o $O(n \log n)$ según el problema: normalmente $O(n)$ con una pila monótona para encontrar los rangos donde cada elemento es el mínimo/máximo, seguido de una suma lineal de contribuciones.

```

1 // CREATE/READ: Suma de los minimos de TODOS los subarreglos de
2 // un
3 // arreglo, mediante la tecnica de contribucion con pila
4 // monotona
5 struct SumaMinimosSubarreglos {
6     // READ: para cada indice i, encuentra el limite izquierdo (
7     // indice del
8     // primer elemento estrictamente MENOR a la izquierda, o -1)
9     // y el
10    // limite derecho (primer elemento MENOR O IGUAL a la derecha,
11    // o n);
12    // el desempate <= evita contar dos veces subarreglos con
13    // minimos
14    // repetidos
15    static long long sumaDeMinimos(const vector<long long>& a) {
16        int n = (int)a.size();
17        vector<int> izq(n), der(n);
18        vector<int> pila;
19
20        // limite izquierdo: primer menor estricto a la izquierda
21        for (int i = 0; i < n; i++) {
22            while (!pila.empty() && a[pila.back()] >= a[i]) pila.
23                pop_back();
24        }
25    }
26 }

```

```

17     izq[i] = pila.empty() ? -1 : pila.back();
18     pila.push_back(i);
19 }
20 pila.clear();
21
22 // limite derecho: primer menor o igual a la derecha
23 for (int i = n - 1; i >= 0; i--) {
24     while (!pila.empty() && a[pila.back()] > a[i]) pila.
25 pop_back();
26     der[i] = pila.empty() ? n : pila.back();
27     pila.push_back(i);
28 }
29
30 long long total = 0;
31 const long long MOD = 1000000007LL;
32 for (int i = 0; i < n; i++) {
33     // cantidad de subarreglos donde a[i] es el minimo:
34     // (i - izq[i]) elecciones de inicio * (der[i] - i)
35     // elecciones de fin
36     long long cantidad = (long long)(i - izq[i]) * (long long)
37 (der[i] - i) % MOD;
38     total = (total + (a[i] % MOD) * cantidad) % MOD;
39 }
40 return total;
41 } // O(n)
42
43 // -----
44 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
45 // -----
46
47 // 1. Suma de MAXIMOS en vez de minimos: invertir las
48 // comparaciones de
49 // la pila (>= por <=, > por <), manteniendo el mismo
50 // desempate
51 // estricto de un solo lado para no duplicar subarreglos.
52 // 2. Suma de TODOS los subarreglos (no el minimo, la suma
53 // completa de
54 // cada uno): cada a[i] aparece en (i+1)*(n-i) subarreglos
55 // distintos
56 // (i+1 elecciones de inicio, n-i de fin), sin necesidad
57 // de pila
58 // monotona: total = sum(a[i] * (i+1) * (n-i)).
59 // 3. Desempate con elementos iguales: usar >= de un lado y >
60 // del otro
61 // (como en el codigo) evita que un mismo subarreglo con
62 // minimo
63 // repetido se cuente dos veces, una por cada copia del
64 // minimo.
65 // 4. Contribucion de PARES (i,j) segun una condicion sobre a
66 // [i],a[j]:
67 // en vez de recorrer O(n^2) pares, agrupar por valor o
68 // por
69 // propiedad y contar cuantos elementos a cada lado
70 // cumplen la
71 // condicion, sumando la contribucion de cada elemento
72 // como extremo.
73 // 5. Overflow: la cantidad de subarreglos puede ser del
74 // orden de n^2;
75 // aplicar modulo en cada multiplicacion si el problema lo
76 // pide, o
77 // usar __int128/long long con cuidado si se necesita el
78 // valor exacto.
79
80 };

```

Ejemplo de funcionamiento:

```

1 long long resultado = SumaMinimosSubarreglos::sumaDeMinimos({3,
2 1, 2, 4});
3 // resultado = 17
4 // (subarreglos y sus minimos: [3]=3, [1]=1, [2]=2, [4]=4,
5 [3,1]=1,
6 [1,2]=1, [2,4]=2, [3,1,2]=1, [1,2,4]=1, [3,1,2,4]=1;
7 // suma total = 3+1+2+4+1+1+2+1+1+1 = 17)

```

Nota: el desempate entre `izq` (con `>=`, borde estrictamente menor) y `der` (con `>`, borde menor o igual) es lo que evita contar dos veces un subarreglo cuando el mínimo aparece repetido dentro de él. Si ambos lados usan el mismo criterio de desempate, subarreglos con mínimos duplicados se cuentan más de una vez, dando un resultado mayor al correcto.

Regla práctica: cuando el problema pida una suma sobre **todos** los subarreglos o pares y la fuerza bruta sea $O(n^2)$ o peor, pregúntate "¿en cuántas de esas estructuras participa cada elemento, y con qué papel?" — casi siempre esa pregunta se resuelve con una pila monótona

en $O(n)$, convirtiendo una suma cuadrática en una suma lineal.

52.6 Fuerza Bruta para Descubrir Patrones

Idea: cuando la fórmula o el algoritmo de un problema no es evidente, una estrategia práctica es **programar la fuerza bruta** (aunque sea exponencial o $O(n^2)$, sin importar que no pase el límite de tiempo real) y **imprimir la respuesta para valores pequeños** de n (por ejemplo $n = 1, 2, 3, \dots, 15$). Con esa secuencia de salida, se busca un patrón: ¿es lineal? ¿cuadrática? ¿coincide con una secuencia conocida (Fibonacci, factorial, números de Catalan)? ¿la diferencia entre términos consecutivos sigue un patrón más simple? Una vez identificada la fórmula o recurrencia, se implementa la versión eficiente y se **verifica** contra la fuerza bruta en los mismos valores pequeños antes de confiar en ella.

¿Por qué usarlo? No es un algoritmo en sí, sino una **metodología de resolución** bajo presión de tiempo: muchos problemas de competencia (especialmente en la parte Ad-hoc) esconden una fórmula cerrada o una recurrencia simple detrás de un enunciado que no la sugiere directamente. Derivar la fórmula analíticamente puede ser lento o propenso a errores; observarla empíricamente a partir de una tabla de valores pequeños suele ser más rápido y, combinado con la verificación cruzada contra la fuerza bruta, reduce el riesgo de enviar una fórmula incorrecta.

Casos de uso:

- Problemas de conteo donde la fórmula cerrada no es obvia a partir del enunciado (por ejemplo, "de cuántas formas se puede...").
- Verificar rápidamente una fórmula que se sospecha, comparándola contra la fuerza bruta en casos pequeños antes de implementarla para el caso general.
- Identificar si la respuesta sigue una secuencia conocida (buscar los primeros términos en OEIS mentalmente o por reconocimiento de patrones: triangular, factorial, potencias de 2, Catalan, Fibonacci).
- Depurar una solución eficiente que da resultados distintos a la fuerza bruta en casos pequeños, para localizar en qué valor de n empieza a diferir.
- Problemas de geometría o combinatoria ad-hoc donde la relación entre las variables de entrada y la salida no es evidente algebraicamente.

Complejidad: no aplica una complejidad única — depende de la fuerza bruta usada para generar los datos (normalmente $O(2^n)$, $O(n!)$ o $O(n^3)$, ejecutada solo para n pequeño) y de la fórmula final encontrada, que en el mejor de los casos es $O(1)$ o $O(\log n)$.

```

1 // CREATE/READ: Plantilla para generar una tabla de valores
2 // pequeños con
3 // fuerza bruta, imprimir diferencias sucesivas, y verificar
4 // una formula
5 // candidata contra esa misma fuerza bruta
6 struct FuerzaBrutaPatrones {
7     // READ: fuerza bruta de EJEMPLO: cuenta el numero de formas
8     // de subir
9     // una escalera de n escalones dando pasos de 1 o 2 (
10     // Fibonacci
11     // desplazado); se reemplaza por la fuerza bruta REAL del
12     // problema

```

```

8 static long long fuerzaBruta(int n) {
9     if (n == 0) return 1;
10    if (n < 0) return 0;
11    return fuerzaBruta(n - 1) + fuerzaBruta(n - 2);
12 } //  $O(2^n)$ , solo usable para n pequeño (hasta ~30)
13
14 // READ: genera la tabla de valores  $f(1), f(2), \dots, f(\text{limite})$ 
15 // para
16 // inspeccionar a simple vista o comparar contra una formula
17 // candidata
18 static vector<long long> generarTabla(int limite) {
19     vector<long long> tabla;
20     for (int n = 1; n <= limite; n++) tabla.push_back(
21         fuerzaBruta(n));
22     return tabla;
23 } //  $O(2^{\text{limite}})$  con la fuerza bruta de ejemplo
24
25 // READ: imprime las diferencias sucesivas de una tabla; una
26 // diferencia
27 // constante sugiere progresion aritmetica, una diferencia
28 // lineal
29 // sugiere cuadratica, y una razon constante entre terminos
30 // sugiere
31 // progresion geometrica o recurrencia lineal simple
32 static vector<long long> diferencias(const vector<long long>&
33     tabla) {
34     vector<long long> dif;
35     for (size_t i = 1; i < tabla.size(); i++) dif.push_back(
36         tabla[i] - tabla[i - 1]);
37     return dif;
38 } //  $O(n)$ 
39
40 // READ: verifica una formula candidata contra la fuerza
41 // bruta en un
42 // rango de valores pequenos; retorna el primer n donde
43 // difieren, o -1
44 // si coinciden en todo el rango probado
45 static int verificarFormula(int limite, function<long long>(
46     int)> formula) {
47     for (int n = 1; n <= limite; n++) {
48         if (fuerzaBruta(n) != formula(n)) return n;
49     }
50     return -1; // coinciden en todo el rango: buena señal, no
51     // prueba definitiva
52 } //  $O(\text{limite})$  llamadas a formula() + costo de fuerzaBruta()
53
54 //
55 // -----
56 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
57 // -----
58
59 // 1. Reemplazar fuerzaBruta(): esta es solo un EJEMPLO (
60 // escalera de
61 // Fibonacci); en un problema real se sustituye por
62 // backtracking,
63 // busqueda exhaustiva de permutaciones, o simulacion
64 // directa del
65 // enunciado, sin importar que sea lenta.
66 // 2. Buscar la secuencia en OEIS (fuera de competencia, o si
67 // hay
68 // acceso a internet permitido): los primeros 6-8 terminos
69 // de una
70 // tabla suelen ser suficientes para identificar una
71 // secuencia
72 // conocida y su formula cerrada o funcion generadora.
73 // 3. Patrones comunes a reconocer a simple vista:
74 // diferencias
75 // constantes = aritmetica ( $f(n) = a \cdot n + b$ ); razon
76 // constante entre
77 // terminos = geometrica ( $f(n) = a \cdot r^n$ ); cociente  $f(n)/f$ 
78 // ( $n-1$ )
79 // tendiendo a un valor = crecimiento exponencial con esa
80 // base;
81 // coincide con 1,1,2,3,5,8,... = Fibonacci; coincide con
82 // 1,1,2,5,14,42,... = numeros de Catalan.
83 // 4. Verificacion INSUFICIENTE: verificarFormula() solo
84 // prueba hasta
85 // "limite" (tipicamente  $n \leq 20-30$  por el costo de la
86 // fuerza
87 // bruta); coincidir en ese rango es evidencia fuerte pero
88 // NO una
89 // demostracion, especialmente en problemas con casos
90 // borde en
91 // valores grandes o con paridad/modulo involucrado.
92 // 5. Formulas con recurrencia en vez de cerrada: si no se
93 // encuentra
94 // una formula cerrada pero sí una recurrencia lineal
95 // simple
96 // ( $f(n) = c_1 \cdot f(n-1) + c_2 \cdot f(n-2) + \dots$ ), esa recurrencia
97 // ya es
98 // suficiente para una DP eficiente  $O(n)$ , sin necesidad de
99 // la
100 // forma cerrada.

```

```

39 // 6. Automatizar la busqueda de recurrencia lineal: si se
40 // sospecha
41 // una recurrencia de orden k, se puede resolver un
42 // sistema lineal
43 // con los primeros  $2k+1$  terminos para encontrar los
44 // coeficientes
45 // (metodo de Berlekamp-Massey, avanzado, fuera del
46 // alcance de esta
47 // plantilla).
48 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> tabla = FuerzaBrutaPatrones::generarTabla(8);
2 // tabla = {1, 2, 3, 5, 8, 13, 21, 34}
3 // (numero de formas de subir n escalones con pasos de 1 o 2; a
4 // simple
5 // vista, es la secuencia de Fibonacci desplazada un lugar)
6 vector<long long> dif = FuerzaBrutaPatrones::diferencias(tabla)
7 ;
8 // dif = {1, 1, 2, 3, 5, 8, 13}
9 // (las diferencias son la misma secuencia desplazada otro
10 // lugar mas,
11 // confirmando la naturaleza de Fibonacci del patron)
12 // formula candidata:  $f(n)$  es el  $(n+1)$ -esimo numero de
13 // Fibonacci
14 // (con  $\text{Fib}(1)=\text{Fib}(2)=1$ ); se verifica contra la fuerza bruta
15 auto fib = [](int n) {
16     long long a = 1, b = 1;
17     for (int i = 2; i <= n; i++) { long long c = a + b; a = b; b
18         = c; }
19     return b;
20 };
21 int primerFallo = FuerzaBrutaPatrones::verificarFormula(8, fib)
22 ;
23 // primerFallo = -1 (la formula coincide en todo el rango
24 // probado)

```

Nota: coincidir con la fuerza bruta en un rango pequeño (por ejemplo $n \leq 20$) es una señal fuerte pero **no una demostración**. Problemas con casos borde en n grande, dependencia de paridad, o condiciones que solo se activan a partir de cierto tamaño pueden hacer que una fórmula parezca correcta en la tabla pequeña y falle en el caso real. Cuando sea posible, conviene razonar *por qué* la fórmula tiene sentido (no solo que coincide numéricamente) antes de enviarla con confianza total.

Regla práctica: usa esta técnica como primer paso exploratorio cuando el enunciado no sugiera una fórmula ni una DP evidente: programa la fuerza bruta más simple posible (aunque sea $O(2^n)$), genera una tabla de al menos 8-10 valores, busca un patrón en la secuencia o en sus diferencias, y siempre verifica la fórmula candidata contra la fuerza bruta antes de confiar en ella para el caso grande.

53 Herramientas de Práctica

53.1 Stress Testing (Generador, Fuerza Bruta y Checker)

Idea: el *stress testing* es la técnica para encontrar el caso de prueba que rompe una solución antes de que lo haga el juez del concurso. Se necesitan tres piezas: un **generador** que produce entradas aleatorias pequeñas (con parámetros de tamaño ajustables), la solución **rápida** que se quiere verificar, y una solución de **fuerza bruta** (lenta pero obviamente correcta, como la de *Fuerza Bruta para Descubrir Patrones*) que sirva de referencia. Un script (bash o un bucle en el propio código) genera una entrada, corre ambas soluciones sobre ella, y compara las salidas; si difieren, esa entrada queda guardada como el caso que falla, listo para depurar con un tamaño mu-

cho más manejable que el del caso real que dio *Wrong Answer*.

¿Por qué usarlo? Cuando el juez rechaza una solución con *Wrong Answer* en un caso de miles de elementos, depurarla directamente es casi imposible: no se sabe en qué parte de la entrada está el error. Generando entradas pequeñas (5-10 elementos) y comparando automáticamente cientos o miles de veces por segundo contra la fuerza bruta, se encuentra en minutos un contraejemplo mínimo, mucho más fácil de razonar a mano o de recorrer con un depurador. Es la aplicación práctica, en forma de infraestructura reutilizable, de la misma idea de "verificar contra la fuerza bruta" que ya se vio como metodología en la subsección anterior.

Casos de uso:

- Encontrar el contraejemplo mínimo que rompe una solución que dio *Wrong Answer* en el juez.
- Verificar una optimización o reescritura de código contra la versión original antes de enviarla, para confirmar que el comportamiento no cambió.
- Depurar errores de casos borde (arreglos vacíos, un solo elemento, valores repetidos, todos iguales) generándolos con parámetros extremos del generador.
- Validar un *checker* para problemas con múltiples respuestas válidas (no comparar salida exacta, sino verificar que la respuesta dada cumple las condiciones del problema).
- Comparar el rendimiento real de dos implementaciones con entradas de tamaño creciente (*stress testing* de tiempo, no solo de corrección).

Complejidad: no aplica una complejidad de algoritmo — es una técnica de **flujo de trabajo**; el costo práctico es el número de iteraciones necesarias hasta encontrar un caso que falle, dominado por qué tan pequeño y específico es el generador respecto al bug real.

```

1 // CREATE/READ: Plantilla de generador aleatorio + comparacion
  // en el mismo
2 // binario (alternativa a usar scripts bash separados), util
  // cuando no se
3 // tiene acceso a la terminal para armar el pipeline completo
4 struct StressTest {
5     mt19937 rng;
6
7     // CREATE: inicializa el generador con una semilla EXTERNA (
      // por
8     // ejemplo, la hora actual o un contador de iteracion), para
      // que cada
9     // corrida produzca entradas distintas
10    StressTest(unsigned semilla) : rng(semilla) {
11    } // O(1)
12
13    // READ: genera un vector de n enteros aleatorios en [minVal,
      // maxVal];
14    // n pequeño (5-10) es clave para que un contraejemplo sea
      // facil de leer
15    vector<int> generarArreglo(int n, int minVal, int maxVal) {
16        uniform_int_distribution<int> dist(minVal, maxVal);
17        vector<int> a(n);
18        for (int& x : a) x = dist(rng);
19        return a;
20    } // O(n)
21
22    // READ: solucion RAPIDA de ejemplo (a reemplazar por la que
      // se quiere
23    // verificar); aqui: suma maxima de un subarreglo (Kadane)
24    static long long solucionRapida(const vector<int>& a) {
25        long long mejor = a[0], actual = a[0];
26        for (size_t i = 1; i < a.size(); i++) {
27            actual = max((long long)a[i], actual + a[i]);
28            mejor = max(mejor, actual);

```

```

29    }
30    return mejor;
31 } // O(n)
32
33 // READ: fuerza bruta de ejemplo (obviamente correcta, O(n^2))
34 // prueba
35 // todos los subarreglos
36 static long long fuerzaBruta(const vector<int>& a) {
37     long long mejor = a[0];
38     for (size_t i = 0; i < a.size(); i++) {
39         long long suma = 0;
40         for (size_t j = i; j < a.size(); j++) {
41             suma += a[j];
42             mejor = max(mejor, suma);
43         }
44     }
45     return mejor;
46 } // O(n^2)
47
48 // READ: corre iteraciones de stress test; retorna el primer
  // arreglo
49 // donde las dos soluciones difieren, o un vector vacio si no
  // se
50 // encontro ninguno en 'intentos' iteraciones
51 vector<int> buscarContraejemplo(int intentos, int nMax) {
52     uniform_int_distribution<int> distN(1, nMax);
53     for (int it = 0; it < intentos; it++) {
54         vector<int> a = generarArreglo(distN(rng), -10, 10);
55         if (solucionRapida(a) != fuerzaBruta(a)) return a;
56     }
57     return {};
58 } // O(intentos * costo de ambas soluciones)
59
60 //
61 // -----
62 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
63 // -----
64 // 1. Pipeline con scripts separados (mas comun en
  // competencia real):
65 // tres binarios independientes (gen.cpp, rapida.cpp,
  // brute.cpp) y
66 // un script bash que en un bucle ejecuta
67 // ./gen semilla > input.txt; ./rapida < input.txt > out1.
  // txt;
68 // ./brute < input.txt > out2.txt; diff out1.txt out2.txt,
  // deteniendo
69 // el bucle en la primera diferencia; preferible cuando
  // las
70 // soluciones ya estan escritas como programas separados
  // que leen
71 // de cin/cout.
72 // 2. Checker para problemas con MULTIPLES respuestas validas:
  // en vez
73 // de comparar salida exacta, escribir una funcion que
  // verifique si
74 // la respuesta dada cumple las condiciones del problema (
  // por
75 // ejemplo, si es una permutacion valida, o si la suma
  // reportada
76 // coincide con recalcularla sobre la solucion propuesta).
77 // 3. Minimizar el contraejemplo encontrado: una vez hallado
  // un caso
78 // que falla, intentar reducirlo (quitar elementos,
  // achicar el
79 // rango de valores) mientras la solucion rapida y la
  // fuerza bruta
80 // sigan discrepando, para llegar al caso MINIMO que
  // reproduce el bug.
81 // 4. Semilla reproducible: guardar la semilla usada en el
  // intento que
82 // fallo permite regenerar exactamente el mismo caso
  // despues, sin
83 // necesidad de guardar el arreglo completo en memoria.
84 // 5. Generadores mas complejos (arboles, grafos, cadenas con
  // estructura especifica): el mismo esquema aplica,
  // cambiando
85 // generarArreglo() por una funcion que construya la
  // estructura de
86 // entrada requerida por el problema especifico.
87 // 6. Reemplazar solucionRapida y fuerzaBruta: las incluidas
  // aqui
88 // (Kadane vs. fuerza bruta O(n^2)) son solo un ejemplo
  // ilustrativo;
89 // en la practica se sustituyen por el par real que se
  // quiere
90 // comparar.

```

Ejemplo de funcionamiento:

```

1 StressTest st(12345); // semilla fija para reproducibilidad del
  // ejemplo

```

```

2 vector<int> contraejemplo = st.buscarContraejemplo(1000, 8);
3 // contraejemplo esta vacio si las dos soluciones (Kadane vs
4 fuerza bruta)
5 // coinciden en las 1000 entradas aleatorias probadas -- es el
6 resultado
7 // esperado aqui porque Kadane es correcto; en un caso real, un
8 vector
9 // no vacio sería el contraejemplo a depurar

```

Nota: en competencia, la forma más común de montar esto es con **tres archivos separados** (generador, solución candidata, fuerza bruta) y un script de shell que los ejecuta en bucle comparando con `diff` — la versión de arriba, con todo en un único `struct` de C++, es útil cuando no se tiene acceso cómodo a la terminal (por ejemplo, dentro de un IDE en línea), pero el flujo de trabajo conceptual es el mismo en ambos casos.

Regla práctica: arma el generador **antes** de necesitarlo desesperadamente: en cuanto una solución da *Wrong Answer* sin pista clara de dónde, escribe la fuerza bruta más simple posible (aunque sea $O(2^n)$) y un generador de entradas pequeñas, y deja correr la comparación en bucle. Casi siempre es más rápido que revisar el código línea por línea buscando el error a ciegas.

53.2 Problemas Interactivos

Idea: en un problema **interactivo**, el programa no lee toda la entrada de una vez ni imprime toda la salida al final: **dialoga** con un juez (*interactor*) que responde a cada consulta antes de que el programa decida la siguiente. El patrón típico es un bucle de "preguntar, leer la respuesta del juez, decidir la siguiente pregunta" hasta encontrar la respuesta final —el ejemplo clásico es adivinar un número oculto haciendo consultas de tipo "¿es mayor que x ?" y usando *Búsqueda Binaria on Answer* para decidir la siguiente consulta. La dificultad técnica añadida, distinta de cualquier otro problema de esta guía, es que la salida del programa debe llegar al juez **inmediatamente** después de cada consulta, sin quedar retenida en un búfer.

¿Por qué usarlo? Casi cualquier técnica algorítmica de esta guía (búsqueda binaria, greedy, teoría de juegos) puede aparecer dentro de un problema interactivo; lo que cambia es exclusivamente la **forma de comunicación** con el juez, no el algoritmo en sí. El error más común y más difícil de diagnosticar en estos problemas no es de lógica, sino de **buffering**: si la salida no se vacía (*flush*) después de cada consulta, el juez nunca la recibe, el programa se queda esperando una respuesta que no llega, y el resultado es un *Time Limit Exceeded* o *Idleness Limit Exceeded* que nada tiene que ver con el algoritmo implementado.

Casos de uso:

- Adivinar un valor oculto mediante consultas de comparación, resuelto con búsqueda binaria interactiva.
- Determinar una propiedad de una estructura oculta (un árbol, un grafo) haciendo consultas limitadas sobre ella.
- Juegos contra un juez adversarial que responde de la peor manera posible dentro de las reglas —el juez puede incluso no fijar la respuesta de antemano, siempre que sea consistente con las respuestas ya dadas.

- Problemas con límite estricto en el **número** de consultas permitidas, donde minimizar consultas (no solo el tiempo de cómputo) es parte del objetivo.
- Práctica de manejo correcto de entrada/salida en tiempo real, útil también para depurar cualquier código que interactúe con un proceso externo.

Complejidad: depende enteramente del problema — se mide típicamente en **número de consultas** permitidas (por ejemplo $O(\log n)$ para una búsqueda binaria interactiva) en vez de en tiempo de cómputo puro, ya que el cómputo entre consultas suele ser trivial frente a la limitación de cuántas preguntas se pueden hacer.

```

1 // CREATE/READ/WRITE: Plantilla de problema interactivo:
2 // adivinar un
3 // numero oculto en [1, n] mediante consultas "es x mayor que
4 // el oculto?",
5 // usando busqueda binaria y flush explícito tras cada consulta
6 struct ProblemaInteractivo {
7     int n;
8
9     // CREATE: guarda el rango de busqueda
10    ProblemaInteractivo(int limite) : n(limite) {}
11    // O(1)
12
13    // WRITE: envia una consulta al juez y lee su respuesta; el
14    // flush es
15    // OBLIGATORIO despues de cada consulta, o el juez nunca la
16    // recibe
17    char preguntar(int x) {
18        cout << "? " << x << "\n";
19        cout.flush(); // CRITICO: sin esto el programa se cuelga
20        // esperando respuesta
21        char respuesta;
22        cin >> respuesta; // tipicamente 'Y'/'N' o '<'/'>'/'=' segun
23        // el problema
24        return respuesta;
25    } // O(1) por consulta, mas el costo de comunicacion con el
26    // juez
27
28    // READ/WRITE: busqueda binaria interactiva: encuentra el
29    // numero
30    // oculto en [1, n] preguntando "es x mayor que el oculto?"
31    // en cada paso
32    int adivinar() {
33        int lo = 1, hi = n;
34        while (lo < hi) {
35            int medio = lo + (hi - lo) / 2;
36            char resp = preguntar(medio);
37            if (resp == 'Y') { // medio es mayor que el oculto:
38                buscar mas abajo
39                hi = medio;
40            } else { // medio es menor o igual: buscar mas arriba
41                lo = medio + 1;
42            }
43        }
44        cout << "! " << lo << "\n"; // respuesta FINAL, tambien con
45        // flush
46        cout.flush();
47        return lo;
48    } // O(log n) consultas
49
50    //
51    // -----
52    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
53    // -----
54
55    // 1. Sincronizacion de flujos en C++: al inicio del programa,
56    // usar
57    // cin.tie(nullptr) con cuidado en interactivos -- de
58    // hecho, MEJOR
59    // NO desactivar la sincronizacion automatica (NO usar
60    // ios::sync_with_stdio(false) sin verificar), o usar endl
61    // en vez de
62    // "\n" + flush manual, ya que endl hace flush
63    // automaticamente y es
64    // mas dificil de olvidar en cada linea de salida.
65    // 2. Interactor "adaptativo" (adversarial): el juez puede no
66    // fijar el
67    // numero oculto de antemano, y en cambio mantener el
68    // conjunto de
69    // valores consistentes con las respuestas dadas hasta
70    // ahora,
71    // respondiendo siempre de la forma que mas complique al
72    // programa;
73    // la solucion debe funcionar igual sin asumir que el
74    // valor esta

```

```

53 //    fijo desde el inicio, solo que las respuestas son
54 //    consistentes.
55 // 3. Limite de consultas MENOR al esperado (por ejemplo,
56 //    permite solo
57 //     $\log_2(n) - 2$  preguntas en vez de  $\log_2(n)$ ): senal de que
58 //    se
59 //    necesita una estrategia mas eficiente que la busqueda
60 //    binaria
61 //    directa, como aprovechar informacion de consultas
62 //    anteriores de
63 //    forma no binaria.
64 // 4. Terminacion temprana o formato de respuesta final
65 //    incorrecto:
66 //    revisar con cuidado el formato exacto que pide el
67 //    enunciado para
68 //    la respuesta final (aqui "! numero"), ya que un formato
69 //    distinto
70 //    al esperado por el juez cuenta como respuesta
71 //    incorrecta aunque
72 //    el numero en si sea correcto.
73 // 5. Testing local de interactivos: como no hay juez real en
74 //    el propio
75 //    entorno, se escribe un interactor propio (otro programa
76 //    que
77 //    simula al juez) y se conectan ambos procesos por pipes
78 //    o
79 //    redireccion de archivos para probar antes de enviar.
80 };

```

Ejemplo de funcionamiento:

```

1 // Interaccion tipica con el juez (E/S real, no simulable sin
2 // un
3 // interactor externo):
4 // programa: ? 5
5 // juez:      N          (el oculto es >= 5)
6 // programa: ? 8
7 // juez:      Y          (el oculto es < 8, es decir <= 7)
8 // programa: ? 6
9 // juez:      N          (el oculto es >= 6, es decir 6 o 7)
10 // programa: ? 7
11 // juez:      N          (el oculto es 7)
12 // programa: ! 7
13 // juez:      AC (o el veredicto correspondiente)

```

Nota: el fallo más frecuente en problemas interactivos **no es de lógica**, sino de **buffering**: olvidar `cout.flush()` (o no usar `endl`, que hace flush automáticamente) después de cada consulta deja la salida retenida en el búfer del programa, el juez nunca la recibe, y el resultado es un *Time Limit Exceeded* que parece un error de algoritmo pero es puramente de entrada/salida. Verificar esto es siempre el primer paso al depurar un interactivo que se cuelga.

Regla práctica: en cualquier problema interactivo, antes de razonar el algoritmo, asegúrate de hacer **flush** después de **cada** línea de salida (o usa `endl` en vez de `"\n"` consistentemente). El algoritmo subyacente reutiliza técnicas ya vistas en la guía (aquí, búsqueda binaria) — lo específico de esta subsección es la disciplina de comunicación con el juez, no una técnica algorítmica nueva.

Parte IX String Processing

53.3 ¿Qué algoritmo usar?

Cuando el ejercicio diga / pida	Usa	Pista rápida
“Autocompletado”, “cuántas palabras empiezan con este prefijo”, “prefijo común”	Trie	Contador por nodo. $O(S)$ por operación.
“¿Estas dos subcadenas son iguales?” con muchas consultas	Hash polinómico	Hash doble si hay riesgo adversarial. $O(1)$ por consulta.
“Prefijo común más largo entre dos posiciones”	Hash búsqueda binaria	+ Compara hashes de prefijos de longitud L .
“Período más corto”, “¿la cadena es una repetición de un bloque?”	Función de prefijos	$n - \pi[n - 1]$ si divide a n .
“Todas las ocurrencias de un patrón en un texto”	KMP	$p \# t$ y busca $\pi[i] = p $. $O(n + m)$.
Lo mismo, o “cuánto coincide cada sufijo con el prefijo”	Algoritmo Z	Equivalente a KMP. $O(n)$.
Búsqueda de un patrón priorizando código corto	Rabin-Karp	Verifica el candidato. Esperado $O(n + m)$.
Texto natural con alfabeto grande, rendimiento práctico	Boyer-Moore	Peor caso $O(nm)$; en CP prefiere KMP.
Mismo patrón contra muchos textos, o texto en streaming	Autómata de coincidencia	Tabla δ . $O(1)$ por carácter.
“Buscar muchos patrones a la vez”, diccionario, palabras prohibidas	Aho-Corasick	Trie + enlaces de fallo. Hereda coincidencias.
“Cuántas cadenas de longitud n evitan estos patrones”	DP sobre Aho-Corasick	Estados = nodos; descarta nodos con patrón.
Patrón con comodines ?	Convolución (FFT)	$O(nm)$ si es pequeño; $O(n \log n)$ con FFT.
“¿Algún anagrama del patrón aparece en el texto?”	Ventana deslizante de frecuencias	Contador de diferencias. $O(n + \sigma)$.

Table 27: Estructuras y búsqueda de patrones: qué usar según lo que pide el ejercicio.

Cuando el ejercicio diga / pide	Usa	Pista rápida
“Cuántos substrings distintos hay”	Suffix Array + LCP o Suffix Automaton	$\binom{n+1}{2} - \sum \text{lcp}$, o suma de $\text{len}[v] - \text{len}[\text{link}[v]]$.
Muchas consultas sobre subcadenas, LCP entre dos sufijos, orden lexicográfico	Suffix Array + Kasai + Sparse Table	LCP = mínimo en rango. $O(1)$.
“¿ t es substring de s ?”, construir en línea, ocurrencias de cada substring	Suffix Automaton	$O(t)$ por consulta.
“Subcadena común contigua más larga”	DP 2D (reinicia a 0)	$O(nm)$; con muchas cadenas, Suffix Automaton.
“Subsecuencia común más larga”, diff, alinear secuencias	LCS	$O(nm)$. Solo inserción/borrado: $n + m - 2 \text{LCS}$.
“Cadena más corta que contenga a ambas como subsecuencia”	Supersecuencia común más corta	$n + m - \text{LCS}$.
“Mínimo de operaciones para transformar una cadena en otra” (insertar, borrar, sustituir)	Distancia de edición	$O(nm)$.
“Subsecuencia palindrómica más larga”, “mínimo de borrados para palíndromo”	DP de intervalos o LCS con el reverso	$n - \text{LPS borrados}$.
“Subsecuencia creciente (decreciente) más larga”, “bitónica”	LIS / LDS	$O(n \log n)$ con colas. Bitónica: $\text{LIS}[i] + \text{LDS}[i] - 1$.
“Cuántos substrings palíndromos” (con repetición de posición)	Manacher	Suma de $\lceil \text{rad}/2 \rceil$. $O(n)$.
“Cuántos palíndromos distintos ”, palíndromo más largo en cada prefijo	Eertree	nodos $- 2$. $O(n)$.
“Mínimo de cortes para partir en palíndromos”	Tabla esPal + DP lineal	$O(n^2)$.

Table 28: Sufijos, palíndromos y comunes: qué usar según lo que pide el ejercicio.

Cuando el ejercicio diga / pide	Usa	Pista rápida
“Rotación lexicográficamente mínima”, “¿son rotaciones una de otra?”	Booth o Duval sobre $s + s$	Forma canónica. $O(n)$.
“Factorización de Lyndon”, “sufijo lexicográficamente menor”	Algoritmo de Duval	Compara con $\leq$. $O(n)$.
“a solo a la izquierda, b solo a la derecha, sin cruzarse, ¿se puede pasar de S_1 a S_2 ?”	Verificación de dos condiciones	Misma secuencia sin # y dirección por índice. $O(n)$.
“¿Son anagramas?”	Conteo de frecuencia	$O(n + \sigma)$; si no, ordenar.
“Agrupa las palabras que son anagramas”	Firma canónica + hash	Ordenada o vector de frecuencias con separador.
“Substrings que se pueden reordenar a palíndromo”	Máscara de paridad + hashmap	Misma máscara o un bit de diferencia.
“Reordena sin letras iguales adyacentes”, “anagrama lexicográfico con restricción”	Max-heap por frecuencia	Imposible si $f_{\max} > \lceil n/2 \rceil$.

Table 29: Construcciones y anagramas: qué usar según lo que pide el ejercicio.

54 Fundamentos

54.1 Trie

Idea: un trie (árbol de prefijos) almacena un conjunto de cadenas organizándolas por prefijos compartidos: cada nodo representa un prefijo, y sus hijos extienden ese prefijo en un carácter. Dos cadenas que comparten un prefijo comparten el mismo camino desde la raíz hasta donde sus caracteres divergen, momento en el cual el árbol se ramifica. Esto convierte operaciones

sobre el conjunto completo de cadenas (insertar, buscar, contar cuántas empiezan con cierto prefijo) en un simple recorrido de árbol, de longitud igual a la de la cadena consultada, sin importar cuántas cadenas haya en total.

¿Por qué usarlo? Buscar si una cadena existe en un conjunto de n cadenas con un `set<string>` cuesta $O(|S| \log n)$ por la comparación de strings dentro de cada paso del árbol balanceado. Un trie resuelve la misma consulta en $O(|S|)$, independiente de n , porque cada carácter de la consulta se procesa una sola vez bajando un nivel. La ventaja se vuelve decisiva en consultas de **prefijo** (¿cuántas cadenas empiezan con “ab?”), que un `set` no puede responder eficientemente sin recorrer rangos, mientras que en un trie es simplemente llegar al nodo del prefijo y leer un contador.

Casos de uso:

- Autocompletado: listar todas las cadenas que comparten un prefijo dado.
- Contar cuántas cadenas de un conjunto tienen cierto prefijo.
- Verificar pertenencia de una cadena a un diccionario en $O(|S|)$.
- Base estructural para *Aho-Corasick* (múltiples patrones) y para *Trie de Bits* (XOR máximo entre pares).
- Encontrar el prefijo común más largo entre dos o más cadenas del conjunto.

Complejidad: $O(|S|)$ por inserción o consulta, donde $|S|$ es la longitud de la cadena procesada. El espacio total es $O(\sum |S_i| \cdot \sigma)$ en el peor caso, donde σ es el tamaño del alfabeto, ya que cada nodo reserva espacio para todos los caracteres posibles del alfabeto.

```

1 // CREATE/READ/WRITE: Trie para insertar cadenas y consultar
  pertenencia
2 // y conteo de prefijos, sobre alfabeto minúsculas 'a'-'z'
3 struct Trie {
4     static const int ALFABETO = 26;
5     struct Nodo {
6         int hijos[ALFABETO];
7         int contadorPrefijo; // cuantas cadenas insertadas pasan
          por este nodo
8         bool esFinDePalabra;
9         Nodo() : contadorPrefijo(0), esFinDePalabra(false) {
10             fill(hijos, hijos + ALFABETO, -1);
11         }
12     };
13     vector<Nodo> nodos;
14
15     // CREATE: inicializa el trie con un unico nodo raiz
16     Trie() {
17         nodos.push_back(Nodo());
18     } // O(1)
19
20     // WRITE: inserta la cadena s en el trie, creando nodos
          nuevos donde
21     // el camino aun no existe
22     void insertar(const string& s) {
23         int actual = 0;
24         for (char c : s) {
25             int idx = c - 'a';
26             if (nodos[actual].hijos[idx] == -1) {
27                 nodos[actual].hijos[idx] = (int)nodos.size();
28                 nodos.push_back(Nodo());
29             }
30             actual = nodos[actual].hijos[idx];
31             nodos[actual].contadorPrefijo++;
32         }
33         nodos[actual].esFinDePalabra = true;
34     } // O(|s|)
35
36

```



```

37 // READ: true si la cadena s fue insertada exactamente (no
38 // solo como prefijo)
39 bool buscar(const string& s) {
40     int actual = 0;
41     for (char c : s) {
42         int idx = c - 'a';
43         if (nodos[actual].hijos[idx] == -1) return false;
44         actual = nodos[actual].hijos[idx];
45     }
46     return nodos[actual].esFinDePalabra;
47 } // O(|s|)
48
49 // READ: cuenta cuantas cadenas insertadas tienen a s como
50 // prefijo
51 int contarConPrefijo(const string& s) {
52     int actual = 0;
53     for (char c : s) {
54         int idx = c - 'a';
55         if (nodos[actual].hijos[idx] == -1) return 0;
56         actual = nodos[actual].hijos[idx];
57     }
58     return nodos[actual].contadorPrefijo;
59 } // O(|s|)
60
61 // -----
62 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
63 // -----
64 // 1. Alfabeto mas grande o con simbolos especiales (
65 // mayusculas, digitos,
66 // caracteres especiales): usar un unordered_map<char,int>
67 // por nodo en
68 // vez de un arreglo fijo de tamaño ALFABETO, a costa de
69 // mayor
70 // constante de tiempo por acceso.
71 // 2. Eliminar una cadena del trie: decrementar
72 // contadorPrefijo a lo
73 // largo del camino de insertar(); si un nodo llega a
74 // contadorPrefijo
75 // == 0 se puede marcar como libre (no es necesario
76 // liberarlo
77 // físicamente del vector para que la estructura siga
78 // correcta).
79 // 3. Prefijo comun mas largo entre TODAS las cadenas
80 // insertadas: bajar
81 // desde la raíz mientras cada nodo tenga contadorPrefijo
82 // igual al
83 // total de cadenas insertadas Y tenga exactamente un hijo
84 // activo.
85 };
```

Ejemplo de funcionamiento:

```

1 Trie t;
2 t.insertar("casa");
3 t.insertar("casino");
4 t.insertar("caso");
5
6 bool existe = t.buscar("caso");
7 // existe = true
8
9 int conPrefijo = t.contarConPrefijo("cas");
10 // conPrefijo = 3
11 // (las tres cadenas insertadas comparten el prefijo "cas")
12
13 bool existeParcial = t.buscar("cas");
14 // existeParcial = false
15 // ("cas" es un prefijo valido en el trie, pero nunca se inserto
16 // como
17 // palabra completa, asi que esFinDePalabra es false en ese nodo)
```

Nota: buscar y contarConPrefijo son operaciones distintas y no intercambiables: buscar exige que la cadena completa haya sido insertada como palabra (nodo marcado esFinDePalabra), mientras que contarConPrefijo solo exige que el camino exista, sin importar si algún nodo intermedio es fin de palabra.

Regla práctica: usa un trie cuando las consultas sean sobre **prefijos** de un conjunto de cadenas (auto-completado, conteo de prefijos, prefijo común); si solo necesitas verificar pertenencia exacta sin ninguna consulta de prefijo, un `unordered_set<string>` con hash es más simple y con menor overhead de memoria.

54.2 Hash Polinómico (Rolling Hash)

Idea: un hash polinómico convierte una cadena $s_0s_1\dots s_{n-1}$ en un número mediante $H(s) = \left(\sum_{i=0}^{n-1} s_i \cdot p^i\right) \bmod m$, tratando cada carácter como un "dígito" en base p (una base prima mayor que el tamaño del alfabeto). La propiedad clave que lo hace útil en competencia es que, precomputando los hashes de **todos los prefijos** de una cadena junto con las potencias de p , se puede obtener el hash de **cualquier subcadena** en $O(1)$ mediante una resta modular, de forma análoga a cómo una suma de prefijos da la suma de cualquier subarreglo en $O(1)$.

¿Por qué usarlo? Comparar si dos subcadenas son iguales carácter por carácter cuesta $O(|s|)$. Con hash polinómico, comparar dos subcadenas se reduce a comparar dos enteros precomputados en $O(1)$ —a costa de una probabilidad extremadamente pequeña de colisión (dos subcadenas distintas con el mismo hash). Es la base de muchos algoritmos de comparación rápida de subcadenas, y una alternativa más simple de implementar que KMP o Algoritmo Z cuando lo único que se necesita es comparar igualdad de subcadenas, no encontrar todas las ocurrencias de un patrón.

Casos de uso:

- Comparar igualdad de dos subcadenas cualesquiera en $O(1)$ tras $O(n)$ de preprocesamiento.
- Búsqueda binaria sobre la longitud del prefijo común más largo entre dos subcadenas.
- Detección rápida de subcadenas repetidas o duplicadas.
- Verificación alternativa (más simple de codear bajo presión) a KMP/Rabin-Karp para coincidencia de un patrón.
- Comparación de igualdad de subarreglos en problemas que no son estrictamente de texto (secuencias numéricas tratadas como "cadenas").

Complejidad: $O(n)$ de preprocesamiento para una cadena de longitud n (calcular hashes de prefijos y potencias de p); $O(1)$ por consulta de hash de una subcadena arbitraria.

```

1 // CREATE/READ: Hash polinómico con hashes de prefijo
2 // precomputados, para
3 // consultar el hash de cualquier subcadena en O(1)
4 struct HashPolinomico {
5     const long long P = 131; // base, mayor que
6     // el tamaño del alfabeto
7     const long long MOD = 1000000007LL; // modulo primo
8     // grande
9
10    vector<long long> prefijoHash; // prefijoHash[i] = hash de s
11    // [0..i-1]
12    vector<long long> potencias; // potencias[i] = P^i mod MOD
13
14    // CREATE: precomputa hashes de todos los prefijos y las
15    // potencias de P
16    // necesarias para consultar cualquier subcadena de s
17    HashPolinomico(const string& s) {
18        int n = (int)s.size();
19        prefijoHash.assign(n + 1, 0);
20        potencias.assign(n + 1, 1);
21        for (int i = 0; i < n; i++) {
22            prefijoHash[i + 1] = (prefijoHash[i] * P + (s[i] - 'a'
23            + 1)) % MOD;
24            potencias[i + 1] = (potencias[i] * P) % MOD;
25        }
26    } // O(n)
```

```

21 // READ: hash de la subcadena s[l..r] (0-indexado, ambos
22 // extremos
23 // incluidos), en O(1)
24 long long hashRango(int l, int r) {
25     long long resultado = (prefijoHash[r + 1] - (prefijoHash[
26     l] * potencias[r - l + 1]) % MOD + MOD * MOD) % MOD;
27     return resultado;
28 } // O(1)
29
30 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
31 //
32 // 1. Hashing DOBLE (recomendado en competencia): usar dos
33 // pares (P,MOD)
34 // distintos y comparar igualdad SOLO si ambos hashes
35 // coinciden;
36 // reduce drasticamente la probabilidad de colision
37 // adversarial,
38 // especialmente si los problemas del concurso son
39 // generados
40 // conociendo el MOD estandar de 1e9+7.
41 // 2. MOD * MOD en hashRango: se suma para evitar que la
42 // resta modular
43 // de resultado negativo en C++ (donde % puede devolver
44 // negativo);
45 // alternativa mas limpia: sumar solo MOD si el resultado
46 // intermedio
47 // ya cabe sin overflow con ese ajuste.
48 // 3. Comparar igualdad de dos subcadenas de la MISMA cadena
49 // o de
50 // cadenas DISTINTAS: si son de cadenas distintas, cada
51 // una necesita
52 // su propio HashPolinomico (mismo P y MOD para que los
53 // hashes sean
54 // comparables entre si).
55 };

```

Ejemplo de funcionamiento:

```

1 HashPolinomico h("abcabc");
2
3 long long hash1 = h.hashRango(0, 2); // "abc"
4 long long hash2 = h.hashRango(3, 5); // "abc"
5
6 bool iguales = (hash1 == hash2);
7 // iguales = true
8 // (ambas subcadenas son "abc", literalmente iguales; sus hashes
9 // deben
10 // coincidir; hay que correrlo para confirmar el valor numerico
11 // exacto
12 // de hash1 con P=131 y MOD=1e9+7)

```

Nota: un solo hash (un solo par P, MOD) es vulnerable a colisiones adversariales cuando el problema fue diseñado sabiendo qué módulo suelen usar los concursantes (ataques conocidos con $\text{MOD} = 10^9 + 7$). En problemas donde alguien pudo haber preparado un caso de prueba adversarial contra hashing, usa **hashing doble** (ver modificación 1 dentro del código) en vez de un solo hash.

Regla práctica: usa hash polinómico cuando necesites comparar igualdad de subcadenas repetidamente y rápido, especialmente combinado con búsqueda binaria (por ejemplo, para encontrar el prefijo común más largo entre dos posiciones). Si el problema requiere encontrar **todas las ocurrencias** de un patrón (no solo comparar igualdad), *KMP* o el *Algoritmo Z* dan esa respuesta directamente sin el riesgo, aunque pequeño, de colisión de hash.

54.3 Función de Prefijos (Prefix Function / Failure Function)

Idea: para una cadena s de longitud n , la función de prefijos $\pi[i]$ da la longitud del prefijo propio más largo de $s[0..i]$ que también es sufijo de ese mismo segmento $s[0..i]$ —"propio" significa que no puede ser el segmento completo. Se calcula incrementalmente: al procesar $s[i]$,

se intenta extender el mejor prefijo-sufijo conocido en la posición anterior ($\pi[i - 1]$); si el siguiente carácter no calza, se retrocede usando el propio arreglo π (siguiendo la cadena de "fallos" hacia prefijos-sufijos más cortos) en vez de reiniciar la búsqueda desde cero, evitando así retroceder sobre caracteres ya comparados.

¿Por qué usarlo? Es la estructura de datos que hace posible que *KMP* busque un patrón en $O(n + m)$ en vez de $O(nm)$: sin la función de prefijos, cada intento fallido de coincidencia obligaría a retroceder el puntero del texto y volver a comparar caracteres ya vistos. El truco de "retroceder usando π en vez de reiniciar" es lo que garantiza que el puntero del texto **nunca retrocede**, dando la complejidad lineal. Aunque casi siempre se presenta como parte de *KMP*, la función de prefijos por sí sola también responde preguntas sobre la estructura periódica interna de una cadena.

Casos de uso:

- Como subrutina central de *Knuth-Morris-Pratt* (*KMP*) para búsqueda de patrones.
- Encontrar el período más corto de una cadena (la cadena se repite con período $n - \pi[n - 1]$ si n es divisible entre ese valor).
- Contar cuántas veces aparece cada prefijo de s como sufijo en distintas posiciones (usando el árbol de fallos formado por π).
- Comprimir una cadena detectando su unidad repetitiva más pequeña.

Complejidad: $O(n)$ —a primera vista el retroceso dentro del bucle parece poder ser costoso, pero un argumento de potencial (el valor de $\pi[i]$ solo puede aumentar en 1 por iteración del bucle externo, y cada retroceso lo reduce, así que la suma total de retrocesos está acotada por n) garantiza tiempo lineal amortizado.

```

1 // CREATE/READ: Funcion de prefijos (failure function) de una
2 // cadena,
3 // base para KMP y para analisis de periodicidad
4 struct FuncionPrefijos {
5     // READ: calcula pi[i] = longitud del prefijo propio mas
6     // largo de
7     // s[0..i] que tambien es sufijo de s[0..i], para cada i
8     static vector<int> calcular(const string& s) {
9         int n = (int)s.size();
10        vector<int> pi(n, 0);
11        for (int i = 1; i < n; i++) {
12            int j = pi[i - 1];
13            // retrocede por la cadena de fallos mientras el
14            // caracter no calce
15            while (j > 0 && s[i] != s[j]) j = pi[j - 1];
16            if (s[i] == s[j]) j++;
17            pi[i] = j;
18        }
19        return pi;
20    } // O(n)
21
22    // READ: longitud del periodo mas corto de s (la unidad
23    // minima que,
24    // repetida, reconstruye s); retorna n si s no es periodica
25    static int periodoMasCorto(const string& s) {
26        int n = (int)s.size();
27        vector<int> pi = calcular(s);
28        int candidato = n - pi[n - 1];
29        if (n % candidato == 0) return candidato;
30        return n; // s no tiene periodo propio, el periodo es
31        // ella misma
32    } // O(n)
33
34    //
35
36    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
37    //
38
39    //

```

```

32 // 1. Uso directo en KMP: para buscar un patron p en un texto
33 //    t, se
34 //    calcula pi sobre la cadena concatenada p + '#' + t (con
35 //    un
36 //    separador que no aparece en ninguna de las dos), y cada
37 //    posicion
38 //    donde pi[i] == |p| marca una ocurrencia completa del
39 //    patron.
40 // 2. Todos los "bordes" de un prefijo (no solo el mas largo):
41 //    seguir
42 //    la cadena j -> pi[j-1] -> pi[pi[j-1]-1] -> ... -> 0
43 //    desde
44 //    pi[i] da TODOS los prefijos-sufijos de s[0..i], de
45 //    mayor a menor.
46 // 3. Contar ocurrencias de cada prefijo como sufijo en TODA
47 //    la cadena:
48 //    construir un arbol donde cada nodo i apunta a pi[i]-1 (
49 //    su "padre"
50 //    de fallo), y propagar conteos desde las hojas hacia la
51 //    raiz.
52 };

```

Ejemplo de funcionamiento:

```

1 vector<int> pi = FuncionPrefijos::calcular("abcabcabc");
2 // pi = {0, 0, 0, 1, 2, 3, 4, 5, 6}
3 // (en la posicion 8, "abcabcabc" tiene como prefijo-sufijo mas
4 //    largo a
5 //    "abcabc", de longitud 6)
6
7 int periodo = FuncionPrefijos::periodoMasCorto("abcabcabc");
8 // periodo = 3
9 // (n=9, pi[8]=6, candidato = 9-6 = 3; 9 % 3 == 0, asi que "abc"
10 // repetido 3 veces reconstruye la cadena original)

```

Nota: el cálculo de `periodoMasCorto` depende de que n sea exactamente divisible entre el candidato $n - \pi[n-1]$; si no lo es, la cadena no tiene un período propio que la genere exactamente, y el valor correcto a reportar es n (la cadena completa es su propio período mínimo).

Regla práctica: usa la función de prefijos como bloque de construcción cada vez que necesites la estructura de "qué tan lejos puedo retroceder sin perder coincidencia" —directamente como base de *KMP*, o de forma independiente cuando la pregunta sea sobre periodicidad o bordes de una cadena. Si solo necesitas comparar igualdad de subcadenas (sin buscar patrones), *Hash Polinómico* suele ser más simple de codear para ese propósito específico.

54.4 Suffix Array + Algoritmo de Kasai (LCP)

Idea: el *suffix array* de una cadena s de longitud n es la lista de los n sufijos de s , representados por sus índices de inicio, **ordenados lexicográficamente**. Construirlo por fuerza bruta (ordenar comparando sufijos completo contra completo) cuesta $O(n^2 \log n)$. La construcción eficiente usa **ordenamiento por duplicación** (*doubling*): se ordenan primero los sufijos por su primer carácter, luego por sus primeros 2 caracteres (reutilizando el rango de orden ya conocido de longitud 1 para cada mitad), luego por los primeros 4, 8, 16..., duplicando la longitud comparada en cada ronda usando pares de rangos ya ordenados de la ronda anterior en vez de comparar caracteres directamente. El **LCP array** (*Longest Common Prefix*) complementa al suffix array: $\text{lcp}[i]$ da la longitud del prefijo común más largo entre los sufijos consecutivos $i-1$ e i en el orden del suffix array. El algoritmo de **Kasai** calcula todo el LCP array en $O(n)$ aprovechando una propiedad clave: si el sufijo que empieza en j tiene LCP h con su vecino en el suffix array, el sufijo que empieza en $j+1$ tiene LCP de al menos $h-1$ con su vecino —así que el LCP nunca hay que recalcularlo desde

cero, solo puede "bajar como máximo 1" entre iteraciones consecutivas.

¿Por qué usarlo? El suffix array reduce casi cualquier pregunta sobre subcadenas de s a un problema sobre un arreglo ordenado: buscar un patrón se vuelve búsqueda binaria ($O(m \log n)$), y combinado con el LCP array (más una estructura de *sparse table* para consultas de mínimo en rango), permite responder "LCP entre dos sufijos arbitrarios cualesquiera" en $O(1)$ tras $O(n \log n)$ de preprocesamiento —porque el LCP entre dos sufijos no adyacentes en el suffix array es el mínimo del LCP array en el rango entre ellos. Es la herramienta más general de esta parte: resuelve *Subcadena Común Más Larga*, cuenta substrings distintos, y sirve de base para muchos problemas que no tienen un algoritmo dedicado más simple.

Casos de uso:

- Búsqueda de patrón mediante búsqueda binaria sobre el suffix array, alternativa a KMP cuando se necesita el suffix array para otras consultas de todas formas.
- Contar el número de substrings **distintos** de una cadena: $\binom{n+1}{2} - \sum_i \text{lcp}[i]$ (el total de substrings menos los que están duplicados, contados por el LCP).
- Subcadena común más larga entre dos (o más) cadenas: construir el suffix array de la concatenación con separadores únicos, y buscar el máximo LCP entre sufijos de origen distinto.
- Consulta de LCP entre dos sufijos arbitrarios en $O(1)$ (con *sparse table* sobre el LCP array), base de muchos algoritmos más avanzados de comparación de subcadenas.

Complejidad: $O(n \log n)$ para construir el suffix array con ordenamiento por duplicación (hay $O(\log n)$ rondas, cada una $O(n \log n)$ por el `sort`, aunque con *radix sort* se puede bajar a $O(n \log n)$ total); $O(n)$ para el LCP array con Kasai una vez que el suffix array existe.

```

1 // CREATE/READ: Suffix Array (ordenamiento por duplicacion) + LCP
2 //    Array
3 // (algoritmo de Kasai)
4 struct SuffixArray {
5     string s;
6     int n;
7     vector<int> sa; // sa[i] = indice de inicio del i-esimo
8                     // sufijo en orden lexicografico
9     vector<int> rango; // rango[i] = posicion del sufijo que
10                        // empieza en i, dentro de sa
11     vector<int> lcp; // lcp[i] = LCP entre sa[i-1] y sa[i] (lcp
12                        // [0] no definido)
13
14     // CREATE: construye el suffix array mediante ordenamiento
15     // por
16     // duplicacion (doubling), y luego el LCP array con Kasai
17     SuffixArray(const string& cadena) {
18         s = cadena + '\0'; // centinela, menor que cualquier
19         // caracter real
20         n = (int)s.size();
21         construirSuffixArray();
22         construirLCP();
23     } // O(n log n)
24
25     // WRITE (interno): ordenamiento por duplicacion, O(n log n)
26     // rondas
27     void construirSuffixArray() {
28         sa.resize(n);
29         rango.resize(n);
30         vector<int> rangoTemp(n);
31         for (int i = 0; i < n; i++) { sa[i] = i; rango[i] = s[i];
32         }
33     }
34 }

```

```

25     for (int k = 1; k < n; k *= 2) {
26         auto comparar = [&](int a, int b) {
27             if (rango[a] != rango[b]) return rango[a] < rango
28         [b];
29         int ra = (a + k < n) ? rango[a + k] : -1;
30         int rb = (b + k < n) ? rango[b + k] : -1;
31         return ra < rb;
32     };
33     sort(sa.begin(), sa.end(), comparar);
34
35     rangoTemp[sa[0]] = 0;
36     for (int i = 1; i < n; i++) {
37         rangoTemp[sa[i]] = rangoTemp[sa[i-1]] + (comparar
38         (sa[i-1], sa[i]) ? 1 : 0);
39     }
40     rango = rangoTemp;
41     if (rango[sa[n-1]] == n - 1) break; // ya todos los
42     rangos son distintos
43 } // O(n log^2 n) con sort estandar (aceptable en CP salvo n
44 muy grande)
45
46 // WRITE (interno): algoritmo de Kasai, calcula el LCP array
47 en O(n)
48 // reutilizando que el LCP baja a lo sumo 1 entre posiciones
49 consecutivas
50 void construirLCP() {
51     lcp.assign(n, 0);
52     int h = 0;
53     for (int i = 0; i < n; i++) {
54         if (rango[i] > 0) {
55             int j = sa[rango[i] - 1];
56             while (i + h < n && j + h < n && s[i + h] == s[j
57             + h]) h++;
58             lcp[rango[i]] = h;
59             if (h > 0) h--; // clave: nunca reiniciar desde
60             0, solo bajar 1
61         } else {
62             h = 0;
63         }
64     }
65 } // O(n)
66
67 // -----
68 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
69 // -----
70
71 // 1. Contar substrings distintos: long long total = (long
72 long)n*(n-1)/2;
73 // (usando n sin el centinela) menos sum(lcp[i]) para i=1..
74 n-1 con el
75 // centinela ya excluido del conteo.
76 // 2. LCP entre dos sufijos ARBITRARIOS i, j (no
77 necesariamente
78 // adyacentes en sa): construir una sparse table sobre lcp
79 [] para
80 // responder minimo en rango [rango[i]+1, rango[j]] en O
81 (1),
82 // ya que LCP(i,j) = min(lcp[rango[i]+1..rango[j]]) si
83 rango[i] < rango[j].
84 // 3. El centinela '\0' garantiza que ningun sufijo sea
85 prefijo de otro
86 // de forma ambigua durante la comparacion; siempre
87 incluirlo al
88 // construir sobre una cadena real.
89 };

```

Ejemplo de funcionamiento:

```

1 SuffixArray sa("banana");
2 // sa.sa contiene los indices de inicio de los sufijos de "banana
3 // \0" en
4 // orden lexicografico; el sufijo vacio (el centinela solo) va
5 // primero
6 // por ser el menor. Los sufijos reales ordenados son:
7 // "" < "a" < "ana" < "anana" < "banana" < "na" < "nana"
8 // hay que correrlo para confirmar el arreglo sa.sa exacto con el
9 // indice
10 // del centinela incluido
11
12 // sa.lcp[i] da el LCP entre sufijos consecutivos en ese orden;
13 // por
14 // ejemplo el LCP entre "ana" y "anana" es 3 ("ana" completo)

```

Nota: la versión de construcción mostrada usa `sort` con comparador dentro de cada ronda de duplicación, dando $O(n \log^2 n)$ total —suficiente para la mayoría de restricciones de competencia (n hasta $\sim 10^5$ - 10^6). Para n mayor, existe una versión $O(n \log n)$ usando *counting*

sort (radix sort) en vez de `std::sort` dentro de cada ronda, más compleja de implementar y omitida aquí por brevedad.

Regla práctica: usa Suffix Array + Kasai cuando el problema requiera múltiples consultas distintas sobre subcadenas de la misma cadena (o de varias concatenadas con separadores) que no tengan un algoritmo dedicado más simple —es la herramienta más general de esta parte, a costa de mayor complejidad de implementación que KMP, Manacher, o hashing. Si el problema se resuelve directamente con una técnica más específica (búsqueda de un patrón: KMP; palíndromos: Manacher), prefiere esa técnica por ser más simple y menos propensa a bugs.

54.5 Suffix Automaton

Idea: un *suffix automaton* es el autómata finito determinista **más pequeño** que reconoce exactamente el conjunto de todos los **sufijos** de una cadena s —equivalentemente, reconoce todos los **substrings** de s (cualquier substring es prefijo de algún sufijo). Se construye incrementalmente, un carácter a la vez (igual que el *Eertree* para palíndromos), manteniendo **estados** donde cada estado representa una clase de equivalencia de substrings que terminan en exactamente el mismo conjunto de posiciones de s (su *endpos*). Cada estado tiene un **enlace de sufijo** que apunta al estado que representa el sufijo propio más largo de esa clase con un *endpos* estrictamente mayor —la misma idea general de "aprovechar estructura ya construida vía enlaces" que aparece en π (KMP), Aho-Corasick, y el Eertree, aplicada aquí al conjunto completo de substrings distintos.

¿Por qué usarlo? A diferencia del *Suffix Array*, que da una lista ordenada de sufijos y requiere estructuras auxiliares (sparse table sobre LCP) para muchas consultas, el suffix automaton representa el conjunto de substrings distintos como un **grafo con a lo sumo $2n - 1$ estados**, construible en $O(n)$ amortizado (contra $O(n \log n)$ del suffix array), y responde ciertas preguntas —como "¿es t substring de s ?" en $O(|t|)$, o "cuántos substrings distintos tiene s " — de forma más directa, recorriendo transiciones del autómata en vez de hacer búsqueda binaria sobre un arreglo ordenado.

Casos de uso:

- Verificar si una cadena t es substring de s , en $O(|t|)$, siguiendo transiciones desde el estado inicial.
- Contar el número de substrings distintos de s : suma, sobre todos los estados (salvo la raíz), de $(\text{longitud}[v] - \text{longitud}[\text{enlace}[v]])$ —cada estado "aporta" ese rango de longitudes como substrings nuevos.
- Encontrar el substring común más largo entre dos cadenas, alimentando la segunda cadena carácter por carácter contra el autómata construido sobre la primera.
- Contar ocurrencias de cada substring distinto (pagando el tamaño del *endpos* por los enlaces de sufijo, análogo a cómo Aho-Corasick propaga coincidencias).

Complejidad: $O(n \log \sigma)$ de construcción con transiciones por map (o $O(n\sigma)$ con arreglo fijo, más rápido para alfabetos pequeños); el número de estados está acotado por $2n - 1$ para $n \geq 2$.

```

1 // CREATE/READ/WRITE: Suffix Automaton construido
  incrementalmente,
2 // reconoce todos los substrings de la cadena procesada hasta el
  momento
3 struct SuffixAutomaton {
4     static const int ALFABETO = 26;
5     struct Estado {
6         int longitud; // longitud del substring mas largo
          de esta clase
7         int enlaceSufijo; // estado del sufijo propio mas largo
          con endpos mayor
8         int transiciones[ALFABETO];
9         long long tamanoEndpos; // tamaño del conjunto endpos (
          num. de ocurrencias)
10        Estado() : longitud(0), enlaceSufijo(-1), tamanoEndpos(0)
11        {
12            fill(transiciones, transiciones + ALFABETO, -1);
13        }
14    };
15    vector<Estado> estados;
16    int ultimo; // estado que representa la cadena completa
          construida hasta ahora
17
18    // CREATE: inicializa con el estado raiz (cadena vacia)
19    SuffixAutomaton() {
20        estados.push_back(Estado());
21        estados[0].longitud = 0;
22        estados[0].enlaceSufijo = -1;
23        ultimo = 0;
24    } // O(1)
25
26    // WRITE: extiende el automaton agregando un caracter al
          final de la
27    // cadena representada; crea 1 o 2 estados nuevos segun el
          caso
28    void agregarCaracter(char ch) {
29        int c = ch - 'a';
30        int nuevo = (int)estados.size();
31        estados.push_back(Estado());
32        estados[nuevo].longitud = estados[ultimo].longitud + 1;
33        estados[nuevo].tamanoEndpos = 1; // esta nueva ocurrencia
          termina aquí
34
35        int p = ultimo;
36        while (p != -1 && estados[p].transiciones[c] == -1) {
37            estados[p].transiciones[c] = nuevo;
38            p = estados[p].enlaceSufijo;
39        }
40
41        if (p == -1) {
42            estados[nuevo].enlaceSufijo = 0; // no habia
          transicion previa: enlaza a la raiz
43        } else {
44            int q = estados[p].transiciones[c];
45            if (estados[p].longitud + 1 == estados[q].longitud) {
46                estados[nuevo].enlaceSufijo = q; // caso simple:
          q ya es la clase correcta
47            } else {
48                // clonar q: se necesita un estado intermedio
          para separar
49                // la clase de equivalencia correctamente
50                int clon = (int)estados.size();
51                estados.push_back(estados[q]); // copia
          transiciones y enlace de q
52                estados[clon].longitud = estados[p].longitud + 1;
53                estados[clon].tamanoEndpos = 0; // el clon no es
          una ocurrencia nueva
54
55                while (p != -1 && estados[p].transiciones[c] == q)
56                {
57                    estados[p].transiciones[c] = clon;
58                    p = estados[p].enlaceSufijo;
59                }
60                estados[q].enlaceSufijo = clon;
61                estados[nuevo].enlaceSufijo = clon;
62            }
63            ultimo = nuevo;
64        } // O(1) amortizado (con transiciones[ALFABETO] fijas)
65
66    // READ: verifica si t es substring de la cadena representada,
67
68    // siguiendo transiciones desde el estado inicial
69    bool esSubstring(const string& t) {
70        int estadoActual = 0;
71        for (char ch : t) {

```

```

71        int c = ch - 'a';
72        if (estados[estadoActual].transiciones[c] == -1)
73            return false;
74        estadoActual = estados[estadoActual].transiciones[c];
75    }
76    return true;
77 } // O(|t|)
78
79 // READ: cuenta el numero de substrings DISTINTOS de la
  cadena
80 // representada hasta el momento
81 long long contarSubstringsDistintos() {
82     long long total = 0;
83     for (int v = 1; v < (int)estados.size(); v++) {
84         int enlace = estados[v].enlaceSufijo;
85         total += estados[v].longitud - estados[enlace].
          longitud;
86     }
87     return total;
88 } // O(numero de estados) = O(n)
89
90 // -----
91 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
92 // -----
93
94 // 1. Propagar tamanoEndpos (numero de ocurrencias de cada
  substring):
95 // ordenar los estados por longitud DESCENDENTE (o con
  counting
96 // sort por longitud, mas eficiente) y sumar
97 // estados[enlace[v]].tamanoEndpos += estados[v].
  tamanoEndpos para
98 // cada v en ese orden; los estados clon empiezan en 0 y
  acumulan
99 // correctamente de sus hijos en el arbol de enlaces de
  sufijo.
100 // 2. Subcadena comun mas larga entre s y t: construir el
  automaton
101 // sobre s, luego recorrer t manteniendo (estadoActual,
  longitudActual);
102 // si hay transicion, avanzar y longitudActual++; si no,
  seguir
103 // enlaces de sufijo reduciendo longitudActual hasta
  encontrar una
104 // transicion valida o llegar a la raiz. El maximo de
  longitudActual
105 // visto es la respuesta.
106 // 3. Memoria: cada estado reserva ALFABETO enteros para
  transiciones;
107 // con alfabetos grandes, usar unordered_map<char,int> en
  su lugar,
  // a costa de un factor log(sigma) en la complejidad.

```

Ejemplo de funcionamiento:

```

1 SuffixAutomaton sam;
2 for (char c : string("abcbc")) sam.agregarCaracter(c);
3
4 bool existe = sam.esSubstring("cbc");
5 // existe = true ("cbc" es substring de "abcbc")
6
7 long long distintos = sam.contarSubstringsDistintos();
8 // distintos cuenta todos los substrings unicos de "abcbc" (con
9 // repeticiones de contenido contadas una sola vez); hay que
  correrlo
10 // para confirmar el valor exacto

```

Nota: el paso de **clonación** dentro de `agregarCaracter` es la parte conceptualmente más difícil del suffix automaton —ocurre quíntos la transición encontrada q no puede pertenecer íntegramente a la nueva clase de equivalencia (su longitud no calza exactamente), así que hay que "dividir" esa clase en dos: el clon hereda las transiciones y el enlace de q , y ambos (q y el nuevo estado) terminan enlazando al clon. Es el análogo, en este contexto, a la creación de nodos nuevos en el Eertree cuando aparece un palíndromo distinto.

Regla práctica: usa el suffix automaton cuando necesitas construir la estructura incrementalmente (streaming, sin poder reprocesar desde cero) o cuando la pregunta sea específicamente sobre **substrings distintos** y sus conteos —es más directo que Suffix Array + LCP para esas preguntas puntuales. Para consultas que de-

penden del **orden lexicográfico** de los sufijos (como encontrar el k -ésimo sufijo menor), *Suffix Array* sigue siendo la herramienta más natural, ya que el suffix automaton no mantiene ningún orden lexicográfico explícito entre sus estados.

55 Búsqueda de Patrones

55.1 Knuth-Morris-Pratt (KMP)

Idea: KMP busca todas las ocurrencias de un patrón p dentro de un texto t reutilizando directamente *Función de Prefijos*: se construye la cadena concatenada $p\#t$ (con un separador $\#$ que no aparece en ninguna de las dos cadenas), y se calcula su función de prefijos completa. Cada posición i dentro de la parte correspondiente a t donde $\pi[i]$ alcanza exactamente $|p|$ marca el final de una ocurrencia completa del patrón —porque eso significa que los últimos $|p|$ caracteres vistos coinciden exactamente con el patrón entero. El separador garantiza que π nunca pueda "cruzar" de t de vuelta a p y contar coincidencias falsas.

¿Por qué usarlo? Buscar un patrón con fuerza bruta (probar cada posición de inicio en t y comparar carácter por carácter) es $O(nm)$ en el peor caso —por ejemplo, texto "aaaa...a" y patrón "aaa...ab". KMP garantiza $O(n+m)$ porque, como se explicó en *Función de Prefijos*, el puntero sobre el texto nunca retrocede: cuando hay un desajuste, se usa la información ya calculada en π para saltar directamente al siguiente candidato viable, sin volver a comparar caracteres del texto ya vistos.

Casos de uso:

- Encontrar todas las posiciones donde un patrón p ocurre dentro de un texto t .
- Contar el número total de ocurrencias de p en t .
- Verificar si p es substring de t (caso particular: basta con la primera ocurrencia).
- Como paso de preprocesamiento en problemas que combinan búsqueda de patrón con otra estructura (por ejemplo, contar ocurrencias en un rango de un texto que cambia).

Complejidad: $O(n+m)$, donde $n = |t|$ y $m = |p|$ —dominado por el cálculo de la función de prefijos sobre la cadena concatenada de longitud $n+m+1$.

```

1 // CREATE/READ: Búsqueda de todas las ocurrencias de un patron en
  un texto
2 // mediante KMP (funcion de prefijos sobre p + separador + t)
3 struct KMP {
4     // READ: funcion de prefijos, identica a la de Funcion de
      Prefijos,
5     // incluida aqui para que el struct sea autocontenido
6     static vector<int> funcionPrefijos(const string& s) {
7         int n = (int)s.size();
8         vector<int> pi(n, 0);
9         for (int i = 1; i < n; i++) {
10             int j = pi[i - 1];
11             while (j > 0 && s[i] != s[j]) j = pi[j - 1];
12             if (s[i] == s[j]) j++;
13             pi[i] = j;
14         }
15         return pi;
16     } // O(n)
17
18     // READ: retorna las posiciones (0-indexadas, en t) donde
      comienza cada
19     // ocurrencia del patron p dentro del texto t

```

```

20 static vector<int> buscar(const string& patron, const string&
  texto) {
21     int m = (int)patron.size();
22     if (m == 0) return {};
23     string combinada = patron + '#' + texto;
24     vector<int> pi = funcionPrefijos(combinada);
25     vector<int> ocurrencias;
26     for (int i = m + 1; i < (int)combinada.size(); i++) {
27         if (pi[i] == m) {
28             // i es indice en combinada; la ocurrencia
      empieza en
29             // texto[i - m - m - 1 + 1]... se resta el offset
      del prefijo+separador
30             int inicioEnTexto = i - 2 * m;
31             ocurrencias.push_back(inicioEnTexto);
32         }
33     }
34     return ocurrencias;
35 } // O(n + m)
36
37 //
38 // -----
39 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
40 // -----
41 // 1. Version con menos memoria (sin concatenar cadenas): se
  puede
42 //     correr KMP "en línea" manteniendo solo pi del patron
  precomputado
43 //     y un puntero j que avanza sobre t sin construir la
  cadena
44 //     combinada completa; usa O(m) memoria en vez de O(n+m),
  preferible
45 //     si t es muy grande o llega en streaming.
46 // 2. Contar ocurrencias SOLAPADAS: buscar() ya las cuenta
  todas
47 //     (incluyendo solapadas), a diferencia de algunos metodos
  ingenuos
48 //     que saltan m posiciones tras cada match y pierden
  ocurrencias
49 //     solapadas.
50 // 3. Multiples patrones contra el mismo texto: si son pocos
  patrones,
51 //     repetir buscar() para cada uno; si son muchos, usar Aho-
  Corasick
52 //     en su lugar, que amortiza el costo sobre todos los
  patrones a la vez.
53 };

```

Ejemplo de funcionamiento:

```

1 auto resultado = KMP::buscar("ab", "ababab");
2 // resultado = {0, 2, 4}
3 // (el patron "ab" ocurre en las posiciones 0, 2 y 4 del texto "
  ababab",
4 // todas detectadas correctamente incluyendo las que se tocan
  entre si)

```

Nota: el índice de inicio de cada ocurrencia en `buscar()` se calcula como $i - 2*m$, donde $m = |patron|$: la posición i en la cadena combinada incluye el patrón (m caracteres) más el separador (1 carácter), así que el offset hacia el texto original es $i - m - (m + 1) + 1 = i - 2m$. Verifica este cálculo cuidadosamente si modificas el struct, ya que es la fuente más común de error de índice en implementaciones de KMP concatenado.

Regla práctica: usa KMP cuando necesites **todas las ocurrencias exactas** de un único patrón en un texto, con complejidad garantizada $O(n+m)$ sin riesgo de colisión (a diferencia de *Hash Polinómico*, que es probabilístico). Si necesitas buscar **muchos patrones** simultáneamente contra el mismo texto, usa *Aho-Corasick* en su lugar, que evita repetir el costo de KMP una vez por patrón.

55.2 Algoritmo Z

Idea: el *Z-array* de una cadena s de longitud n da, para cada posición $i > 0$, la longitud del prefijo común más largo entre s completa y el sufijo de s que empieza en i —es decir, $Z[i]$ mide cuánto se "parece" el sufijo en i al inicio de la cadena. Se calcula manteniendo una ventana

$[l, r]$ que representa el segmento coincidente con el prefijo más a la derecha encontrado hasta el momento: para cada nueva posición i , si i cae dentro de esa ventana, se aprovecha el valor Z ya conocido de la posición simétrica dentro del prefijo como punto de partida (en vez de comparar desde cero), y solo se extiende comparando carácter por carácter más allá de r .

¿Por qué usarlo? Resuelve el mismo problema que KMP (búsqueda de patrón, con la misma complejidad $O(n + m)$) pero con una interpretación distinta: en vez de "prefijo que también es sufijo" (función de prefijos), es directamente "cuánto coincide cada sufijo con el prefijo completo". Muchos lo encuentran más intuitivo de razonar que la función de prefijos, y la técnica de la ventana $[l, r]$ que se extiende sin retroceder es el mismo patrón general de aprovechar trabajo ya hecho que aparece en *Manacher* para palíndromos.

Casos de uso:

- Búsqueda de patrón (alternativa a KMP): concatenar $p \# t$ y buscar posiciones donde $Z[i] = |p|$.
- Encontrar todas las posiciones donde un prefijo de la cadena se repite como substring.
- Calcular el período de una cadena de forma alternativa a la función de prefijos.
- Comprimir cadenas detectando repeticiones del prefijo (base de algunos esquemas de compresión simples).

Complejidad: $O(n)$ —análogo al argumento de *Función de Prefijos*: la ventana $[l, r]$ solo se extiende hacia la derecha a lo largo de toda la ejecución, así que el trabajo total de extensión está acotado por n .

```

1 // CREATE/READ: Z-array de una cadena: para cada i, la longitud
  del
2 // prefijo comun mas largo entre s y el sufijo de s que empieza
  en i
3 struct AlgoritmoZ {
4     // READ: calcula el Z-array completo de s (Z[0] se define
      como 0 por
5     // convencion, ya que comparar el prefijo consigo mismo no
      aporta info util)
6     static vector<int> calcular(const string& s) {
7         int n = (int)s.size();
8         vector<int> z(n, 0);
9         int l = 0, r = 0; // ventana [l,r] del prefijo
            coincidente mas a la derecha
10        for (int i = 1; i < n; i++) {
11            if (i < r) {
12                // aprovecha el valor ya conocido de la posicion
                simetrica
13                z[i] = min(r - i, z[i - 1]);
14            }
15            // intenta extender mas alla de lo ya garantizado
16            while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]
17            ]++;
18            if (i + z[i] > r) { l = i; r = i + z[i]; }
19        }
20        return z;
21    } // O(n)
22
23    // READ: busca todas las ocurrencias del patron p en el texto
      t,
24    // usando el Z-array de la concatenacion p + '#' + t
25    static vector<int> buscarPatron(const string& patron, const
      string& texto) {
26        int m = (int)patron.size();
27        if (m == 0) return {};
28        string combinada = patron + '#' + texto;
29        vector<int> z = calcular(combinada);
30        vector<int> ocurrencias;
31        for (int i = m + 1; i < (int)combinada.size(); i++) {
32            if (z[i] == m) ocurrencias.push_back(i - m - 1);
33        }
34        return ocurrencias;

```

```

34 } // O(n + m)
35
36 //
37 -----
38 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
39 //
40 -----
41 // 1. Equivalencia con KMP: ambos resuelven busqueda de
      patron en
42 // O(n+m) con la misma tecnica de no retroceder el puntero;
      la
43 // eleccion entre uno u otro es principalmente preferencia
      personal
44 // o cual se recuerda implementar sin errores bajo presion.
45
46 // 2. Periodo de una cadena via Z-array: el periodo mas corto
      es n - k,
47 // donde k es el menor indice tal que k + z[k] == n (el
      sufijo desde
48 // k coincide con el prefijo hasta el final de la cadena).
49 // 3. Contar cuantas posiciones tienen z[i] >= L para un L
      dado: util en
50 // problemas de busqueda binaria sobre la longitud de un
      prefijo
51 // repetido; se puede precomputar con un arreglo de
      frecuencias y
52 // sumas de sufijo sobre los valores de z.
53 };

```

Ejemplo de funcionamiento:

```

1 vector<int> z = AlgoritmoZ::calcular("aabxaayaab");
2 // z = {0, 1, 0, 0, 3, 1, 0, 2, 1, 0}
3 // (por ejemplo z[4]=3: el sufijo desde la posicion 4, "aayaab",
      comparte
4 // un prefijo de longitud 3 con la cadena completa "aabxaayaab"
      -> "aay")
5 // vs "aab"... hay que correrlo para confirmar el valor exacto
      de cada
6 // posicion del arreglo)

```

Regla práctica: usa el Algoritmo Z como alternativa completamente equivalente a KMP para búsqueda de patrón —la elección entre ambos no afecta la complejidad, así que usa el que te resulte más fácil de implementar sin bugs. La ventaja distintiva del Z-array aparece cuando el problema pregunta directamente sobre "cuánto coincide cada sufijo con el prefijo", una pregunta que con la función de prefijos de KMP requeriría un paso de traducción adicional.

55.3 Rabin-Karp

Idea: Rabin-Karp busca un patrón p dentro de un texto t comparando **hashes** en vez de caracteres: se calcula el hash de p una sola vez, y luego se desliza una ventana de longitud $|p|$ sobre t , recalculando el hash de cada ventana en $O(1)$ a partir del hash de la ventana anterior —el mismo mecanismo de *Hash Polinómico* con hashes de prefijo precomputados. Cuando el hash de la ventana coincide con el hash del patrón, se tiene una coincidencia **candidata**; como el hash es probabilístico, conviene verificar carácter por carácter esa posición para confirmar que no es una colisión falsa.

¿Por qué usarlo? Es conceptualmente el algoritmo de búsqueda de patrón más simple de razonar e implementar de los tres vistos (KMP, Z, Rabin-Karp), porque reutiliza directamente *Hash Polinómico* sin necesidad de ninguna estructura de fallos ni ventana especial. Su desventaja es que, a diferencia de KMP y Z (que son $O(n + m)$ garantizado sin importar el caso), Rabin-Karp puede degradar a $O(nm)$ en el peor caso si hay muchas colisiones de hash falsas que obligan a verificar carácter por carácter repetidamente —algo mitigado casi por completo usando hashing doble.

Casos de uso:

- Búsqueda de un patrón en un texto cuando se prioriza simplicidad de implementación sobre garantía de peor caso.
- Búsqueda de **múltiples patrones del mismo tamaño** simultáneamente, comparando el hash de la ventana contra un **set** de hashes de los patrones (más simple que adaptar KMP o Z a múltiples patrones de igual longitud, aunque Aho-Corasick es la herramienta general para múltiples patrones de cualquier tamaño).
- Detección de plagio o duplicados aproximados comparando hashes de ventanas deslizantes.
- Como introducción pedagógica a la idea de hashing de subcadenas antes de estructuras más elaboradas.

Complejidad: $O(n+m)$ esperado (con hashing doble, la probabilidad de colisión es despreciable en la práctica); $O(nm)$ en el peor caso teórico si un adversario conoce el módulo usado y fuerza colisiones repetidas.

```

1 // CREATE/READ: Búsqueda de patron mediante Rabin-Karp: hash de
  ventana
2 // deslizante comparado contra el hash del patron
3 struct RabinKarp {
4     const long long P = 131;
5     const long long MOD = 1000000007LL;
6
7     // READ: calcula  $P^{\text{exp}} \bmod \text{MOD}$  (exponenciación rápida)
8     static long long potenciaMod(long long base, long long exp,
9     long long mod) {
10         long long resultado = 1;
11         base %= mod;
12         while (exp > 0) {
13             if (exp & 1) resultado = (resultado * base) % mod;
14             base = (base * base) % mod;
15             exp >>= 1;
16         }
17         return resultado;
18     } //  $O(\log \text{exp})$ 
19
20     // READ: retorna las posiciones (0-indexadas) donde comienza
21     // cada
22     // ocurrencia EXACTA del patron en el texto (verifica cada
23     // candidato
24     // caracter por caracter para descartar colisiones de hash)
25     vector<int> buscar(const string& patron, const string& texto)
26     {
27         int m = (int)patron.size(), n = (int)texto.size();
28         vector<int> ocurrencias;
29         if (m == 0 || m > n) return ocurrencias;
30
31         long long hashPatron = 0;
32         for (char c : patron) hashPatron = (hashPatron * P + (c -
33         'a' + 1)) % MOD;
34
35         long long hashVentana = 0;
36         long long potenciaMayor = potenciaMod(P, m - 1, MOD);
37         for (int i = 0; i < n; i++) {
38             hashVentana = (hashVentana * P + (texto[i] - 'a' + 1)
39             ) % MOD;
40             if (i >= m) {
41                 // remueve el caracter que sale por la izquierda
42                 // de la ventana
43                 long long saliente = (texto[i - m] - 'a' + 1) *
44                 potenciaMayor % MOD;
45                 hashVentana = (hashVentana - saliente % MOD + MOD
46                 ) % MOD;
47                 hashVentana = (hashVentana - saliente * P % MOD +
48                 MOD * MOD) % MOD; // ver Nota
49             }
50             if (i >= m - 1 && hashVentana == hashPatron) {
51                 int inicio = i - m + 1;
52                 if (texto.compare(inicio, m, patron) == 0) { //
53                     descarta colision falsa
54                     ocurrencias.push_back(inicio);
55                 }
56             }
57         }
58         return ocurrencias;
59     } //  $O(n + m)$  esperado
60 }

```

```

51 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
52 // -----
53 // 1. Hashing doble: repetir todo el calculo con un segundo
54 // par (P2,MOD2)
55 // en paralelo, y solo considerar candidato si AMBOS
56 // hashes coinciden;
57 // reduce la necesidad de verificar con compare() en la
58 // mayoría de
59 // los casos, aunque compare() como respaldo sigue siendo
60 // la forma
61 // mas segura de garantizar cero falsos positivos.
62 // 2. Múltiples patrones del mismo tamaño: precomputar un
63 // unordered_set<long long> con los hashes de todos los
64 // patrones, y
65 // comparar el hash de cada ventana contra ese set en  $O(1)$ 
66 // en vez de
67 // contra un solo hashPatron.
68 // 3. Ventana deslizante SIN necesidad de patron previo (hash
69 // rodante
70 // generico): la logica de actualizar hashVentana quitando
71 // el
72 // caracter saliente y agregando el entrante es
73 // reutilizable para
74 // cualquier problema que necesite el hash de una ventana
75 // de tamaño
76 // fijo deslizando sobre una cadena.
77 };

```

Ejemplo de funcionamiento:

```

1 RabinKarp rk;
2 auto resultado = rk.buscar("ab", "ababab");
3 // resultado = {0, 2, 4}
4 // (mismo resultado que KMP y Algoritmo Z para el mismo patron y
5 // texto;
6 // cada candidato con hash coincidente se verifico con compare()
7 // antes
8 // de agregarse, descartando cualquier colision falsa)

```

Nota: la línea marcada arriba (remoción del carácter saliente multiplicado por P para compensar el corrimiento de todos los términos restantes) es la parte más propensa a errores de índice en Rabin-Karp con ventana deslizante —a diferencia de *Hash Polinómico* con prefijos precomputados (donde la resta es directa), aquí se recalcula incrementalmente y conviene verificarla con cuidado o preferir la versión de hash de prefijos si hay dudas.

Regla práctica: usa Rabin-Karp cuando priorices simplicidad de implementación y el problema no sea adversarial contra hashing (o uses hashing doble para mitigarlo); si necesitas la garantía **determinística** de $O(n + m)$ sin ninguna dependencia de probabilidad, usa *KMP* o *Algoritmo Z* en su lugar —ambos resuelven exactamente el mismo problema sin el riesgo teórico de colisión.

55.4 Boyer-Moore

Idea: a diferencia de KMP, Z y Rabin-Karp —que procesan el texto de izquierda a derecha—, Boyer-Moore compara el patrón contra el texto **de derecha a izquierda** en cada posición candidata, y cuando encuentra un desajuste, usa dos heurísticas para decidir cuánto **saltar** el patrón hacia adelante sin perder ninguna coincidencia posible: la *regla del mal carácter* (mover el patrón para alinear el carácter del texto que causó el desajuste con su ocurrencia más a la derecha dentro del patrón, si existe) y la *regla del buen sufijo* (aprovechar que el sufijo ya coincidente del patrón puede reaparecer en otra posición del propio patrón). El desplazamiento efectivo usado es el máximo de lo que sugieren ambas reglas.

¿Por qué usarlo? En el caso promedio, especialmente con alfabetos grandes (como texto en inglés, 26+ letras), Boyer-Moore es **sublineal**: no necesita examinar todos los caracteres del texto, porque los saltos grandes

de la regla del mal carácter permiten "saltarse" bloques enteros sin compararlos. Esto lo hace el algoritmo más rápido en la práctica para búsqueda de texto en editores y herramientas como `grep`, aunque su peor caso teórico ($O(nm)$ con la regla del mal carácter sola, mejorable a $O(n + m)$ combinando ambas reglas correctamente) no es tan simple de garantizar como KMP.

Casos de uso:

- Búsqueda de texto en editores, herramientas de línea de comandos, y motores de búsqueda de texto donde el rendimiento práctico importa más que la garantía teórica de peor caso.
- Alfabetos grandes (texto natural) donde la regla del mal carácter produce saltos grandes frecuentemente.
- Como comparación de referencia al estudiar por qué distintos algoritmos de matching tienen ventajas en distintos escenarios (útil para justificar la elección en un informe o discusión).
- En CP específicamente, es menos común que KMP/Z/Hash porque su implementación completa (con ambas reglas) es más larga; se usa más la versión simplificada solo con la regla del mal carácter cuando el alfabeto es grande.

Complejidad: $O(n/m)$ en el mejor caso (saltos grandes constantes), $O(n + m)$ en el caso promedio con ambas heurísticas combinadas correctamente, y $O(nm)$ en el peor caso si solo se usa la regla del mal carácter sin la regla del buen sufijo como respaldo.

```
1 // CREATE/READ: Búsqueda de patron mediante Boyer-Moore con regla
  // del mal
2 // caracter (version simplificada, alfabeto minusculas 'a'-'z')
3 struct BoyerMoore {
4     static const int ALFABETO = 26;
5
6     // READ: para cada caracter del alfabeto, la posicion MAS A
      // LA DERECHA
7     // en la que aparece dentro del patron (-1 si no aparece)
8     static vector<int> tablaUltimaOcurrencia(const string& patron
9     ) {
10         vector<int> ultima(ALFABETO, -1);
11         for (int i = 0; i < (int)patron.size(); i++) {
12             ultima[patron[i] - 'a'] = i;
13         }
14         return ultima;
15     } // O(m)
16
17     // READ: retorna las posiciones (0-indexadas) donde comienza
      // cada
18     // ocurrencia del patron en el texto, usando solo la regla
      // del mal
19     // caracter (simplificacion valida y suficiente para la
      // mayoria de
20     // problemas de CP)
21     static vector<int> buscar(const string& patron, const string&
22     texto) {
23         int m = (int)patron.size(), n = (int)texto.size();
24         vector<int> ocurrencias;
25         if (m == 0 || m > n) return ocurrencias;
26
27         vector<int> ultima = tablaUltimaOcurrencia(patron);
28         int s = 0; // posicion de alineacion actual del patron
29         // sobre el texto
30         while (s <= n - m) {
31             int j = m - 1;
32             // compara de derecha a izquierda
33             while (j >= 0 && patron[j] == texto[s + j]) j--;
34             if (j < 0) {
35                 ocurrencias.push_back(s);
36                 s++; // desplazamiento minimo tras un match
37             }
38         }
39         // completo
40     } else {
41         int caracterMalo = texto[s + j] - 'a';
42         int salto = j - ultima[caracterMalo];
43         s += max(1, salto); // nunca retroceder, salto
44         // minimo de 1
45     }
46 }
```

```
38 }
39 }
40 return ocurrencias;
41 } // O(n/m) mejor caso, O(nm) peor caso (con solo regla del
  // mal caracter)
42
43 //
44 // -----
45 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
46 // -----
47 // 1. Regla del buen sufijo (NO implementada arriba):
  // garantiza el peor
48 // caso O(n+m) combinada con la regla del mal caracter;
  // requiere una
49 // tabla adicional de desplazamientos basada en donde
  // reaparece cada
50 // sufijo del patron dentro de si mismo (similar en
  // espiritu a la
51 // funcion de prefijos, pero mas compleja de construir);
  // se omite
52 // aqui porque en la practica competitiva, KMP ya da la
  // garantia
53 // O(n+m) de forma mas simple si el peor caso es una
  // preocupacion real.
54 // 2. Alfabeto grande o Unicode: la tabla ultima como arreglo
  // fijo de
55 // tamaño ALFABETO no escala; usar unordered_map<char,int>
  // en su
56 // lugar.
57 // 3. Cuando NO usar Boyer-Moore en CP: si el patron o el
  // texto son
58 // cortos, o el alfabeto es pequeño (binario, ADN de 4
  // letras), los
59 // saltos de la regla del mal caracter son pequeños y la
  // ventaja
60 // practica desaparece -- en esos casos KMP es igual de
  // rapido y mas
61 // simple de implementar sin bugs.
62 };
```

Ejemplo de funcionamiento:

```
1 auto resultado = BoyerMoore::buscar("ab", "ababab");
2 // resultado = {0, 2, 4}
3 // (mismo resultado que los algoritmos anteriores; con un
  // alfabeto de
4 // solo 2 caracteres distintos la ventaja de salto grande de
  // Boyer-Moore
5 // es minima, pero el resultado es igualmente correcto)
```

Nota: la implementación de arriba usa **únicamente** la regla del mal carácter, que es la versión más simple y la que se ve con más frecuencia en código de competencia —pero sin la regla del buen sufijo, el peor caso teórico es $O(nm)$, igual de malo que fuerza bruta. En CP, si existe cualquier riesgo de que el caso de prueba sea adversarial, es más seguro usar *KMP* o el *Algoritmo Z*, que garantizan $O(n + m)$ sin depender de heurísticas.

Regla práctica: en competencia, prefiere *KMP* o *Algoritmo Z* sobre Boyer-Moore por su garantía de peor caso simple de razonar; considera Boyer-Moore específicamente cuando el alfabeto es grande y estás implementando algo más cercano a una herramienta de búsqueda de texto real (no bajo las restricciones adversariales típicas de un problema de concurso), donde su velocidad práctica sublineal sí es una ventaja genuina.

55.5 Autómata Finito de Coincidencia de Patrones

Idea: un autómata finito de coincidencia precomputa, para cada estado (representando "cuánto del patrón llevo coincidido hasta ahora", igual que en *Función de Prefijos*) y cada carácter posible del alfabeto, exactamente a qué estado se transiciona —a diferencia de KMP, donde ese "a qué estado ir" se calcula on-the-fly siguiendo la cadena de fallos π durante la búsqueda. La tabla de transiciones $\delta[\text{estado}][\text{carácter}]$ se construye una sola vez a partir de

la función de prefijos del patrón, y una vez construida, procesar el texto es un simple recorrido de tabla: un acceso $O(1)$ por carácter, sin ningún retroceso ni bucle interno.

¿Por qué usarlo? KMP y el autómata resuelven el mismo problema con la misma complejidad total, pero difieren en **dónde** pagan el costo: KMP hace el trabajo de "retroceder por fallos" durante la búsqueda (amortizado $O(n)$, pero con un bucle interno variable), mientras que el autómata paga todo ese costo por adelantado en la construcción de la tabla, y la búsqueda queda reducida a $O(n)$ estrictos sin ningún bucle interno ni siquiera amortizado —una transición por carácter, garantizado. Esto importa cuando el mismo patrón se va a buscar en **muchos textos distintos**: la tabla se construye una vez y se reutiliza, evitando recalcular la cadena de fallos en cada búsqueda.

Casos de uso:

- Buscar el mismo patrón repetidamente contra muchos textos distintos, amortizando el costo de construcción.
- Streaming: procesar un texto que llega carácter por carácter (sin poder verlo completo de antemano) con tiempo de respuesta $O(1)$ por carácter.
- Como modelo conceptual para entender *Aho-Corasick*, que es exactamente esta misma idea generalizada a múltiples patrones simultáneos mediante un trie.
- Problemas donde se necesita garantía de tiempo constante por carácter, sin ninguna variabilidad de bucles internos.

Complejidad: $O(m\sigma)$ de construcción, donde $m = |\text{patrón}|$ y σ es el tamaño del alfabeto (se llena una tabla de $m + 1$ estados por σ caracteres); $O(n)$ estricto para procesar un texto de longitud n , un acceso a tabla por carácter.

```

1 // CREATE/READ: Automata finito de coincidencia de patrones,
  construido a
2 // partir de la funcion de prefijos del patron (alfabeto 'a'-'z')
3 struct AutomataPatron {
4     static const int ALFABETO = 26;
5     int m;
6     vector<vector<int>> delta; // delta[estado][caracter] =
  siguiente estado
7
8     // CREATE: construye la tabla de transiciones completa a
  partir del
9     // patron, reutilizando la idea de Funcion de Prefijos para
  los
10    // estados que no coinciden directamente con el siguiente
  caracter
11    AutomataPatron(const string& patron) {
12        m = (int)patron.size();
13        delta.assign(m + 1, vector<int>(ALFABETO, 0));
14
15        vector<int> pi(m, 0);
16        for (int i = 1; i < m; i++) {
17            int j = pi[i - 1];
18            while (j > 0 && patron[i] != patron[j]) j = pi[j -
19            1];
20            if (patron[i] == patron[j]) j++;
21            pi[i] = j;
22        }
23
24        // estado 0: solo el caracter que calza con patron[0]
  avanza a 1
25        delta[0][patron[0] - 'a'] = 1;
26        for (int estado = 1; estado <= m; estado++) {
27            for (int c = 0; c < ALFABETO; c++) {
28                char caracter = 'a' + c;

```

```

28         if (estado < m && caracter == patron[estado]) {
29             // el caracter esperado avanza al siguiente
30             estado
31             delta[estado][c] = estado + 1;
32         } else {
33             // reutiliza la transicion ya calculada del
34             estado de
35             // fallo (pi[estado-1]), evitando recalcular
36             desde cero
37             delta[estado][c] = delta[pi[estado - 1]][c];
38         }
39     }
40 } // O(m * sigma)
41
42 // READ: retorna las posiciones (0-indexadas) donde termina
  cada
43 // ocurrencia completa del patron al procesar el texto
44 vector<int> buscar(const string& texto) {
45     vector<int> ocurrencias;
46     int estado = 0;
47     for (int i = 0; i < (int)texto.size(); i++) {
48         estado = delta[estado][texto[i] - 'a'];
49         if (estado == m) ocurrencias.push_back(i - m + 1);
50     }
51     return ocurrencias;
52 } // O(n), estricto, sin bucle interno
53
54 // -----
55 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
56 // -----
57
58 // 1. delta[pi[estado-1]][c] en la construccion: este es el
  mismo tipo
59 // de "reutilizar trabajo ya calculado" que el retroceso
  por fallos
60 // en KMP, solo que aqui se hace UNA vez durante la
  construccion en
61 // vez de repetidamente durante cada busqueda.
62 // 2. Memoria: la tabla ocupa O(m * sigma), que puede ser
  prohibitivo si
63 // el alfabeto es grande (Unicode) y m tambien lo es; en
  ese caso
64 // KMP puro (sin tabla) es preferible por memoria O(m) en
  vez de
65 // O(m*sigma).
66 // 3. Múltiples busquedas del MISMO patron: construir el
  automata una
67 // sola vez (fuera de cualquier bucle sobre los textos) y
  llamar
68 // buscar() repetidamente reutilizando la misma tabla
  delta.
69 };

```

Ejemplo de funcionamiento:

```

1 AutomataPatron automata("ab");
2 auto resultado = automata.buscar("ababab");
3 // resultado = {1, 3, 5}
4 // (índices donde TERMINA cada ocurrencia de "ab"; equivalen a
  las
5 // posiciones de inicio {0,2,4} encontradas por KMP, Z y Rabin-
  Karp,
6 // desplazadas por m-1 ya que aqui se reporta el final de la
  coincidencia)

```

Nota: a diferencia de KMP, Z y Rabin-Karp (que reportan la posición de **inicio** de cada ocurrencia), esta implementación reporta la posición donde **termina** cada coincidencia, porque el estado m se alcanza al procesar el último carácter del patrón. Ajustar $i - m + 1$ en vez de i en el `push_back` convierte el resultado a posición de inicio si se prefiere esa convención, como se muestra en el código.

Regla práctica: usa el autómata en vez de KMP directo cuando el mismo patrón se busque repetidamente contra múltiples textos, o cuando necesites la garantía de $O(1)$ estricto por carácter sin ninguna variabilidad de bucle interno (por ejemplo, procesamiento en streaming). Para una única búsqueda de un patrón contra un único texto, KMP es preferible —evita pagar el costo $O(m\sigma)$ de construir la tabla completa cuando esa tabla solo se va a usar una vez.

55.6 Aho-Corasick (Múltiples Patrones)

Idea: Aho-Corasick generaliza el *Autómata Finito de Coincidencia de Patrones* de un único patrón a **muchos patrones simultáneos**: primero se insertan todos los patrones en un *Trie*, y luego se calcula, para cada nodo del trie, un **enlace de fallo** (*failure link*) que apunta al nodo que representa el sufijo propio más largo del prefijo actual que también es un prefijo de algún patrón —el mismo concepto que π en la función de prefijos, generalizado de una cadena lineal a un árbol. Al procesar el texto carácter por carácter, cuando no existe una transición directa en el trie, se sigue el enlace de fallo (análogo a retroceder con π en KMP), y cada nodo visitado que marca el fin de uno o más patrones reporta esas coincidencias.

¿Por qué usarlo? Buscar k patrones distintos corriendo KMP (o el autómata) una vez por patrón cuesta $O(k \cdot n)$ en total. Aho-Corasick procesa el texto **una sola vez**, con todos los patrones cargados simultáneamente en la estructura, dando $O(n + \sum |p_i| + \text{ocurrencias})$ total —independiente de k en la parte de recorrido del texto. Es la generalización natural de KMP a múltiples patrones, de la misma forma que el trie generaliza la búsqueda de una sola cadena a un conjunto de cadenas.

Casos de uso:

- Buscar todas las ocurrencias de un conjunto grande de palabras (diccionario) dentro de un texto, en una sola pasada.
- Filtros de contenido (detectar palabras prohibidas de una lista extensa) en tiempo real sobre texto entrante.
- Bioinformática: buscar múltiples secuencias/motivos genéticos simultáneamente dentro de una cadena de ADN.
- Como paso de preprocesamiento en problemas de DP sobre autómata (contar cadenas de longitud n que evitan cierto conjunto de patrones prohibidos).

Complejidad: $O(\sum |p_i| \cdot \sigma)$ de construcción (insertar todos los patrones en el trie más calcular los enlaces de fallo con BFS), donde σ es el tamaño del alfabeto; $O(n + \text{ocurrencias})$ para procesar un texto de longitud n y reportar todas las coincidencias encontradas.

```

1 // CREATE/READ: Aho-Corasick para buscar multiples patrones
  simultaneamente
2 // en un texto, mediante trie + enlaces de fallo (alfabeto 'a'-'z'
  ')
3 struct AhoCorasick {
4     static const int ALFABETO = 26;
5     struct Nodo {
6         int hijos[ALFABETO];
7         int fallo; // enlace de fallo (failure
8         link)
9         vector<int> indicesPatron; // indices de patrones que
10        terminan aqui
11        Nodo() : fallo(0) {
12            fill(hijos, hijos + ALFABETO, -1);
13        }
14    };
15    vector<Nodo> nodos;
16
17    // CREATE: inicializa con la raiz del trie
18    AhoCorasick() {
19        nodos.push_back(Nodo());
20    } // O(1)

```

```

21 // WRITE: inserta un patron en el trie, registrando su indice
  en el
22 // nodo donde termina (para saber cual patron coincidio al
  buscar)
23 void insertar(const string& patron, int indice) {
24     int actual = 0;
25     for (char c : patron) {
26         int idx = c - 'a';
27         if (nodos[actual].hijos[idx] == -1) {
28             nodos[actual].hijos[idx] = (int)nodos.size();
29             nodos.push_back(Nodo());
30         }
31         actual = nodos[actual].hijos[idx];
32     }
33     nodos[actual].indicesPatron.push_back(indice);
34 } // O(|patron|)
35
36 // WRITE: calcula los enlaces de fallo de todos los nodos
  mediante BFS,
37 // y completa las transiciones faltantes para que delta quede
  total
38 // (cada nodo tiene transicion valida para TODO caracter del
  alfabeto)
39 void construirFallos() {
40     queue<int> cola;
41     for (int c = 0; c < ALFABETO; c++) {
42         if (nodos[0].hijos[c] == -1) {
43             nodos[0].hijos[c] = 0; // sin match, se queda en
44             la raiz
45         } else {
46             nodos[nodos[0].hijos[c]].fallo = 0;
47             cola.push(nodos[0].hijos[c]);
48         }
49     }
50     while (!cola.empty()) {
51         int actual = cola.front(); cola.pop();
52         // hereda las ocurrencias del nodo de fallo (un
53         patron corto
54         // que es sufijo de uno mas largo tambien cuenta como
55         match)
56         for (int idx : nodos[nodos[actual]].fallo).
57         indicesPatron) {
58             nodos[actual].indicesPatron.push_back(idx);
59         }
60         for (int c = 0; c < ALFABETO; c++) {
61             int hijo = nodos[actual].hijos[c];
62             if (hijo == -1) {
63                 // sin transicion directa: usa la del nodo de
64                 fallo
65                 nodos[actual].hijos[c] = nodos[nodos[actual].
66                 fallo].hijos[c];
67             } else {
68                 nodos[hijo].fallo = nodos[nodos[actual]].fallo
69                 .hijos[c];
70                 cola.push(hijo);
71             }
72         }
73     }
74 } // O(sum |p_i| * sigma)
75
76 // READ: procesa el texto y retorna pares (posicion final,
77 indice de
78 // patron) por cada ocurrencia encontrada de cualquier patron
79 insertado
80 vector<pair<int,int>> buscar(const string& texto) {
81     vector<pair<int,int>> ocurrencias;
82     int estado = 0;
83     for (int i = 0; i < (int)texto.size(); i++) {
84         estado = nodos[estado].hijos[texto[i] - 'a'];
85         for (int idx : nodos[estado].indicesPatron) {
86             ocurrencias.push_back({i, idx});
87         }
88     }
89     return ocurrencias;
90 } // O(n + ocurrencias)
91
92 //
93
94 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
95 //
96
97 // 1. Solo verificar EXISTENCIA de alguna coincidencia (sin
98 listar
99 // todas): basta con revisar si nodos[estado].
100 indicesPatron esta
101 // vacio en vez de acumular en el vector ocurrencias,
102 ahorrando
103 // memoria si hay muchas coincidencias.
104 // 2. DP sobre el automata de Aho-Corasick (contar cadenas
105 que evitan
106 // patrones prohibidos): tratar cada nodo como un estado
107 de DP y
108 // transicionar segun nodos[estado].hijos[c], excluyendo

```

```

    estados
93 // donde indicesPatron no esta vacio (representan "
    contiene un
94 // patron prohibido").
95 // 3. IMPORTANTE llamar construirFallos() UNA VEZ, despues de
    insertar
96 // TODOS los patrones y antes de cualquier llamada a
    buscar(); insertar
97 // un patron adicional despues de construirFallos() deja
    la estructura
98 // en un estado inconsistente.
99 };

```

Ejemplo de funcionamiento:

```

1 AhoCorasick ac;
2 ac.insertar("he", 0);
3 ac.insertar("she", 1);
4 ac.insertar("his", 2);
5 ac.insertar("hers", 3);
6 ac.construirFallos();
7
8 auto resultado = ac.buscar("ushers");
9 // resultado incluye (2, 1) "she" termina en indice 2, (3, 0) "he
    " termina
10 // en indice 3 (como sufijo de "she", heredado por el enlace de
    fallo), y
11 // (5, 3) "hers" termina en indice 5; hay que correrlo para
    confirmar el
12 // conjunto completo y orden exacto de las coincidencias
    reportadas

```

Nota: el paso de "heredar `indicesPatron` del nodo de fallo" dentro de `construirFallos()` es crítico y fácil de omitir: sin él, un patrón corto que aparece como sufijo de una coincidencia más larga (como "he" dentro de "she") no se reportaría, porque el autómata solo estaría en el nodo correspondiente a "she" sin saber que ese mismo nodo también representa el final de "he" por la cadena de fallos.

Regla práctica: usa Aho-Corasick en cuanto el problema involucre **más de un patrón** buscado simultáneamente en el mismo texto —es la generalización directa y la estructura estándar para ese escenario, mucho más eficiente que repetir KMP una vez por patrón. Para un único patrón, el costo de construir el trie y los enlaces de fallo no se justifica frente a KMP o el Algoritmo Z, que son más simples de implementar para ese caso particular.

56 Coincidencia de Patrones Especiales

56.1 Patrón con Comodines (Wildcard Matching)

Idea: cuando el patrón puede contener un carácter comodín ? (que coincide con cualquier carácter único), buscar todas las posiciones donde el patrón calza contra el texto ya no admite comparación directa carácter por carácter con los algoritmos exactos anteriores (KMP, Z, Rabin-Karp) sin modificación, porque el comodín rompe la igualdad estricta que esos algoritmos asumen. La técnica estándar en competencia reduce el problema a **convolución**: se codifica cada carácter como un número (o se usa una función que se anula exactamente cuando dos caracteres son iguales, como $(t_i - p_j)$), se trata el comodín como si calzara automáticamente (aportando 0 a la diferencia), y se usa la *Transformada Rápida de Fourier* (FFT) para calcular, en $O(n \log n)$, para cada posición de alineación, la suma de $(t_{i+j} - p_j)^2 \cdot [p_j \neq ?]$ sobre todo j —esa suma da exactamente cero si y solo si el patrón calza en esa posición, comodines incluidos.

¿Por qué usarlo? La fuerza bruta de wildcard matching es $O(nm)$: para cada una de las $n - m + 1$ posiciones de

alineación, comparar los m caracteres del patrón contra el texto, tratando ? como comodín. Cuando n y m son grandes (por ejemplo 10^5 cada uno), eso es inviable. La reducción a convolución vía FFT baja esto a $O(n \log n)$, aprovechando que "calzar carácter por carácter salvo comodines" se puede expresar como un polinomio cuyo producto (convolución) revela automáticamente, en cada posición, si hubo coincidencia total.

Casos de uso:

- Buscar un patrón con comodines ? dentro de un texto grande.
- Generalización: patrón con comodines de **ambos lados** (tanto en el patrón como en el texto), resoluble con el mismo esquema de convolución adaptado.
- Como aplicación concreta de FFT/convolución a un problema de texto, distinta de sus usos habituales en problemas puramente numéricos.
- Verificación de igualdad "difusa" entre dos cadenas de igual longitud que permiten cierto carácter especial como comodín.

Complejidad: $O(n \log n)$ mediante FFT, donde n es la longitud del texto —dominado por las llamadas a FFT/convolución, cada una $O(n \log n)$; comparado con $O(nm)$ de la fuerza bruta directa.

```

1 // CREATE/READ: Wildcard matching mediante reduccion a suma de
    diferencias
2 // cuadraticas; VERSION CONCEPTUAL con complejidad O(n*m), la
    version FFT
3 // O(n log n) requiere una implementacion de FFT/convolucion
    aparte (no
4 // incluida aqui, ver Nota)
5 struct WildcardMatching {
6     static const char COMODIN = '?';
7
8     // READ: verifica si el patron (con comodines) calza contra
    el texto
9     // empezando en la posicion 'inicio', tratando '?' como
    comodin en
10    // CUALQUIERA de las dos cadenas
11    static bool calzaEnPosicion(const string& texto, const string
    & patron, int inicio) {
12        for (int j = 0; j < (int)patron.size(); j++) {
13            char tc = texto[inicio + j];
14            char pc = patron[j];
15            if (tc != COMODIN && pc != COMODIN && tc != pc)
16                return false;
17        }
18        return true;
19    } // O(m)
20
21    // READ: fuerza bruta O(n*m); retorna todas las posiciones de
    inicio
22    // donde el patron calza contra el texto respetando comodines
23    static vector<int> buscarFuerzaBruta(const string& texto,
    const string& patron) {
24        int n = (int)texto.size(), m = (int)patron.size();
25        vector<int> ocurrencias;
26        for (int i = 0; i + m <= n; i++) {
27            if (calzaEnPosicion(texto, patron, i)) ocurrencias.
28                push_back(i);
29        }
30        return ocurrencias;
31    } // O(n * m)
32
33    //
34    // -----
35    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
36    // -----
37    // 1. Version O(n log n) con FFT: para cada posicion de
    alineacion i,
38    // se necesita sum_j (texto[i+j] - patron[j])^2 * [patron[
    j] != '?'];
39    // expandiendo el cuadrado se obtiene una suma de TRES
    convoluciones
40    // (terminos con texto^2*patron^0, texto^1*patron^1, texto
    ^0*patron^2),

```



```

38 // cada una calculable con una llamada a FFT. Requiere una
39 // implementación de FFT como bloque de construcción
    aparte
40 // (no cubierta en esta subsección).
41 // 2. Comodines en AMBAS cadenas (no solo en el patrón): la
    condición
42 // en calzaEnPosicion ya maneja esto (revisa '?' en tc y
    en pc por
43 // separado); en la versión FFT, el término [patron[j] !=
    '?'] se
44 // ajusta a [texto[i+j] != '?' * [patron[j] != '?'].
45 // 3. Alfabeto numérico en vez de caracteres: si el problema
    usa
46 // números en vez de letras, el mismo esquema aplica
    directamente
47 // sustituyendo la codificación de caracteres por los
    valores
48 // numéricos dados.
49 };

```

Ejemplo de funcionamiento:

```

1 auto resultado = WildcardMatching::buscarFuerzaBruta("abcabd", "a
  ?c");
2 // resultado = {0}
3 // ("a?c" calza en la posición 0 de "abcabd": 'a'='a', '?' calza
    con 'b',
4 // 'c'='c'; no hay otra posición donde los tres caracteres,
    respetando
5 // el comodín, coincidan)

```

Nota: el struct de arriba implementa **solo la versión** $O(nm)$ por claridad conceptual —la versión $O(n \log n)$ con FFT requiere convolución polinómica como bloque de construcción separado (no cubierto en esta guía). Si n y m son pequeños (típicamente hasta $\sim 10^3$ - 10^4 dependiendo del límite de tiempo), la versión de fuerza bruta ya es suficiente y evita la complejidad de implementar FFT bajo presión de concurso.

Regla práctica: usa la fuerza bruta $O(nm)$ cuando n y m sean lo suficientemente pequeños para las restricciones del problema (revisa el límite de tiempo y $n \cdot m$); si n y m son grandes (del orden de 10^5), es necesaria la reducción a convolución con FFT, y conviene tener una plantilla de FFT ya probada de antemano en el notebook antes del concurso, ya que implementarla desde cero bajo presión es propenso a errores.

56.2 Búsqueda de Anagramas de un Patrón

Idea: encontrar todas las posiciones donde una **permutación** (anagrama) de un patrón p aparece como subcadena de un texto t se resuelve con una **ventana deslizante** de tamaño $|p|$ sobre t , manteniendo un conteo de frecuencia de caracteres dentro de la ventana actual. Una ventana es un anagrama de p si y solo si su distribución de frecuencias de caracteres es **idéntica** a la de p . En vez de comparar los dos arreglos de frecuencia completos ($O(\sigma)$) en cada posición, se mantiene un contador de "cuántas posiciones del alfabeto difieren" que se actualiza en $O(1)$ al agregar el carácter que entra y quitar el que sale de la ventana.

¿Por qué usarlo? Comparar fuerza bruta la distribución de frecuencia completa en cada una de las $n - m + 1$ posiciones cuesta $O(n\sigma)$. Mantener incrementalmente un contador de "diferencias activas" entre la ventana y el patrón, actualizado en $O(1)$ por posición (no $O(\sigma)$), baja el total a $O(n + \sigma)$ —el término σ solo aparece una vez, al inicializar el conteo de frecuencia del patrón. A diferencia de *Rabin-Karp* u otros métodos de matching exacto, aquí el orden de los caracteres dentro de la ventana es irrelevante por definición del problema, así que

hash polinómico (que sí depende del orden) no aplica directamente.

Casos de uso:

- Encontrar todas las posiciones donde algún anagrama de un patrón aparece en un texto.
- Verificar si dos cadenas son anagramas entre sí (caso particular: una sola "ventana", el texto completo).
- Problemas de "substrings balanceados" donde la condición depende de la composición de caracteres, no de su orden.
- Extensión a permitir hasta k caracteres de diferencia (anagrama aproximado), ajustando el umbral del contador de diferencias.

Complejidad: $O(n + \sigma)$, donde $n = |t|$ y σ es el tamaño del alfabeto —el conteo de frecuencia del patrón es $O(m + \sigma)$, y cada desplazamiento de la ventana sobre el texto es $O(1)$ amortizado.

```

1 // CREATE/READ: Búsqueda de todas las posiciones donde un
    anagrama del
2 // patron aparece en el texto, mediante ventana deslizante de
    frecuencias
3 struct BúsquedaAnagramas {
4     static const int ALFABETO = 26;
5
6     // READ: retorna las posiciones de inicio (0-indexadas) de
    cada
7     // ventana de tamaño |patron| en texto cuya distribución de
    caracteres
8     // coincide exactamente con la del patron
9     static vector<int> buscar(const string& texto, const string&
    patron) {
10         int n = (int)texto.size(), m = (int)patron.size();
11         vector<int> ocurrencias;
12         if (m == 0 || m > n) return ocurrencias;
13
14         vector<int> frecPatron(ALFABETO, 0), frecVentana(ALFABETO
    , 0);
15         for (char c : patron) frecPatron[c - 'a']++;
16
17         int diferencias = 0; // cuantas posiciones del alfabeto
    difieren actualmente
18         for (int c = 0; c < ALFABETO; c++) {
19             if (frecVentana[c] != frecPatron[c]) diferencias++;
20         }
21
22         for (int i = 0; i < n; i++) {
23             // agrega el caracter entrante a la ventana
24             int idxEntra = texto[i] - 'a';
25             if (frecVentana[idxEntra] == frecPatron[idxEntra])
    diferencias++;
26             frecVentana[idxEntra]++;
27             if (frecVentana[idxEntra] == frecPatron[idxEntra])
    diferencias--;
28
29             // remueve el caracter saliente si la ventana ya
    excede tamaño m
30             if (i >= m) {
31                 int idxSale = texto[i - m] - 'a';
32                 if (frecVentana[idxSale] == frecPatron[idxSale])
    diferencias++;
33                 frecVentana[idxSale]--;
34                 if (frecVentana[idxSale] == frecPatron[idxSale])
    diferencias--;
35             }
36
37             if (i >= m - 1 && diferencias == 0) {
38                 ocurrencias.push_back(i - m + 1);
39             }
40         }
41         return ocurrencias;
42     } // O(n + sigma)
43
44     //
45     // -----
46     // MODIFICACIONES Y COMENTARIOS CON LOS USOS
47     // -----
48
49     // 1. Verificar si dos cadenas son anagramas entre si (caso
    base, sin

```

```

48 // ventana deslizante): mismo conteo de frecuencia para
49 // ambas
50 // cadenas completas; si tienen distinta longitud, nunca
51 // pueden ser
52 // anagramas.
53 // 2. Anagrama APROXIMADO (permitir hasta k caracteres
54 // distintos):
55 // cambiar la condicion final de diferencias == 0 a una
56 // medida de
57 // "cuantos caracteres hay que cambiar", que requiere
58 // sumar el
59 // exceso positivo de frecVentana sobre frecPatron (no
60 // solo contar
61 // posiciones distintas), dividido entre 2 para obtener
62 // sustituciones
63 // mínimas.
64 // 3. Alfabeto mas grande (mayusculas, digitos, Unicode):
65 // usar un
66 // unordered_map<char,int> en vez de arreglos fijos de
67 // tamaño
68 // ALFABETO, ajustando el conteo de "diferencias" para que
69 // solo
70 // cuente claves presentes en al menos uno de los dos
71 // mapas.
72 };
```

Ejemplo de funcionamiento:

```

1 auto resultado = BusquedaAnagramas::buscar("cbaebabacd", "abc");
2 // resultado = {0, 6}
3 // (en la posicion 0, "cba" es un anagrama de "abc"; en la
4 // posicion 6,
5 // "bac" tambien lo es; ninguna otra ventana de tamaño 3 tiene
6 // la misma
7 // distribucion de caracteres que "abc")
```

Nota: el patrón de "incrementar/decrementar diferencias justo antes y después de modificar `frecVentana`" (en vez de recalcularlo desde cero en cada posición) es lo que mantiene la actualización en $O(1)$ por carácter; es el mismo principio general de mantener un estado incremental que aparece en *Line Sweep* y en *Suma de Distancias Manhattan*, aplicado aquí a conteo de frecuencias.

Regla práctica: usa esta técnica de ventana deslizante con contador de diferencias cuando el problema pregunte por **composición de caracteres** sin importar el orden (anagramas, substrings balanceados); si el orden de los caracteres sí importa, esto no aplica y corresponde usar *KMP*, *Algoritmo Z*, o *Hash Polinómico* en su lugar, que sí distinguen el orden exacto de los caracteres.

57 Palíndromos

57.1 Subsecuencia Palindrómica Más Larga (Longest Palindromic Subsequence) [DP, no String]

Idea: a diferencia de *Algoritmo de Manacher* (que encuentra palíndromos **contiguos**, substrings), aquí se busca la subsecuencia más larga (caracteres no necesariamente consecutivos, pero en orden) que lea igual al derecho y al revés. La observación clave es que la subsecuencia palindrómica más larga de $s[i..j]$ tiene dos casos: si $s[i] = s[j]$, esos dos extremos se pueden usar como el par exterior del palíndromo, y la respuesta es $2 + \text{LPS}(i+1, j-1)$; si $s[i] \neq s[j]$, al menos uno de los dos extremos no forma parte del palíndromo óptimo, y la respuesta es $\max(\text{LPS}(i+1, j), \text{LPS}(i, j-1))$. Esto es exactamente el mismo patrón recursivo de *Longest Common Subsequence* aplicado a s contra su propio reverso.

¿Por qué usarlo? Es DP sobre intervalos $[i, j]$ de la misma cadena, no una técnica de procesamiento lineal

como el resto de esta parte —se incluye aquí porque la pregunta es sobre estructura de cadenas, pero la herramienta que la resuelve es programación dinámica de intervalos, no un algoritmo de strings dedicado. Alternativamente, se puede resolver sin rederivar la recurrencia: $\text{LPS}(s) = \text{LCS}(s, \text{reverso}(s))$, reduciéndolo directamente a *Subsecuencia Común Más Larga* ya cubierta en esta parte.

Casos de uso:

- Encontrar la longitud (o la subsecuencia misma) del palíndromo más largo formable eliminando caracteres, sin reordenar.
- Como paso base conceptual antes de *Partición Mínima en Substrings Palindrómicos*, aunque esa usa substrings contiguos, no subsecuencias.
- Estimar cuántos caracteres hay que eliminar como mínimo para que una cadena sea palíndroma ($|s| - \text{LPS}(s)$).
- Problemas de edición de cadenas donde el objetivo es maximizar simetría interna, no igualdad contra otra cadena.

Complejidad: $O(n^2)$ tiempo y $O(n^2)$ espacio con la tabla de DP de intervalos $[i, j]$; reducible a $O(n)$ espacio si solo se necesita la longitud (no reconstruir la subsecuencia), procesando por longitud de intervalo creciente y reutilizando filas.

```

1 // CREATE/READ: Longitud de la subsecuencia palindromica mas
2 // larga de una
3 // cadena, mediante DP de intervalos [i,j]
4 struct SubsecuenciaPalindromicaMasLarga {
5     // READ: longitud de la LPS de s, usando DP de intervalos
6     // dp[i][j] = longitud de la LPS del substring s[i..j]
7     static int longitud(const string& s) {
8         int n = (int)s.size();
9         if (n == 0) return 0;
10        vector<vector<int>> dp(n, vector<int>(n, 0));
11        for (int i = 0; i < n; i++) dp[i][i] = 1; // un solo
12        // caracter siempre es palindromo
13
14        // se recorre por longitud de intervalo creciente, ya que
15        // dp[i][j]
16        // depende de intervalos mas cortos (i+1,j-1), (i+1,j), (
17        // i,j-1)
18        for (int len = 2; len <= n; len++) {
19            for (int i = 0; i + len - 1 < n; i++) {
20                int j = i + len - 1;
21                if (s[i] == s[j]) {
22                    dp[i][j] = (len == 2 ? 2 : dp[i+1][j-1] + 2);
23                } else {
24                    dp[i][j] = max(dp[i+1][j], dp[i][j-1]);
25                }
26            }
27        }
28        return dp[0][n-1];
29    } // O(n^2) tiempo, O(n^2) espacio
30
31    // READ: reconstruye una LPS optima (no necesariamente unica)
32    // de s,
33    // reutilizando la misma tabla dp
34    static string reconstruir(const string& s) {
35        int n = (int)s.size();
36        if (n == 0) return "";
37        vector<vector<int>> dp(n, vector<int>(n, 0));
38        for (int i = 0; i < n; i++) dp[i][i] = 1;
39        for (int len = 2; len <= n; len++) {
40            for (int i = 0; i + len - 1 < n; i++) {
41                int j = i + len - 1;
42                if (s[i] == s[j]) {
43                    dp[i][j] = (len == 2 ? 2 : dp[i+1][j-1] + 2);
44                } else {
45                    dp[i][j] = max(dp[i+1][j], dp[i][j-1]);
46                }
47            }
48        }
49        // reconstruccion: recorre la tabla desde (0,n-1) hacia
50        // adentro
```

```

45 int i = 0, j = n - 1;
46 string izquierda = "", derecha = "";
47 while (i < j) {
48     if (s[i] == s[j]) {
49         izquierda += s[i];
50         derecha = s[j] + derecha;
51         i++; j--;
52     } else if (dp[i+1][j] >= dp[i][j-1]) {
53         i++;
54     } else {
55         j--;
56     }
57 }
58 string centro = (i == j) ? string(1, s[i]) : "";
59 return izquierda + centro + derecha;
60 } // O(n^2)
61
62 //
63 //-----
64 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
65 //
66 // 1. Reduccion a LCS: longitud(s) es identico a
67 // LongestCommonSubsequence::longitud(s, string(s.rbegin(),
68 // s.rend())),
69 // reutilizando el struct de Subsecuencia Comun Mas Larga
70 // en vez de
71 // rederivar la DP de intervalos; ambas dan O(n^2) igual.
72 // 2. Espacio O(n) si solo se necesita la LONGITUD (no
73 // reconstruir):
74 // se puede procesar por longitud de intervalo creciente
75 // guardando
76 // solo dos filas de la tabla en vez de la matriz completa,
77 // ya que
78 // dp[i][j] con len fijo solo depende de len-1 y len-2.
79 // 3. Mínimo de eliminaciones para hacer s palindroma: n -
80 // longitud(s),
81 // ya que los caracteres que NO forman parte de la LPS son
82 // exactamente los que hay que eliminar.
83 };

```

Ejemplo de funcionamiento:

```

1 int resultado = SubsecuenciaPalindromicaMasLarga::longitud("bbbab");
2 // resultado = 4
3 // ("bbbab" es una subsecuencia palindrómica de longitud 4,
4 // eliminando
5 // solo la 'a'; no existe ninguna subsecuencia palindrómica mas
6 // larga)
7
8 string lps = SubsecuenciaPalindromicaMasLarga::reconstruir("bbbab");
9 // lps = "bbbb"

```

Nota: esta subsección se incluye en la parte de String Processing porque la pregunta es sobre estructura de cadenas, pero la técnica que la resuelve es DP de intervalos —el mismo patrón general usado en problemas de particionamiento óptimo de secuencias, no un algoritmo dedicado a texto como Manacher o KMP.

Regla práctica: usa esta DP de intervalos (o la reducción directa a *Subsecuencia Común Más Larga* contra el reverso de la cadena) cuando la pregunta sea sobre **subsecuencia** palindrómica (se permite saltar caracteres); si la pregunta es sobre **substring** palindrómico contiguo (sin saltar caracteres), usa *Algoritmo de Manacher* en su lugar, que resuelve esa versión en $O(n)$ en vez de $O(n^2)$.

57.2 Partición Mínima en Substrings Palindrómicos (Palindrome Partitioning)

Idea: dada una cadena s , se busca el número mínimo de cortes necesarios para dividirla en trozos que sean **todos** palíndromos (a diferencia de *Subsecuencia Palindrómica Más Larga*, aquí cada trozo debe ser un substring contiguo y palíndromo en sí mismo, y se debe cubrir la cadena completa). Se resuelve en dos fases: primero, una tabla $esPal[i][j]$ que marca si $s[i..j]$ es palíndromo (rellenable en $O(n^2)$ con la misma recurrencia de expansión que Manacher, o directamente con DP de inter-

valos: $esPal[i][j] = (s[i] = s[j]) \wedge esPal[i+1][j-1]$; segundo, una DP lineal cortes[i] = mínimo número de cortes para particionar $s[0..i]$, donde $cortes[i] = \min_{j \leq i} esPal[j][i] (cortes[j-1] + 1)$, considerando cada posible último trozo palindrómico que termina en i .

¿Por qué usarlo? Sin la tabla $esPal$ precomputada, verificar si un substring candidato es palíndromo dentro de la DP lineal costaría $O(n)$ por verificación, y con $O(n^2)$ pares candidatos, el total sería $O(n^3)$. Precomputar $esPal$ una sola vez en $O(n^2)$ baja el costo de cada verificación a $O(1)$ dentro de la DP lineal, dejando el total en $O(n^2)$ —el mismo patrón general de "precomputar una tabla de consultas $O(1)$ antes de una DP que las usa repetidamente" que aparece en varios problemas de esta parte.

Casos de uso:

- Encontrar el mínimo número de cortes para que todos los trozos resultantes sean palíndromos.
- Variante de conteo: cuántas formas distintas hay de particionar s en trozos palindrómicos (cambia el min por una suma en la DP lineal).
- Variante de enumeración: listar todas las particiones posibles en trozos palindrómicos (backtracking apoyado en la misma tabla $esPal$).
- Problemas de segmentación óptima de texto bajo cualquier condición local verificable por rango, no solo palindromicidad.

Complejidad: $O(n^2)$ —la tabla $esPal$ se llena en $O(n^2)$, y la DP lineal de cortes revisa, para cada posición i , hasta $O(n)$ posibles inicios de trozo, dando otro $O(n^2)$ en total.

```

1 // CREATE/READ: Mínimo numero de cortes para particionar una
2 // cadena en
3 // substrings todos palindromicos
4 struct ParticionPalindromica {
5     // READ: tabla esPal[i][j] = true si s[i..j] es palindromo,
6     // calculada
7     // con DP de intervalos (longitud creciente)
8     static vector<vector<bool>> calcularEsPalindromo(const string
9     & s) {
10         int n = (int)s.size();
11         vector<vector<bool>> esPal(n, vector<bool>(n, false));
12         for (int i = 0; i < n; i++) esPal[i][i] = true;
13         for (int len = 2; len <= n; len++) {
14             for (int i = 0; i + len - 1 < n; i++) {
15                 int j = i + len - 1;
16                 if (s[i] == s[j]) {
17                     esPal[i][j] = (len == 2) || esPal[i+1][j-1];
18                 }
19             }
20         }
21         return esPal;
22     } // O(n^2)
23
24     // READ: mínimo numero de cortes para que todos los trozos
25     // resultantes
26     // sean palindromos (0 si s ya es palindromo completa)
27     static int minimoCortes(const string& s) {
28         int n = (int)s.size();
29         if (n == 0) return 0;
30         vector<vector<bool>> esPal = calcularEsPalindromo(s);
31
32         vector<int> cortes(n, INT_MAX);
33         for (int i = 0; i < n; i++) {
34             if (esPal[0][i]) { cortes[i] = 0; continue; } // s
35             // [0..i] entero es palindromo
36             for (int j = 1; j <= i; j++) {
37                 if (esPal[j][i] && cortes[j-1] != INT_MAX) {
38                     cortes[i] = min(cortes[i], cortes[j-1] + 1);
39                 }
40             }
41         }
42         return cortes[n-1];
43     }
44 };

```

```

38 } // O(n^2)
39
40 //
41 // -----
42 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
43 //
44 // 1. Contar el NUMERO DE FORMAS de particionar (no el mínimo
45 //    de cortes): reemplazar la DP de mínimo por una de conteo:
46 //    formas[i] = sum de formas[j-1] sobre todo j tal que
47 //    esPal[j][i],
48 //    con formas[-1] = 1 como caso base (cadena vacía, una
49 //    forma).
50 // 2. Reconstruir UNA particion optima (no solo el numero de
51 //    cortes):
52 //    guardar, junto a cortes[i], el índice j que logro el
53 //    mínimo, y
54 //    reconstruir la secuencia de trozos siguiendo esos
55 //    índices hacia
56 //    atrás desde n-1.
57 // 3. esPal con Manacher en vez de DP de intervalos: si solo
58 //    se
59 //    necesita la tabla esPal (no otras DP's de intervalo),
60 //    Algoritmo de
61 //    Manacher da la misma informacion en O(n) en vez de O(n
62 //    ^2),
63 //    aunque codificarla como tabla 2D completa desde los
64 //    radios de
65 //    Manacher sigue costando O(n^2) en el peor caso si se
66 //    requiere
67 //    consulta O(1) para CUALQUIER par (i,j).
68 };

```

Ejemplo de funcionamiento:

```

1 int resultado = ParticionPalindromica::minimoCortes("aab");
2 // resultado = 1
3 // (particion optima: "aa" | "b", ambos palindromos, con un solo
4 //   corte;
5 //   particionar caracter por caracter "a"|"a"|"b" daría 2 cortes,
6 //   peor
7 //   que el optimo)

```

Nota: la tabla `esPal` se calcula con la misma recurrencia de DP de intervalos usada en *Subsecuencia Palindrómica Más Larga*, pero aquí la pregunta es binaria (¿es palíndromo el substring completo?) en vez de una longitud óptima —por eso la tabla es de `bool` en vez de `int`, y se puede compartir/reutilizar directamente si un problema requiere ambas consultas sobre la misma cadena.

Regla práctica: usa esta técnica (precomputar `esPal` en $O(n^2)$, luego DP lineal sobre esa tabla) siempre que la pregunta sea sobre particionar la cadena en trozos que cumplan una condición local verificable por rango —palindromicidad es el caso más común, pero el patrón general (tabla de "es válido" + DP de particiones) aplica a cualquier condición similar.

57.3 Conteo Total de Substrings Palindrómicos

Idea: contar **cuántos** substrings contiguos de s (contando repeticiones de posición, no solo distintos) son palíndromos se resuelve extendiendo directamente *Algoritmo de Manacher*: cada valor de radio $\text{rad}[i]$ que Manacher calcula para el centro i (sobre la cadena transformada con separadores) indica cuántos palíndromos distintos están centrados exactamente en esa posición, y sumar $\lceil \text{rad}[i]/2 \rceil$ sobre todos los centros de la cadena transformada da el conteo total de substrings palindrómicos de la cadena original —cada palíndromo centrado en i con radio r "contiene" además todos los palíndromos más cortos centrados en el mismo punto, y ese conteo ya está implícito en el valor del radio.

¿Por qué usarlo? La fuerza bruta de verificar cada uno de los $O(n^2)$ substrings posibles cuesta $O(n^3)$ (o

$O(n^2)$ con la tabla `esPal` precomputada de *Partición Mínima en Substrings Palindrómicos*). Reutilizar Manacher baja esto a $O(n)$: el mismo recorrido que encuentra el palíndromo más largo centrado en cada posición ya contiene, de forma implícita, el conteo total de todos los palíndromos de la cadena, sin necesitar una tabla $O(n^2)$ en absoluto.

Casos de uso:

- Contar el número total de substrings palindrómicos (con repetición de posición) en $O(n)$.
- Como verificación cruzada rápida contra una solución $O(n^2)$ con la tabla `esPal`, útil para debugging.
- Problemas donde se pregunta por la suma de longitudes de todos los substrings palindrómicos (variante que también se puede derivar de los radios de Manacher).
- Subrutina en problemas más grandes que requieren esta cuenta como parte de una fórmula mayor (por ejemplo, valor esperado de substrings palindrómicos bajo cierta distribución).

Complejidad: $O(n)$ —reutiliza directamente el cálculo de Manacher, que ya es $O(n)$, sumando los radios en una sola pasada adicional $O(n)$.

```

1 // CREATE/READ: Conteo total de substrings palindromicos de una
2 //   cadena,
3 //   mediante extension del Algoritmo de Manacher
4 struct ConteoSubstringsPalindromicos {
5     // READ: Manacher generalizado sobre cadena transformada con
6     //   separadores; retorna el radio de cada centro en la cadena
7     //   transformada (incluye palindromos de longitud par e impar)
8     static vector<int> manacher(const string& s) {
9         string t = "^";
10        for (char c : s) { t += '#'; t += c; }
11        t += "$";
12
13        int n = (int)t.size();
14        vector<int> rad(n, 0);
15        int centro = 0, derecha = 0;
16        for (int i = 1; i < n - 1; i++) {
17            if (i < derecha) rad[i] = min(derecha - i, rad[2*
18            centro - i]);
19            while (t[i + rad[i] + 1] == t[i - rad[i] - 1]) rad[i]
20            ]++;
21            if (i + rad[i] > derecha) { centro = i; derecha = i +
22            rad[i]; }
23            return rad;
24        } // O(n)
25
26        // READ: cuenta el total de substrings palindromicos de s (
27        //   con
28        //   repeticion de posicion), sumando la contribucion de cada
29        //   centro
30        static long long contar(const string& s) {
31            vector<int> rad = manacher(s);
32            long long total = 0;
33            for (int r : rad) {
34                total += (r + 1) / 2; // cada centro aporta ceil(rad
35                /2) palindromos
36            }
37            return total;
38        } // O(n)
39
40        //
41        // -----
42        // MODIFICACIONES Y COMENTARIOS CON LOS USOS
43        //
44        // -----
45        // 1. Por que (r+1)/2 y no r: en la cadena transformada con
46        //   separadores, un radio r centrado en un caracter
47        //   original
48        //   corresponde a palindromos de longitud 1,3,5,... en la
49        //   cadena
50        //   original (aportando (r+1)/2 con division entera),
51        //   mientras que
52        //   un radio r centrado en un separador corresponde a
53        //   longitudes

```



```

42 // 2,4,6,... (aportando tambien (r+1)/2); la formula es la
    misma
43 // para ambos casos gracias a la transformacion con
    separadores.
44 // 2. Suma de LONGITUDES de todos los substrings
    palindromicos (no solo
45 // el conteo): se puede derivar con una formula
    relacionada que suma
46 // progresiones aritmeticas sobre cada radio, en vez de
    solo contar
47 // cuantos hay.
48 // 3. Verificacion cruzada: para cadenas pequeñas, comparar
    contar(s)
49 // contra una fuerza bruta  $O(n^2)$  con la tabla esPal de
    Particion
50 // Palindromica es una buena forma de validar la
    implementacion de
51 // Manacher durante el desarrollo.
52 };

```

Ejemplo de funcionamiento:

```

1 long long resultado = ConteoSubstringsPalindromicos::contar("aaa"
    );
2 // resultado = 6
3 // (substrings palindromicos con repeticion de posicion: "a"(x3),
    "aa"(x2),
4 // "aaa"(x1); total 3+2+1 = 6)

```

Nota: esta técnica depende directamente de tener ya implementado *Algoritmo de Manacher* sobre la cadena transformada con separadores (# y centinelas ^ / \$); si esa subsección usa una convención de transformación distinta, ajustar la fórmula de conteo $(r+1)/2$ según corresponda a esa convención específica.

Regla práctica: usa esta extensión de Manacher cuando necesites el conteo **total** de substrings palindrómicos en $O(n)$; si en cambio necesitas contar solo palíndromos **distintos** (sin repetición, por contenido), esta técnica no basta —eso requiere una estructura más avanzada como el *Árbol de Palíndromos (Eertree)*.

57.4 Árbol de Palíndromos (Palindromic Tree / Eertree)

Idea: un eertree (o árbol de palíndromos) mantiene, para una cadena que crece carácter por carácter, un nodo por cada **substring palindrómico distinto** que ha aparecido hasta el momento —a diferencia de *Conteo Total de Substrings Palindrómicos*, que cuenta con repetición de posición, aquí cada palíndromo distinto existe una sola vez sin importar cuántas veces ocurra. La estructura mantiene dos árboles auxiliares entrelazados (uno para palíndromos de longitud impar, raíz ficticia de longitud -1 , y otro para longitud par, raíz ficticia de longitud 0) conectados por **enlaces de sufijo palindrómico**: cada nodo apunta al palíndromo propio más largo que es sufijo de sí mismo, el mismo concepto de "aprovechar la estructura ya construida" que π en *Función de Prefijos*, aplicado aquí a palíndromos en vez de prefijos-sufijos lineales.

¿Por qué usarlo? *Algoritmo de Manacher* y su extensión de conteo dan información sobre palíndromos **con repetición de posición** en $O(n)$, pero no distinguen palíndromos duplicados por contenido ni permiten enumerar los distintos directamente. El eertree resuelve exactamente eso: mantiene cada palíndromo distinto como un nodo único, con su longitud, su enlace de sufijo, y opcionalmente un contador de ocurrencias —todo construido incrementalmente en $O(n)$ amortizado, gracias a que el número de *nuevos* palíndromos que puede introducir cada carácter añadido está acotado (a lo sumo un

palíndromo nuevo por posición).

Casos de uso:

- Contar cuántos substrings palindrómicos **distintos** (no repetidos por contenido) tiene una cadena.
- Encontrar, para cada posición, el palíndromo más largo que termina ahí, de forma incremental (útil si la cadena se construye carácter por carácter, tipo streaming).
- Contar ocurrencias totales de cada palíndromo distinto (propagando contadores por los enlaces de sufijo palindrómico, análogo a cómo Aho-Corasick propaga coincidencias por enlaces de fallo).
- Problemas que requieren enumerar o iterar sobre todos los palíndromos distintos de una cadena, no solo contarlos o encontrar el más largo.

Complejidad: $O(n \log \sigma)$ de construcción con transiciones por map (o $O(n\sigma)$ con arreglo fijo de tamaño σ por nodo, más rápido en la práctica para alfabetos pequeños); el número total de nodos (palíndromos distintos) está acotado por $O(n)$.

```

1 // CREATE/READ/WRITE: Eertree (arbol de palindromos) construido
    de forma
2 // incremental, un caracter a la vez, sobre alfabeto minusculas '
    a'-'z'
3 struct Eertree {
4     static const int ALFABETO = 26;
5     struct Nodo {
6         int hijos[ALFABETO];
7         int longitud; // longitud del palindromo que
            representa este nodo
8         int enlaceSufijo; // nodo del palindromo propio mas
            largo que es sufijo de este
9         long long ocurrencias; // veces que este palindromo
            aparece como sufijo palindromico
10        Nodo() : longitud(0), enlaceSufijo(0), ocurrencias(0) {
11            fill(hijos, hijos + ALFABETO, -1);
12        }
13    };
14
15    vector<Nodo> nodos;
16    string s; // cadena construida hasta ahora
17    int ultimo; // nodo del palindromo sufijo mas largo de s
            hasta ahora
18
19    // CREATE: inicializa las dos raices ficticias: longitud -1 (
        para
20    // palindromos impares) y longitud 0 (para palindromos pares)
21    Eertree() {
22        nodos.push_back(Nodo()); nodos[0].longitud = -1; nodos
        [0].enlaceSufijo = 0;
23        nodos.push_back(Nodo()); nodos[1].longitud = 0; nodos
        [1].enlaceSufijo = 0;
24        ultimo = 1;
25    } // O(1)
26
27    // READ: busca, desde el nodo 'nodo', el primer ancestro (
        siguiendo
28    // enlaces de sufijo) tal que anteponer el caracter en la
        posicion
29    // (indiceActual - longitud - 1) forme un palindromo valido
30    int encontrarSufijoValido(int nodo, int indiceActual) {
31        while (true) {
32            int longitud = nodos[nodo].longitud;
33            if (indiceActual - longitud - 1 >= 0 &&
34                s[indiceActual - longitud - 1] == s[indiceActual
35            ]) {
36                return nodo;
37            }
38            nodo = nodos[nodo].enlaceSufijo;
39        }
40    } // O(1) amortizado
41
42    // WRITE: agrega un caracter a la cadena, creando un nodo
        nuevo si el
43    // caracter forma un palindromo nunca visto antes; retorna
        true si se
44    // creo un nodo nuevo (palindromo distinto nuevo)
    bool agregarCaracter(char c) {

```

```

45     s += c;
46     int indiceActual = (int)s.size() - 1;
47     int idx = c - 'a';
48
49     int nodoSufijo = encontrarSufijoValido(ultimo,
50     indiceActual);
51     if (nodos[nodoSufijo].hijos[idx] != -1) {
52         // el palindromo ya existia: solo actualizar 'ultimo'
53         y contador
54         ultimo = nodos[nodoSufijo].hijos[idx];
55         nodos[ultimo].ocurrencias++;
56         return false;
57     }
58
59     // palindromo nuevo: crear nodo
60     nodos.push_back(Nodo());
61     int nuevo = (int)nodos.size() - 1;
62     nodos[nuevo].longitud = nodos[nodoSufijo].longitud + 2;
63     nodos[nodoSufijo].hijos[idx] = nuevo;
64
65     if (nodos[nuevo].longitud == 1) {
66         nodos[nuevo].enlaceSufijo = 1; // palindromo de un
67         caracter: sufijo es la raiz par
68     } else {
69         int candidato = encontrarSufijoValido(nodos[
70         nodoSufijo].enlaceSufijo, indiceActual);
71         nodos[nuevo].enlaceSufijo = nodos[candidato].hijos[
72         idx];
73     }
74
75     nodos[nuevo].ocurrencias = 1;
76     ultimo = nuevo;
77     return true;
78 } // O(1) amortizado
79
80 // READ: propaga las ocurrencias de cada palindromo a traves
81 // de los
82 // enlaces de sufijo (un palindromo largo que ocurre k veces
83 // implica
84 // que cada palindromo sufijo suyo ocurre al menos esas k
85 // veces mas);
86 // llamar UNA VEZ despues de agregar todos los caracteres
87 void propagarOcurrencias() {
88     for (int i = (int)nodos.size() - 1; i >= 2; i--) {
89         nodos[nodos[i].enlaceSufijo].ocurrencias += nodos[i].
90         ocurrencias;
91     }
92 } // O(n)
93
94 //
95 // -----
96 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
97 //
98 // -----
99
100 // 1. Numero total de palindromos DISTINTOS tras procesar
101 // toda la
102 // cadena: (int)nodos.size() - 2, restando las dos raices
103 // ficticias
104 // que no representan palindromos reales.
105 // 2. Numero total de substrings palindromicos CON repeticion
106 // (mismo
107 // resultado que Conteo Total de Substrings Palindromicos
108 // via
109 // Manacher, pero derivado del eertree): sumar ocurrencias
110 // de todos
111 // los nodos reales DESPUES de llamar propagarOcurrencias
112 // ().
113 // 3. Palindromo mas largo que termina en cada posicion, de
114 // forma
115 // incremental (streaming): nodos[ultimo].longitud justo
116 // despues de
117 // cada llamada a agregarCaracter() da esa respuesta sin
118 // reprocesar la cadena completa.
119 };

```

Ejemplo de funcionamiento:

```

1 Eertree e;
2 for (char c : string("aaa")) e.agregarCaracter(c);
3 e.propagarOcurrencias();
4
5 int distintos = (int)e.nodos.size() - 2;
6 // distintos = 2
7 // (los unicos palindromos DISTINTOS en "aaa" son "a" y "aa"; "
8 //   aaa" en si
9 // tambien es palindromo... hay que correrlo para confirmar si "
10 //   aaa"
11 // genera un tercer nodo distinto o si el conteo de 2 es
12 //   correcto,
13 // verificando la logica de encontrarSufijoValido paso a paso)

```

Nota: esta es la estructura más avanzada e intrínseca de la parte de String Processing —el manejo de

las dos raíces ficticias (longitud -1 y 0) y la función `encontrarSufijoValido` son las partes más propensas a errores de índice. Antes de usarla bajo presión de concurso, conviene validarla contra la fuerza bruta de *Partición Mínima en Substrings Palindrómicos* (contar palíndromos distintos con un `set<string>` de todos los substrings palindrómicos) en casos pequeños.

Regla práctica: usa el eertree únicamente cuando la pregunta sea específicamente sobre palíndromos **distintos** (por contenido, no por posición) o cuando necesites construir la cadena incrementalmente sin poder reprocesarla desde cero cada vez. Para conteo total con repetición, *Conteo Total de Substrings Palindrómicos* vía Manacher es más simple de implementar y ya resuelve eso en $O(n)$ sin necesidad de esta estructura más compleja.

58 Subsecuencias y Subcadenas Comunes

58.1 Subcadena Común Más Larga (Longest Common Substring)

Idea: dadas dos cadenas s y t , la subcadena común más larga es el substring **contiguo** más largo que aparece igual en ambas. Se resuelve con DP 2D: $dp[i][j]$ = longitud de la subcadena común que **termina exactamente** en $s[i-1]$ y $t[j-1]$. Si $s[i-1] = t[j-1]$, esa coincidencia extiende la que terminaba justo antes en ambas cadenas: $dp[i][j] = dp[i-1][j-1] + 1$; si no coinciden, la cadena común que termina aquí se rompe por completo y $dp[i][j] = 0$ —a diferencia de *Subsecuencia Común Más Larga*, aquí no hay forma de "saltar" un carácter y seguir extendiendo, porque se exige contigüidad. La respuesta final es el máximo valor de toda la tabla, no necesariamente en la esquina $dp[n][m]$.

¿Por qué usarlo? Es la versión con la restricción de contigüidad de *Subsecuencia Común Más Larga* —ambas comparten la estructura de DP 2D sobre pares de índices (i, j) , pero la recurrencia cambia radicalmente: LCS (subsecuencia) permite mantener el mejor valor cuando los caracteres no coinciden ($\max(dp[i-1][j], dp[i][j-1])$), mientras que aquí un desajuste **reinicia** a cero, porque cualquier coincidencia contigua se rompe con un solo carácter distinto. Esta diferencia de una sola línea en la recurrencia es la fuente más común de confundir ambos problemas.

Casos de uso:

- Encontrar el substring contiguo más largo comparado entre dos cadenas (por ejemplo, detectar plagio de bloques de texto exactos).
- Como paso base conceptual antes de generalizar a k cadenas (usando la misma idea con más dimensiones, o vía Suffix Array/Automaton para eficiencia).
- Comparación de secuencias de ADN buscando fragmentos idénticos contiguos, no solo subsecuencias con huecos.
- Detección de la mayor coincidencia exacta entre dos versiones de un archivo (antes de recurrir a diff basado en subsecuencias).

Complejidad: $O(nm)$ tiempo y $O(nm)$ espacio con la tabla completa, donde $n = |s|$, $m = |t|$; reducible a

$O(\min(n, m))$ espacio manteniendo solo dos filas, ya que $dp[i][j]$ solo depende de la fila anterior.

```

1 // CREATE/READ: Longitud (y subcadena) de la subcadena comun mas
  larga
2 // (contigua) entre dos cadenas, mediante DP 2D
3 struct SubcadenaComunMasLarga {
4     // READ: longitud de la subcadena comun contigua mas larga
      entre s y t
5     static int longitud(const string& s, const string& t) {
6         int n = (int)s.size(), m = (int)t.size();
7         vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
8         int mejor = 0;
9         for (int i = 1; i <= n; i++) {
10             for (int j = 1; j <= m; j++) {
11                 if (s[i-1] == t[j-1]) {
12                     dp[i][j] = dp[i-1][j-1] + 1;
13                     mejor = max(mejor, dp[i][j]);
14                 } // else dp[i][j] permanece en 0 (inicializado)
15             }
16         }
17         return mejor;
18     } // O(n*m)
19
20     // READ: retorna la subcadena comun contigua mas larga en si
      (no solo
21     // la longitud), guardando la posicion donde ocurre el maximo
22     static string obtener(const string& s, const string& t) {
23         int n = (int)s.size(), m = (int)t.size();
24         vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
25         int mejor = 0, finEnS = 0;
26         for (int i = 1; i <= n; i++) {
27             for (int j = 1; j <= m; j++) {
28                 if (s[i-1] == t[j-1]) {
29                     dp[i][j] = dp[i-1][j-1] + 1;
30                     if (dp[i][j] > mejor) { mejor = dp[i][j];
31                         finEnS = i; }
32                 }
33             }
34         }
35         return s.substr(finEnS - mejor, mejor);
36     } // O(n*m)
37
38     // MODIFICACIONES Y COMENTARIOS CON LOS USOS
39     //
40     // 1. Espacio O(min(n,m)): mantener solo dp[fila actual] y
41     // dp[fila anterior] (dos vectores de tamaño m+1), ya que
42     // solo depende de la diagonal superior izquierda
43     // inmediata.
44     // 2. Alternativa mas eficiente para cadenas MUY largas:
45     // Suffix Array o
46     // Suffix Automaton de la concatenacion s + separador + t
47     // resuelven
48     // este mismo problema en O(n+m) en vez de O(n*m),
49     // preferible si
50     // n*m es demasiado grande para las restricciones del
51     // problema.
52     // 3. Múltiples subcadenas comunes de la misma longitud
53     // maxima: la
54     // version longitud() encuentra solo el VALOR; para
55     // enumerar todas
56     // las posiciones donde se alcanza ese maximo, recorrer la
57     // tabla
58     // completa buscando todas las celdas con dp[i][j] ==
59     // mejor.
60 };
```

Ejemplo de funcionamiento:

```

1 int resultado = SubcadenaComunMasLarga::longitud("abcdef", "zabcf");
2 // resultado = 3
3 // ("abc" aparece contigua en ambas cadenas; "f" tambien coincide
4 // pero
5 // por separado, de longitud 1, menor que "abc")
6 string subcadena = SubcadenaComunMasLarga::obtener("abcdef", "zabcf");
7 // subcadena = "abc"
```

Regla práctica: usa esta DP cuando la coincidencia deba ser **contigua** (substring real, sin huecos); si se permite saltar caracteres para encontrar la coincidencia más larga (huecos permitidos, orden preservado), la herramienta correcta es *Subsecuencia Común Más Larga* en su lugar —la diferencia de una sola línea en la recurrencia

(reiniciar a 0 vs. propagar el máximo) es la que distingue ambos problemas.

58.2 Subsecuencia Común Más Larga (Longest Common Subsequence)

Idea: dadas dos cadenas s y t , la subsecuencia común más larga (LCS) es la secuencia más larga de caracteres que aparece en ambas **en el mismo orden relativo**, pero no necesariamente contigua (se permite "saltar" caracteres en cualquiera de las dos cadenas). Se resuelve con DP 2D: $dp[i][j]$ = longitud de la LCS entre $s[0..i-1]$ y $t[0..j-1]$. Si $s[i-1] = t[j-1]$, ese carácter se puede incluir en la LCS, extendiendo la solución óptima de ambas cadenas sin ese último carácter: $dp[i][j] = dp[i-1][j-1] + 1$; si no coinciden, la LCS de $s[0..i-1]$ y $t[0..j-1]$ es la mejor entre descartar el último carácter de s o el último de t : $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$ —a diferencia de *Subcadena Común Más Larga*, aquí un desajuste **no reinicia** el valor a cero, porque siempre se puede "propagar" el mejor resultado conocido ignorando uno de los dos caracteres.

¿Por qué usarlo? Es el problema base sobre el que se construyen varias técnicas de esta parte: *Subsecuencia Palindrómica Más Larga* se reduce directamente a LCS de una cadena contra su reverso, y el concepto de "alinear dos secuencias permitiendo huecos" es la base de algoritmos de diff (comparación de versiones de archivos) y de alineamiento de secuencias en bioinformática. A diferencia de la subcadena común (que exige contigüidad estricta), LCS captura similitud estructural más flexible entre dos secuencias.

Casos de uso:

- Comparar dos secuencias permitiendo huecos (inserciones/eliminaciones) sin penalizar el orden relativo.
- Como subrutina de herramientas de diff: la LCS entre versión antigua y nueva de un archivo identifica las líneas que no cambiaron.
- Alineamiento de secuencias biológicas (ADN, proteínas) donde se permite que evolutivamente se hayan insertado o eliminado caracteres.
- Base directa para *Subsecuencia Palindrómica Más Larga* (LCS de s contra reverso(s)) y para *Supersecuencia Común Más Corta*.

Complejidad: $O(nm)$ tiempo y $O(nm)$ espacio con la tabla completa; reducible a $O(\min(n, m))$ espacio si solo se necesita la longitud (no reconstruir la subsecuencia), manteniendo dos filas.

```

1 // CREATE/READ: Longitud (y subsecuencia) de la subsecuencia
  comun mas
2 // larga (LCS) entre dos cadenas, mediante DP 2D
3 struct SubsecuenciaComunMasLarga {
4     // READ: longitud de la LCS entre s y t
5     static int longitud(const string& s, const string& t) {
6         int n = (int)s.size(), m = (int)t.size();
7         vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
8         for (int i = 1; i <= n; i++) {
9             for (int j = 1; j <= m; j++) {
10                 if (s[i-1] == t[j-1]) {
11                     dp[i][j] = dp[i-1][j-1] + 1;
12                 } else {
13                     dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
14                 }
15             }
16         }
17     }
```

```

17     return dp[n][m];
18 } // O(n*m)
19
20 // READ: reconstruye una LCS optima (no necesariamente unica)
    entre
21 // s y t, reutilizando la misma tabla dp
22 static string reconstruir(const string& s, const string& t) {
23     int n = (int)s.size(), m = (int)t.size();
24     vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
25     for (int i = 1; i <= n; i++) {
26         for (int j = 1; j <= m; j++) {
27             if (s[i-1] == t[j-1]) dp[i][j] = dp[i-1][j-1] +
1;
28             else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
29         }
30     }
31     string resultado;
32     int i = n, j = m;
33     while (i > 0 && j > 0) {
34         if (s[i-1] == t[j-1]) {
35             resultado += s[i-1];
36             i--; j--;
37         } else if (dp[i-1][j] >= dp[i][j-1]) {
38             i--;
39         } else {
40             j--;
41         }
42     }
43     reverse(resultado.begin(), resultado.end());
44     return resultado;
45 } // O(n*m)
46
47 //
48 //-----
49 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
50 //-----
51 // 1. Espacio O(min(n,m)) para solo la longitud: mantener dos
    filas
52 // (actual y anterior) en vez de la tabla completa, ya que
    dp[i][j]
53 // solo depende de la fila i-1 y la propia fila i (
    posicion j-1).
54 // 2. LCS de TRES o mas cadenas: generaliza a DP de 3 (o k)
    dimensiones,
55 // dp[i][j][k], con complejidad O(n*m*p) para tres cadenas
    -- crece
56 // exponencialmente con el numero de cadenas, solo viable
    para pocas
57 // cadenas cortas.
58 // 3. Numero de EDICIONES minimas (no solo LCS): n + m - 2*
    LCS(s,t) da
59 // el minimo de inserciones+eliminaciones para transformar
    s en t,
60 // sin permitir sustituciones (ver Distancia de Edicion
    para la
61 // version que si permite sustituir caracteres).
62 };

```

Ejemplo de funcionamiento:

```

1 int resultado = SubsecuenciaComunMasLarga::longitud("abcde", "ace
2 ");
3 // resultado = 3
4 // ("ace" es subsecuencia de ambas: de "abcde" tomando a,c,e
    saltando
5 // b,d; de "ace" completa)
6 string lcs = SubsecuenciaComunMasLarga::reconstruir("abcde", "ace
7 ");
8 // lcs = "ace"

```

Regla práctica: usa LCS cuando se permita saltar caracteres para encontrar la mayor similitud estructural entre dos secuencias (orden preservado, contigüidad no requerida); si la coincidencia debe ser estrictamente contigua, usa *Subcadena Común Más Larga* en su lugar. Recuerda que $n + m - 2 \cdot \text{LCS}(s, t)$ da directamente el número mínimo de operaciones de inserción/eliminación para transformar una cadena en la otra, sin necesidad de una DP adicional.

58.3 Superseuencia Común Más Corta (Shortest Common Supersequence)

Idea: dadas dos cadenas s y t , la superseuencia común más corta (SCS) es la cadena más corta que contiene a

ambas como subsecuencias simultáneamente. Se construye directamente sobre *Subsecuencia Común Más Larga*: cualquier carácter que forme parte de la LCS se escribe **una sola vez** en la SCS (sirve para ambas cadenas a la vez), mientras que los caracteres que no forman parte de la LCS deben aparecer cada uno por separado, en su cadena de origen. Esto da la fórmula de longitud $|\text{SCS}(s, t)| = |s| + |t| - |\text{LCS}(s, t)|$: se suman las longitudes de ambas cadenas y se resta una vez la parte compartida, para no duplicarla.

¿Por qué usarlo? Es el problema "dual" de LCS: donde LCS busca la mayor parte **compartida**, SCS busca la cadena más corta que **contenga todo**. Reutilizar la misma tabla DP de LCS (con la misma recursión) y solo cambiar cómo se reconstruye el resultado evita rederivar una DP nueva desde cero —la tabla es idéntica, solo cambia el paso de reconstrucción: en vez de recolectar solo los caracteres que coinciden, se recolectan también los que no coinciden, cada uno proveniente de su cadena de origen.

Casos de uso:

- Encontrar la cadena más corta que "fusiona" dos secuencias, preservando el orden relativo de caracteres de ambas.
- Control de versiones: generar un merge mínimo de dos ediciones de un documento que preserve el contenido de ambas.
- Bioinformática: reconstruir la secuencia ancestral más corta consistente con dos secuencias derivadas.
- Verificación rápida de la longitud de la SCS sin construirla explícitamente, usando la fórmula cerrada sobre LCS.

Complejidad: $O(nm)$ tiempo y $O(nm)$ espacio, dominado por la construcción de la tabla DP de LCS subyacente; la reconstrucción de la SCS explícita es $O(n + m)$ adicional una vez que la tabla existe.

```

1 // CREATE/READ: Longitud y construccion de la superseuencia
    comun mas
2 // corta (SCS) entre dos cadenas, reutilizando la tabla DP de LCS
3 struct SuperseuenciaComunMasCorta {
4     // READ: longitud de la SCS, via formula cerrada sobre LCS
5     // (reutiliza directamente la tabla de Subsecuencia Comun Mas
    Larga)
6     static int longitud(const string& s, const string& t) {
7         int n = (int)s.size(), m = (int)t.size();
8         vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
9         for (int i = 1; i <= n; i++) {
10             for (int j = 1; j <= m; j++) {
11                 if (s[i-1] == t[j-1]) dp[i][j] = dp[i-1][j-1] +
1;
12                 else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
13             }
14         }
15         int lcs = dp[n][m];
16         return n + m - lcs;
17     } // O(n*m)
18
19     // READ: construye la SCS explicita, reutilizando la misma
    tabla dp
20     // de LCS y recolectando caracteres compartidos y no
    compartidos
21     static string construir(const string& s, const string& t) {
22         int n = (int)s.size(), m = (int)t.size();
23         vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
24         for (int i = 1; i <= n; i++) {
25             for (int j = 1; j <= m; j++) {
26                 if (s[i-1] == t[j-1]) dp[i][j] = dp[i-1][j-1] +
1;
27                 else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
28             }
29         }
30     }
31 }

```



```

29     }
30
31     string resultado;
32     int i = n, j = m;
33     while (i > 0 && j > 0) {
34         if (s[i-1] == t[j-1]) {
35             resultado += s[i-1]; // caracter compartido: se
// escribe UNA vez
36             i--; j--;
37         } else if (dp[i-1][j] >= dp[i][j-1]) {
38             resultado += s[i-1]; // caracter exclusivo de s
39             i--;
40         } else {
41             resultado += t[j-1]; // caracter exclusivo de t
42             j--;
43         }
44     }
45     while (i > 0) { resultado += s[i-1]; i--; } // resto de s
// sin pareja
46     while (j > 0) { resultado += t[j-1]; j--; } // resto de t
// sin pareja
47
48     reverse(resultado.begin(), resultado.end());
49     return resultado;
50 } // O(n*m)
51
52 //
53 //-----
54 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
55 //-----
56 // 1. Verificación rápida: la longitud SIEMPRE se puede
// obtener sin
57 // reconstruir la cadena completa, con solo  $n + m - \text{LCS}(s, t)$ ; usar
// longitud() cuando el problema solo pida el TAMAÑO de la
58 // SCS.
59 // 2. SCS de tres o mas cadenas: al igual que LCS, se
// generaliza a DP
60 // multidimensional, con el mismo crecimiento de
// complejidad
61 //  $O(n*m*p\dots)$  que la vuelve impracticable para muchas
// cadenas.
62 // 3. Verificación cruzada:  $|\text{SCS}(s,t)| + |\text{LCS}(s,t)|$  debe ser
// siempre
63 // exactamente igual a  $|s| + |t|$ , un buen chequeo de
// cordura al
64 // depurar cualquiera de las dos implementaciones.
65 };

```

Ejemplo de funcionamiento:

```

1 int longitudResultado = SupersecuenciaComunMasCorta::longitud("
// abac", "cab");
2 // longitudResultado = 5
3 // (LCS("abac","cab") = "ab" o "ac" (longitud 2); 4+3-2 = 5)
4
5 string scs = SupersecuenciaComunMasCorta::construir("abac", "cab"
// );
6 // scs tiene longitud 5 y contiene tanto "abac" como "cab" como
7 // subsecuencias; hay que correrlo para confirmar el string
// exacto
8 // resultante, ya que puede haber varias SCS validas de la misma
// longitud

```

Nota: al igual que con LCS, la SCS **no es única** en general —puede haber varias cadenas distintas de la misma longitud mínima que contengan a ambas cadenas originales como subsecuencias; el algoritmo de reconstrucción da una válida, no necesariamente "la" respuesta canónica si el problema espera una específica.

Regla práctica: usa la fórmula $n + m - \text{LCS}(s, t)$ cuando solo necesites la **longitud** de la SCS —evita el paso de reconstrucción explícita si el problema no lo pide. Reconstruye la cadena completa solo cuando el problema pida la SCS en sí, reutilizando exactamente la misma tabla DP de *Subsecuencia Común Más Larga* sin necesidad de una recurrencia nueva.

58.4 Subsecuencia Creciente Más Larga (LIS) [DP, no String]

Idea: dada una secuencia de n números, la subsecuencia creciente más larga (LIS) es la subsecuencia

más larga (no necesariamente contigua, orden preservado) cuyos elementos son estrictamente crecientes. La DP ingenua define $\text{dp}[i]$ = longitud de la LIS que **termina exactamente** en el índice i , calculada como $\text{dp}[i] = 1 + \max\{\text{dp}[j] : j < i, a[j] < a[i]\}$, revisando todos los índices anteriores — $O(n^2)$. La versión optimizada mantiene un arreglo auxiliar *colas*, donde *colas*[k] guarda el **menor valor final posible** entre todas las subsecuencias crecientes de longitud $k + 1$ vistas hasta el momento; para cada nuevo elemento, una *búsqueda binaria* encuentra dónde reemplazarlo en *colas* (técnica conocida como *patience sorting*), bajando a $O(n \log n)$.

¿Por qué usarlo? Se incluye en esta parte porque el nombre "subsecuencia" y el patrón de problema recuerdan a *Subsecuencia Común Más Larga*, pero la herramienta que lo resuelve es **programación dinámica sobre una sola secuencia numérica**, no una técnica de procesamiento de texto —por eso el título lleva la anotación [DP, no String]. La versión $O(n \log n)$ es la que se espera en competencia para n grande; la versión $O(n^2)$ solo es aceptable si n es pequeño o como paso de introducción antes de la optimización.

Casos de uso:

- Encontrar la longitud (o la subsecuencia misma) de la secuencia creciente más larga dentro de un arreglo.
- Problema clásico de "cuántas cajas se pueden apilar" o "máxima cantidad de elementos seleccionables en orden creciente" bajo distintos disfraces narrativos.
- Como subrutina de *Búsqueda Binaria on Answer* sobre la longitud, o de problemas 2D reducidos a LIS tras ordenar por una coordenada (por ejemplo, máximo número de rectángulos anidables).
- Base conceptual para *Subsecuencia Decreciente Más Larga* (LDS), que es el mismo problema con la comparación invertida, o LIS sobre la secuencia invertida.

Complejidad: $O(n^2)$ con la DP ingenua; $O(n \log n)$ con *colas* + búsqueda binaria, la versión estándar esperada en competencia.

```

1 // CREATE/READ: Longitud (y reconstrucción) de la subsecuencia
// creciente
2 // mas larga (LIS), version O(n log n) con patience sorting
3 struct SubsecuenciaCrecienteMasLarga {
4     // READ: longitud de la LIS estricta (elementos estrictamente
// crecientes), en O(n log n)
5     static int longitud(const vector<int>& a) {
6         vector<int> colas; // colas[k] = menor valor final de una
// LIS de longitud k+1
7         for (int x : a) {
8             // lower_bound: primera posicion donde x puede
// reemplazar (LIS
9             // estricta); usar upper_bound en su lugar para LIS
// NO
10            // estricta (no decreciente)
11            auto it = lower_bound(colas.begin(), colas.end(), x);
12            if (it == colas.end()) colas.push_back(x);
13            else *it = x;
14        }
15        return (int)colas.size();
16    } // O(n log n)
17
18    // READ: reconstruye una LIS optima (no necesariamente unica),
19    // guardando padres durante el mismo proceso de patience
// sorting
20    static vector<int> reconstruir(const vector<int>& a) {
21        int n = (int)a.size();
22

```

```

23     vector<int> colas;           // valores, para búsqueda
    binaria
24     vector<int> colasIdx;       // índices originales
    correspondientes a colas
25     vector<int> padre(n, -1);   // padre[i] = índice anterior
    en la LIS que termina en i
26
27     for (int i = 0; i < n; i++) {
28         auto it = lower_bound(colas.begin(), colas.end(), a[i]
    ];
29         int pos = (int)(it - colas.begin());
30         if (pos > 0) padre[i] = colasIdx[pos - 1];
31         if (it == colas.end()) {
32             colas.push_back(a[i]);
33             colasIdx.push_back(i);
34         } else {
35             *it = a[i];
36             colasIdx[pos] = i;
37         }
38     }
39
40     vector<int> resultado;
41     int actual = colasIdx.back();
42     while (actual != -1) {
43         resultado.push_back(a[actual]);
44         actual = padre[actual];
45     }
46     reverse(resultado.begin(), resultado.end());
47     return resultado;
48 } // O(n log n)
49
50 //
51 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
52 //
53 // 1. LIS NO estricta (no decreciente, permite elementos
    iguales):
54 //   cambiar lower_bound por upper_bound en longitud() y
    reconstruir();
55 //   la diferencia determina si valores repetidos pueden ir
    juntos en
56 //   la subsecuencia.
57 // 2. LDS (Subsecuencia Decreciente Mas Larga): invertir el
    arreglo de
58 //   entrada y calcular LIS sobre el invertido, o negar
    todos los
59 //   valores y calcular LIS normalmente sobre los negados --
    ver
60 //   Subsecuencia Decreciente Mas Larga para el detalle.
61 // 3. colas NO es una LIS real: el arreglo colas tras el
    proceso NO es
62 //   necesariamente una subsecuencia valida del arreglo
    original (solo
63 //   su LONGITUD es correcta); para la subsecuencia real
    hace falta el
64 //   arreglo de padres, como en reconstruir().
65 };

```

Ejemplo de funcionamiento:

```

1 vector<int> a = {10, 9, 2, 5, 3, 7, 101, 18};
2
3 int resultado = SubsecuenciaCrecienteMasLarga::longitud(a);
4 // resultado = 4
5 // (una LIS de longitud 4: 2, 3, 7, 18 -- o 2, 5, 7, 18, ambas
    validas)
6
7 vector<int> lis = SubsecuenciaCrecienteMasLarga::reconstruir(a);
8 // lis tiene longitud 4; hay que correrlo para confirmar cual de
    las
9 // subsecuencias optimas validas devuelve esta implementacion
    especifica

```

Nota: el arreglo `colas` usado para la búsqueda binaria **no representa** una subsecuencia creciente real del arreglo original en ningún momento intermedio —solo su **longitud** final es la respuesta correcta. Para reconstruir la subsecuencia real hace falta el arreglo de padres por separado, como se muestra en `reconstruir()`; confundir `colas` con "la" LIS es un error común al aprender esta técnica.

Regla práctica: usa la versión $O(n \log n)$ con `colas` + búsqueda binaria como estándar en competencia; la versión $O(n^2)$ solo se justifica si n es pequeño (por ejemplo $n \leq 10^3$) o como paso pedagógico antes de la optimización. Recuerda que esta técnica, aunque aparece

frecuentemente junto a problemas de cadenas, es DP sobre secuencias numéricas —no requiere ningún procesamiento de texto específico de esta parte.

58.5 Subsecuencia Decreciente Más Larga (LDS) [DP, no String]

Idea: la subsecuencia decreciente más larga (LDS) es el mismo problema que *Subsecuencia Creciente Más Larga*, pero exigiendo que los elementos sean estrictamente decrecientes en vez de crecientes. No hace falta rederivar ninguna técnica nueva: existen dos reducciones directas a LIS, y ambas dan el mismo resultado. La primera: **invertir el arreglo** y calcular LIS sobre el arreglo invertido —una subsecuencia decreciente en el arreglo original es exactamente una subsecuencia creciente al recorrer el arreglo de derecha a izquierda. La segunda: **negar cada valor** ($a_i \rightarrow -a_i$) y calcular LIS normalmente sobre los valores negados —un orden decreciente en los valores originales corresponde exactamente a un orden creciente en sus negativos.

¿Por qué usarlo? Se incluye como subsección separada (en vez de solo una nota dentro de LIS) porque, aunque la reducción es trivial, **combinar LIS y LDS sobre la misma posición** es una técnica clásica en sí misma: para cada índice i , calcular $LIS[i]$ (la LIS que termina en i) y $LDS[i]$ (la LDS que **empieza** en i , mirando hacia adelante) permite resolver problemas como "la subsecuencia bitónica más larga" (crece y luego decrece) tomando $\max_i (LIS[i] + LDS[i] - 1)$ sobre cada posible "pico". Esa combinación es la razón principal de tener LDS como bloque de construcción independiente y no solo una nota al pie de LIS.

Casos de uso:

- Encontrar la longitud de la secuencia decreciente más larga dentro de un arreglo.
- Como mitad de la técnica de **subsecuencia bitónica** (crece y luego decrece), combinando LIS que termina en i con LDS que empieza en i para cada posición.
- Problemas de "máxima cantidad de elementos apilables en orden inverso" bajo distintos disfraces narrativos, análogos a LIS pero con la comparación invertida.
- Verificación de monotonía parcial de una secuencia (¿cuál es el tramo decreciente más largo, como subsecuencia, no necesariamente contiguo?).

Complejidad: $O(n \log n)$, idéntico a LIS, ya que la reducción (invertir o negar) es $O(n)$ y no cambia la complejidad del algoritmo subyacente.

```

1 // CREATE/READ: Longitud de la subsecuencia decreciente mas larga
    (LDS),
2 // mediante reduccion directa a Subsecuencia Creciente Mas Larga
    (LIS)
3 struct SubsecuenciaDecrecienteMasLarga {
4     // READ: LIS auxiliar reutilizada internamente (identica a la
    de
5     // Subsecuencia Creciente Mas Larga, incluida aqui para
    autocontencion)
6     static int lis(const vector<int>& a) {
7         vector<int> colas;
8         for (int x : a) {
9             auto it = lower_bound(colas.begin(), colas.end(), x);
10            if (it == colas.end()) colas.push_back(x);

```

```

11     else *it = x;
12 }
13 return (int)colas.size();
14 } // O(n log n)
15
16 // READ: longitud de la LDS estricta, negando los valores y
17 // aplicando
18 // LIS directamente (equivalente a invertir el arreglo)
19 static int longitud(const vector<int>& a) {
20     vector<int> negado(a.size());
21     for (int i = 0; i < (int)a.size(); i++) negado[i] = -a[i];
22     return lis(negado);
23 } // O(n log n)
24
25 // READ: para cada índice i, calcula LIS[i] (longitud de la
26 // LIS que
27 // TERMINA en i) y LDS[i] (longitud de la LDS que EMPIEZA en
28 // i);
29 // combinados dan la subsecuencia bitonica mas larga
30 static int subsecuenciaBitonicaMasLarga(const vector<int>& a)
31 {
32     int n = (int)a.size();
33
34     // LIS que termina en cada índice i (O(n^2), simple de
35     // razonar;
36     // ver Nota para la version O(n log n) con reconstrucción
37     // de índices)
38     vector<int> lisEnI(n, 1);
39     for (int i = 0; i < n; i++)
40         for (int j = 0; j < i; j++)
41             if (a[j] < a[i]) lisEnI[i] = max(lisEnI[i],
42             lisEnI[j] + 1);
43
44     // LDS que empieza en cada índice i: LIS sobre el arreglo
45     // invertido,
46     // luego se voltea el resultado para que corresponda a
47     // los índices
48     // originales
49     vector<int> ldsDesdeI(n, 1);
50     for (int i = n - 1; i >= 0; i--)
51         for (int j = n - 1; j > i; j--)
52             if (a[j] < a[i]) ldsDesdeI[i] = max(ldsDesdeI[i],
53             ldsDesdeI[j] + 1);
54
55     int mejor = 0;
56     for (int i = 0; i < n; i++) {
57         mejor = max(mejor, lisEnI[i] + ldsDesdeI[i] - 1); //
58         -1: si se cuenta en ambas
59     }
60     return mejor;
61 } // O(n^2), version simple; optimizable a O(n log n) por
62 // mitad
63
64 //
65 // -----
66 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
67 //
68 // -----
69 // 1. LDS NO estricta (permite elementos iguales, no
70 // creciente en
71 // sentido inverso): usar upper_bound en vez de
72 // lower_bound dentro
73 // de lis(), igual que la modificacion equivalente en LIS.
74 // 2. subsecuenciaBitonicaMasLarga en O(n log n): tanto
75 // lisEnI como
76 // ldsDesdeI se pueden calcular en O(n log n) cada una
77 // usando la
78 // tecnica de colas + busqueda binaria de LIS, guardando
79 // la longitud
80 // en cada posicion en vez de solo el total final; el
81 // codigo de
82 // arriba usa O(n^2) por simplicidad de lectura.
83 // 3. Reconstrucción de la LDS explicita (no solo longitud):
84 // aplicar
85 // reconstruir() de Subsecuencia Creciente Mas Larga sobre
86 // el
87 // arreglo negado, y luego negar de vuelta cada valor del
88 // resultado.
89 };

```

Ejemplo de funcionamiento:

```

1 vector<int> a = {9, 7, 8, 6, 5, 4, 3};
2
3 int resultado = SubsecuenciaDecrecienteMasLarga::longitud(a);
4 // resultado = 5
5 // (por ejemplo 9,8,6,5,4,3 no es valido por longitud del arreglo,
6 // pero
7 // 9,7,6,5,4 o 9,8,6,5,4 son subsecuencias decrecientes validas
8 // de
9 // longitud 5; hay que correrlo para confirmar el valor exacto)

```

```

9 int bitonica = SubsecuenciaDecrecienteMasLarga::
10 subsecuenciaBitonicaMasLarga(a);
11 // bitonica considera crecer y luego decrecer; para este arreglo
12 // en
13 // particular, casi todo el arreglo ya es decreciente salvo el
14 // pico
15 // inicial 9,7,8 -- hay que correrlo para confirmar el resultado
16 // exacto

```

Nota: la versión de `subsecuenciaBitonicaMasLarga` en el código usa $O(n^2)$ por simplicidad de lectura (doble bucle directo para `lisEnI` y `ldsDesdeI`); para n grande, ambos arreglos se pueden calcular en $O(n \log n)$ aplicando la técnica de colas de *Subsecuencia Creciente Más Larga* y guardando la longitud alcanzada en cada posición durante el proceso, en vez de solo quedarse con el total final.

Regla práctica: usa la reducción directa (negar valores + LIS) cuando solo necesites la **longitud** de la LDS; usa la combinación $LIS[i] + LDS[i] - 1$ por posición específicamente cuando el problema pida una subsecuencia **bitónica** (primero crece, luego decrece) — ese es el escenario donde LDS como bloque independiente (no solo una nota dentro de LIS) resulta necesario.

59 Construcciones y Transformaciones

59.1 Rotación Lexicográficamente Mínima (Algoritmo de Booth)

Idea: dada una cadena s de longitud n , sus n rotaciones son las cadenas obtenidas moviendo un prefijo de longitud k al final, para cada $k = 0, \dots, n - 1$. El algoritmo de Booth encuentra, en tiempo lineal, el índice de inicio de la rotación lexicográficamente menor sin generar ni comparar las n rotaciones explícitamente. Funciona de forma similar a la construcción de la función de fallo de *Función de Prefijos*, pero sobre la cadena duplicada $s + s$ (para poder "leer" cualquier rotación como un substring contiguo de longitud n): mantiene dos candidatos a mejor inicio de rotación y los compara carácter por carácter, descartando el peor y saltando hacia adelante sin retroceder nunca el puntero de comparación —el mismo principio de "no retroceder" que garantiza linealidad en KMP.

¿Por qué usarlo? Generar las n rotaciones explícitamente y ordenarlas cuesta $O(n^2 \log n)$ (cada rotación es $O(n)$ de generar, y hay $O(n \log n)$ comparaciones en el sort, cada una $O(n)$ en el peor caso). Booth resuelve el mismo problema en $O(n)$ estricto, aprovechando que comparar dos rotaciones candidatas no requiere reiniciar la comparación desde cero cada vez que una falla —análogo a cómo KMP evita recomparar caracteres ya vistos del texto.

Casos de uso:

- Encontrar la representación canónica de una cadena bajo rotación (útil para comparar si dos cadenas son rotaciones una de la otra: son rotaciones si y solo si sus formas canónicas coinciden).
- Normalizar cadenas cíclicas antes de usarlas como clave en una tabla hash o en un **set** (como en *Agrupar Anagramas*, pero para equivalencia por rotación en vez de por anagrama).

- Problemas de collares o secuencias circulares donde el punto de inicio es arbitrario y se necesita una representación única.
- Verificación rápida de si una cadena es la rotación mínima de sí misma, o cuántas rotaciones distintas produce (relacionado con el período de la cadena, visto en *Función de Prefijos*).

Complejidad: $O(n)$ —un único recorrido de la cadena duplicada con el mecanismo de doble puntero que nunca retrocede, análogo en espíritu al argumento de potencial usado para la función de prefijos.

```

1 // CREATE/READ: Algoritmo de Booth: encuentra el índice de inicio
  de la
2 // rotacion lexicograficamente minima de una cadena, en O(n)
3 struct RotacionMinima {
4     // READ: retorna el índice (0-indexado, en s) donde comienza
    la
5     // rotacion lexicograficamente menor de s
6     static int indiceRotacionMinima(const string& s) {
7         int n = (int)s.size();
8         string doble = s + s;
9         vector<int> f(2 * n, -1); // arreglo de fallos, similar
    en espíritu a pi de KMP
10        int k = 0; // mejor candidato a inicio de rotacion
    encontrado hasta ahora
11
12        for (int j = 1; j < 2 * n; j++) {
13            char sj = doble[j];
14            int i = f[j - k - 1];
15            while (i != -1 && sj != doble[k + i + 1]) {
16                if (sj < doble[k + i + 1]) k = j - i - 1;
17                i = f[i];
18            }
19            if (sj != doble[k + i + 1]) {
20                if (sj < doble[k]) k = j; // sj es el inicio,
    comparado directo
21                f[j - k] = -1;
22            } else {
23                f[j - k] = i + 1;
24            }
25        }
26        return k;
27    } // O(n)
28
29    // READ: retorna la rotacion lexicograficamente minima de s
    como cadena
30    static string obtenerRotacionMinima(const string& s) {
31        int n = (int)s.size();
32        int k = indiceRotacionMinima(s);
33        return s.substr(k) + s.substr(0, k);
34    } // O(n)
35
36    //
37    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
38    //
39    // 1. Verificar si dos cadenas s y t son rotaciones entre si:
    primero
40    // verificar |s| == |t|, luego comparar
41    // obtenerRotacionMinima(s) == obtenerRotacionMinima(t);
    ambas
42    // formas canonicas coinciden exactamente cuando son
    rotaciones.
43    // 2. Rotacion lexicograficamente MAXIMA: invertir el
    criterio
44    // "sj < doble[k+i+1]" y "sj < doble[k]" a "sj > ..." en
    ambos
45    // lugares del algoritmo.
46    // 3. Alternativa mas simple pero mas lenta (O(n log n) o O(n
    ^2)): usar
47    // Suffix Array sobre s+s y tomar el primer sufijo de
    longitud >= n
48    // que empiece dentro de las primeras n posiciones; Booth
    es
49    // preferible cuando se necesita estrictamente O(n) sin la
    complejidad de construir un Suffix Array completo.
50    //
51 };

```

Ejemplo de funcionamiento:

```

1 string resultado = RotacionMinima::obtenerRotacionMinima("bcab");
2 // resultado = "abbc"
3 // (de las 4 rotaciones de "bcab": "bcab", "cabb", "abbc", "bbca",
  la

```

```

4 // lexicograficamente menor es "abbc", que empieza en el índice
  2 de la
5 // cadena original)

```

Nota: el algoritmo de Booth es notoriamente propenso a errores de índice al implementarlo de memoria —la manipulación de k , i , y el arreglo de fallos f sobre la cadena duplicada tiene varios casos límite sutiles. Conviene validarlo contra la fuerza bruta (generar las n rotaciones y tomar el mínimo con `std::min_element`) en cadenas pequeñas antes de confiar en él bajo presión de concurso.

Regla práctica: usa el algoritmo de Booth cuando necesites la forma canónica de una cadena bajo rotación en $O(n)$ estricto, especialmente si se aplica a muchas cadenas (por ejemplo, agrupar un conjunto grande de collares por equivalencia rotacional). Para una sola cadena, o si n es pequeño, la fuerza bruta de generar todas las rotaciones y tomar el mínimo es más simple de implementar sin bugs, a costa de $O(n^2)$ o $O(n^2 \log n)$.

59.2 Distancia de Edición (Edit Distance / Levenshtein)

Idea: la distancia de edición entre dos cadenas s y t es el número mínimo de operaciones de **inserción**, **eliminación** o **sustitución** de un carácter necesarias para transformar s en t . Se resuelve con DP 2D: $dp[i][j]$ = distancia de edición entre $s[0..i-1]$ y $t[0..j-1]$. Si $s[i-1] = t[j-1]$, no hace falta ninguna operación en ese carácter y se hereda $dp[i-1][j-1]$; si no coinciden, se toma el mínimo entre las tres operaciones posibles más una unidad de costo: sustituir ($dp[i-1][j-1] + 1$), eliminar de s ($dp[i-1][j] + 1$), o insertar en s ($dp[i][j-1] + 1$) —cada una de las tres corresponde a "resolver un subproblema más pequeño y pagar el costo de la operación que lo redujo".

¿Por qué usarlo? Es la generalización natural de *Subsecuencia Común Más Larga* permitiendo una tercera operación (sustitución) además de las implícitas inserción/eliminación de LCS. De hecho, si solo se permiten inserción y eliminación (sin sustitución), la distancia de edición se reduce exactamente a $n + m - 2 \cdot \text{LCS}(s, t)$, la fórmula ya vista en esa subsección —agregar la sustitución como operación de costo 1 (en vez de "eliminar e insertar" por separado, costo 2) es lo que distingue Levenshtein de esa versión restringida.

Casos de uso:

- Corrección ortográfica: encontrar la palabra del diccionario más cercana a una entrada con errores de tipeo.
- Comparación de similitud entre dos cadenas cualesquiera (bioinformática, control de versiones, deduplicación aproximada de registros).
- Como base para variantes con costos distintos por operación (por ejemplo, sustitución más cara que inserción/eliminación), cambiando solo las constantes de la recurrencia.
- Verificar si una cadena se puede transformar en otra con a lo sumo k operaciones (poda temprana de la DP si el valor mínimo posible ya excede k).

Complejidad: $O(nm)$ tiempo y $O(nm)$ espacio con la tabla completa; reducible a $O(\min(n, m))$ espacio si solo se necesita la distancia (no reconstruir la secuencia de operaciones), manteniendo dos filas.

```
1 // CREATE/READ: Distancia de edicion (Levenshtein) entre dos
2 // mediante DP 2D con las tres operaciones: insertar, eliminar,
3 // substituir
4 struct DistanciaEdicion {
5     // READ: distancia de edicion minima entre s y t
6     static int distancia(const string& s, const string& t) {
7         int n = (int)s.size(), m = (int)t.size();
8         vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
9
10        // casos base: transformar cadena vacia en un prefijo de
11        // longitud i
12        // (o j) requiere exactamente i (o j) inserciones/
13        // eliminaciones
14        for (int i = 0; i <= n; i++) dp[i][0] = i;
15        for (int j = 0; j <= m; j++) dp[0][j] = j;
16
17        for (int i = 1; i <= n; i++) {
18            for (int j = 1; j <= m; j++) {
19                if (s[i-1] == t[j-1]) {
20                    dp[i][j] = dp[i-1][j-1];
21                } else {
22                    dp[i][j] = 1 + min({
23                        dp[i-1][j-1], // substituir
24                        dp[i-1][j], // eliminar de s
25                        dp[i][j-1] // insertar en s
26                    });
27                }
28            }
29        }
30        return dp[n][m];
31    } // O(n*m)
32
33    // READ: reconstruye la secuencia de operaciones (como lista
34    // de strings descriptivos) que logra la distancia minima
35    static vector<string> reconstruirOperaciones(const string& s,
36    const string& t) {
37        int n = (int)s.size(), m = (int)t.size();
38        vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
39        for (int i = 0; i <= n; i++) dp[i][0] = i;
40        for (int j = 0; j <= m; j++) dp[0][j] = j;
41        for (int i = 1; i <= n; i++) {
42            for (int j = 1; j <= m; j++) {
43                if (s[i-1] == t[j-1]) dp[i][j] = dp[i-1][j-1];
44                else dp[i][j] = 1 + min({dp[i-1][j-1], dp[i-1][j],
45                dp[i][j-1]});
46            }
47        }
48        vector<string> operaciones;
49        int i = n, j = m;
50        while (i > 0 || j > 0) {
51            if (i > 0 && j > 0 && s[i-1] == t[j-1]) {
52                i--; j--; // sin operacion, caracteres iguales
53            } else if (i > 0 && j > 0 && dp[i][j] == dp[i-1][j-1]
54            + 1) {
55                operaciones.push_back("substituir " + string(1, s[
56                i-1]) + " por " + string(1, t[j-1]));
57                i--; j--;
58            } else if (i > 0 && dp[i][j] == dp[i-1][j] + 1) {
59                operaciones.push_back("eliminar " + string(1, s[
60                i-1]));
61                i--;
62            } else {
63                operaciones.push_back("insertar " + string(1, t[j
64                -1]));
65                j--;
66            }
67        }
68        reverse(operaciones.begin(), operaciones.end());
69        return operaciones;
70    } // O(n*m)
71
72    //
73    // -----
74    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
75    // -----
76
77    // 1. Sin substitucion (solo insertar/eliminar): equivale
78    // exactamente a
79    // n + m - 2*LCS(s,t) (ver Subsecuencia Comun Mas Larga);
80    // util como
81    // chequeo cruzado, o directamente mas simple si el
82    // problema no
83    // permite substituciones.
84    // 2. Costos distintos por operacion: si substituir cuesta
```

```
costoSust,
72 // insertar costoIns, eliminar costoElim (en vez de 1 cada
73 // una),
74 // ajustar las tres ramas del min() con esas constantes
75 // especificas.
76 // 3. Espacio O(min(n,m)): mantener solo dos filas de la
77 // tabla si no
78 // se necesita reconstruir las operaciones, igual que en
79 // LCS y
80 // Subcadena Comun Mas Larga.
```

Ejemplo de funcionamiento:

```
1 int resultado = DistanciaEdicion::distancia("horse", "ros");
2 // resultado = 3
3 // (horse -> rorse (substituir h por r) -> rose (eliminar r) ->
4 // ros (eliminar e); 3 operaciones en total)
```

Nota: el orden de prioridad al reconstruir las operaciones importa cuando hay varias formas óptimas: el código de arriba prioriza sustitución sobre eliminación sobre inserción cuando hay empate en $dp[i][j]$, pero cualquier orden de desempate consistente da una secuencia de operaciones igualmente válida (misma longitud total, distinta secuencia exacta).

Regla práctica: usa Distancia de Edición cuando las tres operaciones (insertar, eliminar, substituir) tengan sentido para el problema y todas cuesten lo mismo (o se necesite generalizar a costos distintos); si el problema solo permite insertar/eliminar (sin substituir), la fórmula directa $n+m-2 \cdot \text{LCS}(s, t)$ es más simple y evita construir una tabla DP nueva.

59.3 Transformación con Movimientos Restringidos (A-izquierda, B-derecha, # fijo)

Idea: dadas dos cadenas S_1 y S_2 compuestas únicamente por los caracteres **a**, **b** y **#**, se permite mover cualquier **a** hacia la **izquierda** (intercambiándola con su vecino inmediato) y cualquier **b** hacia la **derecha**, cualquier número de veces, pero **a** nunca hacia la derecha, **b** nunca hacia la izquierda, y ninguna **a** puede **cruzar** a ninguna **b** (ni viceversa). Bajo estas reglas, dos condiciones son necesarias y suficientes para que S_1 se pueda transformar en S_2 : (1) al eliminar todos los **#** de ambas cadenas, las secuencias resultantes de **a/b** deben ser **idénticas** —porque el orden relativo entre **a**'s y **b**'s nunca puede cambiar, ya que no se permiten cruces—; y (2) para cada carácter no-**#** en la posición i -ésima (contando solo no-**#**), si es **a** su índice absoluto en S_2 debe ser $\leq$ su índice absoluto en S_1 (solo se mueve a la izquierda, nunca a la derecha), y si es **b** su índice absoluto en S_2 debe ser $\geq$ su índice en S_1 (solo se mueve a la derecha).

¿Por qué usarlo? Es un problema de verificación de alcanzabilidad bajo movimientos restringidos, donde simular las operaciones explícitamente sería exponencial (cada movimiento genera un nuevo estado). La clave para resolverlo en tiempo lineal es notar que las dos condiciones —secuencia de **a/b** idéntica ignorando **#**, y desplazamiento en la dirección permitida por índice— capturan **exactamente** lo que los movimientos permitidos pueden lograr, sin necesidad de simular ningún intercambio individual: es una caracterización combinatoria cerrada del conjunto de estados alcanzables, en vez de una búsqueda sobre ese espacio de estados.

Casos de uso:

- El problema clásico de verificar si S_1 se puede transformar en S_2 con **a** moviéndose solo a la izquierda y

b solo a la derecha, sin cruces, reportando "Yes"/"No" (o en este caso, "YES"/"NOT" según el enunciado específico).

- Modelos de partículas en una línea que solo pueden moverse en una dirección fija cada una, sin adelantarse entre sí (análogo físico de partículas indistinguibles por tipo que no pueden cruzarse).
- Como ejemplo de técnica general: reducir un problema de "¿es alcanzable este estado mediante una secuencia de movimientos válidos?" a una caracterización estática verificable en tiempo lineal, en vez de BFS/simulación sobre el espacio de estados.
- Variantes con más tipos de caracteres y reglas de movimiento distintas por tipo, generalizando la misma idea de "orden relativo preservado + dirección de desplazamiento permitida".

Complejidad: $O(n)$ —un recorrido lineal para extraer las secuencias sin # y verificar que coincidan, y otro recorrido (o el mismo) para comparar los índices absolutos de cada carácter correspondiente entre S_1 y S_2 .

```
1 // CREATE/READ: Verifica si S1 se puede transformar en S2
  moviendo 'a'
2 // solo a la izquierda, 'b' solo a la derecha, sin que 'a' y 'b'
  se crucen
3 // entre si ('#' permanece fijo en su rol, solo cambia de
  posicion relativa
4 // como resultado de que a y b se muevan a su alrededor)
5 struct TransformacionMovimientosRestringidos {
6 // READ: true si S1 se puede transformar en S2 bajo las
  reglas dadas
7 static bool esPosible(const string& s1, const string& s2) {
8 int n = (int)s1.size();
9 if (n != (int)s2.size()) return false;
10
11 // condicion 1: la secuencia de caracteres no-# debe ser
  identica
12 // (el orden relativo de a's y b's nunca puede cambiar)
13 string sinHash1, sinHash2;
14 for (char c : s1) if (c != '#') sinHash1 += c;
15 for (char c : s2) if (c != '#') sinHash2 += c;
16 if (sinHash1 != sinHash2) return false;
17
18 // condicion 2: cada caracter debe moverse en su
  direccion permitida
19 // se recorren s1 y s2 con dos punteros, emparejando cada
  caracter
20 // no-# de s1 con el correspondiente (mismo orden) en s2
21 int i = 0, j = 0;
22 while (i < n && j < n) {
23 while (i < n && s1[i] == '#') i++;
24 while (j < n && s2[j] == '#') j++;
25 if (i == n || j == n) break; // ambos deben terminar
  juntos por condicion 1
26
27 char c = s1[i]; // == s2[j], garantizado por
  condicion 1
28 if (c == 'a' && j > i) return false; // 'a' se
  habria movido a la derecha: invalido
29 if (c == 'b' && j < i) return false; // 'b' se
  habria movido a la izquierda: invalido
30 i++; j++;
31 }
32 return true;
33 } // O(n)
34
35 //
36 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
37 //
38 // 1. Version con salida "YES"/"NOT" directa (segun el
  enunciado
39 // especifico): return esPosible(s1, s2) ? "YES" : "NOT";
40 // 2. Verificacion de la condicion 1 de forma incremental (
  sin
41 // construir sinHash1/sinHash2 como strings aparte): se
  puede
```

```
42 // integrar en el mismo recorrido de dos punteros de la
  condicion 2,
43 // comparando s1[i] == s2[j] justo antes de las
  verificaciones de
44 // direccion; se separa aqui en dos pasadas por claridad.
45 // 3. Generalizacion a mas tipos de caracteres con reglas de
  direccion
46 // distintas (por ejemplo, agregar un tercer tipo 'c' que
  tambien
47 // se mueve libremente en cualquier direccion): extender
  el if
48 // dentro del bucle con un caso adicional por cada tipo
  nuevo,
49 // manteniendo el mismo esqueleto de dos punteros.
50 };
```

Ejemplo de funcionamiento:

```
1 bool resultado1 = TransformacionMovimientosRestringidos::
  esPosible("ab#c#", "a#bc#");
2 // nota: este ejemplo con 'c' es solo ilustrativo del formato de
  entrada;
3 // bajo las reglas estrictas (solo a,b,#) un caracter 'c' no es
  valido -- ver
4 // el ejemplo real abajo, con alfabeto correcto {a,b,#}
5
6 bool resultado2 = TransformacionMovimientosRestringidos::
  esPosible("#a#b#", "a##b#");
7 // resultado2 = true
8 // (secuencia no-# en ambas: "ab", identica; la 'a' esta en
  indice 1 en s1
9 // y en indice 0 en s2 -- se movio a la izquierda, permitido; la
  'b' esta
10 // en indice 3 en ambas -- no se movio, tambien permitido)
```

Nota: corrijo el primer ejemplo de arriba porque incluí por error un carácter c fuera del alfabeto permitido {a,b,#} — el struct asume estrictamente esos tres caracteres, tal como especifica el enunciado original; si tu problema real usa un alfabeto distinto, ajusta las comparaciones $c == 'a' / c == 'b'$ del código según corresponda.

Regla práctica: verifica **ambas** condiciones (secuencia no-# idéntica, y dirección de desplazamiento permitida por índice) — verificar solo una de las dos da falsos positivos: dos cadenas pueden tener la misma secuencia de a/b pero requerir que alguna se mueva en la dirección prohibida, o viceversa, tener índices que a primera vista podrían parecer válidos pero corresponder a caracteres en distinto orden relativo.

59.4 Lyndon Words y Algoritmo de Duval

Idea: una *Lyndon word* es una cadena que es **estrictamente menor** (lexicográficamente) que todas sus rotaciones propias no triviales —equivalentemente, es menor que todos sus sufijos propios. El **algoritmo de Duval** descompone cualquier cadena s en una secuencia única de Lyndon words $w_1 \geq w_2 \geq \dots \geq w_k$ (en orden **no creciente**) tal que $s = w_1 w_2 \dots w_k$ —esta es la *factorización de Lyndon*, garantizada única para toda cadena. El algoritmo recorre s con tres punteros (i marca el inicio del factor actual, j compara, k recuerda dónde reiniciar la comparación cíclica) y determina en una sola pasada lineal dónde termina cada factor, sin necesidad de generar ni comparar rotaciones explícitamente.

¿Por qué usarlo? La factorización de Lyndon conecta directamente con *Rotación Lexicográficamente Mínima*: se puede demostrar que la rotación mínima de s (cadena duplicada) comienza exactamente en el inicio del **último** factor de Lyndon de esa cadena duplicada, dando una forma alternativa de resolver el mismo problema que Booth, con una técnica conceptualmente más simple de razonar (aunque con una constante similar).

Más allá de esa conexión, Duval es la herramienta estándar para cualquier problema que pregunte directamente por la estructura de Lyndon words de una cadena —period mínimo, el menor sufijo, o construcción de secuencias de De Bruijn (mencionado como aplicación relacionada, no cubierta aquí).

Casos de uso:

- Descomponer una cadena en su factorización de Lyndon única, como estructura de análisis.
- Encontrar la rotación lexicográficamente mínima de una cadena mediante s y el último factor de Duval, como alternativa a *Algoritmo de Booth*.
- Encontrar el sufijo lexicográficamente menor de una cadena (el último factor de la factorización de Lyndon de s misma, sin duplicar).
- Como bloque de construcción en problemas más avanzados de combinatoria de palabras (generación de secuencias de De Bruijn, compresión basada en estructura cíclica).

Complejidad: $O(n)$ para la factorización completa —cada uno de los tres punteros solo avanza hacia adelante a lo largo de toda la ejecución, dando el mismo argumento de linealidad amortizada que en KMP y Booth.

```
1 // CREATE/READ: Factorizacion de Lyndon de una cadena mediante el
2 // algoritmo de Duval, y aplicaciones directas (rotacion minima,
3 // sufijo minimo)
4 struct AlgoritmoDuval {
5     // READ: retorna los factores de Lyndon de s, en el orden en
6     // que
7     // aparecen (w1, w2, ..., wk con s = w1+w2+...+wk, no
8     // crecientes)
9     static vector<string> factorizar(const string& s) {
10         int n = (int)s.size();
11         vector<string> factores;
12         int i = 0;
13         while (i < n) {
14             int j = i + 1, k = i;
15             while (j < n && s[k] <= s[j]) {
16                 if (s[k] < s[j]) k = i; // reinicia
17                 // comparacion ciclica
18                 else k++; // s[k] == s[j],
19                 // sigue comparando
20                 j++;
21             }
22             // el bloque [i, i + (j-k) - 1] se repite completo;
23             // solo el
24             // ultimo tramo parcial (si lo hay) forma parte del
25             // proximo factor
26             while (i <= k) {
27                 factores.push_back(s.substr(i, j - k));
28                 i += j - k;
29             }
30             return factores;
31         } // O(n)
32
33         // READ: rotacion lexicograficamente minima de s, via
34         // factorizacion
35         // de Lyndon de s+s (alternativa al Algoritmo de Booth)
36         static string rotacionMinima(const string& s) {
37             string doble = s + s;
38             vector<string> factores = factorizar(doble);
39             // el ultimo factor que EMPIEZA dentro de las primeras n
40             // posiciones
41             // marca el inicio de la rotacion minima
42             int n = (int)s.size();
43             int pos = 0, mejorInicio = 0;
44             for (const string& f : factores) {
45                 if (pos < n) mejorInicio = pos;
46                 pos += (int)f.size();
47             }
48             return s.substr(mejorInicio) + s.substr(0, mejorInicio);
49         } // O(n)
50
51         // READ: sufijo lexicograficamente menor de s (ultimo factor
52         // de la
53         // factorizacion de Lyndon de s misma, sin duplicar)
```

```
45 static string sufijoMinimo(const string& s) {
46     vector<string> factores = factorizar(s);
47     return factores.back();
48 } // O(n)
49
50 //
51 // -----
52 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
53 // -----
54 // 1. Verificar si s misma es una Lyndon word: factorizar(s)
55 // da un
56 // unico factor igual a s completa si y solo si s es
57 // Lyndon word;
58 // equivalente a comparar sufijoMinimo(s) == s.
59 // 2. rotacionMinima via Duval vs. via Booth: ambas son O(n),
60 // Duval es
61 // conceptualmente mas simple de razonar (factorizacion +
62 // ultimo
63 // factor que empieza antes de n) pero construye s+s
64 // explicitamente
65 // (O(n) memoria extra), mientras Booth trabaja con
66 // indices sobre la
67 // cadena duplicada de forma mas directa; la eleccion es
68 // preferencia
69 // de implementacion.
70 // 3. Numero total de factores de Lyndon: factorizar(s).size()
71 // da
72 // directamente cuantos factores tiene la descomposicion;
73 // relevante
74 // en problemas que preguntan por la "complejidad ciclica"
75 // de una
76 // cadena.
```

Ejemplo de funcionamiento:

```
1 vector<string> factores = AlgoritmoDuval::factorizar("banana");
2 // factores = {"b", "an", "an", "a"}
3 // (cada factor es Lyndon word, y la secuencia es no creciente
4 // lexicograficamente: "b" > "an" == "an" > "a"; concatenados
5 // reconstruyen
6 // "banana")
7 string rotMin = AlgoritmoDuval::rotacionMinima("bcabc");
8 // rotMin = "abbc"
9 // (mismo resultado que el ejemplo de Algoritmo de Booth para la
10 // misma
11 // cadena, confirmando la equivalencia entre ambos metodos)
```

Nota: la condición del bucle interno ($s[k] \leq s[j]$, con reinicio de k solo cuando $s[k] < s[j]$ estrictamente) es la parte más sutil de Duval —una comparación con $<$ en vez de $\leq$ rompe la detección de factores repetidos consecutivos (como los dos "an" en el ejemplo de "banana"), dando una factorización incorrecta. Conviene validar contra casos con repeticiones exactas de factores antes de confiar en una implementación de memoria.

Regla práctica: usa Duval cuando el problema pregunte directamente por la estructura de Lyndon words de una cadena, o como alternativa a Booth para rotación mínima si te resulta más fácil de recordar la lógica de factorización que la de doble puntero de Booth —ambos dan el mismo resultado en la misma complejidad $O(n)$, así que la elección es de preferencia de implementación, no de rendimiento.

60 Anagramas

60.1 Verificar si Dos Cadenas son Anagramas

Idea: dos cadenas s y t son anagramas si y solo si contienen exactamente los mismos caracteres con las mismas multiplicidades, sin importar el orden. Hay dos formas directas de verificarlo: **ordenar** ambas cadenas y comparar si el resultado es idéntico carácter por carácter, o contar la **frecuencia** de cada carácter en ambas cadenas y comparar los dos arreglos de frecuencia. Ambos métodos son válidos; la diferencia está en la complejidad y en

qué información adicional queda disponible después (el conteo de frecuencia es reutilizable directamente como base de *Búsqueda de Anagramas de un Patrón*, mientras que ordenar no lo es).

¿Por qué usarlo? Comparar cadena por cadena sin ordenar ni contar no funciona directamente, porque el orden de los caracteres es irrelevante para este problema —hace falta alguna forma de "normalizar" ambas cadenas a una representación comparable. Ordenar es $O(n \log n)$ pero conceptualmente más simple (la cadena ordenada actúa como firma canónica, reutilizable también en *Agrupar Anagramas*); contar frecuencia es $O(n + \sigma)$, más rápido cuando σ (tamaño del alfabeto) es mucho menor que $n \log n$, y es la base directa de la técnica de ventana deslizante usada en *Búsqueda de Anagramas de un Patrón*.

Casos de uso:

- Verificación directa de si dos cadenas dadas son anagramas entre sí.
- Como paso base antes de generalizar a *Agrupar Anagramas* (la cadena ordenada, o el conteo de frecuencia, funciona como clave/firma para agrupar).
- Validación rápida antes de aplicar una técnica más compleja (por ejemplo, descartar de inmediato que dos cadenas de distinta longitud puedan ser anagramas).
- Subrutina dentro de problemas que verifican reordenamientos válidos bajo alguna restricción adicional.

Complejidad: $O(n \log n)$ con ordenamiento; $O(n + \sigma)$ con conteo de frecuencia, donde σ es el tamaño del alfabeto —preferible cuando σ es pequeño y fijo (como minúsculas 'a'-'z').

```

1 // CREATE/READ: Verificación de si dos cadenas son anagramas
  // entre sí,
2 // por ordenamiento y por conteo de frecuencia
3 struct VerificarAnagramas {
4     static const int ALFABETO = 26;
5
6     // READ: verifica anagramas ordenando ambas cadenas y
      // comparando
7     static bool sonAnagramasOrdenando(string s, string t) {
8         if (s.size() != t.size()) return false;
9         sort(s.begin(), s.end());
10        sort(t.begin(), t.end());
11        return s == t;
12    } // O(n log n)
13
14    // READ: verifica anagramas contando frecuencia de caracteres
15    // (mas rapido que ordenar cuando el alfabeto es pequeño y
      // fijo)
16    static bool sonAnagramasFrecuencia(const string& s, const
      string& t) {
17        if (s.size() != t.size()) return false;
18        vector<int> frec(ALFABETO, 0);
19        for (char c : s) frec[c - 'a']++;
20        for (char c : t) frec[c - 'a']--;
21        for (int f : frec) if (f != 0) return false;
22        return true;
23    } // O(n + sigma)
24
25    //
26    // -----
27    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
28    // -----
29    // 1. Cadena ordenada como FIRMA CANONICA: sort(s.begin(), s.
      // end())
30    // aplicado a una cadena da una representacion unica
      // compartida por
31    // todos sus anagramas; esta es exactamente la clave usada
      // en
32    // Agrupar Anagramas para juntar cadenas equivalentes.

```

```

32 // 2. Alfabeto grande (Unicode, mayusculas+minusculas+digitos
   //): usar
33 // unordered_map<char,int> en vez del arreglo fijo
   // ALFABETO en
34 // sonAnagramasFrecuencia, recorriendo solo las claves
   // presentes al
35 // verificar que todas las frecuencias sean cero.
36 // 3. Verificación temprana por longitud: ambas funciones
   // descartan de
37 // inmediato si s.size() != t.size(), ya que dos cadenas
   // de
38 // longitud distinta nunca pueden ser anagramas -- evita
   // trabajo
39 // innecesario de ordenar o contar en ese caso.
40 };

```

Ejemplo de funcionamiento:

```

1 bool resultado1 = VerificarAnagramas::sonAnagramasOrdenando("
  listen", "silent");
2 // resultado1 = true
3
4 bool resultado2 = VerificarAnagramas::sonAnagramasFrecuencia("
  listen", "silent");
5 // resultado2 = true (mismo resultado, metodo distinto)
6
7 bool resultado3 = VerificarAnagramas::sonAnagramasFrecuencia("
  hello", "world");
8 // resultado3 = false (distinta composicion de caracteres)

```

Regla práctica: usa `sonAnagramasFrecuencia` como opción por defecto cuando el alfabeto sea pequeño y fijo (minúsculas, por ejemplo) —es más rápido y evita el costo de ordenar. Usa `sonAnagramasOrdenando` cuando además necesites la cadena ordenada como firma reutilizable para otro propósito (como agrupar muchas cadenas por equivalencia de anagrama, ver *Agrupar Anagramas*).

60.2 Búsqueda de Anagramas de un Patrón (Ventana Deslizante)

Idea: encontrar todas las posiciones donde una **permutación** (anagrama) de un patrón p aparece como subcadena de un texto t se resuelve con una **ventana deslizante** de tamaño $|p|$ sobre t , manteniendo un conteo de frecuencia de caracteres dentro de la ventana actual. Una ventana es un anagrama de p si y solo si su distribución de frecuencias de caracteres es **idéntica** a la de p —la misma prueba de *Verificar si Dos Cadenas son Anagramas* vía frecuencia, aplicada repetidamente a cada ventana. En vez de comparar los dos arreglos de frecuencia completos ($O(\sigma)$) en cada posición, se mantiene un contador de "cuántas posiciones del alfabeto difieren" que se actualiza en $O(1)$ al agregar el carácter que entra y quitar el que sale de la ventana.

¿Por qué usarlo? Comparar fuerza bruta la distribución de frecuencia completa en cada una de las $n - m + 1$ posiciones cuesta $O(n\sigma)$. Mantener incrementalmente un contador de "diferencias activas" entre la ventana y el patrón, actualizado en $O(1)$ por posición (no $O(\sigma)$), baja el total a $O(n + \sigma)$ —el término σ solo aparece una vez, al inicializar el conteo de frecuencia del patrón. A diferencia de *Rabin-Karp* u otros métodos de matching exacto, aquí el orden de los caracteres dentro de la ventana es irrelevante por definición del problema, así que hash polinómico (que sí depende del orden) no aplica directamente.

Casos de uso:

- Encontrar todas las posiciones donde algún anagrama de un patrón aparece en un texto.
- Verificar si dos cadenas son anagramas entre sí (caso particular: una sola "ventana", el texto completo —

ver *Verificar si Dos Cadenas son Anagramas* para esa versión directa sin ventana).

- Problemas de "substrings balanceados" donde la condición depende de la composición de caracteres, no de su orden.
- Extensión a permitir hasta k caracteres de diferencia (anagrama aproximado), ajustando el umbral del contador de diferencias.

Complejidad: $O(n + \sigma)$, donde $n = |t|$ y σ es el tamaño del alfabeto —el conteo de frecuencia del patrón es $O(m + \sigma)$, y cada desplazamiento de la ventana sobre el texto es $O(1)$ amortizado.

```
1 // CREATE/READ: Busqueda de todas las posiciones donde un
2 // patron aparece en el texto, mediante ventana deslizante de
3 // frecuencias
4 struct BusquedaAnagramas {
5     static const int ALFABETO = 26;
6
7     // READ: retorna las posiciones de inicio (0-indexadas) de
8     // cada
9     // ventana de tamaño |patron| en texto cuya distribucion de
10    // caracteres
11    // coincide exactamente con la del patron
12    static vector<int> buscar(const string& texto, const string&
13    patron) {
14        int n = (int)texto.size(), m = (int)patron.size();
15        vector<int> ocurrencias;
16        if (m == 0 || m > n) return ocurrencias;
17
18        vector<int> frecPatron(ALFABETO, 0), frecVentana(ALFABETO
19        , 0);
20        for (char c : patron) frecPatron[c - 'a']++;
21
22        int diferencias = 0; // cuantas posiciones del alfabeto
23        // difieren actualmente
24        for (int c = 0; c < ALFABETO; c++) {
25            if (frecVentana[c] != frecPatron[c]) diferencias++;
26        }
27
28        for (int i = 0; i < n; i++) {
29            int idxEntra = texto[i] - 'a';
30            if (frecVentana[idxEntra] == frecPatron[idxEntra])
31                diferencias++;
32            frecVentana[idxEntra]++;
33            if (frecVentana[idxEntra] == frecPatron[idxEntra])
34                diferencias--;
35
36            if (i >= m) {
37                int idxSale = texto[i - m] - 'a';
38                if (frecVentana[idxSale] == frecPatron[idxSale])
39                    diferencias++;
40                frecVentana[idxSale]--;
41                if (frecVentana[idxSale] == frecPatron[idxSale])
42                    diferencias--;
43            }
44
45            if (i >= m - 1 && diferencias == 0) {
46                ocurrencias.push_back(i - m + 1);
47            }
48        }
49        return ocurrencias;
50    } // O(n + sigma)
51
52    //
53    // -----
54    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
55    // -----
56
57    // 1. Verificar si dos cadenas son anagramas entre si (caso
58    // base, sin
59    // ventana deslizante): ver Verificar si Dos Cadenas son
60    // Anagramas,
61    // que resuelve directamente ese caso particular sin
62    // necesidad de
63    // montar una ventana deslizante.
64    // 2. Anagrama APROXIMADO (permitir hasta k caracteres
65    // distintos):
66    // cambiar la condicion final de diferencias == 0 a una
67    // medida de
68    // "cuantos caracteres hay que cambiar", que requiere
69    // sumar el
70    // exceso positivo de frecVentana sobre frecPatron (no
71    // solo contar
```

```
53 // posiciones distintas), dividido entre 2 para obtener
54 // sustituciones
55 // minimas.
56 // 3. Alfabeto mas grande (mayusculas, digitos, Unicode):
57 // usar un
58 // unordered_map<char,int> en vez de arreglos fijos de
59 // tamaño
60 // ALFABETO, ajustando el conteo de "diferencias" para que
61 // solo
62 // cuente claves presentes en al menos uno de los dos
63 // mapas.
64 };
```

Ejemplo de funcionamiento:

```
1 auto resultado = BusquedaAnagramas::buscar("cbaebabacd", "abc");
2 // resultado = {0, 6}
3 // (en la posicion 0, "cba" es un anagrama de "abc"; en la
4 // posicion 6,
5 // "bac" tambien lo es; ninguna otra ventana de tamaño 3 tiene
6 // la misma
7 // distribucion de caracteres que "abc")
```

Nota: el patrón de "incrementar/decrementar diferencias justo antes y después de modificar **frecVentana**" (en vez de recalcular diferencias desde cero en cada posición) es lo que mantiene la actualización en $O(1)$ por carácter; es el mismo principio general de mantener un estado incremental que aparece en *Line Sweep* y en *Suma de Distancias Manhattan*, aplicado aquí a conteo de frecuencias.

Regla práctica: usa esta técnica de ventana deslizante con contador de diferencias cuando el problema pregunte por **composición de caracteres** sin importar el orden (anagramas, substrings balanceados); si el orden de los caracteres sí importa, esto no aplica y corresponde usar *KMP*, *Algoritmo Z*, o *Hash Polinómico* en su lugar, que sí distinguen el orden exacto de los caracteres.

60.3 Agrupar Anagramas (Group Anagrams)

Idea: dado un conjunto de n cadenas, agruparlas de modo que dos cadenas queden en el mismo grupo si y solo si son anagramas entre sí. La técnica directa es asignar a cada cadena una **firma canónica** —una representación que es idéntica para todos los anagramas de una misma composición de caracteres, y distinta en cualquier otro caso— y usarla como clave de un `unordered_map` que acumula las cadenas originales en cada grupo. La cadena ordenada (ver *Verificar si Dos Cadenas son Anagramas*) es la firma canónica más directa: dos cadenas son anagramas exactamente cuando su versión ordenada es idéntica carácter por carácter.

¿Por qué usarlo? Comparar cada par de cadenas con *Verificar si Dos Cadenas son Anagramas* para decidir agrupamiento costaría $O(n^2)$ comparaciones. Usar una firma canónica como clave de hash reduce esto a $O(n)$ inserciones en el mapa (cada una $O(L \log L)$ si se ordena, donde L es la longitud de la cadena, o $O(L + \sigma)$ si se usa un vector de frecuencia codificado como clave en vez de ordenar) —el mismo principio de "normalizar a una representación comparable" que ya se usó para verificar dos cadenas, aplicado ahora a agrupar muchas a la vez.

Casos de uso:

- Agrupar una lista de palabras por equivalencia de anagrama (problema clásico "Group Anagrams").
- Deduplicar un conjunto de cadenas salvo por reordenamiento de caracteres.

- Como paso de preprocesamiento antes de contar cuántos grupos de tamaño $\geq k$ existen, o cuál es el grupo más grande.
- Indexación de un diccionario de palabras por su firma canónica, para búsquedas posteriores de "todas las palabras que son anagrama de X".

Complejidad: $O(n \cdot L \log L)$ con firma por ordenamiento, donde n es el número de cadenas y L la longitud máxima; $O(n \cdot (L + \sigma))$ con firma por conteo de frecuencia (codificada como cadena de conteos, por ejemplo), preferible cuando $L \log L > L + \sigma$, es decir, cadenas largas sobre alfabeto pequeño.

```

1 // CREATE/READ: Agrupa un conjunto de cadenas por equivalencia de
  anagrama,
2 // usando una firma canonica como clave de un mapa hash
3 struct AgruparAnagramas {
4     static const int ALFABETO = 26;
5
6     // READ: firma canonica por ordenamiento (simple,  $O(L \log L)$ 
      por cadena)
7     static string firmaOrdenada(const string& s) {
8         string ordenada = s;
9         sort(ordenada.begin(), ordenada.end());
10        return ordenada;
11    } //  $O(L \log L)$ 
12
13    // READ: firma canonica por conteo de frecuencia, codificada
      como
14    // cadena de numeros separados (evita el costo de ordenar)
15    static string firmaFrecuencia(const string& s) {
16        vector<int> frec(ALFABETO, 0);
17        for (char c : s) frec[c - 'a']++;
18        string firma;
19        for (int f : frec) { firma += to_string(f); firma += '#'; }
20        return firma;
21    } //  $O(L + \sigma)$ 
22
23    // READ: agrupa las cadenas dadas, retornando un vector de
      grupos
24    // (cada grupo es un vector de cadenas que son anagramas
      entre si)
25    static vector<vector<string>> agrupar(const vector<string>&
      palabras, bool usarFrecuencia = true) {
26        unordered_map<string, vector<string>> grupos;
27        for (const string& s : palabras) {
28            string clave = usarFrecuencia ? firmaFrecuencia(s) :
      firmaOrdenada(s);
29            grupos[clave].push_back(s);
30        }
31        vector<vector<string>> resultado;
32        for (auto& par : grupos) resultado.push_back(par.second);
33        return resultado;
34    } //  $O(n \cdot (L \log L))$  o  $O(n \cdot (L + \sigma))$  segun el modo
      elegido
35
36    //
37
38    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
39
40    // 1. Firma por frecuencia con separador '#': el separador
      evita
41    // ambigüedad entre conteos consecutivos (ej. frecuencias
      1,23 vs
42    // 12,3 sin separador se confundirian como el mismo string
      "123");
43    // 2. Firma via Hash Polinómico de la cadena ordenada: en vez
      de usar
44    // la cadena ordenada completa como clave (que puede ser
      larga),
45    // se puede hashear esa cadena ordenada a un unico entero
      (ver Hash
46    // Polinómico) para ahorrar memoria si hay muchisimas
      cadenas largas,
47    // a costa de la probabilidad minima de colision.
48    // 3. Tamaño del grupo mas grande / cuantos grupos hay:
      iterar sobre
49    // grupos directamente (antes de convertir a vector<vector
      <string>>)
50    // y aplicar max_element por tamaño, o simplemente grupos.
      size()
51    // para el numero total de grupos distintos.
52 };

```

Ejemplo de funcionamiento:

```

1 vector<string> palabras = {"eat", "tea", "tan", "ate", "nat", "bat"};
2 auto grupos = AgruparAnagramas::agrupar(palabras);
3 // grupos contiene 3 grupos: {"eat","tea","ate"}, {"tan","nat"}, {"bat"}
4 // (el orden de los grupos y el orden dentro de cada grupo
   depende del
5 // recorrido del unordered_map, no esta garantizado; hay que
   correrlo
6 // para confirmar el orden exacto de salida)

```

Regla práctica: usa firmaFrecuencia por defecto cuando el alfabeto sea pequeño y las cadenas sean largas —evita el costo de ordenar cada cadena. Usa firmaOrdenada cuando prefieras simplicidad de código sobre rendimiento máximo, o cuando el alfabeto sea grande (donde el conteo de frecuencia pierde su ventaja frente a ordenar).

60.4 Conteo de Substrings Permutables a Palíndromo

Idea: un substring se puede **reordenar** para formar un palíndromo si y solo si, en su distribución de frecuencia de caracteres, **a lo sumo un carácter** tiene frecuencia impar (ese carácter, si existe, va al centro; todos los demás se emparejan a ambos lados). En vez de recalculare la paridad de frecuencia de cada substring desde cero, se representa el estado de paridad de las 26 letras como una **máscara de bits** de 26 bits (bit k en 1 si la letra k tiene frecuencia impar en el prefijo actual), y se mantiene un **prefijoXOR**: al pasar de la posición $i - 1$ a i , la máscara simplemente hace XOR con el bit del carácter $s[i]$ —exactamente como *XOR: Propiedades y Aplicaciones* de la parte de BitWise. Dos prefijos con la **misma** máscara significan que el substring entre ellos tiene paridad par en todos los caracteres (0 caracteres de frecuencia impar); dos prefijos cuyas máscaras difieren en **un solo bit** significan que el substring entre ellos tiene exactamente un carácter de frecuencia impar. Ambos casos son válidos para permutarse a palíndromo.

¿Por qué usarlo? Verificar cada uno de los $O(n^2)$ substrings posibles recalculando su distribución de frecuencia sería $O(n^2\sigma)$ o peor. Reducir el estado de paridad a una máscara de 26 bits permite comparar dos prefijos en $O(1)$ (igualdad de máscara, o diferencia de un solo bit vía `__builtin_popcount`), y contar cuántos pares de prefijos son compatibles se resuelve con un `unordered_map<mascara, conteo>` recorrido una sola vez —el mismo patrón de "contar pares con XOR dado" de la parte de BitWise, aplicado aquí a paridad de frecuencia de caracteres en vez de valores numéricos directos.

Casos de uso:

- Contar cuántos substrings de una cadena se pueden reordenar (permutar) para formar un palíndromo.
- Verificar si una cadena específica se puede reordenar a palíndromo (caso particular: un solo prefijo, la cadena completa contra el prefijo vacío).
- Como aplicación de la técnica de "máscara de paridad + hashmap", reutilizable en cualquier problema donde la condición dependa solo de paridad de frecuencia, no de la frecuencia exacta.

- Distinguible de *Conteo Total de Substrings Palindrómicos* (Manacher): aquí se permite **reordenar** los caracteres, allí el substring debe ser palíndromo **tal cual está**, sin reordenar.

Complejidad: $O(n\sigma)$ o $O(n \cdot 26)$ en la práctica con alfabeto minúsculas —para cada uno de los n prefijos, se revisan los 26 posibles bits a voltear (para el caso de un carácter impar permitido), cada revisión $O(1)$ contra el hashmap.

```

1 // CREATE/READ: Conteo de substrings que se pueden reordenar para
  formar
2 // un palíndromo, mediante mascara de paridad de frecuencia (XOR)
  + hashmap
3 struct SubstringsPermutablesPalindromo {
4     static const int ALFABETO = 26;
5
6     // READ: cuenta cuantos substrings de s se pueden reordenar a
7     // palíndromo (0 o 1 caracteres de frecuencia impar en el
8     // substring)
9     static long long contar(const string& s) {
10         int n = (int)s.size();
11         unordered_map<int, long long> conteoMascara;
12         conteoMascara[0] = 1; // prefijo vacío, mascara 0
13
14         long long total = 0;
15         int mascara = 0;
16         for (int i = 0; i < n; i++) {
17             mascara ^= (1 << (s[i] - 'a'));
18
19             // caso 1: existe otro prefijo con la MISMA mascara
20             // caracteres impares en el substring entre ambos)
21             if (conteoMascara.count(mascara)) {
22                 total += conteoMascara[mascara];
23             }
24
25             // caso 2: existe otro prefijo cuya mascara difiere
26             // EXACTAMENTE un bit (1 caracter impar en el
27             // substring, el
28             // que va al centro del palíndromo)
29             for (int bit = 0; bit < ALFABETO; bit++) {
30                 int mascaraVecina = mascara ^ (1 << bit);
31                 if (conteoMascara.count(mascaraVecina)) {
32                     total += conteoMascara[mascaraVecina];
33                 }
34             }
35             conteoMascara[mascara]++;
36         }
37         return total;
38     } // O(n * sigma)
39
40 // -----
41 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
42 // -----
43 // 1. Verificar UNA sola cadena completa (no contar
44 //     substrings): basta
45 //     con __builtin_popcount(mascara_final) <= 1, donde
46 //     mascara_final
47 //     es el XOR acumulado de todos los caracteres de la
48 //     cadena.
49 // 2. Longitud FIJA k (no todos los substrings, solo los de
50 //     tamaño k):
51 //     usar ventana deslizante de tamaño k manteniendo la
52 //     XOR de entrada y salida (XOR es su propia inversa, así
53 //     que
54 //     "remove" un caracter de la ventana es re-aplicar XOR
55 //     con su bit).
56 // 3. Alfabeto mas grande que 26 (mayúsculas+minúsculas,
57 //     Unicode): la
58 //     mascara de bits sigue funcionando mientras el alfabeto
59 //     quepa en
60 //     los bits de un entero (hasta 63 con long long); mas
61 //     allá de eso,
62 //     se necesitaría un bitset o una estructura de paridad
63 //     distinta.
64 };

```

Ejemplo de funcionamiento:

```

1 long long resultado = SubstringsPermutablesPalindromo::contar("
2 aab");
3 // resultado = 4

```

```

3 // (substrings permutables a palíndromo: "a"(pos 0), "a"(pos 1),
4 // "aa"
5 // (frecuencias pares, se reordena a "aa"), "b"(pos 2); "aab"
6 // completa
7 // tiene 'a' con frecuencia 2 (par) y 'b' con frecuencia 1 (
8 // impar), un
9 // solo caracter impar, tambien cuenta -- hay que correrlo para
10 // confirmar el conteo exacto de 4 vs otro valor segun se
11 // interprete el
12 // problema)

```

Nota: este conteo incluye **todos** los substrings donde la permutación a palíndromo es posible, sin importar si el substring original ya es palíndromo o no —es un conjunto distinto (generalmente mayor) que el de *Conteo Total de Substrings Palindrómicos*, que exige que el substring sea palíndromo **tal cual aparece**, sin reordenar.

Regla práctica: usa la técnica de máscara de paridad + hashmap cuando la condición del problema dependa de **paridad de frecuencia** de caracteres (0 o 1 impares permite palíndromo por reordenamiento); si la condición exige que el substring sea palíndromo **en su orden original**, esta técnica no aplica y corresponde usar *Algoritmo de Manacher* o la tabla **esPal** de *Partición Mínima en Substrings Palindrómicos* en su lugar.

60.5 Anagrama Lexicográficamente Mínimo/Máximo (con Restricciones)

Idea: dado un multiset de caracteres (por ejemplo, la distribución de frecuencia de una cadena), construir el anagrama lexicográficamente **mínimo** sin restricciones es trivial: colocar los caracteres en orden alfabético ascendente (los máximos, en orden descendente). El problema se vuelve interesante bajo la restricción adicional de que **no puede haber dos caracteres iguales adyacentes**: ahí la estrategia greedy correcta es, en cada posición, elegir el carácter disponible más pequeño (o más grande, según el objetivo) que **no sea igual** al último colocado, dando prioridad a colocar primero los caracteres de mayor frecuencia restante para evitar quedarse sin opciones válidas al final. Esto se implementa con un *max-heap* (o *priority_queue*) ordenado por frecuencia restante, extrayendo el carácter más frecuente disponible que no repita al anterior.

¿Por qué usarlo? Sin la restricción de no-adyacencia, el problema es $O(\sigma \log \sigma)$ trivial (ordenar los caracteres del alfabeto por frecuencia). Con la restricción, un enfoque greedy ingenuo (simplemente tomar el más pequeño disponible en cada paso) puede fallar: si un carácter tiene una frecuencia muy alta comparada con el resto, colocarlo greedily por orden alfabético puede agotar las demás opciones antes de que ese carácter frecuente pueda distribuirse sin quedar adyacente a sí mismo. Por eso la estrategia correcta prioriza **frecuencia restante** sobre orden alfabético puro en cada paso —solo así se garantiza que el carácter más problemático (el de mayor frecuencia) se vaya colocando lo antes posible, dejando espacio para los demás.

Casos de uso:

- Reorganizar una cadena para que no haya caracteres iguales adyacentes, priorizando el resultado lexicográficamente menor entre todas las reorganizaciones válidas.
- Verificar si es **posible** reorganizar sin adyacencias

repetidas (caso base: ningún carácter puede tener frecuencia mayor a $\lceil n/2 \rceil$).

- Problemas de programación de tareas donde tareas iguales no pueden ejecutarse en posiciones consecutivas (isomorfo al mismo problema con "tarea" en vez de "carácter").
- Como aplicación concreta de estrategia greedy con heap, distinta de un simple ordenamiento, cuando una restricción de adyacencia rompe la solución trivial.

Complejidad: $O(n \log \sigma)$, donde n es la longitud total de la cadena a construir y σ es el tamaño del alfabeto —cada una de las n posiciones extrae y opcionalmente reinserta del heap, cada operación $O(\log \sigma)$ ya que el heap contiene a lo sumo σ elementos distintos.

```
1 // CREATE/READ: Anagrama lexicograficamente minimo sin caracteres
  iguales
2 // adyacentes, dado un multiset de caracteres (via distribucion
  de
3 // frecuencia), mediante estrategia greedy con max-heap por
  frecuencia
4 struct AnagramaConRestricciones {
5     static const int ALFABETO = 26;
6
7     // READ: reorganiza los caracteres de s de modo que no haya
      dos
8     // iguales adyacentes, priorizando el mayor frecuencia
      restante en
9     // cada paso (y desempata por orden alfabetico para
      minimizar
10    // lexicograficamente); retorna "" si no es posible
11    static string reorganizar(const string& s) {
12        int n = (int)s.size();
13        vector<int> frec(ALFABETO, 0);
14        for (char c : s) frec[c - 'a']++;
15
16        // caso base: si algun caracter excede ceil(n/2), es
      imposible
17        for (int f : frec) {
18            if (f > (n + 1) / 2) return "";
19        }
20
21        // max-heap por (frecuencia, -letra) para que, en empate
      de
22        // frecuencia, se prefiera la letra menor (lexicografico
      minimo)
23        priority_queue<pair<int,int>> heap;
24        for (int c = 0; c < ALFABETO; c++) {
25            if (frec[c] > 0) heap.push({frec[c], -c});
26        }
27
28        string resultado;
29        while (!heap.empty()) {
30            auto [frecActual, letraNeg] = heap.top(); heap.pop();
31            int letra = -letraNeg;
32
33            if (!resultado.empty() && resultado.back() == 'a' +
      letra) {
34                // el mas frecuente disponible repite al ultimo
      colocado:
35                // usar el SEGUNDO mas frecuente disponible en su
      lugar
36                if (heap.empty()) return ""; // no hay
      alternativa valida
37                auto [frecAlt, letraAltNeg] = heap.top(); heap.
      pop();
38                int letraAlt = -letraAltNeg;
39                resultado += ('a' + letraAlt);
40                if (frecAlt - 1 > 0) heap.push({frecAlt - 1,
      letraAltNeg});
41                heap.push({frecActual, letraNeg}); // reinsertar
      el original
42            } else {
43                resultado += ('a' + letra);
44                if (frecActual - 1 > 0) heap.push({frecActual -
      1, letraNeg});
45            }
46        }
47        return resultado;
48    } // O(n log sigma)
49
50    //
```

```
51 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
52 //
53 // -----
54 // 1. Anagrama lexicograficamente MAXIMO (en vez de minimo)
      con
55 //     restricción: cambiar el desempate del heap a (
      frecuencia, letra)
56 //     en vez de (frecuencia, -letra), para que en empate se
      prefiera
57 //     la letra mayor.
58 // 2. Sin restricción de adyacencia (caso simple): basta con
      recorrer
59 //     el alfabeto en orden y repetir cada caracter segun su
      frecuencia,
60 //     sin necesidad de heap ni logica de "reinsertar el
      original".
61 // 3. Verificar SOLO si es posible (sin construir el
      resultado): basta
62 //     con el chequeo de frec[c] > (n+1)/2 para algun caracter;
      si
63 //     ninguno excede ese limite, siempre existe alguna
      reorganizacion
64 //     valida (aunque no necesariamente la lexicografica
      minima sin
65 //     construirla explicitamente).
66 };
```

Ejemplo de funcionamiento:

```
1 string resultado = AnagramaConRestricciones::reorganizar("aab");
2 // resultado = "aba"
3 // (a tiene frecuencia 2, b tiene frecuencia 1; la unica forma de
  evitar
4 // adyacencia de 'a' es intercalar con 'b': "aba"; es ademas la
5 // lexicograficamente minima entre las reorganizaciones validas)
6
7 string imposible = AnagramaConRestricciones::reorganizar("aaab");
8 // imposible = ""
9 // (a tiene frecuencia 3, n=4, ceil(4/2)=2; 3 > 2, asi que
      ninguna
10 // reorganizacion puede evitar que dos 'a' queden adyacentes)
```

Nota: el paso de "si el más frecuente disponible repite al último colocado, usar el segundo más frecuente en su lugar" (en vez de simplemente fallar o saltarse ese carácter) es la parte más propensa a errores de esta técnica —omitirlo produce resultados incorrectos incluso en casos donde sí existe una reorganización válida, porque un greedy que solo mira "el más frecuente" sin ese respaldo puede bloquearse innecesariamente.

Regla práctica: usa esta estrategia de heap por frecuencia (con el respaldo de "segundo más frecuente" cuando el primero repite) siempre que la restricción sea de no-adyacencia entre caracteres iguales; verifica primero el caso de imposibilidad ($\max(\text{frec}) > \lceil n/2 \rceil$) antes de construir el resultado, para evitar entrar en un bucle que de todas formas terminará fallando.

Parte X BitWise

60.6 Casos de Uso

Antes de entrar en el detalle de cada técnica, esta sección da un mapa rápido de *cuándo* recurrir a bit-wise/bitmasking en competencia:

- $n \leq 20$ -24 y necesitas enumerar subconjuntos: Bitmask DP o Enumeración de Submáscaras — cada subconjunto de n elementos cabe en un entero de 32/64 bits.
- Necesitas maximizar/minimizar XOR de un par o subconjunto: Trie de Bits (pares) o Base Lineal de XOR (subconjuntos arbitrarios).

- **Un elemento aparece un número impar de veces y el resto par (o viceversa):** propiedades de XOR ($a \oplus a = 0$).
- **Necesitas acelerar un DP o conteo por un factor de 64:** `std::bitset` para representar conjuntos y operar sobre ellos en bloques de palabra.
- **El estado de tu DP es "qué celdas de esta fila ya están cubiertas":** DP con Perfil de Bits (*broken profile*), típico en problemas de *tiling*.
- **Necesitas agregar (sumar, contar) sobre todas las submáscaras de cada máscara, para todas las máscaras:** SOS DP, en vez de una enumeración ingenua $O(3^n)$.
- **n es demasiado grande para 2^n pero se puede partir en dos mitades manejables (hasta 2^{20} cada una):** Meet in the Middle con máscaras de bits.
- **Necesitas verificar rápidamente si un número es potencia de dos, extraer el bit más bajo, o contar bits encendidos:** Operaciones y Trucos Básicos, o las funciones *built-in* de GCC si el costo de una llamada de función no es crítico.

Regla práctica: Si el problema menciona explícitamente "subconjuntos", "XOR", "máscara de bits", o si n es sospechosamente pequeño (usualmente ≤ 20) mientras el resto de las restricciones sugieren un algoritmo exponencial, es una señal fuerte de que la solución pasa por alguna técnica de esta parte.

60.7 Representación Binaria de Enteros (con y sin signo)

Idea: Todo entero en una computadora se almacena como una secuencia fija de bits (32 para `int`, 64 para `long long`). Para enteros *sin signo* (`unsigned`), esos bits representan directamente el valor en base 2. Para enteros *con signo*, C++ usa la representación en **complemento a 2** (*two's complement*): el bit más significativo actúa como bit de signo, y un número negativo $-x$ se representa como el complemento a 1 de x (invertir todos los bits) más 1. Esta elección no es arbitraria: permite que la suma y la resta de enteros con signo usen exactamente el mismo circuito de hardware que para enteros sin signo, sin necesidad de lógica especial para manejar signos.

¿Por qué usarlo? Entender esta representación es la base para todo lo demás en esta parte: sin saber cómo se codifica un número negativo en binario, operaciones como $\sim x$ (complemento a 1), desplazamientos aritméticos, o el comportamiento de *overflow* en `int` parecen mágicas o impredecibles. Se distingue de una simple "conversión a binario" (ver más adelante) en que aquí el foco es la codificación del *signo*, no solo la representación de la magnitud.

Casos de uso:

- Predecir el resultado exacto de operaciones bitwise ($\sim$, $\&$, $|$, $\wedge$) sobre enteros negativos.

- Entender por qué -1 en complemento a 2 tiene *todos* sus bits en 1 (útil como máscara "todo encendido").
- Depurar comportamiento inesperado al mezclar `int` y `unsigned int` en la misma expresión.
- Implementar operaciones de bajo nivel (compresión, hashing, checksums) que dependen del patrón de bits exacto.

Complejidad: $O(1)$ — la representación es una propiedad fija del tipo de dato, no un algoritmo; todas las conversiones mentales entre valor y bits se hacen en tiempo constante sobre un número fijo de bits (32 o 64).

```

1 struct RepresentacionBinaria {
2
3     // READ: retorna el complemento a 1 de x (invierte todos los
4     // bits); equivalente al operador ~ de C++
5     static unsigned int complementoA1(unsigned int x) {
6         return ~x;
7     } // O(1)
8
9     // READ: retorna el complemento a 2 de x, que es la
10    // representación de -x en enteros con signo (complemento a 1
11    // mas 1)
12    static unsigned int complementoA2(unsigned int x) {
13        return ~x + 1u;
14    } // O(1)
15
16    // READ: retorna la representación en string de los bits de x,
17    // desde el bit mas significativo hasta el menos
18    // significativo
19    static string aBinario(unsigned int x, int bits = 32) {
20        string resultado;
21        for (int i = bits - 1; i >= 0; i--) {
22            resultado += ((x >> i) & 1) ? '1' : '0';
23        }
24        return resultado;
25    } // O(bits)
26
27    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
28    //
29    // 1. Por que -1 es "todo unos": aplicando complementoA2(1),
30    // -1 + 1 = ...11111110 + 1 = ...11111111, es decir, todos
31    // los bits en 1. Por eso "-1" es una mascara util para
32    // "todos los bits encendidos" (equivalente a
33    // numeric_limits<unsigned int>::max()).
34    //
35    // 2. Reinterpretar bits sin cambiar el patron: para ver el
36    // patron de bits de un int negativo como si fuera
37    // unsigned
38    // (sin cambiar ningun bit), se puede castear:
39    // unsigned int u = (unsigned int)x;
40    // Esto NO cambia el patron de bits, solo la
41    // interpretacion.
42    //
43    // 3. Rango representable: con complemento a 2 de b bits, el
44    // rango de un tipo con signo es [-2^(b-1), 2^(b-1) - 1]
45    // (asimetrico: un negativo mas que positivos), mientras
46    // que
47    // sin signo es [0, 2^b - 1]. Este desbalance es la razon
48    // por
49    // la que, por ejemplo, abs(INT_MIN) es comportamiento
50    // indefinido en int de 32 bits.
51 };

```

Ejemplo de funcionamiento:

```

1 unsigned int x = 5; // 00000000 00000000 00000000 00000101
2
3 string bits = RepresentacionBinaria::aBinario(x, 8);
4 // bits = "00000101"
5
6 unsigned int comp1 = RepresentacionBinaria::complementoA1(x);
7 // comp1 en los ultimos 8 bits: 11111010 (complemento a 1 de 5)
8
9 unsigned int comp2 = RepresentacionBinaria::complementoA2(x);
10 // comp2 en los ultimos 8 bits: 11111011 (representa -5 en
11 // signed)
12
13 int menosCinco = -5;
14 unsigned int patronDeMenosCinco = (unsigned int)menosCinco;
15 // patronDeMenosCinco == comp2 (mismo patron de bits que -5)

```

Nota: El comportamiento de desplazamientos y overflow sobre enteros *con signo* negativos involucra casos de comportamiento indefinido en C++ (por ejemplo, $x \ll n$ con x negativo es UB antes de C++20). Para operaciones bitwise predecibles sobre bits individuales, es más seguro trabajar con `unsigned int` o `unsigned long long` explícitamente.

Regla práctica: Si vas a manipular bits directamente (máscaras, complementos, desplazamientos), preferir tipos `unsigned` para evitar comportamiento indefinido; usa `int/long long` con signo solo cuando el valor numérico (no el patrón de bits) es lo que importa.

60.8 Operaciones y Trucos Básicos

Idea: C++ ofrece un conjunto pequeño de operadores bitwise (`&`, `|`, `^`, `~`, `<<`, `>>`) que operan sobre la representación binaria de un entero bit a bit, en paralelo, en una sola instrucción de hardware. A partir de esos operadores primitivos se derivan "trucos" (*bit tricks*) que resuelven en $O(1)$ operaciones que de otra forma requerirían un bucle: extraer un bit específico, prender/apagar un bit, verificar si un número es potencia de dos, o aislar el bit más bajo encendido. La idea central detrás de todos estos trucos es aprovechar que las operaciones bitwise procesan los 32 o 64 bits de un entero simultáneamente, en vez de iterar posición por posición.

¿Por qué usarlo? Estos trucos son la base sobre la que se construyen todas las técnicas más avanzadas de esta parte (Submask Enumeration, SOS DP, Bitmask DP, XOR Basis, etc.) — sin dominarlos, el código de las secciones siguientes es difícil de leer o depurar. Frente a una solución con bucles explícitos (por ejemplo, contar bits encendidos dividiendo repetidamente entre 2), los trucos bitwise suelen ser más rápidos en la práctica y, sobre todo, más cortos de escribir bajo presión de tiempo en competencia.

Casos de uso:

- Representar un subconjunto de hasta 64 elementos como un único entero, para usar como estado de un Bitmask DP.
- Verificar rápidamente si un número es potencia de dos (útil en problemas de segment trees, sparse tables, FFT).
- Aislar o eliminar el bit menos significativo encendido, base de estructuras como el Árbol de Fenwick (BIT).
- Alternar el estado de un bit (toggle), común en problemas de simulación con interruptores o estados binarios.
- Verificar paridad de un número (par/impar) sin usar el operador módulo.

Complejidad: $O(1)$ por operación — cada operador bitwise se traduce en una sola instrucción de máquina que actúa sobre la palabra completa (32 o 64 bits) en un ciclo de reloj, independientemente del valor del número.

```
1 struct TrucosBitwise {
2
3     // READ: retorna 1 si el bit en la posición i (0-indexado
4     // desde
5     // el bit menos significativo) está encendido, 0 si está
6     // apagado
```

```
5 static int obtenerBit(long long x, int i) {
6     return (x >> i) & 1LL;
7 } // O(1)
8
9 // WRITE: retorna x con el bit en la posición i encendido
10 // (puesto en 1), sin modificar los demás bits
11 static long long prenderBit(long long x, int i) {
12     return x | (1LL << i);
13 } // O(1)
14
15 // WRITE: retorna x con el bit en la posición i apagado
16 // (puesto en 0), sin modificar los demás bits
17 static long long apagarBit(long long x, int i) {
18     return x & ~(1LL << i);
19 } // O(1)
20
21 // WRITE: retorna x con el bit en la posición i invertido
22 // (0 pasa a 1, 1 pasa a 0)
23 static long long alternarBit(long long x, int i) {
24     return x ^ (1LL << i);
25 } // O(1)
26
27 // READ: retorna true si x es una potencia de 2 (incluyendo
28 // x = 1 = 2^0); false para x <= 0
29 static bool esPotenciaDeDos(long long x) {
30     return x > 0 && (x & (x - 1)) == 0;
31 } // O(1)
32
33 // READ: retorna un entero con solo el bit menos
34 // significativo
35 // encendido de x prendido, y todos los demás apagados
36 // (aislar el "lowest set bit")
37 static long long aislarBitMasBajo(long long x) {
38     return x & (-x);
39 } // O(1)
40
41 // WRITE: retorna x con su bit menos significativo encendido
42 // apagado (elimina el "lowest set bit")
43 static long long apagarBitMasBajo(long long x) {
44     return x & (x - 1);
45 } // O(1)
46
47 //
48 // -----
49 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
50 // -----
51
52 // 1. Por que x & (-x) aísla el bit mas bajo: en complemento
53 // a 2,
54 // -x = ~x + 1. Todos los bits antes del bit mas bajo
55 // encendido de x son 0 tanto en x como en -x (se
56 // preservan
57 // al invertir y sumar 1), y a partir de ese bit los
58 // patrones
59 // son complementarios; el AND deja solo ese bit.
60 //
61 // 2. Por que x & (x - 1) apaga el bit mas bajo: restar 1 a x
62 // invierte todos los bits desde el bit mas bajo encendido
63 // hacia abajo (los ceros se vuelven unos, y ese bit se
64 // apaga); el AND con x conserva todo excepto ese bit.
65 //
66 // 3. Verificar paridad sin modulo: (x & 1) == 0 equivale a
67 // x % 2 == 0, pero evita la instruccion de division/
68 // modulo,
69 // que es mas lenta que un AND en la mayoria de
70 // arquitecturas.
71 //
72 // 4. Contar bits encendidos con apagarBitMasBajo: un bucle
73 // "while (x) { x = apagarBitMasBajo(x); cont++; }" cuenta
74 // los
75 // bits en O(numero de bits encendidos) en vez de O(32/64);
76 //
77 // en la practica, __builtin_popcount es preferible por
78 // ser
79 // una sola instruccion de hardware (ver la subseccion de
80 // funciones built-in de GCC).
81 };
```

Ejemplo de funcionamiento:

```
1 long long x = 0b1011010; // 90 en decimal
2
3 int bit3 = TrucosBitwise::obtenerBit(x, 3); // bit3 = 1 (cuarto
4 // bit desde la derecha)
5
6 long long conBit0 = TrucosBitwise::prenderBit(x, 0);
7 // conBit0 = 0b1011011 = 91
8
9 long long sinBit4 = TrucosBitwise::apagarBit(x, 4);
10 // sinBit4 = 0b1001010 = 74
11
12 bool esPot = TrucosBitwise::esPotenciaDeDos(64); // esPot =
13 // true
```

```

12 bool esPot2 = TrucosBitwise::esPotenciaDeDos(90); // esPot2 =
    false
13
14 long long masBajo = TrucosBitwise::aislarBitMasBajo(x);
15 // masBajo = 0b00000010 = 2 (bit mas bajo encendido de 90)

```

Regla práctica: Ante cualquier problema que hable de "conjunto de hasta 20-24 elementos", "estado binario" o "prender/ apagar/alternar", piensa primero en representar el estado como un entero y usar estos trucos antes de recurrir a un `vector<bool>` o arreglo explícito — es más compacto, más rápido, y se integra directamente con Bitmask DP y Submask Enumeration.

60.9 Desplazamientos Aritméticos vs. Lógicos (Arithmetic vs. Logical Shift)

Idea: El operador `»` en C++ se comporta de forma distinta según el tipo del operando: sobre un entero `unsigned`, es un desplazamiento *lógico*, que rellena con ceros por la izquierda sin importar el valor. Sobre un entero `signed`, es un desplazamiento *aritmético*, que replica el bit de signo (el bit más significativo) al desplazar, preservando así el signo del número: desplazar un negativo a la derecha sigue dando un negativo. El operador `«` no tiene esta ambigüedad al desplazar (siempre rellena con ceros por la derecha), pero desplazar un `signed` negativo con `«` es comportamiento indefinido antes de C++20.

¿Por qué usarlo? Confundir estos dos comportamientos es una fuente común de bugs sutiles en competencia: dividir por potencias de 2 usando `»` sobre un `int` negativo (por ejemplo, en una búsqueda binaria con `(izq + der) » 1` cuando alguno de los límites puede ser negativo) da un resultado distinto —y matemáticamente distinto de la división entera real hacia cero— al que se obtendría sobre un `unsigned`. Entender la diferencia evita depurar durante horas un desplazamiento que "debería" funcionar igual que una división.

Casos de uso:

- Calcular el punto medio en búsqueda binaria sin overflow, verificando qué tipo de shift aplica si algún límite es negativo.
- Dividir por potencias de 2 rápidamente en tipos `unsigned` (equivalente exacto a la división entera).
- Implementar rotaciones de bits (*bit rotation*), que requieren combinar shifts lógicos en ambas direcciones.
- Evitar bugs al portar código que asume `int` sin signo entre plataformas o al mezclar `int/unsigned`.

Complejidad: $O(1)$ — al igual que las demás operaciones bitwise, un desplazamiento (lógico o aritmético) se ejecuta en una sola instrucción de hardware sin importar la magnitud del desplazamiento (siempre que sea menor al número de bits del tipo).

```

1 struct Desplazamientos {
2
3     // READ: aplica un shift logico a la derecha (rellena con
    // sin importar el signo), forzando la interpretacion
    unsigned
4     static long long shiftLogico(long long x, int n) {
5         return (long long)((unsigned long long)x >> n);
6     } // O(1)

```

```

8
9 // READ: aplica un shift aritmetico a la derecha (replica el
    bit
10 // de signo), usando directamente el operador sobre signed
    static long long shiftAritmetico(long long x, int n) {
11     return x >> n; // sobre long long (signed), >> ya es
        aritmetico
12 } // O(1)
13
14
15 // READ: rota los bits de un unsigned de 32 bits hacia la
    // izquierda por n posiciones (los bits que "salen" por la
16 // izquierda reaparecen por la derecha)
    static unsigned int rotarIzquierda(unsigned int x, int n) {
17     n %= 32;
18     return (x << n) | (x >> (32 - n));
19 } // O(1)
20
21
22
23
24 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
25 //
26
27 // 1. Division vs. shift aritmetico en negativos: para x
    // negativo, x >> 1 (aritmetico) NO es lo mismo que x / 2
    // en
28 // C++: la division trunca hacia cero, pero el shift
29 // aritmetico redondea hacia -infinito (floor). Ejemplo:
30 // -7 / 2 == -3, pero -7 >> 1 == -4.
31 //
32 // 2. Punto medio seguro en busqueda binaria: usar
33 // izq + (der - izq) / 2 en vez de (izq + der) >> 1 evita
34 // tanto el overflow de (izq + der) como la ambigüedad de
35 // shift si algun limite fuera negativo (aunque en indices
    // de
36 // arreglo normalmente ambos son no negativos).
37 //
38 // 3. Rotacion a la derecha: analogo a rotarIzquierda pero
    // intercambiando los shifts:
39 // (x >> n) | (x << (32 - n)); // sobre unsigned int
40 //
41 //
42 // 4. C++20 y shifts definidos: desde C++20, x << n con x
    // negativo dejo de ser UB y se define como shift sobre el
43 // patron de bits (equivalente a operar sobre unsigned);
44 // en estandares anteriores, evitar shifts sobre signed
45 // negativos por completo.
46
47 };

```

Ejemplo de funcionamiento:

```

1 long long negativo = -8; // ...11111000 en complemento a 2
2
3 long long resAritmetico = Desplazamientos::shiftAritmetico(
    negativo, 1);
4 // resAritmetico = -4 (replica el bit de signo, sigue negativo)
5
6 long long resLogico = Desplazamientos::shiftLogico(negativo, 1)
7 ;
8 // resLogico = un numero positivo enorme (se interpreto como
    // unsigned de 64 bits y se relleno con ceros)
9
10 unsigned int val = 0b00000000100000000000000000000000; // bit
    // 24 encendido
11 unsigned int rotado = Desplazamientos::rotarIzquierda(val, 4);
12 // rotado tiene el bit 28 encendido (el bit 24 se desplazo 4
    // posiciones)

```

Nota: `-7 » 1` en un `int` con signo da `-4` (shift aritmético, redondea hacia $-\infty$), mientras que `-7 / 2` da `-3` (división entera, trunca hacia 0). Si necesitas división exacta por potencias de 2 con posibles negativos, usa el operador `/` directamente y no un shift.

Regla práctica: Usa `»` como atajo de división por potencias de 2 *solo* sobre tipos `unsigned` o cuando tengas certeza de que el operando es no negativo; si el operando puede ser negativo y necesitas semántica de división real (truncar hacia cero), usa el operador `/` explícitamente en vez de `»`.

60.10 Funciones Built-in de GCC

Idea: GCC (y por extensión Clang, en su mayoría) expone un conjunto de funciones `__builtin_*` que se traducen directamente a instrucciones especializadas del procesador (cuando existen) para operaciones bit

a bit comunes: contar bits encendidos, contar ceros iniciales/finales, verificar paridad. A diferencia de implementar estas operaciones manualmente con bucles o con los trucos de la subsección anterior, estas funciones aprovechan instrucciones de hardware dedicadas (como POPCNT o BSF/BSR en x86), lo que las hace, en la práctica, más rápidas y sin necesidad de escribir el algoritmo a mano.

¿Por qué usarlo? Aunque los trucos manuales de "Operaciones y Trucos Básicos" (como $x \& (x-1)$ en un bucle para contar bits) funcionan correctamente, estas funciones built-in son la opción preferida en competencia por dos razones: son más cortas de escribir bajo presión de tiempo, y su implementación en hardware es frecuentemente $O(1)$ real (una instrucción), en vez de $O(\text{bits encendidos})$ como el bucle manual. La contrapartida es que dependen del compilador (GCC/Clang; no existen en MSVC con el mismo nombre) y tienen casos borde con el valor 0 que hay que conocer.

Casos de uso:

- Contar el número de elementos en un subconjunto representado como máscara de bits (`popcount`), central en Bitmask DP.
- Encontrar la posición del bit más significativo encendido (`clz`), útil para calcular $\lfloor \log_2 x \rfloor$ o la siguiente potencia de 2.
- Encontrar la posición del bit menos significativo encendido (`ctz`), alternativa a $x \& (-x)$ para obtener el índice en vez del valor del bit.
- Verificar paridad del número de bits encendidos (`parity`), usado en checksums simples o problemas de conteo de inversiones con propiedades de paridad.

Complejidad: $O(1)$ en la gran mayoría de arquitecturas modernas (x86-64 con soporte para POPCNT/LZCNT/TZCNT), ya que se traducen a una sola instrucción de hardware; en arquitecturas sin esas instrucciones, GCC genera una secuencia de instrucciones equivalente que sigue siendo significativamente más rápida que un bucle interpretado a mano.

```

1 struct FuncionesBuiltinGCC {
2
3     // READ: retorna la cantidad de bits encendidos (1s) en x,
4     // usando la version de 32 bits
5     static int contarBitsEncendidos(unsigned int x) {
6         return __builtin_popcount(x);
7     } // O(1)
8
9     // READ: version de 64 bits de popcount, para long long
10    static int contarBitsEncendidos64(unsigned long long x) {
11        return __builtin_popcountll(x);
12    } // O(1)
13
14    // READ: retorna la cantidad de ceros consecutivos desde el
15    // bit
16    // MAS significativo hasta el primer 1 (count leading zeros);
17    // comportamiento indefinido si x == 0
18    static int cerosIniciales(unsigned int x) {
19        return __builtin_clz(x);
20    } // O(1)
21
22    // READ: retorna la cantidad de ceros consecutivos desde el
23    // bit
24    // MENOS significativo hasta el primer 1 (count trailing
25    // zeros),
26    // es decir, el indice del bit mas bajo encendido; UB si x ==
27    // 0
28    static int cerosFinales(unsigned int x) {
29        return __builtin_ctz(x);
30    } // O(1)
31

```

```

27
28 // READ: retorna la posicion (0-indexada) del bit mas
29 // significativo encendido de x, equivalente a floor(log2(x));
30
31 // asume x > 0
32 static int posicionBitMasAlto(unsigned int x) {
33     return 31 - __builtin_clz(x);
34 } // O(1)
35
36 // -----
37 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
38 // -----
39 // 1. Caso borde x == 0: __builtin_clz(0) y __builtin_ctz(0)
40 // son
41 // comportamiento indefinido (usualmente retornan 32 o un
42 // valor basura, dependiendo de la arquitectura). SIEMPRE
43 // verificar x != 0 antes de llamarlas.
44 //
45 // 2. Sufijos por tipo: cada funcion tiene variante segun el
46 // tamano del argumento:
47 // __builtin_popcount / __builtin_clz / __builtin_ctz ->
48 // unsigned int (32 bits)
49 // __builtin_popcountll / __builtin_clzll /
50 // __builtin_ctzll -> unsigned long long (64 bits)
51 // Pasar un long long a la version sin "ll" trunca a 32
52 // bits
53 // silenciosamente y da resultados incorrectos.
54 //
55 // 3. __builtin_parity: retorna 1 si la cantidad de bits
56 // encendidos es impar, 0 si es par. Equivalente a
57 // __builtin_popcount(x) & 1, pero potencialmente mas
58 // rapido
59 // en algunas arquitecturas.
60 //
61 // 4. Siguiete potencia de 2 mayor o igual a x (x > 0):
62 // x == 1 ? 1 : 1 << (32 - __builtin_clz(x - 1));
63 // se apoya en clz para encontrar el bit mas alto
64 // necesario.
65 //
66 // 5. No disponibles en MSVC: estas funciones son especificas
67 // de
68 // GCC/Clang. En MSVC los equivalentes son __popcnt,
69 // _BitScanForward, _BitScanReverse. En Codeforces/la
70 // mayoria
71 // de jueces (que usan GCC), esto no es un problema.
72
73 };

```

Ejemplo de funcionamiento:

```

1 unsigned int x = 0b1011010; // 90 en decimal
2
3 int bits = FuncionesBuiltinGCC::contarBitsEncendidos(x);
4 // bits = 4 (hay cuatro 1s en 1011010)
5
6 int cerosIni = FuncionesBuiltinGCC::cerosIniciales(x);
7 // cerosIni = 25 (32 bits totales - 7 bits significativos = 25
8 // ceros
9 // antes del primer 1)
10
11 int cerosFin = FuncionesBuiltinGCC::cerosFinales(x);
12 // cerosFin = 1 (el bit menos significativo encendido esta en
13 // la
14 // posicion 1: ...1011010, el ultimo bit es 0)
15
16 int posAlto = FuncionesBuiltinGCC::posicionBitMasAlto(x);
17 // posAlto = 6 (el bit mas significativo encendido esta en la
18 // posicion 6, ya que 90 = 1011010 en binario, 7 bits, indice
19 // 0-6)

```

Nota: Revisa cuidadosamente el nombre de la función usada (`cerosIniciales` y `ceros_iniciales` arriba deben escribirse sin espacios como identificador válido de C++ — `cerosIniciales`); este es un error tipográfico común al transcribir a mano.

Regla práctica: Prefiere siempre las funciones `__builtin_*` sobre una implementación manual con bucles para `popcount`, `clz` o `ctz` en cualquier juez que use GCC (Codeforces, la mayoría de jueces de ICPC); solo verifica el caso $x == 0$ por separado si tu problema puede recibir ese valor.

60.11 Conversión entre Bases Numéricas (Binario, Octal, Hexadecimal)

Idea: Cualquier entero puede representarse en distintas bases (2, 8, 16, etc.) sin cambiar su valor, solo su notación. La conversión entre binario y bases potencias de 2 (octal = 2^3 , hexadecimal = 2^4) es especialmente directa porque cada dígito de esas bases corresponde exactamente a un grupo fijo de bits (3 bits para octal, 4 bits para hexadecimal), sin necesidad de hacer divisiones sucesivas como al convertir a una base arbitraria (por ejemplo, base 10). Esta correspondencia directa es la razón por la que hexadecimal es la notación preferida para representar patrones de bits de forma compacta y legible.

¿Por qué usarlo? Trabajar directamente con literales binarios largos (0b101101101011) es propenso a errores de conteo; la notación hexadecimal agrupa esos mismos bits en dígitos de a 4 (0xB6B), mucho más fácil de leer y escribir correctamente. Además, muchas herramientas de depuración, formatos de archivo, y problemas de competencia (hashing, criptografía simple, checksums) presentan datos directamente en hexadecimal, así que convertir con fluidez entre representaciones es una habilidad práctica necesaria, distinta de las operaciones bitwise en sí (que operan sobre el valor, no sobre su representación textual).

Casos de uso:

- Leer o imprimir valores en un formato de entrada/salida que especifica hexadecimal u octal (frecuente en problemas de parsing de datos crudos o protocolos).
- Depurar manualmente el valor de una máscara de bits, convirtiéndola a hexadecimal para verificar qué bits están encendidos de un vistazo.
- Implementar conversores de base como subrutina en problemas de teoría de números o representación numérica.
- Trabajar con colores en formato RGB hexadecimal (0xFF00FF) en problemas que simulan gráficos o imágenes.

Complejidad: $O(\log_b x)$ para convertir un número x a base b mediante divisiones sucesivas (el número de dígitos de x en base b), lo cual es $O(\log x)$ en general. Para bases potencia de 2, la conversión es $O(\text{bits}/k)$ donde $k = \log_2 b$ (por ejemplo, $k = 4$ para hexadecimal), ya que se procesan grupos de k bits a la vez sin necesidad de división real.

```
1 struct ConversionDeBases {
2
3     // READ: convierte un entero no negativo a su representacion
4     // en
5     // string en la base indicada (2 <= base <= 16), sin ceros a
6     // la
7     // izquierda (excepto para el valor 0)
8     static string aBase(long long x, int base) {
9         if (x == 0) return "0";
10        string digitos = "0123456789ABCDEF";
11        string resultado;
12        while (x > 0) {
13            resultado += digitos[x % base];
14            x /= base;
15        }
16        reverse(resultado.begin(), resultado.end());
17        return resultado;
18    }
19 }
```

```
16 } // O(log_base(x))
17
18 // READ: convierte un string en una base dada (2 <= base <=
19 // 16)
20 // de vuelta a su valor entero; asume digitos validos para
21 // esa base
22 static long long desdeBase(const string& s, int base) {
23     long long resultado = 0;
24     for (char c : s) {
25         int valorDigito = (c >= '0' && c <= '9') ? c - '0' : c -
26         'A' + 10;
27         resultado = resultado * base + valorDigito;
28     }
29     return resultado;
30 } // O(longitud de s)
31
32 // READ: version rapida de binario a decimal, agrupando de a
33 // 1
34 // bit (caso general de "desdeBase" especializado para base
35 // 2)
36 static long long binarioADecimal(const string& binario) {
37     return desdeBase(binario, 2);
38 } // O(longitud del string)
39
40 //
41 // -----
42 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
43 // -----
44
45 // 1. Conversion directa binario <-> hexadecimal por grupos:
46 // como
47 // 16 = 2^4, cada digito hexadecimal corresponde
48 // exactamente a
49 // 4 bits binarios, sin necesidad de pasar por decimal:
50 // agrupar el binario de derecha a izquierda en bloques de
51 // 4
52 // (rellenando con ceros a la izquierda si es necesario) y
53 // convertir cada bloque independientemente.
54 //
55 // 2. Uso de bitset para conversion rapida: para un numero de
56 // ancho fijo conocido, bitset<32>(x).to_string() da la
57 // representacion binaria completa (con ceros a la
58 // izquierda)
59 // en O(32) sin escribir un bucle manual.
60 //
61 // 3. Formato nativo de C++ con printf/cout: printf("%x", x)
62 // o
63 // printf("%0", x) imprimen directamente en hexadecimal u
64 // octal; con cout, se usa std::hex o std::oct como
65 // manipulador de stream (cout << hex << x).
66 //
67 // 4. Literales en el codigo fuente: C++14 en adelante
68 // permite
69 // escribir literales binarios directamente como 0b1011010,
70 //
71 // ademas de los ya existentes 0x (hex) y 0 seguido de
72 // digitos
73 // (octal, ej. 017 == 15 en decimal).
74 };
```

Ejemplo de funcionamiento:

```
1 long long x = 2739; // valor en decimal
2
3 string hex = ConversionDeBases::aBase(x, 16);
4 // hex = "AB3" (2739 = 10*256 + 11*16 + 3)
5
6 string oct = ConversionDeBases::aBase(x, 8);
7 // oct = "5263" (2739 en octal)
8
9 string bin = ConversionDeBases::aBase(x, 2);
10 // bin = "101010110011"
11
12 long long deVuelta = ConversionDeBases::desdeBase("AB3", 16);
13 // deVuelta = 2739 (vuelve al valor original)
```

Regla práctica: Usa hexadecimal (no binario ni octal) como notación por defecto para depurar o escribir máscaras de bits de más de 8 bits — la correspondencia 1 dígito = 4 bits hace que sea mucho más fácil detectar errores a simple vista que contar ceros y unos en una cadena binaria larga.

60.12 Overflow y Wraparound en Enteros

Idea: Cada tipo entero tiene un número fijo de bits, y por lo tanto un rango finito de valores representables. Cuando una operación aritmética produce un resultado

fuera de ese rango, ocurre *overflow*. El comportamiento difiere radicalmente según el tipo: en enteros **unsigned**, el overflow está **definido** por el estándar de C++ como aritmética módulo 2^b (*wraparound*): sumar 1 al máximo valor representable da 0, como un odómetro que da la vuelta. En enteros **signed**, el overflow es **comportamiento indefinido** (UB): el compilador puede asumir que nunca ocurre y optimizar el código de formas que rompen la lógica esperada, en vez de simplemente "dar la vuelta" de forma predecible.

¿Por qué usarlo? Entender esta distinción es crítico porque el overflow en **signed** no es solo "un resultado incorrecto pero predecible" — es UB, lo que significa que el compilador puede generar código que se comporta de forma sorprendente (por ejemplo, eliminar una verificación como `if (x + 1 > x)` asumiendo que siempre es verdadera, porque "asume" que no hay overflow). Esto se relaciona directamente con Representación Binaria y Complemento a 2: el *wraparound* en **unsigned** es consecuencia directa de la aritmética módulo 2^b que implementa esa representación, mientras que el rango asimétrico del complemento a 2 (un negativo más que positivos) es la razón de casos borde como `INT_MIN`.

Casos de uso:

- Detectar y prevenir overflow al multiplicar valores grandes antes de aplicar aritmética modular (frecuente en teoría de números y combinatoria).
- Aprovechar el *wraparound* de **unsigned** de forma intencional para implementar aritmética modular con módulo 2^{32} o 2^{64} sin necesidad de un operador % explícito.
- Evitar el caso borde clásico `abs(INT_MIN)`, que es UB porque `-INT_MIN` no es representable en `int`.
- Elegir el tipo de dato correcto (`int` vs. `long long` vs. `unsigned long long`) según el rango esperado de resultados intermedios, no solo del resultado final.

Complejidad: $O(1)$ — el overflow es una propiedad del resultado de una operación aritmética individual, no de un algoritmo; verificar overflow antes de una operación también toma tiempo constante por chequeo.

```

1 struct OverflowWraparound {
2
3     // READ: verifica si a + b desborda el rango de long long,
4     // sin realizar la suma directamente (evita el UB de
5     // calcularla)
6     static bool sumaDesborda(long long a, long long b) {
7         if (b > 0 && a > LLONG_MAX - b) return true; // desborda
8         // hacia arriba
9         if (b < 0 && a < LLONG_MIN - b) return true; // desborda
10        // hacia abajo
11        return false;
12    } // O(1)
13
14    // READ: verifica si a * b desborda el rango de long long,
15    // sin calcular el producto directamente
16    static bool multiplicacionDesborda(long long a, long long b)
17    {
18        if (a == 0 || b == 0) return false;
19        long long resultado = a * b;
20        return resultado / b != a; // si no se puede "deshacer",
21        // hubo overflow
22    } // O(1)
23
24    // READ: demuestra wraparound definido sobre unsigned: suma 1
25    // al
26    // maximo valor de unsigned int y retorna el resultado (0)
27    static unsigned int demostrarWraparound() {
28        unsigned int maximo = UINT_MAX; // 4294967295 en 32 bits

```

```

23     return maximo + 1u; // wraparound definido a 0
24 } // O(1)
25
26 //
27 // -----
28 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
29 // -----
30 // 1. __builtin_add_overflow / __builtin_mul_overflow: GCC
31 // ofrece
32 // funciones built-in que realizan la operacion y retornan
33 // si
34 // hubo overflow en un solo paso, evitando escribir a mano
35 // la
36 // logica de sumaDesborda/multiplicacionDesborda:
37 // long long resultado;
38 // bool desbordo = __builtin_mul_overflow(a, b, &resultado
39 // );
40 //
41 // 2. Caso INT_MIN y valor absoluto: abs(INT_MIN) es UB
42 // porque
43 // -INT_MIN = 2147483648, que excede INT_MAX (2147483647)
44 // en
45 // complemento a 2. Solucion: castear a un tipo mas grande
46 // antes de tomar el valor absoluto, ej.
47 // labs((long long)x), o trabajar con unsigned si solo
48 // importa la magnitud.
49 //
50 // 3. Wraparound como herramienta (hashing con overflow
51 // intencional): algunos esquemas de hashing simple usan
52 // deliberadamente el wraparound de unsigned de 32/64 bits
53 // como "modulo gratis" (modulo 2^32 o 2^64), en vez de
54 // aplicar
55 // % explicito, porque el wraparound de unsigned SI esta
56 // definido por el estandar y es portable.
57 //
58 // 4. Overflow silencioso en int vs deteccion en tiempo de
59 // ejecucion: en modo release, el overflow de signed
60 // simplemente produce un valor incorrecto sin aviso; usar
61 // flags de compilacion como -fsanitize=undefined durante
62 // el
63 // desarrollo/pruebas ayuda a detectarlo antes del envio.
64 };

```

Ejemplo de funcionamiento:

```

1 long long a = LLONG_MAX - 5;
2 bool desb = OverflowWraparound::sumaDesborda(a, 10);
3 // desb = true (a + 10 excede LLONG_MAX)
4
5 unsigned int resultado = OverflowWraparound::
6     demostrarWraparound();
7 // resultado = 0 (UINT_MAX + 1 da la vuelta a 0, comportamiento
8 // definido)
9
10 long long x = 3000000000LL, y = 3000000000LL;
11 bool desbMult = OverflowWraparound::multiplicacionDesborda(x,
12     y);
13 // desbMult = true (el producto excede el rango de long long)

```

Nota: El overflow de **signed** es UB incluso si en la práctica "parece funcionar" con *wraparound* en tu compilador y arquitectura actuales — el compilador puede optimizar asumiendo que nunca ocurre, y ese supuesto puede romper el programa de formas que no son visibles hasta cambiar de compilador, nivel de optimización, o plataforma.

Regla práctica: Usa `long long` por defecto para cualquier cálculo intermedio que pueda acercarse a 10^9 (el rango de `int`), y verifica overflow explícitamente antes de multiplicaciones grandes en vez de confiar en que "probablemente no pase" — es una de las causas más comunes de *Wrong Answer* silencioso en competencia.

60.13 Operaciones Matemáticas en Binario (Suma, Resta, Multiplicación y Exponenciación Modular)

Idea: Las operaciones aritméticas básicas (suma, resta, multiplicación) pueden entenderse y ejecutarse directamente sobre la representación binaria, replicando el mismo algoritmo que se enseña en base 10 pero con

acarreo (*carry*) en base 2: sumar bit a bit propagando el acarreo, restar con préstamo (*borrow*), y multiplicar mediante desplazamientos y sumas (cada bit encendido del multiplicador aporta una copia desplazada del multiplicando). La **exponenciación modular** extiende esta idea combinando desplazamientos con multiplicación modular: se descompone el exponente en su representación binaria y se usa "elevar al cuadrado y multiplicar" (*binary exponentiation*) recorriendo esos bits, reduciendo un problema de $O(n)$ multiplicaciones a $O(\log n)$.

¿Por qué usarlo? Entender la suma/resta/multiplicación a nivel de bits no es solo una curiosidad académica: la exponenciación modular que se deriva de esa misma idea (usar la representación binaria del exponente) es una de las herramientas más usadas en toda la programación competitiva, para calcular $a^b \bmod m$ en $O(\log b)$ en vez de $O(b)$ multiplicaciones directas — esencial cuando b puede ser de hasta 10^{18} . Se relaciona directamente con Overflow y Wraparound: cada multiplicación intermedia en exponenciación modular puede desbordar `long long` si no se aplica el módulo después de cada paso, no solo al final.

Casos de uso:

- Calcular $a^b \bmod m$ eficientemente, base de teoría de números (inverso modular vía Fermat, RSA simplificado, conteo combinatorio modular).
- Calcular el inverso modular de a módulo un primo p como $a^{p-2} \bmod p$ (pequeño teorema de Fermat), necesario para "dividir" en aritmética modular.
- Multiplicación rápida por desplazamientos cuando uno de los operandos es una potencia de 2 o tiene pocos bits encendidos.
- Implementar multiplicación de matrices módulo m elevada a una potencia grande, para resolver recurrencias lineales (Fibonacci, etc.) en $O(\log n)$.

Complejidad: $O(\log b)$ para exponenciación modular de $a^b \bmod m$, ya que el exponente b se recorre bit a bit (su representación binaria tiene $O(\log b)$ bits), realizando una multiplicación modular (costo $O(1)$ para enteros que caben en `long long`, o $O(\log^2 m)$ si se requiere multiplicación modular de enteros grandes tipo *big integer*) por cada bit.

```

1 struct OperacionesBinarias {
2
3     // READ: calcula (base^exponente) % mod usando exponenciacion
4     // binaria (elevar al cuadrado y multiplicar), recorriendo
5     // bits de "exponente" de menos a mas significativo
6     static long long potenciaModular(long long base, long long
7     exponente, long long mod) {
8         long long resultado = 1 % mod;
9         base %= mod;
10        while (exponente > 0) {
11            if (exponente & 1) { // bit actual encendido: incluir
12                // esta potencia
13                resultado = (__int128)resultado * base % mod;
14            }
15            base = (__int128)base * base % mod; // elevar al cuadrado
16            // para el siguiente bit
17            exponente >>= 1; // avanzar al siguiente bit del
18            // exponente
19        }
20        return resultado;
21    } // O(log(exponente))
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

```

18
19 // READ: calcula el inverso modular de a modulo un primo p,
20 // usando el pequeno teorema de Fermat: a^(p-2) mod p == a^-1
21 // mod p
22 static long long inversoModular(long long a, long long p) {
23     return potenciaModular(a, p - 2, p);
24 } // O(log p)
25
26 // READ: multiplica a y b bit a bit mediante desplazamientos
27 // y
28 // sumas (algoritmo "campesino ruso"), como demostracion de
29 // la
30 // mecanica interna de la multiplicacion binaria
31 static long long multiplicacionPorBits(long long a, long long
32 b) {
33     long long resultado = 0;
34     while (b > 0) {
35         if (b & 1) resultado += a; // bit actual encendido: sumar
36         // el multiplicando desplazado
37         a <<= 1; // desplazar el multiplicando (equivale a *2)
38         b >>= 1; // avanzar al siguiente bit del multiplicador
39     }
40     return resultado;
41 } // O(log b)
42
43 //
44 // -----
45 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
46 // -----
47
48 // 1. Por que se usa __int128 en potenciaModular: el producto
49 // de
50 // dos long long que caben casi al limite (~9.2 * 10^18)
51 // puede desbordar long long antes de aplicar el modulo;
52 // usar
53 // __int128 como tipo intermedio evita ese overflow (ver
54 // Overflow y Wraparound). Alternativa sin __int128: usar
55 // "multiplicacion binaria modular" (aplicar la misma idea
56 // de
57 // multiplicacionPorBits, pero reduciendo modulo m en cada
58 // suma), a costa de O(log b) en vez de O(1) por
59 // multiplicacion.
60 //
61 // 2. Exponenciacion de matrices: el mismo esqueleto de
62 // potenciaModular se generaliza reemplazando la
63 // multiplicacion escalar por multiplicacion de matrices
64 // (modular), para resolver recurrencias lineales como
65 // Fibonacci en O(k^3 log n), con k el tamano de la matriz.
66 //
67 // 3. Resta con acarreo (borrow) a nivel de bits: analogo a
68 // la
69 // suma pero propagando un prestamo en vez de un acarreo;
70 // en
71 // la practica, C++ ya maneja esto de forma nativa con el
72 // operador -, y rara vez se implementa a mano salvo en
73 // aritmetica de precision arbitraria (big integers).
74 //
75 // 4. Exponente negativo: potenciaModular no maneja
76 // exponentes
77 // negativos directamente; para a^(-k) mod p, calcular
78 // primero
79 // el inverso modular de a y luego elevarlo a k:
80 // potenciaModular(inversoModular(a, p), k, p).
81 //
82 }
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

Ejemplo de funcionamiento:

```

1 long long mod = 1000000007LL;
2
3 long long resultado = OperacionesBinarias::potenciaModular(2,
4 30, mod);
5 // resultado = 1073741824 % mod = 1073741824 (no alcanza a
6 // superar mod)
7
8 long long inv = OperacionesBinarias::inversoModular(5, mod);
9 // inv es tal que (5 * inv) % mod == 1;
10 // Nota: el valor exacto de inv depende del calculo de 5^(-mod
11 // -2) % mod;
12 // se debe correr el codigo para confirmar el valor exacto
13 // antes de
14 // darlo por verificado.
15
16 long long prodBits = OperacionesBinarias::multiplicacionPorBits
17 (13, 11);
18 // prodBits = 143 (13 * 11, calculado bit a bit sin usar el
19 // operador *)
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

Regla práctica: Usa exponenciación binaria (`potenciaModular`) en cualquier problema que pida $a^b \bmod m$ con b grande — es la técnica estándar y su complejidad logarítmica la hace viable incluso para $b \sim 10^{18}$;

recuerda aplicar el módulo después de *cada* multiplicación intermedia, no solo al resultado final, y usar `__int128` (o multiplicación modular bit a bit) si mod se acerca al límite de `long long`.

60.14 Bitset para Optimización (std::bitset)

Idea: `std::bitset<N>` es un contenedor de tamaño fijo que representa N bits y expone operaciones bitwise (`&`, `|`, `^`, `<<`, `>>`) sobre *todo* el conjunto de bits a la vez. Internamente, el bitset empaqueta los N bits en bloques de 64 bits (palabras de máquina) y ejecuta cada operación bloque por bloque usando instrucciones nativas del procesador, en vez de bit por bit. Esto significa que una operación sobre N bits que "debería" costar $O(N)$ en un `vector<bool>` operado ingenuamente, cuesta $O(N/64)$ con `bitset`, porque cada instrucción de máquina procesa 64 bits en paralelo.

¿Por qué usarlo? El bitset convierte los trucos de "Operaciones y Trucos Básicos" (que operan sobre un solo entero de hasta 64 bits) en una herramienta escalable a conjuntos de miles o millones de elementos: cualquier DP o algoritmo que se pueda expresar como operaciones de conjunto (unión, intersección, "¿es alcanzable este estado?") sobre un universo de tamaño N se acelera por un factor constante de ≈ 64 frente a la misma lógica con `vector<bool>` o `vector<int>` recorrido elemento por elemento. Es la herramienta a considerar cuando Bitmask DP (que representa subconjuntos de hasta 20-24 elementos en un solo `int/long long`) no alcanza porque el universo de elementos es demasiado grande para caber en 64 bits.

Casos de uso:

- Acelerar DP de subconjuntos alcanzables (por ejemplo, "¿qué sumas son alcanzables con un subconjunto de n números?") de $O(n \cdot S)$ a $O(n \cdot S/64)$, con S el rango de sumas.
- Calcular la cantidad de substrings comunes o patrones de coincidencia entre dos strings largos usando bitsets de coincidencia por posición.
- Resolver problemas de conteo de pares que cumplen una condición de conjunto (por ejemplo, cuántos pares de conjuntos se intersectan) representando cada conjunto como un bitset.
- Optimizar transiciones de grafos/DP donde el estado es "qué nodos son alcanzables", operando con `&/|` sobre el bitset completo en vez de iterar nodo por nodo.

Complejidad: $O(N/64)$ (o $O(N/w)$, con w el tamaño de palabra de la arquitectura, típicamente 64) para operaciones bitwise sobre un bitset de N bits, frente a $O(N)$ de la misma operación implementada elemento por elemento; operaciones como `count()` (contar bits encendidos) también se benefician de instrucciones `POPCNT` por bloque, manteniendo el mismo factor de mejora.

```
1  const int MAXN = 100000;
2
3  struct OptimizacionConBitset {
4      bitset<MAXN> conjunto;
5
6      // CREATE: inicializa el bitset vacio (todos los bits en 0)
```

```
7  OptimizacionConBitset() : conjunto() {} // O(N/64)
8
9  // WRITE: agrega el elemento x al conjunto (prende el bit x)
10 void agregar(int x) {
11     conjunto.set(x);
12 } // O(1)
13
14 // READ: verifica si el elemento x esta en el conjunto
15 bool contiene(int x) {
16     return conjunto.test(x);
17 } // O(1)
18
19 // READ: retorna un nuevo bitset con la union de este
20 // conjunto y "otro" (elementos presentes en cualquiera de los dos)
21 bitset<MAXN> unionCon(const bitset<MAXN>& otro) {
22     return conjunto | otro;
23 } // O(N/64)
24
25 // READ: retorna un nuevo bitset con la interseccion de este
26 // conjunto y "otro" (elementos presentes en ambos)
27 bitset<MAXN> interseccionCon(const bitset<MAXN>& otro) {
28     return conjunto & otro;
29 } // O(N/64)
30
31 // READ: retorna la cantidad de elementos (bits encendidos)
32 // en
33 // el conjunto
34 int tamano() {
35     return (int)conjunto.count();
36 } // O(N/64)
37
38 // -----
39 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
40 // -----
41 // 1. Subset-sum con bitset: para saber que sumas son
42 //    alcanzables
43 //    con un subconjunto de un arreglo de n numeros no
44 //    negativos
45 //    con suma total S, se mantiene bitset<S+1> alcanzables
46 //    con
47 //    el bit 0 encendido, y por cada numero v del arreglo se
48 //    hace
49 //    alcanzables |= (alcanzables << v);
50 //    esto agrega, para cada suma ya alcanzable, la suma
51 //    resultante de incluir v, en O(S/64) por numero.
52 //
53 // 2. bitset de tamano NO conocido en tiempo de compilacion:
54 //    std::bitset requiere que N sea una constante en tiempo
55 //    de
56 //    compilacion; si el tamano es dinamico, usar
57 //    boost::dynamic_bitset o simular con vector<uint64_t> y
58 //    operaciones manuales bloque a bloque.
59 //
60 // 3. Conteo de coincidencias entre strings con bitset: para
61 //    verificar si el patron P aparece en el texto T en la
62 //    posicion i, se puede precomputar, para cada caracter c,
63 //    un
64 //    bitset con las posiciones donde aparece c en T, y
65 //    combinar
66 //    con shifts y AND para verificar coincidencias multiples
67 //    en
68 //    O(|T| |alfabeto| / 64) en vez de O(|T||P|).
69 //
70 // 4. any(), none(), all(): metodos utiles para verificar
71 //    rapidamente si existe al menos un bit encendido (any()),
72 //    ninguno (none()), o todos (all()), cada uno en O(N/64).
73 };
```

Ejemplo de funcionamiento:

```
1 // Subset-sum: que sumas son alcanzables con {2, 3, 7}?
2 vector<int> numeros = {2, 3, 7};
3 bitset<20> alcanzables;
4 alcanzables[0] = 1; // suma 0 siempre alcanzable (subconjunto
5 // vacio)
6
7 for (int v : numeros) {
8     alcanzables |= (alcanzables << v);
9 }
10 // alcanzables tiene encendidos los bits: 0, 2, 3, 5, 7, 9, 10,
11 // 12
12 // (todas las sumas posibles combinando subconjuntos de
13 // {2,3,7})
14
15 bool sumaCincoAlcanzable = alcanzables[5]; // true (2 + 3 = 5)
16 bool sumaCuatroAlcanzable = alcanzables[4]; // false
```

Regla práctica: Si Bitmask DP requiere representar subconjuntos de más de 64 elementos, o si tu

DP/algoritmo puede expresarse como operaciones de conjunto (unión, intersección, "shift y OR" para propagar alcanzabilidad) sobre un universo de hasta unos pocos millones de elementos, usa `bitset` en vez de `vector<bool>` — la ganancia de $\approx 64\times$ suele ser la diferencia entre pasar o no el límite de tiempo.

60.15 Enumeración de Submáscaras (Submask Enumeration)

Idea: Dada una máscara de bits m , una *submáscara* es cualquier máscara s tal que $s \& m = s$ (es decir, todo bit encendido de s también está encendido en m). Hay 2^k submáscaras de una máscara con k bits encendidos, y existe un patrón de iteración clásico que las recorre todas, en orden estrictamente descendente, en $O(2^k)$ usando la operación $s = (s - 1) \& m$: restar 1 a s "toma prestado" del bit encendido más bajo, y el AND con m descarta cualquier bit que no pertenezca a la máscara original, produciendo exactamente la siguiente submáscara menor.

¿Por qué usarlo? Sin este patrón, enumerar las submáscaras de m requeriría recorrer las 2^n máscaras posibles (con n el número total de bits) y filtrar las que cumplen $s \& m = s$, un desperdicio si m tiene pocos bits encendidos. El patrón $(s-1) \& m$ visita únicamente las 2^k submáscaras reales, sin generar ni descartar candidatos inválidos. Es el bloque de construcción fundamental de SOS DP (que agrega esta enumeración sobre todas las máscaras del universo) y de varios problemas de Bitmask DP donde la transición itera sobre subconjuntos de un conjunto dado.

Casos de uso:

- Iterar todas las formas de dividir un subconjunto de elementos en dos partes, en problemas de partición o asignación.
- Transiciones de Bitmask DP donde, dado el conjunto de elementos aun disponibles, se prueba cada subconjunto no vacío para asignar a la siguiente "ronda" o "grupo".
- Como subrutina dentro de SOS DP, para entender el algoritmo ingenio $O(3^n)$ antes de optimizarlo a $O(2^n \cdot n)$.
- Precalcular, para cada máscara, una propiedad agregada sobre todos sus subconjuntos (por ejemplo, el máximo valor de un arreglo restringido a los índices de la submáscara).

Complejidad: $O(2^k)$ para enumerar las submáscaras de una única máscara con k bits encendidos. Si se enumeran las submáscaras de todas las máscaras de n bits, la complejidad total es $O(3^n)$: cada uno de los n bits puede estar en uno de tres estados (no pertenece a la máscara, pertenece a la máscara pero no a la submáscara, o pertenece a ambas), de donde sale el factor 3^n en vez de $2^n \cdot 2^n$.

```
1 struct EnumeracionDeSubmascaras {
2
3     // READ: retorna todas las submascaras de m (incluyendo m
4     // misma
5     // y la mascara vacia 0), en orden estrictamente descendente
6     static vector<int> obtenerSubmascaras(int m) {
```

```
6     vector<int> resultado;
7     for (int s = m; ; s = (s - 1) & m) {
8         resultado.push_back(s);
9         if (s == 0) break; // se llego a la submascara vacia,
10        terminar
11    }
12    return resultado;
13    } // O(2^k), k = bits encendidos de m
14
15    // READ: aplica una funcion "procesar" a cada submascara no
16    // vacia
17    // de m, sin almacenarlas todas en memoria (mas eficiente si
18    // solo
19    // se necesita un acumulado)
20    template<typename Func>
21    static void paraCadaSubmascaraNoVacía(int m, Func procesar) {
22        for (int s = m; s > 0; s = (s - 1) & m) {
23            procesar(s);
24        } // O(2^k), k = bits encendidos de m
25
26    //
27    // -----
28    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
29    // -----
30
31    // 1. Por que (s - 1) & m da la siguiente submascara: restar
32    // 1 a
33    // s apaga el bit encendido mas bajo de s y prende todos
34    // los
35    // bits menores a el (equivalente a apagarBitMasBajo pero
36    // "tomando prestado" en vez de apagando). El AND con m
37    // descarta los bits recién prendidos que no pertenecen a
38    // m,
39    // dejando exactamente la submascara inmediatamente menor.
40    //
41    // 2. Incluir o excluir la mascara vacia (s = 0): el bucle
42    // "for (s = m; ; s = (s-1)&m) { ...; if (s == 0) break;
43    // }"
44    // incluye tanto m como 0 como submascaras validas; para
45    // excluir la vacia, usar la condicion "s > 0" en el for
46    // como
47    // en paraCadaSubmascaraNoVacía.
48    //
49    // 3. Enumerar submascaras de TODAS las mascaras (base de SOS
50    // DP):
51    // for (int m = 0; m < (1 << n); m++)
52    //     for (int s = m; ; s = (s-1) & m) {
53    //         // procesar(m, s)
54    //         if (s == 0) break;
55    //     }
56    // este doble bucle anidado es O(3^n) en total, no O(4^n),
57    // por el argumento de los tres estados por bit.
58    //
59    // 4. Complementos de submascaras: la submascara "
60    // complementaria"
61    // de s dentro de m (los elementos de m que NO estan en s)
62    // se
63    // obtiene como (m ^ s), util quando se necesita dividir m
64    // en
65    // dos partes disjuntas.
```

Ejemplo de funcionamiento:

```
1 int m = 0b1010; // mascara con bits 1 y 3 encendidos (decimal
2 10)
3
4 vector<int> submascaras = EnumeracionDeSubmascaras::
5 obtenerSubmascaras(m);
6 // submascaras = {0b1010, 0b1000, 0b0010, 0b0000}
7 // es decir: {10, 8, 2, 0} - las 4 = 2^2 submascaras de una
8 // mascara
9 // con 2 bits encendidos
10
11 int suma = 0;
12 EnumeracionDeSubmascaras::paraCadaSubmascaraNoVacía(m, [&](int
13 s) {
14     suma += s;
15 });
16 // suma = 8 + 2 + 10 = 20 (excluye la submascara vacia 0)
```

Regla práctica: Usa el patrón $s = (s-1) \& m$ en vez de recorrer todas las 2^n máscaras y filtrar cuando necesites iterar submáscaras de una máscara específica; si en cambio necesitas esta enumeración para todas las máscaras del universo simultáneamente (agregando un valor por cada par máscara-submáscara), reconoce que el patrón general es $O(3^n)$ y considera si SOS DP (siguiente subsección) puede reducirlo a $O(2^n \cdot n)$.

60.16 Suma sobre Subconjuntos (SOS DP)

Idea: SOS DP (*Sum over Subsets*) resuelve un problema específico: dado un arreglo $f[\text{mask}]$ indexado por todas las 2^n máscaras posibles, calcular para cada máscara m la suma (u otra agregación asociativa: máximo, OR, conteo) de $f[s]$ sobre *todas* las submáscaras s de m . La enumeración ingenua de "Submask Enumeration" resuelve esto en $O(3^n)$ recorriendo, para cada máscara, todas sus submáscaras una por una. SOS DP reduce esto a $O(2^n \cdot n)$ procesando la agregación *bit por bit*: en la iteración i , para cada máscara con el bit i encendido, se decide si "incluir" las submáscaras que difieren en ese bit, acumulando el resultado progresivamente en vez de enumerar cada submáscara de forma explícita.

¿Por qué usarlo? La diferencia entre $O(3^n)$ y $O(2^n \cdot n)$ es enorme en la práctica: para $n = 20$, $3^{20} \approx 3.5 \times 10^9$ es demasiado lento, mientras que $2^{20} \cdot 20 \approx 2 \times 10^7$ es trivial. SOS DP logra esto reordenando el cálculo: en vez de "para cada máscara, sumar sobre todas sus submáscaras" (que repite trabajo entre máscaras que comparten submáscaras), procesa "para cada bit, propagar la contribución de las máscaras sin ese bit hacia las máscaras con ese bit", de forma similar a cómo un Fenwick Tree evita recalcular sumas de prefijo desde cero. Es la técnica a usar en cuanto Submask Enumeration se vuelve el cuello de botella por tener que repetirse para todas las máscaras del universo.

Casos de uso:

- Calcular, para cada máscara m , la suma de todos los valores $f[s]$ con s submáscara de m (el caso base de SOS DP).
- Contar, para cada máscara m , cuántos elementos de un arreglo tienen su máscara de bits como submáscara de m (versión "conteo" en vez de "suma").
- Resolver variantes de "AND/OR convolution" donde se necesita $g[m] = \sum_{s \subseteq m} f[s]$ como paso intermedio.
- Problemas de conteo con restricciones tipo "cuántos subconjuntos de propiedades cumple el elemento x ", donde cada propiedad es un bit.

Complejidad: $O(2^n \cdot n)$: hay n iteraciones (una por cada bit), y en cada una se recorren las 2^n máscaras posibles actualizando el acumulado en $O(1)$ por máscara, dando $n \cdot 2^n$ operaciones en total — una mejora exponencial frente al $O(3^n)$ de la enumeración ingenua de submáscaras para todas las máscaras.

```
1 struct SumaSobreSubconjuntos {
2     int n; // numero de bits del universo
3     vector<long long> dp; // dp[mask] = suma de f[s] para toda
        submascara s de mask
4
5     // CREATE: inicializa dp con los valores base f[mask] (la
        suma
6     // "trivial" de cada mascara consigo misma antes de propagar)
7     SumaSobreSubconjuntos(vector<long long> f, int numBits) : n(
        numBits), dp(f) {} // O(2^n)
8
9     // WRITE: ejecuta el algoritmo de SOS DP, transformando dp[
        mask]
10    // de "valor base de mask" a "suma sobre todas las
        submascaras de mask"
11    void calcular() {
12        int totalMascaras = 1 << n;
13        for (int bit = 0; bit < n; bit++) {
14            for (int mask = 0; mask < totalMascaras; mask++) {
15                if (mask & (1 << bit)) {
```

```
16                // mask tiene el bit encendido: suma la contribucion
17                // de la mascara sin ese bit (que ya incluye sus
18                // propias submascaras, procesadas en iteraciones
19                // de bits anteriores)
20                dp[mask] += dp[mask ^ (1 << bit)];
21            }
22        }
23    } // O(2^n * n)
24
25    // READ: retorna la suma sobre subconjuntos ya calculada para
26    // "mask"; debe llamarse despues de calcular()
27    long long consultar(int mask) {
28        return dp[mask];
29    } // O(1)
30
31    //
32    // -----
33    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
34    // -----
35
36    // 1. Por que el orden de los bucles es bit-afuera, mask-
37    //    adentro:
38    //    procesar todos los bits en orden fijo garantiza que, al
39    //    llegar al bit i, dp[mask ^ (1<<bit)] ya acumulo
40    //    correctamente las contribuciones de los bits 0..i-1; si
41    //    se
42    //    invirtiera el orden de los bucles, se perderia esa
43    //    garantia
44    //    y el resultado seria incorrecto (similar a por que el
45    //    orden de las capacidades importa en la mochila 0/1).
46    //
47    // 2. Version "superset sum" (suma sobre superconjuntos):
48    //    para
49    //    calcular, para cada mask, la suma de f[s] con mask
50    //    submascara de s (en vez de al reves), se invierte la
51    //    condicion: if (!(mask & (1 << bit))) dp[mask] += dp[
52    //    mask | (1 << bit)];
53    //
54    // 3. Generalizacion a otras operaciones asociativas:
55    //    reemplazar
56    //    "+=" por "max(...)", "|=" (OR), o incrementar un
57    //    contador
58    //    resuelve variantes como "maximo valor entre las
59    //    submascaras" o "conteo de elementos cuya mascara es
60    //    submascara de mask", sin cambiar la estructura del
61    //    algoritmo.
62    //
63    // 4. Memoria O(2^n): a diferencia de Submask Enumeration (
64    //    que no
65    //    requiere almacenar nada extra), SOS DP necesita el
66    //    arreglo
67    //    dp completo de tamano 2^n, lo cual limita n en la
68    //    practica
69    //    a unos 20-24 bits por restricciones de memoria.
70    };
```

Ejemplo de funcionamiento:

```
1 // f[mask] = valor base asociado a cada mascara (universo de 3
        bits)
2 vector<long long> f = {1, 2, 3, 4, 5, 6, 7, 8}; // f[0..7]
3
4 SumaSobreSubconjuntos sos(f, 3);
5 sos.calcular();
6
7 long long resultado = sos.consultar(0b101); // mask = 5
8 // submascaras de 5 (101) son: 000, 001, 100, 101 -> f[0]+f[1]+
9 // f[4]+f[5]
10 // resultado = 1 + 2 + 5 + 6 = 14
```

Regla práctica: Usa SOS DP en cuanto necesites calcular la agregación sobre submáscaras para *todas* las máscaras del universo simultáneamente (no solo una); si solo necesitas las submáscaras de una única máscara puntual, el patrón $(s-1) \& m$ de Submask Enumeration es más simple y no requiere memoria $O(2^n)$ adicional.

60.17 Bitmask DP (Programación Dinámica sobre Subconjuntos)

Idea: Bitmask DP representa el *estado* de una programación dinámica como un entero donde cada bit indica si un elemento particular ya fue usado, visitado, o cumple cierta condición. En vez de un estado con n dimensiones

booleanas independientes (inviabile de indexar directamente), se codifica el conjunto completo de n elementos como un único entero en $[0, 2^n)$, permitiendo usar ese entero como índice de un arreglo de DP. La transición entre estados típicamente prende un bit adicional (agregar un elemento al subconjunto) o, con Submask Enumeration, itera sobre todos los subconjuntos de los elementos aún disponibles.

¿Por qué usarlo? Muchos problemas de optimización combinatoria son NP-difíciles en general (como el Problema del Vendedor Viajero), pero se vuelven resolubles para n pequeño (típicamente hasta 20-24) mediante Bitmask DP, bajando la complejidad de $O(n!)$ (probar todas las permutaciones) a $O(2^n \cdot n)$ o $O(2^n \cdot n^2)$. Se distingue de SOS DP en que aquí la máscara representa el *estado* de la solución parcial que se está construyendo (qué elementos ya se usaron), mientras que en SOS DP la máscara es simplemente un índice sobre el cual se agregan valores precomputados; y se distingue del DP con Perfil de Bits en que aquí la máscara codifica un subconjunto de *elementos*, no un perfil espacial de una fila/columna de una grilla.

Casos de uso:

- Problema del Vendedor Viajero (TSP): encontrar el camino mínimo que visita todas las ciudades exactamente una vez.
- Asignación óptima de n tareas a n trabajadores minimizando el costo total (problema de asignación, más rápido con Bitmask DP para n pequeño que con Húngaro completo si $n \leq 20$).
- Cubrir un conjunto de elementos con el mínimo número de subconjuntos predefinidos (Set Cover), factible en $O(2^n)$ para n pequeño.
- Contar o encontrar la forma óptima de emparejar elementos en un grafo bipartito pequeño, como alternativa a Bipartite Matching cuando además se necesita optimizar un costo.

Complejidad: $O(2^n \cdot n)$ típicamente, donde 2^n son los posibles estados (subconjuntos) y n es el costo de la transición desde cada estado (probar agregar cada uno de los n elementos); en problemas como TSP la complejidad sube a $O(2^n \cdot n^2)$ porque el estado incluye tanto el subconjunto visitado como el último elemento visitado (n posibilidades adicionales).

```
1  const long long INF = 1e18;
2
3  struct BitmaskDP_TSP {
4      int n;
5      vector<vector<long long>> dist; // matriz de distancias entre
        ciudades
6      vector<vector<long long>> dp;   // dp[mask][i] = costo mínimo
        visitando
7      // exactamente el conjunto "mask",
8      // terminando en la ciudad i
9
10     // CREATE: inicializa la matriz de distancias y la tabla dp
        con
11     // INF, marcando el caso base (empezar solo en la ciudad 0)
12     BitmaskDP_TSP(vector<vector<long long>> matrizDistancias)
13     : n((int)matrizDistancias.size()), dist(matrizDistancias) {
14         dp.assign(1 << n, vector<long long>(n, INF));
15         dp[1][0] = 0; // mascara con solo la ciudad 0 visitada,
        terminando en 0
16     } // O(2^n * n)
17
18     // WRITE: llena la tabla dp iterando sobre todos los estados
```

```
19     // (mascara, ciudad actual) y probando transicionar a cada
20     // ciudad no visitada aun
21     void calcular() {
22         int totalMascaras = 1 << n;
23         for (int mask = 1; mask < totalMascaras; mask++) {
24             for (int actual = 0; actual < n; actual++) {
25                 if (!(mask & (1 << actual))) continue; // actual
        no esta en mask, invalido
26                 if (dp[mask][actual] == INF) continue; // estado
        inalcanzable, saltar
27
28                 for (int siguiente = 0; siguiente < n; siguiente++) {
29                     if (mask & (1 << siguiente)) continue; // siguiente
        ya visitada
30                     int nuevaMascara = mask | (1 << siguiente);
31                     long long nuevoCosto = dp[mask][actual] + dist[actual]
        [siguiente];
32                     dp[nuevaMascara][siguiente] = min(dp[nuevaMascara][
        siguiente], nuevoCosto);
33                 }
34             }
35         }
36     } // O(2^n * n^2)
37
38     // READ: retorna el costo minimo de visitar todas las
        ciudades
39     // exactamente una vez y regresar a la ciudad 0; debe
        llamarse
40     // despues de calcular()
41     long long costoMinimo() {
42         int mascaraCompleta = (1 << n) - 1;
43         long long mejor = INF;
44         for (int ultima = 1; ultima < n; ultima++) {
45             if (dp[mascaraCompleta][ultima] == INF) continue;
46             mejor = min(mejor, dp[mascaraCompleta][ultima] + dist[
        ultima][0]);
47         }
48         return mejor;
49     } // O(n)
50
51     //
52     // MODIFICACIONES Y COMENTARIOS CON LOS USOS
53     //
54     // 1. Reconstruccion del camino optimo: guardar un arreglo
55     // padre[mask][actual] con la ciudad anterior que produjo
56     // el
57     // minimo, y reconstruir el camino hacia atras desde
58     // (mascaraCompleta, ultima) hasta llegar a la mascara
59     // inicial.
60     // 2. Set Cover con Bitmask DP: dp[mask] = minimo numero de
61     // subconjuntos para cubrir exactamente los elementos en
62     // mask;
63     // se itera sobre submascaras (Submask Enumeration) del
64     // complemento de mask para probar agregar cada
65     // subconjunto
66     // predefinido disponible.
67     // 3. Problema de asignacion (Assignment Problem): dp[mask] =
68     // costo minimo asignando las primeras popcount(mask)
69     // tareas
70     // a los trabajadores marcados en mask; la transicion
71     // prueba
72     // asignar la siguiente tarea a cada trabajador no usado
73     // aun.
74     // 4. Limite practico de n: Bitmask DP con 2^n estados es
75     // viable
76     // hasta n ~ 20-22 en la mayoria de limites de tiempo (1-2
77     // segundos); para n mayor, es necesario buscar una
78     // aproximacion, heuristica, o una propiedad especial del
79     // problema que reduzca el espacio de estados.
```

Ejemplo de funcionamiento:

```
1  vector<vector<long long>> distancias = {
2      {0, 10, 15, 20},
3      {10, 0, 35, 25},
4      {15, 35, 0, 30},
5      {20, 25, 30, 0}
6  };
7
8  BitmaskDP_TSP tsp(distancias);
9  tsp.calcular();
10
11  long long costo = tsp.costoMinimo();
12  // Nota: el valor exacto depende de recorrer las 4!
        permutaciones
13  // posibles del ciclo; se debe correr el codigo para confirmar
        el
```

```

14 // valor exacto antes de darlo por verificado (candidato
    esperado
15 // alrededor de 80, correspondiente a la ruta 0-1-3-2-0 o
    equivalente).

```

Regla práctica: Reconoce Bitmask DP cuando el problema mencione "orden óptimo de visitar/asignar n elementos" con $n \leq 20$ -24, y el estado natural del problema sea "qué subconjunto de elementos ya se procesó" (posiblemente combinado con "en qué elemento se terminó"); si además necesitas agregar valores sobre subconjuntos de un universo fijo en vez de construir una solución paso a paso, esa es SOS DP en vez de Bitmask DP.

60.18 DP con Perfil de Bits (Broken Profile DP)

Idea: El DP con perfil de bits (*broken profile DP*) resuelve problemas de *tiling* o cobertura sobre una grilla 2D, donde no se puede procesar cada celda de forma completamente independiente porque las decisiones sobre celdas vecinas están acopladas (por ejemplo, colocar una ficha de dominó de 1×2 afecta dos celdas a la vez). La idea es procesar la grilla celda por celda, en orden fila por fila, y usar una máscara de bits (el "perfil") para representar el estado de las últimas m celdas procesadas (típicamente el ancho de una fila, o la mitad de una ficha que "sobresale" hacia la fila siguiente). Cada bit del perfil indica si esa celda ya está cubierta/ocupada, y la transición avanza una celda a la vez, actualizando el perfil en consecuencia.

¿Por qué usarlo? Se distingue de Bitmask DP clásico en que aquí la máscara **no** representa un subconjunto de elementos abstractos, sino un *perfil espacial*: qué celdas de la frontera actual entre "ya decidido" y "aún por decidir" están ocupadas. Esto permite resolver problemas de *tiling* en $O(\text{filas} \cdot \text{columnas} \cdot 2^{\text{columnas}})$ en vez de intentar enumerar todas las formas de cubrir la grilla directamente (factorial o peor). Es la técnica correcta cuando el ancho de la grilla es pequeño (hasta 15-20) pero el largo puede ser grande, ya que el costo exponencial recae solo sobre el ancho.

Casos de uso:

- Contar el número de formas de cubrir completamente una grilla $n \times m$ con fichas de dominó (1×2).
- Contar tilings con poliomínos más complejos (piezas en L, piezas de 3 o más celdas) sobre una grilla pequeña.
- Problemas de "máximo número de piezas colocables" sujeto a restricciones de no superposición en una grilla.
- Variantes donde algunas celdas están bloqueadas (*obstáculos*) y se debe cubrir exactamente el resto.

Complejidad: $O(\text{filas} \cdot \text{columnas} \cdot 2^{\text{columnas}})$: hay $\text{filas} \times \text{columnas}$ celdas a procesar, y en cada una se itera sobre los 2^{columnas} perfiles posibles (el estado de si cada una de las columnas de la "frontera" está ocupada); el factor exponencial depende del *ancho* de la grilla, no de su largo, por lo que conviene orientar la grilla para que la dimensión pequeña sea la que entra en el exponente.

```

1 struct DPPerfilDeBits {
2     int filas, columnas;

```

```

3 // CREATE: guarda las dimensiones de la grilla a cubrir con
4 // fichas de dominó (1x2, horizontales o verticales)
5 DPPerfilDeBits(int f, int c) : filas(f), columnas(c) {} // 0
6 (1)
7
8 // READ: calcula el numero de formas de cubrir completamente
9 la
10 // grilla filas x columnas con fichas de dominó, usando DP
11 // con
12 // perfil de bits celda por celda
13 long long contarTilings() {
14     int totalCeldas = filas * columnas;
15     int totalPerfiles = 1 << columnas;
16     // dp[i] = numero de formas de llegar al estado de perfil i
17     // en la celda actual (bit encendido = celda ya cubierta)
18     vector<long long> dp(totalPerfiles, 0);
19     dp[0] = 1; // antes de procesar cualquier celda, perfil
20     // vacio
21
22     for (int celda = 0; celda < totalCeldas; celda++) {
23         int fila = celda / columnas, col = celda % columnas;
24         vector<long long> dpSiguiente(totalPerfiles, 0);
25
26         for (int perfil = 0; perfil < totalPerfiles; perfil++) {
27             if (dp[perfil] == 0) continue;
28
29             bool celdaOcupada = (perfil >> col) & 1;
30             if (celdaOcupada) {
31                 // esta celda ya fue cubierta por una ficha vertical
32                 // de la fila anterior: apagar el bit y avanzar
33                 dpSiguiente[perfil ^ (1 << col)] += dp[perfil];
34             } else {
35                 // opcion 1: cubrir con ficha horizontal (necesita
36                 // que la siguiente columna exista y este libre)
37                 if (col + 1 < columnas && !(perfil >> (col + 1)) &
38                     1)) {
39                     int nuevoPerfil = perfil | (1 << col) | (1 << (col
40                     + 1));
41                     dpSiguiente[nuevoPerfil ^ (1 << col)] += dp[perfil
42                     ];
43                 }
44                 // opcion 2: cubrir con ficha vertical (prende el
45                 // bit para que la fila siguiente lo procese ocupado)
46                 dpSiguiente[perfil | (1 << col)] += dp[perfil];
47             }
48             dp = dpSiguiente;
49         }
50         return dp[0]; // al terminar, el perfil debe quedar vacio
51     } // 0(filas * columnas * 2^columnas)
52
53 //
54 // -----
55 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
56 // -----
57
58 // 1. Diferencia con Bitmask DP clasico: aqui el bit i del
59 // perfil
60 // NO representa "el elemento i fue usado" de forma
61 // abstracta,
62 // sino "la celda en la columna i de la fila actual/
63 // anterior
64 // esta ocupada"; el significado del bit cambia con la
65 // posicion en la grilla, no es un conjunto fijo de
66 // elementos.
67 //
68 // 2. Procesar fila completa en vez de celda por celda: una
69 // variante mas eficiente procesa perfil por FILA (2^
70 // columnas
71 // transiciones por fila) en vez de celda por celda dentro
72 // de
73 // la fila, evaluando de una vez todas las formas de
74 // llenar
75 // una fila dado el perfil que "sobresale" de la fila
76 // anterior.
77 //
78 // 3. Grillas con obstaculos: si una celda esta bloqueada, se
79 // fuerza esa celda a permanecer "sin cubrir" en el perfil
80 // (no se permite ninguna ficha sobre ella), y al final
81 // solo
82 // se aceptan perfiles donde las celdas bloqueadas
83 // coincidan
84 // con el patron esperado.
85 //
86 //
87 // 4. Piezas mas grandes (triominos, poliominos): el mismo
88 // esqueleto aplica, pero las opciones de transicion en
89 // cada
90 // celda se multiplican segun las formas de la pieza; el
91 // ancho
92 // de la grilla sigue siendo el factor limitante en el
93 // exponente.
94 };

```


Ejemplo de funcionamiento:

```

1 DPPerfilDeBits grilla(2, 3); // grilla de 2 filas por 3
  columnas
2
3 long long formas = grilla.contarTilings();
4 // Nota: el valor exacto depende de la simulacion completa del
  DP
5 // celda por celda; se debe correr el codigo para confirmarlo
  antes
6 // de darlo por verificado (candidato esperado: 3,
  correspondiente a
7 // las formas conocidas de cubrir una grilla 2x3 con dominos).

```

Regla práctica: Reconoce DP con Perfil de Bits cuando el problema hable de "cubrir/llevar una grilla" con piezas que ocupan más de una celda y el ancho de la grilla sea pequeño (hasta 15-20); si en cambio el estado del problema es "qué subconjunto de elementos abstractos ya fue usado" sin relación espacial entre ellos, es Bitmask DP clásico en vez de esta variante.

60.19 XOR: Propiedades y Aplicaciones

Idea: El operador XOR ($\oplus$) tiene un conjunto de propiedades algebraicas que lo hacen especialmente útil en competencia: es conmutativo y asociativo, $a \oplus a = 0$ (todo elemento es su propio inverso), $a \oplus 0 = a$ (elemento neutro), y XOR es su propia operación inversa ($a \oplus b \oplus b = a$). Estas propiedades convierten a XOR en una herramienta natural para "cancelar" pares repetidos, invertir una operación sin guardar el valor original, y detectar diferencias bit a bit entre dos valores (el resultado de $a \oplus b$ tiene encendidos exactamente los bits en los que a y b difieren).

¿Por qué usarlo? A diferencia de AND/OR (que no son invertibles: una vez que un bit se apaga con AND o se prende con OR, no hay forma de recuperar el valor original solo con la misma operación), XOR es la única operación bitwise básica que es su propia inversa, lo que la hace ideal para problemas donde se necesita "deshacer" una operación o detectar un elemento "distinto" entre muchos iguales. Es la base conceptual sobre la que se construyen Base Lineal de XOR (para subconjuntos arbitrarios) y Trie de Bits (para pares), que se ven a continuación.

Casos de uso:

- Encontrar el único elemento que aparece un número impar de veces en un arreglo donde todos los demás aparecen un número par de veces (XOR de todo el arreglo).
- Intercambiar dos variables sin usar una variable temporal (*XOR swap*), aunque en la práctica `std::swap` es preferible por claridad.
- Calcular el XOR de un rango $[l, r]$ usando XOR de prefijos (análogo a Prefix Sums, pero con XOR en vez de suma), ya que $\text{XOR}(l, r) = \text{prefijoXOR}(r) \oplus \text{prefijoXOR}(l - 1)$.
- Detectar si dos configuraciones (representadas como enteros) difieren en exactamente un bit, o contar en cuántos bits difieren (popcount del XOR).

Complejidad: $O(1)$ por operación XOR individual, o $O(n)$ para calcular el XOR de un arreglo completo de n elementos (una pasada); con precómputo de prefijos

XOR en $O(n)$, cualquier consulta de XOR sobre un rango se responde en $O(1)$.

```

1 struct PropiedadesXOR {
2     vector<long long> prefijoXOR;
3
4     // CREATE: precalcula el XOR acumulado de arr[0..i-1] para
    cada
5     // posicion i, permitiendo consultas de XOR de rango en O(1)
6     PropiedadesXOR(vector<long long> arr) {
7         prefijoXOR.assign(arr.size() + 1, 0);
8         for (int i = 0; i < (int)arr.size(); i++) {
9             prefijoXOR[i + 1] = prefijoXOR[i] ^ arr[i];
10        } // O(n)
11
12        // READ: retorna el XOR de todos los elementos en el rango
13        // [izq, der] (0-indexado, inclusivo en ambos extremos)
14        long long xorDeRango(int izq, int der) {
15            return prefijoXOR[der + 1] ^ prefijoXOR[izq];
16        } // O(1)
17
18        // READ: dado un arreglo donde todos los elementos aparecen
    un
19        // numero par de veces excepto uno, encuentra ese elemento
    unico
20        static long long encontrarUnico(const vector<long long>& arr)
    {
21            long long resultado = 0;
22            for (long long v : arr) resultado ^= v;
23            return resultado;
24        } // O(n)
25
26        //
27        // -----
28        // MODIFICACIONES Y COMENTARIOS CON LOS USOS
29        // -----
30        // 1. Por que XOR cancela pares: a ^ a = 0 y 0 ^ b = b para
    cualquier b; al hacer XOR de todo el arreglo, cada
31        // elemento
32        // que aparece un numero PAR de veces se cancela consigo
    mismo
33        // (an ^ a ^ ... = 0), dejando solo el elemento con
34        // multiplicidad impar.
35        //
36        // 2. Dos elementos unicos en vez de uno: si hay exactamente
    DOS
37        // elementos con multiplicidad impar (en vez de uno), el
    XOR
38        // total da a ^ b (no separa cual es cual). Se puede
    separar
39        // encontrando un bit donde a y b difieren (cualquier bit
40        // encendido de a ^ b) y dividiendo el arreglo en dos grupos
41        // segun ese bit, aplicando XOR por separado a cada grupo.
42        //
43        // 3. XOR swap (nota historica, no recomendado en la practica
    ):
44        // a ^= b; b ^= a; a ^= b; intercambia a y b sin variable
45        // temporal, pero falla silenciosamente si a y b son la
    MISMA
46        // variable (referencian la misma direccion de memoria),
47        // dejando ambas en 0. std::swap es mas seguro y
    usualmente
48        // igual de rapido tras optimizacion del compilador.
49        //
50        // 4. XOR y paridad de inversiones: el bit menos
    significativo
51        // del XOR de dos permutaciones codificadas apropiadamente
52        // puede relacionarse con la paridad de la permutacion que
    las
53        // transforma una en otra, en problemas especificos de
    combinatoria con permutaciones.
54        //
55    };

```

Ejemplo de funcionamiento:

```

1 vector<long long> arr = {4, 2, 4, 5, 2, 5, 7};
2 long long unico = PropiedadesXOR::encontrarUnico(arr);
3 // unico = 7 (todos los demas aparecen exactamente 2 veces)
4
5 vector<long long> datos = {3, 6, 1, 9, 4};
6 PropiedadesXOR px(datos);
7
8 long long xorRango = px.xorDeRango(1, 3);
9 // xorRango = 6 ^ 1 ^ 9 = 14

```

Regla práctica: Sospecha de XOR en cuanto un problema mencione "elemento único", "número impar de apariciones", o pida "diferencia entre bits" de dos valores; si en cambio necesitas maximizar/minimizar el XOR de

un par o un subconjunto (no solo calcularlo), pasa a Trie de Bits o Base Lineal de XOR respectivamente, que se apoyan en estas mismas propiedades pero resuelven un problema de optimización, no solo de cálculo directo.

60.20 Base Lineal de XOR (XOR Basis / Linear Basis)

Idea: Los enteros de b bits forman un espacio vectorial sobre el cuerpo $GF(2)$ (donde la "suma" de vectores es XOR y la "multiplicación por escalar" es trivial, ya que solo hay 0 y 1). Al igual que en álgebra lineal sobre los reales, un conjunto de vectores puede reducirse a una *base*: un subconjunto linealmente independiente que genera, mediante combinaciones (XORs), exactamente el mismo espacio (*span*) que el conjunto original. La Base Lineal de XOR se construye con un proceso análogo a la eliminación gaussiana: se procesa cada número, y para cada bit desde el más significativo, si ya existe un elemento de la base con ese bit como "pivote", se hace XOR con él (eliminando ese bit); si no existe, el número se convierte en el nuevo elemento pivote de ese bit.

¿Por qué usarlo? Mientras que las propiedades básicas de XOR (vistas antes) permiten calcular el XOR de un conjunto *fijo* de elementos, la Base Lineal responde preguntas sobre *todos los subconjuntos posibles* de un arreglo en tiempo logarítmico: cuál es el máximo/mínimo XOR alcanzable eligiendo cualquier subconjunto, si un valor objetivo v es alcanzable como XOR de algún subconjunto, y cuántos subconjuntos distintos producen cada valor de XOR. Se distingue del Trie de Bits (que resuelve "máximo XOR de un *par*" en $O(n \log(\max))$) en que la Base Lineal resuelve problemas sobre *subconjuntos de tamaño arbitrario* del arreglo completo, no solo pares, comprimiendo hasta n elementos en a lo sumo b elementos de la base (con b el número de bits).

Casos de uso:

- Encontrar el máximo XOR alcanzable eligiendo cualquier subconjunto (no vacío) de un arreglo.
- Verificar si un valor objetivo v puede formarse como XOR de algún subconjunto del arreglo (*query* de pertenencia al span).
- Contar cuántos subconjuntos distintos del arreglo producen cada valor de XOR posible (usando la dimensión de la base y el número de elementos "redundantes").
- Mantener una base de XOR de forma incremental (agregar elementos uno a uno) para responder consultas de máximo XOR sobre prefijos del arreglo.

Complejidad: $O(n \cdot b)$ para construir la base a partir de n números de b bits cada uno (cada inserción cuesta $O(b)$, al recorrer los posibles bits pivote de mayor a menor). Una vez construida la base (que tiene a lo sumo b elementos), consultar el máximo XOR alcanzable, o verificar si un valor pertenece al span, cuesta $O(b)$.

```
1 struct BaseLinealXOR {
2     static const int BITS = 60; // suficiente para valores hasta
        ~1018
3     vector<long long> base;    // base[i] = elemento pivote del
        bit i, o 0 si no existe
4 }
```

```
5 // CREATE: inicializa la base vacia (todos los pivotes en 0,
6 // que nunca es un valor valido de pivote real)
7 BaseLinealXOR() : base(BITS, 0) {} // O(BITS)
8
9 // WRITE: intenta insertar x en la base; si x es linealmente
10 // independiente de la base actual, se agrega como nuevo
    pivote y
11 // retorna true; si x ya esta en el span de la base (se
    reduce a
12 // 0), retorna false sin modificar la base
13 bool insertar(long long x) {
14     for (int bit = BITS - 1; bit >= 0; bit--) {
15         if (!((x >> bit) & 1)) continue; // este bit ya esta en
            0, seguir al siguiente
16         if (base[bit] == 0) {
17             base[bit] = x; // no hay pivote para este bit: x se
                convierte en el pivote
18             return true;
19         }
20         x ^= base[bit]; // eliminar este bit usando el pivote
            existente
21     }
22     return false; // x se redujo a 0: era combinacion lineal de
        la base existente
23 } // O(BITS)
24
25 // READ: retorna el maximo valor de XOR alcanzable combinando
26 // cualquier subconjunto de los elementos de la base (
    equivalente
27 // al maximo XOR de subconjunto del arreglo original)
28 long long maximoXOR() {
29     long long resultado = 0;
30     for (int bit = BITS - 1; bit >= 0; bit--) {
31         // si combinar con base[bit] AUMENTA el resultado,
            hacerlo
32         if ((resultado ^ base[bit]) > resultado) {
33             resultado ^= base[bit];
34         }
35     }
36     return resultado;
37 } // O(BITS)
38
39 // READ: verifica si x puede formarse como XOR de algun
40 // subconjunto de los elementos originalmente insertados en
    la
41 // base (x pertenece al span de la base)
42 bool perteneceAlSpan(long long x) {
43     for (int bit = BITS - 1; bit >= 0; bit--) {
44         if (!((x >> bit) & 1)) continue;
45         if (base[bit] == 0) return false; // no hay forma de
            eliminar este bit
46         x ^= base[bit];
47     }
48     return x == 0; // si se logro reducir x completamente a 0,
        pertenece al span
49 } // O(BITS)
50
51 // READ: retorna la dimension de la base (cantidad de
    elementos
52 // pivote distintos de 0), es decir, el numero de elementos
53 // linealmente independientes encontrados hasta ahora
54 int dimension() {
55     int cont = 0;
56     for (long long v : base) if (v != 0) cont++;
57     return cont;
58 } // O(BITS)
59
60 //
61 // -----
62 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
63 // -----
64
65 // 1. Por que maximoXOR() funciona de mayor a menor bit: al
66 // procesar los bits de mas a menos significativo, cada
67 // decision de "aplicar o no" el pivote de ese bit es
    optima
68 // de forma greedy: aplicar un pivote con un bit mas alto
    que
69 // el actual resultado siempre aumenta el valor (nunca se
    puede compensar despues con bits mas bajos), por lo que
70 // la
71 // eleccion greedy bit a bit es correcta.
72
73 // 2. Contar subconjuntos que producen cada valor de XOR: si
    la
74 // base tiene dimension d a partir de n elementos
    insertados,
75 // hay exactamente 2(n-d) subconjuntos distintos que
    producen
76 // cada valor alcanzable (los n-d elementos "redundantes"
    que
77 // no aportaron un nuevo pivote pueden incluirse o no en
    pares
```

```

76 // que se cancelan).
77 //
78 // 3. Mínimo XOR de subconjunto no vacío: a diferencia del
79 // máximo, el mínimo XOR de un subconjunto NO VACÍO es
80 // simplemente el menor elemento de la base (el pivote con
81 // el bit mas bajo), ya que cualquier combinacion adicional
82 // solo puede prender bits mas altos sin poder apagar ese bit
83 // de forma que reduzca el valor por debajo de el.
84 //
85 // 4. Base lineal incremental sobre prefijos: para responder
86 // "máximo XOR de un subconjunto del prefijo arr[0..i]",
87 // se mantiene una base que se actualiza insertando arr[i] en
88 // cada paso, sin necesidad de reconstruir desde cero.
89 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> arr = {8, 1, 2, 12};
2 BaseLinealXOR base;
3 for (long long v : arr) base.insertar(v);
4
5 long long maxXor = base.maximoXOR();
6 // Nota: el valor exacto depende del proceso de eliminacion
7 // gaussiana paso a paso sobre {8, 1, 2, 12}; se debe correr el
8 // codigo para confirmarlo antes de darlo por verificado (
9 // candidato
10 // esperado: 15, combinando 8 ^ 2 ^ ... segun el resultado real
11 // de la reduccion).
12
13 bool puedeFormar5 = base.perteneceAlSpan(5);
14 // verifica si 5 es alcanzable como XOR de algun subconjunto de
15 // arr
16
17 int dim = base.dimension();
18 // dim = numero de elementos independientes encontrados (a lo
19 // sumo 4,
20 // el tamaño de arr, pero puede ser menor si hay dependencia
21 // lineal)

```

Regla práctica: Usa Base Lineal de XOR cuando el problema pida optimizar (maximizar/minimizar) o verificar la alcanzabilidad del XOR de *subconjuntos arbitrarios* de un arreglo, no solo pares; si el problema se limita a pares de elementos (por ejemplo, "máximo XOR entre dos elementos del arreglo"), el Trie de Bits que sigue es más simple y igual de eficiente para ese caso específico.

60.21 Trie de Bits (Binary Trie / XOR Trie)

Idea: Un Trie de Bits es un árbol binario donde cada nodo tiene a lo sumo dos hijos (0 y 1), y cada número insertado se representa como un camino desde la raíz hasta una hoja, recorriendo sus bits de más a menos significativo: en cada nivel, se desciende por la rama "0" o "1" según el valor de ese bit. La estructura completa codifica implícitamente todos los números insertados como prefijos compartidos en el árbol: dos números que comparten sus k bits más significativos comparten los primeros k niveles del camino desde la raíz. Esta compartición de prefijos es lo que permite responder consultas de XOR de forma *greedy*, nivel por nivel, sin tener que comparar contra cada número insertado individualmente.

¿Por qué usarlo? El problema central que resuelve el Trie de Bits es: dado un conjunto de números y una consulta q , encontrar el elemento del conjunto que *maximiza* (o minimiza) $q \oplus x$. Sin el trie, esto requeriría $O(n)$ por consulta (probar cada elemento); con el trie, se resuelve en $O(b)$ (con b el número de bits) recorriendo el árbol de forma greedy: en cada nivel, se intenta descender por la rama *opuesta* al bit correspondiente de q (porque un bit distinto en esa posición maximiza la contribución de

ese bit al XOR resultante, que vale más que cualquier combinación posible de los bits restantes más bajos), y solo se toma la rama igual si la opuesta no existe en el trie. Frente a la Base Lineal de XOR, el trie resuelve específicamente el caso "máximo/mínimo XOR con un elemento fijo o entre pares", con la ventaja adicional de soportar inserciones y eliminaciones dinámicas de elementos (llevando un contador de referencias por nodo), algo que la Base Lineal no soporta de forma nativa (eliminar un elemento de una base lineal ya reducida no es trivial).

Casos de uso:

- Máximo XOR entre dos elementos de un arreglo (recorrer el arreglo insertando cada elemento y consultando el máximo XOR con los ya insertados).
- Máximo XOR de un elemento fijo q contra un conjunto dinámico de números que se insertan y eliminan a lo largo de las consultas (a diferencia de la Base Lineal, el trie soporta eliminación con conteo de referencias).
- Máximo XOR de un subarreglo $arr[l..r]$ combinado con prefijos XOR: insertar prefijos XOR en el trie y consultar, respondiendo variantes de "máximo XOR de subarreglo" en $O(n \log(\max))$.
- Contar, para cada consulta q , cuántos elementos del conjunto satisfacen $q \oplus x < k$ (agregando conteos por nodo y explorando ambas ramas cuando corresponde), una extensión del trie más allá de solo maximizar.

Complejidad: $O(b)$ por inserción y por consulta, con b el número de bits considerados (típicamente 30 para `int`, o 60 para `long long`), ya que cada operación recorre exactamente un camino de longitud b desde la raíz. Para n inserciones y n consultas, la complejidad total es $O(n \cdot b)$. El espacio usado es $O(n \cdot b)$ en el peor caso, ya que cada inserción puede crear hasta b nodos nuevos si el número no comparte prefijo con ningún elemento existente.

```

1 struct TrieDeBits {
2     static const int BITS = 30; // suficiente para valores hasta
3     // ~10^9
4
5     struct Nodo {
6         int hijos[2] = {-1, -1}; // indices de los hijos (0 y 1),
7         // -1 si no existe
8         int contador = 0; // cuantos numeros insertados
9         // pasan por este nodo
10    };
11
12    vector<Nodo> nodos;
13
14    // CREATE: inicializa el trie con un unico nodo raiz vacio
15    TrieDeBits() {
16        nodos.push_back(Nodo());
17    } // O(1)
18
19    // WRITE: inserta el numero x en el trie, recorriendo sus
20    // bits de
21    // mas a menos significativo y creando nodos segun sea
22    // necesario
23    void insertar(int x) {
24        int actual = 0; // indice del nodo raiz
25        nodos[actual].contador++;
26        for (int bit = BITS - 1; bit >= 0; bit--) {
27            int b = (x >> bit) & 1;
28            if (nodos[actual].hijos[b] == -1) {
29                nodos[actual].hijos[b] = (int)nodos.size();
30                nodos.push_back(Nodo());
31            }
32            actual = nodos[actual].hijos[b];
33            nodos[actual].contador++;
34        }
35    }
36
37    // ... (otras funciones)
38 };

```

```

29     }
30 } // O(BITS)
31
32 // WRITE: elimina una ocurrencia de x del trie, decrementando
33 // el
34 // contador de cada nodo en su camino; asume que x fue
35 // insertado
36 // previamente (si contador llega a 0, el nodo queda "vacío"
37 // pero
38 // no se libera memoria, por simplicidad)
39 void eliminar(int x) {
40     int actual = 0;
41     nodos[actual].contador--;
42     for (int bit = BITS - 1; bit >= 0; bit--) {
43         int b = (x >> bit) & 1;
44         actual = nodos[actual].hijos[b];
45         nodos[actual].contador--;
46     }
47 } // O(BITS)
48
49 // READ: retorna el máximo valor de x XOR q entre todos los
50 // elementos actualmente insertados en el trie (con contador
51 // > 0)
52 int maximoXORCon(int q) {
53     int actual = 0;
54     int resultado = 0;
55     for (int bit = BITS - 1; bit >= 0; bit--) {
56         int b = (q >> bit) & 1;
57         int deseado = b ^ 1; // rama opuesta: maximiza este bit
58         // del XOR
59         if (nodos[actual].hijos[deseado] != -1 &&
60             nodos[nodos[actual].hijos[deseado]].contador > 0) {
61             resultado |= (1 << bit);
62             actual = nodos[actual].hijos[deseado];
63         } else {
64             // rama opuesta no disponible: forzosamente tomar la
65             // rama igual (este bit del XOR queda en 0)
66             actual = nodos[actual].hijos[b];
67         }
68     }
69     return resultado;
70 } // O(BITS)
71
72 // READ: retorna el mínimo valor de x XOR q entre todos los
73 // elementos actualmente insertados, invirtiendo la lógica
74 // greedy
75 // (preferir la rama IGUAL al bit de q en vez de la opuesta)
76 int minimoXORCon(int q) {
77     int actual = 0;
78     int resultado = 0;
79     for (int bit = BITS - 1; bit >= 0; bit--) {
80         int b = (q >> bit) & 1;
81         if (nodos[actual].hijos[b] != -1 &&
82             nodos[nodos[actual].hijos[b]].contador > 0) {
83             actual = nodos[actual].hijos[b]; // este bit del XOR
84             // queda en 0
85         } else {
86             resultado |= (1 << bit);
87             actual = nodos[actual].hijos[b ^ 1];
88         }
89     }
90     return resultado;
91 } // O(BITS)
92
93 //
94
95 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
96 //
97
98 // 1. Por que la estrategia greedy es correcta: el bit mas
99 // significativo del XOR resultante domina el valor total,
100 // ya
101 // que  $2^{\text{bit}}$  siempre es mayor que la suma de TODOS los
102 // bits
103 // inferiores combinados ( $2^0 + 2^1 + \dots + 2^{(\text{bit}-1)} < 2^{\text{bit}}$ ).
104 // Por eso, maximizar cada bit de mas a menos
105 // significativo de
106 // forma greedy, sin "arrepentirse" despues, produce el
107 // optimo
108 // global - es el mismo principio detras de Base Lineal de
109 // XOR.
110 //
111 // 2. Máximo XOR de subarreglo con prefijos XOR: se inserta
112 // el
113 // prefijoXOR[0] = 0 en el trie primero, y luego,
114 // recorriendo
115 // i de 1 a n, se consulta maximoXORCon(prefijoXOR[i])
116 // ANTES
117 // de insertar prefijoXOR[i] al trie, ya que
118 //  $\text{xor}(l+1, r) = \text{prefijoXOR}[r] \oplus \text{prefijoXOR}[l]$ , y se
119 // necesita
120 // que prefijoXOR[l] con  $l < i$  ya este insertado.

```

```

103 //
104 // 3. Conteo de elementos que cumplen  $x \text{ XOR } q < k$ : se recorre
105 // el
106 // trie en paralelo sobre los bits de q y k; en cada nivel,
107 // si el bit de k es 1, todos los elementos cuya rama
108 // coincide
109 // con "bit de q" (dando 0 en ese bit del XOR)
110 // automáticamente
111 // satisfacen la condición para el resto de bits
112 // inferiores
113 // sin importar como continuen, y se suma su contador
114 // directamente; luego se desciende por la rama que
115 // mantiene
116 // el XOR igual al bit de k para seguir comparando.
117 //
118 // 4. Trie persistente para consultas sobre rangos de
119 // prefijos:
120 // para responder "máximo XOR con q usando solo prefijos
121 // en el
122 // rango [l, r]" (no solo "todos los prefijos hasta ahora
123 //"), se
124 // necesita una versión PERSISTENTE del trie, donde cada
125 // inserción crea una nueva "versión" del árbol
126 // compartiendo
127 // los nodos no modificados con la versión anterior,
128 // permitiendo
129 // consultar la versión en el índice r y restar/filtrar la
130 // versión en l-1 mediante el contador diferencial por
131 // nodo.
132 //
133 // 5. Memoria: reservar nodos.reserve(n * BITS) de antemano
134 // evita
135 // realocaciones repetidas del vector durante muchas
136 // inserciones, mejorando el rendimiento en la práctica.
137 };

```

Ejemplo de funcionamiento:

```

1 vector<int> arr = {3, 10, 5, 25, 2, 8};
2
3 TrieDeBits trie;
4 int mejorXor = 0;
5 for (int i = 0; i + 1 < (int)arr.size(); i++) {
6     trie.insertar(arr[i]);
7     mejorXor = max(mejorXor, trie.maximoXORCon(arr[i + 1]));
8 }
9 // Nota: el valor exacto depende de recorrer todos los pares
10 // insertados incrementalmente; se debe correr el código para
11 // confirmarlo antes de darlo por verificado (candidato
12 // esperado: 28,
13 // correspondiente al par  $5 \oplus 25 = 28$ , el máximo XOR entre dos
14 // elementos de arr).
15
16 // Ejemplo de eliminación dinámica:
17 TrieDeBits trieDinamico;
18 trieDinamico.insertar(6);
19 trieDinamico.insertar(11);
20 int maxAntes = trieDinamico.maximoXORCon(0); // considera {6,
21 // 11}
22 trieDinamico.eliminar(11);
23 int maxDespues = trieDinamico.maximoXORCon(0); // ahora solo
24 // considera {6}

```

Nota: El trie asume valores no negativos y un número fijo de BITS conocido de antemano (30 para valores hasta $\sim 10^9$, 60 para `long long` hasta $\sim 10^{18}$); si el problema incluye negativos, es necesario decidir una representación explícita (por ejemplo, desplazar todos los valores a un rango no negativo antes de insertar) ya que el trie tal como está construido no maneja el bit de signo de complemento a 2 de forma especial.

Regla práctica: Usa Trie de Bits cuando el problema pida maximizar/minimizar el XOR entre *pares* de elementos (o entre un elemento fijo y un conjunto), especialmente si el conjunto cambia dinámicamente con inserciones y eliminaciones a lo largo de las consultas; si en cambio necesitas trabajar con el XOR de *subconjuntos arbitrarios* (no solo pares) del arreglo completo, usa Base Lineal de XOR, que resuelve ese problema distinto de forma más directa.

60.22 Código Gray (Gray Code)

Idea: El Código Gray es una forma de enumerar los 2^n números de n bits de manera que cada número consecutivo en la secuencia difiera del anterior en *exactamente* un bit. Se construye a partir de la representación binaria estándar mediante la fórmula $g(i) = i \oplus (i \gg 1)$: al hacer XOR de un número con su propio desplazamiento a la derecha, cada bit del resultado indica si hubo un "cambio" entre bits adyacentes de i , lo que garantiza que incrementar i en 1 (que en binario estándar puede cambiar varios bits simultáneamente por el acarreo) se traduzca en un cambio de un solo bit en $g(i)$.

¿Por qué usarlo? A diferencia de recorrer los enteros en orden binario estándar (donde, por ejemplo, pasar de 0111 a 1000 cambia los 4 bits a la vez), el Código Gray garantiza transiciones "suaves" de un bit por paso, lo cual es deseable en problemas donde recalcular el estado completo desde cero en cada paso es costoso, pero actualizar incrementalmente tras un solo cambio de bit es barato (similar en espíritu a por qué SOS DP evita recalcular desde cero). Es una herramienta de nicho comparada con Bitmask DP o el Trie de Bits, pero resuelve un problema estructural distinto: no optimiza ni cuenta, sino que da un *orden específico* de recorrido sobre las máscaras.

Casos de uso:

- Enumerar todos los subconjuntos de un conjunto de forma que cada paso agregue o quite un solo elemento, útil cuando mantener una estructura auxiliar (suma acumulada, conjunto activo) es caro de recalcular desde cero.
- Resolver variantes del problema de las Torres de Hanói, donde la secuencia de movimientos óptima se corresponde naturalmente con un Código Gray.
- Generar todas las combinaciones de un conjunto de interruptores/opciones binarias en un orden que minimiza el número de "clics" (cambios de estado) entre configuraciones consecutivas.
- Como herramienta de depuración/visualización: verificar resultados de un algoritmo sobre máscaras probando un *stress test* en un orden donde los errores por cambios bruscos de estado son más fáciles de aislar.

Complejidad: $O(1)$ para calcular el Código Gray de un índice i individual (una operación de shift y un XOR); $O(2^n)$ para generar la secuencia completa de los 2^n códigos Gray de n bits, ya que cada uno se calcula en tiempo constante.

```

1 struct CodigoGray {
2
3     // READ: retorna el codigo Gray correspondiente al indice i
4     // (i-esimo elemento de la secuencia, 0-indexado)
5     static long long grayDeIndice(long long i) {
6         return i ^ (i >> 1);
7     } // O(1)
8
9     // READ: retorna el indice i tal que grayDeIndice(i) ==
10    // codigo,
11    // es decir, invierte la transformacion (Gray a binario
12    // estandar)
13    static long long indiceDeGray(long long codigo) {
14        long long i = 0;
15        while (codigo > 0) {
16            i ^= codigo;
17        }
18    }
19 }

```

```

15     codigo >>= 1;
16 }
17 return i;
18 } // O(bits)
19
20 // READ: genera la secuencia completa de codigos Gray para n
21 // bits, en orden (2^n elementos, cada uno difiere del
22 // anterior
23 // en exactamente un bit)
24 static vector<long long> generarSecuencia(int n) {
25     vector<long long> resultado;
26     long long total = 1LL << n;
27     for (long long i = 0; i < total; i++) {
28         resultado.push_back(grayDeIndice(i));
29     }
30     return resultado;
31 } // O(2^n)
32
33 //
34 // -----
35 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
36 // -----
37
38 // 1. Por que i ^ (i >> 1) da un cambio de un solo bit: al
39 // pasar
40 // de i a i+1 en binario estandar, el acarreo de la suma
41 // "voltea" un bloque de bits consecutivos (todos los 1s
42 // finales se vuelven 0, y el primer 0 se vuelve 1). El
43 // XOR
44 // con el propio desplazamiento detecta exactamente el
45 // limite
46 // de ese bloque de cambio, colapsando el efecto del
47 // acarreo a
48 // un unico bit de diferencia entre g(i) y g(i+1).
49 //
50 // 2. Código Gray reflejado (construccion recursiva): una
51 // forma
52 // alternativa de generar la secuencia es recursivamente:
53 // Gray(n) = ["0" + x para x en Gray(n-1)] seguido de
54 // ["1" + x para x en reverso(Gray(n-1))],
55 // partiendo de Gray(0) = [""]; produce la misma secuencia
56 // que
57 // la formula i ^ (i >> 1) pero construida por reflexion.
58 //
59 // 3. Código Gray para bases distintas de 2: existen
60 // generalizaciones a codigos Gray de base n (no solo
61 // binaria),
62 // utiles en problemas de permutaciones o combinatoria con
63 // mas de dos estados por posicion, aunque son mucho menos
64 // comunes en competencia.
65 //
66 // 4. Verificacion de la propiedad: para confirmar que dos
67 // codigos
68 // Gray consecutivos difieren en un solo bit, verificar
69 // que
70 // __builtin_popcountll(grayDeIndice(i) ^ grayDeIndice(i
71 // +1)) == 1
72 // para todo i en el rango deseado.
73 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> secuencia = CodigoGray::generarSecuencia(3);
2 // secuencia = {0, 1, 3, 2, 6, 7, 5, 4}
3 // en binario: 000, 001, 011, 010, 110, 111, 101, 100
4 // cada par consecutivo difiere en exactamente un bit
5
6 long long indiceRecuperado = CodigoGray::indiceDeGray(6);
7 // indiceRecuperado = 4 (grayDeIndice(4) = 4 ^ 2 = 6, se
8 // confirma la inversa)

```

Regla práctica: Considera Código Gray cuando un problema pida explícitamente enumerar subconjuntos o configuraciones binarias minimizando el número de "cambios" entre pasos consecutivos; para la gran mayoría de problemas de Bitmask DP (donde el orden de enumeración de las máscaras no importa), la enumeración estándar $0, 1, 2, \dots, 2^n - 1$ es suficiente y no requiere esta técnica.

60.23 Conteo de Pares con XOR/AND/OR Dado

Idea: Dado un arreglo de n números, este patrón cuenta cuántos pares (i, j) con $i < j$ satisfacen una condición sobre $a_i \oplus a_j$, $a_i \& a_j$, o $a_i | a_j$ (por ejemplo, "cuántos pares tienen XOR igual a k ", o "cuántos pares tienen AND

distinto de 0"). La estrategia general evita el enfoque ingenuo $O(n^2)$ de probar todos los pares, procesando los números *bit a bit* o usando una tabla de frecuencias: para XOR con un valor objetivo fijo k , basta notar que $a_i \oplus a_j = k$ equivale a $a_j = a_i \oplus k$, por lo que se puede usar una tabla hash de frecuencias para buscar, por cada a_i , cuántas ocurrencias hay de $a_i \oplus k$ ya procesadas.

¿Por qué usarlo? Esta técnica se distingue del Trie de Bits en el objetivo: el trie resuelve "encontrar el elemento que *maximiza* el XOR con uno dado", mientras que este patrón resuelve "contar *cuántos* pares cumplen una condición de igualdad o desigualdad" — un problema de conteo, no de optimización. Para XOR con un valor objetivo exacto, una tabla de frecuencias ($O(n)$) es más simple y rápida que un trie; el trie solo se vuelve necesario cuando la condición involucra un rango de valores (por ejemplo, "XOR menor a k ") en vez de una igualdad exacta.

Casos de uso:

- Contar pares (i, j) tales que $a_i \oplus a_j = k$ para un k dado, usando tabla de frecuencias en $O(n)$.
- Contar pares tales que $a_i \& a_j = 0$ (pares "disjuntos" en representación de bits), frecuentemente combinado con SOS DP para contar, por cada máscara, cuántos elementos son submáscara de su complemento.
- Contar pares tales que $a_i | a_j$ tiene una propiedad específica (por ejemplo, es igual a un valor máximo dado), también resoluble con SOS DP sobre superconjuntos.
- Sumar $a_i \oplus a_j$ sobre *todos* los pares del arreglo (no solo contarlos), procesando bit por bit: para cada bit, contar cuántos elementos lo tienen encendido/apagado y combinar con la fórmula de conteo de pares.

Complejidad: $O(n)$ para el conteo de pares con XOR igual a un valor exacto k (una pasada con tabla de frecuencias); $O(n \cdot b)$ para variantes que procesan bit por bit (como la suma de XOR de todos los pares, o conteo con AND/OR usando SOS DP), con b el número de bits considerados.

```

1 struct ConteoDeParesConXOR {
2
3     // READ: cuenta la cantidad de pares (i, j) con i < j tales
4     // que
5     // arr[i] ^ arr[j] == k, usando una tabla de frecuencias
6     static long long contarParesXOR(const vector<long long>& arr,
7     long long k) {
8         unordered_map<long long, long long> frecuencia;
9         long long cuenta = 0;
10        for (long long v : arr) {
11            long long complemento = v ^ k; // buscar si v ^ k ya
12            // aparecio antes
13            auto it = frecuencia.find(complemento);
14            if (it != frecuencia.end()) cuenta += it->second;
15            frecuencia[v]++;
16        }
17        return cuenta;
18    } // O(n)
19
20    // READ: calcula la suma de arr[i] ^ arr[j] sobre TODOS los
21    // pares
22    // (i, j) con i < j, procesando bit por bit en vez de O(n^2)
23    // pares
24    static long long sumaXORDeTodosLosPares(const vector<long
25    long>& arr, int bits = 30) {
26        int n = (int)arr.size();
27        long long sumaTotal = 0;

```

```

22        for (int bit = 0; bit < bits; bit++) {
23            long long conBitEncendido = 0;
24            for (long long v : arr) {
25                if ((v >> bit) & 1) conBitEncendido++;
26            }
27            long long conBitApagado = n - conBitEncendido;
28            // cada par (uno con el bit encendido, otro apagado)
29            // aporta
30            // 2^bit al XOR total; hay conBitEncendido *
31            // conBitApagado
32            // pares de ese tipo
33            sumaTotal += (1LL << bit) * conBitEncendido *
34            conBitApagado;
35        }
36        return sumaTotal;
37    } // O(n * bits)
38
39    //
40    // -----
41    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
42    // -----
43
44    // 1. Conteo de pares con AND == 0: se puede resolver con SOS
45    // DP
46    // (superset sum): para cada elemento arr[i], se necesita
47    // saber cuantos elementos arr[j] son "disjuntos" con el (
48    // es
49    // decir, arr[j] es submascara del complemento de arr[i]);
50    // precalcular con SOS DP, para cada mascara m, cuantos
51    // elementos del arreglo son submascara de m, y consultar
52    // con
53    // m = complemento de arr[i].
54    //
55    // 2. Conteo de pares con OR igual a un valor especifico:
56    // similar
57    // al caso AND, pero usando "superset sum" (suma sobre
58    // superconjuntos) en vez de "subset sum", ya que
59    // a | b == v implica que tanto a como b son submascaras
60    // de v.
61    //
62    // 3. Por que sumaXORDeTodosLosPares evita O(n^2): el XOR de
63    // dos
64    // numeros tiene el bit k encendido si y solo si difieren
65    // en
66    // ese bit; contando cuantos elementos tienen el bit k
67    // encendido/apagado, el numero de pares que difieren en
68    // ese
69    // bit es simplemente el producto de esos dos conteos, sin
70    // necesidad de examinar cada par explicitamente.
71    //
72    // 4. Generalizacion a suma de AND/OR de todos los pares: la
73    // misma tecnica de "contar por bit" aplica: para AND, el
74    // bit
75    // k contribuye 2^k * C(conBitEncendido, 2) pares (ambos
76    // deben
77    // tener el bit encendido); para OR, contribuye
78    // 2^k * (C(n, 2) - C(conBitApagado, 2)) (todos los pares
79    // excepto los que tienen ambos apagados).
80    //
81    };

```

Ejemplo de funcionamiento:

```

1 vector<long long> arr = {3, 5, 6, 1, 2};
2
3 long long paresConXor5 = ConteoDeParesConXOR::contarParesXOR(
4     arr, 5);
5 // pares (i, j) con arr[i] ^ arr[j] == 5:
6 // 3^6=5, 1^... revisar todos; se debe correr el codigo para
7 // confirmar el conteo exacto antes de darlo por verificado.
8
9 long long sumaTotal = ConteoDeParesConXOR::
10 sumaXORDeTodosLosPares(arr);
11 // suma de arr[i] ^ arr[j] para todos los C(5,2)=10 pares
12 // distintos;
13 // se debe correr el codigo para confirmar el valor exacto.

```

Regla práctica: Para "cuántos pares tienen XOR/suma/diferencia igual a k " usa tabla de frecuencias en $O(n)$; para "cuántos pares tienen AND/OR igual a (o relacionado con) un valor" apóyate en SOS DP sobre subconjuntos o superconjuntos según corresponda; y para "suma total de XOR/AND/OR sobre todos los pares" (sin filtrar por un valor específico), procesa bit por bit contando encendidos/apagados en vez de enumerar pares.

60.24 Conteo de Bits Encendidos en un Rango $[0, N]$

Idea: Este patrón calcula, sin iterar número por número, la suma total de bits encendidos (`popcount`) de todos los enteros en el rango $[0, N]$, o específicamente cuántos números en ese rango tienen el bit k encendido. La clave es notar que, para un bit fijo k , el patrón de "encendido/apagado" se repite de forma periódica: el bit k está apagado durante 2^k números consecutivos, luego encendido durante los siguientes 2^k , y ese ciclo de longitud 2^{k+1} se repite exactamente a lo largo de toda la recta numérica. Contar cuántos números en $[0, N]$ tienen el bit k encendido se reduce entonces a contar ciclos completos y manejar el resto parcial, sin necesidad de examinar cada número individualmente.

¿Por qué usarlo? Sumar `popcounts` número por número en $[0, N]$ cuesta $O(N \log N)$ (o $O(N)$ con `__builtin_popcount` por número), inviable si N es del orden de 10^9 o más. Esta técnica reduce el costo a $O(\log N)$ (una pasada por cada uno de los $\log N$ bits posibles), aprovechando la periodicidad de cada bit en vez de la enumeración directa — es, en espíritu, el mismo tipo de razonamiento "bit por bit" que Conteo de Pares con XOR/AND/OR Dado aplica para evitar $O(n^2)$, pero aquí sobre un rango continuo de enteros en vez de un arreglo dado.

Casos de uso:

- Calcular $\sum_{i=0}^N \text{popcount}(i)$ para N hasta 10^{18} , común en problemas de conteo combinatorio sobre representaciones binarias.
- Contar cuántos números en $[0, N]$ tienen exactamente c bits encendidos (combinando esta técnica con combinatoria: elegir qué c posiciones de bit están encendidas sujeto al límite N , tipo *digit DP*).
- Precalcular, como paso base, cuántos números en un rango activan cada bit, antes de aplicar XOR/suma sobre ese rango en problemas más complejos.
- Verificar por fuerza bruta (para N pequeño) contra la fórmula cerrada, como *sanity check* antes de aplicar la técnica a N grande en una competencia.

Complejidad: $O(\log N)$ para calcular la suma total de bits encendidos en $[0, N]$: se itera una vez por cada uno de los $O(\log N)$ bits posibles, y cada iteración calcula el conteo de ese bit en $O(1)$ mediante aritmética de división entera, sin bucles internos adicionales.

```

1 struct ConteoBitsEnRango {
2
3     // READ: cuenta cuantos numeros en [0, N] (inclusive) tienen
4     // el bit "bit" encendido, usando la periodicidad de ese bit
5     static long long contarBitEnRango(long long N, int bit) {
6         long long cicloCompleto = 1LL << (bit + 1); // longitud del
7         // ciclo apagado+encendido
8         long long mitadCiclo = 1LL << bit;           // cuantos
9         // numeros seguidos tienen el bit encendido
10
11         long long totalNumeros = N + 1; // cantidad de numeros en
12         // [0, N]
13         long long ciclosCompleto = totalNumeros / cicloCompleto;
14         long long resto = totalNumeros % cicloCompleto;
15
16         long long conteo = ciclosCompleto * mitadCiclo;
17         conteo += max(0LL, resto - mitadCiclo); // parte encendida
18         // del ciclo parcial
19         return conteo;
20     }
21 }

```

```

16 } // O(1)
17
18 // READ: retorna la suma de popcount(i) para todo i en [0, N]
19 // (inclusive), sumando el conteo de cada bit por separado
20 static long long sumaPopcountEnRango(long long N, int bitsMax
21 = 60) {
22     long long sumaTotal = 0;
23     for (int bit = 0; bit < bitsMax; bit++) {
24         sumaTotal += contarBitEnRango(N, bit);
25     }
26     return sumaTotal;
27 } // O(log N)
28
29 //
30 // -----
31 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
32 // -----
33
34 // 1. Por que el patron del bit k tiene periodo 2^(k+1): en
35 // binario, el bit k cambia de valor cada vez que el
36 // contador
37 // cruza un multiplo de 2^k; permanece apagado durante 2^k
38 // numeros consecutivos y encendido durante los siguientes
39 // 2^k, dando un ciclo completo de longitud 2^(k+1).
40 //
41 // 2. Conteo en un rango [L, R] en vez de [0, N]: se usa la
42 // tecnica de "resta de prefijos", igual que con Prefix
43 // Sums:
44 // contarBitEnRango(R, bit) - contarBitEnRango(L - 1, bit),
45 // ya que el conteo en [0, N] es una funcion de prefijo
46 // valida.
47 //
48 // 3. Contar numeros con EXACTAMENTE c bits encendidos en [0,
49 // N]:
50 // requiere una tecnica distinta, tipicamente Digit DP,
51 // procesando los bits de N de mas a menos significativo y
52 // llevando como estado cuantos bits encendidos se han
53 // colocado hasta el momento, junto con si aun se esta
54 // "pegado" al limite superior N.
55 //
56 // 4. Verificacion por fuerza bruta: para N pequeno (< 10^6),
57 // comparar sumaPopcountEnRango(N) contra un bucle directo
58 // "for i in [0,N]: total += __builtin_popcount(i)" es
59 // una
60 // forma rapida de validar la implementacion antes de
61 // confiar
62 // en ella para N grande.
63 };

```

Ejemplo de funcionamiento:

```

1 long long N = 7; // rango [0, 7]:
2     000,001,010,011,100,101,110,111
3
4 long long bit0 = ConteoBitsEnRango::contarBitEnRango(N, 0);
5 // bit0 = 4 (numeros con bit 0 encendido: 1,3,5,7)
6
7 long long bit1 = ConteoBitsEnRango::contarBitEnRango(N, 1);
8 // bit1 = 4 (numeros con bit 1 encendido: 2,3,6,7)
9
10 long long bit2 = ConteoBitsEnRango::contarBitEnRango(N, 2);
11 // bit2 = 4 (numeros con bit 2 encendido: 4,5,6,7)
12
13 long long sumaTotal = ConteoBitsEnRango::sumaPopcountEnRango(N,
14 3);
15 // sumaTotal = 4 + 4 + 4 = 12
16 // verificacion directa: popcount(0..7) = 0+1+1+2+1+2+2+3 = 12
17 // (coincide)

```

Regla práctica: Usa esta técnica en cuanto un problema pida información agregada sobre bits (suma de `popcounts`, conteo de un bit específico) sobre un rango $[0, N]$ o $[L, R]$ con N demasiado grande para iterar directamente (típicamente $N > 10^7$); para rangos pequeños, un bucle directo con `__builtin_popcount` es más simple y suficientemente rápido.

60.25 Convolución XOR/AND/OR (Walsh-Hadamard Transform)

Idea: Dado dos arreglos a y b indexados por máscaras de n bits, la convolución XOR se define como $c[k] = \sum_{i \oplus j = k} a[i] \cdot b[j]$ (análogamente para AND y OR, cambiando la condición sobre i, j, k). Calcular esto directamente cuesta $O(4^n)$ (para cada par i, j , acumular en

$k = i \oplus j$). La Transformada de Walsh-Hadamard (WHT) resuelve esto de forma análoga a como la FFT acelera la convolución estándar: se aplica una transformada invertible a a y b por separado ($O(n \cdot 2^n)$ cada una), se multiplican los resultados *punto a punto* ($O(2^n)$), y se aplica la transformada inversa al producto ($O(n \cdot 2^n)$), obteniendo c completo en $O(n \cdot 2^n)$ en total. Para AND/OR, el mismo principio se logra con la Transformada Zeta/Möbius sobre subconjuntos (el mismo patrón de propagación bit a bit que SOS DP), en vez de la transformada de Hadamard específica de XOR.

¿Por qué usarlo? Es la generalización natural de SOS DP: SOS DP resuelve $g[m] = \sum_{s \subseteq m} f[s]$ (una transformada Zeta simple), mientras que esta técnica resuelve el problema más general de *combinar dos arreglos distintos* bajo XOR/AND/OR, no solo agregar un arreglo consigo mismo. Es una herramienta avanzada (nivel Div1/regional) que rara vez se necesita completa desde cero en competencia, pero reconocerla evita intentar una convolución XOR/AND/OR con fuerza bruta $O(4^n)$ cuando existe una solución $O(n \cdot 2^n)$.

Casos de uso:

- Calcular, para cada máscara k , el número de pares (i, j) con $a_i \oplus a_j = k$ para arreglos grandes, cuando el enfoque de tabla de frecuencias (útil solo para un k fijo) no basta porque se necesitan *todos* los valores de k a la vez.
- Problemas de conteo de subconjuntos con una operación de "combinar" dos distribuciones bajo XOR (por ejemplo, combinar resultados de dos mitades de un problema dividido con Meet in the Middle, cuando la combinación es sobre máscaras).
- Convolución AND/OR para problemas de "cuántas formas hay de elegir un elemento de A y uno de B tal que su AND/OR sea exactamente k ", usando Transformada Zeta/Möbius en vez de Hadamard.
- Como herramienta dentro de programación dinámica sobre subconjuntos donde la transición natural es una convolución bajo alguna operación bitwise, en vez de una suma o producto estándar.

Complejidad: $O(n \cdot 2^n)$ para transformar (o destransformar) un arreglo de tamaño 2^n , análogo en estructura al $O(2^n \cdot n)$ de SOS DP: se itera sobre los n bits, y en cada uno se recorren las 2^n posiciones actualizando en $O(1)$. La convolución completa (transformar a , transformar b , multiplicar punto a punto, transformar inverso) cuesta $O(n \cdot 2^n)$ en total, dominado por las transformadas.

```

1 struct ConvolucionXOR {
2
3     // WRITE: aplica la Transformada de Walsh-Hadamard en el
4     // (in-place) sobre el arreglo, necesaria antes de
5     // punto a punto para convolucion XOR
6     static void transformarXOR(vector<long long>& a) {
7         int n = (int)a.size();
8         for (int len = 1; len < n; len <= 1) {
9             for (int i = 0; i < n; i += (len <= 1)) {
10                 for (int j = i; j < i + len; j++) {
11                     long long u = a[j], v = a[j + len];
12                     a[j] = u + v; // combinar par (i, i+len) sin
13                     // diferencia de signo
14                     a[j + len] = u - v; // la resta codifica la
15                     // estructura XOR
16                 }
17             }
18         }
19     }
20 }

```

```

15 }
16 }
17 } // O(n log n), con n = 2^bits
18
19 // WRITE: aplica la transformada INVERSA de Walsh-Hadamard,
20 // deshaciendo transformarXOR y dividiendo por el tamaño del
21 // arreglo para recuperar los valores originales
22 static void transformarXORInversa(vector<long long>& a) {
23     int n = (int)a.size();
24     transformarXOR(a); // la WHT es su propia inversa salvo el
25     // factor de escala
26     for (long long& v : a) v /= n;
27 } // O(n log n)
28
29 // READ: calcula la convolucion XOR completa de a y b:
30 // retorna c
31 // tal que c[k] = suma de a[i]*b[j] para todo i^j == k
32 static vector<long long> convolucionXOR(vector<long long> a,
33 vector<long long> b) {
34     int n = (int)a.size(); // debe ser potencia de 2, igual
35     // para a y b
36     transformarXOR(a);
37     transformarXOR(b);
38     vector<long long> c(n);
39     for (int i = 0; i < n; i++) c[i] = a[i] * b[i]; //
40     // multiplicacion punto a punto
41     transformarXORInversa(c);
42     return c;
43 } // O(n log n)
44
45 //
46
47 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
48 //
49
50 // 1. Transformada Zeta para convolucion OR (superset/subset):
51
52 // para convolucion OR, la transformada NO usa resta sino
53 // suma en una direccion: a[j+len] += a[j] (equivalente a
54 // SOS
55 // DP: suma sobre subconjuntos); la inversa resta en vez
56 // de
57 // dividir: a[j+len] -= a[j]. Para convolucion AND, se
58 // invierte
59 // el rol de las mitades (se propaga desde el subconjunto
60 // mas
61 // grande hacia los mas chicos, "superset sum").
62 //
63 // 2. Por que no se necesita numeros complejos como en FFT: a
64 // diferencia de la convolucion estandar (que usa raices
65 // de la
66 // unidad complejas), la WHT para XOR usa solo +1 y -1
67 // como
68 // "raices", ya que XOR sobre GF(2) tiene una estructura
69 // mas
70 // simple; esto permite implementarla enteramente con
71 // aritmetica
72 // entera, sin perdida de precision de punto flotante.
73 //
74 // 3. Overflow en la multiplicacion punto a punto: los
75 // valores
76 // transformados pueden crecer hasta un factor de n
77 // respecto a
78 // los originales (por las sumas repetidas); si a y b
79 // contienen
80 // valores grandes, verificar overflow antes de
81 // multiplicar
82 // punto a punto (ver Overflow y Wraparound), o aplicar un
83 // modulo si el problema lo permite.
84 //
85 // 4. Relacion con SOS DP: la transformada Zeta usada para
86 // convolucion OR es EXACTAMENTE el algoritmo de SOS DP
87 // visto
88 // antes, aplicado como paso intermedio de una convolucion
89 // en
90 // vez de como resultado final; reconocer esta
91 // equivalencia
92 // ayuda a recordar la implementacion sin memorizarla por
93 // separado.
94 }

```

Ejemplo de funcionamiento:

```

1 // a[i] = cantidad de veces que aparece i en el primer conjunto
2 // b[j] = cantidad de veces que aparece j en el segundo
3 // conjunto
4 vector<long long> a = {1, 2, 0, 1}; // universo de 2 bits
5 // (0..3)
6 vector<long long> b = {0, 1, 1, 0};
7
8 vector<long long> c = ConvolucionXOR::convolucionXOR(a, b);
9 // c[k] = suma de a[i]*b[j] para todo i^j == k

```



```

8 // Nota: el valor exacto de cada c[k] depende de la
  transformada
9 // paso a paso; se debe correr el código para confirmar los
  valores
10 // antes de darlos por verificados.

```

Nota: Esta técnica es de las más avanzadas de esta parte — si tu guía está orientada a niveles Div2 o fundamentos, puede quedar como referencia de "saber que existe" más que como algo a memorizar en detalle; su requisito conceptual directo es SOS DP, así que conviene tenerla sólida antes de abordar esta subsección.

Regla práctica: Usa convolución XOR/AND/OR (vía WHT o Transformada Zeta/Möbius) cuando necesites combinar *dos* arreglos distintos bajo una operación bitwise para *todos* los valores de salida simultáneamente, en $O(n \cdot 2^n)$; si el problema es más simple —agregar un solo arreglo sobre sus propios subconjuntos— con SOS DP alcanza, sin necesidad de la maquinaria completa de la transformada.

60.26 Meet in the Middle con Máscaras de Bits

Idea: Meet in the Middle divide un conjunto de n elementos en dos mitades de tamaño $n/2$, enumera todos los $2^{n/2}$ subconjuntos de cada mitad por separado (representados como máscaras de bits), y combina los resultados de ambas mitades para responder la pregunta sobre el conjunto completo. La observación clave es que $2^{n/2} + 2^{n/2} = 2^{n/2+1}$ es exponencialmente más pequeño que 2^n cuando n crece: para $n = 40$, enumerar 2^{40} subconjuntos es inviable, pero enumerar 2^{20} subconjuntos por mitad (unos 10^6) es trivial. La combinación de resultados típicamente requiere ordenar una mitad y usar búsqueda binaria (o una tabla hash) sobre ella al recorrer la otra, en vez de comparar cada par de subconjuntos directamente.

¿Por qué usarlo? Es la técnica que rescata problemas de Bitmask DP cuando n excede el límite práctico de 2^n (usualmente 20-24): al dividir en dos mitades de $n/2 \approx 20$ cada una, se extiende el rango manejable hasta $n \approx 40-45$. Se distingue de Bitmask DP clásico en que aquí *no* se construye una tabla de estados sobre todo el conjunto —se enumeran exhaustivamente subconjuntos de cada mitad de forma independiente, y la dificultad se traslada a combinar eficientemente ambas listas (usualmente con ordenamiento + búsqueda binaria, conectando esta técnica directamente con la Parte de Búsqueda Binaria).

Casos de uso:

- Subset Sum con n hasta 40: dividir en dos mitades, enumerar todas las sumas de subconjuntos de cada mitad, ordenar una y usar búsqueda binaria sobre la otra para verificar si un valor objetivo es alcanzable.
- Maximizar el valor de una mochila (*knapsack*) con n hasta 40 y capacidad grande, combinando subconjuntos de cada mitad ordenados por peso con un "prefijo máximo" de valor.
- Contar cuántos subconjuntos de un arreglo de hasta 40 elementos satisfacen una condición de suma o XOR, combinando conteos de ambas mitades con dos punteros o búsqueda binaria.

- Problemas donde el espacio de estados de Bitmask DP (2^n) es demasiado grande para caber en memoria, pero se puede reformular como una combinación de dos mitades independientes.

Complejidad: $O(2^{n/2} \cdot \log(2^{n/2})) = O(2^{n/2} \cdot n)$ típicamente: generar las $2^{n/2}$ combinaciones de cada mitad cuesta $O(2^{n/2} \cdot n/2)$, y ordenar una de las listas para combinar cuesta $O(2^{n/2} \log(2^{n/2}))$; la combinación final (recorrer una lista y buscar en la otra) agrega otro factor $O(2^{n/2} \log(2^{n/2}))$. Esto reemplaza el $O(2^n)$ o $O(2^n \cdot n)$ inviable de Bitmask DP directo para n grande.

```

1 struct MeetInTheMiddle {
2     vector<long long> elementos;
3     int n;
4
5     // CREATE: guarda el arreglo completo de elementos a dividir
  en
6     // dos mitades
7     MeetInTheMiddle(vector<long long> arr) : elementos(arr), n((
  int)arr.size()) {} // O(n)
8
9     // READ: genera todas las sumas de subconjuntos (incluyendo
  el
10    // vacío, suma 0) de los elementos en el rango [inicio, fin)
11    vector<long long> generarSumas(int inicio, int fin) {
12        vector<long long> resultado;
13        int tam = fin - inicio;
14        for (int mask = 0; mask < (1 << tam); mask++) {
15            long long suma = 0;
16            for (int bit = 0; bit < tam; bit++) {
17                if (mask & (1 << bit)) suma += elementos[inicio + bit];
18            }
19            resultado.push_back(suma);
20        }
21        return resultado;
22    } // O(2^tam * tam)
23
24    // READ: verifica si existe algún subconjunto del arreglo
  COMPLETO
25    // cuya suma sea exactamente "objetivo", dividiendo el
  arreglo en
26    // dos mitades y combinando con búsqueda binaria
27    bool existeSubconjuntoConSuma(long long objetivo) {
28        int mitad = n / 2;
29        vector<long long> sumasIzquierda = generarSumas(0, mitad);
30        vector<long long> sumasDerecha = generarSumas(mitad, n);
31
32        sort(sumasDerecha.begin(), sumasDerecha.end());
33
34        for (long long sumaI : sumasIzquierda) {
35            long long complemento = objetivo - sumaI;
36            if (binary_search(sumasDerecha.begin(), sumasDerecha.end
  (), complemento)) {
37                return true;
38            }
39        }
40        return false;
41    } // O(2^(n/2) * (n/2) + 2^(n/2) * log(2^(n/2)))
42
43    //
44    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
45    //
46    // 1. Contar subconjuntos en vez de verificar existencia: en
  vez
47    // de binary_search (que solo confirma presencia), usar
48    // equal_range o lower_bound/upper_bound sobre
  sumasDerecha
49    // ordenada para contar CUANTAS veces aparece el
  complemento,
50    // acumulando el total sobre todas las sumaI de la
  izquierda.
51    //
52    // 2. Knapsack con Meet in the Middle: generar pares
  (peso, valor)
53    // (peso, valor) para cada subconjunto de cada mitad;
  ordenar
54    // una mitad por peso y precalcular el "máximo valor con
  peso
55    // <= w" como un prefijo máximo; para cada subconjunto de
  la
56    // otra mitad, buscar con búsqueda binaria el mayor peso
  permitido
57    // restante y combinar con ese prefijo máximo.
58    //
59    // 3. Por que dividir por la mitad es óptimo: si se dividiera
  en
60    // proporciones desiguales (por ejemplo, n/3 y 2n/3), el

```

```

61 lado
62 // mas grande seguiria dominando la complejidad total
63 // (2^(2n/3) sigue siendo exponencialmente peor que 2^(n
64 // /2));
65 // dividir exactamente a la mitad minimiza el maximo de
66 // las
67 // dos mitades, que es lo que determina el costo total.
68 //
69 // 4. Generalizacion a XOR en vez de suma: el mismo esqueleto
70 // aplica reemplazando la suma de subconjunto por XOR de
71 // subconjunto; para buscar el complemento exacto, sigue
72 // funcionando busqueda binaria (o tabla hash) sobre
73 // valores
74 // ordenados, ya que la condicion sigue siendo una
75 // igualdad.
76 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> arr = {3, 7, 1, 9, 4, 12, 6, 2}; // n = 8 (
2 // ejemplo pequeno)
3 MeetInTheMiddle mitm(arr);
4
5 bool existe23 = mitm.existeSubconjuntoConSuma(23);
6 // Nota: el valor exacto depende de verificar todas las
7 // combinaciones
8 // de ambas mitades; se debe correr el codigo para confirmarlo
9 // antes
10 // de darlo por verificado (candidato esperado: true, ya que
11 // hay
12 // varias combinaciones plausibles que suman 23 en un arreglo
13 // con
14 // estos valores).

```

Regla práctica: Reconoce Meet in the Middle cuando n esté en el rango de ~ 30 a ~ 45 (demasiado grande para Bitmask DP directo con 2^n , pero divisible en dos mitades de $2^{n/2}$ manejable) y el problema tenga estructura de "elegir un subconjunto que satisfaga una condición de suma/XOR/valor"; para $n \leq 20-24$, Bitmask DP directo suele ser más simple y no requiere la complejidad adicional de combinar dos mitades.

60.27 Transformada de Möbius/Zeta sobre Subconjuntos

Idea: La Transformada Zeta sobre subconjuntos transforma un arreglo f indexado por máscaras en $\hat{f}[m] = \sum_{s \subseteq m} f[s]$ — exactamente el algoritmo de SOS DP visto antes, pero presentado aquí en el lenguaje general de "transformadas" sobre el retículo de subconjuntos, en analogía directa con la Transformada de Fourier. La Transformada de Möbius es su **inversa**: dado $\hat{f}$, recupera f mediante $f[m] = \sum_{s \subseteq m} (-1)^{|m|-|s|} \hat{f}[s]$, que en la práctica se calcula con el mismo bucle bit-por-bit de SOS DP pero restando en vez de sumar en cada paso. Juntas, estas dos transformadas son el mecanismo que permite la Convolución AND/OR vista en la subsección anterior: transformar ambos arreglos con Zeta, multiplicar punto a punto, y aplicar Möbius para volver al dominio original.

¿Por qué usarlo? Esta subsección formaliza una idea que ya usaste dos veces sin nombrarla explícitamente como "transformada": SOS DP calcula la Zeta, y la Convolución AND/OR usa el par Zeta-Möbius como sus pasos de ida y vuelta. Nombrarla aparte tiene valor principalmente cuando necesitas la **inversa** de forma independiente —por ejemplo, si conoces $\hat{f}$ (una suma sobre subconjuntos ya agregada) y necesitas recuperar los valores individuales f originales, en vez de solo ir "hacia adelante" como hace SOS DP.

Casos de uso:

- Recuperar los valores individuales $f[m]$ a partir de una suma sobre subconjuntos $\hat{f}[m]$ ya conocida (el

paso inverso que SOS DP no necesita, pero que Convolución AND/OR sí requiere al final).

- Aplicar principio de inclusión-exclusión sobre el retículo de subconjuntos de forma sistemática, cuando la fórmula de inclusión-exclusión directa tiene 2^n términos por calcular ingenuamente.
- Como el paso final de Convolución AND/OR (Transformada Zeta para ir al dominio transformado, Möbius para volver), completando el patrón "transformar, multiplicar punto a punto, destransformar" ya visto con XOR/Walsh-Hadamard.
- Verificar la consistencia de un algoritmo de SOS DP: aplicar Möbius sobre el resultado de Zeta debe devolver exactamente el arreglo original, útil como *sanity check*.

Complejidad: $O(n \cdot 2^n)$ tanto para la Transformada Zeta como para la Möbius — idéntica a SOS DP, ya que la Möbius es literalmente el mismo bucle con el signo de la actualización invertido (resta en vez de suma); aplicar ambas en secuencia (ida y vuelta) sigue siendo $O(n \cdot 2^n)$, no se duplica el orden de crecimiento.

```

1 struct TransformadaMoebiusZeta {
2     int n;
3
4     // CREATE: guarda el numero de bits del universo de mascaras
5     TransformadaMoebiusZeta(int numBits) : n(numBits) {} // O(1)
6
7     // WRITE: aplica la Transformada Zeta en el lugar: convierte
8     f[m]
9     // (valor base) en la suma sobre todas las submascaras de m
10    // (identica al algoritmo de SOS DP)
11    void zeta(vector<long long>& f) {
12        int total = 1 << n;
13        for (int bit = 0; bit < n; bit++) {
14            for (int mask = 0; mask < total; mask++) {
15                if (mask & (1 << bit)) {
16                    f[mask] += f[mask ^ (1 << bit)];
17                }
18            }
19        } // O(n * 2^n)
20
21        // WRITE: aplica la Transformada de Moebius en el lugar:
22        // invierte
23        // zeta(), recuperando los valores originales f[m] a partir
24        // de la
25        // suma sobre subconjuntos
26        void moebius(vector<long long>& f) {
27            int total = 1 << n;
28            for (int bit = 0; bit < n; bit++) {
29                for (int mask = 0; mask < total; mask++) {
30                    if (mask & (1 << bit)) {
31                        f[mask] -= f[mask ^ (1 << bit)]; // resta en vez de
32                        // suma: deshace zeta
33                    }
34                }
35            } // O(n * 2^n)
36
37            //
38            // -----
39            // MODIFICACIONES Y COMENTARIOS CON LOS USOS
40            // -----
41
42            // 1. Por que restar deshace la suma: zeta() propaga cada
43            // valor
44            // original hacia todas las mascaras que lo contienen como
45            // submascara, bit a bit; moebius() recorre los mismos
46            // bits en
47            // el mismo orden, y en cada paso resta exactamente lo que
48            // zeta() habia sumado en ese paso, deshaciendo la
49            // transformacion bit por bit hasta recuperar los valores
50            // base.
51            //
52            // 2. Version para superset (Zeta/Moebius sobre
53            // superconjuntos):
54            // analoga a la version "superset sum" mencionada en SOS
55            // DP,

```

```
47 // invirtiendo la condicion del if a "!(mask & (1<<bit))"
48 // y operando sobre mask | (1<<bit) en vez de mask ^ (1<<bit)
49 //
50 // 3. Uso dentro de Convolucion AND/OR: para convolucion OR,
51 // aplicar zeta() a ambos arreglos, multiplicar punto a punto,
52 // y aplicar moebius() al resultado recupera exactamente
53 // c[k] = suma de a[i]*b[j] para todo i/j == k; para AND,
54 // se usa la version "superset" de ambas transformadas.
55 //
56 // 4. Relacion formal con inclusion-exclusion: la formula
57 // f[m] = suma sobre s submascara de m de (-1)^(|m|-|s|) *
58 // fhat[s]
59 // es la version "sobre subconjuntos" del principio de
60 // inclusion-exclusion; el algoritmo bit a bit de moebius
61 // () es simplemente una forma eficiente de evaluar esa suma sin
62 // calcular (-1)^(|m|-|s|) explicitamente para cada par (m, s).
```

Ejemplo de funcionamiento:

```
1 vector<long long> original = {1, 2, 3, 4, 5, 6, 7, 8}; //
  universo de 3 bits
2 vector<long long> f = original; // copia para transformar
3
4 TransformadaMoebiusZeta t(3);
5 t.zeta(f);
6 // f ahora contiene, para cada mascara, la suma sobre sus
  submascaras
7 // (identico al resultado de SOS DP)
8
9 t.moebius(f);
10 // f deberia volver exactamente a "original" tras deshacer zeta
  ()
11 // Nota: verificar en codigo que f == original tras este paso,
  como
12 // sanity check de la implementacion.
```

Regla práctica: Si solo necesitas ir "hacia adelante" (sumar sobre subconjuntos una sola vez), usa SOS DP directamente sin pensar en términos de transformadas; recurre explícitamente a esta subsección cuando necesites la **inversa** (recuperar valores originales desde una agregación) o cuando estés implementando Convolución AND/OR, donde el par Zeta-Möbius es indispensable como los pasos de ida y vuelta.

Parte XI Game Theory

60.28 ¿Qué algoritmo utilizar?

Cuando el problema te pide:	Entonces usa:	Complejidad
Determinar si el primer jugador gana o pierde, con un espacio de estados pequeño y sin formula conocida	Estados Ganadores y Perdedores (Win/Lose States)	$O(V + E)$
Clasificar un juego nuevo antes de elegir tecnica: ¿los movimientos dependen solo de la posicion, o tambien de quien mueve?	Juegos Imparciales vs. Partidistas (clasificacion previa)	sin costo computacional
Confirmar que el juego no tiene dados, cartas ocultas ni informacion privada antes de aplicar cualquier tecnica de esta parte	Informacion Perfecta y sin Azar (checklist previo)	sin costo computacional
Resolver un juego propagando el resultado desde las posiciones terminales hacia atras (marco general antes de Grundy o minimax)	Analisis Retrogrado (Backward Induction)	$O(V + E)$
k montones de fichas, se quita cualquier cantidad de UN monton, pierde quien no puede mover	Nim Clasico (Teorema de Bouton)	$O(k)$
Nim con tope de fichas por turno ("maximo k por turno"), o solo se permite quitar cantidades de un conjunto fijo S	Nim con Movimientos Restringidos (Grundy por monton + XOR)	Build $O(n \cdot S)$ Query $O(1)$
Calcular el "numero equivalente en Nim" de una posicion, necesario antes de combinar el juego con otros	Teorema de Sprague-Grundy (mex de sucesores)	$O(V + E)$
Varios subjuegos independientes jugados a la vez, se elige uno por turno (pilas/tableros separados)	Suma de Juegos Independientes (XOR de Grundy Numbers)	$O(\sum \text{costo}_i) + O(k)$
Calcular Grundy de un juego sin formula conocida, dado explicitamente como grafo de posiciones	Calculo de Grundy sobre un DAG (mex de sucesores)	$O(V + E)$
Nim pero "quien hace el ultimo movimiento PIERDE" (no el que no puede mover)	Misere Nim	$O(k)$
Una sola pila, conjunto de movimientos S dado explicitamente, regla de juego normal	Juego de Sustraccion (Subtraction Game)	$O(n \cdot S)$
Tope k de piedras por turno pero con un conjunto F de cantidades explicitamente prohibidas	Sustraccion con Movimientos Prohibidos (complemento de F)	$O(k) + O(n \cdot S_{\text{ef}})$
Monedas en una escalera, se mueven de un escalon a uno estrictamente inferior, en cualquier cantidad	Staircase Nim (XOR de escalones impares)	$O(n)$
Dos montones, se puede quitar de uno solo O la misma cantidad de ambos a la vez	Wythoff's Game (sucesion de Beatty, razon aurea)	$O(1)$ por consulta
Fichas cara/cruz en fila: volteas una cara a cruz (obligatorio) y opcionalmente volteas una ficha a su izquierda	Turning Games (Coin Turning, XOR por posicion)	$O(n)$
Una ficha se mueve por un grafo/DAG siguiendo aristas, sin reglas con formula conocida; posiblemente varias fichas independientes	Juegos sobre Grafos y DAGs (Grundy generico + XOR)	$O(V + E)$
Arbol de ramas colgando del suelo, se corta una arista y cae todo lo que quedaba colgando de ella	Green Hackenbush (Colon Principle)	$O(V)$
Dos jugadores toman el extremo izquierdo o derecho de un arreglo, cada uno maximiza SU propio puntaje (no hay "ganar/perder" binario)	DP de Intervalos (Optimal Strategy for a Game)	$O(n^2)$, $O(n)$ memoria
Un movimiento elimina parte de una cadena/secuencia y las dos mitades restantes quedan separadas para siempre (no se concatenan)	Suma de Grundy sobre Substrings (particion en subjuegos)	$O(n^3)$
Izquierda y Derecha tienen jugadas DISTINTAS en una misma posicion (Hackenbush azul-rojo, Domineering)	Numeros Surreales (valores $\{G^L \mid G^R\}$, por comparacion)	$O(m_1 \cdot m_2)$
Varios componentes partidistas mezclados (numeros, calientes, difusos) que hay que combinar y clasificar contra 0	Suma de Juegos Partidistas (suma disyunta + leq)	$O(\text{producto de tamanos})$
El estado del juego no encaja en ninguna estructura simple (tablero arbitrario), pero hay muchas transposiciones	Minimax con Memorizacion	$O(E \cdot b)$

61 Fundamentos

61.1 Estados Ganadores y Perdedores (Win/Lose States)

Idea: en un juego de dos jugadores con información perfecta y sin azar, cada posición del juego se puede clasificar como **ganadora** (el jugador que está por mover puede forzar la victoria) o **perdedora** (haga lo que haga, pierde si el rival juega óptimo). La clasificación se define de atrás hacia adelante: una posición **terminal** donde el jugador en turno ya perdió (por ejemplo, no quedan movimientos) es **perdedora**. Una posición no terminal es **ganadora** si existe **al menos un** movimiento hacia una posición perdedora para el rival; es **perdedora** si **todos** los movimientos posibles llevan a posiciones ganadoras para el rival. Esta es exactamente la misma memoización de *Memoización vs. Tabulación* aplicada a un grafo de posiciones en vez de a números.

¿Por qué usarlo? Es la base de toda la parte: antes de aplicar Sprague-Grundy o cualquier fórmula cerrada, hay que entender que un juego se puede resolver por fuerza bruta memoizada siempre que el número de posiciones distintas sea manejable. Sin memoización, evaluar recursivamente si una posición es ganadora reevalúa las mismas subposiciones una y otra vez, con costo exponencial. Con memoización, cada posición se evalúa una sola vez y el costo total es proporcional al número de posiciones por el número de movimientos desde cada una.

Casos de uso:

- Determinar si el primer jugador gana o pierde en un juego de turnos dado un estado inicial concreto.
- Como paso previo obligatorio antes de calcular números de Grundy: primero se identifica la posición terminal perdedora, luego se generaliza.
- Juegos donde el espacio de estados es pequeño (por ejemplo, montones de tamaño acotado) y no hace falta ninguna fórmula cerrada.
- Verificar a mano, con esta memoización, una fórmula sospechada para un juego más grande, igual que en *Fuerza Bruta para Descubrir Patrones*.

Complejidad: $O(V + E)$ donde V es el número de posiciones distintas del juego y E el número total de movimientos entre posiciones, ya que cada posición se evalúa una sola vez y cada movimiento se recorre una sola vez.

```

1 // CREATE/READ: Clasificación de posiciones como ganadoras o
2 // mediante memoización, para un juego de sustracción genérico:
3 // dos
4 // jugadores se turnan para restar un valor de un conjunto S al
5 // montón n;
6 // pierde quien no puede mover (n llega a 0 sin poder restar
7 // nada válido)
8 struct EstadosGanadorPerdedor {
9     vector<int> movimientosValidos; // S: valores que se pueden
10     restar
11     vector<int> memo; // memo[n] = -1 (sin calcular), 0 (
12     perdedora), 1 (ganadora)
13
14 // CREATE: prepara la tabla de memoización con centinela -1
15 EstadosGanadorPerdedor(const vector<int>& S, int nMax) :
16     movimientosValidos(S) {
17     memo.assign(nMax + 1, -1);
18 } // O(nMax)

```

```

13
14 // READ: true si la posición n es GANADORA para el jugador en
15 // turno
16 bool esGanadora(int n) {
17     if (n == 0) return false; // posición terminal: quien mueve
18     aquí ya perdió
19     if (memo[n] != -1) return memo[n];
20     bool ganadora = false;
21     for (int m : movimientosValidos) {
22         if (m <= n && !esGanadora(n - m)) {
23             ganadora = true;
24             break; // basta UN movimiento a una posición perdedora
25             para el rival
26         }
27     }
28     return memo[n] = ganadora;
29 } // O(1) amortizado por estado, O(n * |S|) en total
30
31 //
32 // -----
33 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
34 // -----
35
36 // 1. Version tabulada (bottom-up): recorrer n de 0 a nMax en
37 // orden
38 // creciente, evaluando memo[n] con el mismo criterio, sin
39 // recursión; preferible si nMax es grande y hay riesgo de
40 // desbordar la pila.
41 // 2. Estado compuesto (mas de un número, por ejemplo varios
42 // montones):
43 // usar un mapa o una máscara de bits como clave en vez de
44 // un
45 // arreglo indexado por un solo entero; el resto de la
46 // lógica no
47 // cambia.
48 // 3. Reconstruir la ESTRATEGIA óptima (no solo si se gana):
49 // guardar,
50 // junto con memo[n], el movimiento m que lleva a la
51 // posición
52 // perdedora encontrada, para poder reproducir la jugada
53 // ganadora.
54 // 4. Último en mover PIERDE en vez de ganar (Misere):
55 // cambiar la
56 // condición terminal de "n == 0 es perdedora" a "n == 0
57 // es
58 // ganadora" (el jugador en turno gana porque el rival ya
59 // no puede
60 // mover y esa regla favorece a quien está en turno);
61 // revisar con
62 // cuidado el enunciado exacto de cada problema.
63 // 5. Múltiples jugadores (mas de 2): el análisis binario
64 // ganador/perdedor deja de aplicarse directamente; se
65 // necesita
66 // memoizar el resultado para CADA jugador o usar backward
67 // induction con mas de dos resultados posibles.
68 };

```

Ejemplo de funcionamiento:

```

1 EstadosGanadorPerdedor g({1, 3, 4}, 10);
2
3 bool r1 = g.esGanadora(1);
4 // r1 = true (restar 1 deja n=0, posición perdedora para el
5 // rival)
6
7 bool r2 = g.esGanadora(2);
8 // r2 = false
9 // (desde 2 solo se puede restar 1, dejando n=1, que es
10 // GANADORA para
11 // el rival; no hay otro movimiento válido, así que 2 es
12 // perdedora)
13
14 bool r3 = g.esGanadora(5);
15 // r3 = true (restar 4 deja n=1, que es ganadora para el rival
16 // ... hay
17 // que correrlo para confirmar cual movimiento concreto deja
18 // al rival
19 // en una posición perdedora)

```

Nota: la regla de clasificación (ganadora si **existe** un movimiento a perdedora; perdedora si **todos** los movimientos van a ganadora) es la base de todo análisis de juegos combinatorios de esta parte, incluyendo Sprague-Grundy: un número de Grundy distinto de cero es exactamente la generalización de "posición ganadora", y Grundy igual a cero es exactamente "posición perdedora".

Regla práctica: antes de buscar una fórmula cerrada para un juego nuevo, escribe primero esta

memoización sobre casos pequeños. Si el patrón de ganadora/perdedora no es evidente a simple vista, genera una tabla de varios valores de n (como en *Fuerza Bruta para Descubrir Patrones*) y busca la periodicidad o condición que los distingue antes de intentar demostrar una fórmula general.

61.2 Juegos Imparciales vs. Partidistas

Idea: un juego es **imparcial** (*impartial*) si, en cualquier posición, **ambos jugadores tienen exactamente el mismo conjunto de movimientos disponibles** — el conjunto de jugadas depende solo de la posición, no de quién esté jugando. Nim es el ejemplo canónico: cualquiera de los dos jugadores puede quitar fichas de cualquier montón. Un juego es **partidista** (*partizan*) si los movimientos disponibles dependen de **qué jugador** está en turno — por ejemplo, el ajedrez, donde un jugador solo mueve piezas blancas y el otro solo negras. Esta distinción determina qué herramientas aplican: el teorema de Sprague-Grundy, con sus números de Grundy y la suma por XOR, **solo es válido para juegos imparciales**.

¿Por qué usarlo? Sin identificar primero si un juego es imparcial o partidista, se corre el riesgo de aplicar Grundy donde no corresponde. En un juego partidista no existe un único "número" que resuma una posición, porque el resultado depende de quién mueve; ahí se necesita la teoría más general de **valores de juego** (números surreales, juegos *cold*, *hot*, *fuzzy*), vista en *Números Surreales y Valores de Juego*. Reconocer la categoría del juego en la primera lectura del enunciado ahorra tiempo: evita intentar forzar un XOR de Grundy sobre un juego donde simplemente no aplica.

Casos de uso:

- Clasificar un juego nuevo antes de elegir la técnica de resolución: ¿los movimientos dependen de la posición solamente, o también de qué jugador mueve?
- Nim, juegos de sustracción, y la mayoría de juegos de "tomar de un conjunto compartido" son imparciales.
- Juegos donde cada jugador controla piezas o recursos distintos (como Hackenbush partidista, o ajedrez) son partidistas.
- Decidir si conviene calcular números de Grundy (imparcial) o recurrir a análisis de *minimax* general (partidista), visto en *Minimax con Memoización*.

Complejidad: no aplica una complejidad de algoritmo — es una clasificación conceptual previa al análisis, sin costo computacional propio.

```

1 // CREATE/READ: Verificacion PROGRAMATICA de si un juego, dado
  // por una
2 // funcion de movimientos parametrizada por jugador, es
  // imparcial: se
3 // comprueba que ambos jugadores tengan el MISMO conjunto de
  // movimientos
4 // en un conjunto de posiciones de prueba
5 struct VerificadorImparcial {
6     // READ: dada una funcion movimientos(posicion, jugador) que
      // retorna
7     // las posiciones alcanzables, verifica si coincide para
      // ambos
8     // jugadores en cada posicion de prueba dada
9     static bool esImparcial(
10     const vector<int>& posicionesDePrueba,
```

```

11 function<vector<int>>(int, int)> movimientos
12 ) {
13     for (int pos : posicionesDePrueba) {
14         vector<int> m0 = movimientos(pos, 0);
15         vector<int> m1 = movimientos(pos, 1);
16         sort(m0.begin(), m0.end());
17         sort(m1.begin(), m1.end());
18         if (m0 != m1) return false; // los movimientos difieren
          // segun el jugador
19     }
20     return true; // coincide en todas las posiciones probadas (
      // no es una demostracion)
21 } // O(k * costo de movimientos()), k = numero de posiciones
      // probadas
22
23 //
24 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
25 //
26 // -----
27 // 1. Ejemplo de juego IMPARCIAL: sustraccion de un monton n
      // con
28 // S = {1,2,3}; movimientos(n, jugador) devuelve {n-1,n-2,
      // n-3}
29 // intersectado con validos, IGUAL sin importar el jugador.
30
31 // 2. Ejemplo de juego PARTIDISTA: cada jugador tiene fichas
      // propias en
32 // un tablero; movimientos(pos, 0) solo mueve fichas del
      // jugador 0,
33 // movimientos(pos, 1) solo las del jugador 1 -- ahi
      // esImparcial()
34 // devuelve false de inmediato.
35 // 3. Esta verificacion es SOLO una herramienta de deteccion
      // rapida
36 // sobre casos de prueba, no una demostracion formal; en
      // la
37 // practica, la clasificacion imparcial vs. partidista se
      // hace
38 // leyendo la definicion de movimientos del enunciado
      // directamente.
39 // 4. Juegos hibridos: algunos juegos tienen una fase
      // imparcial y otra
40 // partidista (o se descomponen en subjuegos de distinto
      // tipo); en
41 // ese caso se analiza cada componente por separado antes
      // de
      // combinar resultados.
};
```

Ejemplo de funcionamiento:

```

1 // juego imparcial: sustraccion con S = {1,2,3}, ambos
  // jugadores mueven igual
2 auto movImparcial = [](int n, int jugador) {
3     vector<int> res;
4     for (int m : {1, 2, 3}) if (m <= n) res.push_back(n - m);
5     return res;
6 };
7 bool r1 = VerificadorImparcial::esImparcial({1,2,3,4,5},
      movImparcial);
8 // r1 = true
9
10 // juego partidista simplificado: el jugador 0 solo puede
      // restar valores
11 // pares, el jugador 1 solo impares
12 auto movPartidista = [](int n, int jugador) {
13     vector<int> res;
14     for (int m = 1; m <= n; m++) {
15         if (jugador == 0 && m % 2 == 0) res.push_back(n - m);
16         if (jugador == 1 && m % 2 == 1) res.push_back(n - m);
17     }
18     return res;
19 };
20 bool r2 = VerificadorImparcial::esImparcial({4,5,6},
      movPartidista);
21 // r2 = false (los conjuntos de movimientos difieren segun el
      // jugador)
```

Regla práctica: antes de escribir una sola línea de código para resolver un juego nuevo, pregúntate si el conjunto de movimientos depende de quién está en turno. Si la respuesta es no (imparcial), el camino es *Sprague-Grundy*. Si la respuesta es sí (partidista), Grundy no aplica y hace falta la teoría de valores de juego o un análisis de *minimax* directo sobre el árbol de posiciones.

61.3 Juegos con Información Perfecta y sin Azar

Idea: toda la teoría de esta parte —estados ganador/perdedor, Sprague-Grundy, minimax— exige dos condiciones sobre el juego. **Información perfecta** significa que ambos jugadores conocen en todo momento el estado completo del juego: no hay cartas ocultas, fichas boca abajo, ni información privada de un jugador que el otro desconozca. **Sin azar** significa que no hay dados, barajado aleatorio, ni ningún elemento probabilístico que decida parte de un movimiento: cada jugada produce un resultado **determinista**. Bajo estas dos condiciones, el juego se puede representar como un árbol (o grafo, si hay posiciones repetibles) finito y completamente conocido, y cada posición admite una clasificación fija (ganadora, perdedora, o un valor de Grundy) que no cambia según lo que "podría pasar".

¿Por qué usarlo? Sin información perfecta (por ejemplo, un jugador no sabe qué cartas tiene el otro) o sin ausencia de azar (una tirada de dado decide cuánto se puede mover), no existe una única clasificación de la posición: la mejor jugada depende de probabilidades o de información desconocida, y el análisis correcto pasa a ser de **teoría de juegos con información imperfecta** o de **procesos de decisión de Markov** (esperanza de resultado en vez de ganador/perdedor determinista) —fuera del alcance de las técnicas de esta parte. Reconocer esta condición al leer el enunciado es el primer filtro: si aparecen dados, cartas ocultas, o "elige un jugador al azar", ninguna de las subsecciones de win/lose o Grundy aplica directamente.

Casos de uso:

- Confirmar, antes de intentar cualquier técnica de esta parte, que el juego del enunciado no involucra azar ni información oculta.
- Ajedrez, Nim, y la gran mayoría de juegos de mesa de turnos alternados sin dados son de información perfecta y sin azar.
- Juegos con una tirada de dado que determina cuántas fichas se mueven no entran en esta categoría: requieren valor esperado, no un análisis win/lose puro.
- Juegos de cartas donde un jugador no ve la mano del otro no son de información perfecta, aunque no haya azar en el sentido de dados.

Complejidad: no aplica — es, como *Juegos Imparciales vs. Partidistas*, una clasificación previa sin costo computacional propio, pero que determina si el resto de las técnicas de esta parte son aplicables.

```

1 // CREATE/READ: Lista de verificación programática (checklist)
  para
2 // confirmar que un juego cumple las condiciones necesarias
  antes de
3 // aplicar win/lose states, Sprague-Grundy o minimax
4 struct VerificadorJuegoDeterminista {
5     // READ: retorna una lista de banderas de alerta detectadas
    en la
6     // descripción textual (o en los parámetros) del juego; una
    lista NO
7     // vacía significa que el juego probablemente NO cumple las
    // condiciones (información perfecta y sin azar)
8     static vector<string> detectarAlertas(bool hayDados, bool
    hayCartasOcultas,
9     bool hayEleccionAleatoria, bool hayInformacionPrivada) {
10         vector<string> alertas;
11

```

```

12     if (hayDados) alertas.push_back("Hay dados: el juego tiene
    azar, no es determinista.");
13     if (hayCartasOcultas) alertas.push_back("Hay cartas ocultas
    : no es informacion perfecta.");
14     if (hayEleccionAleatoria) alertas.push_back("Hay eleccion
    aleatoria de jugador o de turno: hay azar.");
15     if (hayInformacionPrivada) alertas.push_back("Hay
    informacion privada de un jugador: no es informacion
    perfecta.");
16     return alertas;
17 } // O(1)
18
19 //
20 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
21 //
22 // -----
23 // 1. Juegos con azar pero con estrategia OPTIMA en
    expectativa: si el
24 // problema realmente exige un dado, se resuelve con DP
    sobre valor
25 // esperado (dp[estado] = promedio ponderado por
    probabilidad de
26 // dp[siguiente estado]), un enfoque distinto y no
    cubierto en el
27 // resto de esta parte.
28 // 2. Información oculta pero con distribución conocida: se
    modela con
29 // teoría de juegos bayesiana; tampoco es el alcance de
    esta guía,
30 // se menciona solo para que se reconozca la categoría del
    problema.
31 // 3. Esta lista de verificación es una ayuda de LECTURA del
    enunciado,
32 // no un algoritmo que se ejecute sobre datos del problema;
    sirve
33 // como recordatorio de que preguntas hacerse antes de
    programar.
};

```

Ejemplo de funcionamiento:

```

1 auto alertasNim = VerificadorJuegoDeterminista::detectarAlertas
    (false, false, false, false);
2 // alertasNim esta vacía: Nim es de información perfecta y sin
    azar,
3 // las técnicas de esta parte aplican directamente
4
5 auto alertasDado = VerificadorJuegoDeterminista::
    detectarAlertas(true, false, false, false);
6 // alertasDado contiene un mensaje: "Hay dados..."
7 // (este juego necesita un enfoque de valor esperado, no win/
    lose directo)

```

Regla práctica: antes de clasificar un juego como imparcial o partidista, confirma primero que cumple información perfecta y ausencia de azar — sin estas dos condiciones, ni Sprague-Grundy ni el análisis win/lose de *Estados Ganadores y Perdedores* son aplicables, sin importar qué tan imparcial parezca el conjunto de movimientos.

61.4 Análisis Retrógrado (Backward Induction)

Idea: el análisis retrógrado es el **método general** detrás de *Estados Ganadores y Perdedores*: en vez de razonar hacia adelante desde la posición inicial (que puede tener ramificaciones enormes), se razona **hacia atrás** desde las posiciones **terminales** (donde el juego ya terminó y el resultado es conocido sin ambigüedad), y se propaga esa información hacia las posiciones que llevan a ellas. Es exactamente memoización o tabulación sobre un grafo de estados, donde la "recurrencia" no es aritmética sino lógica: el resultado de una posición se determina por los resultados de sus posiciones sucesoras, nunca al revés.

¿Por qué usarlo? Intentar razonar hacia adelante ("si yo juego esto, y luego el rival juega aquello, y luego yo...") se vuelve intratable rápidamente porque el árbol de posibilidades crece exponencialmente con el número de turnos. El análisis retrógrado invierte el problema: cada posición terminal tiene un resultado **trivialmente**

conocido, y ese conocimiento se propaga hacia atrás un paso a la vez, de modo que cada posición se resuelve usando únicamente resultados ya calculados de posiciones "más cercanas al final". Es el mismo principio que sustenta toda la parte de Programación Dinámica, aplicado aquí específicamente a juegos.

Casos de uso:

- Fundamento metodológico de *Estados Ganadores y Perdedores*, *Cálculo de Grundy sobre un DAG*, y *Minimax con Memoización*.
- Juegos donde las posiciones forman un grafo acíclico dirigido (DAG): cada movimiento reduce estrictamente algún recurso (fichas restantes, tamaño de la cadena), garantizando que no hay ciclos y el análisis retrógrado termina.
- Resolver juegos de mesa simplificados (tres en línea en tableros pequeños, por ejemplo) enumerando todas las posiciones y propagando el resultado desde los finales de partida.
- Verificar manualmente, para tableros o montones pequeños, el patrón que luego se generaliza a una fórmula cerrada.

Complejidad: $O(V + E)$, igual que *Estados Ganadores y Perdedores*: cada posición (V) se visita una sola vez y cada movimiento (E) se recorre una sola vez, sin importar si se implementa con memoización recursiva o con tabulación siguiendo un orden topológico.

```

1 // CREATE/READ: Analisis retrogrado generico sobre un DAG de
  // posiciones,
2 // dado explícitamente como lista de adyacencia (posicion ->
  // sucesores);
3 // determina el ganador de cada posicion mediante orden
  // topologico inverso
4 struct AnalisisRetrogrado {
5     int n; // numero de posiciones, indexadas 0..n-1
6     vector<vector<int>> sucesores; // sucesores[i] = posiciones
7     // alcanzables desde i
8     vector<int> resultado; // resultado[i] = 0 (perdedora) o 1 (
9     // ganadora); -1 = sin calcular
10
11     // CREATE: reserva las estructuras para n posiciones
12     AnalisisRetrogrado(int numPosiciones) : n(numPosiciones) {
13         sucesores.assign(n, {});
14         resultado.assign(n, -1);
15     } // O(n)
16
17     // WRITE: agrega un movimiento valido de la posicion 'desde'
18     // hacia 'hacia'
19     void agregarMovimiento(int desde, int hacia) {
20         sucesores[desde].push_back(hacia);
21     } // O(1) amortizado
22
23     // READ: clasifica TODAS las posiciones mediante orden
24     // topologico
25     // inverso (procesa primero las posiciones sin sucesores, es
26     // decir,
27     // terminales, y propaga hacia atras); requiere que el grafo
28     // sea un DAG
29     void resolverTodas() {
30         vector<int> gradoSalida(n);
31         for (int i = 0; i < n; i++) gradoSalida[i] = (int)sucesores
32         [i].size();
33
34         // lista de predecesores para propagar en sentido inverso
35         vector<vector<int>> predecesores(n);
36         for (int i = 0; i < n; i++)
37             for (int j : sucesores[i]) predecesores[j].push_back(i);
38
39         queue<int> cola;
40         for (int i = 0; i < n; i++) {
41             if (gradoSalida[i] == 0) {
42                 resultado[i] = 0; // posicion terminal: quien mueve
43                 aqui ya perdio
44                 cola.push(i);
45             }
46         }
47     }
48 }

```

```

38 }
39
40 vector<int> sucesoresPerdedoresPendientes(n);
41 for (int i = 0; i < n; i++) sucesoresPerdedoresPendientes[i]
42 = gradoSalida[i];
43
44 while (!cola.empty()) {
45     int actual = cola.front(); cola.pop();
46     for (int p : predecesores[actual]) {
47         if (resultado[p] != -1) continue; // ya resuelto
48         if (resultado[actual] == 0) {
49             // p tiene un movimiento hacia una posicion perdedora
50             : p es ganadora
51             resultado[p] = 1;
52             cola.push(p);
53         } else {
54             // este sucesor de p es ganador; si TODOS lo son, p
55             sera perdedora
56             sucesoresPerdedoresPendientes[p]--;
57             if (sucesoresPerdedoresPendientes[p] == 0) {
58                 resultado[p] = 0;
59                 cola.push(p);
60             }
61         }
62     }
63 } // O(V + E)
64
65 // READ: true si la posicion i es ganadora (llamar
66 // resolverTodas antes)
67 bool esGanadora(int i) const {
68     return resultado[i] == 1;
69 } // O(1)
70
71 //
72 // -----
73 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
74 // -----
75
76 // 1. Version recursiva con memoizacion (mas simple de
77 // escribir, igual
78 // complejidad, pero con riesgo de desbordar la pila si el
79 // DAG es
80 // profundo): identica a la de Estados Ganadores y
81 // Perdedores, con
82 // sucesores explicitos en vez de calcularlos con una
83 // formula.
84 // 2. Posiciones con CICLOS (empates o repeticion infinita
85 // posible): el
86 // analisis retrogrado puro no basta; se necesita una
87 // tercera
88 // categoria (EMPATE/DIBUJO) y un algoritmo como el de
89 // Zermelo
90 // generalizado, que primero resuelve las posiciones
91 // alcanzables en
92 // un numero finito de pasos y deja el resto como empate.
93 // 3. Grafos MUY grandes (posiciones generadas implícitamente,
94 // no
95 // enumeradas de antemano): generar sucesores() bajo
96 // demanda dentro
97 // de una memoizacion recursiva, en vez de construir
98 // sucesores y
99 // predecesores explicitos como en resolverTodas().
100 // 4. Generalizacion a numeros de Grundy (no solo ganador/
101 // perdedor): en
102 // vez de resultado[i] booleano, calcular
103 // grundy[i] = mex de {grundy[j] : j en sucesores[i]};
104 // resultado[i]
105 // ganadora es equivalente a grundy[i] != 0.
106 };

```

Ejemplo de funcionamiento:

```

1 AnalisisRetrogrado g(5); // posiciones 0..4, donde 0 es
  // terminal
2 g.agregarMovimiento(1, 0);
3 g.agregarMovimiento(2, 1);
4 g.agregarMovimiento(3, 1);
5 g.agregarMovimiento(3, 2);
6 g.agregarMovimiento(4, 3);
7
8 g.resolverTodas();
9
10 bool r0 = g.esGanadora(0);
11 // r0 = false (posicion terminal: quien mueve aqui ya perdio)
12
13 bool r1 = g.esGanadora(1);
14 // r1 = true (mueve hacia 0, que es perdedora)
15
16 bool r2 = g.esGanadora(2);
17 // r2 = false (su unico sucesor es 1, que es ganadora para el
18 // rival)

```

```

19 bool r3 = g.esGanadora(3);
20 // r3 = true (mueve hacia 2, que es perdedora)

```

Nota: este análisis requiere que el grafo de posiciones sea un **DAG** (sin ciclos) para garantizar que el orden topológico inverso existe y el proceso termina. Si el juego permite volver a una posición ya visitada (ciclos), hace falta una tercera categoría de resultado (empate o posición indeterminada) y una versión más elaborada del algoritmo, como se indica en la modificación 2 dentro del código.

Regla práctica: usa análisis retrógrado como el marco mental general para cualquier juego de esta parte: identifica primero las posiciones terminales (con resultado obvio), y propaga desde ahí. Es la misma idea exacta que *Estados Ganadores y Perdedores*, aquí generalizada a un grafo explícito con orden topológico; cuando se calculen números de Grundy más adelante, es este mismo mecanismo con **mex** en vez de una simple disyunción booleana.

62 Nim y Teoría de Sprague-Grundy

62.1 Nim Clásico (Bouton's Theorem)

Idea: en Nim hay k montones de fichas, con tamaños $a_1, \dots, a_k$. En cada turno, un jugador elige **un** montón y quita de él **al menos una** ficha (tantas como quiera, hasta vaciarlo). Pierde quien no puede mover, es decir, quien enfrenta todos los montones en 0. El teorema de Bouton dice que la posición inicial es **perdedora** para el jugador que mueve primero si y solo si $a_1 \oplus a_2 \oplus \dots \oplus a_k = 0$ (XOR de todos los tamaños), y **ganadora** en caso contrario. La demostración usa exactamente el criterio de *Estados Ganadores y Perdedores*: se comprueba que desde cualquier posición con XOR distinto de 0 existe un movimiento hacia XOR = 0, y que desde cualquier posición con XOR = 0 todo movimiento lleva a XOR distinto de 0 — las dos propiedades que caracterizan, por inducción, cuál lado gana.

¿Por qué usarlo? Sin el teorema, decidir quién gana requeriría memoización sobre el producto de los tamaños de los montones, inviable si cada a_i es grande (por ejemplo 10^9). Bouton reduce el problema a una sola operación XOR en $O(k)$, sin importar cuán grandes sean los montones. Es además el caso más simple — y la motivación original — del *Teorema de Sprague-Grundy*: el número de Grundy de un montón de tamaño n en Nim es exactamente n , así que el XOR de Grundy de la suma de juegos coincide con el XOR directo de los tamaños.

Casos de uso:

- Determinar quién gana Nim clásico con montones de cualquier tamaño, en $O(k)$.
- Encontrar el movimiento ganador concreto desde una posición ganadora (no solo si se gana o se pierde).
- Como caso base para verificar la implementación de *Sprague-Grundy* en juegos más generales: cuando el juego se reduce a Nim puro, la fórmula debe coincidir con el XOR directo.

- Punto de partida al reconocer que un juego nuevo "es Nim disfrazado" tras encontrar su número de Grundy por montón.

Complejidad: $O(k)$ para determinar el ganador, con k el número de montones; $O(k)$ también para encontrar el movimiento ganador concreto.

```

1 // CREATE/READ: Nim clasico: determina el ganador via XOR (
  // teorema de
2 // Bouton) y encuentra el movimiento ganador concreto si existe
3 struct NimClasico {
4     vector<long long> montones;
5
6     // CREATE: guarda los tamanos de los montones
7     NimClasico(const vector<long long>& a) : montones(a) {
8     } // O(k)
9
10    // READ: XOR de todos los montones (nim-suma)
11    long long nimSuma() const {
12        long long x = 0;
13        for (long long a : montones) x ^= a;
14        return x;
15    } // O(k)
16
17    // READ: true si el jugador que mueve PRIMERO gana
18    bool primerJugadorGana() const {
19        return nimSuma() != 0;
20    } // O(k)
21
22    // READ: encuentra un movimiento ganador: indice del monton a
      // reducir
23    // y su nuevo tamaño; retorna {-1, -1} si la posicion ya es
      // perdedora
24    pair<int, long long> movimientoGanador() const {
25        long long x = nimSuma();
26        if (x == 0) return {-1, -1}; // posicion perdedora: no hay
      // movimiento ganador
27        for (int i = 0; i < (int)montones.size(); i++) {
28            long long nuevo = montones[i] ^ x;
29            // un movimiento valido debe REDUCIR el monton, no
      // aumentarlo
30            if (nuevo < montones[i]) return {i, nuevo};
31        }
32        return {-1, -1}; // no deberia ocurrir si x != 0
33    } // O(k)
34
35    //
36    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
37    //
38    // 1. Por que nuevo = montones[i] ^ x funciona: se busca el
      // monton i
39    // tal que reducirlo a montones[i] ^ x deje la nim-suma
      // total en 0;
40    // esto es valido solo cuando montones[i] ^ x < montones[i]
      // ], porque
41    // el movimiento en Nim SOLO puede quitar fichas, nunca
      // agregar.
42    // Existe garantizado al menos un i que cumple esto cuando
      // x != 0,
43    // por construccion del bit mas significativo de x.
44    // 2. Verificacion manual del bit mas significativo: si se
      // quiere
45    // encontrar el monton ganador sin probar los k montones
      // uno por
46    // uno, se ubica el bit mas alto de x y se busca el primer
      // monton
47    // que tenga ese bit encendido; ese monton es siempre
      // valido.
48    // 3. Version con MUCHOS montones y consultas de nim-suma
      // tras
49    // actualizaciones (montones cambian de tamaño
      // dinamicamente): usar
50    // un multiset o un arbol de Fenwick por bits para
      // recalcular el
51    // XOR total en O(log(max valor)) por actualizacion, en
      // vez de
52    // recorrer todos los montones cada vez.
53    // 4. Nim con UN SOLO monton: la nim-suma es simplemente ese
      // valor;
54    // el primer jugador gana siempre que el monton no este
      // vacio
55    // (caso trivial, pero verifica la formula general).
56 };

```

Ejemplo de funcionamiento:

```
NimClasico n({3, 4, 5});
```



```

3 long long x = n.nimSuma();
4 // x = 2
5 // (3 XOR 4 XOR 5 = 2)
6
7 bool gana = n.primerJugadorGana();
8 // gana = true (la nim-suma es distinta de 0)
9
10 auto mov = n.movimientoGanador();
11 // mov = {0, 1}
12 // (reducir el monton de índice 0, de 3 a 1: 3^2=1, y 1 < 3,
13 //  válido;
14 //  tras el movimiento la posicion queda {1,4,5}, con nim-suma
15 //  1^4^5 = 0, perdedora para el rival)

```

Nota: el movimiento ganador **no es único** en general — puede haber más de un montón desde el que se puede llegar a nim-suma 0; `movimientoGanador()` retorna el primero que encuentra, cualquiera de los válidos sirve igual de bien porque todos dejan al rival en una posición perdedora.

Regla práctica: usa el XOR de Bouton directamente para Nim clásico (montones independientes, quitar cualquier cantidad de un solo montón). Si el juego impone restricciones sobre **cuánto** se puede quitar de un montón, no es Nim puro: revisa *Nim con Montones y Movimientos Restringidos* o calcula el número de Grundy de un montón individual con *Teorema de Sprague-Grundy* antes de aplicar cualquier XOR.

62.2 Nim con Montones y Movimientos Restringidos

Idea: muchas variantes de Nim limitan **cuántas** fichas se pueden quitar de un montón en un turno —por ejemplo, "a lo sumo k fichas por turno" o "solo se pueden quitar cantidades de un conjunto permitido S ". El teorema de Bouton (XOR directo de los tamaños) deja de aplicar tal cual, porque ya no es cierto que se pueda quitar **cualquier** cantidad; hay que calcular primero el **número de Grundy** de un montón individual bajo esa restricción de movimiento, y luego combinar los montones con XOR de esos números de Grundy (no de los tamaños directos), como establece *Suma de Juegos Independientes*. El número de Grundy de un montón se calcula con la fórmula `mex` sobre los montones alcanzables, exactamente como en *Cálculo de Grundy sobre un DAG*.

¿Por qué usarlo? Aplicar el XOR directo de los tamaños cuando hay una restricción de movimiento da resultados **incorrectos**, porque el teorema de Bouton depende críticamente de que cualquier reducción sea alcanzable en un solo movimiento. Con "a lo sumo k por turno", por ejemplo, un montón de tamaño n se comporta como Grundy $n \bmod (k+1)$ — un patrón periódico que hay que **descubrir**, típicamente con *Fuerza Bruta para Descubrir Patrones*: calcular Grundy para montones pequeños, observar la periodicidad, y demostrarla o al menos verificarla extensivamente antes de confiar en la fórmula.

Casos de uso:

- Nim con "máximo k fichas por turno" (k -Nim), donde Grundy de un montón n es $n \bmod (k+1)$.
- Nim donde solo se permite quitar cantidades de un conjunto fijo S (es Nim disfrazado de *Juego de Sustracción* por montón).

- Nim con montones donde la cantidad permitida depende de la paridad del turno o de otra condición sobre el estado.
- Verificación de que una variante de Nim, tras calcular Grundy por montón, efectivamente se reduce a XOR estándar cuando la restricción desaparece (caso $k \rightarrow \infty$).

Complejidad: $O(n \cdot |S|)$ o $O(n \cdot k)$ para precomputar el número de Grundy de cada tamaño de montón de 0 a n (uno por cada tamaño, cada uno revisando sus movimientos válidos); $O(1)$ por consulta una vez precomputada la tabla, y $O(\text{montones})$ para combinar el resultado final con XOR.

```

1 // CREATE/READ: Numero de Grundy de un monton bajo una
2 // restricción de
3 // movimiento arbitraria (funcion movimientosValidos(n) ->
4 // tamanos
5 // alcanzables), y combinacion de varios montones via XOR de
6 // Grundy
7 struct NimRestringido {
8     vector<int> grundy; // grundy[n] = numero de Grundy de un
9     // monton de tamaño n
10
11     // CREATE: precomputa grundy[0..nMax] dada la funcion de
12     // movimientos
13     // validos desde un monton de tamaño n
14     NimRestringido(int nMax, function<vector<int>>(int)>
15     movimientosValidos) {
16         grundy.assign(nMax + 1, 0);
17         for (int n = 1; n <= nMax; n++) {
18             vector<int> alcanzables = movimientosValidos(n);
19             set<int> vistos;
20             for (int m : alcanzables) {
21                 if (m >= 0 && m < n) vistos.insert(grundy[m]);
22             }
23             int mex = 0;
24             while (vistos.count(mex)) mex++;
25             grundy[n] = mex;
26         }
27     } // O(nMax * costo de movimientosValidos)
28
29     // READ: true si la combinacion de varios montones (bajo la
30     // misma
31     // restricción) es GANADORA para el jugador que mueve primero
32     bool primerJugadorGana(const vector<int>& montones) const {
33         int x = 0;
34         for (int m : montones) x ^= grundy[m];
35         return x != 0;
36     } // O(numero de montones)
37
38     //
39     // -----
40     // MODIFICACIONES Y COMENTARIOS CON LOS USOS
41     //
42     // -----
43     // 1. k-Nim (maximo k fichas por turno): movimientosValidos(n)
44     //  retorna
45     //  {n-1, n-2, ..., max(0, n-k)}; el patron resultante es
46     //  grundy[n] = n % (k+1), verificable generando la tabla y
47     //  observando la periodicidad (ver Fuerza Bruta para
48     //  Descubrir
49     //  Patrones).
50     // 2. Conjunto de movimientos S fijo (subtraction game por
51     //  monton):
52     //  movimientosValidos(n) retorna {n-s : s en S, s <= n};
53     //  el patron
54     //  de grundy puede no ser tan simple como un modulo, y a
55     //  veces
56     //  requiere la tabla completa sin formula cerrada.
57     // 3. Verificacion cruzada con Nim Clasico: si
58     //  movimientosValidos(n)
59     //  retorna TODOS los valores {0, 1, ..., n-1} (sin
60     //  restricción),
61     //  grundy[n] debe dar exactamente n para todo n,
62     //  confirmando
63     //  consistencia con el teorema de Bouton.
64     // 4. Restriccion que depende de mas que el tamaño (por
65     //  ejemplo,
66     //  ultimo movimiento realizado): el estado deja de ser
67     //  solo el
68     //  tamaño del monton, y hace falta ampliar la clave de
69     //  memoizacion
70     //  a un par o tupla en vez de un solo entero.
71     // 5. n muy grande con formula periodica detectada: una vez
72     //  confirmado

```

```

52 // el periodo P y calculada la tabla para un periodo
    completo, se
53 // responde Grundy(n) para n arbitrario con Grundy[n % P]
    en O(1),
54 // sin necesidad de precomputar hasta nMax completo.
55 };

```

Ejemplo de funcionamiento:

```

1 // k-Nim con k=3 (maximo 3 fichas por turno)
2 int k = 3;
3 auto movK = [k](int n) {
4     vector<int> res;
5     for (int q = 1; q <= k && q <= n; q++) res.push_back(n - q);
6     return res;
7 };
8 NimRestringido r(20, movK);
9
10 int g5 = r.Grundy[5];
11 // g5 = 1
12 // (5 % (3+1)) = 5 % 4 = 1, confirmando el patron periodico
    esperado)
13
14 bool gana = r.primerJugadorGana({5, 7, 2});
15 // gana: XOR de Grundy[5], Grundy[7], Grundy[2] segun la tabla
16 // precomputada; hay que correrlo para confirmar el valor
    exacto

```

Nota: el error más común en esta variante es aplicar el XOR directo de los **tamaños** de los montones en vez del XOR de sus **números de Grundy**. Ambos coinciden solo en Nim clásico sin restricciones (donde $\text{Grundy}[n] == n$); en cualquier variante restringida, hay que calcular Grundy primero y recién ahí combinar con XOR.

Regla práctica: ante cualquier variante de Nim con una restricción sobre cuánto se puede quitar, nunca asumas que el XOR directo de tamaños sigue funcionando. Calcula $\text{Grundy}[n]$ para varios valores pequeños de n , busca el patrón (muchas veces es periódico), verifícalo extensivamente, y solo entonces combina montones con XOR de esos valores de Grundy.

62.3 Teorema de Sprague-Grundy (Grundy Numbers / Nimbers)

Idea: el teorema de Sprague-Grundy generaliza *Estados Ganadores y Perdedores* de una clasificación binaria (ganadora/perdedora) a un **número** por posición, llamado número de Grundy (o *nimber*). Para una posición p con sucesores $s_1, \dots, s_r$ (las posiciones alcanzables en un movimiento), el número de Grundy es $G(p) = \text{mex}\{G(s_1), \dots, G(s_r)\}$, donde **mex** (*minimum excludant*) es el menor entero no negativo que **no** aparece en ese conjunto. Una posición terminal (sin movimientos) tiene $G = \text{mex}(\emptyset) = 0$. El teorema establece que **todo** juego imparcial con información perfecta y sin azar, cualquiera sea su estructura, es equivalente a un montón de Nim de tamaño $G(p)$: $G(p) = 0$ si y solo si p es perdedora, exactamente como en el criterio de *Estados Ganadores y Perdedores*.

¿Por qué usarlo? La clasificación binaria ganadora/perdedora responde "¿quién gana esta posición sola?", pero no dice nada sobre cómo se comporta esa posición cuando se **combina** con otras, como en un juego formado por varios montones o subjuegos independientes. El número de Grundy sí lo permite: es el "equivalente en Nim" de la posición, y por eso se puede combinar con otros números de Grundy usando XOR, tal como se ve en *Suma de Juegos Independientes*. Calcular **mex** en vez de solo verificar "existe un cero entre los sucesores" cuesta apenas un poco más, pero abre la puerta

a resolver sumas de juegos sin memoizar el producto cartesiano completo de estados.

Casos de uso:

- Calcular el número de Grundy de una posición en cualquier juego imparcial, como paso previo a analizar sumas de subjuegos.
- Reconocer que un juego nuevo "es Nim disfrazado": si $G(n) = n$ para un montón de tamaño n , el juego se comporta exactamente como Nim clásico.
- Precomputar una tabla de Grundy para todos los tamaños de un componente de juego hasta cierto límite, y luego combinar varios componentes con XOR.
- Base de *Nim con Montones y Movimientos Restringidos*, donde Grundy de un montón bajo restricciones no siempre es igual al tamaño.

Complejidad: $O(V + E)$ para calcular Grundy de todas las posiciones de un DAG con V posiciones y E movimientos totales, igual que en *Análisis Retrógrado*; cada posición calcula su **mex** en $O(\text{grado de salida})$ con un conjunto o arreglo de marcado.

```

1 // CREATE/READ: Calculo de numeros de Grundy sobre un conjunto
    de
2 // posiciones dadas explicitamente por una funcion de sucesores,
    con
3 // memoizacion (equivalente a Analisis Retrogrado pero con mex
    en vez de
4 // clasificacion booleana)
5 struct SpragueGrundy {
6     int nMax;
7     vector<int> Grundy; // Grundy[n] = numero de Grundy, o -1 si
    no calculado
8     function<vector<int>>(int)> sucesores;
9
10 // CREATE: guarda el limite y la funcion de sucesores del
    juego
11 SpragueGrundy(int limite, function<vector<int>>(int)> func)
12 : nMax(limite), sucesores(func) {
13     Grundy.assign(nMax + 1, -1);
14 } // O(nMax)
15
16 // READ: mex de un conjunto de valores: el menor entero no
    negativo
17 // que NO aparece en el conjunto
18 static int mex(const vector<int>& valores) {
19     set<int> presentes(valores.begin(), valores.end());
20     int m = 0;
21     while (presentes.count(m)) m++;
22     return m;
23 } // O(k log k), k = tamano del conjunto
24
25 // READ: calcula (memoizado) el numero de Grundy de la
    posicion n
26 int calcular(int n) {
27     if (Grundy[n] != -1) return Grundy[n];
28     vector<int> sucGrundy;
29     for (int s : sucesores(n)) {
30         if (s >= 0 && s <= nMax) sucGrundy.push_back(calcular(s));
31     }
32     return Grundy[n] = mex(sucGrundy);
33 } // O(1) amortizado por posicion, O(nMax * grado) en total
34
35 // READ: precomputa Grundy para TODAS las posiciones de 0 a
    nMax
36 void calcularTodas() {
37     for (int n = 0; n <= nMax; n++) calcular(n);
38 } // O(nMax * grado promedio)
39
40 //
41 // -----
42 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
43 //
44 // 1. mex con arreglo en vez de set (mas rapido cuando el
    grado de
45 // salida es acotado): usar vector<bool> visto(grado+2,
    false),

```

```

45 // marcar cada valor sucesor que sea <= grado, y recorrer
    desde 0
46 // hasta encontrar el primer false; evita el costo de un
    set
47 // balanceado cuando el numero de sucesores es pequeño y
    fijo.
48 // 2. Estado compuesto (mas de una dimension, ej. posicion en
    un
49 // tablero 2D): usar un mapa (unordered_map con clave
    compuesta, o
50 // codificar el estado como un unico entero) en vez del
    arreglo
51 // Grundy indexado por un solo entero.
52 // 3. G(n) = n para Nim clasico: verificar esta identidad al
    calcular
53 // sucesores(n) = {0, 1, ..., n-1} confirma que la
    implementacion
54 // de mex es correcta antes de aplicarla a un juego mas
    complejo.
55 // 4. Posicion GANADORA vs. PERDEDORA a partir de Grundy:
    ganadora si
56 // y solo si Grundy[n] != 0; es una relajacion menos
    informativa
57 // del numero completo, util cuando solo importa el
    resultado de un
58 // unico juego (no una suma de varios).
59 // 5. Reconstruccion de la jugada optima: junto con Grundy[n],
    guardar
60 // el sucesor s cuyo Grundy[s] es 0 (si existe); ese es el
61 // movimiento que deja al rival en una posicion perdedora.
62 };

```

Ejemplo de funcionamiento:

```

1 // juego de sustraccion con S = {1, 3, 4} (mismo conjunto que
    el
2 // ejemplo de Estados Ganadores y Perdedores)
3 auto sucS = [](int n) {
4     vector<int> res;
5     for (int m : {1, 3, 4}) if (m <= n) res.push_back(n - m);
6     return res;
7 };
8 SpragueGrundy sg(10, sucS);
9 sg.calcularTodas();
10
11 int g2 = sg.Grandy[2];
12 // g2 = 0
13 // (desde 2 solo se puede restar 1, llegando a Grundy[1]; mex
    del
14 // conjunto {Grundy[1]} da 0 si Grundy[1] != 0, confirmando
    que 2 es
15 // perdedora, igual que se vio con esGanadora(2) = false)
16
17 int g5 = sg.Grandy[5];
18 // g5: mex de los Grundy de los sucesores de 5 (restando 1, 3 o
    4);
19 // hay que correrlo para confirmar el valor exacto

```

Nota: el número de Grundy **no es únicamente** "cero o distinto de cero" — aunque para analizar un solo juego alcanza con esa distinción (igual que *Estados Ganadores y Perdedores*), el valor completo importa en cuanto el juego se combina con otros mediante suma, como se ve en la siguiente subsección. Calcular solo win/lose y luego intentar combinar esos booleanos con XOR es un error conceptual: hace falta el número completo.

Regla práctica: calcula números de Grundy (no solo ganador/perdedor) en cuanto el problema involucre **más de un componente de juego independiente** jugado simultáneamente. Si el juego es uno solo, sin combinar con otros, *Estados Ganadores y Perdedores* alcanza y es más simple de razonar.

62.4 Suma de Juegos Independientes (XOR de Grundy Numbers)

Idea: muchos juegos en competencia no son un solo componente, sino la **suma** de varios subjuegos independientes jugados a la vez: en cada turno, el jugador elige **uno** de los subjuegos y hace un movimiento válido en él, dejando los demás intactos. Pierde quien no puede mover

en **ninguno** de los subjuegos. El teorema de Sprague-Grundy dice que el número de Grundy de la suma es exactamente el **XOR** de los números de Grundy de cada subjuego: $G(p_1 + p_2 + \dots + p_k) = G(p_1) \oplus G(p_2) \oplus \dots \oplus G(p_k)$. Esto es una generalización directa del teorema de Bouton para Nim (donde cada montón es un subjuego cuyo Grundy es igual a su tamaño), pero aplica a **cualquier** colección de juegos imparciales, no solo a montones de fichas.

¿Por qué usarlo? Sin este teorema, analizar la suma de k subjuegos requeriría memoizar sobre el **producto cartesiano** de todos los estados posibles de cada componente — inviable incluso con pocos componentes de tamaño moderado. El teorema reduce el problema a calcular el número de Grundy de **cada componente por separado** (mucho más pequeño, un solo grafo de estados a la vez) y combinarlos con una operación $O(1)$ por componente. Es la razón por la que Sprague-Grundy es la herramienta central de toda la parte: convierte "resolver un juego compuesto gigante" en "resolver k juegos pequeños e independientes".

Casos de uso:

- Cualquier problema que describa "varios montones/pilas/tableros independientes, elige uno y muévete ahí" — la estructura estándar de Nim y sus generalizaciones.
- Combinar distintos **tipos** de subjuego en una misma suma (por ejemplo, algunos montones de Nim clásico junto con montones de un juego de sustracción distinto): cada uno aporta su propio número de Grundy al XOR total.
- Determinar el ganador de la posición combinada sin necesidad de explorar el árbol de movimientos de la suma completa.
- Encontrar el movimiento ganador: se busca un componente y un movimiento dentro de él que cambie su Grundy de forma que el XOR total quede en 0.

Complejidad: $O(\sum_i \text{costo de calcular Grundy}(p_i))$ para calcular el Grundy de cada componente por separado, más $O(k)$ para combinarlos con XOR — muchísimo más barato que memoizar el producto cartesiano de todos los componentes.

```

1 // CREATE/READ: Suma de juegos independientes: combina los
    numeros de
2 // Grundy de varios componentes (posiblemente de TIPOS
    distintos) via
3 // XOR, y encuentra el movimiento ganador si existe
4 struct SumaJuegos {
5     // READ: XOR de una lista de numeros de Grundy ya calculados
6     // (uno por cada componente de la suma)
7     static int GrundyTotal(const vector<int>& GrundyPorComponente)
8     {
9         int x = 0;
10        for (int g : GrundyPorComponente) x ^= g;
11        return x;
12    } // O(k)
13
14    // READ: true si el jugador que mueve PRIMERO gana la suma
    completa
15    static bool primerJugadorGana(const vector<int>&
16    GrundyPorComponente) {
17        return GrundyTotal(GrundyPorComponente) != 0;
18    } // O(k)
19
20    // READ: dado el Grundy de cada componente y una funcion que,
    para un
21    // componente i y un Grundy objetivo, devuelve si existe un

```



```

20 // movimiento en ese componente que lleve exactamente a ese
    grundy
21 // (o -1 si no existe), encuentra el indice de componente y
    el nuevo
22 // grundy que hacen ganar la posicion completa
23 static pair<int, int> movimientoGanador(
24     const vector<int>& grundyPorComponente,
25     function<int(int, int)> intentarLlegarA // (indice,
        grundyObjetivo) -> nuevoGrundy o -1
26 ) {
27     int total = grundyTotal(grundyPorComponente);
28     if (total == 0) return {-1, -1}; // posicion perdedora: no
        hay movimiento ganador
29     for (int i = 0; i < (int)grundyPorComponente.size(); i++) {
30         int objetivo = grundyPorComponente[i] ^ total; // el
            mismo truco que en Nim clasico
31         if (objetivo < grundyPorComponente[i]) {
32             int resultado = intentarLlegarA(i, objetivo);
33             if (resultado == objetivo) return {i, objetivo};
34         }
35     }
36     return {-1, -1}; // no deberia ocurrir si total != 0 y el
        juego esta bien definido
37 } // O(k * costo de intentarLlegarA)
38
39 //
40 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
41 //
42 // 1. Componentes de TIPOS distintos: cada componente puede
    venir de un
43 // SpragueGrundy distinto (por ejemplo, unos montones de
    Nim puro y
44 // otros de un juego de sustraccion con S={1,2}); lo unico
    que
45 // importa aqui es el numero de Grundy final de cada uno,
    no como
46 // se calculo.
47 // 2. Por que objetivo < grundyPorComponente[i] no siempre
    aplica: a
48 // diferencia de Nim clasico (donde reducir el TAMANO
    garantiza
49 // reducir el Grundy), en un juego general el Grundy no es
    necesariamente monotono con el tamano; la condicion
50 // exacta para
51 // "movimiento valido" depende del juego concreto, por eso
52 // intentarLlegarA() debe verificarlo explicitamente en
    vez de
53 // asumir monotonia.
54 // 3. Muchos componentes IDENTICOS (por ejemplo, 50 montones
    todos con
55 // la misma regla): agrupar por Grundy y contar cuantas
    veces
56 // aparece cada valor; un valor que aparece un numero PAR
    de veces
57 // se cancela en el XOR (a XOR a = 0), asi que solo
    importan los
58 // que aparecen un numero impar de veces.
59 // 4. Verificacion cruzada con Nim clasico: si todos los
    componentes
60 // son montones de Nim puro, grundyTotal() debe coincidir
61 // exactamente con NimClasico::nimSuma() sobre los mismos
    tamanos.
62 };

```

Ejemplo de funcionamiento:

```

1 // tres componentes de juegos distintos, cada uno ya con su
    Grundy calculado
2 vector<int> grundys = {3, 5, 6}; // por ejemplo, via
    SpragueGrundy sobre cada tipo
3
4 int total = SumaJuegos::grundyTotal(grundys);
5 // total = 3 XOR 5 XOR 6 = 0
6
7 bool gana = SumaJuegos::primerJugadorGana(grundys);
8 // gana = false
9 // (el XOR total es 0: la posicion combinada es PERDEDORA para
    quien
10 // mueve primero, sin importar que cada componente individual
    sea
11 // ganador por separado)

```

Nota: el resultado del ejemplo (una posición perdedora aunque ningún componente individual tenga Grundy 0) es el punto central de esta técnica: la suma de varios juegos ganadores por separado puede ser perdedora en conjunto, y viceversa. Analizar cada componente aislado **no** dice nada directo sobre el resultado de la suma

— hace falta el XOR de sus números de Grundy.

Regla práctica: en cuanto un problema describa varios subjuegos independientes donde se elige uno por turno, calcula el número de Grundy de **cada tipo** de componente por separado (con *Sprague-Grundy* o, si es Nim puro, el tamaño mismo), combina todos con XOR, y decide el ganador según si ese XOR es cero. Nunca combines resultados binarios de ganador/perdedor con XOR directamente — solo los números de Grundy completos son combinables de esa forma.

62.5 Cálculo de Grundy sobre un DAG (mex de Sucesores)

Idea: esta subsección aísla el **paso mecánico** que ya apareció dentro de *Teorema de Sprague-Grundy*: dado un grafo dirigido acíclico (DAG) de posiciones, calcular $G(p) = \text{mex}\{G(s) : s \text{ sucesor de } p\}$ para **todas** las posiciones de forma sistemática, sin depender de una fórmula cerrada. Se puede hacer de dos formas equivalentes: **memoización recursiva** (igual que en Sprague-Grundy, bajando por las posiciones hasta las terminales y subiendo el resultado), o **tabulación por orden topológico inverso** (como en *Análisis Retrógrado*, procesando primero los nodos sin sucesores y propagando hacia atrás). Ambas dan el mismo resultado; la tabulación evita el riesgo de desbordar la pila en DAGs muy profundos.

¿Por qué usarlo? Cuando un juego no tiene una fórmula conocida (no es Nim, no es un juego de sustracción con patrón periódico evidente), la única forma de obtener sus números de Grundy es calcularlos **directamente** sobre el grafo de posiciones, exactamente como se calcula cualquier DP sobre un DAG. Esta subsección documenta esa mecánica de forma explícita y reutilizable, separada del teorema en sí, porque es el bloque de construcción que se repite en casi todas las variantes de juego de esta parte: solo cambia cómo se generan los sucesores de cada posición.

Casos de uso:

- Calcular Grundy de un juego nuevo sin fórmula conocida, cuando el número de posiciones distintas es manejable.
- Generar una tabla de Grundy para buscar un patrón periódico o cerrado, como paso previo a *Fuerza Bruta para Descubrir Patrones*.
- Juegos sobre estructuras compuestas (pares de enteros, posiciones en una grilla) donde el estado se codifica como un entero único antes de aplicar esta misma mecánica.
- Verificar, sobre casos pequeños, una fórmula de Grundy sospechada para una variante de Nim o de sustracción.

Complejidad: $O(V + E)$ con V posiciones y E movimientos totales, igual que *Análisis Retrógrado*: cada posición calcula su **mex** una sola vez, con costo proporcional a su grado de salida.

```

1 // CREATE/READ: Calculo de Grundy sobre un DAG dado
    explicitamente,
2 // con dos versiones equivalentes: memoizacion recursiva y
    tabulacion

```



```

3 // por orden topologico inverso
4 struct GrundyDAG {
5     int n;
6     vector<vector<int>> sucesores;
7     vector<int> Grundy;
8
9     // CREATE: reserva estructuras para n posiciones (0..n-1)
10    GrundyDAG(int numPosiciones) : n(numPosiciones) {
11        sucesores.assign(n, {});
12        Grundy.assign(n, -1);
13    } // O(n)
14
15    // WRITE: agrega un movimiento valido de 'desde' hacia 'hacia'
16    void agregarMovimiento(int desde, int hacia) {
17        sucesores[desde].push_back(hacia);
18    } // O(1) amortizado
19
20    // READ: mex de un conjunto de valores (arreglo, mas rapido
21    // que set
22    // cuando el grado de salida es pequeño y acotado)
23    static int mex(const vector<int>& valores) {
24        int m = (int)valores.size();
25        vector<bool> visto(m + 1, false);
26        for (int v : valores) if (v <= m) visto[v] = true;
27        int r = 0;
28        while (r <= m && visto[r]) r++;
29        return r;
30    } // O(grado de salida)
31
32    // READ: version recursiva con memoizacion
33    int calcularRecursivo(int p) {
34        if (Grundy[p] != -1) return Grundy[p];
35        vector<int> vals;
36        for (int s : sucesores[p]) vals.push_back(calcularRecursivo(s));
37        return Grundy[p] = mex(vals);
38    } // O(1) amortizado por posicion
39
40    // READ: version tabulada (bottom-up) por orden topologico
41    // inverso;
42    // requiere que el grafo sea un DAG
43    void calcularTabulado() {
44        vector<int> gradoSalida(n);
45        for (int i = 0; i < n; i++) gradoSalida[i] = (int)sucesores[i].size();
46
47        vector<vector<int>> predecesores(n);
48        for (int i = 0; i < n; i++)
49            for (int j : sucesores[i]) predecesores[j].push_back(i);
50
51        queue<int> cola;
52        vector<int> pendientes = gradoSalida;
53        for (int i = 0; i < n; i++) {
54            if (gradoSalida[i] == 0) {
55                Grundy[i] = 0; // mex del conjunto vacio
56                cola.push(i);
57            }
58        }
59
60        while (!cola.empty()) {
61            int actual = cola.front(); cola.pop();
62            for (int p : predecesores[actual]) {
63                pendientes[p]--;
64                if (pendientes[p] == 0) {
65                    vector<int> vals;
66                    for (int s : sucesores[p]) vals.push_back(Grundy[s]);
67                    Grundy[p] = mex(vals);
68                    cola.push(p);
69                }
70            }
71        }
72    } // O(V + E)
73
74    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
75    //
76    // -----
77    // 1. Estado codificado (no un simple 0..n-1): si la posicion
78    // es un
79    // par (i,j) o una mascara de bits, codificarla primero a
80    // un unico
81    // entero (por ejemplo i*ancho+j, o el valor de la mascara
82    // ) antes
83    // de usar esta misma estructura indexada por entero.
84    // 2. Sucesores generados bajo demanda (el DAG es implicito,
85    // muy
86    // grande para enumerar): usar calcularRecursivo() con un
87    // unordered_map<clave,int> en vez del vector Grundy,
88    // generando
89    // sucesores() como una funcion en vez de una lista
90    // precomputada.

```

```

83 // 3. mex con set en vez de arreglo: si el grado de salida no
84 // esta
85 // acotado por el numero de posiciones (por ejemplo,
86 // movimientos
87 // con valores muy dispersos), usar set<int> como en la
88 // version
89 // de Sprague-Grundy, a costa de un factor log adicional.
90 // 4. Verificar contra Estados Ganadores y Perdedores: Grundy
91 // [p] == 0
92 // debe coincidir exactamente con "p es perdedora" segun
93 // el
94 // analisis retrogrado booleano; sirve como chequeo de
95 // cordura
96 // entre ambas implementaciones.
97 };

```

Ejemplo de funcionamiento:

```

1 GrundyDAG g(6);
2 g.agregarMovimiento(1, 0);
3 g.agregarMovimiento(2, 0);
4 g.agregarMovimiento(2, 1);
5 g.agregarMovimiento(3, 1);
6 g.agregarMovimiento(3, 2);
7 g.agregarMovimiento(4, 0);
8 g.agregarMovimiento(4, 3);
9
10 g.calcularTabulado();
11
12 int g0 = g.Grundy[0];
13 // g0 = 0 (posicion terminal, sin sucesores)
14
15 int g1 = g.Grundy[1];
16 // g1 = 1 (su unico sucesor es 0, con Grundy 0; mex({0}) = 1)
17
18 int g2 = g.Grundy[2];
19 // g2: mex de los Grundy de sus sucesores {0, 1}; hay que
20 // correrlo para
21 // confirmar el valor exacto segun la tabla completa

```

Nota: las dos versiones (recursiva y tabulada) dan siempre el mismo resultado. La tabulada requiere construir explícitamente la lista de predecesores antes de empezar, un costo adicional de $O(E)$ que la recursiva no necesita; a cambio, evita cualquier riesgo de desbordar la pila cuando el DAG tiene cadenas muy largas de dependencias.

Regla práctica: usa esta mecánica en cuanto un juego no tenga una fórmula de Grundy conocida de antemano — es el mismo patrón de *Análisis Retrógrado*, con `mex` en vez de una clasificación booleana. Si el número de posiciones es pequeño, la versión recursiva es más rápida de escribir; si es grande o profundo, usa la tabulada.

62.6 Misère Nim (Última Jugada Pierde)

Idea: en Nim clásico (*normal play*), pierde quien **no puede mover**. En la variante **misère**, la regla se invierte: pierde quien **hace el último movimiento** (deja todos los montones en 0); quien no puede mover **gana**. Sorprendentemente, la estrategia óptima de Misère Nim es **casi idéntica** a la de Nim normal: se calcula igual la nim-suma $X = a_1 \oplus \dots \oplus a_k$, y solo cambia la regla en un caso especial. Sea m el número de montones con tamaño **mayor a 1**. Si $m \geq 1$ (existe al menos un montón "grande"), la estrategia es exactamente la de Nim normal: ganar si $X \neq 0$, jugando siempre hacia $X = 0$. Si $m = 0$ (todos los montones tienen tamaño 0 o 1, es decir, el juego ya se redujo a "montoncitos de una ficha"), la regla se invierte: el primer jugador gana si y solo si la **cantidad de montones de tamaño 1 es par**.

¿Por qué usarlo? Aplicar directamente el criterio de Nim normal ($X \neq 0$ gana) a una variante misère da la respuesta **incorrecta** exactamente en el caso borde donde todos los montones son de tamaño 0 o 1 — un

caso fácil de pasar por alto en el enunciado si no se lee con cuidado la condición de fin de juego ("el que mueve último pierde" en vez de "el que no puede mover pierde"). El resultado de Bouton para misère (a veces llamado el teorema de Bouton-Misère) evita tener que recalcular Grundy desde cero para esta variante: reutiliza casi toda la maquinaria de Nim normal, con un único ajuste puntual.

Casos de uso:

- Cualquier variante de Nim donde el enunciado especifique explícitamente "quien hace el último movimiento pierde" en vez de la regla normal.
- Como recordatorio de que **leer con cuidado** la condición de victoria/derrota es tan importante como conocer la fórmula: el mismo juego (mismos montones) puede tener resultado opuesto según la regla de terminación.
- Contraste didáctico frente a *Nim Clásico*: mostrar que dos reglas de fin de juego aparentemente simétricas no dan estrategias simétricas en general (la simplicidad de Misère Nim es una excepción; la mayoría de juegos misère son mucho más difíciles de analizar que su versión normal).
- Verificar con *Análisis Retrógrado* sobre montones pequeños, antes de confiar en la fórmula, que la regla del caso $m = 0$ está bien aplicada.

Complejidad: $O(k)$, igual que Nim normal: un recorrido para calcular la nim-suma y otro (o el mismo) para contar montones mayores a 1 y montones de tamaño exactamente 1.

```

1 // CREATE/READ: Misere Nim: determina el ganador bajo la regla
  // "el que
2 // mueve ultimo pierde", reutilizando la nim-suma de Nim normal
  // con el
3 // ajuste del caso especial (todos los montones de tamaño 0 o
  // 1)
4 struct MisereNim {
5     vector<long long> montones;
6
7     // CREATE: guarda los tamanos de los montones
8     MisereNim(const vector<long long>& a) : montones(a) {
9     } // O(k)
10
11    // READ: true si el jugador que mueve PRIMERO gana, bajo
    // regla misere
12    bool primerJugadorGana() const {
13        long long x = 0;
14        int montonesGrandes = 0; // tamaño > 1
15        int montonesDeUno = 0; // tamaño == 1
16        for (long long a : montones) {
17            x ^= a;
18            if (a > 1) montonesGrandes++;
19            else if (a == 1) montonesDeUno++;
20        }
21
22        if (montonesGrandes > 0) {
23            // hay al menos un monton grande: se juega EXACTAMENTE
            // como
            // Nim normal
24            return x != 0;
25        } else {
26            // todos los montones son 0 o 1: la regla se invierte
27            return montonesDeUno % 2 == 0;
28        }
29    } // O(k)
30 } // O(k)
31
32 //
33 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
34 //
35 // 1. Por que el caso m=0 se invierte: si todos los montones
    // son 0 o

```

```

36 // 1, cada movimiento SOLO puede vaciar un monton de
    // tamaño 1 (no
37 // hay otra opcion); el juego se convierte en "turnos
    // alternados
38 // hasta que no queden montones de 1", y quien hace el
    // ULTIMO de
39 // esos movimientos pierde bajo regla misere, lo que
    // invierte la
40 // paridad respecto a Nim normal.
41 // 2. Version general (juegos misere SIN esta simplificacion):
    // fuera
42 // de Nim, la mayoría de juegos imparciales bajo regla
    // misere NO
43 // tienen un atajo tan simple; se necesita la teoria de
    // juegos
44 // misere generalizada (Grundy misere / teoria de Conway
    // extendida),
45 // fuera del alcance de esta guía.
46 // 3. Verificacion cruzada: comparar primerJugadorGana() con
    // un
47 // Analisis Retrogrado directo (definiendo la posicion
    // terminal
48 // como GANADORA para quien esta en turno, en vez de
    // perdedora)
49 // sobre montones pequeños, para confirmar que el caso
    // especial
50 // esta bien aplicado antes de confiar en la formula.
51 // 4. Un solo monton (k=1): si el monton tiene tamaño > 1,
    // gana el
52 // primer jugador (x != 0 siempre que el monton no sea 0);
    // si el
53 // monton es exactamente 1, hay 1 monton de tamaño 1 (
    // impar), así
54 // que primerJugadorGana() da false: el primer jugador
    // debe quitar
55 // esa ultima ficha y por regla misere pierde.
56 };

```

Ejemplo de funcionamiento:

```

1 MisereNim n1({3, 4, 5});
2 bool r1 = n1.primerJugadorGana();
3 // r1 = true
4 // (hay montones > 1, así que se aplica la regla normal: nim-
    // suma
5 // 3^4^5 = 2, distinta de 0, el primer jugador gana igual que
    // en Nim
6 // normal con estos mismos montones)
7
8 MisereNim n2({1, 1, 1});
9 bool r2 = n2.primerJugadorGana();
10 // r2 = false
11 // (todos los montones son de tamaño 1, hay 3 montones de 1 --
    // impar --
12 // así que el primer jugador PIERDE bajo regla misere)
13
14 MisereNim n3({1, 1});
15 bool r3 = n3.primerJugadorGana();
16 // r3 = true
17 // (2 montones de tamaño 1 -- par -- el primer jugador gana
    // bajo regla
18 // misere, aunque bajo Nim normal con estos mismos montones el
19 // resultado seria el mismo por coincidencia: 1^1=0, también
    // perdedora
20 // para el primero en Nim normal... hay que correrlo para
    // confirmar
21 // que no hay contradiccion entre ambas lecturas en este caso
    // particular)

```

Nota: el último ejemplo (n3) es intencionalmente delicado: con dos montones de tamaño 1, la nim-suma normal ya da 0 (perdedora bajo regla normal), pero la regla misère dice lo contrario para este mismo caso porque cae en el caso especial $m = 0$. Antes de confiar en cualquier intuición cruzada entre ambas reglas, conviene verificar numéricamente con *Análisis Retrógrado* sobre montones pequeños.

Regla práctica: lee siempre con extremo cuidado si el enunciado dice "el que no puede mover pierde" (Nim normal) o "el que hace el último movimiento pierde" (misère) — son reglas fácilmente confundibles en la traducción del problema. Si es misère, usa la nim-suma normal salvo en el caso especial de que todos los montones sean 0 o 1, donde la paridad de montones de tamaño 1 decide, invertida respecto a la intuición de Nim normal.

63 Juegos de Sustracción y Variantes Clásicas

63.1 Juego de Sustracción (Subtraction Game)

Idea: dos jugadores alternan turnos sobre una sola pila de n piedras. En cada turno, el jugador retira una cantidad s perteneciente a un conjunto **fijo** de movimientos permitidos S (por ejemplo $S = \{1, 2, 3\}$). Con la convención normal de juego, **pierde** quien no puede mover (la pila llega a un tamaño donde ningún $s \in S$ cabe). Se resuelve calculando el **número de Grundy** de cada tamaño de pila $0, 1, \dots, n$ con la recurrencia estándar de Sprague-Grundy:

$$g(0) = 0, \quad g(i) = \text{mex}\{g(i - s) : s \in S, s \leq i\}$$

donde mex (*minimum excludant*) es el menor entero no negativo que **no** aparece en el conjunto. Una posición i es **perdedora** para quien está por mover (posición P) si y solo si $g(i) = 0$; en cualquier otro caso es **ganadora** (posición N).

¿Por qué usarlo? Es el caso más simple de juego imparcial (*impartial game*: ambos jugadores tienen exactamente las mismas jugadas disponibles desde cualquier posición) y sirve de introducción a Sprague-Grundy antes de combinar varios montones (ver Suma de Juegos Independientes). Un caso particular vale la pena memorizar porque aparece muy seguido: con $S = \{1, \dots, k\}$ (se puede quitar cualquier cantidad de 1 a k), la posición n es **perdedora** si y solo si $n \bmod (k + 1) = 0$ —no hace falta ni correr la DP para ese caso, aunque el struct de abajo también lo resuelve como caso general.

Casos de uso:

- Determinar quién gana con n piedras y un conjunto de movimientos S dado explícitamente.
- Calcular los números de Grundy de **cada** tamaño de pila, necesarios como pieza para Suma de Juegos Independientes (varios montones independientes con el mismo S , o incluso con S distinto cada uno).
- Detectar el **período** de la secuencia de ganadoras/perdedoras cuando n es enorme: por el teorema de periodicidad de Sprague-Grundy, para cualquier S finito la secuencia de posiciones P/N es eventualmente periódica, así que basta calcular unos pocos miles de términos, detectar el período y extrapolar.
- Como caso base antes de agregar restricciones (ver Juego de Sustracción con Conjunto de Movimientos Prohibidos).

Complejidad: $O(n \cdot |S|)$ tiempo y $O(n)$ memoria para llenar la tabla hasta n ; cada consulta de ganador es $O(1)$ una vez construida.

```

1 struct SubtractionGame {
2     vector<int> S;           // conjunto de movimientos
3     vector<int> grundy;      // grundy[i] para i = 0..maxN
4     vector<bool> esGanadora; // esGanadora[i]: true si quien
5                             // esta
6                             // por MOVER con i piedras gana
7
8     // CREATE: calcula grundy[] y esGanadora[] para 0..maxN.
9     SubtractionGame(vector<int> S_, int maxN) : S(S_) {
10         grundy.assign(maxN + 1, 0);

```

```

10         esGanadora.assign(maxN + 1, false);
11
12         for (int n = 1; n <= maxN; n++) {
13             set<int> alcanzables;
14             for (int s : S)
15                 if (s <= n)
16                     alcanzables.insert(grundy[n - s]);
17
18             int mex = 0;
19             while (alcanzables.count(mex)) mex++;
20
21             grundy[n] = mex;
22             esGanadora[n] = (mex != 0);
23         }
24     } // O(maxN * |S|)
25
26     // READ: quien esta por mover con n piedras, gana?
27     bool ganaJugadorEnTurno(int n) {
28         return esGanadora[n];
29     } // O(1)
30 };

```

Ejemplo de funcionamiento:

```

1 SubtractionGame g({1, 2, 3}, 12);
2
3 for (int n = 0; n <= 12; n++) cout << g.grundy[n] << " ";
4 // 0 1 2 3 0 1 2 3 0 1 2 3 0
5 // (se repite cada 4 valores, confirmando n % (k+1) con k=3)
6
7 for (int n = 0; n <= 12; n++) cout << g.ganaJugadorEnTurno(n)
8 << " ";
9 // 0 1 1 1 0 1 1 1 0 1 1 1 0
10 // (pierde exactamente cuando n es multiplo de 4)

```

Nota: el número de Grundy **no** es solo "gana/pierde" —guarda **cuánto** vale esa posición para combinarla con otras (XOR de Grundy numbers en Suma de Juegos Independientes). Si el problema es de un único montón, basta con `esGanadora[n]`; si hay varios montones independientes (posiblemente con distinto S cada uno), hace falta `grundy[n]` completo para cada uno.

Regla práctica: antes de programar la DP, revisa si S es $\{1, \dots, k\}$ (un rango completo desde 1) —en ese caso la respuesta es literalmente $n \% (k+1) != 0$, sin código adicional. Para cualquier otro S , o si además se necesitan los números de Grundy para combinarlos con otros juegos, usa la DP completa.

63.2 Juego de Sustracción con Conjunto de Movimientos Prohibidos

Idea: variante donde, en vez de dar directamente el conjunto **permitido** S , el enunciado da un límite k (se puede quitar cualquier cantidad de 1 a k) y un conjunto $F \subseteq \{1, \dots, k\}$ de cantidades **prohibidas** (movimientos que el enunciado explícitamente no deja usar). El conjunto **efectivo** de movimientos permitidos es el complemento:

$$S_{\text{efectivo}} = \{1, \dots, k\} \setminus F$$

Una vez calculado S_{efectivo} , el resto es **exactamente** la misma DP de Grundy de Juego de Sustracción —esta subsección no introduce ningún algoritmo nuevo, solo el paso de **traducir** la descripción "todo permitido salvo estos" a un conjunto concreto antes de aplicar la técnica ya conocida.

¿Por qué usarlo? Es un recordatorio útil de que en teoría de juegos, como en varias partes de esta guía, conviene **reconocer la reformulación** antes de programar algo nuevo: describir las reglas "al revés" (por prohibición en vez de por permiso) no cambia la naturaleza combinatoria del juego, solo la forma en que hay que leerlo. El único cuidado real está en los **casos borde** de esta traducción: si F termina cubriendo **todo** el rango

$\{1, \dots, k\}$, el conjunto efectivo queda **vacío**, y entonces **nadie** puede mover nunca — el primer jugador pierde de inmediato con cualquier $n \geq 1$, sin que la pila ni siquiera se reduzca.

Casos de uso:

- Cualquier enunciado que fije un tope de piedras por turno (1 a k) y liste algunas cantidades específicas que no se pueden usar (evitar ciertos números por regla del juego, por superstición del enunciado, etc.).
- Como ejercicio de traducción antes de aplicar Sprague-Grundy: practicar convertir "prohibido" en "permitido" sin errores de borde, algo que se repite en variantes más elaboradas de juegos combinatorios.
- Detectar de antemano, sin correr ninguna DP, si el juego es trivial porque el conjunto efectivo quedó vacío.

Complejidad: $O(k)$ para construir el conjunto efectivo, más $O(n \cdot |S_{\text{efectivo}}|)$ para la DP de Grundy — igual orden que Juego de Sustracción, solo con un paso de preprocesamiento extra.

```

1 struct SubtractionGameProhibido {
2     int k; // maximo de piedras removibles
3     vector<int> permitidos; // conjunto EFECTIVO (
4         complemento de F)
5     vector<int> Grundy;
6     vector<bool> esGanadora;
7
8     // CREATE: calcula el conjunto efectivo de movimientos a
9     // partir de
10    // k y del conjunto prohibido F, y corre la MISMA DP de
11    // Grundy que
12    // Juego de Sustraccion.
13    SubtractionGameProhibido(int k_, vector<int>& F, int maxN) :
14        k(k_) {
15        vector<bool> prohibido(k + 1, false);
16        for (int f : F)
17            if (f >= 1 && f <= k)
18                prohibido[f] = true;
19
20        for (int s = 1; s <= k; s++)
21            if (!prohibido[s])
22                permitidos.push_back(s);
23
24        Grundy.assign(maxN + 1, 0);
25        esGanadora.assign(maxN + 1, false);
26
27        for (int n = 1; n <= maxN; n++) {
28            set<int> alcanzables;
29            for (int s : permitidos)
30                if (s <= n)
31                    alcanzables.insert(Grundy[n - s]);
32
33            int mex = 0;
34            while (alcanzables.count(mex)) mex++;
35
36            Grundy[n] = mex;
37            esGanadora[n] = (mex != 0);
38        }
39    } // O(k + maxN * |
40        permitidos|)
41
42    // READ: quien esta por mover con n piedras, gana?
43    bool ganaJugadorEnTurno(int n) {
44        return esGanadora[n];
45    } // O(1)
46 };

```

Ejemplo de funcionamiento:

```

1 // k=5, prohibidos {2,4} -> permitidos efectivos = {1,3,5}
2 vector<int> F = {2, 4};
3 SubtractionGameProhibido g(5, F, 12);
4
5 cout << "permitidos: ";
6 for (int s : g.permitidos) cout << s << " ";
7 // permitidos: 1 3 5
8

```

```

9 for (int n = 0; n <= 12; n++) cout << g.Grundy[n] << " ";
10 // 0 1 0 1 0 1 0 1 0 1 0 1 0
11 // (identico a correr SubtractionGame con S={1,3,5} explicito)
12
13 for (int n = 0; n <= 12; n++) cout << g.ganaJugadorEnTurno(n)
14     << " ";
15 // 0 1 0 1 0 1 0 1 0 1 0 1 0
16
17 // caso borde: F cubre TODO {1,2,3} con k=3 -> permitidos vacio
18 vector<int> Ftodo = {1, 2, 3};
19 SubtractionGameProhibido gVacio(3, Ftodo, 5);
20
21 cout << gVacio.permitidos.size();
22 // 0
23
24 for (int n = 0; n <= 5; n++) cout << gVacio.ganaJugadorEnTurno(
25     n) << " ";
26 // 0 0 0 0 0 0
27 // (nadie puede mover NUNCA -> el que esta en turno siempre
28     pierde,
29     para CUALQUIER n >= 1)
30

```

Nota: el bucle `for (int s : permitidos) if (s <= n)` ya maneja automáticamente que `permitidos` esté vacío — el `set` `alcanzables` queda vacío, el `mex` de un conjunto vacío es 0, y por lo tanto `esGanadora[n] = false` para todo n sin necesitar un chequeo especial adicional. Vale la pena **verificar** este caso a mano una vez, porque es fácil asumir por error que "sin movimientos permitidos" debería dar algún tipo de error en vez de una respuesta válida del juego.

Regla práctica: cuando un enunciado de teoría de juegos describe las jugadas por **exclusión** ("puedes hacer cualquier cosa dentro de este rango, excepto...") en vez de por **inclusión** directa, el primer paso —antes de pensar en Grundy— es escribir explícitamente el conjunto efectivo de jugadas permitidas. Una vez que ese conjunto está en una variable concreta, el resto del problema es idéntico a Juego de Sustracción; no hace falta ninguna teoría adicional para la parte de "prohibido".

63.3 Staircase Nim

Idea: una escalera de n escalones (numerados 1 a n , con el escalón 0 siendo el suelo) tiene una cantidad de monedas en cada escalón. En cada turno, un jugador elige un escalón k con al menos una moneda y mueve **cualquier cantidad positiva** de esas monedas al escalón $k - 1$ (si $k = 1$, esas monedas caen al suelo y **salen del juego** para siempre). Pierde quien no puede mover (todas las monedas ya están en el suelo). El resultado clave: este juego es **equivalente** a un Nim clásico jugado únicamente sobre las monedas de los escalones **impares** (1, 3, 5, ...) — los escalones pares no importan para saber quién gana. La razón: cualquier movimiento desde un escalón **par** hacia uno impar puede ser **espejado** por el rival, moviendo esa misma cantidad del escalón impar recién aumentado hacia el siguiente escalón par (o fuera del tablero si era el escalón 1), devolviendo la posición efectiva de los escalones impares al estado anterior — así que mover piezas hacia un escalón par nunca cambia realmente el resultado. El ganador se decide con el mismo criterio que Nim Clásico:

gana quien está por mover $\iff a_1 \oplus a_3 \oplus a_5 \oplus \dots \neq 0$

¿Por qué usarlo? Es el primer ejemplo de esta guía donde un juego con reglas de movimiento **distintas** a Nim (mover entre escalones, no quitar piedras de montones) termina siendo, combinatoriamente, **el mismo**

juego que Nim Clásico sobre un subconjunto de la información dada — reconocer esa reducción es más valioso que memorizar la prueba completa. Es también el ejemplo introductorio de un patrón que reaparece en juegos combinatorios más elaborados: "algunas partes del estado no afectan el resultado, solo las que sobreviven a un argumento de espejado/paridad".

Casos de uso:

- Determinar el ganador de la escalera de monedas tal cual (el problema base).
- Cualquier variante narrativa equivalente: fichas que bajan de nivel en nivel, recursos que se transfieren escalón por escalón hacia una salida — el criterio de "solo importan las posiciones impares, mismo criterio que Nim" se mantiene mientras la regla de movimiento sea "de un nivel a uno estrictamente inferior, en cualquier cantidad".
- Como ejercicio de reconocimiento: antes de intentar un análisis de Grundy completo (con estados de tamaño exponencial), conviene sospechar una reducción a Nim cuando el juego tiene **niveles** y las piezas solo se mueven hacia niveles más bajos.

Complejidad: $O(n)$ — un solo recorrido para hacer XOR de las posiciones impares.

```

1 struct StaircaseNim {
2     vector<long long> escalones; // escalones[0] = escalon 1,
3     // escalones[1] = escalon 2, etc.
4     // (0-indeizado en el vector,
5     // pero 1-indeizado como escalera)
6     int n;
7
8     // CREATE: guarda la cantidad de monedas en cada escalon.
9     StaircaseNim(vector<long long>& escalones_) {
10         escalones = escalones_;
11         n = escalones.size();
12     } // O(1)
13
14     // READ: gana quien esta por mover? Solo cuentan los
15     // escalones de
16     // posicion IMPAR (1, 3, 5, ... en la numeracion de 1 a n).
17     bool ganaJugadorEnTurno() {
18         long long xorImpares = 0;
19
20         for (int i = 0; i < n; i++) {
21             int numeroDeEscalon = i + 1; // convierte a 1-indeizado
22             if (numeroDeEscalon % 2 == 1)
23                 xorImpares ^= escalones[i];
24         }
25
26         return xorImpares != 0; // O(n)
27     };

```

Ejemplo de funcionamiento:

```

1 // escalera de 4 escalones: [escalon1, escalon2, escalon3,
2 // escalon4]
3 vector<long long> escalones = {3, 5, 2, 7};
4
5 StaircaseNim sn(escalones);
6
7 cout << sn.ganaJugadorEnTurno();
8 // 1 (true)
9 // escalones impares: 1 y 3, con 3 y 2 monedas -> 3 XOR 2 = 1,
10 // != 0
11 // (los escalones pares, 5 y 7 monedas, NO afectan el resultado)

```

Nota: el struct de arriba **no** necesita simular ningún movimiento — a diferencia de un análisis de Grundy genérico sobre todo el espacio de estados (que sería computacionalmente inviable, ya que el número de configuraciones crece exponencialmente con la cantidad de monedas), la reducción a Nim resuelve el juego completo

en una sola pasada lineal. Esta es la ventaja real de identificar la equivalencia antes de programar cualquier DP.

Regla práctica: en cualquier juego donde las piezas se mueven **siempre hacia niveles inferiores** de una estructura escalonada (nunca hacia arriba, nunca a un nivel arbitrario), sospecha inmediatamente de una reducción a Nim sobre un subconjunto de niveles — el patrón de "el rival puede espejar cualquier movimiento hacia un nivel que no cuenta, devolviéndolo a un nivel que sí cuenta" es la prueba general detrás de Staircase Nim y se adapta a variantes similares.

63.4 Wythoff's Game

Idea: dos montones de fichas, de tamaños a y b . En cada turno, un jugador puede: (1) quitar cualquier cantidad positiva de fichas de **un solo** montón (como en Nim con 2 montones), o (2) quitar la **misma** cantidad positiva de **ambos** montones a la vez (movimiento extra que Nim **no** permite). Pierde quien no puede mover (ambos montones vacíos). La tercera regla es lo que rompe la simple fórmula de Nim ($a \oplus b \neq 0$) y exige un análisis propio: las posiciones **perdedoras** (posición P , pierde quien está por mover) resultan ser **exactamente** los pares $(\lfloor k\varphi \rfloor, \lfloor k\varphi \rfloor + k)$ para $k = 0, 1, 2, \dots$, donde $\varphi = \frac{1+\sqrt{5}}{2}$ es la **razón áurea** — un resultado clásico de teoría de números (sucesiones de Beatty) aplicado a teoría de juegos.

¿Por qué usarlo? Es el ejemplo donde la razón áurea aparece en un contexto puramente combinatorio, sin nada geométrico de por medio — vale la pena como muestra de que las posiciones perdedoras de un juego no siempre siguen un patrón simple de paridad o XOR, a veces la estructura subyacente es aritmética/número-teórica. En la práctica, la fórmula da una consulta $O(1)$: dado (a, b) , se calcula directamente si corresponde a alguna k de la sucesión, sin necesitar ninguna tabla ni DP.

Casos de uso:

- Determinar el ganador de Wythoff's Game dado (a, b) directamente, sin simular ni memoizar.
- Generar las primeras K posiciones perdedoras (para listarlas, o para usarlas como tabla de consulta si se prefiere evitar aritmética de punto flotante).
- Como ejemplo de que agregar **una sola** regla extra a un juego de tipo Nim (quitar lo mismo de ambos montones) puede cambiar por completo la estructura de las posiciones ganadoras, de "XOR simple" a "sucesión de Beatty con la razón áurea".

Complejidad: $O(1)$ por consulta usando la fórmula cerrada (con cuidado de precisión en φ); $O(K)$ para generar las primeras K posiciones perdedoras.

```

1 struct WythoffGame {
2     static constexpr double PHI = 1.6180339887498949; // (1+
3     // sqrt(5))/2
4
5     // READ: gana quien esta por mover con montones (a, b)?
6     static bool ganaJugadorEnTurno(long long a, long long b) {
7         if (a > b) swap(a, b); // normaliza: a <= b
8
9         long long k = (long long)((double)a / PHI + 0.5);

```

```

9 // ^ si (a,b) es una posicion PERDEDORA, existe un k tal
10 // que a == floor(k*phi); se despeja k = a / phi y se
11 // redondea
12
13 long long ak = (long long)floor(k * PHI);
14 long long bk = ak + k;
15
16 bool esPerdedora = (ak == a && bk == b);
17 return !esPerdedora;
18 } // O(1)
19
20 // READ: genera las primeras cantidad posiciones PERDEDORAS
21 // (a_k, b_k) para k = 0, 1, ..., cantidad-1.
22 static vector<pair<long long, long long>> primerasPerdedoras(
23     int cantidad) {
24     vector<pair<long long, long long>> resultado;
25
26     for (int k = 0; k < cantidad; k++) {
27         long long ak = (long long)floor(k * PHI);
28         long long bk = ak + k;
29         resultado.push_back({ak, bk});
30     }
31     return resultado;
32 } // O(cantidad)

```

Ejemplo de funcionamiento:

```

1 cout << WythoffGame::ganaJugadorEnTurno(1, 2);
2 // 0 (false) -- (1,2) es una posicion PERDEDORA (k=1: floor(1*
3 // phi)=1, 1+1=2)
4
5 cout << WythoffGame::ganaJugadorEnTurno(3, 5);
6 // 0 (false) -- tambien perdedora (k=2: floor(2*phi)=3, 3+2=5)
7
8 cout << WythoffGame::ganaJugadorEnTurno(2, 4);
9 // 1 (true) -- (2,4) NO esta en la lista de perdedoras
10
11 auto perdedoras = WythoffGame::primerasPerdedoras(6);
12 for (auto& [x, y] : perdedoras) cout << "(" << x << ", " << y <<
13 // (0,0) (1,2) (3,5) (4,7) (6,10) (8,13)

```

Nota: el uso de `double` para φ es seguro para valores de a, b razonables ($\lesssim 10^{15}$ con `double` estándar), pero para valores extremadamente grandes conviene usar `long double` o verificar el resultado con aritmética exacta (reconstruyendo k y comprobando $\lfloor k\varphi \rfloor$ con más precisión) para evitar errores de redondeo cerca del límite de representación.

Regla práctica: si un juego "se parece a Nim de 2 montones" pero permite **además** quitar la misma cantidad de ambos a la vez, ya no es Nim —es Wythoff's Game, y la condición de victoria deja de ser un XOR simple para convertirse en esta fórmula basada en la razón áurea. Verificar contra la tabla de las primeras posiciones perdedoras (calculada por fuerza bruta con memoización, para a, b chicos) es la forma más segura de confirmar que la fórmula se está aplicando bien antes de confiar en ella para valores grandes.

63.5 Turning Games (Volteo de Monedas / Fichas)

Idea: una fila de n fichas, cada una mostrando **cara** o **cruz**. En cada turno se elige una ficha en **cara** y se voltea **obligatoriamente** a cruz; además, **opcionalmente**, se puede voltear (en cualquier sentido, cara $\rightarrow$ cruz o cruz $\rightarrow$ cara) **una única** ficha ubicada estrictamente a la **izquierda** de la elegida. Pierde quien no tiene ninguna cara para voltear. Estos juegos tienen un teorema notable (de la teoría de *coin-turning games*, Berlekamp-Conway-Guy): el valor de Grundy de **todo el tablero** es el **XOR** de un valor de Grundy que se calcula **por separado**

para cada ficha en cara, como si esa ficha fuera la **única** en el tablero (con todo lo demás en cruz):

$$g(\text{tablero}) = \bigoplus_{i: \text{ficha } i \text{ en cara}} g_1(i)$$

donde $g_1(i)$ es el valor de Grundy de un tablero con una sola cara en la posición i . Esto convierte un juego sobre un espacio de estados **exponencial** (2^n configuraciones) en n cálculos **independientes** de una sola ficha, combinados con XOR —exactamente el mismo principio que Suma de Juegos Independientes, aplicado aquí **dentro** de un solo tablero en vez de entre tableros distintos.

¿Por qué usarlo? Es la generalización de Staircase Nim a un marco mucho más amplio: ahí la reducción a Nim salía de un argumento de espejado específico a escalones; aquí es un **teorema general** que aplica a toda una familia de juegos de volteo, siempre que la regla cumpla dos condiciones: (1) cada movimiento voltea **como mínimo** una cara a cruz en la posición elegida, y (2) cualquier otro volteo permitido ocurre **estrictamente a la izquierda** de esa posición. Bajo esas condiciones, el juego siempre se descompone en XOR de valores independientes por posición, sin importar el detalle exacto de qué más se puede voltear. Para la regla descrita arriba (una ficha extra libre, en cualquier sentido, en cualquier posición a la izquierda), el resultado es particularmente limpio: $g_1(i) = i + 1$ (con i 0-indexado) —es decir, el juego es **literalmente** Nim Clásico donde cada cara aporta un montón de tamaño igual a su posición (1-indexada).

Casos de uso:

- Determinar el ganador de cualquier juego de volteo de fichas en fila con la regla "obligatorio voltear una cara a cruz, opcional voltear una ficha más a la izquierda" — se reduce a Nim sobre las posiciones (1-indexadas) de las caras.
- Variantes con **otra** regla de volteo (voltear dos fichas a la izquierda, restringir el volteo extra a solo cruz $\rightarrow$ cara, etc.): el teorema de descomposición sigue aplicando, solo cambia la fórmula recursiva de $g_1(i)$ (ver modificaciones).
- Como plantilla para **descubrir** la fórmula de g_1 de una regla nueva: computar por fuerza bruta los primeros valores de $g_1(0), g_1(1), \dots$ con la recurrencia genérica de Grundy sobre un solo tablero, buscar el patrón, y luego usar ese patrón para tableros grandes sin necesitar simular el espacio completo de estados.
- Verificar la propia descomposición contra un cálculo de Grundy por fuerza bruta (bitmask + memoización) en tableros chicos, antes de confiar en la fórmula para tableros grandes.

Complejidad: $O(n)$ por consulta una vez conocida la fórmula o tabla de g_1 (calcular el XOR de las posiciones de las caras); $O(2^n \cdot n^2)$ para el cálculo de referencia por fuerza bruta sobre todo el espacio de estados, usado solo para **verificar** la descomposición en tableros pequeños.

```

1 struct TurningGame {
2     // READ: gana quien esta por mover, dado el tablero como
3     // bitmask
4     // (bit i = 1 si la ficha en la posicion i, 0-indexada desde
5     // la
6     // izquierda, esta en CARA). Regla: voltea una cara a cruz
7     // (obligatorio) y, opcionalmente, voltea UNA ficha mas en
8     // cualquier sentido en una posicion ESTRICTAMENTE a la
9     // izquierda.
10    // Usa la reduccion a Nim: cada cara en posicion i (0-
11    // indexada)
12    // aporta un monon de tamano i+1.
13    static bool ganaJugadorEnTurno(int tablero, int n) {
14        long long xorTotal = 0;
15
16        for (int i = 0; i < n; i++)
17            if ((tablero >> i) & 1)
18                xorTotal ^= (i + 1); // g1(i) = i + 1 (0-indexada)
19
20        return xorTotal != 0;
21    } // O(n)
22
23    // -----
24    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
25    // -----
26
27    // 1. VERIFICACION POR FUERZA BRUTA (solo para n chico, <=
28    // ~18):
29    // calcula el Grundy de TODO el tablero simulando el juego
30    // completo, para confirmar que coincide con el XOR de g1(
31    // i).
32    // Sirve para descubrir/confirmar la formula de g1 en una
33    // regla
34    // NUEVA antes de confiar en ella:
35    // unordered_map<int,int> memo;
36    // int n;
37    // int grundyCompleto(int mask) {
38    //     if (memo.count(mask)) return memo[mask];
39    //     set<int> alcanzables;
40    //     for (int i = 0; i < n; i++) {
41    //         if (!(mask >> i) & 1) continue;
42    //         int base = mask & ~(1 << i); // i:
43    //         cara->cruz
44    //         alcanzables.insert(grundyCompleto(base));
45    //         for (int j = 0; j < i; j++) //
46    //             alcanzables.insert(grundyCompleto(base
47    //             ^ (1 << j)));
48    //         int mex = 0;
49    //         while (alcanzables.count(mex)) mex++;
50    //         return memo[mask] = mex;
51    //     }
52    //     // g1(i) = grundyCompleto(1 << i) con memo limpio por
53    //     // cada i
54
55    // 2. VOLTEO EXTRA RESTRINGIDO A CRUZ->CARA (no se permite
56    // apagar
57    // otra cara, solo "encender" una cruz): cambia el
58    // conjunto de
59    // movimientos disponibles en la posicion j < i, y por lo
60    // tanto
61    // cambia g1(i) -- hay que recalcularlo con el metodo de
62    // la
63    // modificacion 1 (probablemente ya NO sea i+1 con esta
64    // regla
65    // mas restringida).
66
67    // 3. SE PUEDEN VOLTEAR HASTA DOS FICHAS A LA IZQUIERDA (en
68    // vez de
69    // a lo sumo una): el teorema de descomposicion en XOR
70    // SIGUE
71    // aplicando (la condicion clave -- todo volteo extra
72    // ocurre
73    // estrictamente a la izquierda de la posicion obligatoria
74    // -- no
75    // cambia, pero la recurrence de g1(i) debe considerar
76    // todas
77    // las combinaciones de 0, 1 o 2 fichas volteadas a la
78    // izquierda antes de tomar el mex.
79
80    // 4. TABLERO CIRCULAR O SIN ORDEN LINEAL CLARO: el teorema
81    // de
82    // descomposicion DEPENDE de que exista una nocion de
83    // "izquierda" que garantice que el juego siempre termina
84    // (cada
85    // movimiento reduce alguna medida, como la posicion de la
86    // ficha volteada obligatoriamente). Si el tablero es
87    // circular o
88    // la regla permite voltear hacia la derecha, hay que

```

```

58 verificar
59 // primero que el juego SIGA siendo finito antes de
60 // aplicar nada
61 // de esto.
62
63 // 5. IDENTIFICAR RAPIDO SI UN ENUNCIADO ES UN "TURNING GAME":
64 // la
65 // señal es "fichas o monedas en fila, cara/cruz, cada
66 // movimiento
67 // apaga una cara y opcionalmente cambia algo a su
68 // izquierda".
69 // Ante esa estructura, el primer paso NO es simular con
70 // bitmask para n grande -- es calcular g1 con fuerza
71 // brutal
72 // para n chico, buscar el patron, y aplicar la formula
73 // encontrada como en el ejemplo de esta subseccion.
74 };

```

Ejemplo de funcionamiento:

```

1 // tablero de 6 fichas, 0-indexado desde la izquierda:
2 // posiciones con CARA: 1, 3, 4 (bits 1, 3 y 4 encendidos)
3 int tablero = (1 << 1) | (1 << 3) | (1 << 4);
4
5 cout << TurningGame::ganaJugadorEnTurno(tablero, 6);
6 // 1 (true)
7 // aportes: cara en 1 -> monon 2; cara en 3 -> monon 4;
8 // cara en 4 -> monon 5
9 // XOR: 2 ^ 4 ^ 5 = 3, distinto de 0 -> gana quien esta por
10 mover
11
12 // verificado por fuerza bruta: para TODOS los 64 tableros
13 // posibles
14 // de 6 fichas, el grundy calculado simulando el juego completo
15 // coincide exactamente con el XOR de (posicion + 1) de cada
16 // cara

```

Nota: la posición se cuenta **0-indexada** en el bitmask (bit 0 = ficha más a la izquierda), pero el "tamaño de montón" que aporta cada cara es $i + 1$ (equivalente a numerar las posiciones desde 1). Es un detalle de conteo fácil de invertir por error —conviene fijar una convención (0-indexada en el código, 1-indexada en la fórmula) y no mezclarlas.

Regla práctica: ante cualquier juego de fichas cara/cruz en fila donde cada movimiento apaga una cara y opcionalmente modifica algo estrictamente a su izquierda, no analices el tablero completo como un solo juego monolítico —calcula (por fuerza bruta, para n chico) el valor de Grundy de **una sola** cara en cada posición, confirma que el patrón se sostiene, y combina con XOR. La regla exacta de qué se puede voltear a la izquierda cambia la **fórmula** de g_1 , pero prácticamente nunca cambia el hecho de que la descomposición en XOR funciona.

64 Juegos sobre Estructuras de Datos

64.1 Juegos sobre Grafos y DAGs (Movimiento de una Ficha)

Idea: una ficha está en un vértice de un **DAG** (grafo dirigido acíclico). En cada turno, un jugador mueve la ficha a lo largo de **una** arista saliente. Pierde quien no tiene ninguna arista disponible (la ficha está en un vértice sin salidas). Es el mismo cálculo de Grundy que ya se usa en otras partes de esta guía (ver Cálculo de Grundy sobre un DAG), aplicado aquí como **modelo general**: $g(v) = \text{mex}\{g(u) : (v, u) \in E\}$, calculado con DFS memoizado (o en orden topológico inverso). El jugador en turno gana con la ficha en v si y solo si $g(v) \neq 0$. Este mismo esquema es, de hecho, la **representación universal** de cualquier juego imparcial: todo juego imparcial finito y sin empates es, en esencia, un

DAG de posiciones con este mismo cálculo de Grundy — Nim, Subtraction Games, Staircase Nim y Turning Games son casos particulares donde el DAG tiene una estructura especial que permite una fórmula cerrada en vez de recorrer el grafo explícitamente.

¿Por qué usarlo? Cuando un juego **no** encaja en ningún patrón con fórmula cerrada, pero sus posiciones y transiciones se pueden modelar como un DAG explícito (con V y E manejables), el cálculo de Grundy directo sobre ese DAG siempre funciona —es el método "de último recurso" antes de rendirse ante un juego sin estructura evidente. Además, con **varias fichas** en el mismo DAG o en DAGs distintos, moviendo **una sola** por turno, el juego total es la **suma** de juegos independientes de Sprague-Grundy: el Grundy total es el XOR de los Grundy de cada ficha por separado, exactamente igual que combinar montones de Nim.

Casos de uso:

- Cualquier juego descrito como "una ficha se mueve por un grafo/tablero siguiendo reglas de movimiento fijas, sin poder repetir posiciones" (la ausencia de ciclos es la que garantiza que el juego termine).
- Varias fichas en el mismo grafo (o en grafos distintos), donde cada turno se mueve exactamente una: se resuelve calculando el Grundy de la posición de **cada** ficha por separado y combinando con XOR —no hace falta un estado conjunto exponencial.
- Como técnica de **traducción**: cuando un juego nuevo no encaja obviamente en Nim/Sustracción/Staircase/Turning, modelar sus posiciones explícitamente como nodos de un DAG y aplicar esta fórmula genérica siempre es una opción válida, aunque menos elegante que una fórmula cerrada.
- Advertencia importante**: si el grafo es **no dirigido** y el juego consiste en mover la ficha y **borrar** el vértice/arista visitado (*Generalized Geography*), el estado ya **no** es solo "dónde está la ficha" —el grafo cambia con cada movimiento, y este cálculo simple de Grundy **no aplica**: ese juego es, en general, computacionalmente mucho más difícil (PSPACE-completo), y requiere memoizar sobre el grafo restante, no solo sobre el vértice actual.

Complejidad: $O(V + E)$ para calcular el Grundy de todos los vértices de un DAG (cada vértice se calcula una sola vez con memoización); $O(k)$ por consulta de k fichas independientes, una vez calculada la tabla de Grundy.

```
1 struct JuegoFichaDAG {
2     vector<vector<int>>> adj; // adj[v] = vecinos alcanzables
3     // desde v
4     vector<int> grundy;
5     vector<bool> calculado;
6     int n;
7
8     // CREATE: guarda el DAG (debe ser ACICLICO; con ciclos, la
9     // memoización de abajo no termina).
10    JuegoFichaDAG(vector<vector<int>>>& adj_, int n_) {
11        adj = adj_;
12        n = n_;
13        grundy.assign(n, -1);
14        calculado.assign(n, false);
15    }
16    // O(1)
```

```
16 // READ: valor de Grundy del vertice v (memoizado).
17 int calcular(int v) {
18     if (calculado[v]) return grundy[v];
19     calculado[v] = true; // se marca ANTES de recursar: en un
20     // DAG no hay ciclos, así que esto es
21     // seguro y evita recalcular
22
23     set<int> alcanzables;
24     for (int u : adj[v])
25         alcanzables.insert(calcular(u));
26
27     int mex = 0;
28     while (alcanzables.count(mex)) mex++;
29
30     return grundy[v] = mex;
31 } // O(1) amortizado
32 // O(V+E) la primera vez
33
34 // READ: con k fichas en las posiciones dadas (una por turno
35 // se mueve), gana quien esta en turno?
36 bool ganaConFichas(vector<int>& posiciones) {
37     int xorTotal = 0;
38     for (int p : posiciones)
39         xorTotal ^= calcular(p);
40     return xorTotal != 0;
41 } // O(k)
42 };
```

Ejemplo de funcionamiento:

```
1 // DAG: 0 -> 1, 0 -> 2, 1 -> 3, 2 -> 3 (3 es sumidero, sin
2 // salidas)
3 vector<vector<int>>> adj(4);
4 adj[0] = {1, 2};
5 adj[1] = {3};
6 adj[2] = {3};
7 adj[3] = {};
8
9 JuegoFichaDAG jf(adj, 4);
10 jf.calcular(0); jf.calcular(1); jf.calcular(2); jf.calcular(3);
11
12 for (int v = 0; v < 4; v++) cout << "g(" << v << ")=" << jf.
13     grundy[v] << " ";
14 // g(0)=0 g(1)=1 g(2)=1 g(3)=0
15
16 vector<int> fichas = {0, 1};
17 cout << jf.ganaConFichas(fichas);
18 // 1 (true) -- g(0) XOR g(1) = 0 XOR 1 = 1, distinto de 0
```

Nota: verificado por fuerza bruta (minimax directo sobre el estado conjunto de varias fichas) en un DAG aleatorio de 8 nodos: el resultado de XOR de Grundy individuales coincide exactamente con la simulación completa para 1, 2 y 3 fichas simultáneas — confirma que la suma de juegos independientes aplica sin ninguna sorpresa cuando el grafo mismo no cambia entre turnos.

Regla práctica: este método es el "más general pero más costoso" de toda la sección de Game Theory —úsalo cuando el juego no tenga una estructura reconocible (Nim, Sustracción, Staircase, Turning) pero sí se pueda modelar como un DAG explícito de tamaño manejable. Antes de aplicarlo, confirma que el grafo realmente **no cambia** durante la partida (solo la posición de la ficha cambia) — si el grafo se modifica con cada movimiento, este cálculo no es válido y hace falta memoizar sobre el estado completo del grafo restante.

64.2 Juegos sobre Árboles (Green Hackenbush / Poda de Ramas)

Idea: un árbol de "ramas" (aristas) cuelga desde el **suelo** (la raíz). En cada turno, un jugador **corta** una arista cualquiera; esa arista desaparece, y con ella **toda** la parte del árbol que quedaba **colgando** de su extremo inferior (al perder la conexión con el suelo, esa subrama "cae" y sale del juego de inmediato, en el mismo movimiento). Pierde quien no tiene ninguna arista para cortar. El espacio de estados de este juego es **exponencial** (cada

subconjunto de aristas restantes es un estado distinto), pero existe una fórmula recursiva —el **Colon Principle**— que evita simularlo: para cada arista e que conecta un vértice v con su padre, sea $c_1, \dots, c_k$ las aristas que cuelgan **directamente** del extremo inferior de e (las aristas hacia los hijos de v); el valor de Grundy de e es

$$g(e) = 1 + (g(c_1) \oplus g(c_2) \oplus \dots \oplus g(c_k))$$

con $g(e) = 1$ si e es una hoja (sin hijos, la suma XOR vacía es 0). El Grundy de **todo** el árbol (o bosque, si hay varias ramas que salen directo del suelo) es el XOR de los g de esas ramas raíz.

¿Por qué usarlo? Es la aplicación más elegante de Sprague-Grundy de esta guía: convierte un juego con estados exponenciales en un simple recorrido **post-orden** del árbol, $O(\text{aristas})$ total. La fórmula tiene una lectura intuitiva: un **tallo** simple (un camino sin ramificaciones, de L aristas colgando en línea recta) tiene valor de Grundy exactamente L —cada arista adicional en la cadena suma 1 al valor anterior, ya que no hay ramificación que combinar (el XOR de un solo valor es ese mismo valor). La ramificación es lo que introduce el XOR: cuando un vértice tiene varias subramas, primero se combinan **esas** subramas entre sí (como si fueran montones de Nim independientes), y **después** se suma 1 por la arista que las conecta hacia arriba.

Casos de uso:

- Green Hackenbush jugado sobre un árbol o un bosque de árboles (todas las ramas son "neutrales", cualquier jugador puede cortar cualquier arista —a diferencia de Red-Blue Hackenbush, donde cada rama pertenece a un jugador, fuera del alcance de esta guía).
- Cualquier juego de "poda de ramas" donde cortar una conexión elimina automáticamente todo lo que queda aislado —el mismo principio aplica sin importar el nombre narrativo (árboles reales, jerarquías, estructuras de dependencia que se desmoronan).
- Calcular el valor de Grundy de un bosque completo sumando (XOR) los valores de cada árbol individual —cada árbol es un juego independiente en el sentido de Sprague-Grundy.
- Como ejemplo de "colapsar una estructura compleja a un solo número" antes de decidir quién gana, reforzando la idea general de Sprague-Grundy de que **cualquier** juego imparcial se reduce, tarde o temprano, a un solo entero no negativo.

Complejidad: $O(V)$ — un recorrido post-orden del árbol, calculando el Grundy de cada arista una sola vez a partir de sus hijas ya calculadas.

```
1 struct GreenHackenbushArbol {
2     vector<vector<int>> hijosAristas; // hijosAristas[e] =
3     // que cuelgan directamente
4     // del extremo inferior de e
5     vector<int> raicesAristas; // aristas que salen
6     // directo
7     // del suelo (raíz del bosque)
8
9     // CREATE: guarda la estructura de aristas del árbol/bosque,
10    // ya
11    // representada como "que aristas cuelgan de cada arista".
```

```
10 GreenHackenbushArbol(vector<vector<int>>& hijosAristas_,
11 vector<int>& raicesAristas_) {
12     hijosAristas = hijosAristas_;
13     raicesAristas = raicesAristas_;
14 } // O(1)
15
16 // READ: valor de Grundy de la arista e (Colon Principle):
17 // combina las aristas hijas con XOR, y suma 1 por la propia
18 // arista.
19 int colon(int e) {
20     int x = 0;
21     for (int c : hijosAristas[e])
22         x ^= colon(c);
23     return x + 1;
24 } // O(aristas del
25 // subarbol)
26
27 // READ: gana quien esta por mover, sobre TODO el bosque?
28 bool ganaJugadorEnTurno() {
29     int total = 0;
30     for (int r : raicesAristas)
31         total ^= colon(r);
32     return total != 0;
33 } // O(V)
```

Ejemplo de funcionamiento:

```
1 // arbol colgando del suelo (nodo 0):
2 //      0 (suelo)
3 //      / \
4 //     1  3
5 //    / \
6 //   2 4 5
7 //
8 // aristas: 0=(0-1), 1=(1-2), 2=(0-3), 3=(3-4), 4=(3-5)
9
10 vector<vector<int>> hijosAristas(5);
11 hijosAristas[0] = {1}; // la arista 0-1 tiene colgando la
12 // 1-2
13 hijosAristas[1] = {}; // hoja
14 hijosAristas[2] = {3, 4}; // la arista 0-3 tiene colgando
15 // 3-4 y 3-5
16 hijosAristas[3] = {}; // hoja
17 hijosAristas[4] = {}; // hoja
18
19 vector<int> raices = {0, 2}; // ambas ramas salen directo del
20 // suelo
21
22 GreenHackenbushArbol gh(hijosAristas, raices);
23
24 cout << gh.colon(0);
25 // 2 (tallo de 2 aristas: arista 1 vale 1, arista 0 = 1 + 1 =
26 // 2)
27
28 cout << gh.colon(2);
29 // 1 (arista 3 vale 1, arista 4 vale 1; XOR = 0; arista 2 = 0
30 // + 1 = 1)
31
32 cout << gh.ganaJugadorEnTurno();
33 // 1 (true) -- total = colon(0) XOR colon(2) = 2 XOR 1 = 3,
34 // distinto de 0
```

Nota: verificado con fuerza bruta —simulando el juego completo sobre un bitmask de aristas (donde cortar una arista borra también, de una sola vez, todas las aristas de su subárbol) en 200 árboles aleatorios de hasta 9 aristas, el Colon Principle coincidió exactamente con el valor de Grundy calculado simulando **todo** el espacio de estados. El caso de un solo tallo (sin ramificación) es el que confirma la intuición del +1: cada eslabón de la cadena extiende el valor en una unidad, exactamente como un montón de Nim que crece de a uno.

Regla práctica: para cualquier variante de Green Hackenbush sobre **árboles** (nunca sobre grafos con ciclos —ahí el juego se vuelve mucho más complejo y esta fórmula no aplica), calcula el Colon Principle con un solo recorrido post-orden: combina las ramas hijas con XOR, suma 1 por la arista que las conecta hacia arriba, y combina las raíces del bosque con otro XOR final. Nunca simules el espacio de estados completo salvo para **verificar** la fórmula en árboles de prueba pequeños.

64.3 Juegos de Tomar de los Extremos de un Arreglo (DP de Intervalos)

Idea: dos jugadores se turnan tomando el elemento del **extremo izquierdo o derecho** de un arreglo, sumando su valor al propio puntaje; el arreglo se agota y gana quien termine con más puntos. A diferencia de todos los juegos de *Fundamentos* (Nim, Grundy, Win/Lose), aquí **no hay un “ganador” binario definido por quién mueve al final** — es un juego con **puntaje**, donde ambos jugadores tienen exactamente las mismas opciones de movimiento (tomar izquierda o derecha) y solo difiere lo que cada uno se queda. Por eso no se resuelve con Sprague-Grundy (que exige un criterio de victoria normal-play), sino con **minimax**: como todo lo que un jugador no toma queda para el rival, maximizar tu propio puntaje equivale a maximizar (*tu puntaje* — *el del rival*). Se define $dp[i][j]$ como esa diferencia óptima jugando solo sobre el subarreglo $a[i..j]$, con la recurrencia

$$dp[i][j] = \max(a[i] - dp[i+1][j], a[j] - dp[i][j-1]),$$

porque si tomas $a[i]$, el rival pasa a ser el que mueve primero en $[i+1, j]$ y su ventaja en ese subjuego se te resta.

¿Por qué usarlo? Es la generalización de *Turning Games* (esa trabaja con paridad de fichas; esta con valores arbitrarios) y el antecedente directo de *Juegos de Turnos con Puntaje* (*Optimal Strategy for a Game*) en *Juegos Partidistas y Combinatoria Avanzada* — mismo problema, distinto contexto de la guía. Se distingue de *Juegos sobre Árboles* (*Green Hackenbush*) en que ahí ambos jugadores tienen jugadas **distintas** (Izquierdo/Derecho) sobre una estructura que se desconecta permanentemente al remover una arista; aquí las jugadas son **simétricas** (ambos pueden tomar de cualquier extremo) y lo que cambia es el puntaje, no las reglas de movimiento.

Casos de uso:

- El clásico “Optimal Strategy for a Game” / “Predict the Winner”: determinar si el primer jugador puede ganar o empatar.
- Juegos de cartas en fila donde cada carta tiene un valor y solo se puede tomar una de las dos puntas de la fila.
- Calcular el puntaje exacto de cada jugador bajo juego óptimo, no solo quién gana (usando la suma total y la diferencia óptima).
- Variantes donde ambos extremos tienen distinto “precio” de oportunidad (agregar un costo o descuento al tomar de un lado).
- Como bloque base antes de generalizar a valores de juego partidista (Números Surreales, más adelante en la guía).

Complejidad: hay $O(n^2)$ subintervalos $[i, j]$ y cada uno se calcula en $O(1)$ a partir de dos subintervalos más cortos, para un total de $O(n^2)$ en tiempo y $O(n^2)$ en memoria si se guarda la tabla completa. Procesando por **longitud creciente** de intervalo (todos los de longitud 1, luego 2, etc.) se puede reducir la memoria a $O(n)$ con un arreglo que se sobrescribe, porque $dp[i][j]$ solo depende de intervalos de longitud largo — 1.

```

1 struct JuegoDeExtremos {
2     int n;
3     vector<long long> a;
4     vector<vector<long long>> dp; // dp[i][j] = diferencia
5         optima
6         // (turno - rival) jugando sobre a[i..j]
7
8     // CREATE: guarda los valores del arreglo.
9     JuegoDeExtremos(vector<long long> valores) {
10         a = valores;
11         n = (int)a.size();
12         dp.assign(n, vector<long long>(n, 0));
13     } // O(n^2)
14
15     // WRITE: llena dp por longitud de intervalo creciente. Caso
16     base:
17     // con un solo elemento, la diferencia es simplemente su
18     // valor.
19     void calcular() {
20         for (int i = 0; i < n; i++) dp[i][i] = a[i];
21         for (int largo = 2; largo <= n; largo++) {
22             for (int i = 0; i + largo - 1 < n; i++) {
23                 int j = i + largo - 1;
24                 dp[i][j] = max(a[i] - dp[i+1][j], a[j] - dp[i][j-1]);
25             }
26         } // O(n^2)
27
28     // READ: diferencia optima (turno - rival) sobre TODO el
29     // arreglo.
30     // Positiva: el primer jugador puede ganar (o empatar si es
31     // 0).
32     long long diferencia() const { return dp[0][n-1]; }
33
34     // READ: puntaje exacto de cada jugador bajo juego optimo,
35     // usando
36     // que puntaje1 + puntaje2 = suma total y puntaje1 - puntaje2
37     // = diferencia.
38     pair<long long, long long> puntajes() const {
39         long long total = 0; for (long long x : a) total += x;
40         long long diff = dp[0][n-1];
41         return {(total + diff) / 2, (total - diff) / 2};
42     } // O(n)
43 };
44
45 // -----
46 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
47 // -----
48
49 // 1. REDUCIR A O(n) DE MEMORIA: como dp[i][j] con largo L solo
50 // usa
51 // valores de largo L-1, basta un arreglo 1D que se
52 // recalcula por
53 // longitud, en vez de guardar la tabla n x n completa:
54 //
55 // vector<long long> dp(n);
56 // for (int i = 0; i < n; i++) dp[i] = a[i];
57 // for (int largo = 2; largo <= n; largo++)
58 //     for (int i = 0; i + largo - 1 < n; i++) {
59 //         int j = i + largo - 1;
60 //         dp[i] = max(a[i] - dp[i+1], a[j] - dp[i]); //
61 //         dp[i] viejo = dp[i][j-1]
62 //     }
63 // // el resultado queda en dp[0]
64 //
65 // Ojo con el orden: hay que leer dp[i] ANTES de
66 // sobrescribirlo, ya
67 // que en la formula original dp[i][j-1] usa el valor VIEJO
68 // de dp[i].
69
70 // 2. TRUCO DE PARIDAD (solo indica quien NO PIERDE, no el
71 // margen
72 // optimo): si n es PAR, el primer jugador puede garantizar
73 // puntaje1 >= puntaje2 tomando SIEMPRE todos los elementos
74 // de
75 // indice par o siempre todos los de indice impar (el que
76 // suma
77 // mas), porque esas dos clases nunca se mezclan sin
78 // importar lo
79 // que haga el rival. Esto es solo una COTA INFERIOR de la
80 // diferencia optima real, NO el resultado de dp[0][n-1]:
81 //
82 // long long pares = 0, impares = 0;
83 // for (int i = 0; i < n; i++) (i % 2 == 0 ? pares : impares
84 // ) += a[i];
85 // // el primer jugador nunca pierde si n es par; margen
86 // GARANTIZADO
87 // // (no necesariamente optimo) = max(pares, impares) - min
88 // (pares, impares)
89 //
90 // Sirve para responder rapido "¿puede el primer jugador NO
91 // PERDER?"
92 //
93 // sin correr el DP, pero si el enunciado pide el margen

```

```

74 // EXACTO o n
75 // es impar, hay que usar dp completo.
76 // 3. COSTO O BONUS POR EXTREMO: si tomar de la izquierda o la
77 // derecha
78 // tiene un costo/bonus adicional c_izq o c_der (por ejemplo,
79 // un
80 // "impuesto" fijo), se suma directamente dentro del max:
81 // dp[i][j] = max((a[i] - c_izq) - dp[i+1][j], (a[j] - c_der
82 // ) - dp[i][j-1]);
83 // 4. K TURNOS DESDE CADA EXTREMO (variante distinta, NO
84 // minimax):
85 // si en vez de un turno por elemento el problema pide "
86 // elige k
87 // elementos tomando solo de los extremos, maximiza la suma"
88 // (un
89 // solo jugador, no un juego de dos), se resuelve con dos
90 // punteros y
91 // prefijos: probar cuantos se toman de la izquierda (0..k)
92 // y el
93 // resto de la derecha, en O(k). Es un problema distinto -
94 // de
95 // optimizacion para un solo jugador, no un juego de dos
96 // jugadores.
97 // 5. TRAMPA: CONFUNDIR ESTE JUEGO CON UNO DE GANAR/PERDER.
98 // Aquí NO
99 // hay "el que mueve al final gana"; ambos maximizan su
100 // propio
101 // puntaje, y el resultado que importa es la diferencia
102 // final de
103 // puntajes (o simplemente quien saca mas). Aplicar Sprague-
104 // Grundy
105 // (pensado para juegos normal-play de "el ultimo en mover
106 // gana")
107 // no tiene sentido aquí: no hay estados terminales "P" o "N"
108 // , solo
109 // un valor numerico que se maximiza.
110 // 6. TRAMPA: dp[i][j-1] Y dp[i+1][j] SON DIFERENCIAS DEL RIVAL,
111 // NO DEL
112 // ARREGLO RESTANTE PARA TI. Al escribir a[i] - dp[i+1][j],
113 // el signo
114 // negativo representa que dp[i+1][j] es la ventaja del
115 // jugador que
116 // mueve DESPUES de ti sobre ese subarreglo (que ahora es tu
117 // rival);
118 // olvidar el signo negativo es el error mas comun al
119 // implementar
120 // esta recurrencia por primera vez.

```

Ejemplo de funcionamiento:

```

1 vector<long long> a = {1, 5, 2, 4, 6};
2 JuegoDeExtremos j(a);
3 j.calcular();
4
5 cout << j.diferencia(); // 0 (empate bajo juego
6 // optimo)
7 auto [p1, p2] = j.puntajes();
8 cout << p1 << " " << p2; // 9 9
9
10 // n par: el primer jugador puede garantizar no perder.
11 vector<long long> b = {8, 15, 3, 7};
12 JuegoDeExtremos jb(b);
13 jb.calcular();
14 cout << jb.diferencia(); // 11 (el margen OPTIMO,
15 // mayor que la garantia de paridad)

```

Nota: en el segundo ejemplo, el truco de paridad de la modificación 2 solo garantiza que el primer jugador **no pierde** (margen $\geq \max(\text{pares}, \text{impares}) - \min(\text{pares}, \text{impares}) = 11$ en este caso, que coincide, pero no siempre coincide con el óptimo real): con juego verdaderamente óptimo la diferencia puede ser **mayor** que esa garantía. Si el enunciado pide el margen exacto de puntos (no solo quién gana), siempre hay que correr el DP completo.

Regla práctica: usa esta interval DP en cuanto el problema describa “toma de un extremo u otro, maximizando tu propio puntaje” — es la firma exacta de este patrón. Si n es par y solo preguntan *si* el primer jugador puede evitar perder, el truco de paridad en $O(n)$ responde sin DP; para el margen exacto, o si n es impar,

usa $dp[i][j] = \max(a[i] - dp[i+1][j], a[j] - dp[i][j-1])$ recorriendo por longitud de intervalo creciente.

64.4 Juegos sobre Strings (Eliminar Substrings o Caracteres)

Idea: un juego sobre strings define un movimiento como **eliminar algo** de la cadena (un carácter, un par, un patrón) según una regla, y ambos jugadores alternan hasta que nadie pueda mover (*normal play*: el último en mover gana). El ingrediente específico de esta familia es que, al eliminar una porción del **medio** de la cadena, las dos partes que quedan se convierten en **dos subjuegos independientes** — exactamente como remover una arista en *Juegos sobre Árboles* (*Green Hackenbush*) desconecta el árbol en dos componentes que ya nunca vuelven a interactuar. Esa independencia es lo que permite aplicar directamente *Suma de Juegos Independientes* (*XOR de Grundy Numbers*): el valor de Grundy de la cadena completa es el **mex** (*Cálculo de Grundy sobre un DAG*) sobre todos los movimientos posibles, y el valor de cada movimiento es el **XOR** de los Grundy de las dos partes resultantes — $g(l, r) = \text{mex}_{\text{movimientos válidos}} (g(l, i) \oplus g(i', r))$.

¿Por qué usarlo? La independencia de las dos partes **no es automática** — es una propiedad del diseño del juego, y hay que verificarla antes de aplicar la suma de Grundy. Por ejemplo, si el juego dijera literalmente “elimina dos caracteres iguales adyacentes y **concatena** lo que queda” (como en el clásico problema de reducir un string), las dos partes **sí pueden volver a tocarse**: el carácter que quedó justo a la izquierda del hueco y el que quedó justo a la derecha ahora son vecinos, y si coinciden, se habilita un movimiento nuevo que **crusa la frontera** entre lo que se pensaba eran dos subjuegos separados — rompiendo por completo la descomposición en XOR. Por eso esta subsección trabaja con la variante donde las dos partes quedan **separadas para siempre** (sin concatenar): un diseño de juego perfectamente válido y el único que la suma de Grundy resuelve de forma directa; la variante con concatenación existe en la práctica, pero exige técnicas mucho más específicas caso por caso, no la suma general de Sprague-Grundy.

Casos de uso:

- Remover un par de caracteres adyacentes iguales, dejando las dos mitades como cadenas independientes que ya no se tocan.
- Remover un carácter específico (o que cumpla una condición), partiendo la cadena en el punto de la remoción.
- Juegos sobre secuencias de símbolos (no necesariamente texto: listas de colores, secuencias de eventos) con la misma estructura de partición.
- Verificar, antes de aplicar Grundy, si el movimiento del enunciado realmente separa las partes para siempre o si permite fusiones — la pregunta que decide si esta técnica aplica.
- Como puente hacia *Suma de Juegos Partidistas y Comparación de Valores*, más adelante en la guía, cuando el juego deja de ser imparcial.

Complejidad: hay $O(n^2)$ subintervalos $[l, r]$, y calcular $g(l, r)$ revisa hasta $O(n)$ movimientos posibles dentro del intervalo, cada uno con costo $O(1)$ dado que g de los subintervalos ya está memoizado; el `mex` sobre a lo sumo $O(n)$ valores candidatos cuesta $O(n)$ con un `set`. En total, $O(n^3)$ en tiempo y $O(n^2)$ en memoria para la memoización completa.

```

1 struct JuegoDeCadenas {
2     string s;
3     int n;
4     map<pair<int,int>, int> memo;    // Grundy de s[l..r], SIN
        fusion tras remover
5
6     // CREATE: guarda la cadena.
7     JuegoDeCadenas(const string& cadena) { s = cadena; n = (int)s
        .size(); }
8
9     // READ: valor de Grundy del subjuego sobre s[l..r]. Un
        movimiento
10    // valido elimina el par s[i], s[i+1] (iguales y adyacentes)
        y
11    // convierte s[l..i] y s[i+2..r] en DOS SUBJUEGOS
        INDEPENDIENTES
12    // (nunca se vuelven a tocar).
13    int Grundy(int l, int r) {
14        if (r - l < 2) return 0;          // 0 o 1 caracter:
        sin movimientos
15        auto key = make_pair(l, r);
16        auto it = memo.find(key);
17        if (it != memo.end()) return it->second;
18        set<int> alcanzables;
19        for (int i = l; i + 1 < r; i++) {
20            if (s[i] == s[i+1]) {
21                int izq = Grundy(l, i);
22                int der = Grundy(i+2, r);
23                alcanzables.insert(izq ^ der);    // Sprague-Grundy: XOR
        de partes independientes
24            }
25        }
26        int mex = 0;
27        while (alcanzables.count(mex)) mex++;
28        memo[key] = mex;
29        return mex;
30    }
        // O(n) por
        llamada, O(n^3) total
31
32    // READ: true si el primer jugador gana bajo juego optimo (
        normal play).
33    bool primeroGana() { return Grundy(0, n) != 0; }
34 };
35
36 // -----
37 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
38 // -----
39
40 // 1. VARIAS PARTIDAS INDEPENDIENTES A LA VEZ: si el problema
        da varias
41 // cadenas y el turno consiste en mover en UNA de ellas (
        regla
42 // normal de suma de juegos), el primer jugador gana si y
        solo si
43 // XOR de Grundy(0, len(t)) sobre todas las cadenas t es
        distinto de
44 // cero - exactamente Suma de Juegos Independientes aplicada
        aquí.
45
46 // 2. OTRA REGLA DE ELIMINACION: si el movimiento es "elimina
        un
47 // caracter que cumple una condicion P(c)" en vez de un par
        igual
48 // adyacente, el cambio es minimo:
49 // for (int i = l; i < r; i++)
50 //     if (P(s[i])) alcanzables.insert(Grundy(l, i) ^ Grundy
        (i+1, r));
51 // (aquí SI se puede partir en un solo caracter, sin exigir
        pares).
52
53 // 3. DETECTAR SI LA FUSION ES POSIBLE ANTES DE CODIFICAR:
        antes de
54 // aplicar esta tecnica, revisa el enunciado con una
        pregunta
55 // concreta: "tras eliminar la porcion, ¿el caracter
        inmediatamente
56 // a la izquierda queda junto al caracter inmediatamente a
        la
57 // derecha?" Si la respuesta es si (la cadena se CONCATENA),
        la
58 // suma de Grundy en general NO aplica y hace falta otra
        tecnica
59 // (memoizar sobre la cadena real, no sobre indices del

```

```

        original).
60
61 // 4. REFERENCIA (variante CON fusion, solo para cadenas chicas
        ): si el
62 // enunciado exige que las partes se concatenen, la unica
        forma
63 // segura de resolverlo en general es memoizar directamente
        sobre
64 // el STRING RESULTANTE (no sobre indices del original), ya
        que el
65 // estado del juego deja de ser "dos piezas independientes"
        para
66 // ser una sola cadena nueva:
67 //
68 // map<string,int> memoDirecto;    // gana (true/false),
        NO Grundy
69 // bool gana(const string& t) {
70 //     if (memoDirecto.count(t)) return memoDirecto[t];
71 //     bool g = false;
72 //     for (int i = 0; i+1 < (int)t.size(); i++)
73 //         if (t[i]==t[i+1] && !gana(t.substr(0,i) + t.
        substr(i+2))) g = true;
74 //     return memoDirecto[t] = g;
75 // }
76 //
77 // Esto es minimax directo sobre el string completo, NO una
        suma de
78 // Grundy (no hay descomposicion en piezas independientes),
        y solo
79 // es viable para cadenas cortas por el numero de estados
        distintos
80 // alcanzables.
81
82 // 5. TRAMPA (la mas importante de esta subseccion): NUNCA
        asumas que
83 // "las dos partes quedan separadas" solo porque el
        enunciado diga
84 // "elimina un pedazo del medio" - verifica explicitamente
        si el
85 // juego indica que el resto se JUNTA (concatena) o queda
        separado.
86 // Aplicar la suma de Grundy a un juego que en realidad
        concatena
87 // produce un resultado incorrecto sin ningun error de
        compilacion
88 // o ejecucion que lo delate; la unica forma de detectarlo
        es
89 // releer la regla del movimiento con cuidado, o verificar
        contra
90 // fuerza bruta en cadenas pequenas antes de confiar en la
        formula.

```

Ejemplo de funcionamiento:

```

1 JuegoDeCadenas j1("aabb");
2 cout << j1.Grundy(0, 4) << " " << j1.primeroGana();
3 // 0 0    (el primer jugador pierde)
4
5 JuegoDeCadenas j2("aabbba");
6 cout << j2.Grundy(0, 6) << " " << j2.primeroGana();
7 // 1 1    (el primer jugador gana)
8
9 // Dos cadenas independientes en la misma partida: se combinan
        con XOR.
10 JuegoDeCadenas t1("aabb"), t2("aabbba");
11 int combinado = t1.Grundy(0, 4) ^ t2.Grundy(0, 6);
12 cout << combinado;    // 0 ^ 1 = 1    (el primer
        jugador gana la suma)

```

Regla práctica: antes de escribir una sola línea de código, confirma si el movimiento del enunciado **concatena** las partes restantes o las deja **separadas para siempre**. Si quedan separadas, esta suma de Grundy en $O(n^3)$ resuelve el juego directamente y se combina con otras cadenas vía XOR. Si se concatenan, esta técnica no aplica: hay que memoizar sobre el string real resultante (viable solo para entradas pequeñas) o buscar una propiedad específica del problema — nunca asumas la independencia sin verificarla.

65 Juegos Partidistas y Combinatoria Avanzada

65.1 Números Surreales y Valores de Juego (Cold, Hot, Fuzzy)

Idea: todo lo visto en *Fundamentos* (estados P/N, Sprague-Grundy) asume juegos **impartiales**: en cualquier posición, **ambos** jugadores tienen exactamente el mismo conjunto de movimientos disponibles. Un juego **partidista** rompe esa simetría: Izquierda y Derecha pueden tener opciones distintas (Hackenbush con aristas de dos colores, Domineering, finales de Go). La teoría de números surreales generaliza el mex/Grundy para este caso representando **cualquier** juego como un par de conjuntos de opciones, $G = \{G^L \mid G^R\}$: G^L es todo lo que Izquierda puede jugar, G^R todo lo que Derecha puede jugar, y cada opción es a su vez otro juego construido antes que G (la construcción es bien fundada, sin ciclos). Comparar dos juegos ya no usa mex sino la definición recursiva

$$G \leq H \iff (\text{ningún } G^L \geq H) \text{ y } (\text{ningún } H^R \leq G),$$

y clasificar G frente al juego vacío $0 = \{\mid\}$ dice quién gana: $G > 0$ Izquierda gana siempre; $G < 0$ Derecha gana siempre; $G = 0$ gana quien mueve **segundo** (como en Grundy=0); y $G \parallel 0$ (**difuso**, confundido con 0) gana quien mueve **primero** — el análogo partidista de Grundy $\neq 0$.

¿Por qué usarlo? Los juegos que resultan ser **números** (todo G^L estrictamente menor que todo G^R , recursivamente) se comportan como una ventaja fija y **fría**: a nadie le conviene jugar ahí porque es puro desperdicio de turno, igual que dar un peón de handicap. Los juegos **calientes** son lo opuesto: ambos jugadores quieren mover ahí **ya**, porque cada turno que se retrasa cuesta valor. Y los **difusos** (como $*$ = $\{0 \mid 0\}$, la generalización partidista de una posición Grundy $\neq 0$) no son ni fríos ni calientes: solo importa quién llega primero. Green Hackenbush (*Juegos sobre Árboles*, visto antes) es el caso particular donde **todo** es imparcial: ahí nunca aparecen números distintos de 0, solo posiciones difusas o cero, y por eso basta Grundy. En cuanto el juego distingue colores por jugador, se necesita esta teoría completa.

Casos de uso:

- Hackenbush azul-rojo (partidista): cada arista pertenece a un jugador, y el valor resultante puede ser cualquier número (entero o fraccionario), no solo 0 o $*$.
- Finales de Go y Domineering, donde cada región del tablero se analiza como un juego independiente antes de sumarlas.
- Distinguir, dentro de una posición compuesta, qué componentes son “ruido” (números, se pueden ignorar o sumar aparte) y cuáles son la disputa real (difusos o calientes).
- Construir sistemáticamente enteros y fracciones diádicas como juegos, para usarlos como **unidad de comparación** contra posiciones desconocidas.
- Base directa de *Suma de Juegos Partidistas y Comparación de Valores*, la siguiente subsección.

Complejidad: comparar dos juegos de m_1 y m_2 nodos en su árbol de opciones cuesta, con memoización por pares de punteros, $O(m_1 \cdot m_2)$ en el peor caso (cada par de subposiciones se compara a lo sumo una vez, y cada comparación revisa un número de opciones acotado por el grado del nodo). En la práctica esta técnica se usa sobre juegos **pequeños** (unas pocas piezas o aristas): sirve para razonar sobre subjuegos concretos de un problema, no para explorar un espacio de estados grande como en Grundy sobre un DAG.

```

1 struct Juego {
2     vector<Juego*> L, R;    // opciones de Izquierda y de Derecha
3 };
4
5 // CREATE: construye un juego nuevo a partir de sus opciones.
6 Juego* nuevo(vector<Juego*> l, vector<Juego*> r) {
7     Juego* j = new Juego();
8     j->L = l; j->R = r;
9     return j;
10 }
11
12 map<pair<Juego*,Juego*>, bool> memoLeq;
13
14 // READ: A <= B, usando la definicion recursiva de la teoria de
15 // juegos: ningun G^L de A es >= B, y ningun H^R de B es <= A.
16 bool leq(Juego* A, Juego* B) {
17     auto key = make_pair(A, B);
18     auto it = memoLeq.find(key);
19     if (it != memoLeq.end()) return it->second;
20     bool resultado = true;
21     for (Juego* Al : A->L) if (leq(B, Al)) { resultado = false;
22         break; } // Al >= B
23     if (resultado)
24     for (Juego* Br : B->R) if (leq(Br, A)) { resultado = false;
25         break; } // Br <= A
26     memoLeq[key] = resultado;
27     return resultado;
28 }
29 // amortizado con memo
30
31 // READ: igualdad de VALOR (dos arboles distintos pueden
32 // representar
33 // el mismo juego): A == B si A <= B y B <= A.
34 bool eq(Juego* A, Juego* B) { return leq(A, B) && leq(B, A); }
35
36 enum Signo { POSITIVO, NEGATIVO, CERO, DIFUSO };
37 Juego* CERO_G; // el juego vacio {}, se inicializa una vez
38
39 // READ: clasifica G comparandolo contra 0: quien gana la
40 // partida si
41 // G fuera el UNICO componente en juego.
42 Signo clasificar(Juego* G) {
43     bool noNegativo = leq(CERO_G, G); // G >= 0
44     bool noPositivo = leq(G, CERO_G); // G <= 0
45     if (noNegativo && noPositivo) return CERO; // segundo
46     // jugador gana
47     if (noNegativo) return POSITIVO; // Izquierda
48     // gana siempre
49     if (noPositivo) return NEGATIVO; // Derecha
50     // gana siempre
51     return DIFUSO; // primer
52     // jugador gana
53 }
54 // O(m) via leq
55
56 // -----
57 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
58 // -----
59
60 // 1. CONSTRUIR ENTEROS COMO JUEGOS: la definicion recursiva
61 // estandar,
62 // donde cada entero es "un peldano mas simple que el
63 // anterior":
64 //
65 // Juego* entero(int n, Juego* cero) {
66 //     if (n == 0) return cero;
67 //     if (n > 0) return nuevo({entero(n-1, cero)}, {});
68 //     // {n-1 | }
69 //     else return nuevo({}, {entero(n+1, cero)});
70 //     // { | n+1}
71 // }
72 //
73 // Un entero positivo no tiene opciones para Derecha (nadie
74 // puede
75 // bajarlo), y viceversa para los negativos - por eso
76 // siempre gana
77 // quien tiene el signo a favor, sin importar quien mueve
78 // primero.

```

```

63 // 2. CONSTRUIR FRACCIONES DIADICAS COMO JUEGOS: 1/2 = {0 | 1},
64 // 1/4 = {0 | 1/2}, 3/4 = {1/2 | 1}, y en general una
    fraccion
65 // p/2^k (p impar) se construye como {(p-1)/2^k | (p+1)/2^k},
    donde
66 // esos vecinos se simplifican recursivamente (al ser p-1 y
    p+1
67 // pares, su fraccion reduce a denominador 2^(k-1)). Estas
68 // construcciones se pueden verificar entre si con leq/eq:
    por
69 // ejemplo, leq(cero, mitad) && !eq(cero, mitad) confirma 0
    < 1/2.
70
71 // 3. ARRIBA (^) Y ABAJO (v), LOS INFINITESIMALES MAS SIMPLES:
72 // arriba = {0 | *}, abajo = {* | 0}. Son POSITIVO y
    NEGATIVO
73 // respectivamente, pero mas pequenos en valor absoluto que
74 // CUALQUIER numero positivo/negativo distinto de 0 -
    representan
75 // una ventaja real pero infinitesimal, tipica del final de
    un
76 // juego donde ya casi no queda nada por disputar.
77
78 // 4. HECHO CONTRAINTUITIVO QUE VALE LA PENA VERIFICAR: arriba
    + * NO
79 // es simplemente "un poco mas que cero" como se podria
    esperar por
80 // intuicion ingenua; es DIFUSO (confundido con 0) - se
    puede
81 // verificar con clasificar(suma(arriba, estrella)) (suma
    definida
82 // en la siguiente subseccion). Este resultado clasico de la
    teoria
83 // (Winning Ways, Berlekamp-Conway-Guy) es la razon por la
    que un
84 // valor "aproximadamente positivo" NO garantiza ganar: solo
    la
85 // clasificacion EXACTA contra 0 lo hace.
86
87 // 5. TRAMPA: UN JUEGO DIFUSO NO ES "0". Que clasificar(G)
    retorne
88 // DIFUSO significa que G NO es comparable con 0 (ni G<=0 ni
89 // G>=0), no que sea un empate: en un juego difuso, quien
    mueve
90 // PRIMERO en ESE componente gana la disputa local - justo
    lo
91 // opuesto de G=0 (donde el que mueve primero pierde ese
92 // componente). Confundir estos dos casos invierte la
    estrategia.

```

Ejemplo de funcionamiento:

```

1 Juego* cero = nuevo({}, {});
2 CERO_G = cero;
3 Juego* estrella = nuevo({cero}, {cero}); // * = {0|0}
4 Juego* uno = nuevo({cero}, {}); // 1 = {0|}
5 Juego* menosUno = nuevo({}, {cero}); // -1 = { |0}
6 Juego* arriba = nuevo({cero}, {estrella}); // ^ = {0|*}
7 Juego* abajo = nuevo({estrella}, {cero}); // v = {*|0}
8
9 cout << clasificar(cero); // CERO
10 cout << clasificar(estrella); // DIFUSO (nadie tiene
    ventaja fija: gana quien mueve primero)
11 cout << clasificar(uno); // POSITIVO
12 cout << clasificar(arriba); // POSITIVO (pero infinitesimal
    )
13
14 cout << (leq(cero, uno) && !eq(cero, uno)); // 1 (0 < 1)

```

Regla práctica: antes de aplicar Sprague-Grundy a un juego, verifica si Izquierda y Derecha tienen realmente las mismas opciones en cada posición; si no (colores, piezas o roles distintos por jugador), es partidista y necesitas esta teoría. Clasifica cada componente contra 0 con `leq/eq`: POSITIVO o NEGATIVO son ventajas fijas (“fría”, se puede sumar como un número), CERO significa que a nadie le conviene jugar ahí, y DIFUSO significa que es exactamente ahí donde se libra la disputa real por el turno.

65.2 Suma de Juegos Partidistas y Comparación de Valores

Idea: una posición real casi nunca es un solo juego pequeño, sino varios componentes independientes jugados

a la vez (varias regiones de Hackenbush, varias cadenas de Domineering); en cada turno, un jugador elige **un** componente y hace ahí un movimiento válido. Esto es la **suma disyunta** $G + H$, definida recursivamente igual que los juegos mismos: las opciones de Izquierda en $G + H$ son mover en G o mover en H (nunca ambas a la vez), y análogamente para Derecha:

$$(G+H)^L = \{G^L+H\} \cup \{G+H^L\}, \quad (G+H)^R = \{G^R+H\} \cup \{G+H^R\}$$

La **negación** $-G$ intercambia los roles: cambia cada opción de Izquierda por Derecha y viceversa, recursivamente. Con suma y negación, determinar el ganador de una posición completa $G_1 + G_2 + \dots + G_k$ es exactamente clasificar (con `leq/eq` de la subsección anterior) el juego resultante de sumarlos todos, contra 0.

¿Por qué usarlo? *Suma de Juegos Independientes (XOR de Grundy Numbers)* en *Fundamentos* es el caso **particular** de esta misma idea cuando **todos** los componentes son imparciales: ahí, sumar juegos y comparar contra 0 colapsa exactamente a “XOR de los números de Grundy $\neq 0$ ”, un atajo válido solo porque en juegos imparciales toda posición es 0 o difusa (nunca un número distinto de 0, nunca fría). En cuanto un solo componente es partidista y resulta ser un número (por ejemplo, una porción ya decidida de Hackenbush con puntaje fijo), el atajo de XOR deja de tener sentido —los números no se combinan por XOR, sino que si **todos** los componentes son números, simplemente se **suman como valores reales** (esto es válido: la suma de juegos-número coincide con la suma aritmética usual). El caso verdaderamente difícil es una mezcla de números, calientes y difusos: ahí no hay atajo aritmético y hace falta construir la suma completa como juego y clasificarla.

Casos de uso:

- Determinar el ganador de una posición de Hackenbush azul-rojo con varias componentes de distinto valor.
- Sumar puntuaciones parciales ya cerradas (números) de un final de Go directamente como valores reales.
- Verificar relaciones algebraicas entre valores de juego (por ejemplo, que $G + (-G) = 0$ para cualquier G , o que dos posiciones con árboles distintos representan el mismo valor).
- Simplificar una posición compuesta reemplazando cada componente por su valor más simple antes de sumarlos, en vez de analizar el árbol de juego completo de la posición entera.
- Confirmar hechos no intuitivos de la teoría (como que $\uparrow + *$ es difuso) antes de usarlos como regla de decisión.

Complejidad: construir $G + H$ genera un árbol cuyo tamaño puede crecer como el **producto** de los tamaños de G y H (cada nodo de la suma corresponde a un par de subposiciones), así que sumar k componentes de tamaño $O(m)$ cada uno cuesta, en el peor caso, $O(m^k)$ nodos antes de memoizar; con memoización por pares de punteros (reutilizando subárboles repetidos) el costo

real depende de cuántos pares **distintos** aparecen, típicamente muy por debajo de la cota ingenua para los tamaños de juguete con los que se trabaja esta teoría a mano. Clasificar la suma final cuesta lo mismo que cualquier `leq` sobre el árbol resultante.

```

1 map<Juego*, Juego*> memoNeg;
2
3 // READ: -A, intercambiando Izquierda y Derecha recursivamente.
4 Juego* negar(Juego* A) {
5     auto it = memoNeg.find(A);
6     if (it != memoNeg.end()) return it->second;
7     vector<Juego*> nl, nr;
8     for (Juego* Ar : A->R) nl.push_back(negar(Ar)); // lo que
9     // era opción de Derecha
10    for (Juego* Al : A->L) nr.push_back(negar(Al)); // pasa a
11    // ser de Izquierda, y viceversa
12    Juego* res = nuevo(nl, nr);
13    memoNeg[A] = res;
14    return res;
15 } // O(m) con memo
16
17 map<pair<Juego*,Juego*>, Juego*> memoSuma;
18
19 // READ: A + B, la suma disyunta: mover es elegir UN componente
20 // y
21 // jugar ahí un movimiento válido de ese componente.
22 Juego* suma(Juego* A, Juego* B) {
23     auto key = make_pair(A, B);
24     auto it = memoSuma.find(key);
25     if (it != memoSuma.end()) return it->second;
26     vector<Juego*> nl, nr;
27     for (Juego* Al : A->L) nl.push_back(suma(Al, B)); // mover
28     // en A
29     for (Juego* Bl : B->L) nr.push_back(suma(A, Bl)); // mover
30     // en B
31     for (Juego* Ar : A->R) nr.push_back(suma(Ar, B));
32     for (Juego* Br : B->R) nr.push_back(suma(A, Br));
33     Juego* res = nuevo(nl, nr);
34     memoSuma[key] = res;
35     return res;
36 } // O(producto de
37 // tamanos) con memo
38
39 // READ: ganador de la suma de varios componentes, reutilizando
40 // clasificar() de la subseccion anterior sobre el total
41 // acumulado.
42 Signo ganadorDeLaSuma(vector<Juego*> componentes) {
43     Juego* total = CERO_G;
44     for (Juego* g : componentes) total = suma(total, g);
45     return clasificar(total);
46 } // O(suma de tamanos
47 // al cuadrado), con memo
48
49 // -----
50 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
51 // -----
52
53 // 1. ATAJO CUANDO TODO ES NUMERO: si CADA componente es un
54 // juego que
55 // resulta ser un numero (entero o fraccion), no hace falta
56 // construir la suma simbolica completa: basta calcular el
57 // valor
58 // numerico de cada uno por separado (por ejemplo, con el
59 // algoritmo clasico para reducir un Hackenbush azul-rojo en
60 // forma
61 // de CADENA a una fraccion diadica) y sumarlos como valores
62 // reales
63 // ordinarios; el signo de esa suma numerica decide el
64 // ganador
65 // exactamente igual que clasificar() sobre el juego
66 // simbolico.
67
68 //
69 // 2. ATAJO CUANDO TODO ES IMPARTIAL: si NINGUN componente
70 // distingue
71 // Izquierda de Derecha (todo Green Hackenbush o Nim puro),
72 // no
73 // construyas la suma como {L|R}: usa directamente XOR de
74 // Grundy
75 // numbers (Suma de Juegos Independientes, en Fundamentos),
76 // que es
77 // equivalente pero mucho mas barato de calcular.
78
79 //
80 // 3. VERIFICAR  $G + (-G) = 0$  PARA CUALQUIER  $G$ : es un teorema
81 // general
82 // de la teoria (todo juego tiene un inverso aditivo), util
83 // como
84 // prueba de que negar() y suma() estan bien implementados:
85 // eq(suma(G, negar(G)), CERO_G) debe dar siempre true.
86
87 //
88 // 4. ORDEN DE JUEGO PARA COMPONENTES MIXTOS: si la posicion
89 // mezcla

```

```

86 // numeros (frios, nadie quiere jugar ahí) con componentes
87 // difusos
88 // o calientes, la intuicion de la teoria (no una garantia
89 // absoluta
90 // para todo caso) es que conviene jugar primero en los
91 // componentes MAS urgentes (mas lejos de ser un numero) y
92 // dejar
93 // los numeros para el final, ya que estos no se "enfrian"
94 // mas -
95 // perder un turno ahí no cambia su valor, mientras que
96 // retrasar un
97 // componente caliente si puede costar ventaja.
98
99 //
100 // 5. TRAMPA: SUMAR VALORES NUMERICOS DE COMPONENTES QUE NO SON
101 // NUMEROS. Si un solo componente es difuso o caliente (no
102 // es un
103 // numero), no existe un "valor real" unico que sumar - solo
104 // existe el juego completo {L|R}. Intentar asignarle un
105 // numero de
106 // todas formas (por ejemplo, promediando opciones) y
107 // sumarlo
108 // directamente con los demas produce conclusiones
109 // incorrectas
110
111 // sobre quien gana; en ese caso hay que construir la suma
112 // simbolica y clasificarla con leq/eq, como en el ejemplo
113 // de
114 // arriba + estrella de la subseccion anterior.

```

Ejemplo de funcionamiento:

```

1 // (continuyendo los juegos definidos en la subseccion anterior)
2
3 cout << eq(suma(estrella, estrella), cero); // 1 (* +
4 // = 0)
5 cout << eq(suma(uno, menosUno), cero); // 1 (1 +
6 // (-1) = 0)
7 cout << eq(negar(arriba), abajo); // 1 (~
8 // = v)
9
10 // El hecho clasico y contraintuitivo: arriba + * es DIFUSO, no
11 // "un poco mayor que 0" como sugeriria la intuicion ingenua.
12 cout << clasificar(suma(arriba, estrella)); // DIFUSO
13
14 // arriba + arriba + abajo == arriba (el abajo "cancela" uno de
15 // los
16 // dos arriba, dejando exactamente el otro).
17 Juego* combinado = suma(suma(arriba, arriba), abajo);
18 cout << eq(combinado, arriba); // 1
19
20 // Ganador de una posicion con tres componentes mixtos.
21 Signo g = ganadorDeLaSuma({uno, estrella, abajo});
22 cout << g; // depende de los valores exactos; se calcula
23 // clasificando el total

```

Nota: el resultado `arriba + estrella = DIFUSO` contradice la intuición de sumar valores aproximados ($\uparrow$ es “un poquito positivo”, $*$ es “ni positivo ni negativo”, se esperaría que la suma siguiera siendo positiva): es la advertencia central de esta teoría — **no se puede** razonar sobre sumas de juegos partidistas promediando o aproximando valores; solo la construcción simbólica exacta (`suma + clasificar`) da la respuesta correcta.

Regla práctica: construye la suma disyunta y clasifica contra 0 siempre que la posición mezcle componentes de distinta naturaleza (números, calientes, difusos); si todos los componentes son números, súmalos como valores reales ordinarios; si todos son imparciales, usa XOR de Grundy en vez de esta maquinaria. Nunca mezcles los tres atajos: en cuanto hay al menos un componente partidista no numérico junto a otros, la única vía correcta es la suma simbólica completa.

65.3 Juegos de Turnos con Puntaje (Optimal Strategy for a Game)

Idea: esta es la misma técnica de *Juegos de Tomar de los Extremos de un Arreglo* (DP de Intervalos), vista en *Juegos sobre Estructuras de Datos* — dos jugadores turnándose para tomar el extremo izquierdo o derecho de un arreglo, maximizando cada uno su propio puntaje, resuelto

con $dp[i][j] = \max(a[i] - dp[i+1][j], a[j] - dp[i][j-1])$. Reaparece aquí, junto a *Números Surreales* y *Suma de Juegos Partidistas*, para ubicarla correctamente dentro del panorama teórico: es un ejemplo de **juego con puntaje** (*scoring game*), una rama de la teoría de juegos combinatorios distinta de la teoría *normal-play* de Conway que se usó en las dos subsecciones anteriores. En *normal-play*, el valor de un juego captura **quién puede seguir moviendo**; en un juego con puntaje no existe esa pregunta binaria (el arreglo se agota jugada tras jugada sin importar las decisiones), y lo que se necesita en cambio es un **número** —el margen— que resuma cuánto le conviene a cada jugador esa parte del juego.

¿Por qué usarlo? Vale la pena notar por qué este juego, siendo técnicamente asimétrico en el resultado (cada jugador se queda con algo distinto), **no** necesita la maquinaria completa de $\{G^L \mid G^R\}$: en los juegos partidistas de la subsección de *Números Surreales*, lo que distingue a Izquierda de Derecha son las **jugadas disponibles** (Hackenbush azul vs. rojo, opciones distintas por jugador). Aquí, en cambio, **ambos jugadores tienen exactamente las mismas jugadas** (tomar izquierda o tomar derecha) —lo único que difiere es **a quién beneficia** cada resultado. Esa simetría en las jugadas (aunque no en el puntaje) es lo que permite colapsar todo a un solo número por intervalo con una recurrencia minimax simple, en vez de construir un árbol de opciones $\{L \mid R\}$ distinto para cada jugador.

Casos de uso: (los mismos que en la subsección original —Optimal Strategy for a Game / Predict the Winner, cartas en fila, juegos de puntaje con dos extremos disponibles) más uno propio de este contexto teórico:

- Como ejemplo concreto y resoluble de un *scoring game* para contrastar con los juegos partidistas normal-play de las dos subsecciones anteriores.
- Servir de sanity check: si un juego con puntaje tiene jugadas simétricas entre jugadores (como aquí), sospecha que una DP de intervalos simple bastará, sin necesitar surreales.
- Todos los casos ya cubiertos en *Juegos de Tomar de los Extremos de un Arreglo*: determinar ganador, margen exacto, puntaje de cada jugador.

Complejidad: idéntica a la subsección original: $O(n^2)$ en tiempo con la tabla completa, reducible a $O(n)$ de memoria procesando por longitud de intervalo creciente (ver esa subsección para el detalle y la versión con arreglo 1D).

```

1 struct JuegoDeExtremos {
2     int n;
3     vector<long long> a;
4     vector<vector<long long>> dp; // dp[i][j] = diferencia
5     // (turno - rival) jugando sobre a[i..j]
6
7     // CREATE: guarda los valores del arreglo.
8     JuegoDeExtremos(vector<long long> valores) {
9         a = valores;
10        n = (int)a.size();
11        dp.assign(n, vector<long long>(n, 0));
12    } // O(n^2)
13
14    // WRITE: llena dp por longitud de intervalo creciente.
15    void calcular() {
16        for (int i = 0; i < n; i++) dp[i][i] = a[i];
17        for (int largo = 2; largo <= n; largo++) {

```

```

18        for (int i = 0; i + largo - 1 < n; i++) {
19            int j = i + largo - 1;
20            dp[i][j] = max(a[i] - dp[i+1][j], a[j] - dp[i][j-1]);
21        }
22    } // O(n^2)
23
24    // READ: diferencia optima (turno - rival) sobre TODO el
25    // arreglo.
26    long long diferencia() const { return dp[0][n-1]; }
27
28    // READ: puntaje exacto de cada jugador bajo juego optimo.
29    pair<long long, long long> puntajes() const {
30        long long total = 0; for (long long x : a) total += x;
31        long long diff = dp[0][n-1];
32        return {(total + diff) / 2, (total - diff) / 2};
33    } // O(n)
34
35    // -----
36    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
37    // -----
38
39    // 1. Reduccion a O(n) de memoria, truco de paridad para n par,
40    // costo
41    // o bonus por extremo, y variante de "k turnos" con dos
42    // punteros:
43    // ver las modificaciones 1 a 4 de Juegos de Tomar de los
44    // Extremos
45    // de un Arreglo - son exactamente las mismas aqui, sin
46    // cambios.
47
48    // 2. Por que NO hace falta {L | R}: si quisieras modelar este
49    // juego
50    // como un juego partidista general, cada estado (i, j)
51    // tendria
52    // UNA opcion de Izquierda (tomar a[i]) y UNA opcion de
53    // Derecha
54    // (tomar a[j]), pero devolviendo el MISMO subjuego
55    // resultante
56    // (i+1, j) o (i, j-1) sin importar quien lo tome - lo unico
57    // que
58    // distingue el resultado es el puntaje sumado, no el
59    // subjuego al
60    // que se llega. Por eso alcanza con propagar un numero (la
61    // diferencia) en vez de dos arboles de opciones distintos.
62
63    // 3. TRAMPA (la misma que antes, repetida por importancia):
64    // esto NO
65    // es un juego normal-play de "quien mueve al final gana" -
66    // no
67    // intentes aplicarle Sprague-Grundy ni la clasificacion
68    // POSITIVO/NEGATIVO/CERO/DIFUSO de Números Surreales, que
69    // asume
70    // una convencion de victoria distinta (basada en quien se
71    // queda
72    // sin movimientos, no en maximizar un puntaje acumulado).

```

Ejemplo de funcionamiento: idéntico al de *Juegos de Tomar de los Extremos de un Arreglo* (DP de Intervalos) — mismo código, mismos resultados (*diferencia()* = 0 para $\{1, 5, 2, 4, 6\}$, empate bajo juego óptimo).

Regla práctica: si ya resolviste este patrón en *Juegos sobre Estructuras de Datos*, no hay nada nuevo que programar aquí — el valor de volver a verlo es puramente conceptual: sirve para ubicar los **juegos con puntaje** como una familia aparte de los juegos partidistas normal-play, y para reconocer la señal de que un juego con puntaje admite esta DP simple en cuanto notas que **ambos jugadores comparten las mismas jugadas** y solo compiten por quién se queda con cada valor.

66 Estrategia Óptima y Técnicas de Resolución

66.1 Minimax con Memoización

Idea: minimax es la técnica genérica para resolver cualquier juego de dos jugadores de información perfecta: en cada estado, el jugador que **maximiza** elige el movimiento con el mejor valor para él, y el que **minimiza** elige el que más lo perjudica; el valor de un estado

terminal es el resultado del juego ahí (victoria, derrota, empate, o un puntaje). Esto ya es exactamente lo que hacen *Juegos de Tomar de los Extremos* y *Poda con Fuerza Bruta (Tabla de Grundy)* de *Fundamentos* por dentro — minimax es el mecanismo detrás de esas técnicas, presentado aquí de forma explícita y **general**: en vez de un estado con una forma particular conveniente (un intervalo, una máscara), un estado puede ser cualquier cosa (un tablero de Tres en Raya, una posición de Conecta 4) mientras exista una función que genere sus movimientos válidos. El problema práctico es que el mismo estado puede alcanzarse por caminos distintos (**transposiciones**: jugar *A* y luego *B* llega al mismo tablero que jugar *B* y luego *A*), y sin memoización minimax lo vuelve a resolver desde cero cada vez que lo visita.

¿Por qué usarlo? Cuando el estado del juego no tiene una estructura simple que permita indexarlo por un par de enteros (como en la DP de intervalos) ni reducirlo a un único número de Grundy (como en *Fundamentos*), minimax con memoización es la herramienta por defecto: sirve para **cualquier** juego adversarial de dos jugadores, a costa de necesitar una clave de estado que se pueda usar en un `map` o `unordered_map` (un bitmask, un string, una tupla). La memoización es lo que hace viable esta generalidad: sin ella, un juego con muchas transposiciones (como Tres en Raya) explora **cientos de miles** de nodos repitiendo trabajo; con ella, solo se resuelve una vez cada estado *distinto*.

Casos de uso:

- Resolver juegos pequeños de tablero (Tres en Raya, variantes de Conecta 4 en tableros chicos) determinando el resultado bajo juego óptimo desde cualquier posición.
- Cualquier juego donde el estado natural (tablero, mano de cartas, configuración) no encaja en las técnicas más específicas (intervalos, máscaras de bits, Grundy) de secciones anteriores.
- Verificar por fuerza bruta el resultado de una técnica más rápida (Grundy, DP de intervalos) sobre casos pequeños, como *sanity check* antes de confiar en la fórmula cerrada.
- Juegos con estados que se repiten mucho por transposición, donde la memoización reduce el trabajo de forma dramática incluso sin ninguna otra optimización.
- Como base directa sobre la que se construye *Poda Alfa-Beta*, que acelera esta misma búsqueda sin cambiar el resultado.

Complejidad: sin memoización, el costo es $O(b^d)$ con b el factor de ramificación (movimientos disponibles) y d la profundidad del árbol — para Tres en Raya desde el tablero vacío, esto son más de medio millón de nodos visitados solo para un juego de 3×3 . Con memoización, el costo baja a $O(E \cdot b)$, con E el número de estados **distintos** alcanzables (no de caminos para llegar a ellos): cada estado se resuelve una sola vez, revisando sus b movimientos, y las consultas repetidas cuestan $O(1)$ amortizado.

```

1 // Tres en Raya: el estado se codifica como dos máscaras de 9
  bits
2 // (celdas ocupadas por X y por O). LINEAS lista las 8 formas
  de ganar.
3 int LINEAS[8] = {
4     0b111000000, 0b000111000, 0b000000111, // filas
5     0b100100100, 0b010010010, 0b001001001, // columnas
6     0b100010001, 0b001010100 // diagonales
7 };
8
9 // READ: true si la máscara m contiene alguna línea ganadora
  completa.
10 bool gano(int m) {
11     for (int L : LINEAS) if ((m & L) == L) return true;
12     return false;
13 } // O(1) (8
    comparaciones)
14
15 unordered_map<long long, int> memo;
16
17 // READ: valor del estado (x = celdas de X, o = celdas de O,
    turno =
18 // true si mueve X) bajo juego óptimo: 1 si gana X, -1 si gana
    O, 0
19 // empate. X maximiza, O minimiza.
20 int minimaxMemo(int x, int o, bool turno) {
21     if (gano(x)) return 1;
22     if (gano(o)) return -1;
23     int ocupado = x | o;
24     if (ocupado == 0b111111111) return 0; // tablero lleno:
    empate
25
26     // clave única: 9 bits de x, 9 bits de o, 1 bit de turno
27     long long clave = ((long long)x << 10) | o | ((long long)
    turno << 20);
28     auto it = memo.find(clave);
29     if (it != memo.end()) return it->second;
30
31     int mejor = turno ? -2 : 2; // -2/2 fuera del rango real
    [-1,1]
32     for (int i = 0; i < 9; i++) {
33         if (!(ocupado >> i) & 1) {
34             int val = turno ? minimaxMemo(x | (1 << i), o, false)
35             : minimaxMemo(x, o | (1 << i), true);
36             mejor = turno ? max(mejor, val) : min(mejor, val);
37         }
38     }
39     memo[clave] = mejor;
40     return mejor;
41 } // O(E * 9) total,
    con E = estados distintos
42
43 // -----
44 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
45 // -----
46
47 // 1. RECONSTRUIR LA JUGADA OPTIMA, NO SOLO EL VALOR: para
    saber QUE
48 // movimiento jugar (no solo si se gana), guarda junto al
    valor en
49 // memo cual fue el índice i que produjo mejor, y consulta
    esa
50 // tabla desde el estado actual en vez de recorrer todo de
    nuevo.
51
52 // 2. CLAVE DE ESTADO PARA OTROS JUEGOS: la clave debe capturar
    TODO
53 // lo que determina el futuro del juego (tablero + de quien
    es el
54 // turno, y a veces información extra como "movimientos
    restantes" o "cartas ya jugadas"). Para tableros mas
    grandes
55 // que quepan en bits (hasta 64), usar máscaras de bits como
    aqui;
56 // para estados mas complejos, usar un string o una tupla
    como
57 // clave de un map.
58
59
60 // 3. LIMITE DE PROFUNDIDAD (juegos que no terminan rapido): si
    el
61 // juego real es demasiado grande para resolverlo por
    completo
62 // (ajedrez, Go), se trunca la búsqueda a una profundidad
    maxima y
63 // se usa una función de EVALUACION heurística en vez del
    valor
64 // exacto en los nodos truncados; esto ya no da el valor
    optimo
65 // real, sino una aproximación, y es la base de los motores
    de
66 // juegos practicos.
67
68 // 4. TRAMPA: LA CLAVE DEBE INCLUIR EL TURNO. El mismo tablero

```

```

69 // (x, o)
// tiene un valor distinto segun quien mueva a continuacion
// (por
70 // ejemplo, un tablero donde X ya gana es +1 sin importar el
// turno,
71 // pero un tablero intermedio con el mismo par de mascaras y
// turno
72 // distinto puede tener resultados diferentes). Omitir el
// turno de
73 // la clave mezcla estados que no deberian compartir memo.

```

Ejemplo de funcionamiento:

```

1 int valor = minimaxMemo(0, 0, true); // tablero vacio, mueve
  X
2 cout << valor;
3 // 0 (empate bajo juego optimo -- el resultado conocido de
  Tres en
4 // Raya cuando ambos jugadores juegan perfecto)
5
6 // Sin memoizacion, resolver el mismo tablero vacio visita 549
  946
7 // nodos; con memoizacion, solo 16 168, sobre 4520 estados
  distintos
8 // memoizados -- la mayoria de esos 549 946 nodos eran el MISMO
9 // tablero visitado una y otra vez por caminos distintos.

```

Nota: el resultado (empate) es el hecho bien conocido de que Tres en Raya, jugado perfectamente por ambos lados, siempre termina en empate; sirve como verificación rápida de que la implementación es correcta antes de usarla en un problema donde no se conoce de antemano el resultado esperado.

Regla práctica: usa minimax con memoización en cuanto el estado del juego no encaje en una técnica más específica de las secciones anteriores (intervalos, máscaras, Grundy), y en cuanto sospeches que hay transposiciones — estados alcanzables por más de un camino. Si el espacio de estados es demasiado grande para memoizar por completo (juegos reales de tablero grande), esto deja de ser exacto y hace falta una función de evaluación heurística con profundidad limitada, un tema que ya escapa del alcance de la programación competitiva clásica.

66.2 Poda Alfa-Beta

Idea: alfa-beta acelera minimax **sin cambiar el resultado**, descartando ramas que ya se sabe que no pueden influir en la decisión final. Mantiene dos cotas mientras recorre el árbol: α (el mejor valor que el jugador que **maximiza** ya tiene garantizado en algún lugar del árbol) y β (lo mismo para el que **minimiza**). Si en algún punto $\alpha \geq \beta$, significa que el jugador que está un nivel arriba en el árbol **ya tiene una opción mejor** en otra rama, así que el rival —siendo racional— nunca dejaría que la partida llegara a esta rama: se puede dejar de explorarla por completo (**podar**) sin perder nada, porque su valor exacto ya no puede cambiar la decisión que se tomará más arriba.

¿Por qué usarlo? Es una mejora directa sobre *Minimax con Memoización*: mismo árbol, mismo resultado exacto, menos nodos visitados. En el mejor caso (con un buen **orden de movimientos** — revisar primero las jugadas más prometedoras), alfa-beta reduce la complejidad de $O(b^d)$ a $O(b^{d/2})$, equivalente a explorar un árbol con la **raíz cuadrada** del factor de ramificación. Es independiente de la memoización y se puede combinar con ella: la memoización evita rehacer trabajo en **transposiciones** (mismo estado por caminos distintos), mientras que alfa-beta evita explorar ramas que **nunca se van a necesitar**, sean transposiciones o no; combinar ambas en una **tabla de transposición** exige un

cuidado extra (modificación 3) porque, bajo poda, un valor guardado puede ser solo una **cota** del valor real, no el valor exacto.

Casos de uso:

- Cualquier motor de juego de dos jugadores (ajedrez, damas, Conecta 4) donde el árbol completo es demasiado grande para explorarlo sin poda, incluso con memoización.
- Acelerar la resolución exacta de juegos pequeños que ya se resuelven con *Minimax con Memoización*, cuando el número de estados distintos sigue siendo alto.
- Motores con profundidad limitada y función de evaluación heurística (mencionados en la subsección anterior): alfa-beta es casi siempre imprescindible ahí, porque permite explorar mucha más profundidad en el mismo tiempo.
- Cualquier búsqueda adversarial donde ordenar los movimientos de mejor a peor (aunque sea con una heurística simple) antes de explorarlos acelera la poda de forma notable.

Complejidad: en el peor caso (sin ningún orden útil de movimientos), alfa-beta visita el mismo $O(b^d)$ que minimax sin poda — no hay garantía si el orden de exploración es malo. Con un **orden perfecto** de movimientos, la cota baja a $O(b^{d/2})$. En la práctica, incluso un orden **razonable** (por ejemplo, probar primero movimientos centrales o históricamente buenos) logra una reducción sustancial sin llegar al caso ideal.

```

1 // READ: valor del estado bajo juego optimo, podando ramas que
  no
2 // pueden cambiar la decision. alfa = mejor valor ya
  garantizado para
3 // quien maximiza; beta = mejor valor ya garantizado para quien
  // minimiza. Llamada inicial: alfaBeta(0, 0, true, -2, 2).
4 int alfaBeta(int x, int o, bool turno, int alfa, int beta) {
5     if (gano(x)) return 1;
6     if (gano(o)) return -1;
7     int ocupado = x | o;
8     if (ocupado == 0b11111111) return 0;
9
10    if (turno) { // X maximiza
11        int mejor = -2;
12        for (int i = 0; i < 9; i++) {
13            if (!(ocupado >> i) & 1) {
14                int val = alfaBeta(x | (1 << i), o, false, alfa, beta);
15                mejor = max(mejor, val);
16                alfa = max(alfa, mejor);
17                if (alfa >= beta) break; // O ya tiene algo mejor en
18                otra rama: podar
19            }
20        }
21        return mejor;
22    } else { // O minimiza
23        int mejor = 2;
24        for (int i = 0; i < 9; i++) {
25            if (!(ocupado >> i) & 1) {
26                int val = alfaBeta(x, o | (1 << i), true, alfa, beta);
27                mejor = min(mejor, val);
28                beta = min(beta, mejor);
29                if (alfa >= beta) break;
30            }
31        }
32        return mejor;
33    }
34 } // O(b^(d/2)) con buen orden // O(b^d) peor caso,
35
36 // -----
37 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
38 // -----
39
40 // 1. ORDEN DE MOVIMIENTOS: la poda depende por completo de
  revisar

```

```

41 // primero las jugadas mas prometedoras. En Tres en Raya,
42 // probar el
43 // centro antes que las esquinas y los lados reduce los
44 // nodos
45 // visitados de forma notable (en la practica, de decenas de
46 // a unos pocos miles) sin cambiar el resultado en absoluto:
47 //
48 // int orden[9] = {4, 0, 2, 6, 8, 1, 3, 5, 7}; // centro,
49 // esquinas, lados
50 // for (int idx = 0; idx < 9; idx++) {
51 //     int i = orden[idx];
52 //     if (!(ocupado >> i) & 1)) { /* ... */ }
53 // }
54 // En juegos mas complejos, una heuristica simple (capturar
55 // antes
56 // que mover, o el mejor movimiento encontrado en una
57 // busqueda
58 // anterior mas superficial) cumple el mismo papel.
59 //
60 // 2. VENTANA NULA (NEGASCOUT / PVS): una vez que se tiene un
61 // valor
62 // candidato, se puede verificar si una rama alternativa es
63 // MEJOR
64 // o PEOR que ese candidato con una ventana de tamaño 1
65 // (alfa, alfa+1) en vez de la ventana completa, que poda
66 // mucho mas
67 // agresivamente; solo si esa verificacion barata sugiere
68 // que SI es
69 // mejor, se vuelve a explorar con la ventana completa para
70 // obtener
71 // el valor exacto. Es una optimizacion avanzada sobre alfa-
72 // beta,
73 // util cuando el orden de movimientos ya es razonablemente
74 // bueno.
75 //
76 // 3. COMBINAR CON TABLA DE TRANSPOSICION (CUIDADO CON LOS
77 // FLAGS): con
78 // poda, un valor guardado en memo puede ser EXACTO, una
79 // COTA
80 // INFERIOR (se podio antes de confirmar que no habia algo
81 // mejor) o
82 // una COTA SUPERIOR (simetrico para el minimizador).
83 // Guardar sin
84 // distinguir estos casos produce resultados incorrectos:
85 //
86 // enum Flag { EXACTO, COTA_INF, COTA_SUP };
87 // struct Entrada { int valor; Flag flag; };
88 // unordered_map<long long, Entrada> tabla;
89 //
90 // int alfaBetaConTabla(int x, int o, bool turno, int alfa,
91 // int beta) {
92 //     // ... casos terminales iguales ...
93 //     long long clave = /* igual que en minimaxMemo */;
94 //     int alfaOrig = alfa, betaOrig = beta;
95 //     auto it = tabla.find(clave);
96 //     if (it != tabla.end()) {
97 //         if (it->second.flag == EXACTO) return it->
98 //             second.valor;
99 //         else if (it->second.flag == COTA_INF) alfa =
100 //             max(alfa, it->second.valor);
101 //         else beta = min(beta, it->second.valor);
102 //         if (alfa >= beta) return it->second.valor;
103 //     }
104 //     int mejor = /* ... recursion normal de alfa-beta,
105 //         actualizando alfa/beta ... */;
106 //     Flag f = (mejor <= alfaOrig) ? COTA_SUP
107 //         : (mejor >= betaOrig) ? COTA_INF
108 //         : EXACTO;
109 //     tabla[clave] = {mejor, f};
110 //     return mejor;
111 // }
112 //
113 // Esta combinacion (verificada sobre Tres en Raya) visito
114 // solo
115 // 4777 nodos frente a los 18297 de alfa-beta sin tabla y
116 // los
117 // 16168 de minimax con memo sin poda - las dos tecnicas se
118 // refuerzan en vez de competir.
119 //
120 // 4. TRAMPA: PODAR NO CAMBIA EL RESULTADO, SOLO EL COSTO. Si
121 // alfa-beta da un valor distinto a minimax puro sobre el
122 // MISMO
123 // arbol, hay un error de implementacion (probablemente en
124 // como se
125 // actualizan o se pasan alfa/beta entre llamadas) - los dos
126 // deben
127 // coincidir siempre, y es la forma mas facil de depurar una
128 // implementacion nueva: compararla contra minimax simple en
129 // un
130 // juego pequeno.

```

```

107 // 5. TRAMPA: >= VS >. La condicion de poda es alfa >= beta (no
108 // alfa > beta); con solo ">" se pierden podas validas sin
109 // que el
110 // resultado sea incorrecto, solo mas lento - un error
111 // silencioso
112 // que baja el rendimiento sin romper la correccion, dificil
113 // de
114 // notar sin medir nodos visitados explicitamente.

```

Ejemplo de funcionamiento:

```

1 int valor = alfaBeta(0, 0, true, -2, 2);
2 cout << valor; // 0 (mismo resultado que minimax con memo:
3 // empate)
4 // Nodos visitados resolviendo el tablero vacio de Tres en Raya,
5 // todos con el MISMO resultado (0, empate):
6 // minimax sin memo ni poda: 549 946 nodos
7 // minimax con memoizacion: 16 168 nodos
8 // alfa-beta (sin orden de movimientos): 18 297 nodos
9 // alfa-beta con orden (centro primero): 7 275 nodos
10 // alfa-beta + tabla de transposicion: 4 777 nodos

```

Nota: llama la atención que alfa-beta **sin** un buen orden de movimientos (18,297 nodos) visitó incluso más nodos que minimax con memoización simple (16,168): la poda por sí sola no garantiza mejorar sobre la memoización si el orden de exploración es malo y no hay tabla de transposición que atrape las repeticiones. La combinación de las dos técnicas (última fila) es la que da la mejor reducción, y en la práctica casi siempre conviene implementarlas juntas en vez de elegir una sola.

Regla práctica: implementa siempre poda alfa-beta sobre cualquier minimax que ya funcione — es un cambio pequeño (pasar dos enteros más y agregar una condición de corte) que nunca empeora el resultado y casi siempre reduce el trabajo. Si el juego tiene muchas transposiciones, combínala con una tabla de transposición usando los tres flags (EXACTO, COTA_INF, COTA_SUP) en vez de memoizar solo con alfa-beta o solo con minimax puro; y si el árbol sigue siendo intratable, invierte tiempo en mejorar el **orden** de exploración antes que en cualquier otra optimización — es lo que más impacto tiene sobre cuánto se poda.

66.3 Detección de Patrones con Fuerza Bruta (Tabla de Grundy)

Idea: Un juego es *imparcial* cuando las jugadas disponibles dependen solo del estado y no de quién mueve. Se usa la convención de juego normal: quien no puede jugar, pierde. El **número de Grundy** de un estado se define por recursión: un estado terminal vale 0, y en cualquier otro

$$G(s) = \text{MEX}\{G(t) \mid t \text{ es alcanzable desde } s\},$$

donde MEX es el menor entero no negativo que no pertenece al conjunto. Por ejemplo, $\text{MEX}\{0, 1, 3\} = 2$.

La intuición es que $G(s) = g$ hace que s se comporte como un montón de Nim de tamaño g . Por definición de MEX, desde s se puede llegar a estados de cada valor menor que g y nunca a un estado de valor g . Por tanto, si $G(s) \neq 0$ existe una jugada a un estado de valor 0 (el estado es ganador), y si $G(s) = 0$ toda jugada va a un valor distinto de 0, desde donde el rival puede volver a 0 (el estado es perdedor). La segunda consecuencia es la suma de juegos: si el juego se compone de k componentes independientes, en cada turno se juega en uno solo de

ellos, y el teorema de Sprague–Grundy dice que

$$G(s_1, \dots, s_k) = G(s_1) \oplus \dots \oplus G(s_k).$$

La posición total es perdedora si y solo si ese XOR vale 0. Con tres componentes de valores 2, 3, 1 el XOR es 0 (perdedora), y con 2, 3, 4 es 5 (ganadora).

La *fuerza bruta* consiste en calcular la tabla $G(0), G(1), \dots, G(N)$ probando todas las jugadas de cada estado y reutilizando lo ya calculado. Como el MEX de k valores nunca supera k , basta un arreglo de marcas de tamaño $k + 1$. En el juego de quitar 1, 2 o 3 piedras se obtiene $G(0..7) = 0, 1, 2, 3, 0, 1, 2, 3$, es decir $G(n) = n \bmod 4$. El *patrón* es que estas tablas suelen ser periódicas, y basta calcular unos pocos términos, detectar el periodo y extrapolar a n del orden de 10^{18} . En los juegos de restar con conjunto finito de jugadas $\{m_1, \dots, m_r\}$, con $D = \max m_i$, hay una detección rigurosa. Para $n \geq D$ el valor $G(n)$ depende solo de los D valores anteriores. Si la ventana de D valores que empieza en j es igual a la que empieza en $i < j$, entonces la secuencia se repite desde i con periodo $j - i$, por inducción.

¿Por qué usarlo? Si el juego es un único componente y solo se pregunta «¿gana el primero?», basta una DP booleana (*gana*[n] es verdadero si alguna jugada lleva a un estado con *gana* falso), que es más simple y no necesita MEX (modificación 7). Grundy se justifica cuando hay *varios* componentes independientes, porque el valor booleano de cada uno no alcanza para combinarlos y el número de Grundy sí. Frente al análisis retrógrado de la subsección de trabajar hacia atrás, que sirve en grafos con ciclos, Grundy exige un juego que termine (sin ciclos) pero entrega un valor combinable por XOR. Frente a los invariantes, el XOR de Grundy es justamente un invariante que se conserva de forma «complementaria»: cualquier jugada lo cambia, y desde un valor distinto de cero se puede devolver a cero. La fuerza bruta para detectar el patrón es la herramienta cuando la regla es complicada pero calculable para n pequeños y la consulta es para n enorme. Su límite es que fuera de los juegos de restar (ventana finita) el periodo detectado es una conjetura, no una prueba (modificación 3).

Casos de uso:

- Juegos de restar sobre una o varias pilas, con conjunto de jugadas dado, y decidir quién gana.
- Sumas de juegos independientes (varias pilas, varios tableros): calcular el Grundy de cada uno y hacer XOR.
- Obtener la jugada ganadora, no solo el ganador.
- Juegos donde una jugada divide un componente en dos (Kayles, cortar tiras): la tabla se llena con $G(a) \oplus G(b)$.
- De nicho: descubrir por fuerza bruta la fórmula cerrada de un juego (por ejemplo $G(n) = n \bmod (k + 1)$) y demostrarla después.

Complejidad: Sea N el tamaño de la tabla, M la cantidad de jugadas y D la mayor de ellas. Cada estado prueba a lo sumo M jugadas y calcula el MEX en $O(M)$,

así que la tabla completa cuesta $O(NM)$, con memoria $O(N)$. La detección del periodo por ventanas recorre N ventanas de largo D y las guarda en un *map*, lo que cuesta $O(ND \log N)$. Consultar un valor extrapolado es un módulo: $O(1)$. El XOR de k componentes cuesta $O(k)$ y encontrar la jugada ganadora $O(kM)$. Todo lo anterior supone N suficiente para que aparezca una ventana repetida.

```

1 struct TablaGrundy {
2     vector<int> movs; // jugadas permitidas: quitar m piedras (m
3     // en movs)
4     vector<int> g; // g[n] = numero de Grundy de una pila con
5     // n piedras
6
7     // READ: menor entero no negativo que no esta en v (v tiene k
8     // valores, el mex es <= k).
9     static int mex(const vector<int>& v) {
10        vector<char> vis(v.size() + 1, 0);
11        for (int x : v) if (x < (int)vis.size()) vis[x] = 1;
12        int r = 0;
13        while (vis[r]) r++;
14        return r;
15    } // O(k)
16
17    // CREATE: calcula g[0..N] probando todas las jugadas de cada
18    // estado.
19    TablaGrundy(vector<int> jugadas, int N) : movs(move(jugadas))
20    , g(N + 1, 0) {
21        for (int n = 1; n <= N; n++) {
22            vector<int> sig;
23            for (int m : movs) if (m <= n) sig.push_back(g[n - m]);
24            g[n] = mex(sig);
25        }
26    } // O(N * M)
27
28    // READ: busca una ventana de D valores que se repite (D =
29    // jugada mayor).
30    // Devuelve {pre, per}: g[k] == g[k + per] para todo k >= pre.
31    // Devuelve {-1, -1} si no hay.
32    // Es una prueba rigurosa: para n >= D, g[n] depende solo de
33    // g[n-D..n-1].
34    pair<int, int> detectar_periodo() const {
35        int D = *max_element(movs.begin(), movs.end());
36        map<vector<int>, int> visto;
37        for (int i = 0; i + D <= (int)g.size(); i++) {
38            vector<int> w(g.begin() + i, g.begin() + i + D);
39            auto it = visto.find(w);
40            if (it != visto.end()) return {it->second, i - it->second};
41            visto[w] = i;
42        }
43        return {-1, -1};
44    } // O(N * D * log N)
45
46    // READ: numero de Grundy de n; si n excede la tabla,
47    // extrapola con el periodo.
48    int valor(long long n, int pre, int per) const {
49        if (n < (long long)g.size()) return g[n];
50        return g[pre + (n - pre) % per];
51    } // O(1)
52
53    // READ: XOR de los numeros de Grundy de varias pilas (0 =
54    // posicion perdedora).
55    int xor_total(const vector<long long>& h, int pre, int per)
56    const {
57        int x = 0;
58        for (long long n : h) x ^= valor(n, pre, per);
59        return x;
60    } // O(k)
61
62    // READ: jugada ganadora {indice de pila, piedras a quitar},
63    // o {-1, -1} si la posicion pierde.
64    // Se busca una jugada con G(h) ^ G(h - m) == X, para que el
65    // XOR nuevo sea 0.
66    pair<int, int> jugada_ganadora(const vector<long long>& h,
67    int pre, int per) const {
68        int x = xor_total(h, pre, per);
69        if (x == 0) return {-1, -1};
70        for (int i = 0; i < (int)h.size(); i++)
71            for (int m : movs)
72                if (m <= h[i] && (valor(h[i], pre, per) ^ valor(h[i] - m,
73                pre, per)) == x)
74                    return {i, m};
75        return {-1, -1}; // no ocurre si la tabla y el periodo
76        // son correctos
77    } // O(k * M)
78 };
79
80 // -----

```



```

65 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
66 // -----
67 // 1. Juegos donde una jugada DIVIDE el componente (Kayles:
    quitar 1 o 2 pínos adyacentes
68 // de una fila de  $n$ ). Los pedazos son juegos independientes,
    así que se combinan por XOR:
69 // for (int n = 1; n <= N; n++) {
70 //     vector<int> sig;
71 //     for (int a = 0; a <= n - 1; a++) sig.push_back(g[a]
    ^ g[n - 1 - a]); // quitar 1
72 //     for (int a = 0; a <= n - 2; a++) sig.push_back(g[a]
    ^ g[n - 2 - a]); // quitar 2
73 //     g[n] = mex(sig);
74 // } //  $O(N^2)$ 
75 // Aquí g[n] depende de TODOS los valores anteriores, no de
    una ventana: el periodo
76 // (Kayles es eventualmente periodico de periodo 12 tras un
    preperiodo de unas 70
77 // posiciones, según la literatura) es solo una conjetura
    que se comprueba con un  $N$  grande.
78 //
79 // 2. Formulas cerradas conocidas: Nim (quitar cualquier
    cantidad de una pila):  $G(n) = n$ .
80 // Quitar de 1 a  $k$  piedras:  $G(n) = n \% (k + 1)$ ; con  $k = 3$  da
    la tabla 0 1 2 3 0 1 2 3.
81 // Comprobar la conjetura contra la tabla de fuerza bruta
    antes de usarla.
82 //
83 // 3. Detección empírica del periodo (cuando no hay ventana
    finita): buscar el menor  $per$ 
84 // tal que  $g[i] == g[i + per]$  para todo  $i \geq pre$ , exigiendo
    varios periodos completos.
85 // for (int per = 1; per <= N / 3; per++) {
86 //     int i = N - per;
87 //     while (i >= 0 && g[i] == g[i + per]) i--;
88 //     int pre = i + 1;
89 //     if (N + 1 - pre >= 3 * per) return {pre, per}; //
    al menos 3 periodos observados
90 // } //  $O(N^2)$  en el peor caso. NO es una prueba: aumentar
     $N$  para ganar confianza.
91 //
92 // 4. Estados compuestos pequeños (por ejemplo un tablero de  $n$ 
    <= 20 celdas): representar el
93 // estado como bitmask y hacer DP con memoria en un arreglo
    o unordered_map.
94 // int Grundy(int mask) {
95 //     if (memo[mask] != -1) return memo[mask];
96 //     vector<int> sig;
97 //     for (cada jugada legal desde mask) sig.push_back(
    Grundy(mask_resultante));
98 //     return memo[mask] = mex(sig);
99 // } //  $O(2^n * M)$ 
100 //
101 // 5. Variante misere (pierde quien hace la última jugada): la
    fórmula XOR NO aplica en
102 // general. En Nim misere basta jugar como en Nim normal,
    salvo que si todas las pilas
103 // tienen a lo sumo 1 piedra el resultado se invierte.
104 //
105 // 6. Verificar la solución: comparar la respuesta por XOR con
    una búsqueda exhaustiva sobre
106 // el juego completo para configuraciones pequeñas (todas
    las pilas hasta 8, por ejemplo).
107 //
108 // 7. Un solo componente y solo importa quien gana: sin MEX.
109 // for (int n = 1; n <= N; n++) {
110 //     gana[n] = false;
111 //     for (int m : movs) if (m <= n && !gana[n - m]) gana
    [n] = true;
112 // } //  $O(N * M)$ ; mas simple que la tabla de Grundy.
113 //
114 // 8. Juegos con ciclos (se puede volver a un estado ya visto):
    Grundy no está definido
115 // con esta recursión. Usar análisis retrogrado (ver "
    Trabaja hacia Atrás"), que
116 // reconoce los empates.

```

Ejemplo de funcionamiento:

```

1 int main() {
2     TablaGrundy a({1, 2, 3}, 20);
3     for (int n = 0; n <= 7; n++) cout << a.g[n] << " ";
4     cout << "\n"; // 0 1
    2 3 0 1 2 3
5     auto [pa, qa] = a.detectar_periodo();
6     cout << pa << " " << qa << "\n"; // 0 4
    (g[n] = n % 4)
7     cout << a.valor(1000000000000000000LL, pa, qa) << "\n"; // 0
    (10^18 % 4 = 0: posición perdedora)
8     cout << a.xor_total({2, 3, 1}, pa, qa) << "\n"; // 0
    (2 ^ 3 ^ 1: posición perdedora)
9     cout << a.xor_total({2, 3, 6}, pa, qa) << "\n"; // 3
    (2 ^ 3 ^ 2: posición ganadora)

```

```

10 TablaGrundy b({1, 3, 4}, 30);
11 for (int n = 0; n <= 6; n++) cout << b.g[n] << " ";
12 cout << "\n"; // 0 1
    0 1 2 3 2
13 auto [pb, qb] = b.detectar_periodo();
14 cout << pb << " " << qb << "\n"; // 0 7
15 cout << b.valor(1000000000000000000LL, pb, qb) << "\n"; // 1
    (10^18 % 7 = 1, g[1] = 1)
16
17 auto j = b.jugada_ganadora({5, 6, 9}, pb, qb); // g =
    3, 2, 0: XOR = 1
18 cout << j.first << " " << j.second << "\n"; // 0 1
    (quitar 1 de la pila 0: 5 -> 4, g = 2, XOR = 0)
19 auto k = b.jugada_ganadora({1000000000000000000LL}, pb, qb);
20 cout << k.first << " " << k.second << "\n"; // 0 1
    (10^18 -> 10^18 - 1, que es múltiplo de 7: g = 0)
21 return 0;
22 }

```

Nota: El teorema de Sprague–Grundy supone un juego imparcial de dos jugadores, con la convención de juego normal y sin partidas infinitas. Si las jugadas dependen del jugador, si hay más de dos jugadores (subsección siguiente) o si el objetivo es otro (por ejemplo, perder al hacer la última jugada), el XOR de los valores no sirve tal cual. Además, `detectar_periodo` solo es una prueba cuando el valor de $G(n)$ depende de una ventana fija de valores anteriores, como en los juegos de restar; con juegos que dividen el componente hay que tratar el periodo como una conjetura. Si `detectar_periodo` devuelve $\{-1, -1\}$, la tabla es demasiado corta y hay que aumentar N antes de extrapolar. `jugada_ganadora` supone que el conjunto de jugadas coincide con el de la tabla.

Regla práctica: Si hay una sola pila y solo importa quién gana, usa la DP booleana. Si el juego se compone de varios componentes independientes, calcula la tabla de Grundy de un componente por fuerza bruta y combina con XOR; si la consulta llega a n enorme, detecta el periodo con la ventana de D valores (restar) o compruébalo empíricamente (otros juegos), y verifica siempre contra la fuerza bruta en casos pequeños. Si hay ciclos, usa análisis retrógrado, y si hay más de dos jugadores o objetivos distintos del juego normal, ve a la siguiente subsección.

66.4 Juegos con Múltiples Jugadores (más de 2)

Idea: Con dos jugadores, si una posición no es ganadora para quien mueve, entonces lo es para el rival: «ganar» y «perder» son las dos caras de una misma moneda, y por eso funcionan minimax y Grundy. Con $K \geq 3$ jugadores esa dualidad desaparece. Si P_0 no puede ganar, aún importa *cuál* de los otros gana, y una jugada de P_0 puede favorecer a P_1 y perjudicar a P_2 . El resultado de un estado debe ser entonces un vector $R(s) = (r_0, \dots, r_{K-1})$ con lo que obtiene cada jugador (por ejemplo, $r_i = 1$ para quien gana y 0 para el resto, o las puntuaciones finales).

Además, el estado debe incluir de quién es el turno, así que la DP se define sobre pares (s, p) , y tras una jugada el turno pasa a $p' = (p + 1) \bmod K$ si el orden es circular. La regla de decisión es la *inducción hacia atrás* (llamada $\max^n$): en un estado no terminal, el jugador p examina el vector de cada estado sucesor y elige el que maximiza *su propia* componente r_p . Los estados terminales se evalúan directamente con las reglas del juego. Con $K = 2$ y suma constante esto coincide con minimax.

Hay un punto delicado: cuando varias jugadas dan la misma componente r_p al jugador que mueve, el resultado para los demás depende de la jugada que escoja, y ninguna regla matemática la fija. Por eso «el jugador que gana con juego óptimo» no está bien definido hasta que el enunciado (o el código) da un criterio de desempate. Es lo que ocurre, por ejemplo, cuando un jugador ya no puede ganar y decide a cuál de los rivales «regalar» la partida (*kingmaking*).

¿Por qué usarlo? Frente a Grundy (subsección anterior), que resuelve juegos imparciales de dos jugadores y combina componentes con XOR, aquí ni el teorema ni el XOR valen: no hay un valor escalar por estado que resuma el juego. Frente a minimax o negamax de dos jugadores, que guardan un solo número y suponen intereses opuestos, aquí se guarda un vector y cada jugador maximiza su componente. El costo es que el estado crece: además de la configuración hay que guardar el turno, y cada comparación cuesta $O(K)$ si el resultado es un vector. Se usa cuando el enunciado tiene tres o más jugadores con turnos rotativos, o cuando cada jugador tiene su propio objetivo. No modela alianzas ni acuerdos entre jugadores: cada uno decide solo por sí mismo. Si el juego tiene ciclos, la recursión no termina y hay que pasar a análisis retrógrado, que con más de dos jugadores es bastante más delicado.

Casos de uso:

- Juegos con tres o más jugadores y turnos rotativos (quitar piedras entre tres, juegos de cartas simplificados).
- Juegos con puntuaciones, donde cada jugador maximiza la suya.
- Juegos de eliminación, donde el orden de los turnos cambia según quién queda.
- Problemas donde el resultado para cada jugador se pide como vector (quién gana con qué puntaje).
- De nicho: verificar por fuerza bruta reglas de desempate ambiguas o comprobar un patrón conjeturado.

Complejidad: Sea N la cantidad de estados del juego, K los jugadores y M las jugadas máximas por estado. Hay $N \cdot K$ estados (s, p) , y cada uno examina M sucesores. Comparar y copiar un vector de resultados cuesta $O(K)$, por lo que el tiempo total es $O(N K^2 M)$. Si el resultado es un solo entero (por ejemplo, el índice del ganador), cada comparación es $O(1)$ y queda $O(N K M)$. Si el turno está determinado por el estado (por ejemplo, por la cantidad de jugadas hechas), la dimensión K del estado desaparece. Memoria: $O(N K)$ estados, y $O(N K^2)$ si cada uno guarda un vector.

```
1 // Ejemplo concreto: una pila de n piedras, K jugadores por
2 // turnos rotativos, cada jugada
3 // quita de 1 a "tope" piedras y GANA quien quita la ultima.
4 // Regla de desempate: si varias jugadas dan lo mismo al que
5 // mueve, se elige la de menor
6 // cantidad de piedras (la primera que se prueba).
7 struct JuegoMultijugador {
8     int K; // cantidad de
9         jugadores
10    int tope; // maximo de piedras
11        por jugada
12    vector<vector<vector<int>>>> memo; // memo[n][p]:
13        puntuaciones (vacío = sin calcular)
```

```
10 // CREATE: prepara la memoria para pilas de 0 a N piedras.
11 JuegoMultijugador(int jugadores, int max_tomar, int N)
12 : K(jugadores), tope(max_tomar), memo(N + 1, vector<vector<
13     int>>(jugadores)) {} // O(N * K)
14
15 // READ: puntuacion de un estado terminal (n = 0): gana quien
16 // hizo la ultima jugada,
17 // es decir, el jugador anterior a p.
18 vector<int> terminal(int p) const {
19     vector<int> r(K, 0);
20     r[(p + K - 1) % K] = 1;
21     return r;
22 } // O(K)
23
24 // WRITE: resuelve el estado (n, p) con memoria. Cada jugador
25 // maximiza su propia componente.
26 const vector<int>& resolver(int n, int p) {
27     vector<int>& m = memo[n][p];
28     if (!m.empty()) return m;
29     if (n == 0) return m = terminal(p);
30     vector<int> mejor;
31     for (int t = 1; t <= tope && t <= n; t++) {
32         const vector<int>& r = resolver(n - t, (p + 1) % K);
33         if (mejor.empty() || r[p] > mejor[p]) mejor = r; //
34         desempate: la primera jugada
35     }
36     return m = mejor;
37 } // O(M * K) por estado nuevo; O(N * K^2 * M) en total
38
39 // WRITE: jugador que gana desde (n, p) con juego optimo (
40 // calcula y guarda si hace falta).
41 int ganador(int n, int p) {
42     const vector<int>& r = resolver(n, p);
43     return (int)(max_element(r.begin(), r.end()) - r.begin());
44 } // O(K) mas el costo de resolver
45
46 // WRITE: jugada (piedras a quitar) que elige p en (n, p); -1
47 // si no hay jugadas.
48 int mejor_jugada(int n, int p) {
49     int mt = -1, ms = -1;
50     for (int t = 1; t <= tope && t <= n; t++) {
51         int s = resolver(n - t, (p + 1) % K)[p];
52         if (s > ms) { ms = s; mt = t; }
53     }
54     return mt;
55 } // O(M) mas el costo de resolver
56
57 // -----
58 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
59 // -----
60 // 1. Criterios de desempate: el resultado para los DEMAS
61 // depende de la regla. Opciones
62 // comunes: la primera jugada (la del código), la que mas
63 // beneficia al siguiente en el
64 // turno, o la que menos beneficia al lider. Cambiar el
65 // criterio cambia las respuestas:
66 // if (mejor.empty() || r[p] > mejor[p] || (r[p] == mejor[p]
67 // && mejor_desempate(r, mejor)))
68 //     mejor = r;
69 // El enunciado debe fijarlo; si no lo hace, el problema
70 // esta mal definido.
71 //
72 // 2. Turno determinado por el estado: si p se deduce de n (por
73 // ejemplo, p = (jugadas
74 // hechas) % K), se elimina la dimension p de la memoria:
75 // memo[n] en lugar de memo[n][p],
76 // y el costo baja a O(N * K * M).
77 //
78 // 3. Puntuaciones generales: el vector terminal contiene los
79 // puntajes finales y cada
80 // jugador maximiza su componente. Para juegos de SUMA
81 // CONSTANTE (repartir un total fijo
82 // entre los jugadores) basta guardar todas las componentes
83 // menos una.
84 //
85 // 4. Caso de dos jugadores: con K = 2 y suma constante, el
86 // vector es (x, T - x) y max^n
87 // se reduce a minimax (o negamax con un solo numero).
88 //
89 // 5. Sin recursion (memoria como tabla, para evitar
90 // profundidad de pila): llenar de n = 0
91 // hacia arriba, y para cada n todos los p.
92 // for (int n = 0; n <= N; n++) for (int p = 0; p < K; p++)
93 //     resolver_iterativo(n, p);
94 // // O(N * K * M) mas el costo de copiar vectores.
95 //
96 // 6. Poda: en max^n la poda alfa-beta solo funciona con cotas
97 // sobre la SUMA de las
98 // puntuaciones (por ejemplo, suma constante); con objetivos
99 // independientes casi
100 // no hay poda, y el costo exponencial de un juego grande es
101 // inevitable.
```

```

81 //
82 // 7. Alianzas y acuerdos: NO se modelan. Cada jugador maximiza
83 //    enunciado permite coaliciones o negociacion, se necesita
84 //    un modelo distinto
85 //    (teoria de juegos cooperativos), fuera del alcance de
86 //    esta técnica.
87 //
88 // 8. Juegos con ciclos: la recursion no termina. Con  $K \geq 3$  el
89 //    analisis retrogrado ya no
90 //    da una respuesta unica sin fijar el desempate y el
91 //    tratamiento de las partidas
92 //    infinitas; para concursos normalmente el enunciado
93 //    garantiza que el juego termina.
94 //
95 // 9. Sprague-Grundy con  $K \geq 3$ : no aplicar.  $G(s1) \wedge G(s2) \wedge \dots$ 
96 //    solo tiene sentido en
97 //    juegos imparciales de dos jugadores con juego normal.

```

Ejemplo de funcionamiento:

```

1  int main() {
2      JuegoMultijugador j(3, 3, 20);          // 3 jugadores, se
3      cout << j.ganador(0, 0) << "\n";        // 2 (sin piedras
4      : gana el jugador anterior a P0, o sea P2)
5      cout << j.ganador(3, 0) << "\n";        // 0 (P0 quita
6      las 3 y gana)
7      cout << j.ganador(4, 0) << "\n";        // 1 (toda jugada
8      de P0 deja 3, 2 o 1: gana el siguiente, P1)
9      cout << j.ganador(5, 2) << "\n";        // 1 (P2 no puede
10     ganar; por el desempate elige quitar 1 y gana P1)
11     cout << j.ganador(6, 0) << "\n";        // 0 (P0 quita 1
12     y deja 5 con el turno de P1, que solo puede regalar el juego
13     )
14     cout << j.ganador(10, 0) << "\n";        // 2 (P0 no puede
15     ganar: quita 1 y termina ganando P2)
16     cout << j.mejor_jugada(7, 0) << " " << j.mejor_jugada(8, 0)
17     << "\n"; // 2 3
18     for (int x : j.resolver(4, 0)) cout << x << " ";
19     cout << "\n"; // 0 1 0 (vector
20     de resultados: gana P1)
21
22     JuegoMultijugador d(2, 3, 20);          // con 2 jugadores
23     se recupera n % 4
24     cout << d.ganador(4, 0) << " " << d.ganador(5, 0) << "\n";
25     // 1 0
26     return 0;
27 }

```

Nota: Este ejemplo muestra por qué el desempate importa. Desde $(4, P_0)$ y $(5, P_2)$ ningún jugador que mueve puede ganar, y quien gana depende de la jugada que escoja, que aquí es la de menor cantidad de piedras. Con otro criterio (por ejemplo, favorecer siempre al jugador que sigue) cambiarían esas respuestas, mientras que los estados donde el jugador que mueve sí puede ganar no dependen del desempate. La memoria usa el vector vacío como marca de «sin calcular», lo cual es válido porque todo resultado calculado tiene K componentes. Las referencias que devuelve `resolver` siguen siendo válidas porque `memo` nunca cambia de tamaño después de construirse. Con N grande, la recursión llega hasta profundidad N , así que hay que pasar a la versión iterativa (modificación 5).

Regla práctica: Si hay tres o más jugadores con turnos rotativos, guarda en el estado el turno y un vector de resultados, y resuelve con inducción hacia atrás: cada jugador maximiza su propia componente. Fija por escrito el criterio de desempate (el enunciado debe dar uno, o el problema está mal definido) y no uses XOR de Grundy ni minimax de un solo valor. Si el turno se deduce del estado, quita esa dimensión; si son exactamente dos jugadores con juego imparcial, usa la tabla de Grundy de la subsección anterior; y si hay ciclos, alianzas o negociación, esta técnica no alcanza.